

GATE DA

Programming, Data Structures & Algorithms

50 FULL MOCK TESTS · 1000 QUESTIONS · DETAILED SOLUTIONS

Data Science and Artificial Intelligence (DA) — Section 4 Sectional Test Series

Python · Stacks · Queues · Linked Lists · Trees · Hash Tables

Linear & Binary Search · Selection / Bubble / Insertion Sort

Merge Sort · Quicksort · Graph Theory · Traversals · Shortest Paths

MCQ · MSQ · NAT in the exact GATE pattern — 1-mark and 2-mark questions

Every answer machine-verified · Diagrams for trees, graphs, lists and hash tables

Answer keys · Step-by-step solutions · Traps & tips · Quick-revision handbook

Aligned to the GATE 2027 DA syllabus (Section 4)



Contents

How to Use This Book	7
Syllabus and Topic Map	8
Quick-Revision Handbook	11
Mock Test 01	16
Foundations I	16
Detailed Solutions — Mock Test 01	25
Mock Test 02	35
Stacks & Expression Evaluation	35
Detailed Solutions — Mock Test 02	44
Mock Test 03	55
Queues, Deques & Circular Buffers	55
Detailed Solutions — Mock Test 03	64
Mock Test 04	73
Linked Lists	73
Detailed Solutions — Mock Test 04	83
Mock Test 05	93
Binary Trees & Traversals	93
Detailed Solutions — Mock Test 05	101
Mock Test 06	112
Binary Search Trees	112
Detailed Solutions — Mock Test 06	120
Mock Test 07	131
Heaps & Priority Queues	131
Detailed Solutions — Mock Test 07	140
Mock Test 08	151
Hash Tables	151
Detailed Solutions — Mock Test 08	160
Mock Test 09	172
Linear & Binary Search	172
Detailed Solutions — Mock Test 09	180
Mock Test 10	190
Elementary Sorting	190
Detailed Solutions — Mock Test 10	198

Mock Test 11	209
Merge Sort & Recurrences	209
Detailed Solutions — Mock Test 11	217
Mock Test 12	226
Quicksort & Partitioning	226
Detailed Solutions — Mock Test 12	234
Mock Test 13	244
Graph Theory Fundamentals	244
Detailed Solutions — Mock Test 13	251
Mock Test 14	260
Graph Traversals: BFS & DFS	260
Detailed Solutions — Mock Test 14	272
Mock Test 15	282
Shortest Paths	282
Detailed Solutions — Mock Test 15	291
Mock Test 16	301
Recursion & Memoization in Python	301
Detailed Solutions — Mock Test 16	312
Mock Test 17	322
Iterators, Generators & Comprehensions	322
Detailed Solutions — Mock Test 17	331
Mock Test 18	340
Scoping, Closures, OOP & Exceptions	340
Detailed Solutions — Mock Test 18	350
Mock Test 19	359
Complexity Analysis	359
Detailed Solutions — Mock Test 19	366
Mock Test 20	375
Stack & Queue Applications	375
Detailed Solutions — Mock Test 20	386
Mock Test 21	396
Counting & Structure of Trees	396
Detailed Solutions — Mock Test 21	403
Mock Test 22	412
DAGs & Topological Sorting	412
Detailed Solutions — Mock Test 22	423

Mock Test 23	433
Hashing Analysis	433
Detailed Solutions — Mock Test 23	440
Mock Test 24	448
Sorting Properties	448
Detailed Solutions — Mock Test 24	456
Mock Test 25	465
Binary Search Variants	465
Detailed Solutions — Mock Test 25	473
Mock Test 26	481
Linked-List Algorithms	481
Detailed Solutions — Mock Test 26	490
Mock Test 27	498
Graph Representations	498
Detailed Solutions — Mock Test 27	506
Mock Test 28	516
Connectivity & Bipartiteness	516
Detailed Solutions — Mock Test 28	526
Mock Test 29	536
Weighted Shortest Paths	536
Detailed Solutions — Mock Test 29	546
Mock Test 30	557
Divide & Conquer Applications	557
Detailed Solutions — Mock Test 30	566
Mock Test 31	574
Full-Syllabus Mock — Paper 1	574
Detailed Solutions — Mock Test 31	583
Mock Test 32	592
Full-Syllabus Mock — Paper 2	592
Detailed Solutions — Mock Test 32	599
Mock Test 33	609
Full-Syllabus Mock — Paper 3	609
Detailed Solutions — Mock Test 33	616
Mock Test 34	624
Full-Syllabus Mock — Paper 4	624

Detailed Solutions — Mock Test 34	631
Mock Test 35	640
Full-Syllabus Mock — Paper 5	640
Detailed Solutions — Mock Test 35	648
Mock Test 36	657
Full-Syllabus Mock — Paper 6	657
Detailed Solutions — Mock Test 36	664
Mock Test 37	673
Full-Syllabus Mock — Paper 7	673
Detailed Solutions — Mock Test 37	680
Mock Test 38	691
Full-Syllabus Mock — Paper 8	691
Detailed Solutions — Mock Test 38	699
Mock Test 39	708
Full-Syllabus Mock — Paper 9	708
Detailed Solutions — Mock Test 39	715
Mock Test 40	724
Full-Syllabus Mock — Paper 10	724
Detailed Solutions — Mock Test 40	732
Mock Test 41	741
Full-Syllabus Mock — Paper 11	741
Detailed Solutions — Mock Test 41	748
Mock Test 42	756
Full-Syllabus Mock — Paper 12	756
Detailed Solutions — Mock Test 42	764
Mock Test 43	773
Full-Syllabus Mock — Paper 13	773
Detailed Solutions — Mock Test 43	780
Mock Test 44	788
Full-Syllabus Mock — Paper 14	788
Detailed Solutions — Mock Test 44	796
Mock Test 45	805
Full-Syllabus Mock — Paper 15	805
Detailed Solutions — Mock Test 45	812
Mock Test 46	821

Challenge Mock — Paper 1	821
Detailed Solutions — Mock Test 46	829
Mock Test 47	838
Challenge Mock — Paper 2	838
Detailed Solutions — Mock Test 47	846
Mock Test 48	854
Challenge Mock — Paper 3	854
Detailed Solutions — Mock Test 48	864
Mock Test 49	873
Challenge Mock — Paper 4	873
Detailed Solutions — Mock Test 49	882
Mock Test 50	891
Challenge Mock — Paper 5	891
Detailed Solutions — Mock Test 50	900
Consolidated Answer Keys	907
Score Tracker	910

How to Use This Book

This book is a complete sectional test series for **Section 4 — Programming, Data Structures and Algorithms** of the GATE Data Science and Artificial Intelligence (DA) paper. It contains **50 mock tests of 20 questions each**, built to the pattern, difficulty and question styles of the real GATE DA paper, followed by an answer key and fully worked solutions for every test.

The tests are graded in three tiers:

- **Tests 01–15 (Moderate):** one focus topic per test — ideal while you are still revising a topic. The rest of each paper still spans the whole section so that you keep earlier topics warm.
- **Tests 16–30 (GATE-level):** deeper themes — recursion, generators, closures, complexity, counting trees, topological orders, hashing analysis, sorting properties, binary-search variants, weighted shortest paths.
- **Tests 31–45 (Full-syllabus GATE-level papers) and Tests 46–50 (Challenge papers):** balanced, timed papers exactly like the section you will face in the exam; the challenge papers are deliberately harder and trap-heavy.

Recommended routine. Sit each test in one go with a timer (54 minutes for 30 marks, the GATE rate of about 1.8 minutes per mark). Use only rough paper — no IDE. Mark answers on the response grid, score yourself with the answer key using the GATE marking scheme, and only then read the solutions. Every solution explains the concept, traces the computation step by step, analyses every option and names the common trap. Log your score in the tracker at the end of the book and revisit every question you got wrong after a week.

Conventions used throughout. Programs are Python 3. Unless a question says otherwise: arrays/lists are 0-indexed; the *height* of a tree is the number of edges on the longest root-to-leaf path (a single node has height 0); log means $\log_2$; adjacency lists are explored in the order stated in the question (usually alphabetical or increasing numerical order); a *comparison* means a key comparison between two array elements.

Exam pattern and marking scheme

Question type	What you do	Marks	Negative marking
MCQ — Multiple Choice	Choose the single correct option out of four	1 or 2	−1/3 for a wrong 1-mark answer, −2/3 for a wrong 2-mark answer
MSQ — Multiple Select	Choose ALL correct options (one or more of four)	1 or 2	None. No partial credit: you must select exactly the correct set
NAT — Numerical Answer	Type a number using the virtual keypad	1 or 2	None. Answers given as a range accept any value in the range

The actual GATE DA paper has 65 questions for 100 marks in 180 minutes (General Aptitude: 15 marks; subject: 85 marks), so roughly one in seven to one in five subject questions comes from this section. Each mock test here is a 30-mark sectional slice: **Q1–Q10 carry 1 mark** and **Q11–Q20 carry 2 marks**.

Score interpretation (per 30-mark test)

Score	Reading	What to do next
24 – 30	Exam-ready on this material	Move to the next tier; time yourself more strictly
18 – 23.9	Strong, but leaking marks	Re-read the traps for every miss; redo after a week
12 – 17.9	Concepts present, execution shaky	Revise the handbook pages for the weak topics, then retest
below 12	Foundation gap	Study the topic first; attempt the Moderate tier tests on it

Syllabus and Topic Map

GATE 2027 DA — Section 4: Programming, Data Structures and Algorithms (official text):

Programming in Python, basic data structures: stacks, queues, linked lists, trees, hash tables; Search algorithms: linear search and binary search, basic sorting algorithms: selection sort, bubble sort and insertion sort; divide and conquer: mergesort, quicksort; introduction to graph theory; basic graph algorithms: traversals and shortest path.

What is examined under each syllabus item

Syllabus item	Sub-topics covered in these tests
Programming in Python	Data types, integer vs float division, <code>//</code> and <code>%</code> with negatives, strings and slicing, lists/tuples/dicts/sets, mutability and aliasing, shallow vs deep copy, functions, default arguments, <code>*args/**kwargs</code> , scope (LEGB), <code>global/nonlocal</code> , closures and late binding, recursion, comprehensions, generators and iterators, <code>lambda/map/filter/zip/sorted</code> , classes and objects, exceptions and <code>finally</code> .
Stacks	Push/pop traces, infix→postfix/prefix, postfix evaluation, balanced brackets, stack permutations (Catalan numbers), recursion as a stack, monotonic stacks, two stacks in one array.
Queues	Linear and circular queues (front/rear arithmetic, full/empty tests), dequeues, queue using two stacks (amortised cost), stack using queues, priority queues.
Linked lists	Singly, doubly and circular lists; insertion/deletion/reversal; cycle detection; middle element; merging; costs of operations with and without tail pointers.
Trees	Binary trees, traversals and reconstruction, height/size bounds, full/complete/perfect trees, BSTs (search/insert/delete, valid search sequences), number of BSTs/binary trees, binary heaps and heap operations.
Hash tables	Division/multiplication hashing, chaining, linear/quadratic probing, double hashing, deletion with tombstones, load factor and expected probe counts, clustering, collision probabilities.
Searching	Linear search (with/without sentinel, expected comparisons), binary search traces, lower/upper bounds, number of comparisons, searching rotated arrays, binary search on the answer.
Elementary sorting	Selection, bubble (with early exit), insertion sort: passes, swaps, comparisons, inversions, best/worst cases, stability, in-place property, adaptivity.
Divide and conquer	Merge sort (merge comparisons, recursion tree, space), quicksort (Lomuto/Hoare partition, pivot effects, worst/best cases), recurrences and the master theorem, D&C applications.
Graph theory	Degree sequences, handshaking lemma, simple/complete/bipartite graphs, trees and forests, connectivity and components, cycles, representations (matrix/list) and their costs.
Graph algorithms	BFS and DFS orders, BFS levels and shortest hops, DFS trees, discovery/finish times, edge classification, topological sort, connected components, Dijkstra, Bellman-Ford, shortest-path trees.

Test-by-test focus

Test	Title	Difficulty	Focus
01	Foundations I	Moderate	Python basics, lists, strings, slicing, arrays
02	Stacks & Expression Evaluation	Moderate	stacks, infix/postfix/prefix, balanced brackets, stack permutations
03	Queues, Deques & Circular Buffers	Moderate	queues, circular queue arithmetic, deque, queue via stacks
04	Linked Lists	Moderate	singly/doubly/circular lists, pointer manipulation, traversal costs
05	Binary Trees & Traversals	Moderate	traversals, reconstruction from traversals, height, node counts
06	Binary Search Trees	Moderate	BST insert/delete/search, valid sequences, min/max, successor
07	Heaps & Priority Queues	Moderate	binary heaps, heapify, build-heap, k-th element, heap arrays
08	Hash Tables	Moderate	chaining, linear/quadratic probing, double hashing, deletion
09	Linear & Binary Search	Moderate	binary search traces, comparisons, boundaries, sentinel search
10	Elementary Sorting	Moderate	selection, bubble, insertion sorts — passes, swaps, comparisons
11	Merge Sort & Recurrences	Moderate	merge sort traces, merge comparisons, recurrences, master theorem
12	Quicksort & Partitioning	Moderate	Lomuto/Hoare partition, pivot choice, best/worst case
13	Graph Theory Fundamentals	Moderate	degrees, handshaking, trees, simple graphs, complete/bipartite graphs
14	Graph Traversals: BFS & DFS	Moderate	BFS/DFS orders, BFS levels, DFS trees, discovery/finish times
15	Shortest Paths	Moderate	Dijkstra traces, BFS shortest paths, path counting
16	Recursion & Memoization in Python	GATE-level	recursion tracing, call counts, memoized recursion, recursion trees
17	Iterators, Generators & Comprehensions	GATE-level	generators, iterator exhaustion, comprehensions, map/filter/zip
18	Scoping, Closures, OOP & Exceptions	GATE-level	LEGB, global/nonlocal, closures, classes, try/except/finally
19	Complexity Analysis	GATE-level	loop-nest complexity, asymptotic comparison, recurrence solving
20	Stack & Queue Applications	GATE-level	simulations, monotonic stack, two-stack queue costs, amortised ops
21	Counting & Structure of Trees	GATE-level	number of BSTs/binary trees, complete/full trees, height bounds
22	DAGs & Topological Sorting	GATE-level	topological orders count, DFS edge classification, cycle detection
23	Hashing Analysis	GATE-level	load factor, expected probes, clustering, probability of collisions
24	Sorting Properties	GATE-level	stability, inversions, adaptivity, in-place, comparison lower bounds
25	Binary Search Variants	GATE-level	lower/upper bound, rotated arrays, search on answer, off-by-one bugs
26	Linked-List Algorithms	GATE-level	reversal, cycle detection, merging, middle node, list surgery
27	Graph Representations	GATE-level	adjacency matrix/list, matrix powers & walks, degree sums, storage

Test	Title	Difficulty	Focus
28	Connectivity & Bipartiteness	GATE-level	components, bridges/cut vertices intuition, bipartite checks, cycles
29	Weighted Shortest Paths	GATE-level	Dijkstra with ties, negative edges, Bellman-Ford rounds, path trees
30	Divide & Conquer Applications	GATE-level	inversion counting, max subarray, fast power, D&C recurrences
31	Full-Syllabus Mock — Paper 1	GATE-level	balanced paper across the whole Section 4 syllabus
32	Full-Syllabus Mock — Paper 2	GATE-level	balanced paper across the whole Section 4 syllabus
33	Full-Syllabus Mock — Paper 3	GATE-level	balanced paper across the whole Section 4 syllabus
34	Full-Syllabus Mock — Paper 4	GATE-level	balanced paper across the whole Section 4 syllabus
35	Full-Syllabus Mock — Paper 5	GATE-level	balanced paper across the whole Section 4 syllabus
36	Full-Syllabus Mock — Paper 6	GATE-level	balanced paper across the whole Section 4 syllabus
37	Full-Syllabus Mock — Paper 7	GATE-level	balanced paper across the whole Section 4 syllabus
38	Full-Syllabus Mock — Paper 8	GATE-level	balanced paper across the whole Section 4 syllabus
39	Full-Syllabus Mock — Paper 9	GATE-level	balanced paper across the whole Section 4 syllabus
40	Full-Syllabus Mock — Paper 10	GATE-level	balanced paper across the whole Section 4 syllabus
41	Full-Syllabus Mock — Paper 11	GATE-level	balanced paper across the whole Section 4 syllabus
42	Full-Syllabus Mock — Paper 12	GATE-level	balanced paper across the whole Section 4 syllabus
43	Full-Syllabus Mock — Paper 13	GATE-level	balanced paper across the whole Section 4 syllabus
44	Full-Syllabus Mock — Paper 14	GATE-level	balanced paper across the whole Section 4 syllabus
45	Full-Syllabus Mock — Paper 15	GATE-level	balanced paper across the whole Section 4 syllabus
46	Challenge Mock — Paper 1	Hard	hardest, multi-concept, trap-heavy 2-mark questions
47	Challenge Mock — Paper 2	Hard	hardest, multi-concept, trap-heavy 2-mark questions
48	Challenge Mock — Paper 3	Hard	hardest, multi-concept, trap-heavy 2-mark questions
49	Challenge Mock — Paper 4	Hard	hardest, multi-concept, trap-heavy 2-mark questions
50	Challenge Mock — Paper 5	Hard	hardest, multi-concept, trap-heavy 2-mark questions

Quick-Revision Handbook

A compact reference of the facts, formulas and Python behaviours that the questions in this book test most often. Read it before each test tier, and come back to it whenever a solution names a trap.

1. Python behaviours that GATE loves to test

Behaviour	Example	Result / rule
Floor division & modulo	<code>-7 // 2, -7 % 2, 7 % -3</code>	<code>-4, 1, -2</code> — <code>//</code> floors toward $-\infty$; <code>a % b</code> has the sign of <code>b</code> ; <code>a == (a//b)*b + a%b</code>
True division	<code>7 / 2, 4 / 2</code>	<code>3.5, 2.0</code> (always float)
Slicing	<code>s[::-1], s[1:-1], s[5:1:-2]</code>	Slices never raise <code>IndexError</code> ; stop is exclusive; negative step walks backwards
Aliasing	<code>b = a; b.append(1)</code>	<code>a</code> changes too — same object. <code>a[:]</code> , <code>list(a)</code> , <code>copy.copy(a)</code> make shallow copies
Shallow copy of nested	<code>m = [[0]*3]*3; m[0][0] = 1</code>	All three rows change: the inner list is repeated by reference
Mutable default	<code>def f(x, L=[])</code>	Default created once at <code>def</code> time and shared across calls
<code>is</code> vs <code>==</code>	<code>[1] == [1], [1] is [1]</code>	<code>True, False</code> — identity vs equality
Closures (late binding)	<code>fs = [lambda: i for i in range(3)]</code>	Every <code>f()</code> returns <code>2</code> — <code>i</code> is looked up when called. Fix: <code>lambda i=i: i</code>
Scope	assigning to a name inside a function	makes it local for the whole function → <code>UnboundLocalError</code> if read first; use <code>global/nonlocal</code>
Generators	<code>g = (x*x for x in range(3))</code>	Single pass: <code>list(g)</code> twice gives <code>[0, 1, 4]</code> then <code>[]</code>
Short-circuit	<code>0 or [] or 'a', 1 and 0 and 2</code>	<code>'a', 0</code> — returns the deciding operand, not a bool
Strings immutable	<code>s[0] = 'x'</code>	<code>TypeError</code> ; <code>s.replace</code> returns a new string
Dict order	<code>{'b':1, 'a':2}</code>	Insertion order preserved (Python 3.7+); keys must be hashable
sorted stability	<code>sorted(L, key=len)</code>	Stable: equal keys keep their original relative order
<code>try/finally</code>	return inside <code>try</code> and <code>finally</code>	<code>finally</code> always runs; a <code>return</code> in <code>finally</code> overrides the earlier one
Chained comparison	<code>1 < 3 > 2</code>	<code>True</code> — means <code>1 < 3</code> and <code>3 > 2</code>
Integer size	<code>2**100</code>	Exact — Python ints are unbounded

2. Complexity cheat-sheet

Structure / operation	Average	Worst	Notes
Python list: index, append	$O(1)$	$O(1)$ amortised	<code>insert(0,x)</code> , <code>pop(0)</code> , <code>x in L</code> are $O(n)$
dict / set: lookup, insert	$O(1)$	$O(n)$	Hash-based; worst case under pathological collisions
Stack push/pop (array or list)	$O(1)$	$O(1)$	
Queue via two stacks	$O(1)$ amortised	$O(n)$ for one dequeue	Each element moved at most twice
Linked list: insert at head	$O(1)$	$O(1)$	Insert at tail $O(1)$ only with a tail pointer
Linked list: search / k-th	$O(n)$	$O(n)$	Delete a given node in a singly list needs its predecessor
BST search/insert/delete	$O(\log n)$	$O(n)$	Height $h \Rightarrow O(h)$
Binary heap insert / extract	$O(\log n)$	$O(\log n)$	find-min $O(1)$; build-heap $O(n)$

Structure / operation	Average	Worst	Notes
Hash table (chaining) search	$O(1 + \alpha)$	$O(n)$	$\alpha = n/m$ load factor
Linear search	$O(n)$	$O(n)$	Expected $(n+1)/2$ comparisons for a successful uniform search
Binary search	$O(\log n)$	$O(\log n)$	At most $\lfloor \log_2 n \rfloor + 1$ probes
Graph BFS / DFS	$O(V + E)$	$O(V + E)$	$O(V^2)$ with an adjacency matrix
Dijkstra (binary heap)	$O((V + E) \log V)$		$O(V^2)$ with an array; fails with negative edges
Bellman-Ford	$O(VE)$		$V-1$ rounds; detects negative cycles
Floyd-Warshall	$O(V^3)$		All pairs

3. Sorting at a glance

Algorithm	Best	Average	Worst	Swaps (worst)	Stable	In-place	Adaptive
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$n - 1$	No*	Yes	No
Bubble sort (early exit)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$n(n-1)/2$	Yes	Yes	Yes
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$n(n-1)/2$ shifts	Yes	Yes	Yes
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	—	Yes	No ($\Theta(n)$ extra)	No
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	—	No	Yes (stack $O(\log n) - O(n)$)	No
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	—	No	Yes	No

*Selection sort with swapping is unstable; a shifting variant can be made stable.

- **Inversions:** a pair (i, j) with $i < j$ and $A[i] > A[j]$. Bubble sort performs exactly one swap per inversion; insertion sort performs exactly one shift per inversion and at most $(n-1) + \text{\#inversions}$ comparisons. Maximum inversions = $n(n-1)/2$ (reverse order); expected for a random permutation = $n(n-1)/4$.
- **Merging** two sorted lists of sizes m and n takes between $\min(m, n)$ and $m + n - 1$ comparisons.
- **Comparison sorting lower bound:** any comparison sort needs $\lceil \log_2 n! \rceil$ comparisons in the worst case = $\Omega(n \log n)$.
- **Quicksort worst case** happens when the pivot is always the minimum or maximum (e.g. sorted input with first/last-element pivot): $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$, with $n(n-1)/2$ comparisons.
- **Lomuto partition** (pivot = last):
 $i = \text{lo}-1$; for j in $\text{lo}..\text{hi}-1$: if $A[j] \leq \text{pivot}$: $i += 1$, swap $A[i], A[j]$; finally swap $A[i+1], A[\text{hi}]$. **Hoare partition** uses two inward-moving indices and returns a split point, not the pivot's final position.

4. Recurrences and the master theorem

For $T(n) = a \cdot T(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$:

- $f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow T(n) = \Theta(n^{\log_b a})$.
- $f(n) = \Theta(n^{\log_b a} \log^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity $\Rightarrow T(n) = \Theta(f(n))$.

Recurrence	Solution	Where it appears
$T(n) = T(n/2) + 1$	$\Theta(\log n)$	Binary search
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	Merge sort, balanced quicksort
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Worst-case quicksort, selection sort
$T(n) = T(n-1) + 1$	$\Theta(n)$	Linear recursion, list length
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	Towers of Hanoi ($2^n - 1$ moves)
$T(n) = 2T(n/2) + 1$	$\Theta(n)$	Tree traversal, max by D&C
$T(n) = T(n/2) + n$	$\Theta(n)$	Selection by halving
$T(n) = 7T(n/2) + n^2$	$\Theta(n^{\log_2 7})$	Strassen
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\phi^n)$	Naive Fibonacci; calls = $2F(n+1) - 1$

5. Trees — formulas worth memorising

- A binary tree of height h has at most $2^{h+1} - 1$ nodes and at least $h + 1$ nodes; a binary tree with n nodes has height between $\lfloor \log_2 n \rfloor$ and $n - 1$.
- In any non-empty binary tree, $n_0 = n_2 + 1$ (leaves = nodes with two children + 1). A full (strict) binary tree with i internal nodes has $i + 1$ leaves and $2i + 1$ nodes.
- Number of structurally different binary trees (and of BSTs on n distinct keys) is the Catalan number $C_n = (2n)! / ((n+1)! n!)$: 1, 1, 2, 5, 14, 42, 132, 429, 1430 for $n = 0..8$. C_n also counts valid stack permutations of $1..n$ and balanced bracket strings with n pairs.
- Number of labelled binary trees with n nodes = $n! \cdot C_n$. Number of BSTs of height $n-1$ on n keys = 2^{n-1} .
- Pre-order + in-order (or post-order + in-order) determine a binary tree uniquely; pre-order + post-order do not (unless the tree is full).
- Heap stored in an array (0-indexed): children of i are $2i+1$ and $2i+2$, parent is $\lfloor (i-1)/2 \rfloor$. Leaves are indices $\lfloor n/2 \rfloor .. n-1$. Build-heap is $O(n)$; the k -th smallest element of a min-heap lies at depth $< k$.
- BST in-order traversal is sorted. Deleting a node with two children replaces it by its in-order successor (minimum of right subtree) or predecessor (maximum of left subtree).

6. Hashing formulas

- Load factor $\alpha = n / m$. Chaining: expected unsuccessful search $1 + \alpha$ probes ($\Theta(1 + \alpha)$); successful $\approx 1 + \alpha/2$.
- Open addressing, uniform hashing: expected probes for unsuccessful search $\leq 1/(1 - \alpha)$; successful $\leq (1/\alpha) \cdot \ln(1/(1 - \alpha))$.
- Linear probing (Knuth): unsuccessful $\approx \frac{1}{2}(1 + 1/(1 - \alpha)^2)$, successful $\approx \frac{1}{2}(1 + 1/(1 - \alpha))$.
- Probe sequences: linear $h(k) + i$, quadratic $h(k) + c_1 i + c_2 i^2$, double hashing $h_1(k) + i \cdot h_2(k)$ (all mod m). Quadratic probing may fail to find a free slot even when one exists; double hashing needs $h_2(k) \neq 0$ and coprime with m to visit every slot.
- With n keys hashed uniformly into m slots, expected number of colliding pairs = $C(n, 2)/m$; probability that all n go to distinct slots = $m(m-1) \dots (m-n+1)/m^n$; expected empty slots = $m(1 - 1/m)^n$.
- Deletion in open addressing must leave a tombstone (DELETED marker); otherwise later searches stop early.

7. Graph facts

- Handshaking lemma: $\sum \deg(v) = 2|E|$; the number of odd-degree vertices is even. In a digraph $\sum \text{in-deg} = \sum \text{out-deg} = |E|$.
- Simple undirected graph on n vertices has at most $n(n-1)/2$ edges; number of labelled simple graphs is $2^{n(n-1)/2}$. $K_{m,n}$ has mn edges; K_n has $n(n-1)/2$.
- A tree on n vertices has $n - 1$ edges; a forest with k components has $n - k$ edges. Number of labelled trees on n vertices = n^{n-2} (Cayley).
- A graph is bipartite $\Leftrightarrow$ it has no odd cycle $\Leftrightarrow$ BFS 2-colouring never finds an edge inside one level.
- A graph with n vertices and k components has at least $n - k$ edges; with more than $(n-1)(n-2)/2$ edges a simple graph must be connected.
- BFS from s gives shortest hop counts; edges of an undirected graph go between the same or adjacent BFS levels. DFS in an undirected graph has only tree and back edges; in a digraph, tree/back/forward/cross, and a back edge exists $\Leftrightarrow$ there is a cycle.
- Topological order exists $\Leftrightarrow$ the digraph is acyclic; reverse post-order of DFS is a topological order.
- $(A^k)[i][j]$ = number of walks of length k from i to j in the adjacency matrix A .
- Dijkstra requires non-negative weights; each vertex is finalised once, in non-decreasing order of distance. Bellman-Ford relaxes all edges $V - 1$ times; an improvement in round V signals a negative cycle.

8. Reference implementations

Searching and sorting

```

1 def binary_search(A, x):                # first index with A[i] >= x
2     lo, hi = 0, len(A)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if A[mid] < x: lo = mid + 1
6         else: hi = mid
7     return lo
8
9 def insertion_sort(A):
10    for i in range(1, len(A)):
11        key, j = A[i], i - 1
12        while j >= 0 and A[j] > key:
13            A[j + 1] = A[j]; j -= 1
14        A[j + 1] = key
15
16 def merge(L, R):
17    out, i, j = [], 0, 0
18    while i < len(L) and j < len(R):
19        if L[i] <= R[j]: out.append(L[i]); i += 1
20        else: out.append(R[j]); j += 1
21    return out + L[i:] + R[j:]
22
23 def lomuto(A, lo, hi):
24    p, i = A[hi], lo - 1
25    for j in range(lo, hi):
26        if A[j] <= p:
27            i += 1; A[i], A[j] = A[j], A[i]
28    A[i + 1], A[hi] = A[hi], A[i + 1]
29    return i + 1

```

Graph algorithms

```

1 from collections import deque
2 import heapq
3
4 def bfs(G, s):
5     dist, q = {s: 0}, deque([s])
6     while q:
7         u = q.popleft()

```

```
8         for v in G[u]:
9             if v not in dist:
10                 dist[v] = dist[u] + 1; q.append(v)
11     return dist
12
13 def dfs(G, u, seen, order):
14     seen.add(u); order.append(u)
15     for v in G[u]:
16         if v not in seen: dfs(G, v, seen, order)
17
18 def dijkstra(G, s):
19     # G[u] = [(v, w), ...], w >= 0
20     d, pq = {s: 0}, [(0, s)]
21     while pq:
22         du, u = heapq.heappop(pq)
23         if du > d[u]: continue
24         for v, w in G[u]:
25             if du + w < d.get(v, float('inf')):
26                 d[v] = du + w; heapq.heappush(pq, (d[v], v))
27     return d
```

Mock Test 01

Foundations I

Difficulty: Moderate

Focus areas: Python basics, lists, strings, slicing, arrays

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing with negative steps]

Consider the following Python program. What is printed?

Python

```
1 s = "DATASCIENCE"
2 print(s[-2:-9:-3], s[::4])
```

- (A) EIA DSN
- (B) CIAT DSN
- (C) CIA DSN
- (D) CIA DSNE

Q.2 [NAT · numerical answer | 1 mark | Python — aliasing, += versus +]

Consider the following Python program. The value printed is _____.

Python

```
1 a = [1, 2, 3]
2 b = a
3 c = a[:]
4 b += [4]
5 c.append(5)
6 a = a + [6]
7 b.extend(c)
8 print(len(a) + len(b) + len(c))
```

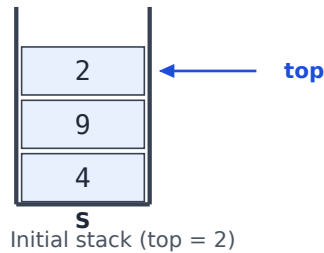
Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — operation trace]

The stack S below is shown bottom → top. POP() removes and returns the top, TOP() returns the top without removing it, and in an expression the operands are evaluated left to right. The following operations are executed in order:

- 1. PUSH(POP() × 3)
- 2. PUSH(POP() + POP())
- 3. PUSH(8)
- 4. t = POP(); PUSH(POP() – t)
- 5. PUSH(TOP() × TOP())

The final contents of S, bottom → top, are



- (A) 4, 7, 14
- (B) 4, 7, 49
- (C) 4, -7, 49
- (D) 4, 15, 8, 49

Q.4 [MCQ · single correct | 1 mark | Linked lists — pointer manipulation]

The singly linked list shown is built and the program below is run on it. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.next = v, nxt
4
5 head = None
6 for v in [4, 6, 1, 8, 3]:
7     head = Node(v, head)
8
9 p = head
10 while p.next and p.next.next:
11     p.next = p.next.next
12     p = p.next
13
14 out, q = [], head
15 while q:
16     out.append(q.v)
17     q = q.next
18 print(out)

```



- (A) [3, 1, 4]
- (B) [3, 8, 1]
- (C) [8, 6]
- (D) [3, 1, 6, 4]

Q.5 [NAT · numerical answer | 1 mark | Binary search — probe trace]

Iterative binary search with $lo = 0$, $hi = 12$, $mid = (lo + hi) // 2$, $lo = mid + 1$ if $A[mid] < key$, $hi = mid - 1$ if $A[mid] > key$, and termination when $lo > hi$, is used to search for key **47** in the sorted array A below. The sum of all array elements that are compared with the key is _____.

A =	3	8	12	17	21	26	30	34	41	45	52	58	63
	0	1	2	3	4	5	6	7	8	9	10	11	12

Answer: _____ (enter a numerical value)

Q.6 [MSQ · one or more correct | 1 mark | Insertion sort — shifts and comparisons]

Standard insertion sort (for $i = 1 \dots n-1$, the key $A[i]$ is compared with $A[i-1]$, $A[i-2]$, ... and each larger element is shifted one place right) sorts $A = [5, 2, 9, 1, 7, 3]$ into ascending order. A *comparison* is one evaluation of $A[j] > key$. Which of the following statements is/are TRUE?

- (A) The total number of element shifts is 8
- (B) The total number of key comparisons is 11
- (C) The element 9 is shifted to the right exactly twice
- (D) After the iterations $i = 1, 2, 3$ the array is $[1, 2, 5, 9, 7, 3]$

Q.7 [NAT · numerical answer | 1 mark | Hashing — linear probing collisions]

The keys 4, 15, 26, 9, 7, 20, 2 are inserted, in that order, into an initially empty hash table with 11 slots (indices 0 ... 10) using $h(k) = (3k + 1) \bmod 11$ and linear probing ($h(k)$, $h(k)+1$, ... mod 11). A *collision* is counted each time a probed slot is found occupied. The total number of collisions is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Binary search trees — traversal]

The keys 40, 25, 61, 18, 33, 52, 70, 29, 36, 55 are inserted in that order into an initially empty binary search tree. The post-order traversal of the resulting tree is

- (A) 29, 36, 18, 33, 25, 52, 55, 70, 61, 40
- (B) 18, 25, 29, 33, 36, 40, 52, 55, 61, 70
- (C) 18, 36, 29, 33, 25, 55, 52, 70, 61, 40
- (D) 18, 29, 36, 33, 25, 55, 52, 70, 61, 40

Q.9 [MCQ · single correct | 1 mark | Python — floor division and modulo with negatives]

Consider the following Python program. What is printed?

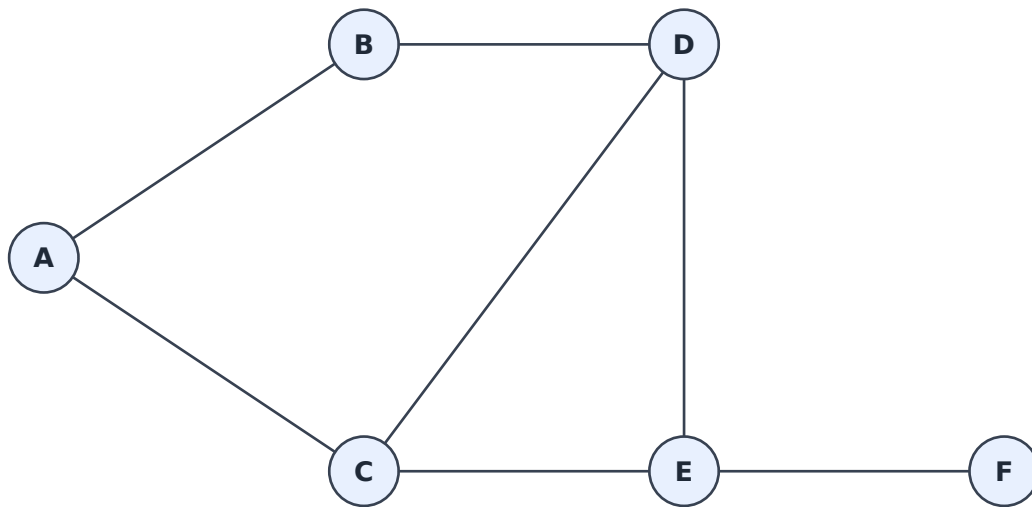
Python

```
1 print(-17 // 5, -17 % 5, 17 // -5, 17 % -5, int(-17 / 5))
```

- (A) -3 -2 -3 2 -3
- (B) -4 -2 -3 -3 -3
- (C) -4 3 -4 3 -4
- (D) -4 3 -4 -3 -3

Q.10 [MSQ · one or more correct | 1 mark | Graphs — degrees, BFS and DFS]

Consider the undirected graph G below. In every traversal, neighbours are visited in alphabetical order. Which of the following statements is/are TRUE?



Graph G

- (A) The recursive DFS order starting from A is A, B, D, E, C, F
- (B) G is bipartite
- (C) In the BFS tree rooted at A, vertex F is at distance 3 from A
- (D) G has exactly 4 vertices of odd degree

Q.11 [NAT · numerical answer | 2 marks | Python — nested lists and shallow copies]

Consider the following Python program. The value printed is _____.

Python

```
1 grid = [[0] * 3] * 3
2 grid[0][1] = 5
3 rows = [row[:] for row in grid]
4 rows[1][2] = 7
5 grid[2][0] += 1
6 total = sum(sum(r) for r in grid)
7 total += sum(sum(r) for r in rows)
8 print(total)
```

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Python — strings, loops and slicing]

Consider the following Python program. What is printed?

Python

```

1 def encode(s):
2     out = []
3     i = 0
4     while i < len(s):
5         j = i
6         while j < len(s) and s[j] == s[i]:
7             j += 1
8             run = j - i
9             out.append(s[i] + str(run) if run > 1 else s[i])
10        i = j
11    return ''.join(out)
12
13 r = encode("aaabccdddaae")
14 print(r, len(r), r[::-2])

```

- (A) a3bc2d4ae 9 ea4dc
- (B) a3bc2d4ae 9 e42ba
- (C) a3b1c2d4a1e1 12 1a1d2b3
- (D) a3bc2d5e 8 e5c3

Q.13 [MSQ · one or more correct | 2 marks | Queues — circular array implementation]

A circular queue of capacity 5 is implemented as follows. Which of the following statements about the state after the program finishes is/are TRUE?

Python

```

1 Q = [None] * 5
2 front = rear = size = 0
3
4 def enq(x):
5     global rear, size
6     Q[rear] = x
7     rear = (rear + 1) % 5
8     size += 1
9
10 def deq():
11     global front, size
12     x = Q[front]
13     front = (front + 1) % 5
14     size -= 1
15     return x
16
17 for x in [3, 6, 9, 12]:
18     enq(x)
19 deq(); deq()
20 for x in [15, 18, 21]:
21     enq(x)
22 s = deq() + deq()
23 enq(s)
24 print(front, rear, size, Q)

```

- (A) size is 3
- (B) The elements logically in the queue, from front to rear, are 15, 18, 21, 21
- (C) The list Q contains 12 at index 3, and 12 is one of the elements logically in the queue
- (D) front is 4 and rear is 3

Q.14 [MSQ · one or more correct | 2 marks | Arrays — rotation by reversal]

Consider the following Python functions. Which of the following statements is/are TRUE? (Each statement starts from a fresh list $a = [1, 2, 3, 4, 5, 6, 7]$; a *swap* is one execution of the tuple-assignment line in *rev*.)

Python

```
1 def rev(a, i, j):
2     while i < j:
3         a[i], a[j] = a[j], a[i]
4         i += 1
5         j -= 1
6
7 def mystery(a, k):
8     n = len(a)
9     k %= n
10    rev(a, 0, n - 1)
11    rev(a, 0, k - 1)
12    rev(a, k, n - 1)
```

- (A) *mystery(a, 10)* performs exactly 6 swaps
- (B) *mystery(a, 7)* leaves *a* unchanged and performs no swaps
- (C) After *mystery(a, -2)*, *a* is $[6, 7, 1, 2, 3, 4, 5]$
- (D) After *mystery(a, 10)*, *a* is $[5, 6, 7, 1, 2, 3, 4]$

Q.15 [MCQ · single correct | 2 marks | Merge sort — counting comparisons]

Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ and the remaining elements, sort each recursively, then merge) is applied to $[38, 12, 55, 7, 41, 29, 63, 18]$.

The merge step compares the front elements of the two runs and stops comparing as soon as one run is exhausted (the rest is copied without comparisons). The total number of element comparisons made by all merges is

A =	38	12	55	7	41	29	63	18
	0	1	2	3	4	5	6	7

- (A) 12
- (B) 16
- (C) 17
- (D) 24

Q.16 [NAT · numerical answer | 2 marks | Bubble sort — early termination]

The following bubble sort is applied to $A = [4, 1, 6, 2, 8, 3, 9]$. The total number of times the comparison $A[j] > A[j + 1]$ is evaluated is _____.

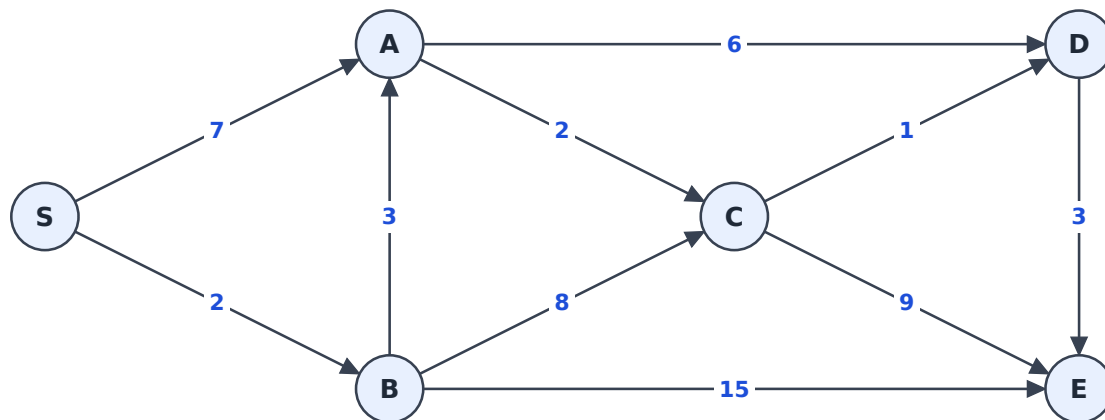
Python

```
1 def bubble(A):
2     n = len(A)
3     for i in range(n - 1):
4         swapped = False
5         for j in range(n - 1 - i):
6             if A[j] > A[j + 1]:
7                 A[j], A[j + 1] = A[j + 1], A[j]
8                 swapped = True
9         if not swapped:
10            break
```

Answer: _____ (enter a numerical value)

Q.17 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

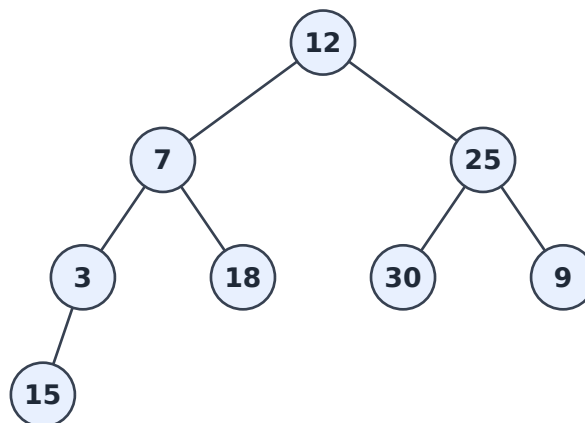
Dijkstra's algorithm is run from source S on the weighted directed graph below. The sum of the shortest-path distances from S to all six vertices (including $d(S) = 0$) is _____.



Answer: _____ (enter a numerical value)

Q.18 [MSQ · one or more correct | 2 marks | Heaps — bottom-up build-heap]

The array $H = [12, 7, 25, 3, 18, 30, 9, 15]$ (0-indexed, children of i at $2i+1$ and $2i+2$) is converted into a **max-heap** using the bottom-up build-heap procedure (sift-down applied to $i = 3, 2, 1, 0$). A *swap* is one parent-child exchange during sift-down. Which of the following statements is/are TRUE?



H viewed as a complete binary tree (before build-heap)

- (A) In the built heap, 12 is stored in a leaf
- (B) The resulting heap array is $[30, 18, 25, 15, 7, 12, 9, 3]$
- (C) After one DELETE-MAX on the built heap (last element moved to the root, then sift-down), the array is $[25, 18, 9, 15, 7, 12, 3]$
- (D) Build-heap performs exactly 5 swaps

Q.19 [MCQ · single correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition procedure below is called once on the whole array $A = [14, 3, 22, 9, 17, 5, 11]$ (pivot = last element). The array after the call returns is

Python

```
1 def partition(A, lo, hi):
2     p = A[hi]
3     i = lo - 1
4     for j in range(lo, hi):
5         if A[j] <= p:
6             i += 1
7             A[i], A[j] = A[j], A[i]
8     A[i + 1], A[hi] = A[hi], A[i + 1]
9     return i + 1
```



- (A) [3, 9, 5, 11, 17, 22, 14]
 (B) [3, 9, 5, 11, 14, 17, 22]
 (C) [5, 3, 9, 11, 17, 22, 14]
 (D) [3, 9, 5, 11, 22, 17, 14]

Q.20 [NAT · numerical answer | 2 marks | Linked lists — recursive pairwise swap]

Consider the following Python program. The value printed is _____.

Python

```
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def build(xs):
6     head = None
7     for x in reversed(xs):
8         head = Node(x, head)
9     return head
10
11 def f(h):
12     if h is None or h.nxt is None:
13         return h
14     second = h.nxt
15     h.nxt = f(second.nxt)
16     second.nxt = h
17     return second
18
19 h = f(build([2, 7, 1, 8, 2, 8, 1]))
20 out = 0
21 while h:
22     out = out * 10 + h.v
23     h = h.nxt
24 print(out)
```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 01

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — string slicing with negative steps
2	NAT	1	17	Python — aliasing, += versus +
3	MCQ	1	(B)	Stacks — operation trace
4	MCQ	1	(A)	Linked lists — pointer manipulation
5	NAT	1	185	Binary search — probe trace
6	MSQ	1	(A), (B), (D)	Insertion sort — shifts and comparisons
7	NAT	1	5	Hashing — linear probing collisions
8	MCQ	1	(D)	Binary search trees — traversal
9	MCQ	1	(D)	Python — floor division and modulo with negatives
10	MSQ	1	(C), (D)	Graphs — degrees, BFS and DFS
11	NAT	2	40	Python — nested lists and shallow copies
12	MCQ	2	(B)	Python — strings, loops and slicing
13	MSQ	2	(B), (D)	Queues — circular array implementation
14	MSQ	2	(A), (D)	Arrays — rotation by reversal
15	MCQ	2	(C)	Merge sort — counting comparisons
16	NAT	2	18	Bubble sort — early termination
17	NAT	2	33	Shortest paths — Dijkstra
18	MSQ	2	(A), (B), (D)	Heaps — bottom-up build-heap
19	MCQ	2	(A)	Quicksort — Lomuto partition
20	NAT	2	7281821	Linked lists — recursive pairwise swap

Detailed Solutions — Mock Test 01

Q.1 [MCQ · 1 mark · Python — string slicing with negative steps]

Answer: (C)

A slice `s[start:stop:step]` begins at `start`, moves by `step`, and stops *before* reaching `stop`. Negative indices are first converted by adding `len(s)`.

Indices of "DATASCIENCE" (length 11): D0 A1 T2 A3 S4 C5 I6 E7 N8 C9 E10.

- `s[-2:-9:-3]`: `start = 11 - 2 = 9`, `stop = 11 - 9 = 2` (exclusive), `step -3`. Visited indices 9, 6, 3 → **C, I, A**. The next index would be 0, which is past the stop (2), so the result is **CIA**.
- `s[:4]`: indices 0, 4, 8 → **D, S, N** → **DSN** (index 12 is out of range).

Output: **CIA DSN**.

- (A) starts at index 10 (**E**) — that would be `s[-1:...]`.
- (B) wrongly includes index 2 — the stop index is never included.
- (D) appends **E** as if index 12 wrapped around; slices never wrap.

Trap: with a negative step, the stop bound is still exclusive; convert negative indices to positive ones before tracing.

Q.2 [NAT · 1 mark · Python — aliasing, += versus +]

Answer: 17

Key idea: `b = a` creates an **alias** (two names, one list); `a[:]` creates a **shallow copy**; `b += [4]` mutates the list in place (it calls `list.__iadd__`), while `a = a + [6]` builds a **new** list and rebinds only the name `a`.

- After line 3: `a, b` → list L1 = [1, 2, 3]; `c` → L2 = [1, 2, 3].
- `b += [4]`: L1 becomes [1, 2, 3, 4] (seen through both `a` and `b`).
- `c.append(5)`: L2 = [1, 2, 3, 5].
- `a = a + [6]`: new list L3 = [1, 2, 3, 4, 6]; `a` now names L3, `b` still names L1.
- `b.extend(c)`: L1 = [1, 2, 3, 4, 1, 2, 3, 5] (length 8).

Lengths: `len(a) = 5`, `len(b) = 8`, `len(c) = 4`, `sum = 17`.

Trap: treating `a = a + [6]` like `+=` gives `len(b) = 9` and the answer 18; treating `c = a[:]` as an alias makes `c` grow too.

Q.3 [MCQ · 1 mark · Stacks — operation trace]

Answer: (B)

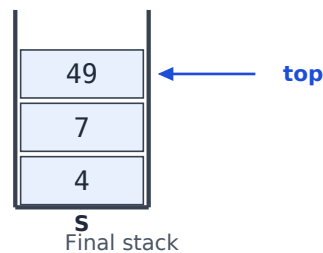
Simulate carefully, remembering that `TOP()` does **not** remove anything.

- Start: [4, 9, 2].
- Op 1: `POP() = 2`, push 6 → [4, 9, 6].
- Op 2: `POP() = 6`, `POP() = 9`, push 15 → [4, 15].
- Op 3: push 8 → [4, 15, 8].
- Op 4: `t = 8`; `POP() = 15`; push `15 - 8 = 7` → [4, 7].
- Op 5: `TOP() = 7` twice (no removal), push 49 → [4, 7, 49].

Option analysis:

- (A) treats `TOP() × TOP()` as `7 + 7` or as `2 × TOP()`; wrong.
- (B) 4, 7, 49 — matches the trace. **Correct.**
- (C) computes `t - POP() = 8 - 15 = -7` in step 4 (operand order reversed).
- (D) forgets that op 4 pops two elements and pushes one.

Tip: for non-commutative operations (`-`, `÷`) write down which popped value is the left operand; it is the classic source of errors in postfix evaluation too.

**Q.4 [MCQ · 1 mark · Linked lists — pointer manipulation]****Answer: (A)**

Building by inserting at the head in the order 4, 6, 1, 8, 3 yields $3 \rightarrow 8 \rightarrow 1 \rightarrow 6 \rightarrow 4$ (the reverse of the insertion order), as in the figure.

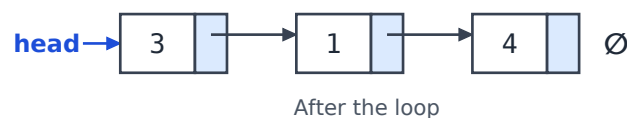
The loop *bypasses* the node after *p* and then jumps to the node that follows it:

- $p = 3$: $p.next$ (8) and $p.next.next$ (1) exist $\rightarrow 3.next = 1$; $p = 1$.
- $p = 1$: $p.next$ (6) and $p.next.next$ (4) exist $\rightarrow 1.next = 4$; $p = 4$.
- $p = 4$: $p.next$ is None $\rightarrow$ loop ends.

The list is now $3 \rightarrow 1 \rightarrow 4$; output **[3, 1, 4]** $\rightarrow$ (A). Every node at an odd position (2nd, 4th, ...) has been unlinked.

- (B) keeps the first three nodes — that is truncation, not alternate deletion.
- (C) lists the deleted nodes.
- (D) assumes only one bypass happens.

Trap: forgetting that head insertion reverses the order of the input loop.

**Q.5 [NAT · 1 mark · Binary search — probe trace]****Answer: 185**

Binary search halves the live interval $[lo, hi]$ each step; an unsuccessful search stops when the interval becomes empty.

- $lo = 0, hi = 12 \rightarrow mid = 6, A[6] = 30 < 47 \rightarrow lo = 7$.
- $lo = 7, hi = 12 \rightarrow mid = 9, A[9] = 45 < 47 \rightarrow lo = 10$.
- $lo = 10, hi = 12 \rightarrow mid = 11, A[11] = 58 > 47 \rightarrow hi = 10$.
- $lo = 10, hi = 10 \rightarrow mid = 10, A[10] = 52 > 47 \rightarrow hi = 9$.
- $lo = 10 > hi = 9 \rightarrow$ stop (not found).

Probed elements: 30, 45, 58, 52. Sum = $30 + 45 + 58 + 52 = \mathbf{185}$.

Note that $4 = \lfloor \log_2 13 \rfloor + 1$ probes is the maximum for $n = 13$, as expected for a key that falls between two array elements deep in the search tree.

Trap: using $mid = \lfloor (lo+hi)/2 \rfloor$ or stopping when $lo == hi$ gives a different set of probes (e.g. missing the final probe of 52).

Q.6 [MSQ · 1 mark · Insertion sort — shifts and comparisons]**Answer: (A), (B), (D)**

In insertion sort each shift removes exactly one inversion, so $\#shifts = \#inversions$. Comparisons = shifts + one extra comparison for every iteration that stops on a failing test (instead of running off the left end).

- $i = 1$ (key 2): $5 > 2$ shift; reached index 0 $\rightarrow$ 1 comparison, 1 shift $\rightarrow [2, 5, 9, 1, 7, 3]$.
- $i = 2$ (key 9): $5 > 9$ false $\rightarrow$ 1 comparison, 0 shifts.
- $i = 3$ (key 1): 9, 5, 2 all shift; reached index 0 $\rightarrow$ 3 comparisons, 3 shifts $\rightarrow [1, 2, 5, 9, 7, 3]$.
- $i = 4$ (key 7): 9 shifts, $5 > 7$ false $\rightarrow$ 2 comparisons, 1 shift $\rightarrow [1, 2, 5, 7, 9, 3]$.
- $i = 5$ (key 3): 9, 7, 5 shift, $2 > 3$ false $\rightarrow$ 4 comparisons, 3 shifts $\rightarrow$ sorted.

Totals: shifts = $1 + 0 + 3 + 1 + 3 = 8$, comparisons = $1 + 1 + 3 + 2 + 4 = 11$.

- (A) TRUE — also equals the inversion count $3 + 1 + 3 + 1 = 8$.
- (B) TRUE.
- (C) FALSE — 9 is shifted at $i = 3$, $i = 4$ and $i = 5$, i.e. three times.
- (D) TRUE — see the state after $i = 3$.

Tip: counting inversions is the fastest way to get the number of shifts.

Q.7 [NAT · 1 mark · Hashing — linear probing collisions]

Answer: 5

Compute the home slots first: $h(4) = 13 \bmod 11 = 2$, $h(15) = 46 \bmod 11 = 2$, $h(26) = 79 \bmod 11 = 2$, $h(9) = 28 \bmod 11 = 6$, $h(7) = 22 \bmod 11 = 0$, $h(20) = 61 \bmod 11 = 6$, $h(2) = 7$.

Keys that differ by a multiple of 11 (4, 15, 26) always share a home slot here, since $3 \cdot 11 \equiv 0 \pmod{11}$.

- 4 $\rightarrow$ slot 2 (0 collisions).
- 15 $\rightarrow$ 2 occupied $\rightarrow$ slot 3 (1).
- 26 $\rightarrow$ 2, 3 occupied $\rightarrow$ slot 4 (2).
- 9 $\rightarrow$ slot 6 (0).
- 7 $\rightarrow$ slot 0 (0).
- 20 $\rightarrow$ 6 occupied $\rightarrow$ slot 7 (1).
- 2 $\rightarrow$ home 7, now occupied by 20 $\rightarrow$ slot 8 (1).

Total collisions = $1 + 2 + 1 + 1 = 5$.

Trap: key 2 has home slot 7, which was free originally but has been taken by 20 through probing — secondary effects of primary clustering are easy to miss.

0	7
1	
2	4
3	15
4	26
5	
6	9
7	20
8	2
9	
10	

Final table

Q.8 [MCQ · 1 mark · Binary search trees — traversal]**Answer: (D)**

Each insertion walks from the root, going left for smaller keys and right for larger ones. The resulting tree is shown in the solution figure: 40 has children 25 and 61; 25 has 18 and 33; 33 has 29 and 36; 61 has 52 and 70; 52 has right child 55.

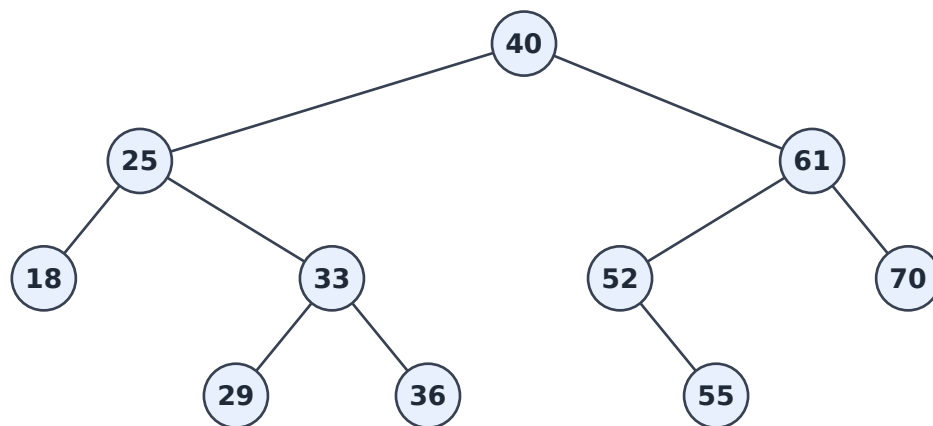
Post-order = left subtree, right subtree, root:

- Left subtree of 40: $\text{post}(18) = 18$; $\text{post}(33\text{-subtree}) = 29, 36, 33$; then $25 \rightarrow 18, 29, 36, 33, 25$.
- Right subtree of 40: $\text{post}(52\text{-subtree}) = 55, 52$; $\text{post}(70) = 70$; then $61 \rightarrow 55, 52, 70, 61$.
- Finally 40.

Result: 18, 29, 36, 33, 25, 55, 52, 70, 61, 40 $\rightarrow$ (D).

- (A) visits 18 after the 33-subtree and puts 52 before its child 55.
- (B) is the in-order (sorted) sequence.
- (C) swaps 29 and 36 — the left child must come before the right child.
- (D) correct.

Tip: in a post-order listing every node appears after all its descendants; use that to eliminate options quickly.



BST after all insertions

Q.9 [MCQ · 1 mark · Python — floor division and modulo with negatives]**Answer: (D)**

Python's `//` is **floor** division (rounds toward $-\infty$) and `%` is defined so that $a == (a // b) * b + a \% b$; hence the remainder takes the sign of the **divisor**. `int()` on a float truncates toward 0.

- $-17 // 5 = \lfloor -3.4 \rfloor = -4$.
- $-17 \% 5 = -17 - (-4)(5) = 3$.
- $17 // -5 = \lfloor -3.4 \rfloor = -4$.
- $17 \% -5 = 17 - (-4)(-5) = -3$.
- $\text{int}(-17 / 5) = \text{int}(-3.4) = -3$.

Output: -4 3 -4 -3 -3 $\rightarrow$ (D).

- (A) is what C/Java would print (truncating division).
- (B) mixes the two conventions.
- (C) gives the remainder the sign of the dividend for $17 \% -5$, and floors in `int()`.

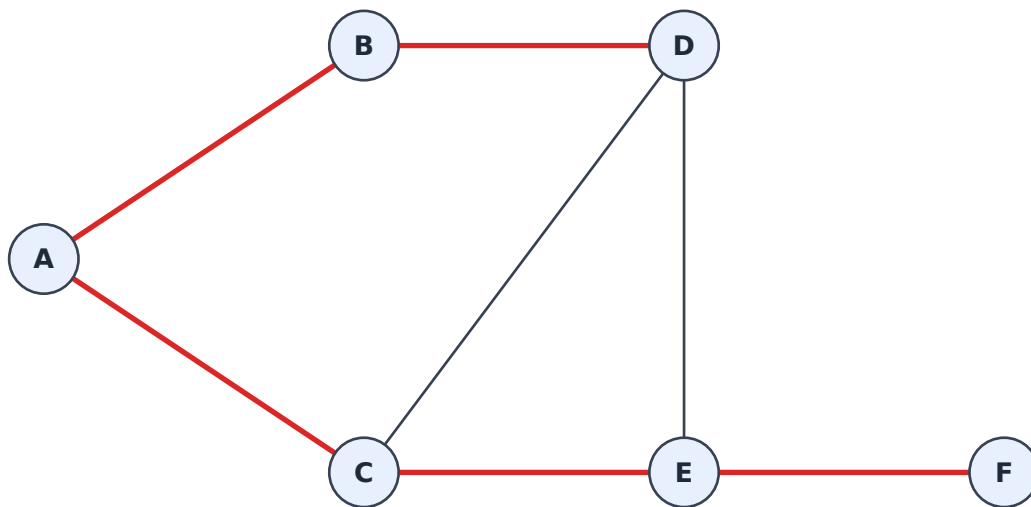
Trap: `int(x / y)` and `x // y` differ for negative operands.

Q.10 [MSQ · 1 mark · Graphs — degrees, BFS and DFS]**Answer: (C), (D)**

Degrees: A 2, B 2, C 3, D 3, E 3, F 1 (sum 14 = 2×7 edges).

- (A) DFS: A → B (first neighbour) → D (B's only unvisited neighbour) → D's neighbours in order B, C, E: C is unvisited → C → C's unvisited neighbour E → E → F. Order A, B, D, C, E, F. The stated order skips C at D. **FALSE.**
- (B) C-D-E is a triangle (odd cycle), so G is not bipartite. **FALSE.**
- (C) BFS from A: level 1 = {B, C}, level 2 = {D, E} (D from B, E from C), level 3 = {F}. **TRUE.**
- (D) Odd-degree vertices: C, D, E, F → 4. **TRUE** (always an even number by the handshaking lemma).

Trap: in (A), at vertex D the alphabetically first unvisited neighbour is C, not E.



BFS tree from A (highlighted)

Q.11 [NAT · 2 marks · Python — nested lists and shallow copies]**Answer: 40**

`[[0] * 3] * 3` makes an outer list holding **three references to the same inner list**. `[0] * 3` itself is fine because integers are immutable, but replicating the outer list copies only the reference.

- `grid[0][1] = 5`: the single shared row becomes `[0, 5, 0]`; every `grid[i]` shows it.
- `rows = [row[:] for row in grid]`: three *independent* copies, each `[0, 5, 0]` (taken **now**, before later changes).
- `rows[1][2] = 7`: only the second copy changes → `rows = [[0,5,0], [0,5,7], [0,5,0]]`.
- `grid[2][0] += 1`: the shared row becomes `[1, 5, 0]`; `rows` is unaffected.

Sums:

- `grid`: three references to `[1, 5, 0]` → $3 \times 6 = 18$.
- `rows`: $5 + 12 + 5 = 22$.

Total = $18 + 22 = 40$.

Common wrong answers:

- 18 — treating `grid` as three independent rows (`grid = [[0,5,0],[0,0,0],[1,0,0]]`, sum 6; `rows` sum $5 + 7 + 0 = 12$);
- 43 — assuming the copies in `rows` also see the later `+= 1` ($6 + 13 + 6 = 25$ for `rows`).

Trap: list replication `*` on a list of lists aliases rows; a slice copy `row[:]` is a snapshot at the moment it is taken. The idiom `[[0] * 3 for _ in range(3)]` creates independent rows.

Q.12 [MCQ · 2 marks · Python — strings, loops and slicing]

Answer: (B)

`encode` is run-length encoding: for each maximal run of equal characters it emits the character, followed by the run length only when the run is longer than 1.

Runs in `aaabccdddaae`: aaa (3), b (1), cc (2), dddd (4), a (1), e (1).

Encoded pieces: `a3, b, c2, d4, a, e` → `r = a3bc2d4ae`, `len(r) = 9`.

Indexing `r`: `a0 31 b2 c3 24 d5 46 a7 e8`. `r[::-2]` starts at the last index (8) and takes every second character backwards: indices 8, 6, 4, 2, 0 → `e, 4, 2, b, a` → `e42ba`.

Option analysis:

- (A) `ea4dc` takes indices 8, 7, 6, 5, 3 — irregular; with step `-2` starting at index 8 only the even indices 8, 6, 4, 2, 0 are taken.
- (B) `a3bc2d4ae 9 e42ba` — correct.
- (C) appends counts of 1 as well; the conditional expression suppresses them.
- (D) merges the final `a` into the `d` run; a run stops at the first different character.

Trap: the conditional expression `X if cond else Y` binds looser than `+`, so `s[i] + str(run) if run > 1 else s[i]` means `(s[i] + str(run)) if run > 1 else s[i]`.

Q.13 [MSQ · 2 marks · Queues — circular array implementation]

Answer: (B), (D)

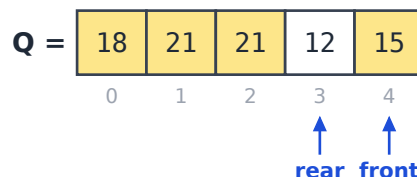
In a circular queue, `front` points at the oldest element and `rear` at the next free slot; both advance modulo the capacity. Dequeueing does **not** erase the slot.

- `enq 3, 6, 9, 12` → `Q = [3, 6, 9, 12, None]`, `front 0`, `rear 4`, `size 4`.
- `deq twice (3, 6)` → `front 2`, `size 2`.
- `enq 15` → `Q[4]`, `rear 0`; `enq 18` → `Q[0]`, `rear 1`; `enq 21` → `Q[1]`, `rear 2`; `size 5 (full)`.
- `s = deq() + deq() = Q[2] + Q[3] = 9 + 12 = 21`; `front 4`, `size 3`.
- `enq 21` → `Q[2] = 21`, `rear 3`, `size 4`.

Final: `front 4`, `rear 3`, `size 4`, `Q = [18, 21, 21, 12, 15]`. Logical contents from `front`: `Q[4]`, `Q[0]`, `Q[1]`, `Q[2] = 15, 18, 21, 21`.

- (A) FALSE — `size` is 4.
- (B) TRUE.
- (C) FALSE — 12 is still physically present at index 3 (a stale value), but index 3 is outside the live range `front ... rear-1` (indices 4, 0, 1, 2).
- (D) TRUE.

Trap: reading the raw array instead of walking from `front` for `size` elements.



Final array (live slots highlighted)

Q.14 [MSQ · 2 marks · Arrays — rotation by reversal]**Answer: (A), (D)**

Reversing the whole array, then reversing the first k and the last $n - k$ elements, **rotates the array right by k** in place.

(D)/(A): $k = 10 \% 7 = 3$.

- $\text{rev}(0, 6)$: [7, 6, 5, 4, 3, 2, 1] — 3 swaps.
- $\text{rev}(0, 2)$: [5, 6, 7, 4, 3, 2, 1] — 1 swap.
- $\text{rev}(3, 6)$: [5, 6, 7, 1, 2, 3, 4] — 2 swaps.

Result is the right-rotation by 3 and $\text{swaps} = 3 + 1 + 2 = 6$. Both **TRUE**.

(C): Python's `%` with a positive modulus is non-negative: $-2 \% 7 = 5$. So the call rotates right by 5 (= left by 2) giving [3, 4, 5, 6, 7, 1, 2]. The listed array is the right-rotation by 2. **FALSE**.

(B): $k = 0$. $\text{rev}(0, 6)$ reverses (3 swaps), $\text{rev}(0, -1)$ does nothing ($i > j$), and $\text{rev}(0, 6)$ reverses back (3 swaps). The array is unchanged but **6 swaps** occur. **FALSE**.

Trap: assuming $-2 \% 7$ is -2 (C/Java behaviour) or that $k = 0$ short-circuits.

Q.15 [MCQ · 2 marks · Merge sort — counting comparisons]**Answer: (C)**

Merging runs of lengths p and q needs between $\min(p, q)$ and $p + q - 1$ comparisons; the exact count depends on when one run is exhausted.

Level 1 (pairs), 1 comparison each: [38]+[12], [55]+[7], [41]+[29], [63]+[18] → 4 comparisons; runs [12, 38], [7, 55], [29, 41], [18, 63].

Level 2:

- [12, 38] + [7, 55]: 12 vs 7 → 7; 12 vs 55 → 12; 38 vs 55 → 38; left run exhausted, copy 55. 3 comparisons.
- [29, 41] + [18, 63]: 29 vs 18 → 18; 29 vs 63 → 29; 41 vs 63 → 41; copy 63. 3 comparisons.

Level 3: [7, 12, 38, 55] + [18, 29, 41, 63]: 7|18, 12|18, 38|18, 38|29, 38|41, 55|41, 55|63 → 7 comparisons, then 63 is copied.

Total = 4 + 3 + 3 + 7 = **17** → (C).

- (A) 12 is the best-case count ($n/2 \cdot \log_2 n = 4 \cdot 3$).
- (B) 16 forgets one comparison in the final merge.
- (D) $24 = n \log_2 n$ is only an upper-bound estimate; the worst case for $n = 8$ is 17 as well (each merge using $p + q - 1$), so this input happens to be a worst case.

Tip: worst-case comparisons for $n = 2^k$ is $n \cdot k - n + 1 = 8 \cdot 3 - 8 + 1 = 17$.

Runs after each merge level

	0	1	2	3	4	5	6	7
start	38	12	55	7	41	29	63	18
level 1	12	38	7	55	29	41	18	63
level 2	7	12	38	55	18	29	41	63
level 3	7	12	18	29	38	41	55	63

Q.16 [NAT · 2 marks · Bubble sort — early termination]**Answer: 18**

Each pass i compares $n - 1 - i$ adjacent pairs; the flag stops the algorithm after the first pass that makes **no** swap (that pass's comparisons still count).

$n = 7$.

- Pass 0 (6 comparisons): swaps (4,1), (6,2), (8,3) $\rightarrow$ [1, 4, 2, 6, 3, 8, 9].
- Pass 1 (5 comparisons): swaps (4,2), (6,3) $\rightarrow$ [1, 2, 4, 3, 6, 8, 9].
- Pass 2 (4 comparisons): swap (4,3) $\rightarrow$ [1, 2, 3, 4, 6, 8, 9].
- Pass 3 (3 comparisons): no swap $\rightarrow$ stop.

Comparisons = $6 + 5 + 4 + 3 = 18$. (Swaps = 6, equal to the number of inversions of the input.)

Without the flag the algorithm would make $6 + 5 + 4 + 3 + 2 + 1 = 21$ comparisons. The number of passes needed equals 1 + the largest distance any element must move **left** (element 3 moves from index 5 to index 2, i.e. 3 places, so 3 swapping passes + 1 verification pass).

Trap: forgetting to count the final, swap-free pass (answer 15).

Array after each pass							
	0	1	2	3	4	5	6
input	4	1	6	2	8	3	9
pass 0	1	4	2	6	3	8	9
pass 1	1	2	4	3	6	8	9
pass 2	1	2	3	4	6	8	9
pass 3	1	2	3	4	6	8	9

Q.17 [NAT · 2 marks · Shortest paths — Dijkstra]

Answer: 33

Dijkstra repeatedly extracts the unfinished vertex with the smallest tentative distance and relaxes its out-edges; with non-negative weights each extracted distance is final.

- Init: $S = 0$, rest ∞ .
- Extract S (0): $A = 7$, $B = 2$.
- Extract B (2): $A = \min(7, 2 + 3) = 5$, $C = 10$, $E = 17$.
- Extract A (5): $C = \min(10, 5 + 2) = 7$, $D = 11$.
- Extract C (7): $D = \min(11, 7 + 1) = 8$, $E = \min(17, 16) = 16$.
- Extract D (8): $E = \min(16, 8 + 3) = 11$.
- Extract E (11).

Distances: S 0, A 5, B 2, C 7, D 8, E 11. Sum = $0 + 5 + 2 + 7 + 8 + 11 = 33$.

Shortest-path tree: $S \rightarrow B$, $B \rightarrow A$, $A \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$ — a single chain, so the path to E is $S \rightarrow B \rightarrow A \rightarrow C \rightarrow D \rightarrow E$ with cost $2 + 3 + 2 + 1 + 3 = 11$.

Trap: stopping relaxation early and keeping $E = 16$ (via $C \rightarrow E$) or $E = 17$ (via $B \rightarrow E$); a direct edge with few hops is not necessarily shortest.

Tentative distances after each extraction

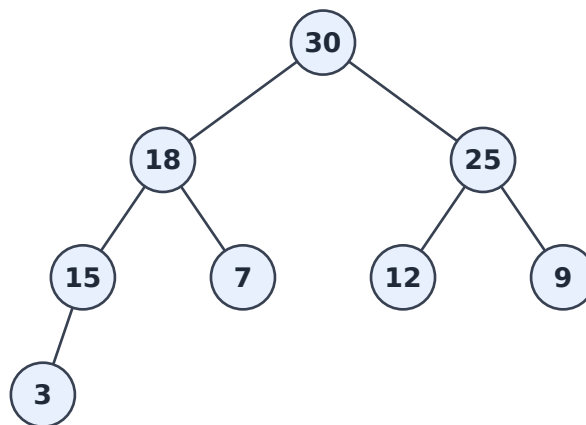
	S	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
S	0	7	2	∞	∞	∞
B	0	5	2	10	∞	17
A	0	5	2	7	11	17
C	0	5	2	7	8	16
D	0	5	2	7	8	11

Q.18 [MSQ · 2 marks · Heaps — bottom-up build-heap]**Answer: (A), (B), (D)**

Bottom-up build-heap sifts down every internal node, from the last internal node (index $\lfloor n/2 \rfloor - 1 = 3$) back to the root. Sift-down swaps with the **larger** child while that child is larger.

- $i = 3$ (3): child 15 $\rightarrow$ swap. [12, 7, 25, 15, 18, 30, 9, 3] (1 swap)
- $i = 2$ (25): children 30, 9 $\rightarrow$ swap with 30. [12, 7, 30, 15, 18, 25, 9, 3] (2)
- $i = 1$ (7): children 15, 18 $\rightarrow$ swap with 18; index 4 has no children. [12, 18, 30, 15, 7, 25, 9, 3] (3)
- $i = 0$ (12): children 18, 30 $\rightarrow$ swap with 30 (4); at index 2 children 25, 9 $\rightarrow$ swap with 25 (5). [30, 18, 25, 15, 7, 12, 9, 3]
- (A) TRUE — 12 is at index 5; indices 4 ... 7 are leaves in an 8-element heap.
- (B) TRUE.
- (C) FALSE. Move 3 to the root: [3, 18, 25, 15, 7, 12, 9]; 3 swaps with 25 (larger child), then at index 2 children 12, 9 $\rightarrow$ swap with 12: [25, 18, 12, 15, 7, 3, 9]. The option swaps with the wrong child.
- (D) TRUE — 5 swaps.

Trap: sift-down must continue after the first swap (the 12 at the root sinks two levels).



Max-heap after build-heap

Q.19 [MCQ · 2 marks · Quicksort — Lomuto partition]**Answer: (A)**

Lomuto keeps the invariant $A[\text{lo} \dots i] \leq \text{pivot} < A[i+1 \dots j-1]$. Whenever $A[j] \leq \text{pivot}$, i advances and $A[i] \leftrightarrow A[j]$. Finally the pivot is swapped into position $i + 1$.

Pivot $p = 11$, $i = -1$.

- $j = 0$ (14): > 11 , nothing.
- $j = 1$ (3): $i = 0$, swap $A[0] \leftrightarrow A[1] \rightarrow [3, 14, 22, 9, 17, 5, 11]$.
- $j = 2$ (22): nothing.
- $j = 3$ (9): $i = 1$, swap $A[1] \leftrightarrow A[3] \rightarrow [3, 9, 22, 14, 17, 5, 11]$.
- $j = 4$ (17): nothing.
- $j = 5$ (5): $i = 2$, swap $A[2] \leftrightarrow A[5] \rightarrow [3, 9, 5, 14, 17, 22, 11]$.
- Final: swap $A[3] \leftrightarrow A[6] \rightarrow [3, 9, 5, 11, 17, 22, 14]$; returns 3.
- (A) correct.
- (B) has a sorted right part — partition does not sort the sides.
- (C) moves 5 to the front; Lomuto keeps the elements $\leq \text{pivot}$ in the order they are discovered: 3, 9, 5.
- (D) puts 22 before 17; but only 14 (displaced by the final pivot swap) moves — to index 6 — while 17 and 22 keep their positions 4 and 5.

Tip: the element originally at index $i + 1$ is the one sent to the pivot's old slot.

Q.20 [NAT · 2 marks · Linked lists — recursive pairwise swap]

Answer: 7281821

build inserts at the head while iterating over the **reversed** input, so the list is $2 \rightarrow 7 \rightarrow 1 \rightarrow 8 \rightarrow 2 \rightarrow 8 \rightarrow 1$ (same order as the Python list).

f swaps nodes in adjacent pairs recursively: it makes the second node of a pair the new head of that pair, points the first node to the result of processing the rest, and returns the second node. An odd node left at the end is returned unchanged.

Trace of the pairs:

- $(2, 7) \rightarrow 7, 2$
- $(1, 8) \rightarrow 8, 1$
- $(2, 8) \rightarrow 8, 2$
- (1) alone $\rightarrow 1$

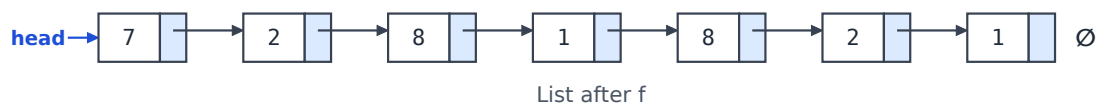
Resulting list: $7 \rightarrow 2 \rightarrow 8 \rightarrow 1 \rightarrow 8 \rightarrow 2 \rightarrow 1$.

The final loop reads the values as decimal digits: $\text{out} = 7281821$.

Recursion depth is $\lceil n/2 \rceil = 4$ calls with a non-trivial body, plus the base call, so the function runs in $\Theta(n)$ time and $\Theta(n)$ stack space.

Traps:

- Forgetting that **reversed** in **build** cancels the head-insertion reversal (would give the list $1 \rightarrow 8 \rightarrow 2 \rightarrow 8 \rightarrow 1 \rightarrow 7 \rightarrow 2$).
- Swapping only values rather than relinking — the result is the same here, but **second.next = h** must come **after** **h.next** is set, otherwise the rest of the list is lost.



Mock Test 02

Stacks & Expression Evaluation

Difficulty: Moderate

Focus areas: stacks, infix/postfix/prefix, balanced brackets, stack permutations

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing with negative steps]

Consider the following Python program. What is printed?

Python

```
1 s = "POSTFIX"
2 t = s[::-2] + s[1:-1:2]
3 print(t, len(s[5:2]))
```

- (A) XFSP0SF 0
- (B) XFSP0TI 0
- (C) XFSP0TI 3
- (D) XIFTSOP 0

Q.2 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated using an operand stack. All operators are binary; ^ denotes exponentiation. The tokens are separated by spaces.

5 9 3 - 2 ^ * 7 4 2 * - +

The value of the expression is ____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — infix to postfix conversion]

The infix expression below is converted to postfix using the standard operator-stack algorithm. Precedence: ^ (highest, **right**-associative) > *, / (left-associative) > +, - (lowest, left-associative).

a - b / (c + d) * e ^ f ^ g + h

The postfix form is

- (A) a b c d + / e f ^ g ^ * - h +
- (B) a b c d + e f g ^ ^ * / - h +
- (C) a b c d + / e f g ^ ^ * - h +
- (D) a b c d + / e f g ^ ^ * h + -

Q.4 [MSQ · one or more correct | 1 mark | Stacks — stack permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack **in this order**. Pops may be interleaved with the pushes arbitrarily, and every popped value is written to the output. Which of the following output sequences is/are possible?

- (A) 2, 4, 3, 6, 1, 5
- (B) 4, 5, 3, 6, 2, 1
- (C) 1, 5, 2, 4, 3, 6
- (D) 3, 2, 5, 6, 4, 1

Q.5 [NAT · numerical answer | 1 mark | Python + stacks — bracket matching]

Consider the following Python program. The value printed is _____.

Python

```
1 def check(s):
2     pairs = {'(': ')', '[': ']', '{': '}'
3     st, peak = [], 0
4     for i, ch in enumerate(s):
5         if ch in '([{':
6             st.append(ch)
7             peak = max(peak, len(st))
8         elif not st or st.pop() != pairs[ch]:
9             return -i
10    return peak if not st else len(st)
11
12 print(check("{}[()]([{}])") + check("([])"))
```

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Queues — circular array]

A circular queue is stored in an array $Q[0..5]$ (size 6). **front** is the index of the first element and **rear** is the index of the next free slot; the queue is **full** when $(\text{rear} + 1) \bmod 6 == \text{front}$ (one slot is always left empty). Initially $\text{front} = \text{rear} = 0$. An enqueue on a full queue is rejected and changes nothing.

The operations performed are: 5 enqueues, then 3 dequeues, then 4 enqueues. The final values of (front, rear) are

- (A) (3, 2)
- (B) (3, 3)
- (C) (3, 1)
- (D) (4, 2)

Q.7 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

The keys 23, 46, 14, 57, 35, 68, 9, 80, 19, 41 are inserted in this order into an initially empty hash table with 11 slots using $h(k) = k \bmod 11$ and separate chaining (new keys are appended at the end of their chain). Assume each key is equally likely to be searched for. The average number of key comparisons in a **successful** search, rounded off to two decimal places, is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Linked lists — slow/fast pointers in Python]

Consider the following Python program. What is printed?

Python

```

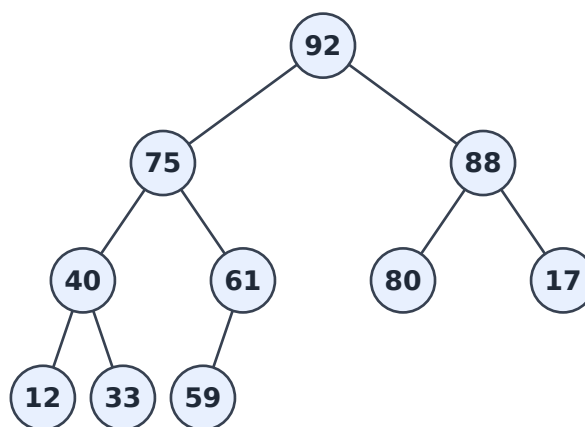
1 class N:
2     def __init__(s, v, nx=None):
3         s.v, s.nx = v, nx
4
5 h = None
6 for v in [4, 7, 1, 9, 3]:
7     h = N(v, h)
8     p = q = h
9     while q and q.nx:
10        p, q = p.nx, q.nx.nx
11    p.nx = p.nx.nx
12    out = []
13    while h:
14        out.append(h.v)
15        h = h.nx
16    print(out)

```

- (A) [4, 7, 9, 3]
- (B) [3, 9, 1, 4]
- (C) [3, 9, 7, 4]
- (D) [3, 9, 1, 7]

Q.9 [MSQ · one or more correct | 1 mark | Heaps — max-heap operations]

The array below (0-based) stores a binary max-heap. Which of the following statements is/are TRUE? (Each statement refers to the original heap.)



Max-heap H = [92, 75, 88, 40, 61, 80, 17, 12, 33, 59]

- (A) Inserting 65 (append, then sift-up) leaves 65 at index 1
- (B) Inserting 90 (append, then sift-up) performs exactly 2 swaps
- (C) The third-largest key, 80, is a child of the root
- (D) After one delete-max (last element moved to the root, then sift-down), the array is [88, 75, 80, 40, 61, 59, 17, 12, 33]

Q.10 [NAT · numerical answer | 1 mark | Binary search — counting probes]

Consider the following Python program. The value printed is _____.

Python

```
1 def bs(a, x):
2     lo, hi, c = 0, len(a) - 1, 0
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         c += 1
6         if a[mid] == x:
7             return c
8         elif a[mid] < x:
9             lo = mid + 1
10        else:
11            hi = mid - 1
12    return -c
13
14 A = [3, 8, 14, 19, 23, 27, 31, 36, 42, 47, 50, 58, 63]
15 print(bs(A, 47) - bs(A, 30))
```

Answer: _____ (enter a numerical value)

Q.11 [NAT · numerical answer | 2 marks | Stacks — counting stack permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack in this order, with pops interleaved arbitrarily; popped values form the output sequence, and at the end the stack is emptied. The number of distinct output sequences whose **first** element is 4 is _____.

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Python + stacks — prefix evaluation]

The function below evaluates a prefix (Polish) expression by scanning it from right to left. What does the program print?

Python

```
1 def ev(tokens):
2     st = []
3     for t in reversed(tokens):
4         if t.isdigit():
5             st.append(int(t))
6         else:
7             a, b = st.pop(), st.pop()
8             st.append({'+': a + b, '-': a - b,
9                       '*': a * b, '/': a // b}[t])
10    return st
11
12 print(ev("+ * 3 / - 2 9 4 - 8 5".split()))
```

- (A) [0]
- (B) -3
- (C) [3]
- (D) [-3]

Q.13 [MSQ · one or more correct | 2 marks | Stacks — infix to postfix, operator-stack trace]

The infix expression $a * (b + c * d) - e / f ^ g$ is converted to postfix using an operator stack (precedence $^ > *, / > +, -$; $^$ right-associative, others left-associative; $($ is pushed onto the same stack). Which of the following statements is/are TRUE?

- (A) With $a = 2, b = 3, c = 4, d = 5, e = 64, f = 2, g = 3$ the expression evaluates to 38
- (B) The postfix expression is $a b c d * + * e f g ^ / -$
- (C) The maximum number of symbols (operators and $($) on the stack at any moment is 4
- (D) When $-$ is scanned, exactly two operators are popped to the output

Q.14 [NAT · numerical answer | 2 marks | Python — mutable default argument in a Stack class]

Consider the following Python program. The value printed is _____.

Python

```

1 class Stack:
2     def __init__(self, items=[]):
3         self.items = items
4     def push(self, x):
5         self.items.append(x)
6     def pop(self):
7         return self.items.pop()
8
9 s1 = Stack()
10 s2 = Stack()
11 s3 = Stack([9])
12 for k in range(1, 5):
13     (s1 if k % 2 else s2).push(k * k)
14 s3.push(s2.pop())
15 print(sum(s1.items) + len(s2.items) * 10 + s3.pop())

```

Answer: _____ (enter a numerical value)

Q.15 [MSQ · one or more correct | 2 marks | Elementary sorting — selection, insertion, bubble]

The array $A = [5, 1, 4, 2, 8, 3]$ is to be sorted in ascending order. Consider:

- **Selection sort:** in pass i ($i = 0..n-2$) the minimum of $A[i..n-1]$ is found and swapped into position i ; a swap is counted only if the minimum is not already at i .
- **Insertion sort:** a *shift* is one move of an element one position to the right.
- **Bubble sort** with early termination: a pass that makes no swap ends the algorithm (that pass is counted).

Which of the following statements is/are TRUE?

- (A) After 2 passes of selection sort the array is $[1, 2, 4, 5, 8, 3]$
- (B) Bubble sort performs exactly 4 passes
- (C) Selection sort performs exactly 4 swaps
- (D) Insertion sort performs exactly 7 shifts

Q.16 [MCQ · single correct | 2 marks | Quicksort — recursion stack depth]

Consider the quicksort below (Lomuto partition, last element as pivot). It is called as `qs(a, 0, 7, 1)` on `a = [1, 2, 3, 4, 5, 6, 7, 8]`. What does the final `print` show?

Python

```
1 def part(a, lo, hi):
2     x, i = a[hi], lo - 1
3     for j in range(lo, hi):
4         if a[j] <= x:
5             i += 1
6             a[i], a[j] = a[j], a[i]
7     a[i + 1], a[hi] = a[hi], a[i + 1]
8     return i + 1
9
10 deepest = calls = 0
11 def qs(a, lo, hi, d):
12     global deepest, calls
13     calls += 1
14     deepest = max(deepest, d)
15     if lo < hi:
16         p = part(a, lo, hi)
17         qs(a, lo, p - 1, d + 1)
18         qs(a, p + 1, hi, d + 1)
19
20 a = [1, 2, 3, 4, 5, 6, 7, 8]
21 qs(a, 0, 7, 1)
22 print(deepest, calls)
```

- (A) 7 15
- (B) 8 8
- (C) 8 15
- (D) 8 16

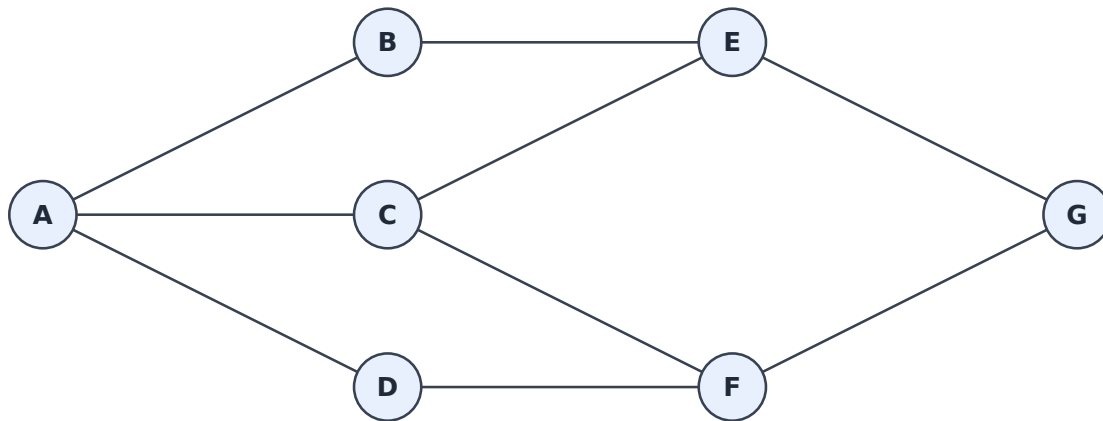
Q.17 [MSQ · one or more correct | 2 marks | Graph traversal — iterative DFS with an explicit stack]

The following iterative DFS is run on the undirected graph shown, starting at A:

Initially `stack = [A]` and `visited = []`.

Repeat while the stack is non-empty: pop `u`; if `u` is already visited, skip it; otherwise append `u` to `visited` and push every **unvisited** neighbour of `u` onto the stack in **alphabetical order** (so the alphabetically last neighbour ends on top).

Which of the following statements is/are TRUE?



Undirected graph

- (A) The visiting order equals that of recursive DFS that explores neighbours alphabetically
- (B) Vertex C is pushed onto the stack exactly 3 times
- (C) The maximum number of entries on the stack at any moment is 5
- (D) The visiting order is A, D, F, G, E, C, B

Q.18 [NAT · numerical answer | 2 marks | Queue using two stacks — operation count]

A queue is implemented with two stacks `S1` and `S2`. **Enqueue(x)** pushes `x` onto `S1`. **Dequeue()**: if `S2` is empty, pop every element of `S1` and push it onto `S2`; then pop `S2`. Each individual push or pop on `S1` or `S2` costs 1 (so moving one element costs 2).

Starting with both stacks empty, the following 12 operations are performed (E = enqueue, D = dequeue):

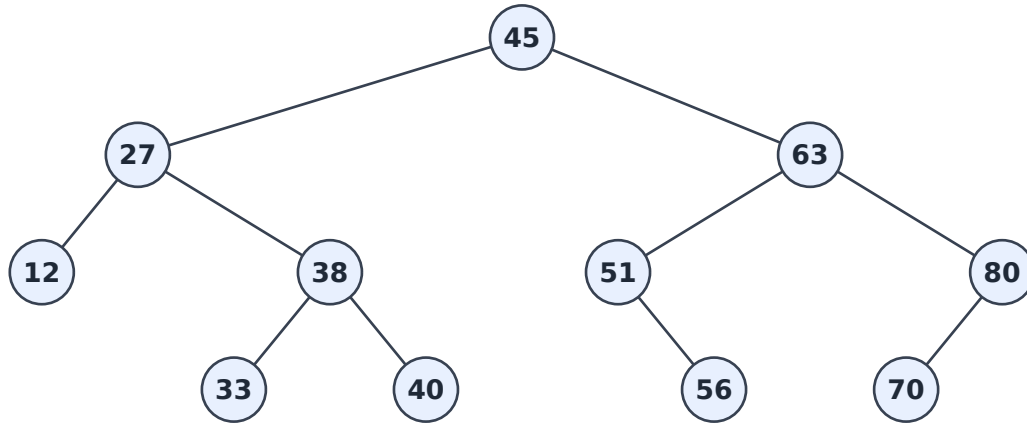
E E E D E E D D D E D D

The total cost of the sequence is _____.

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Binary search trees — deletion]

The keys 45, 27, 63, 12, 38, 51, 80, 33, 40, 70, 56 are inserted in this order into an initially empty binary search tree (shown below). Then 45 is deleted and afterwards 27 is deleted. A node with two children is deleted by copying its **in-order successor** into it and deleting the successor. The post-order traversal of the final tree is

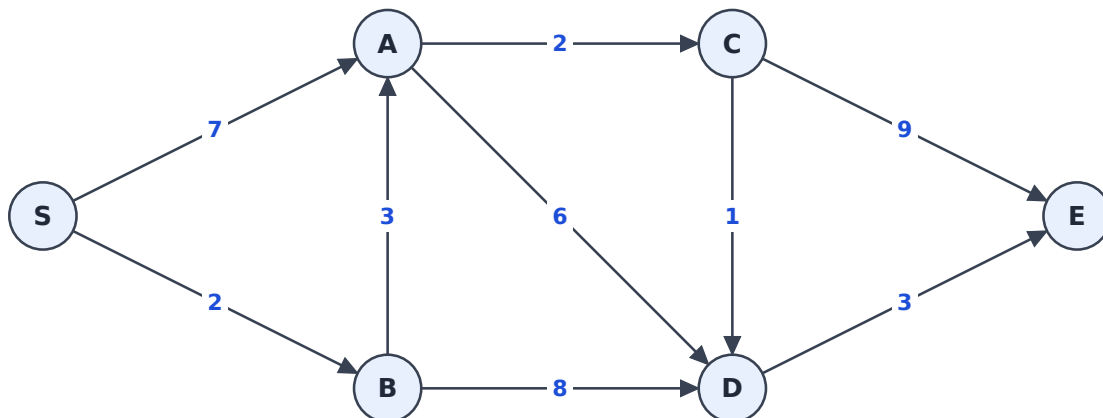


BST after all insertions

- (A) 12, 40, 38, 33, 56, 70, 80, 63, 51
 (B) 40, 12, 38, 33, 56, 70, 80, 63, 51
 (C) 12, 33, 40, 38, 56, 70, 80, 63, 51
 (D) 12, 38, 40, 33, 70, 56, 80, 63, 51

Q.20 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from source S on the weighted directed graph shown. Let $d(v)$ be the final shortest-path distance of v . The value of $d(A) + d(B) + d(C) + d(D) + d(E)$ is _____.



Weighted digraph

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 02

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — string slicing with negative steps
2	NAT	1	179	Stacks — postfix evaluation
3	MCQ	1	(C)	Stacks — infix to postfix conversion
4	MSQ	1	(B), (D)	Stacks — stack permutations
5	NAT	1	2	Python + stacks — bracket matching
6	MCQ	1	(A)	Queues — circular array
7	NAT	1	1.79 to 1.81	Hashing — separate chaining
8	MCQ	1	(B)	Linked lists — slow/fast pointers in Python
9	MSQ	1	(B), (D)	Heaps — max-heap operations
10	NAT	1	6	Binary search — counting probes
11	NAT	2	14	Stacks — counting stack permutations
12	MCQ	2	(D)	Python + stacks — prefix evaluation
13	MSQ	2	(A), (B), (C)	Stacks — infix to postfix, operator-stack trace
14	NAT	2	60	Python — mutable default argument in a Stack class
15	MSQ	2	(A), (B), (D)	Elementary sorting — selection, insertion, bubble
16	MCQ	2	(C)	Quicksort — recursion stack depth
17	MSQ	2	(B), (C), (D)	Graph traversal — iterative DFS with an explicit stack
18	NAT	2	24	Queue using two stacks — operation count
19	MCQ	2	(A)	Binary search trees — deletion
20	NAT	2	33	Shortest paths — Dijkstra

Detailed Solutions — Mock Test 02

Q.1 [MCQ · 1 mark · Python — string slicing with negative steps]

Answer: (B)

Concept: a slice `s[start:stop:step]` walks from `start` towards `stop` in jumps of `step`; with a negative step and omitted bounds it starts at the last character. A slice whose direction cannot reach `stop` is simply empty — it never raises an error.

Indices of "POSTFIX": P0 O1 S2 T3 F4 I5 X6.

- `s[::-2]` → indices 6, 4, 2, 0 → X F S P = "XFSP".
- `s[1:-1:2]` → stop is index 6 (excluded), so indices 1, 3, 5 → "OTI".
- `t = "XFSP" + "OTI" = "XFSPOTI".`
- `s[5:2]` has the default step +1, but $5 > 2$, so the slice is empty and its length is 0.

Options:

- (A) re-uses the reversed pattern for the second slice; wrong.
- (B) XFSPOTI 0 — correct.
- (C) treats `s[5:2]` as if it went backwards and had 3 characters; a positive step never moves leftwards.
- (D) is the full reversal `s[::-1]` — forgets the step of 2.

Trap: `s[5:2]` is not `s[5:2:-1]`; the default step is +1.

Q.2 [NAT · 1 mark · Stacks — postfix evaluation]

Answer: 179

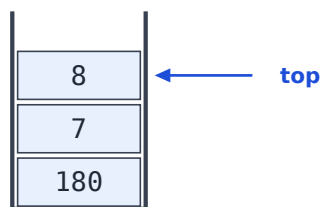
Concept: scan left to right; push operands; on an operator pop the top two values — the **first** pop is the *right* operand, the second pop is the *left* operand — and push the result.

Trace (stack shown bottom → top):

- 5 9 3 → [5, 9, 3]
- - → $9 - 3 = 6$ → [5, 6]
- 2 ^ → $6^2 = 36$ → [5, 36]
- * → $5 \times 36 = 180$ → [180]
- 7 4 2 → [180, 7, 4, 2]
- * → $4 \times 2 = 8$ → [180, 7, 8]
- - → $7 - 8 = -1$ → [180, -1]
- + → $180 + (-1) = 179$

The maximum stack depth reached is 4 (just after pushing the second 2).

Trap: popping in the wrong order turns 9 3 - into -6 and 7 8 - into +1, which gives $5 \times 36 + 1 = 181$ — a very common wrong answer. Always compute *second-pop op first-pop*.



Operand stack just before the first '-' after 7 4 2 *

Q.3 [MCQ · 1 mark · Stacks — infix to postfix conversion]**Answer: (C)**

Concept: an incoming operator pops every stacked operator of *higher* precedence, and also those of *equal* precedence when the incoming operator is left-associative. For a right-associative $^$, an equal $^$ is **not** popped.

Grouping: $^$ first: $e \wedge (f \wedge g)$. Then $/$ and $*$ left to right: $(b / (c+d)) * (e \wedge f \wedge g)$. Then $-$ and $+$ left to right: $(a - \text{that}) + h$.

Trace of key moments:

- a out; $-$ pushed; b out; $/$ pushed above $-$.
- $(c + d)$ emits $c d +$.
- $*$ arrives: pops $/$ (equal precedence, left-assoc) $\rightarrow$ output so far $a b c d + /$.
- $e, ^, f, ^$ (second $^$ does not pop the first), $g \rightarrow$ stack $- * ^ ^$.
- $+$ arrives: pops $^ ^ * - \rightarrow \dots e f g ^ ^ * -$; then h , end pops $+$.

Result: $a b c d + / e f g ^ ^ * - h +$.

Options: (A) treats $^$ as left-associative $((e \wedge f) \wedge g)$. (B) applies $*$ before $/$, breaking left-to-right order. (D) evaluates h before the subtraction, i.e. $a - (\dots + h)$. Only (C) is right.

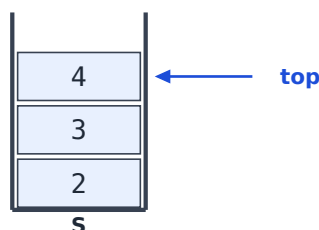
Tip: check associativity of $^$ first — it is the most-tested detail.

Q.4 [MSQ · 1 mark · Stacks — stack permutations]**Answer: (B), (D)**

Concept: simulate greedily — to output x , push every not-yet-pushed value up to x , then x must be on top. Equivalently, a sequence is impossible iff it contains a pattern $i < j < k$ output in the order $k \dots i \dots j$ (a '3-1-2' pattern).

- (A) after 2, 4, 3 the stack holds [1]; push 5, 6 and pop 6 $\rightarrow$ stack [1, 5]. Next we need 1 but 5 is on top. **Impossible** (pattern 6 ... 1 ... 5).
- (B) push 1-4 pop 4; push 5 pop 5; pop 3; push 6 pop 6; pop 2; pop 1. **Possible.**
- (C) pop 1 at once; push 2-5 pop 5 $\rightarrow$ stack [2, 3, 4]; we need 2 but 4 is on top. **Impossible** (pattern 5 ... 2 ... 4).
- (D) push 1,2,3 pop 3; pop 2; push 4,5 pop 5; push 6 pop 6; pop 4; pop 1. **Possible.**

Trap: do not check only the first few outputs; (A) looks fine until the last two pops.



(D) after outputting 1 and 5: 2 is buried under 3, 4

Q.5 [NAT · 1 mark · Python + stacks — bracket matching]**Answer: 2**

Concept: a stack checks bracket balance — push every opener, and each closer must pop the matching opener. This function returns the peak stack depth for a balanced string, and $-(\text{index of the first bad closer})$ on a mismatch.

First call `{[()()]}([{}])` — depth after each character:

- `{ 1 [ 2 ( 3 ) 2 ( 3 ) 2 ] 1 ( 2 [ 3 { 4 } 3 ] 2 ) 1 } 0`
- Every closer matches and the stack ends empty $\rightarrow$ returns peak = 4.

Second call `([ ])`:

- $i = 0$ (push, $i = 1$ [push.
- $i = 2$): pop gives [but `pairs['')']` is ($\rightarrow$ mismatch $\rightarrow$ returns -2 .

Printed value: $4 + (-2) = 2$.

Trap: `([]]` has equal counts of each bracket type, so a counter-based check would call it balanced; only a stack catches the crossing. Note also that `-i` uses the index of the offending character (2), not the number of characters read (3).

Q.6 [MCQ · 1 mark · Queues — circular array]

Answer: (A)

Concept: with one slot sacrificed, an array of size 6 holds at most 5 elements; both indices advance modulo 6.

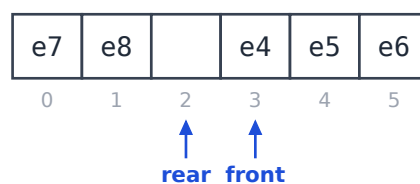
Trace:

- 5 enqueues: rear goes $0 \rightarrow 5$; size 5. Now $(5 + 1) \bmod 6 = 0 = \text{front} \rightarrow$ **full**.
- 3 dequeues: front goes $0 \rightarrow 3$; size 2.
- Enqueue #1: slot 5 used, rear = 0 (wrap-around); size 3.
- Enqueue #2: rear = 1; size 4.
- Enqueue #3: rear = 2; size 5. Now $(2 + 1) \bmod 6 = 3 = \text{front} \rightarrow$ **full**.
- Enqueue #4: rejected.

Final: front = 3, rear = 2 $\rightarrow$ option **(A)**.

Options: (B) accepts the 4th enqueue, ignoring the reserved empty slot. (C) loses one enqueue at the wrap-around. (D) performs an extra dequeue.

Tip: with this convention, $\text{size} = (\text{rear} - \text{front} + 6) \bmod 6 = (2 - 3 + 6) \bmod 6 = 5$.



Final state (e1..e3 dequeued; e9 rejected)

Q.7 [NAT · 1 mark · Hashing — separate chaining]

Answer: 1.79 to 1.81

Concept: in a chain of length L , the key in position p (1-based) costs p comparisons; the chain contributes $1 + 2 + \dots + L = L(L + 1)/2$.

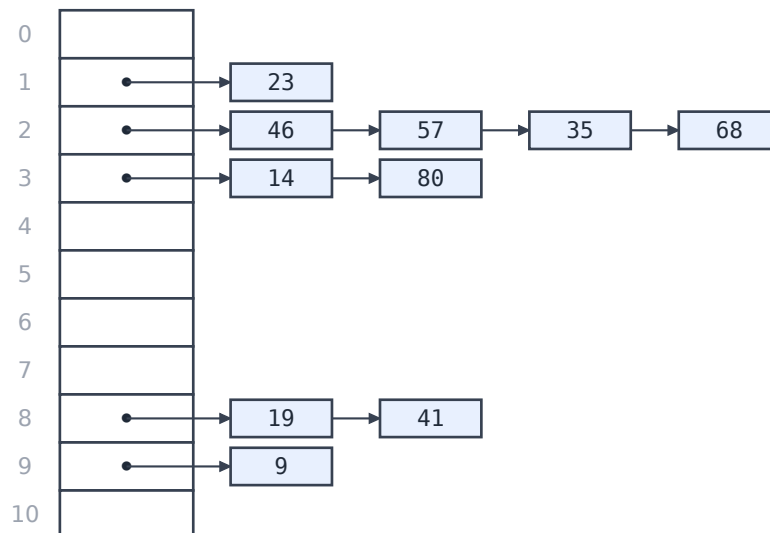
Home slots ($k \bmod 11$): $23 \rightarrow 1$, $46 \rightarrow 2$, $14 \rightarrow 3$, $57 \rightarrow 2$, $35 \rightarrow 2$, $68 \rightarrow 2$, $9 \rightarrow 9$, $80 \rightarrow 3$, $19 \rightarrow 8$, $41 \rightarrow 8$.

Chains:

- slot 1: 23 ($L = 1$) $\rightarrow 1$
- slot 2: $46 \rightarrow 57 \rightarrow 35 \rightarrow 68$ ($L = 4$) $\rightarrow 10$
- slot 3: $14 \rightarrow 80$ ($L = 2$) $\rightarrow 3$
- slot 8: $19 \rightarrow 41$ ($L = 2$) $\rightarrow 3$
- slot 9: 9 ($L = 1$) $\rightarrow 1$

Total = $1 + 10 + 3 + 3 + 1 = 18$ comparisons over 10 keys $\rightarrow$ average = **1.80**.

Trap: dividing by the table size (11) or using the load factor formula $1 + \alpha/2 = 1.45$ gives the *expected* cost for random keys, not the cost for this particular key set.



Final chained table

Q.8 [MCQ · 1 mark · Linked lists — slow/fast pointers in Python]**Answer: (B)**

Concept: $h = N(v, h)$ inserts at the **head**, so the list is built in reverse. The slow/fast pointer loop stops with p at the middle node; the program then unlinks the node *after* the middle.

- After the loop over $[4, 7, 1, 9, 3]$, the list is $3 \rightarrow 9 \rightarrow 1 \rightarrow 7 \rightarrow 4$.
- $p = q = 3$. Iteration 1: $p = 9, q = 1$. Iteration 2: $p = 1, q = 4$. Now $q.nx$ is None $\rightarrow$ stop.
- $p.nx = p.nx.nx$ makes 1 point to 4, skipping 7.
- Traversal prints $[3, 9, 1, 4]$.

Options:

- (A) assumes tail insertion (list $4, 7, 1, 9, 3$) and deletes 1.
- (C) deletes the middle node itself instead of its successor.
- (D) deletes the last node.

Trap: forgetting that prepending reverses the input order.



List after node 7 is unlinked

Q.9 [MSQ · 1 mark · Heaps — max-heap operations]**Answer: (B), (D)**

Concept: children of index i are $2i + 1$ and $2i + 2$; the parent is $\lfloor (i - 1)/2 \rfloor$.

- (A) 65 at index 10 swaps with 61 (index 4), then its parent 75 is larger $\rightarrow$ stops at **index 4. False.**
- (B) 90 goes to index 10; parent index 4 (61) $\rightarrow$ swap; parent index 1 (75) $\rightarrow$ swap; parent index 0 (92) is larger $\rightarrow$ stop. 2 swaps. **True.**
- (C) The root's children are 75 and 88. 80 is a child of 88, at index 5. **False.**
- (D) Move 59 to the root: $[59, 75, 88, 40, 61, 80, 17, 12, 33]$. Larger child 88 $\rightarrow$ swap $\rightarrow$ 59 at index 2; children 80, 17 $\rightarrow$ swap with 80 $\rightarrow$ 59 at index 5 (leaf). Result $[88, 75, 80, 40, 61, 59, 17, 12, 33]$. **True.**

Trap: the k -th largest element of a heap need not be at depth $k - 1$ or less in any fixed spot; for $k = 3$ it can be a child or a grandchild of the root.

Q.10 [NAT · 1 mark · Binary search — counting probes]**Answer: 6**

Concept: each loop iteration probes one middle element; **bs** returns the probe count for a hit and the *negated* probe count for a miss.

Array has 13 elements, indices 0..12.

Search 47:

- lo = 0, hi = 12, mid = 6 → A[6] = 31 < 47 → lo = 7 (c = 1)
- lo = 7, hi = 12, mid = 9 → A[9] = 47 → found, c = 2

Search 30 (absent):

- mid = 6 → 31 > 30 → hi = 5 (c = 1)
- lo = 0, hi = 5, mid = 2 → 14 < 30 → lo = 3 (c = 2)
- lo = 3, hi = 5, mid = 4 → 23 < 30 → lo = 5 (c = 3)
- lo = 5, hi = 5, mid = 5 → 27 < 30 → lo = 6 (c = 4); loop ends → returns **-4**

Printed: 2 - (-4) = 6.

Trap: missing the sign: 2 - 4 = -2 is the most common slip. Also note an unsuccessful search on 13 elements may take 3 or 4 probes ($\lceil \log_2 13 \rceil + 1 = 4$ at most).

3	8	14	19	23	27	31	36	42	47	50	58	63
0	1	2	3	4	5	6	7	8	9	10	11	12

↑
lo/hi end

Probes for x = 30: indices 6, 2, 4, 5

Q.11 [NAT · 2 marks · Stacks — counting stack permutations]**Answer: 14**

Concept: an output sequence is fixed by the sequence of push/pop operations, and distinct valid operation sequences give distinct outputs. So we count operation sequences.

Step 1 — forced prefix. To output 4 first, we must push 1, 2, 3, 4 and pop 4 immediately (any earlier pop would output something other than 4). The stack is now [1, 2, 3] (3 on top) and inputs 5, 6 remain.

Step 2 — count the rest. We still need 2 pushes (U) and 5 pops (D), in any order such that the stack height, starting at 3, never goes below 0.

- Unrestricted arrangements: $C(7, 2) = 21$.
- Bad paths touch height -1. Reflecting the part before the first touch maps them to paths from -5 to 0 in 7 steps, i.e. $U - D = 5$, $U + D = 7 \rightarrow U = 6$, $D = 1$: $C(7, 1) = 7$.
- Good paths: $21 - 7 = 14$.

Cross-check by cases: let k = number of pops before 5 is pushed (k = 0..3). The stack then holds 4 - k items, and 6 can be pushed after j = 0, 1, ..., 4 - k further pops: 5 - k ways. Total $5 + 4 + 3 + 2 = 14$.

Trap: answering Catalan(2) = 2 by thinking only about the order of 5 and 6, or answering $C_6 = 132$ (all stack permutations). The buried 1, 2, 3 must still come out in the order 3, 2, 1 but 5 and 6 can be interleaved among them.

Q.12 [MCQ · 2 marks · Python + stacks — prefix evaluation]**Answer: (D)**

Concept: for prefix evaluation scanned right-to-left, the **first** value popped is the *left* operand (the reverse of postfix). Here `a = first pop = left operand` — correct. Python's `//` is *floor* division, rounding towards $-\infty$.

Trace (tokens reversed: `5 8 - 4 9 2 - / 3 * +`), stack bottom $\rightarrow$ top:

- push 5, 8 $\rightarrow$ [5, 8]
- `-`: `a = 8, b = 5  $\rightarrow$  8 - 5 = 3  $\rightarrow$  [3]`
- push 4, 9, 2 $\rightarrow$ [3, 4, 9, 2]
- `-`: `a = 2, b = 9  $\rightarrow$  2 - 9 = -7  $\rightarrow$  [3, 4, -7]`
- `/`: `a = -7, b = 4  $\rightarrow$  -7 // 4 = -2 (floor of -1.75)  $\rightarrow$  [3, -2]`
- push 3 $\rightarrow$ [3, -2, 3]
- `*`: `a = 3, b = -2  $\rightarrow$  -6  $\rightarrow$  [3, -6]`
- `+`: `a = -6, b = 3  $\rightarrow$  -3  $\rightarrow$  [-3]`

`ev` returns the *list* `st`, so the output is `[-3]`.

(Note: the dict literal evaluates all four expressions each time, but no divisor is ever 0, so no exception arises.)

Options:

- (A) `[0]` uses truncation (`-7 / 4  $\rightarrow$  -1`, then `-3 + 3 = 0`) — C-style division, not Python.
- (B) `-3` forgets that a list is returned and printed.
- (C) `[3]` comes from a sign slip in `2 - 9`.

Trap: `//` with a negative operand — floor, not truncate.

Q.13 [MSQ · 2 marks · Stacks — infix to postfix, operator-stack trace]**Answer: (A), (B), (C)**

Concept: operators wait on the stack until a lower-or-equal precedence operator (or `)` or end of input) forces them out.

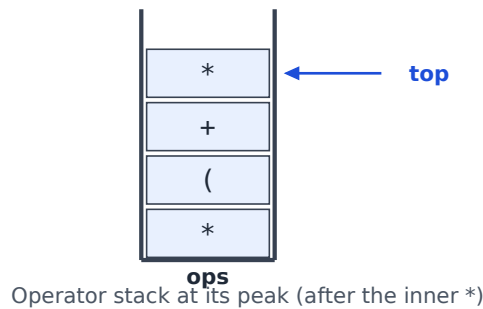
Trace (stack bottom $\rightarrow$ top, output so far):

- `a` $\rightarrow$ out: `a`
- `*` $\rightarrow$ [`*`]
- `(` $\rightarrow$ [`*`, `(`]
- `b` $\rightarrow$ out: `a b`
- `+` $\rightarrow$ [`*`, `(`, `+`]
- `c` $\rightarrow$ out: `a b c`
- `*` $\rightarrow$ higher than `+`, push $\rightarrow$ [`*`, `(`, `+`, `*`] (**size 4**, the maximum)
- `d` $\rightarrow$ out: `a b c d`
- `)` $\rightarrow$ pop `*`, `+`, discard `(` $\rightarrow$ out: `a b c d * +`; stack [`*`]
- `-` $\rightarrow$ pops `*` only (stack then empty) $\rightarrow$ push `-` $\rightarrow$ out: `a b c d * + *`
- `e`, `/` (push above `-`), `f`, `^` (push above `/`), `g` $\rightarrow$ stack [`-`, `/`, `^`]
- end $\rightarrow$ pop `^` `/` `-` $\rightarrow$ final `a b c d * + * e f g ^ / -`.

Verdicts:

- (A) $2 \times (3 + 4 \times 5) - 64 / 2^3 = 2 \times 23 - 64/8 = 46 - 8 = 38$. **True.**
- (B) **True** (see final output).
- (C) **True** — size 4 after the inner `*`; later the stack holds at most 3.
- (D) **False** — only one operator (`*`) is on the stack when `-` arrives.

Trap: in (D) people forget that `)` already flushed `+` and the inner `*`.

**Q.14 [NAT · 2 marks · Python — mutable default argument in a Stack class]****Answer: 60**

Concept: the default list `[]` in `__init__` is created **once** when the function is defined. `s1 = Stack()` and `s2 = Stack()` both store a reference to that same list, so they are two names for one stack. `s3` gets its own list `[9]`.

Trace:

- `k = 1` (odd) $\rightarrow$ `s1.push(1)` $\rightarrow$ shared list `[1]`
- `k = 2` (even) $\rightarrow$ `s2.push(4)` $\rightarrow$ shared list `[1, 4]`
- `k = 3` $\rightarrow$ `s1.push(9)` $\rightarrow$ `[1, 4, 9]`
- `k = 4` $\rightarrow$ `s2.push(16)` $\rightarrow$ `[1, 4, 9, 16]`
- `s2.pop()` removes 16 from the shared list $\rightarrow$ `[1, 4, 9]`; `s3` becomes `[9, 16]`.

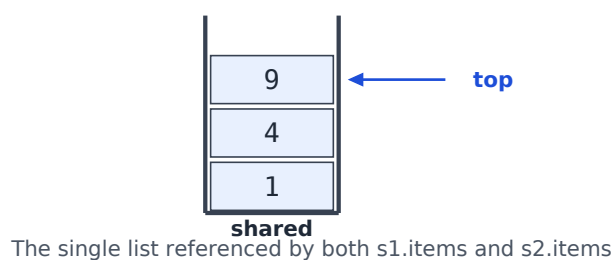
Final expression:

- `sum(s1.items)` = `1 + 4 + 9` = 14
- `len(s2.items) * 10` = `3 × 10` = 30 (same list!)
- `s3.pop()` = 16

Total = `14 + 30 + 16` = **60**.

What a naive reading gives: with independent lists, `s1 = [1, 9]` (sum 10), `s2 = [4]` after the pop (length 1 $\rightarrow$ 10) and `s3.pop()` = 16, total 36.

Trap/Tip: never use a mutable default; write `items=None` and `self.items = [] if items is None else items`.

**Q.15 [MSQ · 2 marks · Elementary sorting — selection, insertion, bubble]****Answer: (A), (B), (D)**

Concepts: insertion-sort shifts = number of inversions; selection sort swaps depend on where each minimum sits; bubble sort stops after the first swap-free pass.

Selection sort (see table): pass 1 swaps `5↔1`; pass 2 swaps `5↔2`; pass 3 swaps `4↔3`; pass 4 swaps `5↔4`; pass 5 swaps `8↔5`. Every pass swaps $\rightarrow$ **5 swaps**. (C) **False**. After 2 passes: `[1, 2, 4, 5, 8, 3]` $\rightarrow$ (A) **True**.

Insertion sort: inversions of `[5, 1, 4, 2, 8, 3]` are `(5,1)`, `(5,4)`, `(5,2)`, `(5,3)`, `(4,2)`, `(4,3)`, `(8,3)` = 7. Each shift removes exactly one inversion $\rightarrow$ **7 shifts**. (D) **True**.

Bubble sort:

- pass 1 → [1, 4, 2, 5, 3, 8]
- pass 2 → [1, 2, 4, 3, 5, 8]
- pass 3 → [1, 2, 3, 4, 5, 8]
- pass 4 → no swap → stop.

So **4 passes**; (B) **True**.

Trap: in (B), people stop at 3 because the array is already sorted after pass 3 — but the algorithm only *knows* that after a swap-free pass. In (C), 5 swaps happen because no element is ever already in its final selection position.

Selection sort — array after each pass

	0	1	2	3	4	5
start	5	1	4	2	8	3
pass 1	1	5	4	2	8	3
pass 2	1	2	4	5	8	3
pass 3	1	2	3	5	8	4
pass 4	1	2	3	4	8	5
pass 5	1	2	3	4	5	8

Q.16 [MCQ · 2 marks · Quicksort — recursion stack depth]

Answer: (C)

Concept: on already-sorted input, Lomuto with the last element as pivot always puts the pivot at **hi**, so the left part has size $n - 1$ and the right part is empty — the worst case, with recursion depth $\Theta(n)$ and running time $\Theta(n^2)$ ($T(n) = T(n - 1) + \Theta(n)$).

Depth: the subarray sizes along the left spine are 8, 7, 6, 5, 4, 3, 2, 1 at depths 1, 2, ..., 8. The size-1 call ($lo = hi$) still executes **calls += 1** and updates **deepest** before failing the **lo < hi** test. So **deepest = 8**.

Calls: every call with $lo < hi$ (sizes 8 down to 2: 7 calls) partitions and makes exactly 2 further calls. Counting the root: **calls = 1 + 2 × 7 = 15** (the 7 empty right-side calls and the one size-1 call are included).

Output: **8 15**.

Options:

- (A) **7 15** stops the depth at the last *partitioning* call (size 2) and forgets that the size-1 call also records its depth.
- (B) counts only the left-spine calls.
- (D) counts an extra call; a quicksort recursion tree is a full binary tree with 7 internal nodes and therefore exactly 8 leaves, 15 nodes.

Tip: recursing on the smaller side first (and looping on the larger) bounds the stack depth by $O(\log n)$ even in this worst case.

Q.17 [MSQ · 2 marks · Graph traversal — iterative DFS with an explicit stack]

Answer: (B), (C), (D)

Concept: pushing neighbours in alphabetical order makes the *last* one pop first, so the traversal goes 'reverse-alphabetical'. Because a vertex may be pushed several times (it is only checked when popped), stack entries can exceed $|V| - 1$ in general.

Trace (stack bottom → top after processing u):

- pop A → visit A; push B, C, D → [B, C, D]

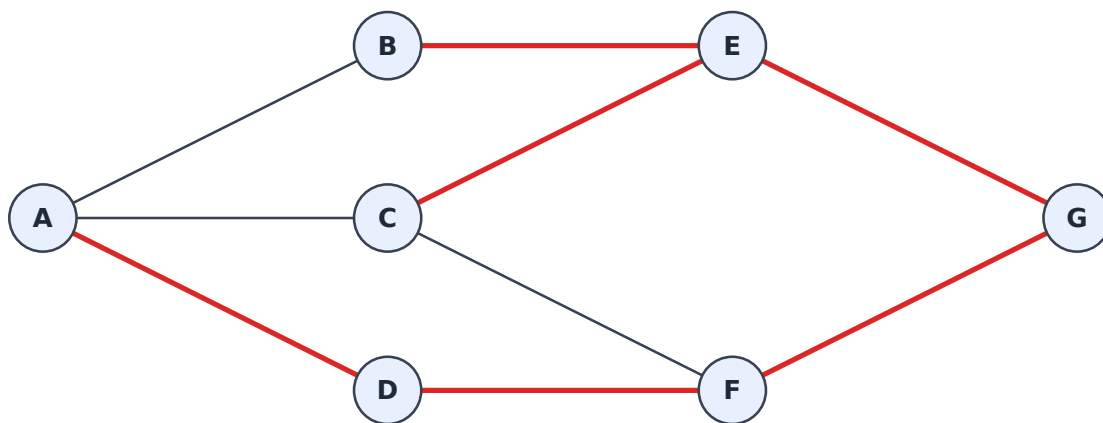
- pop D → visit D; push F → [B, C, F]
- pop F → visit F; unvisited nbrs C, G → [B, C, C, G]
- pop G → visit G; push E → [B, C, C, E]
- pop E → visit E; unvisited nbrs B, C → [B, C, C, B, C] (**5 entries**)
- pop C → visit C; no unvisited nbrs → [B, C, C, B]
- pop B → visit B → [B, C, C]; the remaining entries are popped and skipped.

Visiting order: **A, D, F, G, E, C, B.**

Verdicts:

- (A) **False** — recursive alphabetical DFS gives A, B, E, C, F, D, G.
- (B) C is pushed by A, by F and by E → 3 times. **True.**
- (C) **True** — peak is 5, right after E is processed.
- (D) **True.**

Trap: assuming an explicit-stack DFS automatically reproduces the recursive order; to do that you must push neighbours in *reverse* order.



DFS tree of the iterative traversal (vertex → the one that pushed it when it was visited)

Q.18 [NAT · 2 marks · Queue using two stacks — operation count]

Answer: 24

Concept: each element is pushed onto S1 once, moved to S2 at most once (cost 2) and popped from S2 once — at most 4 stack operations per element, so the amortised cost per queue operation is $O(1)$. Here we count exactly.

Trace (sizes of S1 / S2 after each operation):

- E, E, E: 3 pushes → cost 3; S1 = 3, S2 = 0
- D: S2 empty → move 3 elements (6) + pop (1) = 7 → total 10; S1 = 0, S2 = 2
- E, E: cost 2 → total 12; S1 = 2, S2 = 2
- D: pop S2 (1) → 13; S2 = 1
- D: pop S2 (1) → 14; S2 = 0
- D: move 2 (4) + pop (1) = 5 → 19; S1 = 0, S2 = 1
- E: 1 → 20; S1 = 1
- D: pop S2 (1) → 21; S2 = 0
- D: move 1 (2) + pop (1) = 3 → **24**

Check with the per-element bound: 6 elements × (1 push + 2 move + 1 pop) = 24 — every element was transferred exactly once, so the bound is attained.

Trap: counting a transfer as cost 1 per element (giving 18), or moving S1 into S2 when S2 is *not* empty, which would also break FIFO order.

Stack sizes and running cost

	op	S1	S2	cost	total
1	E	1	0	1	1
2	E	2	0	1	2
3	E	3	0	1	3
4	D	0	2	7	10
5	E	1	2	1	11
6	E	2	2	1	12
7	D	2	1	1	13
8	D	2	0	1	14
9	D	0	1	5	19
10	E	1	1	1	20
11	D	1	0	1	21
12	D	0	0	3	24

Q.19 [MCQ · 2 marks · Binary search trees — deletion]**Answer: (A)**

Concept: the in-order successor of a node with two children is the minimum of its right subtree; it has no left child, so removing it is easy (splice in its right child).

Delete 45: successor = min of right subtree 63 → 51 → 51 (no left child). Copy 51 into the root; remove 51 from its old place, its right child 56 takes its place as left child of 63.

Delete 27: two children; successor = min of right subtree 38 → 33. Copy 33 into 27's node; delete the leaf 33. Node 38 now has only the right child 40.

Final tree: root 51; left 33 (children 12, and 38 → right 40); right 63 (children 56 and 80 → left 70).

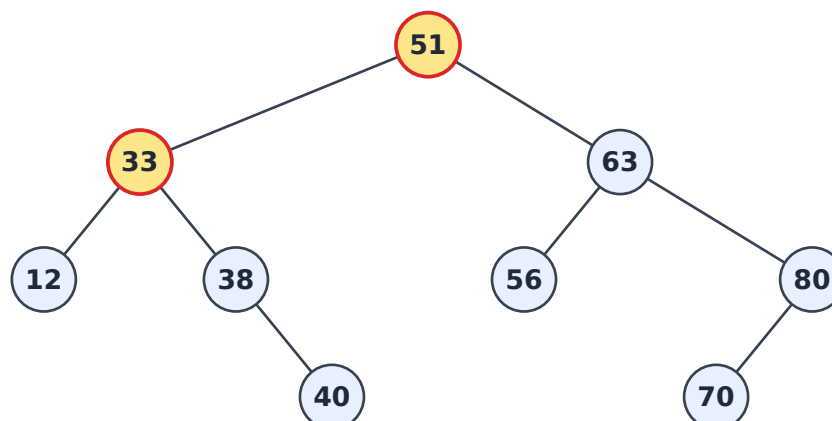
Post-order (left, right, root):

- left subtree of 33: 12; right subtree: 40, 38; then 33 → 12, 40, 38, 33
- right subtree: 56, then 70, 80, then 63 → 56, 70, 80, 63
- root 51

Result: **12, 40, 38, 33, 56, 70, 80, 63, 51** → (A).

Options: (C) puts 33 before 40 — that is not a valid post-order for this tree (38's subtree contains only 40). (B) lists 40 first as if it were the leftmost leaf. (D) swaps the order inside both subtrees.

Trap: using the in-order predecessor (40 for 45, 12 for 27) produces a different tree.



Final BST after deleting 45 and 27

Q.20 [NAT · 2 marks · Shortest paths — Dijkstra]**Answer: 33**

Concept: Dijkstra repeatedly extracts the unfinished vertex with the smallest tentative distance and relaxes its outgoing edges; with non-negative weights each extracted distance is final.

Trace (see table):

- Extract S (0): $A = 7$, $B = 2$.
- Extract B (2): $A = \min(7, 2 + 3) = 5$; $D = 2 + 8 = 10$.
- Extract A (5): $C = 5 + 2 = 7$; $D = \min(10, 5 + 6) = 10$ (unchanged).
- Extract C (7): $D = \min(10, 7 + 1) = 8$; $E = 7 + 9 = 16$.
- Extract D (8): $E = \min(16, 8 + 3) = 11$.
- Extract E (11).

Final: $d(A) = 5$, $d(B) = 2$, $d(C) = 7$, $d(D) = 8$, $d(E) = 11 \rightarrow \text{sum} = \mathbf{33}$.

Shortest-path tree: $S \rightarrow B$, $B \rightarrow A$, $A \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$ — a single chain, so every vertex's path goes through all earlier ones.

Trap: stopping at the first finite estimate ($A = 7$ via the direct edge, $D = 10$ via B) gives $7 + 2 + 9 + 10 + 13 = 41$. The direct edge is not the shortest when a detour through a cheap vertex exists.

Tentative distances after each extraction

	S	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
S	0	7	2	∞	∞	∞
B	0	5	2	∞	10	∞
A	0	5	2	7	10	∞
C	0	5	2	7	8	16
D	0	5	2	7	8	11

Mock Test 03

Queues, Deques & Circular Buffers

Difficulty: Moderate

Focus areas: queues, circular queue arithmetic, deque, queue via stacks

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — collections.deque]

Consider the following Python program. What is printed?

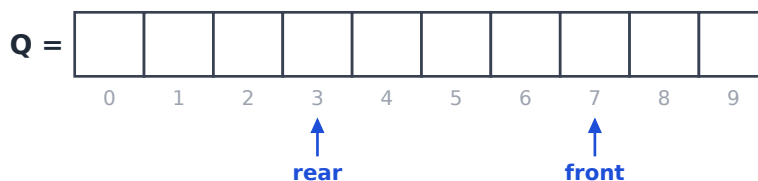
Python

```
1 from collections import deque
2 d = deque([3, 1, 4, 1, 5, 9])
3 d.rotate(2)
4 d.appendleft(d.pop())
5 d.rotate(-1)
6 d.extend([d.popleft(), d.pop()])
7 print(list(d))
```

- (A) [9, 3, 1, 4, 5, 1]
- (B) [9, 3, 1, 4, 1, 5]
- (C) [3, 1, 4, 1, 5, 9]
- (D) [1, 4, 1, 5, 9, 3]

Q.2 [NAT · numerical answer | 1 mark | Circular queue — occupancy arithmetic]

A circular queue is stored in an array $Q[0..9]$ (size 10). **front** holds the index of the first (oldest) element and **rear** holds the index of the last (newest) element. The queue is non-empty and the current pointer values are shown below. The number of elements currently in the queue is ____.



front = 7, rear = 3 (contents not shown)

Answer: _____ (enter a numerical value)

Q.3 [NAT · numerical answer | 1 mark | Python — stack-based postfix evaluation]

Consider the following Python program. The value printed is _____.

Python

```

1 def ev(expr):
2     st = []
3     for t in expr.split():
4         if t.isdigit():
5             st.append(int(t))
6         else:
7             b, a = st.pop(), st.pop()
8             if t == '-':
9                 st.append(a - b)
10            elif t == '/':
11                st.append(a // b)
12            else:
13                st.append(a * b)
14    return st.pop()
15
16 print(ev("3 8 - 2 / 4 6 * -"))

```

Answer: _____ (enter a numerical value)

Q.4 [MSQ · one or more correct | 1 mark | Python — list slicing]

Let `q = [10, 20, 30, 40, 50, 60]`. Which of the following statements is/are TRUE?

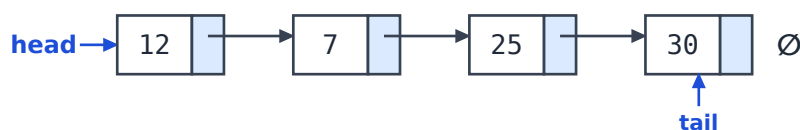
- (A) `q[-3:-1]` evaluates to `[40, 50]`
- (B) `q[4:1:-1]` evaluates to `[50, 40, 30, 20]`
- (C) `q[::-2]` evaluates to `[60, 40, 20]`
- (D) `q[-1:-4]` evaluates to `[60, 50, 40]`

Q.5 [MCQ · single correct | 1 mark | Linked-list queue]

A queue is implemented as a singly linked list with **head** (front) and **tail** (rear) pointers; **enqueue** inserts after **tail**, **dequeue** removes the node at **head** and returns its value. Starting from the queue shown, the following are executed:

`x = dequeue(); enqueue(x + 1); enqueue(dequeue()); dequeue(); enqueue(9)`

The contents of the queue from head to tail are



- (A) `7 → 30 → 13 → 9`
- (B) `30 → 13 → 7 → 9`
- (C) `25 → 30 → 13 → 9`
- (D) `30 → 7 → 13 → 9`

Q.6 [MCQ · single correct | 1 mark | Binary search — probe sequence]

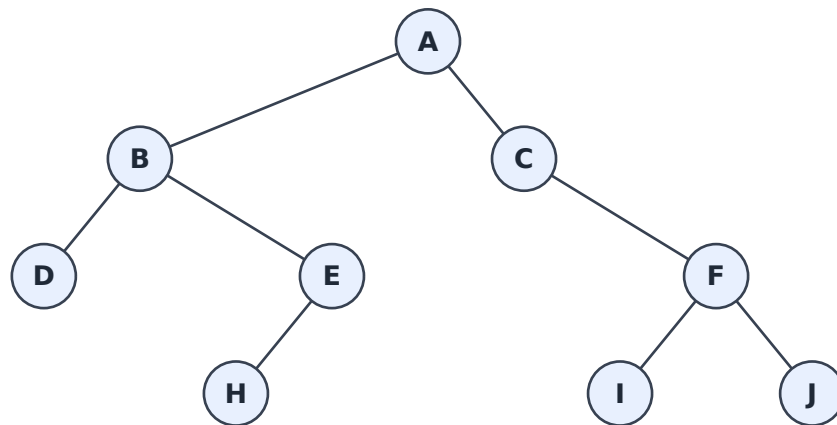
Iterative binary search ($lo = 0, hi = n - 1, mid = (lo + hi) // 2$; if $A[mid] == x$ stop; if $A[mid] < x$ then $lo = mid + 1$ else $hi = mid - 1$; loop while $lo \leq hi$) searches for $x = 37$ in the sorted array below. The sequence of array values compared with x is

A =	3	8	11	15	19	22	26	31	35	40	44	51	57	62	70
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

- (A) 31, 51, 40, 35, 37
- (B) 31, 44, 35, 40
- (C) 31, 51, 40, 35
- (D) 26, 51, 40, 35

Q.7 [MCQ · single correct | 1 mark | Level-order traversal with a queue]

Level-order traversal of the binary tree below is done with a queue: enqueue the root; repeatedly dequeue a node, visit it, then enqueue its left child and then its right child (if present). Immediately after node **E** has been dequeued and its children enqueued, the queue contents from front to rear are



Binary tree

- (A) H
- (B) H, F
- (C) F, H, I, J
- (D) F, H

Q.8 [MCQ · single correct | 1 mark | Hashing — separate chaining]

The keys 24, 17, 40, 11, 45, 61, 9, 52 are inserted in that order into an initially empty hash table of size 7 using $h(k) = k \bmod 7$ and separate chaining, where every new key is inserted at the **head** of its chain. The chain at slot 3, read from head to tail, is

- (A) 24 → 17 → 45 → 52
- (B) 52 → 45 → 17 → 24
- (C) 52 → 45 → 24 → 17
- (D) 45 → 52 → 17 → 24

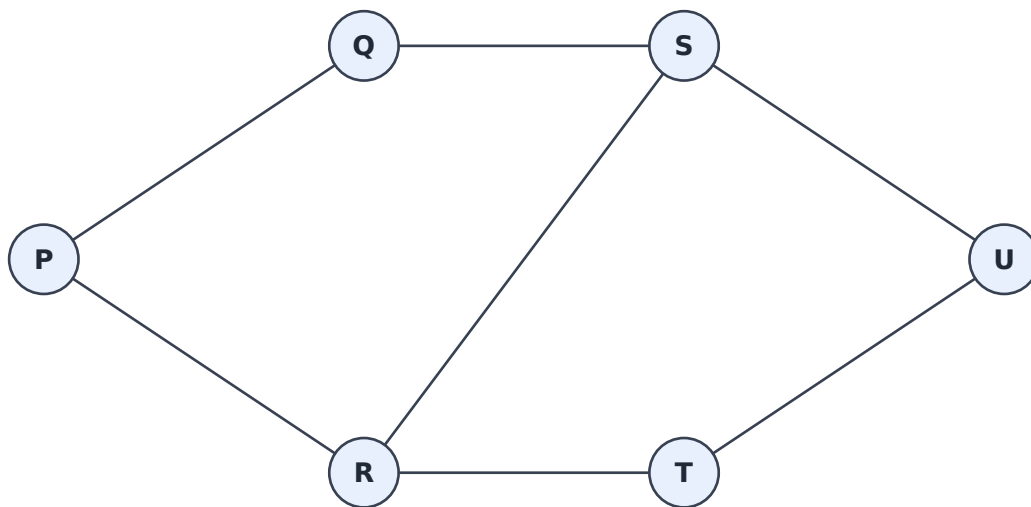
Q.9 [MCQ · single correct | 1 mark | Insertion sort — shifts]

Standard insertion sort (ascending) is applied to the array [5, 2, 8, 1, 9, 3, 7]. A *shift* is one execution of $A[j+1] = A[j]$ inside the inner loop. The total number of shifts is

- (A) 11
- (B) 7
- (C) 9
- (D) 6

Q.10 [MSQ · one or more correct | 1 mark | BFS — valid visit orders]

Breadth-first search is run on the undirected graph below starting at P. Neighbours may be enqueued in **any** order. Which of the following is/are possible BFS visit orders?



- (A) P, R, Q, S, U, T
- (B) P, Q, R, T, S, U
- (C) P, R, Q, T, S, U
- (D) P, Q, R, S, T, U

Q.11 [NAT · numerical answer | 2 marks | Python — deque elimination simulation]

Consider the following Python program. The value printed is _____.

Python

```

1 from collections import deque
2 q = deque(range(1, 12))
3 while len(q) > 1:
4     q.rotate(-(len(q) % 3))
5     q.popleft()
6 print(q[0])
  
```

Answer: _____ (enter a numerical value)

Q.12 [NAT · numerical answer | 2 marks | Queue implemented with two stacks]

A queue is implemented using two stacks S1 and S2. `enqueue(x)` pushes x onto S1. `dequeue()` first checks S2: if S2 is empty, it pops every element of S1 and pushes it onto S2; it then pops and returns the top of S2. Each individual push or pop on S1 or S2 costs one unit (the emptiness test is free). Starting with both stacks empty, the following sequence is executed (E k = enqueue k, D = dequeue):

E1 E2 E3 D E4 D E5 E6 D D D E7 D

The total cost of the whole sequence is _____.

Answer: _____ (enter a numerical value)

Q.13 [MCQ · single correct | 2 marks | Circular queue — wasted-slot convention]

A circular queue uses an array of size 7 (indices 0-6). `front` is the index of the first element and `rear` is the index of the next **free** slot. Initially `front = rear = 0`. The queue is *full* when $(rear + 1) \bmod 7 = front$, and an enqueue on a full queue is ignored. The following operations are performed: enqueue 10, 20, 30, 40, 50; dequeue three times; enqueue 60, 70, 80, 90, 100; dequeue twice; enqueue 110.

At the end, the values of (`front`, `rear`) and the number of elements in the queue are

- (A) `front = 5`, `rear = 3`, 5 elements
- (B) `front = 5`, `rear = 4`, 6 elements
- (C) `front = 5`, `rear = 2`, 4 elements
- (D) `front = 4`, `rear = 3`, 6 elements

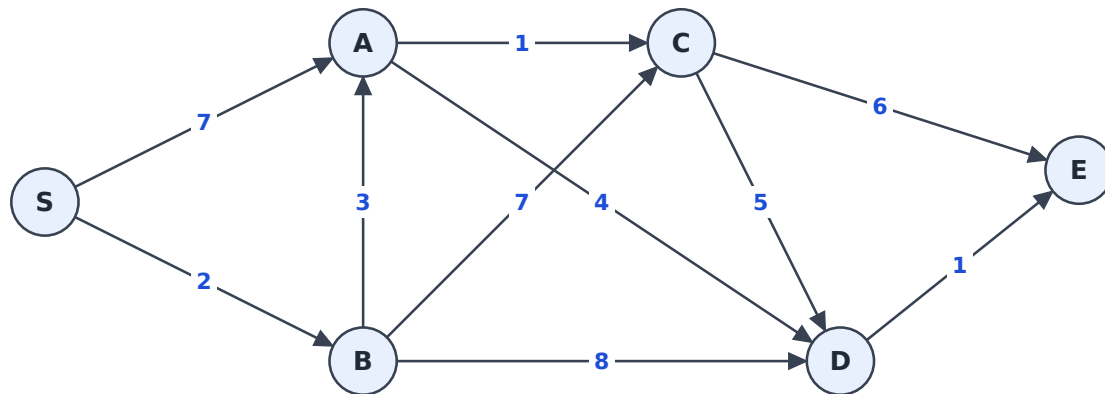
Q.14 [MSQ · one or more correct | 2 marks | Binary min-heap — insert & extract-min]

The keys 14, 9, 21, 6, 17, 3, 11 are inserted one at a time into an initially empty binary min-heap stored in an array (0-based, children of i at $2i+1$ and $2i+2$; each insert appends and sifts up). Then one `extract-min` is performed (the last element moves to the root and sifts down, swapping with the smaller child). Which of the following statements is/are TRUE?

- (A) After the `extract-min`, the element at index 4 is 21
- (B) After the `extract-min`, the right child of the root is 11
- (C) Exactly 4 swaps are performed in total during the seven insertions
- (D) After all seven insertions the array is [3, 9, 6, 14, 17, 21, 11]

Q.15 [NAT · numerical answer | 2 marks | Dijkstra — counting relaxations]

Dijkstra's algorithm (with a min-priority queue) is run from source S on the weighted directed graph below. Initially $d[S] = 0$ and every other d-value is ∞ . Count every event in which some d-value is **strictly decreased** by an edge relaxation (including a decrease from ∞ to a finite value). The total number of such events is _____.



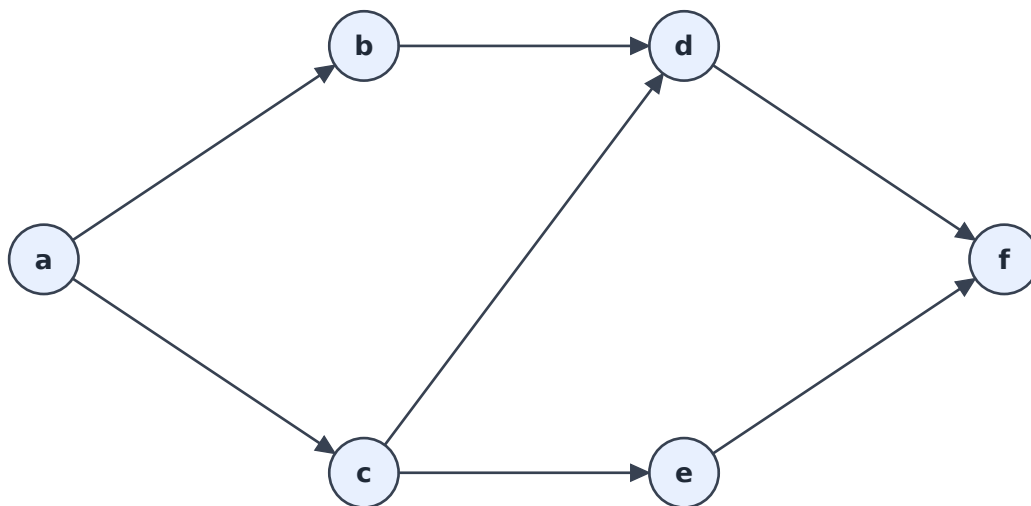
Answer: _____ (enter a numerical value)

Q.16 [MCQ · single correct | 2 marks | Python — list as worklist (stack vs queue)]

The program below was intended to perform BFS on the directed graph shown, but it uses `todo.pop()`. What does it print?

Python

```
1 G = {'a': ['b', 'c'], 'b': ['d'], 'c': ['d', 'e'],
2     'd': ['f'], 'e': ['f'], 'f': []}
3
4 def walk(s):
5     todo, seen, order = [s], {s}, []
6     while todo:
7         u = todo.pop()
8         order.append(u)
9         for v in G[u]:
10            if v not in seen:
11                seen.add(v)
12                todo.append(v)
13     return ''.join(order)
14
15 print(walk('a'))
```



- (A) acedfb
- (B) abcdef
- (C) abdfce
- (D) acefdb

Q.17 [NAT · numerical answer | 2 marks | Merge sort with a queue of runs]

A *queue-based* bottom-up merge sort works as follows. Each element of [9, 4, 7, 1, 8, 2, 6, 3, 5] is enqueued as a one-element sorted run (in array order). While the queue holds more than one run: dequeue run X, dequeue run Y, merge them with the standard two-pointer merge, and enqueue the merged run. A *comparison* is one comparison between an element of X and an element of Y; once either run is exhausted the rest is copied without comparisons. The total number of comparisons is _____.

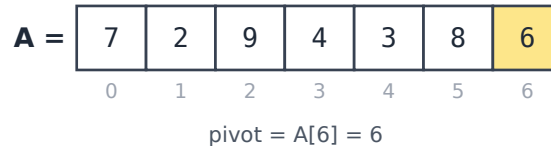
Answer: _____ (enter a numerical value)

Q.18 [MSQ · one or more correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition below is applied once to $A = [7, 2, 9, 4, 3, 8, 6]$ (pivot = last element). Which of the following statements is/are TRUE?

Python

```
1 def partition(A, lo, hi):
2     p, i = A[hi], lo - 1
3     for j in range(lo, hi):
4         if A[j] <= p:
5             i += 1
6             A[i], A[j] = A[j], A[i]
7     A[i + 1], A[hi] = A[hi], A[i + 1]
8     return i + 1
```



- (A) If quicksort used this partition on an already ascending array of n distinct keys, it would take $\Theta(n^2)$ time
- (B) `partition(A, 0, 6)` returns 3
- (C) After the call, the part of A left of the pivot is already in sorted order
- (D) After the call, $A = [2, 4, 3, 6, 9, 8, 7]$

Q.19 [NAT · numerical answer | 2 marks | Monotonic deque — sliding window maximum]

Consider the following Python program. The value printed is _____.

Python

```
1 from collections import deque
2
3 def win_max(a, k):
4     dq, pops, res = deque(), 0, []
5     for i, x in enumerate(a):
6         while dq and a[dq[-1]] <= x:
7             dq.pop()
8             pops += 1
9         dq.append(i)
10        if dq[0] <= i - k:
11            dq.popleft()
12        if i >= k - 1:
13            res.append(a[dq[0]])
14    return pops, res
15
16 p, r = win_max([4, 2, 12, 3, 8, 5, 7, 1, 9, 6], 3)
17 print(p + sum(r))
```

Answer: _____ (enter a numerical value)

Q.20 [MSQ · one or more correct | 2 marks | Output-restricted deque permutations]

The numbers 1, 2, 3, 4 arrive in this order and each must be inserted into an initially empty **output-restricted deque** as soon as it arrives (insertion is allowed at either end, deletion only at the front). Deletions may be performed at any time, and each deleted number is written to the output. Which of the following output sequences CANNOT be produced?

- (A) 4, 2, 3, 1
- (B) 3, 1, 4, 2
- (C) 4, 1, 3, 2
- (D) 4, 3, 1, 2

Answer Key — Mock Test 03

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — collections.deque
2	NAT	1	7	Circular queue — occupancy arithmetic
3	NAT	1	-27	Python — stack-based postfix evaluation
4	MSQ	1	(A), (C)	Python — list slicing
5	MCQ	1	(B)	Linked-list queue
6	MCQ	1	(C)	Binary search — probe sequence
7	MCQ	1	(D)	Level-order traversal with a queue
8	MCQ	1	(B)	Hashing — separate chaining
9	MCQ	1	(C)	Insertion sort — shifts
10	MSQ	1	(C), (D)	BFS — valid visit orders
11	NAT	2	8	Python — deque elimination simulation
12	NAT	2	25	Queue implemented with two stacks
13	MCQ	2	(A)	Circular queue — wasted-slot convention
14	MSQ	2	(B), (D)	Binary min-heap — insert & extract-min
15	NAT	2	9	Dijkstra — counting relaxations
16	MCQ	2	(D)	Python — list as worklist (stack vs queue)
17	NAT	2	21	Merge sort with a queue of runs
18	MSQ	2	(A), (B), (D)	Quicksort — Lomuto partition
19	NAT	2	83	Monotonic deque — sliding window maximum
20	MSQ	2	(A), (C)	Output-restricted deque permutations

Detailed Solutions — Mock Test 03

Q.1 [MCQ · 1 mark · Python — collections.deque]

Answer: (A)

`rotate(k)` with $k > 0$ moves the last k elements to the front (a right rotation); `rotate(-k)` moves the first k to the back. The arguments of `extend` are evaluated left to right **before** anything is appended.

- Start: `[3, 1, 4, 1, 5, 9]`
- `rotate(2)` → `[5, 9, 3, 1, 4, 1]`
- `appendleft(pop())`: `pop` removes the rear 1 → `[1, 5, 9, 3, 1, 4]` (a right rotation by one)
- `rotate(-1)` → `[5, 9, 3, 1, 4, 1]` (undoes the previous step)
- `popleft()` gives 5 → `[9, 3, 1, 4, 1]`; then `pop()` gives 1 → `[9, 3, 1, 4]`; now `extend([5, 1])` → `[9, 3, 1, 4, 5, 1]`.

Option-by-option:

- (A) correct.
- (B) appends the two removed values in the wrong order (`pop` before `popleft`).
- (C) assumes the rotations cancel and `extend` re-adds the same ends — ignores the first `rotate(2)`.
- (D) treats `rotate(2)` as a left rotation.

Trap: the list literal `[d.popleft(), d.pop()]` is fully built before `extend` runs, so both removals happen first. **Tip:** `appendleft(pop())` $\equiv$ `rotate(1)`.

Q.2 [NAT · 1 mark · Circular queue — occupancy arithmetic]

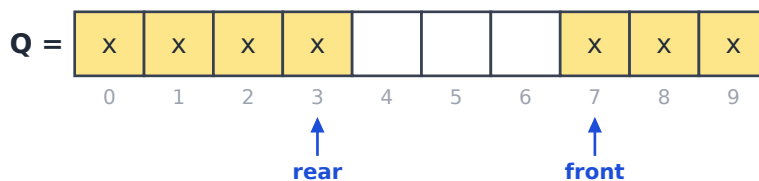
Answer: 7

With `front` and `rear` both pointing at occupied cells, a non-empty circular queue of size n holds $\text{count} = ((\text{rear} - \text{front} + n) \bmod n) + 1$.

Here $n = 10$: $\text{count} = ((3 - 7 + 10) \bmod 10) + 1 = 6 + 1 = 7$.

Check by listing the occupied indices walking forward from `front` with wrap-around: 7, 8, 9, 0, 1, 2, 3 — that is seven cells.

Trap: forgetting the $+1$ gives 6 (that formula is for the convention where `rear` points to the next free slot). Computing $7 - 3 = 4$ ignores the wrap-around and gives the number of **empty** cells minus one. **Tip:** always enumerate the indices once for a sanity check.



Q.3 [NAT · 1 mark · Python — stack-based postfix evaluation]

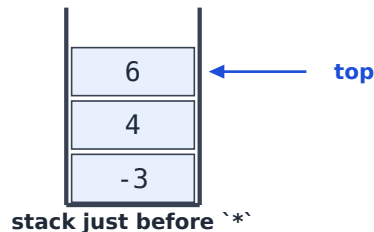
Answer: -27

A Python list used with `append/pop` is a stack. For a binary operator the **first** pop is the right operand `b`, the second is the left operand `a`. `//` is floor division, which rounds toward $-\infty$.

- `3, 8` → stack `[3, 8]`
- `-` → $b = 8, a = 3 \rightarrow 3 - 8 = -5 \rightarrow [-5]$
- `2` → `[-5, 2]`
- `/` → $-5 // 2 = -3$ (floor of -2.5) → `[-3]`

- $4, 6 \rightarrow [-3, 4, 6]; * \rightarrow 24 \rightarrow [-3, 24]$
- $- \rightarrow -3 - 24 = -27$

Traps: (i) truncating toward zero gives -2 and a final answer of -26 ; (ii) swapping the operand order gives $8 - 3 = 5$ at the first step. Note also that `'-5'.isdigit()` is `False` — the program only works because every input token is a non-negative literal.



Q.4 [MSQ · 1 mark · Python — list slicing]

Answer: (A), (C)

A slice `q[start:stop:step]` starts at `start`, moves by `step`, and stops **before** reaching `stop`. Negative indices count from the end (-1 is the last element). For a negative step the defaults are `start = last`, `stop = before-the-first`.

- (A) $-3 \rightarrow$ index 3, $-1 \rightarrow$ index 5 (excluded) $\rightarrow$ indices 3, 4 $\rightarrow [40, 50]$. **True.**
- (B) indices 4, 3, 2 (index 1 is the excluded stop) $\rightarrow [50, 40, 30]$. **False** — the stop bound is exclusive in both directions.
- (C) step -2 from index 5: indices 5, 3, 1 $\rightarrow [60, 40, 20]$. **True.**
- (D) start index 5, stop index 2, step $+1$ (default): you cannot move forward from 5 to 2, so the result is `[]`. **False.**

Trap: (D) looks like it should walk backwards, but without an explicit negative step Python never reverses — it silently returns an empty list.

Q.5 [MCQ · 1 mark · Linked-list queue]

Answer: (B)

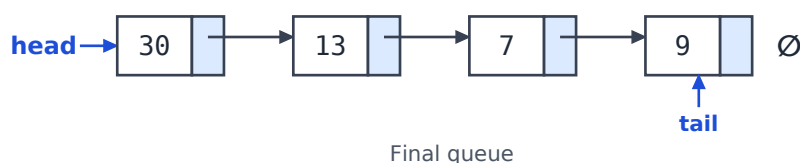
A queue is FIFO: removals come from the head, insertions go after the tail. Both are $\Theta(1)$ with a tail pointer.

- `x = dequeue()` $\rightarrow x = 12$; queue 7, 25, 30
- `enqueue(13)` $\rightarrow 7, 25, 30, 13$
- `enqueue(dequeue())` removes 7 and re-inserts it at the rear $\rightarrow 25, 30, 13, 7$
- `dequeue()` removes 25 $\rightarrow 30, 13, 7$
- `enqueue(9)` $\rightarrow 30, 13, 7, 9$

Option-by-option:

- (A) forgets that `enqueue(dequeue())` moves 7 to the back.
- (B) correct.
- (C) skips the final `dequeue()`.
- (D) puts 7 before 13, as if 7 had been moved before 13 was inserted.

Tip: `enqueue(dequeue())` is a one-step rotation of the queue.



Q.6 [MCQ · 1 mark · Binary search — probe sequence]**Answer: (C)**

Each iteration halves the live range [lo, hi]; mid uses floor division.

- lo=0, hi=14 → mid=7, A[7]=31 < 37 → lo=8
- lo=8, hi=14 → mid=11, A[11]=51 > 37 → hi=10
- lo=8, hi=10 → mid=9, A[9]=40 > 37 → hi=8
- lo=8, hi=8 → mid=8, A[8]=35 < 37 → lo=9 > hi → stop (not found)

Values probed: **31, 51, 40, 35** (4 probes = $\lfloor \log_2 15 \rfloor + 1$).

- (A) adds 37, which is not in the array at all.
- (B) uses mid = 10 in the second step (as if hi were not 14).
- (D) uses a wrong first midpoint (index 6).

Tip: an unsuccessful search in an array of $2^k - 1$ elements always makes exactly k probes.

Q.7 [MCQ · 1 mark · Level-order traversal with a queue]**Answer: (D)**

The queue always holds a suffix of the current level followed by a prefix of the next level.

- dequeue A → enqueue B, C → [B, C]
- dequeue B → enqueue D, E → [C, D, E]
- dequeue C → enqueue F (no left child) → [D, E, F]
- dequeue D (leaf) → [E, F]
- dequeue E → enqueue H → **[F, H]**
- (A) forgets F, which was enqueued when C was processed.
- (B) reverses FIFO order — that would be a stack.
- (C) adds I, J which are enqueued only when F is dequeued.

Tip: full level order is A B C D E F H I J; at any moment the queue is a contiguous window of that sequence.

Q.8 [MCQ · 1 mark · Hashing — separate chaining]**Answer: (B)**

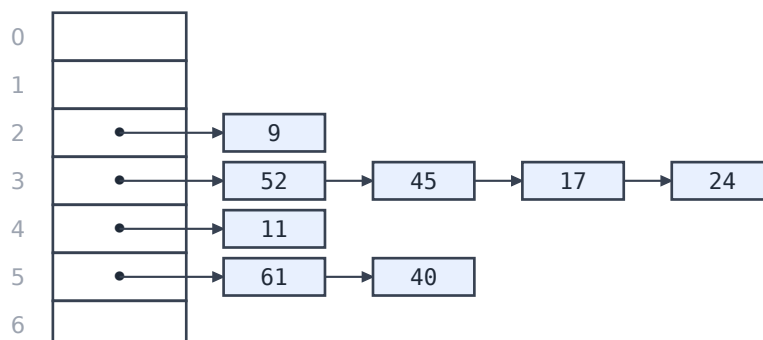
Compute each home slot: 24→3, 17→3, 40→5, 11→4, 45→3, 61→5, 9→2, 52→3.

Keys landing in slot 3, in insertion order: 24, 17, 45, 52. With head insertion each new key goes in front of the previous ones, so the chain reads in **reverse** insertion order: **52 → 45 → 17 → 24**.

- (A) is tail insertion order.
- (C) and (D) are not the reverse of any insertion order of these keys.

Slot 5 similarly holds 61 → 40, slot 4 holds 11, slot 2 holds 9. Load factor $\alpha = 8/7$.

Tip: head insertion makes insertion $\Theta(1)$ without a tail pointer; the most recently inserted key is found fastest.



Final table (chains head first)

Q.9 [MCQ · 1 mark · Insertion sort — shifts]**Answer: (C)**

Each shift fixes exactly one inversion (a pair $i < j$ with $A[i] > A[j]$), so the number of shifts equals the number of inversions.

Count, for each element, how many larger elements precede it:

- 5: 0; 2: 1 (5); 8: 0; 1: 3 (5, 2, 8); 9: 0; 3: 3 (5, 8, 9); 7: 2 (8, 9)

Total = $0 + 1 + 0 + 3 + 0 + 3 + 2 = 9$.

- (A) 11 over-counts — typically by counting the comparison that ends each pass as a shift.
- (B) 7 comes from missing that 3 has **three** larger predecessors (5, 8, 9), counting only 8 and 9.
- (D) 6 is just the number of passes ($n - 1$).

Trap: shifts $\neq$ comparisons. Comparisons = shifts + (number of passes in which the loop stopped because of a smaller element) — here $9 + 4 = 13$.

Worked trace / reference code

```
1 A = [5, 2, 8, 1, 9, 3, 7]; shifts = 0
2 for i in range(1, len(A)):
3     key, j = A[i], i - 1
4     while j >= 0 and A[j] > key:
5         A[j + 1] = A[j]; j -= 1; shifts += 1
6     A[j + 1] = key
7 print(A, shifts) # [1, 2, 3, 5, 7, 8, 9] 9
```

Q.10 [MSQ · 1 mark · BFS — valid visit orders]**Answer: (C), (D)**

BFS visits vertices in non-decreasing distance from P, **and** within a level the order follows the order in which parents were dequeued (FIFO). Levels: {P}, {Q, R}, {S, T}, {U}.

- (A) U (distance 3) appears before T (distance 2). **Invalid.**
- (B) Q is before R, so S (discovered by Q) must precede T (discovered only by R). **Invalid.**
- (C) R dequeued first discovers S and T (any order, here T then S); Q adds nothing new. **Valid.**
- (D) Q dequeued first discovers S; then R discovers T $\rightarrow$ S before T. **Valid.**

Trap: being level-consistent is necessary but not sufficient — (B) respects levels but violates the FIFO parent order.

Q.11 [NAT · 2 marks · Python — deque elimination simulation]**Answer: 8**

`rotate(-r)` moves the first r people to the back; `popleft()` then removes the person now at the front. The step $r = \text{len}(q) \bmod 3$ changes as the queue shrinks — so this is **not** a fixed-step Josephus problem and must be traced.

- len 11, $r=2$: remove 3 $\rightarrow$ [4 5 6 7 8 9 10 11 1 2]
- len 10, $r=1$: remove 5 $\rightarrow$ [6 7 8 9 10 11 1 2 4]
- len 9, $r=0$: remove 6 $\rightarrow$ [7 8 9 10 11 1 2 4]
- len 8, $r=2$: remove 9 $\rightarrow$ [10 11 1 2 4 7 8]
- len 7, $r=1$: remove 11 $\rightarrow$ [1 2 4 7 8 10]
- len 6, $r=0$: remove 1 $\rightarrow$ [2 4 7 8 10]
- len 5, $r=2$: remove 7 $\rightarrow$ [8 10 2 4]
- len 4, $r=1$: remove 10 $\rightarrow$ [2 4 8]
- len 3, $r=0$: remove 2 $\rightarrow$ [4 8]
- len 2, $r=2$: rotate by 2 is a full turn $\rightarrow$ remove 4 $\rightarrow$ [8]

The survivor printed is **8**.

Traps: (i) `rotate(-r)` is a *left* rotation — using a right rotation changes everything; (ii) when $r = 0$ the current front is removed immediately; (iii) in the last step rotating a 2-element deque by 2 leaves it unchanged.

Q.12 [NAT · 2 marks · Queue implemented with two stacks]

Answer: 25

Lifetime (accounting) view: every element that is eventually dequeued costs exactly 4 units — push S1, pop S1, push S2, pop S2. An element still sitting in S1 at the end has cost only 1 unit. Count the D's carefully: positions 4, 6, 9, 10, 11, 13 → **6** dequeues for 7 enqueues, so element 7 is never transferred.

Detailed trace (S1/S2 shown bottom→top, running cost):

- E1 E2 E3: S1=[1,2,3]; cost 3
- D: transfer 3 elements (6 units), pop 1 → S2=[3,2]; cost 10
- E4: S1=[4]; cost 11. D: pop 2 → cost 12
- E5 E6: S1=[4,5,6]; cost 14. D: pop 3 → S2 empty; cost 15
- D: transfer 3 (6 units), pop 4 → S2=[6,5]; cost 22
- D: pop 5 → cost 23. E7: cost 24. D: pop 6 → cost **25**

Check with the lifetime argument: 6 dequeued elements $\times 4 = 24$, plus 1 for element 7 left in S1 → 25.

Trap: miscounting the dequeues, or charging a transfer on every dequeue (only when S2 is empty).

Tip: the amortised cost per operation is $O(1)$.

Running state

op	S1 (bottom→top)	S2 (bottom→top)	cost
E1 E2 E3	1 2 3	—	3
D → 1	—	3 2	10
E4	4	3 2	11
D → 2	4	3	12
E5 E6	4 5 6	3	14
D → 3	4 5 6	—	15
D → 4	—	6 5	22
D → 5	—	6	23
E7	7	6	24
D → 6	7	—	25

Q.13 [MCQ · 2 marks · Circular queue — wasted-slot convention]

Answer: (A)

With the wasted-slot convention an array of size 7 holds at most **6** elements; $\text{count} = (\text{rear} - \text{front} + 7) \bmod 7$.

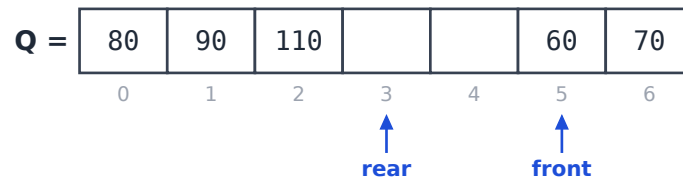
- Enqueue 10...50 into slots 0-4 → front 0, rear 5 (5 elements).
- Three dequeues → front 3 (2 elements: 40, 50).
- 60 → slot 5, 70 → slot 6, 80 → slot 0, 90 → slot 1 → rear 2, $\text{count} = (2 - 3 + 7) \bmod 7 = 6 \rightarrow$ **full**.
- 100: $(2 + 1) \bmod 7 = 3 = \text{front} \rightarrow$ full → **ignored**.
- Two dequeues remove 40, 50 → front 5.

- 110 → slot 2 → rear 3.

Final: front = 5, rear = 3, count = $(3 - 5 + 7) \bmod 7 = 5$ (60, 70, 80, 90, 110).

- (B) assumes all 7 slots are usable, so 100 is accepted.
- (C) forgets the final enqueue of 110.
- (D) performs only one of the last two dequeues.

Trap: the 'one empty slot' rule exists precisely so that front = rear can mean *empty*; it costs one cell of capacity.



Final state (slot 3, 4 free)

Q.14 [MSQ · 2 marks · Binary min-heap — insert & extract-min]

Answer: (B), (D)

A priority queue implemented as a binary heap: insert = append + sift-up, extract-min = move last to root + sift-down.

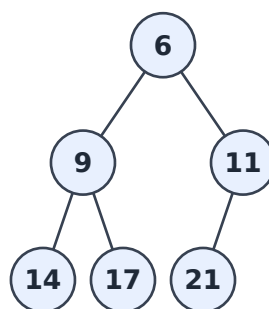
Insertions (swaps in brackets):

- 14 → [14] (0)
- 9 → swap with 14 → [9, 14] (1)
- 21 → [9, 14, 21] (0)
- 6 at index 3: swap with 14, then with 9 → [6, 9, 21, 14] (2)
- 17 at index 4: parent 9 < 17 → [6, 9, 21, 14, 17] (0)
- 3 at index 5: swap with 21, then with 6 → [3, 9, 6, 14, 17, 21] (2)
- 11 at index 6: parent 6 < 11 → [3, 9, 6, 14, 17, 21, 11] (0)

Total swaps = 5. Extract-min: remove 3, move 11 to root → [11, 9, 6, 14, 17, 21]; smaller child is 6 → swap → [6, 9, 11, 14, 17, 21]; 11's only child 21 is larger → stop.

- (A) index 4 holds 17; 21 is at index 5. **False.**
- (B) index 2 holds 11. **True.**
- (C) **False** — 5 swaps, not 4.
- (D) **True.**

Trap: in sift-down always swap with the **smaller** child (9 vs 6 → choose 6); swapping with the left child by default breaks the heap property.



Min-heap after extract-min

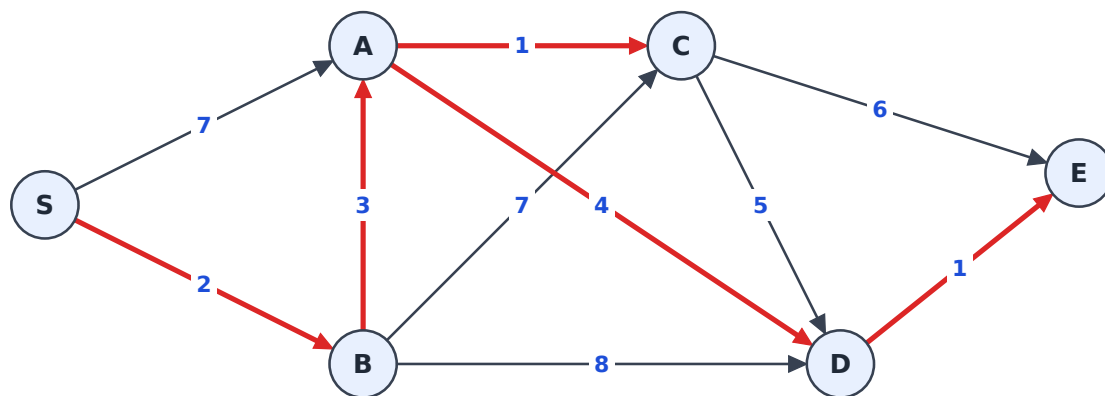
Q.15 [NAT · 2 marks · Dijkstra — counting relaxations]**Answer: 9**

Dijkstra extracts the vertex with the smallest tentative distance and relaxes its out-edges; a relaxation succeeds when $d[u] + w < d[v]$.

- Extract S (0): $A \infty \rightarrow 7$, $B \infty \rightarrow 2$ — **2** decreases
- Extract B (2): $A 7 \rightarrow 5$, $D \infty \rightarrow 10$, $C \infty \rightarrow 9$ — **3**
- Extract A (5): $C 9 \rightarrow 6$, $D 10 \rightarrow 9$ — **2**
- Extract C (6): $E \infty \rightarrow 12$; $D: 6 + 5 = 11 > 9$ ✗ — **1**
- Extract D (9): $E 12 \rightarrow 10$ — **1**
- Extract E (10): no out-edges

Total = 2 + 3 + 2 + 1 + 1 = **9**. Final distances: A 5, B 2, C 6, D 9, E 10.

Trap: counting every edge examination (10 edges) instead of only successful ones; the relaxation of $C \rightarrow D$ fails. **Tip:** with a binary-heap PQ each successful relaxation is one decrease-key, so this count drives the $O((V + E) \log V)$ bound.



Shortest-path tree

Q.16 [MCQ · 2 marks · Python — list as worklist (stack vs queue)]**Answer: (D)**

`list.pop()` removes the **last** element, so `todo` behaves as a stack (LIFO). Vertices are marked **seen** when *pushed*, so each is pushed at most once — this is a stack-based traversal, not the recursive DFS order.

- `todo [a] → pop a`; push b, c $\rightarrow [b, c]$; order a
- `pop c`; push d, e $\rightarrow [b, d, e]$; order ac
- `pop e`; f unseen $\rightarrow$ push f $\rightarrow [b, d, f]$; order ace
- `pop f` $\rightarrow [b, d]$; order acef
- `pop d` (f already seen) $\rightarrow [b]$; order acefd
- `pop b` (d seen) $\rightarrow$ order **acefdb**
- (A) **acedfb** would arise if d were popped before f — impossible since f was pushed last.
- (B) **abcdef** is the BFS order you would get with `todo.pop(0)`.
- (C) **abdfce** is the recursive DFS order (visit neighbours in list order).

Trap: marking on push (not on pop) means f is pushed by e, not d. **Tip:** use `collections.deque.popleft()` for BFS; `list.pop(0)` works but is $O(n)$.

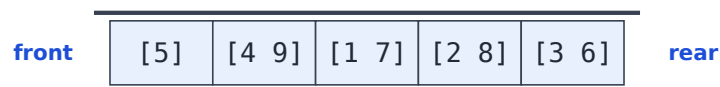
Q.17 [NAT · 2 marks · Merge sort with a queue of runs]**Answer: 21**

Merging runs of sizes p and q costs between $\min(p, q)$ and $p + q - 1$ comparisons; the exact count is $p + q - (\text{number of elements left over when one run empties})$.

- $[9]+[4] \rightarrow 1$, $[7]+[1] \rightarrow 1$, $[8]+[2] \rightarrow 1$, $[6]+[3] \rightarrow 1$ (queue now: $[5]$, $[4\ 9]$, $[1\ 7]$, $[2\ 8]$, $[3\ 6]$)
- $[5]+[4\ 9] \rightarrow 4$ vs 5 , 5 vs $9 \rightarrow 2 \rightarrow [4\ 5\ 9]$
- $[1\ 7]+[2\ 8] \rightarrow 1|2, 7|2, 7|8 \rightarrow 3 \rightarrow [1\ 2\ 7\ 8]$
- $[3\ 6]+[4\ 5\ 9] \rightarrow 3|4, 6|4, 6|5, 6|9 \rightarrow 4 \rightarrow [3\ 4\ 5\ 6\ 9]$
- $[1\ 2\ 7\ 8]+[3\ 4\ 5\ 6\ 9] \rightarrow 1|3, 2|3, 7|3, 7|4, 7|5, 7|6, 7|9, 8|9 \rightarrow 8$

Total = $4 + 2 + 3 + 4 + 8 = 21$.

Trap: the odd element 5 does *not* wait for the end — FIFO order pairs it with the first merged run $[4\ 9]$, so the merge tree is unbalanced compared with recursive merge sort. **Tip:** this queue scheme still makes $O(n \log n)$ comparisons overall.



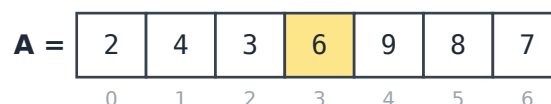
Queue after the four singleton merges

Q.18 [MSQ · 2 marks · Quicksort — Lomuto partition]**Answer: (A), (B), (D)**

Lomuto keeps the invariant $A[\text{lo}..i] \leq \text{pivot} < A[i+1..j-1]$; at the end the pivot is swapped into position $i + 1$.

- $j=0$ ($7 > 6$) skip
- $j=1$ ($2 \leq 6$): $i=0$, swap $A_0 \leftrightarrow A_1 \rightarrow [2, 7, 9, 4, 3, 8, 6]$
- $j=2$ (9) skip
- $j=3$ (4): $i=1$, swap $A_1 \leftrightarrow A_3 \rightarrow [2, 4, 9, 7, 3, 8, 6]$
- $j=4$ (3): $i=2$, swap $A_2 \leftrightarrow A_4 \rightarrow [2, 4, 3, 7, 9, 8, 6]$
- $j=5$ (8) skip
- final swap $A_3 \leftrightarrow A_6 \rightarrow [2, 4, 3, 6, 9, 8, 7]$, return 3
- (A) on sorted input the last element is the maximum, every split is $(n-1, 0)$, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. **True.**
- (B) **True.**
- (C) left part is $[2, 4, 3]$ — not sorted. **False.** Partition only guarantees $\leq$ pivot, not order.
- (D) **True.**

Trap: partitioning is not sorting, and Lomuto is not stable: the right part was 7, 9, 8 in the input but ends up as 9, 8, 7 because the pivot swap moved 7 to the end.



After partition: pivot 6 at index 3

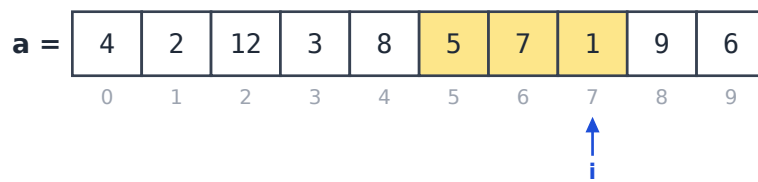
Q.19 [NAT · 2 marks · Monotonic deque — sliding window maximum]**Answer: 83**

The deque stores **indices** whose values are strictly decreasing from front to rear. A new value pops every smaller-or-equal value from the rear (they can never be a future maximum); the front is dropped once it slides out of the window. The front is always the window max.

- i0 4: dq [0]
- i1 2: dq [0,1]
- i2 12: pops 2 and 4 (**2 pops**) → [2]; max 12
- i3 3: [2,3]; max 12
- i4 8: pops 3 (**1**) → [2,4]; max 12
- i5 5: [2,4,5]; index $2 \leq 5-3 \rightarrow$ popleft → [4,5]; max 8
- i6 7: pops 5 (**1**) → [4,6]; max 8
- i7 1: [4,6,7]; index $4 \leq 4 \rightarrow$ popleft → [6,7]; max 7
- i8 9: pops 1 and 7 (**2**) → [8]; max 9
- i9 6: [8,9]; max 9

pops = 2 + 1 + 1 + 2 = 6; r = [12, 12, 12, 8, 8, 7, 9, 9], sum = 77. Printed: 6 + 77 = **83**.

Trap: `popleft()` (window expiry) is not counted in **pops** — only rear pops are. **Tip:** each index is pushed once and popped at most once, so the algorithm is $\Theta(n)$ regardless of k.



Window at i = 7: deque holds indices 6, 7 → max a[6] = 7

Q.20 [MSQ · 2 marks · Output-restricted deque permutations]

Answer: (A), (C)

If 4 is output first, then 1, 2, 3 are all inside the deque before any deletion, and the remaining output is just the deque read front-to-rear. Building the deque by inserting 1, then 2 at either end, then 3 at either end gives only:

3 2 1, 2 1 3, 3 1 2, 1 2 3

(the values must form a sequence where each newly inserted number is at an **end**). With 4 inserted at the front and deleted first, the possible outputs starting with 4 are 4321, 4213, 4312, 4123.

- (A) 4 2 3 1 → deque 2 3 1 with 3 in the middle, but 3 was the last of the three inserted. **Cannot.**
- (B) insert 1, insert 2 at the rear → [1 2]; insert 3 at front → [3 1 2]; delete 3, delete 1; insert 4 at front → [4 2]; delete 4, delete 2 → 3 1 4 2. **Can.**
- (C) 4 1 3 2 → would need deque 1 3 2 with 3 in the middle. **Cannot.**
- (D) 4 3 1 2 → deque 3 1 2 (1; 2 at rear; 3 at front). **Can.**

Of the 24 permutations exactly 22 are achievable; the only two impossible ones are (C) and (A).

Tip: the last-inserted element of a deque must sit at one of its ends — a quick test for any 'restricted deque' question.

Mock Test 04

Linked Lists

Difficulty: Moderate

Focus areas: singly/doubly/circular lists, pointer manipulation, traversal costs

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Linked lists — pointer manipulation in Python]

Consider the following Python program. What is printed?

Python

```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 head = None
6 for v in [7, 3, 9, 4, 6]:
7     head = Node(v, head)
8 p = head
9 while p.next and p.next.next:
10    p.next = p.next.next
11    p = p.next
12 out = []
13 while head:
14    out.append(head.val)
15    head = head.next
16 print(out)

```



List after the building loop

- (A) [6, 4, 9, 3, 7]
- (B) [7, 9, 6]
- (C) [6, 9, 7]
- (D) [6, 9]

Q.2 [NAT · numerical answer | 1 mark | Doubly linked lists — unlink and move]

The program below builds the doubly linked list shown in the figure and then performs two pointer operations. What value does it print?

Python

```

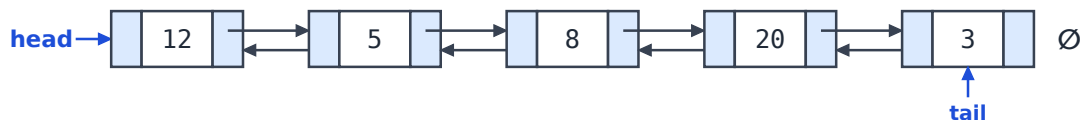
1 class D:
2     def __init__(self, v):
3         self.v, self.prev, self.next = v, None, None
4
5 nodes = [D(v) for v in [12, 5, 8, 20, 3]]
6 for a, b in zip(nodes, nodes[1:]):
7     a.next, b.prev = b, a
8 head, tail = nodes[0], nodes[-1]
9
10 x = head.next.next
11 x.prev.next = x.next

```

```

12 x.next.prev = x.prev
13
14 t = tail
15 tail = t.prev
16 tail.next = None
17 t.next, head.prev = head, t
18 head = t
19
20 p, s, i = head, 0, 0
21 while p:
22     s += i * p.v
23     i, p = i + 1, p.next
24 print(s)

```



Initial doubly linked list

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Queues using linked lists — operation costs]

A FIFO queue holding n elements is implemented with a singly linked list (each node has only a `next` field). Which of the following statements is/are TRUE?

- (A) If the list is non-circular and only a head pointer is kept, enqueue at the tail takes $\Theta(n)$ time
- (B) If both head and tail pointers are kept, enqueueing at the tail and dequeuing at the head both take $O(1)$ worst-case time
- (C) If the list is circular and only a pointer to the last node is kept, both enqueue and dequeue take $O(1)$ worst-case time
- (D) If both head and tail pointers are kept, enqueueing at the head and dequeuing at the tail both take $O(1)$ worst-case time

Q.4 [MCQ · single correct | 1 mark | Python — string slicing with negative steps]

What is printed by the following Python statements?

Python

```

1 s = "LINKEDLIST"
2 print(s[-2:2:-2] + s[1::4])

```

- (A) ELSIDT
- (B) SLEKIDT
- (C) TSLEIDT
- (D) SLEIDT

Q.5 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation]

The postfix expression

8 2 5 * + 1 3 2 * + 4 - /

is evaluated with an operand stack in the usual way (push operands; for an operator pop two, apply, push the result). The maximum number of elements present on the stack at any instant during the evaluation is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Hashing — separate chaining]

Keys 19, 26, 13, 40, 33, 20, 6, 14 are inserted, in this order, into an initially empty hash table with 7 slots (0-6), hash function $h(k) = k \bmod 7$, and separate chaining in which every new key is inserted at the **front** of its chain. Reading the chain of slot 5 from front to back gives

- (A) 19, 26, 40, 33
- (B) 33, 26, 40, 19
- (C) 40, 33, 26, 19
- (D) 33, 40, 26, 19

Q.7 [MCQ · single correct | 1 mark | Binary search — probe sequence]

Binary search with $\text{mid} = (\text{lo} + \text{hi}) // 2$ (0-based indices, initially $\text{lo} = 0$, $\text{hi} = 11$) is applied to the sorted array A below to search for the key **47**. The sequence of array **values** compared with the key is

A =	4	9	13	18	22	27	31	36	40	45	51	58
	0	1	2	3	4	5	6	7	8	9	10	11

Sorted array A[0..11]

- (A) 31, 45, 58, 51
- (B) 27, 40, 51, 45
- (C) 27, 40, 45
- (D) 27, 40, 51, 45, 51

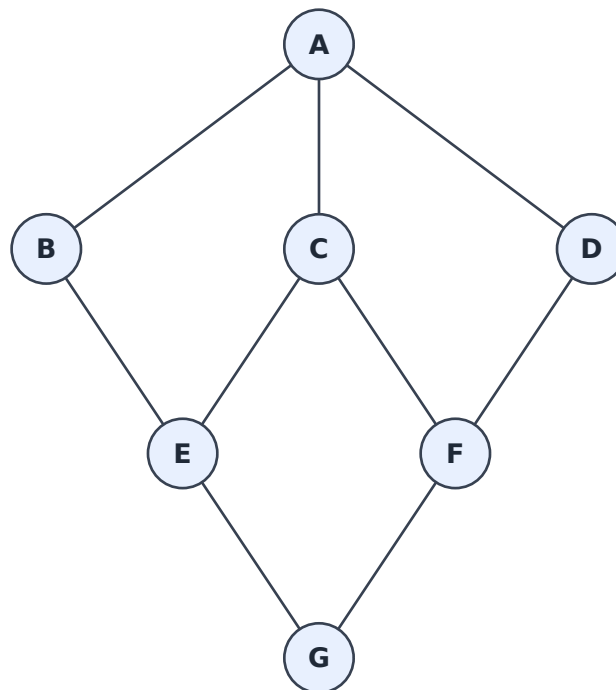
Q.8 [MCQ · single correct | 1 mark | Insertion sort — intermediate state]

Insertion sort (ascending) is applied to the array [31, 12, 47, 8, 25, 19]. In pass i ($i = 1, 2, \dots, 5$) the element at index i is inserted into the sorted prefix $A[0..i-1]$. What is the array immediately after pass 3 completes?

- (A) [8, 12, 31, 47, 25, 19]
- (B) [12, 31, 47, 8, 25, 19]
- (C) [8, 12, 25, 31, 47, 19]
- (D) [8, 12, 31, 25, 47, 19]

Q.9 [MSQ · one or more correct | 1 mark | Graph traversal — valid BFS orders]

Consider the undirected graph below. Breadth-first search is started at vertex A; the neighbours of a vertex may be enqueued in **any** order. Which of the following is/are possible BFS visiting orders?



- (A) A, B, C, D, F, E, G
- (B) A, B, C, D, E, F, G
- (C) A, C, B, D, F, E, G
- (D) A, D, C, B, F, E, G

Q.10 [NAT · numerical answer | 1 mark | Binary heaps — insertion swaps]

The keys 40, 35, 50, 20, 45, 10, 30, 15 are inserted one by one, in this order, into an initially empty binary **min**-heap (array representation, sift-up after each insertion). The total number of parent-child swaps performed over all eight insertions is _____.

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Linked lists — reversal in groups of k]

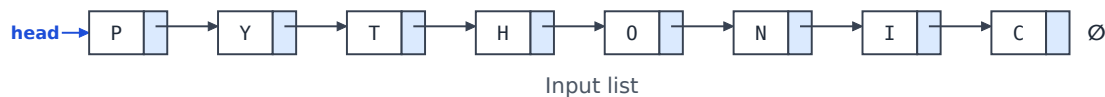
The singly linked list shown below holds one character per node. Consider the following Python program, which uses the function `rev_k`. What is printed?

Python

```

1 class Node:
2     def __init__(self, val):
3         self.val, self.next = val, None
4
5 def rev_k(head, k):
6     prev, cur, n = None, head, 0
7     while cur and n < k:
8         cur.next, prev, cur = prev, cur, cur.next
9         n += 1
10    if cur:
11        head.next = rev_k(cur, k)
12    return prev
13
14 head = None
15 for ch in reversed("PYTHONIC"):
16     nd = Node(ch); nd.next = head; head = nd
17 head = rev_k(head, 3)
18 s = ""
19 while head:
20     s, head = s + head.val, head.next
21 print(s)

```



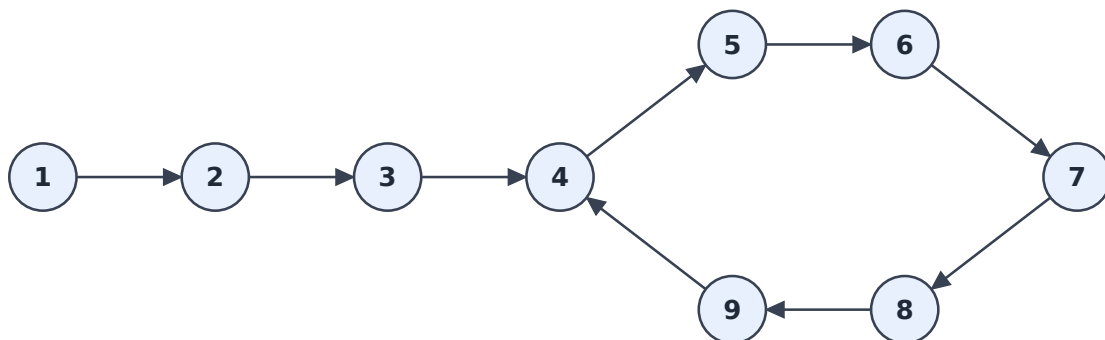
- (A) TYPNOHIC
- (B) TYPNOHCI
- (C) CINOHTYP
- (D) TYPHONCI

Q.12 [NAT · numerical answer | 2 marks | Linked lists — Floyd's cycle detection]

A singly linked list has nodes 1, 2, ..., 9 linked in order, and the `next` pointer of node 9 points back to node 4 (see figure). Floyd's algorithm is run:

- **Phase 1:** `slow` and `fast` both start at node 1. In each step, `slow` moves one node and `fast` moves two nodes; the step count m is the number of steps until they are first on the same node.
- **Phase 2:** `slow` is reset to node 1, `fast` stays where they met; both now move one node per step until they meet; let r be the number of steps of this phase.

The value of $m + r$ is ____.



Answer: _____ (enter a numerical value)

Q.13 [NAT · numerical answer | 2 marks | Circular linked lists — elimination game]

What value is printed by the following Python program?

Python

```

1 class Node:
2     def __init__(self, v):
3         self.v, self.next = v, None
4
5 n, k = 11, 4
6 first = Node(1)
7 p = first
8 for v in range(2, n + 1):
9     p.next = Node(v)
10    p = p.next
11 p.next = first      # make it circular; p = last node
12 while p.next is not p:
13     for _ in range(k - 1):
14         p = p.next
15     p.next = p.next.next
16 print(p.v)

```



Circular list before the elimination loop

Answer: _____ (enter a numerical value)

Q.14 [MSQ · one or more correct | 2 marks | Doubly linked lists — deletion code]

A doubly linked list has nodes with fields **val**, **prev**, **next**; the first node has **prev = None** and the last node has **next = None**. The following function is meant to delete node **x** (which is in the list) and return the new head.

Which of the following statements is/are TRUE?

Python

```

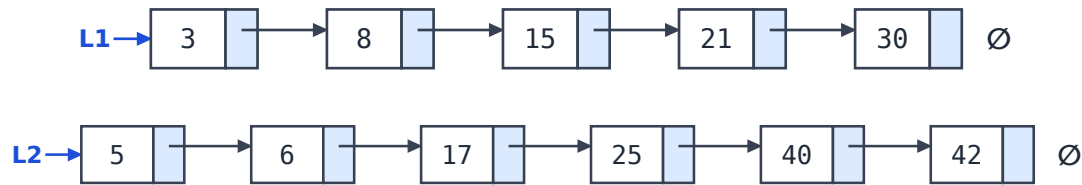
1 def remove(head, x):
2     x.prev.next = x.next
3     if x.next:
4         x.next.prev = x.prev
5     if x is head:
6         head = x.next
7     return head

```

- (A) Called on a one-node list (**x** is head), it returns **None** without raising an error
- (B) It correctly deletes any node that is neither the first nor the last node
- (C) It correctly deletes the last node of a list having at least two nodes
- (D) It correctly deletes the first node of a list having at least two nodes

Q.15 [NAT · numerical answer | 2 marks | Linked lists — merging sorted lists]

The two sorted singly linked lists L1 and L2 shown below are merged into one sorted list by the standard merge procedure: repeatedly compare the front nodes of the two lists, detach the smaller (L1 on ties) and append it to the output; as soon as one list becomes empty, the remainder of the other is attached with a single pointer assignment, without any comparisons. The number of key comparisons performed is _____.



Answer: _____ (enter a numerical value)

Q.16 [MCQ · single correct | 2 marks | Recursion on linked lists]

Consider the following Python program. What is its complete output?

Python

```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 def g(p, d=0):
6     if p is None:
7         return 0
8     r = g(p.next, d + 1)
9     if d % 2:
10        print(p.val, end=" ")
11    return p.val - r
12
13 head = None
14 for v in [3, 9, 1, 7, 4]:
15     head = Node(v, head)
16 print(g(head))
  
```

- (A) 9 7 -8
- (B) 7 9 -8
- (C) 9 7 8
- (D) 4 1 3 -8

Q.17 [MCQ · single correct | 2 marks | Python — shallow vs deep copy]

What is printed by the following Python program?

Python

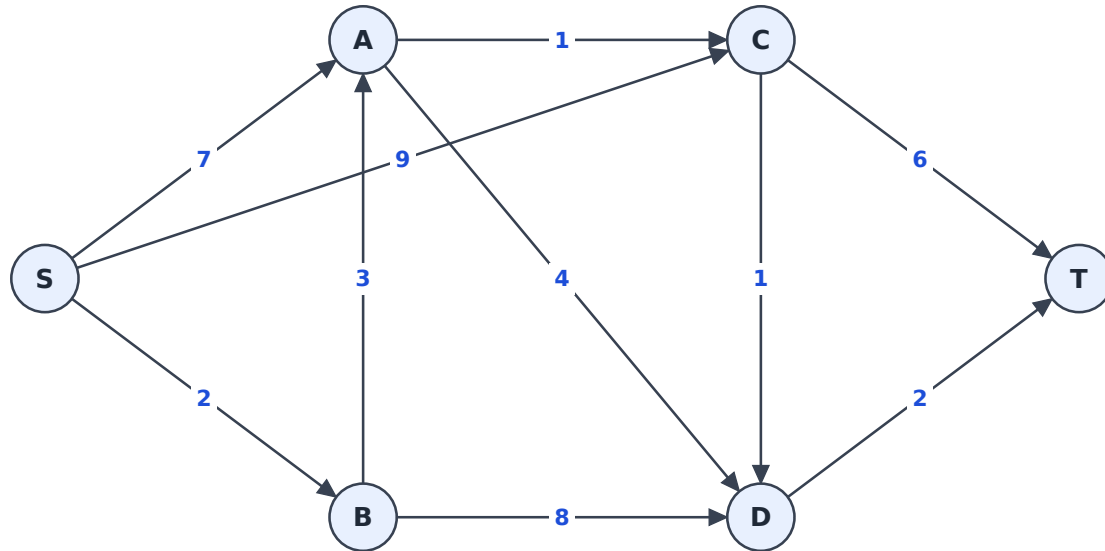
```

1 import copy
2 a = [[1, 2], [3, 4]]
3 b = a[:]
4 c = copy.deepcopy(a)
5 b[0].append(5)
6 b[1] = [6]
7 c[0][0] = 9
8 print(a, b[0] is a[0], c[0])
  
```

- (A) [[1, 2, 5], [6]] True [9, 2]
- (B) [[1, 2], [3, 4]] False [9, 2]
- (C) [[1, 2, 5], [3, 4]] True [9, 2]
- (D) [[9, 2, 5], [3, 4]] True [9, 2]

Q.18 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra's algorithm trace]

Dijkstra's algorithm is run from source S on the weighted directed graph below. Initially $d[S] = 0$ and every other $d[v] = \infty$. Each time a vertex u is extracted, every edge (u, v) is relaxed. Count one **update** every time some $d[v]$ is strictly decreased (the first change from ∞ to a finite value counts as an update). The total number of updates is _____.



Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition procedure below is called as `partition(A, 0, 6)` on $A = [7, 2, 9, 4, 1, 8, 5]$. Which of the following statements is/are TRUE?

Python

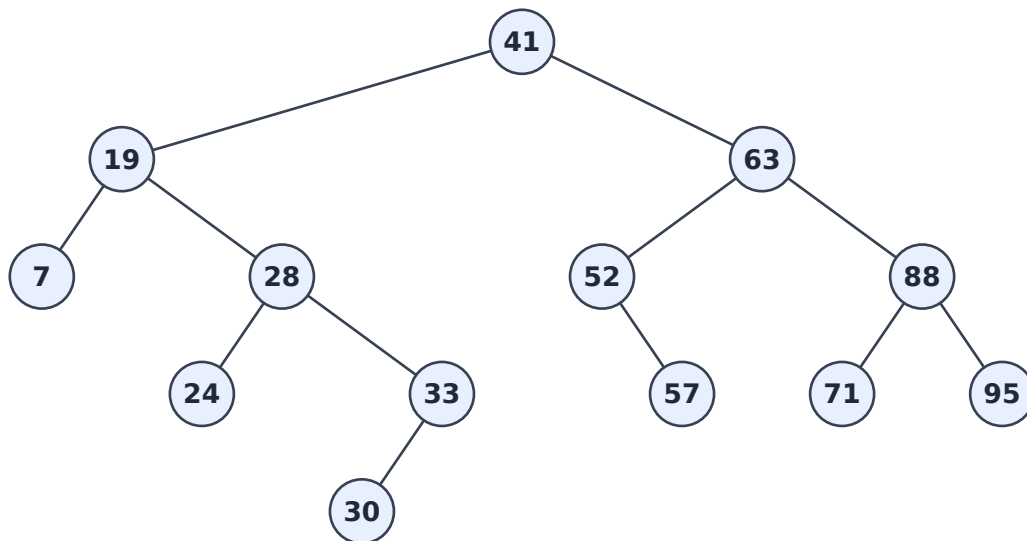
```

1 def partition(A, lo, hi):
2     pivot, i = A[hi], lo - 1
3     for j in range(lo, hi):
4         if A[j] <= pivot:
5             i += 1
6             A[i], A[j] = A[j], A[i]
7     A[i + 1], A[hi] = A[hi], A[i + 1]
8     return i + 1
  
```

- (A) The number of swap statements executed (including the final one) is 3
- (B) If the same procedure is then called on the right part $[9, 8, 7]$, its pivot ends at the first position of that part, so the left sub-part is empty
- (C) After the call $A = [2, 4, 1, 5, 9, 8, 7]$
- (D) The returned index (final position of the pivot) is 3

Q.20 [MSQ · one or more correct | 2 marks | Binary search trees — deletion]

The BST shown below was built by inserting 41, 19, 63, 7, 28, 52, 88, 24, 33, 57, 71, 95, 30 in that order. Now key 41 is deleted and then key 19 is deleted; a node with two children is deleted by copying its **in-order successor** into it and deleting the successor. Which of the following statements about the final tree is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)



Initial BST

- (A) The root of the final tree is 52
- (B) The final tree has exactly 4 leaves
- (C) The pre-order traversal of the final tree begins 52, 24, 7, 28, 33
- (D) The height of the final tree is 4

Answer Key — Mock Test 04

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Linked lists — pointer manipulation in Python
2	NAT	1	82	Doubly linked lists — unlink and move
3	MSQ	1	(A), (B), (C)	Queues using linked lists — operation costs
4	MCQ	1	(D)	Python — string slicing with negative steps
5	NAT	1	4	Stacks — postfix evaluation
6	MCQ	1	(D)	Hashing — separate chaining
7	MCQ	1	(B)	Binary search — probe sequence
8	MCQ	1	(A)	Insertion sort — intermediate state
9	MSQ	1	(B), (C), (D)	Graph traversal — valid BFS orders
10	NAT	1	7	Binary heaps — insertion swaps
11	MCQ	2	(B)	Linked lists — reversal in groups of k
12	NAT	2	9	Linked lists — Floyd's cycle detection
13	NAT	2	9	Circular linked lists — elimination game
14	MSQ	2	(B), (C)	Doubly linked lists — deletion code
15	NAT	2	9	Linked lists — merging sorted lists
16	MCQ	2	(A)	Recursion on linked lists
17	MCQ	2	(C)	Python — shallow vs deep copy

Q	Type	Marks	Key	Topic
18	NAT	2	10	Shortest paths — Dijkstra's algorithm trace
19	MSQ	2	(B), (C), (D)	Quicksort — Lomuto partition
20	MSQ	2	(A), (C), (D)	Binary search trees — deletion

Detailed Solutions — Mock Test 04

Q.1 [MCQ · 1 mark · Linked lists — pointer manipulation in Python]

Answer: (C)

Concept. `head = Node(v, head)` inserts every new node at the *front*, so the list is built in reverse order of the loop. The second loop then bypasses every second node.

Trace.

- After building: `head → 6 → 4 → 9 → 3 → 7 → None`.
- `p = 6`: `p.next (4)` and `p.next.next (9)` exist → `6.next = 9`, `p = 9`.
- `p = 9`: `p.next (3)` and `p.next.next (7)` exist → `9.next = 7`, `p = 7`.
- `p = 7`: `p.next` is `None` → loop stops.
- Final list: `6 → 9 → 7`, printed as `[6, 9, 7]`.

Options.

- (A) Ignores the deletions made by the `while` loop.
- (B) Assumes the list is in insertion order 7, 3, 9, 4, 6 — but front insertion reverses it.
- (C) Correct.
- (D) Would require 7 to be skipped too; after `p = 9`, the node 7 is the new successor and the loop moves onto it instead of deleting it.

Trap: forgetting that prepending reverses order, and that `p = p.next` moves onto the node that was *kept*, so deletions are alternate, not consecutive.



List after bypassing alternate nodes

Q.2 [NAT · 1 mark · Doubly linked lists — unlink and move]

Answer: 82

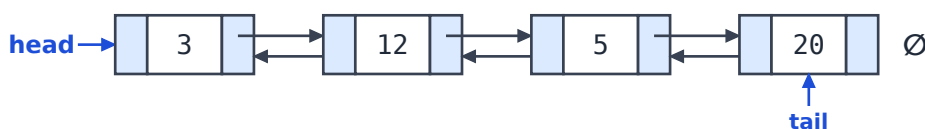
Concept. Unlinking an interior node `x` of a doubly linked list needs exactly two pointer writes: `x.prev.next = x.next` and `x.next.prev = x.prev`. Moving the tail to the front needs the old tail's predecessor (available in $O(1)$ through `prev`).

Trace.

- `x = head.next.next` is the node 8. After the two writes, the list is `12 ⇄ 5 ⇄ 20 ⇄ 3`.
- `t = node 3`; `tail` becomes 20 and `20.next = None`.
- `3.next = 12`, `12.prev = 3`, `head = 3`. The list is `3 ⇄ 12 ⇄ 5 ⇄ 20`.
- The final loop computes $\sum i \times \text{value}$ with 0-based position `i`:

$$0 \times 3 + 1 \times 12 + 2 \times 5 + 3 \times 20 = 0 + 12 + 10 + 60 = \mathbf{82}.$$

Trap: students often think the removed node 8 is still reachable (it is not — no forward pointer leads to it any more), or forget that position counting starts at 0 for the new head 3. Using 1-based weights would give $3 + 24 + 15 + 80 = 122$.



Final list

Q.3 [MSQ · 1 mark · Queues using linked lists — operation costs]**Answer: (A), (B), (C)**

Concept. In a singly linked list you can insert after any node you hold, and delete the node *after* any node you hold, in $O(1)$. Deleting a node needs its predecessor.

- (A) **True.** Without a tail pointer, reaching the last node needs $n - 1$ hops.
- (B) **True.** Enqueue: `tail.next = new; tail = new`. Dequeue: `head = head.next`. Both $O(1)$.
- (C) **True.** With a circular list and pointer `last`, the front is `last.next`. Enqueue: `new.next = last.next; last.next = new; last = new`. Dequeue: `last.next = last.next.next`. Both $O(1)$ — one pointer suffices.
- (D) **False.** Dequeuing at the tail needs the predecessor of the tail so that its `next` can be set to `None`; finding it requires walking from the head — $\Theta(n)$.

Tip: the circular-list-with-last-pointer trick is a favourite GATE question — it gives both ends for the price of one pointer.

Q.4 [MCQ · 1 mark · Python — string slicing with negative steps]**Answer: (D)**

Concept. A slice `s[a:b:c]` with $c < 0$ starts at `a`, moves left by $|c|$, and stops *before* reaching `b`. Negative indices count from the end (-1 is the last character).

Indices: L0 I1 N2 K3 E4 D5 L6 I7 S8 T9.

- `s[-2:2:-2]`: start at $-2 = \text{index } 8 \text{ (S)}$, then 6 (L) , 4 (E) ; the next index 2 is the stop bound, so it is excluded $\rightarrow \text{"SLE"}$.
- `s[1::4]`: indices 1 (I) , 5 (A) , 9 (T) $\rightarrow \text{"IDT"}$.
- Concatenation $\rightarrow \text{SLEIDT}$.

Options.

- (A) lists the first slice in increasing index order (4, 6, 8) — but a negative step walks right-to-left.
- (B) wrongly includes index 3 (K) — but stepping by -2 from 8 never lands on 3.
- (C) starts at index 9 (T), confusing -2 with -1 .

Trap: the stop index is exclusive in both directions, and the step decides which indices are actually visited.

Q.5 [NAT · 1 mark · Stacks — postfix evaluation]**Answer: 4**

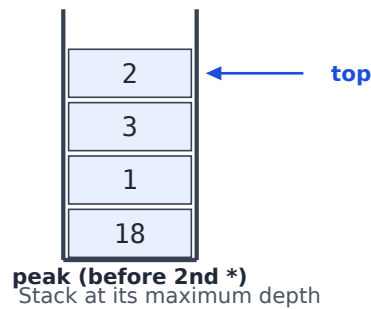
Concept. Each operand is pushed; each binary operator pops two and pushes one (net -1). The peak depth is reached just before an operator that follows a long run of operands.

Trace (stack shown bottom $\rightarrow$ top):

- $8 \rightarrow [8]; 2 \rightarrow [8, 2]; 5 \rightarrow [8, 2, 5]$ (size 3).
- $* \rightarrow [8, 10]; + \rightarrow [18]$.
- $1 \rightarrow [18, 1]; 3 \rightarrow [18, 1, 3]; 2 \rightarrow [18, 1, 3, 2]$ (size 4).
- $* \rightarrow [18, 1, 6]; + \rightarrow [18, 7]; 4 \rightarrow [18, 7, 4]; - \rightarrow [18, 3]; / \rightarrow [6]$.

Maximum size = 4 (the expression's value is 6).

Trap: answering 3 by looking only at the first operand run, or forgetting that the earlier partial result 18 is still sitting at the bottom of the stack.

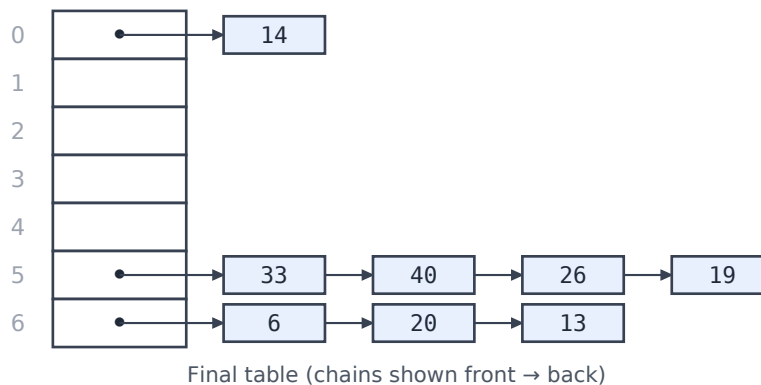
**Q.6 [MCQ · 1 mark · Hashing — separate chaining]****Answer: (D)****Concept.** With front insertion, a chain lists its keys in *reverse* order of arrival.**Hash values.** $19 \rightarrow 5$, $26 \rightarrow 5$, $13 \rightarrow 6$, $40 \rightarrow 5$, $33 \rightarrow 5$, $20 \rightarrow 6$, $6 \rightarrow 6$, $14 \rightarrow 0$.

Slot 5 receives 19, 26, 40, 33 in that order. Front insertion gives:

- after 19: [19]
- after 26: [26, 19]
- after 40: [40, 26, 19]
- after 33: [33, 40, 26, 19]

Options.

- (A) is the order for *tail* insertion.
- (B) is not obtainable by either policy.
- (C) swaps the last two arrivals.

Tip: front insertion is the default in textbook chaining because it is $O(1)$ without a tail pointer; a successful search for 19 now costs 4 key comparisons.**Q.7 [MCQ · 1 mark · Binary search — probe sequence]****Answer: (B)****Concept.** Each probe halves the interval $[lo, hi]$; the search ends unsuccessfully when $lo > hi$.**Trace.**

- $lo = 0$, $hi = 11 \rightarrow mid = 5$, $A[5] = 27 < 47 \rightarrow lo = 6$.
- $lo = 6$, $hi = 11 \rightarrow mid = 8$, $A[8] = 40 < 47 \rightarrow lo = 9$.
- $lo = 9$, $hi = 11 \rightarrow mid = 10$, $A[10] = 51 > 47 \rightarrow hi = 9$.
- $lo = 9$, $hi = 9 \rightarrow mid = 9$, $A[9] = 45 < 47 \rightarrow lo = 10 > hi \rightarrow stop$ (not found).

Values compared: **27, 40, 51, 45**.**Options.**

- (A) uses the *ceiling* midpoint $\lceil (lo+hi)/2 \rceil$ (6, 9, 11, 10).
- (C) stops one probe early — after 51 the interval $[9, 9]$ is still non-empty.

- (D) probes 51 twice, which never happens because hi moves past it.

Trap: always recompute mid with floor division and update lo/hi to $\text{mid} \pm 1$.

Q.8 [MCQ · 1 mark · Insertion sort — intermediate state]

Answer: (A)

Concept. After pass i of insertion sort, the prefix $A[0..i]$ is sorted and contains exactly the original first $i + 1$ elements; the suffix is untouched.

Trace.

- Pass 1 (insert 12): [12, 31, 47, 8, 25, 19].
- Pass 2 (insert 47): no shift $\rightarrow$ [12, 31, 47, 8, 25, 19].
- Pass 3 (insert 8): shift 47, 31, 12 right $\rightarrow$ [8, 12, 31, 47, 25, 19].

Options.

- (B) is the state after pass 2.
- (C) is the state after pass 4 — 25 has been inserted too.
- (D) mixes 25 into the prefix without fully sorting it — insertion sort never produces this.

Contrast: selection sort after 3 passes would instead hold the 3 *smallest overall* (8, 12, 19) at the front. Insertion sort's prefix is sorted but not final.

Array after each pass						
start	31	12	47	8	25	19
pass 1	12	31	47	8	25	19
pass 2	12	31	47	8	25	19
pass 3	8	12	31	47	25	19

Q.9 [MSQ · 1 mark · Graph traversal — valid BFS orders]

Answer: (B), (C), (D)

Concept. In BFS, when a vertex u is dequeued, all its *not-yet-discovered* neighbours are enqueued together. So the next-level vertices appear grouped by the parent that discovered them, in the order of the parents.

Level 1 = {B, C, D} in any order; level 2 = {E, F}; level 3 = {G}.

- (A) B is dequeued first and discovers E, so E must come before F. **Invalid.**
- (B) B first discovers E; C then discovers F; D nothing. Order E, F. **Valid.**
- (C) C is dequeued first and discovers both E and F (any order, here F then E); B and D add nothing. **Valid.**
- (D) D first discovers F; C then discovers E; B nothing new. Order F, E. **Valid.**

Trap: checking only the *levels* is not enough — (A) has correct levels but violates the FIFO order of discovery.

Q.10 [NAT · 1 mark · Binary heaps — insertion swaps]

Answer: 7

Concept. Insertion places the key at the next free leaf and swaps it with its parent while it is smaller than the parent. Parent of index i is $(i - 1) // 2$.

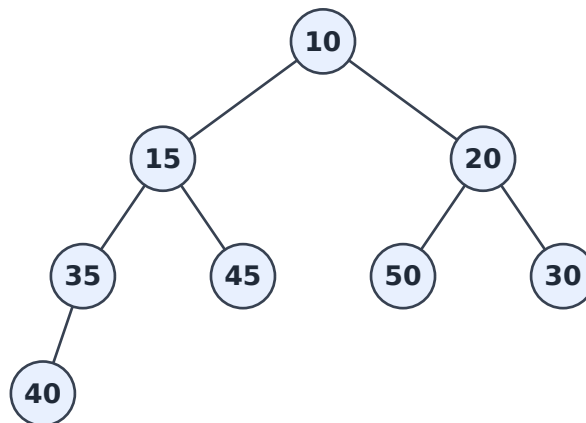
Trace (heap array after each insertion, swaps in brackets):

- 40 $\rightarrow$ [40] (0)
- 35 $\rightarrow$ [35, 40] (1)
- 50 $\rightarrow$ [35, 40, 50] (0)

- 20 → at index 3, swaps with 40, then with 35 → [20, 35, 50, 40] (2)
- 45 → parent 35 < 45 → [20, 35, 50, 40, 45] (0)
- 10 → swaps with 50, then with 20 → [10, 35, 20, 40, 45, 50] (2)
- 30 → parent 20 < 30 → [10, 35, 20, 40, 45, 50, 30] (0)
- 15 → swaps with 40, then with 35; parent 10 < 15 stops → [10, 15, 20, 35, 45, 50, 30, 40] (2)

Total = 1 + 2 + 2 + 2 = **7**.

Trap: do not count the final comparison that stops sift-up as a swap, and do not confuse repeated insertion with bottom-up build-heap (which would perform a different number of swaps).



Final min-heap

Q.11 [MCQ · 2 marks · Linked lists — reversal in groups of k]

Answer: (B)

Concept. `rev_k` reverses the first k nodes in place, then links the *old* head (which is now the last node of the reversed block) to the result of recursively processing the rest. A final block with fewer than k nodes is also reversed, because the *while* loop simply stops when *cur* becomes None.

The tuple assignment. In `cur.next, prev, cur = prev, cur, cur.next` the right side is evaluated first, giving (old prev, old cur, old cur.next); then targets are assigned left to right. `cur.next` is assigned while *cur* still refers to the old node, so the step is a correct pointer reversal.

Trace.

- Block 1: P Y T → T Y P; P.next ← `rev_k`(H...).
- Block 2: H O N → N O H; H.next ← `rev_k`(I...).
- Block 3: I C (only 2 nodes) → C I; *cur* is None, so no further call.
- Result: T Y P N O H C I → **TYPNOHCI**.

Options.

- (A) assumes an incomplete last group is left as is (that variant needs a length check first).
- (C) is a full reversal of the whole list.
- (D) reverses only the first group.

Trap: if the targets were written as `prev, cur, cur.next = cur, cur.next, prev, cur.next` would be assigned on the *new* cur and the list would be corrupted — order of targets matters.



Output list

Q.12 [NAT · 2 marks · Linked lists — Floyd's cycle detection]**Answer: 9**

Concept. If the tail before the cycle has μ nodes and the cycle has length λ , then phase 1 ends at a step count that is a multiple of λ and $\geq \mu$, and phase 2 always takes exactly μ steps, ending at the first node of the cycle.

Here $\mu = 3$ (nodes 1, 2, 3) and $\lambda = 6$ (cycle $4 \rightarrow 5 \rightarrow \dots \rightarrow 9 \rightarrow 4$).

Phase 1 trace (slow, fast after each step):

- step 1: (2, 3)
- step 2: (3, 5)
- step 3: (4, 7)
- step 4: (5, 9)
- step 5: (6, 5) [fast: $9 \rightarrow 4 \rightarrow 5$]
- step 6: (7, 7) $\rightarrow$ meet. **m = 6.**

Phase 2 trace (slow from 1, fast from 7, one step each):

- (2, 8), (3, 9), (4, 4) $\rightarrow$ meet at node 4, the cycle entry. **r = 3.**

$m + r = 6 + 3 = 9$.

Check with the theory: m is the smallest multiple of $\lambda = 6$ that is $\geq \mu = 3$, i.e. 6; $r = \mu = 3$.

Trap: counting the starting position as a step, or moving **fast** only once in the step when it wraps from 9 to 4.

Q.13 [NAT · 2 marks · Circular linked lists — elimination game]**Answer: 9**

Concept. p always sits on the node *before* the one to be removed. Starting from the last node, moving $k - 1 = 3$ steps lands on the node preceding the 4th node, which is then unlinked. This is the Josephus problem with $n = 11$, $k = 4$ (every 4th person removed, counting resumes from the node after the removed one).

Trace (removed node each round, remaining circle after it):

- remove 4 $\rightarrow$ 1 2 3 5 6 7 8 9 10 11
- remove 8 $\rightarrow$ 1 2 3 5 6 7 9 10 11
- remove 1 $\rightarrow$ 2 3 5 6 7 9 10 11
- remove 6 $\rightarrow$ 2 3 5 7 9 10 11
- remove 11 $\rightarrow$ 2 3 5 7 9 10
- remove 7 $\rightarrow$ 2 3 5 9 10
- remove 3 $\rightarrow$ 2 5 9 10
- remove 2 $\rightarrow$ 5 9 10
- remove 5 $\rightarrow$ 9 10
- remove 10 $\rightarrow$ 9

Only node 9 remains, its **next** points to itself, the loop ends and **9** is printed.

Cross-check with the recurrence $J(1) = 0$, $J(m) = (J(m-1) + k) \bmod m$ (0-based): $J(11) = 8$, i.e. person 9.

Trap: starting p at **first** instead of the last node shifts every removal by one and gives a different survivor.

Q.14 [MSQ · 2 marks · Doubly linked lists — deletion code]**Answer: (B), (C)**

Concept. Deleting x needs $x.\text{prev}.next = x.next$ (unless x is first) and $x.next.prev = x.prev$ (unless x is last). Every dereference of $x.\text{prev}$ or $x.next$ must be guarded by a None check.

- (A) **False.** Same reason: the single node is also the first node, `x.prev` is `None`, and an `AttributeError` is raised.
- (B) **True.** Interior node: both neighbours exist; the two writes splice them together, and `x is head` is false.
- (C) **True.** Last node: `x.prev` exists, so `x.prev.next = None`; the `if x.next` guard skips the second write. The predecessor becomes the new last node.
- (D) **False.** For the first node `x.prev` is `None`, so the very first line evaluates `None.next = ...` and raises `AttributeError` before the `x is head` check is reached.

Fix: write `if x.prev: x.prev.next = x.next else: head = x.next` — i.e. test the head case *before* dereferencing `x.prev`.

Trap: the head check exists in the code, so it *looks* handled — but its position after the unguarded dereference makes it unreachable for the head node.

Q.15 [NAT · 2 marks · Linked lists — merging sorted lists]

Answer: 9

Concept. Each comparison moves exactly one node to the output. The process stops when one list is empty, so $\# \text{comparisons} = (m + n) - (\text{number of nodes left in the other list at that moment})$.

Trace (comparison $\rightarrow$ node moved):

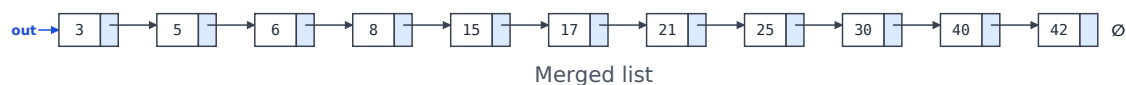
1. 3 vs 5 $\rightarrow$ 3
2. 8 vs 5 $\rightarrow$ 5
3. 8 vs 6 $\rightarrow$ 6
4. 8 vs 17 $\rightarrow$ 8
5. 15 vs 17 $\rightarrow$ 15
6. 21 vs 17 $\rightarrow$ 17
7. 21 vs 25 $\rightarrow$ 21
8. 30 vs 25 $\rightarrow$ 25
9. 30 vs 40 $\rightarrow$ 30 $\rightarrow$ L1 is now empty.

The remaining 40 $\rightarrow$ 42 of L2 is attached with one pointer assignment. Comparisons = $11 - 2 = 9$.

Bounds to remember: merging lists of sizes m and n needs at least $\min(m, n) = 5$ and at most $m + n - 1 = 10$ comparisons.

Tip: a linked-list merge needs no extra array — it only relinks `next` pointers, which is why merge sort is the preferred sort for linked lists.

Trap: answering $m + n - 1 = 10$ (the worst case) without tracing.



Q.16 [MCQ · 2 marks · Recursion on linked lists]

Answer: (A)

Concept. The recursive call is made *before* the print, so printing happens while the recursion unwinds — i.e. from the tail back towards the head. The return value is an alternating sum computed right to left.

Step 1 — the list. Front insertion of 3, 9, 1, 7, 4 gives head $\rightarrow 4 \rightarrow 7 \rightarrow 1 \rightarrow 9 \rightarrow 3$.

Step 2 — depths. 4 ($d=0$), 7 ($d=1$), 1 ($d=2$), 9 ($d=3$), 3 ($d=4$). Odd depths: 7 and 9.

Step 3 — unwinding.

- $g(\text{None}) = 0$.
- $d=4$, node 3: no print, returns $3 - 0 = 3$.
- $d=3$, node 9: prints 9, returns $9 - 3 = 6$.

- $d=2$, node 1: no print, returns $1 - 6 = -5$.
- $d=1$, node 7: prints 7, returns $7 - (-5) = 12$.
- $d=0$, node 4: no print, returns $4 - 12 = -8$.

The outer `print` then writes -8 on the same line $\rightarrow 9\ 7\ -8$.

Options.

- (B) prints in forward order — that would need the print before the recursive call.
- (C) loses the sign: $4 - (7 - (1 - (9 - 3))) = -8$, not 8.
- (D) prints even-depth nodes.

Trap: the output of `g` appears only after all the prints *inside* `g`, because the argument of the outer `print` must be fully evaluated first.

Q.17 [MCQ · 2 marks · Python — shallow vs deep copy]

Answer: (C)

Concept. `a[:]` makes a *shallow* copy: a new outer list whose elements are the **same** inner list objects. `copy.deepcopy` recursively copies the inner lists too. Mutating an object is visible through every reference to it; rebinding a slot of one outer list is not.

Trace.

- `b = a[:]` $\rightarrow$ `b` is a new list, but `b[0]` is `a[0]` and `b[1]` is `a[1]`.
- `c` is completely independent.
- `b[0].append(5)` mutates the shared inner list $\rightarrow$ `a[0]` is now `[1, 2, 5]`.
- `b[1] = [6]` rebinds only `b`'s slot 1; `a[1]` is still `[3, 4]`.
- `c[0][0] = 9` changes only `c`'s private copy $\rightarrow$ `c[0] = [9, 2]`.
- `b[0] is a[0]` is still `True`.

Output: `[[1, 2, 5], [3, 4]] True [9, 2]`.

Options.

- (A) treats `b[1] = [6]` as if it changed `a` — rebinding a slot never affects another list.
- (B) treats `a[:]` as a deep copy.
- (D) lets the deep copy's change leak back into `a`.

Trap: distinguish *mutation* (`append`, item assignment on a shared object) from *rebinding* (assigning a new object to a name or slot).

Q.18 [NAT · 2 marks · Shortest paths — Dijkstra's algorithm trace]

Answer: 10

Concept. Dijkstra repeatedly extracts the unfinished vertex with the smallest tentative distance and relaxes its outgoing edges: $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$.

Trace (updates in brackets):

- Extract S (0): `A = 7, B = 2, C = 9` [3 updates; total 3].
- Extract B (2): `A = min(7, 5) = 5 ✓, D = 10 ✓` [2; total 5].
- Extract A (5): `C = min(9, 6) = 6 ✓, D = min(10, 9) = 9 ✓` [2; total 7].
- Extract C (6): `T = 12 ✓, D = min(9, 7) = 7 ✓` [2; total 9].
- Extract D (7): `T = min(12, 9) = 9 ✓` [1; total 10].
- Extract T (9): no outgoing edges.

Total updates = **10**. Final distances: S 0, B 2, A 5, C 6, D 7, T 9; shortest path to T is `S  $\rightarrow$  B  $\rightarrow$  A  $\rightarrow$  C  $\rightarrow$  D  $\rightarrow$  T`.

Trap: D is improved three times ($10 \rightarrow 9 \rightarrow 7$) — miscounting such repeated improvements is the usual error. A relaxation that does not decrease the value is *not* an update.

Tentative distances after each extraction

	S	A	B	C	D	T
init	0	∞	∞	∞	∞	∞
S	0	7	2	9	∞	∞
B	0	5	2	9	10	∞
A	0	5	2	6	9	∞
C	0	5	2	6	7	12
D	0	5	2	6	7	9

Q.19 [MSQ · 2 marks · Quicksort — Lomuto partition]**Answer: (B), (C), (D)**

Concept. Lomuto keeps $A[\text{lo}..i] \leq \text{pivot}$. Each element $\leq \text{pivot}$ found by j is swapped into position $i + 1$; at the end the pivot is swapped into $i + 1$.

Trace (pivot 5):

- $j=0$: $7 > 5$.
- $j=1$: $2 \leq 5 \rightarrow i=0$, swap $A[0], A[1] \rightarrow [2, 7, 9, 4, 1, 8, 5]$ (swap 1).
- $j=2$: $9 > 5$.
- $j=3$: $4 \leq 5 \rightarrow i=1$, swap $A[1], A[3] \rightarrow [2, 4, 9, 7, 1, 8, 5]$ (swap 2).
- $j=4$: $1 \leq 5 \rightarrow i=2$, swap $A[2], A[4] \rightarrow [2, 4, 1, 7, 9, 8, 5]$ (swap 3).
- $j=5$: $8 > 5$.
- Final: swap $A[3], A[6] \rightarrow [2, 4, 1, 5, 9, 8, 7]$ (swap 4); return 3.

Statements.

- (A) **False** — 4 swap statements execute (3 in the loop + the final one).
- (B) **True** — for $[9, 8, 7]$ the pivot 7 is the minimum, no j satisfies $A[j] \leq 7$, i stays at $\text{lo} - 1$, and the pivot is swapped to the first position: $[7, 8, 9]$ with an empty left part — the classic unbalanced split.
- (C) **True** — matches the trace.
- (D) **True** — returned index is 3 (three keys are smaller than 5).

Trap: forgetting the final pivot swap when counting swaps.

Q.20 [MSQ · 2 marks · Binary search trees — deletion]**Answer: (A), (C), (D)**

Concept. The in-order successor of a node with two children is the minimum of its right subtree; it has no left child, so removing it is a 0- or 1-child deletion.

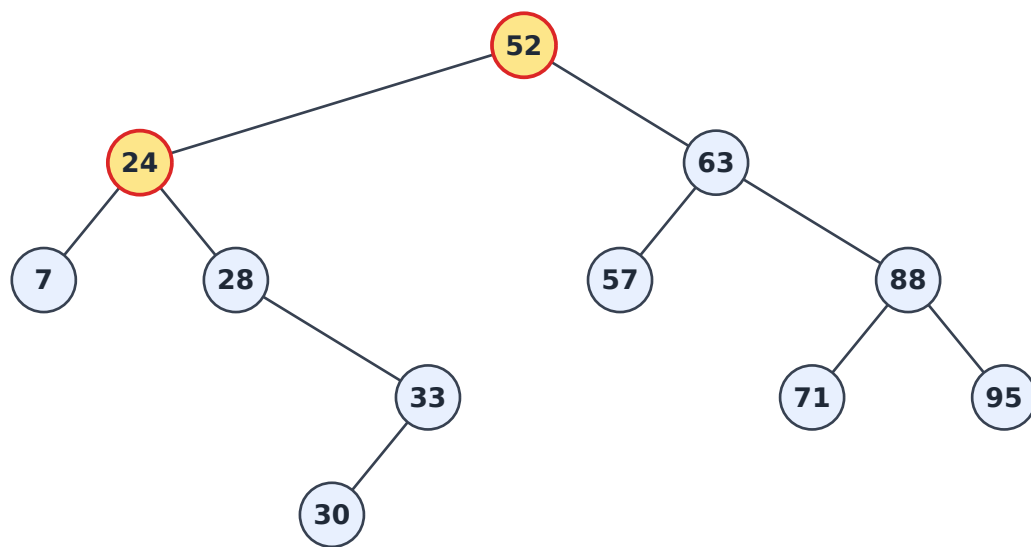
Delete 41. Successor = leftmost node of the subtree rooted at 63 $\rightarrow 63 \rightarrow 52$ (no left child). Copy 52 into the root and delete the old 52, whose right child 57 moves up to become 63's left child.

Delete 19. 19 has two children; successor = leftmost of subtree 28 $\rightarrow 24$ (a leaf). Copy 24 into 19's node and delete the leaf 24. Now 28 has no left child.

Final tree: 52 $\rightarrow$ left 24 (children 7 and 28), 28 $\rightarrow$ right 33, 33 $\rightarrow$ left 30; 52 $\rightarrow$ right 63 (children 57 and 88), 88 $\rightarrow$ children 71 and 95.

- (A) **True.**
- (B) **False** — the leaves are 7, 30, 57, 71, 95: five leaves.
- (C) **True** — pre-order is 52, 24, 7, 28, 33, 30, 63, 57, 88, 71, 95.
- (D) **True** — path 52 $\rightarrow$ 24 $\rightarrow$ 28 $\rightarrow$ 33 $\rightarrow$ 30 has 4 edges.

Trap: using the in-order predecessor (33 for 41) gives a completely different tree; always follow the convention stated in the question.



Final BST

Mock Test 05

Binary Trees & Traversals

Difficulty: Moderate

Focus areas: traversals, reconstruction from traversals, height, node counts

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Binary trees — reconstruction from traversals]

A binary tree with distinct labels has

- pre-order traversal: K, C, A, G, E, H, T, X, V
- in-order traversal: A, C, E, G, H, K, T, V, X

Which of the following is its post-order traversal?

- (A) A, E, H, G, C, V, X, T, K
- (B) A, E, H, G, C, X, V, T, K
- (C) A, E, G, H, C, V, X, T, K
- (D) E, H, G, A, C, V, X, T, K

Q.2 [NAT · numerical answer | 1 mark | Binary trees — node counts]

A binary tree has 40 nodes in total. Exactly 7 of its nodes have exactly one child. The number of leaves in the tree is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Python — string slicing]

Consider the following Python program. What is printed?

Python

```
1 s = "TRAVERSAL"
2 print(s[::-2], s[-3:1:-1], s[1::3])
```

- (A) LSEAT SREVAR RVA
- (B) LSEAT SREVA REA
- (C) LAEST SREVA REA
- (D) LSEAT SREV REA

Q.4 [MSQ · one or more correct | 1 mark | Python — aliasing and shallow copy]

Consider the following Python code. After it executes, which of the following expressions evaluate to **True**?

Python

```
1 x = [1, 2, [3, 4]]
2 y = x[:]
3 z = x
4 y[2].append(5)
5 z.append(6)
6 y[0] = 9
```

- (A) `y == [9, 2, [3, 4, 5]]`
- (B) `x == [1, 2, [3, 4, 5], 6]`
- (C) `len(y) == 4`
- (D) `x[2] is y[2]`

Q.5 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with a stack, where every operand is a single integer token and `/` denotes integer (floor) division. The second operand popped is the left operand.

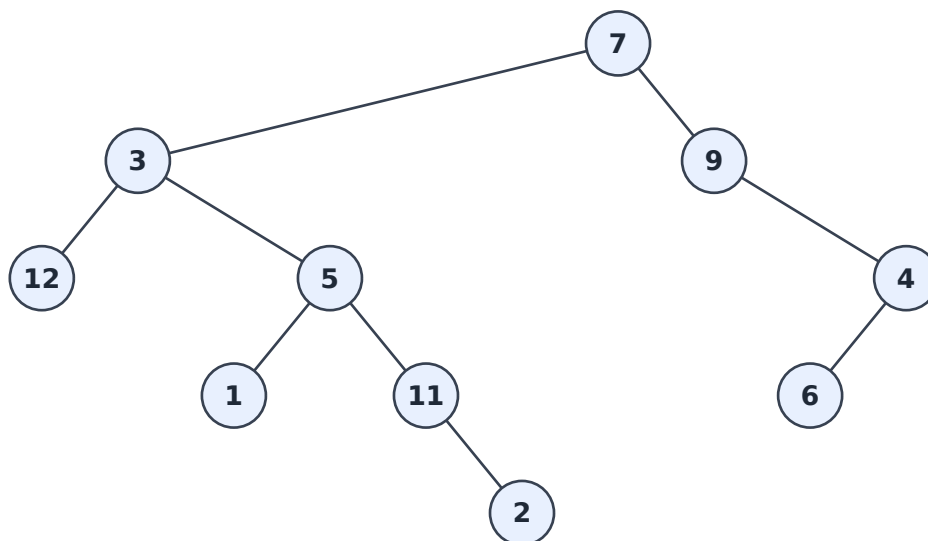
9 4 2 - 3 * + 7 2 / -

The value of the expression is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Binary trees — level-order traversal]

Consider the binary tree shown. Which of the following is its level-order (breadth-first, left to right within each level) traversal?



Binary tree T

- (A) 7, 9, 3, 12, 5, 4, 6, 11, 1, 2
- (B) 7, 3, 12, 5, 1, 11, 2, 9, 4, 6
- (C) 7, 3, 9, 12, 5, 4, 1, 11, 6, 2
- (D) 7, 3, 9, 12, 5, 4, 1, 11, 2, 6

Q.7 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

The keys 44, 17, 82, 59, 31, 96, 23, 70, 38, 65 are inserted into an initially empty hash table with 7 slots (indices 0–6) using $h(k) = k \bmod 7$ and separate chaining. Let L be the length of the longest chain and E the number of empty slots after all insertions. The value of $L + E$ is _____.

Answer: _____ (enter a numerical value)

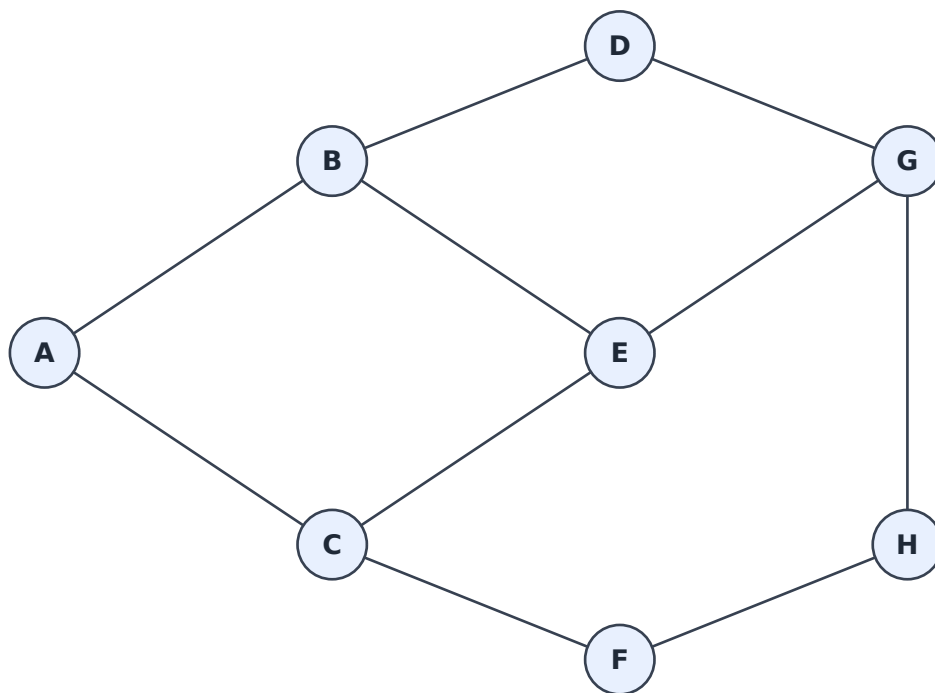
Q.8 [MCQ · single correct | 1 mark | Insertion sort — number of shifts]

Insertion sort (ascending) is applied to the array [5, 8, 2, 9, 1, 7, 3]. A *shift* is one move of an element one position to the right to make room for the key being inserted. The total number of shifts performed is

- (A) 11
- (B) 12
- (C) 13
- (D) 21

Q.9 [NAT · numerical answer | 1 mark | Graphs — BFS levels]

Breadth-first search is run from vertex A on the undirected graph shown. The number of vertices whose BFS distance (number of edges) from A is exactly 2 is _____.



Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Python — recursion on trees]

A binary tree is stored as nested tuples (value, left, right). Consider the following Python program. What is printed?

Python

```
1 def f(t, d=0):
2     if t is None:
3         return 0
4     v, l, r = t
5     if l is None and r is None:
6         return v if d % 2 == 0 else -v
7     return f(l, d + 1) + f(r, d + 1)
8
9 T = (1, (2, (4, None, None), (5, (8, None, None), None)),
10      (3, None, (6, (9, None, None), (7, None, None))))
11 print(f(T))
```

- (A) -20
- (B) 20
- (C) -12
- (D) 4

Q.11 [MSQ · one or more correct | 2 marks | Binary trees — reconstruction from post-order and in-order]

A binary tree with distinct keys has

- in-order traversal: 4, 2, 9, 5, 7, 1, 3, 8, 6
- post-order traversal: 4, 9, 2, 7, 5, 8, 6, 3, 1

Which of the following statements about this tree is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) Node 3 has a left child
- (B) Its height is 3
- (C) Its pre-order traversal is 1, 5, 2, 4, 9, 7, 3, 6, 8
- (D) It has exactly 4 leaves

Q.12 [NAT · numerical answer | 2 marks | Binary trees — counting trees with given traversals]

Consider binary trees on the five labelled nodes P, Q, R, S, T. The number of distinct such binary trees whose pre-order traversal is P, Q, R, S, T **and** whose post-order traversal is T, S, R, Q, P is _____.

Answer: _____ (enter a numerical value)

Q.13 [MCQ · single correct | 2 marks | Python — closures and late binding]

Consider the following Python program. What is printed?

Python

```
1 fs = []
2 for i in range(4):
3     fs.append(lambda x, i=i: x * i if i % 2 else x + i)
4 gs = [lambda x: x * i for i in range(4)]
5 print(sum(f(3) for f in fs), sum(g(3) for g in gs))
```

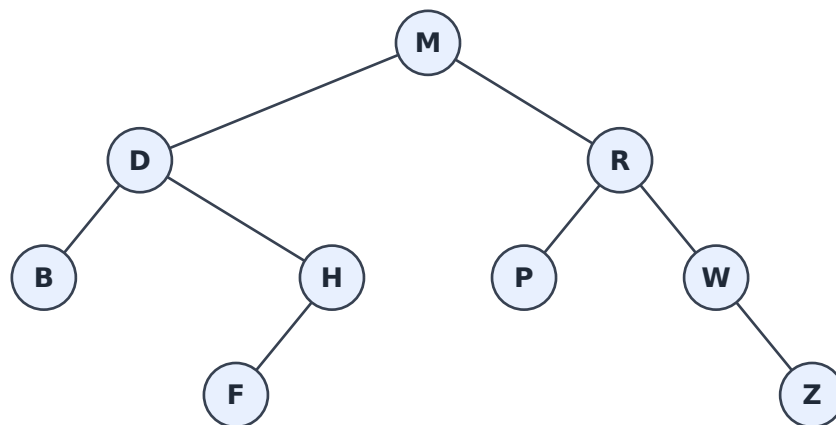
- (A) 18 36
- (B) 20 18
- (C) 36 36
- (D) 20 36

Q.14 [MCQ · single correct | 2 marks | Binary trees — iterative traversal with a stack]

The function `walk` below is applied to the tree shown (stored as nested tuples `(label, left, right)`). What list does `walk(T)` return?

Python

```
1 def walk(t):
2     out, st = [], [t]
3     while st:
4         node = st.pop()
5         if node is None:
6             continue
7         out.append(node[0])
8         st.append(node[1])
9         st.append(node[2])
10    return out
```



Tree T

- (A) M, D, B, H, F, R, P, W, Z
- (B) B, F, H, D, P, Z, W, R, M
- (C) M, R, W, Z, P, D, H, F, B
- (D) M, R, P, W, Z, D, B, H, F

Q.15 [MSQ · one or more correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition procedure below is applied once to $A = [7, 2, 9, 4, 3, 8, 5]$ with $lo = 0$ and $hi = 6$. Which of the following statements is/are TRUE?

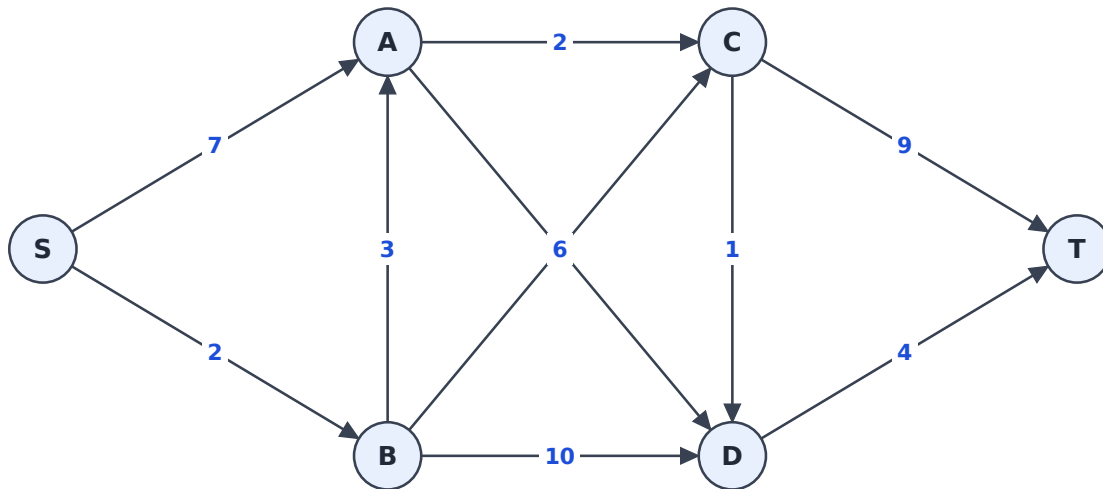
Python

```
1 def partition(A, lo, hi):
2     pivot, i = A[hi], lo - 1
3     for j in range(lo, hi):
4         if A[j] <= pivot:
5             i += 1
6             A[i], A[j] = A[j], A[i]
7     A[i + 1], A[hi] = A[hi], A[i + 1]
8     return i + 1
```

- (A) Exactly 3 swap statements (including the final one) are executed
- (B) After partition $A = [2, 4, 3, 5, 9, 8, 7]$
- (C) The sub-array to the right of the pivot is $[9, 8, 7]$
- (D) The returned index is 3

Q.16 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from source S on the weighted directed graph shown. The sum of the final shortest-path distances of **all six** vertices (including S itself) is _____.



Answer: _____ (enter a numerical value)

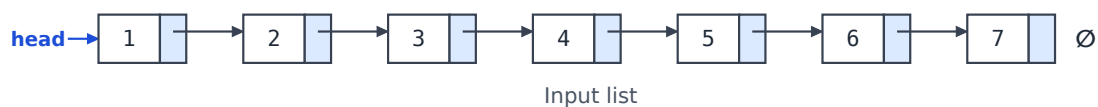
Q.17 [MCQ · single correct | 2 marks | Linked lists — recursive pointer manipulation]

Consider the following Python program operating on the singly linked list shown. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def build(vals):
6     head = None
7     for v in reversed(vals):
8         head = Node(v, head)
9     return head
10
11 def mystery(h):
12     if h is None or h.nxt is None:
13         return h
14     second = h.nxt
15     h.nxt = mystery(second.nxt)
16     second.nxt = h
17     return second
18
19 h = mystery(build([1, 2, 3, 4, 5, 6, 7]))
20 out = []
21 while h:
22     out.append(h.v)
23     h = h.nxt
24 print(out)
  
```



- (A) [1, 3, 2, 5, 4, 7, 6]
- (B) [7, 6, 5, 4, 3, 2, 1]
- (C) [2, 1, 4, 3, 6, 5]
- (D) [2, 1, 4, 3, 6, 5, 7]

Q.18 [NAT · numerical answer | 2 marks | Binary search — probe counting]

The function `bsearch` below is used on the sorted array `A` shown (0-based indices). Let p_1 be the value of `probes` returned when searching for $x = 50$, and p_2 the value returned when searching for $x = 69$. The value of $p_1 + p_2$ is _____.

Python

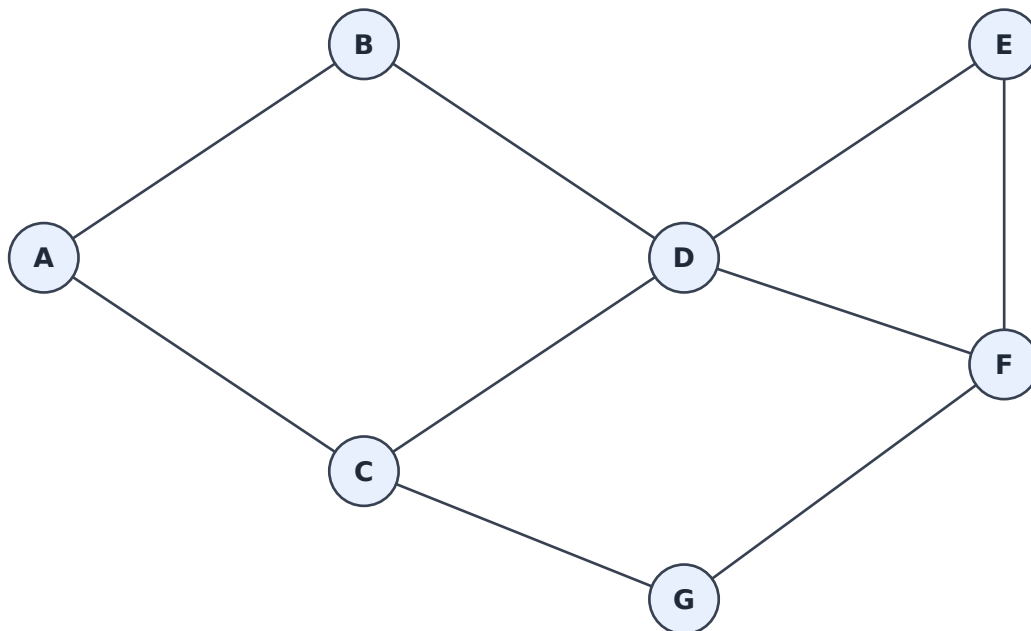
```
1 def bsearch(A, x):
2     lo, hi, probes = 0, len(A) - 1, 0
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         probes += 1
6         if A[mid] == x:
7             return mid, probes
8         if A[mid] < x:
9             lo = mid + 1
10        else:
11            hi = mid - 1
12    return -1, probes
```

A =	4	9	13	18	22	27	31	36	42	47	53	58	64	69	75
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Graphs — DFS and BFS trees]

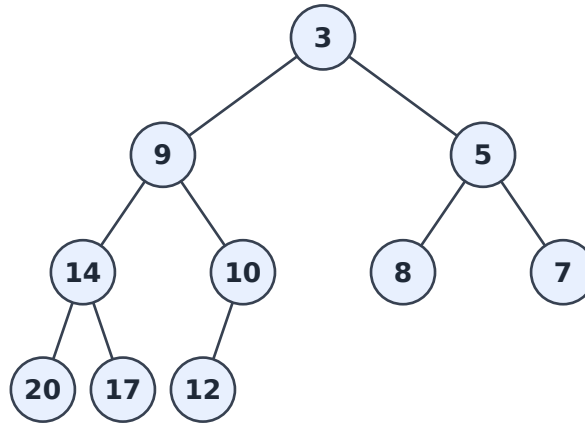
Consider the undirected graph shown. Depth-first search (recursive) and breadth-first search are both started at A; whenever there is a choice, neighbours are explored in alphabetical order. Which of the following statements is/are TRUE?



- (A) In the BFS tree, E is at level 3
- (B) Exactly 3 edges of the graph are not DFS-tree edges
- (C) The DFS visit order is A, B, D, C, G, F, E
- (D) In the DFS tree, the depth of E (edges from A) is 5

Q.20 [MSQ · one or more correct | 2 marks | Heaps — repeated delete-min]

The min-heap shown is stored in the array $H = [3, 9, 5, 14, 10, 8, 7, 20, 17, 12]$ (0-based; children of index i are $2i + 1$ and $2i + 2$). Two delete-min operations are performed; each moves the last element to the root and sifts it down, swapping with the **smaller** child while that child is smaller. Which of the following statements is/are TRUE?



Initial min-heap

- (A) After the second delete-min, the root is 7 and its children are 9 and 8
- (B) After the second delete-min, key 17 is at index 5
- (C) After the second delete-min, key 14 is a leaf
- (D) After the first delete-min, $H = [5, 9, 7, 14, 10, 8, 12, 20, 17]$

Answer Key — Mock Test 05

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Binary trees — reconstruction from traversals
2	NAT	1	17	Binary trees — node counts
3	MCQ	1	(B)	Python — string slicing
4	MSQ	1	(A), (B), (D)	Python — aliasing and shallow copy
5	NAT	1	12	Stacks — postfix evaluation
6	MCQ	1	(C)	Binary trees — level-order traversal
7	NAT	1	7	Hashing — separate chaining
8	MCQ	1	(B)	Insertion sort — number of shifts
9	NAT	1	3	Graphs — BFS levels
10	MCQ	1	(A)	Python — recursion on trees
11	MSQ	2	(B), (C), (D)	Binary trees — reconstruction from post-order and in-order
12	NAT	2	16	Binary trees — counting trees with given traversals
13	MCQ	2	(D)	Python — closures and late binding
14	MCQ	2	(C)	Binary trees — iterative traversal with a stack
15	MSQ	2	(B), (C), (D)	Quicksort — Lomuto partition
16	NAT	2	34	Shortest paths — Dijkstra
17	MCQ	2	(D)	Linked lists — recursive pointer manipulation
18	NAT	2	7	Binary search — probe counting
19	MSQ	2	(A), (B), (C)	Graphs — DFS and BFS trees
20	MSQ	2	(A), (B), (D)	Heaps — repeated delete-min

Detailed Solutions — Mock Test 05

Q.1 [MCQ · 1 mark · Binary trees — reconstruction from traversals]

Answer: (A)

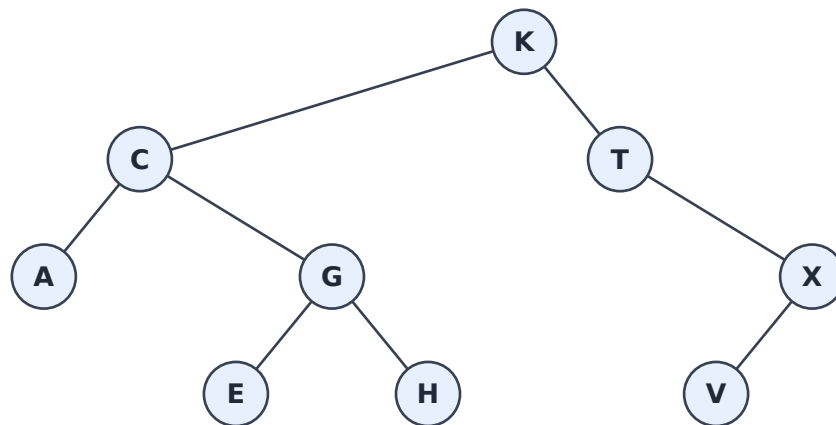
Concept. The first pre-order symbol is the root; its position in the in-order sequence splits the remaining symbols into the left and right subtrees. Recurse.

- Root K. In-order left of K: {A, C, E, G, H}; right of K: {T, V, X}.
- Left part, pre-order C, A, G, E, H → root C; in-order A | C | E, G, H, so A is C's left child and the subtree {E, G, H} (pre-order G, E, H) has root G with children E and H.
- Right part, pre-order T, X, V → root T; in-order T, V, X has nothing left of T, so T has only a right subtree {V, X} with root X; V lies left of X in in-order, so V is X's **left** child.

Post-order (left, right, root): A, E, H, G, C, V, X, T, K → option (A).

- (B) puts X before V, i.e. treats V as a child *after* X — wrong because V is X's left child and post-order lists children before the parent.
- (C) swaps G and H: G is the parent of H, so G must follow H.
- (D) moves A after E, H, G: A is in the left subtree of C, visited first.

Tip: always verify by re-deriving the in-order of the reconstructed tree.



Reconstructed tree

Q.2 [NAT · 1 mark · Binary trees — node counts]

Answer: 17

Concept. In any non-empty binary tree, if n_0 , n_1 , n_2 are the numbers of nodes with 0, 1, 2 children, then $n_0 = n_2 + 1$. Proof by edge counting: there are $n - 1$ edges, and every edge leaves a parent, so $n - 1 = n_1 + 2n_2$. Substituting $n = n_0 + n_1 + n_2$ gives $n_0 = n_2 + 1$.

Computation.

- $n_0 + n_1 + n_2 = 40$ with $n_1 = 7 \rightarrow n_0 + n_2 = 33$.
- $n_0 = n_2 + 1 \rightarrow 2n_2 + 1 = 33 \rightarrow n_2 = 16$.
- Leaves $n_0 = 17$.

Check: edges = $n_1 + 2n_2 = 7 + 32 = 39 = 40 - 1$. ✓

Trap: the count of one-child nodes does not affect the relation $n_0 = n_2 + 1$; it only enters through the total. Answering 33/2 or forgetting the +1 gives 16.

Q.3 [MCQ · 1 mark · Python — string slicing]**Answer: (B)**

Concept. `s[start:stop:step]` includes `start`, excludes `stop`; a negative step walks right-to-left, and omitted bounds default to the appropriate end.

Indices: T0 R1 A2 V3 E4 R5 S6 A7 L8.

- `s[::-2]`: start at index 8 and step $-2 \rightarrow 8, 6, 4, 2, 0 \rightarrow L, S, E, A, T = \text{LSEAT}.$
- `s[-3:1:-1]`: -3 means index 6; go down while index $> 1 \rightarrow 6, 5, 4, 3, 2 \rightarrow S, R, E, V, A = \text{SREVA}.$
- `s[1::3]`: $1, 4, 7 \rightarrow R, E, A = \text{REA}.$

Output: `LSEAT SREVA REA` $\rightarrow$ (B).

- (A) includes index 1 (R) in the second slice and uses step 2 in the third.
- (C) reads the even-indexed characters left to right and then reverses wrongly.
- (D) stops one early, as if the stop index 1 were index 2.

Trap: the stop index is excluded even when stepping backwards.

Q.4 [MSQ · 1 mark · Python — aliasing and shallow copy]**Answer: (A), (B), (D)**

Concept. `x[:]` is a *shallow* copy: a new outer list whose elements are the same objects. `z = x` creates no copy at all, only a second name.

Trace:

- `y = x[:]` $\rightarrow$ `y` is a new list, but `y[2]` and `x[2]` are the *same* inner list.
- `y[2].append(5)` mutates that shared inner list $\rightarrow$ both see `[3, 4, 5]`.
- `z.append(6)` mutates `x` (`z` is `x`) $\rightarrow x = [1, 2, [3, 4, 5], 6]$; `y` unaffected.
- `y[0] = 9` rebinds slot 0 of `y` only $\rightarrow y = [9, 2, [3, 4, 5]]$.
- (A) True.
- (B) True.
- (C) False — `y` has 3 elements; the append went to `x`.
- (D) True — the shallow copy shares the inner list object.

Trap: mutating a nested object through a shallow copy is visible in the original; rebinding a top-level slot is not.

Q.5 [NAT · 1 mark · Stacks — postfix evaluation]**Answer: 12**

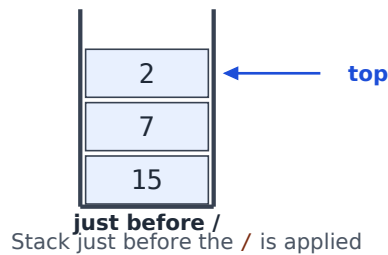
Concept. Scan left to right: push operands; on an operator pop the right operand `b`, then the left operand `a`, push `a op b`.

Trace (stack shown bottom $\rightarrow$ top):

- `9, 4, 2` $\rightarrow [9, 4, 2]$
- `-` $\rightarrow 4 - 2 = 2 \rightarrow [9, 2]$
- `3` $\rightarrow [9, 2, 3]$; `*` $\rightarrow 2 \times 3 = 6 \rightarrow [9, 6]$
- `+` $\rightarrow 9 + 6 = 15 \rightarrow [15]$
- `7, 2` $\rightarrow [15, 7, 2]$; `/` $\rightarrow 7 // 2 = 3 \rightarrow [15, 3]$
- `-` $\rightarrow 15 - 3 = 12 \rightarrow [12]$

Value = **12** (infix: $9 + (4 - 2) \times 3 - 7 / 2$).

Trap: popping in the wrong order turns `4 2 -` into -2 and `7 2 /` into 0 , giving a different result. The maximum stack depth here is only 3.

**Q.6 [MCQ · 1 mark · Binary trees — level-order traversal]****Answer: (C)**

Concept. Level-order uses a FIFO queue: dequeue a node, output it, enqueue its left then right child. Nodes therefore appear level by level, left to right.

Levels of T:

- depth 0: 7
- depth 1: 3, 9
- depth 2: 12, 5, 4
- depth 3: 1, 11 (children of 5), 6 (child of 4)
- depth 4: 2

Sequence: 7, 3, 9, 12, 5, 4, 1, 11, 6, 2 → (C).

- (A) is a zig-zag order: it reverses alternate levels.
- (B) is the pre-order traversal (depth-first).
- (D) places 2 (depth 4) before 6 (depth 3) — it treats 2 as being on 11's level.

Tip: 6 is at depth 3 even though it hangs from the right side; depth, not horizontal position, decides the level.

Q.7 [NAT · 1 mark · Hashing — separate chaining]**Answer: 7**

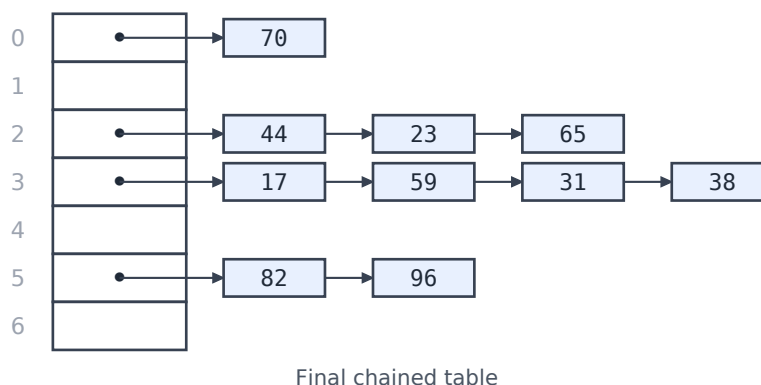
Concept. With chaining every key goes into the list at its home slot; there is no probing, so just compute $k \bmod 7$ for every key.

- $44 \rightarrow 2$, $17 \rightarrow 3$, $82 \rightarrow 5$, $59 \rightarrow 3$, $31 \rightarrow 3$
- $96 \rightarrow 5$, $23 \rightarrow 2$, $70 \rightarrow 0$, $38 \rightarrow 3$, $65 \rightarrow 2$

Chains: slot 0: [70]; slot 2: [44, 23, 65]; slot 3: [17, 59, 31, 38]; slot 5: [82, 96]; slots 1, 4, 6 are empty.

$L = 4$ (slot 3), $E = 3$, so $L + E = 7$.

Trap: $59 = 56 + 3$ and $38 = 35 + 3$ are easy to mis-reduce; also note that 10 keys in 7 slots must leave at least one chain of length ≥ 2 (pigeonhole), but the load factor $10/7$ says nothing about the actual maximum.



Q.8 [MCQ · 1 mark · Insertion sort — number of shifts]**Answer: (B)**

Concept. Every shift in insertion sort removes exactly one inversion (a pair $i < j$ with $A[i] > A[j]$), and sorting removes them all. So $\#shifts = \#inversions$.

Count inversions element by element (elements to its right that are smaller):

- 5: {2, 1, 3} → 3
- 8: {2, 1, 7, 3} → 4
- 2: {1} → 1
- 9: {1, 7, 3} → 3
- 1: none → 0
- 7: {3} → 1
- 3: → 0

Total = $3 + 4 + 1 + 3 + 0 + 1 = 12 \rightarrow (B)$.

- (A) 11 and (C) 13 come from missing or double-counting one pair.
- (D) $21 = 7 \cdot 6/2$ is the worst case (reverse-sorted input), not this input.

Tip: the number of *comparisons* differs from the number of shifts — it is shifts plus one for each insertion that stops before reaching the front.

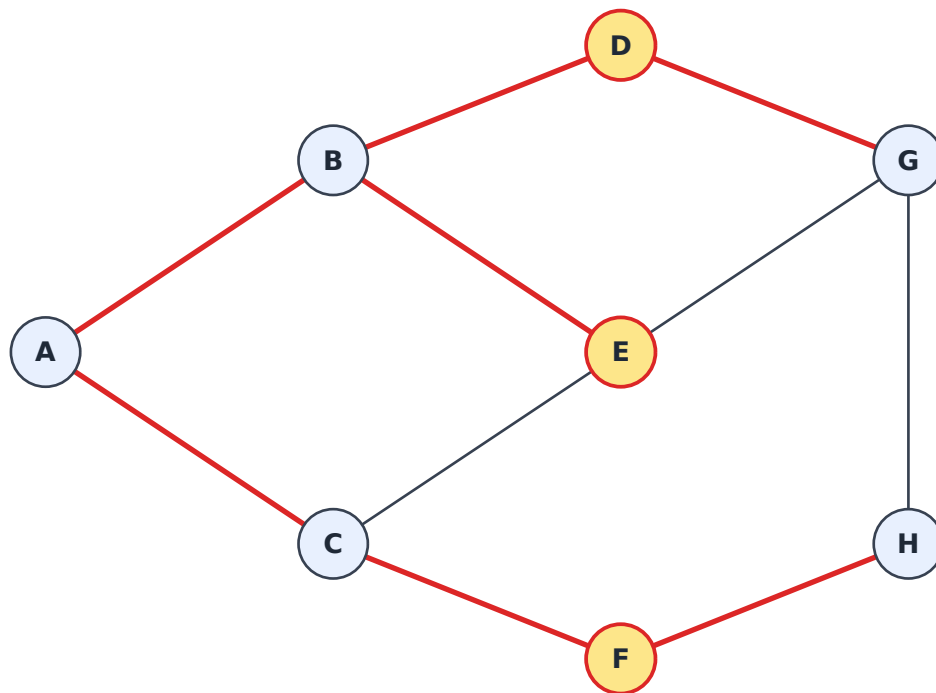
Q.9 [NAT · 1 mark · Graphs — BFS levels]**Answer: 3**

Concept. BFS discovers vertices in non-decreasing order of hop distance; the level of a vertex is its shortest-path length in an unweighted graph.

- Level 0: A
- Level 1: neighbours of A → B, C
- Level 2: new neighbours of B (D, E) and of C (F; E already found) → D, E, F
- Level 3: G (from D or E), H (from F)

Exactly **3** vertices (D, E, F) are at distance 2.

Trap: E is adjacent to both B and C, so it must be counted once; G is adjacent to E but is at distance 3, not 2, because no level-1 vertex touches it.



BFS tree (red) — level-2 vertices highlighted

Q.10 [MCQ · 1 mark · Python — recursion on trees]**Answer: (A)**

Concept. `f` adds the values of **leaves only**, with sign $+$ at even depth and $-$ at odd depth (root depth 0). Internal nodes contribute nothing; `None` children contribute 0.

Leaves and depths:

- 4: depth 2 (1→2→4) → +4
- 8: depth 3 (1→2→5→8) → -8
- 9: depth 3 (1→3→6→9) → -9
- 7: depth 3 (1→3→6→7) → -7

Sum = 4 - 8 - 9 - 7 = **-20** → (A).

- (B) 20 results from counting the root at depth 1, which flips every sign.
- (C) -12 = 4 - 9 - 7 misses leaf 8 (it hangs below the one-child node 5).
- (D) 4 counts only even-depth leaves.

Trap: a node with one child (5 and 3) is *not* a leaf, and the depth passed to children is `d + 1` regardless of which side they are on.

Q.11 [MSQ · 2 marks · Binary trees — reconstruction from post-order and in-order]**Answer: (B), (C), (D)**

Concept. In post-order the root is the **last** symbol. Locate it in the in-order sequence to split left/right subtrees; the post-order sequence splits at the same sizes (left block first, then right block).

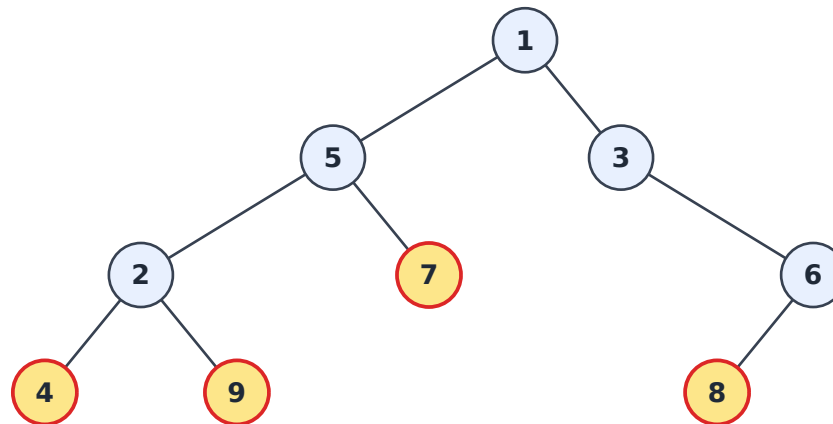
Reconstruction:

- Root = 1. In-order: [4, 2, 9, 5, 7] | 1 | [3, 8, 6]. Post-order blocks: [4, 9, 2, 7, 5] and [8, 6, 3].
- Left block: root 5 (last of 4, 9, 2, 7, 5). In-order [4, 2, 9] | 5 | [7]. So 5 has right child 7 and left subtree {4, 2, 9} whose post-order 4, 9, 2 gives root 2 with children 4 (left) and 9 (right).
- Right block: root 3 (last of 8, 6, 3). In-order 3 | [8, 6] → nothing left of 3, so 3 has only a right subtree {8, 6}; post-order 8, 6 → root 6, and 8 precedes 6 in in-order, so 8 is 6's **left** child.

Option analysis:

- (A) 3 has only a right child (6). **False.**
- (B) Longest paths $1 \rightarrow 5 \rightarrow 2 \rightarrow 4$, $1 \rightarrow 5 \rightarrow 2 \rightarrow 9$ and $1 \rightarrow 3 \rightarrow 6 \rightarrow 8$ all have 3 edges. **True.**
- (C) Pre-order (root, left, right): 1, 5, 2, 4, 9, 7, 3, 6, 8. **True.**
- (D) Leaves are 4, 9, 7, 8 — exactly 4. **True.**

Trap: students often read the post-order from the front and pick 4 as the root. Another slip is attaching 8 as the right child of 6; the in-order sequence (... , 8, 6) forces it to be the left child.



Reconstructed tree (leaves highlighted)

Q.12 [NAT · 2 marks · Binary trees — counting trees with given traversals]

Answer: 16

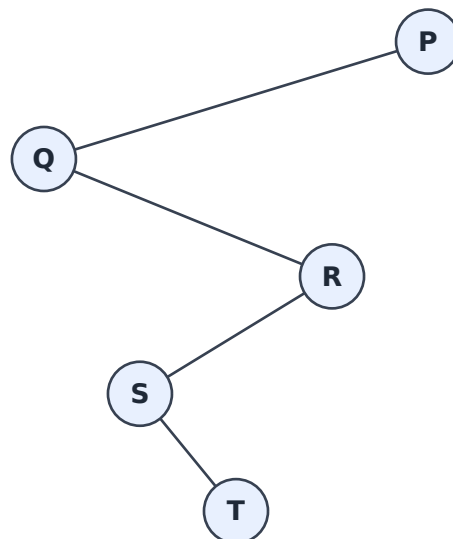
Concept. Pre-order and post-order together do **not** determine a binary tree when some node has exactly one child, because neither traversal records whether that child is a left or a right child.

Structure forced by the traversals:

- Pre-order P, Q, R, S, T says P is the root, and post-order ending with P agrees.
- Take any node X. In pre-order X precedes all its descendants; in post-order X follows them. If X had two non-empty subtrees L and R, both traversals would list all of L before all of R — but the given post-order is the exact reverse of the pre-order, so no two nodes keep their relative order. Hence **no node has two children.**
- So the tree is a **chain** P-Q-R-S-T (each of P, Q, R, S has exactly one child), and any such chain does give pre-order PQRST and post-order TSRQP.

Counting: each of the 4 one-child nodes independently chooses whether its child is a left or right child $\rightarrow 2^4 = \mathbf{16}$ trees, all with the given pre- and post-order.

Trap: answering 1 (assuming pre + post determine the tree) or 2 (only left-skewed and right-skewed). In-order would distinguish them; pre + post do not.



One of the 16 trees (zig-zag chain)

Q.13 [MCQ · 2 marks · Python — closures and late binding]**Answer: (D)**

Concept. A lambda's free variable is looked up **when the lambda is called** (late binding). A default argument (`i=i`) is evaluated **when the lambda is created**, freezing the current value.

First sum (`fs`, defaults freeze `i = 0, 1, 2, 3`); body is a conditional expression `x*i if i % 2 else x+i`:

- `i = 0` (even): $3 + 0 = 3$
- `i = 1` (odd): $3 \times 1 = 3$
- `i = 2` (even): $3 + 2 = 5$
- `i = 3` (odd): $3 \times 3 = 9$
- total = 20

Second sum (`gs`): all four lambdas share the comprehension's variable `i`, which is 3 once the comprehension finishes. Each call returns $3 \times 3 = 9 \rightarrow 4 \times 9 = 36$.

Output: 20 36 → (D).

- (A) mis-evaluates the conditional as always `x*i` for `fs` ($0+3+6+9 = 18$).
- (B) assumes `gs` captured 0, 1, 2, 3 → $0 + 3 + 6 + 9 = 18$ (no late binding).
- (C) applies late binding to `fs` as well, ignoring the default argument.

Trap: the comprehension has its own scope, but the closures still see the *final* value of that scope's `i`.

Q.14 [MCQ · 2 marks · Binary trees — iterative traversal with a stack]**Answer: (C)**

Concept. The stack is LIFO. Pushing left **then** right means the right child is popped first, so the function visits root, right subtree, left subtree — the *mirror pre-order*, which equals the **reverse of the post-order**.

Trace (stack top on the right; `·` = None):

- pop M → out M; push D, R
- pop R → out R; push P, W
- pop W → out W; push `·`, Z → pop Z → out Z; pop `·`, pop `·` (Z's children); pop `·` (W's left)
- pop P → out P (children `·`)
- pop D → out D; push B, H

- pop H → out H; push F, · → pop · → pop F → out F
- pop B → out B

Result: M, R, W, Z, P, D, H, F, B → (C). Post-order is B, F, H, D, P, Z, W, R, M, and (C) is exactly its reverse.

- (A) is ordinary pre-order (would need right pushed first).
- (B) is the post-order itself.
- (D) visits R's children left-first, i.e. treats the structure as a queue at one level — not what a stack does.

Tip: this push order is the basis of the 'two-stack post-order' trick: run it, then reverse the output.

Q.15 [MSQ · 2 marks · Quicksort — Lomuto partition]

Answer: (B), (C), (D)

Concept. Lomuto keeps $A[\text{lo}..i] \leq \text{pivot}$. Each element $\leq \text{pivot}$ advances i and is swapped into place; finally the pivot is swapped into position $i + 1$.

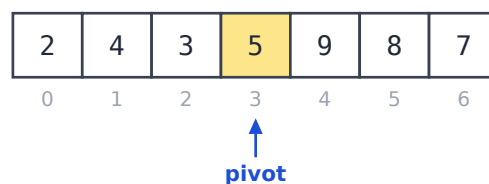
Pivot = 5, $i = -1$:

- $j = 0$: $7 > 5 \rightarrow$ nothing.
- $j = 1$: $2 \leq 5 \rightarrow i = 0$, swap $A[0], A[1] \rightarrow [2, 7, 9, 4, 3, 8, 5]$
- $j = 2$: $9 > 5$.
- $j = 3$: $4 \leq 5 \rightarrow i = 1$, swap $A[1], A[3] \rightarrow [2, 4, 9, 7, 3, 8, 5]$
- $j = 4$: $3 \leq 5 \rightarrow i = 2$, swap $A[2], A[4] \rightarrow [2, 4, 3, 7, 9, 8, 5]$
- $j = 5$: $8 > 5$.
- final: swap $A[3], A[6] \rightarrow [2, 4, 3, 5, 9, 8, 7]$; return 3.

Option analysis:

- (A) **False** — swaps happen at $j = 1, 3, 4$ plus the final one: **4** in total.
- (B) **True** — matches the final state.
- (C) **True** — $A[4..6] = [9, 8, 7]$.
- (D) **True** — pivot lands at index 3.

Trap: forgetting the final pivot swap when counting swaps, or assuming the right part keeps its original relative order (it does not: 7 moved to the end). Lomuto is not stable.



Array after partition

Q.16 [NAT · 2 marks · Shortest paths — Dijkstra]

Answer: 34

Concept. Dijkstra repeatedly extracts the unfinished vertex with the smallest tentative distance and relaxes its outgoing edges; with non-negative weights the extracted value is final.

Trace (tentative distances after each extraction):

- Extract S (0): $A = 7, B = 2$.
- Extract B (2): $A = \min(7, 2+3) = 5$; $C = 2+8 = 10$; $D = 2+10 = 12$.
- Extract A (5): $C = \min(10, 5+2) = 7$; $D = \min(12, 5+6) = 11$.
- Extract C (7): $D = \min(11, 7+1) = 8$; $T = 7+9 = 16$.
- Extract D (8): $T = \min(16, 8+4) = 12$.
- Extract T (12).

Final: S 0, A 5, B 2, C 7, D 8, T 12. Sum = $0 + 5 + 2 + 7 + 8 + 12 = \mathbf{34}$.

Trap: the direct edge $S \rightarrow A$ (7) and $B \rightarrow D$ (10) look attractive but are beaten by detours through B and C; T's distance is improved twice ($16 \rightarrow 12$).

	Tentative distances					
	S	A	B	C	D	T
after S	0	7	2	∞	∞	∞
after B	0	5	2	10	12	∞
after A	0	5	2	7	11	∞
after C	0	5	2	7	8	16
after D	0	5	2	7	8	12

Q.17 [MCQ · 2 marks · Linked lists — recursive pointer manipulation]

Answer: (D)

Concept. `mystery` swaps the first two nodes and recursively processes the list starting at the third node. It therefore swaps nodes in adjacent pairs; an odd last node is returned unchanged by the base case.

Trace of the recursion:

- `mystery(1)`: `second = 2`; `1.next = mystery(3)`; `2.next = 1`; return 2.
- `mystery(3)`: `second = 4`; `3.next = mystery(5)`; `4.next = 3`; return 4.
- `mystery(5)`: `second = 6`; `5.next = mystery(7)`; `6.next = 5`; return 6.
- `mystery(7)`: `7.next` is None $\rightarrow$ return 7 (base case).

Unwinding: $5 \rightarrow 7$, $3 \rightarrow 6$, $1 \rightarrow 4$, so the list reads 2, 1, 4, 3, 6, 5, 7 $\rightarrow$ (D).

- (A) starts pairing from the second node.
- (B) is a full reversal — this function never reverses beyond a pair.
- (C) drops the unpaired 7; the base case keeps it.

Trap: the assignment `h.next = mystery(second.next)` must happen before `second.next = h`; otherwise `second.next` would already point back to h.



Q.18 [NAT · 2 marks · Binary search — probe counting]

Answer: 7

Concept. Each loop iteration inspects one middle element (one probe) and halves the range. With $15 = 2^4 - 1$ elements, at most 4 probes are ever needed.

Search $x = 50$:

- lo 0, hi 14 $\rightarrow$ mid 7 (36) $< 50 \rightarrow$ lo = 8
- lo 8, hi 14 $\rightarrow$ mid 11 (58) $> 50 \rightarrow$ hi = 10
- lo 8, hi 10 $\rightarrow$ mid 9 (47) $< 50 \rightarrow$ lo = 10
- lo 10, hi 10 $\rightarrow$ mid 10 (53) $> 50 \rightarrow$ hi = 9; loop ends. $p_1 = 4$.

Search $x = 69$:

- mid 7 (36) $< 69 \rightarrow$ lo = 8
- mid 11 (58) $< 69 \rightarrow$ lo = 12
- lo 12, hi 14 $\rightarrow$ mid 13 (69) found. $p_2 = 3$.

$p_1 + p_2 = 4 + 3 = 7$.

Trap: an unsuccessful search on a perfectly balanced 15-element array always costs exactly 4 probes; the final `lo > hi` test is not a probe because no array element is examined.

Q.19 [MSQ · 2 marks · Graphs — DFS and BFS trees]

Answer: (A), (B), (C)

Concept. Recursive DFS goes as deep as possible before backtracking; in an undirected graph every non-tree edge is a back edge. BFS levels equal hop distances.

DFS from A (alphabetical):

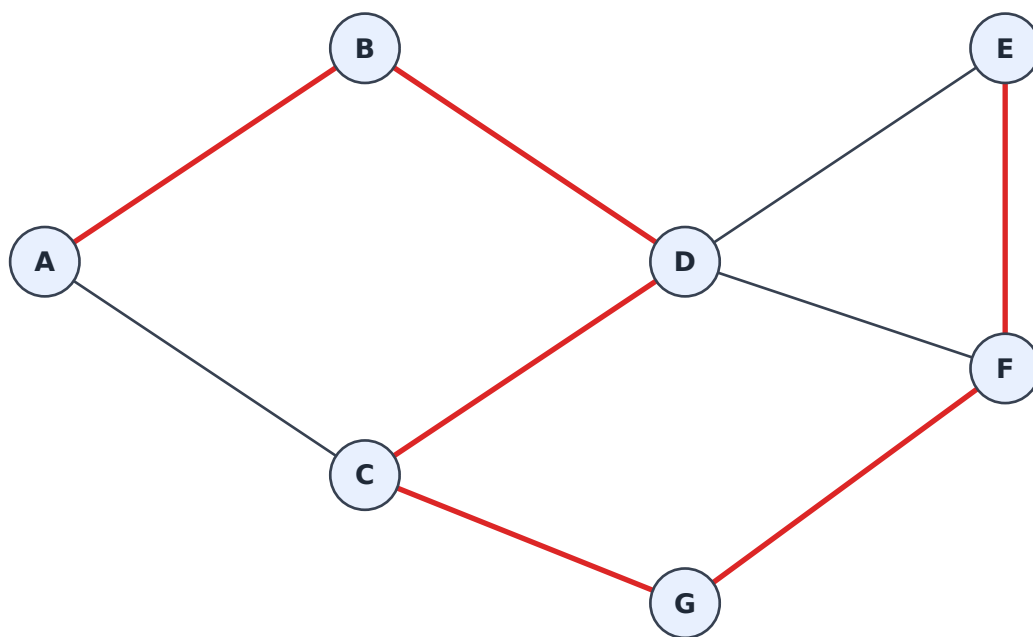
- A → B (first neighbour) → D (B's only unvisited neighbour)
- D's neighbours B, C, E, F → C first
- C's neighbours A (seen), D (seen), G → G
- G's neighbours C (seen), F → F
- F's neighbours D (seen), E → E; E has no unvisited neighbour; backtrack all the way.
- Order: A, B, D, C, G, F, E. Tree edges: AB, BD, DC, CG, GF, FE (a single path).

BFS from A: level 1 {B, C}, level 2 {D, G}, level 3 {E, F}.

Option analysis:

- (A) **True** — E is reached via D (level 2) → level 3.
- (B) **True** — 9 edges — 6 tree edges = 3 back edges (AC, DE, DF).
- (C) **True** (derived above).
- (D) **False** — the DFS tree is the path A-B-D-C-G-F-E, so E is at depth 6.

Trap: E is adjacent to D, so it is tempting to place it at DFS depth 3; but DFS commits to C before ever looking at E, and E is finally reached from F.



DFS tree edges in red

Q.20 [MSQ · 2 marks · Heaps — repeated delete-min]

Answer: (A), (B), (D)

Concept. delete-min removes `H[0]`, moves the last element to the root, then sifts down along a path of smaller children: $O(\log n)$ work.

First delete-min (remove 3, move 12 to root): [12, 9, 5, 14, 10, 8, 7, 20, 17]

- children 9, 5 → swap with 5 → [5, 9, 12, 14, 10, 8, 7, 20, 17]

- 12 at index 2, children 8, 7 → swap with 7 → [5, 9, 7, 14, 10, 8, 12, 20, 17]
- index 6 has no children → stop.

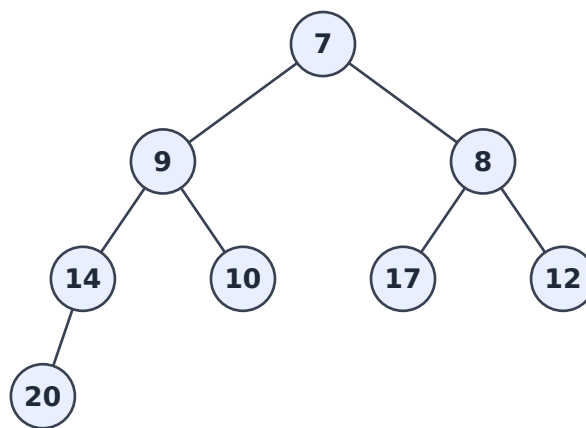
Second delete-min (remove 5, move 17 to root): [17, 9, 7, 14, 10, 8, 12, 20]

- children 9, 7 → swap with 7 → [7, 9, 17, 14, 10, 8, 12, 20]
- 17 at index 2, children 8, 12 → swap with 8 → [7, 9, 8, 14, 10, 17, 12, 20]
- index 5 has no children ($11 > 7$) → stop.

Option analysis:

- (A) **True** — $H[0] = 7$, $H[1] = 9$, $H[2] = 8$.
- (B) **True** — 17 ends at index 5.
- (C) **False** — with 8 elements, index 3 (key 14) has child index 7 (key 20).
- (D) **True**.

Trap: sifting toward the *left* child by habit (9 instead of 5) breaks the heap property; always compare both children.



Heap after two delete-min operations

Mock Test 06

Binary Search Trees

Difficulty: Moderate

Focus areas: BST insert/delete/search, valid sequences, min/max, successor

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing with negative steps]

Consider the following Python program. What is printed?

Python

```
1 s = "PREORDER"
2 print(s[-2:0:-3], s[::-2], s[5:2:-1])
```

- (A) EO RDOR DR0
- (B) EOP RDOR DR0
- (C) EO REDR ORD
- (D) ER RDOR DROE

Q.2 [NAT · numerical answer | 1 mark | BST insertion — height]

The keys 52, 38, 71, 45, 60, 83, 41, 47, 43, 90, 65 are inserted in the given order into an initially empty binary search tree (no balancing). The height of the resulting tree, measured as the number of **edges** on the longest root-to-leaf path, is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | BST search paths]

A binary search tree stores distinct integers from 1 to 100. A search for the key **55** examines nodes in the order listed. Which of the following sequences **cannot** be the sequence of nodes examined?

- (A) 10, 90, 20, 80, 30, 70, 55
- (B) 91, 12, 85, 30, 61, 40, 55
- (C) 70, 20, 60, 30, 65, 55
- (D) 5, 99, 50, 75, 60, 52, 55

Q.4 [MSQ · one or more correct | 1 mark | Stacks — permutations]

The integers 1, 2, 3, 4, 5 are pushed onto an initially empty stack **in this order**; pops may be interleaved with pushes at any time, and every popped value is printed. Which of the following output sequences is/are possible?

- (A) 2 4 1 3 5
- (B) 3 2 5 4 1
- (C) 4 5 3 2 1
- (D) 1 5 4 2 3

Q.5 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

The keys 27, 14, 50, 36, 23, 41, 61, 19, 72, 8 are inserted in this order into a hash table with 9 slots (indices 0–8) using $h(k) = k \bmod 9$ and separate chaining; each new key is inserted at the **head** of its chain. Assuming every stored key is equally likely to be searched for, the average number of key comparisons in a successful search is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | BST — in-order successor and predecessor]

For the binary search tree shown below, let x be the in-order **successor** of 37 and y the in-order **predecessor** of 47. The pair (x, y) is

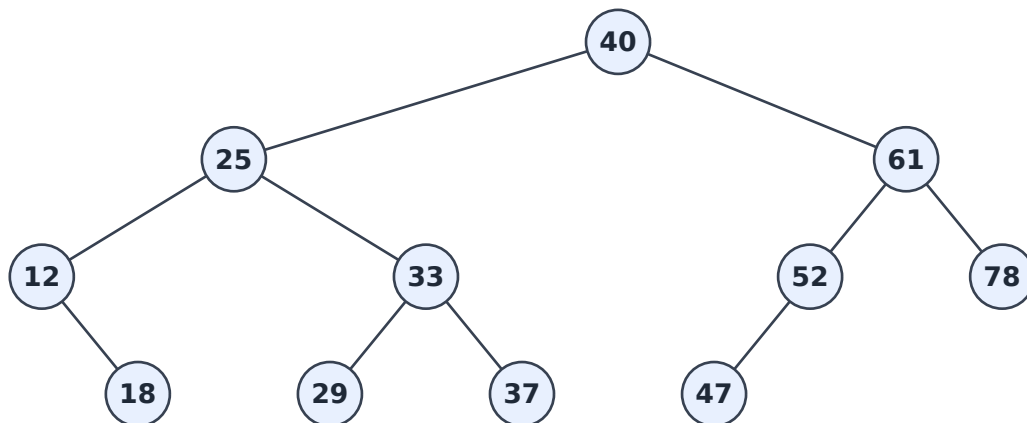


Figure: BST

- (A) (33, 40)
- (B) (40, 40)
- (C) (40, 52)
- (D) (29, 61)

Q.7 [NAT · numerical answer | 1 mark | Insertion sort — number of shifts]

Insertion sort (ascending) is applied to the array [7, 3, 9, 1, 6, 2, 8]. A *shift* is one execution of $A[j+1] = A[j]$ that moves a larger element one place to the right. The total number of shifts performed is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Python — list aliasing and shallow copies]

Consider the following Python program. What is printed?

Python

```

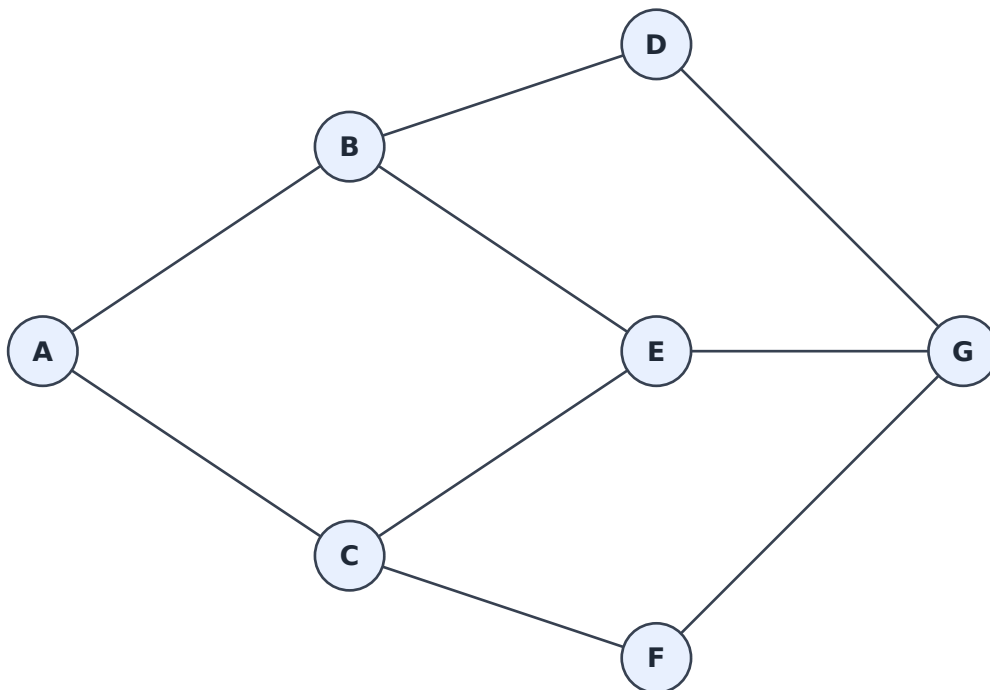
1 a = [[0] * 2] * 3
2 b = [row[:] for row in a]
3 a[0][1] = 5
4 b[1][0] = 7
5 c = a + b
6 c[0][0] += 1
7 print(sum(map(sum, c)))

```

- (A) 13
- (B) 40
- (C) 23
- (D) 25

Q.9 [MSQ · one or more correct | 1 mark | Graph traversal — BFS and DFS]

Consider the undirected graph below. BFS and DFS are started from **A**; whenever there is a choice, neighbours are visited in alphabetical order. Which of the following statements is/are TRUE?



- (A) The BFS visit order is A, B, C, D, E, F, G
- (B) DFS visits F before E
- (C) Edge E-G is an edge of the BFS tree
- (D) The BFS level (distance from A) of G is 3

Q.10 [NAT · numerical answer | 1 mark | Binary search — counting probes]

Consider the following Python program. The value printed is ____.

Python

```

1 def bs(A, x):
2     lo, hi, c = 0, len(A) - 1, 0
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         c += 1
6         if A[mid] == x:
7             return c
8         elif A[mid] < x:
9             lo = mid + 1
10        else:
11            hi = mid - 1
12    return c
13
14 A = list(range(3, 60, 4))
15 print(bs(A, 39) + bs(A, 42))
  
```

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | BST deletion — successor replacement]

The BST shown below is modified by deleting the keys **50**, **30** and **20**, in that order. A node with two children is deleted by copying its in-order **successor** into it and then deleting the successor; a node with one child is replaced by that child. The pre-order traversal of the final tree is

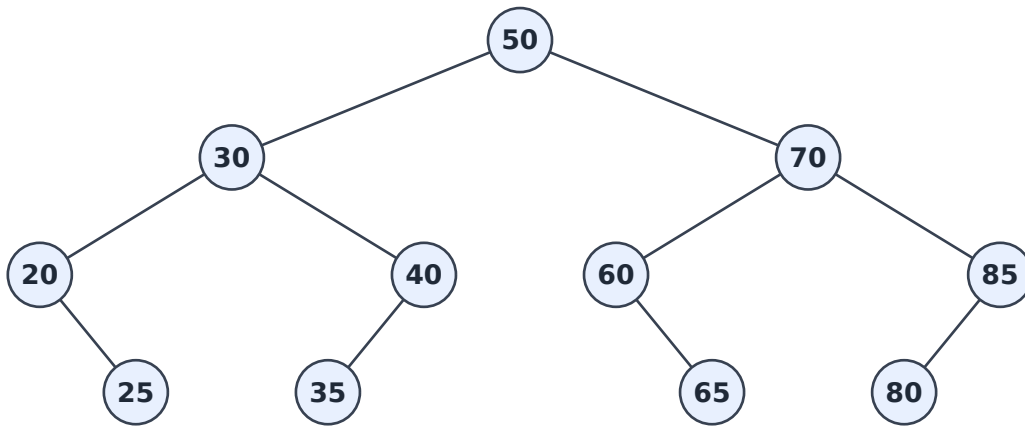


Figure: initial BST

- (A) 40, 25, 35, 70, 60, 65, 85, 80
 (B) 60, 35, 25, 40, 70, 65, 85, 80
 (C) 60, 40, 25, 35, 70, 65, 85, 80
 (D) 60, 35, 25, 40, 70, 85, 80

Q.12 [NAT · numerical answer | 2 marks | BST — counting insertion orders]

The eight keys of the BST shown below are inserted one by one, in some order, into an initially empty BST. The number of insertion orders (permutations of the eight keys) that produce **exactly** this tree is _____.

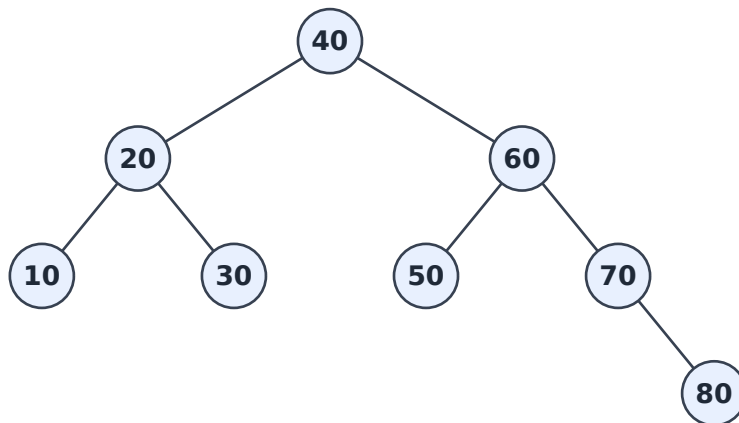


Figure: target BST

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | BST — valid pre-order sequences]

Which of the following sequences is/are valid **pre-order** traversals of some binary search tree with distinct keys?

- (A) 30, 20, 25, 10, 40, 50
 (B) 15, 10, 12, 11, 20, 18, 25, 17
 (C) 50, 30, 20, 40, 70, 60, 55, 65, 80
 (D) 30, 20, 10, 25, 40, 35, 50

Q.14 [MCQ · single correct | 2 marks | Python BST — range query with pruning]

Consider the following Python program. What is printed?

Python

```

1 class N:
2     def __init__(s, k):
3         s.k, s.l, s.r = k, None, None
4
5     def ins(t, k):
6         if t is None:
7             return N(k)
8         if k < t.k: t.l = ins(t.l, k)
9         else:     t.r = ins(t.r, k)
10        return t
11
12    calls = 0
13    def f(t, lo, hi):
14        global calls
15        calls += 1
16        if t is None: return 0
17        if t.k < lo: return f(t.r, lo, hi)
18        if t.k > hi: return f(t.l, lo, hi)
19        return t.k + f(t.l, lo, hi) + f(t.r, lo, hi)
20
21    root = None
22    for k in [45, 22, 67, 15, 31, 58, 80, 27, 36, 62]:
23        root = ins(root, k)
24    print(f(root, 27, 62), calls)

```

- (A) 259 15
- (B) 259 10
- (C) 232 15
- (D) 259 21

Q.15 [NAT · numerical answer | 2 marks | Python — closures and late binding]

Consider the following Python program. The value printed is _____.

Python

```

1 fs = []
2 for i in range(4):
3     fs.append(lambda x, i=i: x * i if i % 2 else x + i)
4
5 gs = [lambda x: x * i for i in range(4)]
6
7 print(sum(f(3) for f in fs) + sum(g(3) for g in gs))

```

Answer: _____ (enter a numerical value)

Q.16 [NAT · numerical answer | 2 marks | Quicksort — Lomuto partition swaps]

Quicksort with the Lomuto partition below is used to sort [6, 11, 3, 9, 2, 8, 5] by calling `quicksort(A, 0, 6)`, where `quicksort(A, lo, hi)` does nothing if $lo \geq hi$ and otherwise calls `partition` and recurses on both sides of the returned index. Counting **every** execution of a swap statement (including swaps of an element with itself), the total number of swaps is _____.

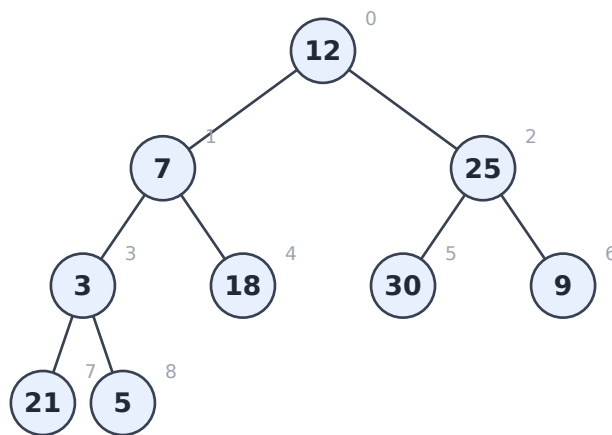
Python

```
1 def partition(A, lo, hi):
2     p, i = A[hi], lo - 1
3     for j in range(lo, hi):
4         if A[j] <= p:
5             i += 1
6             A[i], A[j] = A[j], A[i]      # swap 1
7     A[i + 1], A[hi] = A[hi], A[i + 1]  # swap 2
8     return i + 1
```

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Heaps — bottom-up build-heap]

The array [12, 7, 25, 3, 18, 30, 9, 21, 5] (0-indexed) is converted into a **max-heap** using the bottom-up build-heap procedure (sift-down applied at indices 3, 2, 1, 0 in that order; at each step the larger child is chosen). Which of the following statements is/are TRUE?

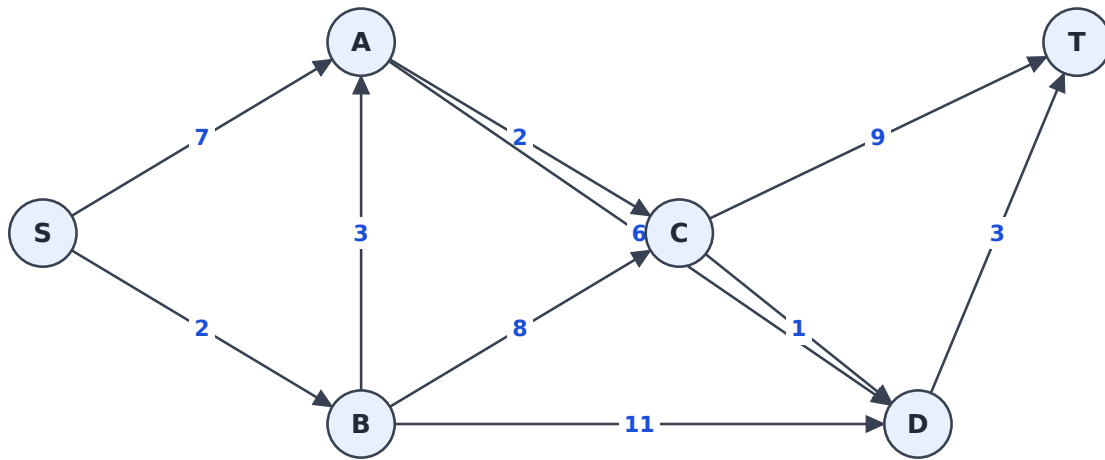


Initial array viewed as a complete binary tree

- (A) After build-heap, 3 and 5 are the children of 7
- (B) After build-heap, key 12 is at index 5
- (C) After build-heap, index 2 holds 25
- (D) Exactly 4 swaps are performed in total

Q.18 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from source **S** on the weighted directed graph shown below. The sum of the final shortest-path distances from **S** to all six vertices (including **S** itself) is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — pointer manipulation in Python]

Consider the following Python program. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 head = None
6 for v in [4, 8, 15, 16, 23, 42]:
7     head = Node(v, head)
8
9 p = head
10 while p and p.nxt:
11     p.nxt = p.nxt.nxt
12     p = p.nxt
13
14 out = []
15 while head:
16     out.append(head.v)
17     head = head.nxt
18 print(out)

```

- (A) [4, 15, 23]
- (B) [23, 15, 4]
- (C) [42, 16, 8]
- (D) [42, 16, 8, 4]

Q.20 [MSQ · one or more correct | 2 marks | BST — reconstruction from post-order]

The **post-order** traversal of a binary search tree with distinct keys is

9, 17, 14, 26, 23, 35, 48, 41, 30.

Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) The in-order successor of 26 is 35
- (B) The tree has exactly 4 leaves
- (C) The pre-order traversal is 30, 23, 14, 9, 17, 26, 41, 35, 48
- (D) The height of the tree is 3

Answer Key — Mock Test 06

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — string slicing with negative steps
2	NAT	1	4	BST insertion — height
3	MCQ	1	(C)	BST search paths
4	MSQ	1	(B), (C)	Stacks — permutations
5	NAT	1	1.89 to 1.91	Hashing — separate chaining
6	MCQ	1	(B)	BST — in-order successor and predecessor
7	NAT	1	11	Insertion sort — number of shifts
8	MCQ	1	(D)	Python — list aliasing and shallow copies
9	MSQ	1	(A), (D)	Graph traversal — BFS and DFS
10	NAT	1	7	Binary search — counting probes
11	MCQ	2	(B)	BST deletion — successor replacement
12	NAT	2	210	BST — counting insertion orders
13	MSQ	2	(C), (D)	BST — valid pre-order sequences
14	MCQ	2	(A)	Python BST — range query with pruning
15	NAT	2	56	Python — closures and late binding
16	NAT	2	7	Quicksort — Lomuto partition swaps
17	MSQ	2	(A), (B), (C)	Heaps — bottom-up build-heap
18	NAT	2	33	Shortest paths — Dijkstra
19	MCQ	2	(C)	Linked lists — pointer manipulation in Python
20	MSQ	2	(C), (D)	BST — reconstruction from post-order

Detailed Solutions — Mock Test 06

Q.1 [MCQ · 1 mark · Python — string slicing with negative steps]

Answer: (A)

A slice `s[start:stop:step]` with a negative step walks **leftwards** from `start` and stops *before* reaching `stop`; omitted ends default to the far ends of the string.

Index the string: P0 R1 E2 O3 R4 D5 E6 R7.

- `s[-2:0:-3]`: start at index 6 (E), then 3 (0); the next index 0 equals `stop`, which is excluded → E0.
- `s[::-2]`: start at the last index 7 → 7, 5, 3, 1 → R D O R → RDOR.
- `s[5:2:-1]`: indices 5, 4, 3 → D R O → DRO.

Output: E0 RDOR DRO.

- (A) Correct.
- (B) Includes index 0 (P), but `stop` is always excluded.
- (C) REDR would come from starting at index 7 with step `-2` but reading wrong letters; ORD reverses the third slice incorrectly.
- (D) Treats `-3` as reaching index 4 and includes the stop index in the third slice.

Trap: with negative steps the *stop* bound is still exclusive, and `s[::-k]` starts at the last character, not the first.

Q.2 [NAT · 1 mark · BST insertion — height]

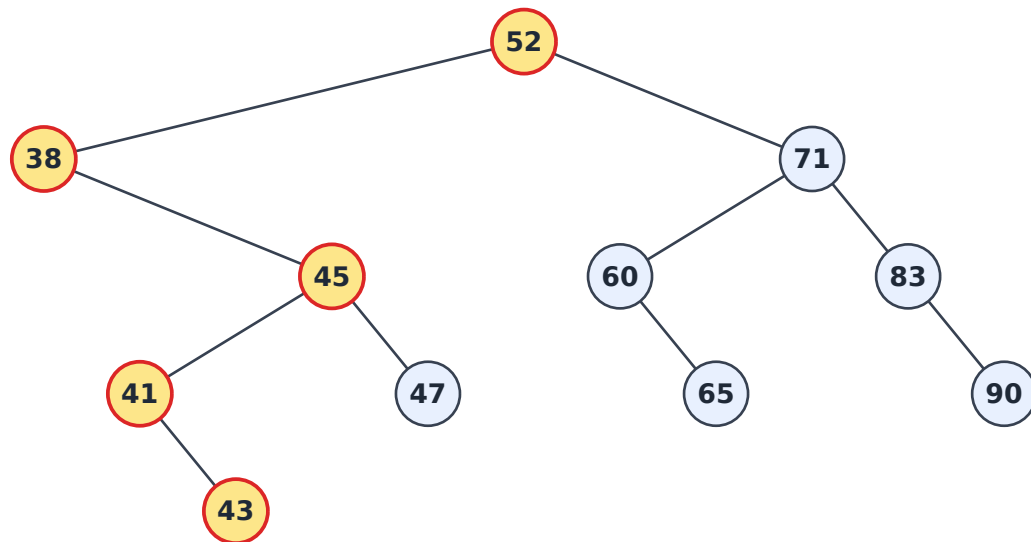
Answer: 4

Each key is inserted by walking from the root (left if smaller, right if larger) and attached at the first empty position.

- 52 is the root; 38 → left of 52; 71 → right of 52.
- 45 → 52 L → 38 R; 60 → 71 L; 83 → 71 R.
- 41 → 38 R → 45 L; 47 → 45 R.
- 43 → 52 L → 38 R → 45 L → 41 R (depth 4).
- 90 → 71 R → 83 R; 65 → 71 L → 60 R.

The deepest node is 43 on the path 52 → 38 → 45 → 41 → 43, which has **4 edges**. The right subtree only reaches depth 3 (52 → 71 → 83 → 90).

Trap: counting nodes instead of edges gives 5. Also note 41 is placed under 45 (not under 38's left) because 41 > 38.



Resulting BST; the longest path is highlighted

Q.3 [MCQ · 1 mark · BST search paths]**Answer: (C)**

While searching for 55, every node visited narrows an open interval (low, high) that must contain 55; each later node on the path must lie strictly inside the current interval.

- (A) $10 \rightarrow (10, \infty)$; $90 \rightarrow (10, 90)$; $20 \rightarrow (20, 90)$; $80 \rightarrow (20, 80)$; $30 \rightarrow (30, 80)$; $70 \rightarrow (30, 70)$; 55 inside. **Valid.**
- (B) $91 \rightarrow (-\infty, 91)$; $12 \rightarrow (12, 91)$; $85 \rightarrow (12, 85)$; $30 \rightarrow (30, 85)$; $61 \rightarrow (30, 61)$; $40 \rightarrow (40, 61)$; 55 inside. **Valid.**
- (C) $70 \rightarrow (-\infty, 70)$; $20 \rightarrow (20, 70)$; $60 \rightarrow (20, 60)$; $30 \rightarrow (30, 60)$; now **65** is not in $(30, 60)$ — after going left at 60 every later key must be < 60 . **Invalid.**
- (D) $5 \rightarrow (5, \infty)$; $99 \rightarrow (5, 99)$; $50 \rightarrow (50, 99)$; $75 \rightarrow (50, 75)$; $60 \rightarrow (50, 60)$; $52 \rightarrow (52, 60)$; 55 inside. **Valid.**

Tip: check each element only against the tightest bounds so far — a zig-zag path is perfectly fine as long as the interval keeps shrinking around the target.

Q.4 [MSQ · 1 mark · Stacks — permutations]**Answer: (B), (C)**

Simulate greedily: to output x , push everything up to x (if not yet pushed) and pop; if x is already pushed it must be on **top**, otherwise the sequence is impossible.

- (A) push 1,2 pop 2; push 3,4 pop 4; stack is [1, 3] with 3 on top but 1 is needed. **Impossible.**
- (B) push 1,2,3 pop 3; pop 2; push 4,5 pop 5; pop 4; pop 1. **Possible.**
- (C) push 1..4 pop 4; push 5 pop 5; pop 3, 2, 1. **Possible.**
- (D) push 1 pop 1; push 2..5 pop 5; pop 4; stack is [2, 3], top is 3 but 2 is needed. **Impossible.**

Tip: a sequence is impossible exactly when it contains a pattern $a \dots b \dots c$ with $c < a < b$ (the '3-1-2' pattern), e.g. 4 ... 1 ... 3 in (A) and 5 ... 2 ... 3 in (D).

**(B) after printing 2, 4**

In (B) 1 is buried under 3, so it cannot be printed next

Q.5 [NAT · 1 mark · Hashing — separate chaining]**Answer: 1.89 to 1.91**

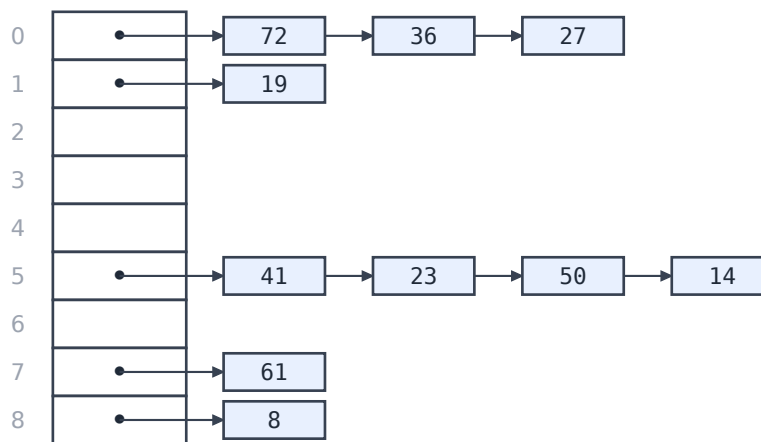
In a successful search the number of comparisons equals the position of the key in its chain. For a chain of length L the positions are 1, 2, ..., L whatever the insertion order, so the chain contributes $L(L+1)/2$ in total.

Hash values: $27 \rightarrow 0$, $14 \rightarrow 5$, $50 \rightarrow 5$, $36 \rightarrow 0$, $23 \rightarrow 5$, $41 \rightarrow 5$, $61 \rightarrow 7$, $19 \rightarrow 1$, $72 \rightarrow 0$, $8 \rightarrow 8$.

- Slot 0: $72 \rightarrow 36 \rightarrow 27$ (length 3, total 6)
- Slot 1: 19 (1)
- Slot 5: $41 \rightarrow 23 \rightarrow 50 \rightarrow 14$ (length 4, total 10)
- Slot 7: 61 (1); Slot 8: 8 (1)

Total = $6 + 1 + 10 + 1 + 1 = 19$ comparisons over 10 keys $\rightarrow$ **1.90**.

Trap: head-insertion changes *which* key is at which position (e.g. 14 now needs 4 comparisons) but not the multiset of positions, so the average is unaffected.



Final table (chains listed head first)

Q.6 [MCQ · 1 mark · BST — in-order successor and predecessor]**Answer: (B)**

If a node has **no right child**, its successor is the lowest ancestor whose *left* subtree contains the node. Symmetrically, if a node has **no left child**, its predecessor is the lowest ancestor whose *right* subtree contains the node.

- 37 is a leaf. Its path from the root is $40 \text{ (go L)} \rightarrow 25 \text{ (go R)} \rightarrow 33 \text{ (go R)} \rightarrow 37$. The last ancestor where we turned **left** is 40, so $x = 40$.
- 47 is a leaf. Path: $40 \text{ (go R)} \rightarrow 61 \text{ (go L)} \rightarrow 52 \text{ (go L)} \rightarrow 47$. The last ancestor where we turned **right** is 40, so $y = 40$.

Check with the in-order sequence 12, 18, 25, 29, 33, 37, **40**, 47, 52, 61, 78: the key after 37 is 40 and the key before 47 is 40.

- (A) 33 is 37's parent, but it is smaller than 37.
- (C) 52 is 47's parent, but it is larger than 47.
- (D) 29 and 61 are unrelated neighbours in the drawing.

Trap: the parent is the successor/predecessor only when the node is a left/right child respectively.

Q.7 [NAT · 1 mark · Insertion sort — number of shifts]**Answer: 11**

Each shift fixes exactly one **inversion** (a pair $i < j$ with $A[i] > A[j]$), so the number of shifts equals the number of inversions.

Count, for every element, how many earlier elements are larger:

- 7: 0; 3: 1 (7); 9: 0; 1: 3 (7, 3, 9); 6: 2 (7, 9); 2: 4 (7, 3, 9, 6); 8: 1 (9).

Total = 0 + 1 + 0 + 3 + 2 + 4 + 1 = **11**.

Pass by pass the array becomes [3,7,9,1,6,2,8] → [3,7,9,...] → [1,3,7,9,6,2,8] → [1,3,6,7,9,2,8] → [1,2,3,6,7,9,8] → [1,2,3,6,7,8,9].

Trap: do not count comparisons — they exceed shifts by one for every pass that stops because it found a smaller element.

Q.8 [MCQ · 1 mark · Python — list aliasing and shallow copies]

Answer: (D)

`[[0] * 2] * 3` creates **one** inner list referenced three times. `row[:]` makes genuine copies, and `a + b` creates a new outer list whose elements are the *same* row objects.

- After line 2: `b` has three independent rows `[0, 0]`.
- `a[0][1] = 5` changes the shared row → every row of `a` is `[0, 5]`.
- `b[1][0] = 7` → `b = [[0,0],[7,0],[0,0]]`.
- `c[0]` is the shared row of `a`; `c[0][0] += 1` → that row is `[1, 5]`.

Sum: rows from `a` contribute $3 \times 6 = 18$, rows from `b` contribute 7 → **25**.

- (A) 13 assumes the rows of `a` were independent ($6 + 0 + 0 + 7$).
- (B) 40 assumes `b` was copied *after* the 5 was written ($18 + 22$).
- (C) 23 assumes `a + b` copies the rows, so the `+= 1` changes only one row ($16 + 7$).

Trap: list concatenation copies references, not the inner lists.

Q.9 [MSQ · 1 mark · Graph traversal — BFS and DFS]

Answer: (A), (D)

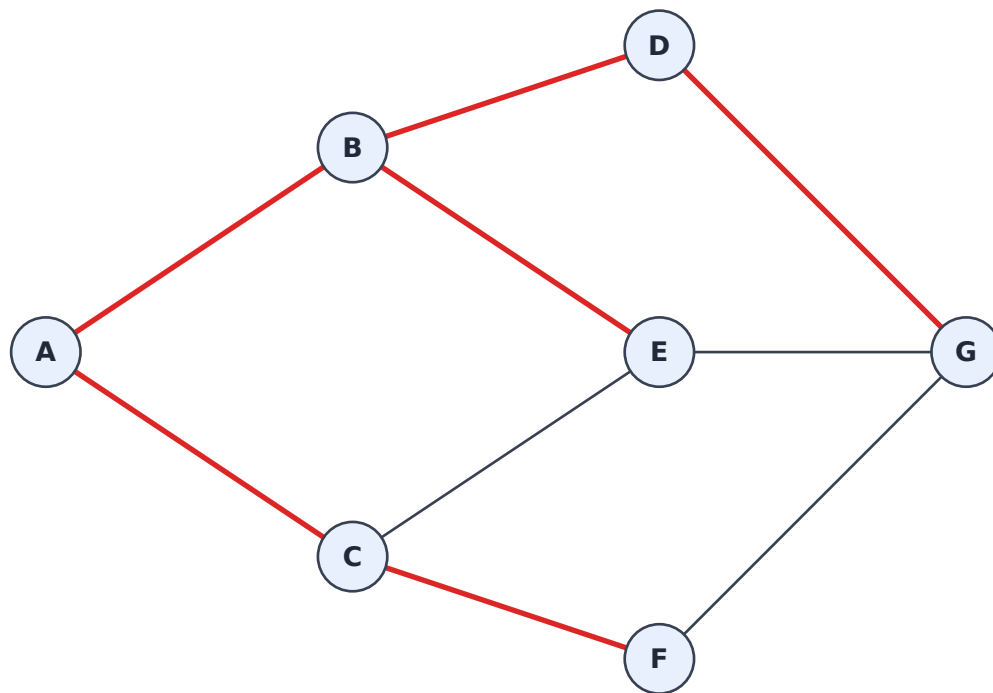
BFS explores level by level with a FIFO queue; DFS goes as deep as possible (recursively) before backtracking.

BFS: queue A → dequeue A, enqueue B, C → dequeue B, enqueue D, E → dequeue C, enqueue F (E already seen) → dequeue D, enqueue G (via D) → E, F, G. Order A, B, C, D, E, F, G; levels A0, B1, C1, D2, E2, F2, G3.

DFS: A → B → D → G → E (G's smallest unvisited neighbour) → back to E: C → F. Order A, B, D, G, E, C, F.

- (A) **True**.
- (B) **False** — DFS reaches E (from G) before F.
- (C) **False** — G is discovered from D, so the tree edge is D-G; E-G is a non-tree edge.
- (D) **True** — G is first reached via D at level 3.

Trap: in BFS a vertex's parent is the *first* vertex that discovers it, not every neighbour one level up.



BFS tree edges highlighted

Q.10 [NAT · 1 mark · Binary search — counting probes]**Answer: 7**

$A = [3, 7, 11, \dots, 59]$ has 15 elements (indices 0-14), $A[i] = 3 + 4i$. The counter c counts loop iterations (probes).

Search 39 (index 9):

- $lo=0, hi=14, mid=7, A[7]=31 < 39 \rightarrow lo=8$ ($c=1$)
- $mid=11, A[11]=47 > 39 \rightarrow hi=10$ ($c=2$)
- $mid=9, A[9]=39 \rightarrow$ found, returns **3**.

Search 42 (absent):

- $mid=7$ (31) $\rightarrow lo=8$; $mid=11$ (47) $\rightarrow hi=10$; $mid=9$ (39) $\rightarrow lo=10$; $mid=10$ (43) $\rightarrow hi=9 \rightarrow$ loop ends. $c = 4$.

Printed value $3 + 4 = 7$.

Tip: with $15 = 2^4 - 1$ elements every unsuccessful search makes exactly 4 probes, while a successful one takes 1-4 depending on depth in the implicit decision tree.

Q.11 [MCQ · 2 marks · BST deletion — successor replacement]**Answer: (B)**

The in-order successor of a node with two children is the **minimum of its right subtree** (go right once, then left as far as possible). That successor has no left child, so removing it is a 0- or 1-child deletion.

Delete 50 (two children): right subtree 70 $\rightarrow$ left 60 $\rightarrow$ no left child, so successor = 60. Copy 60 into the root and delete the old 60, which has one child 65 $\rightarrow$ 65 becomes 70's left child.

Delete 30 (two children): right subtree 40 $\rightarrow$ left 35 $\rightarrow$ successor = 35 (a leaf). Copy 35 into 30's node, remove the leaf; 40 now has no children.

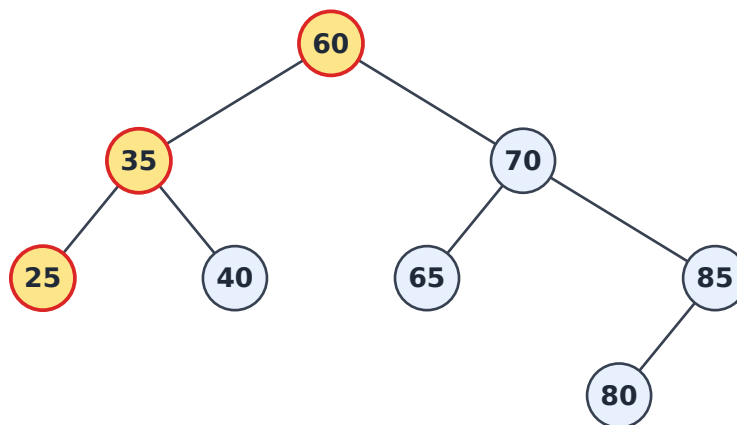
Delete 20 (one child, 25): 25 takes its place as left child of 35.

Final tree: root 60; left 35 (children 25, 40); right 70 (children 65, 85), 85 has left 80.

Pre-order: **60, 35, 25, 40, 70, 65, 85, 80.**

- (A) Result of using the in-order *predecessor* (40 replaces 50, 25 replaces 30 ...).
- (B) Correct.
- (C) Takes 40 (30's right child) as the successor of 30 instead of the leftmost node 35.
- (D) Loses 65 — forgetting to re-attach the successor's right child.

Trap: the successor need not be a leaf — it may have a right child that must be spliced into the successor's old position.



Final BST (changed positions highlighted)

Q.12 [NAT · 2 marks · BST — counting insertion orders]

Answer: 210

An insertion order produces the tree iff the root comes first and, recursively, the keys of each subtree appear in a valid order for that subtree; the left and right subtree sequences may be **interleaved arbitrarily**. Hence

$$f(T) = C(n_L + n_R, n_L) \cdot f(L) \cdot f(R).$$

- Subtree 20 (keys 10, 30): 20 first, then 10 and 30 in any order $\rightarrow f = C(2,1) = 2$.
- Subtree 70 $\rightarrow$ 80: a chain, only one order $\rightarrow f = 1$.
- Subtree 60 (left {50}, right {70, 80}): $f = C(3,1) \cdot 1 \cdot 1 = 3$.
- Root 40 (left 3 keys, right 4 keys): $f = C(7,3) \cdot 2 \cdot 3 = 35 \cdot 6 = \mathbf{210}$.

Out of $8! = 40320$ permutations, only 210 build this particular shape.

Trap: multiplying only $f(L) \cdot f(R) = 6$ forgets that insertions into the two subtrees can be interleaved; the binomial factor $C(7,3)$ accounts for that.

Q.13 [MSQ · 2 marks · BST — valid pre-order sequences]

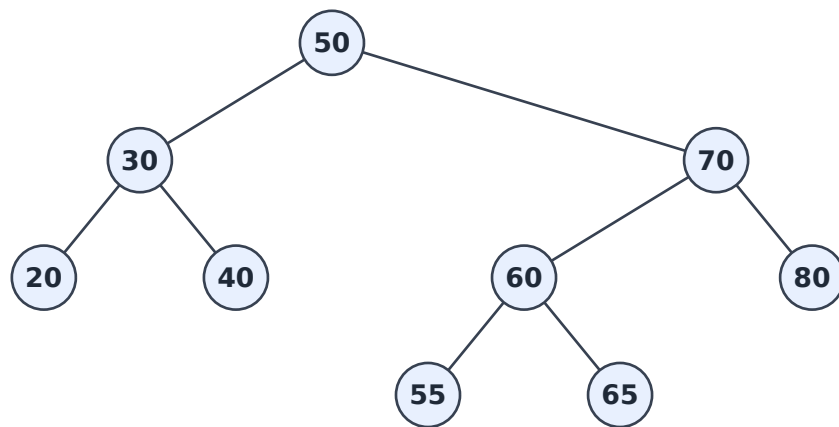
Answer: (C), (D)

In a BST pre-order, once we move into the right subtree of a node v (i.e. see a key larger than v after v 's left part), **no later key may be smaller than v** . A stack-based check keeps a running lower bound: when a key larger than the stack top arrives, pop and raise the bound to the popped key.

- (A) 30, 20, then $25 > 20$ means we are in 20's right subtree (bound 20). Then $10 < 20$ violates the bound. **Invalid.**
- (B) $15 \rightarrow$ left 10 $\rightarrow$ right 12 $\rightarrow$ left 11; then 20 (bound 15), 18 left of 20, 25 right of 20 (bound now 20); then $17 < 20$ would have to lie in 20's right subtree — impossible. **Invalid.**
- (C) $50 \rightarrow$ left 30 (children 20, 40); right 70 $\rightarrow$ left 60 (children 55, 65), right 80. **Valid.**

- (D) $30 \rightarrow$ left $\{20, 10, 25\} = 20$ with children 10, 25; right $\{40, 35, 50\} = 40$ with children 35, 50.
Valid.

Tip: a sequence is a valid BST pre-order iff it avoids the pattern 'a, ..., c, ..., b' with $b < a < c$ — exactly what the stack check detects.



The unique BST with pre-order (C)

Q.14 [MCQ · 2 marks · Python BST — range query with pruning]

Answer: (A)

f returns the sum of keys in $[lo, hi]$ but **prunes**: if a key is below **lo** only the right subtree is explored, if above **hi** only the left. **calls** counts every invocation, including calls on **None**.

Tree: 45 (L 22, R 67); 22 (L 15, R 31); 31 (L 27, R 36); 67 (L 58, R 80); 58 (R 62).

Trace ($\checkmark$ = key in range, added to the sum):

- $f(45) \checkmark \rightarrow f(22), f(67)$
- $f(22)$: $22 < 27 \rightarrow$ only $f(31) \checkmark \rightarrow f(27) \checkmark [f(\text{None}), f(\text{None})], f(36) \checkmark [f(\text{None}), f(\text{None})]$
- $f(67)$: $67 > 62 \rightarrow$ only $f(58) \checkmark \rightarrow f(\text{None}), f(62) \checkmark [f(\text{None}), f(\text{None})]$

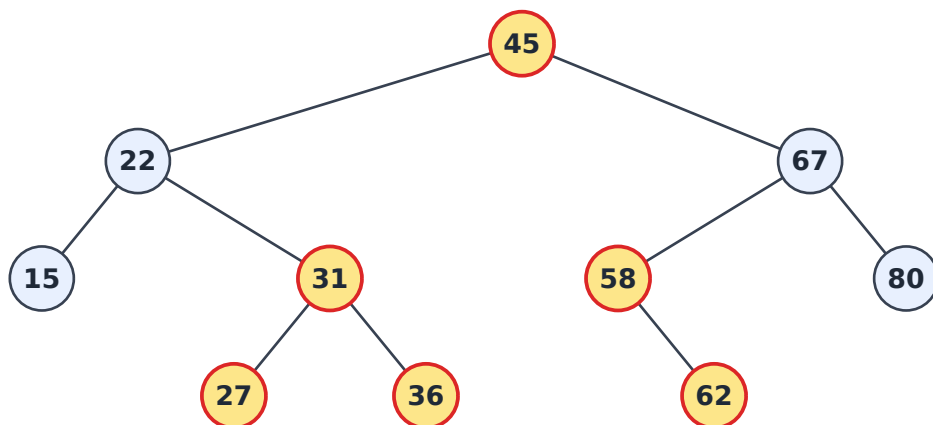
Calls on real nodes: 45, 22, 31, 27, 36, 67, 58, 62 = 8; calls on **None**: $2 + 2 + 1 + 2 = 7$. Total **15**.

Nodes 15 and 80 are never visited.

Sum = $45 + 31 + 27 + 36 + 58 + 62 = 259$.

- (B) 10 counts each node once, ignoring pruning and **None** calls.
- (C) 232 omits 27 (treating the range as exclusive).
- (D) 21 would be the count with no pruning (10 nodes + 11 **None** calls).

Trap: **calls** $+= 1$ runs *before* the **None** check, so empty subtrees are counted too.



Keys in $[27, 62]$ highlighted; 15 and 80 are pruned

Q.15 [NAT · 2 marks · Python — closures and late binding]**Answer: 56**

A default argument (`i=i`) is evaluated when the lambda is **created**, freezing the current value. A plain free variable is looked up when the lambda is **called** (late binding).

fs (frozen `i = 0, 1, 2, 3`; the body is `x*i` if `i` is odd else `x+i`):

- `i=0`: $3 + 0 = 3$
- `i=1`: $3 \times 1 = 3$
- `i=2`: $3 + 2 = 5$
- `i=3`: $3 \times 3 = 9$

Sum = 20.

gs: all four lambdas share the comprehension's variable `i`, whose final value is 3, so each returns $3 \times 3 = 9 \rightarrow$ sum 36.

Output $20 + 36 = 56$.

Trap: reading **gs** as $0 + 3 + 6 + 9 = 18$ (giving 38) ignores late binding. Also note the conditional expression groups as `(x*i) if (i%2) else (x+i)`; misreading it as `x * (i if i%2 else x+i)` gives a different **fs** sum.

Q.16 [NAT · 2 marks · Quicksort — Lomuto partition swaps]**Answer: 7**

Lomuto partition executes swap 1 once for every element $\leq$ pivot and swap 2 exactly once per call. So swaps per call = (number of elements $\leq$ pivot, excluding the pivot) + 1.

- partition(0, 6), pivot 5: elements ≤ 5 are 3 and 2 $\rightarrow 2 + 1 = 3$ swaps. Array $\rightarrow [3, 2, 5, 9, 11, 8, 6]$, pivot index 2.
- partition(0, 1) on [3, 2], pivot 2: none $\leq 2 \rightarrow 1$ swap $\rightarrow [2, 3]$.
- partition(3, 6) on [9, 11, 8, 6], pivot 6: none $\rightarrow 1$ swap $\rightarrow [6, 11, 8, 9]$, pivot index 3.
- partition(4, 6) on [11, 8, 9], pivot 9: only 8 $\rightarrow 1 + 1 = 2$ swaps $\rightarrow [8, 9, 11]$.
- Remaining ranges have size ≤ 1 : no calls to partition.

Total = $3 + 1 + 1 + 2 = 7$.

Trap: forgetting the final pivot swap in calls where nothing is $\leq$ pivot (it still executes), or mis-tracking the array after the first partition — the right part becomes [9, 11, 8, 6], not the original order of those keys.

Array after each partition call (pivot position highlighted)

	0	1	2	3	4	5	6	swaps
start	6	11	3	9	2	8	5	-
P(0,6)	3	2	5	9	11	8	6	3
P(0,1)	2	3	5	9	11	8	6	1
P(3,6)	2	3	5	6	11	8	9	1
P(4,6)	2	3	5	6	8	9	11	2

Q.17 [MSQ · 2 marks · Heaps — bottom-up build-heap]**Answer: (A), (B), (C)**

Bottom-up build-heap sifts down every internal node from the last one (index $\lfloor n/2 \rfloor - 1 = 3$) back to the root; a sifted key may sink several levels.

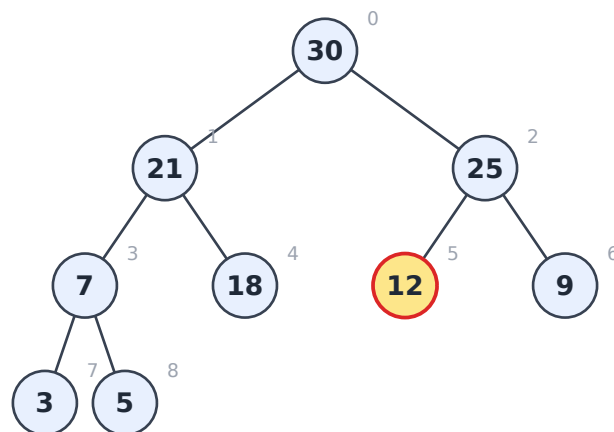
- `i = 3` (key 3, children 21, 5): swap with 21 $\rightarrow [12, 7, 25, 21, 18, 30, 9, 3, 5]$. (1 swap)
- `i = 2` (key 25, children 30, 9): swap with 30 $\rightarrow [12, 7, 30, 21, 18, 25, 9, 3, 5]$. (1 swap)

- $i = 1$ (key 7, children 21, 18): swap with 21 $\rightarrow$ index 3 now 7 with children 3, 5 $\rightarrow$ stop. [12, 21, 30, 7, 18, 25, 9, 3, 5]. (1 swap)
- $i = 0$ (key 12, children 21, 30): swap with 30 $\rightarrow$ at index 2, children 25, 9 $\rightarrow$ swap with 25 $\rightarrow$ index 5 is a leaf. [30, 21, 25, 7, 18, 12, 9, 3, 5]. (2 swaps)

Final heap: [30, 21, 25, 7, 18, 12, 9, 3, 5]; total swaps = 5.

- (A) index 3 = 7, its children (indices 7, 8) are 3 and 5. **True.**
- (B) 12 sank two levels to index 5. **True.**
- (C) index 2 = 25. **True.**
- (D) 5 swaps, not 4. **False** — the root's key sinks twice.

Trap: stopping the root's sift-down after one swap; a sift-down continues until the key is $\geq$ both children.



Max-heap after build-heap (index 5 = key 12 highlighted)

Q.18 [NAT · 2 marks · Shortest paths — Dijkstra]

Answer: 33

Dijkstra repeatedly extracts the unfinished vertex with the smallest tentative distance and relaxes its outgoing edges (valid because all weights are non-negative).

- Extract S (0): $A = 7$, $B = 2$.
- Extract B (2): $A = \min(7, 5) = 5$; $C = 10$; $D = 13$.
- Extract A (5): $C = \min(10, 7) = 7$; $D = \min(13, 11) = 11$.
- Extract C (7): $D = \min(11, 8) = 8$; $T = 16$.
- Extract D (8): $T = \min(16, 11) = 11$.
- Extract T (11).

Final distances: S 0, B 2, A 5, C 7, D 8, T 11. Sum = $0 + 2 + 5 + 7 + 8 + 11 = 33$.

Trap: the direct-looking edges $S \rightarrow A$ (7), $B \rightarrow D$ (11) and $C \rightarrow T$ (9) are all beaten by longer chains of cheap edges; the shortest path to T is $S \rightarrow B \rightarrow A \rightarrow C \rightarrow D \rightarrow T$.

Tentative distances after each extraction

	S	A	B	C	D	T
init	0	∞	∞	∞	∞	∞
S	0	7	2	∞	∞	∞
B	0	5	2	10	13	∞
A	0	5	2	7	11	∞
C	0	5	2	7	8	16
D	0	5	2	7	8	11

Q.19 [MCQ · 2 marks · Linked lists — pointer manipulation in Python]**Answer: (C)**

`Node(v, head)` puts each new node **in front**, so the list is built in reverse: $42 \rightarrow 23 \rightarrow 16 \rightarrow 15 \rightarrow 8 \rightarrow 4$.

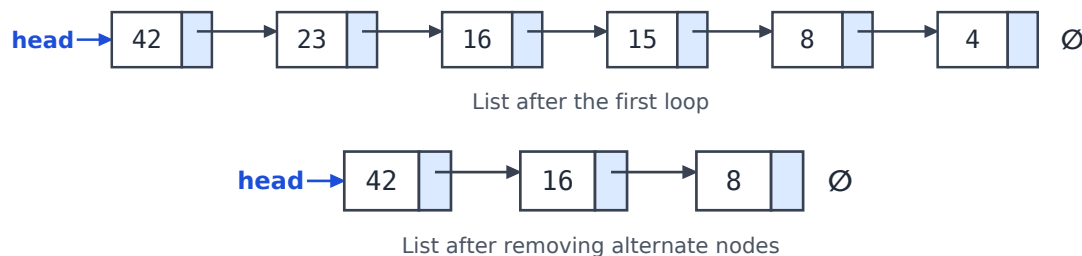
The **while** loop unlinks every second node: it makes `p` skip its successor, then advances `p` to the new successor.

- `p = 42`: `42.nxt = 16` (23 removed); `p = 16`.
- `p = 16`: `16.nxt = 8` (15 removed); `p = 8`.
- `p = 8`: `8.nxt = 4.nxt = None` (4 removed); `p = None` → loop stops (the `p and` guard prevents an `AttributeError`).

Remaining list: $42 \rightarrow 16 \rightarrow 8$, so `[42, 16, 8]` is printed.

- (A) assumes the list is in insertion order $4 \rightarrow 8 \rightarrow \dots$.
- (B) keeps exactly the nodes that were removed.
- (D) assumes the last node survives, i.e. that the loop stops before processing `p = 8`.

Trap: head-insertion reverses order; always draw the list before tracing pointer surgery.

**Q.20 [MSQ · 2 marks · BST — reconstruction from post-order]****Answer: (C), (D)**

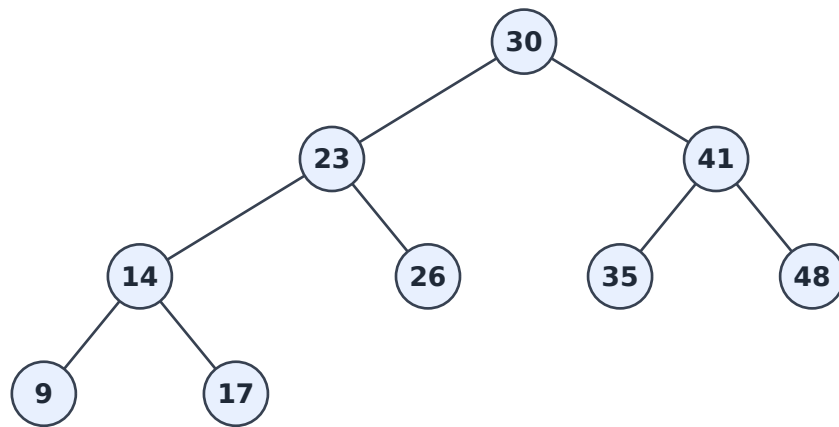
In post-order the **last** key is the root; the earlier keys split into those smaller (left subtree) and larger (right subtree), each again in post-order. Since the in-order of a BST is the sorted order, the post-order alone determines the BST.

- Root 30. Left part: 9, 17, 14, 26, 23 (all < 30). Right part: 35, 48, 41.
- Left subtree root 23: smaller $\{9, 17, 14\}$ → root 14 with children 9, 17; larger $\{26\}$.
- Right subtree root 41 with children 35 and 48.

Tree: $30 \rightarrow (23 \rightarrow (14 \rightarrow 9, 17), 26), (41 \rightarrow 35, 48)$.

- (A) 26 has no right child; it is in 30's left subtree, so its successor is **30. False.**
- (B) Leaves are 9, 17, 26, 35, 48 — five, not four. **False.**
- (C) Pre-order 30, 23, 14, 9, 17, 26, 41, 35, 48. **True.**
- (D) Longest path $30 \rightarrow 23 \rightarrow 14 \rightarrow 9$ has 3 edges. **True.**

Trap: in (A), 35 is the next key *within the right subtree*, but the root 30 lies between 26 and 35.



BST reconstructed from the post-order

Mock Test 07

Heaps & Priority Queues

Difficulty: Moderate

Focus areas: binary heaps, heapify, build-heap, k-th element, heap arrays

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — slicing with negative steps]

Consider the following Python program. What is printed?

Python

```
1 a = [3, 8, 1, 9, 4, 7, 6]
2 print(a[-2:0:-2], a[::-3])
```

- (A) [7, 9] [6, 9, 3]
- (B) [7, 9, 8, 3] [6, 9, 3]
- (C) [7, 9, 8] [6, 9, 3]
- (D) [7, 9, 8] [3, 9, 6]

Q.2 [NAT · numerical answer | 1 mark | Max-heap insertion (sift-up)]

The keys 12, 25, 7, 31, 18, 40, 29 are inserted one at a time, in this order, into an initially empty binary max-heap. Each insertion places the key in the next free array slot and sifts it up, exchanging it with its parent while the parent is smaller. The total number of parent-child exchanges performed over all seven insertions is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Heap property in arrays]

Arrays below are 0-indexed; the children of index i are $2i + 1$ and $2i + 2$. Which one of the following arrays represents a valid binary **max-heap**?

- (A) [90, 72, 85, 60, 75, 81, 30, 55]
- (B) [90, 85, 72, 60, 81, 70, 30, 55]
- (C) [90, 85, 72, 55, 81, 75, 30, 60]
- (D) [90, 81, 85, 60, 72, 30, 87, 55]

Q.4 [MSQ · one or more correct | 1 mark | Stacks — permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack in this order. Pops may be interleaved with pushes arbitrarily, and every popped value is written to the output immediately. Which of the following output sequences is/are possible?

- (A) 2 4 3 6 1 5
- (B) 1 5 2 4 3 6
- (C) 4 5 3 6 2 1
- (D) 3 2 5 4 6 1

Q.5 [NAT · numerical answer | 1 mark | Hashing — linear probing]

Keys 15, 26, 4, 37, 9, 20 are inserted, in this order, into an initially empty hash table with 11 slots (indices 0–10) using $h(k) = (3k + 2) \bmod 11$ and linear probing (on a collision try the next slot, wrapping around). A *probe* is one examination of a slot, including the slot where the key is finally placed. The total number of probes over all six insertions is _____.

Answer: _____ (enter a numerical value)

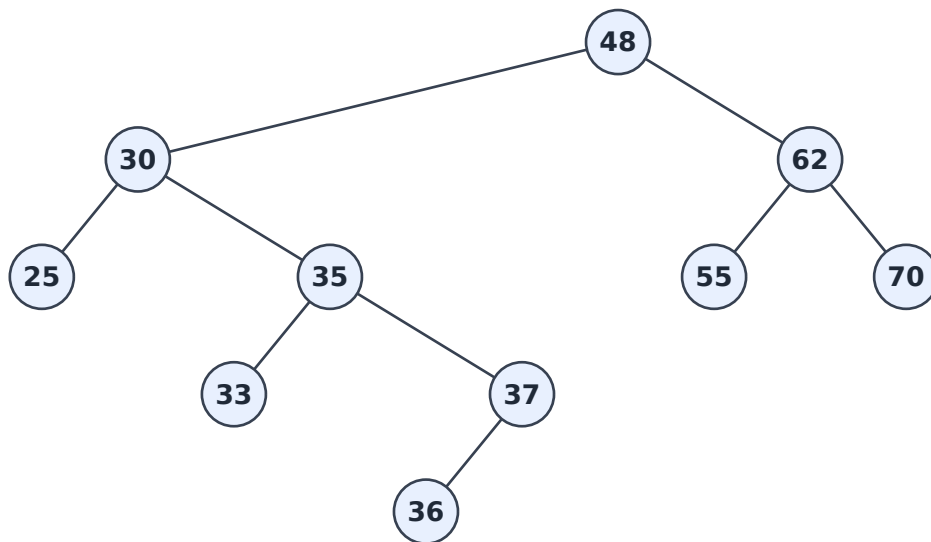
Q.6 [NAT · numerical answer | 1 mark | Selection sort — swaps]

Selection sort is applied to the array [29, 10, 14, 37, 13, 5, 41, 22] to sort it in ascending order. In pass i ($i = 0, 1, \dots, n - 2$) it finds the index m of the minimum of $A[i..n-1]$ and exchanges $A[i]$ and $A[m]$ **only if $m \neq i$** . The number of exchanges performed is _____.

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | BST insertion and deletion]

The keys 48, 30, 62, 25, 35, 55, 70, 33, 37, 36 are inserted in this order into an initially empty binary search tree, giving the tree shown. Key 30 is then deleted; a node with two children is replaced by its **in-order successor**. The pre-order traversal of the resulting tree is

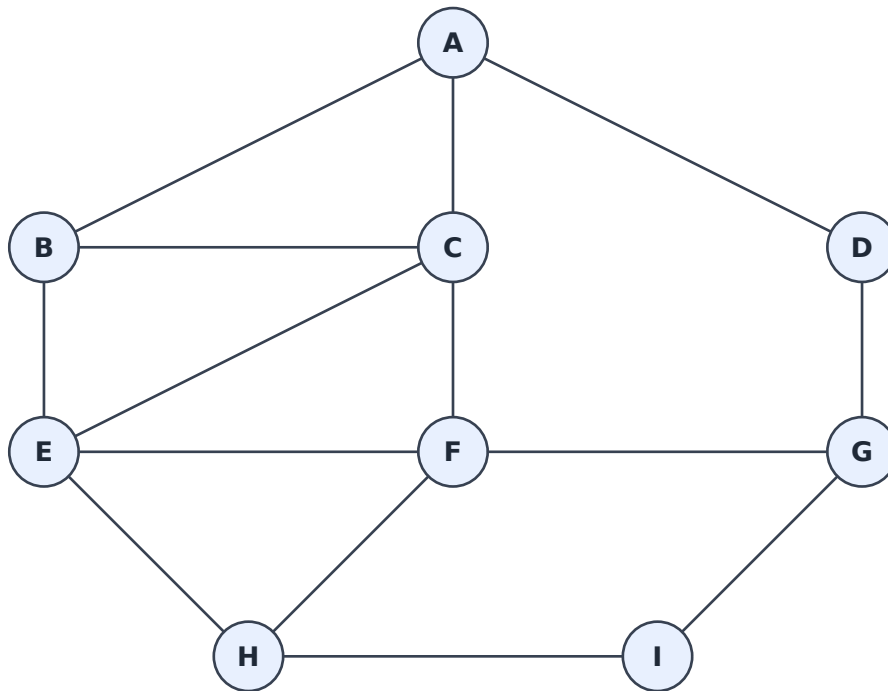


BST before deleting 30

- (A) 48, 33, 25, 35, 36, 37, 62, 55, 70
- (B) 48, 35, 25, 33, 37, 36, 62, 55, 70
- (C) 48, 25, 35, 33, 37, 36, 62, 55, 70
- (D) 48, 33, 25, 35, 37, 36, 62, 55, 70

Q.8 [NAT · numerical answer | 1 mark | BFS levels]

Breadth-first search is run from vertex A on the undirected graph shown. Each vertex gets a level equal to its BFS distance from A. The number of edges of the graph whose two endpoints have the **same** level is _____.



Answer: _____ (enter a numerical value)

Q.9 [MSQ · one or more correct | 1 mark | Linked lists — pointer manipulation]

A singly linked list `head → 4 → 9 → 2 → 7 → 5` is built and the loop below is executed. Which of the following statements is/are TRUE after the loop terminates?

Python

```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 head = None
6 for v in [5, 7, 2, 9, 4]:
7     head = Node(v, head)
8
9 p = head
10 while p.next and p.next.next:
11     p.next = p.next.next
12     p = p.next

```



List before the loop

- (A) The node holding 9 is no longer reachable from `head`
- (B) For any list of $n \geq 1$ nodes, the loop leaves exactly $\lceil n/2 \rceil$ nodes reachable from `head`
- (C) When the loop ends, `p` refers to the node holding 7
- (D) Traversing from `head` now visits 4, 2, 5

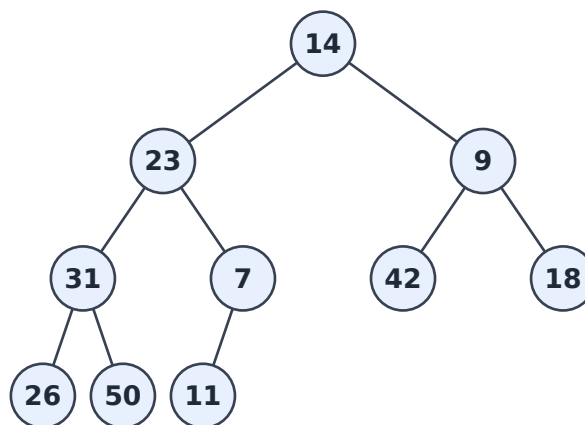
Q.10 [MCQ · single correct | 1 mark | Merge — comparison count]

The standard merge procedure of merge sort merges the sorted lists $L = [2, 9, 14, 20]$ and $R = [5, 6, 15, 30, 31]$. It compares the current heads of L and R , moves the smaller to the output, and when one list becomes empty it copies the rest of the other list without comparisons. The number of key comparisons made is

- (A) 6
- (B) 7
- (C) 8
- (D) 9

Q.11 [NAT · numerical answer | 2 marks | Build-heap (bottom-up heapify)]

The array $[14, 23, 9, 31, 7, 42, 18, 26, 50, 11]$ (0-indexed) is converted into a max-heap by the bottom-up build-heap procedure: for $i = \lfloor n/2 \rfloor - 1$ down to 0, `sift_down(i)` repeatedly exchanges the key with its **larger** child while that child is larger. The total number of exchanges performed is _____.

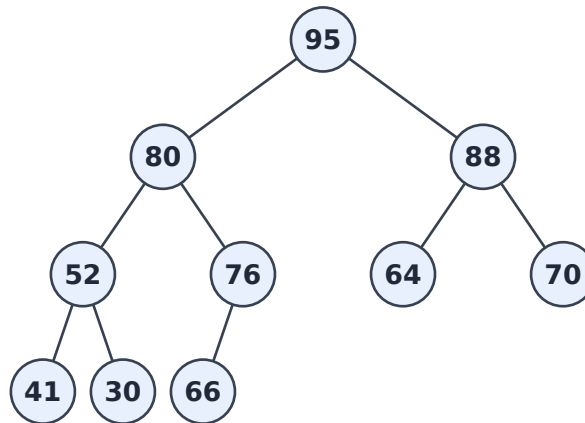


Initial array viewed as a complete binary tree

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Heap delete-max]

The max-heap shown is stored as [95, 80, 88, 52, 76, 64, 70, 41, 30, 66]. Two **delete-max** operations are performed. Each one moves the last array element to the root, shrinks the array by one, and sifts the root down by exchanging it with its larger child while that child is larger. The array after both deletions is



Initial max-heap

- (A) [80, 76, 70, 52, 30, 64, 66, 41]
- (B) [80, 76, 66, 52, 30, 64, 70, 41]
- (C) [80, 66, 70, 52, 76, 64, 30, 41]
- (D) [76, 80, 70, 52, 30, 64, 66, 41]

Q.13 [MSQ · one or more correct | 2 marks | Positions of k-th largest in a heap]

A binary max-heap holds 20 **distinct** keys in $H[1..20]$ (1-indexed; the children of i are $2i$ and $2i + 1$). Which of the following statements is/are TRUE for every such heap?

- (A) The smallest key is stored at an index in the range 11..20
- (B) The second-largest key is stored at index 2 or index 3
- (C) There exists such a heap in which the third-largest key is stored at index 6
- (D) There exists such a heap in which the fourth-largest key is stored at index 16

Q.14 [NAT · numerical answer | 2 marks | Counting heaps]

Consider all binary **min**-heaps that can be formed with the seven distinct keys 1, 2, 3, 4, 5, 6, 7 (stored as a complete binary tree of height 2). In how many of these heaps is the key 3 a **leaf**?

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Python heapq with tuples]

Consider the following Python program. What is printed?

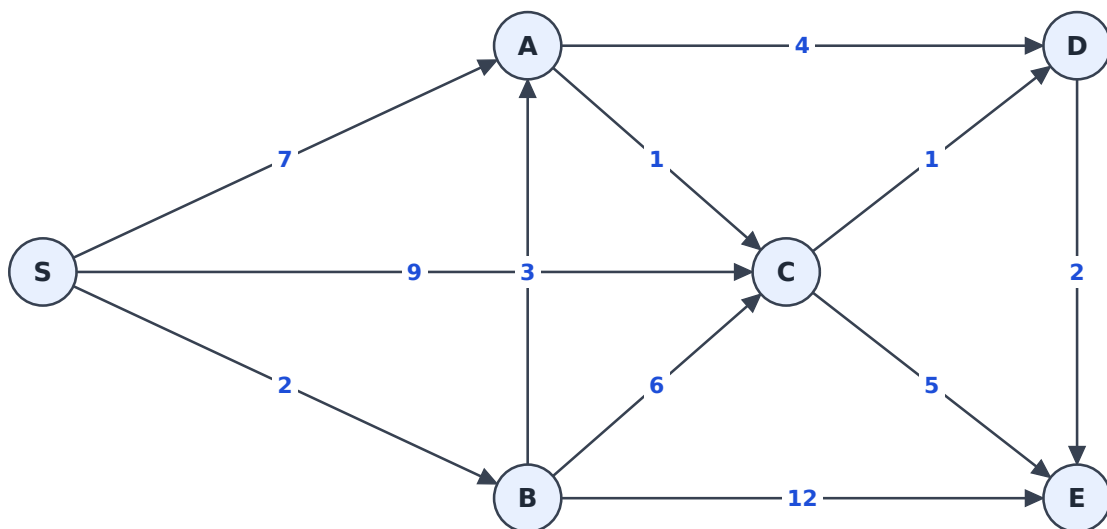
Python

```
1 import heapq
2 tasks = [(3, 'e'), (1, 'b'), (4, 'a'), (1, 'a'), (5, 'c'), (2, 'd')]
3 h = []
4 for t in tasks:
5     heapq.heappush(h, t)
6 out = []
7 while len(h) > 2:
8     p, s = heapq.heappop(h)
9     out.append(s)
10    if p % 2:
11        heapq.heappush(h, (p + 3, s.upper()))
12 print(''.join(out), h[0])
```

- (A) abdeABa (5, 'c')
- (B) abdeaAB (5, 'c')
- (C) abdeAB (4, 'a')
- (D) badeABa (5, 'c')

Q.16 [NAT · numerical answer | 2 marks | Dijkstra with a lazy priority queue]

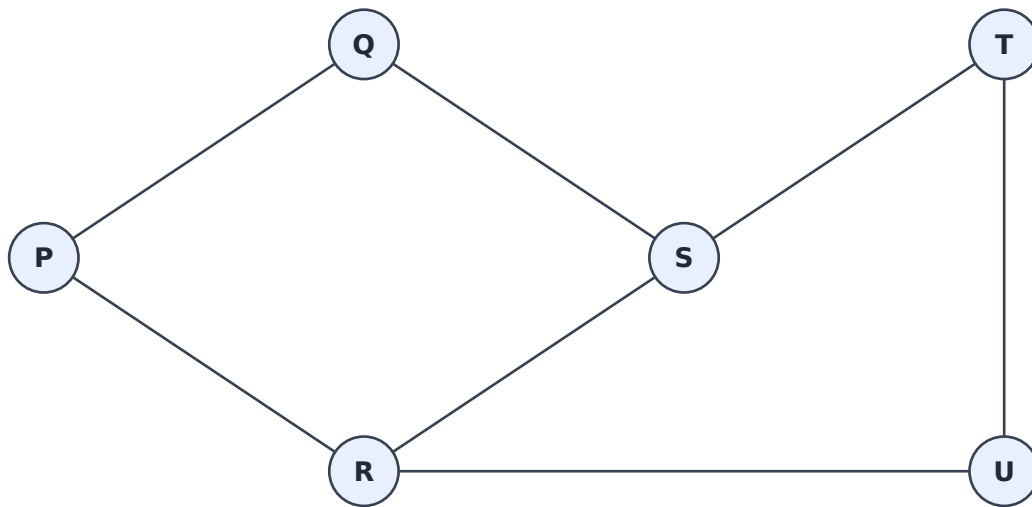
Dijkstra's algorithm from source S is implemented with a binary min-heap **without** decrease-key: initially (0, S) is pushed; whenever an edge relaxation strictly improves $\text{dist}[v]$, the pair $(\text{dist}[v], v)$ is pushed; popped pairs whose distance exceeds the current $\text{dist}[v]$ are discarded. For the directed graph shown, the total number of push operations (including the initial push of S) is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Valid DFS orders]

Depth-first search (recursive, any neighbour order allowed) is started at vertex P of the undirected graph shown. Which of the following is/are possible orders in which DFS **discovers** the vertices?



- (A) P, R, S, U, T, Q
- (B) P, Q, S, T, R, U
- (C) P, Q, S, R, U, T
- (D) P, R, U, T, S, Q

Q.18 [NAT · numerical answer | 2 marks | Quicksort — Lomuto partition]

Consider the following quicksort with Lomuto partitioning (last element as pivot). Every execution of a swap statement is counted, even if it exchanges an element with itself. The value printed is _____.

Python

```

1 cnt = 0
2 def partition(a, lo, hi):
3     global cnt
4     p, i = a[hi], lo - 1
5     for j in range(lo, hi):
6         if a[j] <= p:
7             i += 1
8             a[i], a[j] = a[j], a[i]
9             cnt += 1
10    a[i + 1], a[hi] = a[hi], a[i + 1]
11    cnt += 1
12    return i + 1
13
14 def qsort(a, lo, hi):
15     if lo < hi:
16         q = partition(a, lo, hi)
17         qsort(a, lo, q - 1)
18         qsort(a, q + 1, hi)
19
20 a = [38, 12, 57, 24, 9, 71, 45, 30]
21 qsort(a, 0, len(a) - 1)
22 print(cnt)

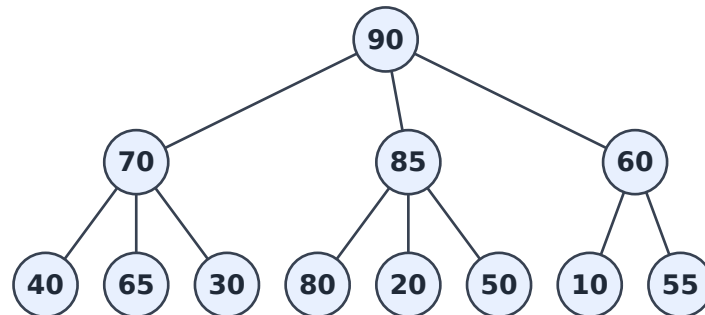
```

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | d-ary heaps]

A **3-ary** max-heap is stored in a 0-indexed array: the children of index i are $3i + 1$, $3i + 2$, $3i + 3$ and the parent of i is $\lfloor (i - 1)/3 \rfloor$. The heap shown is

$[90, 70, 85, 60, 40, 65, 30, 80, 20, 50, 10, 55]$. Keys 75 and then 88 are inserted (append, then sift up while the parent is smaller). The resulting array is



3-ary max-heap before the insertions

- (A) $[90, 70, 88, 75, 40, 65, 30, 80, 20, 50, 10, 55, 60, 85]$
- (B) $[90, 88, 85, 75, 40, 65, 30, 80, 20, 50, 10, 55, 60, 70]$
- (C) $[90, 88, 85, 75, 70, 65, 30, 80, 20, 50, 10, 55, 60, 40]$
- (D) $[90, 88, 85, 60, 70, 65, 30, 80, 20, 50, 10, 55, 75, 40]$

Q.20 [MSQ · one or more correct | 2 marks | Priority-queue operation costs]

Let n be the number of keys in a binary heap stored in an array. Which of the following statements is/are TRUE (worst-case bounds)?

- (A) Given the array index of a key, decrease-key in a binary min-heap takes $O(\log n)$ time
- (B) Dijkstra's algorithm with a binary heap on a graph with V vertices and E edges runs in $O((V + E) \log V)$ time
- (C) The maximum key of a binary **min**-heap can be found in $O(\log n)$ time
- (D) Building a heap bottom-up from n arbitrary keys takes $\Theta(n)$ time

Answer Key — Mock Test 07

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — slicing with negative steps
2	NAT	1	5	Max-heap insertion (sift-up)
3	MCQ	1	(B)	Heap property in arrays
4	MSQ	1	(C), (D)	Stacks — permutations
5	NAT	1	13	Hashing — linear probing
6	NAT	1	5	Selection sort — swaps
7	MCQ	1	(D)	BST insertion and deletion
8	NAT	1	4	BFS levels
9	MSQ	1	(A), (D)	Linked lists — pointer manipulation
10	MCQ	1	(B)	Merge — comparison count
11	NAT	2	8	Build-heap (bottom-up heapify)
12	MCQ	2	(A)	Heap delete-max
13	MSQ	2	(A), (B), (C)	Positions of k -th largest in a heap
14	NAT	2	32	Counting heaps
15	MCQ	2	(A)	Python heapq with tuples

Q	Type	Marks	Key	Topic
16	NAT	2	12	Dijkstra with a lazy priority queue
17	MSQ	2	(C), (D)	Valid DFS orders
18	NAT	2	9	Quicksort — Lomuto partition
19	MCQ	2	(C)	d-ary heaps
20	MSQ	2	(A), (B), (D)	Priority-queue operation costs

Detailed Solutions — Mock Test 07

Q.1 [MCQ · 1 mark · Python — slicing with negative steps]

Answer: (C)

With a negative step a slice walks **right to left**, starts at **start**, and stops *before* reaching **stop** (the stop index is always excluded).

- The list has 7 elements, so index -2 is index 5, i.e. the value 7.
- `a[-2:0:-2]` visits indices 5, 3, 1 and would next visit -1 ... but index 0 is the stop, so it halts after 1. Values: `a[5]=7`, `a[3]=9`, `a[1]=8` → `[7, 9, 8]`.
- `a[::-3]` with omitted start/stop and a negative step starts at the last index 6 and goes down to the beginning: indices 6, 3, 0 → values 6, 9, 3 → `[6, 9, 3]`.

Option analysis:

- (A) Stops one step early — index 1 is still > 0 , so it is included.
- (B) Wrongly includes the stop index 0 (value 3).
- (C) Correct.
- (D) Reverses the result of `a[::-3]`, as if the slice were taken left-to-right and then reversed.

Trap: the stop bound is exclusive in both directions; with step $-k$ the first element taken is the *start* index, not the stop index.

Q.2 [NAT · 1 mark · Max-heap insertion (sift-up)]

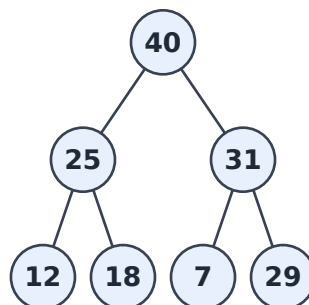
Answer: 5

Insertion = append at the end, then sift up. Count one exchange per swap with the parent.

- 12 → `[12]` (0 swaps)
- 25 → parent 12 $<$ 25, swap → `[25, 12]` (1)
- 7 → parent 25 $>$ 7 → `[25, 12, 7]` (0)
- 31 → index 3, parent 12: swap; then parent 25: swap → `[31, 25, 7, 12]` (2)
- 18 → index 4, parent 25 $>$ 18 → no swap (0)
- 40 → index 5, parent 7: swap; then parent 31: swap → `[40, 25, 31, 12, 18, 7]` (2)
- 29 → index 6, parent 31 $>$ 29 → no swap (0)

Total = $1 + 2 + 2 = 5$. Final heap `[40, 25, 31, 12, 18, 7, 29]`.

Tip: a key sifts up at most $\lfloor \log_2 i \rfloor$ levels when it is the i -th key (1-based), so inserting a new maximum at position 6 costs exactly 2 swaps. **Trap:** do not confuse this with bottom-up build-heap, which would give a different number of swaps and a different final array.



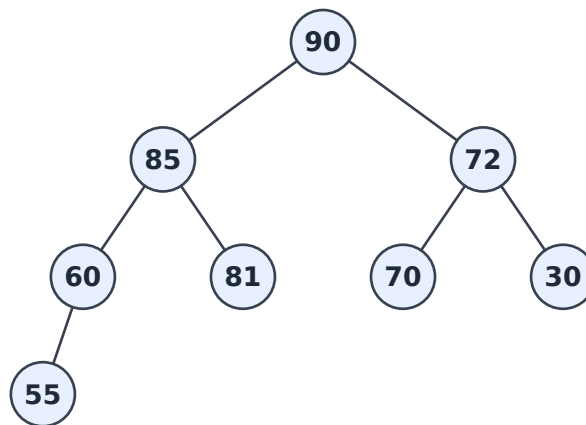
Max-heap after all seven insertions

Q.3 [MCQ · 1 mark · Heap property in arrays]**Answer: (B)**

A max-heap needs $A[\text{parent}(i)] \geq A[i]$ for every $i \geq 1$, where $\text{parent}(i) = \lfloor (i - 1)/2 \rfloor$. Check every parent against its children.

- (A) Index 1 holds 72 and its children (indices 3, 4) are 60 and **75**. $75 > 72 \rightarrow$ **invalid**.
- (B) $90 \geq 85, 72$; $85 \geq 60, 81$; $72 \geq 70, 30$; $60 \geq 55 \rightarrow$ every check passes $\rightarrow$ **valid**.
- (C) Index 2 holds 72 and its child at index 5 is **75** $\rightarrow$ **invalid**. It also fails at index 3: 55 has child 60.
- (D) Index 2 holds 85 and its child at index 6 is **87** $\rightarrow$ **invalid**.

Trap: being sorted in descending order is sufficient but *not* necessary for a max-heap. Siblings can be in either order, and a key may be smaller than a key in a different subtree at a deeper level (in B, 70 at depth 2 is less than 81 at depth 2 in the other subtree, which is fine).



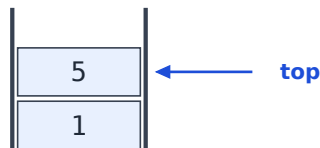
Option (B) drawn as a complete binary tree

Q.4 [MSQ · 1 mark · Stacks — permutations]**Answer: (C), (D)**

Simulate greedily: to output x , push everything up to x (if not yet pushed), then x must be on top.

- (A) push 1,2 pop **2**; push 3,4 pop **4**; pop **3**; push 5,6 pop **6**. Stack is now [1, 5] with 5 on top, but 1 is required $\rightarrow$ **impossible**.
- (B) push 1 pop **1**; push 2..5 pop **5**; top is 4 but 2 is required $\rightarrow$ **impossible**.
- (C) push 1..4 pop **4**; push 5 pop **5**; pop **3**; push 6 pop **6**; pop **2**; pop **1**. **Possible**.
- (D) push 1,2,3 pop **3**; pop **2**; push 4,5 pop **5**; pop **4**; push 6 pop **6**; pop **1**. **Possible**.

Tip: an output sequence is impossible exactly when it contains three values appearing in the order high, low, middle (a , then c , then b with $a > b > c$) — the forbidden “3-1-2” pattern. In (A) 6, 1, 5 forms it; in (B) 5, 2, 4 forms it.



(B) stuck: stack after 6 is popped

Q.5 [NAT · 1 mark · Hashing — linear probing]**Answer: 13**

Compute home slots first: $h(15) = 47 \bmod 11 = 3$, $h(26) = 80 \bmod 11 = 3$, $h(4) = 14 \bmod 11 = 3$, $h(37) = 113 \bmod 11 = 3$, $h(9) = 29 \bmod 11 = 7$, $h(20) = 62 \bmod 11 = 7$.

- 15: slot 3 empty → 1 probe.
- 26: 3 full, 4 empty → 2 probes.
- 4: 3, 4 full, 5 empty → 3 probes.
- 37: 3, 4, 5 full, 6 empty → 4 probes.
- 9: slot 7 empty → 1 probe.
- 20: 7 full, 8 empty → 2 probes.

Total = 1 + 2 + 3 + 4 + 1 + 2 = **13**.

Notice how the cluster 3–6 grows and becomes adjacent to slot 7: any later key hashing to 3..8 will have to walk the whole run (primary clustering).

Trap: counting only *collisions* (failed probes) gives 7; the question counts every slot examined, including the successful one.

0	
1	
2	
3	15
4	26
5	4
6	37
7	9
8	20
9	
10	

Final table

Q.6 [NAT · 1 mark · Selection sort — swaps]

Answer: 5

Selection sort fixes position i in pass i ; an exchange is needed only when the minimum is not already in place.

- $i=0$: min of whole array is 5 (index 5) → swap → [5, 10, 14, 37, 13, 29, 41, 22] (1)
- $i=1$: min of $A[1..]$ is 10, already at index 1 → no swap
- $i=2$: min 13 at index 4 → swap → [5, 10, 13, 37, 14, 29, 41, 22] (2)
- $i=3$: min 14 at index 4 → swap → [5, 10, 13, 14, 37, 29, 41, 22] (3)
- $i=4$: min 22 at index 7 → swap → [5, 10, 13, 14, 22, 29, 41, 37] (4)
- $i=5$: 29 in place → no swap
- $i=6$: min 37 at index 7 → swap → [5, 10, 13, 14, 22, 29, 37, 41] (5)

Total exchanges = **5**.

Tip: selection sort never makes more than $n - 1$ swaps, which is why it is preferred when writes are expensive; but the exact count needs a trace, because each swap moves an element into a new position that later passes see. **Trap:** answering 7 ($= n - 1$) assumes a swap in every pass.

Q.7 [MCQ · 1 mark · BST insertion and deletion]

Answer: (D)

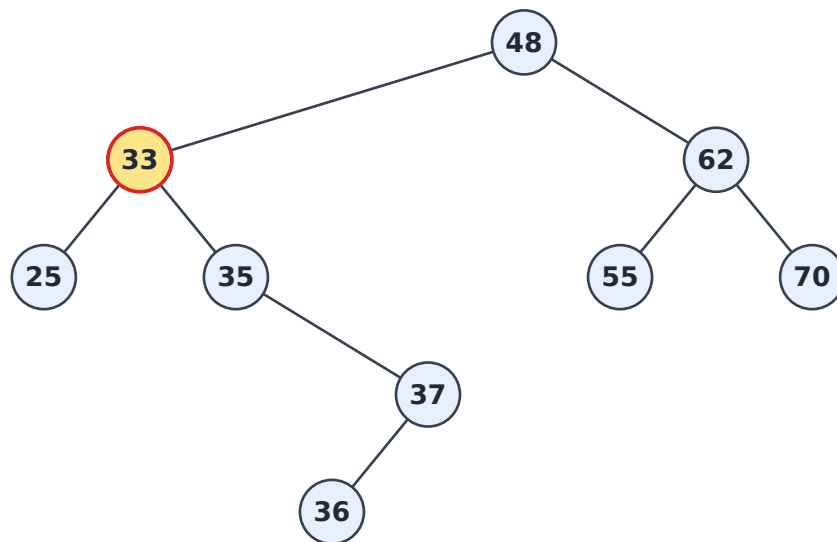
Node 30 has two children, so it is overwritten by its in-order successor = the minimum of its right subtree. From 35 go left as far as possible: $35 \rightarrow 33$, and 33 has no left child, so the successor is **33**. Copy 33 into the node and delete the leaf 33.

Resulting tree: 48 with left child 33 (children 25 and 35), 35 has only a right child 37, and 37 has left child 36; the right subtree 62 (55, 70) is unchanged.

Pre-order (root, left, right): 48, 33, 25, 35, 37, 36, 62, 55, 70.

- (A) Lists 36 before 37, but 36 is the *left child* of 37, so pre-order visits 37 first.
- (B) Puts 35 (the *right child*, not the successor) at the top of the subtree — a common slip.
- (C) Uses the in-order **predecessor** 25 instead of the successor.
- (D) Correct.

Trap: the successor is the leftmost node of the right subtree, not the right child.



After deleting 30

Q.8 [NAT · 1 mark · BFS levels]

Answer: 4

BFS distances do not depend on the order in which neighbours are scanned, so levels are unique.

- Level 0: A
- Level 1: B, C, D (neighbours of A)
- Level 2: E (via B or C), F (via C), G (via D)
- Level 3: H (via E or F), I (via G)

Now classify each of the 14 edges by the levels of its endpoints. Same-level edges:

- B-C (1, 1)
- E-F (2, 2)
- F-G (2, 2)
- H-I (3, 3)

All other edges join consecutive levels. Answer = 4.

Tip: in an undirected graph every BFS non-tree edge joins vertices whose levels differ by at most 1; a same-level edge always closes an **odd** cycle, so the graph is not bipartite. **Trap:** do not count edges like C-E (levels 1 and 2) — they are non-tree edges but cross levels.

Q.9 [MSQ · 1 mark · Linked lists — pointer manipulation]

Answer: (A), (D)

Each iteration bypasses the node after **p** and then advances **p** to the node two ahead.

- Start: $p = 4$. $p.next(9)$ and $p.next.next(2)$ exist $\rightarrow 4.next = 2$, $p = 2$.
- $p = 2$: $p.next = 7$, $p.next.next = 5$ exist $\rightarrow 2.next = 5$, $p = 5$.

- $p = 5$: $p.next$ is None $\rightarrow$ loop ends.

List is now $4 \rightarrow 2 \rightarrow 5$.

- (A) **True** — $4.next$ was redirected to 2, so 9 is bypassed (and 7 likewise).
- (B) **False** — for even n the last node survives too. With $n = 6$ (1..6): $1 \rightarrow 3$, $p=3$; $3 \rightarrow 5$, $p=5$; $5.next.next$ is None, so the list is 1, 3, 5, 6 — 4 nodes, not $\lceil 6/2 \rceil = 3$. In general the length is $\lfloor n/2 \rfloor + 1$.
- (C) **False** — p ends at the node holding 5.
- (D) **True**.

Trap: the loop condition requires *two* nodes ahead; when only one remains it is kept.



Q.10 [MCQ · 1 mark · Merge — comparison count]

Answer: (B)

Merging two lists of sizes m and n uses between $\min(m, n)$ and $m + n - 1$ comparisons; the exact number is $(m + n) - (\text{number of elements copied after one list empties})$.

- 2 vs 5 $\rightarrow$ 2
- 9 vs 5 $\rightarrow$ 5
- 9 vs 6 $\rightarrow$ 6
- 9 vs 15 $\rightarrow$ 9
- 14 vs 15 $\rightarrow$ 14
- 20 vs 15 $\rightarrow$ 15
- 20 vs 30 $\rightarrow$ 20 (L is now empty)

That is **7** comparisons; 30 and 31 are copied for free.

Check with the formula: 9 elements – 2 copied without comparison = 7.

- (A) 6 forgets the last comparison 20 vs 30.
- (C) $8 = m + n - 1$ is the worst case, reached only if the last two output elements come from different lists.
- (D) 9 counts a comparison for every element output.

Tip: look at the *tail* of the merged output: the maximal run of elements from one list at the end is copied for free.

Q.11 [NAT · 2 marks · Build-heap (bottom-up heapify)]

Answer: 8

Bottom-up build-heap fixes the subtrees rooted at the internal nodes, from the last internal node (index $\lfloor 10/2 \rfloor - 1 = 4$) back to the root. A key may travel several levels.

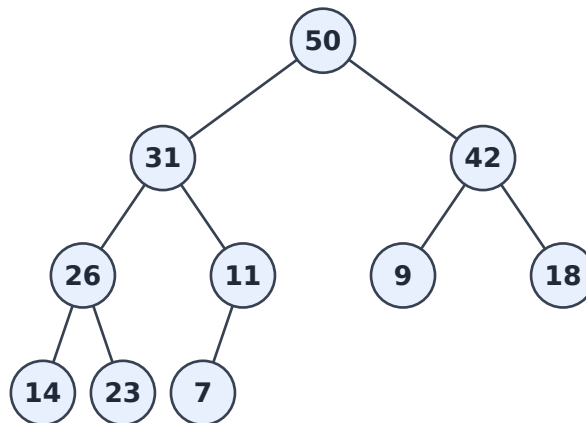
- $i = 4$ (key 7): only child 11 $\rightarrow$ swap (1). [14, 23, 9, 31, 11, 42, 18, 26, 50, 7]
- $i = 3$ (key 31): children 26, 50 $\rightarrow$ swap with 50 (2). [14, 23, 9, 50, 11, 42, 18, 26, 31, 7]
- $i = 2$ (key 9): children 42, 18 $\rightarrow$ swap with 42 (3); 9 is now a leaf.
- $i = 1$ (key 23): children 50, 11 $\rightarrow$ swap with 50 (4); at index 3 its children are 26, 31 $\rightarrow$ swap with 31 (5).
- $i = 0$ (key 14): children 50, 42 $\rightarrow$ swap with 50 (6); at index 1 children 31, 11 $\rightarrow$ swap with 31 (7); at index 3 children 26, 23 $\rightarrow$ swap with 26 (8).

Total = **8** exchanges. Final heap: [50, 31, 42, 26, 11, 9, 18, 14, 23, 7].

Why it is $\Theta(n)$: a node at height h can move at most h levels, and $\sum h \cdot n/2^{h+1} = O(n)$. Here the internal nodes 4, 3, 2, 1, 0 have heights 1, 1, 1, 2, 3, so at most $1 + 1 + 1 + 2 + 3 = 8$ exchanges

are possible — this input attains the maximum.

Trap: stopping a sift-down after one swap (e.g. leaving 23 at index 3 or 14 at index 1) gives 5 or 6 — the key must keep sinking until both children are smaller.



Final max-heap (keys that sank twice or more highlighted)

Q.12 [MCQ · 2 marks · Heap delete-max]

Answer: (A)

Delete-max #1 (removes 95): the last key 66 goes to the root → [66, 80, 88, 52, 76, 64, 70, 41, 30].

- 66 vs children 80, 88 → swap with 88 → 66 at index 2.
- 66 vs children 64, 70 → swap with 70 → 66 at index 6 (leaf).

Heap: [88, 80, 70, 52, 76, 64, 66, 41, 30].

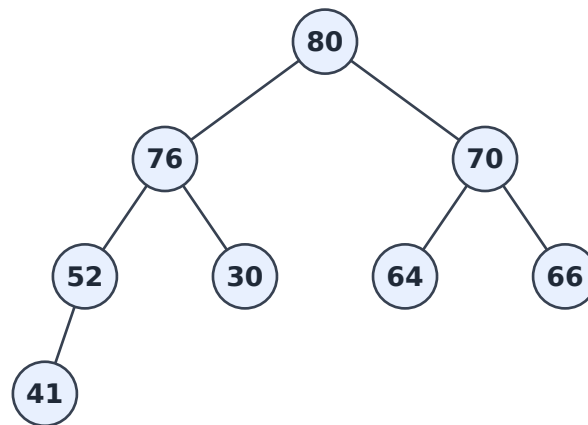
Delete-max #2 (removes 88): the last key 30 goes to the root → [30, 80, 70, 52, 76, 64, 66, 41].

- 30 vs children 80, 70 → swap with 80 → index 1.
- 30 vs children 52, 76 → swap with 76 → index 4.
- Index 4 has children at 9, 10 — outside the array of size 8 → stop.

Heap: [80, 76, 70, 52, 30, 64, 66, 41].

- (A) Correct.
- (B) Has 66 and 70 exchanged — what you get if 66 is assumed to stop at index 2 in the first deletion.
- (C) Sifts with the *smaller* child at some step; 66 at index 1 has child 76 > 66, not a heap.
- (D) Not even a heap (76 at the root with child 80).

Trap: always exchange with the **larger** child; exchanging with the smaller child can leave the larger child above its new parent.



Heap after two delete-max operations

Q.13 [MSQ · 2 marks · Positions of k-th largest in a heap]**Answer: (A), (B), (C)**

Key fact: every ancestor of a node is larger than it. So a key that has exactly $r - 1$ larger keys in the heap (the r -th largest) can only sit at depth $\leq r - 1$.

- (A) The smallest key cannot have a child (the child would have to be smaller). Nodes with no children are indices $\lfloor 20/2 \rfloor + 1 = 11$ to 20. **True.**
- (B) The second-largest key has only one larger key (the root), so its depth is ≤ 1 , i.e. index 2 or 3 (it cannot be the root). **True.**
- (C) Index 6 is at depth 2; its ancestors are 3 and 1. Put the largest at 1, the second-largest at 3 and the third-largest at 6 — this can be completed to a valid heap (e.g. fill the remaining slots in decreasing order). **True** (possible).
- (D) Index 16 has depth $\lfloor \log_2 16 \rfloor = 4$, so it has 4 ancestors, all larger. But the fourth-largest has only 3 larger keys. **False** — the fourth-largest is confined to depth ≤ 3 , i.e. indices 2..15.

Trap: in (C) “third-largest must be a child of the root” is a frequent wrong belief; it can be a grandchild when the second-largest is its parent.

Q.14 [NAT · 2 marks · Counting heaps]**Answer: 32**

A key's ancestors must all be smaller than it. If 3 is a leaf (depth 2) it has two ancestors, and the only keys smaller than 3 are 1 and 2 — so the root is 1, 3's parent is 2, and 3 sits under 2.

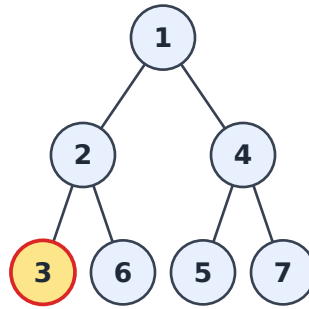
Count the heaps for a fixed leaf position of 3:

- The parent of 3 holds 2 and is one child of the root (fixed by the leaf choice).
- The sibling leaf of 3 can be any of the four keys $\{4, 5, 6, 7\} \rightarrow 4$ ways.
- The other subtree of the root (3 nodes) gets the remaining 3 keys; its root must be the smallest of them and the two leaves can be arranged in 2 ways $\rightarrow 2$ ways.

Per leaf position: $4 \times 2 = 8$. There are 4 leaf positions $\rightarrow 4 \times 8 = \mathbf{32}$.

Sanity check: the total number of min-heaps on 7 keys is $C(6,3) \cdot 2 \cdot 2 = 80$. Key 3 is at depth 1 in the remaining 48 heaps (24 at each child of the root), and it is never at the root.

Trap: assuming the 3 must be near the top because it is “small”; only keys with few smaller keys are restricted, and 3 has exactly two, which permits depth 2.



One of the 32 heaps (3 is a leaf under 2)

Q.15 [MCQ · 2 marks · Python heapq with tuples]

Answer: (A)

`heapq` is a min-heap; tuples compare lexicographically (first the number, then the string). Uppercase letters have smaller code points than lowercase ('A' = 65 < 'a' = 97).

Pops while more than two items remain:

- (1,'a') → out a; p odd → push (4,'A')
- (1,'b') → out b; push (4,'B')
- (2,'d') → out d; p even, nothing pushed
- (3,'e') → out e; push (6,'E')
- Heap now holds (4,'A'), (4,'B'), (4,'a'), (5,'c'), (6,'E').
- (4,'A') → out A (even); (4,'B') → out B; (4,'a') → out a.
- Two items (5,'c'), (6,'E') remain → loop stops. h[0] is the minimum (5,'c').

Output: **abdeABa** (5, 'c').

- (B) Assumes (4,'a') < (4,'A') — lowercase does not sort first.
- (C) Stops one pop early (treats the condition as ≥ 3 remaining after pop).
- (D) Ignores the tie-break on the second field: (1,'a') < (1,'b').

Trap: p + 3 flips parity, so each task is re-pushed at most once — the loop terminates.

Q.16 [NAT · 2 marks · Dijkstra with a lazy priority queue]

Answer: 12

Every strict improvement of a tentative distance causes one push. Trace (dist after each pop):

- push (0,S). [1]
- Pop S(0): A=7, B=2, C=9 → 3 pushes. [4]
- Pop B(2): A: 5 < 7 push; C: 8 < 9 push; E: 14 push. [7]
- Pop A(5): D = 9 push; C: 6 < 8 push. [9]
- Pop C(6): D: 7 < 9 push; E: 11 < 14 push. [11]
- Pop (7,A) — stale, discarded. Pop D(7): E: 9 < 11 push. [12]
- Remaining pops (8,C), (9,C), (9,D) are stale; E(9) relaxes nothing; (11,E), (14,E) stale.

Total pushes = **12** (1 initial + 11 improvements). Final distances: S 0, B 2, A 5, C 6, D 7, E 9.

Note the heap ends up handling 12 pops for 6 vertices: lazy deletion costs $O(E \log E) = O(E \log V)$ time, the same bound as decrease-key.

Trap: counting one push per vertex (6) or one per edge (11) — the count is the number of strict improvements plus one.

dist[] after each non-stale pop

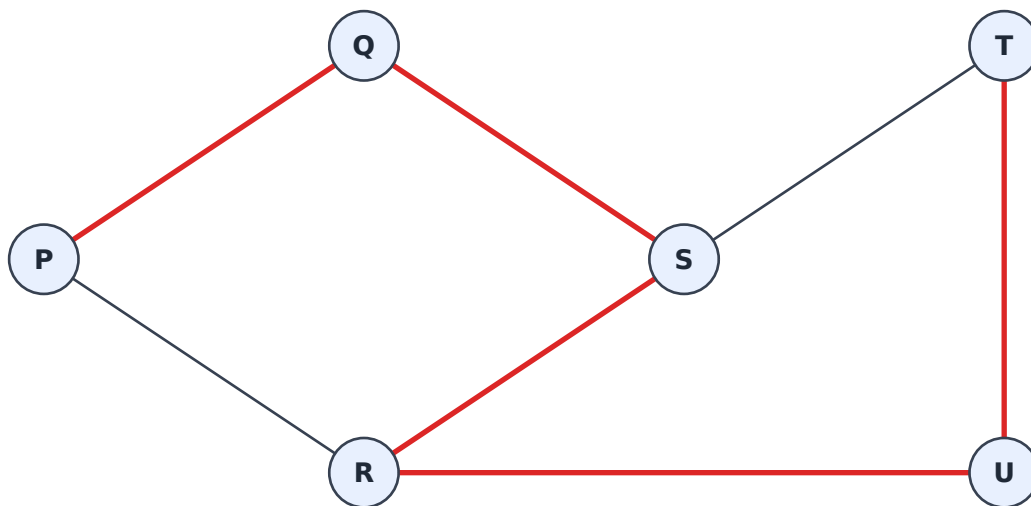
	S	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
pop S	0	7	2	9	∞	∞
pop B	0	5	2	8	∞	14
pop A	0	5	2	6	9	14
pop C	0	5	2	6	7	11
pop D	0	5	2	6	7	9

Q.17 [MSQ · 2 marks · Valid DFS orders]**Answer: (C), (D)**

Rule: each newly discovered vertex must be adjacent to the **deepest vertex on the current recursion path that still has an undiscovered neighbour** (DFS only backtracks when the current vertex is exhausted).

- (A) $P \rightarrow R \rightarrow S \rightarrow U$? S and U are **not** adjacent; at that moment S still has undiscovered neighbours Q and T, so the next vertex must be one of them. **Invalid.**
- (B) $P \rightarrow Q \rightarrow S \rightarrow T$. T still has the undiscovered neighbour U, so DFS must go $T \rightarrow U$ next; it cannot jump to R. **Invalid.**
- (C) $P \rightarrow Q \rightarrow S \rightarrow R$ (S-R edge) $\rightarrow U$ (R-U) $\rightarrow T$ (U-T). Every step goes to a neighbour of the current vertex. **Valid.**
- (D) $P \rightarrow R \rightarrow U \rightarrow T \rightarrow S$ (T-S) $\rightarrow Q$ (S-Q). **Valid.**

Trap: BFS-like thinking (“visit R since it is adjacent to P”) breaks DFS order; a jump back to an ancestor's neighbour is allowed only after the current vertex has no unvisited neighbours.



DFS tree for order (A)

Q.18 [NAT · 2 marks · Quicksort — Lomuto partition]**Answer: 9**

Lomuto performs one swap for every element $\leq$ pivot in the scanned range plus one final swap that places the pivot. So for a call on a range: $\text{swaps} = (\# \text{elements} \leq \text{pivot, excluding pivot}) + 1$.

- Call $[0..7]$, pivot 30: elements ≤ 30 are 12, 24, 9 $\rightarrow 3 + 1 = 4$ swaps. Array $[12, 24, 9, 30, 57, 71, 45, 38]$, pivot at 3.

- Call $[0..2] = [12, 24, 9]$, pivot 9: $\text{none} \leq 9 \rightarrow 0 + 1 = 1$ swap $\rightarrow [9, 24, 12]$, pivot at 0.
- Call $[1..2] = [24, 12]$, pivot 12: $\text{none} \rightarrow 1$ swap $\rightarrow [12, 24]$.
- Call $[4..7] = [57, 71, 45, 38]$, pivot 38: $\text{none} \leq 38 \rightarrow 1$ swap $\rightarrow [38, 71, 45, 57]$, pivot at 4.
- Call $[5..7] = [71, 45, 57]$, pivot 57: $45 \rightarrow 1 + 1 = 2$ swaps $\rightarrow [45, 57, 71]$.
- Remaining ranges have size ≤ 1 (no partition call).

Total = $4 + 1 + 1 + 1 + 2 = 9$.

Trap: forgetting the self-swaps (when $i == j$) or the final pivot swap; the problem counts every execution of the swap statement.

Array after each partition call

	0	1	2	3	4	5	6	7
start	38	12	57	24	9	71	45	30
[0..7]	12	24	9	30	57	71	45	38
[0..2]	9	24	12	30	57	71	45	38
[1..2]	9	12	24	30	57	71	45	38
[4..7]	9	12	24	30	38	71	45	57
[5..7]	9	12	24	30	38	45	57	71

Q.19 [MCQ · 2 marks · d-ary heaps]

Answer: (C)

In a d-ary heap the sift-up is the same as in a binary heap, only the parent formula changes: $\text{parent}(i) = \lfloor (i - 1)/3 \rfloor$.

Insert 75 at index 12: $\text{parent}[11/3] = 3$ holds $60 < 75 \rightarrow$ swap; 75 now at 3, parent 0 holds 90 $\rightarrow$ stop.

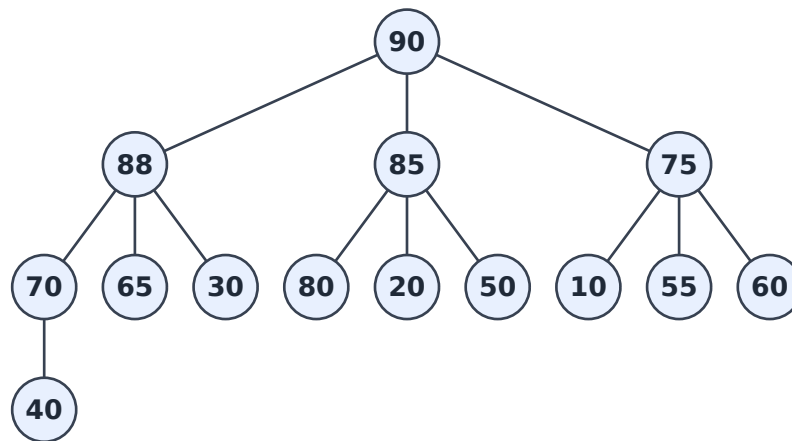
$[90, 70, 85, 75, 40, 65, 30, 80, 20, 50, 10, 55, 60]$

Insert 88 at index 13: $\text{parent}[12/3] = 4$ holds $40 \rightarrow$ swap; 88 at 4, parent $[3/3] = 1$ holds 70 $\rightarrow$ swap; 88 at 1, parent 0 holds 90 $\rightarrow$ stop.

$[90, 88, 85, 75, 70, 65, 30, 80, 20, 50, 10, 55, 60, 40]$

- (A) Exchanges 88 with 85, but 85 (index 2) is not an ancestor of index 13; the ancestors of 13 are 4, 1 and 0.
- (B) Moves 70 to the end instead of shifting it down one level along the path.
- (C) Correct.
- (D) Forgets to sift up 75 (left at index 12).

Tip: a d-ary heap has height $\approx \log_d n$, so insert/sift-up is $O(\log_d n)$ but delete-max costs $O(d \log_d n)$ — it must examine d children per level.



After inserting 75 and 88

Q.20 [MSQ · 2 marks · Priority-queue operation costs]

Answer: (A), (B), (D)

- (A) **True.** Decreasing a key can only violate the heap property with its parent; one sift-up of at most $\lfloor \log_2 n \rfloor$ levels fixes it. Knowing the index avoids an $O(n)$ search.
- (B) **True.** At most V extract-min operations and at most E decrease-key (or lazy push) operations, each $O(\log V) \rightarrow O((V + E) \log V)$.
- (C) **False.** The maximum of a min-heap must be a leaf, but it can be *any* of the $\lfloor n/2 \rfloor$ leaves; the heap order gives no information comparing different leaves, so $\Theta(n)$ comparisons are needed.
- (D) **True.** Sift-down from a node of height h costs $O(h)$; there are at most $\lfloor n/2^{h+1} \rfloor$ nodes of height h , and $\sum h/2^h$ converges, so the total is $\Theta(n)$ — not $\Theta(n \log n)$.

Trap: “heaps support $O(\log n)$ everything” — a heap is only partially ordered; search for an arbitrary key or the opposite extreme (max in a min-heap) is linear.

Tip: for dense graphs ($E \approx V^2$) an array-based Dijkstra in $O(V^2)$ beats the binary heap's $O(V^2 \log V)$.

Mock Test 08

Hash Tables

Difficulty: Moderate

Focus areas: chaining, linear/quadratic probing, double hashing, deletion

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

The keys 38, 15, 74, 24, 63, 50, 29, 9, 57, 44 are inserted in that order into an initially empty hash table with 7 slots (indices 0 to 6) using $h(k) = k \bmod 7$ and separate chaining. Each new key is appended at the **end** of its chain.

Every one of the 10 keys is then searched for exactly once. A search walks its chain from the front and each key examined counts as one comparison. The total number of key comparisons over all 10 successful searches is _____.

Answer: _____ (enter a numerical value)

Q.2 [MCQ · single correct | 1 mark | Python — dictionary keys and hashing]

Consider the following Python program. What is printed?

Python

```
1 d = {1: 'a', True: 'b', 1.0: 'c', 2: 'd'}
2 print(d)
```

- (A) {True: 'c', 2: 'd'}
- (B) {1: 'c', 2: 'd'}
- (C) {1: 'a', True: 'b', 1.0: 'c', 2: 'd'}
- (D) {1.0: 'c', 2: 'd'}

Q.3 [MCQ · single correct | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with a stack (operands are pushed; an operator pops the right operand first, then the left operand, and pushes the result). All operators are binary and $/$ is exact division.

8 6 2 - 3 * 4 / + 5 -

What are the value of the expression and the maximum number of elements ever on the stack, respectively?

- (A) −2 and 3
- (B) 6 and 4
- (C) 11 and 3
- (D) 6 and 3

Q.4 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

Keys 35, 20, 57, 9, 46, 31 are inserted in that order into an initially empty hash table of size 11 (indices 0–10) using quadratic probing:

$h(k, i) = (k + i^2) \bmod 11, i = 0, 1, 2, \dots$

The index of the slot in which key 31 is stored is _____.

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Queues — circular array]

A circular queue is stored in an array $Q[0..5]$ with indices **front** and **rear**, both initially 0.

enqueue(x) stores x at $Q[\text{rear}]$ and sets $\text{rear} = (\text{rear} + 1) \bmod 6$; **dequeue()** returns $Q[\text{front}]$ and sets $\text{front} = (\text{front} + 1) \bmod 6$. The queue is full when $(\text{rear} + 1) \bmod 6 = \text{front}$.

Starting empty, the following operations are performed:

enqueue a, b, c, d; dequeue; dequeue; enqueue e, f, g; dequeue; enqueue h.

After these operations, the values of (front, rear) and the element at the front are:

- (A) (3, 2) and e
- (B) (2, 1) and c
- (C) (3, 2) and d
- (D) (4, 2) and d

Q.6 [MSQ · one or more correct | 1 mark | Binary search trees]

The keys 45, 22, 67, 11, 30, 56, 89, 26, 35, 60 are inserted in that order into an initially empty binary search tree, giving the tree T shown. Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

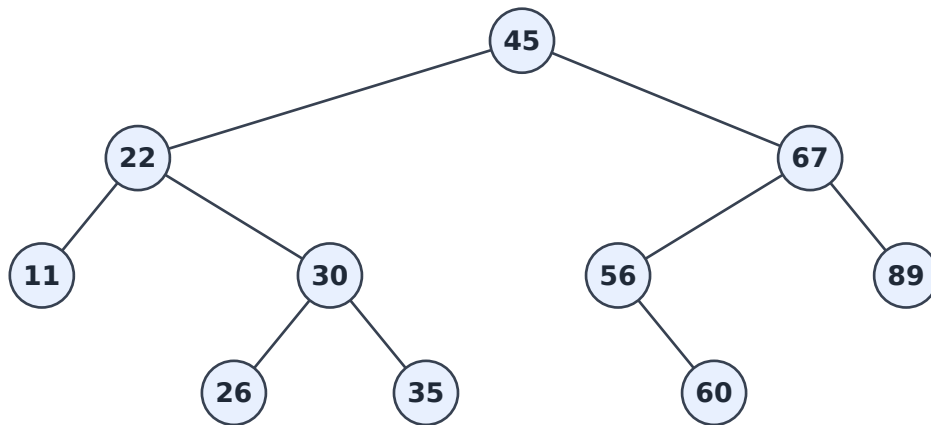


Figure: BST T

- (A) T has exactly 4 leaves
- (B) The height of T is 4
- (C) If 22 is deleted by replacing it with its in-order successor, 26 becomes the left child of 45
- (D) The in-order successor of 35 is 45

Q.7 [MCQ · single correct | 1 mark | Binary search — iteration count]

Consider the following Python program operating on the sorted array shown. What is printed?

Python

```

1 def bs(a, x):
2     lo, hi, c = 0, len(a) - 1, 0
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         c += 1
6         if a[mid] == x:
7             return c
8         elif a[mid] < x:
9             lo = mid + 1
10        else:
11            hi = mid - 1
12    return c
13
14 a = [3, 8, 14, 19, 25, 31, 36, 42, 47, 53, 60, 66]
15 print(bs(a, 50), bs(a, 31))

```

a =	3	8	14	19	25	31	36	42	47	53	60	66
	0	1	2	3	4	5	6	7	8	9	10	11

- (A) 4 1
 (B) 3 1
 (C) 4 6
 (D) 5 1

Q.8 [NAT · numerical answer | 1 mark | Insertion sort — shifts]

Standard insertion sort (ascending) is applied to the array

[31, 12, 47, 5, 28, 19, 40, 3].

A *shift* is one execution of $a[j+1] = a[j]$ in the inner loop. The total number of shifts performed is _____.

Answer: _____ (enter a numerical value)

Q.9 [MCQ · single correct | 1 mark | Linked lists in Python]

Consider the following Python program. What is printed?

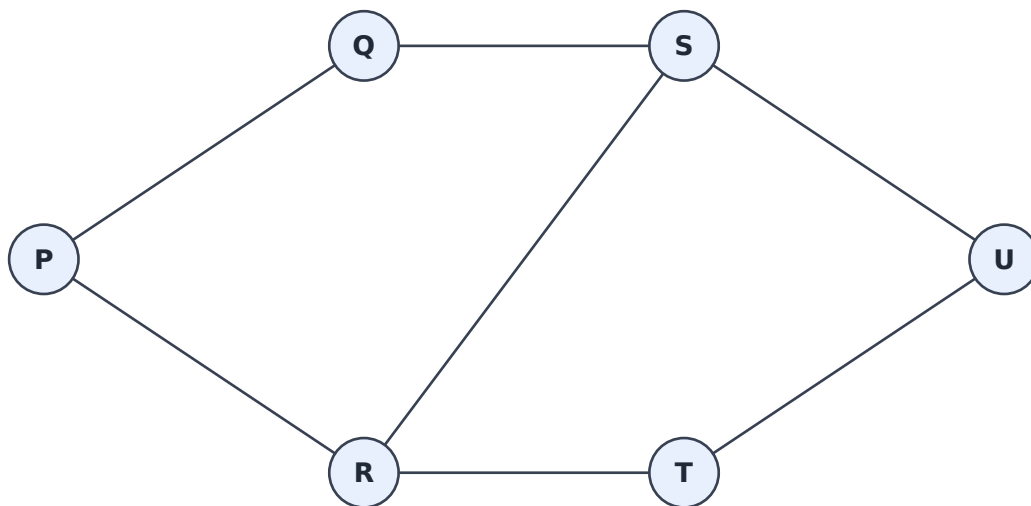
Python

```
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 head = None
6 for v in [4, 8, 15, 16, 23]:
7     head = Node(v, head)
8
9 p = head
10 while p and p.nxt:
11     p.nxt = p.nxt.nxt
12     p = p.nxt
13
14 out, p = [], head
15 while p:
16     out.append(p.v)
17     p = p.nxt
18 print(out)
```

- (A) [4, 15, 23]
- (B) [23, 15, 4]
- (C) [23, 16, 8]
- (D) [16, 8]

Q.10 [MSQ · one or more correct | 1 mark | Graph traversal — BFS orders]

Breadth-first search is started at vertex P of the undirected graph shown. The neighbours of a vertex may be enqueued in **any** order. Which of the following is/are possible BFS visit orders?



- (A) P, R, S, Q, T, U
- (B) P, R, Q, T, S, U
- (C) P, Q, R, T, S, U
- (D) P, Q, R, S, T, U

Q.11 [NAT · numerical answer | 2 marks | Hashing — double hashing]

Keys 41, 58, 22, 70, 35, 96, 19, 48, 32 are inserted in that order into an initially empty table of size 13 (indices 0–12) using double hashing:

$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 7 - (k \bmod 7)$, $i = 0, 1, 2, \dots$

The index of the slot in which the last key, 32, is stored is _____.

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Hashing — deletion in open addressing]

A hash table of size 10 uses $h(k) = k \bmod 10$ with linear probing. Keys 52, 72, 13, 92, 25, 33 are inserted in that order. Then key 72 is deleted. Two deletion policies are considered:

(i) **lazy deletion** — the slot is marked DELETED (a tombstone); searches continue past tombstones and stop only at a never-used empty slot;

(ii) **naive deletion** — the slot is simply made empty.

A search for key 33 is then performed. Counting every slot inspected as one probe, what happens under policies (i) and (ii) respectively?

- (A) (i) 33 found after 5 probes; (ii) 33 reported absent after 1 probe
- (B) (i) 33 found after 4 probes; (ii) 33 found after 4 probes
- (C) (i) 33 found after 5 probes; (ii) 33 found after 4 probes
- (D) (i) 33 reported absent after 1 probe; (ii) 33 reported absent after 1 probe

Q.13 [MSQ · one or more correct | 2 marks | Hashing — reconstructing insertion order]

A hash table of size 10 uses $h(k) = k \bmod 10$ and linear probing. After six insertions into an initially empty table it looks as shown. Which of the following insertion sequences could have produced this table?

0	
1	
2	42
3	23
4	34
5	52
6	33
7	46
8	
9	

Figure: final table

- (A) 23, 34, 42, 52, 33, 46
- (B) 42, 23, 52, 34, 33, 46
- (C) 42, 34, 23, 52, 46, 33
- (D) 34, 42, 23, 52, 33, 46

Q.14 [NAT · numerical answer | 2 marks | Hashing — expected empty slots]

Fifteen keys are inserted into a hash table with 10 slots using separate chaining. Assume simple uniform hashing: each key independently hashes to each slot with probability $1/10$. The expected number of slots that remain **empty** after all insertions is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Python — __hash__ and __eq__ contract]

Consider the following Python program, in which `__eq__` and `__hash__` are deliberately inconsistent. What is printed?

Python

```
1 class P:
2     def __init__(self, x, y):
3         self.x, self.y = x, y
4     def __eq__(self, other):
5         return self.x == other.x
6     def __hash__(self):
7         return hash(self.y)
8
9 s = set()
10 for x, y in [(1, 2), (1, 3), (2, 2), (1, 2)]:
11     s.add(P(x, y))
12 print(len(s), P(1, 99) in s)
```

- (A) 2 True
- (B) 4 False
- (C) 3 False
- (D) 3 True

Q.16 [MSQ · one or more correct | 2 marks | Merge sort — comparison counts]

Top-down merge sort splits an array of length n into a left half of length $\lfloor n/2 \rfloor$ and a right half, sorts both recursively (arrays of length ≤ 1 are returned unchanged) and merges them; the merge compares the two current front elements until one half is exhausted. Which of the following statements is/are TRUE?

- (A) On the already sorted array $[1, 2, \dots, 8]$ it makes exactly 12 comparisons
- (B) The recurrence $C(n) = 2C(n/2) + n - 1$, $C(1) = 0$ gives $C(8) = 24$
- (C) No array of 8 distinct elements needs more than 17 comparisons
- (D) On $[5, 9, 2, 7, 1, 8, 3, 6]$ the algorithm makes exactly 17 element comparisons

Q.17 [MCQ · single correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition scheme (pivot = last element; i starts at $lo - 1$; for each j from lo to $hi - 1$, if $a[j] \leq \text{pivot}$ then i is incremented and $a[i]$, $a[j]$ are swapped; finally $a[i+1]$ and $a[hi]$ are swapped) is applied once to

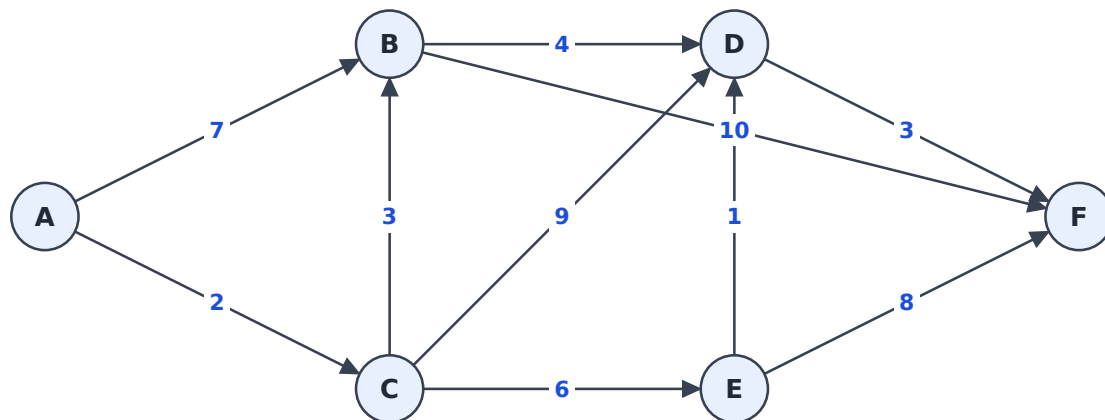
$[42, 17, 63, 8, 55, 29, 71, 36]$.

What is the array after this single partition call?

- (A) $[17, 8, 29, 36, 63, 55, 71, 42]$
- (B) $[17, 8, 29, 36, 42, 55, 63, 71]$
- (C) $[8, 17, 29, 36, 55, 63, 71, 42]$
- (D) $[17, 8, 29, 36, 55, 63, 71, 42]$

Q.18 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from source A on the weighted directed graph shown. The sum of the shortest-path distances from A to the vertices B, C, D, E and F is _____.



Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Binary heaps — insertion and delete-max]

The keys 19, 42, 7, 33, 58, 25, 61, 12 are inserted one at a time, in that order, into an initially empty binary **max**-heap stored in an array (0-based; children of index i are $2i+1$ and $2i+2$). Each insertion appends the key and sifts it up by swapping with its parent while it is larger. Which of the following statements is/are TRUE?

- (A) The level-order sequence of the final heap is 61, 58, 42, 33, 19, 25, 7, 12
- (B) After one delete-max (last element moved to the root, then sift-down) the array is [58, 42, 25, 19, 33, 7, 12]
- (C) A total of 7 swaps are performed during the eight insertions
- (D) The root is 61 and its right child is 58

Q.20 [NAT · numerical answer | 2 marks | Hashing — rehashing with linear probing]

Consider the following Python implementation of an open-addressing hash table that doubles its size whenever an insertion would push the load factor above 0.5. The value printed by the program is _____.

Python

```

1 class HT:
2     def __init__(self):
3         self.m, self.n = 4, 0
4         self.t = [None] * 4
5         self.probes = 0
6     def put(self, k):
7         if (self.n + 1) / self.m > 0.5:
8             self.grow()
9         i = k % self.m
10        while self.t[i] is not None:
11            i = (i + 1) % self.m
12            self.probes += 1
13        self.t[i] = k
14        self.n += 1
15    def grow(self):
16        old = [k for k in self.t if k is not None]
17        self.m *= 2
18        self.t, self.n = [None] * self.m, 0
19        for k in old:
20            self.put(k)
21
22 h = HT()
23 for k in [10, 22, 6, 14, 30, 18, 26]:
24     h.put(k)
25 print(h.probes)

```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 08

Q	Type	Marks	Key	Topic
1	NAT	1	18	Hashing — separate chaining
2	MCQ	1	(B)	Python — dictionary keys and hashing
3	MCQ	1	(D)	Stacks — postfix evaluation
4	NAT	1	7	Hashing — quadratic probing
5	MCQ	1	(C)	Queues — circular array
6	MSQ	1	(C), (D)	Binary search trees
7	MCQ	1	(A)	Binary search — iteration count
8	NAT	1	17	Insertion sort — shifts
9	MCQ	1	(B)	Linked lists in Python
10	MSQ	1	(B), (D)	Graph traversal — BFS orders
11	NAT	2	12	Hashing — double hashing
12	MCQ	2	(A)	Hashing — deletion in open addressing
13	MSQ	2	(A), (D)	Hashing — reconstructing insertion order
14	NAT	2	2.05 to 2.07	Hashing — expected empty slots
15	MCQ	2	(C)	Python — <code>__hash__</code> and <code>__eq__</code> contract
16	MSQ	2	(A), (C), (D)	Merge sort — comparison counts
17	MCQ	2	(D)	Quicksort — Lomuto partition
18	NAT	2	36	Shortest paths — Dijkstra

Q	Type	Marks	Key	Topic
19	MSQ	2	(B), (C), (D)	Binary heaps — insertion and delete-max
20	NAT	2	7	Hashing — rehashing with linear probing

Detailed Solutions — Mock Test 08

Q.1 [NAT · 1 mark · Hashing — separate chaining]

Answer: 18

With chaining, a successful search for the key at position p (1-based) in its chain costs p comparisons, so the total cost of searching every key once is $\sum$ over chains of $1 + 2 + \dots + L = L(L+1)/2$.

Home slots ($k \bmod 7$):

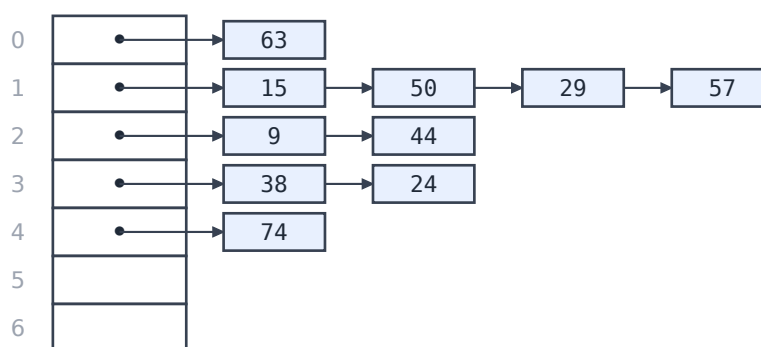
- $38 \rightarrow 3$, $15 \rightarrow 1$, $74 \rightarrow 4$, $24 \rightarrow 3$, $63 \rightarrow 0$, $50 \rightarrow 1$, $29 \rightarrow 1$, $9 \rightarrow 2$, $57 \rightarrow 1$, $44 \rightarrow 2$.

Chains (front $\rightarrow$ back):

- slot 0: 63 ($L = 1$) $\rightarrow$ cost 1
- slot 1: 15, 50, 29, 57 ($L = 4$) $\rightarrow$ cost $1+2+3+4 = 10$
- slot 2: 9, 44 ($L = 2$) $\rightarrow$ cost 3
- slot 3: 38, 24 ($L = 2$) $\rightarrow$ cost 3
- slot 4: 74 ($L = 1$) $\rightarrow$ cost 1
- slots 5, 6 empty.

Total = $1 + 10 + 3 + 3 + 1 = 18$ (average 1.8 comparisons per successful search).

Trap: counting only the chain lengths (10) or the longest chain (4). The cost of a successful search depends on the key's *position* inside the chain, so a long chain contributes quadratically, not linearly.



Final chained table (tail insertion)

Q.2 [MCQ · 1 mark · Python — dictionary keys and hashing]

Answer: (B)

A Python dict treats two keys as the *same* key when they have equal hashes **and** compare equal with `==`. Here `1 == True == 1.0` and `hash(1) == hash(True) == hash(1.0) == 1`, so all three literals denote one dictionary entry.

Building the literal left to right:

- `1: 'a'` creates the entry with key object `1`.
- `True: 'b'` finds an equal existing key $\rightarrow$ only the **value** is replaced; the stored key object stays `1`.
- `1.0: 'c'` again overwrites the value $\rightarrow$ `'c'`.
- `2: 'd'` is a new key.

Result: `{1: 'c', 2: 'd'}` $\rightarrow$ option **(B)**.

- (A) and (D) assume the *latest* key object replaces the original key — it does not; only the value is updated.
- (C) assumes `int`, `bool` and `float` keys are distinct — they are not, because hash tables decide identity by hash + equality, not by type.

Tip: `bool` is a subclass of `int`; `True` behaves as `1` in every hash-based container (`{1, True, 1.0}` has length 1).

Q.3 [MCQ · 1 mark · Stacks — postfix evaluation]

Answer: (D)

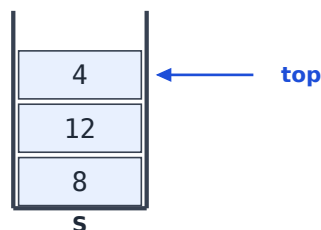
Scan left to right, pushing operands and applying each operator to the top two entries:

- $8 \rightarrow [8]$; $6 \rightarrow [8, 6]$; $2 \rightarrow [8, 6, 2]$ (size 3)
- $- \rightarrow 6 - 2 = 4 \rightarrow [8, 4]$
- $3 \rightarrow [8, 4, 3]$ (size 3)
- $* \rightarrow 4 \times 3 = 12 \rightarrow [8, 12]$
- $4 \rightarrow [8, 12, 4]$ (size 3)
- $/ \rightarrow 12 / 4 = 3 \rightarrow [8, 3]$
- $+ \rightarrow 8 + 3 = 11 \rightarrow [11]$
- $5 \rightarrow [11, 5]$; $- \rightarrow 11 - 5 = 6 \rightarrow [6]$

Value **6**, maximum stack size **3** $\rightarrow$ option **(D)**.

- (A) pops operands in the wrong order for the last operator ($5 - 11 + \dots$).
- (B) would need four operands pending at once, which never happens.
- (C) stops before the final $5 -$.

Trap: for non-commutative operators the *second* popped value is the left operand.



Stack just before $/$ is applied (one of the size-3 peaks)

Q.4 [NAT · 1 mark · Hashing — quadratic probing]

Answer: 7

Quadratic probing tries offsets 0, 1, 4, 9, 16, ... from the home slot $k \bmod 11$, wrapping modulo 11.

- 35: home $35 \bmod 11 = 2 \rightarrow$ slot **2**.
- 20: home $9 \rightarrow$ slot **9**.
- 57: home 2 (full) $\rightarrow i=1$: 3 $\rightarrow$ slot **3**.
- 9: home 9 (full) $\rightarrow i=1$: 10 $\rightarrow$ slot **10**.
- 46: home 2 (full) $\rightarrow i=1$: 3 (full) $\rightarrow i=2$: $2+4 = 6 \rightarrow$ slot **6**.
- 31: home 9 (full) $\rightarrow i=1$: 10 (full) $\rightarrow i=2$: $13 \bmod 11 = 2$ (full) $\rightarrow i=3$: $18 \bmod 11 = 7$ (free).

Key 31 is stored in slot **7** after three collisions.

Trap: using i instead of i^2 (linear probing would give $9 \rightarrow 10 \rightarrow 0$, i.e. slot 0), or forgetting to wrap 13 to 2 and finding it occupied by 35.

0	
1	
2	35
3	57
4	
5	
6	46
7	31
8	
9	20
10	9

Table after all six insertions

Q.5 [MCQ · 1 mark · Queues — circular array]**Answer: (C)**

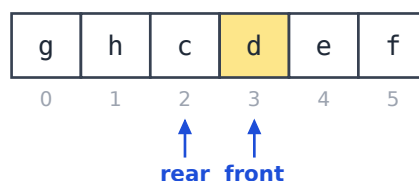
Track both indices modulo 6 (capacity is 5 because one slot is kept empty to distinguish full from empty):

- enqueue a, b, c, d → slots 0-3, rear = 4, front = 0.
- dequeue, dequeue (a, b) → front = 2.
- enqueue e (slot 4), f (slot 5), g (slot 0) → rear = 7 mod 6 = 1. Now (rear+1) mod 6 = 2 = front → the queue is **full** (c, d, e, f, g).
- dequeue (c) → front = 3.
- enqueue h at slot 1 → rear = 2.

Final: front = 3, rear = 2, Q[3] = d → option **(C)**.

- (A) has the right indices but mis-reads the front element (e is at index 4).
- (B) is the state before the last dequeue/enqueue pair.
- (D) dequeues once too often.

Tip: with this convention the queue length is $(\text{rear} - \text{front}) \bmod 6 = (2 - 3) \bmod 6 = 5$, i.e. the queue is full again.



Array Q at the end (slot 2 still holds the dequeued c)

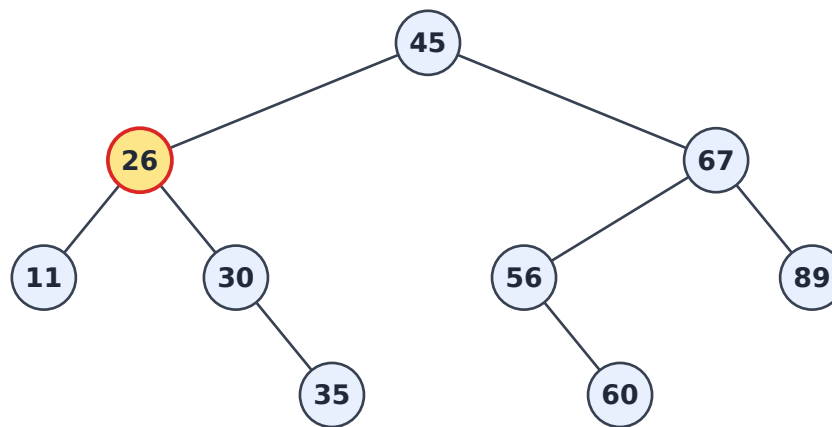
Q.6 [MSQ · 1 mark · Binary search trees]**Answer: (C), (D)**

In a BST the in-order successor of a node with no right child is the nearest ancestor of which the node lies in the *left* subtree.

- (A) Leaves are 11, 26, 35, 60, 89 → 5 leaves. **False.**
- (B) Longest paths: 45→22→30→26, 45→22→30→35, 45→67→56→60 — each has 3 edges. Height = 3, not 4. **False.**
- (C) 22 has two children; its successor is the minimum of its right subtree (rooted at 30) = 26. 26 replaces 22, so 26 becomes the left child of 45. **True.**

- (D) 35 has no right child. Walking up: 35 is the right child of 30, 30 is the right child of 22, 22 is the **left** child of 45 → successor = 45. **True.**

Trap: in (A) it is easy to miss 60, which hangs as the *right* child of 56 and makes 56 an internal node.



T after deleting 22 (successor 26 moved up)

Q.7 [MCQ · 1 mark · Binary search — iteration count]

Answer: (A)

c counts loop iterations (one middle-element probe each).

Search for 50 (absent), $n = 12$:

- $lo=0, hi=11 \rightarrow mid=5, a[5]=31 < 50 \rightarrow lo=6$ ($c=1$)
- $lo=6, hi=11 \rightarrow mid=8, a[8]=47 < 50 \rightarrow lo=9$ ($c=2$)
- $lo=9, hi=11 \rightarrow mid=10, a[10]=60 > 50 \rightarrow hi=9$ ($c=3$)
- $lo=9, hi=9 \rightarrow mid=9, a[9]=53 > 50 \rightarrow hi=8$ ($c=4$)
- $lo > hi \rightarrow$ return 4.

Search for 31: the very first mid is $(0+11)//2 = 5$ and $a[5] = 31 \rightarrow$ return 1.

Output **4 1** → option **(A)**.

- (B) stops one probe early (forgets the $lo = hi$ iteration).
- (C) confuses the count with the index of 31.
- (D) adds an extra probe; $\lfloor \log_2 12 \rfloor + 1 = 4$ is the maximum possible here.

Tip: an unsuccessful binary search on n elements uses $\lfloor \log_2 n \rfloor$ or $\lfloor \log_2 n \rfloor + 1$ iterations.

Q.8 [NAT · 1 mark · Insertion sort — shifts]

Answer: 17

Each shift moves one larger element past the key being inserted, so it removes exactly one inversion. Total shifts = number of inversions (pairs $i < j$ with $a[i] > a[j]$).

Count, for each element, the larger elements to its left:

- 31: 0 · 12: 1 (31) · 47: 0 · 5: 3 (31, 12, 47)
- 28: 2 (31, 47) · 19: 3 (31, 47, 28) · 40: 1 (47)
- 3: 7 (all earlier elements)

Total = $0+1+0+3+2+3+1+7 = 17$.

Pass-by-pass the shift counts are 1, 0, 3, 2, 3, 1, 7 (see table).

Trap: counting *comparisons* instead of shifts — comparisons also include the final failing test of each pass that does not stop at index 0.

Array after inserting each key (shifts in last column)

	0	1	2	3	4	5	6	7	shifts
start	31	12	47	5	28	19	40	3	-
12	12	31	47	5	28	19	40	3	1
47	12	31	47	5	28	19	40	3	0
5	5	12	31	47	28	19	40	3	3
28	5	12	28	31	47	19	40	3	2
19	5	12	19	28	31	47	40	3	3
40	5	12	19	28	31	40	47	3	1
3	3	5	12	19	28	31	40	47	7

Q.9 [MCQ · 1 mark · Linked lists in Python]**Answer: (B)**

`Node(v, head)` prepends, so the list is built in reverse: $23 \rightarrow 16 \rightarrow 15 \rightarrow 8 \rightarrow 4$.

The second loop unlinks the node after `p` and then advances `p` to the new next node:

- `p = 23`: `23.nxt = 15` (16 removed); `p = 15`.
- `p = 15`: `15.nxt = 4` (8 removed); `p = 4`.
- `p = 4`: `p.nxt` is `None` → stop.

Final list $23 \rightarrow 15 \rightarrow 4$, printed as `[23, 15, 4]` → option **(B)**.

- (A) forgets that insertion at the head reverses the order.
- (C) keeps the nodes that were removed (16, 8) instead of the ones kept.
- (D) assumes the head itself is deleted.

Trap: after `p.nxt = p.nxt.nxt`, the statement `p = p.nxt` jumps to the node that was two positions ahead — the loop deletes every second node.



List after the deletion loop

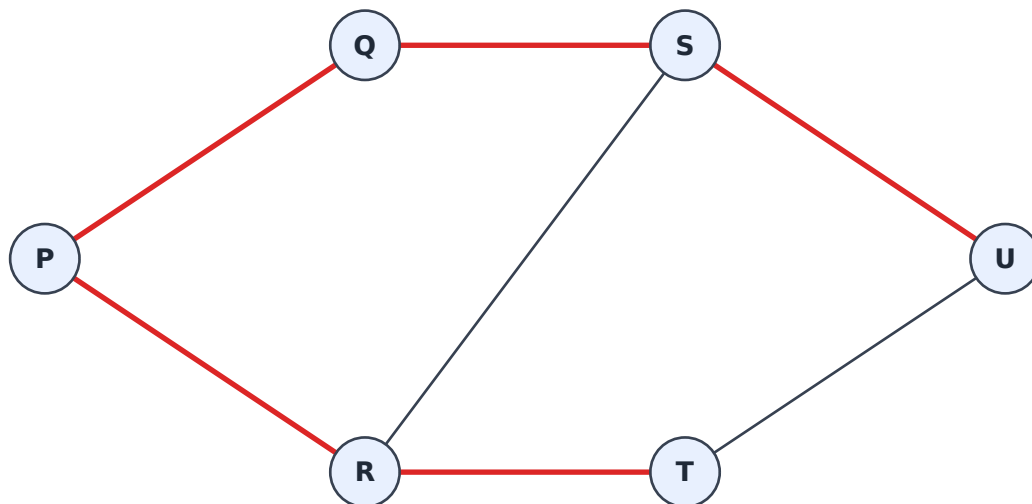
Q.10 [MSQ · 1 mark · Graph traversal — BFS orders]**Answer: (B), (D)**

BFS visits vertices level by level, and *within* a level the order is fixed by the order in which their parents were dequeued.

Levels from P: $L1 = \{Q, R\}$, $L2 = \{S, T\}$, $L3 = \{U\}$.

- (A) S is at level 2 but appears before Q (level 1). **Impossible.**
- (B) R first: R enqueues T then S (any order allowed); Q adds nothing new. Order P R Q T S U. **Possible.**
- (C) Q is dequeued before R, so S (discovered by Q) must precede T (discovered only by R). T before S is **impossible.**
- (D) Q first: Q enqueues S; then R enqueues T (S already seen). Order P Q R S T U. **Possible.**

Trap: checking only the level sets. A sequence can respect levels yet violate the FIFO parent order, as in (C).



BFS tree for order (A)

Q.11 [NAT · 2 marks · Hashing — double hashing]**Answer: 12**

In double hashing the probe step $h_2(k)$ depends on the key, so keys that share a home slot follow different probe sequences (no secondary clustering). h_2 is never 0 here (values 1–7), and 13 is prime, so every probe sequence visits all slots.

Trace ($h_1, h_2 \rightarrow$ probes):

- 41: $h_1 = 2 \rightarrow$ slot **2**.
- 58: $h_1 = 6 \rightarrow$ slot **6**.
- 22: $h_1 = 9 \rightarrow$ slot **9**.
- 70: $h_1 = 5 \rightarrow$ slot **5**.
- 35: $h_1 = 9$ (full), $h_2 = 7 - 0 = 7 \rightarrow 16 \bmod 13 = \mathbf{3}$.
- 96: $h_1 = 5$ (full), $h_2 = 7 - 5 = 2 \rightarrow \mathbf{7}$.
- 19: $h_1 = 6$ (full), $h_2 = 7 - 5 = 2 \rightarrow \mathbf{8}$.
- 48: $h_1 = 9$ (full), $h_2 = 7 - 6 = 1 \rightarrow \mathbf{10}$.
- 32: $h_1 = 6$ (full: 58), $h_2 = 7 - 4 = 3 \rightarrow 9$ (full: 22) $\rightarrow 12 \rightarrow$ **free**.

Key 32 lands in slot **12** after 3 probes.

Trap: using $k \bmod 7$ itself as the step (that gives step 4 for $32 \rightarrow 6, 10$ (full), $14 \bmod 13 = 1$), or forgetting that slot 9 is already occupied by 22.

0	
1	
2	41
3	35
4	
5	70
6	58
7	96
8	19
9	22
10	48
11	
12	32

Final table after nine insertions

Q.12 [MCQ · 2 marks · Hashing — deletion in open addressing]**Answer: (A)**

Open addressing relies on probe chains being *unbroken*: a search for k stops at the first empty slot because an insertion of k would have stopped there too.

Insertions ($\text{home} = k \bmod 10$):

- $52 \rightarrow 2$; $72 \rightarrow 2$ full $\rightarrow 3$; $13 \rightarrow 3$ full $\rightarrow 4$;
- $92 \rightarrow 2, 3, 4$ full $\rightarrow 5$; $25 \rightarrow 5$ full $\rightarrow 6$;
- $33 \rightarrow 3, 4, 5, 6$ full $\rightarrow 7$.

Delete 72 (slot 3).

(i) Tombstone: search 33 probes 3 (DELETED, keep going), 4 (13), 5 (92), 6 (25), 7 (33 ✓) $\rightarrow$ found after **5** probes.

(ii) Naive: slot 3 is now empty, so the search stops at its first probe and wrongly reports 33 as **absent** after **1** probe.

Option **(A)**.

- (B) and (C) assume slot 3 can be skipped for free or that the naive policy still works — the naive policy breaks the chain.
- (D) treats a tombstone as empty, which is exactly what tombstones prevent.

Tip: a later insert of, say, 43 may *reuse* the tombstone at slot 3 (after verifying 43 is not already present), so lazy deletion does not waste space forever.

0	
1	
2	52
3	DEL
4	13
5	92
6	25
7	33
8	
9	

Table after lazy deletion of 72

Q.13 [MSQ · 2 marks · Hashing — reconstructing insertion order]**Answer: (A), (D)**

Derive *precedence constraints* from where each key sits relative to its home slot: a key displaced from home h to slot s needs slots $h \dots s-1$ to be occupied when it arrives.

- 42, 23, 34 sit at their homes (2, 3, 4) → each must arrive before any other key spills into its slot.
- 52 (home 2) is at 5 → 42, 23, 34 must all precede 52.
- 33 (home 3) is at 6 → 23, 34, 52 precede 33.
- 46 (home 6) is at 7 → 33 precedes 46.

So the valid orders are: {42, 23, 34} in any order, then 52, then 33, then 46 ($3! = 6$ sequences).

- (A) 23, 34, 42 | 52 | 33 | 46 — fits. **Possible.**
- (B) 52 arrives before 34, so 52 probes 2, 3 and lands in **4**; then 34 is displaced. **Not possible.**
- (C) 46 arrives before 33 and finds its home 6 empty → 46 at 6, 33 later goes to 7. **Not possible.**
- (D) 34, 42, 23 | 52 | 33 | 46 — fits. **Possible.**

Tip: simulate only the options that pass the constraint check — it saves time in the exam.

Q.14 [NAT · 2 marks · Hashing — expected empty slots]**Answer: 2.05 to 2.07**

Use *linearity of expectation* with an indicator per slot.

- Let $X_j = 1$ if slot j is empty after all insertions. A single key misses slot j with probability $1 - 1/10 = 0.9$, and the 15 keys are independent, so $P(X_j = 1) = 0.9^{15}$.
- Expected number of empty slots $E[\sum X_j] = 10 \cdot 0.9^{15}$.

Compute 0.9^{15} : $0.9^2 = 0.81$, $0.9^4 = 0.6561$, $0.9^8 = 0.43047$, $0.9^{15} = 0.9^8 \cdot 0.9^4 \cdot 0.9^2 \cdot 0.9 = 0.43047 \times 0.6561 \times 0.81 \times 0.9 \approx 0.20589$.

Expected empty slots $\approx 10 \times 0.20589 \approx \mathbf{2.06}$.

Interpretation: even with load factor $\alpha = 1.5$ about a fifth of the table is still unused, i.e. roughly $m \cdot e^{-\alpha} \approx 10 \times 0.2231 = 2.23$ for large m .

Trap: computing $m - n$ (negative here) or assuming keys fill distinct slots until the table is full. Another common slip is using $(1/10)^{15}$ (probability that *all* keys hit one slot) instead of 0.9^{15} .

Q.15 [MCQ · 2 marks · Python — `__hash__` and `__eq__` contract]**Answer: (C)**

A set (a hash table) finds a candidate entry only through its **hash**; `__eq__` is consulted only among entries whose stored hash equals the probe hash. The rule $a == b \Rightarrow \text{hash}(a) == \text{hash}(b)$ is violated here, so 'equal' objects can coexist.

Insertions (`hash = hash(y)`):

- P(1,2): hash 2, set empty → added (size 1).
- P(1,3): hash 3 — no stored entry has hash 3 → `__eq__` never called → added (size 2), even though its x equals P(1,2).x.
- P(2,2): hash 2 matches P(1,2); `__eq__` compares x: $2 \neq 1 \rightarrow$ added (size 3).
- P(1,2): hash 2; compared with P(1,2) → x equal → duplicate, not added.

P(1, 99) in s: hash 99 matches no stored hash → **False**, although `P(1, 99) == P(1, 2)` is **True**.

Output 3 **False** → option **(C)**.

- (A) uses only `__eq__` (x values 1, 2).
- (B) ignores `__eq__` entirely (the final P(1,2) is rejected).
- (D) assumes membership falls back to `==` over all elements.

Tip: always hash exactly the fields used by `__eq__`.

Q.16 [MSQ · 2 marks · Merge sort — comparison counts]**Answer: (A), (C), (D)**

Merging lists of sizes p and q costs between $\min(p, q)$ (one list runs out early) and $p + q - 1$ (they interleave to the very end) comparisons.

(D) [5, 9, 2, 7, 1, 8, 3, 6]:

- Size-1 merges: (5|9), (2|7), (1|8), (3|6) → 1 each = 4.
- Size-2 merges: [5,9]+[2,7] → 2,5,7,9 needs 3; [1,8]+[3,6] → 1,3,6,8 needs 3 → 6.
- Final: [2,5,7,9]+[1,3,6,8] fully interleaves → 7.

Total $4 + 6 + 7 = 17$. **True.**

(A) Sorted input: every merge stops as soon as the left half is exhausted → $4 \cdot 1 + 2 \cdot 2 + 1 \cdot 4 = 12$. **True.**

(C) The worst case equals $C(8)$ with $C(n) = 2C(n/2) + n - 1$: $C(2) = 1$, $C(4) = 5$, $C(8) = 17 = n \log_2 n - n + 1$. The array in (D) already achieves it, and nothing can exceed it. **True.**

(B) As just computed, $C(8) = 17$, not 24 ($24 = n \log_2 n$ would need n comparisons per merge level). **False.**

Trap: using $n \log_2 n$ as the exact count — each merge of total size s needs at most $s - 1$ comparisons, not s.

Comparisons per merge level for (A)

	merges	result(s)	comparisons
level 1	$4 \times (1 1)$	59 27 18 36	4
level 2	$2 \times (2 2)$	2579 1368	6
level 3	$1 \times (4 4)$	12356789	7

Q.17 [MCQ · 2 marks · Quicksort — Lomuto partition]**Answer: (D)**

Lomuto keeps the invariant $a[\text{lo}..i] \leq \text{pivot} < a[i+1..j-1]$. Pivot = 36.

- $j=0$: $42 > 36 \rightarrow$ no change.
- $j=1$: $17 \leq 36 \rightarrow i=0$, swap $a[0]$, $a[1] \rightarrow [17, 42, 63, 8, 55, 29, 71, 36]$.
- $j=2$: $63 > 36$.
- $j=3$: $8 \leq 36 \rightarrow i=1$, swap $a[1]$, $a[3] \rightarrow [17, 8, 63, 42, 55, 29, 71, 36]$.
- $j=4$: $55 > 36$.
- $j=5$: $29 \leq 36 \rightarrow i=2$, swap $a[2]$, $a[5] \rightarrow [17, 8, 29, 42, 55, 63, 71, 36]$.
- $j=6$: $71 > 36$.
- Final: swap $a[3]$, $a[7] \rightarrow [17, 8, 29, \mathbf{36}, 55, 63, 71, 42]$.

Option **(D)**; the pivot 36 is now at its final index 3.

- (A) mis-tracks the swap at $j = 5$ (63 moves to index 5, not 4).
- (B) sorts the right part, which partition does not do.
- (C) sorts the left part (17 and 8 keep their swapped order).

Trap: the final swap sends the element at $i+1$ (here 42) to the *end*, so the right part is not in original order.

Array after each swap (pivot 36)

	0	1	2	3	4	5	6	7
start	42	17	63	8	55	29	71	36
$j=1$	17	42	63	8	55	29	71	36
$j=3$	17	8	63	42	55	29	71	36
$j=5$	17	8	29	42	55	63	71	36
final	17	8	29	36	55	63	71	42

Q.18 [NAT · 2 marks · Shortest paths — Dijkstra]

Answer: 36

Dijkstra repeatedly extracts the unsettled vertex with the smallest tentative distance and relaxes its outgoing edges (all weights are non-negative, so this is correct).

- Extract A (0): $B = 7$, $C = 2$.
- Extract C (2): $B = \min(7, 2+3) = 5$; $D = 2+9 = 11$; $E = 2+6 = 8$.
- Extract B (5): $D = \min(11, 5+4) = 9$; $F = 5+10 = 15$.
- Extract E (8): $D = \min(9, 8+1) = 9$ (tie, no change); $F = \min(15, 16) = 15$.
- Extract D (9): $F = \min(15, 9+3) = 12$.
- Extract F (12).

Distances: $B = 5$, $C = 2$, $D = 9$, $E = 8$, $F = 12$.

Sum = $5 + 2 + 9 + 8 + 12 = \mathbf{36}$.

Trap: taking the direct edge $A \rightarrow B$ (7) or $B \rightarrow F$ (10) at face value. Both are improved through C and D respectively. Note the tie at D ($A \rightarrow C \rightarrow B \rightarrow D$ and $A \rightarrow C \rightarrow E \rightarrow D$ both cost 9) does not change any distance.

Tentative distances after each extraction

	A	B	C	D	E	F
A	0	7	2	∞	∞	∞
C	0	5	2	11	8	∞
B	0	5	2	9	8	15
E	0	5	2	9	8	15
D	0	5	2	9	8	12
F	0	5	2	9	8	12

Q.19 [MSQ · 2 marks · Binary heaps — insertion and delete-max]**Answer: (B), (C), (D)**

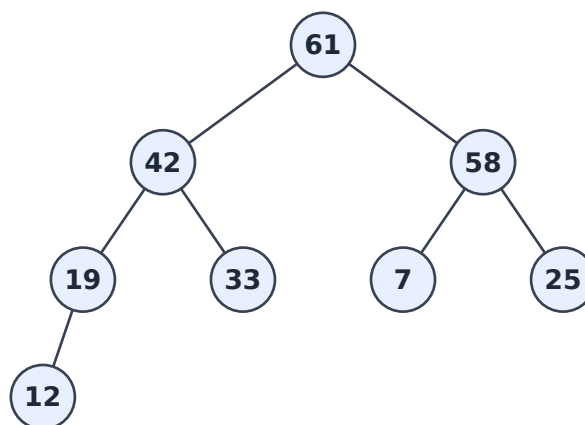
Heap insertion: place at the next free index, then swap upward while the parent is smaller.

- 19 → [19]
- 42 → swaps with 19 (1) → [42, 19]
- 7 → [42, 19, 7]
- 33 at idx 3, parent 19 → swap (2) → [42, 33, 7, 19]
- 58 at idx 4: swap with 33 (3), then with 42 (4) → [58, 42, 7, 19, 33]
- 25 at idx 5: swap with 7 (5) → [58, 42, 25, 19, 33, 7]
- 61 at idx 6: swap with 25 (6), with 58 (7) → [61, 42, 58, 19, 33, 7, 25]
- 12 at idx 7, parent 19 → no swap.

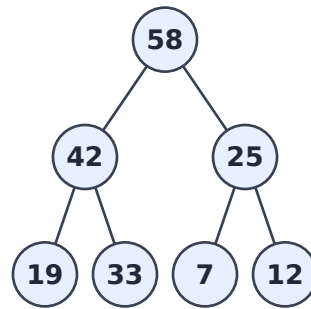
Final array: [61, 42, 58, 19, 33, 7, 25, 12].

- (A) The level order *is* the array order 61, 42, 58, ... — the statement swaps 42 and 58 and is otherwise wrong too. **False.**
- (B) Move 12 to the root: [12, 42, 58, 19, 33, 7, 25]. 12 swaps with larger child 58 → [58, 42, 12, 19, 33, 7, 25]; then with 25 → [58, 42, 25, 19, 33, 7, 12]. **True.**
- (C) 7 swaps in total. **True.**
- (D) Root 61, right child (idx 2) 58. **True.**

Trap: in sift-down always swap with the *larger* child; swapping with 42 here would break the heap.



Max-heap after the eight insertions



After one delete-max

Q.20 [NAT · 2 marks · Hashing — rehashing with linear probing]

Answer: 7

probes counts every *extra* slot inspected (collisions), including those made while re-inserting keys during **grow**. Re-insertion happens in **table order**, not original insertion order.

- $m = 4$: put 10 → slot 2. put 22: load $(1+1)/4 = 0.5$ (not > 0.5) → home 2 full → slot 3 (**probes 1**).
- put 6: $(2+1)/4 > 0.5$ → grow to $m = 8$; old = [10, 22] → 10 → 2, 22 → 6. Then 6 → home 6 full → 7 (**2**).
- put 14: $(3+1)/8 = 0.5$ → no grow; home 6 full, 7 full, → 0 (**4**). Table: [14, _, 10, _, _, _, 22, 6].
- put 30: $(4+1)/8 > 0.5$ → grow to $m = 16$; old (table order) = [14, 10, 22, 6] → 14 → 14, 10 → 10, 22 → 6, 6 → home 6 full → 7 (**5**). Then 30 → home 14 full → 15 (**6**).
- put 18: $6/16$ → home 2 → free.
- put 26: $7/16$ → home 10 full → 11 (**7**).

Printed value: **7**.

Trap: forgetting that **grow** calls **put**, so collisions during rehashing are also counted, or re-inserting keys in their original order (10, 22, 6, 14), which happens to change nothing here but generally can.

0	
1	
2	18
3	
4	
5	
6	22
7	6
8	
9	
10	10
11	26
12	
13	
14	14
15	30

Final table ($m = 16$)

Mock Test 09

Linear & Binary Search

Difficulty: Moderate

Focus areas: binary search traces, comparisons, boundaries, sentinel search

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing and find/rfind]

Consider the following Python program. What is printed?

Python

```
1 s = "BINARYSEARCH"
2 t = s[-2:3:-3]
3 u = s[:4]
4 print(t, u, s.find("AR"), s.rfind("AR"))
```

- (A) CERA BRA 3 8
- (B) CER BRA 3 8
- (C) HAS BRA 3 8
- (D) CER BYA 3 9

Q.2 [NAT · numerical answer | 1 mark | Binary search — comparison count]

The sorted array $A[0..14]$ shown below is searched with the standard iterative binary search:

$lo = 0, hi = 14, mid = (lo + hi) // 2$; if $A[mid]$ equals the key the search stops, if $A[mid] < key$ then $lo = mid + 1$, else $hi = mid - 1$; the loop runs while $lo \leq hi$. One iteration (a three-way comparison of the key with $A[mid]$) counts as **one** comparison.

The key 44 is searched first and then the key 5. The total number of comparisons made by the two searches together is _____.

A =	3	8	11	15	19	24	27	31	36	40	44	52	57	63	70
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Sorted array A (indices 0-14)

Answer: _____ (enter a numerical value)

Q.3 [NAT · numerical answer | 1 mark | Linear search — sentinel technique]

An array $A[0..n-1]$ with $n = 40$ elements is searched for a key that is **not** present.

Version 1 (ordinary): $i = 0$; while $i < n$ and $A[i] != key$: $i += 1$

Version 2 (sentinel, A has a spare cell $A[n]$): $A[n] = key$; $i = 0$; while $A[i] != key$: $i += 1$, followed by one final test $i < n$ to decide the result.

Count every evaluation of $i < n$ and every evaluation of $A[i] != key$ as one comparison. (Version 1 uses short-circuit **and**.) The number of comparisons made by Version 1 minus the number made by Version 2 is _____.

Answer: _____ (enter a numerical value)

Q.4 [MSQ · one or more correct | 1 mark | Stacks — permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack **in this order**; pops may be interleaved with pushes arbitrarily and every popped value is printed. Which of the following output sequences is/are possible?

- (A) 4 5 3 6 1 2
- (B) 1 5 4 6 2 3
- (C) 3 2 5 6 4 1
- (D) 2 4 3 6 5 1

Q.5 [MCQ · single correct | 1 mark | Queues — circular array]

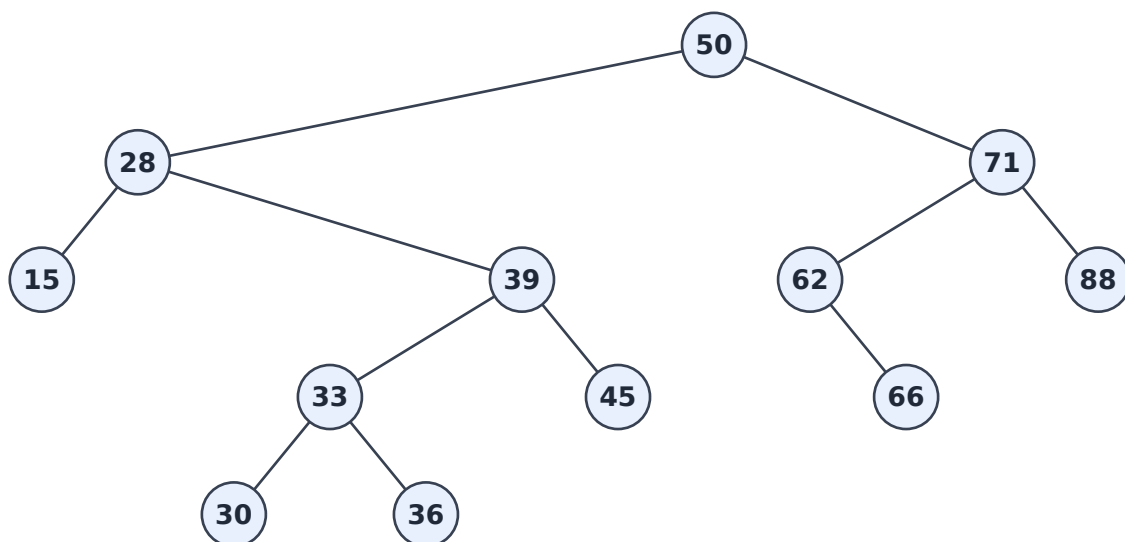
A circular queue is stored in an array $Q[0..7]$ (size 8). **front** is the index of the first element and **rear** is the index where the **next** element will be written; the queue is empty when $\text{front} == \text{rear}$ and full when $(\text{rear} + 1) \% 8 == \text{front}$ (one cell is always left unused). Initially $\text{front} = \text{rear} = 5$.

The operations performed are: 6 enqueues, then 4 dequeues, then 5 enqueues. None of them fails. Which of the following describes the final state?

- (A) $\text{front} = 1$, $\text{rear} = 7$, and the queue is full
- (B) $\text{front} = 1$, $\text{rear} = 0$, and the queue holds 6 elements
- (C) $\text{front} = 4$, $\text{rear} = 0$, and the queue is full
- (D) $\text{front} = 1$, $\text{rear} = 0$, and the queue is full

Q.6 [NAT · numerical answer | 1 mark | BST — unsuccessful search path]

The binary search tree below was built by inserting 50, 28, 71, 15, 39, 62, 88, 33, 45, 66, 30, 36 (in that order) into an empty BST. A search for the key **34** is performed. The number of nodes whose key is compared with 34 is _____.



BST built from the given insertion order

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Hashing — separate chaining]

The keys 15, 22, 8, 30, 44, 17, 50, 13, 37 are inserted in this order into a hash table with 7 slots using $h(k) = k \bmod 7$ and separate chaining (each new key is inserted at the **head** of its chain). Each of the 9 keys is equally likely to be searched for. The average number of key comparisons in a **successful** search is

- (A) 2.00
- (B) 1.80
- (C) 2.25
- (D) 1.50

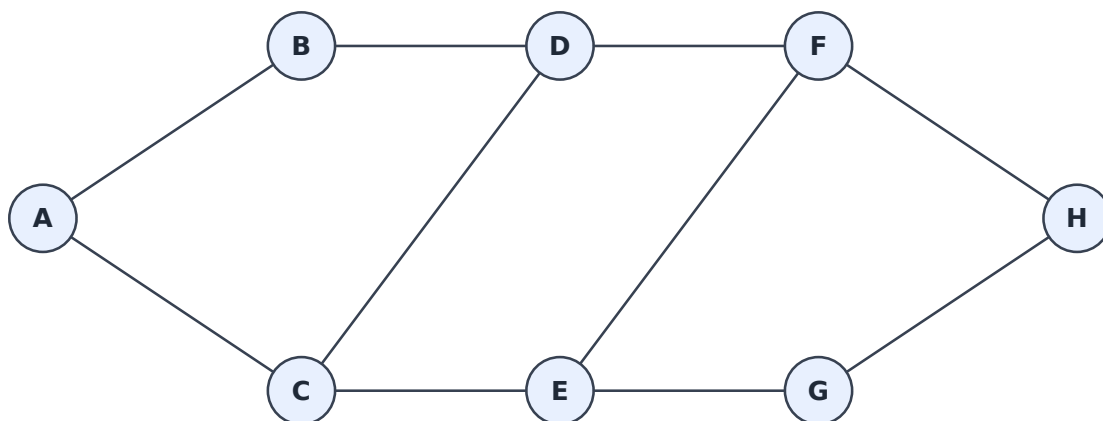
Q.8 [MCQ · single correct | 1 mark | Insertion sort with binary search]

Binary insertion sort is insertion sort in which the position of $A[i]$ inside the sorted prefix $A[0..i-1]$ is found by binary search, after which the elements to its right in the prefix are shifted one place. For an input of n distinct keys in **strictly decreasing** order, the number of key comparisons and the number of element shifts are, respectively,

- (A) $\Theta(n \log n)$ and $\Theta(n \log n)$
- (B) $\Theta(n \log n)$ and $\Theta(n^2)$
- (C) $\Theta(n^2)$ and $\Theta(n^2)$
- (D) $\Theta(n)$ and $\Theta(n^2)$

Q.9 [MSQ · one or more correct | 1 mark | Graph traversal — BFS levels]

Breadth-first search is run on the undirected graph below starting at **A**; whenever a vertex is processed its unvisited neighbours are enqueued in **alphabetical order**. Which of the following statements is/are TRUE?



- (A) Every non-tree edge joins two vertices whose BFS levels differ by at most 1
- (B) The BFS visiting order is A, B, C, D, E, F, G, H
- (C) Exactly three vertices are at BFS distance 2 from A
- (D) Edge E-F is not an edge of the BFS tree

Q.10 [MCQ · single correct | 1 mark | Python — bisect module boundaries]

Consider the following Python program. What is printed?

Python

```
1 from bisect import bisect_left, bisect_right
2 a = [2, 4, 4, 7, 7, 7, 9, 12]
3 print(bisect_left(a, 7), bisect_right(a, 7),
4       bisect_left(a, 8), bisect_right(a, 1))
```

- (A) 4 6 7 0
- (B) 3 5 6 0
- (C) 3 6 6 0
- (D) 3 6 7 1

Q.11 [NAT · numerical answer | 2 marks | Binary search — decision-tree depth]

A sorted array of $n = 100$ distinct keys $A[0..99]$ is searched with the following function. Each execution of the loop body counts as one *iteration*.

Considering only searches for keys that **are present** in the array, the number of keys for which the search needs exactly 7 iterations is _____.

Python

```
1 def search(A, x):
2     lo, hi = 0, len(A) - 1
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         if A[mid] == x:
6             return mid
7         if A[mid] < x:
8             lo = mid + 1
9         else:
10            hi = mid - 1
11    return -1
```

Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | Binary search — lower-bound boundaries]

Consider the following Python function, where **A** is a list sorted in non-decreasing order. Which of the following statements is/are TRUE?

Python

```
1 def find(A, x):
2     lo, hi = 0, len(A) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if A[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
```

- (A) `find([], 5)` raises an `IndexError`
- (B) If **A** is non-empty and $x > \max(A)$, `find` returns `len(A) - 1`
- (C) If the line `mid = (lo + hi) // 2` is changed to `mid = (lo + hi + 1) // 2`, the call `find([10, 20], 15)` never terminates
- (D) If **A** is non-empty and $x \leq \max(A)$, `find` returns the index of the first element of **A** that is $\geq x$

Q.13 [NAT · numerical answer | 2 marks | Exponential (galloping) search]

The function below performs *exponential search* on the sorted array $A[0..19]$ shown in the figure, for the key **47**. Count one *examination* each time an array element is compared with the key: the test $A[0] == \text{key}$, each evaluation of $A[b] < \text{key}$ in the doubling loop, and **one** per iteration of the binary-search loop (regardless of how many relational operators that iteration evaluates). The total number of examinations is _____.

Python

```

1 def exp_search(A, key):
2     if A[0] == key:
3         return 0
4     b = 1
5     while b < len(A) and A[b] < key:
6         b *= 2
7     lo, hi = b // 2, min(b, len(A) - 1)
8     while lo <= hi:
9         mid = (lo + hi) // 2
10        if A[mid] == key:
11            return mid
12        elif A[mid] < key:
13            lo = mid + 1
14        else:
15            hi = mid - 1
16    return -1

```

A =	2	5	9	12	16	21	23	28	31	35	38	42	44	47	53	58	60	66	71	75
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Sorted array $A[0..19]$

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Python — generator-based search and exhaustion]

Consider the following Python program. What is printed?

Python

```

1 data = [4, 9, 2, 9, 7, 9]
2 pos = (i for i, v in enumerate(data) if v == 9)
3 first = next(pos)
4 rest = list(pos)
5 again = list(pos)
6 print(first, rest, again, sum(1 for _ in pos))

```

- (A) 1 [3, 5] [3, 5] 2
 (B) 1 [1, 3, 5] [] 0
 (C) 1 [3, 5] [] 3
 (D) 1 [3, 5] [] 0

Q.15 [NAT · numerical answer | 2 marks | Merge sort — counting comparisons]

Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ and the remaining elements, sort both recursively, then merge) is applied to

[38, 12, 55, 7, 26, 49, 3, 61].

During a merge, one comparison is made each time the front elements of the two runs are compared; when one run becomes empty the rest of the other is copied without comparisons. The total number of element comparisons performed by the whole sort is _____.

Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Quicksort — Lomuto partition]

The Lomuto partition below is applied once to

$A = [23, 41, 8, 56, 17, 35, 12, 29]$ ($lo = 0, hi = 7$).

Which of the following statements is/are TRUE? (A call `swap(i, j)` with $i == j$ counts as a swap.)

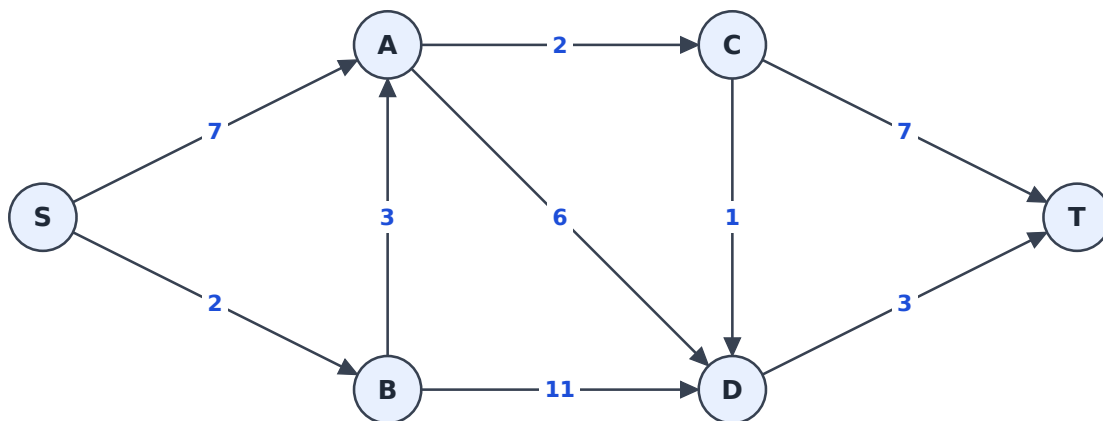
Python

```
1 def partition(A, lo, hi):
2     pivot = A[hi]
3     i = lo - 1
4     for j in range(lo, hi):
5         if A[j] <= pivot:
6             i += 1
7             A[i], A[j] = A[j], A[i]      # swap(i, j)
8     A[i + 1], A[hi] = A[hi], A[i + 1]  # swap(i+1, hi)
9     return i + 1
```

- (A) The function returns 4
- (B) The right sub-array passed to the next recursive call is $[35, 56, 41]$
- (C) Exactly 4 swaps are performed (including the final one)
- (D) After the call, $A = [23, 8, 17, 12, 29, 35, 56, 41]$

Q.17 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra]

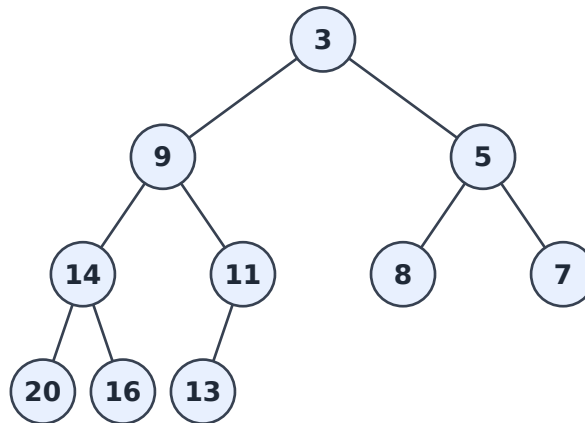
Dijkstra's algorithm is run from source **S** on the weighted directed graph below. Let $d(v)$ be the final shortest-path distance of vertex v . The value of $d(A) + d(B) + d(C) + d(D) + d(T)$ is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Binary heaps — repeated delete-min]

The array below represents a binary **min-heap** (index 0 is the root; children of i are $2i+1$ and $2i+2$). Two **deleteMin** operations are performed. Each **deleteMin** moves the last element to the root and sifts it down, always swapping with the **smaller** child while that child is smaller than it. The resulting array is



Min-heap [3, 9, 5, 14, 11, 8, 7, 20, 16, 13]

- (A) [7, 9, 8, 14, 11, 16, 13, 20]
- (B) [7, 9, 8, 14, 11, 13, 16, 20]
- (C) [7, 9, 13, 14, 11, 8, 16, 20]
- (D) [7, 8, 9, 14, 11, 16, 13, 20]

Q.19 [MSQ · one or more correct | 2 marks | Binary search — rotated sorted array]

A *rotated sorted list* is a sorted list that has been cut at some point and the two pieces swapped (e.g. [5, 8, 1, 3]). Consider the following function. Which of the following statements is/are TRUE?

Python

```

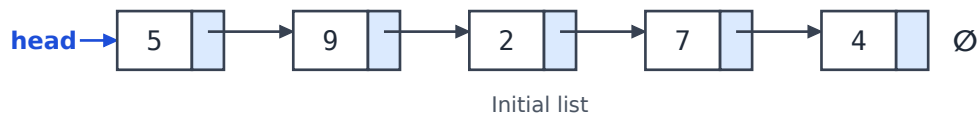
1 def find_min(A):
2     lo, hi = 0, len(A) - 1
3     steps = 0
4     while lo < hi:
5         steps += 1
6         mid = (lo + hi) // 2
7         if A[mid] > A[hi]:
8             lo = mid + 1
9         else:
10            hi = mid
11    return lo, steps

```

- (A) For every rotation of a sorted list of distinct values, the returned index holds the minimum
- (B) `find_min([2, 2, 2, 0, 2])` returns the index of the value 0
- (C) If `A[mid] > A[hi]` is replaced by `A[mid] >= A[hi]`, the function still returns the index of the minimum for every rotated list of **distinct** values
- (D) `find_min([41, 47, 52, 60, 3, 9, 15, 22, 30, 36])` returns (4, 3)

Q.20 [MCQ · single correct | 2 marks | Linked lists — self-organising linear search]

A singly linked list initially holds $5 \rightarrow 9 \rightarrow 2 \rightarrow 7 \rightarrow 4$ (head first). It is searched with the **move-to-front** heuristic: a search scans from the head, comparing the key with each node, and the found node is then unlinked and re-inserted at the head. The keys 7, 4, 7, 7, 2 are searched in this order (all present). What are the total number of key comparisons and the final list?



- (A) 20 comparisons; $5 \rightarrow 9 \rightarrow 2 \rightarrow 7 \rightarrow 4$
 (B) 17 comparisons; $2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 9$
 (C) 17 comparisons; $2 \rightarrow 7 \rightarrow 4 \rightarrow 5 \rightarrow 9$
 (D) 16 comparisons; $2 \rightarrow 7 \rightarrow 4 \rightarrow 5 \rightarrow 9$

Answer Key — Mock Test 09

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — string slicing and find/rfind
2	NAT	1	8	Binary search — comparison count
3	NAT	1	39	Linear search — sentinel technique
4	MSQ	1	(C), (D)	Stacks — permutations
5	MCQ	1	(D)	Queues — circular array
6	NAT	1	5	BST — unsuccessful search path
7	MCQ	1	(A)	Hashing — separate chaining
8	MCQ	1	(B)	Insertion sort with binary search
9	MSQ	1	(A), (B), (D)	Graph traversal — BFS levels
10	MCQ	1	(C)	Python — bisect module boundaries
11	NAT	2	37	Binary search — decision-tree depth
12	MSQ	2	(B), (C), (D)	Binary search — lower-bound boundaries
13	NAT	2	9	Exponential (galloping) search
14	MCQ	2	(D)	Python — generator-based search and exhaustion
15	NAT	2	17	Merge sort — counting comparisons
16	MSQ	2	(A), (B), (D)	Quicksort — Lomuto partition
17	NAT	2	33	Shortest paths — Dijkstra
18	MCQ	2	(A)	Binary heaps — repeated delete-min
19	MSQ	2	(A), (C), (D)	Binary search — rotated sorted array
20	MCQ	2	(C)	Linked lists — self-organising linear search

Detailed Solutions — Mock Test 09

Q.1 [MCQ · 1 mark · Python — string slicing and find/rfind]

Answer: (B)

Index the string once and everything follows: B0 I1 N2 A3 R4 Y5 S6 E7 A8 R9 C10 H11.

- `s[-2:3:-3]`: start at index $-2 = 10$ ('C'), step -3 , stop *before* index 3. Indices visited: 10, 7, 4 → 'C', 'E', 'R'. The next index would be 1, which is past the stop (3) when moving left, so `t = 'CER'`.
- `s[::-4]`: indices 0, 4, 8 → 'B', 'R', 'A' → 'BRA'.
- `find` is a left-to-right linear search for the substring: first 'AR' starts at 3.
- `rfind` searches from the right: the last 'AR' starts at 8 (A8 R9).

Output: CER BRA 3 8 → option (B).

- (A) wrongly includes index 3 — the stop index is never included.
- (C) starts at -1 ('H') instead of -2 .
- (D) reads index 5 instead of 4 and gives the index of 'R' instead of where 'AR' starts.

Trap: with a negative step the slice runs right-to-left and stops *before* the stop index; `find/rfind` return the start index of the match.

Q.2 [NAT · 1 mark · Binary search — comparison count]

Answer: 8

Binary search halves the live range `[lo, hi]` each iteration; we simply trace both searches.

Key 44

- `lo=0, hi=14 → mid=7, A[7]=31 < 44 → lo=8.`
- `lo=8, hi=14 → mid=11, A[11]=52 > 44 → hi=10.`
- `lo=8, hi=10 → mid=9, A[9]=40 < 44 → lo=10.`
- `lo=10, hi=10 → mid=10, A[10]=44 → found. 4 comparisons.`

Key 5 (absent)

- `mid=7 (31) > 5 → hi=6; mid=3 (15) > 5 → hi=2; mid=1 (8) > 5 → hi=0; mid=0 (3) < 5 → lo=1 > hi → stop. 4 comparisons.`

Total = 4 + 4 = **8**.

With $n = 15 = 2^4 - 1$ the decision tree is perfect with 4 levels, so *every* search (successful at the bottom level or unsuccessful) takes at most 4 comparisons.

Trap: do not count the final failed `lo ≤ hi` test as a key comparison — it compares indices, not the key.

Q.3 [NAT · 1 mark · Linear search — sentinel technique]

Answer: 39

A sentinel guarantees that the scan stops, so the bound test `i < n` can be removed from the inner loop — it is done once at the end.

Version 1 (key absent):

- For $i = 0 \dots 39$ the test `i < n` is true and `A[i] != key` is evaluated: $40 + 40 = 80$.
- At $i = 40$, `i < n` is false; short-circuit skips `A[40] != key`: +1.
- Total = $2n + 1 = 81$.

Version 2:

- `A[i] != key` is true for $i = 0 \dots 39$ (40 evaluations) and false at $i = 40$ (the sentinel): 41 evaluations.

- One final $i < n$ ($40 < 40$ is false $\rightarrow$ not found): +1.
- Total = $n + 2 = 42$.

Difference = $81 - 42 = 39$ (in general $n - 1$).

Trap: forgetting that the sentinel cell itself is compared ($n + 1$ element tests), or forgetting the single bound test after the loop. Asymptotically both are $\Theta(n)$; the sentinel only halves the constant.

Q.4 [MSQ · 1 mark · Stacks — permutations]

Answer: (C), (D)

Greedy check: to output x , push everything up to x if x has not been pushed yet; if x was already pushed it must be on the **top** of the stack, otherwise the sequence is impossible.

- (A) push 1–4 pop **4**; push 5 pop **5**; pop **3**; push 6 pop **6**; now stack (bottom $\rightarrow$ top) is 1, 2 — the next output 1 is *under* 2. **Impossible.**
- (B) push 1 pop **1**; push 2–5 pop **5**, pop **4**; push 6 pop **6**; stack is 2, 3 — next output 2 is under 3. **Impossible.**
- (C) push 1,2,3 pop **3**; pop **2**; push 4,5 pop **5**; push 6 pop **6**; pop **4**; pop **1**. Possible.
- (D) push 1,2 pop **2**; push 3,4 pop **4**; pop **3**; push 5,6 pop **6**; pop **5**; pop **1**. Possible.

Answer: (C) and (D).

Tip: a sequence is a stack permutation iff it contains no pattern $i < j < k$ output in the order k, i, j . In (A), 3 ... 1 ... 2 is exactly such a forbidden 3-1-2 pattern.

Q.5 [MCQ · 1 mark · Queues — circular array]

Answer: (D)

Each enqueue does $\text{rear} = (\text{rear} + 1) \% 8$; each dequeue does $\text{front} = (\text{front} + 1) \% 8$.

- 6 enqueues: $\text{rear} = (5 + 6) \% 8 = 3$; size 6 (≤ 7 , OK).
- 4 dequeues: $\text{front} = (5 + 4) \% 8 = 1$; size 2.
- 5 enqueues: $\text{rear} = (3 + 5) \% 8 = 0$; size 7.

Size = $(\text{rear} - \text{front} + 8) \% 8 = (0 - 1 + 8) \% 8 = 7 = \text{capacity } 8 - 1$, and indeed $(\text{rear} + 1) \% 8 = 1 = \text{front}$, so the queue is **full**. Answer (D).

- (A) computes rear without the wrap-around ($(3 + 5) = 8 \rightarrow 0$, not 7).
- (B) uses the correct indices but miscounts the size (7, not 6).
- (C) forgets to apply the 4 dequeues to **front** correctly ($5 + 4 = 9 \rightarrow 1$, not 4).

Trap: with the “one empty cell” convention a size-8 array holds at most 7 items — a sixth enqueue in the last phase would have failed.

Q.6 [NAT · 1 mark · BST — unsuccessful search path]

Answer: 5

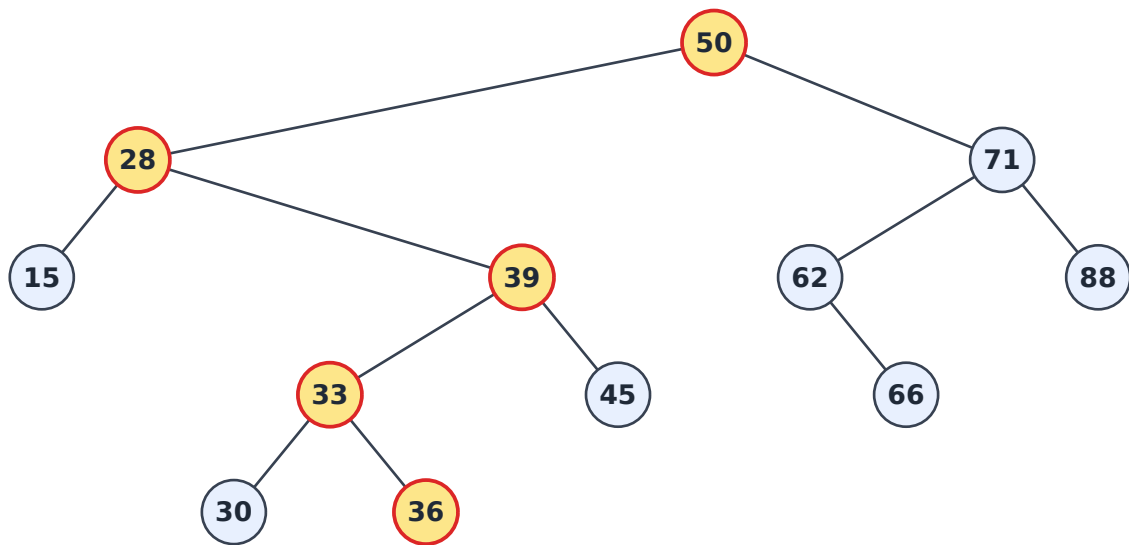
A BST search follows a single root-to-leaf path: go left if $\text{key} < \text{node}$, right if $\text{key} > \text{node}$, and stop at a match or at an empty child.

- 34 vs **50** $\rightarrow$ smaller $\rightarrow$ left.
- 34 vs **28** $\rightarrow$ larger $\rightarrow$ right.
- 34 vs **39** $\rightarrow$ smaller $\rightarrow$ left.
- 34 vs **33** $\rightarrow$ larger $\rightarrow$ right.
- 34 vs **36** $\rightarrow$ smaller $\rightarrow$ left child of 36 is empty $\rightarrow$ unsuccessful.

Nodes compared: 50, 28, 39, 33, 36 $\rightarrow$ **5**.

Note that 34 would be inserted as the left child of 36 — an unsuccessful search always ends exactly where an insertion of that key would occur.

Trap: counting the empty child as a comparison (giving 6), or stopping at 33 because $33 < 34 < 36$ 'looks like' it should be between them.



Search path for 34 (highlighted)

Q.7 [MCQ · 1 mark · Hashing — separate chaining]

Answer: (A)

In chaining, a successful search for a key at position p of its chain costs p comparisons.

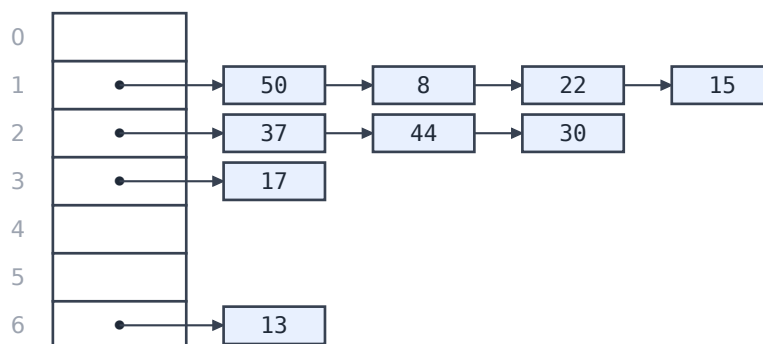
Hash values: $15 \rightarrow 1$, $22 \rightarrow 1$, $8 \rightarrow 1$, $30 \rightarrow 2$, $44 \rightarrow 2$, $17 \rightarrow 3$, $50 \rightarrow 1$, $13 \rightarrow 6$, $37 \rightarrow 2$.

- Slot 1 (head insertion): $50 \rightarrow 8 \rightarrow 22 \rightarrow 15$ — costs $1 + 2 + 3 + 4 = 10$.
- Slot 2: $37 \rightarrow 44 \rightarrow 30$ — costs $1 + 2 + 3 = 6$.
- Slot 3: 17 — cost 1. Slot 6: 13 — cost 1.

Total = $10 + 6 + 1 + 1 = 18$, average = $18 / 9 = 2.00 \rightarrow$ option (A).

- (B) 1.80 divides by 10 instead of 9.
- (C) $2.25 = 18 / 8$ divides by 8 — one key forgotten when counting.
- (D) 1.50 is an underestimate, in the spirit of the textbook estimate $1 + \alpha/2 \approx 1.64$ for *uniform* hashing; this data piles 4 keys into slot 1, so the real cost is higher.

Tip: head- vs tail-insertion changes *which* key is cheap, but not the multiset of costs $\{1, 2, \dots, \text{chain length}\}$, so the average is the same either way.



Final chained table (head insertion)

Q.8 [MCQ · 1 mark · Insertion sort with binary search]

Answer: (B)

Binary search reduces the *search* for the insertion point, but not the *movement* of data.

- Comparisons: inserting the i -th element needs about $\lceil \log_2(i + 1) \rceil$ comparisons, so the total is $\sum \log_2 i = \log_2(n!) = \Theta(n \log n)$. (This holds for any input order.)
- Shifts: in a decreasing array every new element is the smallest so far and must go to index 0, so all i prefix elements shift: $0 + 1 + \dots + (n - 1) = n(n - 1)/2 = \Theta(n^2)$.

Therefore the answer is (B).

- (A) assumes shifts also become logarithmic — impossible in an array, each shift moves one element one slot.
- (C) is plain insertion sort's comparison count.
- (D) $\Theta(n)$ comparisons is the *best* case of ordinary insertion sort (sorted input), not binary insertion sort.

Trap: binary insertion sort is still $\Theta(n^2)$ time in the worst case.

Q.9 [MSQ · 1 mark · Graph traversal — BFS levels]

Answer: (A), (B), (D)

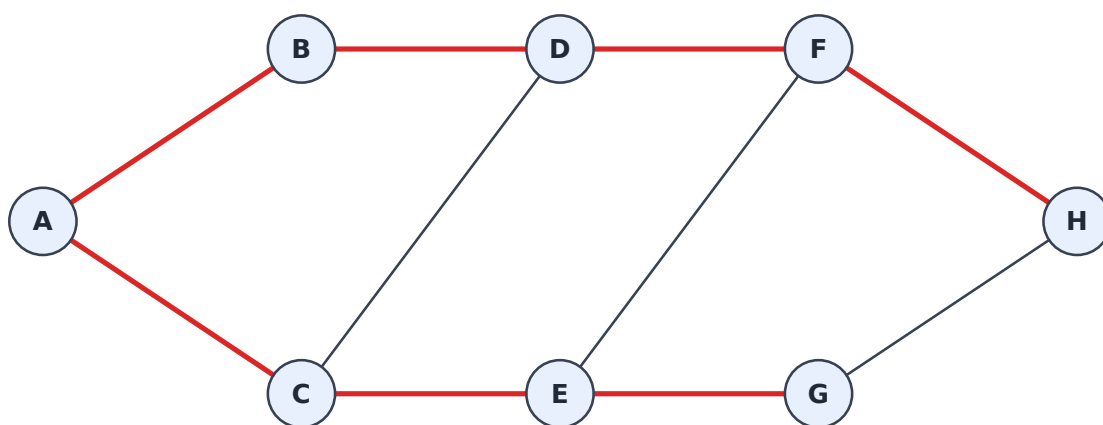
BFS processes vertices level by level using a FIFO queue.

- Dequeue A: enqueue B, C (level 1).
- Dequeue B: enqueue D (level 2, parent B).
- Dequeue C: D already seen; enqueue E (level 2, parent C).
- Dequeue D: enqueue F (level 3, parent D).
- Dequeue E: F seen; enqueue G (level 3, parent E).
- Dequeue F: enqueue H (level 4, parent F). Then G, H.

Order A B C D E F G H; levels: A0 | B1 C1 | D2 E2 | F3 G3 | H4.

- (A) **True** — in BFS of an undirected graph a non-tree edge (u, v) always has $|\text{level}(u) - \text{level}(v)| \leq 1$ (here C-D: 1/2, E-F: 2/3, G-H: 3/4).
- (B) **True** — shown above.
- (C) **False** — only D and E are at distance 2.
- (D) **True** — F was discovered from D, so E-F is a non-tree (cross) edge.

Trap: assuming F is discovered from E because E-F is drawn horizontally; D is dequeued before E, so D claims F first.



BFS tree edges highlighted

Q.10 [MCQ · 1 mark · Python — bisect module boundaries]**Answer: (C)**

`bisect_left(a, x)` returns the first index i with $a[i] \geq x$ (the *lower bound*); `bisect_right(a, x)` returns the first index i with $a[i] > x$ (the *upper bound*). Both are binary searches returning an insertion point that keeps `a` sorted.

Indices: $2_0 \ 4_1 \ 4_2 \ 7_3 \ 7_4 \ 7_5 \ 9_6 \ 12_7$.

- `bisect_left(a, 7)` → first 7 is at **3**.
- `bisect_right(a, 7)` → one past the last 7 → **6**.
- `bisect_left(a, 8)` → 8 is absent; first element ≥ 8 is 9 at **6**.
- `bisect_right(a, 1)` → every element is > 1 → **0**.

Output **3 6 6 0** → (C). Note `bisect_right(a,7) - bisect_left(a,7) = 3` = number of 7s.

- (A) is off by one on the lower bound.
- (B) returns the index of the *last* 7 (5) instead of one past it.
- (D) treats an absent key as if it were inserted after its successor.

Tip: for an absent key both functions return the same value.

Q.11 [NAT · 2 marks · Binary search — decision-tree depth]**Answer: 37**

Binary search on n keys corresponds to a binary **decision tree**: the root is the first mid (index 49), its children are the mids of the two halves, and so on. A key at depth d (root depth 0) is found after exactly $d + 1$ iterations.

Because `mid` always splits the range $[lo, hi]$ as evenly as possible (the two halves differ in size by at most 1), the decision tree is a *minimum-height* tree: every level except the last is completely full.

- Levels 0–5 are full: $1 + 2 + 4 + 8 + 16 + 32 = 63$ keys (found in 1 ... 6 iterations).
- The tree has height $\lceil \log_2 100 \rceil = 6$, so the remaining $100 - 63 = \mathbf{37}$ keys sit on level 6 and need 7 iterations.

Iteration distribution: 1 key needs 1, 2 need 2, 4 need 3, 8 need 4, 16 need 5, 32 need 6, 37 need 7 (total 100).

Consequently the average successful search costs $(1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 8 \cdot 4 + 16 \cdot 5 + 32 \cdot 6 + 37 \cdot 7) / 100 = 580 / 100 = 5.80$ iterations — less than one below the worst case, which is typical for binary search.

Trap: answering 50 ('half the keys are leaves') — that holds only for a perfect tree with $n = 2^k - 1$. Here the last level is only partially filled.

Q.12 [MSQ · 2 marks · Binary search — lower-bound boundaries]**Answer: (B), (C), (D)**

Invariant: the answer (first index with $A[i] \geq x$, if any) always lies in $[lo, hi]$. `A[mid] < x` means `mid` and everything left of it are too small → `lo = mid + 1`; otherwise `mid` itself may be the answer → `hi = mid` (not `mid - 1`). The loop stops when `lo == hi`.

- (A) **False.** `lo = 0`, `hi = -1`, the loop condition $0 < -1$ is false at once, and the function returns 0 without touching the list.
- (B) **True.** Every comparison says `A[mid] < x`, so `lo` keeps moving right until `lo = hi = len(A) - 1`. (The caller must check `A[lo] ≥ x` to detect 'not found'.)
- (C) **True.** With the upper mid: `lo=0`, `hi=1` → `mid=1`, `A[1]=20 ≥ 15` → `hi = mid = 1`. Nothing changed, so the loop repeats forever. With `hi = mid` the lower mid is *required* so that the range always shrinks.

- (D) **True.** E.g. $A = [3, 5, 5, 8]$, $x = 5$: $(lo, hi, mid) = (0, 3, 1)$ $A[1] = 5 \geq 5 \rightarrow hi = 1$; $(0, 1, 0)$ $A[0] = 3 < 5 \rightarrow lo = 1$; stop $\rightarrow 1$, the first 5.

Tip: pair $lo = mid + 1$ / $hi = mid$ with the lower mid, and $lo = mid$ / $hi = mid - 1$ with the upper mid; mixing them causes an infinite loop on two-element ranges.

Q.13 [NAT · 2 marks · Exponential (galloping) search]

Answer: 9

Exponential search first finds a range $[b/2, b]$ that must contain the key by doubling b , then binary-searches inside it. Cost is $O(\log i)$ where i is the key's position — useful for unbounded or very long arrays.

Phase 0: $A[0] == 47$? $2 \neq 47 \rightarrow 1$ examination.

Phase 1 (doubling):

- $b=1$: $A[1]=5 < 47 \rightarrow b=2$.
- $b=2$: $A[2]=9 < 47 \rightarrow b=4$.
- $b=4$: $A[4]=16 < 47 \rightarrow b=8$.
- $b=8$: $A[8]=31 < 47 \rightarrow b=16$.
- $b=16$: $A[16]=60 < 47$ is false $\rightarrow$ stop. 5 examinations.

Range: $lo = 16 // 2 = 8$, $hi = \min(16, 19) = 16$.

Phase 2 (binary search on $A[8..16]$):

- $mid=12$: $A[12]=44 < 47 \rightarrow lo=13$.
- $mid=14$: $A[14]=53 > 47 \rightarrow hi=13$.
- $mid=13$: $A[13]=47 \rightarrow$ found. 3 examinations.

Total = $1 + 5 + 3 = 9$.

Trap: forgetting the initial $A[0]$ test or the final, *failing* doubling test ($A[16] < 47$), which are both genuine examinations.

2	5	9	12	16	21	23	28	31	35	38	42	44	47	53	58	60	66	71	75
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

lo

hi

Examined cells; binary-search range $[8, 16]$

Q.14 [MCQ · 2 marks · Python — generator-based search and exhaustion]

Answer: (D)

A generator expression is a **one-shot iterator**: it produces values lazily and remembers where it stopped; once exhausted it stays exhausted.

- The generator performs a linear search yielding every index i with $data[i] == 9$: the indices are 1, 3, 5.
- `next(pos)` runs the search until the first hit $\rightarrow first = 1$. The generator is now paused just after index 1.
- `list(pos)` resumes and drains the remaining hits $\rightarrow rest = [3, 5]$. The generator is now exhausted.
- `list(pos)` again $\rightarrow$ nothing left $\rightarrow again = []$.
- `sum(1 for _ in pos)` iterates over the exhausted generator $\rightarrow 0$.

Output: `1 [3, 5] [] 0` $\rightarrow$ option (D).

- (A) treats the generator like a list that can be re-iterated.
- (B) assumes `list(pos)` restarts from the beginning after `next`.
- (C) counts the original number of matches, again assuming a restart.

Trap: lists, tuples and ranges are re-iterable; generators, `map`, `filter`, `zip` and file objects are not. Store the results in a list if you need two passes.

Q.15 [NAT · 2 marks · Merge sort — counting comparisons]

Answer: 17

Merging runs of sizes p and q costs between $\min(p, q)$ and $p + q - 1$ comparisons; the exact number is $(p + q) - (\text{number of elements left over when one run empties})$.

Level 1 (pairs): $[38|12] \rightarrow 1$, $[55|7] \rightarrow 1$, $[26|49] \rightarrow 1$, $[3|61] \rightarrow 1$. Total 4.

Runs: $[12, 38]$, $[7, 55]$, $[26, 49]$, $[3, 61]$.

Level 2:

- $[12, 38] + [7, 55]$: $12 \text{ vs } 7 \rightarrow 7$; $12 \text{ vs } 55 \rightarrow 12$; $38 \text{ vs } 55 \rightarrow 38$; left run empty, copy 55. **3** comparisons $\rightarrow [7, 12, 38, 55]$.
- $[26, 49] + [3, 61]$: $26 \text{ vs } 3 \rightarrow 3$; $26 \text{ vs } 61 \rightarrow 26$; $49 \text{ vs } 61 \rightarrow 49$; copy 61. **3** $\rightarrow [3, 26, 49, 61]$.

Level 3: $[7, 12, 38, 55] + [3, 26, 49, 61]$: $7 \text{v} 3 \rightarrow 3$, $7 \text{v} 26 \rightarrow 7$, $12 \text{v} 26 \rightarrow 12$, $38 \text{v} 26 \rightarrow 26$, $38 \text{v} 49 \rightarrow 38$, $55 \text{v} 49 \rightarrow 49$, $55 \text{v} 61 \rightarrow 55$, copy 61. **7** comparisons.

Total = $4 + 6 + 7 = 17$.

For $n = 8$ the possible range is 12 (each merge stops as early as possible) to 17 ($n \log_2 n - n + 1$); this input happens to hit the maximum, because every merge is fully interleaved — only a single element is left over to copy.

Trap: counting $n - 1$ for every merge regardless of data, or adding a comparison for the copied tail.

Array after each merge level

	0	1	2	3	4	5	6	7	comps
input	38	12	55	7	26	49	3	61	—
level 1	12	38	7	55	26	49	3	61	4
level 2	7	12	38	55	3	26	49	61	6
level 3	3	7	12	26	38	49	55	61	7

Q.16 [MSQ · 2 marks · Quicksort — Lomuto partition]

Answer: (A), (B), (D)

Lomuto keeps $A[\text{lo}..i] \leq \text{pivot}$ and $A[i+1..j-1] > \text{pivot}$; the pivot is $A[\text{hi}] = 29$.

- $j=0$: $23 \leq 29 \rightarrow i=0$, $\text{swap}(0,0) \rightarrow$ unchanged. (swap 1)
- $j=1$: $41 > 29 \rightarrow$ skip.
- $j=2$: $8 \leq 29 \rightarrow i=1$, $\text{swap}(1,2) \rightarrow [23, 8, 41, 56, 17, 35, 12, 29]$. (swap 2)
- $j=3$: $56 \rightarrow$ skip.
- $j=4$: $17 \rightarrow i=2$, $\text{swap}(2,4) \rightarrow [23, 8, 17, 56, 41, 35, 12, 29]$. (swap 3)
- $j=5$: $35 \rightarrow$ skip.
- $j=6$: $12 \rightarrow i=3$, $\text{swap}(3,6) \rightarrow [23, 8, 17, 12, 41, 35, 56, 29]$. (swap 4)
- Final $\text{swap}(4, 7) \rightarrow [23, 8, 17, 12, 29, 35, 56, 41]$. (swap 5) Return 4.

Verdicts:

- (A) **True** — the pivot lands at index $i + 1 = 4$ (four keys are ≤ 29 besides it).
- (B) **True** — quicksort next recurses on $A[0..3] = [23, 8, 17, 12]$ and $A[5..7] = [35, 56, 41]$.
- (C) **False** — there are 4 swaps inside the loop (one is the self-swap at $j = 0$) plus the final one = **5**.
- (D) **True** — exact final array above.

Trap: Lomuto does not preserve the relative order of the $>$ pivot elements (41 moved from index 1 to 7) — the algorithm is not stable.

23	8	17	12	29	35	56	41
0	1	2	3	4	5	6	7

↑
pivot

Array after partition (pivot 29 in final place)

Q.17 [NAT · 2 marks · Shortest paths — Dijkstra]

Answer: 33

Dijkstra repeatedly extracts the unvisited vertex with the smallest tentative distance and relaxes its outgoing edges; with non-negative weights an extracted distance is final.

- Extract S (0): $A = 7$, $B = 2$.
- Extract B (2): $A = \min(7, 2+3) = 5$; $D = 2+11 = 13$.
- Extract A (5): $C = 5+2 = 7$; $D = \min(13, 5+6) = 11$.
- Extract C (7): $D = \min(11, 7+1) = 8$; $T = 7+7 = 14$.
- Extract D (8): $T = \min(14, 8+3) = 11$.
- Extract T (11).

Final: $d(A)=5$, $d(B)=2$, $d(C)=7$, $d(D)=8$, $d(T)=11$. Sum = $5 + 2 + 7 + 8 + 11 = \mathbf{33}$.

Shortest-path tree: $S \rightarrow B \rightarrow A \rightarrow C \rightarrow D \rightarrow T$ is a single chain — every direct 'shortcut' edge ($S \rightarrow A$, $B \rightarrow D$, $A \rightarrow D$, $C \rightarrow T$) turns out to be longer than the detour.

Trap: finalising D at 11 via $A \rightarrow D$ (or T at 14 via $C \rightarrow T$) because those values were seen first. A tentative distance is only final when its vertex is extracted.

Tentative distances after each extraction

	S	A	B	C	D	T
init	0	∞	∞	∞	∞	∞
S	0	7	2	∞	∞	∞
B	0	5	2	∞	13	∞
A	0	5	2	7	11	∞
C	0	5	2	7	8	14
D	0	5	2	7	8	11

Q.18 [MCQ · 2 marks · Binary heaps — repeated delete-min]

Answer: (A)

deleteMin removes the root, moves the last leaf to the root and restores the heap order by sift-down ($O(\log n)$ swaps).

First deleteMin (remove 3; move 13 to the root):

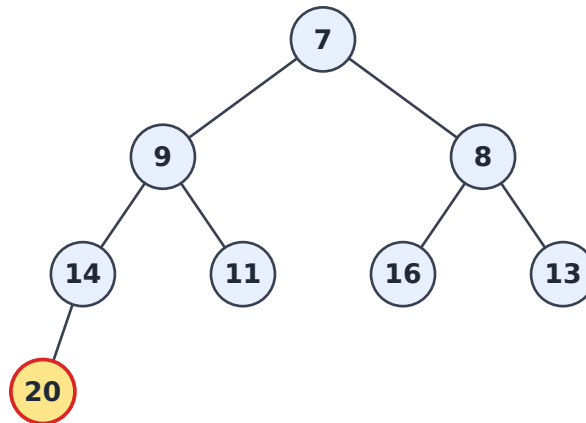
- [13, 9, 5, 14, 11, 8, 7, 20, 16]: children 9, 5 $\rightarrow$ swap with 5 $\rightarrow$ index 2.
- Index 2 children are 8 (idx 5) and 7 (idx 6) $\rightarrow$ swap with 7 $\rightarrow$ index 6 (leaf).
- Result: [5, 9, 7, 14, 11, 8, 13, 20, 16].

Second deleteMin (remove 5; move 16 to the root):

- [16, 9, 7, 14, 11, 8, 13, 20]: children 9, 7 $\rightarrow$ swap with 7 $\rightarrow$ index 2.
- Index 2 children are 8 (idx 5) and 13 (idx 6) $\rightarrow$ swap with 8 $\rightarrow$ index 5 (leaf).

- Result: **[7, 9, 8, 14, 11, 16, 13, 20]** → option (A).
- (B) puts 13 at index 5 and 16 at index 6 — but 16 sinks into the cell vacated by 8 (index 5), while 13 never moves in the second deletion.
- (C) sifts 16 toward the *larger* child (13) at the second step, leaving 13 above its smaller child 8 — heap order is violated.
- (D) wrongly swaps 8 up to index 1 (8 is not a child of the root).

Trap: always compare with **both** children and pick the smaller; choosing the left child by default breaks the heap property.



Heap after two deleteMin operations

Q.19 [MSQ · 2 marks · Binary search — rotated sorted array]

Answer: (A), (C), (D)

Key idea: compare $A[mid]$ with $A[hi]$. If $A[mid] > A[hi]$, the 'drop' (and hence the minimum) lies strictly right of mid; otherwise $A[mid..hi]$ is sorted and the minimum is at mid or to its left.

- (A) **True.** The invariant 'minimum $\in [lo, hi]$ ' holds after each step and the range strictly shrinks ($mid < hi$), so it ends on the minimum.
- (B) **False.** (0,4): $mid=2$, $A[2]=2 > A[4]=2$? no $\rightarrow hi=2$; (0,2): $mid=1 \rightarrow hi=1$; (0,1): $mid=0 \rightarrow hi=0$. Returns index 0 (value 2). With duplicates, $A[mid] == A[hi]$ gives no information about which side holds the drop.
- (C) **True.** While $lo < hi$ we have $mid < hi$, so for distinct values $A[mid] == A[hi]$ can never happen; $\geq$ and $>$ behave identically.
- (D) **True.** (lo,hi)=(0,9): $mid=4$, $A[4]=3 > 36$? no $\rightarrow hi=4$. (0,4): $mid=2$, $52 > 3 \rightarrow lo=3$. (3,4): $mid=3$, $60 > 3 \rightarrow lo=4$. Stop: (4, 3).

Tip: with duplicates the standard fix is `elif A[mid] == A[hi]: hi -= 1`, which makes the worst case $\Theta(n)$.

Q.20 [MCQ · 2 marks · Linked lists — self-organising linear search]

Answer: (C)

Linear search in a list costs (position of the key) comparisons; move-to-front makes recently used keys cheap, which pays off when accesses repeat.

- Search 7: 5, 9, 2, 7 → **4** comparisons; list becomes 7 → 5 → 9 → 2 → 4.
- Search 4: 7, 5, 9, 2, 4 → **5**; list 4 → 7 → 5 → 9 → 2.
- Search 7: 4, 7 → **2**; list 7 → 4 → 5 → 9 → 2.
- Search 7: 7 → **1**; list unchanged.
- Search 2: 7, 4, 5, 9, 2 → **5**; list 2 → 7 → 4 → 5 → 9.

Total = 4 + 5 + 2 + 1 + 5 = **17**; final list 2 → 7 → 4 → 5 → 9 → option (C).

- (A) is what a plain list (no reorganisation) would give: $4 + 5 + 4 + 4 + 3 = 20$, list unchanged.
- (B) has the right count but swaps 4 and 7 — after the last access to 7, 7 is ahead of 4.
- (D) under-counts the third search (7 is at position 2 after 4 was moved in front).

Note: moving a node to the front of a singly linked list is $O(1)$ once the scan has reached it, because the scan keeps the predecessor pointer.



Final list after the five searches

Mock Test 10

Elementary Sorting

Difficulty: Moderate

Focus areas: selection, bubble, insertion sorts — passes, swaps, comparisons

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing]

Consider the following Python program. What is printed?

Python

```
1 s = "PARTITIONS"
2 print(s[7:1:-2], s[::-4], s[-3:])
```

- (A) OTTR SIA ONS
- (B) OIT STA ONS
- (C) OTT STA ONS
- (D) OTT SIO NS

Q.2 [NAT · numerical answer | 1 mark | Selection sort — swaps]

Selection sort is applied to the array below to sort it in ascending order. In pass i ($i = 0, 1, \dots, n-2$) the minimum of $A[i..n-1]$ is located, and a swap is performed **only if** the minimum is not already at index i . The total number of swaps performed is _____.

A =	34	12	45	7	23	56	18
	0	1	2	3	4	5	6

Input array $A[0..6]$

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Bubble sort — early termination]

Bubble sort with an early-exit flag is run on the array

3, 9, 2, 11, 14, 6, 17, 20

In each pass adjacent pairs of the unsorted part are compared left to right and swapped if out of order; the algorithm stops after the first pass in which no swap occurs. How many passes are executed in total (including the final swap-free pass)?

- (A) 3
- (B) 4
- (C) 5
- (D) 7

Q.4 [NAT · numerical answer | 1 mark | Insertion sort — shifts and inversions]

Standard insertion sort (shift larger elements one place right, then drop the key into the gap) sorts the array below into ascending order. The total number of element **shifts** (assignments of the form $A[j+1] \leftarrow A[j]$) is _____.

A =	6	2	9	4	1	7
	0	1	2	3	4	5

Input array

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with an operand stack (all operands are single integers; / denotes integer division and the right operand is the one popped first).

6 2 3 + * 8 4 2 / - 3 * -

Which pair gives (value of the expression, maximum number of items on the stack at any time)?

- (A) (12, 3)
- (B) (12, 4)
- (C) (-12, 4)
- (D) (18, 4)

Q.6 [MSQ · one or more correct | 1 mark | Binary search trees]

The keys 41, 23, 67, 15, 30, 52, 80, 27, 35, 74, 33 are inserted in this order into an initially empty binary search tree (no rebalancing). Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) The tree has exactly 4 leaves
- (B) The post-order traversal begins 15, 27, 35
- (C) The height of the tree is 4
- (D) The in-order successor of 41 is 52

Q.7 [MCQ · single correct | 1 mark | Hashing — separate chaining]

The keys 15, 22, 9, 36, 3, 29, 44, 10 are inserted into an initially empty hash table with 7 slots (indices 0–6) using $h(k) = k \bmod 7$ and separate chaining. Which of the following correctly describes the final table?

- (A) Longest chain has length 3; 4 slots are empty
- (B) Longest chain has length 4; 3 slots are empty
- (C) Longest chain has length 4; 4 slots are empty
- (D) Longest chain has length 3; 3 slots are empty

Q.8 [MSQ · one or more correct | 1 mark | Binary search — probe counts]

Binary search is performed on the sorted array $A[0..14]$ shown below using $\text{mid} = \lfloor (\text{lo} + \text{hi})/2 \rfloor$, starting with $\text{lo} = 0$, $\text{hi} = 14$. Each examination of $A[\text{mid}]$ counts as one probe. For which of the following keys does a **successful** search take exactly 4 probes?

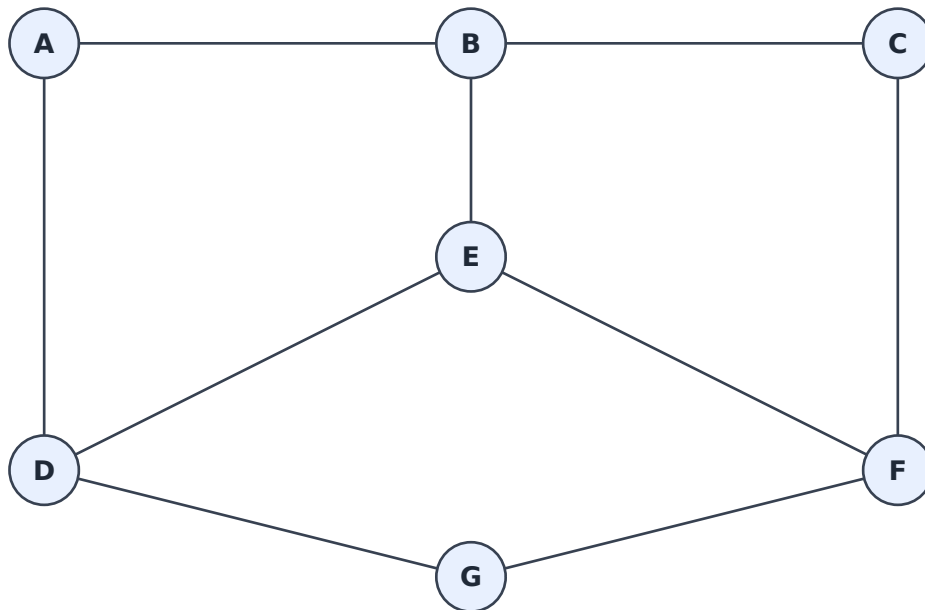
A =	4	9	13	18	22	27	31	36	40	45	52	58	63	70	77
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Sorted array of 15 keys

- (A) 70
 (B) 22
 (C) 36
 (D) 4

Q.9 [MCQ · single correct | 1 mark | Graph traversal — BFS]

Breadth-first search is run on the undirected graph below starting at vertex A. Whenever a vertex is dequeued, its unvisited neighbours are enqueued in **alphabetical order**. Which of the following is the order in which vertices are dequeued?



- (A) A B C F E D G
 (B) A D B G E C F
 (C) A B D E C G F
 (D) A B D C E G F

Q.10 [MSQ · one or more correct | 1 mark | Python — sort vs sorted, aliasing]

Consider the following Python code.

Python

```
1 a = [5, 3, 8, 1]
2 b = a
3 c = a[:]
4 d = a.sort(reverse=True)
5 e = sorted(c, key=lambda x: -x)
```

After it executes, which of the following expressions evaluate to **True**?

- (A) `c[0] == 8`
- (B) `b[0] == 8`
- (C) `e == b`
- (D) `d is None`

Q.11 [MCQ · single correct | 2 marks | Insertion sort — intermediate state]

The array below is sorted in ascending order by the insertion sort shown. What is the content of A immediately after the outer loop has finished the iteration with **i = 3**?

Python

```
1 def insertion_sort(A):
2     for i in range(1, len(A)):
3         key = A[i]
4         j = i - 1
5         while j >= 0 and A[j] > key:
6             A[j + 1] = A[j]
7             j -= 1
8         A[j + 1] = key
```

A =	21	8	30	14	5	26	11	17
	0	1	2	3	4	5	6	7

Initial array

- (A) 8, 14, 21, 30, 5, 26, 11, 17
- (B) 8, 5, 14, 11, 17, 21, 26, 30
- (C) 5, 8, 14, 21, 30, 26, 11, 17
- (D) 5, 8, 11, 14, 21, 26, 30, 17

Q.12 [NAT · numerical answer | 2 marks | Bubble sort — comparison count]

The following bubble sort (with a shrinking range and an early-exit flag) is called on

`A = [2, 6, 1, 5, 9, 3, 10, 12]`. The total number of times the comparison `A[j] > A[j + 1]` is evaluated is _____.

Python

```
1 def bubble(A):
2     n = len(A)
3     for p in range(n - 1):
4         swapped = False
5         for j in range(n - 1 - p):
6             if A[j] > A[j + 1]:
7                 A[j], A[j + 1] = A[j + 1], A[j]
8                 swapped = True
9         if not swapped:
10            break
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Sorting — stability]

Five records (key, tag) are stored in a list in the order

(6, p), (4, q), (6, r), (2, s), (9, t)

and are sorted by **key only** (ascending). Consider these four procedures:

- Selection sort: for $i = 0..n-2$, find the *first* index m of the minimum key in positions $i..n-1$ (strict $<$) and swap positions i and m .
- Insertion sort that shifts while $A[j].key > key$.
- Bubble sort ($n-1$ full passes) that swaps adjacent records while $A[j].key \geq A[j+1].key$.
- Python's `sorted(R, key=lambda x: x[0])`.

Which of the following statements is/are TRUE?

- (A) Selection sort outputs (6, r) before (6, p)
- (B) The $\geq$ bubble sort outputs (6, r) before (6, p)
- (C) Insertion sort outputs (6, p) before (6, r)
- (D) `sorted` outputs (6, r) before (6, p)

Q.14 [MCQ · single correct | 2 marks | Python — bubble passes and copying]

Consider the following Python program. What is printed?

Python

```
1 def pass_k(a, k):
2     a = a[:]
3     for p in range(k):
4         for j in range(len(a) - 1 - p):
5             if a[j] > a[j + 1]:
6                 a[j], a[j + 1] = a[j + 1], a[j]
7     return a
8
9 data = [7, 3, 9, 1, 6, 2]
10 r = pass_k(data, 2)
11 print(r, data[0])
```

- (A) [3, 1, 6, 2, 7, 9] 7
- (B) [3, 1, 6, 2, 7, 9] 3
- (C) [3, 7, 1, 6, 2, 9] 7
- (D) [1, 3, 2, 6, 7, 9] 7

Q.15 [NAT · numerical answer | 2 marks | Quicksort — Lomuto partition swaps]

The quicksort below (Lomuto partition, last element as pivot) is called as `quicksort(A, 0, 7)` on $A = [12, 5, 17, 3, 20, 9, 14, 10]$. Every execution of a swap statement is counted, **including** swaps of an element with itself. The total number of swaps executed is _____.

Python

```
1 def quicksort(A, lo, hi):
2     if lo < hi:
3         pivot, i = A[hi], lo - 1
4         for j in range(lo, hi):
5             if A[j] <= pivot:
6                 i += 1
7                 A[i], A[j] = A[j], A[i]      # swap 1
8             A[i + 1], A[hi] = A[hi], A[i + 1] # swap 2
9         quicksort(A, lo, i)
10        quicksort(A, i + 2, hi)
```

Answer: _____ (enter a numerical value)

Q.16 [NAT · numerical answer | 2 marks | Merge sort — comparisons]

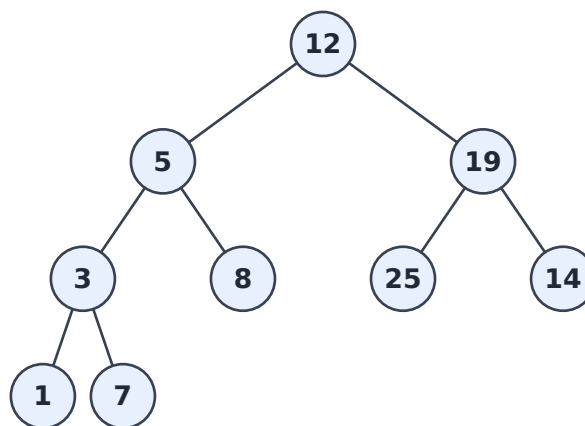
Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ elements and the rest) is applied to 4, 11, 2, 9, 15, 6, 1, 13

The merge step compares the front elements of the two runs, outputs the smaller, and stops comparing as soon as one run is exhausted (the rest of the other run is copied without comparisons). The total number of element comparisons over the whole sort is _____.

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Heaps — bottom-up build-heap]

The array $A = [12, 5, 19, 3, 8, 25, 14, 1, 7]$ (0-indexed, drawn below as a complete binary tree) is converted into a **max-heap** by the bottom-up method: sift-down is called for $i = \lfloor n/2 \rfloor - 1$ down to 0; sift-down swaps a node with its larger child while that child is larger. Which of the following statements is/are TRUE?

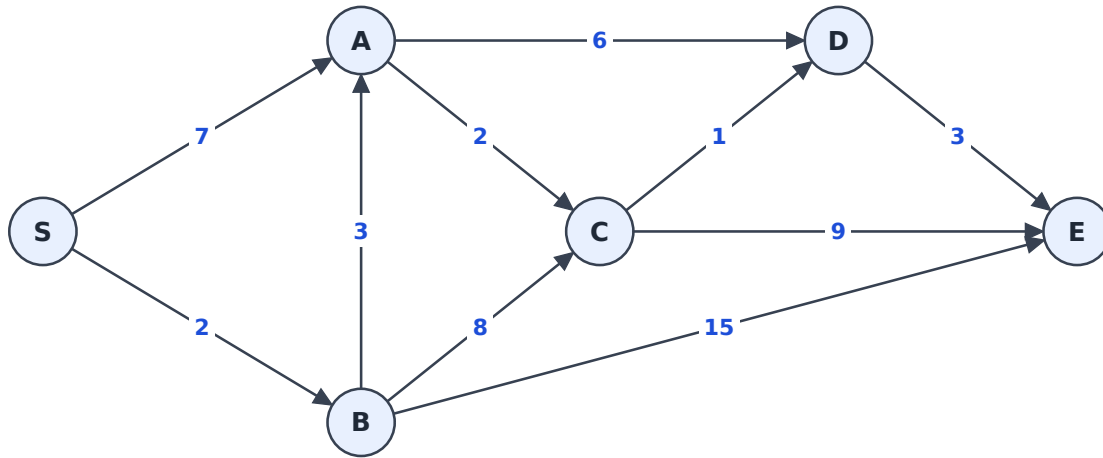


Initial array viewed as a complete binary tree

- (A) If one delete-max is then performed (move last element to the root and sift down), the array becomes 19, 8, 14, 7, 5, 12, 1, 3
- (B) Exactly 5 swaps are performed during the build
- (C) In the resulting heap the children of the root are 8 and 19
- (D) In the resulting heap, key 3 is at index 8

Q.18 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra relaxations]

Dijkstra's algorithm is run from source S on the directed graph below. Initially $d[S] = 0$ and all other d-values are ∞ . Count every time some $d[v]$ is **strictly decreased** during relaxation (the first change from ∞ also counts). The total count is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — selection by relinking]

The function `f` below moves a node of a singly linked list to the front by relinking pointers. The list `6 → 3 → 8 → 2 → 5 → 1` (shown below) is processed as in the program. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def f(head):
6     prev, cur = None, head
7     best_prev, best = None, head
8     while cur:
9         if cur.v < best.v:
10             best_prev, best = prev, cur
11             prev, cur = cur, cur.nxt
12     if best_prev:
13         best_prev.nxt = best.nxt
14         best.nxt = head
15         head = best
16     return head
17
18 h = None
19 for x in reversed([6, 3, 8, 2, 5, 1]):
20     h = Node(x, h)
21 h = f(h)
22 h.nxt = f(h.nxt)
23 h.nxt.nxt = f(h.nxt.nxt)
24 out = []
25 while h:
26     out.append(h.v); h = h.nxt
27 print(*out)

```



(A) 1 2 3 5 6 8

(B) 1 2 6 3 8 5

(C) 1 6 3 8 2 5**(D)** 1 2 3 6 8 5**Q.20 [NAT · numerical answer | 2 marks | Insertion sort — comparison count]**

Consider the insertion sort below, called on $A = [7, 4, 10, 1, 8, 3, 12]$. Because of short-circuit evaluation, the comparison $A[j] > \text{key}$ is evaluated only when $j \geq 0$ is true. The total number of times $A[j] > \text{key}$ is evaluated is _____.

Python

```

1 def isort(A):
2     for i in range(1, len(A)):
3         key, j = A[i], i - 1
4         while j >= 0 and A[j] > key:
5             A[j + 1] = A[j]
6             j -= 1
7         A[j + 1] = key

```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 10

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — string slicing
2	NAT	1	4	Selection sort — swaps
3	MCQ	1	(B)	Bubble sort — early termination
4	NAT	1	8	Insertion sort — shifts and inversions
5	MCQ	1	(B)	Stacks — postfix evaluation
6	MSQ	1	(C), (D)	Binary search trees
7	MCQ	1	(C)	Hashing — separate chaining
8	MSQ	1	(B), (D)	Binary search — probe counts
9	MCQ	1	(D)	Graph traversal — BFS
10	MSQ	1	(B), (C), (D)	Python — sort vs sorted, aliasing
11	MCQ	2	(A)	Insertion sort — intermediate state
12	NAT	2	22	Bubble sort — comparison count
13	MSQ	2	(A), (C)	Sorting — stability
14	MCQ	2	(A)	Python — bubble passes and copying
15	NAT	2	13	Quicksort — Lomuto partition swaps
16	NAT	2	16	Merge sort — comparisons
17	MSQ	2	(B), (C), (D)	Heaps — bottom-up build-heap
18	NAT	2	10	Shortest paths — Dijkstra relaxations
19	MCQ	2	(D)	Linked lists — selection by relinking
20	NAT	2	13	Insertion sort — comparison count

Detailed Solutions — Mock Test 10

Q.1 [MCQ · 1 mark · Python — string slicing]

Answer: (C)

Concept. `s[start:stop:step]` with a negative step walks *leftwards* from `start` and stops *before* reaching `stop`. Omitted bounds with a negative step mean “from the last character down to the first”.

Index the string: P0 A1 R2 T3 I4 T5 I6 O7 N8 S9.

- `s[7:1:-2]` → indices 7, 5, 3 (index 1 is excluded) → O, T, T → **OTT**.
- `s[:-4]` → indices 9, 5, 1 → S, T, A → **STA**.
- `s[-3:]` → the last three characters, indices 7, 8, 9 → **ONS**.

`print` separates its arguments with one space, giving **OTT STA ONS**.

Options. (B) takes index 6 (I) instead of 5 — a step of -1 then -2 confusion. (A) wrongly includes the stop index 1 (R is at index 2, not 1, so it is doubly wrong). (D) starts `s[:-4]` from index 7 instead of 9.

Trap: the stop index is *never* included, whatever the sign of the step.

Q.2 [NAT · 1 mark · Selection sort — swaps]

Answer: 4

Concept. Selection sort always makes $n-1$ passes; a pass swaps only when the minimum of the unsorted suffix is not already in front.

Trace (sorted prefix grows by one each pass):

- $i=0$: min of whole array is 7 (index 3) → swap $34 \leftrightarrow 7 \rightarrow 7\ 12\ 45\ 34\ 23\ 56\ 18$ (swap 1).
- $i=1$: min of $A[1..6]$ is 12, already at index 1 → no swap.
- $i=2$: min of $A[2..6]$ is 18 → swap $45 \leftrightarrow 18 \rightarrow 7\ 12\ 18\ 34\ 23\ 56\ 45$ (swap 2).
- $i=3$: min is 23 → swap $34 \leftrightarrow 23 \rightarrow 7\ 12\ 18\ 23\ 34\ 56\ 45$ (swap 3).
- $i=4$: min is 34, already in place → no swap.
- $i=5$: min of $\{56, 45\}$ is 45 → swap → $7\ 12\ 18\ 23\ 34\ 45\ 56$ (swap 4).

Total swaps = **4**.

Trap: answering $n-1 = 6$ assumes every pass swaps. Selection sort performs *at most* $n-1$ swaps, which is why it is preferred when writes are expensive.

Array after each pass

	0	1	2	3	4	5	6	swap?
i=0	7	12	45	34	23	56	18	yes
i=1	7	12	45	34	23	56	18	no
i=2	7	12	18	34	23	56	45	yes
i=3	7	12	18	23	34	56	45	yes
i=4	7	12	18	23	34	56	45	no
i=5	7	12	18	23	34	45	56	yes

Q.3 [MCQ · 1 mark · Bubble sort — early termination]**Answer: (B)**

Concept. Each bubble pass carries the largest remaining element to the end, but also moves every small element at most **one** position left. The number of passes needed equals the largest leftward distance any element must travel, plus one swap-free confirmation pass.

- Pass 1: 3 2 9 11 6 14 17 20 (swaps $9 \leftrightarrow 2$ and $14 \leftrightarrow 6$).
- Pass 2: 2 3 9 6 11 14 17 20 (swaps $3 \leftrightarrow 2$, $11 \leftrightarrow 6$).
- Pass 3: 2 3 6 9 11 14 17 20 (swap $9 \leftrightarrow 6$). The array is now sorted, but the algorithm does not know that yet.
- Pass 4: no swap $\rightarrow$ stop.

Element 6 started at index 5 and must reach index 2 (3 steps left) $\rightarrow$ 3 sorting passes + 1 check = **4**.

Options. (A) forgets the confirming pass. (C) and (D) ignore the early exit ($n-1 = 7$ is the count for plain bubble sort).

Tip: “turtles” (small values near the end) govern bubble-sort passes; “rabbits” (large values near the front) move fast.

Q.4 [NAT · 1 mark · Insertion sort — shifts and inversions]**Answer: 8**

Concept. Every shift in insertion sort removes exactly one inversion (a pair $i < j$ with $A[i] > A[j]$), and the algorithm stops when no inversions remain. So shifts = number of inversions.

Count inversions element by element (larger elements to the left of each):

- 6: none $\rightarrow 0$
- 2: {6} $\rightarrow 1$
- 9: none $\rightarrow 0$
- 4: {6, 9} $\rightarrow 2$
- 1: {6, 2, 9, 4} $\rightarrow 4$
- 7: {9} $\rightarrow 1$

Total = $0+1+0+2+4+1 = \mathbf{8}$ shifts.

Cross-check by tracing: inserting 2 shifts 1; 9 shifts 0; 4 shifts 2 (9, 6); 1 shifts 4; 7 shifts 1 (9) — again 8.

Trap: do not confuse shifts with comparisons. Comparisons are higher (here 11) because a pass that stops on a smaller element still performs one failing comparison.

Q.5 [MCQ · 1 mark · Stacks — postfix evaluation]**Answer: (B)**

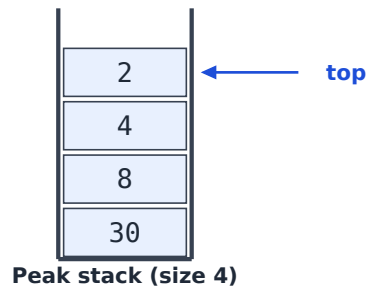
Concept. Push operands; on an operator pop the right operand, then the left one, and push the result.

- 6, 2, 3 $\rightarrow$ stack [6, 2, 3] (size 3)
- + $\rightarrow$ [6, 5]; * $\rightarrow$ [30]
- 8, 4, 2 $\rightarrow$ [30, 8, 4, 2] (size **4**, the maximum)
- / $\rightarrow 4/2 = 2 \rightarrow$ [30, 8, 2]
- - $\rightarrow 8 - 2 = 6 \rightarrow$ [30, 6]
- 3 $\rightarrow$ [30, 6, 3]; * $\rightarrow$ [30, 18]
- - $\rightarrow 30 - 18 = \mathbf{12} \rightarrow$ [12]

So the answer is (12, 4).

Options. (A) misses the moment when 8, 4, 2 sit on top of 30. (C) reverses the operand order of the last subtraction ($18 - 30$). (D) reports the intermediate 18 instead of the final result.

Trap: for non-commutative operators the *first* pop is the **right** operand.



Q.6 [MSQ · 1 mark · Binary search trees]

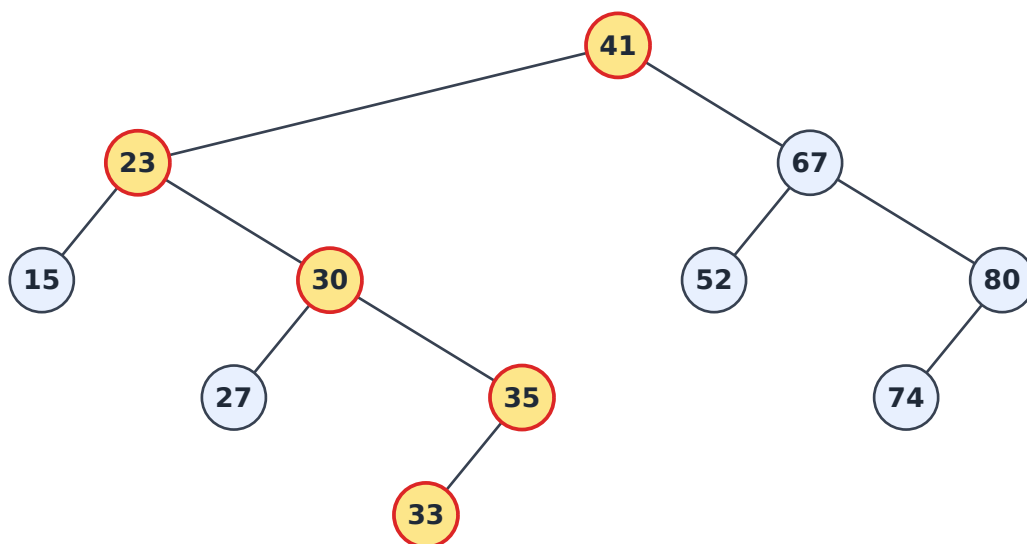
Answer: (C), (D)

Concept. Each key walks down from the root, going left if smaller and right otherwise, and is attached where it falls off.

Resulting tree: 41 has children 23 and 67; 23 → (15, 30); 30 → (27, 35); 35 → (33, -); 67 → (52, 80); 80 → (74, -).

- (A) Leaves are 15, 27, 33, 52, 74 — that is **5** leaves. **FALSE.**
- (B) Post-order = 15, 27, 33, 35, 30, 23, 52, 74, 80, 67, 41. It begins 15, 27, **33** (the child 33 is finished before its parent 35). **FALSE.**
- (C) Longest path 41→23→30→35→33 has **4 edges**. **TRUE.**
- (D) Successor of 41 = minimum of its right subtree = leftmost node under 67 = 52. **TRUE.**

Trap: in (B) students often list 35 before 33 because 35 was inserted first — post-order visits a node only after *both* of its subtrees.



BST with its longest root-to-leaf path highlighted

Q.7 [MCQ · 1 mark · Hashing — separate chaining]

Answer: (C)

Concept. With chaining every key goes to the list at slot $h(k)$; collisions just lengthen the list.

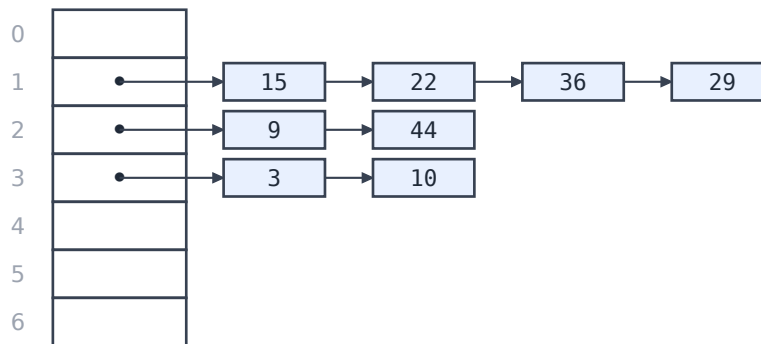
- $15 \bmod 7 = 1$, $22 \bmod 7 = 1$, $36 \bmod 7 = 1$, $29 \bmod 7 = 1 \rightarrow$ slot 1 gets **4** keys.
- $9 \bmod 7 = 2$, $44 \bmod 7 = 2 \rightarrow$ slot 2 gets 2 keys.
- $3 \bmod 7 = 3$, $10 \bmod 7 = 3 \rightarrow$ slot 3 gets 2 keys.

Occupied slots: 1, 2, 3. Empty slots: 0, 4, 5, 6 → **4** empty.

So the longest chain has length 4 and 4 slots are empty → (C).

Options. (A)/(D) miss that $29 = 4 \cdot 7 + 1$ also lands in slot 1. (B) counts occupied slots instead of empty ones.

Tip: keys that differ by a multiple of m always collide — here 15, 22, 29, 36 form an arithmetic progression with difference 7.



Final chained table (new keys appended at the tail)

Q.8 [MSQ · 1 mark · Binary search — probe counts]

Answer: (B), (D)

Concept. For $n = 2^4 - 1 = 15$ elements the binary-search decision tree is perfect: one key at depth 1, two at depth 2, four at depth 3 and eight at depth 4.

- Probe 1: index 7 (36).
- Probe 2: index 3 (18) or 11 (58).
- Probe 3: index 1 (9), 5 (27), 9 (45), 13 (70).
- Probe 4: even indices 0, 2, 4, ..., 14 → 4, 13, 22, 31, 40, 52, 63, 77.

Option by option:

- (A) 70 is at index $13 \rightarrow 7 \rightarrow 11 \rightarrow 13$: 3 probes. **FALSE.**
- (B) 22 is at index $4 \rightarrow 7 \rightarrow 3 \rightarrow 5 \rightarrow 4$: 4 probes. **TRUE.**
- (C) 36 is found on the very first probe. **FALSE.**
- (D) 4 is at index $0 \rightarrow 4$ probes ($7 \rightarrow 3 \rightarrow 1 \rightarrow 0$). **TRUE.**

Tip: with $n = 2^k - 1$, exactly half the keys (the even indices) need the maximum k probes.

Q.9 [MCQ · 1 mark · Graph traversal — BFS]

Answer: (D)

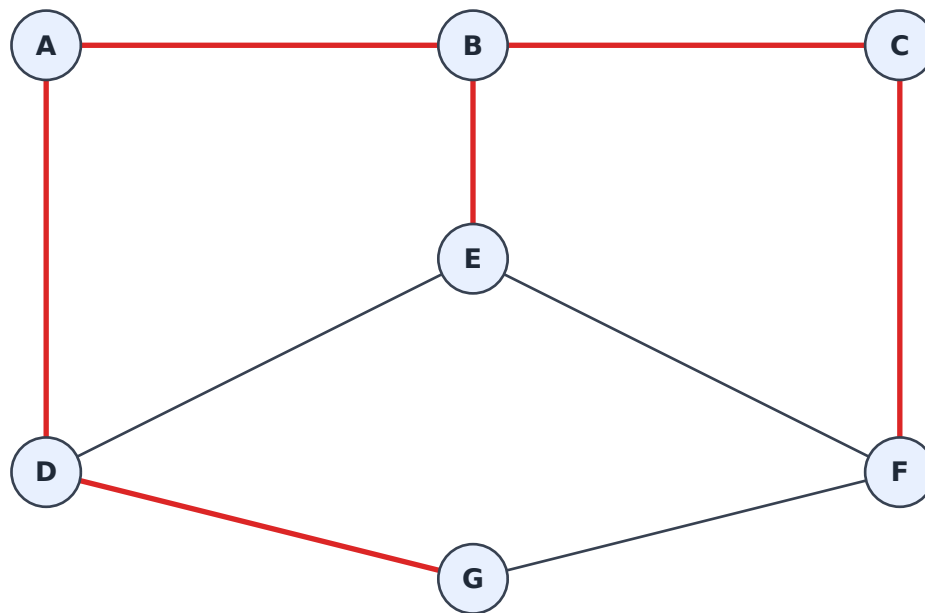
Concept. BFS dequeues vertices level by level; within a level the order is the order in which they were enqueued.

- Dequeue A → enqueue B, D. Queue: B D
- Dequeue B → neighbours A, C, E; enqueue C, E. Queue: D C E
- Dequeue D → neighbours A, E, G; only G is new. Queue: C E G
- Dequeue C → neighbour F is new. Queue: E G F
- Dequeue E, then G, then F (nothing new).

Order: **A B D C E G F** → (D).

Options. (A) is the DFS order (alphabetical). (C) puts E before C, i.e. treats E as a neighbour of D enqueued earlier — but E was already enqueued by B *after* C. (B) starts with D, violating alphabetical order.

Trap: a vertex is marked when it is *enqueued*, not when dequeued; E is not re-added by D.



BFS tree (highlighted edges)

Q.10 [MSQ · 1 mark · Python — sort vs sorted, aliasing]**Answer: (B), (C), (D)**

Concept. `list.sort()` sorts *in place* and returns `None`; `sorted()` builds and returns a *new* list. `b = a` creates an alias, `a[:]` a shallow copy.

- After `a.sort(reverse=True)`, the list object shared by `a` and `b` is `[8, 5, 3, 1]`.
- `c` is a separate copy taken *before* sorting, so it is still `[5, 3, 8, 1]`.
- `d` receives the return value of `sort`, which is `None`.
- `e` sorts the copy by key `-x`, i.e. descending $\rightarrow [8, 5, 3, 1]$.

Option by option:

- (A) `c[0]` is 5 — the copy is untouched. **False.**
- (B) `b[0]` is 8 because `b` aliases the sorted list. **True.**
- (C) `e = [8, 5, 3, 1] = b` (equal contents). **True.**
- (D) `d is None`. **True.**

Trap: writing `x = x.sort()` is a classic bug that silently replaces the list by `None`.

Q.11 [MCQ · 2 marks · Insertion sort — intermediate state]**Answer: (A)**

Concept. After the iteration with index `i`, insertion sort guarantees that the prefix `A[0..i]` is a sorted arrangement of the *original* first `i+1` elements; the suffix is untouched.

Trace:

- `i = 1` (key 8): `21 > 8` $\rightarrow$ shift $\rightarrow$ 8, 21, 30, 14, 5, 26, 11, 17
- `i = 2` (key 30): `21 > 30` is false $\rightarrow$ no change
- `i = 3` (key 14): shift 30, then 21; `8 > 14` false $\rightarrow$ 8, 14, 21, 30, 5, 26, 11, 17

So the array is **8, 14, 21, 30, 5, 26, 11, 17** $\rightarrow$ (A).

Option by option:

- (A) Correct: the prefix is a sorted version of `{21, 8, 30, 14}` and the tail is unchanged.
- (B) is the state after three passes of **bubble** sort (the three largest, 21, 26, 30, are parked at the end).

- (C) is one iteration too far (after $i = 4$, key 5 has been inserted).
- (D) is the state after three passes of **selection** sort (the three global minima 5, 8, 11 are in front). Insertion sort cannot bring 5 forward before it reaches index 4.

Tip: a quick fingerprint test — selection sort's prefix contains the *global* minima, bubble sort's suffix contains the *global* maxima, insertion sort's prefix contains the *original* first elements in sorted order.

Insertion-sort trace

	0	1	2	3	4	5	6	7
start	21	8	30	14	5	26	11	17
i=1	8	21	30	14	5	26	11	17
i=2	8	21	30	14	5	26	11	17
i=3	8	14	21	30	5	26	11	17

Q.12 [NAT · 2 marks · Bubble sort — comparison count]

Answer: 22

Concept. Pass p (0-based) compares positions $0..n-2-p$, i.e. $n-1-p$ comparisons, whether or not it swaps. The total is therefore determined by *how many passes run*.

$n = 8$. Trace:

- Pass $p=0$ (7 comparisons): $2 < 6$ keep; $6 > 1$ swap; $6 > 5$ swap; $6 < 9$ keep; $9 > 3$ swap; $9 < 10$ keep; $10 < 12$ keep → **2 1 5 6 3 9 10 12** (12 is in its final place).
- Pass $p=1$ (6 comparisons): 1 2 5 3 6 9 10 12 (swaps $2 \leftrightarrow 1$, $6 \leftrightarrow 3$).
- Pass $p=2$ (5 comparisons): 1 2 3 5 6 9 10 12 (swap $5 \leftrightarrow 3$).
- Pass $p=3$ (4 comparisons): no swap → exit.

Total comparisons = $7 + 6 + 5 + 4 = \mathbf{22}$.

Why 4 passes? Element 3 starts at index 5 and must reach index 2; each pass moves a small element left by at most one place, so 3 passes are needed to sort plus one swap-free pass to detect it.

Trap: counting only swaps (6) or assuming the full $n(n-1)/2 = 28$ comparisons. The early exit saves $3 + 2 + 1 = 6$ comparisons here.

Array after each pass

	0	1	2	3	4	5	6	7	comps
p=0	2	1	5	6	3	9	10	12	7
p=1	1	2	5	3	6	9	10	12	6
p=2	1	2	3	5	6	9	10	12	5
p=3	1	2	3	5	6	9	10	12	4

Q.13 [MSQ · 2 marks · Sorting — stability]

Answer: (A), (C)

Concept. A sort is *stable* if records with equal keys keep their input order. Swapping non-adjacent elements (selection sort) or swapping equal neighbours ($a \geq b$ bubble sort) can break stability — but whether it *actually* happens depends on the input.

Selection sort trace:

- $i=0$: minimum key 2 at index 3 → swap (6,p) with (2,s): (2,s) (4,q) (6,r) (6,p) (9,t). The long-distance swap has jumped (6,p) **over** (6,r).

• $i=1$: (4,q) already minimal. $i=2$: first minimum key 6 is (6,r) at index 2 $\rightarrow$ no swap. $i=3$: no swap. Output: (2,s) (4,q) **(6,r) (6,p)** (9,t) $\rightarrow$ (A) **TRUE**.

Insertion sort with strict $>$ never moves a record past an equal key, so it is stable: (6,p) stays ahead of (6,r) $\rightarrow$ (C) **TRUE**.

$\geq$ **bubble sort**: equal neighbours are swapped every time they meet. Pass 1: (6,p) $>$ (4,q) swap; (6,p) $\geq$ (6,r) **swap** $\rightarrow$ r ahead; (6,p) $>$ (2,s) swap; ... giving (4,q) (6,r) (2,s) (6,p) (9,t). Pass 2: (6,r) $>$ (2,s) swap; (6,r) $\geq$ (6,p) **swap again** $\rightarrow$ p ahead: (4,q) (2,s) (6,p) (6,r) (9,t). Passes 3–4 only move (2,s) to the front. The two 6s were swapped an even number of times, so the final order is (6,p) (6,r) $\rightarrow$ (B) **FALSE** (the algorithm is unstable in general, but not on this input).

sorted uses Timsort, which is guaranteed stable $\rightarrow$ (6,p) first $\rightarrow$ (D) **FALSE**.

Trap: “unstable algorithm” does not mean “always reorders equal keys” — always trace.

Q.14 [MCQ · 2 marks · Python — bubble passes and copying]

Answer: (A)

Concept. Two things are tested: (i) what k bubble passes do, and (ii) that `a = a[:]` rebinds the *local* name to a copy, so the caller’s list is untouched.

Pass $p = 0$ ($j = 0..4$) on 7 3 9 1 6 2:

- 7 $>$ 3 swap $\rightarrow$ 3 7 9 1 6 2; 7 $<$ 9 keep; 9 $>$ 1 swap $\rightarrow$ 3 7 1 9 6 2; 9 $>$ 6 swap $\rightarrow$ 3 7 1 6 9 2; 9 $>$ 2 swap $\rightarrow$ **3 7 1 6 2 9**.

Pass $p = 1$ ($j = 0..3$) on 3 7 1 6 2 9:

- 3 $<$ 7 keep; 7 $>$ 1 swap $\rightarrow$ 3 1 7 6 2 9; 7 $>$ 6 swap $\rightarrow$ 3 1 6 7 2 9; 7 $>$ 2 swap $\rightarrow$ **3 1 6 2 7 9**.

`r = [3, 1, 6, 2, 7, 9]`. Because the function sorted a copy, `data` is still [7, 3, 9, 1, 6, 2] and `data[0]` is 7.

Option by option:

- (A) Correct.
- (B) assumes the function mutated `data` (it would if the line `a = a[:]` were absent).
- (C) is the result after only one pass (off-by-one on `range(k)`).
- (D) is the result after three passes.

Tip: after k bubble passes the last k positions hold the k largest values in order — here 7, 9 — a quick sanity check on the options.

Q.15 [NAT · 2 marks · Quicksort — Lomuto partition swaps]

Answer: 13

Concept. In one Lomuto partition of $A[\text{lo}..\text{hi}]$, swap 1 runs once for every element $\leq$ pivot (among $A[\text{lo}..\text{hi}-1]$) and swap 2 runs exactly once. So a call contributes (number of elements $\leq$ pivot) + 1 swaps; calls on ranges of size ≤ 1 contribute nothing.

Trace of the recursion:

- quicksort(0,7), pivot 10: elements ≤ 10 are 5, 3, 9 $\rightarrow 3 + 1 = 4$ swaps $\rightarrow$ [5, 3, 9, **10**, 20, 17, 14, 12].
- quicksort(0,2) on [5, 3, 9], pivot 9: 5 and 3 are $\leq 9 \rightarrow 2 + 1 = 3$ swaps (all are self-swaps) $\rightarrow$ [5, 3, 9].
- quicksort(0,1) on [5, 3], pivot 3: none $\leq 3 \rightarrow 1$ swap $\rightarrow$ [3, 5].
- quicksort(4,7) on [20, 17, 14, 12], pivot 12: none $\leq 12 \rightarrow 1$ swap $\rightarrow$ [12, 17, 14, 20].
- quicksort(5,7) on [17, 14, 20], pivot 20: 17, 14 $\leq 20 \rightarrow 2 + 1 = 3$ swaps $\rightarrow$ unchanged.
- quicksort(5,6) on [17, 14], pivot 14: **1** swap $\rightarrow$ [14, 17].

Total = 4 + 3 + 1 + 1 + 3 + 1 = **13**.

Trap: forgetting the self-swaps (when $i = j$) or the final pivot swap of each call; also note that when the pivot is the maximum of its range, every element triggers swap 1.

Partition calls

range	pivot	$\leq$ pivot	swaps
0..7	10	3	4
0..2	9	2	3
0..1	3	0	1
4..7	12	0	1
5..7	20	2	3
5..6	14	0	1

Q.16 [NAT · 2 marks · Merge sort — comparisons]

Answer: 16

Concept. Merging runs of lengths p and q costs between $\min(p, q)$ and $p+q-1$ comparisons: fewer when one run empties early.

Level by level (bottom-up view of the recursion):

- Pairs: $[4]+[11]$, $[2]+[9]$, $[15]+[6]$, $[1]+[13] \rightarrow 1$ comparison each = **4**.
- Merge $[4, 11]$ with $[2, 9]$: $2 < 4 \rightarrow 2$; $4 < 9 \rightarrow 4$; $9 < 11 \rightarrow 9$; right run empty $\rightarrow$ copy 11. **3** comparisons $\rightarrow [2, 4, 9, 11]$.
- Merge $[6, 15]$ with $[1, 13]$: $1 < 6$; $6 < 13$; then copy 15. **3** comparisons $\rightarrow [1, 6, 13, 15]$.
- Final merge $[2, 4, 9, 11]$ with $[1, 6, 13, 15]$: $1, 2, 4, 6, 9, 11$ are output after **6** comparisons; the left run is now empty and $13, 15$ are copied.

Total = $4 + 3 + 3 + 6 = \mathbf{16}$.

Bounds check: the worst case for $n = 8$ is $1 \cdot 4 + 3 \cdot 2 + 7 = 17$, the best case is $4 + 2 \cdot 2 + 4 = 12$; 16 lies between.

Trap: using the formula $n \log_2 n - n + 1 = 17$ blindly — that is only the worst case.

Every merge performed

left run	right run	result	comps
4	11	4 11	1
2	9	2 9	1
15	6	6 15	1
1	13	1 13	1
4 11	2 9	2 4 9 11	3
6 15	1 13	1 6 13 15	3
2 4 9 11	1 6 13 15	1 2 4 6 9 11 13 15	6

Q.17 [MSQ · 2 marks · Heaps — bottom-up build-heap]

Answer: (B), (C), (D)

Concept. Build-heap sifts down each internal node, starting from the last internal node (index $\lfloor 9/2 \rfloor - 1 = 3$).

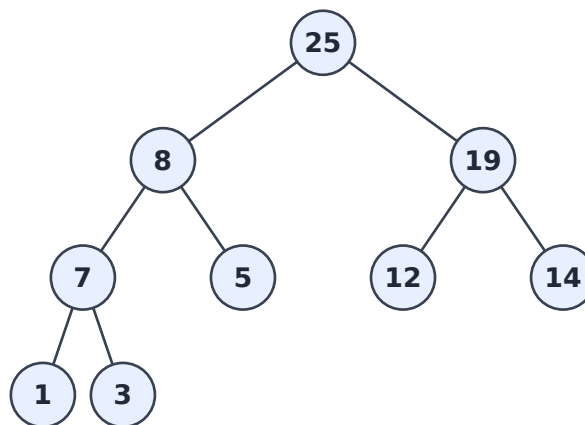
- $i=3$ (3): children 1, 7 $\rightarrow$ swap with 7 (**swap 1**). $A = 12\ 5\ 19\ 7\ 8\ 25\ 14\ 1\ 3$
- $i=2$ (19): children 25, 14 $\rightarrow$ swap with 25 (**swap 2**). $A = 12\ 5\ 25\ 7\ 8\ 19\ 14\ 1\ 3$

- $i=1$ (5): children 7, 8 $\rightarrow$ swap with 8 (**swap 3**); index 4 is a leaf. $A = 12\ 8\ 25\ 7\ 5\ 19\ 14\ 1\ 3$
- $i=0$ (12): children 8, 25 $\rightarrow$ swap with 25 (**swap 4**); now at index 2, children 19, 14 $\rightarrow$ swap with 19 (**swap 5**). $A = \mathbf{25\ 8\ 19\ 7\ 5\ 12\ 14\ 1\ 3}$

Option by option:

- (A) Delete-max: move 3 to the root $\rightarrow 3\ 8\ 19\ 7\ 5\ 12\ 14\ 1$; swap with 19 $\rightarrow$ index 2; children 12, 14 $\rightarrow$ swap with 14 $\rightarrow 19\ 8\ 14\ 7\ 5\ 12\ \mathbf{3\ 1}$. The option has the last two reversed (it forgets that 3 moved down to index 6 while 1 stayed at index 7). **FALSE**.
- (B) Five swaps in total. **TRUE**.
- (C) Root 25 has children $A[1] = 8$ and $A[2] = 19$. **TRUE**.
- (D) Key 3 sits at index 8 (a leaf). **TRUE**.

Trap: in sift-down, keep going after the first swap — 12 had to fall two levels.



Max-heap after build-heap

Q.18 [NAT · 2 marks · Shortest paths — Dijkstra relaxations]

Answer: 10

Concept. Dijkstra extracts the vertex with the smallest tentative distance and relaxes its outgoing edges; a relaxation *succeeds* when $d[u] + w < d[v]$.

- Extract S (0): $A = 7$, $B = 2 \rightarrow \mathbf{2}$ decreases.
- Extract B (2): $A = \min(7, 5) = 5 \checkmark$, $C = 10 \checkmark$, $E = 17 \checkmark \rightarrow \mathbf{3}$.
- Extract A (5): $C = \min(10, 7) = 7 \checkmark$, $D = 11 \checkmark \rightarrow \mathbf{2}$.
- Extract C (7): $D = \min(11, 8) = 8 \checkmark$, $E = \min(17, 16) = 16 \checkmark \rightarrow \mathbf{2}$.
- Extract D (8): $E = \min(16, 11) = 11 \checkmark \rightarrow \mathbf{1}$.
- Extract E (11): no outgoing edges.

Total = $2 + 3 + 2 + 2 + 1 = \mathbf{10}$.

Final distances: A 5, B 2, C 7, D 8, E 11 (path $S \rightarrow B \rightarrow A \rightarrow C \rightarrow D \rightarrow E$).

Trap: E is improved three times ($17 \rightarrow 16 \rightarrow 11$) and D twice ($11 \rightarrow 8$) — students who only record final values or skip the initial $\infty \rightarrow$ finite updates undercount.

d-values after each extraction

	S	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
S	0	7	2	∞	∞	∞
B	0	5	2	10	∞	17
A	0	5	2	7	11	17
C	0	5	2	7	8	16
D	0	5	2	7	8	11

Q.19 [MCQ · 2 marks · Linked lists — selection by relinking]**Answer: (D)**

Concept. f finds the node with the minimum value (first occurrence) and splices it to the front of the list it is given. Calling it on successive suffixes is exactly **selection sort on a linked list** — but only three passes are made here.

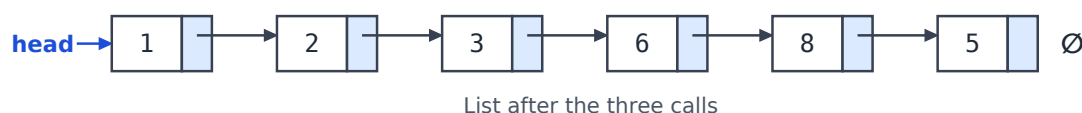
- Pass 1, $f(6\ 3\ 8\ 2\ 5\ 1)$: minimum 1 (its predecessor is 5). Unlink 1, point it at 6 $\rightarrow$ **1** 6 3 8 2 5.
- Pass 2, $f(6\ 3\ 8\ 2\ 5)$ on the suffix after 1: minimum 2 $\rightarrow$ 2 6 3 8 5; $h.next$ is re-attached to it $\rightarrow$ **1** 2 6 3 8 5.
- Pass 3, $f(6\ 3\ 8\ 5)$: minimum 3 $\rightarrow$ 3 6 8 5 $\rightarrow$ 1 2 **3** 6 8 5.

Output: **1 2 3 6 8 5** $\rightarrow$ (D).

Option by option:

- (A) assumes the list ends up fully sorted — three passes fix only the first three positions.
- (B) is after two passes.
- (C) is the list after one pass only (forgetting that the returned suffix is reattached).

Trap: the reassignments $h.next = f(h.next)$ are essential: f may return a *different* node as the new head of the suffix; without reattaching, nodes would be lost.

**Q.20** [NAT · 2 marks · Insertion sort — comparison count]**Answer: 13**

Concept. For each key, the number of evaluations of $A[j] > \text{key}$ equals the number of shifts, plus **one** extra failing comparison if the scan stops on a smaller-or-equal element ($j \geq 0$). If the key travels all the way to index 0, the loop ends on $j \geq 0$ being false and no extra comparison happens.

- $i=1$, key 4: $7 > 4$ shift; $j = -1$ stops $\rightarrow$ shifts 1, comparisons **1**.
- $i=2$, key 10: $7 > 10$ false $\rightarrow$ shifts 0, comparisons **1**.
- $i=3$, key 1: 10, 7, 4 all shift; $j = -1 \rightarrow$ shifts 3, comparisons **3**.
- $i=4$, key 8: $10 > 8$ shift; $7 > 8$ false $\rightarrow$ shifts 1, comparisons **2**.
- $i=5$, key 3: 10, 8, 7, 4 shift; $1 > 3$ false $\rightarrow$ shifts 4, comparisons **5**.
- $i=6$, key 12: $10 > 12$ false $\rightarrow$ comparisons **1**.

Total = $1 + 1 + 3 + 2 + 5 + 1 =$ **13** (shifts = inversions = 9; plus 4 keys that stopped on a smaller element).

Formula: comparisons = inversions + $(n - 1)$ – (number of keys that become the new minimum of the prefix) = $9 + 6 - 2 = 13$.

Trap: adding one comparison per pass to the shifts gives 15 — it overcounts the two keys (4 and 1) that reached index 0.

Mock Test 11

Merge Sort & Recurrences

Difficulty: Moderate

Focus areas: merge sort traces, merge comparisons, recurrences, master theorem

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Recurrences — master theorem (extended case 2)]

Let $T(n) = 3T(n/3) + n \log_2 n$ for $n > 1$, with $T(1) = 1$. Which of the following is a tight asymptotic bound for $T(n)$?

- (A) $\Theta(n \log n)$
- (B) $\Theta(n^2)$
- (C) $\Theta(n^{1.5})$
- (D) $\Theta(n \log^2 n)$

Q.2 [NAT · numerical answer | 1 mark | Merge — counting comparisons]

The standard merge procedure is used to merge the two sorted arrays X and Y below into one sorted array. Each step compares the current front elements of X and Y; as soon as one array is exhausted, the remaining elements of the other are copied without comparisons. The number of element comparisons is _____.

X =

2	9	14	20	31
0	1	2	3	4

Y =

5	6	15	18	22	40	41
0	1	2	3	4	5	6

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Python — closures and late binding]

Consider the following Python program. What is printed?

Python

```
1 fs = [lambda x: x * i for i in range(1, 4)]
2 print([f(2) for f in fs], sum(f(1) for f in fs))
```

- (A) [6, 6, 6] 9
- (B) [2, 4, 6] 6
- (C) [6, 6, 6] 6
- (D) [2, 4, 6] 9

Q.4 [MSQ · one or more correct | 1 mark | Stacks — feasible pop sequences]

The integers 1, 2, 3, 4, 5 are pushed onto an initially empty stack in this order; pops may be interleaved arbitrarily with the pushes, and every popped value is printed. Which of the following output sequences is/are possible?

- (A) 3, 2, 5, 4, 1
- (B) 4, 2, 3, 1, 5
- (C) 1, 5, 4, 2, 3
- (D) 2, 4, 3, 5, 1

Q.5 [NAT · numerical answer | 1 mark | Merge sort — counting recursive calls]

Consider the following merge sort variant, whose base case handles lists of length at most 2 directly. The value printed is _____.

Python

```
1 import heapq
2 calls = 0
3
4 def msort(a):
5     global calls
6     calls += 1
7     if len(a) <= 2:
8         return sorted(a)
9     m = len(a) // 2
10    return list(heapq.merge(msort(a[:m]), msort(a[m:])))
11
12 msort(list(range(13, 0, -1)))
13 print(calls)
```

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Selection sort — swaps]

Selection sort (in pass i , find the minimum of $A[i \dots n-1]$ and swap it with $A[i]$ **only if** it is not already at index i) is applied to $A = [29, 10, 14, 37, 13, 5]$. Which option gives the total number of swaps performed and the array after the first two passes?

- (A) 4 swaps; [5, 10, 13, 37, 14, 29]
- (B) 5 swaps; [5, 10, 14, 37, 13, 29]
- (C) 4 swaps; [5, 10, 14, 37, 13, 29]
- (D) 3 swaps; [5, 10, 29, 37, 13, 14]

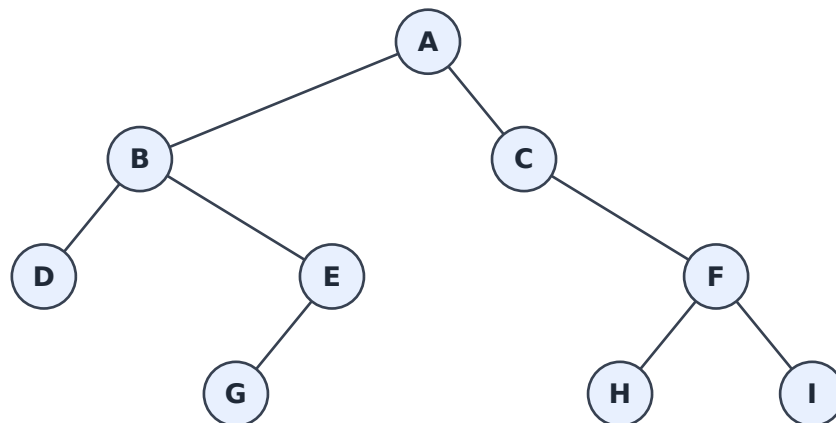
Q.7 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

The keys 27, 45, 13, 31, 22, 40, 8, 63, 17 are inserted in that order into a hash table with 9 slots using $h(k) = k \bmod 9$ and separate chaining; each new key is appended at the **end** of its chain. Afterwards, every one of the nine keys is searched for once. Counting one key comparison per chain node examined, the total number of key comparisons over all nine successful searches is _____.

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Binary trees — traversals]

Consider the binary tree shown below (G is the **left** child of E, and H, I are the left and right children of F). Which of the following statements is/are TRUE?



- (A) The post-order traversal is D, G, E, B, H, I, F, C, A
- (B) The pre-order traversal is A, B, D, E, G, C, F, H, I
- (C) Exactly three nodes have exactly one child
- (D) The in-order traversal is D, B, E, G, A, C, H, F, I

Q.9 [MCQ · single correct | 1 mark | Graph theory — handshaking lemma]

A simple undirected graph G is required to have 12 vertices: 5 vertices of degree 3, 4 vertices of degree 4 and 3 vertices of degree 2. The number of edges of G is

- (A) 18
- (B) No such graph exists
- (C) 37
- (D) 19

Q.10 [NAT · numerical answer | 1 mark | Linked lists — recursive merge of sorted lists]

Two sorted singly linked lists (shown below) are merged by the recursive function `merge`. The value printed is _____.

Python

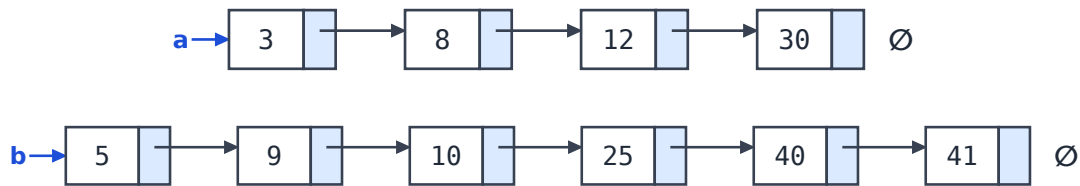
```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def build(xs):
6     h = None
7     for x in reversed(xs):
8         h = Node(x, h)
9     return h
10
11 calls = 0
12 def merge(a, b):
13     global calls
14     calls += 1
15     if a is None: return b
16     if b is None: return a
17     if a.v <= b.v:
18         a.nxt = merge(a.nxt, b)
19         return a
20     b.nxt = merge(a, b.nxt)
21     return b
  
```

```

22
23 merge(build([3, 8, 12, 30]), build([5, 9, 10, 25, 40, 41]))
24 print(calls)

```



Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Bottom-up merge sort – pass trace]

Bottom-up (iterative) merge sort is applied to the array A below. In the pass with run width $w = 1$, 2, 4, 8, ..., adjacent runs $A[i \dots i+w-1]$ and $A[i+w \dots i+2w-1]$ are merged for $i = 0, 2w, 4w, \dots$; a final run with no partner is left unchanged. What is the array immediately **after the pass with $w = 2$** ?

A =	27	4	19	33	8	15	2	41	11	6
	0	1	2	3	4	5	6	7	8	9

- (A) [4, 19, 27, 33, 2, 8, 15, 41, 11, 6]
- (B) [4, 27, 19, 33, 8, 15, 2, 41, 6, 11]
- (C) [2, 4, 8, 15, 19, 27, 33, 41, 6, 11]
- (D) [4, 19, 27, 33, 2, 8, 15, 41, 6, 11]

Q.12 [NAT · numerical answer | 2 marks | Merge sort – worst-case comparison recurrence]

For top-down merge sort that splits n elements into $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, the worst-case number of element comparisons $W(n)$ satisfies

$W(1) = 0$, $W(n) = W(\lfloor n/2 \rfloor) + W(\lceil n/2 \rceil) + n - 1$ for $n \geq 2$.

The value of $W(13)$ is _____.

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Recurrences – which are $\Theta(n \log n)$?]

Which of the following recurrences has/have the solution $T(n) = \Theta(n \log n)$? (Assume $T(n) = \Theta(1)$ for small n .)

- (A) $T(n) = 2T(n/2) + 5n$
- (B) $T(n) = 4T(n/2) + n \log n$
- (C) $T(n) = T(n/3) + T(2n/3) + n$
- (D) $T(n) = 8T(n/4) + n^{1.5}$

Q.14 [NAT · numerical answer | 2 marks | Merge sort — counting during merge (inversions)]

Consider the following Python program. The value printed is _____.

Python

```
1 def sort_count(a):
2     if len(a) < 2:
3         return a, 0
4     m = len(a) // 2
5     L, x = sort_count(a[:m])
6     R, y = sort_count(a[m:])
7     out, i, j, c = [], 0, 0, 0
8     while i < len(L) and j < len(R):
9         if L[i] <= R[j]:
10            out.append(L[i]); i += 1
11        else:
12            out.append(R[j]); j += 1
13            c += len(L) - i
14    return out + L[i:] + R[j:], x + y + c
15
16 print(sort_count([6, 2, 9, 4, 1, 8, 3, 7])[1])
```

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Recurrences — complexity of a recursive function]

Consider the following Python function. Which of the following best describes its running time as a function of n ?

Python

```
1 def work(n):
2     if n <= 1:
3         return 1
4     t, i = 0, 1
5     while i < n:
6         t += 1
7         i *= 2
8     a = work(n // 2)
9     b = work(n // 2)
10    c = work(n // 2)
11    return a + b + c + t
```

- (A) $\Theta(n^{\log_2 3})$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n^{\log_2 3} \cdot \log n)$
- (D) $\Theta(n^2)$

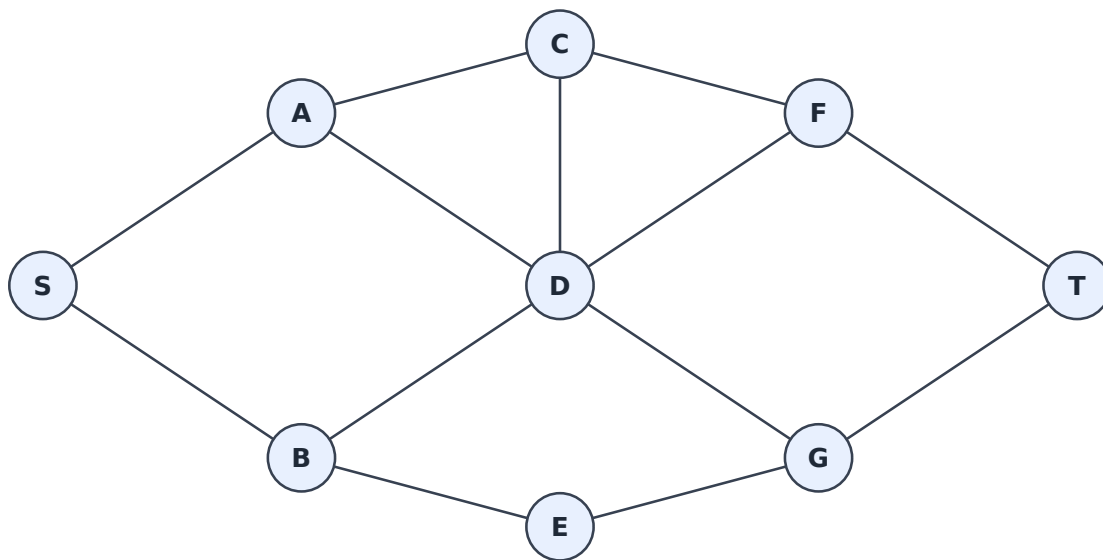
Q.16 [MSQ · one or more correct | 2 marks | Heaps — repeated insertion into a min-heap]

The keys 35, 22, 41, 18, 9, 27, 14, 30 are inserted one at a time, in that order, into an initially empty binary **min-heap** stored in an array (0-indexed; each insertion appends the key and sifts it up). A swap is one child-parent exchange. Which of the following statements is/are TRUE?

- (A) The insertions perform 7 swaps in total
- (B) The final heap array is [9, 18, 14, 30, 22, 41, 27, 35]
- (C) Bottom-up build-heap applied to the array [35, 22, 41, 18, 9, 27, 14, 30] produces the same final array
- (D) After one DELETE-MIN on the final heap, the root holds 14

Q.17 [NAT · numerical answer | 2 marks | BFS — counting shortest paths]

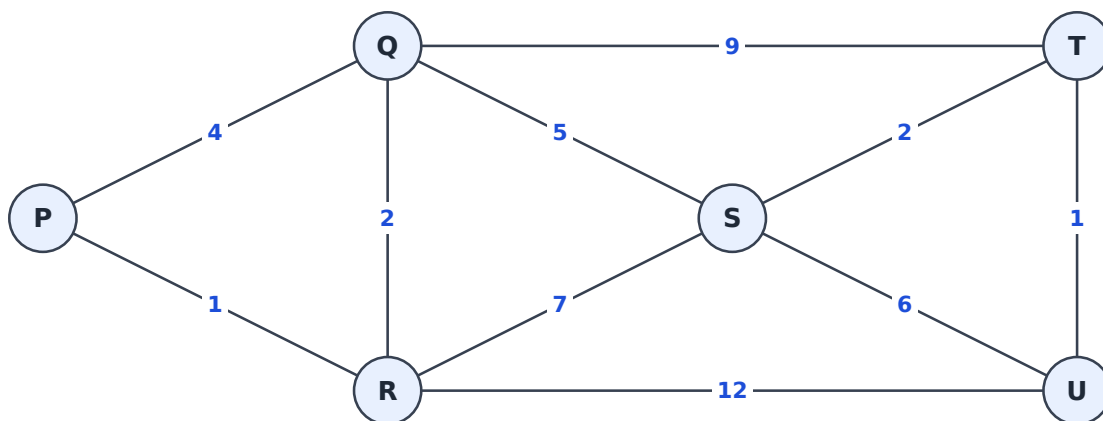
In the unweighted undirected graph below, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Dijkstra — order of finalisation]

Dijkstra's algorithm is run from P on the weighted undirected graph below. Which option gives the order in which vertices are removed from the priority queue (finalised), together with the final distance $d(U)$?



- (A) P, R, Q, S, U, T with $d(U) = 13$
- (B) P, Q, R, S, T, U with $d(U) = 11$
- (C) P, R, Q, S, T, U with $d(U) = 11$
- (D) P, R, Q, T, S, U with $d(U) = 12$

Q.19 [MCQ · single correct | 2 marks | Python — dictionaries and stable sorting]

Consider the following Python program. What is printed?

Python

```
1 text = ("data beats models but more data beats "
2         "better models and data wins")
3 freq = {}
4 for w in text.split():
5     freq[w] = freq.get(w, 0) + 1
6 r = sorted(freq, key=lambda w: (-freq[w], len(w)))
7 print(r[2:6])
```

- (A) ['models', 'and', 'but', 'wins']
- (B) ['models', 'but', 'and', 'more']
- (C) ['models', 'and', 'but', 'more']
- (D) ['beats', 'models', 'but', 'and']

Q.20 [MSQ · one or more correct | 2 marks | Merge sort — stability]

Consider the following Python merge sort, applied to records (key, tag). Which of the following statements is/are TRUE?

Python

```
1 def msort(a):
2     if len(a) <= 1:
3         return a
4     m = len(a) // 2
5     L, R = msort(a[:m]), msort(a[m:])
6     out = []
7     while L and R:
8         if L[0][0] < R[0][0]:
9             out.append(L.pop(0))
10        else:
11            out.append(R.pop(0))
12    return out + L + R
13
14 recs = [(2, 'a'), (1, 'b'), (2, 'c'), (1, 'd'), (2, 'e')]
15 res = msort(recs)
16 print(''.join(t for _, t in res))
```

- (A) The list `recs` is modified by the call `msort(recs)`
- (B) The program prints `dbeca`
- (C) In the output, the records with key 2 appear in the exact reverse of their input order
- (D) If `<` is replaced by `<=`, the program prints `bdace`

Answer Key — Mock Test 11

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Recurrences — master theorem (extended case 2)
2	NAT	1	10	Merge — counting comparisons
3	MCQ	1	(A)	Python — closures and late binding
4	MSQ	1	(A), (D)	Stacks — feasible pop sequences
5	NAT	1	15	Merge sort — counting recursive calls
6	MCQ	1	(C)	Selection sort — swaps
7	NAT	1	19	Hashing — separate chaining
8	MSQ	1	(A), (B)	Binary trees — traversals
9	MCQ	1	(B)	Graph theory — handshaking lemma
10	NAT	1	9	Linked lists — recursive merge of sorted lists
11	MCQ	2	(D)	Bottom-up merge sort — pass trace
12	NAT	2	37	Merge sort — worst-case comparison recurrence
13	MSQ	2	(A), (C)	Recurrences — which are $\Theta(n \log n)$?
14	NAT	2	14	Merge sort — counting during merge (inversions)
15	MCQ	2	(A)	Recurrences — complexity of a recursive function
16	MSQ	2	(B), (D)	Heaps — repeated insertion into a min-heap
17	NAT	2	6	BFS — counting shortest paths
18	MCQ	2	(C)	Dijkstra — order of finalisation
19	MCQ	2	(B)	Python — dictionaries and stable sorting
20	MSQ	2	(B), (C), (D)	Merge sort — stability

Detailed Solutions — Mock Test 11

Q.1 [MCQ · 1 mark · Recurrences — master theorem (extended case 2)]

Answer: (D)

Compare $f(n) = n \log n$ with $n^{\log_b a} = n^{\log_3 3} = n$. The two differ only by a log factor: $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with $k = 1$. The extended case 2 of the master theorem then gives $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n \log^2 n)$.

Recursion-tree check ($n = 3^m$): level i has 3^i subproblems of size $n/3^i$, each costing $(n/3^i) \log(n/3^i)$, so level i costs $n \cdot \log(n/3^i) = n(m - i) \log 3$. Summing $i = 0 \dots m - 1$ gives $n \log 3 \cdot m(m + 1)/2 = \Theta(n \log^2 n)$.

Option analysis:

- (A) $\Theta(n \log n)$ would be correct for $f(n) = n$ (plain case 2); the extra log in f adds one more log factor overall.
- (C) and (B) would need $f(n)$ to be polynomially larger than n , e.g. $n^{1.5}$.
- (D) **Correct.**

Trap: $n \log n$ is *not* polynomially larger than n , so case 3 does not apply.

Q.2 [NAT · 1 mark · Merge — counting comparisons]

Answer: 10

Every comparison outputs exactly one element, so #comparisons = (number of elements output before one array runs out). Merging arrays of sizes p and q needs between $\min(p, q)$ and $p + q - 1$ comparisons.

- 2 vs 5 $\rightarrow$ 2
- 9 vs 5 $\rightarrow$ 5
- 9 vs 6 $\rightarrow$ 6
- 9 vs 15 $\rightarrow$ 9
- 14 vs 15 $\rightarrow$ 14
- 20 vs 15 $\rightarrow$ 15
- 20 vs 18 $\rightarrow$ 18
- 20 vs 22 $\rightarrow$ 20
- 31 vs 22 $\rightarrow$ 22
- 31 vs 40 $\rightarrow$ 31 — X is now exhausted.

The remaining 40, 41 are copied directly. Comparisons = **10**.

Shortcut: X runs out when its last element 31 is output. Everything ≤ 31 in Y (5, 6, 15, 18, 22 $\rightarrow$ 5 elements) is output before it, so comparisons = $5 + 5 = 10$.

Trap: answering the worst case $p + q - 1 = 11$; the two largest elements of Y are never compared.

Q.3 [MCQ · 1 mark · Python — closures and late binding]

Answer: (A)

A lambda does not capture the *value* of a free variable; it captures the variable itself and looks it up **when it is called** (late binding). All three lambdas refer to the same comprehension variable i .

- After the comprehension finishes, i is 3 (the last value of `range(1, 4)`).
- `[f(2) for f in fs]` $\rightarrow$ each lambda computes $2 \times 3 \rightarrow [6, 6, 6]$.
- `sum(f(1) for f in fs)` $\rightarrow 3 + 3 + 3 = 9$.

Output: `[6, 6, 6]` 9 $\rightarrow$ (A).

- (B) assumes each lambda froze its own i (that requires `lambda x, i=i: x * i`); its sum would then be $1 + 2 + 3 = 6$.

- (C) mixes late binding for the list with early binding for the sum.
- (D) mixes them the other way.

Tip: the comprehension has its own scope in Python 3, but the closures all share that one cell for `i`.

Q.4 [MSQ · 1 mark · Stacks — feasible pop sequences]

Answer: (A), (D)

Rule: when x is popped, every value smaller than x that is still on the stack must come out later in **decreasing** order (they are stacked in increasing order). Equivalently, a sequence is infeasible iff it contains a pattern $\dots c \dots a \dots b \dots$ with $a < b < c$ (a 312-pattern).

- (A) 3, 2, 5, 4, 1: push 1, 2, 3, pop 3, pop 2, push 4, 5, pop 5, pop 4, pop 1. **Possible.**
- (B) 4, 2, 3, 1, 5: when 4 is popped the stack holds 1, 2, 3 (3 on top), so 2 cannot come out before 3. **Impossible** (pattern 4, 2, 3).
- (C) 1, 5, 4, 2, 3: after popping 5 and 4 the stack holds 2, 3 with 3 on top, so 2 cannot precede 3. **Impossible** (pattern 5, 2, 3).
- (D) 2, 4, 3, 5, 1: push 1, 2, pop 2, push 3, 4, pop 4, pop 3, push 5, pop 5, pop 1. **Possible.**

Tip: scan for any later pair that appears in increasing order but both smaller than an earlier element — that pair kills the sequence.

Q.5 [NAT · 1 mark · Merge sort — counting recursive calls]

Answer: 15

The number of calls $C(n)$ satisfies $C(n) = 1$ for $n \leq 2$ and $C(n) = 1 + C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil)$ otherwise; it depends only on the length, not on the contents of the list.

- $C(1) = C(2) = 1$
- $C(3) = 1 + C(1) + C(2) = 3$
- $C(4) = 1 + C(2) + C(2) = 3$
- $C(6) = 1 + C(3) + C(3) = 7$
- $C(7) = 1 + C(3) + C(4) = 7$
- $C(13) = 1 + C(6) + C(7) = 15$

Check: the recursion tree is a full binary tree whose leaves are the base-case calls. Leaves: lengths 1, 2, 1, 2 (from the two 3s in $C(6)$) and 1, 2, 2, 2 (from $C(7)$) $\rightarrow$ 8 leaves, so the tree has $2 \cdot 8 - 1 = 15$ nodes.

Trap: the classic formula $2n - 1 = 25$ holds only when the base case is length ≤ 1 .

Q.6 [MCQ · 1 mark · Selection sort — swaps]

Answer: (C)

Selection sort makes at most $n - 1$ swaps; a pass whose minimum is already in place makes none.

- Pass 0: min of all = 5 (index 5) $\rightarrow$ swap with 29 $\rightarrow$ [5, 10, 14, 37, 13, 29] (1).
- Pass 1: min of $A[1..5] = 10$, already at index 1 $\rightarrow$ no swap.
- Pass 2: min of $A[2..5] = 13$ (index 4) $\rightarrow$ [5, 10, 13, 37, 14, 29] (2).
- Pass 3: min of $A[3..5] = 14$ (index 4) $\rightarrow$ [5, 10, 13, 14, 37, 29] (3).
- Pass 4: min of $A[4..5] = 29$ (index 5) $\rightarrow$ [5, 10, 13, 14, 29, 37] (4).

Total = **4 swaps**; after two passes the array is [5, 10, 14, 37, 13, 29] $\rightarrow$ (C).

- (A) shows the array after three passes.
- (B) counts a swap in pass 1 ($n - 1 = 5$ assumes every pass swaps).
- (D) is not a state selection sort can reach.

Tip: selection sort always does exactly $n(n-1)/2 = 15$ comparisons here, whatever the input; only the number of swaps depends on the data.

Q.7 [NAT · 1 mark · Hashing — separate chaining]**Answer: 19**

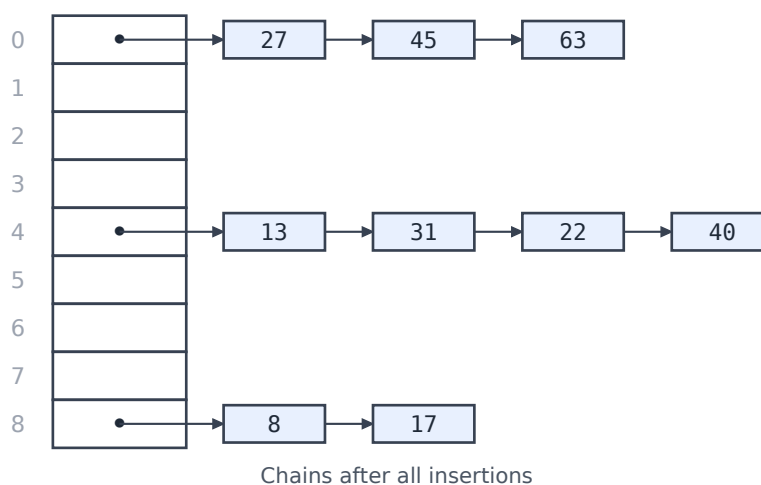
With chaining, a successful search for the j -th key of a chain examines j nodes. So the total cost is $\sum$ over chains of $1 + 2 + \dots + L = L(L + 1)/2$.

Home slots: $27 \rightarrow 0$, $45 \rightarrow 0$, $13 \rightarrow 4$, $31 \rightarrow 4$, $22 \rightarrow 4$, $40 \rightarrow 4$, $8 \rightarrow 8$, $63 \rightarrow 0$, $17 \rightarrow 8$.

- Slot 0: $27 \rightarrow 45 \rightarrow 63$ ($L = 3$) $\rightarrow 1 + 2 + 3 = 6$.
- Slot 4: $13 \rightarrow 31 \rightarrow 22 \rightarrow 40$ ($L = 4$) $\rightarrow 10$.
- Slot 8: $8 \rightarrow 17$ ($L = 2$) $\rightarrow 3$.

Total = $6 + 10 + 3 = 19$ (average $19/9 \approx 2.11$ comparisons per successful search, even though the load factor is only 1).

Trap: using the textbook estimate $1 + \alpha/2 = 1.5$ per search assumes uniform hashing; these keys cluster badly because many are multiples of 9 or $\equiv 4 \pmod{9}$.

**Q.8 [MSQ · 1 mark · Binary trees — traversals]****Answer: (A), (B)**

Pre-order = node, left, right; in-order = left, node, right; post-order = left, right, node.

- (A) Post-order: D, G, E, B (left subtree), H, I, F, C (right subtree), A. **TRUE.**
- (B) Pre-order: A, then subtree B (B, D, E, G), then subtree C (C, F, H, I) $\rightarrow$ A, B, D, E, G, C, F, H, I. **TRUE.**
- (C) Nodes with exactly one child: E (only G) and C (only F) $\rightarrow$ two nodes. **FALSE.**
- (D) In subtree E, G is the *left* child, so in-order lists G before E: D, B, G, E, A, C, H, F, I. The option puts E before G. **FALSE.**

Trap: in-order depends on whether a lone child is left or right; for pre-order and post-order the side of a lone child does not change the sequence — which is why a pre-order + post-order pair cannot always determine a binary tree uniquely.

Q.9 [MCQ · 1 mark · Graph theory — handshaking lemma]**Answer: (B)**

Handshaking lemma: $\sum \deg(v) = 2|E|$, so the degree sum must be **even**; equivalently, the number of odd-degree vertices must be even.

Degree sum = $5 \cdot 3 + 4 \cdot 4 + 3 \cdot 2 = 15 + 16 + 6 = 37$, which is odd. Hence no graph (simple or not) can have this degree sequence $\rightarrow$ (B).

- (A) 18 and (D) 19 are $\lceil 37/2 \rceil$ and $\lfloor 37/2 \rfloor$ — rounding an impossible quantity.
- (C) 37 forgets to divide by 2 at all.

Another view: there are 5 vertices of odd degree (the degree-3 ones), and every graph has an even number of odd-degree vertices.

Tip: always check parity before computing $|E| = (\sum \text{deg})/2$; GATE likes to plant an infeasible degree sequence.

Q.10 [NAT · 1 mark · Linked lists — recursive merge of sorted lists]

Answer: 9

Each non-base call fixes exactly one node of the output and recurses on the rest. The recursion stops with one extra (base-case) call as soon as one list is empty; the remainder of the other list is attached in $O(1)$.

Nodes fixed in order: 3, 5, 8, 9, 10, 12, 25, 30. After 30 is fixed, list a is empty, so the next call `merge(None, 40 → 41)` returns immediately.

- Non-base calls: 8 (one per node up to and including 30).
- Base call: 1.

$\text{calls} = 8 + 1 = 9$.

In general, $\text{calls} = (\text{number of elements} \leq \text{the last element of the list that runs out first, across both lists}) + 1$. The recursion depth is also 9, so merging long lists this way can hit Python's recursion limit (default 1000).

Trap: answering 10 (= total nodes) — 40 and 41 are linked in without any call.

Q.11 [MCQ · 2 marks · Bottom-up merge sort — pass trace]

Answer: (D)

Bottom-up merge sort performs $\lceil \log_2 n \rceil$ passes; after the pass with width w every aligned block of $2w$ elements is sorted (the last block may be shorter).

Pass $w = 1$ (merge pairs):

- $(27, 4) \rightarrow 4, 27$; $(19, 33) \rightarrow 19, 33$; $(8, 15) \rightarrow 8, 15$; $(2, 41) \rightarrow 2, 41$; $(11, 6) \rightarrow 6, 11$.
- $A = [4, 27, 19, 33, 8, 15, 2, 41, 6, 11]$.

Pass $w = 2$ (merge blocks of 2 into blocks of 4):

- $[4, 27] + [19, 33] \rightarrow [4, 19, 27, 33]$.
- $[8, 15] + [2, 41] \rightarrow [2, 8, 15, 41]$.
- $[6, 11]$ has no partner ($i = 8, i + w = 10 = n$) $\rightarrow$ unchanged.
- $A = [4, 19, 27, 33, 2, 8, 15, 41, 6, 11]$.

Later passes: $w = 4$ gives $[2, 4, 8, 15, 19, 27, 33, 41, 6, 11]$ and $w = 8$ finishes the sort.

Option analysis:

- (A) forgets that the $w = 1$ pass already sorted the leftover pair $(11, 6)$.
- (B) is the state after the $w = 1$ pass.
- (C) is the state after the $w = 4$ pass.
- (D) **Correct.**

Trap: unlike top-down merge sort (which splits 10 as $5 + 5$), bottom-up merging uses power-of-two aligned blocks, so the intermediate states differ.

Array after each pass										
	0	1	2	3	4	5	6	7	8	9
input	27	4	19	33	8	15	2	41	11	6
w = 1	4	27	19	33	8	15	2	41	6	11
w = 2	4	19	27	33	2	8	15	41	6	11
w = 4	2	4	8	15	19	27	33	41	6	11
w = 8	2	4	6	8	11	15	19	27	33	41

Q.12 [NAT · 2 marks · Merge sort — worst-case comparison recurrence]**Answer: 37**

The term $n - 1$ is the worst case of one merge of runs whose sizes add up to n (every element except the last is output after a comparison).

Evaluate bottom-up:

- $W(2) = W(1) + W(1) + 1 = 1$
- $W(3) = W(1) + W(2) + 2 = 3$
- $W(4) = W(2) + W(2) + 3 = 5$
- $W(6) = W(3) + W(3) + 5 = 11$
- $W(7) = W(3) + W(4) + 6 = 14$
- $W(13) = W(6) + W(7) + 12 = 11 + 14 + 12 = \mathbf{37}$

Closed form check: $W(n) = n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$. For $n = 13$, $\lceil \log_2 13 \rceil = 4$, so $W(13) = 52 - 16 + 1 = 37$. ✓

Compare with the information-theoretic lower bound $\lceil \log_2 13! \rceil = 33$: merge sort is within a few comparisons of optimal.

Traps:

- Using $n \log_2 n \approx 48.1$ (only an asymptotic estimate).
- Writing $W(13) = 2W(6) + 12 = 34$ — the halves are 6 and **7**, not 6 and 6.

Q.13 [MSQ · 2 marks · Recurrences — which are $\Theta(n \log n)$?]**Answer: (A), (C)**

For $T(n) = aT(n/b) + f(n)$ compare $f(n)$ with $n^{\log_b a}$.

- (A) $a = 2, b = 2 \rightarrow n^{\log_2 2} = n$; $f = 5n = \Theta(n) \rightarrow$ case 2 $\rightarrow \Theta(n \log n)$. **TRUE.**
- (B) $a = 4, b = 2 \rightarrow n^2$; $f = n \log n = O(n^{2-\epsilon}) \rightarrow$ case 1 $\rightarrow \Theta(n^2)$. **FALSE.**
- (C) Not in master-theorem form; use the recursion tree. Every level's subproblem sizes add up to at most n , so each full level costs n . The shortest root-to-leaf path (always $n/3$) has $\log_3 n$ levels and the longest (always $2n/3$) has $\log_{3/2} n$ levels. So $n \log_3 n \leq T(n) \leq n \log_{3/2} n$, i.e. $\Theta(n \log n)$. **TRUE.**
- (D) $a = 8, b = 4 \rightarrow n^{\log_4 8} = n^{1.5} = f(n) \rightarrow$ case 2 $\rightarrow \Theta(n^{1.5} \log n)$. **FALSE** (it has a log factor, but the polynomial part is $n^{1.5}$).

Trap: in (D) the matching of $f(n)$ with $n^{\log_b a}$ produces the “log n ” factor, tempting a hasty $\Theta(n \log n)$. In (C) an unbalanced split still gives $n \log n$ because the split is a constant fraction.

Q.14 [NAT · 2 marks · Merge sort — counting during merge (inversions)]**Answer: 14**

When $R[j]$ is output before the remaining $L[i \dots]$, $R[j]$ is smaller than all $\text{len}(L) - i$ elements still waiting in L , each of which stood **before** it in the original array. So c counts exactly the inversions that cross the split, and the function returns the total number of **inversions** (pairs $i < j$ with $a[i] > a[j]$).

Direct count for $[6, 2, 9, 4, 1, 8, 3, 7]$:

- $6 > 2, 4, 1, 3 \rightarrow 4$
- $2 > 1 \rightarrow 1$
- $9 > 4, 1, 8, 3, 7 \rightarrow 5$
- $4 > 1, 3 \rightarrow 2$
- $1 \rightarrow 0$
- $8 > 3, 7 \rightarrow 2$
- $3 \rightarrow 0$

Total = $4 + 1 + 5 + 2 + 2 = 14$.

Cross-check via the recursion: $[6, 2, 9, 4]$ has 3 inversions $((6,2), (6,4), (9,4))$, $[1, 8, 3, 7]$ has 2 $((8,3), (8,7))$, and the top-level merge of $[2, 4, 6, 9]$ with $[1, 3, 7, 8]$ adds 4 (when 1 is output) + 3 (for 3) + 1 (for 7) + 1 (for 8) = 9; $3 + 2 + 9 = 14$.

Trap: adding 1 per out-of-order step (instead of $\text{len}(L) - i$) counts only the number of times a right element wins, not the inversions.

Q.15 [MCQ · 2 marks · Recurrences — complexity of a recursive function]**Answer: (A)**

The while-loop doubles i , so it runs $\lceil \log_2 n \rceil$ times. Three recursive calls on $n/2$ follow, giving $T(n) = 3T(n/2) + \Theta(\log n)$.

Master theorem: $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$. The driving function $f(n) = \log n$ is $O(n^{1.585 - \epsilon})$ for, e.g., $\epsilon = 0.5$, so **case 1** applies and $T(n) = \Theta(n^{\log_2 3})$.

Intuition: the recursion tree has $3^{\log_2 n} = n^{\log_2 3}$ leaves, each $\Theta(1)$, and the internal work (a log at each node) is dominated by the leaf count since the number of nodes grows geometrically by a factor of 3 per level.

Option analysis:

- (A) **Correct.**
- (B) would need 2 calls and linear work per call.
- (C) would require $f(n) = \Theta(n^{\log_2 3})$ (case 2).
- (D) would need 4 calls on $n/2$.

Trap: storing the three results in a, b, c does **not** memoise anything — each call recomputes the same subproblem. Replacing them by a single call times 3 would give $T(n) = T(n/2) + \log n = \Theta(\log^2 n)$.

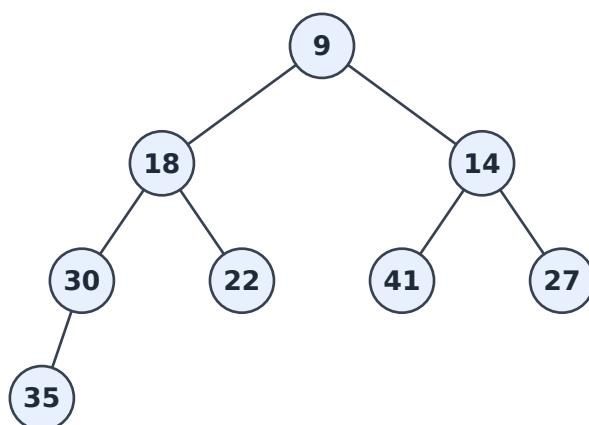
Q.16 [MSQ · 2 marks · Heaps — repeated insertion into a min-heap]**Answer: (B), (D)**

Insertion = append at the end, then swap upward while the parent is larger.

- $35 \rightarrow [35]$
- $22 \rightarrow$ swap with 35 $\rightarrow [22, 35]$ (1)
- $41 \rightarrow [22, 35, 41]$ (0)
- $18 \rightarrow$ swap with 35, then with 22 $\rightarrow [18, 22, 41, 35]$ (2)
- $9 \rightarrow$ swap with 22, then with 18 $\rightarrow [9, 18, 41, 35, 22]$ (2)

- 27 → swap with 41 → [9, 18, 27, 35, 22, 41] (1)
- 14 → swap with 27 → [9, 18, 14, 35, 22, 41, 27] (1)
- 30 → swap with 35 → [9, 18, 14, 30, 22, 41, 27, 35] (1)
- (A) Swaps = 1 + 0 + 2 + 2 + 1 + 1 + 1 = 8, not 7. **FALSE.**
- (B) **TRUE.**
- (C) Bottom-up build-heap (sift-down at $i = 3, 2, 1, 0$) gives [9, 18, 14, 30, 22, 27, 41, 35]: at $i = 2$ the key 41 is swapped with the *smaller* child 14, leaving 27 at index 5. The arrays differ at indices 5, 6. **FALSE.**
- (D) DELETE-MIN moves 35 to the root: [35, 18, 14, 30, 22, 41, 27]; 35 swaps with the smaller child 14, then with 27 → [14, 18, 27, 30, 22, 41, 35]. Root = 14. **TRUE.**

Trap: the heap built by n insertions need not equal the one built bottom-up — both are valid heaps on the same keys.



Min-heap after the eight insertions

Q.17 [NAT · 2 marks · BFS — counting shortest paths]

Answer: 6

Run BFS from S and, for every vertex v , let $\sigma(v)$ be the number of shortest paths. $\sigma(S) = 1$ and $\sigma(v) = \sum \sigma(u)$ over neighbours u with $\text{dist}(u) = \text{dist}(v) - 1$. Edges between vertices on the **same** level never lie on a shortest path.

- Level 0: S ($\sigma = 1$).
- Level 1: A ($\sigma = 1$), B ($\sigma = 1$).
- Level 2: C (from A) $\sigma = 1$; D (from A and B) $\sigma = 2$; E (from B) $\sigma = 1$.
- Level 3: F (from C, D) $\sigma = 1 + 2 = 3$; G (from D, E) $\sigma = 2 + 1 = 3$.
- Level 4: T (from F, G) $\sigma = 3 + 3 = 6$.

The edge $C-D$ joins two level-2 vertices and is ignored.

Listing them: $S-A-C-F-T$, $S-A-D-F-T$, $S-B-D-F-T$, $S-A-D-G-T$, $S-B-D-G-T$, $S-B-E-G-T$.

Trap: counting paths that use $C-D$ (e.g. $S-A-C-D-F-T$) — they have length 5, not 4.

BFS distance and path count σ

	S	A	B	C	D	E	F	G	T
dist	0	1	1	2	2	2	3	3	4
σ	1	1	1	1	2	1	3	3	6

Q.18 [MCQ · 2 marks · Dijkstra — order of finalisation]**Answer: (C)**

Dijkstra always finalises the vertex with the smallest tentative distance; with non-negative weights the finalisation order is the order of increasing true distance.

- Extract P (0): $Q = 4$, $R = 1$.
- Extract R (1): $Q = \min(4, 1 + 2) = 3$, $S = 8$, $U = 13$.
- Extract Q (3): $S = \min(8, 3 + 5) = 8$ (tie, unchanged), $T = 12$.
- Extract S (8): $T = \min(12, 8 + 2) = 10$, $U = \min(13, 8 + 6) = 13$.
- Extract T (10): $U = \min(13, 10 + 1) = 11$.
- Extract U (11).

Order: P, R, Q, S, T, U and $d(U) = 11 \rightarrow (C)$. S is reached with cost 8 along two paths, P-R-S ($1 + 7$) and P-R-Q-S ($3 + 5$); either way S-T-U adds $2 + 1 = 3$, so $d(U) = 11$ — cheaper than the direct edges R-U (13) and S-U (14).

- (A) keeps the direct edge R-U (13) and finalises U too early.
- (B) extracts Q before R although $d(R) = 1 < 3$.
- (D) finalises T (12) before S (8) — T's value 12 is only tentative at that time.

Trap: the edge P-Q (4) is not Q's final distance; it improves to 3 through R.

Q.19 [MCQ · 2 marks · Python — dictionaries and stable sorting]**Answer: (B)**

Since Python 3.7 a dict remembers **insertion order**, and **sorted** is **stable**: items with equal keys keep their original relative order.

Frequencies (insertion order): data 3, beats 2, models 2, but 1, more 1, better 1, and 1, wins 1.

Sort key = ($-\text{frequency}$, length):

- data (-3 , 4)
- beats (-2 , 5), models (-2 , 6)
- frequency-1 words by length: but (3), and (3), more (4), wins (4), better (6). Ties but/and and more/wins keep insertion order: but before and, more before wins.

$r = [\text{data}, \text{beats}, \text{models}, \text{but}, \text{and}, \text{more}, \text{wins}, \text{better}]$; $r[2:6] = [\text{'models'}, \text{'but'}, \text{'and'}, \text{'more'}] \rightarrow (B)$.

- (A) breaks the but/and tie alphabetically and also mis-orders more/wins.
- (C) breaks the but/and tie alphabetically — **sorted** never compares the words themselves here, only the key tuples.
- (D) uses $r[1:5]$ (off-by-one in the slice start).

Tip: to break ties alphabetically you must add **w** to the key tuple explicitly, e.g. `key=lambda w: (-freq[w], len(w), w)`; then 'and' would precede 'but'.

Q.20 [MSQ · 2 marks · Merge sort — stability]**Answer: (B), (C), (D)**

Merge sort is stable only if, on equal keys, the merge takes the element from the **left** run. With **<**, ties go to the right run, so equal keys can be reordered.

Trace (split 2 | 3):

- Left [(2,a), (1,b)]: $2 < 1$ false $\rightarrow b$, then $a \rightarrow [b, a]$.
 - Right [(2,c)] | [(1,d), (2,e)]: inner merge gives $[d, e]$; then c vs d : $2 < 1$ false $\rightarrow d$; c vs e : $2 < 2$ false $\rightarrow e$; then $c \rightarrow [d, e, c]$.
 - Top: b vs d : $1 < 1$ false $\rightarrow d$; b vs e : $1 < 2 \rightarrow b$; a vs e : $2 < 2$ false $\rightarrow e$; a vs c : false $\rightarrow c$; then a .
- Output **dbeca**.

- (A) FALSE — `msort` only slices (`a[:m]` makes copies) and pops from those new lists; a 1-element `recs` would be returned as is, but nothing ever mutates it.
- (B) TRUE.
- (C) TRUE — input order a, c, e; output order e, c, a.
- (D) With `<=` ties go left and the sort is stable: key-1 records b, d then key-2 records a, c, e → `bdace`. TRUE.

Trap: the sorted keys are correct either way; only the tags reveal the instability. (Also note `pop(0)` is $O(\text{len})$ on Python lists, so this version is slower than index-based merging.)

Mock Test 12

Quicksort & Partitioning

Difficulty: Moderate

Focus areas: Lomuto/Hoare partition, pivot choice, best/worst case

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — slice assignment and aliasing]

Consider the following Python program. What is printed?

Python

```
1 a = [3, 1, 4, 1, 5, 9, 2]
2 b = a
3 c = a[:]
4 b[1:3] = [7]
5 c[::2] = c[1::2] + [0]
6 print(len(a), a[2], c)
```

- (A) 6 1 [1, 1, 1, 1, 9, 9, 0]
- (B) 7 4 [1, 1, 1, 1, 9, 9, 0]
- (C) 6 1 [3, 1, 4, 1, 5, 9, 2]
- (D) A `ValueError` is raised

Q.2 [NAT · numerical answer | 1 mark | Quicksort — Lomuto partition swaps]

The Lomuto partition below is applied once to $A = [9, 4, 12, 7, 15, 3, 10, 8]$ with $lo = 0$ and $hi = 7$ (pivot = last element). The total number of times the statement marked `#S` or `#F` is executed (count every execution, even if $i == j$) is ____.

Python

```
1 def partition(a, lo, hi):
2     x, i = a[hi], lo - 1
3     for j in range(lo, hi):
4         if a[j] <= x:
5             i += 1
6             a[i], a[j] = a[j], a[i]      #S
7     a[i + 1], a[hi] = a[hi], a[i + 1]  #F
8     return i + 1
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Quicksort — Hoare partition]

Hoare's partition scheme (pivot $p = a[lo]$) is shown below. It is called once on $a = [6, 11, 2, 9, 6, 1, 14, 4, 8]$ with $lo = 0$, $hi = 8$. What are the array and the returned index immediately after the call?

Python

```

1 def hoare(a, lo, hi):
2     p, i, j = a[lo], lo - 1, hi + 1
3     while True:
4         i += 1
5         while a[i] < p:
6             i += 1
7         j -= 1
8         while a[j] > p:
9             j -= 1
10        if i >= j:
11            return j
12        a[i], a[j] = a[j], a[i]
```

- (A) [4, 1, 2, 6, 9, 11, 14, 6, 8], returns 3
- (B) [4, 1, 2, 6, 6, 11, 14, 9, 8], returns 4
- (C) [4, 1, 2, 6, 9, 11, 14, 6, 8], returns 4
- (D) [1, 4, 2, 6, 6, 9, 14, 11, 8], returns 4

Q.4 [MSQ · one or more correct | 1 mark | Quicksort — identifying the pivot]

The array below shows the state of an array of distinct integers after the **first** partitioning step of some quicksort (the pivot is placed at its final sorted position; nothing else is assumed about the partition scheme). Which of the following elements could have been the pivot?

4	2	7	5	9	12	10	15	13	18
0	1	2	3	4	5	6	7	8	9

Array after the first partition

- (A) 9
- (B) 18
- (C) 7
- (D) 12

Q.5 [NAT · numerical answer | 1 mark | Python — deque as a double-ended queue]

Consider the following Python program. The value printed is _____.

Python

```

1 from collections import deque
2
3 d = deque([5, 2, 8])
4 for k in range(1, 6):
5     if k % 2:
6         d.append(d.popleft() * k)
7     else:
8         d.appendleft(d.pop() - k)
9 print(d[0] - d[-1])
```

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Circular linked lists — elimination game]

Nine people numbered 1..9 are stored in a circular singly linked list. The program repeatedly skips ahead and removes a node. What does it print?

Python

```

1 class Node:
2     def __init__(self, v):
3         self.v, self.next = v, None
4
5 n, k = 9, 4
6 head = Node(1)
7 cur = head
8 for v in range(2, n + 1):
9     cur.next = Node(v)
10    cur = cur.next
11 cur.next = head          # cur is the tail
12 out = []
13 while cur.next is not cur:
14     for _ in range(k - 1):
15         cur = cur.next
16     out.append(cur.next.v)
17     cur.next = cur.next.next
18 print(out[4], cur.v)

```

- (A) 6 5
- (B) 9 1
- (C) 6 1
- (D) 5 1

Q.7 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

Keys 25, 36, 14, 47, 58, 3 are inserted in this order into an initially empty table of size 11 using quadratic probing: probe i ($i = 0, 1, 2, \dots$) examines slot $(k + i^2) \bmod 11$. Next, key 69 is to be inserted and probes $i = 0, 1, \dots, 10$ are tried. The number of **distinct** slots examined by these 11 probes for key 69 is _____.

Answer: _____ (enter a numerical value)

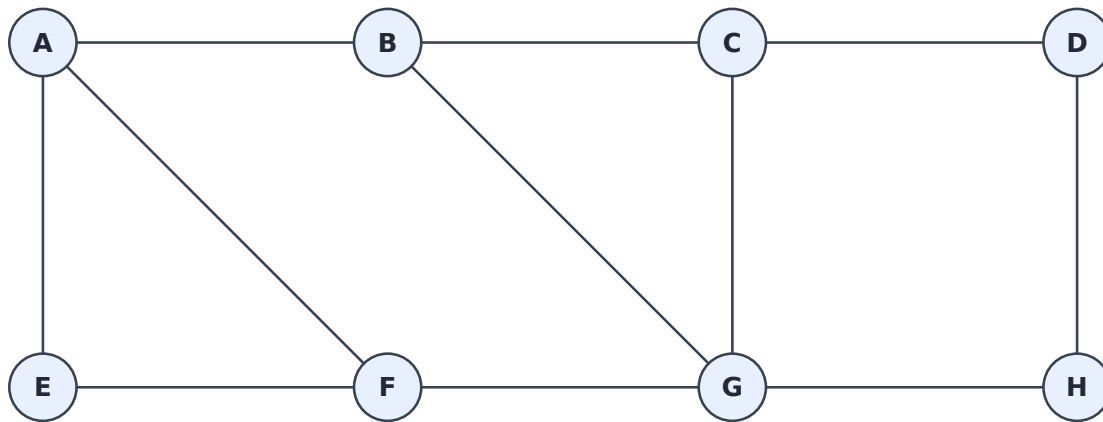
Q.8 [MCQ · single correct | 1 mark | Binary search trees — search paths]

Each sequence below is claimed to be the sequence of keys compared, in order, while searching for the key 47 in some binary search tree containing distinct integers (the search ends at 47). Which sequence **cannot** occur?

- (A) 90, 25, 80, 30, 70, 40, 60, 47
- (B) 10, 75, 64, 43, 58, 53, 47
- (C) 20, 60, 30, 55, 50, 35, 47
- (D) 52, 15, 48, 22, 39, 50, 47

Q.9 [MCQ · single correct | 1 mark | Graph traversal — BFS order]

Breadth-first search is run on the undirected graph below starting at A. When a vertex is dequeued, its unvisited neighbours are enqueued in **alphabetical order**. The order in which vertices are visited (enqueued) is



- (A) A, B, F, E, C, G, D, H
- (B) A, B, C, D, H, G, F, E
- (C) A, B, E, F, G, C, H, D
- (D) A, B, E, F, C, G, D, H

Q.10 [MSQ · one or more correct | 1 mark | Graph theory — graphical degree sequences]

Which of the following sequences is/are the degree sequence of some **simple** undirected graph (no self-loops, no multi-edges)?

- (A) (5, 5, 4, 3, 2, 1)
- (B) (4, 4, 3, 2, 1, 0)
- (C) (6, 2, 2, 2, 2, 2, 2)
- (D) (3, 3, 3, 3, 2, 2)

Q.11 [MCQ · single correct | 2 marks | Python — list-comprehension quicksort with duplicates]

Consider the following Python program. What is printed?

Python

```

1 def qs(a):
2     if len(a) <= 1:
3         return a
4     p, rest = a[0], a[1:]
5     return (qs([x for x in rest if x < p]) + [p] +
6           qs([x for x in rest if x > p]))
7
8 data = [4, 7, 4, 1, 7, 3, 4]
9 r = qs(data)
10 print(len(r), r[-2:], data is r)

```

- (A) 7 [7, 7] False
- (B) 5 [4, 7] False
- (C) 4 [4, 7] False
- (D) 4 [4, 7] True

Q.12 [NAT · numerical answer | 2 marks | Quicksort — total comparisons (Lomuto)]

Quicksort with the Lomuto partition of Q.2 (pivot = last element of the subarray; one comparison $a[j] \leq x$ per loop iteration) is used to sort $A = [7, 3, 9, 1, 6, 8, 2, 5]$. Recursive calls on subarrays of size 0 or 1 do not partition. The total number of comparisons $a[j] \leq x$ made over the whole sort is _____.

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Quicksort — best/worst case and recurrences]

Which of the following statements is/are TRUE?

- (A) If the pivot is always the second-smallest element of its subarray, quicksort runs in $\Theta(n \log n)$ time
- (B) Quicksort with the Lomuto partition (pivot = last, test $a[j] \leq x$) makes exactly $n(n - 1)/2$ comparisons on an array of n equal keys
- (C) If every partition step splits its subarray in the ratio 1 : 9, quicksort runs in $\Theta(n \log n)$ time
- (D) The recurrence $T(n) = 2T(n/2) + c$, $T(1) = c$, solves to $\Theta(n \log n)$

Q.14 [NAT · numerical answer | 2 marks | Quickselect — partition-based selection]

The function below finds the k -th smallest element (k is a 0-based index) using the Lomuto partition of Q.2. It is called as `select(a, 0, 9, 3)` on $a = [14, 3, 22, 9, 17, 5, 11, 26, 8, 19]$. The total number of comparisons $a[j] \leq x$ performed inside `partition` during this call is _____.

Python

```
1 def select(a, lo, hi, k):
2     if lo == hi:
3         return a[lo]
4     p = partition(a, lo, hi)
5     if p == k:
6         return a[p]
7     if p < k:
8         return select(a, p + 1, hi, k)
9     return select(a, lo, p - 1, k)
```

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Quicksort — three-way (Dutch national flag) partition]

The three-way partition below is applied to $a = [5, 8, 2, 5, 9, 1, 5, 7, 3]$ with pivot value $p = 5$. What are the final array and the values of `lt` and `gt`?

Python

```
1 def three_way(a, p):
2     lt, i, gt = 0, 0, len(a) - 1
3     while i <= gt:
4         if a[i] < p:
5             a[lt], a[i] = a[i], a[lt]
6             lt += 1
7             i += 1
8         elif a[i] > p:
9             a[i], a[gt] = a[gt], a[i]
10            gt -= 1
11        else:
12            i += 1
13    return lt, gt
```

- (A) $[3, 2, 1, 5, 5, 5, 9, 7, 8]$, $lt = 3$, $gt = 5$
- (B) $[3, 2, 1, 5, 5, 5, 7, 9, 8]$, $lt = 3$, $gt = 5$
- (C) $[2, 1, 3, 5, 5, 5, 7, 9, 8]$, $lt = 3$, $gt = 5$
- (D) $[3, 2, 1, 5, 5, 5, 7, 9, 8]$, $lt = 3$, $gt = 6$

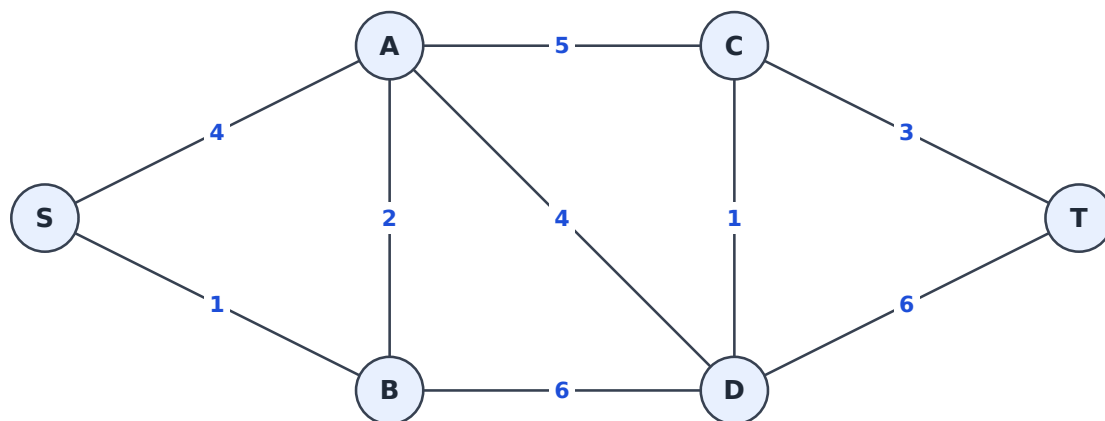
Q.16 [NAT · numerical answer | 2 marks | Merge sort — counting merge comparisons]

Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ and the remaining elements) sorts $[6, 2, 9, 4, 1, 8, 3, 7]$. During a merge, one comparison is made for each element output while both halves are non-empty; once one half is exhausted, the rest is copied without comparisons. The total number of comparisons is _____.

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra with ties]

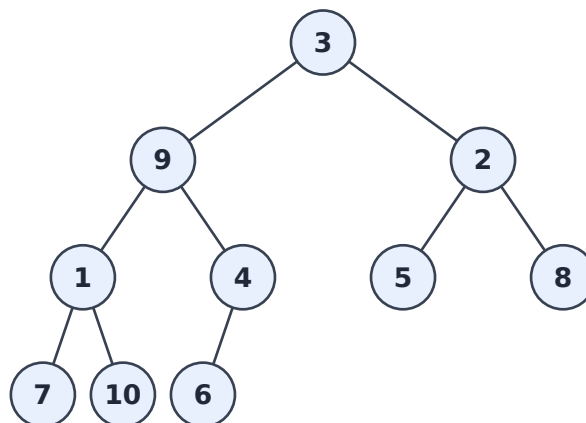
Dijkstra's algorithm is run from S on the weighted undirected graph below. A tentative distance (and the predecessor) is updated only on a **strict** improvement. Which of the following statements is/are TRUE?



- (A) The shortest-path distance from S to T is 11
- (B) There are exactly two distinct shortest paths from S to C
- (C) The vertices are extracted (finalised) in the order S, B, A, D, C, T
- (D) In the shortest-path tree produced, the parent of C is D

Q.18 [NAT · numerical answer | 2 marks | Heaps — bottom-up build-heap]

The array $[3, 9, 2, 1, 4, 5, 8, 7, 10, 6]$ (0-based) is converted into a **max**-heap using the bottom-up build-heap procedure: sift-down is called for $i = \lfloor n/2 \rfloor - 1$ down to 0, and each sift-down repeatedly swaps a node with its larger child while that child is larger. The total number of swaps performed is _____.



Initial array viewed as a complete binary tree

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Insertion sort — comparisons and shifts]

Insertion sort (below) sorts $a = [2, 3, 5, 1, 4, 6, 8, 7]$ in ascending order. A *comparison* is one evaluation of $a[j] > \text{key}$ and a *shift* is one execution of $a[j + 1] = a[j]$. The pair (number of comparisons, number of shifts) is

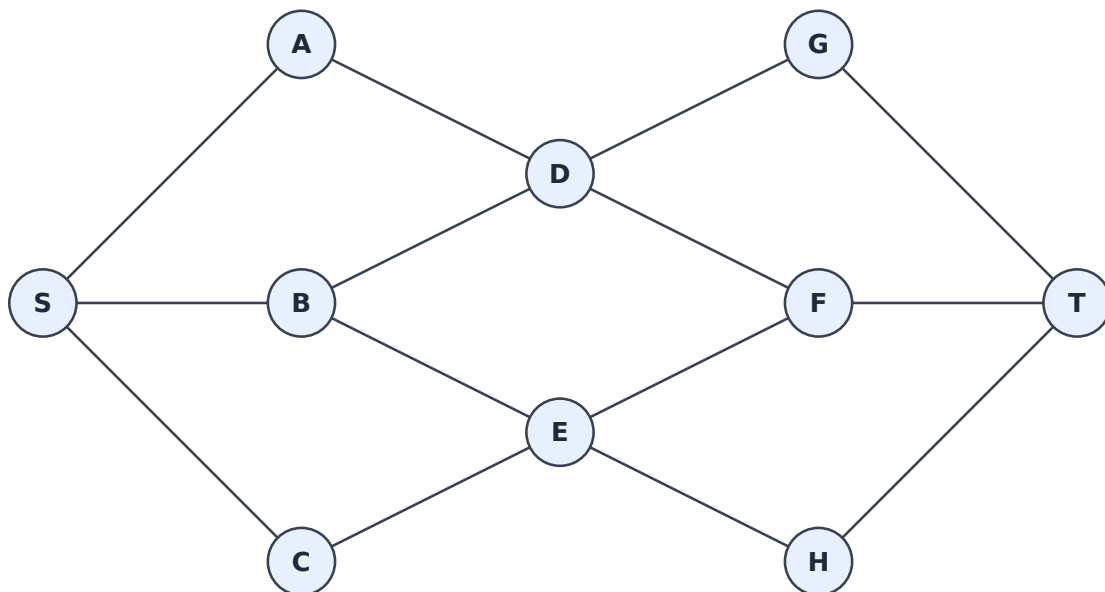
Python

```
1 for i in range(1, len(a)):
2     key, j = a[i], i - 1
3     while j >= 0 and a[j] > key:
4         a[j + 1] = a[j]
5         j -= 1
6     a[j + 1] = key
```

- (A) (12, 5)
- (B) (11, 5)
- (C) (11, 4)
- (D) (16, 5)

Q.20 [MSQ · one or more correct | 2 marks | BFS — counting shortest paths]

Consider the unweighted undirected graph below. A *shortest S-T path* is one with the minimum number of edges. BFS from S enqueues neighbours in alphabetical order. Which of the following statements is/are TRUE?



- (A) Every shortest S-T path passes through E
- (B) There are exactly 8 shortest S-T paths
- (C) Exactly 4 shortest S-T paths pass through D
- (D) In the BFS tree, the parent of T is F

Answer Key — Mock Test 12

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — slice assignment and aliasing
2	NAT	1	4	Quicksort — Lomuto partition swaps
3	MCQ	1	(A)	Quicksort — Hoare partition
4	MSQ	1	(A), (B)	Quicksort — identifying the pivot
5	NAT	1	-23	Python — deque as a double-ended queue
6	MCQ	1	(C)	Circular linked lists — elimination game
7	NAT	1	6	Hashing — quadratic probing
8	MCQ	1	(D)	Binary search trees — search paths
9	MCQ	1	(D)	Graph traversal — BFS order
10	MSQ	1	(C), (D)	Graph theory — graphical degree sequences
11	MCQ	2	(C)	Python — list-comprehension quicksort with duplicates
12	NAT	2	13	Quicksort — total comparisons (Lomuto)
13	MSQ	2	(B), (C)	Quicksort — best/worst case and recurrences
14	NAT	2	18	Quickselect — partition-based selection
15	MCQ	2	(B)	Quicksort — three-way (Dutch national flag) partition
16	NAT	2	17	Merge sort — counting merge comparisons
17	MSQ	2	(A), (C)	Shortest paths — Dijkstra with ties
18	NAT	2	7	Heaps — bottom-up build-heap
19	MCQ	2	(B)	Insertion sort — comparisons and shifts
20	MSQ	2	(B), (C), (D)	BFS — counting shortest paths

Detailed Solutions — Mock Test 12

Q.1 [MCQ · 1 mark · Python — slice assignment and aliasing]

Answer: (A)

Concept: `b = a` makes an alias (same list); `a[:]` makes a shallow copy. Assigning to a *simple* slice may change the length; assigning to an *extended* slice (with a step) requires the right-hand side to have exactly the same number of elements.

- `b[1:3] = [7]` replaces the two elements at indices 1, 2 (values 1, 4) by one element. Since `b` is `a`, now `a = [3, 7, 1, 5, 9, 2]`: `len(a) = 6`, `a[2] = 1`.
- `c` is an independent copy `[3, 1, 4, 1, 5, 9, 2]`.
- `c[::2]` addresses indices 0, 2, 4, 6 (4 slots). The right side is `c[1::2] + [0] = [1, 1, 9] + [0] = 4` values — sizes match, so no error.
- After assignment: `c[0] = 1`, `c[2] = 1`, `c[4] = 9`, `c[6] = 0` → `[1, 1, 1, 1, 9, 9, 0]`.

Options:

- (B) treats `b` as a copy, so `a` would be unchanged.
- (C) assumes `c` aliases `a` (and would anyway not match).
- (D) would happen only if the extended-slice sizes differed (e.g. without `+ [0]`).

Trap: the length check applies only to extended slices — simple slices can shrink or grow.

Q.2 [NAT · 1 mark · Quicksort — Lomuto partition swaps]

Answer: 4

Concept: Lomuto keeps the invariant $a[\text{lo}..i] \leq \text{pivot} < a[i+1..j-1]$. A swap **#S** happens once for every element $\leq \text{pivot}$; the final swap **#F** places the pivot at index $i + 1$.

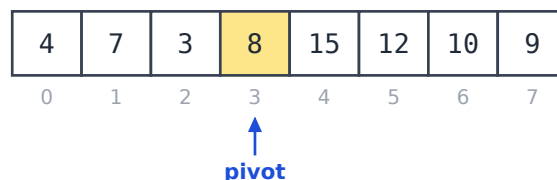
Pivot $x = 8$. Trace (i starts at -1):

- $j = 0$: $9 > 8 \rightarrow$ nothing.
- $j = 1$: $4 \leq 8 \rightarrow i = 0$, swap $a[0] \leftrightarrow a[1] \rightarrow [4, 9, 12, 7, 15, 3, 10, 8]$ (**#S 1**)
- $j = 2$: $12 > 8$.
- $j = 3$: $7 \leq 8 \rightarrow i = 1$, swap $a[1] \leftrightarrow a[3] \rightarrow [4, 7, 12, 9, 15, 3, 10, 8]$ (**#S 2**)
- $j = 4$: $15 > 8$.
- $j = 5$: $3 \leq 8 \rightarrow i = 2$, swap $a[2] \leftrightarrow a[5] \rightarrow [4, 7, 3, 9, 15, 12, 10, 8]$ (**#S 3**)
- $j = 6$: $10 > 8$.
- Final: swap $a[3] \leftrightarrow a[7] \rightarrow [4, 7, 3, 8, 15, 12, 10, 9]$ (**#F**)

Total = $3 + 1 = 4$; the pivot ends at index 3.

Tip: the number of **#S** executions equals the number of non-pivot elements $\leq \text{pivot}$ (here 4, 7, 3) — you can answer without a full trace.

Trap: forgetting the final pivot swap (answer 3).



After partition: $\leq 8 \mid 8 \mid > 8$

Q.3 [MCQ · 1 mark · Quicksort — Hoare partition]**Answer: (A)**

Concept: Hoare's scheme moves i right past elements $< p$ and j left past elements $> p$, swapping when both stop. It guarantees $a[lo..j] \leq p \leq a[j+1..hi]$, but the pivot is **not** necessarily placed at its final position. Elements *equal* to p stop both scans.

Trace with $p = 6$:

- i stops at 0 (6 is not < 6); j moves from 8: $8 > 6 \rightarrow j = 7$ (4) stops. Swap $\rightarrow [4, 11, 2, 9, 6, 1, 14, 6, 8]$.
- $i \rightarrow 1$ (11 stops); $j \rightarrow 6$ ($14 > 6$) $\rightarrow 5$ (1 stops). Swap $\rightarrow [4, 1, 2, 9, 6, 11, 14, 6, 8]$.
- $i \rightarrow 2$ ($2 < 6$) $\rightarrow 3$ (9 stops); $j \rightarrow 4$ (6 stops, not > 6). Swap $\rightarrow [4, 1, 2, 6, 9, 11, 14, 6, 8]$.
- $i \rightarrow 4$ (9 stops); $j \rightarrow 3$ (6 stops). Now $i \geq j \rightarrow$ return **3**.

Check: $a[0..3] = 4, 1, 2, 6$ are all ≤ 6 and $a[4..8] = 9, 11, 14, 6, 8$ are all ≥ 6 . ✓ Note a second 6 sits in the right part — allowed by Hoare's invariant.

Options: (B) and (D) put both 6s next to each other as Lomuto/three-way partitioning would; (C) has the right array but returns i instead of j .

Trap: assuming Hoare leaves the pivot at the returned index.

Q.4 [MSQ · 1 mark · Quicksort — identifying the pivot]**Answer: (A), (B)**

Concept: after partitioning, the pivot is in its final place: every element to its left is smaller and every element to its right is larger. So x at index i is a candidate iff $\max(a[0..i-1]) < x < \min(a[i+1..n-1])$.

Prefix maxima / suffix minima check:

- (A) 9 at index 4: left $\{4, 2, 7, 5\}$, $\max 7 < 9$; right $\{12, 10, 15, 13, 18\}$, $\min 10 > 9$. **Possible.**
- (B) 18 at index 9 (last): everything to its left is smaller and the right side is empty. **Possible** (e.g. Lomuto with the maximum as pivot).
- (C) 7 at index 2: right part contains $5 < 7$. **Not possible.**
- (D) 12 at index 5: right part contains $10 < 12$. **Not possible.**

Also 4, 2, 5, 15, 10, 13 fail; only 9 and 18 qualify.

Trap: forgetting that a pivot can sit at an end of the array — an extreme pivot gives an empty side, which is exactly the worst case of quicksort.

Q.5 [NAT · 1 mark · Python — deque as a double-ended queue]**Answer: -23**

Concept: a deque supports $O(1)$ insertion/removal at both ends: **append/pop** work on the right, **appendleft/popleft** on the left. Odd k rotates the front element to the back (multiplied by k); even k rotates the back element to the front (minus k).

Trace (left $\rightarrow$ right):

- start: $[5, 2, 8]$
- $k = 1$: **popleft** 5, **append** $5 \times 1 = 5 \rightarrow [2, 8, 5]$
- $k = 2$: **pop** 5, **appendleft** $5 - 2 = 3 \rightarrow [3, 2, 8]$
- $k = 3$: **popleft** 3, **append** 9 $\rightarrow [2, 8, 9]$
- $k = 4$: **pop** 9, **appendleft** 5 $\rightarrow [5, 2, 8]$
- $k = 5$: **popleft** 5, **append** 25 $\rightarrow [2, 8, 25]$

$d[0] - d[-1] = 2 - 25 = \mathbf{-23}$.

Trap: mixing up **pop()** (right end) with **popleft()**; with **pop()** on the left the answer would be completely different. Note too that the deque returns to $[5, 2, 8]$ after $k = 4$ — a nice self-check.



Final deque (front on the left)

Q.6 [MCQ · 1 mark · Circular linked lists — elimination game]**Answer: (C)**

Concept: to delete from a singly linked list you need the *predecessor*. *cur* starts at the tail (9), moves $k - 1 = 3$ steps and removes *cur.next* — i.e. every 4th person counting from the person after the last removal.

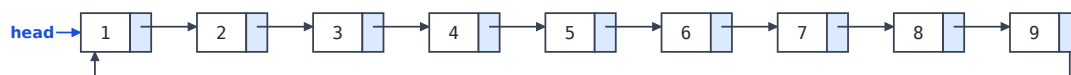
Trace (removed value, list afterwards):

- from 9 move to 3 → remove **4** → 1 2 3 5 6 7 8 9
- from 3 move to 7 → remove **8** → 1 2 3 5 6 7 9
- from 7 move to 2 → remove **3** → 1 2 5 6 7 9
- from 2 move to 7 → remove **9** → 1 2 5 6 7
- from 7 move to 5 → remove **6** → 1 2 5 7
- from 5 move to 2 → remove **5** → 1 2 7
- from 2 move to 2 → remove **7** → 1 2
- from 2 move to 1 → remove **2** → 1

out = [4, 8, 3, 9, 6, 5, 7, 2]; *out*[4] = 6 (5th removal) and the survivor is 1. Output **6 1**.

Options: (B) uses 1-based indexing for *out*[4] (that would be 9, the 4th removal). (A) and (D) report a removed person as the survivor or the wrong removal.

Trap: off-by-one in where counting restarts after a deletion.



Initial circular list; removal order 4, 8, 3, 9, 6, 5, 7, 2

Q.7 [NAT · 1 mark · Hashing — quadratic probing]**Answer: 6**

Concept: with quadratic probing and a prime table size m , $i^2 \bmod m$ takes only $(m + 1)/2$ distinct values, so a key can examine only about half the table — insertion can fail although free slots exist.

Building the table:

- 25 → 3. 36 → 3 (full), 4 → slot 4.
- 14 → 3, 4, 7 → slot 7.
- 47 → 3, 4, 7, $(3 + 9) \bmod 11 = 1$ → slot 1.
- 58 → 3, 4, 7, 1, $(3 + 16) \bmod 11 = 8$ → slot 8.
- 3 → 3, 4, 7, 1, 8, $(3 + 25) \bmod 11 = 6$ → slot 6.

Key 69 (home $69 \bmod 11 = 3$): offsets $i^2 \bmod 11$ for $i = 0..10$ are 0, 1, 4, 9, 5, 3, 3, 5, 9, 4, 1, so the slots are 3, 4, 7, 1, 8, 6, 6, 8, 1, 7, 4 — only **6** distinct slots, all occupied. The insertion fails even though slots 0, 2, 5, 9, 10 are empty.

Trap: assuming 11 probes visit 11 different slots (true for linear probing, false here).

0	
1	47
2	
3	25
4	36
5	
6	3
7	14
8	58
9	
10	

Table before inserting 69: its 6 reachable slots are all full

Q.8 [MCQ · 1 mark · Binary search trees — search paths]

Answer: (D)

Concept: during a search every visited key narrows an open interval (low, high) that must contain the target; each subsequent key must lie inside the current interval.

- (A) $90 \rightarrow (-\infty, 90)$; $25 \rightarrow (25, 90)$; $80 \rightarrow (25, 80)$; $30 \rightarrow (30, 80)$; $70 \rightarrow (30, 70)$; $40 \rightarrow (40, 70)$; $60 \rightarrow (40, 60)$; 47 inside. **Valid.**
- (B) $10 \rightarrow (10, \infty)$; $75 \rightarrow (10, 75)$; $64 \rightarrow (10, 64)$; $43 \rightarrow (43, 64)$; $58 \rightarrow (43, 58)$; $53 \rightarrow (43, 53)$; 47 inside. **Valid.**
- (C) $20 \rightarrow (20, \infty)$; $60 \rightarrow (20, 60)$; $30 \rightarrow (30, 60)$; $55 \rightarrow (30, 55)$; $50 \rightarrow (30, 50)$; $35 \rightarrow (35, 50)$; 47 inside. **Valid.**
- (D) $52 \rightarrow (-\infty, 52)$; $15 \rightarrow (15, 52)$; $48 \rightarrow (15, 48)$; $22 \rightarrow (22, 48)$; $39 \rightarrow (39, 48)$; next key 50 is **not** inside $(39, 48)$ — after going left at 48 every later key must be < 48 . **Invalid.**

Trap: checking only that the sequence zig-zags around 47; you must check every key against *all* previous bounds, not just the immediately preceding key.

Q.9 [MCQ · 1 mark · Graph traversal — BFS order]

Answer: (D)

Concept: BFS visits vertices level by level; within a level the order is inherited from the order in which parents were dequeued, and each parent adds children alphabetically.

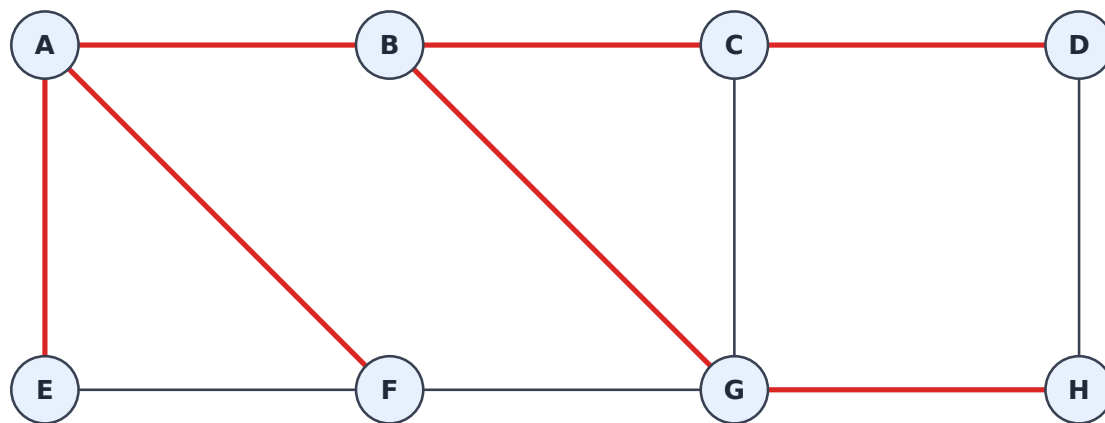
Trace (queue after each dequeue):

- dequeue A $\rightarrow$ enqueue B, E, F $\rightarrow$ [B, E, F]
- dequeue B $\rightarrow$ unvisited nbrs C, G $\rightarrow$ [E, F, C, G]
- dequeue E $\rightarrow$ nbrs A, F already seen $\rightarrow$ [F, C, G]
- dequeue F $\rightarrow$ nbrs A, E, G seen $\rightarrow$ [C, G]
- dequeue C $\rightarrow$ nbr D new $\rightarrow$ [G, D]
- dequeue G $\rightarrow$ nbr H new $\rightarrow$ [D, H]
- dequeue D, H $\rightarrow$ nothing new.

Order: **A, B, E, F, C, G, D, H** (levels: {A}, {B, E, F}, {C, G}, {D, H}).

Options: (B) is a DFS order. (C) lets F (dequeued after B and E) discover G before B does. (A) enqueues A's neighbours out of alphabetical order.

Tip: the level sets are fixed by distances; only the order inside a level depends on tie-breaking.



BFS tree (highlighted edges)

Q.10 [MSQ · 1 mark · Graph theory — graphical degree sequences]**Answer: (C), (D)**

Concept: the degree sum must be even (handshaking lemma), and the Havel-Hakimi test applies: remove the largest degree d and subtract 1 from the next d largest; the sequence is graphical iff this ends in all zeros.

- (A) sum 20 (even), but on 6 vertices two vertices of degree 5 are adjacent to *all* others, so every vertex has degree ≥ 2 — the 1 is impossible. **Not graphical.**
- (B) drop the isolated vertex: (4, 4, 3, 2, 1) on 5 vertices; two degree-4 vertices force every other degree ≥ 2 , contradicting the 1. **Not graphical.**
- (C) sum 18. The degree-6 vertex is adjacent to all six others; each of those needs one more edge among themselves — a perfect matching on 6 vertices works. **Graphical.**
- (D) sum 16. HH: 3,3,3,3,2,2 $\rightarrow$ 2,2,2,2,2 $\rightarrow$ 1,1,2,2 $\rightarrow$ sort 2,2,1,1 $\rightarrow$ 1,0,1 $\rightarrow$ sort 1,1,0 $\rightarrow$ 0,0. **Graphical** (e.g. a 6-cycle plus two chords).

Trap: an even sum is necessary but not sufficient — (A) passes the parity test.

Q.11 [MCQ · 2 marks · Python — list-comprehension quicksort with duplicates]**Answer: (C)**

Concept: this 'functional' quicksort partitions with strict $<$ and strict $>$, so every element **equal** to the pivot (other than the pivot itself) is silently dropped. It also builds new lists, so the result is never the input object (except for inputs of length ≤ 1 , which are returned as is).

Trace:

- `qs([4, 7, 4, 1, 7, 3, 4]):` $p = 4$; `less = [1, 3]`; `greater = [7, 7]`; the two other 4s vanish.
- `qs([1, 3]):` $p = 1$, `less = []`, `greater = [3]` $\rightarrow$ `[1, 3]`.
- `qs([7, 7]):` $p = 7$, `less = []`, `greater = []` (the second 7 is dropped) $\rightarrow$ `[7]`.
- Result: `[1, 3] + [4] + [7] = [1, 3, 4, 7]`.

So `len(r) = 4`, `r[-2:] = [4, 7]`, `data is r` is False $\rightarrow$ 4 `[4, 7]` False.

Options:

- (A) assumes duplicates are kept (a correct sort would give length 7 and end with `[7, 7]`).
- (B) keeps one duplicate too many.
- (D) claims the returned list is the input object — impossible for length > 1 .

Tip: the fix is `<=` for one side (or a three-way split `<, ==, >`). The bug only shows up with repeated keys — a classic test case to include.

Q.12 [NAT · 2 marks · Quicksort — total comparisons (Lomuto)]**Answer: 13**

Concept: partitioning a subarray of size m costs exactly $m - 1$ comparisons; the total is the sum over all partition calls, and the call sizes depend on where each pivot lands.

Trace (see table):

- Call on $[0..7]$, pivot 5: elements ≤ 5 are 3, 1, 2 $\rightarrow$ array $[3, 1, 2, \mathbf{5}, 6, 8, 9, 7]$, pivot at 3. Cost 7.
- Call on $[0..2] = [3, 1, 2]$, pivot 2: $\rightarrow [1, \mathbf{2}, 3]$, pivot at 1. Cost 2. Its two sides have size 1 $\rightarrow$ no further work.
- Call on $[4..7] = [6, 8, 9, 7]$, pivot 7: $6 \leq 7 \rightarrow [6, \mathbf{7}, 9, 8]$, pivot at 5. Cost 3.
- Left side $[4..4]$ size 1; right side $[6..7] = [9, 8]$, pivot 8 $\rightarrow [\mathbf{8}, 9]$. Cost 1.

Total = $7 + 2 + 3 + 1 = \mathbf{13}$.

For comparison, the worst case for $n = 8$ (e.g. already-sorted input) costs $7 + 6 + \dots + 1 = 28$ comparisons; this input's well-placed first pivot saves more than half.

Trap: counting n comparisons per partition instead of $n - 1$, or forgetting the small size-2 partition at the end.

Array after each partition call (pivot highlighted)

	0	1	2	3	4	5	6	7	cost
start	7	3	9	1	6	8	2	5	
[0..7]	3	1	2	5	6	8	9	7	7
[0..2]	1	2	3	5	6	8	9	7	2
[4..7]	1	2	3	5	6	7	9	8	3
[6..7]	1	2	3	5	6	7	8	9	1

Q.13 [MSQ · 2 marks · Quicksort — best/worst case and recurrences]**Answer: (B), (C)**

Concept: quicksort's cost depends on the *depth* of the recursion tree. Any constant-fraction split gives $O(\log n)$ depth and $\Theta(n)$ work per level; a split that removes only a constant number of elements gives $\Theta(n)$ depth.

- (A) Each partition removes the pivot and one element to its left: $T(n) = T(n - 2) + T(1) + \Theta(n) \rightarrow \Theta(n^2)$. **False.**
- (B) With all keys equal, $a[j] \leq x$ is always true, so the pivot lands at hi every time; subproblem sizes are $n, n - 1, \dots, 2$ with $n - 1, n - 2, \dots, 1$ comparisons — total $n(n - 1)/2$. **True.** (Hoare's scheme or three-way partitioning avoid this.)
- (C) $T(n) = T(n/10) + T(9n/10) + \Theta(n)$. Every level of the recursion tree does at most cn work and the depth is $\log_{10/9} n = \Theta(\log n)$; the first $\log_{10} n$ levels are full, so the total is $\Theta(n \log n)$. **True.**
- (D) Master theorem: $a = 2, b = 2, n^{\log_2 2} = n$ dominates $f(n) = c \rightarrow \Theta(n)$. Exactly, $T(n) = (2n - 1)c$ for n a power of 2. **False.**

Trap: (A) 'looks' like a non-extreme pivot, but a split of $(1, n - 2)$ is just as bad as $(0, n - 1)$ asymptotically.

Q.14 [NAT · 2 marks · Quickselect — partition-based selection]**Answer: 18**

Concept: quickselect partitions like quicksort but recurses into **one** side only — the side containing index k . Expected time is $\Theta(n)$, worst case $\Theta(n^2)$.

Target: index 3 = the 4th smallest. (Sorted: 3, 5, 8, **9**, 11, ...)

- Call [0..9], pivot 19: elements ≤ 19 are 14, 3, 9, 17, 5, 11, 8 $\rightarrow$ [14, 3, 9, 17, 5, 11, 8, **19**, 22, 26], $p = 7$. Cost **9**. $7 > 3 \rightarrow$ recurse left [0..6].
- Call [0..6], pivot 8: ≤ 8 are 3, 5 $\rightarrow$ [3, 5, **8**, 17, 14, 11, 9, ...], $p = 2$. Cost **6**. $2 < 3 \rightarrow$ recurse right [3..6].
- Call [3..6] = [17, 14, 11, 9], pivot 9: nothing $\leq 9 \rightarrow$ pivot swapped to index 3 $\rightarrow$ [3, 5, 8, **9**, 14, 11, 17, ...], $p = 3$. Cost **3**. $p == k \rightarrow$ return 9.

Total comparisons = $9 + 6 + 3 = \mathbf{18}$, and the answer returned is 9.

Trap: recursing into both sides (as full quicksort would) or counting n instead of $n - 1$ comparisons per partition (which gives $10 + 7 + 4 = 21$).

Quickselect trace (pivot highlighted)

	0	1	2	3	4	5	6	7	8	9	cost
start	14	3	22	9	17	5	11	26	8	19	
[0..9]	14	3	9	17	5	11	8	19	22	26	9
[0..6]	3	5	8	17	14	11	9	19	22	26	6
[3..6]	3	5	8	9	14	11	17	19	22	26	3

Q.15 [MCQ · 2 marks · Quicksort — three-way (Dutch national flag) partition]

Answer: (B)

Concept: invariant $a[0..lt-1] < p$, $a[lt..i-1] = p$, $a[gt+1..n-1] > p$, $a[i..gt]$ unseen. After a swap with gt , i is **not** advanced because the element brought in is unseen.

Trace (lt, i, gt start at 0, 0, 8):

- $i=0$: $5 = p \rightarrow i = 1$.
- $i=1$: $8 > p \rightarrow$ swap with $a[8] \rightarrow [5, 3, 2, 5, 9, 1, 5, 7, 8]$, $gt = 7$.
- $i=1$: $3 < p \rightarrow$ swap $a[0] \leftrightarrow a[1] \rightarrow [3, 5, 2, 5, 9, 1, 5, 7, 8]$, $lt = 1$, $i = 2$.
- $i=2$: $2 < p \rightarrow$ swap $a[1] \leftrightarrow a[2] \rightarrow [3, 2, 5, 5, 9, 1, 5, 7, 8]$, $lt = 2$, $i = 3$.
- $i=3$: $5 \rightarrow i = 4$.
- $i=4$: $9 > p \rightarrow$ swap with $a[7] \rightarrow [3, 2, 5, 5, 7, 1, 5, 9, 8]$, $gt = 6$.
- $i=4$: $7 > p \rightarrow$ swap with $a[6] \rightarrow [3, 2, 5, 5, 5, 1, 7, 9, 8]$, $gt = 5$.
- $i=4$: $5 \rightarrow i = 5$.
- $i=5$: $1 < p \rightarrow$ swap $a[2] \leftrightarrow a[5] \rightarrow [3, 2, 1, 5, 5, 5, 7, 9, 8]$, $lt = 3$, $i = 6 > gt \rightarrow$ stop.

Final: **[3, 2, 1, 5, 5, 5, 7, 9, 8]**, **$lt = 3$, $gt = 5$** $\rightarrow$ (B). The equal block is $a[3..5]$.

Options: (A) gets the $>$ block order wrong (9 and 7 were swapped twice). (C) sorts the $<$ block, which partitioning never does. (D) has gt one too large — gt must point at the last element equal to p .

Tip: three-way partitioning makes quicksort $\Theta(n)$ on arrays with all keys equal.

Q.16 [NAT · 2 marks · Merge sort — counting merge comparisons]

Answer: 17

Concept: merging lists of sizes p and q costs between $\min(p, q)$ and $p + q - 1$ comparisons; the exact count is $p + q - (\text{number of elements left over when one side runs out})$.

Recursion:

- $[6, 2] \rightarrow 1$ comparison $\rightarrow [2, 6]$; $[9, 4] \rightarrow 1 \rightarrow [4, 9]$.
- merge $[2, 6] + [4, 9]$: 2 vs 4 , 6 vs 4 , 6 vs $9 \rightarrow 3$ comparisons, 9 copied $\rightarrow [2, 4, 6, 9]$.
- $[1, 8] \rightarrow 1 \rightarrow [1, 8]$; $[3, 7] \rightarrow 1 \rightarrow [3, 7]$.
- merge $[1, 8] + [3, 7]$: 1 vs 3 , 8 vs 3 , 8 vs $7 \rightarrow 3$ comparisons, 8 copied $\rightarrow [1, 3, 7, 8]$.
- final merge $[2, 4, 6, 9] + [1, 3, 7, 8]$: $2-1$, $2-3$, $4-3$, $4-7$, $6-7$, $9-7$, $9-8 \rightarrow 7$ comparisons, 9 copied.

Total = $1 + 1 + 3 + 1 + 1 + 3 + 7 = 17$ (the maximum possible for $n = 8$ is 17 as well: $4 \cdot 1 + 2 \cdot 3 + 7$).

Contrast: on already-sorted input the counts are $1, 1, 2, 1, 1, 2, 4 = 12$ (minimum).

Trap: counting a comparison for the copied tail elements (giving $8 + 4 + 4 + \dots = 24$).

Merges and their comparison counts

	left	right	result	comps
1	[6]	[2]	[2, 6]	1
2	[9]	[4]	[4, 9]	1
3	[2, 6]	[4, 9]	[2, 4, 6, 9]	3
4	[1]	[8]	[1, 8]	1
5	[3]	[7]	[3, 7]	1
6	[1, 8]	[3, 7]	[1, 3, 7, 8]	3
7	[2, 4, 6, 9]	[1, 3, 7, 8]	[1, 2, 3, 4, 6, 7, 8, 9]	7

Q.17 [MSQ · 2 marks · Shortest paths — Dijkstra with ties]

Answer: (A), (C)

Concept: Dijkstra finalises vertices in non-decreasing order of distance. Ties in path length do not change distances, but with strict updates the *first* vertex that achieves the best value stays as predecessor.

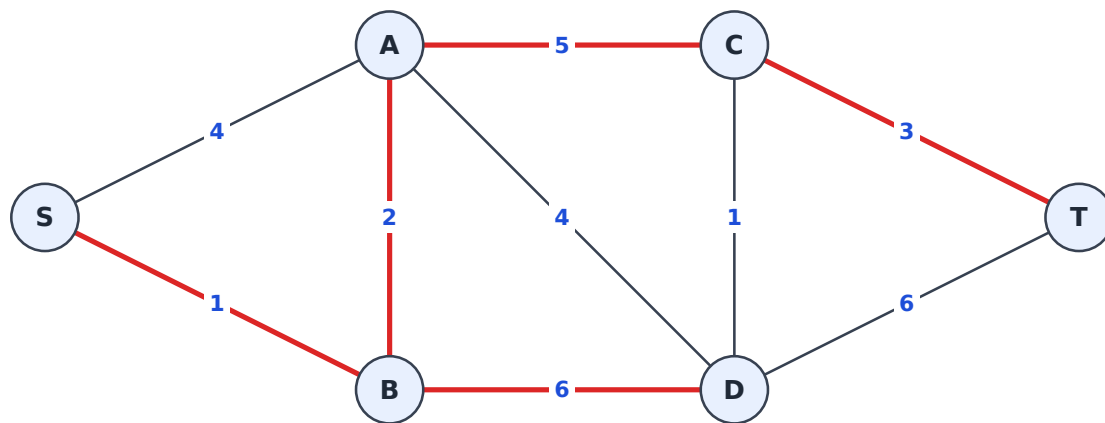
Trace:

- Extract S (0): $A = 4, B = 1$.
- Extract B (1): $A = \min(4, 1 + 2) = 3$ (parent B); $D = 1 + 6 = 7$ (parent B).
- Extract A (3): $C = 3 + 5 = 8$ (parent A); $D: 3 + 4 = 7$, not $< 7 \rightarrow$ unchanged.
- Extract D (7): $C: 7 + 1 = 8$, not $< 8 \rightarrow$ parent stays A; $T = 7 + 6 = 13$.
- Extract C (8): $T = \min(13, 8 + 3) = 11$ (parent C).
- Extract T (11).

Verdicts:

- (A) $d(T) = 11$ (S-B-A-C-T). **True.**
- (B) Shortest S-C paths of length 8: S-B-A-C ($1 + 2 + 5$), S-B-D-C ($1 + 6 + 1$) and S-B-A-D-C ($1 + 2 + 4 + 1$) — **three**, not two. **False.**
- (C) Order S, B, A, D, C, T. **True.**
- (D) C's parent is A, because D offered only an equal distance. **False.**

Trap: missing the path through the diagonal A-D in (B); and in (D) assuming the *last* equal offer wins.



Shortest-path tree produced (strict updates)

Q.18 [NAT · 2 marks · Heaps — bottom-up build-heap]**Answer: 7**

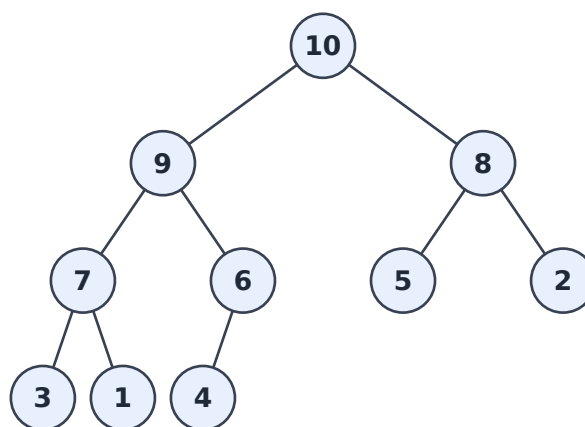
Concept: bottom-up heapify processes internal nodes from the last one (index $\lfloor n/2 \rfloor - 1 = 4$) back to the root; a node at height h causes at most h swaps, giving $O(n)$ total.

Trace:

- $i = 4$ (4): only child index 9 (6) $> 4 \rightarrow$ swap (1). $\rightarrow [3, 9, 2, 1, 6, 5, 8, 7, 10, 4]$
- $i = 3$ (1): children 7, 10 $\rightarrow$ swap with 10 (2). $\rightarrow [3, 9, 2, 10, 6, 5, 8, 7, 1, 4]$
- $i = 2$ (2): children 5, 8 $\rightarrow$ swap with 8 (3). $\rightarrow [3, 9, 8, 10, 6, 5, 2, 7, 1, 4]$
- $i = 1$ (9): children 10, 6 $\rightarrow$ swap with 10 (4); at index 3, children 7, 1 $< 9 \rightarrow$ stop. $\rightarrow [3, 10, 8, 9, 6, 5, 2, 7, 1, 4]$
- $i = 0$ (3): children 10, 8 $\rightarrow$ swap (5); at index 1 children 9, 6 $\rightarrow$ swap (6); at index 3 children 7, 1 $\rightarrow$ swap with 7 (7); index 7 is a leaf.

Final heap $[10, 9, 8, 7, 6, 5, 2, 3, 1, 4]$, total swaps = 7.

Trap: inserting the elements one by one (top-down, sift-up) is a *different* algorithm and generally performs a different number of swaps and can yield a different heap.



Max-heap after build-heap

Q.19 [MCQ · 2 marks · Insertion sort — comparisons and shifts]**Answer: (B)**

Concept: shifts = number of inversions. Each insertion makes (shifts for that key) + 1 comparisons, **except** when the key travels all the way to index 0 — then $j \geq 0$ fails first and the comparison $a[j] > \text{key}$ is short-circuited away.

Inversions of [2, 3, 5, 1, 4, 6, 8, 7]: (2,1), (3,1), (5,1), (5,4), (8,7) = **5 shifts**.

Per insertion (key: comparisons / shifts):

- 3: 1 / 0
- 5: 1 / 0
- 1: compares with 5, 3, 2 (all shift), then $j = -1 \rightarrow 3 / 3$ (no 4th comparison)
- 4: 5 (shift), 3 (stop) $\rightarrow 2 / 1$
- 6: 1 / 0
- 8: 1 / 0
- 7: 8 (shift), 6 (stop) $\rightarrow 2 / 1$

Comparisons = $1 + 1 + 3 + 2 + 1 + 1 + 2 = \mathbf{11}$; shifts = **5** $\rightarrow$ (B).

Options: (A) adds a phantom comparison when 1 reaches the front (the formula $\text{inversions} + n - 1 = 12$ overcounts by the number of keys that reach index 0). (C) misses an inversion. (D) counts 2 comparisons per shift.

Trap: short-circuit **and** — $a[j] > \text{key}$ is never evaluated once $j \geq 0$ is False.

Q.20 [MSQ · 2 marks · BFS — counting shortest paths]

Answer: (B), (C), (D)

Concept: in BFS, the number of shortest paths to v is the sum of the counts of its neighbours one level closer: $\sigma(v) = \sum \sigma(u)$ over edges $u-v$ with $\text{dist}(u) = \text{dist}(v) - 1$.

Levels and counts σ :

- level 0: S ($\sigma = 1$)
- level 1: A, B, C ($\sigma = 1$ each)
- level 2: $D \leftarrow A, B$ ($\sigma = 2$); $E \leftarrow B, C$ ($\sigma = 2$)
- level 3: $F \leftarrow D, E$ ($\sigma = 4$); $G \leftarrow D$ ($\sigma = 2$); $H \leftarrow E$ ($\sigma = 2$)
- level 4: $T \leftarrow F, G, H$ ($\sigma = 4 + 2 + 2 = \mathbf{8}$)

Verdicts:

- (A) **False** — paths through D (e.g. S-A-D-G-T) avoid E.
- (B) **True** — 8 shortest paths of length 4.
- (C) Paths through D = $\sigma(D) \times (\text{shortest } D \rightarrow T \text{ paths}) = 2 \times 2$ (via F or G) = 4. **True.** (D and E are on the same level, so no shortest path uses both; the other 4 go through E.)
- (D) Level-3 vertices are dequeued in the order F, G, H (F was discovered first, by D); F is the first to see T. **True.**

Trap: counting paths by enumerating by hand often misses the 'cross' paths S-B-D-F-T and S-B-E-F-T; the σ recurrence avoids that.

BFS distance and path count σ

	S	A	B	C	D	E	F	G	H	T
dist	0	1	1	1	2	2	3	3	3	4
σ	1	1	1	1	2	2	4	2	2	8

Mock Test 13

Graph Theory Fundamentals

Difficulty: Moderate

Focus areas: degrees, handshaking, trees, simple graphs, complete/bipartite graphs

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — dict.fromkeys aliasing in adjacency sets]

Consider the following Python program that builds an undirected graph. What is printed?

Python

```
1 nodes = 'abcd'
2 adj = dict.fromkeys(nodes, set())
3 for u, v in [('a', 'b'), ('b', 'c'), ('c', 'd')]:
4     adj[u].add(v)
5     adj[v].add(u)
6 print(len(adj['a']), sum(len(s) for s in adj.values()))
```

- (A) 1 6
- (B) 4 16
- (C) 1 3
- (D) 4 4

Q.2 [NAT · numerical answer | 1 mark | Handshaking lemma]

A simple undirected graph has 8 vertices and 15 edges. Every vertex has degree either 3 or 5. The number of vertices of degree 5 is ____.

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Graphical degree sequences]

A sequence of non-negative integers is *graphical* if it is the degree sequence of some **simple** undirected graph. Which of the following sequences is/are graphical?

- (A) (4, 3, 3, 2, 2, 2)
- (B) (5, 4, 3, 2, 1, 1)
- (C) (3, 3, 3, 3, 2, 2)
- (D) (3, 3, 3, 1)

Q.4 [NAT · numerical answer | 1 mark | Trees — counting leaves from degrees]

A tree has exactly 4 vertices of degree 3, exactly 2 vertices of degree 2, and every other vertex is a leaf (degree 1). The number of leaves is ____.

Answer: _____ (enter a numerical value)

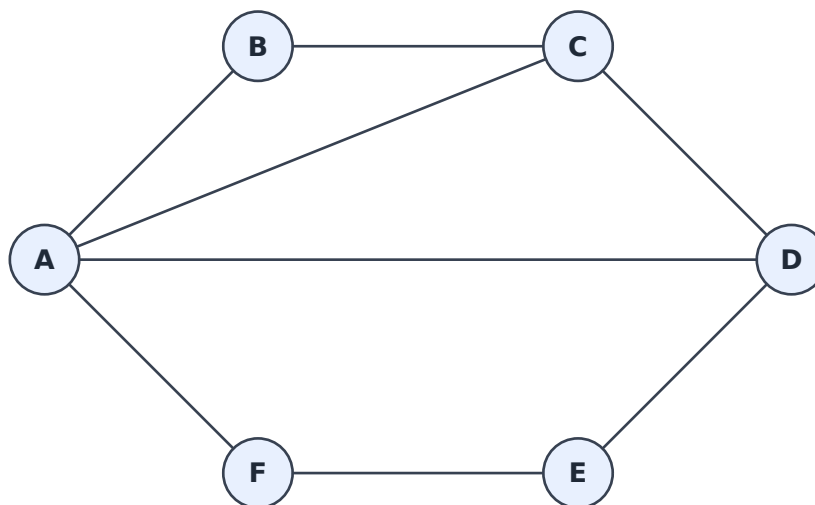
Q.5 [MCQ · single correct | 1 mark | Stacks — feasible pop sequences]

The letters a, b, c, d, e are pushed onto an initially empty stack in this order. Pops may be interleaved with pushes at any time, and every popped letter is printed. Which of the following printed sequences is NOT possible?

- (A) b a d c e
- (B) c d b e a
- (C) a c b e d
- (D) d e c a b

Q.6 [MCQ · single correct | 1 mark | Bipartite graphs and odd cycles]

Consider the undirected graph G below (a 6-cycle A-B-C-D-E-F-A with two chords A-C and A-D). Removing which **single** edge makes G bipartite?



- (A) A-C
- (B) A-B
- (C) A-D
- (D) C-D

Q.7 [NAT · numerical answer | 1 mark | Hashing — linear probing cost]

The keys 23, 47, 13, 35, 58, 2, 24 are inserted in that order into an initially empty hash table of size 11 with $h(k) = k \bmod 11$ and linear probing (step +1). A *probe* is one inspection of a slot, including the slot where the key is finally placed. The total number of probes for all seven insertions is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Selection sort — swaps]

Selection sort (ascending) is applied to [4, 9, 2, 7, 1, 8, 3]. In pass i it finds the minimum of $A[i..n-1]$ and swaps it with $A[i]$ **only if** it is not already at index i . The number of swaps performed is

- (A) 4
- (B) 5
- (C) 6
- (D) 3

Q.9 [MSQ · one or more correct | 1 mark | Python — sets, tuples and frozensets for edges]

Which of the following Python expressions evaluate to **True**?

- (A) `sorted({3: 'a', 1: 'b'}) == ['a', 'b']`
- (B) `len({(1, 2), (2, 1)}) == 1`
- (C) `len({frozenset((1, 2)), frozenset((2, 1))}) == 1`
- (D) `(({1, 2, 3} ^ {2, 3, 4}) == {1, 4})`

Q.10 [MCQ · single correct | 1 mark | Adjacency lists — counting list nodes]

A directed graph has vertices 1, 2, ..., 20. There is an edge $i \rightarrow j$ if and only if $j = i + 1$ or $j = 2i$ (with $j \leq 20$). The graph is stored as adjacency lists, one singly linked list node per edge. The total number of linked-list nodes is

- (A) 29
- (B) 28
- (C) 56
- (D) 30

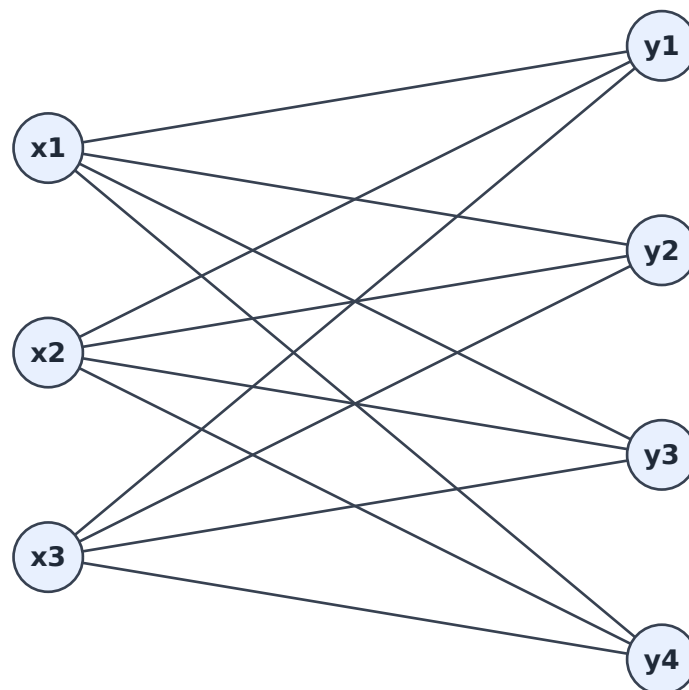
Q.11 [NAT · numerical answer | 2 marks | Labelled graphs and trees — counting]

Consider all simple undirected graphs on the labelled vertex set $\{1, 2, 3, 4\}$ that have **exactly 3 edges**. The number of these graphs that are connected is ____.

Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | Complete bipartite graphs]

Let G be the complete bipartite graph $K_{3,4}$ shown below (parts $X = \{x1, x2, x3\}$ and $Y = \{y1, y2, y3, y4\}$). Which of the following statements is/are TRUE?

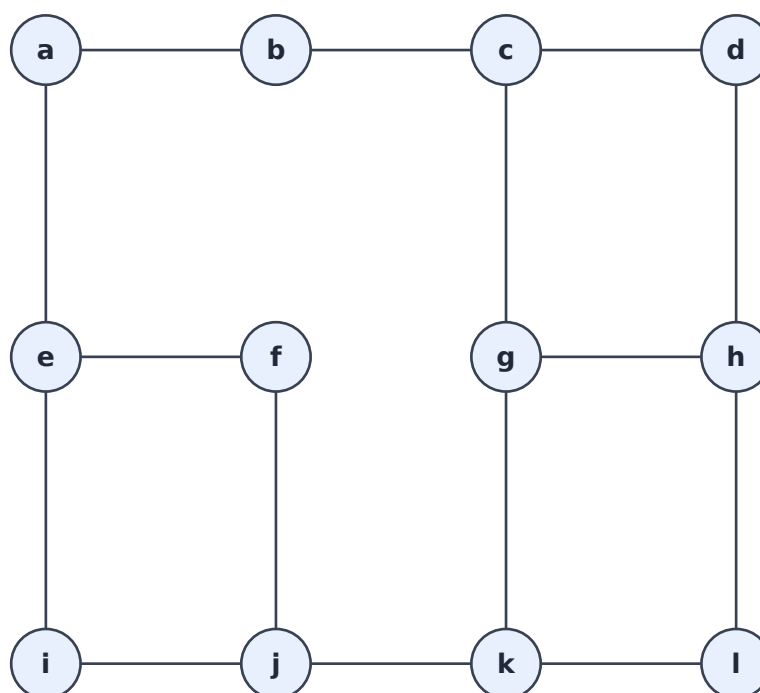


$K_{3,4}$

- (A) The complement of G (with respect to K_7 on the same 7 vertices) has exactly 9 edges
- (B) G contains a cycle of length 5
- (C) G has an Euler circuit
- (D) G has exactly 12 edges

Q.13 [NAT · numerical answer | 2 marks | BFS — counting shortest paths]

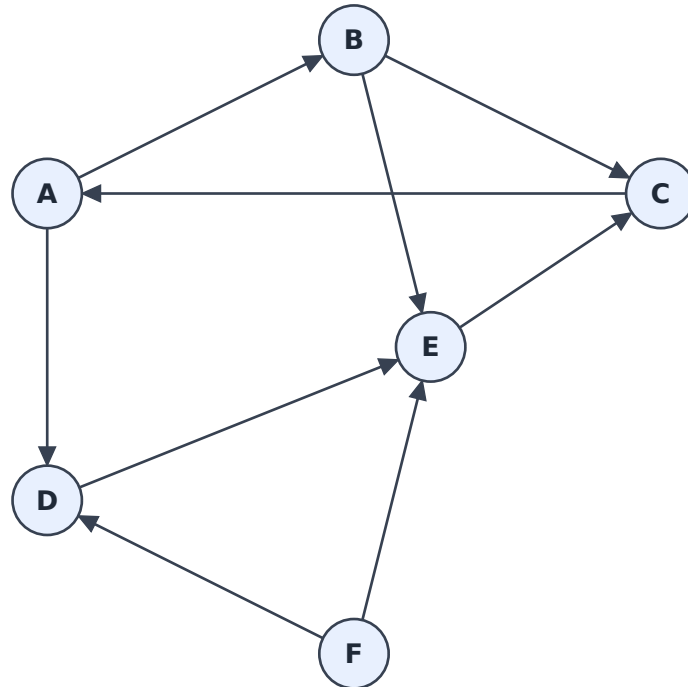
The undirected unweighted graph below is a 3×4 grid from which the edges b-f and f-g have been removed. The number of distinct shortest paths from a to l is _____.



Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | DFS — discovery and finish times]

Depth-first search is run on the directed graph below. The outer loop considers start vertices in alphabetical order, and each vertex's out-neighbours are explored in alphabetical order. The clock starts at 1 and is incremented at every discovery and every finish. The (discovery, finish) times of vertex **D** are



- (A) (4, 9)
- (B) (5, 6)
- (C) (8, 9)
- (D) (9, 10)

Q.15 [NAT · numerical answer | 2 marks | Python — connected components]

Consider the following Python program. The value printed is _____.

Python

```

1 edges = [(1, 2), (3, 4), (2, 5), (6, 6),
2         (7, 8), (5, 1), (8, 9)]
3 n = 10
4 adj = {v: [] for v in range(1, n + 1)}
5 for u, v in edges:
6     adj[u].append(v)
7     adj[v].append(u)
8 seen, comps, sizes = set(), 0, []
9 for s in adj:
10    if s in seen:
11        continue
12    comps += 1
13    stack, c = [s], 0
14    seen.add(s)
15    while stack:
16        u = stack.pop()
17        c += 1
18        for w in adj[u]:
19            if w not in seen:
20                seen.add(w)
21                stack.append(w)
22    sizes.append(c)
23 print(comps * 10 + max(sizes))

```

Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Edges, components and cycles]

G is a simple undirected graph with 10 vertices and 7 edges. Which of the following statements is/are **necessarily** TRUE?

- (A) If G has no cycle, then G has exactly 3 connected components
- (B) G has at least 3 connected components
- (C) G has a vertex of degree at most 1
- (D) G contains a cycle

Q.17 [NAT · numerical answer | 2 marks | Quicksort — counting comparisons]

Quicksort with the Lomuto partition (pivot = last element of the subarray; for each j from lo to hi – 1 compare $A[j] \leq \text{pivot}$; finally place the pivot) is used to sort [5, 3, 8, 1, 9, 2, 7]. Subarrays of size 0 or 1 are not partitioned. The total number of element comparisons $A[j] \leq \text{pivot}$ made over the whole sort is _____.

Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Hashing — quadratic probing]

The keys 20, 31, 42, 9, 53 are inserted in that order into an initially empty hash table of size 11 using quadratic probing: the i-th probe ($i = 0, 1, 2, \dots$) examines slot $(h(k) + i^2) \bmod 11$, where $h(k) = k \bmod 11$. The slot in which key 53 is stored is

- (A) 4
- (B) 1
- (C) 3
- (D) 7

Q.19 [MSQ · one or more correct | 2 marks | Binary search trees — build and delete]

The keys 41, 17, 63, 8, 29, 55, 72, 25, 34, 60, 50 are inserted in that order into an initially empty binary search tree T. Height is the number of **edges** on the longest root-to-leaf path. Which of the following statements is/are TRUE?

- (A) T has exactly 5 leaves
- (B) If 41 is deleted by replacing it with its in-order successor, the new root is 50 and 55 then has no left child
- (C) The pre-order traversal of T begins 41, 17, 8, 29, 25
- (D) The height of T is 3

Q.20 [MCQ · single correct | 2 marks | Extremal graphs — edges vs components]

The maximum possible number of edges in a simple undirected graph with 10 vertices and **exactly 3** connected components is

- (A) 27
- (B) 36
- (C) 21
- (D) 28

Answer Key — Mock Test 13

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — dict.fromkeys aliasing in adjacency sets
2	NAT	1	3	Handshaking lemma
3	MSQ	1	(A), (C)	Graphical degree sequences
4	NAT	1	6	Trees — counting leaves from degrees
5	MCQ	1	(D)	Stacks — feasible pop sequences
6	MCQ	1	(A)	Bipartite graphs and odd cycles
7	NAT	1	20	Hashing — linear probing cost
8	MCQ	1	(A)	Selection sort — swaps
9	MSQ	1	(C), (D)	Python — sets, tuples and frozensets for edges
10	MCQ	1	(B)	Adjacency lists — counting list nodes
11	NAT	2	16	Labelled graphs and trees — counting
12	MSQ	2	(A), (D)	Complete bipartite graphs
13	NAT	2	5	BFS — counting shortest paths
14	MCQ	2	(C)	DFS — discovery and finish times
15	NAT	2	53	Python — connected components
16	MSQ	2	(A), (B), (C)	Edges, components and cycles
17	NAT	2	11	Quicksort — counting comparisons
18	MCQ	2	(C)	Hashing — quadratic probing
19	MSQ	2	(B), (C), (D)	Binary search trees — build and delete
20	MCQ	2	(D)	Extremal graphs — edges vs components

Detailed Solutions — Mock Test 13

Q.1 [MCQ · 1 mark · Python — dict.fromkeys aliasing in adjacency sets]

Answer: (B)

`dict.fromkeys(keys, value)` stores the **same value** object under every key. Here all four keys refer to one single set, so every `add` goes into that shared set.

- After the loop the shared set contains every endpoint ever added: {a, b, c, d}.
- `len(adj['a']) = 4`.
- The sum iterates over four references to the same 4-element set $\rightarrow 4 \times 4 = 16$.

Output: **4 16**.

Option-by-option:

- (A) **1 6** is what a correct adjacency structure gives (degrees 1, 2, 2, 1; sum = $2|E| = 6$).
- (C) **1 3** additionally confuses the degree sum with $|E|$.
- (D) **4 4** realises the set is shared but forgets that `values()` yields it four times.

Trap: the same aliasing bug occurs with `[[]] * n`. **Fix:** `adj = {v: set() for v in nodes}` creates a fresh set per key.

Q.2 [NAT · 1 mark · Handshaking lemma]

Answer: 3

Handshaking lemma: the sum of all vertex degrees equals $2|E|$, because each edge contributes one to the degree of each of its two endpoints.

Let a vertices have degree 3 and b vertices have degree 5:

- $a + b = 8$
- $3a + 5b = 2 \times 15 = 30$

Substituting $a = 8 - b$: $24 - 3b + 5b = 30 \rightarrow 2b = 6 \rightarrow \mathbf{b = 3}$ (and $a = 5$).

Sanity check: the number of odd-degree vertices must be even — here all 8 vertices have odd degree, which is fine. Such a graph exists (e.g. three degree-5 hubs plus a suitable arrangement of the others).

Trap: using $|E| = 15$ instead of $2|E| = 30$ gives $3a + 5b = 15$, which has no solution with $a + b = 8$ — a clear sign of the error.

Q.3 [MSQ · 1 mark · Graphical degree sequences]

Answer: (A), (C)

Use the **Havel-Hakimi** test: remove the largest degree d , subtract 1 from the next d largest entries, re-sort, repeat. The sequence is graphical iff we reach all zeros without a negative entry or a d larger than the remaining length. (An odd sum fails at once.)

- (A) $4,3,3,2,2,2 \rightarrow 2,2,1,1,2 \rightarrow$ sort $2,2,2,1,1 \rightarrow 1,1,1,1 \rightarrow 0,1,1 \rightarrow 0,0$. **Graphical**.
- (B) sum 16 is even, but 5 on 6 vertices means that vertex is adjacent to **all** others, so every other degree is ≥ 1 ; remove it $\rightarrow 3,2,1,0,0$; then $3 \rightarrow 1,0,-1$ **X**. **Not graphical** — the two degree-1 vertices are both used up by the degree-5 vertex, so the degree-4 vertex can reach at most 3 others.
- (C) $3,3,3,3,2,2 \rightarrow 2,2,2,2,2 \rightarrow 1,1,2,2 \rightarrow$ sort $2,2,1,1 \rightarrow 1,0,1 \rightarrow 1,1,0 \rightarrow 0,0$. **Graphical** (e.g. a 6-cycle with two chords).
- (D) sum 10 is even, but $3,3,3$ on 4 vertices: each degree-3 vertex is adjacent to all others, so the fourth vertex would have degree 3, not 1. **Not graphical**.

Trap: an even degree sum is necessary but **not** sufficient — (B) and (D) both pass the parity check.

Q.4 [NAT · 1 mark · Trees — counting leaves from degrees]**Answer: 6**

A tree on n vertices has $n - 1$ edges, so its degree sum is $2(n - 1)$.

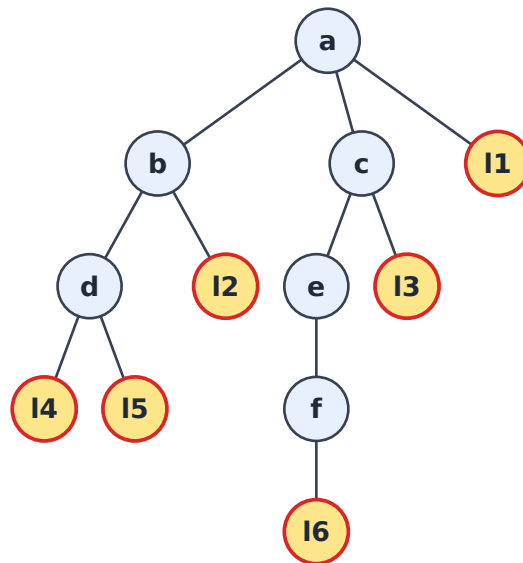
Let L be the number of leaves. Then $n = 4 + 2 + L = 6 + L$ and

- degree sum = $4 \cdot 3 + 2 \cdot 2 + L \cdot 1 = 16 + L$
- $2(n - 1) = 2(5 + L) = 10 + 2L$

$$16 + L = 10 + 2L \rightarrow L = 6.$$

General shortcut: $L = 2 + \sum \text{over internal vertices of } (\deg - 2) = 2 + 4 \cdot 1 + 2 \cdot 0 = 6$. Degree-2 vertices never change the leaf count.

Trap: forgetting the leaves in the degree sum, or using $|E| = n$ instead of $n - 1$.



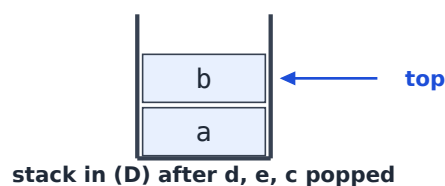
One such tree: a, b, c, d have degree 3; e, f degree 2

Q.5 [MCQ · 1 mark · Stacks — feasible pop sequences]**Answer: (D)**

Key rule: once x is popped, every element pushed before x that is still on the stack must come out in **reverse** push order.

- (A) push a, b ; pop b, a ; push c, d ; pop d, c ; push e ; pop e . **Possible.**
- (B) push a, b, c ; pop c ; push d ; pop d ; pop b ; push e ; pop e ; pop a . **Possible.**
- (C) push a ; pop a ; push b, c ; pop c, b ; push d, e ; pop e, d . **Possible.**
- (D) push a, b, c, d ; pop d ; push e ; pop e ; pop c . The stack now holds a (bottom), b (top). The next pop must give **b** , but the sequence asks for a . **Impossible.**

Tip: a sequence is infeasible exactly when it contains a pattern $\dots k \dots i \dots j \dots$ with $i < j < k$ in push order (here $d \dots a \dots b$).



Q.6 [MCQ · 1 mark · Bipartite graphs and odd cycles]**Answer: (A)**

A graph is bipartite **iff it has no odd cycle**. So we need an edge that lies on every odd cycle.

Odd cycles of G: A-B-C (length 3), A-C-D (length 3), A-C-D-E-F (length 5). The only edge common to all three is **A-C**.

After deleting A-C the remaining cycles are A-B-C-D (4), A-D-E-F (4) and the outer hexagon (6) — all even. A valid 2-colouring: {A, C, E} vs {B, D, F}.

- (B) removing A-B leaves triangle A-C-D. ✗
- (C) removing A-D leaves triangle A-B-C. ✗
- (D) removing C-D leaves triangle A-B-C. ✗

Tip: try a BFS 2-colouring; a conflict edge (both ends same colour) pinpoints an odd cycle.

Q.7 [NAT · 1 mark · Hashing — linear probing cost]**Answer: 20**

Linear probing inspects $h(k)$, $h(k)+1$, ... until an empty slot is found.

- 23 → home 1, empty → 1 probe
- 47 → home 3, empty → 1 probe
- 13 → home 2, empty → 1 probe
- 35 → home 2 (full), 3 (full), 4 → **3** probes
- 58 → home 3 (full), 4 (full), 5 → **3** probes
- 2 → home 2, 3, 4, 5 full, 6 → **5** probes
- 24 → home 2, 3, 4, 5, 6 full, 7 → **6** probes

Total = 1 + 1 + 1 + 3 + 3 + 5 + 6 = **20**.

Trap: counting only *collisions* (failed probes) gives $20 - 7 = 13$. Read the definition. **Note:** the cluster 1-7 grows quickly — primary clustering makes later keys with homes 2 or 3 increasingly expensive.

0	
1	23
2	13
3	47
4	35
5	58
6	2
7	24
8	
9	
10	

Final table

Q.8 [MCQ · 1 mark · Selection sort — swaps]**Answer: (A)**

Selection sort always makes $n(n-1)/2 = 21$ comparisons, but the number of swaps depends on the data.

- $i=0$: min 1 at index 4 → swap → [1, 9, 2, 7, 4, 8, 3]

- $i=1$: min 2 at index 2 $\rightarrow$ swap $\rightarrow [1, 2, 9, 7, 4, 8, 3]$
- $i=2$: min 3 at index 6 $\rightarrow$ swap $\rightarrow [1, 2, 3, 7, 4, 8, 9]$
- $i=3$: min 4 at index 4 $\rightarrow$ swap $\rightarrow [1, 2, 3, 4, 7, 8, 9]$
- $i=4$: 7 already in place — no swap
- $i=5$: 8 already in place — no swap

Total = **4** swaps.

- (B) 5 counts the pass $i=4$ as a swap although 7 is already in place.
- (C) 6 is the count if every one of the $n - 1$ passes swapped unconditionally.
- (D) 3 misses that the swap at $i=2$ sends 9 to the end, so $i=3$ still needs a swap.

Tip: swaps = $n - (\text{number of cycles of the permutation})$. Index cycles here: $(0 \rightarrow 3 \rightarrow 4)$, $(1 \rightarrow 6 \rightarrow 2)$ and the fixed point 5 (value 8) — 3 cycles, so $7 - 3 = 4$.

Q.9 [MSQ · 1 mark · Python — sets, tuples and frozensets for edges]

Answer: (C), (D)

These are the usual ways to store undirected edges and neighbour sets.

- (A) iterating a dict yields its **keys**; **sorted** gives `[1, 3]`. **False**.
- (B) tuples are ordered, so $(1, 2) \neq (2, 1)$; the set has 2 elements. **False**. This is why storing an undirected edge as a tuple can double-count it.
- (C) a frozenset is unordered and hashable, so both describe the same edge $\rightarrow$ set of size 1. **True**.
- (D) $\wedge$ is symmetric difference: elements in exactly one set $\rightarrow \{1, 4\}$. **True**.

Tip: for undirected edges either normalise to `(min(u, v), max(u, v))` or use `frozenset({u, v})`.

Q.10 [MCQ · 1 mark · Adjacency lists — counting list nodes]

Answer: (B)

In a directed graph each edge appears in exactly one adjacency list (that of its tail), so the number of nodes equals $|E|$.

- Edges $i \rightarrow i + 1$ for $i = 1 \dots 19$: **19** edges.
- Edges $i \rightarrow 2i$ for $i = 1 \dots 10$ ($2i \leq 20$): **10** edges.
- Overlap: $i + 1 = 2i$ only when $i = 1$, i.e. the edge $1 \rightarrow 2$ is described twice but is one edge. Subtract 1.

$|E| = 19 + 10 - 1 = \mathbf{28}$.

- (A) 29 forgets the overlap.
- (C) 56 doubles the count, as for an undirected graph (each edge stored twice).
- (D) 30 uses 20 and 10 edges, wrongly including $20 \rightarrow 21$.

Tip: adjacency lists use $\Theta(V + E)$ space: 20 head pointers + 28 nodes here.

Q.11 [NAT · 2 marks · Labelled graphs and trees — counting]

Answer: 16

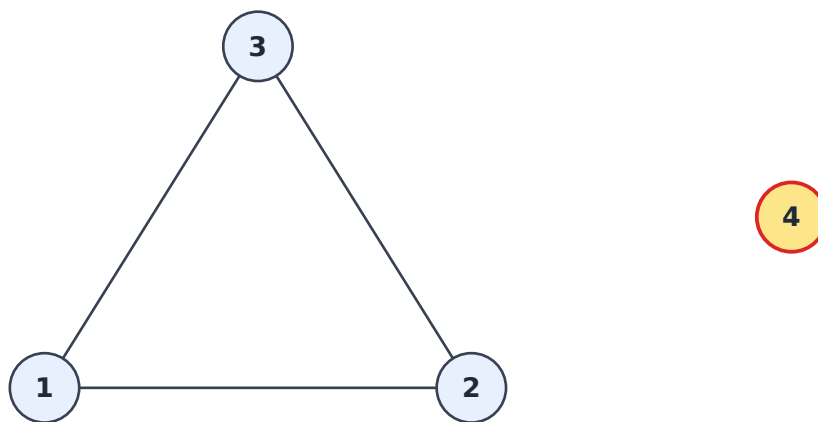
K_4 has $C(4, 2) = 6$ possible edges, so there are $C(6, 3) = \mathbf{20}$ graphs with exactly 3 edges.

A graph on 4 vertices with $3 = n - 1$ edges is connected **iff** it is a tree (a connected graph needs $\geq n - 1$ edges, and connected with exactly $n - 1$ edges means acyclic). So we can count either way:

- **Disconnected** 3-edge graphs: with only 3 edges the only way to waste an edge is a cycle, and the only cycle that fits is a triangle; a triangle on 3 of the 4 vertices leaves the fourth vertex isolated. There are $C(4, 3) = \mathbf{4}$ triangles.
- Connected = $20 - 4 = \mathbf{16}$.

Cross-check with **Cayley's formula**: the number of labelled trees on n vertices is $n^{n-2} = 4^2 = 16$. ✓
(Shape split: 4 stars $K_{1,3}$ + 12 paths $P_4 = 16$.)

Trap: counting unlabelled shapes (only 2: star and path) instead of labelled graphs.



One of the 4 disconnected cases: triangle + isolated vertex

Q.12 [MSQ · 2 marks · Complete bipartite graphs]

Answer: (A), (D)

In $K_{m,n}$ every vertex of one part is adjacent to every vertex of the other part and to none in its own part.

- (A) K_7 has $C(7, 2) = 21$ edges; the complement has $21 - 12 = 9$ edges — exactly a K_3 on X (3 edges) plus a K_4 on Y (6 edges). **True.**
- (B) Every edge goes $X \leftrightarrow Y$, so any closed walk alternates sides and has **even** length. A bipartite graph has no odd cycle. **False.**
- (C) An Euler circuit needs every degree even. The x -vertices have degree 4 (even) but the y -vertices have degree 3 (odd). **False.** (Four odd vertices also rule out an Euler trail.)
- (D) $|E| = m \cdot n = 3 \cdot 4 = 12$. **True.** (Check by handshaking: $3 \cdot 4 + 4 \cdot 3 = 24 = 2 \cdot 12$.)

Trap: in (C), the even degree of the X side is irrelevant — *all* vertices must have even degree. **Tip**: the complement of $K_{m,n}$ is always $K_m \cup K_n$.

Q.13 [NAT · 2 marks · BFS — counting shortest paths]

Answer: 5

In BFS, the number of shortest paths to v is $\sigma(v) = \sum \sigma(u)$ over neighbours u with $\text{dist}(u) = \text{dist}(v) - 1$. Process level by level ($\sigma(a) = 1$):

- dist 1: b ($\sigma=1$), e ($\sigma=1$)
- dist 2: $c \leftarrow b$ (1); $f \leftarrow e$ only, since $b-f$ is gone (1); $i \leftarrow e$ (1)
- dist 3: $d \leftarrow c$ (1); $g \leftarrow c$ only, since $f-g$ is gone (1); $j \leftarrow f, i$ ($1 + 1 = 2$)
- dist 4: $h \leftarrow d, g$ ($1 + 1 = 2$); $k \leftarrow g, j$ ($1 + 2 = 3$)
- dist 5: $l \leftarrow h, k$ ($2 + 3 = 5$)

The 5 paths: $a-b-c-d-h-l$, $a-b-c-g-h-l$, $a-b-c-g-k-l$, $a-e-f-j-k-l$, $a-e-i-j-k-l$. $\text{dist}(a, l)$ is still $5 = 2 + 3$, the Manhattan distance.

Trap: the complete 3×4 grid would give $C(5, 2) = 10$ paths; the two deleted edges remove exactly half of them. Do not reuse the binomial formula once edges are missing.

BFS distance / path count σ				
	col 1	col 2	col 3	col 4
row 1	a: 0 / 1	b: 1 / 1	c: 2 / 1	d: 3 / 1
row 2	e: 1 / 1	f: 2 / 1	g: 3 / 1	h: 4 / 2
row 3	i: 2 / 1	j: 3 / 2	k: 4 / 3	l: 5 / 5

Q.14 [MCQ · 2 marks · DFS — discovery and finish times]**Answer: (C)**

DFS finishes a vertex only after everything newly reachable from it has finished.

- A discovered 1 $\rightarrow$ B 2 $\rightarrow$ C 3; C $\rightarrow$ A is a **back edge**; C finishes 4
- back at B: E discovered 5; E $\rightarrow$ C goes to a finished vertex (**cross edge**); E finishes 6
- B finishes 7
- back at A: D discovered **8**; D $\rightarrow$ E is a cross edge; D finishes **9**
- A finishes 10; F discovered 11 (new DFS tree), F $\rightarrow$ D, F $\rightarrow$ E cross; F finishes 12

So D = **(8, 9)**.

- (A) (4, 9) would come from exploring D right after C, ignoring that B's subtree (including E) must finish first.
- (B) (5, 6) are E's times.
- (D) (9, 10) forgets that the clock also ticks when B finishes.

Tip: the cycle A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A is detected by the back edge C $\rightarrow$ A (C's ancestor A is still on the recursion stack).

DFS timestamps						
	A	B	C	D	E	F
discovery	1	2	3	8	5	11
finish	10	7	4	9	6	12

Q.15 [NAT · 2 marks · Python — connected components]**Answer: 53**

The program counts connected components with an iterative DFS and records each component's size (number of vertices popped).

Components of the graph on vertices 1...10:

- {1, 2, 5} (edges 1-2, 2-5, 5-1 form a triangle) — size 3
- {3, 4} — size 2
- {6} — the self-loop (6, 6) appends 6 twice to `adj[6]` but adds no new vertex — size 1
- {7, 8, 9} — size 3
- {10} — isolated, still a component — size 1

comps = 5, max size = 3 $\rightarrow$ printed $5 \cdot 10 + 3 = \mathbf{53}$.

Because vertices are marked **seen** when pushed, no vertex is counted twice even though the triangle offers two routes to 5.

Traps: forgetting the isolated vertex 10 (gives 43), or thinking the self-loop joins 6 to something.

Tip: comps - 1 = 4 extra edges are needed to connect this graph.



Five components (self-loop at 6 not drawn)

Q.16 [MSQ · 2 marks · Edges, components and cycles]**Answer: (A), (B), (C)**

Key facts: a connected graph on c vertices has $\geq c - 1$ edges; a forest with n vertices and k components has exactly $n - k$ edges.

- (A) For a forest, $|E| = n - k \rightarrow 7 = 10 - k \rightarrow k = 3$. **True.**
- (B) If G has k components of sizes $c_1, \dots, c_k$, then $7 = |E| \geq \sum (c_i - 1) = 10 - k$, so $k \geq 3$. **True.**
- (C) Degree sum = $14 < 2 \cdot 10 = 20$, so the average degree is $1.4 < 2$; some vertex must have degree ≤ 1 . **True.**
- (D) A forest of 3 trees, e.g. a path on 8 vertices plus 2 isolated vertices, has 7 edges and no cycle. **False** (not necessarily).

Trap: (D) confuses 'too few edges to be connected' with 'must have a cycle'. A cycle is forced only when $|E| \geq n$, which is not the case here.

Q.17 [NAT · 2 marks · Quicksort — counting comparisons]**Answer: 11**

A Lomuto partition of a subarray of size s makes exactly $s - 1$ comparisons, so we only need the sizes of all partitioned subarrays.

- $[5 \ 3 \ 8 \ 1 \ 9 \ 2 \ 7]$, pivot 7: 6 comparisons $\rightarrow [5 \ 3 \ 1 \ 2 \ | \ 7 \ | \ 8 \ 9]$
- left $[5 \ 3 \ 1 \ 2]$, pivot 2: 3 comparisons $\rightarrow [1 \ | \ 2 \ | \ 5 \ 3]$
- $[1]$ size 1 — skip; $[5 \ 3]$, pivot 3: 1 comparison $\rightarrow [3 \ 5]$
- right $[8 \ 9]$, pivot 9: 1 comparison $\rightarrow [8 \ 9]$

Total = $6 + 3 + 1 + 1 = 11$.

Compare: the best possible for $n = 7$ is 10 (perfect 3|3 split, then 1|1 twice $\rightarrow 6 + 2 + 2$) and the worst is 21 (sorted input, $n(n-1)/2$).

Trap: counting the final pivot swap as a comparison, or partitioning size-1 subarrays.



After the first partition (pivot 7 at index 4)

Q.18 [MCQ · 2 marks · Hashing — quadratic probing]**Answer: (C)**

All five keys have home slot 9 ($20, 31, 42, 9, 53$ are all $\equiv 9 \pmod{11}$), so each new key walks further along the same quadratic sequence $9, 10, 13 \equiv 2, 18 \equiv 7, 25 \equiv 3, \dots$

- $20 \rightarrow$ slot 9 ($i = 0$)
- $31 \rightarrow$ 9 full; $i=1$: $10 \rightarrow$ slot **10**
- $42 \rightarrow$ 9, 10 full; $i=2$: $9+4 = 13 \equiv 2$

- $9 \rightarrow 9, 10, 2$ full; $i=3: 9+9 = 18 \equiv 7$
- $53 \rightarrow 9, 10, 2, 7$ full; $i=4: 9+16 = 25 \equiv 3$

Key 53 lands in slot **3** after 5 probes.

- (A) 4 is $(9 + 5^2) \bmod 11$, i.e. one probe too many.
- (B) 1 comes from mistakenly applying linear probing *after* 9, 10 wrap (9, 10, 0, 1) — with linear probing the keys would actually occupy 9, 10, 0, 1, 2.
- (D) 7 is where key 9 went.

Note: this is *secondary clustering*: keys with the same home slot follow the identical probe sequence.

0	
1	
2	42
3	53
4	
5	
6	
7	9
8	
9	20
10	31

Final table

Q.19 [MSQ · 2 marks · Binary search trees — build and delete]

Answer: (B), (C), (D)

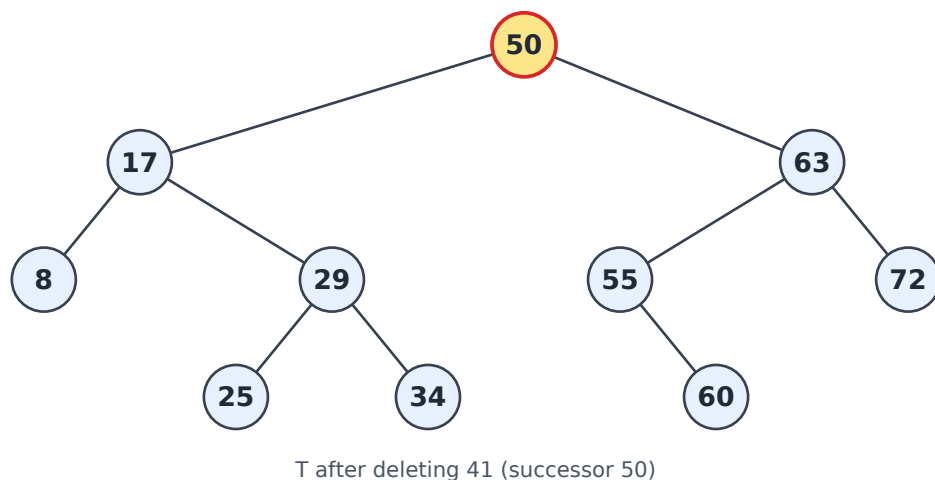
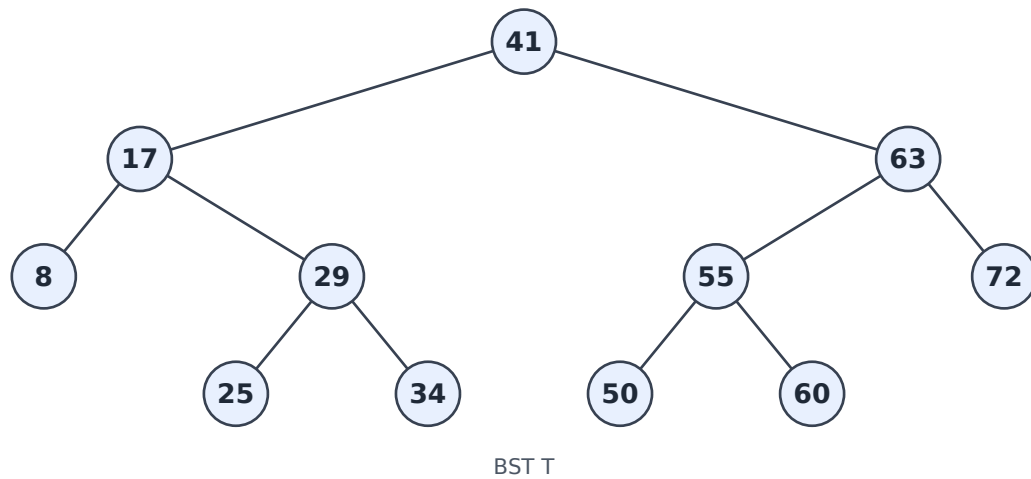
Build T by standard BST insertion:

- 41 root; 17 L; 63 R; 8 under 17 (L); 29 under 17 (R); 55 under 63 (L); 72 under 63 (R)
- $25 \rightarrow 41 \text{ L} \rightarrow 17 \text{ R} \rightarrow 29 \text{ L}$; $34 \rightarrow 29 \text{ R}$; $60 \rightarrow 63 \text{ L} \rightarrow 55 \text{ R}$; $50 \rightarrow 55 \text{ L}$

Now the statements:

- (A) leaves: 8, 25, 34, 50, 60, 72 $\rightarrow$ **6. False.**
- (B) in-order successor of 41 = minimum of the right subtree = 50 ($63 \rightarrow 55 \rightarrow 50$). 50 is a leaf, so it is simply moved to the root and 55's left pointer becomes empty. **True.**
- (C) pre-order (root, left, right): 41, 17, 8, 29, 25, 34, 63, 55, 50, 60, 72. **True.**
- (D) deepest nodes 25, 34, 50, 60 are at depth 3. **True.**

Trap: in (B), students often pick 55 or 63 as the successor; the successor is the **leftmost** node of the right subtree.

**Q.20 [MCQ · 2 marks · Extremal graphs — edges vs components]****Answer: (D)**

With components of sizes $n_1 + n_2 + n_3 = 10$ (each ≥ 1), the best we can do is make each component complete, giving $\sum C(n_i, 2)$ edges. Because $C(x, 2)$ is convex, the sum is maximised by making the sizes as **unequal** as possible: (8, 1, 1).

- (8, 1, 1): $C(8, 2) = 28$
- (7, 2, 1): $21 + 1 + 0 = 22$
- (6, 2, 2): $15 + 1 + 1 = 17$; (4, 3, 3): $6 + 3 + 3 = 12$ (the balanced split is the *minimum*)

General formula: $C(n - k + 1, 2)$ for n vertices and k components $\rightarrow C(8, 2) = 28$.

- (A) 27 is one short — the large component may be complete.
- (B) $36 = C(9, 2)$ is the maximum for exactly **2** components.
- (C) $21 = C(7, 2)$ is the maximum for 4 components.

Tip: the same convexity argument shows that a simple graph on n vertices with more than $C(n - 1, 2)$ edges must be connected.

Mock Test 14

Graph Traversals: BFS & DFS

Difficulty: Moderate

Focus areas: BFS/DFS orders, BFS levels, DFS trees, discovery/finish times

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — list aliasing in adjacency lists]

A student builds adjacency lists for a directed graph with 4 vertices as follows. What is printed?

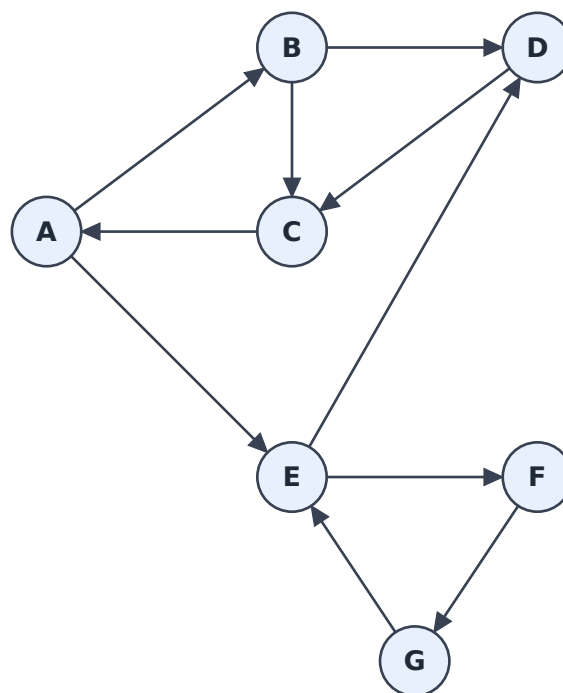
Python

```
1 adj = [[]] * 4
2 for u, v in [(0, 1), (1, 2), (2, 3)]:
3     adj[u].append(v)
4 print(len(adj[3]), adj[0] is adj[2])
```

- (A) 0 False
- (B) 3 True
- (C) 1 False
- (D) 3 False

Q.2 [NAT · numerical answer | 1 mark | DFS — discovery and finishing times]

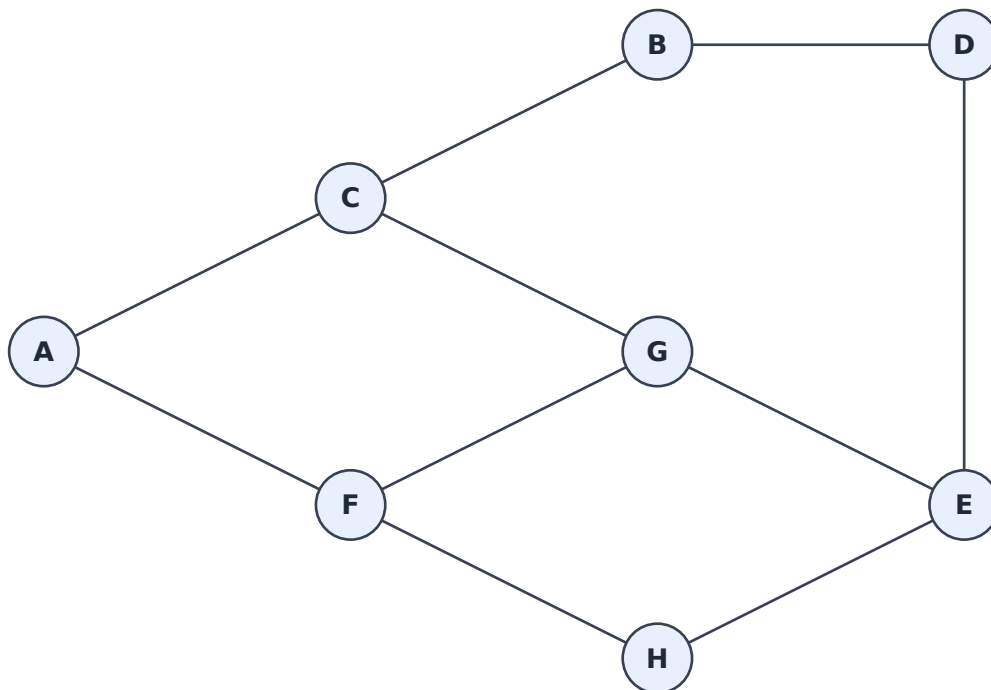
Depth-first search is run on the directed graph below starting at vertex A. Out-neighbours are explored in **alphabetical** order. A single clock starts at 0 and is incremented just before each discovery and each finish (so A is discovered at time 1). The finishing time of vertex E is _____.



Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | BFS — visiting order with tie-breaking]

Breadth-first search is run on the undirected graph below from vertex A. When a vertex is dequeued, its undiscovered neighbours are enqueued in **alphabetical** order. The order in which vertices are dequeued is



- (A) A, C, F, B, G, H, D, E
- (B) A, C, F, B, G, D, H, E
- (C) A, C, B, G, F, H, D, E
- (D) A, C, F, G, B, H, E, D

Q.4 [MSQ · one or more correct | 1 mark | BFS/DFS — complexity and edge properties]

Let G be a graph with n vertices and m edges. Which of the following statements is/are TRUE?

- (A) BFS on G stored as an adjacency matrix takes $\Theta(n^2)$ time
- (B) DFS on G stored as adjacency lists takes $\Theta(n + m)$ time
- (C) If G is undirected, every non-tree edge of a BFS tree joins two vertices whose BFS levels differ by at most 1
- (D) If G is undirected, a DFS of G can produce cross edges

Q.5 [NAT · numerical answer | 1 mark | Queues — circular array]

A queue is stored in a circular array $Q[0..6]$ of capacity 7 (all 7 slots usable; a separate counter tracks the size). Initially the queue is empty and the rear index $r = 0$, where r is the slot that the next enqueued element will occupy; after each enqueue, $r = (r + 1) \bmod 7$. The following operations are performed:

5 enqueues, 3 dequeues, 4 enqueues, 2 dequeues, 3 enqueues.

The index of the slot that holds the most recently enqueued element is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Hashing — quadratic probing]

Keys 22, 33, 44, 13, 55 are inserted in that order into an initially empty hash table of size 11 (slots 0–10) using $h(k) = k \bmod 11$ and quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) examines slot $(h(k) + i^2) \bmod 11$. In which slot is key 55 stored?

- (A) 3
- (B) 5
- (C) 9
- (D) 6

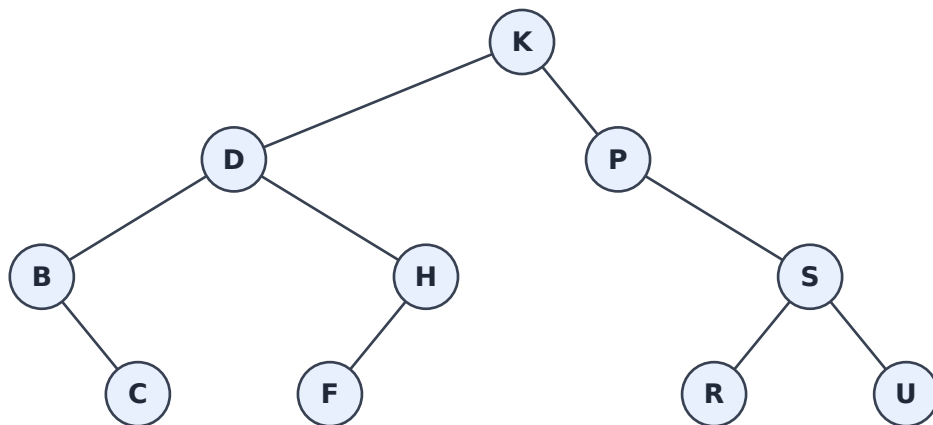
Q.7 [NAT · numerical answer | 1 mark | Selection sort — swap count]

Selection sort (ascending) is applied to [29, 10, 14, 37, 13, 5, 41]. In pass i ($i = 0, 1, \dots, n - 2$) the minimum of $A[i..n-1]$ is found and swapped with $A[i]$ **only if** it is not already at position i . The number of swaps performed is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Binary trees — traversals]

For the binary tree shown below, the **post-order** traversal is

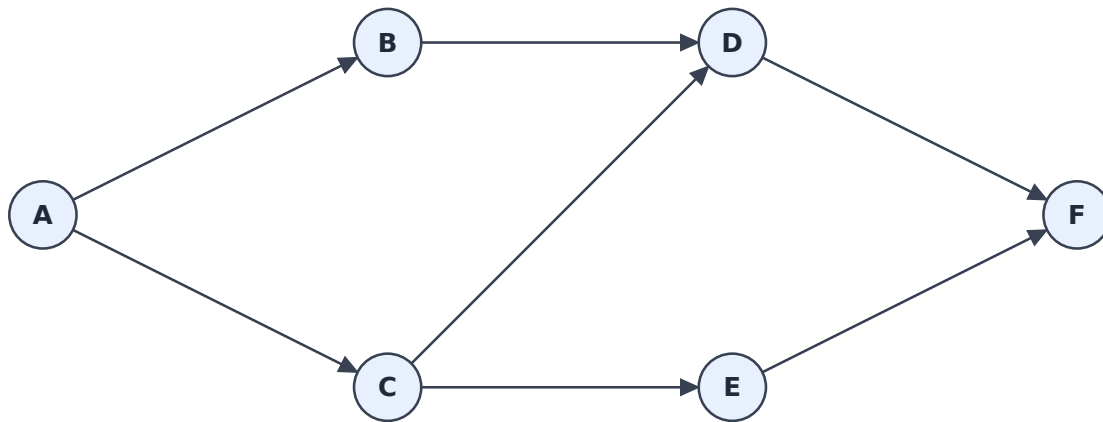


Binary tree

- (A) K, D, B, C, H, F, P, S, R, U
- (B) B, C, D, F, H, K, P, R, S, U
- (C) C, B, F, H, D, P, R, U, S, K
- (D) C, B, F, H, D, R, U, S, P, K

Q.9 [MSQ · one or more correct | 1 mark | DFS — possible visiting orders]

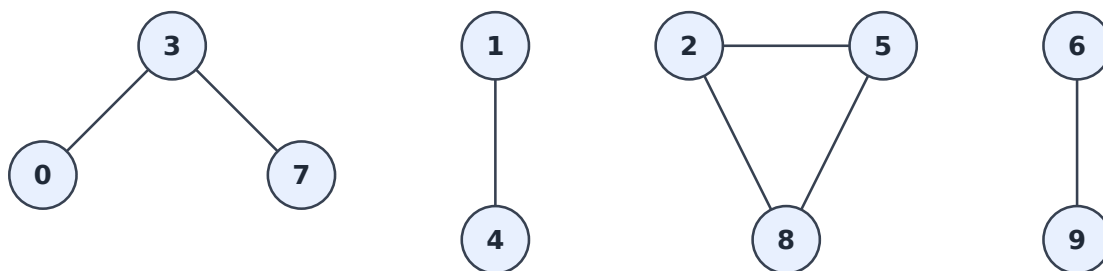
Depth-first search is started at vertex A of the directed graph below; out-neighbours may be explored in **any** order. Which of the following is/are possible orders in which the vertices are discovered?



- (A) A, B, D, F, C, E
- (B) A, C, E, F, D, B
- (C) A, B, D, F, E, C
- (D) A, C, D, F, B, E

Q.10 [MCQ · single correct | 1 mark | Graph traversal — connected components]

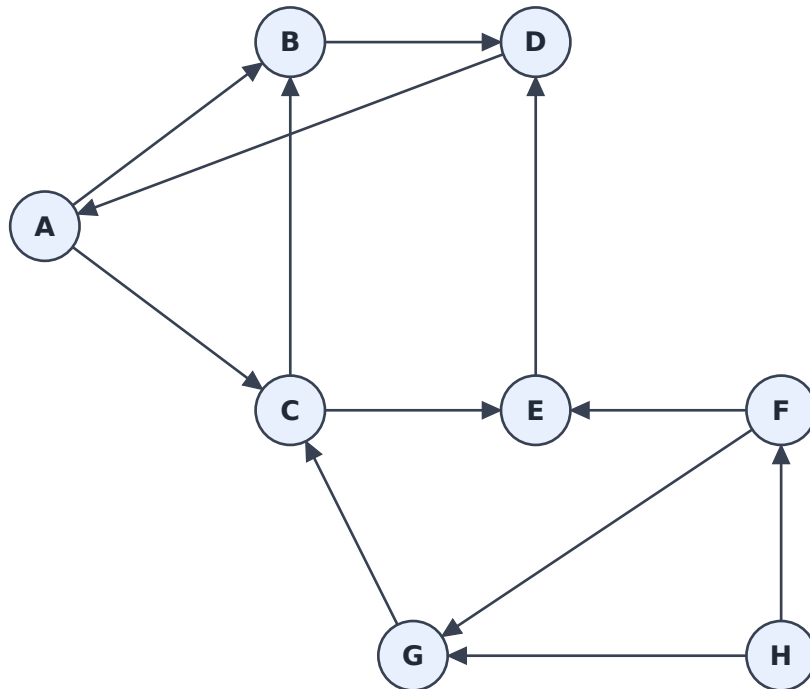
Consider the undirected graph on vertices 0–9 shown below. A program runs `for v in range(10): if not visited[v]: BFS(v)`. What is the minimum number of edges that must be added to make the graph connected? (This equals the number of BFS calls made by the loop, minus one.)



- (A) 2
- (B) 5
- (C) 4
- (D) 3

Q.11 [NAT · numerical answer | 2 marks | DFS — edge classification with restarts]

DFS is run on the directed graph below. The outer loop tries start vertices in alphabetical order (A, B, ..., H), starting a new DFS tree from each vertex that is still undiscovered, and out-neighbours are always explored in alphabetical order. Every edge is classified as a tree, back, forward or cross edge with respect to the resulting DFS forest. The number of **cross** edges is _____.



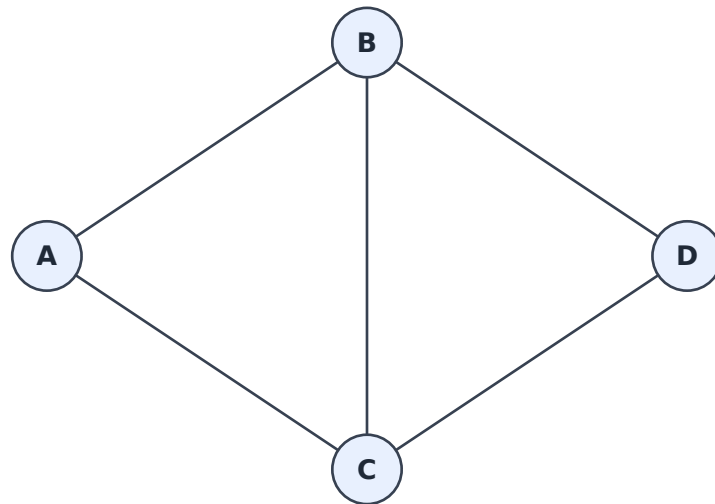
Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | BFS — buggy Python implementation]

The following BFS marks a vertex as seen only when it is **dequeued**. What does the program print?

Python

```
1 from collections import deque
2
3 def bfs(G, s):
4     q, seen, out = deque([s]), set(), []
5     while q:
6         u = q.popleft()
7         out.append(u)
8         seen.add(u)
9         for v in G[u]:
10             if v not in seen:
11                 q.append(v)
12     return out
13
14 G = {'A': 'BC', 'B': 'ACD', 'C': 'ABD', 'D': 'BC'}
15 print("".join(bfs(G, 'A')))
```

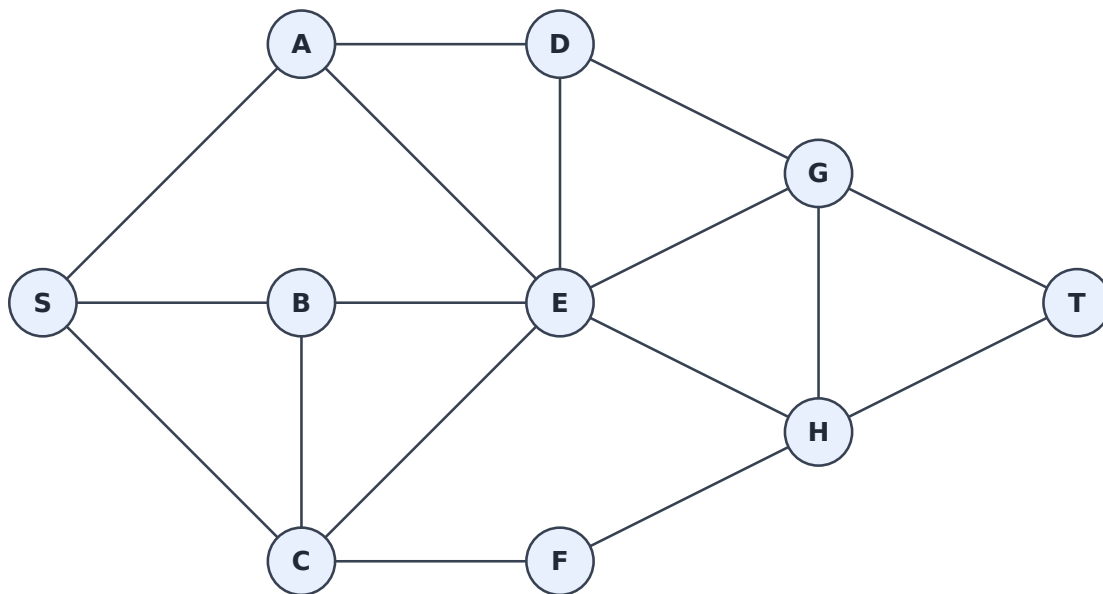


The graph G

- (A) ABCD
- (B) ABCCDD
- (C) ABCCDDD
- (D) ABCDCD

Q.13 [NAT · numerical answer | 2 marks | BFS — counting shortest paths]

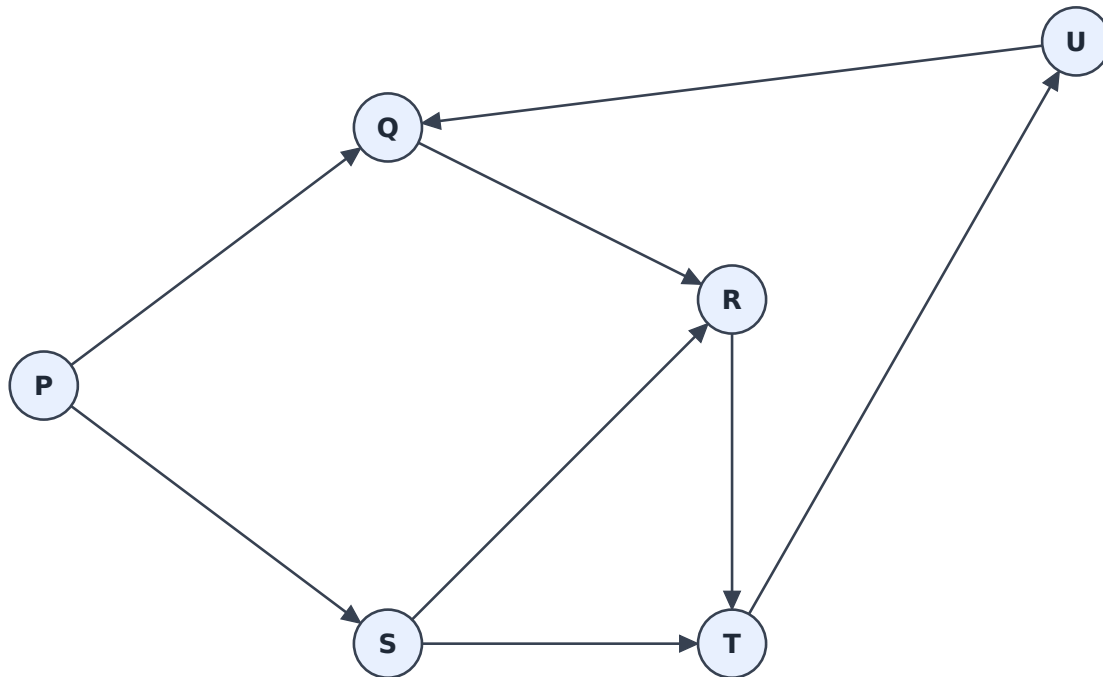
In the unweighted undirected graph below, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.14 [MSQ · one or more correct | 2 marks | DFS — parenthesis structure and edge types]

DFS is run on the directed graph below from vertex P, exploring out-neighbours in alphabetical order, with discovery/finish times from a clock that starts at 1 for the discovery of P. Which of the following statements is/are TRUE?



- (A) The interval $[d(S), f(S)]$ is nested inside the interval $[d(Q), f(Q)]$
- (B) Edge (U, Q) is a back edge
- (C) Edge (S, R) is a cross edge
- (D) The graph contains a directed cycle

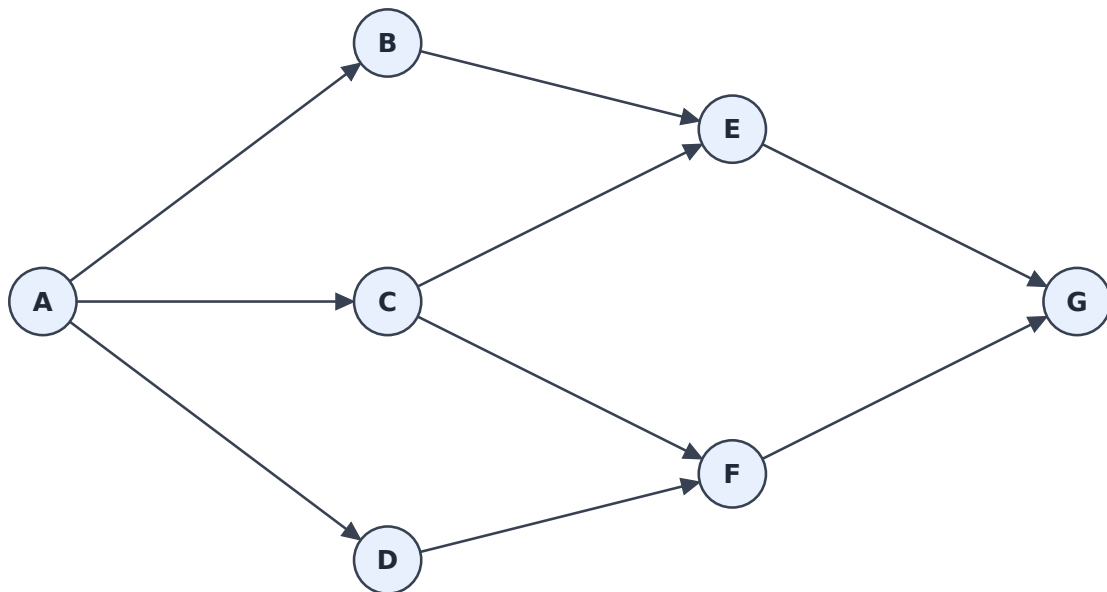
Q.15 [MCQ · single correct | 2 marks | DFS — iterative stack implementation]

Consider the following iterative DFS on the directed graph shown (each adjacency string lists out-neighbours in alphabetical order). What is printed?

Python

```

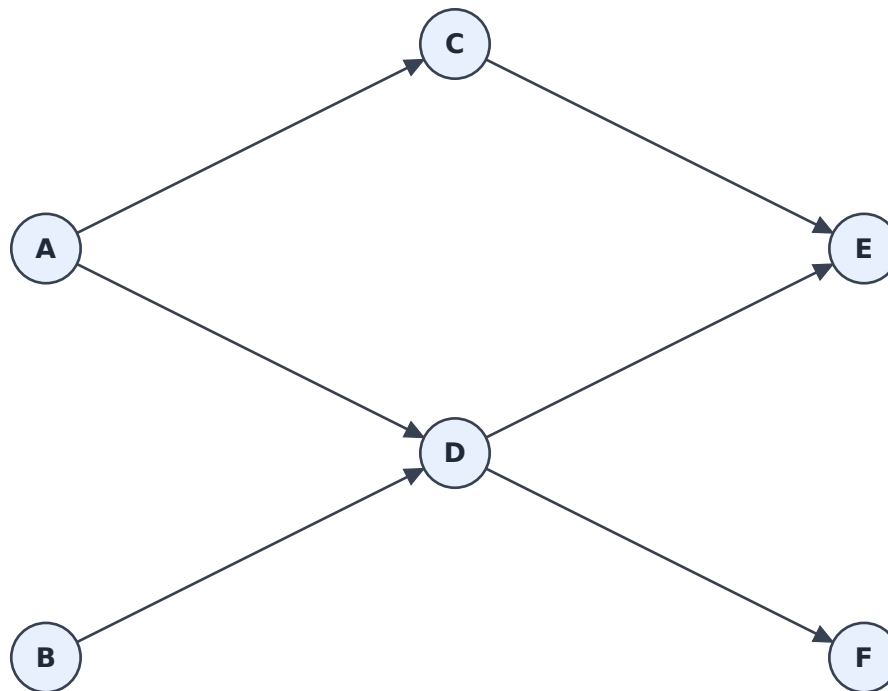
1 def dfs(G, s):
2     seen, st, out = set(), [s], []
3     while st:
4         u = st.pop()
5         if u in seen:
6             continue
7         seen.add(u)
8         out.append(u)
9         for v in G[u]:
10             if v not in seen:
11                 st.append(v)
12     return out
13
14 G = {'A': 'BCD', 'B': 'E', 'C': 'EF', 'D': 'F',
15      'E': 'G', 'F': 'G', 'G': ''}
16 print("".join(dfs(G, 'A')))
```



- (A) ABEGCFD
- (B) ADFGCEB
- (C) ABCDEFG
- (D) ADCBFEG

Q.16 [NAT · numerical answer | 2 marks | DAGs — counting topological orders]

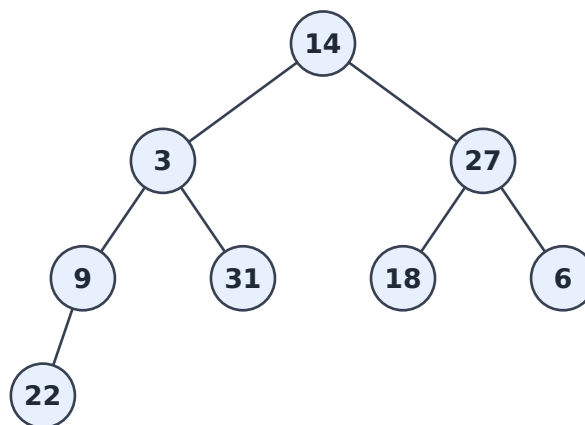
The number of distinct topological orderings of the directed acyclic graph below is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Heaps — bottom-up build-heap]

The array [14, 3, 27, 9, 31, 18, 6, 22] (indices 0-7) is converted into a **max**-heap by the standard bottom-up procedure: call sift-down on $i = 3, 2, 1, 0$ in that order, where sift-down swaps a node with its larger child while that child is larger. Which of the following statements is/are TRUE?

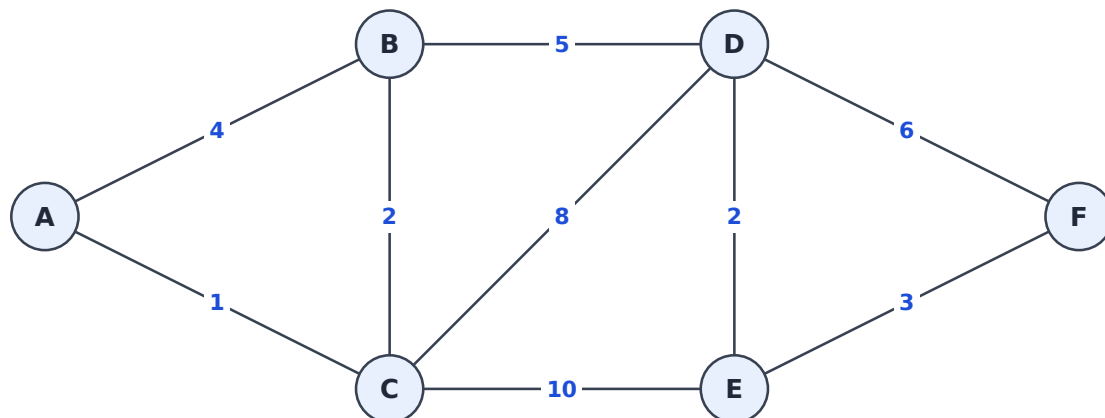


Initial array viewed as a complete binary tree

- (A) Exactly 4 swaps are performed in total
- (B) Inserting the same keys one at a time (in array order) into an empty max-heap with sift-up produces the same final array
- (C) Key 3 ends at a leaf position
- (D) The resulting heap array is [31, 22, 27, 14, 3, 18, 6, 9]

Q.18 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra on an undirected graph]

Dijkstra's algorithm is run from vertex A on the weighted undirected graph below. The sum of the final shortest-path distances from A to all six vertices (including A itself) is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — removing duplicates (buggy loop)]

The following program is intended to remove duplicate values from a sorted singly linked list. What does it print?

Python

```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 head = None
6 for v in reversed([1, 1, 1, 2, 3, 3, 3, 3, 4]):
7     head = Node(v, head)
8 p = head
9 while p and p.next:
10     if p.val == p.next.val:
11         p.next = p.next.next
12     p = p.next
13 out = []
14 while head:
15     out.append(head.val)
16     head = head.next
17 print(out)

```

- (A) [1, 1, 2, 3, 3, 4]
- (B) [1, 2, 3, 4]
- (C) [1, 1, 2, 3, 4]
- (D) [1, 2, 3, 3, 4]

Q.20 [MSQ · one or more correct | 2 marks | Merge sort — comparison counts]

Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ elements and the rest) sorts [21, 7, 33, 14, 2, 40, 19, 11]. Merging compares the front elements of the two halves and stops comparing as soon as one half is exhausted. Which of the following statements is/are TRUE?

- (A) The final (top-level) merge performs 7 comparisons
- (B) Merging [2, 40] with [11, 19] performs 2 comparisons
- (C) No other permutation of 8 distinct keys makes this merge sort perform more comparisons
- (D) The total number of element comparisons is 17

Answer Key — Mock Test 14

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — list aliasing in adjacency lists
2	NAT	1	13	DFS — discovery and finishing times
3	MCQ	1	(A)	BFS — visiting order with tie-breaking
4	MSQ	1	(A), (B), (C)	BFS/DFS — complexity and edge properties
5	NAT	1	4	Queues — circular array
6	MCQ	1	(C)	Hashing — quadratic probing
7	NAT	1	4	Selection sort — swap count
8	MCQ	1	(D)	Binary trees — traversals
9	MSQ	1	(A), (B)	DFS — possible visiting orders
10	MCQ	1	(D)	Graph traversal — connected components
11	NAT	2	6	DFS — edge classification with restarts
12	MCQ	2	(C)	BFS — buggy Python implementation
13	NAT	2	8	BFS — counting shortest paths
14	MSQ	2	(B), (C), (D)	DFS — parenthesis structure and edge types
15	MCQ	2	(B)	DFS — iterative stack implementation
16	NAT	2	12	DAGs — counting topological orders
17	MSQ	2	(A), (C), (D)	Heaps — bottom-up build-heap
18	NAT	2	35	Shortest paths — Dijkstra on an undirected graph
19	MCQ	2	(A)	Linked lists — removing duplicates (buggy loop)
20	MSQ	2	(A), (C), (D)	Merge sort — comparison counts

Detailed Solutions — Mock Test 14

Q.1 [MCQ · 1 mark · Python — list aliasing in adjacency lists]

Answer: (B)

Concept. `[[ ]] * 4` does **not** create four empty lists. It creates one empty list and a new outer list holding four references to that *same* object.

Trace.

- `adj[0].append(1)`, `adj[1].append(2)`, `adj[2].append(3)` all append to the single shared list, which becomes `[1, 2, 3]`.
- `adj[3]` is that same list $\rightarrow \text{len}(\text{adj}[3]) = 3$.
- `adj[0] is adj[2]` compares identities $\rightarrow \text{True}$.

Output: 3 True.

Options.

- (A) is what the student *intended* (vertex 3 has no out-neighbours, lists distinct).
- (C) imagines each append going to a separate list but somehow reaching index 3.
- (D) gets the length right but forgets that the lists are the same object.

Fix: `adj = [[] for _ in range(4)]` evaluates `[]` four times. In a BFS/DFS this bug makes every vertex appear adjacent to every recorded target — a very common silent error.

Q.2 [NAT · 1 mark · DFS — discovery and finishing times]

Answer: 13

Concept. In DFS a vertex finishes only after every vertex reachable from it through white (undiscovered) vertices has finished, so its interval `[d, f]` encloses those of its descendants (parenthesis theorem).

Trace (d / f):

- A d=1 $\rightarrow$ B d=2 $\rightarrow$ C d=3; C $\rightarrow$ A is grey (back edge); C f=4.
- B $\rightarrow$ D d=5; D $\rightarrow$ C is black; D f=6. B f=7.
- A $\rightarrow$ E d=8; E $\rightarrow$ D black (cross edge); E $\rightarrow$ F d=9 $\rightarrow$ G d=10; G $\rightarrow$ E grey (back edge); G f=11, F f=12, **E f=13**.
- A f=14.

So $f(E) = 13$.

Trap: exploring E before B (it is not alphabetical) gives E an earlier interval, and forgetting that every finish also ticks the clock gives values that are too small.

Discovery / finish times

	A	B	C	D	E	F	G
d	1	2	3	5	8	9	10
f	14	7	4	6	13	12	11

Q.3 [MCQ · 1 mark · BFS — visiting order with tie-breaking]

Answer: (A)

Concept. BFS dequeues vertices in non-decreasing order of distance from the source, and within a level in the order they were discovered.

Trace (queue after each dequeue):

- Dequeue A $\rightarrow$ enqueue C, F. Queue: C F.
- Dequeue C $\rightarrow$ neighbours A, B, G; new: B, G. Queue: F B G.
- Dequeue F $\rightarrow$ neighbours A, G, H; new: H. Queue: B G H.

- Dequeue B → new: D. Queue: G H D.
- Dequeue G → new: E. Queue: H D E.
- Dequeue H, D, E (nothing new).

Order: **A, C, F, B, G, H, D, E**. Levels: 0 {A}, 1 {C, F}, 2 {B, G, H}, 3 {D, E}.

Options.

- (B) puts D (level 3) before H (level 2) — impossible in BFS.
- (C) is a depth-first style order (goes from C to its neighbours before F).
- (D) places G before B although both were discovered by C, B first alphabetically; and E before D, though D was enqueued first.

Tip: write the queue explicitly; do not try to read levels off the picture alone.

Q.4 [MSQ · 1 mark · BFS/DFS — complexity and edge properties]

Answer: (A), (B), (C)

- (A) **True.** For each of the n dequeued vertices BFS scans a whole matrix row of length n to find neighbours → $\Theta(n^2)$ regardless of m .
- (B) **True.** Each vertex is visited once and each adjacency list is scanned once: $\Theta(n + \sum \deg) = \Theta(n + m)$.
- (C) **True.** If edge (u, v) had $\text{level}(v) \geq \text{level}(u) + 2$, then when u was dequeued v was still undiscovered and would have been put at $\text{level}(u) + 1$ — contradiction. So every edge, tree or not, spans levels differing by 0 or 1.
- (D) **False.** In an undirected DFS every edge is a tree edge or a back edge. A would-be cross edge (u, v) with v finished before u was discovered is impossible, since v would have explored the edge to the white vertex u before finishing.

Tip: property (C) is why BFS gives shortest paths in unweighted graphs; property (D) is why DFS detects cycles in undirected graphs by looking for any non-tree edge to a non-parent.

Q.5 [NAT · 1 mark · Queues — circular array]

Answer: 4

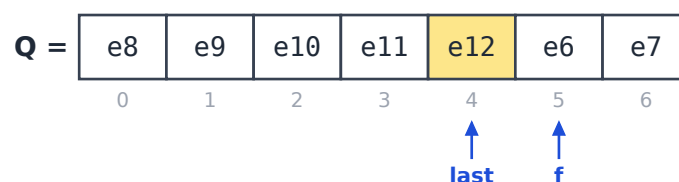
Concept. In a circular buffer only the enqueues move r ; dequeues move the front index f . Size must stay $\leq$ capacity, so first check no overflow occurs.

Trace (size, f , r):

- 5 enqueues → slots 0–4 used; size 5, $f = 0$, $r = 5$.
- 3 dequeues → size 2, $f = 3$.
- 4 enqueues → slots 5, 6, 0, 1; size 6, $r = 2$.
- 2 dequeues → size 4, $f = 5$.
- 3 enqueues → slots 2, 3, 4; size 7 (full, but no overflow), $r = 5$.

The last element went into slot **4** ($= (\text{total enqueues} - 1) \bmod 7 = 11 \bmod 7$).

Trap: answering $r = 5$ — r points to the *next free* slot, one past the last element. Also, the queue is exactly full at the end; with the 'one slot kept empty' convention the last enqueue would have overflowed.



Final buffer ($eK = K$ -th enqueued element)

Q.6 [MCQ · 1 mark · Hashing — quadratic probing]**Answer: (C)**

Concept. Quadratic probing jumps by 1, 4, 9, ... from the home slot, which avoids the long primary clusters of linear probing (but keys with the same home still follow the same probe sequence — secondary clustering).

Trace.

- $22 \rightarrow 0$.
- $33 \rightarrow 0$ (full), $0 + 1 = 1$.
- $44 \rightarrow 0, 1$ full, $0 + 4 = 4$.
- $13 \rightarrow 13 \bmod 11 = 2$.
- $55 \rightarrow 0, 1, 4$ all full; $0 + 9 = 9$ is free.

Options.

- (A) 3 is what *linear* probing would give ($0, 1, 2$ full $\rightarrow 3$).
- (B) 5 adds i instead of i^2 after the third probe.
- (D) 6 is not on 55's probe sequence $0, 1, 4, 9, 5, 3, \dots$

Trap: 22, 33, 44, 55 all hash to 0, so they share one probe sequence — a textbook example of secondary clustering.

0	22
1	33
2	13
3	
4	44
5	
6	
7	
8	
9	55
10	

Final table

Q.7 [NAT · 1 mark · Selection sort — swap count]**Answer: 4**

Concept. Selection sort always makes $n - 1$ passes and $n(n - 1)/2$ comparisons; only the number of actual swaps depends on the data.

Trace.

- $i = 0$: min of whole array is 5 (index 5) $\rightarrow$ swap $\rightarrow [5, 10, 14, 37, 13, 29, 41]$. (1)
- $i = 1$: min of $A[1..]$ is 10, already at index 1 $\rightarrow$ no swap.
- $i = 2$: min 13 at index 4 $\rightarrow$ swap $\rightarrow [5, 10, 13, 37, 14, 29, 41]$. (2)
- $i = 3$: min 14 at index 4 $\rightarrow$ swap $\rightarrow [5, 10, 13, 14, 37, 29, 41]$. (3)
- $i = 4$: min 29 at index 5 $\rightarrow$ swap $\rightarrow [5, 10, 13, 14, 29, 37, 41]$. (4)
- $i = 5$: min 37 already in place $\rightarrow$ no swap.

Total swaps = **4** (with 21 comparisons).

Trap: answering $n - 1 = 6$, which counts the 'swaps' of an element with itself; the question excludes those.

Array after each pass							
start	29	10	14	37	13	5	41
i=0	5	10	14	37	13	29	41
i=1	5	10	14	37	13	29	41
i=2	5	10	13	37	14	29	41
i=3	5	10	13	14	37	29	41
i=4	5	10	13	14	29	37	41
i=5	5	10	13	14	29	37	41

Q.8 [MCQ · 1 mark · Binary trees — traversals]**Answer: (D)**

Concept. Post-order = left subtree, right subtree, root (applied recursively). It is exactly the order in which a DFS of the tree *finishes* the vertices.

Trace.

- Left subtree of K (root D): post(B-subtree) = C, B; post(H-subtree) = F, H; then D → C, B, F, H, D.
- Right subtree of K (root P): P has no left child; post(S-subtree) = R, U, S; then P → R, U, S, P.
- Finally K.

Post-order: **C, B, F, H, D, R, U, S, P, K.**

Options.

- (A) is the pre-order traversal.
- (B) is the in-order traversal (sorted, since this happens to be a BST).
- (C) places P before its own descendants R, U, S — P must come after its right subtree.

Tip: the last element of post-order is always the root; the first element of pre-order is always the root. That alone eliminates (B) and (A).

Q.9 [MSQ · 1 mark · DFS — possible visiting orders]**Answer: (A), (B)**

Concept. DFS always continues from the most recently discovered vertex that still has an undiscovered out-neighbour. It may backtrack only when the current vertex is exhausted.

- (A) A → B → D → F; F, D, B exhausted; back at A → C → E (D, F already seen). **Possible.**
- (B) A → C → E → F; F, E exhausted; back at C → D (F seen); back at A → B. **Possible.**
- (C) After A → B → D → F, backtracking reaches A, whose only undiscovered out-neighbour is C. E is reachable only through C, so E cannot appear before C. **Not possible.**
- (D) A → C → D → F; back at D, then C — but C still has the undiscovered neighbour E, so E must be discovered before returning to A for B. **Not possible.**

Trap: an order can look plausible because every vertex is adjacent to *some* earlier vertex — that is not enough; it must be adjacent to the deepest unfinished vertex.

Q.10 [MCQ · 1 mark · Graph traversal — connected components]**Answer: (D)**

Concept. Each call of BFS from an unvisited vertex discovers exactly one connected component. A graph with c components needs exactly $c - 1$ extra edges to become connected (join the components in a chain).

Trace of the outer loop.

- $v = 0$: BFS(0) visits {0, 3, 7}.

- $v = 1$: BFS(1) visits {1, 4}.
- $v = 2$: BFS(2) visits {2, 5, 8}.
- $v = 3, 4, 5$: already visited.
- $v = 6$: BFS(6) visits {6, 9}.
- $v = 7, 8, 9$: already visited.

4 BFS calls $\rightarrow c = 4$ components $\rightarrow$ **3** edges needed.

Options. (C) 4 is the number of components itself; (A) and (B) miscount the components (e.g. treating the triangle 2-5-8 as two pieces).

Tip: the whole loop still costs $\Theta(n + m)$ — each vertex and edge is processed once across all BFS calls.

Q.11 [NAT · 2 marks · DFS — edge classification with restarts]

Answer: 6

Concept. For edge (u, v) examined from u : v white $\rightarrow$ tree; v grey (on the recursion stack) $\rightarrow$ back; v black and $d[u] < d[v] \rightarrow$ forward; v black and $d[v] < d[u] \rightarrow$ cross.

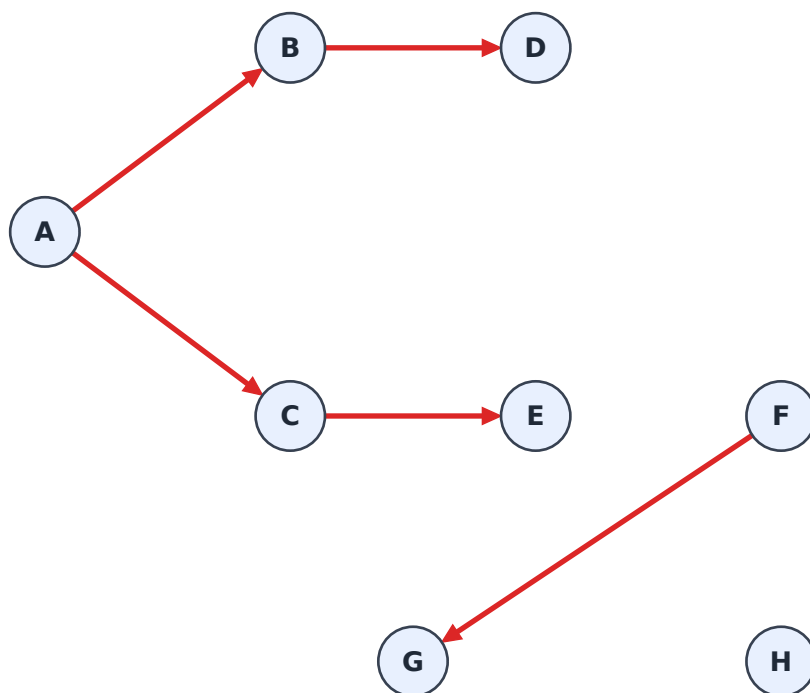
Trace (d/f with a clock starting at 1):

- Tree 1 from A: A 1 $\rightarrow$ B 2 $\rightarrow$ D 3; D $\rightarrow$ A grey: **back**. D f=4, B f=5.
A $\rightarrow$ C 6; C $\rightarrow$ B black, $d[B] = 2 < 6$: **cross**; C $\rightarrow$ E 7; E $\rightarrow$ D black: **cross**; E f=8, C f=9, A f=10.
- B, C, D, E already black. Tree 2 from F: F 11; F $\rightarrow$ E black: **cross**; F $\rightarrow$ G 12; G $\rightarrow$ C black: **cross**; G f=13, F f=14.
- Tree 3 from H: H 15; H $\rightarrow$ F black: **cross**; H $\rightarrow$ G black: **cross**; H f=16.

Classification: tree edges AB, BD, AC, CE, FG (5); back edge DA (1); forward edges none; cross edges CB, ED, FE, GC, HF, HG $\rightarrow$ **6**.

Check: $5 + 1 + 0 + 6 = 12 =$ total number of edges.

Trap: edges into an *earlier* DFS tree (FE, GC, HF, HG) are always cross edges — they can never be back or forward edges because the target is in a different, already finished tree.



DFS forest (trees rooted at A, F, H)

Q.12 [MCQ · 2 marks · BFS — buggy Python implementation]**Answer: (C)**

Concept. Correct BFS marks a vertex when it is *enqueued*. Marking on dequeue lets the same vertex be enqueued several times (once by every neighbour dequeued before it), so it is also output several times.

Trace (queue after processing each vertex):

- pop A → out A; seen {A}; push B, C → [B, C]
- pop B → out AB; seen {A, B}; A seen; push C, D → [C, C, D]
- pop C → out ABC; seen {A, B, C}; push D → [C, D, D]
- pop C → out ABCC; push D → [D, D, D]
- pop D → out ABCCD; seen adds D; neighbours B, C seen → [D, D]
- pop D, pop D → out ABCCDDD.

Printed: **ABCCDDD**.

Options.

- (A) is the output of a correct BFS.
- (B) misses that the *second* copy of C (dequeued before any D) pushes a third D.
- (D) interleaves the duplicates, but FIFO order keeps both Cs ahead of all Ds.

Trap: this bug does not cause an infinite loop (a seen vertex is never pushed again after it is dequeued), so it is easy to miss — but the queue can grow to $\Theta(m)$ and outputs repeat.

Q.13 [NAT · 2 marks · BFS — counting shortest paths]**Answer: 8**

Concept. Run BFS from S. A shortest path to v ends with an edge (u, v) where $\text{level}(u) = \text{level}(v) - 1$, so $\text{ways}(v) = \sum \text{ways}(u)$ over such neighbours u. Edges joining two vertices of the **same** level never lie on a shortest path.

Levels. 0: S; 1: A, B, C; 2: D, E, F; 3: G, H; 4: T.

Same-level edges B-C, D-E, G-H are ignored.

Counting.

- $\text{ways}(A) = \text{ways}(B) = \text{ways}(C) = 1$.
- $\text{ways}(D) = \text{ways}(A) = 1$; $\text{ways}(E) = A + B + C = 3$; $\text{ways}(F) = \text{ways}(C) = 1$.
- $\text{ways}(G) = D + E = 1 + 3 = 4$; $\text{ways}(H) = E + F = 3 + 1 = 4$.
- $\text{ways}(T) = G + H = 4 + 4 = 8$.

The shortest distance is 4 and there are **8** shortest paths.

Trap: counting a path such as S-B-C-E-... (which uses the same-level edge B-C) — it has length 5 to reach the next level and is not shortest.

BFS level and path counts

	S	A	B	C	D	E	F	G	H	T
level	0	1	1	1	2	2	2	3	3	4
ways	1	1	1	1	1	3	1	4	4	8

Q.14 [MSQ · 2 marks · DFS — parenthesis structure and edge types]**Answer: (B), (C), (D)**

Trace (d / f):

- P 1 → Q 2 → R 3 → T 4 → U 5; U → Q: Q is grey → back edge. U f=6, T f=7, R f=8, Q f=9.
- P → S 10; S → R: R black, d(R) = 3 < 10 → cross edge; S → T: black, d(T) = 4 < 10 → cross edge. S f=11. P f=12.

Times: P 1/12, Q 2/9, R 3/8, S 10/11, T 4/7, U 5/6.

- (A) **False** — [10, 11] and [2, 9] are disjoint; S and Q are siblings under P, not ancestor/descendant. By the parenthesis theorem two intervals are either nested or disjoint.
- (B) **True** — Q is an ancestor of U still on the recursion stack.
- (C) **True** — R is finished and in a different subtree (discovered earlier).
- (D) **True** — a directed graph has a cycle iff DFS finds a back edge; here $Q \rightarrow R \rightarrow T \rightarrow U \rightarrow Q$.

Trap: S → R points 'upward' in the drawing but is not a back edge — R is not an ancestor of S in the DFS tree.

Discovery / finish times						
	P	Q	R	S	T	U
d	1	2	3	10	4	5
f	12	9	8	11	7	6

Q.15 [MCQ · 2 marks · DFS — iterative stack implementation]

Answer: (B)

Concept. With an explicit stack, neighbours pushed in alphabetical order are *popped* in reverse alphabetical order, so this DFS prefers the alphabetically **last** neighbour. Marking on pop (with the **continue** check) keeps it a valid DFS order.

Trace (stack shown bottom → top after each step):

- pop A → out A; push B, C, D → [B, C, D]
- pop D → out AD; push F → [B, C, F]
- pop F → out ADF; push G → [B, C, G]
- pop G → out ADFG → [B, C]
- pop C → out ADFGC; E unseen → push E (F seen) → [B, E]
- pop E → out ADFGCE; G seen → [B]
- pop B → out ADFGCEB.

Printed: **ADFGCEB**.

Options.

- (A) is the recursive DFS order with alphabetical exploration.
- (C) is the BFS order.
- (D) treats the stack like a queue of reversed neighbour lists (a 'reverse BFS').

Tip: to make the iterative version match recursive alphabetical DFS, push neighbours in *reverse* alphabetical order.

Q.16 [NAT · 2 marks · DAGs — counting topological orders]

Answer: 12

Concept. Count by repeatedly choosing which current *source* (in-degree 0 vertex) comes first and recursing on the remaining DAG. (A DFS-based topological sort outputs just one of these orders: reverse finishing order.)

Constraints: $A < C$, $A < D$, $B < D$, $C < E$, $D < E$, $D < F$.

Case 1: B first. Then A must come next (it is the only source). The rest {C, D, E, F} with $C < E$, $D < E$, $D < F$:

- C next → remaining D, E, F with D first, then E, F in 2 orders → 2.
- D next → remaining C, E, F with only $C < E$: F can be in any of 3 slots around C, E → 3.
- Subtotal $X = 5$.

Case 2: A first. Sources now B, C.

- B next $\rightarrow$ the same sub-problem $\{C, D, E, F\} \rightarrow 5$.
- C next $\rightarrow$ remaining B, D, E, F: $B < D, D < E, D < F \rightarrow B, D$, then E/F in 2 orders $\rightarrow 2$.
- Subtotal = 7.

Total = $5 + 7 = 12$.

Trap: multiplying independent-looking counts (e.g. 2 sources $\times$...) double counts; always branch on the first vertex and recurse.

Q.17 [MSQ · 2 marks · Heaps — bottom-up build-heap]

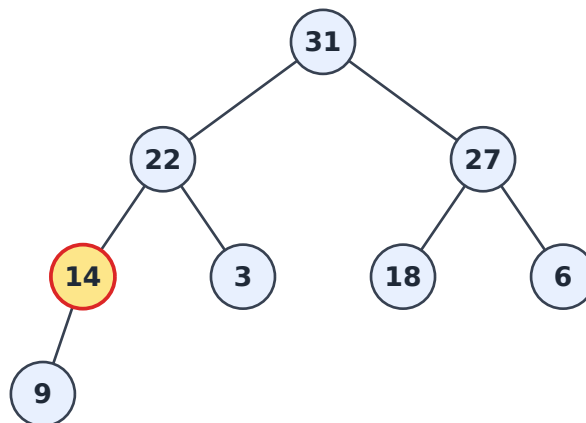
Answer: (A), (C), (D)

Concept. Bottom-up heap construction sifts down every internal node from the last one (index $\lfloor n/2 \rfloor - 1 = 3$) to the root. Total work is $O(n)$.

Trace.

- $i = 3$ (9): child index 7 = 22 $>$ 9 $\rightarrow$ swap. [14, 3, 27, 22, 31, 18, 6, 9] (1 swap)
- $i = 2$ (27): children 18, 6 $\rightarrow$ no swap.
- $i = 1$ (3): children 22, 31 $\rightarrow$ swap with 31 (index 4); index 4 has no children. [14, 31, 27, 22, 3, 18, 6, 9] (2)
- $i = 0$ (14): children 31, 27 $\rightarrow$ swap with 31 $\rightarrow$ index 1: children 22, 3 $\rightarrow$ swap with 22 $\rightarrow$ index 3: child 9 $<$ 14 $\rightarrow$ stop. [31, 22, 27, 14, 3, 18, 6, 9] (4)
- (A) **True** — $1 + 0 + 1 + 2 = 4$ swaps.
- (B) **False** — repeated insertion gives [31, 27, 18, 22, 9, 14, 6, 3]. The two methods generally produce different (both valid) heaps.
- (C) **True** — 3 is at index 4, whose children 9, 10 are beyond $n - 1 = 7$.
- (D) **True**.

Trap: stopping a sift-down after one swap; the key 14 must keep sinking until it is $\geq$ both children.



Max-heap after build-heap

Q.18 [NAT · 2 marks · Shortest paths — Dijkstra on an undirected graph]

Answer: 35

Concept. Dijkstra finalises vertices in increasing order of distance; once extracted, a vertex's distance never changes (all weights are non-negative).

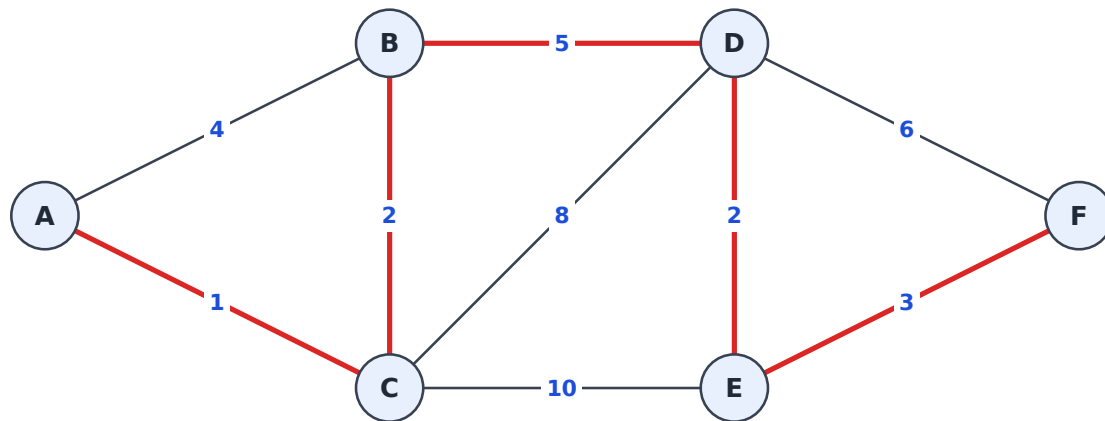
Trace (extracted vertex: updates):

- A (0): B = 4, C = 1.
- C (1): B = $\min(4, 1 + 2) = 3$; D = $1 + 8 = 9$; E = $1 + 10 = 11$.
- B (3): D = $\min(9, 3 + 5) = 8$.
- D (8): E = $\min(11, 8 + 2) = 10$; F = $8 + 6 = 14$.
- E (10): F = $\min(14, 10 + 3) = 13$.

- F (13).

Distances: A 0, B 3, C 1, D 8, E 10, F 13. Sum = $0 + 3 + 1 + 8 + 10 + 13 = 35$.

Trap: taking the direct edges $A-B = 4$ or $C-E = 10$ at face value; both are beaten by detours ($A-C-B = 3$, $A-C-B-D-E = 10$ vs 11). Note also F is reached via E (13), not directly from D (14).



Shortest-path tree from A

Q.19 [MCQ · 2 marks · Linked lists — removing duplicates (buggy loop)]

Answer: (A)

Concept. After deleting `p.next`, the correct algorithm must **stay** on `p` (the new `p.next` may also equal `p`). This loop advances unconditionally, so in a run of equal values it deletes only every second extra copy.

Trace (subscripts mark copies): list 1a 1b 1c 2 3a 3b 3c 3d 4.

- `p = 1a`: next 1b equal → unlink 1b ($1a \rightarrow 1c$); `p = 1c`.
- `p = 1c`: next 2 differs; `p = 2`.
- `p = 2`: next 3a differs; `p = 3a`.
- `p = 3a`: next 3b equal → unlink 3b ($3a \rightarrow 3c$); `p = 3c`.
- `p = 3c`: next 3d equal → unlink 3d ($3c \rightarrow 4$); `p = 4`.
- `p = 4`: `p.next` is None → stop.

Result: 1a, 1c, 2, 3a, 3c, 4 → [1, 1, 2, 3, 3, 4].

Options.

- (B) is the intended result (needs `else: p = p.next`).
- (C) assumes the run of four 3s is fixed completely.
- (D) assumes the run of three 1s is fixed completely.

Trap: a run of length k keeps $\lfloor k/2 \rfloor$ copies with this bug: 3 ones → 2, 4 threes → 2.



List after the buggy loop

Q.20 [MSQ · 2 marks · Merge sort — comparison counts]

Answer: (A), (C), (D)

Concept. Merging lists of sizes p and q takes between $\min(p, q)$ and $p + q - 1$ comparisons. For $n = 8$ the worst case total is $\sum \text{over levels} = 4 \times 1 + 2 \times 3 + 1 \times 7 = 17$ ($= n \log_2 n - n + 1$).

Trace.

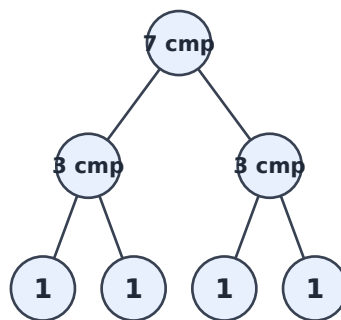
- Level 1 (pairs): $[21] [7] \rightarrow 1$; $[33] [14] \rightarrow 1$; $[2] [40] \rightarrow 1$; $[19] [11] \rightarrow 1$. Subtotal 4.

- Level 2: [7, 21] + [14, 33]: $7 < 14$, $21 > 14$, $21 < 33$, then left empty $\rightarrow 3$. [2, 40] + [11, 19]: $2 < 11$, $40 > 11$, $40 > 19$, right empty $\rightarrow 3$. Subtotal 6.
- Level 3: [7, 14, 21, 33] + [2, 11, 19, 40]: outputs 2, 7, 11, 14, 19, 21, 33 each after one comparison, then 40 is copied $\rightarrow 7$.
- Total = $4 + 6 + 7 = 17$.

Statements.

- (A) **True** — the halves interleave perfectly, so every merge step but the last needs a comparison.
- (B) **False** — it needs 3 comparisons, because 40 outlasts both 11 and 19.
- (C) **True** — every merge here hit its maximum $p + q - 1$, and 17 is the worst case for $n = 8$.
- (D) **True**.

Trap: assuming merging two lists of size 2 always takes 2 comparisons; it is 2 only when one list is entirely smaller than the other.



Comparisons per merge in the recursion tree

Mock Test 15

Shortest Paths

Difficulty: Moderate

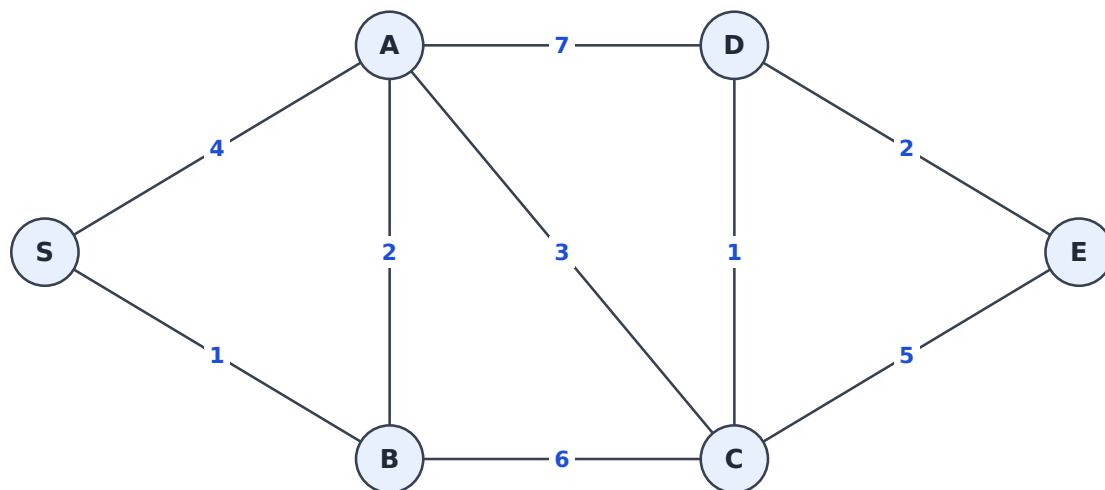
Focus areas: Dijkstra traces, BFS shortest paths, path counting

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Shortest paths — Dijkstra extraction order]

Dijkstra's algorithm is run from S on the weighted undirected graph shown. In which order are the vertices removed from the priority queue (finalised)?



- (A) S, B, A, C, D, E
- (B) S, B, A, D, C, E
- (C) S, B, C, A, D, E
- (D) S, A, B, C, D, E

Q.2 [NAT · numerical answer | 1 mark | Binary search — tracing mid values]

The function below is called as `bs(A, 60)` on the sorted array A shown (0-based). The sum of all values taken by `mid` during the call is _____.

Python

```
1 def bs(A, x):
2     lo, hi = 0, len(A) - 1
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         if A[mid] == x:
6             return mid
7         if A[mid] < x:
8             lo = mid + 1
9         else:
10            hi = mid - 1
11    return -1
```

A =	2	5	8	12	16	23	38	56	72	91	99
	0	1	2	3	4	5	6	7	8	9	10

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Python — dictionaries and max with key]

Consider the following Python program. What is printed?

Python

```
1 d = {}
2 for w in "the cat and the hat and the bat".split():
3     d[w[-2:]] = d.get(w[-2:], 0) + len(w)
4 print(len(d), max(d, key=d.get), d["at"])
```

- (A) 4 he 9
- (B) 3 he 9
- (C) 3 at 9
- (D) 3 he 3

Q.4 [MSQ · one or more correct | 1 mark | Python — floor division and modulo with negatives]

Which of the following Python 3 expressions evaluate to `True`?

- (A) `-7 % 3 == 2`
- (B) `7 % -3 == -2`
- (C) `int(-7 / 2) == -7 // 2`
- (D) `-7 // 2 == -3`

Q.5 [NAT · numerical answer | 1 mark | Queues — circular array arithmetic]

A circular queue is implemented in an array `Q[0..7]`. `front` is the index of the first element and `rear` is the index of the next free slot; enqueue writes `Q[rear]` then sets `rear = (rear + 1) mod 8`, and dequeue sets `front = (front + 1) mod 8`. Initially `front = rear = 5` (empty queue). The operations performed are: 6 enqueues, then 4 dequeues, then 3 enqueues. The index of the slot that holds the **most recently enqueued** element is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Binary search trees — insertion]

The keys 50, 27, 73, 14, 39, 61, 88, 33, 45, 30 are inserted in that order into an initially empty binary search tree. The sequence of keys on the path from the root to the node containing 30 is

- (A) 50, 27, 33, 30
- (B) 50, 27, 14, 30
- (C) 50, 27, 39, 30
- (D) 50, 27, 39, 33, 30

Q.7 [NAT · numerical answer | 1 mark | Hashing — linear probing]

The keys 33, 46, 18, 57, 24, 13, 70, 35 are inserted in that order into an initially empty hash table with 11 slots (0-10) using $h(k) = k \bmod 11$ and linear probing (next slot = $(i + 1) \bmod 11$). A *probe* is one inspection of a slot, including the slot where the key is finally placed. The total number of probes over all eight insertions is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Selection sort — intermediate state]

Selection sort (ascending) is applied to [29, 10, 14, 37, 13, 5, 41]. In pass i ($i = 0, 1, 2, \dots$) the minimum of $A[i..n-1]$ is found and swapped with $A[i]$ (a swap with itself changes nothing). What is the array after **three** passes?

- (A) [5, 10, 13, 37, 14, 29, 41]
- (B) [5, 10, 13, 14, 29, 37, 41]
- (C) [5, 10, 14, 37, 13, 29, 41]
- (D) [5, 10, 13, 29, 14, 37, 41]

Q.9 [NAT · numerical answer | 1 mark | Graphs — counting walks with the adjacency matrix]

The adjacency matrix M of a directed graph on vertices 1, 2, 3, 4 is shown ($M[i][j] = 1$ iff there is an edge $i \rightarrow j$). The number of directed walks of length exactly 3 (three edges, vertices may repeat) from vertex 4 to vertex 1 is _____.

	M			
	1	2	3	4
1	0	1	1	0
2	0	0	1	1
3	1	0	0	1
4	1	1	0	0

Answer: _____ (enter a numerical value)

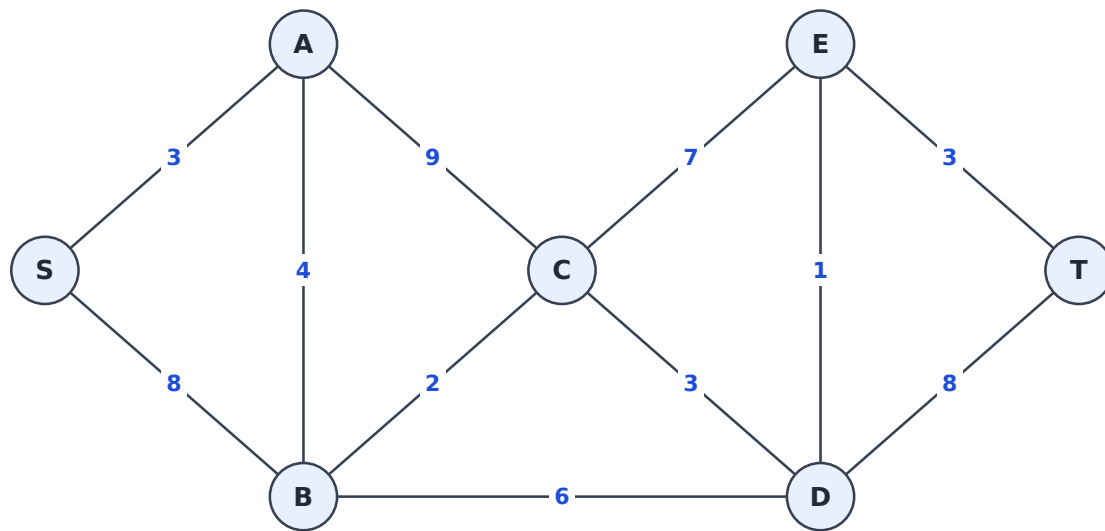
Q.10 [MCQ · single correct | 1 mark | Merge sort — comparisons in a merge]

The standard merge procedure combines the sorted lists $L = [4, 11, 15, 32, 38]$ and $R = [6, 9, 12, 13]$. It compares the front elements, outputs the smaller, and when one list is exhausted copies the rest of the other **without comparisons**. The number of key comparisons made is

- (A) 8
- (B) 7
- (C) 9
- (D) 6

Q.11 [NAT · numerical answer | 2 marks | Shortest paths — Dijkstra on an undirected graph]

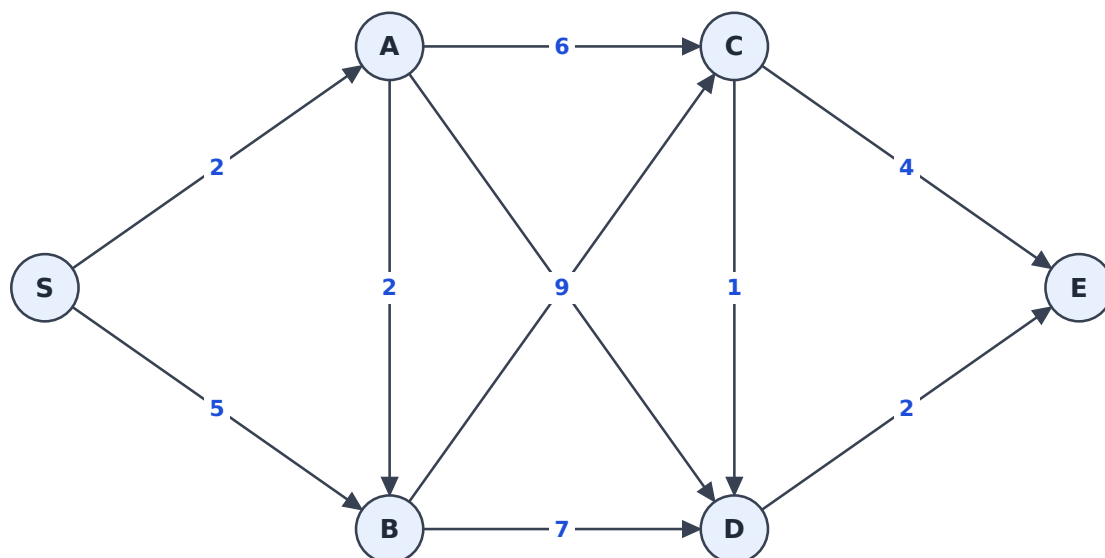
Dijkstra's algorithm is run from S on the weighted undirected graph shown. The shortest-path distance from S to T is _____.



Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra trace and shortest-path tree]

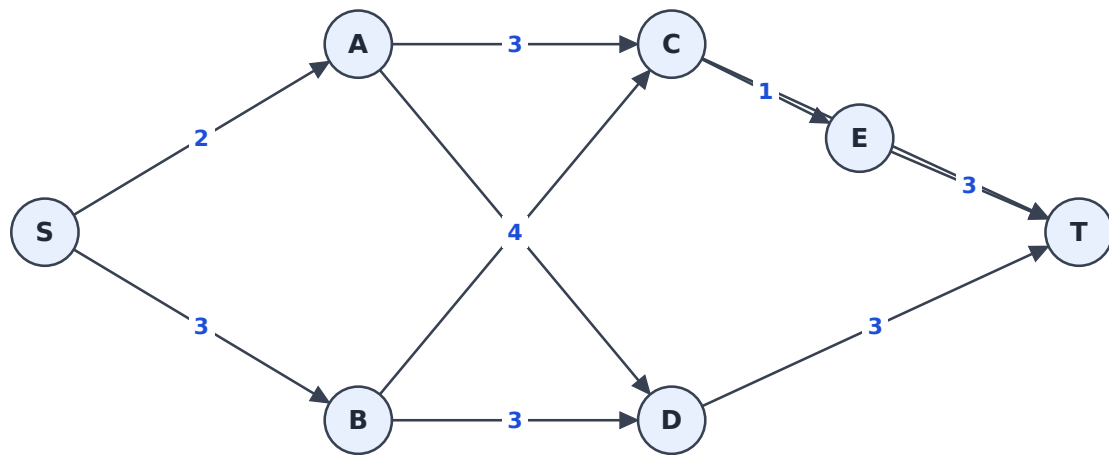
Dijkstra's algorithm is run from S on the weighted directed graph shown. An *update* means a tentative distance that is already finite is decreased (the first assignment of a finite value to a vertex is **not** an update). Which of the following statements is/are TRUE?



- (A) The shortest path from S to E has exactly 5 edges
- (B) Exactly 4 updates occur during the run
- (C) In the shortest-path tree, the parent of D is B
- (D) The vertices are finalised in the order S, A, B, C, D, E

Q.13 [NAT · numerical answer | 2 marks | Shortest paths — counting shortest paths]

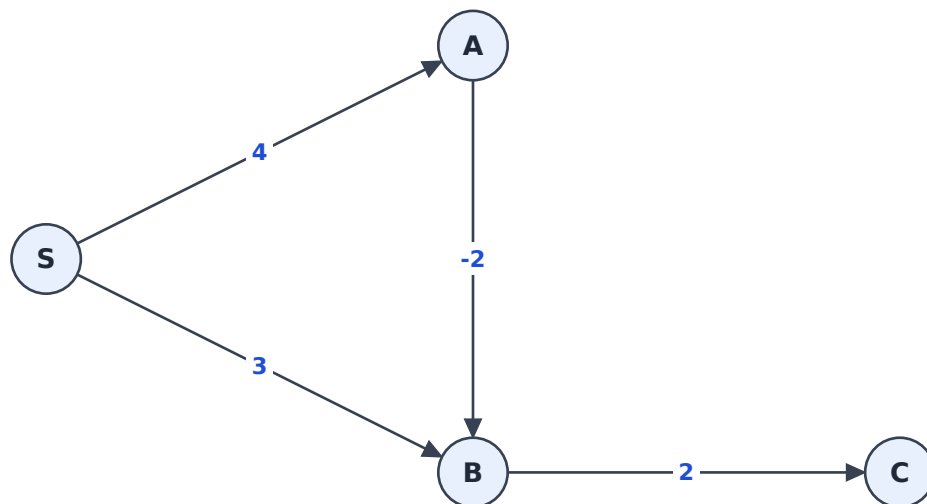
In the weighted directed graph shown, the number of distinct shortest paths (all of minimum total weight) from S to T is _____.



Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Shortest paths — Dijkstra with a negative edge]

The standard Dijkstra algorithm (once a vertex is extracted from the priority queue its distance is never changed again) is run from S on the directed graph shown, which has one negative edge. Which pair gives the distances that the algorithm **reports** for B and C?



- (A) B = 3, C = 4
- (B) B = 2, C = 4
- (C) B = 3, C = 5
- (D) B = 2, C = 5

Q.15 [MCQ · single correct | 2 marks | Python — a buggy BFS (stack instead of queue)]

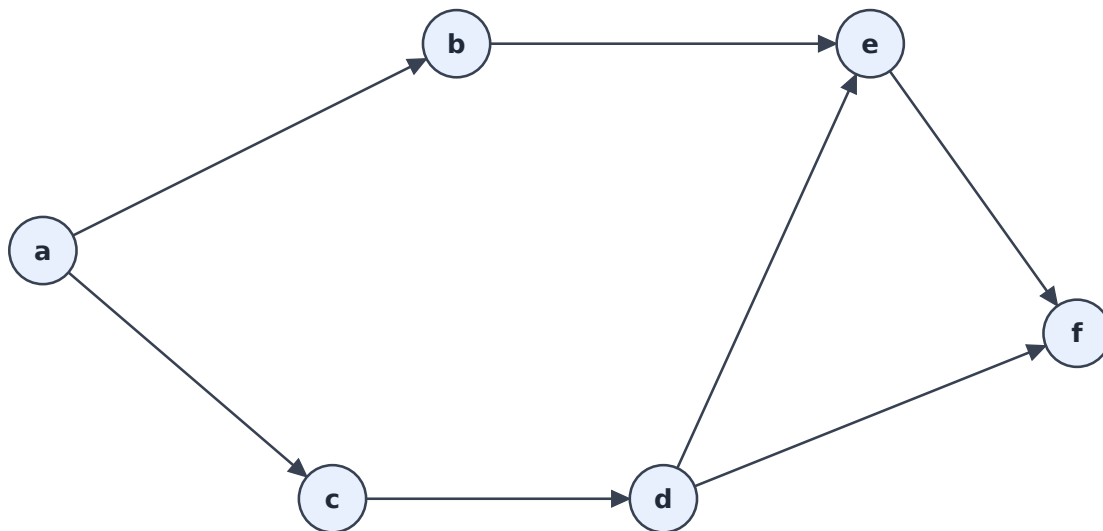
The function below was meant to compute BFS distances, but it uses a Python list as a stack. It is run on the directed graph shown. What is printed?

Python

```

1 def bfs(adj, s):
2     dist = {s: 0}
3     frontier = [s]
4     while frontier:
5         u = frontier.pop()
6         for v in adj[u]:
7             if v not in dist:
8                 dist[v] = dist[u] + 1
9                 frontier.append(v)
10    return dist
11
12 adj = {'a': ['b', 'c'], 'b': ['e'], 'c': ['d'],
13        'd': ['e', 'f'], 'e': ['f'], 'f': []}
14 d = bfs(adj, 'a')
15 print([d[k] for k in sorted(d)])

```



- (A) [0, 1, 1, 2, 2, 3]
- (B) [0, 1, 1, 2, 3, 4]
- (C) [0, 1, 1, 2, 3, 3]
- (D) [0, 1, 1, 2, 2, 4]

Q.16 [MSQ · one or more correct | 2 marks | Stacks — permutations obtainable with a stack]

The integers 1, 2, 3, 4, 5 are pushed onto an initially empty stack in this order; pops may be interleaved with the pushes arbitrarily, and each popped value is appended to an output sequence. Which of the following output sequences is/are possible?

- (A) 3, 2, 5, 4, 1
- (B) 2, 5, 3, 4, 1
- (C) 4, 3, 5, 1, 2
- (D) 1, 4, 3, 5, 2

Q.17 [NAT · numerical answer | 2 marks | Python — memoised path counting on a grid]

Consider the following Python program, which counts monotone lattice paths (each step decreases r or c by 1) that avoid blocked cells. The value printed is _____.

Python

```
1 from functools import lru_cache
2
3 BLOCK = {(1, 1), (2, 3), (3, 0)}
4
5 @lru_cache(maxsize=None)
6 def ways(r, c):
7     if (r, c) in BLOCK or r < 0 or c < 0:
8         return 0
9     if r == 0 and c == 0:
10        return 1
11    return ways(r - 1, c) + ways(r, c - 1)
12
13 print(ways(3, 4))
```

Answer: _____ (enter a numerical value)

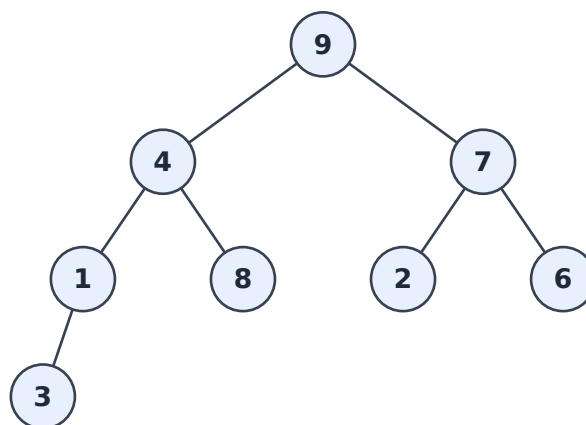
Q.18 [MSQ · one or more correct | 2 marks | Elementary sorting — bubble and insertion sort]

Consider the array $[6, 1, 8, 3, 7, 2]$ to be sorted in ascending order. Bubble sort makes left-to-right passes swapping adjacent out-of-order pairs; insertion sort inserts $A[1]$, $A[2]$, ... in turn into the sorted prefix. Which of the following statements is/are TRUE?

- (A) The first pass of bubble sort performs exactly 5 swaps
- (B) The array contains exactly 8 inversions
- (C) After insertion sort has inserted $A[1]$, $A[2]$ and $A[3]$, the array is $[1, 3, 6, 8, 7, 2]$
- (D) After the first pass of bubble sort the array is $[1, 6, 3, 7, 2, 8]$

Q.19 [MSQ · one or more correct | 2 marks | Heaps — bottom-up build-heap]

The bottom-up build-heap procedure (sift-down at indices $\lfloor n/2 \rfloor - 1$ down to 0, each time swapping with the **smaller** child while it is smaller) converts the array $[9, 4, 7, 1, 8, 2, 6, 3]$ (0-based) into a min-heap. Which of the following statements is/are TRUE?



Input array viewed as a complete binary tree

- (A) Key 9 ends up in a leaf
- (B) The resulting heap array is $[1, 3, 2, 4, 8, 7, 6, 9]$
- (C) The sift-down started at index 1 performs exactly one swap
- (D) Exactly 6 swaps are performed in total

Q.20 [MCQ · single correct | 2 marks | Linked lists — repeated move-last-to-front]

Consider the following Python program on the singly linked list shown. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 head = None
6 for v in [50, 40, 30, 20, 10]:
7     head = Node(v, head)
8
9 steps = 0
10 for _ in range(7):
11     prev, cur = None, head
12     while cur.nxt:
13         prev, cur = cur, cur.nxt
14         steps += 1
15     prev.nxt = None
16     cur.nxt = head
17     head = cur
18
19 out = []
20 while head:
21     out.append(head.v)
22     head = head.nxt
23 print(out, steps)

```



List built by the first loop

- (A) [30, 40, 50, 10, 20] 28
 (B) [40, 50, 10, 20, 30] 28
 (C) [40, 50, 10, 20, 30] 35
 (D) [20, 30, 40, 50, 10] 28

Answer Key — Mock Test 15

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Shortest paths — Dijkstra extraction order
2	NAT	1	26	Binary search — tracing mid values
3	MCQ	1	(B)	Python — dictionaries and max with key
4	MSQ	1	(A), (B)	Python — floor division and modulo with negatives
5	NAT	1	5	Queues — circular array arithmetic
6	MCQ	1	(D)	Binary search trees — insertion
7	NAT	1	22	Hashing — linear probing
8	MCQ	1	(A)	Selection sort — intermediate state
9	NAT	1	3	Graphs — counting walks with the adjacency matrix
10	MCQ	1	(D)	Merge sort — comparisons in a merge
11	NAT	2	16	Shortest paths — Dijkstra on an undirected graph
12	MSQ	2	(A), (B), (D)	Shortest paths — Dijkstra trace and shortest-path tree
13	NAT	2	6	Shortest paths — counting shortest paths
14	MCQ	2	(C)	Shortest paths — Dijkstra with a negative edge
15	MCQ	2	(C)	Python — a buggy BFS (stack instead of queue)

Q	Type	Marks	Key	Topic
16	MSQ	2	(A), (D)	Stacks — permutations obtainable with a stack
17	NAT	2	6	Python — memoised path counting on a grid
18	MSQ	2	(B), (C), (D)	Elementary sorting — bubble and insertion sort
19	MSQ	2	(A), (B), (D)	Heaps — bottom-up build-heap
20	MCQ	2	(B)	Linked lists — repeated move-last-to-front

Detailed Solutions — Mock Test 15

Q.1 [MCQ · 1 mark · Shortest paths — Dijkstra extraction order]

Answer: (A)

Concept. Dijkstra always finalises the unvisited vertex with the smallest tentative distance, so vertices are finalised in non-decreasing order of their true shortest distance.

Trace:

- S (0): A = 4, B = 1.
- B (1): A = $\min(4, 1 + 2) = 3$, C = $1 + 6 = 7$.
- A (3): C = $\min(7, 3 + 3) = 6$, D = $3 + 7 = 10$.
- C (6): D = $\min(10, 6 + 1) = 7$, E = $6 + 5 = 11$.
- D (7): E = $\min(11, 7 + 2) = 9$.
- E (9).

Order: S, B, A, C, D, E → (A).

- (B) finalises D before C — but D's distance (7) depends on C (6).
- (C) finalises C at 7 via B, before A's improvement to 6 is seen.
- (D) takes A before B, ignoring that B (1) is closer than A (4).

Trap: the direct edges S-A (4) and A-D (7) are never on a shortest path.

Q.2 [NAT · 1 mark · Binary search — tracing mid values]

Answer: 26

Concept. Each iteration computes $\text{mid} = (\text{lo} + \text{hi}) // 2$, compares A[mid] with x and discards half of the range. 60 is absent, so the loop runs until $\text{lo} > \text{hi}$.

Trace:

- lo 0, hi 10 → mid 5 (23) < 60 → lo = 6
- lo 6, hi 10 → mid 8 (72) > 60 → hi = 7
- lo 6, hi 7 → mid 6 (38) < 60 → lo = 7
- lo 7, hi 7 → mid 7 (56) < 60 → lo = 8; now lo > hi, return -1.

Sum of mids = $5 + 8 + 6 + 7 = 26$.

Trap: with `//` the mid of (6, 7) is 6, not 7. Note also that the final value of lo (8) is the insertion point of 60 — the index of the first element > 60.

Q.3 [MCQ · 1 mark · Python — dictionaries and max with key]

Answer: (B)

Concept. `d.get(k, 0)` returns 0 for a missing key, so the loop accumulates word lengths grouped by the last two letters. `max(d, key=d.get)` returns the **first** key (in insertion order) attaining the maximum value.

Trace (all words have length 3):

- "the" ×3 → key **he**: 9
- "cat", "hat", "bat" → key **at**: 9
- "and" ×2 → key **nd**: 6

Insertion order is **he**, **at**, **nd**. Both **he** and **at** have 9; **max** keeps the first maximum it meets, which is **he**. Output: `3 he 9` → (B).

- (A) counts four distinct words-endings, but **cat/hat/bat** share **at**.
- (C) assumes ties go to the later key.
- (D) counts occurrences instead of summing lengths.

Tip: `max/min` are stable in the sense that they return the first of equal candidates; dicts preserve insertion order since Python 3.7.

Q.4 [MSQ · 1 mark · Python — floor division and modulo with negatives]

Answer: (A), (B)

Concept. Python's `//` floors (rounds toward $-\infty$) and `%` is defined so that $a == (a // b) * b + a \% b$; hence the remainder takes the **sign of the divisor**. `int()` truncates toward 0.

- (A) $-7 // 3 = -3$ (floor of -2.33), remainder $-7 - (-9) = 2$. **True.**
- (B) $7 // -3 = -3$ (floor of -2.33), remainder $7 - 9 = -2$. **True.**
- (C) `int(-3.5) = -3` (truncation) but $-7 // 2 = -4$. **False.**
- (D) $-7 / 2 = -3.5$, floor $\rightarrow -4$. So $-7 // 2$ is $-4 \neq -3$. **False.**

Trap: C and Java truncate toward zero, so $-7 / 2$ is -3 there and $-7 \% 3$ is -1 . Python differs whenever the operands have opposite signs.

Q.5 [NAT · 1 mark · Queues — circular array arithmetic]

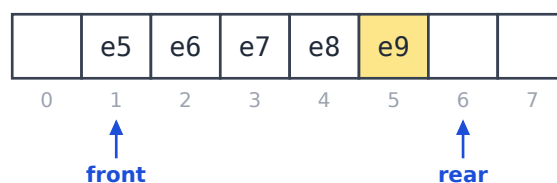
Answer: 5

Concept. Index arithmetic is modulo the array size. Count enqueues for `rear` and dequeues for `front`; the last element written sits at $(rear - 1) \bmod 8$.

- Total enqueues = $6 + 3 = 9 \rightarrow rear = (5 + 9) \bmod 8 = 14 \bmod 8 = 6$.
- Total dequeues = $4 \rightarrow front = (5 + 4) \bmod 8 = 1$.
- Size never exceeds 6 (after the first 6 enqueues) and ends at $9 - 4 = 5$, so the queue never overflows (capacity 7 with one slot kept empty).
- Last enqueued element is at $(6 - 1) \bmod 8 = 5$.

Slot-by-slot: the 9 enqueues write slots 5, 6, 7, 0, 1, 2, 3, 4, 5 — the 9th write reuses slot 5, which was freed by the first dequeue.

Trap: answering 6 (the value of `rear`, which is the next free slot).



Final state (e_i = i -th enqueued element)

Q.6 [MCQ · 1 mark · Binary search trees — insertion]

Answer: (D)

Concept. Each key is inserted as a leaf by walking down from the root: go left if smaller, right if larger.

Insertions:

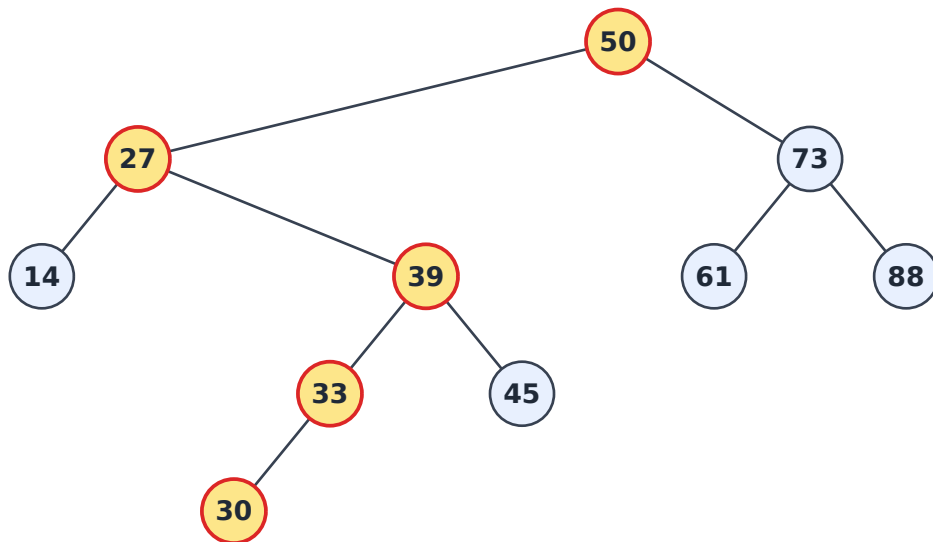
- 50 root; 27 $\rightarrow$ L of 50; 73 $\rightarrow$ R of 50.
- 14 $\rightarrow$ L of 27; 39 $\rightarrow$ R of 27; 61 $\rightarrow$ L of 73; 88 $\rightarrow$ R of 73.
- 33: 50 $\rightarrow$ L, 27 $\rightarrow$ R, 39 $\rightarrow$ L $\Rightarrow$ left child of 39.
- 45: 50 $\rightarrow$ L, 27 $\rightarrow$ R, 39 $\rightarrow$ R $\Rightarrow$ right child of 39.
- 30: 50 $\rightarrow$ L ($30 < 50$), 27 $\rightarrow$ R ($30 > 27$), 39 $\rightarrow$ L ($30 < 39$), 33 $\rightarrow$ L ($30 < 33$) $\Rightarrow$ left child of 33.

Path: 50, 27, 39, 33, 30 $\rightarrow$ (D).

- (B) goes left at 27; but $30 > 27$.
- (C) skips 33, which was inserted before 30 and is on the way.

- (A) skips 39.

Tip: the depth of 30 is 4 — later keys always hang below earlier keys on their path.



Final BST, path to 30 highlighted

Q.7 [NAT · 1 mark · Hashing — linear probing]

Answer: 22

Concept. Linear probing scans consecutive slots from the home slot until an empty one is found. Runs of occupied slots (clusters) grow and lengthen later probe sequences — primary clustering.

Trace (home slot → final slot, probes):

- 33 → 0 → 0 (1)
- 46 → 2 → 2 (1)
- 18 → 7 → 7 (1)
- 57 → 2 → 3 (2)
- 24 → 2 → 4 (3)
- 13 → 2 → 5 (4)
- 70 → 4 → 6 (3: slots 4, 5, 6)
- 35 → 2 → 8 (7: slots 2, 3, 4, 5, 6, 7, 8)

Total = 1 + 1 + 1 + 2 + 3 + 4 + 3 + 7 = **22**.

Trap: 70 hashes to 4, which is *not* its own collision chain, yet it collides because the cluster started at slot 2 has grown over slot 4.

0	33
1	
2	46
3	57
4	24
5	13
6	70
7	18
8	35
9	
10	

Final table

Q.8 [MCQ · 1 mark · Selection sort — intermediate state]**Answer: (A)**

Concept. After k passes of selection sort, $A[0..k-1]$ holds the k smallest elements in order; the remainder is permuted only by the swaps performed.

- Pass 0: min of whole array = 5 (index 5); swap with 29 → [5, 10, 14, 37, 13, 29, 41].
- Pass 1: min of $A[1..]$ = 10 at index 1 → self-swap, unchanged.
- Pass 2: min of $A[2..]$ = 13 (index 4); swap with 14 → [5, 10, 13, 37, 14, 29, 41].

Answer (A).

- (B) is the fully sorted array.
- (C) is the state after only two passes.
- (D) assumes the unsorted tail is also re-ordered, which selection sort never does.

Trap: 14 moves *to the right* (to index 4) in pass 2 — selection sort is not stable and can carry elements far from their final place.

Q.9 [NAT · 1 mark · Graphs — counting walks with the adjacency matrix]**Answer: 3**

Concept. $(M^k)[i][j]$ counts walks of length k from i to j , because each matrix product sums over the possible intermediate vertex.

Edges: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 1$, $3 \rightarrow 4$, $4 \rightarrow 1$, $4 \rightarrow 2$.

Enumerate $4 \rightarrow x \rightarrow y \rightarrow 1$ directly. Vertices with an edge into 1 are 3 and 4, so $y \in \{3, 4\}$:

- $4 \rightarrow 1 \rightarrow 3 \rightarrow 1$ ✓ ($4 \rightarrow 1$, $1 \rightarrow 3$, $3 \rightarrow 1$)
- $4 \rightarrow 2 \rightarrow 3 \rightarrow 1$ ✓
- $4 \rightarrow 2 \rightarrow 4 \rightarrow 1$ ✓
- $4 \rightarrow 1 \rightarrow 4$? no edge $1 \rightarrow 4$; $4 \rightarrow 4$? no self-loop.

Total **3** walks, which equals $(M^3)[4][1]$.

Trap: walks may revisit vertices ($4 \rightarrow 2 \rightarrow 4 \rightarrow 1$ is allowed). Counting only simple paths would give only 1 ($4 \rightarrow 2 \rightarrow 3 \rightarrow 1$).

Q.10 [MCQ · 1 mark · Merge sort — comparisons in a merge]**Answer: (D)**

Concept. Each comparison outputs exactly one element; comparisons stop as soon as one list empties. For lists of sizes m and n the count lies between $\min(m, n)$ and $m + n - 1$.

Trace (L front vs R front $\rightarrow$ output):

- 4 vs 6 $\rightarrow$ 4
- 11 vs 6 $\rightarrow$ 6
- 11 vs 9 $\rightarrow$ 9
- 11 vs 12 $\rightarrow$ 11
- 15 vs 12 $\rightarrow$ 12
- 15 vs 13 $\rightarrow$ 13 — R is now empty.
- Copy 15, 32, 38 with no comparisons.

Total = **6** $\rightarrow$ (D).

- (B) 7 counts one extra comparison after R empties.
- (C) $9 = m + n - 1$ is the worst case (lists interleave until the very end).

Tip: the number of comparisons = $(m + n) - (\text{number of elements copied after exhaustion}) = 9 - 3 = 6$.

Q.11 [NAT · 2 marks · Shortest paths — Dijkstra on an undirected graph]**Answer: 16**

Concept. With non-negative weights, the vertex extracted with the minimum tentative distance is final; relax every edge out of it.

Trace:

- Extract S (0): A = 3, B = 8.
- Extract A (3): B = $\min(8, 3 + 4) = 7$, C = $3 + 9 = 12$.
- Extract B (7): C = $\min(12, 7 + 2) = 9$, D = $7 + 6 = 13$.
- Extract C (9): D = $\min(13, 9 + 3) = 12$, E = $9 + 7 = 16$.
- Extract D (12): E = $\min(16, 12 + 1) = 13$, T = $12 + 8 = 20$.
- Extract E (13): T = $\min(20, 13 + 3) = 16$.
- Extract T (16).

Shortest path: S $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ T, cost $3 + 4 + 2 + 3 + 1 + 3 = \mathbf{16}$.

Trap: the 'obvious' routes S-B-D-T ($8 + 6 + 8 = 22$) or S-A-C-E-T ($3 + 9 + 7 + 3 = 22$) use fewer edges but cost more. The shortest path here uses every intermediate vertex — fewest edges $\neq$ least weight.

Tentative distances after each extraction

	S	A	B	C	D	E	T
S	0	3	8	∞	∞	∞	∞
A	0	3	7	12	∞	∞	∞
B	0	3	7	9	13	∞	∞
C	0	3	7	9	12	16	∞
D	0	3	7	9	12	13	20
E	0	3	7	9	12	13	16

Q.12 [MSQ · 2 marks · Shortest paths — Dijkstra trace and shortest-path tree]**Answer: (A), (B), (D)**

Concept. Track tentative distances and predecessors; the predecessor recorded at the last decrease of a vertex is its parent in the shortest-path tree.

Trace ($\infty \rightarrow$ value is a first assignment; $\text{old} \rightarrow \text{new}$ is an update):

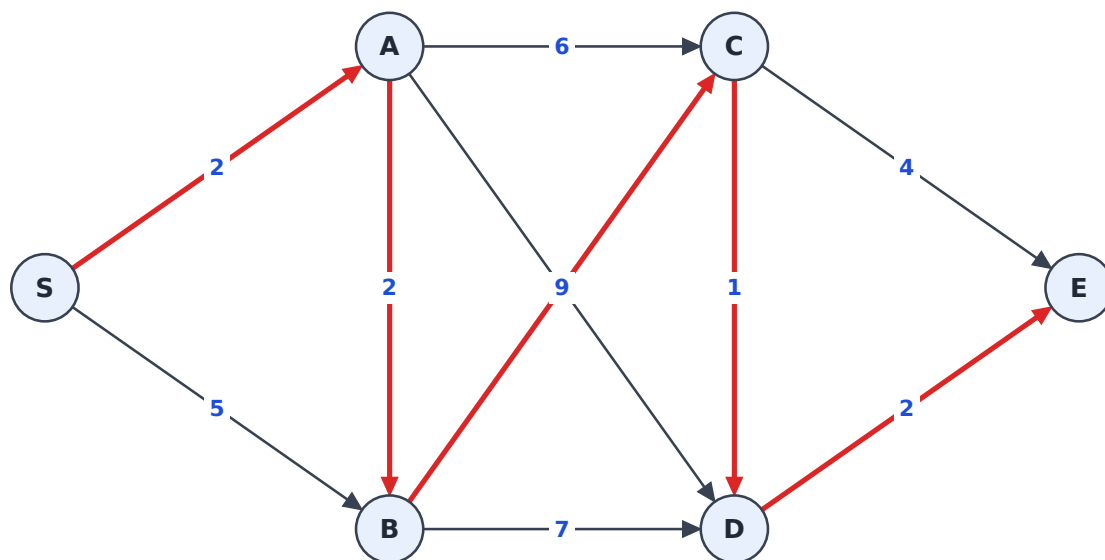
- Extract S (0): $A \infty \rightarrow 2$, $B \infty \rightarrow 5$.
- Extract A (2): $B 5 \rightarrow 4$ (**update 1**), $C \infty \rightarrow 8$, $D \infty \rightarrow 11$.
- Extract B (4): $C 8 \rightarrow 7$ (**update 2**); $D: 4 + 7 = 11$, not smaller $\rightarrow$ no change.
- Extract C (7): $D 11 \rightarrow 8$ (**update 3**), $E \infty \rightarrow 11$.
- Extract D (8): $E 11 \rightarrow 10$ (**update 4**).
- Extract E (10).

Final: A 2, B 4, C 7, D 8, E 10; parents $A \leftarrow S$, $B \leftarrow A$, $C \leftarrow B$, $D \leftarrow C$, $E \leftarrow D$.

Option analysis:

- (A) **True** — $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ has 5 edges, cost $2 + 2 + 3 + 1 + 2 = 10$.
- (B) **True** — exactly four strict decreases.
- (C) **False** — B offers 11 (not better than A's 11); D's final parent is C.
- (D) **True** — order S, A, B, C, D, E.

Trap: an equal-cost offer ($B \rightarrow D$ gives $11 = \text{current } 11$) does not trigger an update with the usual strict $<$ test.



Shortest-path tree (red)

Q.13 [NAT · 2 marks · Shortest paths — counting shortest paths]**Answer: 6**

Concept. Run Dijkstra while keeping $\text{cnt}[v]$ = number of shortest paths to v . When relaxing $u \rightarrow v$: if $d[u] + w < d[v]$, set $d[v] = d[u] + w$ and $\text{cnt}[v] = \text{cnt}[u]$; if $d[u] + w = d[v]$, add $\text{cnt}[u]$ to $\text{cnt}[v]$.

Processing in order of distance:

- S: d 0, cnt 1.
- A: d 2, cnt 1 (from S). B: d 3, cnt 1 (from S).
- C: via A $2 + 3 = 5$, via B $3 + 2 = 5 \rightarrow$ tie, $\text{cnt } 1 + 1 = 2$.
- D: via A $2 + 4 = 6$, via B $3 + 3 = 6 \rightarrow$ tie, $\text{cnt } 2$.
- E: via C $5 + 1 = 6$, $\text{cnt} = \text{cnt}[C] = 2$.
- T: via C $5 + 4 = 9$, via D $6 + 3 = 9$, via E $6 + 3 = 9 \rightarrow$ three-way tie, $\text{cnt} = 2 + 2 + 2 = 6$.

The six paths of weight 9: S-A-C-T, S-B-C-T, S-A-D-T, S-B-D-T, S-A-C-E-T, S-B-C-E-T.

Trap: shortest paths need not have the same number of edges — the two paths through E have 4 edges and are still shortest. Plain BFS-style counting by hops would miss them.

Q.14 [MCQ · 2 marks · Shortest paths — Dijkstra with a negative edge]

Answer: (C)

Concept. Dijkstra's correctness proof assumes that a path cannot get shorter by adding edges. A negative edge breaks this, and a vertex finalised too early is never revisited.

Trace:

- Extract S (0): $A = 4$, $B = 3$.
- Extract B (3) — B is now **final**. Relax $B \rightarrow C$: $C = 5$.
- Extract A (4): $A \rightarrow B$ gives $4 - 2 = 2 < 3$, but B is already finalised, so it is not changed (and $B \rightarrow C$ is never relaxed again).
- Extract C (5).

Reported: $B = 3$, $C = 5 \rightarrow (C)$. True values: $B = 2$ ($S \rightarrow A \rightarrow B$), $C = 4$ ($S \rightarrow A \rightarrow B \rightarrow C$).

- (A) mixes the two.
- (B) gives the true distances — what Bellman-Ford would produce.
- (D) is what an implementation would print if it lowered B's label without re-relaxing B's out-edges.

Trap: 'Dijkstra just needs no negative *cycles*' is wrong — even one negative edge without a cycle can produce wrong answers.

Q.15 [MCQ · 2 marks · Python — a buggy BFS (stack instead of queue)]

Answer: (C)

Concept. `list.pop()` removes the **last** element, so the frontier behaves as a stack (DFS-like order) while distances are still fixed at first discovery. Then a vertex can be discovered first via a longer route, and its label is never corrected.

Trace (frontier shown left $\rightarrow$ right, top on the right):

- pop a: $b = 1$, $c = 1 \rightarrow [b, c]$
- pop c: $d = 2 \rightarrow [b, d]$
- pop d: $e = 3$, $f = 3 \rightarrow [b, e, f]$
- pop f: no out-edges. pop e: f already seen.
- pop b: e already seen (with the wrong value 3).

Distances a..f: 0, 1, 1, 2, 3, 3 $\rightarrow (C)$.

- (A) is the correct BFS answer ($e = 2$ via $a \rightarrow b \rightarrow e$), which a deque with `popleft()` would print.
- (B) assumes f is reached via e ($3 + 1$), but f is discovered from d first.
- (D) mixes a correct e with a wrong f.

Trap: the code looks like BFS but the choice of `pop()` vs `popleft()` decides whether the labels are shortest-path distances.

Q.16 [MSQ · 2 marks · Stacks — permutations obtainable with a stack]

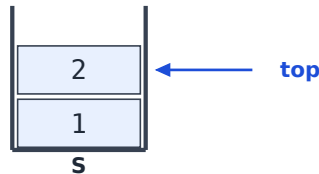
Answer: (A), (D)

Concept. Simulate greedily: to output x, push until x is on top (if x has already been pushed it must be on top right now), then pop. A sequence is impossible exactly when it contains a pattern $\dots c \dots a \dots b \dots$ with $a < b < c$ (the '312' pattern).

- (A) 3, 2, 5, 4, 1: push 1, 2, 3 pop 3; pop 2; push 4, 5 pop 5; pop 4; pop 1. **Possible.**

- (B) 2, 5, 3, 4, 1: push 1, 2 pop 2; push 3, 4, 5 pop 5; top is now 4 but 3 is required → **impossible** (pattern 5, 3, 4).
- (C) 4, 3, 5, 1, 2: after 4, 3, 5 the stack holds [1, 2] with 2 on top, but 1 is required → **impossible** (pattern 4/5, 1, 2).
- (D) 1, 4, 3, 5, 2: push 1 pop 1; push 2, 3, 4 pop 4; pop 3; push 5 pop 5; pop 2. **Possible.**

Tip: a quick test — after any element x is output, the not-yet-output elements smaller than x must appear in *decreasing* order.



Option (C): stack after outputting 4, 3, 5 — 1 is buried

Q.17 [NAT · 2 marks · Python — memoised path counting on a grid]

Answer: 6

Concept. $\text{ways}(r, c)$ = number of paths from $(0, 0)$ to (r, c) moving only in $+r$ or $+c$ steps, avoiding blocked cells. Each cell's value is the sum of the cell above and the cell to the left; blocked cells contribute 0. Memoisation makes it $O(\text{rows} \times \text{cols})$.

Fill the table row by row ($\times$ = blocked):

- $r = 0$: 1, 1, 1, 1, 1 (only one way along the top row)
- $r = 1$: 1, $\times$, $0 + 1 = 1$, $1 + 1 = 2$, $2 + 1 = 3$
- $r = 2$: 1, $0 + 1 = 1$, $1 + 1 = 2$, $\times$, $3 + 0 = 3$
- $r = 3$: $\times$, $0 + 1 = 1$, $2 + 1 = 3$, $0 + 3 = 3$, $3 + 3 = 6$

$\text{ways}(3, 4) = 6$.

Trap: forgetting that a blocked cell must return 0 *before* the base-case check, or treating the blocked $(3, 0)$ as reachable — it kills every path that runs down column 0 to the bottom row. Without blocks the answer would be $C(7, 3) = 35$.

	ways(r, c)				
	c=0	c=1	c=2	c=3	c=4
r=0	1	1	1	1	1
r=1	1	$\times$	1	2	3
r=2	1	1	2	$\times$	3
r=3	$\times$	1	3	3	6

Q.18 [MSQ · 2 marks · Elementary sorting — bubble and insertion sort]

Answer: (B), (C), (D)

Concept. One bubble pass moves the maximum to the end; insertion sort keeps a sorted prefix. Every adjacent swap (in either algorithm) removes exactly one inversion.

Bubble pass 1 on [6, 1, 8, 3, 7, 2]:

- (6, 1) swap → [1, 6, 8, 3, 7, 2]
- (6, 8) ok
- (8, 3) swap → [1, 6, 3, 8, 7, 2]
- (8, 7) swap → [1, 6, 3, 7, 8, 2]
- (8, 2) swap → [1, 6, 3, 7, 2, 8]

Insertion sort: insert 1 → [1, 6, 8, ...]; insert 8 → unchanged; insert 3 → [1, 3, 6, 8, 7, 2].

Inversions: $6 > \{1, 3, 2\}$ (3), $8 > \{3, 7, 2\}$ (3), $3 > 2$ (1), $7 > 2$ (1) $\rightarrow 8$.

- (A) **False** — 4 swaps (the pair 6, 8 is already in order).
- (B) **True** — so bubble sort makes 8 swaps in total and insertion sort 8 shifts.
- (C) **True**.
- (D) **True**.

Trap: counting comparisons (5 in a pass) instead of swaps for (A).

Q.19 [MSQ · 2 marks · Heaps — bottom-up build-heap]

Answer: (A), (B), (D)

Concept. Build-heap sifts down every internal node from the last one up to the root; total work is $O(n)$ because most nodes are near the bottom.

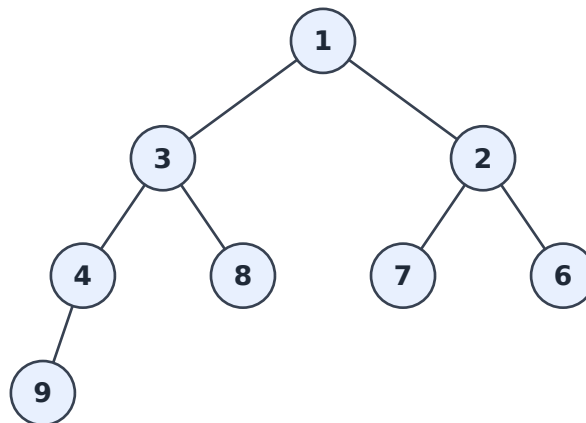
$n = 8$, so sift-downs at $i = 3, 2, 1, 0$:

- $i = 3$ (key 1): child $3 \rightarrow 1 < 3$, no swap. [9, 4, 7, 1, 8, 2, 6, 3]
- $i = 2$ (key 7): children 2, 6 $\rightarrow$ swap with 2. [9, 4, 2, 1, 8, 7, 6, 3] (1 swap)
- $i = 1$ (key 4): children 1, 8 $\rightarrow$ swap with 1 $\rightarrow$ index 3; child $3 < 4 \rightarrow$ swap. [9, 1, 2, 3, 8, 7, 6, 4] (2 swaps)
- $i = 0$ (key 9): children 1, 2 $\rightarrow$ swap with 1 $\rightarrow$ index 1; children 3, 8 $\rightarrow$ swap with 3 $\rightarrow$ index 3; child 4 $\rightarrow$ swap $\rightarrow$ index 7. [1, 3, 2, 4, 8, 7, 6, 9] (3 swaps)

Total swaps = $0 + 1 + 2 + 3 = 6$.

- (A) **True** — 9 sinks to index 7, a leaf.
- (B) **True**.
- (C) **False** — the sift-down from index 1 swaps twice ($4 \leftrightarrow 1$, then $4 \leftrightarrow 3$).
- (D) **True**.

Trap: stopping a sift-down after one level; it continues until the key is no larger than both children.



Resulting min-heap

Q.20 [MCQ · 2 marks · Linked lists — repeated move-last-to-front]

Answer: (B)

Concept. Each iteration walks to the last node, detaches it and makes it the new head — a right rotation by one position. Seven rotations of a 5-node list equal $7 \bmod 5 = 2$ rotations.

Building: inserting 50, 40, 30, 20, 10 at the head gives $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$.

Rotations:

- 1: 50, 10, 20, 30, 40
- 2: 40, 50, 10, 20, 30
- 3: 30, 40, 50, 10, 20
- 4: 20, 30, 40, 50, 10

- 5: 10, 20, 30, 40, 50 (back to the start)
- 6: 50, 10, 20, 30, 40
- 7: 40, 50, 10, 20, 30

Step count: reaching the last of 5 nodes takes 4 advances each time, so steps = $7 \times 4 = 28$. Output [40, 50, 10, 20, 30] 28 → (B).

- (A) is a rotation by 3 (or a left rotation by 2).
- (C) counts 5 advances per rotation (one per node instead of per link).
- (D) is a left rotation by 1.

Trap: the list length never changes, so every rotation costs exactly $n - 1 = 4$ pointer advances: rotating k times this way costs $\Theta(k \cdot n)$, whereas finding $k \bmod n$ first and re-linking once costs $\Theta(n)$.



Mock Test 16

Recursion & Memoization in Python

Difficulty: GATE-level

Focus areas: recursion tracing, call counts, memoized recursion, recursion trees

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Recursion — tracing output order]

Consider the following Python program. What is printed?

Python

```

1 def f(n):
2     if n <= 0:
3         return
4     print(n, end=' ')
5     f(n - 2)
6     print(n % 3, end=' ')
7     f(n - 3)
8
9 f(5)

```

- (A) 5 3 1 2 0 1 2 2
- (B) 5 3 1 1 0 2 2 2
- (C) 5 3 1 1 0 2 2
- (D) 5 2 3 1 1 0 2 2

Q.2 [NAT · numerical answer | 1 mark | Recursion — counting calls]

Consider the function below. The total number of calls to `g` (including the initial call) made when `g(9)` is evaluated is _____.

Python

```

1 def g(n):
2     if n < 3:
3         return 1
4     return g(n - 1) + g(n - 3)

```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with an operand stack (^ is exponentiation, / is exact division). Let v be its value and d the maximum number of operands on the stack at any time. The pair (v, d) is

5 9 3 2 ^ / 4 2 - * + 3 -

- (A) (4, 4)
- (B) (4, 5)
- (C) (−4, 4)
- (D) (0, 4)

Q.4 [MSQ · one or more correct | 1 mark | Memoization — mutable default dictionary]

Consider the function below, which uses a mutable default argument as a memo table. Assume these calls are made in a fresh interpreter, in the order the statements describe. Which of the following statements is/are TRUE?

Python

```
1 def fib(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n if n < 2 else fib(n - 1) + fib(n - 2)
5     return memo[n]
```

- (A) A later call `fib(12)` makes 12 calls to `fib` in total
- (B) A second call `fib(10)` afterwards makes exactly one call to `fib`
- (C) After the first call `fib(10)`, the dictionary `fib.__defaults__[0]` has 11 keys
- (D) The first call `fib(10)` makes 19 calls to `fib` in total (including itself)

Q.5 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

Keys 22, 33, 44, 13, 24, 35 are inserted in this order into an initially empty table of size 11 using quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) examines slot $(k \bmod 11 + i^2) \bmod 11$. Counting every slot examined (including the one where the key is finally placed), the total number of probes for all six insertions is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Recursive binary search — hidden cost of slicing]

Consider the recursive binary search below on a sorted Python list of n elements. Its worst-case running time is

Python

```
1 def bs(A, x):
2     if not A:
3         return False
4     m = len(A) // 2
5     if A[m] == x:
6         return True
7     if A[m] < x:
8         return bs(A[m + 1:], x)
9     return bs(A[:m], x)
```

- (A) $\Theta(\log n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n)$
- (D) $\Theta(\log^2 n)$

Q.7 [NAT · numerical answer | 1 mark | Recursion on binary trees]

The binary tree T shown below is stored as nested lists [value, left, right] (with None for an empty subtree). The value returned by `g(T)` is _____.

Python

```
1 def g(t):
2     if t is None:
3         return 0
4     a, b = g(t[1]), g(t[2])
5     return max(a, b) + (1 if t[0] % 2 else 0)
```

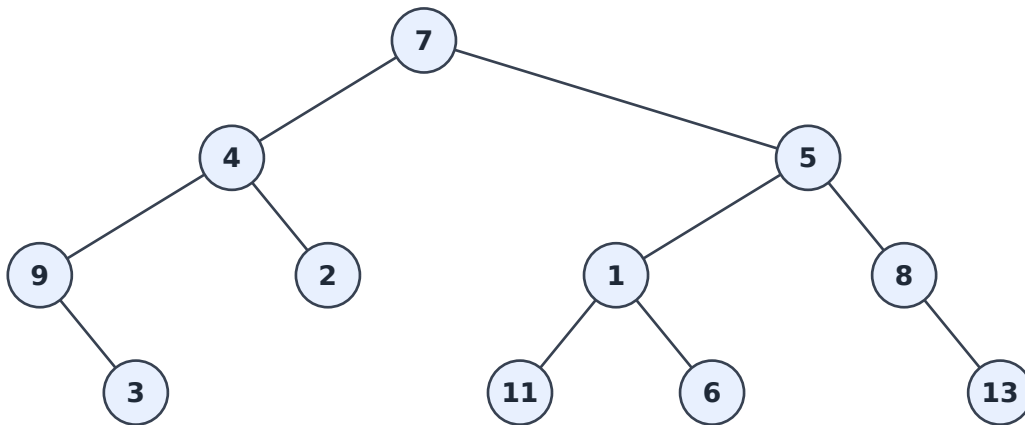


Figure: tree T

Answer: _____ (enter a numerical value)

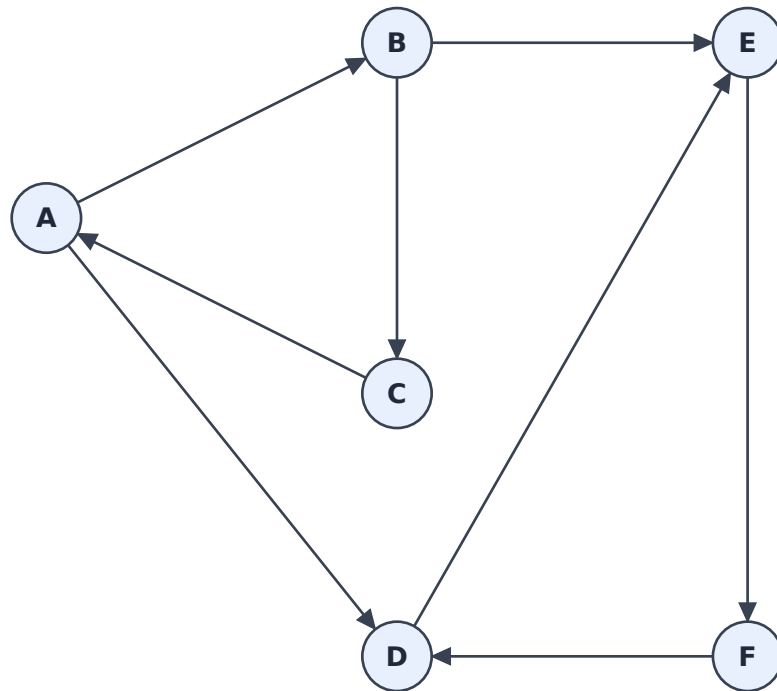
Q.8 [MCQ · single correct | 1 mark | Bubble sort with early termination]

Bubble sort with an early-exit flag (stop after a pass that makes no swap; pass i compares adjacent pairs in positions $0 \dots n-1-i$) sorts $[3, 1, 2, 9, 5, 4, 8, 7]$ in ascending order. The total number of element comparisons made is

- (A) 13
- (B) 28
- (C) 18
- (D) 8

Q.9 [MSQ · one or more correct | 1 mark | Recursive DFS — discovery/finish times]

Recursive DFS is run on the directed graph below starting from **A**, visiting out-neighbours in alphabetical order. A single clock starts at 1 and is incremented at every discovery and every finish. Which of the following statements is/are TRUE?



- (A) E finishes before F
- (B) $A \rightarrow D$ is a back edge
- (C) The finish time of D is 8
- (D) The discovery order is A, B, C, E, F, D

Q.10 [MCQ · single correct | 1 mark | Deques — rotate semantics]

Consider the following Python program. What is printed?

Python

```

1 from collections import deque
2 d = deque([1, 2, 3, 4, 5])
3 d.rotate(2)
4 d.appendleft(d.pop())
5 d.rotate(-3)
6 x = d.popleft() * 10 + d[-1]
7 print(x, len(d))

```

- (A) 15 4
- (B) 35 4
- (C) 45 5
- (D) 12 4

Q.11 [NAT · numerical answer | 2 marks | Memoization — lru_cache call counting]

Consider the following Python program. The **second** number printed (the value of **calls**) is _____.

Python

```
1 from functools import lru_cache
2
3 calls = 0
4
5 @lru_cache(maxsize=None)
6 def p(n, k):
7     global calls
8     calls += 1
9     if k == 0 or k == n:
10         return 1
11     return p(n - 1, k - 1) + p(n - 1, k)
12
13 print(p(8, 3), calls)
```

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Recursion — Tower of Hanoi move sequence]

Consider the following Python program. What is printed?

Python

```
1 moves = []
2
3 def hanoi(n, src, dst, via):
4     if n == 0:
5         return
6     hanoi(n - 1, src, via, dst)
7     moves.append((n, src, dst))
8     hanoi(n - 1, via, dst, src)
9
10 hanoi(4, 'A', 'C', 'B')
11 print(moves[9])
```

- (A) (3, 'B', 'C')
- (B) (1, 'B', 'C')
- (C) (2, 'A', 'B')
- (D) (2, 'B', 'A')

Q.13 [NAT · numerical answer | 2 marks | Mutual recursion]

Consider the following Python program. The value printed is _____.

Python

```
1 def a(n):
2     return 1 if n <= 0 else n + b(n - 1)
3
4 def b(n):
5     return 0 if n <= 0 else a(n // 2) * 2
6
7 print(a(10))
```

Answer: _____ (enter a numerical value)

Q.14 [MSQ · one or more correct | 2 marks | Merge sort — recursion structure]

Top-down merge sort is applied to an array of **13** elements. Each call on a sub-array of length $L > 1$ splits it into the first $\lfloor L/2 \rfloor$ and the remaining $\lfloor L/2 \rfloor$ elements, recursively sorts both halves and merges them; a call on length 1 returns immediately. Which of the following statements is/are TRUE?

- (A) At most 5 merge-sort calls are simultaneously active on the call stack
- (B) The merge-sort function is called 25 times in total (including the first call)
- (C) The final (top-level) merge can perform up to 13 key comparisons
- (D) Exactly 13 calls are made on sub-arrays of length 1

Q.15 [MCQ · single correct | 2 marks | Python — recursion with nonlocal state]

Consider the following Python program. What is printed?

Python

```
1 def make():
2     count = 0
3     def walk(n):
4         nonlocal count
5         if n <= 1:
6             count += 1
7             return n
8         return walk(n // 2) + walk(n - n // 2)
9     return walk, lambda: count
10
11 w, get = make()
12 print(w(7), get(), w(4), get())
```

- (A) 7 7 4 7
- (B) 7 7 4 4
- (C) 7 13 4 20
- (D) 7 7 4 11

Q.16 [MSQ · one or more correct | 2 marks | Recursive BST validation]

Two recursive functions are written to check whether a binary tree (nested lists `[key, left, right]`) is a BST. They are applied to the tree T shown below. Which of the following statements is/are TRUE?

Python

```

1 def local_ok(t):
2     if t is None:
3         return True
4     k, l, r = t
5     if l and l[0] > k:
6         return False
7     if r and r[0] < k:
8         return False
9     return local_ok(l) and local_ok(r)
10
11 INF = float('inf')
12 def range_ok(t, lo=-INF, hi=INF):
13     if t is None:
14         return True
15     k, l, r = t
16     if not lo < k < hi:
17         return False
18     return range_ok(l, lo, k) and range_ok(r, k, hi)

```

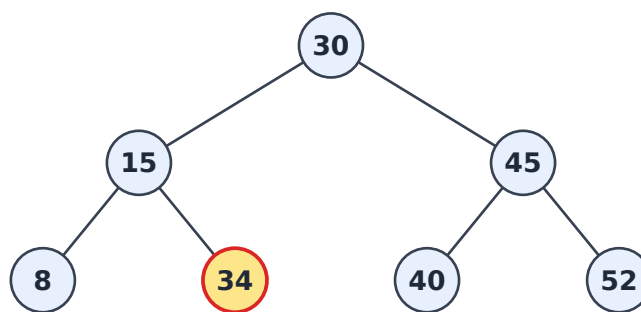


Figure: tree T

- (A) The in-order traversal of T is 8, 15, 34, 30, 40, 45, 52
- (B) `local_ok(T)` returns True
- (C) `range_ok(T)` returns False
- (D) If the key 34 is changed to 31, `range_ok(T)` returns True

Q.17 [MSQ · one or more correct | 2 marks | Recurrences from recursive code]

Consider the four recursive Python functions below (the running time is measured as a function of n , the argument). Which of the following statements is/are TRUE?

Python

```

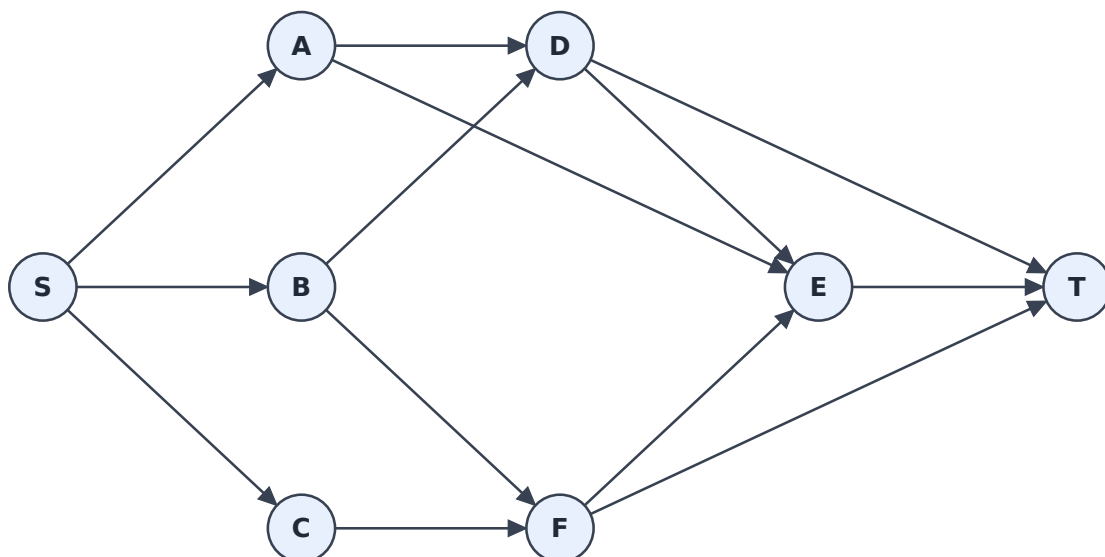
1 def a(n):
2     if n <= 1: return 1
3     s = sum(range(int(n ** 0.5)))
4     return a(n // 2) + a(n // 2) + s
5
6 def b(n):
7     if n == 0: return 0
8     return b(n - 1) + len(list(range(n)))
9
10 def c(n):
11     if n <= 1: return 1
12     s = 0
13     for i in range(n):
14         for j in range(n):
15             s += 1
16     return s + c(n // 2) + c(n // 2) + c(n // 2) + c(n // 2)
17
18 def d(n):
19     return 1 if n <= 0 else d(n - 1) + d(n - 1)

```

- (A) The running time of $a(n)$ is $\Theta(n)$
- (B) The running time of $c(n)$ is $\Theta(n^2)$
- (C) The running time of $d(n)$ is $\Theta(2^n)$
- (D) The running time of $b(n)$ is $\Theta(n^2)$

Q.18 [NAT · numerical answer | 2 marks | Memoized recursion on a DAG — path counting]

The number of distinct directed paths from **S** to **T** in the DAG shown below (as computed by the memoized recursion $\text{paths}(u) = 1$ if $u == T$ else $\sum \text{paths}(v)$ over the out-neighbours v of u) is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Recursive linked-list surgery and aliasing]

Consider the following Python program. What is printed?

Python

```
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5     def sw(h):
6         if h is None or h.nxt is None:
7             return h
8         nxt = h.nxt
9         h.nxt = sw(nxt.nxt)
10        nxt.nxt = h
11        return nxt
12
13    def show(h):
14        out = []
15        while h:
16            out.append(h.v)
17            h = h.nxt
18        return out
19
20    head = None
21    for v in range(7, 0, -1):
22        head = Node(v, head)
23    a = head
24    head = sw(head)
25    print(show(head), show(a))
```

- (A) [2, 1, 4, 3, 6, 5, 7] [1, 2, 3, 4, 5, 6, 7]
- (B) [2, 1, 4, 3, 6, 5, 7] [1, 4, 3, 6, 5, 7]
- (C) [2, 1, 4, 3, 6, 5] [1, 4, 3, 6, 5]
- (D) [2, 1, 4, 3, 6, 5, 7] [1]

Q.20 [NAT · numerical answer | 2 marks | Memoized recursion — grid paths with obstacles]

A robot starts at cell (0, 0) (top-left) of the 6 × 6 grid shown below and must reach cell (5, 5) (bottom-right), moving only **right** or **down** one cell at a time and never entering a blocked cell (marked X). The program below counts such paths. The value printed is _____.

Python

```

1 from functools import lru_cache
2 N = 6
3 blocked = {(1, 3), (2, 1), (3, 3), (4, 0)}
4
5 @lru_cache(maxsize=None)
6 def P(r, c):
7     if r >= N or c >= N or (r, c) in blocked:
8         return 0
9     if (r, c) == (N - 1, N - 1):
10        return 1
11    return P(r + 1, c) + P(r, c + 1)
12
13 print(P(0, 0))

```

Grid (X = blocked)

	0	1	2	3	4	5
0	S					
1				X		
2		X				
3				X		
4	X					
5						T

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 16

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Recursion — tracing output order
2	NAT	1	37	Recursion — counting calls
3	MCQ	1	(A)	Stacks — postfix evaluation
4	MSQ	1	(B), (C), (D)	Memoization — mutable default dictionary
5	NAT	1	12	Hashing — quadratic probing
6	MCQ	1	(C)	Recursive binary search — hidden cost of slicing
7	NAT	1	4	Recursion on binary trees
8	MCQ	1	(C)	Bubble sort with early termination
9	MSQ	1	(C), (D)	Recursive DFS — discovery/finish times
10	MCQ	1	(A)	Deque — rotate semantics
11	NAT	2	23	Memoization — lru_cache call counting
12	MCQ	2	(D)	Recursion — Tower of Hanoi move sequence
13	NAT	2	22	Mutual recursion
14	MSQ	2	(A), (B), (D)	Merge sort — recursion structure
15	MCQ	2	(D)	Python — recursion with nonlocal state
16	MSQ	2	(A), (B), (C)	Recursive BST validation

Q	Type	Marks	Key	Topic
17	MSQ	2	(A), (C), (D)	Recurrences from recursive code
18	NAT	2	9	Memoized recursion on a DAG — path counting
19	MCQ	2	(B)	Recursive linked-list surgery and aliasing
20	NAT	2	39	Memoized recursion — grid paths with obstacles

Detailed Solutions — Mock Test 16

Q.1 [MCQ · 1 mark · Recursion — tracing output order]

Answer: (B)

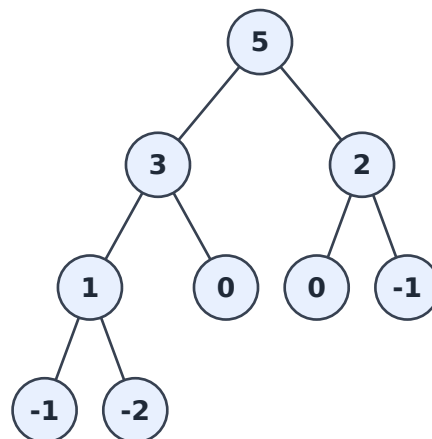
Each call prints n , recurses on $n - 2$, prints $n \bmod 3$, then recurses on $n - 3$. Calls with $n \leq 0$ print nothing. Expand depth-first:

- $f(5)$: print **5**, call $f(3)$
- $f(3)$: print **3**, call $f(1)$
- $f(1)$: print **1**; $f(-1)$ nothing; print $1 \bmod 3 = \mathbf{1}$; $f(-2)$ nothing
- back in $f(3)$: print $3 \bmod 3 = \mathbf{0}$; $f(0)$ nothing
- back in $f(5)$: print $5 \bmod 3 = \mathbf{2}$; call $f(2)$
- $f(2)$: print **2**; $f(0)$ nothing; print $2 \bmod 3 = \mathbf{2}$; $f(-1)$ nothing

Output: **5 3 1 1 0 2 2 2**.

- (A) swaps the two values printed by $f(1)$ and $f(5)$'s middle print — it prints the remainders in the wrong frames.
- (C) forgets that $f(2)$ prints twice (once before and once after its first recursive call).
- (D) runs $f(n - 3)$ before $f(n - 2)$.

Tip: for a function with code between two recursive calls, write the recursion tree and read it like an in-order walk: pre-print, left subtree, mid-print, right subtree.



Recursion tree of $f(5)$ (node = argument)

Q.2 [NAT · 1 mark · Recursion — counting calls]

Answer: 37

Let $C(n)$ be the number of calls made by $g(n)$, counting itself. Base cases $n < 3$ make exactly one call; otherwise $C(n) = 1 + C(n - 1) + C(n - 3)$.

- $C(0) = C(1) = C(2) = 1$
- $C(3) = 1 + C(2) + C(0) = 3$
- $C(4) = 1 + C(3) + C(1) = 5$
- $C(5) = 1 + C(4) + C(2) = 7$
- $C(6) = 1 + C(5) + C(3) = 11$
- $C(7) = 1 + C(6) + C(4) = 17$
- $C(8) = 1 + C(7) + C(5) = 25$
- $C(9) = 1 + C(8) + C(6) = \mathbf{37}$

(The returned value $g(9)$ is 19; it satisfies the same recurrence without the '+1'.)

Trap: do not confuse the *value* returned with the *number of calls*; and remember that base-case calls ($n = 0, 1, 2$) are calls too. Note $C(n) = 2 \cdot g(n) - 1$ because every internal node of the recursion tree has exactly two children.

Q.3 [MCQ · 1 mark · Stacks — postfix evaluation]

Answer: (A)

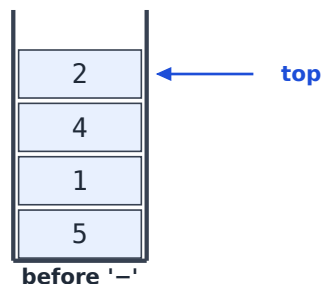
Scan left to right: push operands; on an operator pop the **right** operand first, then the left, and push left $\circ$ right.

- $5, 9, 3, 2 \rightarrow [5, 9, 3, 2]$ (size 4)
- $\wedge \rightarrow 3^2 = 9 \rightarrow [5, 9, 9]$
- $/ \rightarrow 9 / 9 = 1 \rightarrow [5, 1]$
- $4, 2 \rightarrow [5, 1, 4, 2]$ (size 4)
- $- \rightarrow 4 - 2 = 2 \rightarrow [5, 1, 2]$
- $* \rightarrow 1 \times 2 = 2 \rightarrow [5, 2]$
- $+ \rightarrow 7 \rightarrow [7]; 3 \rightarrow [7, 3]; - \rightarrow 7 - 3 = 4$

Maximum stack size = **4**. So $(v, d) = (4, 4)$.

- (B) overcounts the depth (the stack never holds 5 operands).
- (C) computes the final subtraction as $3 - 7$.
- (D) reverses every subtraction ($2 - 4 = -2$, then $5 - 2 = 3$, then $3 - 3 = 0$).

Trap: operand order matters for $-$, $/$ and $\wedge$: the element popped *first* is the right operand.



Stack at maximum depth (second time)

Q.4 [MSQ · 1 mark · Memoization — mutable default dictionary]

Answer: (B), (C), (D)

The default `{}` is created once, so the memo **persists across top-level calls**.

First fib(10): $\text{fib}(n)$ calls $\text{fib}(n-1)$ first (a new value), and when it later calls $\text{fib}(n-2)$ that value is already memoised (a single, immediately-returning call). So each n from 10 down to 2 makes two calls, $\text{fib}(1)$ and $\text{fib}(0)$ make none: total = $1 + 2 \times 9 = 19$.

- (A) $\text{fib}(12) \rightarrow \text{fib}(11)$ (new) $\rightarrow \text{fib}(10)$ hit, $\text{fib}(9)$ hit; $\text{fib}(12)$ also calls $\text{fib}(10)$ hit. Calls: $\text{fib}(12)$, $\text{fib}(11)$, $\text{fib}(10)$, $\text{fib}(9)$, $\text{fib}(10) = 5$, not 12. **False**.
- (B) 10 is in the memo, so the call returns at once — 1 call. **True**.
- (C) Keys 0, 1, ..., 10 are stored $\rightarrow 11$ keys. **True**.
- (D) **True** (19 calls).

Trap: a mutable default is shared state. Here that is exploited deliberately; in ordinary code it is a classic bug.

Q.5 [NAT · 1 mark · Hashing — quadratic probing]

Answer: 12

With quadratic probing the probe offsets are 0, 1, 4, 9, ... from the home slot.

- 22: home 0, empty $\rightarrow$ slot 0 (1 probe)

- 33: home 0 full; $0+1 = 1$ empty $\rightarrow$ slot 1 (2 probes)
- 44: home 0 full; 1 full; $0+4 = 4$ empty $\rightarrow$ slot 4 (3 probes)
- 13: home 2, empty $\rightarrow$ slot 2 (1 probe)
- 24: home 2 full; 3 empty $\rightarrow$ slot 3 (2 probes)
- 35: home 2 full; 3 full; $2+4 = 6$ empty $\rightarrow$ slot 6 (3 probes)

Total = $1 + 2 + 3 + 1 + 2 + 3 = \mathbf{12}$.

Trap: with *linear* probing 44 would go to slot 2 and push 13 and 24 further, giving a different count. Quadratic probing jumps to $0 + 4 = 4$, skipping slots 2 and 3.

0	22
1	33
2	13
3	24
4	44
5	
6	35
7	
8	
9	
10	

Final table

Q.6 [MCQ · 1 mark · Recursive binary search — hidden cost of slicing]

Answer: (C)

Each call does $O(1)$ comparisons, but the **slice** `A[m+1:]` or `A[:m]` builds a new list of about $n/2$ elements, which costs $\Theta(n/2)$ time.

Recurrence: $T(n) = T(n/2) + \Theta(n)$.

Unrolling: $n/2 + n/4 + n/8 + \dots \leq n$, so $T(n) = \mathbf{\Theta(n)}$ (master theorem case 3: $a = 1$, $b = 2$, $f(n) = n$ dominates $n^0 = 1$).

- (A) $\Theta(\log n)$ is the number of *calls*, true only if each call were $O(1)$ (e.g. passing lo/hi indices instead of slicing).
- (B) $\Theta(n \log n)$ would need $\Theta(n)$ work at each of $\log n$ levels — but the slice sizes halve.
- (D) has no basis here.

Trap: Python slicing copies. A 'logarithmic' algorithm written with slices is linear.

Q.7 [NAT · 1 mark · Recursion on binary trees]

Answer: 4

$g(t)$ returns the **maximum number of odd keys on any root-to-leaf path** of t : each node adds 1 if its key is odd to the best of its two subtrees.

Bottom-up values:

- Leaves: $g(3) = 1$, $g(2) = 0$, $g(11) = 1$, $g(6) = 0$, $g(13) = 1$.
- $g(9) = \max(0, 1) + 1 = 2$; $g(4) = \max(2, 0) + 0 = 2$.
- $g(1) = \max(1, 0) + 1 = 2$; $g(8) = \max(0, 1) + 0 = 1$; $g(5) = \max(2, 1) + 1 = 3$.
- $g(7) = \max(2, 3) + 1 = \mathbf{4}$.

The best path is $7 \rightarrow 5 \rightarrow 1 \rightarrow 11$, all four keys odd.

Trap: the path $7 \rightarrow 4 \rightarrow 9 \rightarrow 3$ is the longest-looking left path but contains the even key 4, so it scores only 3.

Q.8 [MCQ · 1 mark · Bubble sort with early termination]

Answer: (C)

Each pass bubbles the largest remaining element to the end; the algorithm stops after the first pass that performs no swaps (that pass still makes its comparisons).

- Pass 1 (7 comparisons): [1, 2, 3, 5, 4, 8, 7, 9] — swaps $3 \leftrightarrow 1$, $3 \leftrightarrow 2$, $9 \leftrightarrow 5$, $9 \leftrightarrow 4$, $9 \leftrightarrow 8$, $9 \leftrightarrow 7$.
- Pass 2 (6 comparisons): [1, 2, 3, 4, 5, 7, 8, 9] — swaps $5 \leftrightarrow 4$, $8 \leftrightarrow 7$.
- Pass 3 (5 comparisons): no swaps $\rightarrow$ stop.

Comparisons = $7 + 6 + 5 = 18$. (Total swaps = 8 = number of inversions.)

- (A) 13 forgets the final swap-free pass, which is needed to *detect* sortedness.
- (B) $28 = n(n-1)/2$ ignores the early exit.
- (D) 8 is the number of swaps, not comparisons.

Tip: the number of passes is (max number of positions any element must move **left**) + 1.

Array after each pass								
	0	1	2	3	4	5	6	7
start	3	1	2	9	5	4	8	7
pass 1	1	2	3	5	4	8	7	9
pass 2	1	2	3	4	5	7	8	9
pass 3	1	2	3	4	5	7	8	9

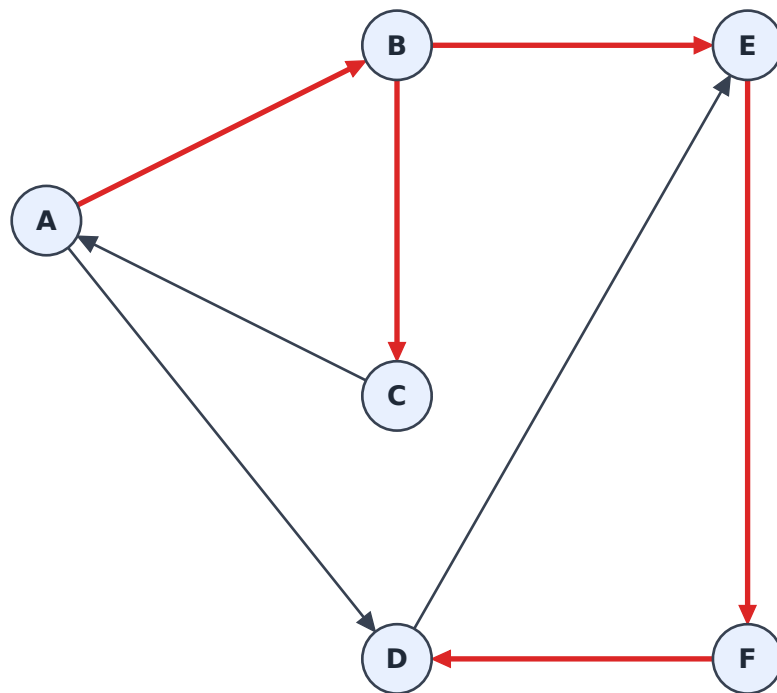
Q.9 [MSQ · 1 mark · Recursive DFS — discovery/finish times]

Answer: (C), (D)

Trace (d/f = discovery/finish):

- A d1 $\rightarrow$ B d2 $\rightarrow$ C d3; C $\rightarrow$ A is to a vertex still on the stack (back edge); C f4.
- B $\rightarrow$ E d5 $\rightarrow$ F d6 $\rightarrow$ D d7; D $\rightarrow$ E: E is on the stack (back edge); D f8; F f9; E f10; B f11.
- A $\rightarrow$ D: D is already finished and was discovered after A $\rightarrow$ **forward** edge. A f12.
- (A) F (9) finishes before E (10) since F is E's descendant. **False.**
- (B) A $\rightarrow$ D goes to a *descendant* that is already finished: a forward edge, not a back edge. **False.**
- (C) D finishes at 8. **True.**
- (D) Discovery order A, B, C, E, F, D. **True.**

Tip: for an edge $u \rightarrow v$ to an already-discovered v : v still active $\Rightarrow$ back; v finished with $d[u] < d[v] \Rightarrow$ forward; otherwise cross.



DFS tree edges highlighted

Discovery / finish times

	A	B	C	D	E	F
d	1	2	3	7	5	6
f	12	11	4	8	10	9

Q.10 [MCQ · 1 mark · Deques — rotate semantics]**Answer: (A)**

`rotate(k)` with $k > 0$ moves k elements from the right end to the left end; $k < 0$ rotates the other way. `pop()` removes from the right, `appendleft` adds on the left.

- start: [1, 2, 3, 4, 5]
- rotate(2): [4, 5, 1, 2, 3]
- pop() → 3, appendleft(3): [3, 4, 5, 1, 2] (this is one more right-rotation)
- rotate(-3): move 3 elements from left to right → [1, 2, 3, 4, 5]
- popleft() → 1; deque [2, 3, 4, 5]; $d[-1] = 5 \rightarrow x = 10 + 5 = \mathbf{15}$; len = 4.

Net effect: $+2 + 1 - 3 = 0$ rotations, so the deque is back to its original order.

- (B) 35 uses the deque after rotate(2) only.
- (C) 45 forgets the `pop` and treats rotate(-3) as rotating right; also the length is 4.
- (D) 12 reads $d[-1]$ as the second element.

Tip: track the net rotation count rather than individual moves.

Q.11 [NAT · 2 marks · Memoization — lru_cache call counting]**Answer: 23**

With `lru_cache`, the function **body** runs exactly once per distinct argument pair; repeated calls are answered by the cache without executing `calls += 1`. So `calls` = number of distinct states (n, k) reachable from $(8, 3)$.

From (n, k) we reach $(n-1, k-1)$ and $(n-1, k)$, stopping when $k = 0$ or $k = n$. Distinct states per level:

- $n = 8$: $k = 3$ (1)
- $n = 7$: $k = 2, 3$ (2)
- $n = 6$: $k = 1, 2, 3$ (3)
- $n = 5$: $k = 0, 1, 2, 3$ (4)
- $n = 4$: $k = 0$ (from $(5,1)$), $1, 2, 3$ (4)
- $n = 3$: $k = 0, 1, 2, 3$ (4) — $(4,0)$ is a base case, so $(3,0)$ comes from $(4,1)$
- $n = 2$: $k = 0, 1, 2$ (3) — $(3,3)$ and $(3,0)$ are base cases
- $n = 1$: $k = 0, 1$ (2) — from $(2,1)$

Total = $1 + 2 + 3 + 4 + 4 + 4 + 3 + 2 = \mathbf{23}$. (The first number printed is $C(8,3) = 56$.)

Trap: without memoization the number of calls would be $2 \cdot C(8,3) - 1 = 111$; with memoization it is the number of distinct subproblems, not the size of the recursion tree.

States (n, k) whose body executes (✓)

	k=0	k=1	k=2	k=3
n=8				✓
n=7			✓	✓
n=6		✓	✓	✓
n=5	✓	✓	✓	✓
n=4	✓	✓	✓	✓
n=3	✓	✓	✓	✓
n=2	✓	✓	✓	
n=1	✓	✓		

Q.12 [MCQ · 2 marks · Recursion — Tower of Hanoi move sequence]

Answer: (D)

$\text{hanoi}(n)$ makes $2^n - 1$ moves: the $2^{n-1} - 1$ moves of the first sub-call, then the move of disk n (move number 2^{n-1}), then the second sub-call. `moves[9]` is the **10th** move.

- $\text{hanoi}(4, A, C, B)$: moves 1-7 = $\text{hanoi}(3, A, B, C)$; move 8 = disk 4 $A \rightarrow C$; moves 9-15 = $\text{hanoi}(3, B, C, A)$.
- Move 10 is the 2nd move of $\text{hanoi}(3, B, C, A)$, whose moves are: $\text{hanoi}(2, B, A, C)$ (moves 9-11), disk 3 $B \rightarrow C$ (move 12), $\text{hanoi}(2, A, C, B)$ (13-15).
- $\text{hanoi}(2, B, A, C)$: disk 1 $B \rightarrow C$ (move 9), **disk 2 $B \rightarrow A$ (move 10)**, disk 1 $C \rightarrow A$ (move 11).

So `moves[9] = (2, 'B', 'A')`.

- (A) (3, 'B', 'C') is move 12.
- (B) (1, 'B', 'C') is move 9 (`moves[8]`) — an off-by-one with 0-based indexing.
- (C) (2, 'A', 'B') is move 2 (pattern of the first half).

Tip: disk d moves at steps whose number has exactly $d - 1$ trailing zero bits in binary; $10 = 1010_2$ has one trailing zero $\rightarrow$ disk 2.

Q.13 [NAT · 2 marks · Mutual recursion]

Answer: 22

`a` and `b` call each other; expand until a base case is reached and then substitute back.

- $a(10) = 10 + b(9)$
- $b(9) = 2 \cdot a(9 // 2) = 2 \cdot a(4)$

- $a(4) = 4 + b(3)$
- $b(3) = 2 \cdot a(1)$
- $a(1) = 1 + b(0) = 1 + 0 = 1$ ($b(0)$ is a base case returning 0)

Substituting back: $b(3) = 2$, $a(4) = 6$, $b(9) = 12$, $a(10) = 10 + 12 = 22$.

Only 9 calls are made in total (a and b alternate), so the depth is small despite $n = 10$.

Trap: $b(0)$ returns **0**, not 1 — the two functions have different base values. Using $a(0) = 1$ by mistake for $b(0)$ gives $a(1) = 2$ and a final answer of 26.

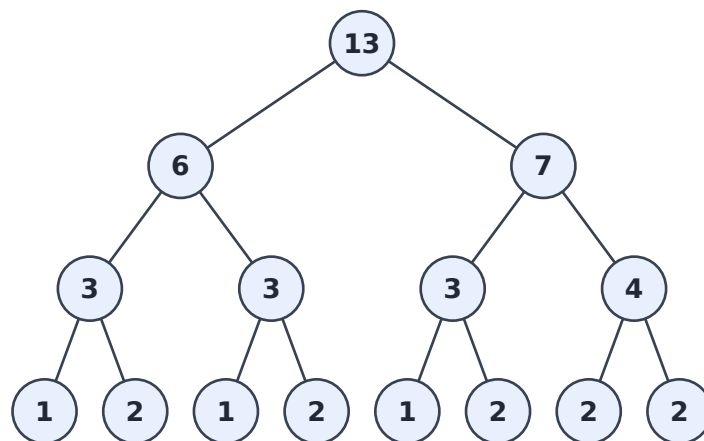
Q.14 [MSQ · 2 marks · Merge sort — recursion structure]

Answer: (A), (B), (D)

The recursion tree is a full binary tree whose leaves are the length-1 calls.

- (A) Longest chain of sizes: $13 \rightarrow 7 \rightarrow 4 \rightarrow 2 \rightarrow 1$ (always following the larger half). That is **5** frames. **True.**
- (B) A full binary tree with 13 leaves has 12 internal nodes $\rightarrow 13 + 12 = 25$ calls. **True.**
- (C) Merging sorted lists of sizes 6 and 7 needs at most $6 + 7 - 1 = 12$ comparisons (the last element is placed without comparing). **False.**
- (D) Every element ends up alone in exactly one leaf $\rightarrow 13$ leaf calls. **True.**

Trap: depth is governed by $\lceil \log_2 13 \rceil + 1 = 5$ frames (4 edges); and the merge worst case is $n - 1$, not n .



Recursion tree by sub-array length (each 2 has two leaf children, not drawn)

Q.15 [MCQ · 2 marks · Python — recursion with nonlocal state]

Answer: (D)

`walk` splits n into $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$ until pieces of size 1 remain, so it returns n and its recursion tree has exactly n leaves. Only leaf calls increment `count`. Because `count` lives in the enclosing scope of `make`, it is **shared and persistent** across calls of `w`, and `get` (a closure) reads its current value.

Arguments of `print` are evaluated left to right:

- $w(7) \rightarrow 7$; leaves: $7 \rightarrow (3, 4) \rightarrow (1, 2), (2, 2) \rightarrow \dots = 7$ leaves $\rightarrow \text{count} = 7$.
- `get()` $\rightarrow 7$.
- $w(4) \rightarrow 4$; 4 more leaves $\rightarrow \text{count} = 11$.
- `get()` $\rightarrow 11$.

Output: **7 7 4 11**.

- (A) assumes the lambda captured the value 7 at creation — closures see the variable, not a snapshot.
- (B) assumes `count` restarts at 0 for each call.

- (C) counts every call ($2n - 1 = 13$ for $n = 7$, then 7 more), but only leaves increment.

Trap: without `nonlocal`, `count += 1` would raise `UnboundLocalError`; with it, the state accumulates across calls.

Q.16 [MSQ · 2 marks · Recursive BST validation]

Answer: (A), (B), (C)

`local_ok` only compares each node with its **immediate children**, while `range_ok` passes down an interval (lo, hi) that every key in the subtree must satisfy — the correct BST definition.

- (A) In-order (left, root, right): 8, 15, 34, 30, 40, 45, 52 — not sorted, confirming T is not a BST. **True.**
- (B) Every parent-child pair is ordered correctly ($15 < 30 < 45$, $8 < 15 < 34$, $40 < 45 < 52$), so `local_ok` returns **True. True.**
- (C) `range_ok`: root 30 → left subtree must lie in $(-\infty, 30)$; node 15 → its right subtree must lie in $(15, 30)$; $34 \notin (15, 30)$ → **False. True.**
- (D) 31 is still > 30 , so it still violates the bound $(15, 30)$. **False.** (Any key in $(15, 30)$, e.g. 29, would fix it.)

Trap: checking only parent-child relations is the classic wrong BST test; the constraint from the root (30) reaches all the way down to 34.

Q.17 [MSQ · 2 marks · Recurrences from recursive code]

Answer: (A), (C), (D)

Write the recurrence for each function from its code, then solve it.

- (A) Two calls on $n/2$ plus $\Theta(\sqrt{n})$ work: $T(n) = 2T(n/2) + \sqrt{n}$. Here $n^{\log_2 2} = n$ dominates $\sqrt{n}$ (master case 1) → **$\Theta(n)$. True.**
- (B) Four calls on $n/2$ plus $\Theta(n^2)$ work: $T(n) = 4T(n/2) + n^2$. Now $n^{\log_2 4} = n^2$ equals $f(n)$ (master case 2) → **$\Theta(n^2 \log n)$, not $\Theta(n^2)$. False.**
- (C) Two calls on $n - 1$ plus $O(1)$: $T(n) = 2T(n-1) + 1 = 2^{n+1} - 1 \rightarrow$ **$\Theta(2^n)$. True.**
- (D) One call on $n - 1$ plus building a list of n elements: $T(n) = T(n-1) + \Theta(n) = \Theta(1 + 2 + \dots + n) = \Theta(n^2)$. **True.**

Trap: in (B) each of the $\log_2 n$ levels of the recursion tree does n^2 total work (4^i calls $\times (n/2^i)^2$), which adds the log factor. In (D) the `len(list(range(n)))` hides a linear cost.

Q.18 [NAT · 2 marks · Memoized recursion on a DAG — path counting]

Answer: 9

In a DAG, $\text{paths}(u) = \sum \text{paths}(v)$ over edges $u \rightarrow v$, with $\text{paths}(T) = 1$. Memoization evaluates each vertex once, in reverse topological order:

- $\text{paths}(T) = 1$
- $\text{paths}(E) = \text{paths}(T) = 1$
- $\text{paths}(D) = \text{paths}(E) + \text{paths}(T) = 2$
- $\text{paths}(F) = \text{paths}(E) + \text{paths}(T) = 2$
- $\text{paths}(A) = \text{paths}(D) + \text{paths}(E) = 3$
- $\text{paths}(B) = \text{paths}(D) + \text{paths}(F) = 4$
- $\text{paths}(C) = \text{paths}(F) = 2$
- $\text{paths}(S) = 3 + 4 + 2 = \mathbf{9}$

Listing them: S-A-D-E-T, S-A-D-T, S-A-E-T, S-B-D-E-T, S-B-D-T, S-B-F-E-T, S-B-F-T, S-C-F-E-T, S-C-F-T.

Tip: without memoization the recursion would recompute $\text{paths}(D)$, $\text{paths}(E)$, $\text{paths}(F)$ repeatedly; with it the cost is $\Theta(V + E)$.

Memo table (reverse topological order)

	T	E	D	F	A	B	C	S
paths	1	1	2	2	3	4	2	9

Q.19 [MCQ · 2 marks · Recursive linked-list surgery and aliasing]**Answer: (B)**

The loop builds $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ (prepending 7, 6, ..., 1). **sw** swaps nodes in **pairs** recursively: for the pair (h, h.next) it first swaps the rest of the list (starting at the third node), hangs that result after h, and puts h.next in front of h.

- $\text{sw}(7) \rightarrow 7$ (single node is returned unchanged).
- $\text{sw}(5)$: $5.\text{next} = \text{sw}(7) = 7$; $6.\text{next} = 5 \rightarrow$ returns $6 \rightarrow 5 \rightarrow 7$.
- $\text{sw}(3)$: $3.\text{next} = 6\dots$; $4.\text{next} = 3 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 7$.
- $\text{sw}(1)$: $1.\text{next} = 4\dots$; $2.\text{next} = 1 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 7$.

a still refers to the **node** holding 1, which is now the second node; walking from it gives $1 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 7$.

- (A) assumes **a** still sees the original list — but the nodes were relinked in place.
- (C) drops the unpaired last node 7, which the base case keeps.
- (D) assumes node 1 was detached; in fact its **next** was set to the swapped remainder.

Trap: a variable that refers to a node is not a snapshot of the list.



After **sw**: **a** points at the node holding 1 (second node)

Q.20 [NAT · 2 marks · Memoized recursion — grid paths with obstacles]**Answer: 39**

$P(r, c)$ = number of valid paths from (r, c) to the target = $P(r+1, c) + P(r, c+1)$, with 0 for blocked or out-of-grid cells and 1 at the target. Fill the table from the bottom-right corner:

- Row 5 and column 5 are all 1 (only one way: straight along the edge).
- Row 4: (4,4)=2, (4,3)=3, (4,2)=4, (4,1)=5, (4,0)=X.
- Row 3: (3,4)=3, (3,3)=X, (3,2)=4, (3,1)=9, (3,0)=9.
- Row 2: (2,4)=4, (2,3)=4, (2,2)=8, (2,1)=X, (2,0)=9.
- Row 1: (1,4)=5, (1,3)=X, (1,2)=8, (1,1)=8, (1,0)=17.
- Row 0: (0,4)=6, (0,3)=6, (0,2)=14, (0,1)=22, (0,0)=**39**.

Memoization makes each of the 36 states compute once, versus an exponential number of calls for plain recursion.

Trap: cell (4, 0) blocks the left column, so $(3, 0) = (3, 1) + 0 = 9$ rather than $9 + 1$; and blocked cells contribute 0, not 1.

P(r, c) = paths from each cell

	0	1	2	3	4	5
0	39	22	14	6	6	1
1	17	8	8	X	5	1
2	9	X	8	4	4	1
3	9	9	4	X	3	1
4	X	5	4	3	2	1
5	1	1	1	1	1	1

Mock Test 17

Iterators, Generators & Comprehensions

Difficulty: GATE-level

Focus areas: generators, iterator exhaustion, comprehensions, map/filter/zip

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Generator exhaustion and `in`]

Consider the following Python program. What is printed?

Python

```
1 nums = [1, 2, 3, 4, 5, 6]
2 g = (n for n in nums if n % 2 == 0)
3 print(4 in g, list(g), sum(g))
```

- (A) True [2, 4, 6] 12
- (B) True [6] 6
- (C) True [6] 0
- (D) True [] 0

Q.2 [NAT · numerical answer | 1 mark | Nested list comprehension — counting]

The value printed by the following Python program is ____.

Python

```
1 pairs = [(i, j) for i in range(6)
2           for j in range(i, 6)
3           if (i + j) % 3 == 0]
4 print(len(pairs))
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | `zip` over the same iterator]

Consider the following Python program. What is printed?

Python

```
1 it = iter([10, 20, 30, 40, 50, 60, 70])
2 pairs = list(zip(it, it))
3 print(pairs, next(it, -1))
```

- (A) [(10, 20), (30, 40), (50, 60)] -1
- (B) [(10, 20), (30, 40), (50, 60)] 70
- (C) [(10, 10), (20, 20), (30, 30), (40, 40), (50, 50), (60, 60), (70, 70)] -1
- (D) [(10, 20), (30, 40), (50, 60), (70, None)] -1

Q.4 [MSQ · one or more correct | 1 mark | Stacks — postfix evaluation]

The postfix expression $9\ 4\ 2\ -\ 3\ *\ +\ 6\ 2\ /\ -$ is evaluated with a stack: an operand is pushed; an operator pops the right operand, then the left operand, and pushes the result ($/$ is exact division here). Which of the following statements is/are TRUE?

- (A) At some moment the value 6 is on top of the stack
- (B) The maximum number of items on the stack at any time is 3
- (C) At some moment the stack holds 4 items
- (D) The final result is 12

Q.5 [NAT · numerical answer | 1 mark | Lazy `map`/`filter` pipelines]

The value printed by the following Python program is _____.

Python

```
1 log = []
2 def f(x):
3     log.append(x)
4     return x % 3 == 0
5
6 m = map(lambda x: x * x, filter(f, range(1, 20)))
7 found = any(v > 50 for v in m)
8 print(len(log))
```

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Binary search — `bisect` boundaries]

Consider the following Python program. What is printed?

Python

```
1 import bisect
2 a = [2, 4, 4, 4, 7, 9, 9, 12]
3 x = bisect.bisect_right(a, 4) - bisect.bisect_left(a, 9)
4 print(x, bisect.bisect_left(a, 8))
```

- (A) -1 5
- (B) -2 5
- (C) -1 4
- (D) 0 5

Q.7 [NAT · numerical answer | 1 mark | Hashing — separate chaining]

Keys 22, 15, 30, 44, 9, 63, 51, 36, 13 are inserted in this order into a hash table with 7 buckets using $h(k) = k \bmod 7$ and separate chaining, where each new key is inserted at the **head** of its chain. Searches are then performed for 44, 36, 58 and 70; each search scans its chain from the head until the key is found or the chain ends, and every key examined counts as one comparison. The total number of comparisons over the four searches is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Bubble sort with early exit]

Bubble sort with the early-exit flag (stop after the first pass that makes no swap; pass i compares adjacent pairs in $A[0..n-1-i]$) sorts $[4, 1, 2, 3, 7, 5, 6, 9, 8]$ in ascending order. The total number of key comparisons is

- (A) 8
- (B) 15
- (C) 21
- (D) 36

Q.9 [MSQ · one or more correct | 1 mark | Comprehension scope]

Consider the following Python 3 program. Which of the following statements is/are TRUE?

Python

```
1 x = 'outer'
2 sq = [x * 2 for x in range(3)]
3 print(x)
4 for x in range(3):
5     pass
6 print(x)
7 y = [i for i in range(3)]
```

- (A) `sq` is `[0, 2, 4]`
- (B) The second `print` outputs `2`
- (C) After the last line, the name `i` is defined at module level
- (D) The first `print` outputs `outer`

Q.10 [NAT · numerical answer | 1 mark | Graph from a comprehension — degrees]

An undirected graph G on vertices $1, 2, \dots, 8$ has edge set

$E = [(u, v) \text{ for } u \text{ in range}(1, 9) \text{ for } v \text{ in range}(u + 1, 9) \text{ if } v \% u == 0]$. The number of vertices of **odd** degree in G is _____.

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Late binding: list vs generator of lambdas]

Consider the following Python program. What is printed?

Python

```
1 fs = [lambda: i * 10 for i in range(3)]
2 gs = (lambda: i * 10 for i in range(3))
3 print([f() for f in fs], [g() for g in gs])
```

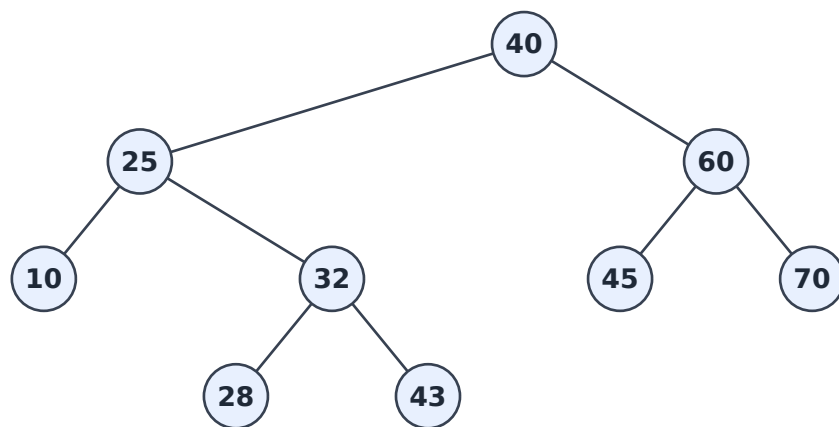
- (A) `[20, 20, 20] [20, 20, 20]`
- (B) `[0, 10, 20] [0, 10, 20]`
- (C) `[20, 20, 20] [0, 10, 20]`
- (D) `[0, 10, 20] [20, 20, 20]`

Q.12 [NAT · numerical answer | 2 marks | Recursive generators — in-order traversal]

The binary tree T shown is encoded as nested lists `[value, left, right]` (None for an empty child). The value printed by the following program is _____.

Python

```
1 def inorder(t):
2     if t:
3         yield from inorder(t[1])
4         yield t[0]
5         yield from inorder(t[2])
6
7 def check(t):
8     it = inorder(t)
9     prev = next(it)
10    for k, v in enumerate(it, 2):
11        if v <= prev:
12            return k * 100 + prev
13        prev = v
14    return 0
15
16 L = lambda v: [v, None, None]
17 T = [40, [25, L(10), [32, L(28), L(43)]], [60, L(45), L(70)]]
18 print(check(T))
```



Tree T

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Iterator vs iterable classes]

Consider the following Python program. Which of the following statements about its output is/are TRUE?

Python

```
1 class Countdown:
2     def __init__(self, n):
3         self.n = n
4     def __iter__(self):
5         return self
6     def __next__(self):
7         if self.n <= 0:
8             raise StopIteration
9         self.n -= 1
10        return self.n
11
12 class Down:
13     def __init__(self, n):
14         self.n = n
15     def __iter__(self):
16         k = self.n
17         while k > 0:
18             k -= 1
19             yield k
20
21 c, r = Countdown(3), Down(3)
22 print(list(c), list(c))
23 print(list(r), list(r))
24 print(list(zip(r, r)))
25 print(sum(c), max(r))
```

- (A) Line 1 is [2, 1, 0] []
- (B) Line 2 is [2, 1, 0] []
- (C) Line 3 is [(2, 2), (1, 1), (0, 0)]
- (D) Line 4 is 0 2

Q.14 [NAT · numerical answer | 2 marks | Linked list mutated during generator traversal]

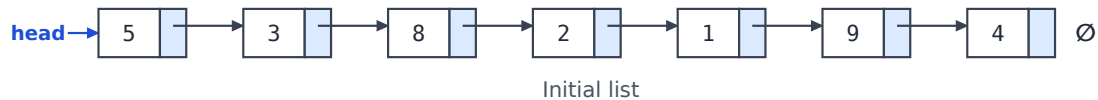
A singly linked list $\text{head} \rightarrow 5 \rightarrow 3 \rightarrow 8 \rightarrow 2 \rightarrow 1 \rightarrow 9 \rightarrow 4$ is processed by the program below. The value printed is _____.

Python

```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 def nodes(h):
6     while h:
7         yield h
8         h = h.next
9
10 head = None
11 for v in reversed([5, 3, 8, 2, 1, 9, 4]):
12     head = Node(v, head)
13
14 for n in nodes(head):
15     if n.next and n.val > n.next.val:
16         n.next = n.next.next
17
18 print(''.join(str(n.val) for n in nodes(head)))

```



Answer: _____ (enter a numerical value)

Q.15 [NAT · numerical answer | 2 marks | Merge sort — comparison count]

Top-down merge sort (split at $m = \lfloor n/2 \rfloor$, left part $A[0..m-1]$) sorts $[7, 2, 9, 4, 3, 8, 6, 1]$. The merge step compares the heads of the two runs and stops comparing as soon as one run is exhausted. The total number of key comparisons over all merges is _____.

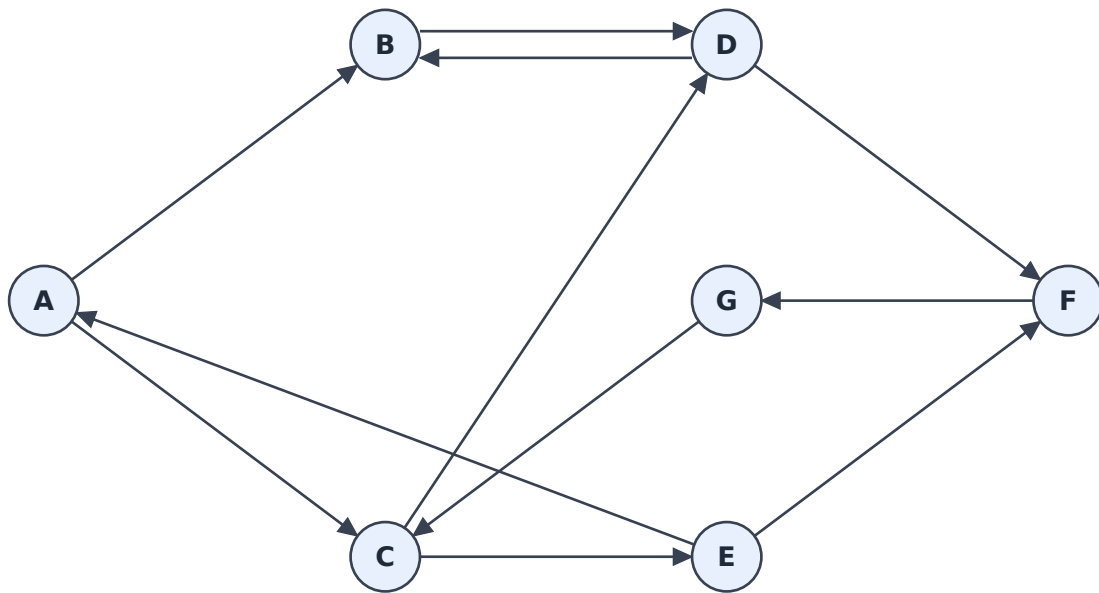
Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Frontier-based BFS with set comprehensions]

The program below performs a level-by-level BFS from A on the directed graph shown. Which of the following statements is/are TRUE?

Python

```
1 G = {'A': 'BC', 'B': 'D', 'C': 'DE', 'D': 'FB',
2     'E': 'FA', 'F': 'G', 'G': 'C'}
3 seen = {'A'}
4 frontier = {'A'}
5 levels = []
6 while frontier:
7     levels.append(''.join(sorted(frontier)))
8     frontier = {v for u in frontier for v in G[u]} - seen
9     seen |= frontier
10 print(levels)
```



- (A) Exactly 5 strings are printed in the list
- (B) The string at index 2 of `levels` is 'DE'
- (C) G appears in the string at index 3 of `levels`
- (D) If the statement `seen |= frontier` were deleted, the loop would never terminate

Q.17 [NAT · numerical answer | 2 marks | Quicksort with comprehensions — call count]

The value printed by the following Python program is _____.

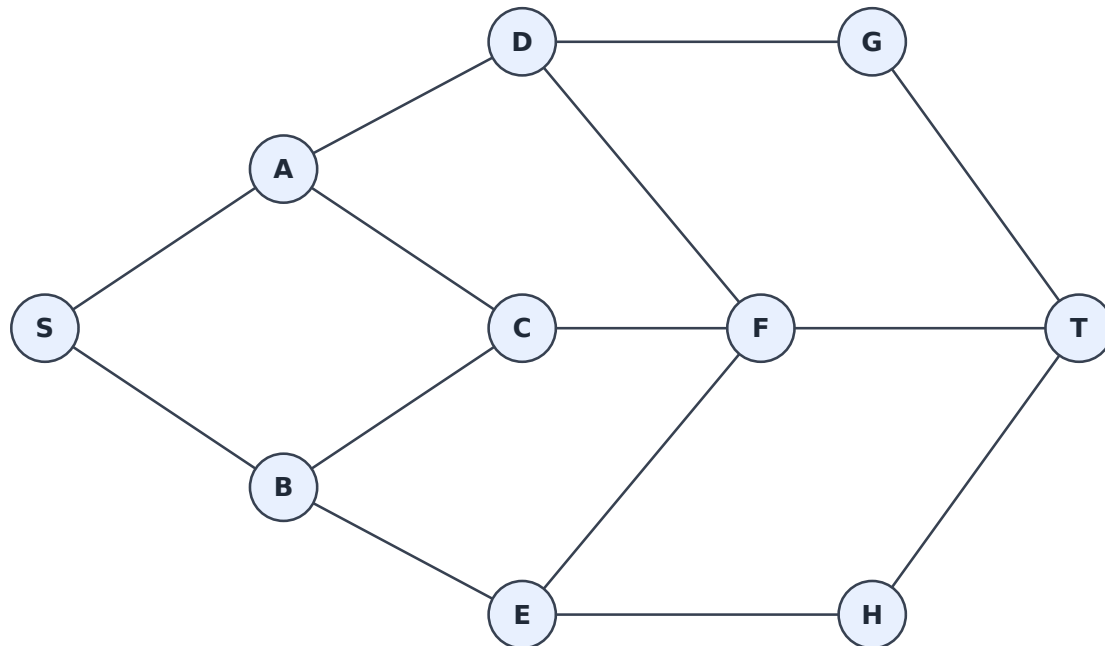
Python

```
1 calls = 0
2 def qs(a):
3     global calls
4     calls += 1
5     if len(a) <= 1:
6         return a
7     p = a[0]
8     lo = [x for x in a[1:] if x < p]
9     hi = [x for x in a[1:] if x >= p]
10    return qs(lo) + [p] + qs(hi)
11
12 qs([5, 3, 8, 1, 9, 2, 7])
13 print(calls)
```

Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Counting shortest paths with BFS]

In the unweighted undirected graph shown, the number of distinct shortest paths from S to T is



- (A) 4
- (B) 6
- (C) 5
- (D) 8

Q.19 [MCQ · single correct | 2 marks | Complexity of a generator expression]

Consider the Python expression below, where n is a positive integer. The number of items produced by the generator (and hence the value of `count`) grows as

```
count = sum(1 for i in range(n) for j in range(i, n, i + 1))
```

- (A) $\Theta(n)$
- (B) $\Theta(n^2)$
- (C) $\Theta(n \sqrt{n})$
- (D) $\Theta(n \log n)$

Q.20 [MSQ · one or more correct | 2 marks | Dict/set comprehensions and stable sorting]

Let `words = ['kiwi', 'fig', 'apple', 'date', 'plum', 'pear', 'banana']`. Which of the following expressions evaluate to True?

- (A) `{len(w): w for w in words}[4] == 'pear'`
- (B) `sorted(words, key=len)[1] == 'kiwi'`
- (C) `sorted(words, key=lambda w: (-len(w), w))[2] == 'date'`
- (D) `len({w[0] for w in words}) == 7`

Answer Key — Mock Test 17

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Generator exhaustion and <code>in</code>
2	NAT	1	7	Nested list comprehension — counting
3	MCQ	1	(A)	<code>zip</code> over the same iterator
4	MSQ	1	(A), (B), (D)	Stacks — postfix evaluation
5	NAT	1	9	Lazy <code>map/filter</code> pipelines
6	MCQ	1	(A)	Binary search — <code>bisect</code> boundaries
7	NAT	1	9	Hashing — separate chaining
8	MCQ	1	(B)	Bubble sort with early exit
9	MSQ	1	(A), (B), (D)	Comprehension scope
10	NAT	1	6	Graph from a comprehension — degrees
11	MCQ	2	(C)	Late binding: list vs generator of lambdas
12	NAT	2	643	Recursive generators — in-order traversal
13	MSQ	2	(A), (C), (D)	Iterator vs iterable classes
14	NAT	2	5819	Linked list mutated during generator traversal
15	NAT	2	17	Merge sort — comparison count
16	MSQ	2	(A), (B), (D)	Frontier-based BFS with set comprehensions
17	NAT	2	9	Quicksort with comprehensions — call count
18	MCQ	2	(B)	Counting shortest paths with BFS
19	MCQ	2	(D)	Complexity of a generator expression
20	MSQ	2	(A), (B), (C)	Dict/set comprehensions and stable sorting

Detailed Solutions — Mock Test 17

Q.1 [MCQ · 1 mark · Generator exhaustion and `in`]

Answer: (C)

A generator expression is a **one-shot iterator**. The `in` operator on an iterator consumes items until it finds a match (or exhausts it), and whatever was consumed is gone.

Arguments of `print` are evaluated left to right:

- `4 in g`: the generator yields 2 ($\neq 4$), then 4 (match) $\rightarrow$ `True`. Items 2 and 4 are consumed.
- `list(g)`: continues from where it stopped: 5 is filtered out, 6 is yielded $\rightarrow$ `[6]`. The generator is now exhausted.
- `sum(g)`: nothing left $\rightarrow$ `sum` of an empty iterable is 0.

Output: `True [6] 0`.

- (A) treats `g` like a list that can be re-read.
- (B) assumes `sum` restarts the generator, or forgets that `list` exhausted it.
- (D) assumes `in` exhausts the whole generator even after a match.

Trap: membership tests stop at the first match; they do not necessarily drain the iterator.

Q.2 [NAT · 1 mark · Nested list comprehension — counting]

Answer: 7

In a comprehension with several `for` clauses, the **leftmost** loop is the outer loop and later clauses may use earlier variables (here `j` starts at `i`). The `if` filters each pair.

Enumerate $i \leq j < 6$ with $(i + j)$ divisible by 3:

- $i = 0$: $j \in \{0, 3\} \rightarrow 2$ pairs
- $i = 1$: $j \in \{2, 5\} \rightarrow 2$ pairs
- $i = 2$: $j \in \{4\} \rightarrow 1$ pair ($j = 1$ is not allowed since $j \geq i$)
- $i = 3$: $j \in \{3\} \rightarrow 1$ pair
- $i = 4$: $j \in \{5\} \rightarrow 1$ pair
- $i = 5$: $j = 5$ gives 10, not divisible $\rightarrow 0$

Total = $2 + 2 + 1 + 1 + 1 + 0 = 7$: (0,0), (0,3), (1,2), (1,5), (2,4), (3,3), (4,5).

Trap: counting ordered pairs over the full 6×6 grid (12 pairs) or forgetting the diagonal pairs (0,0) and (3,3).

Q.3 [MCQ · 1 mark · `zip` over the same iterator]

Answer: (A)

Both arguments of `zip` are the **same** iterator object, so each tuple takes two consecutive items: (10, 20), (30, 40), (50, 60).

For the fourth tuple, `zip` asks its first argument for an item and receives 70; it then asks the second argument, which is exhausted, so `zip` stops. The 70 already fetched is **discarded** — it is not put back. Hence `next(it, -1)` finds nothing and returns the default `-1`.

- (B) assumes the leftover 70 is still available.
- (C) would happen with `zip(lst, lst)` on a *list*, which creates two independent iterators.
- (D) is the behaviour of `itertools.zip_longest`, not `zip`.

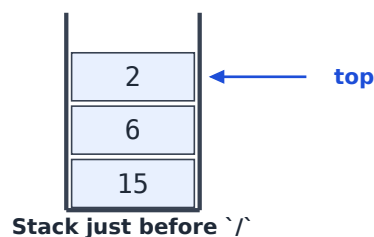
Trap: `zip` silently drops a partially-built tuple; with a shared iterator that also silently loses an element.

Q.4 [MSQ · 1 mark · Stacks — postfix evaluation]**Answer: (A), (B), (D)**

Trace the stack (bottom → top) after each token:

- $9 \rightarrow [9]$
- $4 \rightarrow [9, 4]$
- $2 \rightarrow [9, 4, 2]$
- $- \rightarrow 4 - 2 = 2 \rightarrow [9, 2]$
- $3 \rightarrow [9, 2, 3]$
- $* \rightarrow 2 \times 3 = 6 \rightarrow [9, 6]$
- $+ \rightarrow 9 + 6 = 15 \rightarrow [15]$
- $6 \rightarrow [15, 6]$
- $2 \rightarrow [15, 6, 2]$
- $/ \rightarrow 6 / 2 = 3 \rightarrow [15, 3]$
- $- \rightarrow 15 - 3 = 12 \rightarrow [12]$
- (A) **True** — 6 is on top after $*$ and again when the operand 6 is pushed.
- (B) **True** — depth 3 is reached (after the first 2, after 3, after the second 2) and never exceeded.
- (C) **False** — the depth never reaches 4.
- (D) **True** — result 12.

Trap: operand order — for $-$ and $/$ the *first* pop is the right operand. Popping in the wrong order gives $2 - 4$ and $2 / 6$.

**Q.5 [NAT · 1 mark · Lazy `map`/`filter` pipelines]****Answer: 9**

In Python 3 `map` and `filter` return **lazy iterators**; nothing is computed until a value is requested. `any` stops at the first true value (short-circuit), so only the prefix of `range(1, 20)` needed to find it is ever passed to `f`.

- `filter` calls `f(1)`, `f(2)`, `f(3)` → 3 passes; `map` yields 9 (not > 50).
- `f(4)`, `f(5)`, `f(6)` → 6 passes; yields 36 (not > 50).
- `f(7)`, `f(8)`, `f(9)` → 9 passes; yields 81 > 50 → `any` returns True and stops.

`f` was called for `x = 1..9`, so `len(log) = 9`.

Trap: answering 19 assumes `filter` processes the whole range eagerly (Python 2 behaviour, or wrapping it in `list`).

Q.6 [MCQ · 1 mark · Binary search — `bisect` boundaries]**Answer: (A)**

`bisect_left(a, x)` returns the first index `i` with `a[i] ≥ x` (the leftmost insertion point); `bisect_right(a, x)` returns the first index with `a[i] > x`. Both are binary searches.

- `bisect_right(a, 4)`: the 4s occupy indices 1..3, so the first element > 4 is at index **4**.
- `bisect_left(a, 9)`: the first element ≥ 9 is `a[5] = 9` → **5**.
- `x = 4 - 5 = -1`.
- `bisect_left(a, 8)`: 8 is absent; first element ≥ 8 is `a[5] = 9` → **5**.

Output: -1 5.

- (B) uses `bisect_right(a, 4) = 3` (the index of the last 4, off by one).
- (C) returns 4 for 8, i.e. the index of the last element < 8 rather than the insertion point.
- (D) confuses `bisect_left(a, 9)` with `bisect_left(a, 7) = 4`.

Tip: `bisect_right(a, x) - bisect_left(a, x)` counts occurrences of x in $O(\log n)$.

Q.7 [NAT · 1 mark · Hashing — separate chaining]

Answer: 9

Bucket of each key: $22 \rightarrow 1$, $15 \rightarrow 1$, $30 \rightarrow 2$, $44 \rightarrow 2$, $9 \rightarrow 2$, $63 \rightarrow 0$, $51 \rightarrow 2$, $36 \rightarrow 1$, $13 \rightarrow 6$. With head insertion each chain lists keys in **reverse** insertion order:

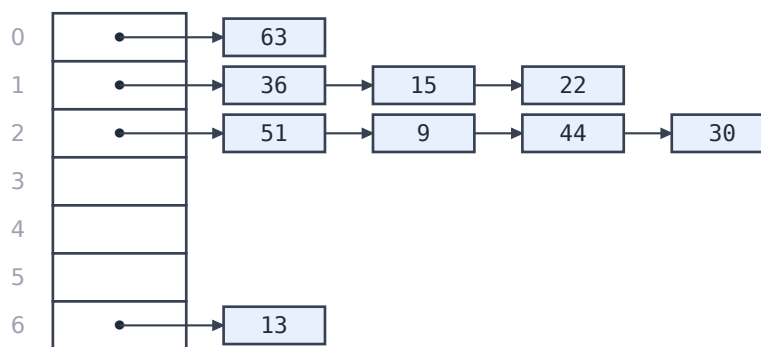
- bucket 0: 63
- bucket 1: $36 \rightarrow 15 \rightarrow 22$
- bucket 2: $51 \rightarrow 9 \rightarrow 44 \rightarrow 30$
- bucket 6: 13

Searches:

- 44 (bucket 2): 51, 9, 44 $\rightarrow$ 3 comparisons.
- 36 (bucket 1): found at the head $\rightarrow$ 1.
- 58 ($58 \bmod 7 = 2$): 51, 9, 44, 30, end $\rightarrow$ 4 (unsuccessful, whole chain).
- 70 ($70 \bmod 7 = 0$): 63, end $\rightarrow$ 1.

Total = $3 + 1 + 4 + 1 = 9$.

Trap: building chains in insertion order (tail insertion) would make 44 cost 2 and 36 cost 3. Also, an unsuccessful search in an empty bucket would cost 0, but bucket 0 is not empty here.



Chains after all insertions (head first)

Q.8 [MCQ · 1 mark · Bubble sort with early exit]

Answer: (B)

Bubble sort pass i scans $n - 1 - i$ adjacent pairs and carries the largest element of the unsorted part to the end. With the flag it stops after a pass with zero swaps.

Pass 0 (8 comparisons): 4 swaps past 1, 2, 3; 7 swaps past 5 and 6; 9 swaps past 8 $\rightarrow$ [1, 2, 3, 4, 5, 6, 7, 8, 9] (6 swaps).

Pass 1 (7 comparisons): the array is sorted, no swaps $\rightarrow$ stop.

Total comparisons = $8 + 7 = 15$.

- (A) 8 forgets that one more pass is needed to *detect* that the array is sorted.
- (C) $21 = 8 + 7 + 6$ adds a non-existent third pass.
- (D) $36 = n(n - 1)/2$ is the cost without the early-exit flag.

Tip: a pass can move a large element many places right, but a small element moves only one place left per pass — here every element was at most one “left-move” from its place.

Array after each pass									
	0	1	2	3	4	5	6	7	8
start	4	1	2	3	7	5	6	9	8
pass 0	1	2	3	4	5	6	7	8	9
pass 1	1	2	3	4	5	6	7	8	9

Q.9 [MSQ · 1 mark · Comprehension scope]**Answer: (A), (B), (D)**

In Python 3 a comprehension runs in its **own scope**: its loop variable does not leak into the enclosing scope. An ordinary **for** loop, by contrast, binds its variable in the current scope, and the variable keeps its last value after the loop.

- (A) **True** — inside the comprehension **x** is an int, so **x * 2** is 0, 2, 4 (not string repetition).
- (B) **True** — the **for** loop rebinds the module-level **x**; after the loop it is 2.
- (C) **False** — **i** exists only inside the comprehension; referring to **i** afterwards raises **NameError**.
- (D) **True** — the comprehension's **x** is local to it, so the module-level **x** is still 'outer'.

Trap: Python 2 list comprehensions *did* leak their variable; Python 3 does not. Do not carry that habit over to **for** loops, which still leak.

Q.10 [NAT · 1 mark · Graph from a comprehension — degrees]**Answer: 6**

An edge joins $u < v$ exactly when u divides v . List the edges by u :

- $u = 1$: (1,2) ... (1,8) → 7 edges
- $u = 2$: (2,4), (2,6), (2,8)
- $u = 3$: (3,6)
- $u = 4$: (4,8)
- $u = 5, 6, 7$: none (no multiple ≤ 8)

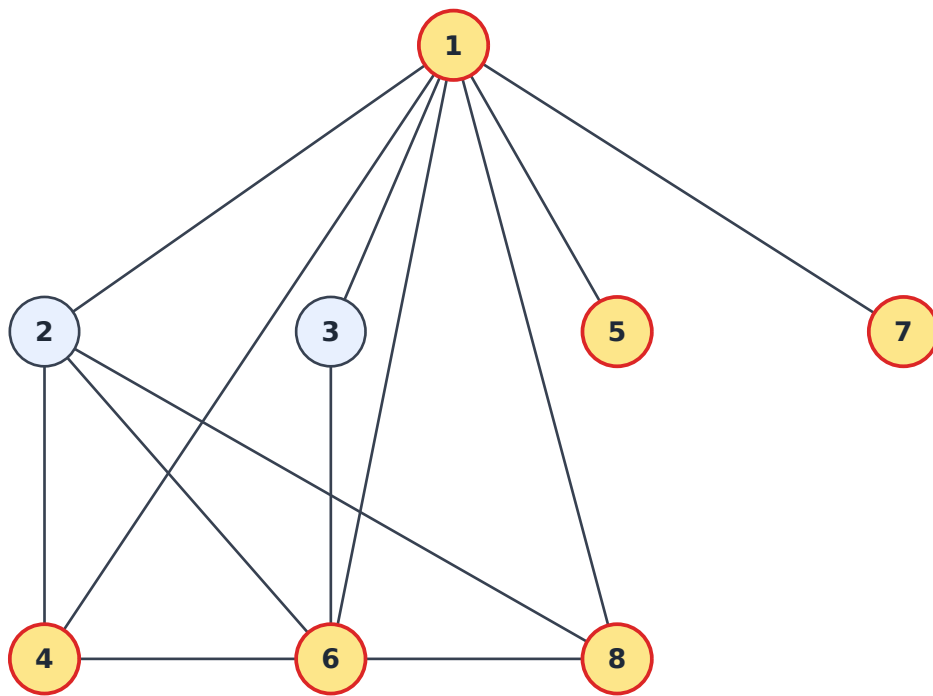
That is 12 edges. Degrees (proper divisors $< v$ plus multiples ≤ 8):

- $\text{deg } 1 = 7$, $\text{deg } 2 = 1 + 3 = 4$, $\text{deg } 3 = 1 + 1 = 2$, $\text{deg } 4 = 2 + 1 = 3$,
- $\text{deg } 5 = 1$, $\text{deg } 6 = 3$ (1, 2, 3), $\text{deg } 7 = 1$, $\text{deg } 8 = 3$ (1, 2, 4).

Odd degrees: vertices 1, 4, 5, 6, 7, 8 → **6**.

Check with the handshake lemma: $\text{sum of degrees} = 7+4+2+3+1+3+1+3 = 24 = 2 \times 12$ ✓, and the number of odd-degree vertices is even ✓.

Trap: forgetting that vertex 1 divides everything (degree 7, odd).



Divisibility graph (odd-degree vertices highlighted)

Q.11 [MCQ · 2 marks · Late binding: list vs generator of lambdas]**Answer: (C)**

A lambda does not store the *value* of `i`; it stores a reference to the variable `i` of the enclosing comprehension scope and looks it up **when it is called** (late binding).

List fs: the list comprehension runs to completion immediately, creating three lambdas that all close over the same `i`. When the loop finishes, `i = 2`. Calling them later gives 2×10 three times $\rightarrow [20, 20, 20]$.

Generator gs: nothing runs until `[g() for g in gs]` pulls items. The generator creates the first lambda while `i = 0` and *pauses*; the consumer calls it right away $\rightarrow 0$. Then the generator resumes, sets `i = 1`, yields the second lambda, which is called at once $\rightarrow 10$, and so on $\rightarrow [0, 10, 20]$.

- (A) assumes the generator also runs to completion before the calls.
- (B) ignores late binding for the list.
- (D) swaps the two behaviours.

Tip: to freeze the value use a default argument: `lambda i=i: i * 10`. **Trap:** the difference here comes only from *when* each lambda is called relative to the loop.

Q.12 [NAT · 2 marks · Recursive generators — in-order traversal]**Answer: 643**

`inorder` is a recursive generator: `yield from` forwards every value of the sub-generator, so values come out in in-order sequence, one at a time. `check` looks for the first place where the sequence is not strictly increasing, i.e. where `T` fails to be a BST.

In-order sequence of `T`: 10, 25, 28, 32, **43, 40**, 45, 60, 70.

- `prev = next(it)` consumes 10 (position 1).
- `enumerate(it, 2)` numbers the *remaining* values starting at 2: 25 $\rightarrow$ 2, 28 $\rightarrow$ 3, 32 $\rightarrow$ 4, 43 $\rightarrow$ 5, 40 $\rightarrow$ 6.
- At `k = 6`, `v = 40` $\leq$ `prev = 43` $\rightarrow$ return $6 \times 100 + 43 = \mathbf{643}$.

The violation exists because 43 is in the **left** subtree of 40 yet larger than 40 — every parent-child pair is locally fine ($32 < 43$), which is why checking only children is not enough to validate a BST.

Trap: starting the count at 1 for the remaining values (giving 543) or returning v instead of $prev$ (giving 640).

Q.13 [MSQ · 2 marks · Iterator vs iterable classes]

Answer: (A), (C), (D)

`Countdown` is an **iterator**: `__iter__` returns `self`, so all loops share one cursor (`self.n`) and it can be consumed only once. `Down` is an **iterable**: its `__iter__` is a generator function, so each call to `iter(r)` returns a **fresh** generator with its own k ; `self.n` is never modified.

- Line 1: the first `list(c)` gives 2, 1, 0 and leaves $n = 0$; the second gets nothing $\rightarrow [2, 1, 0] []$. (A) **True**.
- Line 2: each `list(r)` gets a new generator $\rightarrow [2, 1, 0] [2, 1, 0]$. (B) **False**.
- Line 3: `zip(r, r)` calls `iter(r)` twice $\rightarrow$ two independent generators advancing in step $\rightarrow [(2, 2), (1, 1), (0, 0)]$. (C) **True**. (Contrast with `zip` on a single shared iterator, which would pair consecutive items.)
- Line 4: `c` is exhausted $\rightarrow \text{sum}(c) = 0$; `max(r)` uses a fresh generator $\rightarrow 2$. (D) **True**.

Trap: assuming every class with `__iter__` behaves like a list. Whether re-iteration works depends on whether `__iter__` returns `self` or a new iterator.

Q.14 [NAT · 2 marks · Linked list mutated during generator traversal]

Answer: 5819

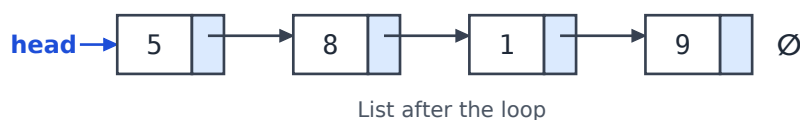
The generator yields a node and, **when resumed**, reads `h.next` — *after* the loop body has possibly modified that pointer. So a bypassed node is never visited, and the scan continues from the new successor.

- $n = 5$: next is 3, $5 > 3 \rightarrow 5.\text{next} = 8$. Generator resumes: $h = 5.\text{next} = 8$.
- $n = 8$: next is 2, $8 > 2 \rightarrow 8.\text{next} = 1$. Next $h = 1$.
- $n = 1$: next is 9, $1 < 9 \rightarrow$ no change. Next $h = 9$.
- $n = 9$: next is 4, $9 > 4 \rightarrow 9.\text{next} = \text{None}$. Next $h = \text{None} \rightarrow$ generator ends.

Final list $5 \rightarrow 8 \rightarrow 1 \rightarrow 9$; the print joins the values $\rightarrow 5819$.

Note that $8 > 1$ was never re-examined after 2 was removed — each node is compared only once, so the result is not “remove every element smaller than its predecessor in the final list”.

Trap: if the generator had saved `nxt = h.next` *before* yielding, it would visit the removed nodes 3 and 2 too, but the printed list would still be built from the live links.



Q.15 [NAT · 2 marks · Merge sort — comparison count]

Answer: 17

Merge sort on 8 keys has three levels of merges. Count comparisons per merge (a merge of runs of sizes p and q costs $p + q$ — (elements copied after one run empties)).

Level 1 (pairs): $[7|2], [9|4], [3|8], [6|1] \rightarrow 1$ comparison each $\rightarrow 4$. Runs: $[2,7] [4,9] [3,8] [1,6]$.

Level 2:

- $[2,7] + [4,9]$: $2 < 4, 7 > 4, 7 < 9 \rightarrow$ left empty, copy 9 $\rightarrow 3 \rightarrow [2,4,7,9]$
- $[3,8] + [1,6]$: $3 > 1, 3 < 6, 8 > 6 \rightarrow$ right empty, copy 8 $\rightarrow 3 \rightarrow [1,3,6,8]$

Level 3: [2,4,7,9] + [1,3,6,8]: $2 > 1$, $2 < 3$, $4 > 3$, $4 < 6$, $7 > 6$, $7 < 8$, $9 > 8 \rightarrow$ right empty, copy 9 $\rightarrow$ 7.

Total = $4 + 3 + 3 + 7 = 17$.

Bounds check: for $n = 8$ merge sort uses between 12 ($= (n/2) \cdot \log_2 n$) and 17 ($= n \log_2 n - n + 1$) comparisons. This input is a worst case: in every merge the last two output elements come from different runs.

Trap: counting $p + q$ comparisons per merge (giving 24) or omitting the level-1 merges.

Runs after each merge level

	runs	comparisons
start	7 2 9 4 3 8 6 1	—
level 1	7 4 9 3 8 1	4
level 2	2 4 7 9 1 3 6 8	3 + 3
level 3	1 2 3 4 6 7 8 9	7

Q.16 [MSQ · 2 marks · Frontier-based BFS with set comprehensions]

Answer: (A), (B), (D)

Each iteration replaces the frontier by all out-neighbours of the current frontier that have never been seen — exactly the next BFS level.

- Level 0: {A}
- Level 1: neighbours {B, C} — {A} $\rightarrow$ 'BC'; seen = {A, B, C}
- Level 2: B $\rightarrow$ D; C $\rightarrow$ D, E $\rightarrow$ {D, E} $\rightarrow$ 'DE'; seen adds D, E
- Level 3: D $\rightarrow$ F, B; E $\rightarrow$ F, A $\rightarrow$ {F, B, A} — seen = {F} $\rightarrow$ 'F'
- Level 4: F $\rightarrow$ G $\rightarrow$ 'G'
- Next: G $\rightarrow$ C, already seen $\rightarrow$ empty set $\rightarrow$ loop ends.

Printed: ['A', 'BC', 'DE', 'F', 'G'].

- (A) **True** — 5 levels.
- (B) **True**.
- (C) **False** — G is at index 4 (distance 4 from A); index 3 is 'F'.
- (D) **True** — without updating **seen**, only A is ever excluded. The cycle $C \rightarrow D \rightarrow F \rightarrow G \rightarrow C$ never involves A, so the frontier keeps cycling (e.g. ... {G} $\rightarrow$ {C} $\rightarrow$ {D, E} $\rightarrow$ {F, B} $\rightarrow$...) and never becomes empty.

Trap: thinking the - **seen** alone guarantees termination; it only works if **seen** grows.

Q.17 [NAT · 2 marks · Quicksort with comprehensions — call count]

Answer: 9

Every call with ≥ 2 elements makes exactly two recursive calls (one possibly on an empty list); calls on lists of length 0 or 1 are leaves. So calls = $2 \times (\text{internal calls}) + 1$.

Recursion tree (pivot = first element):

- $qs([5,3,8,1,9,2,7]) \rightarrow lo = [3,1,2], hi = [8,9,7]$
- $qs([3,1,2]) \rightarrow lo = [1,2], hi = []$
- $qs([1,2]) \rightarrow lo = [], hi = [2]$
- $qs([]), qs([2])$ — leaves
- $qs([])$ (hi of 3) — leaf
- $qs([8,9,7]) \rightarrow lo = [7], hi = [9]$
- $qs([7]), qs([9])$ — leaves

Internal calls: [5,...], [3,1,2], [1,2], [8,9,7] $\rightarrow$ 4. Total calls = $2 \times 4 + 1 = 9$.

Trap: forgetting the calls on empty lists (which still increment **calls**) gives 6. Note: this version is not in-place — each level builds new lists, using $\Theta(n)$ extra memory per level.

All calls in the order they are made

	argument	pivot	lo	hi
1	5 3 8 1 9 2 7	5	3 1 2	8 9 7
2	3 1 2	3	1 2	–
3	1 2	1	–	2
4	–			
5	2			
6	–			
7	8 9 7	8	7	9
8	7			
9	9			

Q.18 [MCQ · 2 marks · Counting shortest paths with BFS]

Answer: (B)

Run BFS from S and propagate counts: $\text{paths}(v) = \sum \text{paths}(u)$ over neighbours u with $\text{dist}(u) = \text{dist}(v) - 1$.

- dist 0: S (1 path)
- dist 1: A (1), B (1)
- dist 2: C from A and B $\rightarrow 2$; D from A $\rightarrow 1$; E from B $\rightarrow 1$
- dist 3: F from C, D, E $\rightarrow 2 + 1 + 1 = 4$; G from D $\rightarrow 1$; H from E $\rightarrow 1$
- dist 4: T from F, G, H $\rightarrow 4 + 1 + 1 = 6$

So the shortest distance is 4 and there are **6** shortest paths.

- (A) 4 counts only the paths through F.
- (C) 5 misses one of the two routes into C.
- (D) 8 adds paths of length 5 or counts edges D-F/E-F twice.

Trap: only predecessors **one level closer** contribute; edges inside a level or going back do not.

BFS distance and number of shortest paths

	S	A	B	C	D	E	F	G	H	T
dist	0	1	1	2	2	2	3	3	3	4
#paths	1	1	1	2	1	1	4	1	1	6

Q.19 [MCQ · 2 marks · Complexity of a generator expression]

Answer: (D)

For a fixed i , `range(i, n, i + 1)` has $\lceil (n - i)/(i + 1) \rceil$ elements. Summing over i :

$$\text{count} = \sum_{i=0}^{n-1} \lceil (n - i)/(i + 1) \rceil \approx \sum_{k=1}^n (n + 1 - k)/k = (n + 1) \cdot H_n - n,$$

where $k = i + 1$ and $H_n = 1 + 1/2 + \dots + 1/n \approx \ln n$. Hence $\text{count} = \Theta(n \log n)$.

Numerical check: $n = 12$ gives $\text{count} = 35$; $n = 1000$ gives $7069 \approx 1.02 \cdot n \ln n$; $n = 10000$ gives $93668 \approx 1.02 \cdot n \ln n$ — the ratio to $n \ln n$ stays constant.

- (A) $\Theta(n)$ would hold only if the inner loop did $O(1)$ work on average.
- (B) $\Theta(n^2)$ ignores the growing step — that would be the cost with step 1.

- (C) $\Theta(n\sqrt{n})$ would come from step sizes like $\sqrt{n}$, not $i + 1$.

Tip: an inner loop with step i (or $i + 1$) is the harmonic-series pattern, as in the sieve of Eratosthenes.

Q.20 [MSQ · 2 marks · Dict/set comprehensions and stable sorting]

Answer: (A), (B), (C)

- (A) In a dict comprehension a repeated key is **overwritten**; the last word of length 4 in the list is 'pear' (kiwi, date, plum, pear in order). **True.**
- (B) **sorted** is **stable**: equal keys keep their original relative order. By length: fig(3), then kiwi, date, plum, pear (4, original order), apple(5), banana(6). Index 1 is 'kiwi'. **True.**
- (C) Key $(-len, word)$: longest first, ties alphabetical $\rightarrow$ banana, apple, date, kiwi, pear, plum, fig. Index 2 is 'date'. **True.**
- (D) First letters: k, f, a, d, p, p, b $\rightarrow$ as a set $\{k, f, a, d, p, b\}$ has **6** elements. **False.**

Trap: in (A) assuming the *first* occurrence wins (it would give 'kiwi'); in (D) forgetting that sets drop duplicates; in (C) applying stability instead of the secondary key — the tuple key makes the original order irrelevant.

Mock Test 18

Scoping, Closures, OOP & Exceptions

Difficulty: GATE-level

Focus areas: LEGB, global/nonlocal, closures, classes, try/except/finally

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — closures and enclosing scope]

Consider the following Python program. What is printed?

Python

```

1 x = 'g'
2
3 def outer():
4     x = 'e'
5     def inner():
6         return x
7     x = 'e2'
8     return inner
9
10 f = outer()
11 print(f(), x)
```

- (A) e2 g
- (B) e g
- (C) g g
- (D) e2 e2

Q.2 [NAT · numerical answer | 1 mark | Python — late binding in lambdas]

Consider the following Python program. The value printed is ____.

Python

```

1 fs = [lambda: i * i for i in range(4)]
2 gs = [lambda i=i: i * i for i in range(4)]
3 print(sum(f() for f in fs) + sum(g() for g in gs))
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Python — try / except / finally with return]

Consider the following Python program. What is printed?

Python

```

1 def f():
2     try:
3         return 'try'
4     finally:
5         print('fin', end=' ')
6
7 def g():
8     try:
9         raise ValueError
10    except ValueError:
11        return 1
12    finally:
13        return 2
14
15 print(f(), g())

```

- (A) try fin 2
- (B) try 1 fin
- (C) fin try 1
- (D) fin try 2

Q.4 [MSQ · one or more correct | 1 mark | Python — class vs instance attributes]

Consider the following Python code. Which of the following statements is/are TRUE immediately after it runs (each statement is evaluated independently, starting from the state after the last line shown)?

Python

```

1 class C:
2     items = []
3     count = 0
4     def __init__(self, v):
5         self.items.append(v)
6         self.count += 1
7
8 a = C(1)
9 b = C(2)

```

- (A) `C.count == 0` evaluates to True
- (B) After executing `a.items = [9]`, the expression `len(b.items) == 2` is True
- (C) `a.count == 2` evaluates to True
- (D) `a.items is b.items` evaluates to True

Q.5 [NAT · numerical answer | 1 mark | Queues — queue using two stacks]

A queue is implemented with two stacks IN and OUT. `enqueue(x)` pushes `x` onto IN. `dequeue()` pops from OUT; if OUT is empty it first pops **every** element of IN and pushes it onto OUT (each such pop-push pair is one *transfer*).

Starting empty, the operations are:

enqueue 1, 2, 3, 4, 5; dequeue; dequeue; enqueue 6, 7; dequeue $\times 4$.

The total number of transfers performed is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | BST — preorder to postorder]

The pre-order traversal of a binary search tree with distinct keys is 52, 31, 17, 44, 38, 49, 70, 63, 88, 91.

What is its post-order traversal?

- (A) 17, 38, 49, 44, 31, 63, 88, 91, 70, 52
- (B) 17, 31, 38, 44, 49, 52, 63, 70, 88, 91
- (C) 17, 38, 49, 44, 31, 63, 91, 88, 70, 52
- (D) 17, 49, 38, 44, 31, 63, 91, 88, 70, 52

Q.7 [NAT · numerical answer | 1 mark | Selection sort — swaps]

Selection sort (ascending) is applied to [8, 21, 5, 34, 40, 13, 55, 47]. In pass i ($i = 0, 1, \dots, n-2$) it finds the index m of the minimum of $a[i..n-1]$ and swaps $a[i]$ with $a[m]$ **only if $m \neq i$** . The number of swaps performed is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Python — exception hierarchy and clause order]

Consider the following Python program. What is printed?

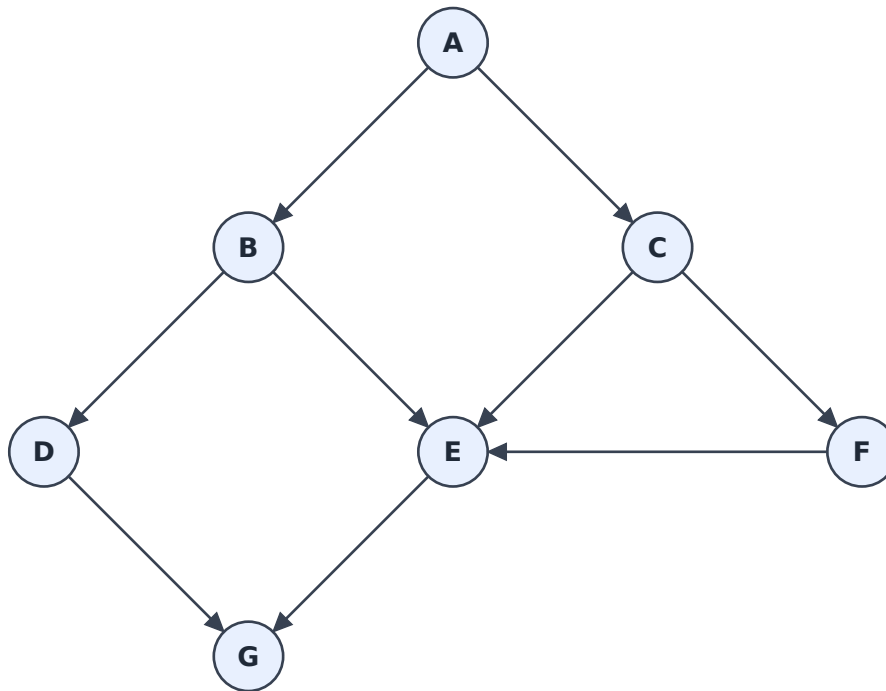
Python

```
1 log = []
2
3 def f(d, k):
4     try:
5         d[k] += 1
6     except LookupError:
7         return 'L'
8     except KeyError:
9         return 'K'
10    else:
11        return 'E'
12    finally:
13        log.append(k)
14
15 r = f({'a': 1}, 'a') + f({'a': 1}, 'b') + f([], 0)
16 print(r, len(log))
```

- (A) EKL 3
- (B) ELL 3
- (C) ELL 2
- (D) EKL 2

Q.9 [MSQ · one or more correct | 1 mark | Graph traversal — DFS orders]

Recursive depth-first search is started at A on the directed graph shown. The out-neighbours of a vertex may be explored in **any** order. Which of the following is/are possible orders in which vertices are first visited?



- (A) A, B, D, E, G, C, F
- (B) A, B, D, G, E, C, F
- (C) A, C, F, E, G, B, D
- (D) A, C, E, F, G, B, D

Q.10 [NAT · numerical answer | 1 mark | Binary search — comparison distribution]

Iterative binary search with $\text{mid} = \lfloor (\text{lo} + \text{hi})/2 \rfloor$ is performed on a sorted array of 100 distinct keys (indices 0–99). Each iteration compares the key with $a[\text{mid}]$ once (a three-way comparison counts as one). Among the 100 keys present in the array, the number that are found after **exactly 7** iterations is _____.

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — nonlocal and closure state]

Consider the following Python program. What is printed?

Python

```

1 def make():
2     n = 0
3     def inc(k=1):
4         nonlocal n
5         n += k
6         return n
7     return inc
8
9 a = make()
10 b = make()
11 a(); a(5); b(2)
12 c = a
13 c(10)
14 print(a(), b())

```

- (A) 17 3
- (B) 7 3
- (C) 17 13
- (D) 1 1

Q.12 [MSQ · one or more correct | 2 marks | Python — inheritance, MRO and super()]

Consider the following class definitions. Which of the following statements is/are TRUE?

Python

```

1 class A:
2     def __init__(self):
3         self.v = 1
4     def f(self):
5         return self.v
6
7 class B(A):
8     def __init__(self):
9         super().__init__()
10        self.v += 10
11    def f(self):
12        return 2 * super().f()
13
14 class C(A):
15     def f(self):
16         return self.v + 100
17
18 class D(B, C):
19     pass

```

- (A) `D().f()` returns 222
- (B) The method resolution order of D is D, B, C, A, object
- (C) `D().v` equals 1, because C has no `__init__` and so A. `__init__` is called last and resets v
- (D) `B().f()` returns 22

Q.13 [MSQ · one or more correct | 2 marks | Python — finally with break/continue and loop-else]

Consider the following Python function. Which of the following statements is/are TRUE?

Python

```
1 def risky(n):
2     log = []
3     for i in range(n):
4         try:
5             if i % 3 == 2:
6                 raise KeyError(i)
7             if i == 4:
8                 break
9             log.append(i)
10        except KeyError:
11            log.append(-i)
12            continue
13        finally:
14            log.append('f')
15    else:
16        log.append('E')
17    return log
```

- (A) `len(risky(4))` is 9
- (B) `-2` appears in both `risky(4)` and `risky(6)`
- (C) `risky(6).count('f')` is 5
- (D) `risky(6)` ends with 'E'

Q.14 [NAT · numerical answer | 2 marks | Python — decorators, closures and memoised recursion]

Consider the following Python program. The value printed is _____.

Python

```
1 from functools import lru_cache
2
3 def counted(f):
4     def w(*a):
5         w.calls += 1
6         return f(*a)
7     w.calls = 0
8     return w
9
10 @counted
11 @lru_cache(maxsize=None)
12 def fib(n):
13     return n if n < 2 else fib(n - 1) + fib(n - 2)
14
15 fib(7)
16 fib(5)
17 print(fib.calls)
```

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Hashing — mutating a key after insertion]

Consider the following Python program. What is printed?

Python

```
1 class K:
2     def __init__(self, v):
3         self.v = v
4     def __hash__(self):
5         return hash(self.v)
6     def __eq__(self, o):
7         return isinstance(o, K) and self.v == o.v
8
9 k = K(1)
10 d = {k: 'x'}
11 k.v = 2
12 print(k in d, K(1) in d, K(2) in d, len(d))
```

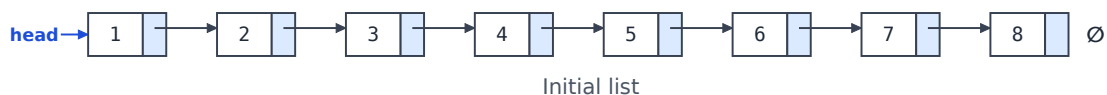
- (A) True False True 1
- (B) False True False 1
- (C) True False False 1
- (D) False False False 1

Q.16 [MCQ · single correct | 2 marks | Linked lists — reversal in groups]

The function `rev_k` below is applied to the singly linked list shown with $k = 3$. What list is printed?

Python

```
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def rev_k(h, k):
6     prev, cur, n = None, h, 0
7     while cur and n < k:
8         cur.nxt, prev, cur = prev, cur, cur.nxt
9         n += 1
10    if cur:
11        h.nxt = rev_k(cur, k)
12    return prev
13
14 head = None
15 for x in range(8, 0, -1):
16     head = Node(x, head)
17 head = rev_k(head, 3)
18 out = []
19 while head:
20     out.append(head.v)
21     head = head.nxt
22 print(out)
```



- (A) [3, 2, 1, 6, 5, 4, 7, 8]
- (B) [3, 2, 1, 6, 5, 4, 8, 7]
- (C) [8, 7, 6, 5, 4, 3, 2, 1]
- (D) [3, 2, 1]

Q.17 [NAT · numerical answer | 2 marks | Quicksort — comparison count]

The following Python quicksort uses the first element as pivot and counts one comparison for every element compared with the pivot. The value printed is _____.

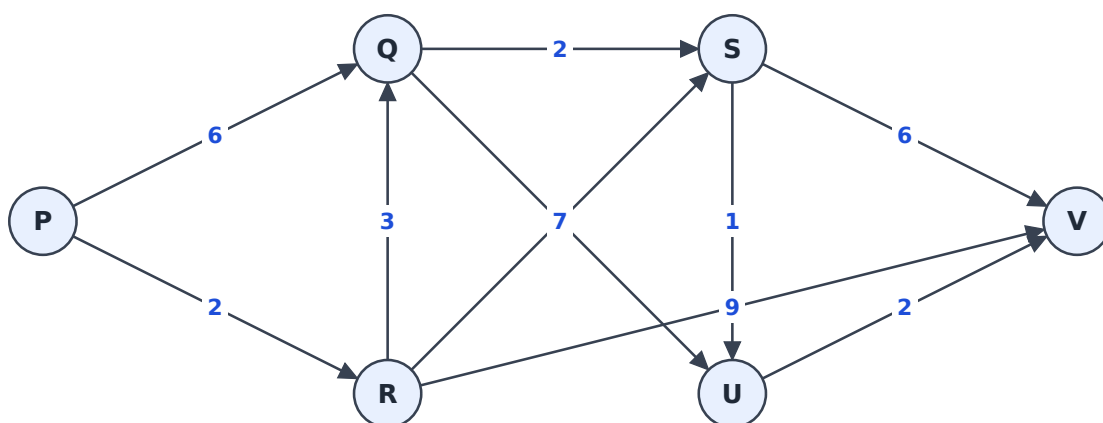
Python

```
1 comps = 0
2
3 def qs(a):
4     global comps
5     if len(a) < 2:
6         return a
7     p = a[0]
8     comps += len(a) - 1
9     left = [x for x in a[1:] if x < p]
10    right = [x for x in a[1:] if x >= p]
11    return qs(left) + [p] + qs(right)
12
13 qs([50, 23, 9, 18, 61, 32, 75, 54, 41])
14 print(comps)
```

Answer: _____ (enter a numerical value)

Q.18 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from source P on the weighted directed graph shown. Which of the following statements is/are TRUE?



- (A) If the weight of edge P→Q were reduced from 6 to 4, the shortest distance to U would decrease by exactly 1
- (B) Vertices are extracted from the priority queue in the order P, R, Q, S, U, V
- (C) In the shortest-path tree, the parent of V is R
- (D) The shortest distance from P to V is 10

Q.19 [MCQ · single correct | 2 marks | Binary trees in Python — mutable default argument]

The following program builds the binary tree shown and collects its leaves. What is printed?

Python

```
1 class Node:
2     def __init__(self, v, l=None, r=None):
3         self.v, self.l, self.r = v, l, r
4     def leaves(self, acc=[]):
5         if self.l is None and self.r is None:
6             acc.append(self.v)
7         for c in (self.l, self.r):
8             if c:
9                 c.leaves(acc)
10        return acc
11
12 t = Node(1, Node(2, Node(4), Node(5, Node(8))),
13         Node(3, None, Node(6, Node(7), Node(9))))
14 x = t.leaves()
15 y = t.l.leaves()
16 print(len(x), len(y), x is y)
```

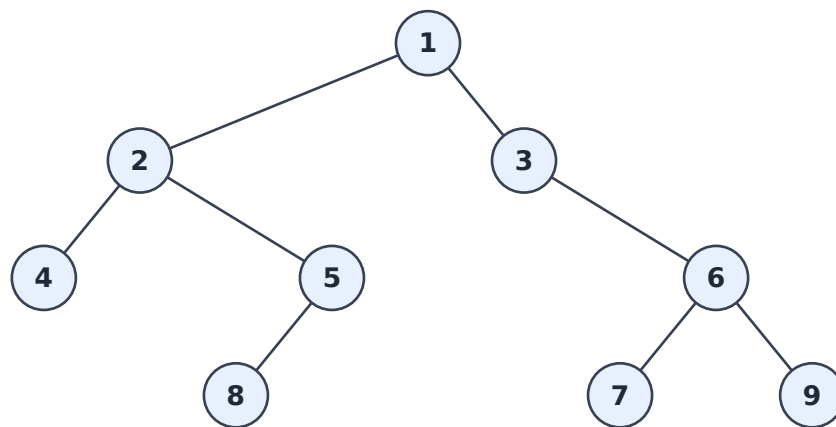
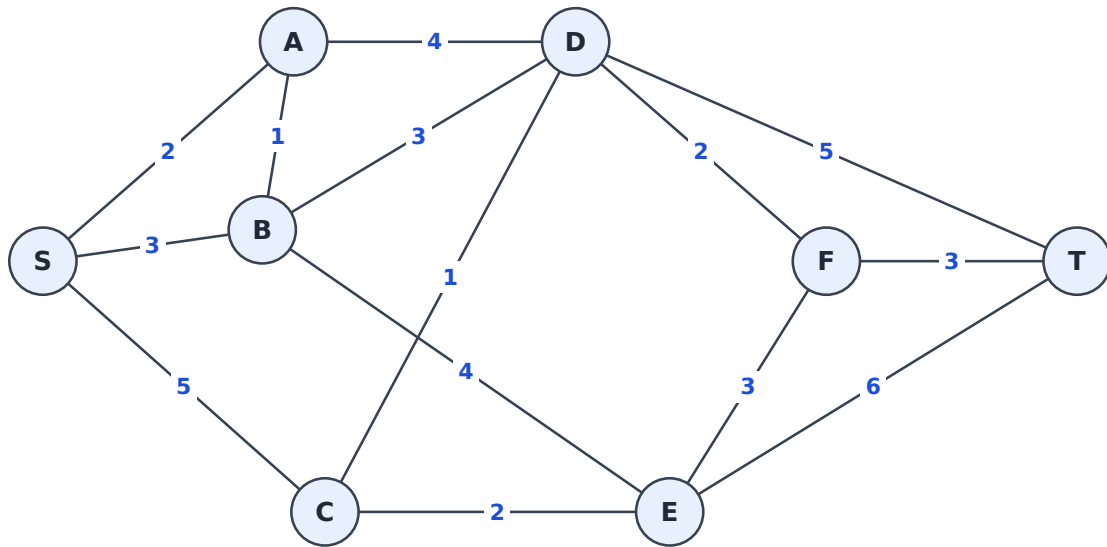


Figure: tree t

- (A) 4 2 False
- (B) 4 6 True
- (C) 6 6 True
- (D) 6 2 True

Q.20 [NAT · numerical answer | 2 marks | Shortest paths — counting shortest paths]

In the weighted undirected graph shown, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Answer Key — Mock Test 18

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — closures and enclosing scope
2	NAT	1	50	Python — late binding in lambdas
3	MCQ	1	(D)	Python — try / except / finally with return
4	MSQ	1	(A), (B), (D)	Python — class vs instance attributes
5	NAT	1	7	Queues — queue using two stacks
6	MCQ	1	(C)	BST — preorder to postorder
7	NAT	1	6	Selection sort — swaps
8	MCQ	1	(B)	Python — exception hierarchy and clause order
9	MSQ	1	(B), (C)	Graph traversal — DFS orders
10	NAT	1	37	Binary search — comparison distribution
11	MCQ	2	(A)	Python — nonlocal and closure state
12	MSQ	2	(A), (B), (D)	Python — inheritance, MRO and super()
13	MSQ	2	(A), (B), (C)	Python — finally with break/continue and loop-else
14	NAT	2	14	Python — decorators, closures and memoised recursion
15	MCQ	2	(D)	Hashing — mutating a key after insertion
16	MCQ	2	(B)	Linked lists — reversal in groups
17	NAT	2	16	Quicksort — comparison count
18	MSQ	2	(A), (B), (D)	Shortest paths — Dijkstra
19	MCQ	2	(C)	Binary trees in Python — mutable default argument
20	NAT	2	8	Shortest paths — counting shortest paths

Detailed Solutions — Mock Test 18

Q.1 [MCQ · 1 mark · Python — closures and enclosing scope]

Answer: (A)

Name resolution follows **LEGB** (Local, Enclosing, Global, Built-in), and a closure captures the enclosing *variable* (a cell), not the value it had when `inner` was defined.

- Inside `outer`, `x` is local because it is assigned there. `inner` does not assign `x`, so its `x` refers to `outer`'s cell.
- After `def inner`, `outer` rebinds that cell to `'e2'` and returns `inner`.
- Calling `f()` reads the cell at call time → `'e2'`.
- The module-level `x` was never touched → `'g'`.

Output `e2 g` → option **(A)**.

- (B) assumes the value is frozen at definition time (*early binding*).
- (C) assumes `inner` skips the enclosing scope and reads the global.
- (D) assumes assignments in `outer` leak into the global scope (they would only with a `global x` declaration).

Tip: closures bind late; to freeze a value use a default argument (`def inner(x=x)`).

Q.2 [NAT · 1 mark · Python — late binding in lambdas]

Answer: 50

All lambdas in `fs` close over the **same** variable `i` of the comprehension's scope. They are called only after the comprehension has finished, when `i = 3`.

- Each `f()` returns $3 \times 3 = 9$; four of them sum to **36**.
- In `gs`, `i=i` is a default parameter, evaluated **when each lambda is created**, so the lambdas remember 0, 1, 2, 3 → $0 + 1 + 4 + 9 = 14$.

Printed value: $36 + 14 = 50$.

Trap: answering 28 ($14 + 14$) by assuming the loop variable is captured per iteration. In Python a closure stores a reference to the variable's cell; only default-argument values are evaluated eagerly. (Note: the comprehension variable does not leak to module scope in Python 3, but it still lives on in the closures' cell.)

Q.3 [MCQ · 1 mark · Python — try / except / finally with return]

Answer: (D)

A `finally` block always runs when control leaves the `try` statement — even via `return` — and a `return` inside `finally` **overrides** any pending return value or exception.

Evaluation of `print(f(), g())`:

- The arguments are evaluated first. `f()`: `return 'try'` is pending; `finally` prints `fin` immediately; then `'try'` is returned.
- `g()`: `ValueError` is caught, `return 1` is pending; `finally` executes `return 2`, which replaces it → 2.
- Only now does the outer `print` run, writing `try 2`.

Full output: `fin try 2` → option **(D)**.

- (A) assumes `print` emits `f()`'s value before `f`'s own side effect.
- (B) gets both the order and the value wrong.
- (C) forgets that `return` in `finally` wins.

Trap: side effects inside argument evaluation happen *before* the enclosing call prints anything.

Q.4 [MSQ · 1 mark · Python — class vs instance attributes]**Answer: (A), (B), (D)**

Attribute *lookup* searches the instance, then the class; attribute *assignment* (`self.x = ...`, including augmented `self.x += ...`) always writes to the instance.

- `self.items.append(v)` looks up `items` (found on the class) and **mutates** the one shared list → `C.items == [1, 2]`.
- `self.count += 1` means `self.count = self.count + 1`: it reads `C.count` (0) and **creates an instance attribute** `count = 1`. The class attribute stays 0.

Statements:

- (A) The class's `count` was never reassigned. **True.**
- (B) `a.items = [9]` creates an instance attribute on `a` only; `b.items` is still the class list `[1, 2]`. **True.**
- (C) Each instance has its own `count == 1`. **False.**
- (D) Both names resolve to the class list. **True.**

Trap: treating `+=` on an immutable class attribute like `append` on a mutable one — the former rebinds on the instance, the latter mutates the shared object.

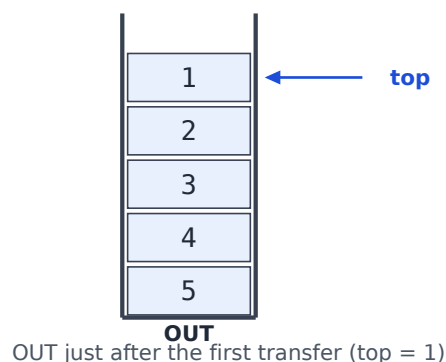
Q.5 [NAT · 1 mark · Queues — queue using two stacks]**Answer: 7**

Each element is transferred at most once (IN → OUT), which is why the two-stack queue has $O(1)$ amortised cost per operation.

- enqueue 1–5: IN = [1, 2, 3, 4, 5] (bottom→top), OUT empty.
- dequeue #1: OUT empty → transfer 5 elements → OUT = [5, 4, 3, 2, 1]; pop 1. Transfers = **5**.
- dequeue #2: pop 2 from OUT (no transfer).
- enqueue 6, 7: IN = [6, 7].
- dequeue #3, #4, #5: pop 3, 4, 5 from OUT (OUT becomes empty).
- dequeue #6: OUT empty → transfer 7 then 6 → OUT = [7, 6]; pop 6. Transfers += **2**.

Total transfers = 5 + 2 = **7**; the dequeued sequence 1, 2, 3, 4, 5, 6 is correct FIFO order.

Trap: transferring on *every* dequeue (or moving elements back to IN after each dequeue) gives a much larger count and is the naive $O(n)$ -per-operation design.

**Q.6 [MCQ · 1 mark · BST — preorder to postorder]****Answer: (C)**

In a BST, the pre-order sequence determines the tree: the first key is the root, the following keys smaller than it form the left subtree's pre-order, the rest the right subtree's.

- Root 52; left part 31, 17, 44, 38, 49; right part 70, 63, 88, 91.
- Left subtree: root 31, left {17}, right 44 with children 38 and 49.

- Right subtree: root 70, left {63}, right 88 whose right child is 91.

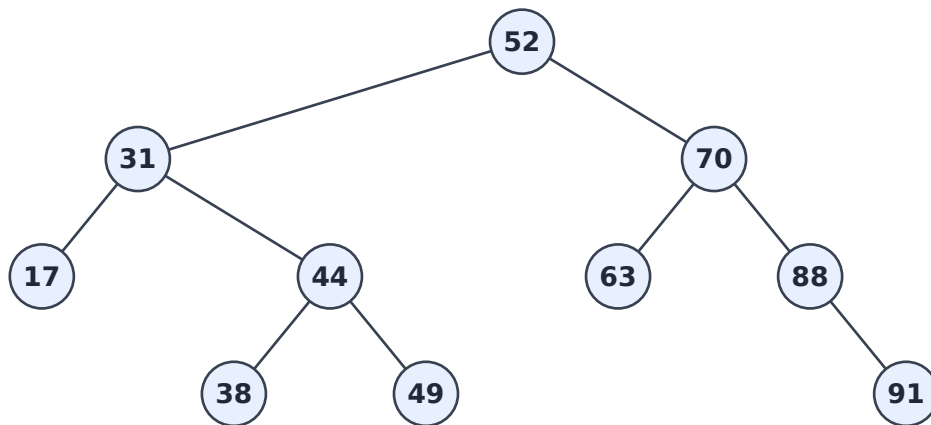
Post-order (left, right, root):

- left subtree: 17, 38, 49, 44, 31
- right subtree: 63, 91, 88, 70
- root: 52

→ 17, 38, 49, 44, 31, 63, 91, 88, 70, 52 → option **(C)**.

- (A) visits 88 before its child 91 — 91 is the *right child* of 88, so it must come first.
- (B) is the in-order (sorted) sequence.
- (D) swaps 38 and 49, i.e. visits the right child of 44 before the left.

Tip: post-order always ends with the root, and every node appears after all its descendants.



BST reconstructed from the pre-order

Q.7 [NAT · 1 mark · Selection sort — swaps]

Answer: 6

Selection sort makes at most $n - 1 = 7$ swaps; a pass is skipped when the minimum of the unsorted suffix is already in place.

- $i=0$: min 5 at 2 → swap → [5, 21, 8, 34, 40, 13, 55, 47]
- $i=1$: min 8 at 2 → swap → [5, 8, 21, 34, 40, 13, 55, 47]
- $i=2$: min 13 at 5 → swap → [5, 8, 13, 34, 40, 21, 55, 47]
- $i=3$: min 21 at 5 → swap → [5, 8, 13, 21, 40, 34, 55, 47]
- $i=4$: min 34 at 5 → swap → [5, 8, 13, 21, 34, 40, 55, 47]
- $i=5$: min 40 at 5 → **no swap**
- $i=6$: min 47 at 7 → swap → sorted.

Total = **6** swaps.

Trap: answering $n - 1 = 7$ automatically. Note how 40 reached its final place as a side-effect of the $i = 4$ swap. The number of *comparisons*, by contrast, is always $n(n-1)/2 = 28$, independent of the input.

Array after each pass									
	0	1	2	3	4	5	6	7	swap?
start	8	21	5	34	40	13	55	47	-
i=0	5	21	8	34	40	13	55	47	yes
i=1	5	8	21	34	40	13	55	47	yes
i=2	5	8	13	34	40	21	55	47	yes
i=3	5	8	13	21	40	34	55	47	yes
i=4	5	8	13	21	34	40	55	47	yes
i=5	5	8	13	21	34	40	55	47	no
i=6	5	8	13	21	34	40	47	55	yes

Q.8 [MCQ · 1 mark · Python — exception hierarchy and clause order]**Answer: (B)**

`except` clauses are tried **top to bottom** and the first one whose class matches (including base classes) wins. `KeyError` and `IndexError` are both subclasses of `LookupError`.

- `f({'a':1}, 'a')`: no exception → `else` returns 'E'; `finally` logs.
- `f({'a':1}, 'b')`: `d['b']` raises `KeyError` → caught by the *first* clause `except LookupError` → 'L'. The `except KeyError` clause is unreachable.
- `f([], 0)`: `[][0]` raises `IndexError` → also a `LookupError` → 'L'.
- `finally` runs in every call → `log` has 3 entries.

`r = 'E' + 'L' + 'L' → output ELL 3 → option (B).`

- (A)/(D) assume the more specific `KeyError` clause is preferred — Python does not look for the best match, only the first.
- (C)/(D) assume `finally` is skipped when `return` executes in `except/else`.

Tip: always order `except` clauses from most specific to most general.

Q.9 [MSQ · 1 mark · Graph traversal — DFS orders]**Answer: (B), (C)**

In DFS, after visiting `v` the search must explore **all** vertices reachable from `v` through unvisited vertices before backtracking. So in a valid order, the vertex after `v` must be an unvisited out-neighbour of `v` if one exists; otherwise of the nearest ancestor that still has one.

- (A) After D, its out-neighbour G is still unvisited, so G must be next; jumping to E is **impossible**.
- (B) `A→B→D→G` (G has no out-edges; D done) → back to B → E (G already seen) → back to A → C → F (E seen). **Possible**.
- (C) `A→C→F→E→G` → back to C (E seen) → back to A → B → D (G seen). **Possible**.
- (D) After E, G is an unvisited out-neighbour of E, so G must come before backtracking to C for F. **Impossible**.

Trap: treating DFS like 'any order that respects edges'. The rule is stricter: you can only move to a neighbour of the *current* vertex or backtrack.

Q.10 [NAT · 1 mark · Binary search — comparison distribution]**Answer: 37**

The probes of binary search form a *decision tree*: the root is the first mid, its children are the mids of the two halves, and so on. A key at depth `d` (root depth 1) is found after exactly `d` iterations.

The decision tree for n keys is as balanced as possible (half sizes differ by at most one), so levels fill completely from the top:

- depth 1: 1 key, depth 2: 2, depth 3: 4, depth 4: 8, depth 5: 16, depth 6: 32 — that is 63 keys in total ($2^6 - 1$).
- The remaining $100 - 63 = 37$ keys sit at depth 7 (which could hold up to 64).

So 37 keys need exactly 7 iterations, and no key needs more ($\lceil \log_2 100 \rceil + 1 = 7$).

Trap: answering 64 (capacity of level 7) or 36 (miscounting the first six levels as 64 keys).

Q.11 [MCQ · 2 marks · Python — nonlocal and closure state]

Answer: (A)

Each call of `make()` creates a **new** frame and hence a new cell for `n`; `inc` closes over that cell and `nonlocal n` lets it rebind it. Assigning `c = a` copies the *reference* to the same function object, not its state.

Trace of the two independent counters:

- `a()` → a's `n = 1`
- `a(5)` → a's `n = 6`
- `b(2)` → b's `n = 2`
- `c(10)` → `c` is `a` → a's `n = 16`
- `a()` → a's `n = 17` (printed first)
- `b()` → b's `n = 3` (printed second)

Output 17 3 → option **(A)**.

- (B) treats `c` as an independent copy, so `c(10)` would not affect `a`.
- (C) assumes `a` and `b` share one counter (as they would with a module-level `global n`).
- (D) assumes every call starts again from `n = 0`, as if `n` were a local of `inc` (in fact, without `nonlocal`, `n += k` would raise `UnboundLocalError`).

Tip: function objects are ordinary objects — aliasing them aliases their closure cells too.

Q.12 [MSQ · 2 marks · Python — inheritance, MRO and super()]

Answer: (A), (B), (D)

Python linearises the class hierarchy with **C3**; `super()` means 'the next class after the current one in the MRO of the *instance's* class', not 'my parent'.

- (B) `D(B, C)` with `B(A)`, `C(A)`: MRO = `D, B, C, A`, object. **True.**
- (A) `D()` runs `B.__init__` (first in MRO with `__init__`). Its `super()` → next after `B` in `D`'s MRO is `C`; `C` has no `__init__`, so lookup continues to `A` → `v = 1`. Back in `B`: `v += 10` → `v = 11`. `D().f()` → `B.f` → `2 * super().f()`; super of `B` in `D`'s MRO is `C`, so `C.f` returns `11 + 100 = 111` → result 222. **True.**
- (C) As traced, `A.__init__` runs *inside* `B.__init__`, before `+= 10`; `v = 11`. **False.**
- (D) For a plain `B()`, the MRO is `B, A`, object: `v = 1 + 10 = 11`, `B.f` → `2 × A.f()` = 22. **True.**

Trap: assuming `super().f()` inside `B` always calls `A.f`. In a diamond, the same line of code dispatches to `C.f` for a `D` instance but to `A.f` for a `B` instance — so `D().f()` is 222, not 22.

Q.13 [MSQ · 2 marks · Python — finally with break/continue and loop-else]

Answer: (A), (B), (C)

Key rules: (1) **finally** runs whenever control leaves the **try** — normally, via **continue**, via **break**, or via an exception; (2) a loop's **else** runs only if the loop ends **without break**.

`risky(4)` (`i = 0..3`):

- `i=0`: append 0, finally 'f'

- `i=1`: append 1, 'f'
 - `i=2`: `KeyError` → append -2, `continue` → finally 'f' still runs
 - `i=3`: append 3, 'f'
 - loop exhausted → else appends 'E'.
- [0, 'f', 1, 'f', -2, 'f', 3, 'f', 'E'] — 9 items.

`risky(6)`: same for `i = 0..3`, then `i=4`: `break` → finally 'f' → loop ends by break, so **no** 'E'.
 → [0, 'f', 1, 'f', -2, 'f', 3, 'f', 'f'] — five 'f'.

- (A) 9 items. **True.**
- (B) -2 is logged at `i = 2` in both calls. **True.**
- (C) five 'f' entries (`i = 0..4`). **True.**
- (D) ends with 'f', not 'E'. **False.**

Trap: thinking `break/continue` skip `finally`, or that loop-`else` means 'runs if the loop body never executed'.

Q.14 [NAT · 2 marks · Python — decorators, closures and memoised recursion]

Answer: 14

Decorators apply bottom-up: `fib = counted(lru_cache(fib_original))`. The global name `fib` is bound to the **counting wrapper**, and the recursive calls inside the body look up the global `fib` at run time — so every call, including recursive ones and cache hits, is counted.

`fib(7)` (cache initially empty):

- each value `n = 7, 6, 5, 4, 3, 2` is computed once (a cache miss) and makes exactly two calls, `fib(n-1)` and `fib(n-2)` → $6 \times 2 = 12$ calls;
- `fib(1)` and `fib(0)` are base cases reached via those calls (already counted);
- plus the outer call `fib(7)` itself → $12 + 1 = 13$.

`fib(5)`: one counted call, answered from the cache → **1**.

Total = $13 + 1 = 14$.

Trap: without memoisation `fib(7)` alone would make 41 calls; and if the decorators were in the other order (`lru_cache` outermost), cache hits would never reach the counter.

Q.15 [MCQ · 2 marks · Hashing — mutating a key after insertion]

Answer: (D)

A dict stores each entry in the slot determined by the key's hash **at insertion time**, together with that hash value. A lookup computes the probe key's *current* hash, goes to that slot, and calls `__eq__` only on entries whose stored hash matches.

The entry was stored with `hash(1)`. Then `k.v = 2` changes `k`'s hash to `hash(2)` but the table is not updated.

- `k in d`: probes using `hash(2)` → no entry with stored hash 2 → **False** (even though it is the very same object).
- `K(1) in d`: `hash(1)` matches the stored hash; `__eq__` compares 1 with the stored key's *current* `v = 2` → **False**.
- `K(2) in d`: `hash(2)` → no matching stored hash → **False**.
- `len(d)` is still 1 — the entry exists but is unreachable.

Output `False False False 1` → option **(D)**.

- (A)/(C) assume `in` checks object identity first without hashing.
- (B) assumes `__eq__` compares against the old value 1.

Tip: this is why Python's built-in mutable containers (`list`, `dict`, `set`) are unhashable — keys must not change their hash while stored.

Q.16 [MCQ · 2 marks · Linked lists — reversal in groups]**Answer: (B)**

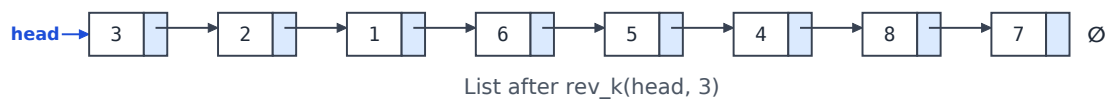
In the tuple assignment `cur.next, prev, cur = prev, cur, cur.next` the right-hand side is evaluated completely first, so it correctly reverses one link per iteration. After k steps, `prev` is the new head of the reversed group and the old head `h` is its tail.

- Group 1: 1, 2, 3 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 1; `cur = 4` is not None, so `1.next = rev_k(4, 3)`.
- Group 2: 4, 5, 6 $\rightarrow$ 6 $\rightarrow$ 5 $\rightarrow$ 4; `cur = 7  $\rightarrow$  4.next = rev_k(7, 3)`.
- Group 3: 7, 8 — the loop stops when `cur` becomes None after 2 steps ($n = 2 < k$), so the short group is **also reversed** $\rightarrow$ 8 $\rightarrow$ 7; `cur` is None so `7.next` stays None (it was set to None when 7 became the group tail).

Result: 3, 2, 1, 6, 5, 4, 8, 7 $\rightarrow$ option **(B)**.

- (A) assumes a final group shorter than k is left untouched — this code has no such check.
- (C) is a full reversal.
- (D) assumes the remainder is lost (it would be, without `h.next = ...`).

Trap: the list is built by prepending 8, 7, ..., 1, so it really starts 1 $\rightarrow$ 2 $\rightarrow$...

**Q.17 [NAT · 2 marks · Quicksort — comparison count]****Answer: 16**

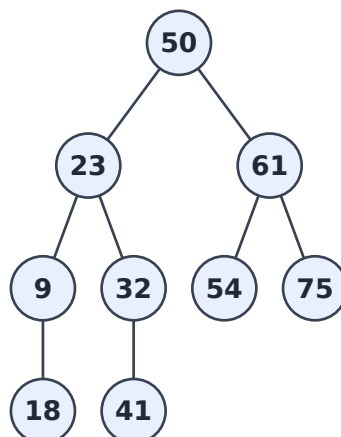
Each call on a sub-list of length $L \geq 2$ adds $L - 1$ comparisons. The comprehensions keep the original relative order, so the next pivots are easy to read off.

- `qs([50, 23, 9, 18, 61, 32, 75, 54, 41])`: $L = 9 \rightarrow$ **8**; left = [23, 9, 18, 32, 41], right = [61, 75, 54].
- `qs([23, 9, 18, 32, 41])`: **4**; left = [9, 18], right = [32, 41].
- `qs([9, 18])`: **1**; left = [], right = [18].
- `qs([32, 41])`: **1**; right = [41].
- `qs([61, 75, 54])`: **2**; left = [54], right = [75].
- All remaining calls have length $\leq 1 \rightarrow 0$.

Total = $8 + 4 + 1 + 1 + 2 =$ **16**.

Compare: the worst case for $n = 9$ is $9 \cdot 8 / 2 = 36$ (already sorted input); the best is about $n \log_2 n - n$.

Trap: `global comps` is required — without it, `comps += ...` makes `comps` local and raises `UnboundLocalError`. Also, each partition step counts as $L - 1$ comparisons even though the code scans the list twice.



Pivot recursion tree (each pivot's call costs size - 1)

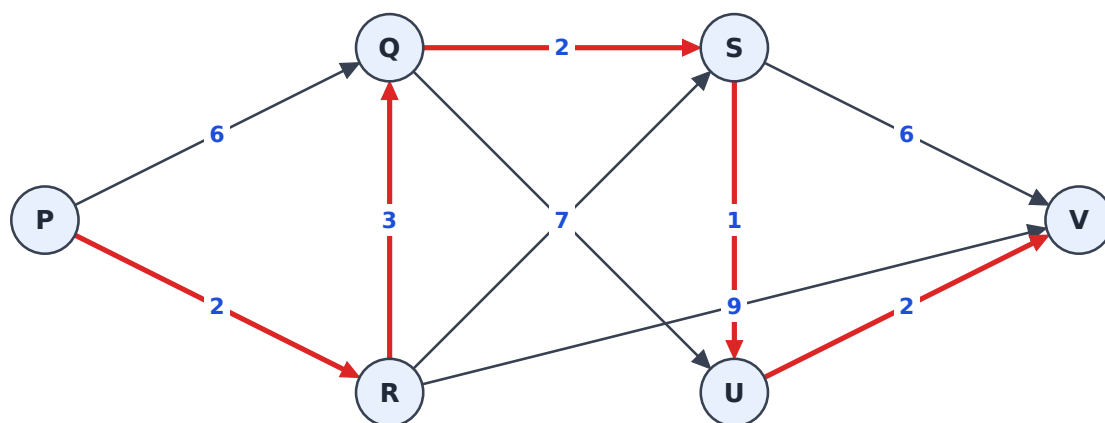
Q.18 [MSQ · 2 marks · Shortest paths — Dijkstra]

Answer: (A), (B), (D)

Dijkstra settles vertices in non-decreasing order of distance.

- Extract P (0): $Q = 6$, $R = 2$.
- Extract R (2): $Q = \min(6, 5) = 5$; $S = 10$; $V = 11$.
- Extract Q (5): $S = \min(10, 7) = 7$; $U = 12$.
- Extract S (7): $U = \min(12, 8) = 8$; $V = \min(11, 13) = 11$.
- Extract U (8): $V = \min(11, 10) = 10$.
- Extract V (10).
- (A) With $P \rightarrow Q = 4$: $Q = \min(4, 5) = 4$, $S = 6$, $U = 7$ (was 8) $\rightarrow$ decrease of exactly 1. **True.**
- (B) Extraction order P, R, Q, S, U, V. **True.**
- (C) V's final distance comes from U ($8 + 2$), so its parent is **U**; the edge $R \rightarrow V$ gave only the temporary value 11. **False.**
- (D) $d(V) = 10$ via $P \rightarrow R \rightarrow Q \rightarrow S \rightarrow U \rightarrow V$. **True.**

Trap: in (A) it is tempting to say 'decrease by 2' because the edge got 2 cheaper — but the old best route to Q (via R, cost 5) was already 1 cheaper than the direct edge, so the saving is only 1.



Shortest-path tree from P

Q.19 [MCQ · 2 marks · Binary trees in Python — mutable default argument]

Answer: (C)

The default `acc=[]` is created **once**, when `def leaves` runs; every call that omits `acc` shares that single list. Recursive calls pass `acc` explicitly, so within one top-level call everything goes into the same list.

- `x = t.leaves()`: pre-order walk appends leaves 4, 8, 7, 9 to the shared default list $\rightarrow$ it now holds 4 items, and `x` refers to it.
- `y = t.l.leaves()`: again uses the *same* default list and appends the leaves of the subtree rooted at 2: 4, 8 $\rightarrow$ the list now holds [4, 8, 7, 9, 4, 8].
- `x` and `y` are the same object, evaluated at print time $\rightarrow \text{len}(x) = \text{len}(y) = 6$, `x is y` $\rightarrow$ True.

Output `6 6 True` $\rightarrow$ option **(C)**.

- (A) is what a fresh list per call would give.
- (B) assumes `len(x)` was frozen when `x` was assigned — `x` is a reference.
- (D) mixes both errors.

Tip: use `acc=None` and `if acc is None: acc = []` inside the function.

Q.20 [NAT · 2 marks · Shortest paths — counting shortest paths]**Answer: 8**

Run Dijkstra and keep, for each vertex v , the number of shortest paths $N(v)$. When an edge (u, v) gives a strictly smaller distance, set $N(v) = N(u)$; when it gives an equal distance, add $N(u)$.

- S: $d = 0$, $N = 1$.
- A: $d = 2$ (S-A), $N = 1$.
- B: S-B = 3 and S-A-B = $2 + 1 = 3 \rightarrow d = 3$, $N = 1 + 1 = 2$.
- C: S-C = 5 (via D would be 7) $\rightarrow d = 5$, $N = 1$.
- D: via A $2 + 4 = 6$, via B $3 + 3 = 6$, via C $5 + 1 = 6 \rightarrow d = 6$, $N = 1 + 2 + 1 = 4$.
- E: via B $3 + 4 = 7$, via C $5 + 2 = 7 \rightarrow d = 7$, $N = 2 + 1 = 3$.
- F: via D $6 + 2 = 8$, via E $7 + 3 = 10 \rightarrow d = 8$, $N = 4$.
- T: via D $6 + 5 = 11$, via F $8 + 3 = 11$, via E $7 + 6 = 13 \rightarrow d = 11$, $N = N(D) + N(F) = 4 + 4 = 8$.

Trap: stopping at the first shortest path found, or adding counts through E (whose route to T is longer). Paths through F all pass through D, but they are still distinct paths from those using D-T directly.

Distance d and path count N

	S	A	B	C	D	E	F	T
d	0	2	3	5	6	7	8	11
N	1	1	2	1	4	3	4	8

Mock Test 19

Complexity Analysis

Difficulty: GATE-level

Focus areas: loop-nest complexity, asymptotic comparison, recurrence solving

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — floor division and modulo with negatives]

Consider the following Python program. What is printed?

Python

```
1 print(-17 // 5, -17 % 5, 17 // -5, int(-17 / 5))
```

- (A) -3 -2 -3 -3
- (B) -4 3 -3 -3
- (C) -4 3 -4 -3
- (D) -4 -2 -4 -4

Q.2 [NAT · numerical answer | 1 mark | Loop counting — geometric loops]

Consider the following Python program. The value printed is _____.

Python

```
1 count = 0
2 i = 1
3 while i < 1000:
4     j = i
5     while j < 1000:
6         count += 1
7         j *= 2
8     i *= 3
9 print(count)
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Asymptotic comparison of functions]

Consider $f_1(n) = 2^{\sqrt{\log_2 n}}$, $f_2(n) = n^{1/3}$, $f_3(n) = (\log_2 n)^{10}$ and $f_4(n) = n / \log_2 n$. Which of the following arranges them in increasing order of asymptotic growth?

- (A) $f_3 < f_1 < f_2 < f_4$
- (B) $f_3 < f_2 < f_1 < f_4$
- (C) $f_1 < f_3 < f_2 < f_4$
- (D) $f_1 < f_3 < f_4 < f_2$

Q.4 [MSQ · one or more correct | 1 mark | Recurrences — identifying $\Theta(n \log n)$]

For each recurrence below, $T(n) = \Theta(1)$ for $n \leq 2$. Which of the following recurrences has/have the solution $T(n) = \Theta(n \log n)$?

- (A) $T(n) = 2T(n/2) + 5n$
- (B) $T(n) = T(n/3) + T(2n/3) + n$
- (C) $T(n) = 3T(n/3) + n / \log n$
- (D) $T(n) = 4T(n/2) + n \log n$

Q.5 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with a stack (^ denotes exponentiation; all operators are binary; for an operator the top of the stack is the **right** operand):

6 2 3 + * 4 2 ^ 7 - 3 / -

The value of the expression is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Complexity of a halving loop nest]

What is the time complexity of the following function, as a function of $n \geq 1$?

Python

```
1 def f(n):
2     s = 0
3     i = n
4     while i > 0:
5         for j in range(i):
6             s += 1
7         i //= 2
8     return s
```

- (A) $\Theta(n^2)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(\log^2 n)$
- (D) $\Theta(n)$

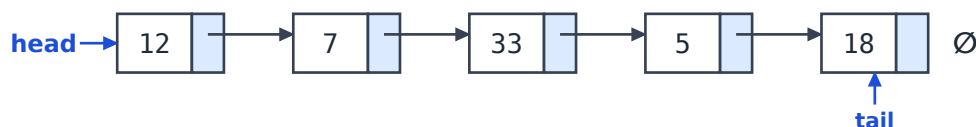
Q.7 [MCQ · single correct | 1 mark | Hashing — linear probing cost]

The keys 42, 17, 64, 31, 75, 53, 20, 86 are inserted, in this order, into an initially empty hash table with 11 slots using $h(k) = k \bmod 11$ and linear probing ($h(k), h(k)+1, \dots \bmod 11$). Counting every slot examined (including the slot finally used) as one probe, the total number of probes for all 8 insertions is

- (A) 22
- (B) 29
- (C) 36
- (D) 28

Q.8 [MCQ · single correct | 1 mark | Linked lists — operation costs]

A singly linked list of n nodes keeps a **head** pointer and a **tail** pointer (as shown). Each node stores only its key and a **next** pointer, and no other auxiliary information is kept. Which of the following operations requires $\Theta(n)$ time in the worst case?



Singly linked list with head and tail pointers

- (A) Inserting a new node before the first node
- (B) Inserting a new node after the last node
- (C) Deleting the first node
- (D) Deleting the last node

Q.9 [NAT · numerical answer | 1 mark | Heaps — nodes by height]

A binary heap is stored in an array of $n = 1000$ elements (a complete binary tree). The *height* of a node is the number of edges on the longest downward path from it to a leaf (leaves have height 0). The number of nodes of height exactly 3 is _____.

Answer: _____ (enter a numerical value)

Q.10 [MSQ · one or more correct | 1 mark | Python — cost of built-in operations]

Let `data` be a list of n integers, many of which may be distinct. Which of the following fragments run(s) in $\Theta(n)$ expected (average-case) time for every such list?

Python

```

1 # (A)
2 s = set()
3 for x in data:
4     if x not in s:
5         s.add(x)
6 # (B)
7 out = []
8 for x in data:
9     out.insert(0, x)
10 # (C)
11 seen = []
12 for x in data:
13     if x not in seen:
14         seen.append(x)
15 # (D)
16 d = {}
17 for x in data:
18     d[x] = d.get(x, 0) + 1

```

- (A) Fragment (B)
- (B) Fragment (C)
- (C) Fragment (D)
- (D) Fragment (A)

Q.11 [MCQ · single correct | 2 marks | Recurrence from recursive code]

Consider the following Python function. Which of the following best describes its running time $T(n)$ for n a power of 2?

Python

```

1 def g(n):
2     if n <= 1:
3         return 1
4     s = 0
5     for i in range(n):
6         s += i
7     return g(n // 2) + g(n // 2) + g(n // 2) + s

```

- (A) $\Theta(n \log n)$
- (B) $\Theta(n^{\log_2 3})$
- (C) $\Theta(n^{1.5})$
- (D) $\Theta(n^2)$

Q.12 [NAT · numerical answer | 2 marks | Loop counting — triple nested dependent loops]

Consider the following Python program. The value printed is _____.

Python

```
1 n = 12
2 c = 0
3 for i in range(n):
4     for j in range(i, n):
5         for k in range(i, j):
6             c += 1
7 print(c)
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Asymptotic notation — true/false statements]

Which of the following statements is/are TRUE?

- (A) $(n + 1)! = O(n!)$
- (B) $2^{n+1} = O(2^n)$
- (C) $\log_2(n!) = \Theta(n \log_2 n)$
- (D) $n \log_2 n = O(n^{1.01})$

Q.14 [NAT · numerical answer | 2 marks | Recurrences — exact solution (merge sort worst case)]

The worst-case number of comparisons of top-down merge sort satisfies

$T(1) = 0$, $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n - 1$ for $n \geq 2$.

The value of $T(100)$ is _____.

Answer: _____ (enter a numerical value)

Q.15 [MSQ · one or more correct | 2 marks | Queue via two stacks — amortised cost]

A queue is implemented with two stacks IN and OUT. `enqueue(x)` pushes x on IN. `dequeue()` first checks OUT; **only if OUT is empty** it pops every element of IN and pushes it on OUT; then it pops OUT and returns that value. Each push and each pop counts as one stack operation (emptiness checks are free).

Starting empty, the following sequence is executed:

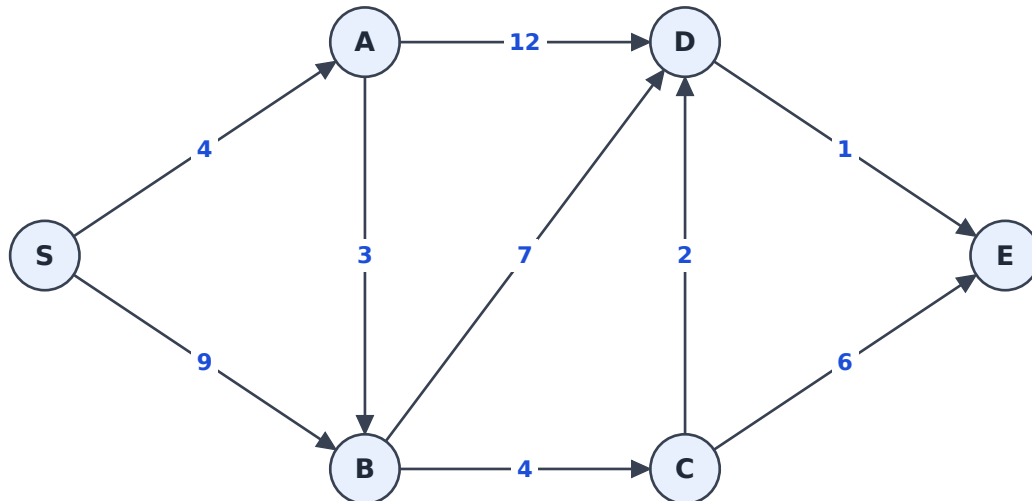
`enqueue 1, 2, 3, 4, 5; dequeue; dequeue; enqueue 6, 7; dequeue × 4; enqueue 8; dequeue.`

Which of the following statements is/are TRUE?

- (A) The last dequeue returns 8
- (B) The sequence performs 29 stack operations in total
- (C) The most expensive single dequeue performs 11 stack operations
- (D) For **any** sequence of m queue operations starting from an empty queue, at most $4m$ stack operations are performed

Q.16 [NAT · numerical answer | 2 marks | Dijkstra — counting distance updates]

Dijkstra's algorithm (with a priority queue supporting decrease-key) is run from source **S** on the directed graph below. An *update* is any assignment that strictly lowers a tentative distance $d[v]$, including the first assignment of a finite value to a vertex whose distance was ∞ ($d[S] = 0$ is the initialisation and is not counted). The total number of updates is _____.



Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Quicksort — comparison count on a skewed input]

Consider the following Python program (each call to `qs` on a list of length $m \geq 2$ is charged $m - 1$ comparisons, one per non-pivot element). What is printed?

Python

```

1 comps = 0
2 def qs(a):
3     global comps
4     if len(a) <= 1:
5         return a
6     p = a[0]
7     comps += len(a) - 1
8     L = [x for x in a[1:] if x < p]
9     R = [x for x in a[1:] if x >= p]
10    return qs(L) + [p] + qs(R)
11
12 qs([5, 1, 2, 3, 4, 10, 9, 8, 7, 6])
13 print(comps)

```

- (A) 45
- (B) 21
- (C) 25
- (D) 24

Q.18 [MSQ · one or more correct | 2 marks | Heaps — complexity facts with numbers]

All heaps below are binary min-heaps stored in arrays (root at index 0) holding **distinct** keys. Which of the following statements is/are TRUE?

- (A) Bottom-up build-heap on 1000 keys never performs more than 994 swaps
- (B) The k smallest of n keys can be output in sorted order in $O(n + k \log n)$ time using a heap
- (C) The maximum key of a min-heap with n keys can always be found with $O(\log n)$ comparisons
- (D) In a min-heap with 1000 keys the second-smallest key is always at index 1 or index 2

Q.19 [NAT · numerical answer | 2 marks | Recursion — counting calls]

Consider the following Python program. The **second** number printed is _____.

Python

```
1 calls = 0
2 def f(n):
3     global calls
4     calls += 1
5     if n < 3:
6         return n
7     return f(n - 1) + f(n - 3)
8
9 v = f(12)
10 print(v, calls)
```

Answer: _____ (enter a numerical value)

Q.20 [MCQ · single correct | 2 marks | Recurrences — change of variable]

Let $W(n)$ be the value of **steps** after the call $r(n)$ (starting from **steps** = 0). Which of the following is $W(n)$ for large n ?

Python

```
1 from math import isqrt
2 steps = 0
3 def r(n):
4     global steps
5     if n <= 2:
6         return
7     i = 1
8     while i < n:
9         i *= 2
10        steps += 1
11    r(isqrt(n))
12    r(isqrt(n))
```

- (A) $\Theta(\log n \cdot \log \log n)$
- (B) $\Theta(\log n)$
- (C) $\Theta(\log^2 n)$
- (D) $\Theta(\sqrt{n})$

Answer Key — Mock Test 19

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — floor division and modulo with negatives
2	NAT	1	40	Loop counting — geometric loops
3	MCQ	1	(A)	Asymptotic comparison of functions
4	MSQ	1	(A), (B)	Recurrences — identifying $\Theta(n \log n)$
5	NAT	1	27	Stacks — postfix evaluation
6	MCQ	1	(D)	Complexity of a halving loop nest
7	MCQ	1	(B)	Hashing — linear probing cost
8	MCQ	1	(D)	Linked lists — operation costs
9	NAT	1	63	Heaps — nodes by height
10	MSQ	1	(C), (D)	Python — cost of built-in operations
11	MCQ	2	(B)	Recurrence from recursive code
12	NAT	2	286	Loop counting — triple nested dependent loops
13	MSQ	2	(B), (C), (D)	Asymptotic notation — true/false statements
14	NAT	2	573	Recurrences — exact solution (merge sort worst case)
15	MSQ	2	(B), (C), (D)	Queue via two stacks — amortised cost
16	NAT	2	9	Dijkstra — counting distance updates
17	MCQ	2	(C)	Quicksort — comparison count on a skewed input
18	MSQ	2	(A), (B), (D)	Heaps — complexity facts with numbers
19	NAT	2	119	Recursion — counting calls
20	MCQ	2	(A)	Recurrences — change of variable

Detailed Solutions — Mock Test 19

Q.1 [MCQ · 1 mark · Python — floor division and modulo with negatives]

Answer: (C)

Python's `//` is **floor** division (rounds toward $-\infty$), and `%` is defined so that $a == (a // b) * b + a \% b$ always holds; hence the remainder takes the sign of the **divisor**. `int()` on a float truncates toward 0.

- `-17 // 5`: $-17 / 5 = -3.4 \rightarrow \text{floor} \rightarrow -4$.
- `-17 \% 5`: $-17 - (-4)(5) = -17 + 20 = 3$ (sign of divisor 5).
- `17 // -5`: $17 / -5 = -3.4 \rightarrow \text{floor} \rightarrow -4$.
- `int(-17 / 5)`: -3.4 truncated toward zero $\rightarrow -3$.

Output `-4 3 -4 -3` $\rightarrow$ option (C).

- (A) is C/Java behaviour (truncation for `/` on ints and a negative remainder).
- (B) floors the first but truncates the third — inconsistent.
- (D) uses a remainder with the sign of the dividend and floors the last value.

Trap: this matters in complexity questions too — loops like `while i != 0: i //= 2` never terminate for negative i , because $-1 // 2$ is -1 .

Q.2 [NAT · 1 mark · Loop counting — geometric loops]

Answer: 40

The outer loop takes $i = 3^0, 3^1, \dots, 3^6 = 1, 3, 9, 27, 81, 243, 729$ ($3^7 = 2187 \geq 1000$). For a fixed i the inner loop runs for $j = i, 2i, 4i, \dots$ while $j < 1000$, i.e. $\lfloor \log_2(999/i) \rfloor + 1$ times.

- $i = 1$: $j = 1 \dots 512 \rightarrow 10$ iterations.
- $i = 3$: $j = 3 \dots 768$ ($3 \cdot 2^8$) $\rightarrow 9$.
- $i = 9$: $j = 9 \dots 576$ ($9 \cdot 2^6$) $\rightarrow 7$.
- $i = 27$: $j = 27 \dots 864$ ($27 \cdot 2^5$) $\rightarrow 6$.
- $i = 81$: $j = 81 \dots 648$ ($81 \cdot 2^3$) $\rightarrow 4$.
- $i = 243$: $j = 243, 486, 972 \rightarrow 3$.
- $i = 729$: $j = 729 \rightarrow 1$.

Total = $10 + 9 + 7 + 6 + 4 + 3 + 1 = 40$.

Asymptotically, $\sum$ over $k = 0 \dots \log_3 n$ of $(\log_2 n - k \cdot \log_2 3) = \Theta(\log^2 n)$ — two nested logarithmic loops with *dependent* bounds still give $\Theta(\log^2 n)$, just with a constant about half that of independent loops.

Trap: multiplying 7 outer iterations by 10 inner ones (70) ignores that the inner loop starts at i , not at 1.

Q.3 [MCQ · 1 mark · Asymptotic comparison of functions]

Answer: (A)

Compare logarithms: put $L = \log_2 n$ ($L \rightarrow \infty$). Then

- $\log_2 f_3 = 10 \cdot \log_2 L$ (doubly logarithmic in n),
- $\log_2 f_1 = \sqrt{L}$,
- $\log_2 f_2 = L / 3$,
- $\log_2 f_4 = L - \log_2 L$.

As $L \rightarrow \infty$: $10 \log_2 L \ll \sqrt{L} \ll L/3 < L - \log_2 L$, and the gaps between these logs tend to infinity, so the functions themselves are separated by unbounded factors.

Hence $f_3 < f_1 < f_2 < f_4 \rightarrow$ option (A).

- (B) puts f_1 above $n^{1/3}$: $\sqrt{L} < L/3$ for $L > 9$.

- (C) puts $2^{\sqrt{\log n}}$ below polylog — wrong: $\sqrt{L}$ eventually beats $10 \log L$ (e.g. $L = 10^6$: 1000 vs ≈ 199).
- (D) puts $n^{1/3}$ above $n / \log n$: $L/3 < L - \log_2 L$ for large L .

Trap: for small n (e.g. $n = 2^{16}$, $L = 16$) $f_3 = 16^{10}$ is astronomically larger than $f_1 = 2^4$ — asymptotic order can disagree badly with small- n intuition.

Q.4 [MSQ · 1 mark · Recurrences — identifying $\Theta(n \log n)$]

Answer: (A), (B)

Use the master theorem (compare $f(n)$ with $n^{\log_b a}$) or the recursion-tree method.

- (A) $a = 2$, $b = 2 \rightarrow n^{\log_2 2} = n$; $f(n) = 5n = \Theta(n) \rightarrow$ case 2 $\rightarrow \Theta(n \log n)$. **Yes.**
- (B) Not master-theorem form, but the recursion tree has cost n at every level ($n/3 + 2n/3 = n$). The shortest root-to-leaf path has $\log_3 n$ levels and the longest $\log_{3/2} n$, both $\Theta(\log n)$, so $T(n) = \Theta(n \log n)$. **Yes.**
- (C) $a = b = 3 \rightarrow n^{\log_3 3} = n$; $f(n) = n / \log n$ is smaller than n only by a log factor, so the master theorem does not apply. Level i of the tree costs $n / \log(n/3^i)$, and summing gives $n \cdot \sum 1/j = \Theta(n \log \log n)$. **No.**
- (D) $a = 4$, $b = 2 \rightarrow n^{\log_2 4} = n^2$; $f(n) = n \log n = O(n^{2-\epsilon}) \rightarrow$ case 1 $\rightarrow \Theta(n^2)$. **No.**

Trap: (C) 'looks like' case 2 — but f must be $\Theta(n^{\log_b a} \cdot \log^k n)$ with $k \geq 0$; here $k = -1$, which gives $n \log \log n$.

Q.5 [NAT · 1 mark · Stacks — postfix evaluation]

Answer: 27

Scan left to right: push operands; on an operator pop the right operand b , then the left operand a , and push $a \text{ op } b$. Each token costs $O(1)$, so evaluation is $\Theta(n)$ for n tokens.

- 6, 2, 3 $\rightarrow$ stack [6, 2, 3].
- $+$ $\rightarrow 2 + 3 = 5 \rightarrow$ [6, 5].
- $*$ $\rightarrow 6 \times 5 = 30 \rightarrow$ [30].
- 4, 2 $\rightarrow$ [30, 4, 2]; $^$ $\rightarrow 4^2 = 16 \rightarrow$ [30, 16].
- 7 $\rightarrow$ [30, 16, 7]; $-$ $\rightarrow 16 - 7 = 9 \rightarrow$ [30, 9].
- 3 $\rightarrow$ [30, 9, 3]; $/$ $\rightarrow 9 / 3 = 3 \rightarrow$ [30, 3].
- $-$ $\rightarrow 30 - 3 = 27$.

The equivalent infix form is $6 \times (2 + 3) - (4^2 - 7) / 3$. The stack never holds more than 3 values.

Trap: popping in the wrong order turns $16 - 7$ into $7 - 16$ and $9 / 3$ into $3 / 9$; the **first** pop is the right operand.

Stack after each operator

	stack (bottom $\rightarrow$ top)
$+$	6 5
$*$	30
$^$	30 16
$-$ (1st)	30 9
$/$	30 3
$-$ (2nd)	27

Q.6 [MCQ · 1 mark · Complexity of a halving loop nest]**Answer: (D)**

The outer loop runs $\lfloor \log_2 n \rfloor + 1$ times, but the inner loop's length *shrinks* geometrically, so we must sum the actual work rather than multiply.

Total inner iterations = $n + \lfloor n/2 \rfloor + \lfloor n/4 \rfloor + \dots + 1 \leq n(1 + 1/2 + 1/4 + \dots) < 2n$.

It is also at least n (the first pass alone). Hence the running time is $\Theta(n)$.

Example: $n = 100 \rightarrow 100 + 50 + 25 + 12 + 6 + 3 + 1 = 197 < 200$.

- (A) would require the inner loop to shrink arithmetically ($i -= 1$), not geometrically.
- (B) $\Theta(n \log n)$ multiplies $\log n$ outer iterations by the *largest* inner cost n — an upper bound, but not tight.
- (C) counts only loop *headers*, not the inner work.
- (D) **Correct** — geometric series bounded by $2n$.

Tip: whenever inner work forms a geometric series, the total is dominated by the largest term (this is also why bottom-up build-heap is $O(n)$).

Q.7 [MCQ · 1 mark · Hashing — linear probing cost]**Answer: (B)**

Hash values: $42 \rightarrow 9$, $17 \rightarrow 6$, $64 \rightarrow 9$, $31 \rightarrow 9$, $75 \rightarrow 9$, $53 \rightarrow 9$, $20 \rightarrow 9$, $86 \rightarrow 9$ — seven of the eight keys collide at slot 9, the worst case for linear probing.

- 42: slot 9 $\rightarrow$ 1 probe.
- 17: slot 6 $\rightarrow$ 1 probe.
- 64: 9, **10** $\rightarrow$ 2.
- 31: 9, 10, **0** (wrap) $\rightarrow$ 3.
- 75: 9, 10, 0, **1** $\rightarrow$ 4.
- 53: 9, 10, 0, 1, **2** $\rightarrow$ 5.
- 20: 9 ... 2, **3** $\rightarrow$ 6.
- 86: 9 ... 3, **4** $\rightarrow$ 7.

Total = $1 + 1 + 2 + 3 + 4 + 5 + 6 + 7 = 29 \rightarrow$ option (B).

- (A) $22 = 29 - 7$ omits the final (successful) probe of the seven keys hashing to 9.
- (C) $36 = 1 + 2 + \dots + 8$ treats 17 as a colliding key too.
- (D) 28 counts only the colliding keys $1 + 2 + \dots + 7$.

Insight: k keys with the same home slot cost $1 + 2 + \dots + k = \Theta(k^2)$ probes in total, which is why a bad hash function turns hashing into $\Theta(n^2)$.

0	31
1	75
2	53
3	20
4	86
5	
6	17
7	
8	
9	42
10	64

Final table — one long cluster 9, 10, 0, 1, 2, 3, 4

Q.8 [MCQ · 1 mark · Linked lists — operation costs]

Answer: (D)

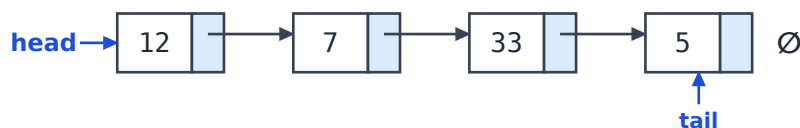
An operation is $O(1)$ only if every pointer it must change is reachable in $O(1)$ steps.

- (A) Insert at front: `new.next = head; head = new` → $O(1)$.
- (B) Insert at end: `tail.next = new; tail = new` → $O(1)$ thanks to the tail pointer.
- (C) Delete first: `head = head.next` → $O(1)$.
- (D) Delete last: we must set the **predecessor** of the tail (node 5 in the figure) to have `next = None` and make it the new tail. A singly linked list has no back pointers, so finding that predecessor requires walking from head: $n - 2$ steps → $\Theta(n)$.

Answer (D).

Consequence: a singly linked list with head and tail pointers is an ideal *queue* (enqueue at tail, dequeue at head, both $O(1)$), but deleting from the tail needs a doubly linked list.

Trap: the tail pointer gives access to the last node, not to the node *before* it.



After deleting 18: the walk to node 5 costs $\Theta(n)$

Q.9 [NAT · 1 mark · Heaps — nodes by height]

Answer: 63

In a heap with n nodes, node i (0-based) is a leaf iff $i \geq \lfloor n/2 \rfloor$. Grouping levels from the bottom, a heap has **at most** $\lfloor n / 2^{h+1} \rfloor$ nodes of height h (the classic build-heap bound); we count them exactly.

Direct count for $n = 1000$:

- height 0 (leaves): indices 500 ... 999 → 500 nodes.
- height 1: parents of leaves only, indices 250 ... 499 → 250 (node 499 has one child).
- height 2: indices 125 ... 249 → 125.
- height 3: indices 62 ... 124 → **63**. Node 62 lies one level above 63 ... 124, but its would-be descendants on the last level (from index 1007) do not exist, so its height is also 3; node 61 reaches index 991 and has height 4.

Here the bound $\lceil 1000 / 16 \rceil = \lceil 62.5 \rceil = 63$ is attained exactly.

This count drives the $O(n)$ bound for build-heap: a node of height h costs $O(h)$ to sift down, and $\sum h \cdot \lfloor n/2^{h+1} \rfloor = O(n)$.

Trap: using depth (distance from the root) instead of height: depth 3 has exactly 8 nodes.

Q.10 [MSQ · 1 mark · Python — cost of built-in operations]

Answer: (C), (D)

Each fragment loops n times, so the question is the cost of the operation inside.

- (A) `list.insert(0, x)` shifts every existing element one place right: the k -th insert costs $\Theta(k)$, total $0 + 1 + \dots + (n - 1) = \Theta(n^2)$. **No.** (`collections.deque.appendleft` would be $O(1)$.)
- (B) `x not in seen` is a **linear search** of a list. If all values are distinct, the k -th test scans k elements → $\Theta(n^2)$. **No.**
- (C) `d.get` and `d[x] = ...` are $O(1)$ expected dict operations → $\Theta(n)$. **Yes.**
- (D) `x not in s` and `s.add(x)` on a hash set are $O(1)$ expected → total $\Theta(n)$. **Yes.**

Answer: (D) and (C).

Trap: the expression `x in container` looks identical for lists and sets, but costs $\Theta(\text{len})$ for a list and $O(1)$ on average for a set or dict. Choosing the right container changes the asymptotic complexity of the whole algorithm.

Q.11 [MCQ · 2 marks · Recurrence from recursive code]

Answer: (B)

Translate the code into a recurrence: each call does $\Theta(n)$ work in the loop and makes **three** recursive calls on $n/2$ (the three `g(n // 2)` calls are evaluated separately — Python does not cache them).

$$T(n) = 3T(n/2) + \Theta(n).$$

Master theorem: $a = 3$, $b = 2$, $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$. Since $f(n) = n = O(n^{1.585 - \epsilon})$, case 1 applies $\rightarrow T(n) = \Theta(n^{\log_2 3})$.

Recursion-tree check ($n = 2^k$): level i has 3^i calls each doing $n/2^i$ work $\rightarrow$ cost $n(3/2)^i$. The series grows geometrically, so the bottom level dominates: $\sum n(3/2)^i$ for $i = 0 \dots k-1$ equals $2(3^k - 2^k) = \Theta(3^k) = \Theta(n^{\log_2 3})$.

- (A) would be the answer with **two** recursive calls (merge-sort shape).
- (C) $n^{1.5}$ is close numerically but $\log_2 3 \approx 1.585 > 1.5$, and exponents that differ give different Θ classes.
- (D) n^2 would need four calls of size $n/2$ ($4T(n/2) + n$).

Trap: writing `3 * g(n // 2)` instead would make it $T(n) = T(n/2) + n = \Theta(n)$. The number of *calls*, not the arithmetic, determines a.

Q.12 [NAT · 2 marks · Loop counting — triple nested dependent loops]

Answer: 286

Count the triples directly. For fixed i and j ($i \leq j \leq n-1$) the innermost loop runs $j-i$ times. Put $d = j-i$, which ranges over $0 \dots n-1-i$:

$$c = \sum \text{over } i \text{ of } (0 + 1 + \dots + (n-1-i)) = \sum \text{over } i \text{ of } (n-i)(n-i-1)/2.$$

Substituting $m = n-i$ ($m = 1 \dots n$): $c = \sum m(m-1)/2 = \sum C(m, 2) = C(n+1, 3)$ (hockey-stick identity).

$$\text{For } n = 12: C(13, 3) = 13 \cdot 12 \cdot 11 / 6 = \mathbf{286}.$$

Combinatorial view: each increment corresponds to a choice $i \leq k < j \leq n-1$ — equivalently three distinct values from $\{0, 1, \dots, 12\}$ after shifting j by 1 — hence $C(13, 3)$.

Partial sums for a check: $m = 1 \dots 12$ contribute 0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66, which add up to 286.

Asymptotically the fragment is $\Theta(n^3)$ with constant $1/6$.

Trap: using $C(n, 3) = 220$ (forgetting that k ranges up to $j-1$ while j can reach $n-1$, which adds one value) or $n^3/6 = 288$ (only the leading term).

Q.13 [MSQ · 2 marks · Asymptotic notation — true/false statements]

Answer: (B), (C), (D)

$f = O(g)$ iff $f(n)/g(n)$ stays bounded as $n \rightarrow \infty$.

- (A) $(n+1)! / n! = n+1 \rightarrow \infty$, unbounded. **False.**
- (B) $2^{n+1} / 2^n = 2$, a constant. **True.**
- (C) $\log(n!) = \sum \log i \leq n \log n$; and the top half of the terms are each $\geq \log(n/2)$, so $\log(n!) \geq (n/2) \log(n/2) = \Omega(n \log n)$. Hence $\Theta(n \log n)$ (Stirling gives $n \log_2 n - 1.44n + O(\log n)$). **True.**

- (D) $(n \log n) / n^{1.01} = \log n / n^{0.01}$. Any positive power of n eventually beats any power of $\log n$, so the ratio $\rightarrow 0$. **True** — although the crossover is enormous ($\log_2 n$ must exceed ≈ 1000 , i.e. $n > 2^{1000}$).

Answer: (D), (B), (C).

Contrast: $2^{2n} = 4^n$ is *not* $O(2^n)$ — a constant *factor* in the exponent changes the class, while a constant *added* to the exponent (B) does not. Similarly $(n+1)!$ differs from $n!$ by a factor that grows with n .

Trap: concluding (D) is false from small- n tables — at $n = 10^6$, $n \log n$ is far bigger than $n^{1.01}$, but O -notation concerns the limit.

Q.14 [NAT · 2 marks · Recurrences — exact solution (merge sort worst case)]

Answer: 573

Use the recursion tree level by level; at each level a sub-problem of size s contributes $s - 1$, so a level costs (sum of sizes) — (number of sub-problems of size ≥ 2).

- Level 0: $100 \rightarrow 99$.
- Level 1: $50, 50 \rightarrow 98$.
- Level 2: four 25s $\rightarrow 96$.
- Level 3: four 12s and four 13s $\rightarrow 44 + 48 = 92$.
- Level 4: $12 \rightarrow 6, 6$ and $13 \rightarrow 6, 7$ — twelve 6s, four 7s $\rightarrow 60 + 24 = 84$.
- Level 5: $6 \rightarrow 3, 3$ and $7 \rightarrow 3, 4$ — twenty-eight 3s, four 4s $\rightarrow 56 + 12 = 68$.
- Level 6: $3 \rightarrow 1, 2$ and $4 \rightarrow 2, 2$ — thirty-six 2s $\rightarrow 36$.
- Level 7: $2 \rightarrow 1, 1$ — all size 1 $\rightarrow 0$.

$T(100) = 99 + 98 + 96 + 92 + 84 + 68 + 36 = \mathbf{573}$.

Closed form check: $T(n) = n[\log_2 n] - 2^{\lceil \log_2 n \rceil} + 1 = 100 \cdot 7 - 128 + 1 = 573$.

Trap: using $n \log_2 n - n + 1$ (valid only for powers of two) gives $100 \cdot 6.64 - 99 \approx 565.4$ — not even an integer.

Recursion tree, level by level

	sub-problem sizes	level cost
0	100	99
1	50 × 2	98
2	25 × 4	96
3	12 × 4, 13 × 4	92
4	6 × 12, 7 × 4	84
5	3 × 28, 4 × 4	68
6	1 × 28, 2 × 36	36

Q.15 [MSQ · 2 marks · Queue via two stacks — amortised cost]

Answer: (B), (C), (D)

Every element is pushed on IN once, popped from IN at most once, pushed on OUT at most once and popped from OUT at most once $\rightarrow \leq 4$ stack operations per element over its life.

Trace (cost in brackets):

- enqueue 1–5 $\rightarrow$ IN = [1,2,3,4,5] (5).
- dequeue: OUT empty $\rightarrow$ move 5 elements (10), pop 1 (1) $\rightarrow$ **11**; OUT = [5,4,3,2] (top 2).
- dequeue $\rightarrow$ 2 (1).
- enqueue 6, 7 $\rightarrow$ IN = [6,7] (2).

- dequeue $\times 4$: 3, 4, 5 (1 each = 3); the 4th finds OUT empty $\rightarrow$ move 6, 7 (4) + pop 6 (1) = 5 $\rightarrow$ total 8.
- enqueue 8 (1); dequeue $\rightarrow$ OUT still holds 7 $\rightarrow$ returns **7** (1).

Total = 5 + 11 + 1 + 2 + 8 + 1 + 1 = **29**.

- (A) **False** — FIFO order: 7 was enqueued before 8, so 7 comes out; 8 remains in IN.
- (B) **True** — 29.
- (C) **True** — the first dequeue costs 11; the transfer of 6, 7 costs only 5.
- (D) **True** — at most 4 operations per enqueued element and the number of enqueues is $\leq m$, so the amortised cost per operation is $O(1)$ even though a single dequeue can cost $\Theta(n)$.

Trap: transferring IN $\rightarrow$ OUT on every dequeue (even when OUT is non-empty) breaks FIFO order and destroys the amortised bound.

Stack contents after each phase

	IN (bottom $\rightarrow$ top)	OUT (bottom $\rightarrow$ top)	cost
enq 1-5	1 2 3 4 5	—	5
deq ($\rightarrow$ 1)	—	5 4 3 2	11
deq ($\rightarrow$ 2)	—	5 4 3	1
enq 6,7	6 7	5 4 3	2
deq $\times 4$ ($\rightarrow$ 3,4,5,6)	—	7	8
enq 8	8	7	1
deq ($\rightarrow$ 7)	8	—	1

Q.16 [NAT · 2 marks · Dijkstra — counting distance updates]

Answer: 9

Each successful relaxation is one decrease-key (or insert) on the priority queue, so the number of updates measures the heap work: with a binary heap Dijkstra costs $O((V + \text{\#updates}) \log V) \leq O((V + E) \log V)$.

- Extract S (0): A = 4 ✓, B = 9 ✓ $\rightarrow$ 2 updates.
- Extract A (4): B = $\min(9, 4+3) = 7$ ✓, D = $4+12 = 16$ ✓ $\rightarrow$ 4.
- Extract B (7): D = $\min(16, 7+7) = 14$ ✓, C = $7+4 = 11$ ✓ $\rightarrow$ 6.
- Extract C (11): D = $\min(14, 11+2) = 13$ ✓, E = $11+6 = 17$ ✓ $\rightarrow$ 8.
- Extract D (13): E = $\min(17, 13+1) = 14$ ✓ $\rightarrow$ 9.
- Extract E (14): no outgoing edges.

Final distances: A 4, B 7, C 11, D 13, E 14. D was improved three times (16 $\rightarrow$ 14 $\rightarrow$ 13) and E twice (17 $\rightarrow$ 14).

Here every one of the 9 edges produced an update — the maximum possible, since an edge is relaxed exactly once (when its tail is extracted).

Trap: counting only the *decrease-key* operations on vertices that already had a finite distance (B, D, D, E $\rightarrow$ 4) — the question counts first assignments as well.

Tentative distances after each extraction

	A	B	C	D	E	updates so far
S	4	9	∞	∞	∞	2
A	4	7	∞	16	∞	4
B	4	7	11	14	∞	6
C	4	7	11	13	17	8
D	4	7	11	13	14	9

Q.17 [MCQ · 2 marks · Quicksort — comparison count on a skewed input]**Answer: (C)**

With the first element as pivot, a sorted (or reverse-sorted) sub-list produces the worst split (0 and $m - 1$). The first pivot 5 happens to be the median, but both halves are then already ordered, so each degenerates.

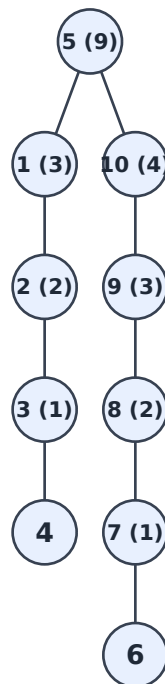
- $qs(10 \text{ elements}), \text{pivot } 5 \rightarrow 9 \text{ comparisons}; L = [1, 2, 3, 4], R = [10, 9, 8, 7, 6]$.
- Left: $[1,2,3,4] \rightarrow 3; [2,3,4] \rightarrow 2; [3,4] \rightarrow 1; [4] \rightarrow 0$. Subtotal 6.
- Right: $[10,9,8,7,6] \text{ pivot } 10 \rightarrow 4, L = [9,8,7,6]; \text{ then } 3, 2, 1$. Subtotal 10.

Total = $9 + 6 + 10 = 25 \rightarrow \text{option (C)}$.

- (A) $45 = C(10, 2)$ is the cost if the whole list were sorted (pivot 1 first).
- (B) 21 assumes balanced splits all the way down ($\approx$ best case for $n = 10$ is 19–21).
- (D) 24 forgets one comparison, e.g. charges $m - 2$ for the root call.

Insight: a good first split is not enough — quicksort's cost depends on every level. In general a sub-list of size m that is already sorted costs $m(m - 1)/2$: here $9 + C(4, 2) + C(5, 2) = 9 + 6 + 10$.

Trap: the $x \geq p$ in R keeps equal keys on the right; with all-equal input this also gives $\Theta(n^2)$.



Pivot tree: node = pivot (comparisons charged)

Q.18 [MSQ · 2 marks · Heaps — complexity facts with numbers]**Answer: (A), (B), (D)**

- (A) **True.** Sifting down a node of height h costs at most h swaps. For $n = 1000$ the numbers of nodes of heights 1 ... 9 are 250, 125, 63, 31, 16, 8, 4, 2, 1, so the worst case is $1 \cdot 250 + 2 \cdot 125 + 3 \cdot 63 + 4 \cdot 31 + 5 \cdot 16 + 6 \cdot 8 + 7 \cdot 4 + 8 \cdot 2 + 9 \cdot 1 = 250 + 250 + 189 + 124 + 80 + 48 + 28 + 16 + 9 = 994 < n$ — the concrete form of the $O(n)$ bound.
- (B) **True.** Build-heap $O(n)$, then k extract-min operations $O(k \log n)$.
- (C) **False.** The maximum can be any of the $\lfloor n/2 \rfloor$ leaves (heap order says nothing about leaves relative to each other), so $\Theta(n)$ comparisons are needed.
- (D) **True.** The second-smallest key's parent must be smaller than it, so the parent is the minimum, i.e. the root; the root's children are indices 1 and 2.

Answer: (A), (B), (D).

Trap for (C): confusing ‘the max is at a leaf’ (true) with ‘the max is at the last index’ or ‘on the rightmost path’ (both false).

Q.19 [NAT · 2 marks · Recursion — counting calls]

Answer: 119

Let $C(n)$ be the number of calls made by $f(n)$ (including itself). Then

$C(n) = 1$ for $n \leq 2$ and $C(n) = 1 + C(n-1) + C(n-3)$ for $n \geq 3$.

Tabulate:

- $C(0) = C(1) = C(2) = 1$
- $C(3) = 1 + 1 + 1 = 3$; $C(4) = 1 + 3 + 1 = 5$; $C(5) = 1 + 5 + 1 = 7$
- $C(6) = 1 + 7 + 3 = 11$; $C(7) = 1 + 11 + 5 = 17$; $C(8) = 1 + 17 + 7 = 25$
- $C(9) = 1 + 25 + 11 = 37$; $C(10) = 1 + 37 + 17 = 55$; $C(11) = 1 + 55 + 25 = 81$
- $C(12) = 1 + 81 + 37 = \mathbf{119}$

(The first number printed is $f(12) = 69$, from $F(n) = F(n-1) + F(n-3)$ with $F(0..2) = 0, 1, 2$.)

Growth: $C(n) \approx c \cdot \rho^n$ where $\rho \approx 1.4656$ is the real root of $x^3 = x^2 + 1$, so the naive recursion is exponential; memoising f reduces it to $n + 1$ distinct calls, i.e. $\Theta(n)$.

Trap: using the value recurrence for the call count (69), or dropping the ‘+1’ for the call itself — that recurrence counts only the base-case (leaf) calls, 60. Since every internal call has exactly two children, $\text{calls} = 2 \times 60 - 1 = 119$.

Q.20 [MCQ · 2 marks · Recurrences — change of variable]

Answer: (A)

The while-loop doubles i until it reaches n : $\lceil \log_2 n \rceil$ iterations. Then two recursive calls on $\sqrt{n}$. So

$W(n) = 2W(\sqrt{n}) + \Theta(\log n)$.

Change of variable: put $m = \log_2 n$, so $\sqrt{n} = 2^{m/2}$, and let $S(m) = W(2^m)$:

$S(m) = 2S(m/2) + \Theta(m)$.

This is the merge-sort recurrence $\rightarrow S(m) = \Theta(m \log m)$. Returning to n : $W(n) = \mathbf{\Theta(\log n \cdot \log \log n)}$ $\rightarrow$ option (A).

Check with $n = 2^{16}$: level 0: 16 steps; level 1: 2 calls on $2^8 \rightarrow 2 \cdot 8$; level 2: 4 calls on $2^4 \rightarrow 4 \cdot 4$; level 3: 8 calls on $4 \rightarrow 8 \cdot 2$; then $n = 2$ stops. Total $16 + 16 + 16 + 16 = 64 = m \log_2 m$ with $m = 16$.

- (B) ignores the doubling of calls at each level (that would be $W = W(\sqrt{n}) + \log n$).
- (C) multiplies $\log n$ by the wrong depth ($\log n$ instead of $\log \log n$ levels).
- (D) confuses the argument $\sqrt{n}$ with the cost.

Trap: the recursion depth is $\log \log n$, not $\log n$ — taking square roots shrinks n far faster than halving.

Mock Test 20

Stack & Queue Applications

Difficulty: GATE-level

Focus areas: simulations, monotonic stack, two-stack queue costs, amortised ops

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — closures and late binding]

Consider the following Python program. What is printed?

Python

```
1 fs = [lambda: i * i for i in range(4)]
2 gs = [lambda i=i: i * i for i in range(4)]
3 print(sum(f() for f in fs), sum(g() for g in gs))
```

- (A) 14 14
- (B) 36 14
- (C) 36 36
- (D) 0 14

Q.2 [NAT · numerical answer | 1 mark | Monotonic stack — next greater element]

The function below computes, for every element, the next greater element to its right (−1 if none) using a stack of indices. What is the value of *s* after

s, p = nge_sum([5, 3, 8, 2, 4, 9, 1, 7])?

Python

```
1 def nge_sum(a):
2     res, st, pops = [-1] * len(a), [], 0
3     for i, x in enumerate(a):
4         while st and a[st[-1]] < x:
5             res[st.pop()] = x
6             pops += 1
7         st.append(i)
8     return sum(res), pops
9
10 s, p = nge_sum([5, 3, 8, 2, 4, 9, 1, 7])
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Queues — circular array]

A queue is stored in a circular array *Q*[0..6] (capacity 7). *front* is the index of the first element and *rear* is the index where the **next** element will be written; both advance by (index + 1) mod 7.

Initially *front* = *rear* = 3 and the queue is empty. The values 10, 20, 30, ... are enqueued in this order, interleaved as follows:

enqueue 5 values, dequeue 3, enqueue 4 values, dequeue 2.

What are (*front*, *rear*, *Q*[*front*]) at the end?

- (A) (1, 4, 60)
- (B) (1, 5, 60)
- (C) (5, 1, 90)
- (D) (1, 5, 50)

Q.4 [NAT · numerical answer | 1 mark | Stacks — valid stack permutations]

The numbers 1, 2, 3, 4, 5 are pushed onto an initially empty stack in this order; pops may be interleaved arbitrarily with the pushes and each popped value is output immediately. How many of the following six sequences can be produced as the output?

- (i) 3 2 5 4 1 (ii) 4 2 3 1 5 (iii) 2 1 5 3 4
 (iv) 1 5 4 3 2 (v) 5 3 4 2 1 (vi) 3 4 2 5 1

Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | Deques — input-restricted deque]

The values 1, 2, 3, 4 arrive in this order and must be inserted at the **rear** of an input-restricted deque (insertion only at the rear, deletion at either end). Deletions may be interleaved with insertions and each deleted value is output. Which of the following output sequences are possible?

- (A) 2 4 1 3
 (B) 4 1 3 2
 (C) 4 2 1 3
 (D) 4 2 3 1

Q.6 [MCQ · single correct | 1 mark | Python — collections.deque]

Consider the following Python program. What is printed?

Python

```
1 from collections import deque
2 d = deque([10, 20, 30, 40, 50])
3 d.rotate(2)
4 d.appendleft(d.pop())
5 d.extendleft([1, 2])
6 x = d.popleft() + d[-1]
7 d.rotate(-3)
8 print(x, list(d))
```

- (A) 21 [50, 10, 20, 2, 30, 40]
 (B) 22 [1, 30, 40, 50, 10, 20]
 (C) 22 [50, 10, 20, 1, 30, 40]
 (D) 12 [40, 50, 10, 1, 20, 30]

Q.7 [NAT · numerical answer | 1 mark | Hashing — double hashing]

Keys 27, 17, 49, 30, 71, 38 are inserted in this order into an empty table of size 11 using double hashing: probe i ($i = 0, 1, 2, \dots$) examines slot $(h_1(k) + i \cdot h_2(k)) \bmod 11$, where $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$. The index of the slot in which 38 is stored is _____.

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Binary search — lower bound]

Consider the following Python function and list.

Python

```
1 def lb(a, x):
2     lo, hi = 0, len(a)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 a = [3, 7, 7, 7, 12, 18, 18, 25]
```

Which of the following statements is/are TRUE?

- (A) `lb(a, 7)` returns 1
- (B) `lb(a, 30)` returns 7
- (C) The `while` loop body executes exactly 3 times during `lb(a, 7)`
- (D) `lb(a, 18) - lb(a, 12)` equals 1

Q.9 [MSQ · one or more correct | 1 mark | Expression trees from postfix]

An expression tree is built from the postfix expression

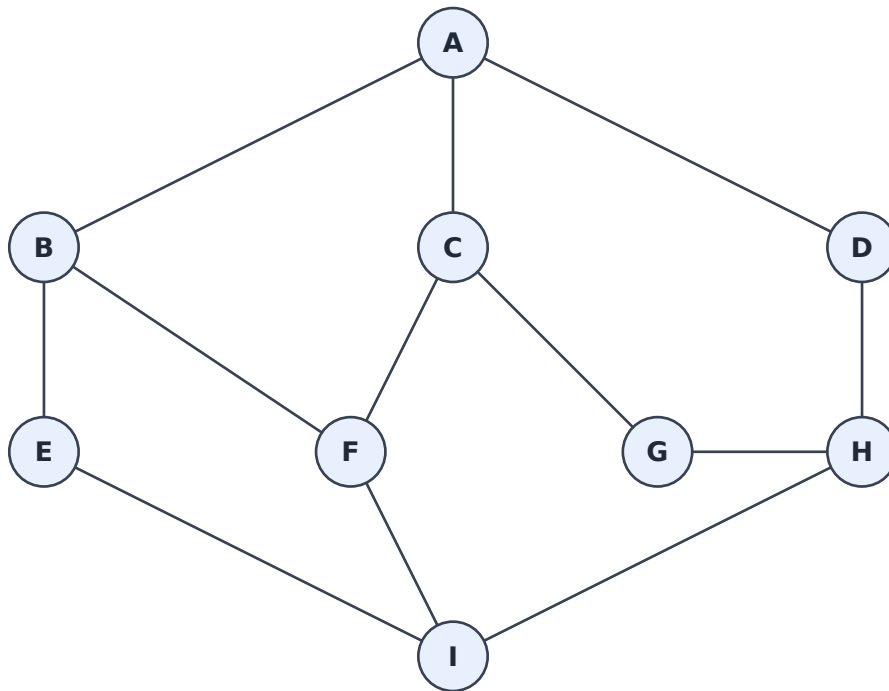
6 3 1 - / 4 2 ^ 5 * +

using a stack of subtrees (an operand is pushed as a leaf; an operator pops its right child, then its left child, and pushes the new subtree). Which of the following statements is/are TRUE? (^ denotes exponentiation.)

- (A) The expression evaluates to 83
- (B) The tree has exactly 5 leaves
- (C) The stack never holds more than 3 subtrees during the construction
- (D) The prefix form of the tree is + / 6 - 3 1 * ^ 4 2 5

Q.10 [NAT · numerical answer | 1 mark | BFS — queue occupancy]

BFS is run on the undirected graph below from vertex A; when a vertex is dequeued, its unvisited neighbours are enqueued in alphabetical order (a vertex is marked visited when it is enqueued). The maximum number of vertices present in the queue at any instant is _____.



Answer: _____ (enter a numerical value)

Q.11 [NAT · numerical answer | 2 marks | Monotonic stack — stock span]

The *span* of day i is the number of consecutive days ending at day i (including day i) whose price is $\leq$ the price on day i . Spans are computed with the stack algorithm below for the prices 30, 25, 28, 22, 26, 35, 31, 33, 40

The sum of all nine spans is _____.

Python

```
1 def spans(p):
2     st, out = [], []
3     for i, x in enumerate(p):
4         while st and p[st[-1]] <= x:
5             st.pop()
6         out.append(i + 1 if not st else i - st[-1])
7         st.append(i)
8     return out
```

p =	30	25	28	22	26	35	31	33	40
	0	1	2	3	4	5	6	7	8

Prices, day 0 to day 8

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Stacks — infix to postfix]

The infix expression

$$a - b^c^d * e + f / (g - h) * i$$

is converted to postfix by the standard operator-stack algorithm. Precedence: $^$ highest (right-associative), then $*$ and $/$ (left-associative), then $+$ and $-$ (left-associative). The resulting postfix expression is

- (A) $a b c d ^ ^ e * - f g h - / i * +$
- (B) $a b c ^ d ^ e * - f g h - / i * +$
- (C) $a b c d ^ ^ e * f g h - / i * + -$
- (D) $a b c d ^ ^ e * - f g h - i * / +$

Q.13 [NAT · numerical answer | 2 marks | Queue using two stacks — cost]

A queue is implemented with two stacks IN and OUT. `enqueue(x)` pushes x on IN. `dequeue()` first checks OUT; **only if OUT is empty**, it pops every element of IN and pushes it onto OUT; then it pops OUT. Each individual push or pop counts as one stack operation (emptiness checks are free). Starting with both stacks empty, the following 14 operations are performed (E = enqueue the next integer 1, 2, 3, ...; D = dequeue):

E E D E E D D E D E E D E

The total number of stack operations performed is _____.

Answer: _____ (enter a numerical value)

Q.14 [MSQ · one or more correct | 2 marks | Monotonic stack — largest rectangle in a histogram]

The largest-rectangle algorithm below is run on bar heights $h = [2, 5, 4, 6, 3, 1, 4]$ (each bar has width 1). A sentinel bar of height 0 is appended. Which of the following statements is/are TRUE?

Python

```
1 def largest(h):
2     H, st, best, pops = h + [0], [], 0, 0
3     for i, x in enumerate(H):
4         while st and H[st[-1]] >= x:
5             t = st.pop(); pops += 1
6             w = i if not st else i - st[-1] - 1
7             best = max(best, H[t] * w)
8         st.append(i)
9     return best, pops
```

- (A) The function returns $pops = 7$
- (B) The function returns $best = 12$
- (C) Exactly one rectangle (one choice of height and contiguous range) attains the maximum area
- (D) When the bar of height 1 (index 5) is processed, exactly 3 bars are popped

Q.15 [MCQ · single correct | 2 marks | Queue simulation — elimination game]

Consider the following Python program. What is printed?

Python

```
1 from collections import deque
2 q = deque(range(1, 9))
3 out = []
4 while len(q) > 1:
5     q.rotate(-3)
6     out.append(q.pop())
7 print(out[:3], q[0])
```

(A) [4, 8, 5] 6

(B) [3, 6, 1] 4

(C) [5, 1, 4] 3

(D) [3, 6, 1] 7

Q.16 [NAT · numerical answer | 2 marks | Quicksort with an explicit stack]

The non-recursive quicksort below (Lomuto partition, last element as pivot) is run on

$A = [5, 1, 6, 2, 7, 3, 4]$. The maximum number of (lo, hi) pairs present on the stack at any time is _____.

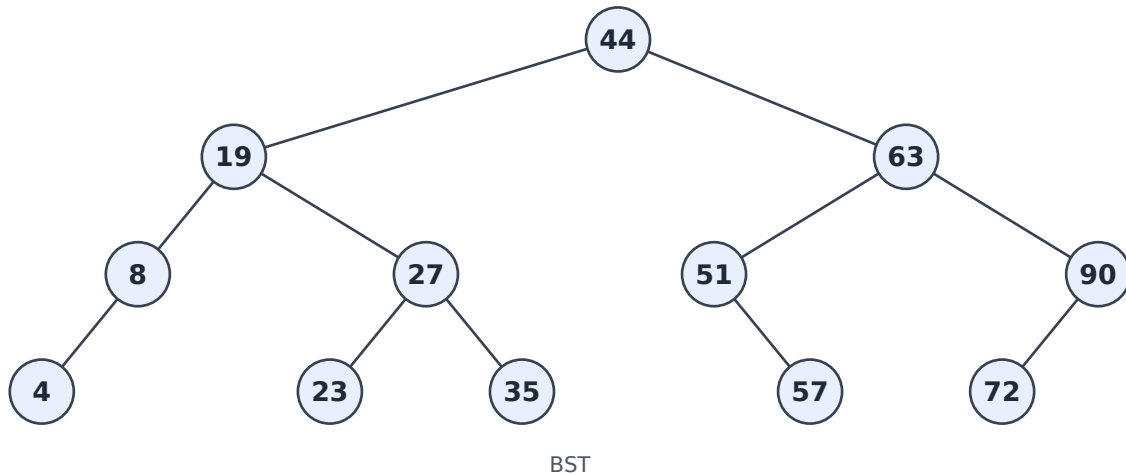
Python

```
1 def qsort(A):
2     st = [(0, len(A) - 1)]
3     while st:
4         lo, hi = st.pop()
5         if lo >= hi:
6             continue
7         p, i = A[hi], lo - 1
8         for j in range(lo, hi):
9             if A[j] <= p:
10                i += 1
11                A[i], A[j] = A[j], A[i]
12        A[i + 1], A[hi] = A[hi], A[i + 1]
13        st.append((lo, i))      # left part
14        st.append((i + 2, hi))  # right part
```

Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Trees — spiral traversal with two stacks]

The BST below is traversed with two stacks S1 and S2. Initially S1 = [root]. Repeat until both are empty: (i) pop every node from S1, printing it and pushing its **left then right** child (if present) onto S2; (ii) pop every node from S2, printing it and pushing its **right then left** child onto S1. What is printed?



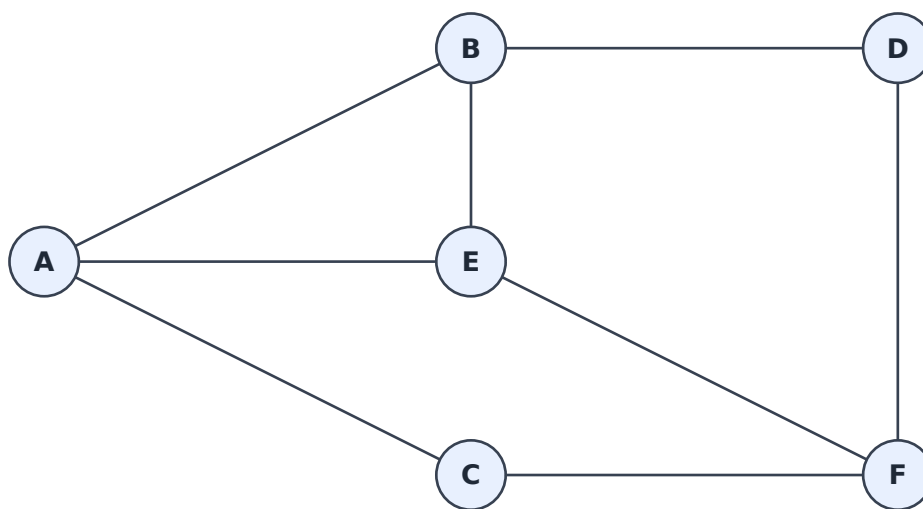
- (A) 44 19 63 90 51 27 8 4 23 35 57 72
- (B) 44 19 63 8 27 51 90 4 23 35 57 72
- (C) 44 63 19 8 27 51 90 72 57 35 23 4
- (D) 44 63 19 8 27 51 90 4 23 35 57 72

Q.18 [MCQ · single correct | 2 marks | DFS with an explicit stack]

The following iterative DFS is run on the undirected graph below from A. Note that a vertex is marked visited when it is **popped**, and neighbours are pushed in alphabetical order. In what order are vertices appended to **vis**?

Python

```
1 def dfs(G, s):
2     st, vis = [s], []
3     while st:
4         u = st.pop()
5         if u in vis:
6             continue
7         vis.append(u)
8         for v in sorted(G[u]):
9             if v not in vis:
10                st.append(v)
11     return vis
```



- (A) A E F D B C
- (B) A E F D C B
- (C) A C F E B D
- (D) A B D F C E

Q.19 [MSQ · one or more correct | 2 marks | Linked lists — circular queue with tail pointer]

A queue is implemented as a circular singly linked list accessed only through a **tail** pointer (**tail.nxt** is the front). Consider the following program.

Python

```
1 class Node:
2     def __init__(self, v):
3         self.v, self.nxt = v, None
4
5 tail = None
6 def enq(v):
7     global tail
8     n = Node(v)
9     if tail is None:
10        n.nxt = n
11    else:
12        n.nxt = tail.nxt
13        tail.nxt = n
14    tail = n
15
16 def deq():
17     global tail
18     head = tail.nxt
19     if head is tail:
20         tail = None
21     else:
22         tail.nxt = head.nxt
23     return head.v
24
25 for x in [4, 8, 15]:
26     enq(x)
27 a = deq()
28 enq(16); enq(23)
29 b = deq() + deq()
30 enq(a + b)
```

After the program finishes, which of the following statements is/are TRUE?

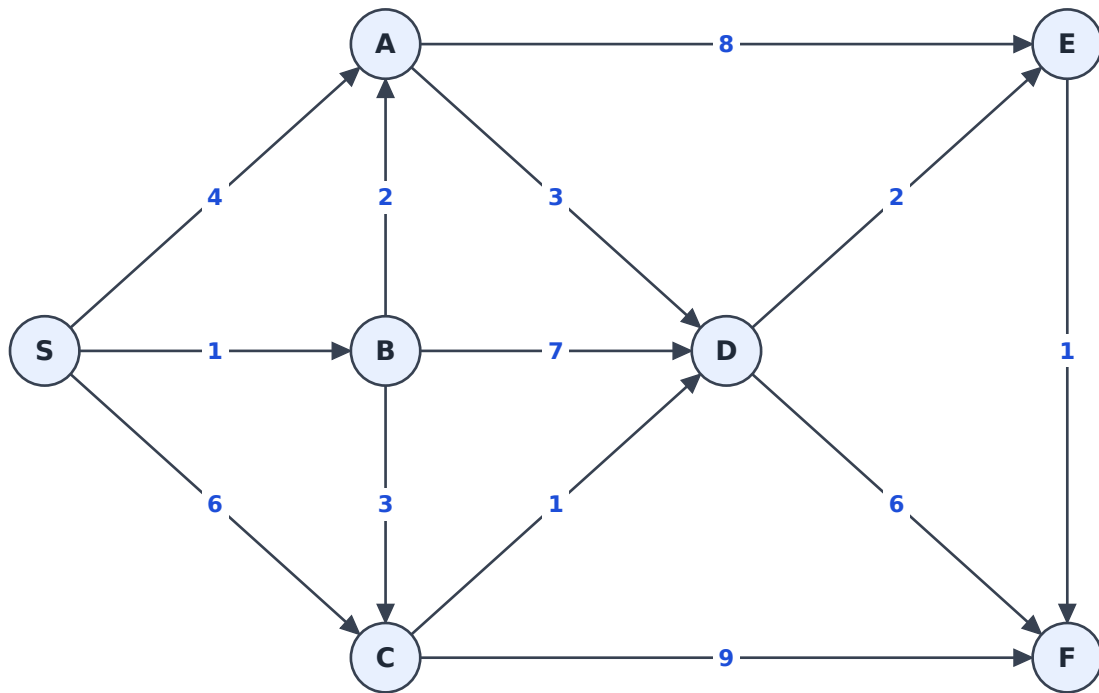
- (A) A further call **deq()** would return 23
- (B) **tail.nxt.v == 16**
- (C) **tail.v == 27**
- (D) **tail.nxt.nxt.v == 27**

Q.20 [NAT · numerical answer | 2 marks | Dijkstra with a binary heap (lazy deletion)]

Dijkstra's algorithm is implemented with Python's `heapq` using *lazy deletion*: whenever a distance improves, a new (distance, vertex) entry is pushed and outdated entries are skipped when popped. It is run from S on the directed graph below. Counting the initial push of (0, S), the total number of `heappush` calls is _____.

Python

```
1 import heapq
2 def dijkstra(G, s):
3     d = {v: float('inf') for v in G}
4     d[s], pq = 0, [(0, s)]
5     while pq:
6         du, u = heapq.heappop(pq)
7         if du > d[u]:
8             continue          # stale entry
9         for v, w in G[u]:
10            if du + w < d[v]:
11                d[v] = du + w
12                heapq.heappush(pq, (d[v], v))
13    return d
```



Answer: _____ (enter a numerical value)

Answer Key — Mock Test 20

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — closures and late binding
2	NAT	1	43	Monotonic stack — next greater element
3	MCQ	1	(B)	Queues — circular array
4	NAT	1	3	Stacks — valid stack permutations
5	MSQ	1	(A), (B)	Deque — input-restricted deque
6	MCQ	1	(C)	Python — collections.deque
7	NAT	1	2	Hashing — double hashing
8	MSQ	1	(A), (D)	Binary search — lower bound
9	MSQ	1	(A), (C), (D)	Expression trees from postfix
10	NAT	1	4	BFS — queue occupancy
11	NAT	2	25	Monotonic stack — stock span
12	MCQ	2	(A)	Stacks — infix to postfix
13	NAT	2	26	Queue using two stacks — cost
14	MSQ	2	(A), (B)	Monotonic stack — largest rectangle in a histogram
15	MCQ	2	(D)	Queue simulation — elimination game
16	NAT	2	4	Quicksort with an explicit stack
17	MCQ	2	(C)	Trees — spiral traversal with two stacks
18	MCQ	2	(A)	DFS with an explicit stack
19	MSQ	2	(B), (C)	Linked lists — circular queue with tail pointer
20	NAT	2	14	Dijkstra with a binary heap (lazy deletion)

Detailed Solutions — Mock Test 20

Q.1 [MCQ · 1 mark · Python — closures and late binding]

Answer: (B)

Concept. A lambda body looks up free variables when it is *called*, not when it is created (late binding). A default argument, by contrast, is evaluated once when the lambda is *defined*.

- In `fs`, all four lambdas refer to the comprehension's variable `i`. When they are called the comprehension has finished, so `i = 3` for every lambda. Each returns $9 \rightarrow \text{sum} = 4 \times 9 = \mathbf{36}$.
- In `gs`, `i=i` captures the current value as a default parameter: the lambdas return 0, 1, 4, 9 $\rightarrow \text{sum} = \mathbf{14}$.

Output: `36 14` $\rightarrow$ (B).

Options. (A) assumes the first list also captures values. (C) assumes the default-arg trick does not help. (D) assumes `i` is 0 in the closures (or that the comprehension variable is not visible) — but each comprehension has its own scope whose `i` ends at 3.

Tip: `i=i` (or `functools.partial`) is the standard fix for late binding in loops.

Q.2 [NAT · 1 mark · Monotonic stack — next greater element]

Answer: 43

Concept. The stack holds indices whose next greater element is still unknown; their values are non-increasing from bottom to top. A new value `x` pops every smaller value — `x` is their answer.

- `x=5`: push. Stack [5]
- `x=3`: $5 \geq 3$, push. [5, 3]
- `x=8`: pops 3 ($\rightarrow 8$) and 5 ($\rightarrow 8$). [8]
- `x=2`: push. [8, 2]
- `x=4`: pops 2 ($\rightarrow 4$). [8, 4]
- `x=9`: pops 4 ($\rightarrow 9$) and 8 ($\rightarrow 9$). [9]
- `x=1`: push. [9, 1]
- `x=7`: pops 1 ($\rightarrow 7$). [9, 7]
- End: 9 and 7 keep -1 .

`res = [8, 8, 9, 4, 9, -1, 7, -1]`; `s = 8+8+9+4+9-1+7-1 = \mathbf{43} (and p = 6 pops).`

Trap: forgetting the two -1 entries (giving 45), or taking the *maximum* to the right instead of the *next greater element* (e.g. 9 for 5).

Q.3 [MCQ · 1 mark · Queues — circular array]

Answer: (B)

Concept. `front` advances once per dequeue and `rear` once per enqueue, both modulo the capacity; the `k`-th value ever enqueued sits at index $(\text{start} + k - 1) \bmod 7$.

- 9 enqueues: `rear = (3 + 9) mod 7 = \mathbf{5}.`
- 5 dequeues: `front = (3 + 5) mod 7 = \mathbf{1}`.
- Queue length = $9 - 5 = 4$ (never exceeds 7: the peak is 6 after the second batch).
- The values removed are 10, 20, 30, 40, 50, so the front value is the 6th one enqueued, **60**, stored at index $(3 + 5) \bmod 7 = 1 \checkmark$.

Remaining contents: `Q[1]=60, Q[2]=70, Q[3]=80, Q[4]=90` $\rightarrow$ (1, 5, 60).

Options. (A) treats `rear` as the index of the last element. (C) swaps the roles of `front` and `rear`. (D) is off by one in the dequeued count.

Tip: with this convention, $\text{length} = (\text{rear} - \text{front}) \bmod \text{capacity}$ whenever the queue is not full: $(5 - 1) \bmod 7 = 4 \checkmark$.



Final circular array (slots 0, 5, 6 hold stale/no data)

Q.4 [NAT · 1 mark · Stacks — valid stack permutations]

Answer: 3

Concept. Simulate greedily: to output x , push every not-yet-pushed value up to x , then x must be on top. Equivalently, a sequence is invalid iff it contains a pattern ... c ... a ... b ... with $a < b < c$ (a “312” pattern).

- (i) 3 2 5 4 1: push 1,2,3 pop 3; pop 2; push 4,5 pop 5; pop 4; pop 1. **Valid.**
- (ii) 4 2 3 1 5: after popping 4 the top is 3, not 2. **Invalid** (pattern 4,2,3).
- (iii) 2 1 5 3 4: after 5, stack holds 3, 4 with 4 on top; 3 is blocked. **Invalid.**
- (iv) 1 5 4 3 2: pop 1; push 2..5, pop 5, 4, 3, 2. **Valid.**
- (v) 5 3 4 2 1: after 5 the top is 4, not 3. **Invalid.**
- (vi) 3 4 2 5 1: pop 3; push 4 pop 4; pop 2; push 5 pop 5; pop 1. **Valid.**

Valid sequences: (i), (iv), (vi) → **3**.

Trap: once a larger value is popped, every smaller value still on the stack must come out in *decreasing* order.

Q.5 [MSQ · 1 mark · Deques — input-restricted deque]

Answer: (A), (B)

Concept. If 4 is output first, all of 1, 2, 3 are already inside, in order 1 2 3 from front to rear; afterwards only the two ends can be taken.

- (A) 2 4 1 3: insert 1, 2; delete 2 from the rear; insert 3, 4; delete 4 from the rear; deque [1, 3] → 1 from front, then 3. **Possible.**
- (B) 4 1 3 2: deque [1, 2, 3] → take 1 from the front → [2, 3] → take 3 from the rear → 2. **Possible.**
- (C) 4 2 1 3: again 2 is in the middle of [1, 2, 3] when it is needed. **Impossible.**
- (D) 4 2 3 1: after removing 4 the deque is [1, 2, 3]; 2 is in the middle and cannot be removed. **Impossible.**

In fact, of the 24 permutations of 1..4 exactly 22 are achievable; the two impossible ones are precisely 4 2 1 3 and 4 2 3 1.

Tip: for an input-restricted deque the hard cases are those where a *middle* element must leave before both its neighbours.

Q.6 [MCQ · 1 mark · Python — collections.deque]

Answer: (C)

Concept. `rotate(k)` with $k > 0$ moves the last k items to the front; with $k < 0$ it moves the first $|k|$ items to the back. `extendleft` inserts items one by one at the left, so they appear **reversed**.

- `rotate(2)` → [40, 50, 10, 20, 30]
- `pop()` removes 30; `appendleft` → [30, 40, 50, 10, 20]
- `extendleft([1, 2])` → 1 goes left, then 2 goes left of it → [2, 1, 30, 40, 50, 10, 20]
- `popleft()` = 2, `d[-1]` = 20 → $x = 22$; `d` = [1, 30, 40, 50, 10, 20]
- `rotate(-3)` → [50, 10, 20, 1, 30, 40]

Output 22 [50, 10, 20, 1, 30, 40] → (C).

Options. (A) forgets that `extendleft` reverses (takes 1 as the leftmost). (B) omits the final rotation. (D) rotates the wrong way at the start.

Trap: `extendleft(iterable) ≠ iterable + d`; it equals `reversed(iterable) + d`.

Q.7 [NAT · 1 mark · Hashing — double hashing]

Answer: 2

Concept. In double hashing the step size depends on the key, so keys sharing a home slot follow *different* probe sequences.

- 27: $h_1 = 5 \rightarrow$ slot **5**.
- 17: $h_1 = 6 \rightarrow$ slot **6**.
- 49: $h_1 = 5$ (full), $h_2 = 1 + 0 = 1 \rightarrow 6$ (full) $\rightarrow$ **7**.
- 30: $h_1 = 8 \rightarrow$ slot **8**.
- 71: $h_1 = 5$ (full), $h_2 = 1 + 1 = 2 \rightarrow 7$ (full) $\rightarrow$ **9**.
- 38: $h_1 = 5$ (full), $h_2 = 1 + 3 = 4 \rightarrow 9$ (full) $\rightarrow 13 \bmod 11 =$ **2** (free).

So 38 is stored at index **2** after three probes.

Trap: applying linear probing (step 1) would put 38 at index 10; and forgetting the “1 +” in h_2 gives step 3 for 38 $\rightarrow 8$ (full) $\rightarrow 0$, a wrong slot.

0	
1	
2	38
3	
4	
5	27
6	17
7	49
8	30
9	71
10	

Final table

Q.8 [MSQ · 1 mark · Binary search — lower bound]

Answer: (A), (D)

Concept. `lb` returns the first index whose value is $\geq x$ (`len(a)` if none), on the half-open interval $[lo, hi)$.

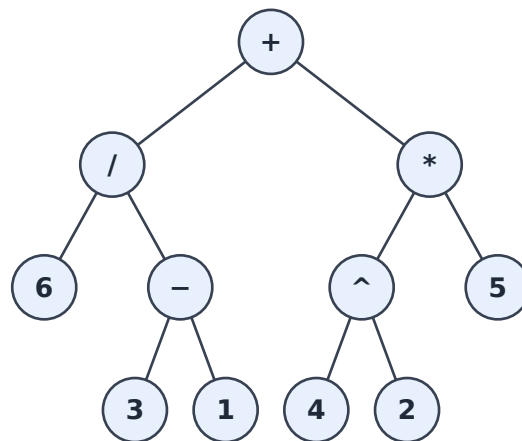
- (A) First value ≥ 7 is at index 1. **TRUE.**
- (B) No value is ≥ 30 , so the function returns `len(a) = 8`, not 7. **FALSE.**
- (C) Trace for $x = 7$: $(lo, hi) = (0, 8) \rightarrow$ mid 4, $12 \geq 7 \rightarrow hi = 4$; $(0, 4) \rightarrow$ mid 2, $7 \geq 7 \rightarrow hi = 2$; $(0, 2) \rightarrow$ mid 1, $hi = 1$; $(0, 1) \rightarrow$ mid 0, $3 < 7 \rightarrow lo = 1$. That is **4** iterations. **FALSE.**
- (D) $lb(a, 18) = 5$ and $lb(a, 12) = 4 \rightarrow$ difference 1 (= number of 12s). **TRUE.**

Trap: on a half-open interval the size goes $8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$, so up to $\lfloor \log_2 n \rfloor + 1 = 4$ iterations are needed — not $\log_2 8 = 3$.

Q.9 [MSQ · 1 mark · Expression trees from postfix]**Answer: (A), (C), (D)****Concept.** Postfix → tree is a direct stack simulation; the last operator becomes the root.

Stack trace (sizes in brackets): 6 [1], 3 [2], 1 [3], − → (3−1) [2], / → 6/(3−1) [1], 4 [2], 2 [3], ^ → 4^2 [2], 5 [3], * → (4^2)*5 [2], + → root [1].

- (A) $6/(3-1) + 4^2 \cdot 5 = 3 + 80 = 83$. **TRUE.**
- (B) Leaves are the operands 6, 3, 1, 4, 2, 5 → **6** leaves. **FALSE.**
- (C) The maximum stack size is 3 (reached three times). **TRUE.**
- (D) Pre-order of the tree: + / 6 − 3 1 * ^ 4 2 5. **TRUE.**

Trap: the first popped subtree is the **right** child — swapping them would turn 3 − 1 into 1 − 3 and the value into −3 + 80.

Expression tree

Q.10 [NAT · 1 mark · BFS — queue occupancy]**Answer: 4****Concept.** The queue length grows when a dequeued vertex discovers several new vertices and shrinks by one per dequeue.

Queue after processing each vertex:

- A → enqueue B, C, D: [B, C, D] (3)
- B → enqueue E, F: [C, D, E, F] (**4**)
- C → F seen; enqueue G: [D, E, F, G] (**4**)
- D → enqueue H: [E, F, G, H] (**4**)
- E → enqueue I: [F, G, H, I] (**4**)
- F, G, H, I discover nothing new: 3, 2, 1, 0.

Maximum = **4**.

A vertex is removed from the queue before its neighbours are added, so the count never momentarily exceeds these values.

Trap: guessing the maximum degree (3) — the queue holds parts of two consecutive BFS levels at once.

Q.11 [NAT · 2 marks · Monotonic stack — stock span]**Answer: 25**

Concept. The stack keeps indices of a strictly decreasing sequence of prices — the only days that can still block a future span. A day's span reaches back to the nearest earlier day with a *greater* price (the stack top after popping).

- Day 0 (30): stack empty → span 1. Stack [30]
- Day 1 (25): top 30 > 25 → span 1. [30, 25]
- Day 2 (28): pop 25; top 30 (day 0) → span 2. [30, 28]
- Day 3 (22): top 28 → span 1. [30, 28, 22]
- Day 4 (26): pop 22; top 28 (day 2) → span 2. [30, 28, 26]
- Day 5 (35): pops 26, 28, 30; empty → span 6. [35]
- Day 6 (31): top 35 → span 1. [35, 31]
- Day 7 (33): pop 31; top 35 (day 5) → span 2. [35, 33]
- Day 8 (40): pops 33, 35; empty → span 9. [40]

Spans = 1, 1, 2, 1, 2, 6, 1, 2, 9 → sum = **25**.

Amortised cost: there are 8 pops in total, never more than $n = 9$, so the algorithm is $\Theta(n)$ despite the nested loop.

Trap: counting only *consecutive* smaller days via the previous day's comparison (e.g. giving day 4 a span of 2 but day 5 a span of 3) — the span jumps over every popped run.

		Span of each day								
		0	1	2	3	4	5	6	7	8
price		30	25	28	22	26	35	31	33	40
span		1	1	2	1	2	6	1	2	9

Q.12 [MCQ · 2 marks · Stacks — infix to postfix]**Answer: (A)**

Concept. When an operator arrives, pop operators of *higher* precedence, and of *equal* precedence only if the incoming operator is left-associative. (is pushed,) pops back to the matching (.

Trace (output | stack):

- a | ; - | -; b | -
- ^ : top - is lower → push | - ^; c
- ^ : equal precedence but right-assoc → **do not pop**, push | - ^ ^; d → a b c d
- * : pop ^, ^ (higher); - is lower → push | - *; e → a b c d ^ ^ e
- + : pop * (higher) and - (equal, left-assoc) → a b c d ^ ^ e * - ; push +
- f | + ; / : push | + /; (g - h) → f g h -
- * : / has equal precedence, left-assoc → pop / → ... f g h - / ; push *
- i, end: pop *, + → **a b c d ^ ^ e * - f g h - / i * +**

Option by option:

- (A) Correct.
- (B) treats ^ as left-associative: $(b^c)^d$ instead of $b^{(c^d)}$.
- (C) fails to pop - when + arrives, i.e. computes $a - (... + ...)$.
- (D) treats * and / as right-associative: $f / ((g-h) \cdot i)$.

Trap: right-associativity of ^ only affects ties between ^ operators; it is the left-associativity of +, - and *, / that forces the pops on equal precedence.

Q.13 [NAT · 2 marks · Queue using two stacks — cost]**Answer: 26**

Concept. Each element costs 1 push on IN, and *if it is ever transferred* 1 pop from IN + 1 push on OUT, and *if dequeued* 1 pop from OUT. Elements still on IN at the end were never transferred.

Trace (IN | OUT, top on the right):

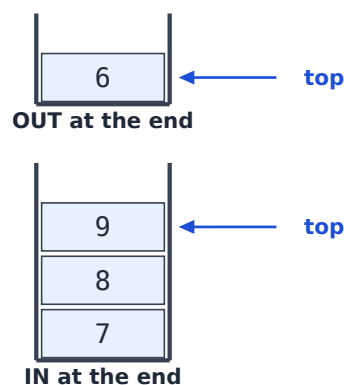
- E E E: IN [1 2 3] — 3 ops
- D: OUT empty → transfer 3 elements (6 ops) → OUT [3 2 1]; pop 1 (1 op)
- E E: IN [4 5] — 2 ops
- D: pop 2 (1); D: pop 3 (1) — OUT now empty
- E: IN [4 5 6] — 1 op
- D: transfer 3 (6 ops) → OUT [6 5 4]; pop 4 (1)
- E E: IN [7 8] — 2 ops; D: pop 5 (1); E: IN [7 8 9] — 1 op

Totals: 9 enqueue pushes + 6 transferred × 2 + 5 dequeue pops = 9 + 12 + 5 = **26**.

Dequeue order 1, 2, 3, 4, 5 confirms FIFO behaviour. Element 6 is left on OUT and 7, 8, 9 on IN.

Amortised view: every element costs at most 4 stack operations over its lifetime, so any sequence of m queue operations costs $O(m)$ even though a single dequeue can cost $\Theta(n)$.

Trap: transferring on *every* dequeue (or moving elements back to IN afterwards) gives much larger counts; also do not forget that the transfer is 2 operations per element.

**Q.14 [MSQ · 2 marks · Monotonic stack — largest rectangle in a histogram]****Answer: (A), (B)**

Concept. When bar t is popped, the current index i is the first bar to its right that is lower, and the new stack top is the nearest lower bar to its left; so $H[t] \times w$ is the largest rectangle using bar t as its *shortest* bar.

Trace (pops with their areas):

- $i=2$ (4): pop 5 (w 1) → 5
- $i=4$ (3): pop 6 (w 1) → 6; pop 4 (index 2, $w = 4 - 0 - 1 = 3$) → **12**
- $i=5$ (1): pop 3 ($w = 5 - 0 - 1 = 4$) → **12**; pop 2 (stack empty, w 5) → 10
- $i=7$ (sentinel 0): pop 4 (w 1) → 4; pop 1 (w 7) → 7

Option by option:

- (A) Every one of the 7 real bars is pushed once and popped once (the sentinel is pushed but never popped) → pops = 7. **TRUE.**
- (B) best = 12. **TRUE.**
- (C) Two different rectangles reach 12: height 4 over bars 1–3 and height 3 over bars 1–4. **FALSE.**
- (D) At $i = 5$ the stack holds the bars of heights 2 and 3 only (5, 6, 4 were already popped earlier), so exactly **2** bars are popped. **FALSE.**

Trap: in (D), counting all bars taller than 1 to the left (4 of them) instead of those still on the stack.



Bars 1-4: height 3 $\times$ width 4 = 12 (also height 4 $\times$ bars 1-3 = 12)

Q.15 [MCQ · 2 marks · Queue simulation — elimination game]

Answer: (D)

Concept. `rotate(-3)` moves the three leftmost items to the right end; `pop()` then removes the rightmost item — which is the **third** of the three just moved. So each round eliminates every third person in a circle (a Josephus process with $k = 3$).

Circle 1..8, counting from the front:

- Round 1: [4 5 6 7 8 1 2 3] $\rightarrow$ remove **3**; front is 4.
- Round 2: count 4, 5, 6 $\rightarrow$ remove **6**; front 7.
- Round 3: count 7, 8, 1 $\rightarrow$ remove **1**; front 2.
- Round 4: 2, 4, 5 $\rightarrow$ remove 5. Round 5: 7, 8, 2 $\rightarrow$ remove 2. Round 6: 4, 7, 8 $\rightarrow$ remove 8. Round 7: 4, 7, 4 $\rightarrow$ remove 4.
- Survivor: **7**.

Elimination order 3, 6, 1, 5, 2, 8, 4 $\rightarrow$ printed [3, 6, 1] 7 $\rightarrow$ (D).

Option by option:

- (A) uses `popleft()` instead of `pop()` (removes every 4th).
- (B) reports the last *eliminated* person (4) instead of the survivor.
- (C) rotates the wrong way (`rotate(3)`), counting backwards.

Tip: the Josephus recurrence $J(n) = (J(n-1) + k) \bmod n$ with $J(1) = 0$ gives $J(8) = 6$ (0-based) $\rightarrow$ person 7 $\checkmark$.

Q.16 [NAT · 2 marks · Quicksort with an explicit stack]

Answer: 4

Concept. Each processed range is replaced on the stack by two sub-ranges (even empty ones, which are popped and discarded later). Because the *right* part is pushed last, it is handled first; the stack grows while right parts keep splitting.

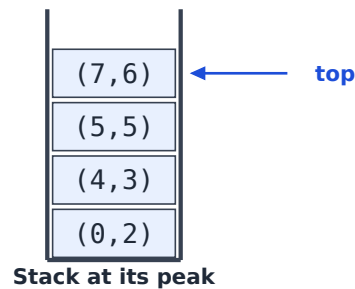
Trace (stack shown bottom $\rightarrow$ top after each step):

- Pop (0,6), pivot 4: $A = [1, 2, 3, \mathbf{4}, 7, 6, 5]$; push (0,2), (4,6) $\rightarrow$ 2 pairs.
- Pop (4,6) on [7, 6, 5], pivot 5: nothing $\leq 5 \rightarrow A[4..6] = [\mathbf{5}, 6, 7]$; push (4,3), (5,6) $\rightarrow$ stack (0,2), (4,3), (5,6): 3 pairs.
- Pop (5,6) on [6, 7], pivot 7: $6 \leq 7 \rightarrow [6, \mathbf{7}]$; push (5,5), (7,6) $\rightarrow$ stack (0,2), (4,3), (5,5), (7,6): **4 pairs** (maximum).
- Pop (7,6), (5,5), (4,3): all trivial $\rightarrow$ stack (0,2).
- Pop (0,2) on [1, 2, 3], pivot 3: push (0,1), (3,2) $\rightarrow$ 2 pairs; pop (3,2) trivial.
- Pop (0,1), pivot 2: push (0,0), (2,1) $\rightarrow$ 2 pairs; both trivial.

Maximum stack size = **4**.

Tip: pushing the *larger* part first (so the smaller part is processed next) bounds the stack size by $O(\log n)$; this code ignores sizes, so on a bad input the stack can grow to $\Theta(n)$.

Trap: forgetting that empty ranges like (4,3) and (7,6) are also pushed.

**Q.17 [MCQ · 2 marks · Trees — spiral traversal with two stacks]****Answer: (C)**

Concept. A stack reverses the order in which a level's children were pushed, so alternating the two stacks (and the push order) prints the levels in alternating directions — a spiral (zigzag) level order.

- Phase S1: print **44**; push 19, 63 onto S2 (63 on top).
- Phase S2: pop 63 → print, push 90, 51 onto S1; pop 19 → print, push 27, 8 onto S1. S1 (bottom→top) = 90, 51, 27, 8. Printed: **63 19**.
- Phase S1: pop 8 (push 4), 27 (push 23, 35), 51 (push 57), 90 (push 72). Printed **8 27 51 90**; S2 = 4, 23, 35, 57, 72.
- Phase S2: pop 72, 57, 35, 23, 4 → printed **72 57 35 23 4** (leaves, nothing pushed).

Output: 44 63 19 8 27 51 90 72 57 35 23 4 → (C).

Option by option:

- (A) is a spiral that starts left-to-right on level 1 (push orders swapped).
- (B) is the plain level order (a queue, not two stacks).
- (D) reverses level 1 but forgets that level 3 is reversed too.

Trap: level 2 is printed left-to-right because it was pushed right-then-left onto S1, and a stack reverses that.

Q.18 [MCQ · 2 marks · DFS with an explicit stack]**Answer: (A)**

Concept. With a stack, the neighbour pushed *last* (alphabetically largest) is explored first, so this DFS follows the reverse alphabetical choice at each vertex. Marking on pop allows duplicates on the stack; stale copies are skipped.

Stack trace (top at right):

- [A] → pop A; push B, C, E → [B, C, E]
- pop E; push B, F → [B, C, B, F]
- pop F; push C, D → [B, C, B, C, D]
- pop D; push B → [B, C, B, C, B]
- pop B (all its neighbours visited) → [B, C, B, C]
- pop C → [B, C, B]; the remaining B, C, B are already visited and skipped.

Order: **A E F D B C** → (A).

Option by option:

- (B) results if vertices are marked when *pushed* (then C is popped before B at the end).
- (C) picks C before E at A, which this stack order never does.
- (D) is the *recursive* DFS with alphabetical order.

Trap: an iterative DFS is not automatically the same as the recursive one; push order and marking time both matter.

Q.19 [MSQ · 2 marks · Linked lists — circular queue with tail pointer]**Answer: (B), (C)**

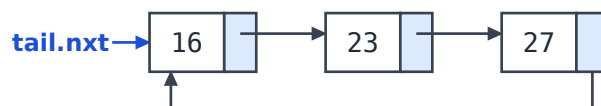
Concept. With only a tail pointer, both enqueue (insert after tail, then advance tail) and dequeue (unlink tail.nxt) are $O(1)$; the front is always `tail.nxt`.

- enq 4, 8, 15 $\rightarrow$ circle $4 \rightarrow 8 \rightarrow 15 \rightarrow$ (back to 4); tail = 15.
- `a = deq()` removes the front 4 $\rightarrow$ a = 4; circle $8 \rightarrow 15$.
- enq 16, 23 $\rightarrow 8 \rightarrow 15 \rightarrow 16 \rightarrow 23$; tail = 23.
- `deq() + deq()` = $8 + 15 = 23 = b$; circle $16 \rightarrow 23$.
- enq(a + b) = enq(27) $\rightarrow 16 \rightarrow 23 \rightarrow 27$; tail = 27.

Option by option:

- (A) The next dequeue returns the front, 16 — not 23. **FALSE.**
- (B) The front (`tail.nxt`) is 16. **TRUE.**
- (C) tail is the last enqueued node, 27. **TRUE.**
- (D) `tail.nxt.nxt` is the second element, 23. **FALSE.**

Trap: thinking that `a + b` is $4 + 8 = 12$ — `b` is the sum of *two* dequeues (8 and 15).



Final circular list (tail = node 27)

Q.20 [NAT · 2 marks · Dijkstra with a binary heap (lazy deletion)]**Answer: 14**

Concept. One push happens for the source plus one for *every successful relaxation*; the number of pushes is therefore $1 + (\text{number of times some } d[v] \text{ decreases})$.

- Pop S (0): A=4, B=1, C=6 $\rightarrow$ 3 pushes.
- Pop B (1): A=3 ✓, C=4 ✓, D=8 ✓ $\rightarrow$ 3 pushes.
- Pop A (3): D = min(8, 6) = 6 ✓, E = 11 ✓ $\rightarrow$ 2 pushes.
- Pop (4, A): stale ($d[A] = 3$) $\rightarrow$ skipped. Pop C (4): D = min(6, 5) = 5 ✓, F = 13 ✓ $\rightarrow$ 2.
- Pop D (5): E = min(11, 7) = 7 ✓, F = min(13, 11) = 11 ✓ $\rightarrow$ 2.
- Stale (6, C), (6, D) skipped. Pop E (7): F = min(11, 8) = 8 ✓ $\rightarrow$ 1.
- Pop (8, D): stale (ties are broken by vertex name). Pop F (8); the remaining entries (11, E), (11, F), (13, F) are stale.

Pushes = $1 + 3 + 3 + 2 + 2 + 2 + 1 = \mathbf{14}$. (Of these, 7 are popped as stale entries.)

Final distances: A 3, B 1, C 4, D 5, E 7, F 8.

Trap: answering $|V| = 7$ (one entry per vertex, as with a decrease-key heap). Lazy deletion may push up to $|E| + 1$ entries, giving $O(|E| \log |E|)$ time.

Distances after each valid pop

	S	A	B	C	D	E	F	pushes
S	0	4	1	6	∞	∞	∞	3
B	0	3	1	4	8	∞	∞	3
A	0	3	1	4	6	11	∞	2
C	0	3	1	4	5	11	13	2
D	0	3	1	4	5	7	11	2
E	0	3	1	4	5	7	8	1

Mock Test 21

Counting & Structure of Trees

Difficulty: GATE-level

Focus areas: number of BSTs/binary trees, complete/full trees, height bounds

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [NAT · numerical answer | 1 mark | Counting BSTs with a fixed root]

The number of distinct binary search trees that can be formed with the keys 1, 2, 3, 4, 5, 6 and whose **root is 3** is _____.

Answer: _____ (enter a numerical value)

Q.2 [MCQ · single correct | 1 mark | Binary trees — node-degree counting]

A binary tree has 20 leaves and exactly 9 nodes that have exactly one child. The total number of nodes in the tree is

- (A) 48
- (B) 47
- (C) 39
- (D) 57

Q.3 [MCQ · single correct | 1 mark | Python — recursion on a tuple-encoded tree]

A binary tree is encoded as nested tuples (**value**, **left**, **right**). Consider the following Python program. What is printed?

Python

```

1 def f(t, d=0):
2     if t is None:
3         return 0
4     v, l, r = t
5     return v * d + f(l, d + 1) + f(r, d + 1)
6
7 T = (5, (3, None, (4, None, None)),
8      (8, (7, None, None), None))
9 print(f(T), f(T, 1))

```

- (A) 33 55
- (B) 33 60
- (C) 27 60
- (D) 60 93

Q.4 [MSQ · one or more correct | 1 mark | Height bounds and complete binary trees]

Height is the number of **edges** on the longest root-to-leaf path. Which of the following statements is/are TRUE?

- (A) A complete binary tree with 50 nodes has exactly 25 leaves
- (B) A complete binary tree with 50 nodes has exactly one node with exactly one child
- (C) Every binary tree with 50 nodes has height at least 5
- (D) Inserting 50 distinct keys into an empty BST can produce a tree of height 50

Q.5 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation]

The postfix expression below is evaluated with an operand stack (for a binary operator, the first value popped is the **right** operand; / is exact division and all intermediate results here are integers):

7 3 - 2 4 * 5 - * 12 6 / 3 + +

The value of the expression is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Queues — Python deque operations]

Consider the following Python program. What is printed?

Python

```
1 from collections import deque
2 d = deque([1, 2, 3, 4, 5])
3 d.rotate(2)
4 d.appendleft(d.pop())
5 d.extend([6, 7])
6 d.rotate(-3)
7 x = d.popleft() + d.pop()
8 print(list(d), x)
```

(A) [6, 7, 3, 4, 5] 3

(B) [1, 6, 7, 2, 3] 9

(C) [2, 6, 7, 3, 4] 6

(D) [2, 6, 7, 3, 4] 8

Q.7 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

The keys 22, 33, 14, 44, 25, 3, 36 are inserted in that order into an initially empty table of 11 slots using quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) for key k examines slot $(k \bmod 11 + i^2) \bmod 11$. The index of the slot in which 36 is stored is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Binary search — decision tree structure]

Binary search with $\text{mid} = (\text{lo} + \text{hi}) // 2$ (three-way comparison at each probe, counted as one comparison) is performed on a sorted array of 20 distinct keys, once for each key in the array. For how many of the 20 keys does the successful search use **exactly 5** comparisons?

(A) 4

(B) 8

(C) 6

(D) 5

Q.9 [MCQ · single correct | 1 mark | Bubble sort — number of swaps]

Bubble sort (repeatedly swapping adjacent out-of-order elements until no swap occurs) is used to sort [3, 7, 1, 8, 2, 6] into ascending order. The total number of swaps performed is

(A) 5

(B) 6

(C) 8

(D) 7

Q.10 [MSQ · one or more correct | 1 mark | Graph theory — trees]

G is a **connected** simple undirected graph with 10 vertices and 9 edges. Which of the following statements is/are TRUE for every such G?

- (A) G has a vertex of degree at least 3
- (B) The sum of the degrees of the vertices of G is 20
- (C) G has at least two vertices of degree 1
- (D) G contains no cycle

Q.11 [NAT · numerical answer | 2 marks | Counting BSTs of maximum height]

Height is measured in edges. The number of distinct binary search trees on the keys 1, 2, ..., 7 that have height exactly 6 is _____.

Answer: _____ (enter a numerical value)

Q.12 [NAT · numerical answer | 2 marks | Counting binary trees by height (Python DP)]

The function `c(n, h)` below counts binary trees (shapes) with n nodes and height at most h , where height is in edges and the empty tree has height -1 . The value printed is _____.

Python

```
1 from functools import lru_cache
2
3 @lru_cache(None)
4 def c(n, h):
5     if n == 0:
6         return 1
7     if h < 0:
8         return 0
9     return sum(c(k, h - 1) * c(n - 1 - k, h - 1)
10              for k in range(n))
11
12 print(c(7, 3) - c(7, 2))
```

Answer: _____ (enter a numerical value)

Q.13 [MCQ · single correct | 2 marks | Tree reconstruction from traversals]

A binary tree with nine distinct labels has

pre-order: G, C, A, E, K, J, H, B, F

in-order: A, E, C, K, G, H, J, F, B

Its post-order traversal is

- (A) A, E, K, C, H, F, B, J, G
- (B) E, A, K, C, H, F, B, J, G
- (C) E, A, K, C, F, B, H, J, G
- (D) E, A, C, K, H, B, F, J, G

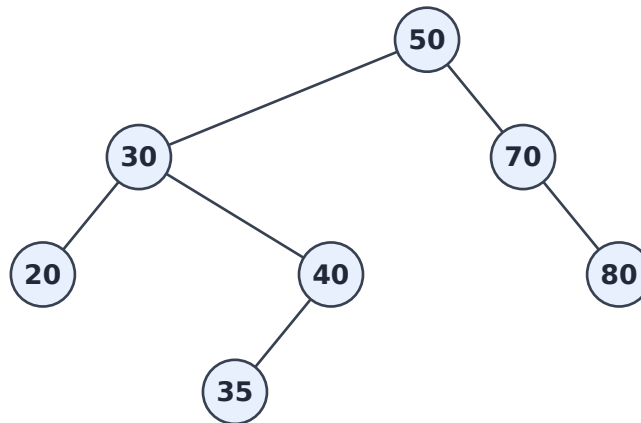
Q.14 [MSQ · one or more correct | 2 marks | Heaps — array layout of a complete tree]

A binary max-heap stores 100 **distinct** keys in an array $A[0 \dots 99]$ (children of index i at $2i + 1$ and $2i + 2$; depth of the root is 0). Which of the following statements is/are TRUE?

- (A) The deepest level (depth 6) contains exactly 36 keys
- (B) Depending on the keys, the minimum key can be at any of exactly 50 indices, and at no other index
- (C) The children of index 24 are at indices 49 and 50
- (D) Index 49 holds a leaf

Q.15 [NAT · numerical answer | 2 marks | Counting insertion orders that yield a BST]

The number of distinct orderings of the keys {20, 30, 35, 40, 50, 70, 80} which, when inserted one by one into an initially empty binary search tree, produce exactly the BST shown below is _____.



Target BST

Answer: _____ (enter a numerical value)

Q.16 [MCQ · single correct | 2 marks | Quicksort — comparisons via the recursion tree]

Consider the following quicksort, which uses the first element as the pivot and builds the two partitions with list comprehensions (preserving relative order). Each call on a list of length m is charged $m - 1$ comparisons. What is printed?

Python

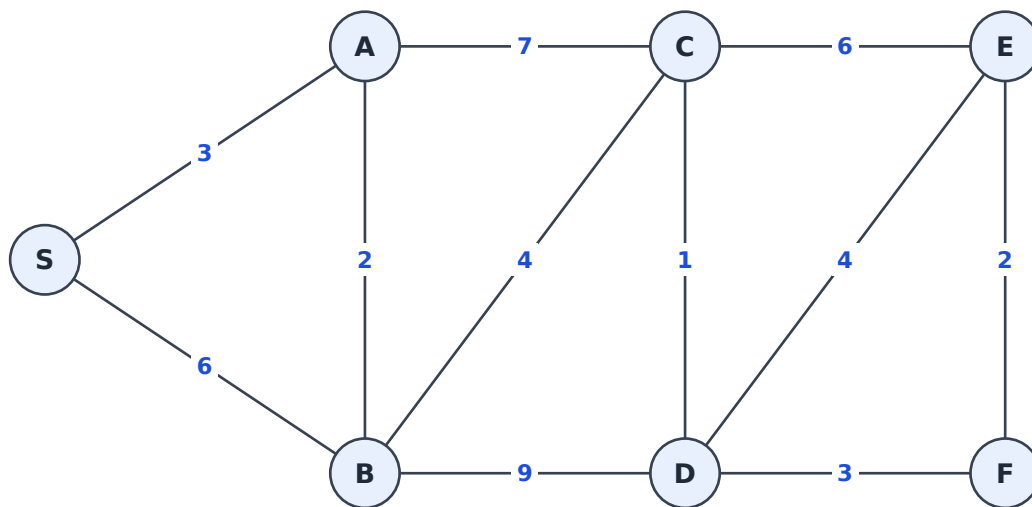
```

1  cmp = 0
2
3  def qs(a):
4      global cmp
5      if len(a) <= 1:
6          return a
7      p = a[0]
8      cmp += len(a) - 1
9      lo = [x for x in a[1:] if x < p]
10     hi = [x for x in a[1:] if x > p]
11     return qs(lo) + [p] + qs(hi)
12
13 qs([5, 3, 8, 1, 4, 7, 9, 2, 6])
14 print(cmp)
  
```

- (A) 16
- (B) 13
- (C) 21
- (D) 36

Q.17 [MSQ · one or more correct | 2 marks | Dijkstra — shortest-path tree]

Dijkstra's algorithm is run from S on the weighted undirected graph below. Which of the following statements is/are TRUE?



- (A) The edge A-B belongs to the shortest-path tree
- (B) The shortest-path distance from S to F is 13
- (C) The shortest-path tree has exactly two leaves
- (D) E is finalised (extracted) before F

Q.18 [MSQ · one or more correct | 2 marks | Merge sort — shape of the recursion tree]

Top-down merge sort is called on an array of $n = 37$ elements. Each call on a subarray of length $m \geq 2$ makes two recursive calls on lengths $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$ and then one merge; a call on length 1 returns immediately. Consider the recursion tree (one node per call; the root is at depth 0). Which of the following statements is/are TRUE?

- (A) The recursion tree has exactly 73 nodes
- (B) Exactly 18 of the merges combine two runs of equal length
- (C) Exactly 27 of the length-1 calls occur at depth 5
- (D) The recursion tree has height 5

Q.19 [MCQ · single correct | 2 marks | Building a balanced BST from a sorted array]

Consider the following Python program. What is printed?

Python

```

1 def build(a, lo, hi):
2     if lo > hi:
3         return None
4     m = (lo + hi + 1) // 2
5     return (a[m], build(a, lo, m - 1),
6           build(a, m + 1, hi))
7
8 def pre(t):
9     if t is None:
10        return []
11    return [t[0]] + pre(t[1]) + pre(t[2])
12
13 a = list(range(10, 101, 10))
14 print(pre(build(a, 0, len(a) - 1)))

```

- (A) [50, 20, 10, 30, 40, 80, 60, 70, 90, 100]
- (B) [60, 30, 20, 10, 40, 50, 90, 80, 70, 100]
- (C) [60, 30, 20, 10, 50, 40, 90, 80, 70, 100]
- (D) [60, 30, 10, 20, 50, 40, 90, 70, 80, 100]

Q.20 [NAT · numerical answer | 2 marks | Linked lists — reversal in groups]

Consider the following Python program. The value printed is _____.

Python

```

1 class N:
2     def __init__(self, v, nxt=None):
3         self.v, self.n = v, nxt
4
5 def rev_k(h, k):
6     prev, cur, cnt = None, h, 0
7     while cur and cnt < k:
8         cur.n, prev, cur = prev, cur, cur.n
9         cnt += 1
10    if cur:
11        h.n = rev_k(cur, k)
12    return prev
13
14 head = None
15 for v in reversed([4, 9, 1, 7, 3, 8, 2, 5]):
16     head = N(v, head)
17 head = rev_k(head, 3)
18 s = 0
19 while head:
20     s = s * 10 + head.v
21     head = head.n
22 print(s)

```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 21

Q	Type	Marks	Key	Topic
1	NAT	1	10	Counting BSTs with a fixed root
2	MCQ	1	(A)	Binary trees — node-degree counting
3	MCQ	1	(B)	Python — recursion on a tuple-encoded tree
4	MSQ	1	(A), (B), (C)	Height bounds and complete binary trees
5	NAT	1	17	Stacks — postfix evaluation
6	MCQ	1	(C)	Queues — Python deque operations
7	NAT	1	6	Hashing — quadratic probing
8	MCQ	1	(D)	Binary search — decision tree structure
9	MCQ	1	(D)	Bubble sort — number of swaps
10	MSQ	1	(C), (D)	Graph theory — trees
11	NAT	2	64	Counting BSTs of maximum height
12	NAT	2	68	Counting binary trees by height (Python DP)
13	MCQ	2	(B)	Tree reconstruction from traversals
14	MSQ	2	(B), (C)	Heaps — array layout of a complete tree
15	NAT	2	45	Counting insertion orders that yield a BST
16	MCQ	2	(A)	Quicksort — comparisons via the recursion tree
17	MSQ	2	(A), (B), (C)	Dijkstra — shortest-path tree
18	MSQ	2	(A), (C)	Merge sort — shape of the recursion tree
19	MCQ	2	(C)	Building a balanced BST from a sorted array
20	NAT	2	19483752	Linked lists — reversal in groups

Detailed Solutions — Mock Test 21

Q.1 [NAT · 1 mark · Counting BSTs with a fixed root]

Answer: 10

In a BST the root splits the keys: everything smaller goes to the left subtree, everything larger to the right, and the two subtrees can be chosen independently. The number of BSTs on m keys is the Catalan number $C_m = (2m)! / ((m+1)! m!)$: $C_0 = 1$, $C_1 = 1$, $C_2 = 2$, $C_3 = 5$, $C_4 = 14$, $C_5 = 42$.

- Root 3 → left subtree on $\{1, 2\}$ (2 keys): $C_2 = 2$ shapes.
- Right subtree on $\{4, 5, 6\}$ (3 keys): $C_3 = 5$ shapes.

Total = $2 \times 5 = 10$.

Sanity check: summing over all six roots gives $C_0C_5 + C_1C_4 + C_2C_3 + C_3C_2 + C_4C_1 + C_5C_0 = 42 + 14 + 10 + 10 + 14 + 42 = 132 = C_6$.

Trap: adding instead of multiplying ($2 + 5 = 7$) — the choices for the two subtrees are independent, so the counts multiply.

Q.2 [MCQ · 1 mark · Binary trees — node-degree counting]

Answer: (A)

Let n_0, n_1, n_2 be the numbers of nodes with 0, 1, 2 children. Counting edges two ways: every node except the root has one parent edge, so $n - 1 = n_1 + 2n_2$, and $n = n_0 + n_1 + n_2$. Subtracting gives the key identity **$n_0 = n_2 + 1$** .

- $n_2 = n_0 - 1 = 20 - 1 = 19$.
- $n = n_0 + n_1 + n_2 = 20 + 9 + 19 = 48 \rightarrow (A)$.

Check with edges: $n - 1 = 47 = n_1 + 2n_2 = 9 + 38$. ✓

Option analysis:

- (B) 47 is the number of edges, not nodes.
- (C) 39 ignores the one-child nodes ($20 + 19$).
- (D) 57 uses $n_2 = n_0 + n_1 - 1 = 28$ (wrong identity) giving $20 + 9 + 28$.

Tip: the number of one-child nodes never affects n_2 ; it just adds to n .

Q.3 [MCQ · 1 mark · Python — recursion on a tuple-encoded tree]

Answer: (B)

$f(t, d)$ returns $\sum \text{value} \times (d + \text{depth of the node})$, i.e. a depth-weighted sum where the root is given weight d .

Depths: 5 at 0; 3 and 8 at 1; 4 and 7 at 2.

- $f(T, d=0)$: $5 \cdot 0 + 3 \cdot 1 + 8 \cdot 1 + 4 \cdot 2 + 7 \cdot 2 = 0 + 3 + 8 + 8 + 14 = 33$.
- $f(T, 1)$: every weight is one larger, so the result increases by the sum of all values $5 + 3 + 8 + 4 + 7 = 27$: $33 + 27 = 60$.

Output **33 60** → (B).

- (A) adds only 22 (forgets the root's own contribution $5 \cdot 1$).
- (C) prints the plain sum of values 27 for the first call.
- (D) shifts both calls by one level.

Tip: $f(T, d+1) - f(T, d)$ always equals the sum of all values — a quick consistency check.

Q.4 [MSQ · 1 mark · Height bounds and complete binary trees]

Answer: (A), (B), (C)

A binary tree of height h has at most $2^{h+1} - 1$ nodes, so n nodes force $h \geq \lceil \log_2(n+1) \rceil - 1$.

- (A) In the array layout (0-indexed) node i has a child iff $2i + 1 < n$, i.e. $i \leq 24$. So nodes 0 ... 24 are internal and 25 ... 49 are leaves: 25 leaves ($\lceil n/2 \rceil$ in general). **TRUE.**
- (B) Node 24 has left child 49 but no right child (index 50 does not exist); every other internal node has two children. Exactly one node with one child (n even). **TRUE.**
- (C) Height 4 allows at most 31 nodes < 50 , height 5 allows 63. So $h \geq 5$. **TRUE.**
- (D) A degenerate BST (sorted insertion) is a path of 50 nodes, which has **49** edges. Height 50 is impossible. **FALSE.**

Trap: mixing up height in edges vs nodes — 50 is the height counted in nodes.

Q.5 [NAT · 1 mark · Stacks — postfix evaluation]

Answer: 17

Scan left to right: push operands; for an operator pop b (right) then a (left) and push $a \text{ op } b$.

- $7, 3 \rightarrow [7, 3]; - \rightarrow 7 - 3 = 4 \rightarrow [4]$
- $2, 4 \rightarrow [4, 2, 4]; * \rightarrow 8 \rightarrow [4, 8]$
- $5 \rightarrow [4, 8, 5]; - \rightarrow 8 - 5 = 3 \rightarrow [4, 3]$
- $* \rightarrow 4 \times 3 = 12 \rightarrow [12]$
- $12, 6 \rightarrow [12, 12, 6]; / \rightarrow 2 \rightarrow [12, 2]$
- $3 \rightarrow [12, 2, 3]; + \rightarrow 5 \rightarrow [12, 5]$
- $+ \rightarrow 17 \rightarrow [17]$

Value = **17**. (The stack never holds more than 3 values.)

The expression tree's infix form is $((7 - 3) \times (2 \times 4 - 5)) + (12 / 6 + 3)$.

Trap: popping in the wrong order gives $3 - 7 = -4$ and $6/12$, producing a completely different value.

Q.6 [MCQ · 1 mark · Queues — Python deque operations]

Answer: (C)

`rotate(k)` with $k > 0$ moves the last k elements to the front (right rotation); $k < 0$ rotates left. `pop()` removes from the right, `popleft()` from the left.

- Start: $[1, 2, 3, 4, 5]$
- `rotate(2)`: $[4, 5, 1, 2, 3]$
- `appendleft(pop())`: `pop() = 3` $\rightarrow [3, 4, 5, 1, 2]$
- `extend([6, 7])`: $[3, 4, 5, 1, 2, 6, 7]$
- `rotate(-3)`: $[1, 2, 6, 7, 3, 4, 5]$
- $x = \text{popleft()} + \text{pop()} = 1 + 5 = 6 \rightarrow d = [2, 6, 7, 3, 4]$

Output **`[2, 6, 7, 3, 4]`** $6 \rightarrow (C)$.

- (A) treats `rotate(-3)` as a right rotation.
- (B) treats `rotate(2)` as a left rotation.
- (D) has the right deque but adds $1 + 7$ (the last element before the rotate).

Tip: a right rotation by k of length n equals a left rotation by $n - k$.

Q.7 [NAT · 1 mark · Hashing — quadratic probing]

Answer: 6

Quadratic probing visits $h, h + 1, h + 4, h + 9, h + 16, h + 25, \dots \pmod{11}$, so keys with the same home slot follow the same probe sequence (secondary clustering).

- $22 \rightarrow h = 0 \rightarrow \text{slot } 0$.
- $33 \rightarrow h = 0$ occupied; $0 + 1 = 1 \rightarrow \text{slot } 1$.
- $14 \rightarrow h = 3 \rightarrow \text{slot } 3$.

- $44 \rightarrow h = 0$: 0, 1 occupied; $0 + 4 = 4 \rightarrow$ slot 4.
- $25 \rightarrow h = 3$: 3, 4 occupied; $3 + 4 = 7 \rightarrow$ slot 7.
- $3 \rightarrow h = 3$: 3, 4, 7 occupied; $3 + 9 = 12 \equiv 1$ occupied; $3 + 16 = 19 \equiv 8 \rightarrow$ slot 8.
- $36 \rightarrow h = 3$: probes 3, 4, 7, 1, 8 all occupied; $3 + 25 = 28 \equiv 6 \rightarrow$ slot 6.

36 needs 6 probes even though the table is only about half full — the keys 14, 25, 3, 36 all share home slot 3 and hence the same probe sequence.

Trap: using linear steps ($h + i$) would put 36 in slot 5.

0	22
1	33
2	
3	14
4	44
5	
6	36
7	25
8	3
9	
10	

Final table

Q.8 [MCQ · 1 mark · Binary search — decision tree structure]

Answer: (D)

The probes made by binary search form a *decision tree*: the first mid is the root, the left/right halves are its subtrees, and a key found at depth d needs $d + 1$ comparisons. Because the halves differ in size by at most one at every step, this tree is balanced: all levels except the last are full.

- Depth 0: 1 key (1 comparison)
- Depth 1: 2 keys (2 comparisons)
- Depth 2: 4 keys (3)
- Depth 3: 8 keys (4)
- Depth 4: the remaining $20 - 15 = 5$ keys (5 comparisons)

So 5 keys need exactly 5 comparisons $\rightarrow$ (D). (Average successful cost = $(1 + 4 + 12 + 32 + 25)/20 = 3.7$.)

- (A) and (C) are off by one in the level counts.
- (B) is the size of the last full level (depth 3).

Tip: for n keys, the worst case is $\lceil \log_2 n \rceil + 1 = 5$ and the number of keys on the last level is $n - (2^{\lceil \log_2 n \rceil} - 1)$.

Q.9 [MCQ · 1 mark · Bubble sort — number of swaps]

Answer: (D)

Each adjacent swap of an out-of-order pair removes **exactly one** inversion, and the algorithm stops when no inversions remain. Therefore $\#swaps = \#inversions$, no matter in which order the passes visit the pairs.

Inversions of [3, 7, 1, 8, 2, 6]:

- $3 > 1, 2 \rightarrow 2$

- $7 > 1, 2, 6 \rightarrow 3$
- $1 \rightarrow 0$
- $8 > 2, 6 \rightarrow 2$
- $2 \rightarrow 0$

Total = **7** $\rightarrow$ (D).

Pass-by-pass check: pass 1 swaps (7,1), (8,2), (8,6) $\rightarrow$ [3, 1, 7, 2, 6, 8]; pass 2 swaps (3,1), (7,2), (7,6) $\rightarrow$ [1, 3, 2, 6, 7, 8]; pass 3 swaps (3,2) $\rightarrow$ sorted. $3 + 3 + 1 = 7$.

- (A), (B), (C) come from counting passes or comparisons instead of swaps.

Tip: insertion sort also performs exactly #inversions shifts.

Q.10 [MSQ · 1 mark · Graph theory — trees]

Answer: (C), (D)

A connected graph on n vertices with $n - 1$ edges is a **tree** (connected + $n - 1$ edges $\Leftrightarrow$ connected + acyclic).

- (A) The path P_{10} is a tree with maximum degree 2. **FALSE.**
- (B) $\sum \deg = 2|E| = 18$, not 20. **FALSE.**
- (C) Every tree with $n \geq 2$ vertices has at least two leaves: the two endpoints of a longest path cannot have any other neighbour (that would extend the path or close a cycle). **TRUE.**
- (D) Trees are acyclic. **TRUE.**

Extra: since $\sum \deg = 18 = 2(n - 1)$, the average degree is $1.8 < 2$, which is another way to see that some vertices must have degree 1.

Trap: confusing $|E| = n - 1$ with $|E| = n$ (which would give 20 in (B) and force a cycle).

Q.11 [NAT · 2 marks · Counting BSTs of maximum height]

Answer: 64

Height 6 with 7 nodes means the tree is a **path**: every node except the last has exactly one child.

Build the path from the root downward. At each step the current node splits the remaining keys into smaller and larger ones; for the tree to stay a path, all remaining keys must lie on one side, so the current node must be the **minimum or the maximum** of the keys still remaining.

- Root: 2 choices (1 or 7).
- Each of the next nodes: again min or max of what remains $\rightarrow$ 2 choices.
- The last node (one key left): 1 choice.

Total = $2^6 = \mathbf{64}$ (in general 2^{n-1} BSTs of height $n - 1$).

Example: root 7 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 6 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4 is valid (the zig-zag path), while root 4 is impossible because both sides would be non-empty.

These are exactly the shapes produced by the 2^{n-1} insertion orders in which every inserted key is the current min or max of the not-yet-inserted keys.

Trap: answering 2 (only the two sorted orders) or $7!$ / something — zig-zag paths count too.

Q.12 [NAT · 2 marks · Counting binary trees by height (Python DP)]

Answer: 68

$c(7, 3) - c(7, 2)$ is the number of 7-node binary trees of height **exactly 3**.

- $c(7, 2) = 1$: height ≤ 2 holds at most 7 nodes, so only the perfect tree qualifies.
- $c(7, 3)$: root + a left subtree of k nodes and a right subtree of $6 - k$ nodes, each of height ≤ 2 .

First tabulate $a(m) = c(m, 2)$ (trees with m nodes and height ≤ 2) and $b(m) = c(m, 1)$:

- $b(0..3) = 1, 1, 2, 1$ (height ≤ 1 holds at most 3 nodes).

- $a(m) = \sum_k b(k) b(m-1-k)$: $a(0) = 1$, $a(1) = 1$, $a(2) = 2$, $a(3) = 5$, $a(4) = 6$, $a(5) = 6$, $a(6) = 4$, $a(7) = 1$.

Then $c(7, 3) = \sum_{k=0..6} a(k) a(6-k) = 1 \cdot 4 + 1 \cdot 6 + 2 \cdot 6 + 5 \cdot 5 + 6 \cdot 2 + 6 \cdot 1 + 4 \cdot 1 = 4 + 6 + 12 + 25 + 12 + 6 + 4 = 69$.

Answer = $69 - 1 = 68$.

Sanity checks: $a(3) = 5 = C_3$ (all 3-node trees have height ≤ 2); $a(4) = 6$ because of the $C_4 = 14$ four-node trees, 8 are paths of height 3.

Trap: forgetting that the empty subtree ($k = 0$) is allowed — its count is 1, not 0 — which drops the $k = 0$ and $k = 6$ terms.

Q.13 [MCQ · 2 marks · Tree reconstruction from traversals]

Answer: (B)

The first pre-order element is the root; its position in the in-order sequence splits the remaining labels into left and right subtrees. Recurse.

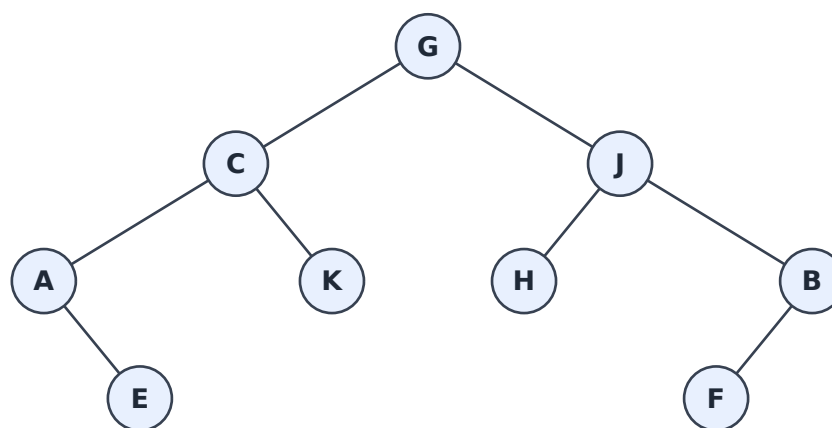
- Root G. In-order left of G: A, E, C, K; right: H, J, F, B.
- Left subtree: pre-order C, A, E, K $\rightarrow$ root C; in-order A, E | C | K $\rightarrow$ left {A, E}, right {K}. In {A, E}: pre-order A, E $\rightarrow$ root A; in-order A, E $\rightarrow$ E is A's **right** child.
- Right subtree: pre-order J, H, B, F $\rightarrow$ root J; in-order H | J | F, B $\rightarrow$ left {H}, right {F, B}. In {F, B}: pre-order B, F $\rightarrow$ root B; in-order F, B $\rightarrow$ F is B's **left** child.

Post-order (left, right, root): E, A, K, C | H, F, B, J | G $\rightarrow$ (B).

Option analysis:

- (A) lists A before its child E — a parent always follows its children in post-order.
- (B) **Correct.**
- (C) visits J's right subtree (F, B) before its left child H.
- (D) puts C before K, but K is in C's subtree, and B before F.

Tip: the last post-order element must equal the first pre-order element (G); use such checks to eliminate options quickly.



Reconstructed tree

Q.14 [MSQ · 2 marks · Heaps — array layout of a complete tree]

Answer: (B), (C)

In a complete tree with $n = 100$ nodes, depths 0 ... 5 are full ($1 + 2 + \dots + 32 = 63$ nodes) and the remaining 37 nodes sit on depth 6. Node i is internal iff $2i + 1 \leq n - 1 = 99$, i.e. $i \leq 49$.

- (A) Depth 6 contains $100 - 63 = 37$ keys, not 36. **FALSE.**

- (B) In a max-heap every internal node is larger than its child, so the minimum must be a leaf: indices 50 ... 99, i.e. 50 positions. Conversely each leaf can hold the minimum (no ordering constraint between a leaf and other leaves). **TRUE.**
- (C) $2 \cdot 24 + 1 = 49$ and $2 \cdot 24 + 2 = 50$. **TRUE.**
- (D) Index 49 has a left child at 99 (but no right child, since 100 is out of range). It is an internal node. **FALSE.**

Trap: in (D), index 49 has exactly one child — the unique one-child node of a complete tree with an even number of nodes — so it is *not* a leaf.

	0	1	2	3	4	5	6
nodes	1	2	4	8	16	32	37

Q.15 [NAT · 2 marks · Counting insertion orders that yield a BST]

Answer: 45

An insertion order produces a given BST iff every node is inserted **before all of its descendants**. Equivalently, the root comes first, and the orders for the left and right subtrees can be interleaved arbitrarily. So

$$N(T) = C(|L| + |R|, |L|) \times N(L) \times N(R).$$

- Subtree 40 (child 35): 40 must precede 35 $\rightarrow N = 1$.
- Subtree 30: left {20} (size 1), right {40, 35} (size 2) $\rightarrow C(3, 1) \times 1 \times 1 = 3$.
- Subtree 70 (child 80): $N = 1$.
- Root 50: left size 4, right size 2 $\rightarrow C(6, 2) \times 3 \times 1 = 15 \times 3 = \mathbf{45}$.

Check by the hook-length style formula: $N = n! / \prod(\text{subtree sizes}) = 7! / (7 \cdot 4 \cdot 1 \cdot 2 \cdot 1 \cdot 2 \cdot 1) = 5040 / 112 = 45$. ✓

Trap: counting only the interleavings at the root (15) and forgetting the factor 3 from the 30-subtree, or treating the order within each subtree as free (giving $6!/(\dots)$ too large).

Q.16 [MCQ · 2 marks · Quicksort — comparisons via the recursion tree]

Answer: (A)

With the first element as pivot and order-preserving partitions, the pivots of the recursive calls form exactly the **BST obtained by inserting the input in order** 5, 3, 8, 1, 4, 7, 9, 2, 6. A call whose pivot is node v handles v 's whole subtree, charging (subtree size $- 1$). Summing over all nodes, each key is charged once per proper ancestor, so

total comparisons = $\sum \text{depth}(v)$ = internal path length of the BST.

BST: 5 $\rightarrow$ left 3 (left 1 $\rightarrow$ right 2; right 4), right 8 (left 7 $\rightarrow$ left 6; right 9).

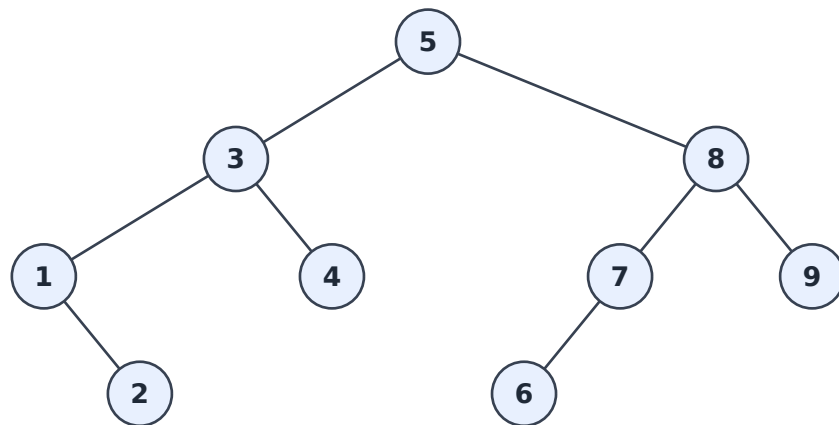
- Depth 1: 3, 8 $\rightarrow 2$
- Depth 2: 1, 4, 7, 9 $\rightarrow 8$
- Depth 3: 2, 6 $\rightarrow 6$

Total = $2 + 8 + 6 = \mathbf{16} \rightarrow \text{(A)}$.

Call-by-call: [9 keys] 8, [3,1,4,2] 3, [1,2] 1, [8,7,9,6] 3, [7,6] 1 $\rightarrow 8 + 3 + 1 + 3 + 1 = 16$.

- (B) 13 is impossible: any 9-node binary tree has internal path length at least $0 + 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 2 = 16$, so this input is in fact a best case.
- (C) 21 charges m instead of $m - 1$ for each of the 5 calls with $m \geq 2$.
- (D) $36 = 9 \cdot 8/2$ is the worst case (sorted input).

Tip: quicksort's comparison count and BST-insertion cost are the same random variable — which is why both average $\Theta(n \log n)$.



Recursion tree of pivots (= BST of the input order)

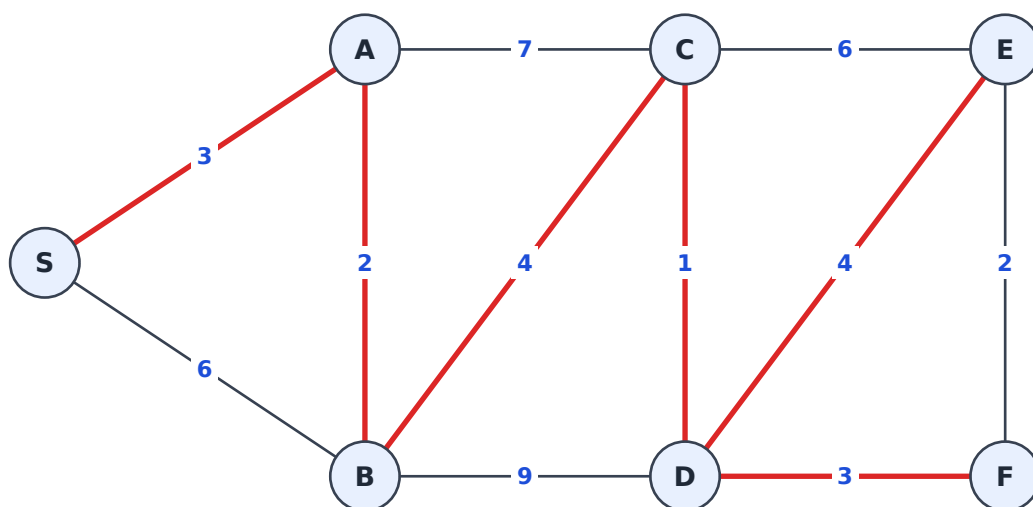
Q.17 [MSQ · 2 marks · Dijkstra — shortest-path tree]**Answer: (A), (B), (C)**

Run Dijkstra (tentative distances updated on extraction):

- S (0): A = 3, B = 6.
- A (3): B = $\min(6, 5) = 5$ (parent A), C = 10.
- B (5): C = $\min(10, 9) = 9$ (parent B), D = 14.
- C (9): D = $\min(14, 10) = 10$ (parent C), E = 15.
- D (10): F = 13, E = $\min(15, 14) = 14$ (parent D).
- F (13): E = $\min(14, 15) = 14$ (unchanged).
- E (14).

Final: S 0, A 3, B 5, C 9, D 10, F 13, E 14. Tree edges: S-A, A-B, B-C, C-D, D-F, D-E.

- (A) B's parent is A ($3 + 2 = 5 < 6$). **TRUE.**
- (B) $d(F) = 13$. **TRUE.**
- (C) The tree is the path S-A-B-C-D with two branches F and E hanging off D: leaves are E and F only. **TRUE.**
- (D) F (13) is extracted before E (14). **FALSE.**

Trap: taking the direct edges S-B (6) or C-E (6) at face value; both are improved by longer, cheaper routes.

Shortest-path tree (highlighted)

Q.18 [MSQ · 2 marks · Merge sort — shape of the recursion tree]**Answer: (A), (C)**

The tree is a full binary tree whose leaves are the n length-1 calls, so it has $n - 1 = 36$ internal nodes (merges) and $2n - 1$ nodes. Splitting into $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$ keeps all leaves within two consecutive depths, $\lfloor \log_2 n \rfloor$ and $\lceil \log_2 n \rceil$.

- (A) $2 \cdot 37 - 1 = 73$. **TRUE.**
- (B) Equal-length merges happen exactly at the internal calls with **even** m . Sizes by depth: 37 | 18, 19 | 9, 9, 9, 10 | 4, 5 × 3 and 5, 5 | ... Counting all internal calls with even m gives 21, not 18. **FALSE.**
- (C) Depth 5 has 32 nodes (all depths ≤ 5 are full). Let x of them have length 2; then the 32 sizes add to 37, so $x = 5$. Those 5 calls create 10 leaves at depth 6, and the other $32 - 5 = 27$ depth-5 calls are leaves. **TRUE.**
- (D) $2^5 = 32 < 37$, so some length-2 calls still exist at depth 5 and their children lie at depth 6 = $\lceil \log_2 37 \rceil$. Height 6. **FALSE.**

Trap: assuming the tree for 37 elements has height $\lceil \log_2 37 \rceil = 5$ — the leftover length-2 subarrays add one more level.

Calls per depth (n = 37)							
	0	1	2	3	4	5	6
calls	1	2	4	8	16	32	10
leaves	0	0	0	0	0	27	10

Q.19 [MCQ · 2 marks · Building a balanced BST from a sorted array]**Answer: (C)**

$m = (\text{lo} + \text{hi} + 1) // 2$ picks the **upper** middle of each range. The result is a height-balanced BST; pre-order lists each root before its subtrees.

$a = [10, 20, \dots, 100]$, indices 0 ... 9.

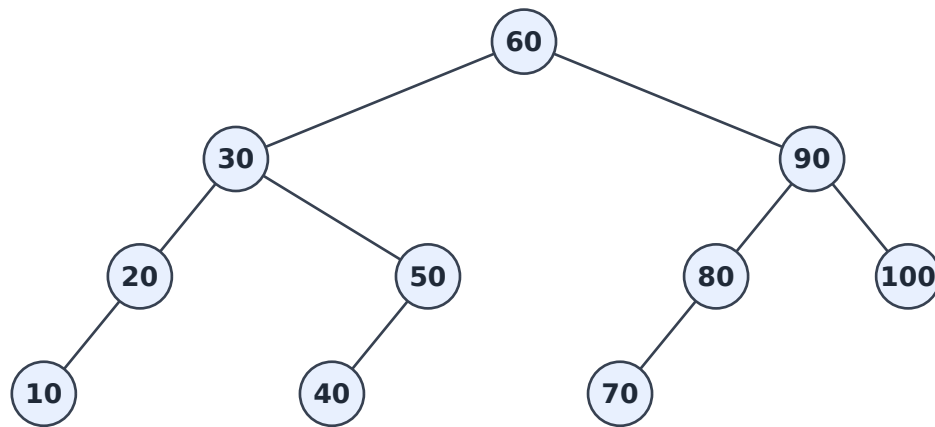
- build(0, 9): $m = 5 \rightarrow 60$; left = build(0, 4), right = build(6, 9).
- build(0, 4): $m = 2 \rightarrow 30$; left build(0, 1): $m = 1 \rightarrow 20$ with left child 10; right build(3, 4): $m = 4 \rightarrow 50$ with left child 40.
- build(6, 9): $m = 8 \rightarrow 90$; left build(6, 7): $m = 7 \rightarrow 80$ with left child 70; right build(9, 9) $\rightarrow 100$.

Pre-order: 60, 30, 20, 10, 50, 40, 90, 80, 70, 100 $\rightarrow$ (C).

- (A) is the result with the lower middle $(\text{lo} + \text{hi}) // 2$.
- (B) lists 40 before its parent 50.
- (D) makes 10 the parent of 20 (lower middle at one level only).

The tree has height 3 (edges) = $\lceil \log_2 11 \rceil - 1$, the minimum for 10 nodes.

Trap: with an even-length range the choice of middle changes the whole shape.



Tree built with the upper middle

Q.20 [NAT · 2 marks · Linked lists — reversal in groups]**Answer: 19483752**

`rev_k` reverses the first k nodes in place, then (if nodes remain) recursively processes the rest and hangs it after the old first node `h`, which is now the last node of its group. A final short group is also reversed.

The tuple assignment `cur.n, prev, cur = prev, cur, cur.n` is safe: the right side (old `prev`, old `cur`, old `cur.n`) is evaluated completely first, and the targets are then assigned left to right — `cur.n` is set while `cur` still names the old node.

The list is $4 \rightarrow 9 \rightarrow 1 \rightarrow 7 \rightarrow 3 \rightarrow 8 \rightarrow 2 \rightarrow 5$. Groups of 3:

- $[4, 9, 1] \rightarrow 1, 9, 4$
- $[7, 3, 8] \rightarrow 8, 3, 7$
- $[2, 5]$ (short group) $\rightarrow 5, 2$

Result: $1 \rightarrow 9 \rightarrow 4 \rightarrow 8 \rightarrow 3 \rightarrow 7 \rightarrow 5 \rightarrow 2$, and the loop reads it as the decimal number **19483752**.

Traps:

- Leaving the final short group unreversed (LeetCode-style) would give 19483725.
- Writing the assignment as `cur, cur.n, prev = cur.n, prev, cur` breaks the list: `cur` is rebound first, so `cur.n = prev` modifies the wrong node.

After `rev_k(head, 3)`

Mock Test 22

DAGs & Topological Sorting

Difficulty: GATE-level

Focus areas: topological orders count, DFS edge classification, cycle detection

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — generator exhaustion]

Consider the following Python program. What is printed?

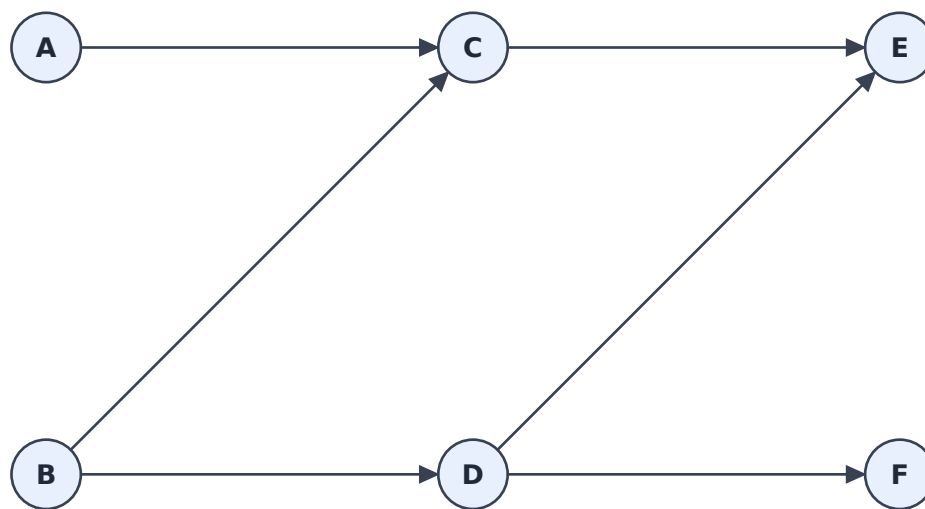
Python

```
1 g = (x * x for x in range(6) if x % 2 == 0)
2 s = sum(g)
3 print(s, list(g), max(g, default=-1))
```

- (A) 20 [0, 4, 16] 16
 (B) 20 [] -1
 (C) 55 [] -1
 (D) A `ValueError` is raised by `max`

Q.2 [NAT · numerical answer | 1 mark | Topological sorting — counting orders]

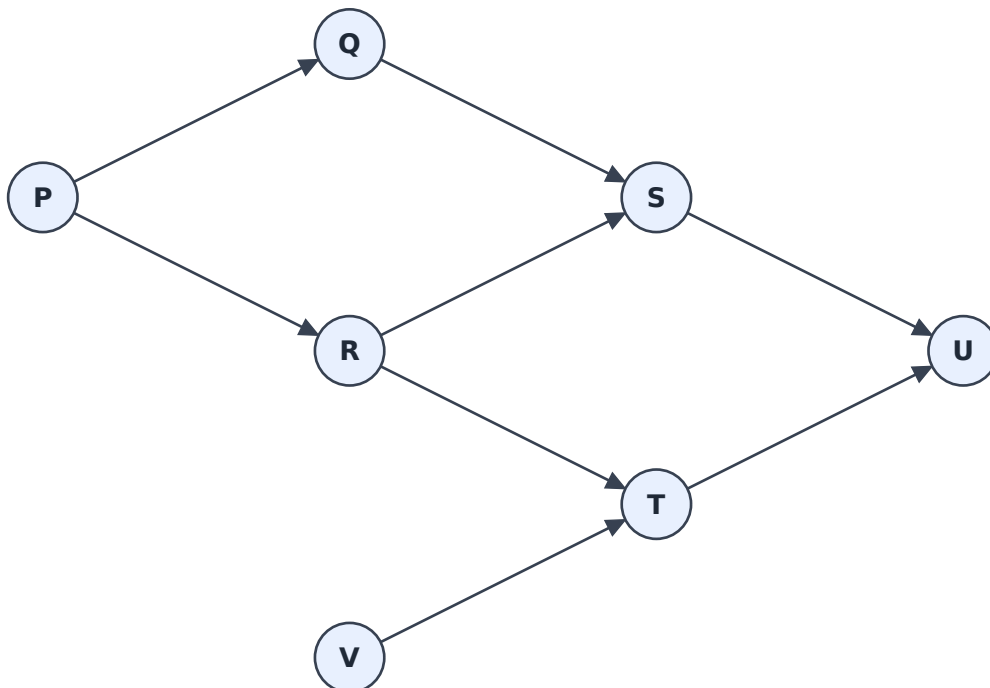
The number of distinct topological orderings of the DAG shown below is ____.



Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Topological sorting — validity of an order]

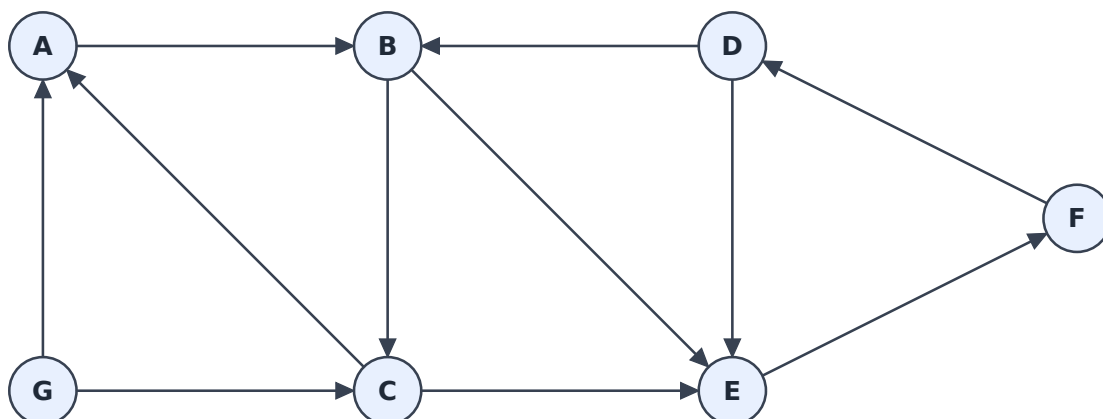
Which of the following is **NOT** a topological ordering of the DAG shown below?



- (A) P, V, R, Q, T, S, U
- (B) V, P, Q, R, S, T, U
- (C) P, Q, V, R, T, U, S
- (D) P, R, V, T, Q, S, U

Q.4 [MSQ · one or more correct | 1 mark | DFS — edge classification]

DFS is run on the directed graph below. The outer loop tries start vertices in alphabetical order, adjacency lists are explored in alphabetical order, and a single clock (starting at 1) is incremented at every discovery and every finish. Which of the following statements is/are TRUE?



- (A) The DFS finds exactly 3 back edges
- (B) (B, E) is a cross edge
- (C) D is discovered at time 6 and finished at time 7
- (D) (C, A) is a back edge

Q.5 [NAT · numerical answer | 1 mark | Stacks — monotonic stack with Python modulo]

Consider the following Python program. The value printed is _____.

Python

```
1 st = []
2 for x in [7, -3, 5, -8, 2]:
3     while st and st[-1] % 4 > x % 4:
4         st.pop()
5     st.append(x)
6 print(sum(st))
```

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Doubly linked lists — swapping adjacent nodes]

The program builds a doubly linked list $P \rightleftharpoons Q \rightleftharpoons R \rightleftharpoons S \rightleftharpoons T$ and then rewires pointers around nodes Q and R. What is printed?

Python

```
1 class D:
2     def __init__(s, v):
3         s.v, s.prev, s.next = v, None, None
4
5 nodes = [D(v) for v in "PQRST"]
6 for x, y in zip(nodes, nodes[1:]):
7     x.next, y.prev = y, x
8 p, q = nodes[1], nodes[2]
9 p.prev.next = q
10 q.next.prev = p
11 p.next = q.next
12 q.prev = p.prev
13 q.next = p
14 p.prev = q
15 h, fw = nodes[0], ""
16 while h:
17     fw, h = fw + h.v, h.next
18 t, bw = nodes[4], ""
19 while t:
20     bw, t = bw + t.v, t.prev
21 print(fw, bw)
```

- (A) PRQST TSRQP
- (B) PRQST TSQRP
- (C) PQRST TSRQP
- (D) PRST TSQRP

Q.7 [NAT · numerical answer | 1 mark | Hashing — double hashing]

Keys 18, 41, 25, 57, 70, 31 are inserted in this order into an initially empty hash table of size 13 using double hashing: probe i ($i = 0, 1, 2, \dots$) examines slot $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. The index of the slot in which 31 is stored is _____.

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Binary trees — reconstruction from traversals]

A binary tree with distinct labels has

- pre-order: M, F, B, A, D, K, H, P, S, R
- in-order: A, B, D, F, H, K, M, P, R, S

Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) The level-order traversal is M, F, P, B, K, S, A, D, H, R
- (B) The tree has exactly 4 leaves
- (C) The height of the tree is 4
- (D) The post-order traversal is A, D, B, H, K, F, R, S, P, M

Q.9 [MCQ · single correct | 1 mark | Binary search — lower bound]

Consider the following Python program. What is printed?

Python

```
1 def lb(a, x):
2     lo, hi = 0, len(a)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 A = [2, 4, 4, 4, 7, 9, 9, 12]
12 print(lb(A, 4), lb(A, 9) - lb(A, 8), lb(A, 13))
```

- (A) 1 0 8
- (B) 1 1 7
- (C) 3 1 8
- (D) 2 0 8

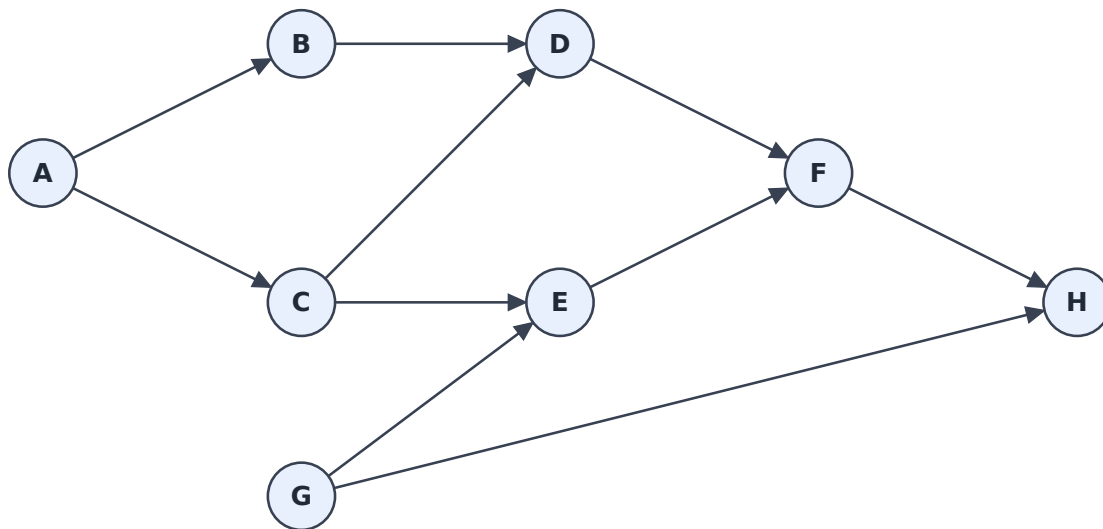
Q.10 [MCQ · single correct | 1 mark | Selection sort — stability]

The records (3, a), (1, b), (3, c), (2, d), (1, e) are sorted by their numeric key using swap-based selection sort: in pass i the record with the minimum key in positions $i..n-1$ is found (scanning left to right and replacing the current minimum only on a strictly smaller key) and swapped with position i . Reading only the letters, the final order is

- (A) b e d a c
- (B) e b d a c
- (C) e b d c a
- (D) b e d c a

Q.11 [NAT · numerical answer | 2 marks | Topological sorting — counting orders]

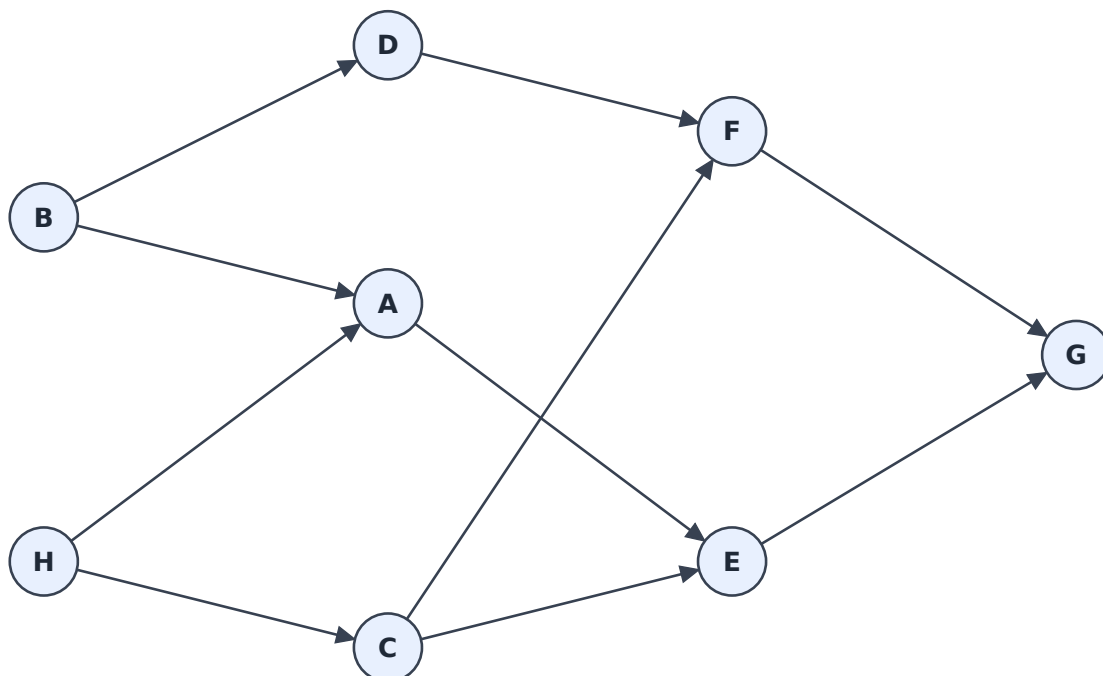
The number of distinct topological orderings of the DAG shown below is _____.



Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Topological sorting — Kahn's algorithm with a min-heap]

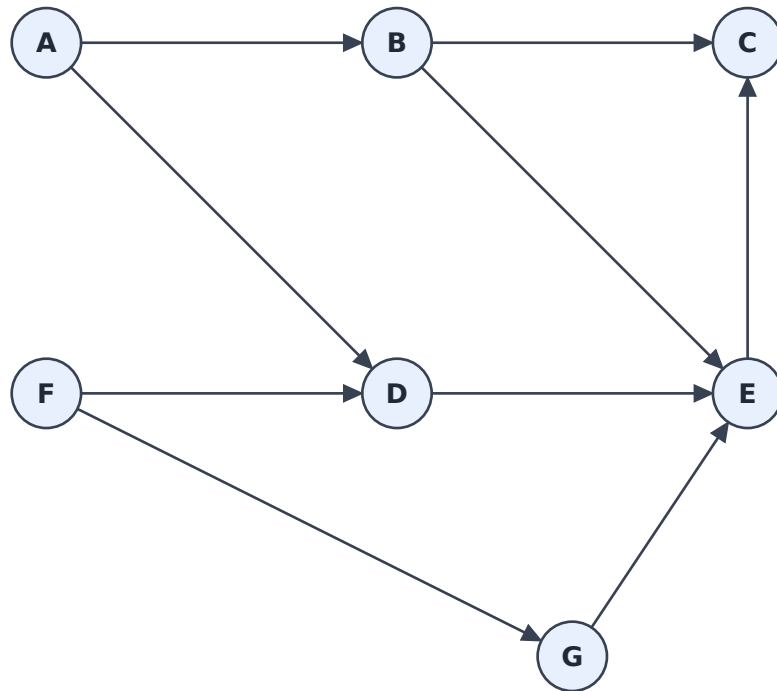
Kahn's algorithm is run on the DAG below, but the set of vertices with in-degree 0 is kept in a **min-priority queue** keyed on the vertex label, so the alphabetically smallest available vertex is always output next. The output order is



- (A) B, D, H, A, C, E, F, G
- (B) B, H, D, A, C, E, F, G
- (C) B, D, H, A, C, F, E, G
- (D) H, B, A, C, D, E, F, G

Q.13 [MSQ · one or more correct | 2 marks | DFS on a DAG — timestamps and topological order]

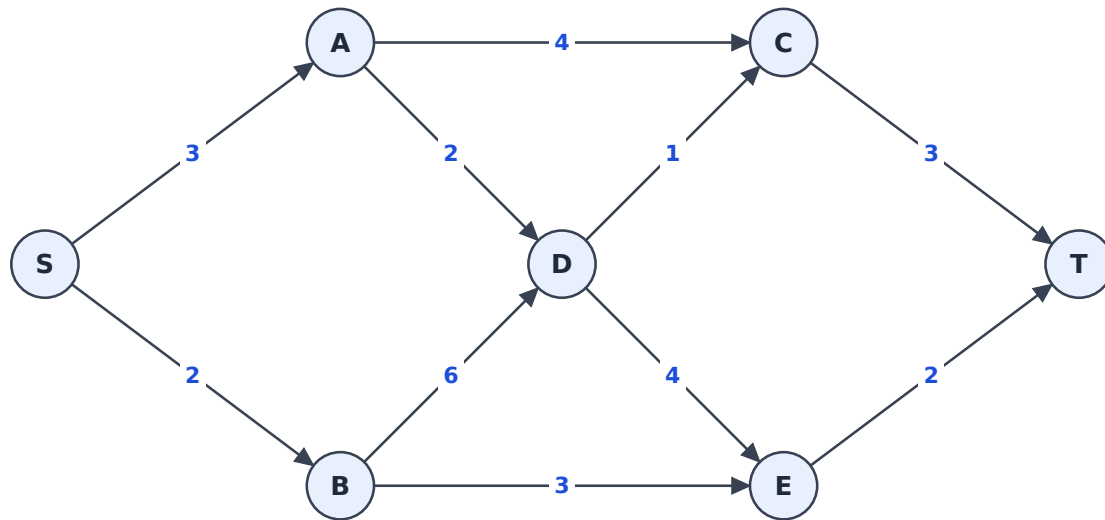
DFS is run on the DAG below with the same conventions as Q.4 (outer loop and adjacency lists in alphabetical order; one clock starting at 1, ticking at every discovery and finish). Let $d[v]$ and $f[v]$ be the discovery and finish times. Which of the following statements is/are TRUE?



- (A) $d[E] = 5$ and $f[E] = 6$
- (B) (D, E) is a back edge
- (C) The DFS classifies exactly 3 edges as cross edges
- (D) Listing the vertices in decreasing order of finish time gives F, G, A, D, B, E, C, which is a topological order

Q.14 [NAT · numerical answer | 2 marks | DAGs — longest (critical) path via topological order]

In the weighted DAG below, an edge weight is the duration of an activity; a vertex can start only after all its incoming activities are complete. The length of the longest path from S to T (the minimum completion time of the project) is ____.



Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Cycle detection — Kahn's algorithm in Python]

Consider the following Python program. What is printed?

Python

```

1 from collections import deque
2
3 G = {'a': ['b', 'c'], 'b': ['d'], 'c': ['d', 'e'],
4      'd': ['f'], 'e': ['f', 'g'], 'f': ['e'], 'g': []}
5 indeg = {v: 0 for v in G}
6 for u in G:
7     for v in G[u]:
8         indeg[v] += 1
9 q = deque(v for v in G if indeg[v] == 0)
10 order = []
11 while q:
12     u = q.popleft()
13     order.append(u)
14     for v in G[u]:
15         indeg[v] -= 1
16         if indeg[v] == 0:
17             q.append(v)
18 print(''.join(order), sum(indeg.values()))

```

- (A) abcdefg 0
- (B) abcd 2
- (C) acbd 3
- (D) abcd 3

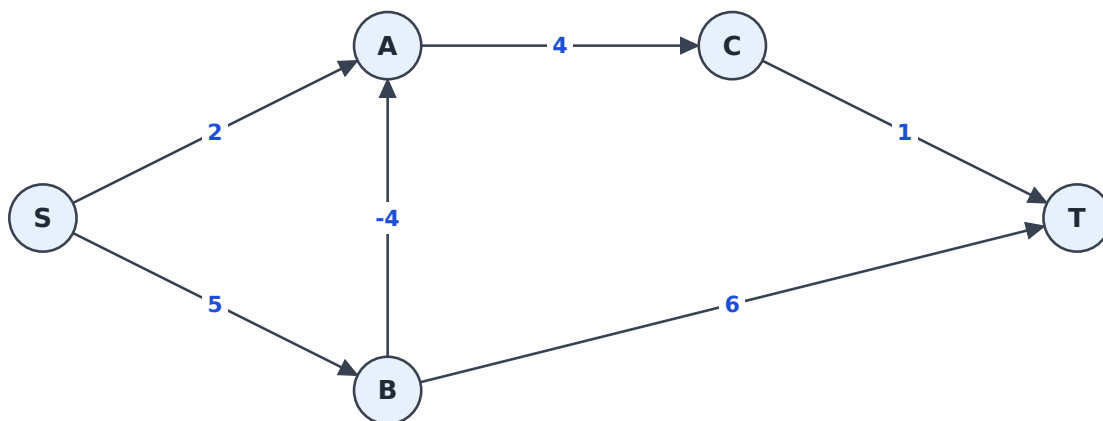
Q.16 [MCQ · single correct | 2 marks | Merge sort — worst-case comparison recurrence]

The worst-case number of key comparisons made by top-down merge sort on n elements satisfies $T(1) = 0$ and $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n - 1$ for $n \geq 2$. The value of $T(13)$ is

- (A) 33
- (B) 49
- (C) 37
- (D) 52

Q.17 [MSQ · one or more correct | 2 marks | Shortest paths in a DAG with a negative edge]

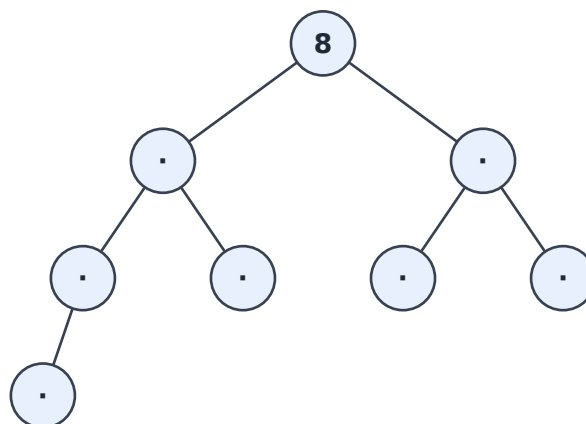
Consider the weighted DAG below (note the negative edge $B \rightarrow A$). Which of the following statements is/are TRUE?



- (A) Adding 4 to every edge weight (making all weights non-negative) and running Dijkstra returns a path that is also a shortest $S \rightarrow T$ path in the original graph
- (B) Relaxing edges in topological order gives the shortest distance $S \rightarrow T$ as 6
- (C) The DAG has exactly 2 topological orderings
- (D) Dijkstra's algorithm, in an implementation where a vertex once extracted is never updated again, reports the distance $S \rightarrow T$ as 7

Q.18 [NAT · numerical answer | 2 marks | Heaps — counting distinct max-heaps]

The number of distinct binary max-heaps (stored as complete binary trees / arrays) that can be formed using all of the keys 1, 2, 3, 4, 5, 6, 7, 8 exactly once is _____.



Shape of every 8-node heap (root must be 8)

Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Python — closures, nonlocal and late binding]

Consider the following Python program, which prints four integers separated by spaces. Which of the following statements is/are TRUE?

Python

```
1 def counter():
2     n = 0
3     def inc(k=1):
4         nonlocal n
5         n += k
6         return n
7     return inc
8
9 f, g = counter(), counter()
10 f(); f(5); g(2)
11 adders = [lambda x: x + i for i in range(3)]
12 muls = [lambda x, i=i: x * i for i in range(3)]
13 print(f(), g(), adders[0](10), muls[2](10))
```

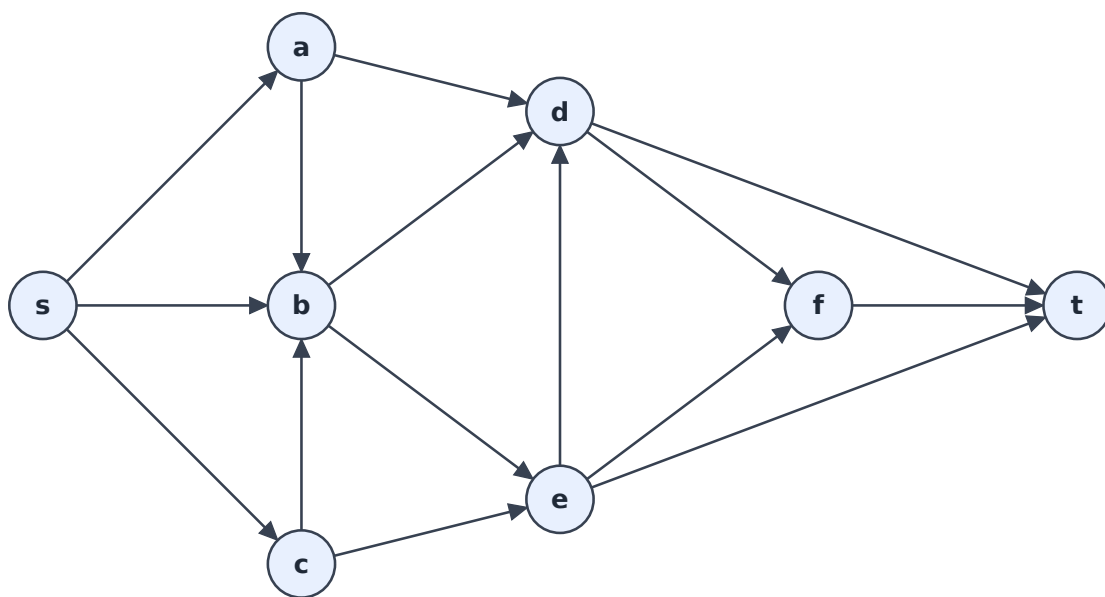
- (A) The first number printed is 7
- (B) The second number printed is 8
- (C) The third number printed is 10
- (D) The fourth number printed is 20

Q.20 [NAT · numerical answer | 2 marks | DAGs — counting paths with memoised recursion]

The program below stores a DAG as a dictionary in which each value is a string of successor labels (the DAG is drawn below). The value printed is _____.

Python

```
1 from functools import lru_cache
2
3 G = {'s': 'abc', 'a': 'bd', 'b': 'de', 'c': 'be',
4      'd': 'ft', 'e': 'dft', 'f': 't', 't': ''}
5
6 @lru_cache(maxsize=None)
7 def paths(u):
8     if u == 't':
9         return 1
10    return sum(paths(v) for v in G[u])
11
12 print(paths('s'))
```



Answer: _____ (enter a numerical value)

Answer Key — Mock Test 22

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — generator exhaustion
2	NAT	1	14	Topological sorting — counting orders
3	MCQ	1	(C)	Topological sorting — validity of an order
4	MSQ	1	(A), (C), (D)	DFS — edge classification
5	NAT	1	-6	Stacks — monotonic stack with Python modulo
6	MCQ	1	(B)	Doubly linked lists — swapping adjacent nodes
7	NAT	1	9	Hashing — double hashing
8	MSQ	1	(A), (B), (D)	Binary trees — reconstruction from traversals
9	MCQ	1	(A)	Binary search — lower bound
10	MCQ	1	(D)	Selection sort — stability
11	NAT	2	21	Topological sorting — counting orders
12	MCQ	2	(A)	Topological sorting — Kahn's algorithm with a min-heap
13	MSQ	2	(A), (D)	DFS on a DAG — timestamps and topological order
14	NAT	2	14	DAGs — longest (critical) path via topological order
15	MCQ	2	(D)	Cycle detection — Kahn's algorithm in Python
16	MCQ	2	(C)	Merge sort — worst-case comparison recurrence
17	MSQ	2	(B), (D)	Shortest paths in a DAG with a negative edge
18	NAT	2	210	Heaps — counting distinct max-heaps
19	MSQ	2	(A), (D)	Python — closures, nonlocal and late binding
20	NAT	2	24	DAGs — counting paths with memoised recursion

Detailed Solutions — Mock Test 22

Q.1 [MCQ · 1 mark · Python — generator exhaustion]

Answer: (B)

Concept: a generator expression is a one-shot iterator. Once it has been consumed, further iteration yields nothing — it does not restart.

- The generator yields x^2 for even x in $0..5$: 0, 4, 16.
- `sum(g)` consumes all of it $\rightarrow 20$.
- `list(g)` $\rightarrow$ the generator is exhausted $\rightarrow []$.
- `max(g, default=-1)` $\rightarrow$ empty iterable, so the `default` is returned $\rightarrow -1$ (no exception because `default` is supplied).

Output: 20 [] -1.

Options:

- (A) treats `g` as if it were a list that can be iterated repeatedly.
- (C) sums the squares of all of $0..5$ ($0 + 1 + 4 + 9 + 16 + 25 = 55$), ignoring the filter.
- (D) would be correct only without `default=` — `max()` of an empty iterable raises `ValueError`, but the default suppresses it.

Tip: use a list comprehension `[...]` when the values are needed more than once.

Q.2 [NAT · 1 mark · Topological sorting — counting orders]

Answer: 14

Concept: count linear extensions by splitting on the first vertex (a source) and counting the orders of what remains.

Constraints: $A < C$, $B < C$, $B < D$, $C < E$, $D < E$, $D < F$. Sources: A and B.

Case 1 — A first. C still needs B, and D needs B, so B must be second. Remaining $\{C, D, E, F\}$ with $C < E$, $D < E$, $D < F$: E must follow both C and D; F follows D.

- C D E F, C D F E, D C E F, D C F E, D F C E $\rightarrow 5$.

Case 2 — B first. Remaining $\{A, C, D, E, F\}$ with $A < C < E$, $D < E$, $D < F$.

- Orders of $\{A, C, D, E\}$ with $A < C < E$ and $D < E$: D in one of 3 places $\rightarrow$ D A C E, A D C E, A C D E.
- Insert F anywhere after D: 4, 3 and 2 places respectively $\rightarrow 4 + 3 + 2 = 9$.

Total = $5 + 9 = 14$.

Trap: assuming both cases are symmetric (giving 2×5 or 2×9) — after choosing A first, B is forced, but after B first, A is *not* forced.

Q.3 [MCQ · 1 mark · Topological sorting — validity of an order]

Answer: (C)

Concept: an ordering is topological iff for every edge $u \rightarrow v$, u appears before v . Check each edge, focusing on the sinks and on vertices with several predecessors (S needs Q, R; T needs R, V; U needs S, T).

- (A) P, V, R, Q, T, S, U: P before Q, R; R before S, T; V before T; Q before S; S, T before U. **Valid.**
- (B) V, P, Q, R, S, T, U: every edge goes forward. **Valid.**
- (C) P, Q, V, R, T, U, S: U appears **before** S although $S \rightarrow U$ is an edge. **Not valid.**
- (D) P, R, V, T, Q, S, U: T after R and V; S after Q and R; U last. **Valid.**

Trap: checking only that each vertex appears after *one* of its predecessors — U comes after T, but it must also come after S. The unique sink U must always be last here.

Q.4 [MSQ · 1 mark · DFS — edge classification]**Answer: (A), (C), (D)**

Concept: for an edge $u \rightarrow v$ examined during DFS: v undiscovered $\rightarrow$ *tree*; v discovered but not finished (an ancestor on the stack) $\rightarrow$ *back*; v finished and $d[u] < d[v] \rightarrow$ *forward*; v finished and $d[u] > d[v] \rightarrow$ *cross*.

Trace (d/f times):

- $A(1) \rightarrow B(2) \rightarrow C(3)$. $C \rightarrow A$: A is on the stack $\rightarrow$ **back**. $C \rightarrow E$: tree, E(4).
- $E \rightarrow F$: tree, F(5). $F \rightarrow D$: tree, D(6). $D \rightarrow B$: B on stack $\rightarrow$ **back**. $D \rightarrow E$: E on stack $\rightarrow$ **back**. D finishes (7); F (8); E (9); C (10).
- Back at B: $B \rightarrow E$: E already finished, $d[B] = 2 < d[E] = 4 \rightarrow$ **forward**. B finishes (11), A (12).
- Restart at G(13): $G \rightarrow A$ and $G \rightarrow C$ go to finished vertices discovered earlier $\rightarrow$ **cross**. G finishes (14).

Verdicts: (D) **True**. (B) **False** — it is a forward edge (E is a descendant of B reached via C). (A) back edges $C \rightarrow A$, $D \rightarrow B$, $D \rightarrow E \rightarrow$ exactly 3, **True**. (C) $d[D] = 6$, $f[D] = 7$, **True**.

Trap: calling (B, E) a cross edge because E was reached 'from another branch' — what matters is that E is a descendant of B in the DFS tree.

Discovery / finish times

	A	B	C	D	E	F	G
d	1	2	3	6	4	5	13
f	12	11	10	7	9	8	14

Q.5 [NAT · 1 mark · Stacks — monotonic stack with Python modulo]**Answer: -6**

Concept: this is a *monotonic stack*: before pushing x , it pops every element whose key (value mod 4) is larger than x 's key, so keys on the stack are non-decreasing from bottom to top. In Python, `%` returns a result with the sign of the divisor, so $-3 \% 4 = 1$ and $-8 \% 4 = 0$.

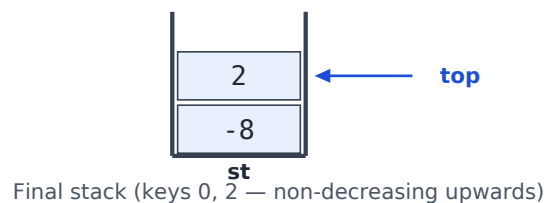
Keys: $7 \rightarrow 3$, $-3 \rightarrow 1$, $5 \rightarrow 1$, $-8 \rightarrow 0$, $2 \rightarrow 2$.

Trace:

- $x = 7$ (key 3): push $\rightarrow [7]$
- $x = -3$ (key 1): top key $3 > 1 \rightarrow$ pop 7; push $\rightarrow [-3]$
- $x = 5$ (key 1): top key $1 > 1$? No $\rightarrow$ push $\rightarrow [-3, 5]$
- $x = -8$ (key 0): pop 5 ($1 > 0$), pop -3 ($1 > 0$); push $\rightarrow [-8]$
- $x = 2$ (key 2): $0 > 2$? No $\rightarrow$ push $\rightarrow [-8, 2]$

Sum = $-8 + 2 = -6$.

Trap: using C-style remainders ($-3 \% 4 = -3$, $-8 \% 4 = 0$) changes the pops: -3 would have key -3 and pop everything, giving a different stack. Python's `%` is never negative for a positive divisor.



Q.6 [MCQ · 1 mark · Doubly linked lists — swapping adjacent nodes]**Answer: (B)**

Concept: swapping two adjacent nodes p, q ($q = p.next$) in a doubly linked list needs six pointer updates: the outer neighbours' links ($P.next, S.prev$) and both links of each of p and q . The order matters only where a pointer is read after being overwritten.

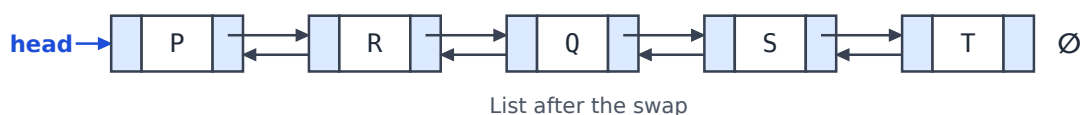
Step by step ($p = Q, q = R$):

- $p.prev.next = q \rightarrow P.next = R$
- $q.next.prev = p \rightarrow S.prev = Q$ ($q.next$ is still S)
- $p.next = q.next \rightarrow Q.next = S$
- $q.prev = p.prev \rightarrow R.prev = P$ ($p.prev$ is still P)
- $q.next = p \rightarrow R.next = Q$
- $p.prev = q \rightarrow Q.prev = R$

Forward from P : $P \rightarrow R \rightarrow Q \rightarrow S \rightarrow T = \text{PRQST}$. Backward from T : $T \rightarrow S \rightarrow Q \rightarrow R \rightarrow P = \text{TSQRP}$. The two traversals are mirror images, confirming the list is consistent.

Options: (A) has a backward chain that was not updated; (C) assumes nothing changed; (D) loses a node.

Trap: if $p.prev = q$ were executed before $q.prev = p.prev$, $R.prev$ would point to R itself and the backward traversal would loop forever — always read old pointers before overwriting them.

**Q.7 [NAT · 1 mark · Hashing — double hashing]****Answer: 9**

Concept: in double hashing the step size depends on the key, so keys with the same home slot follow different probe sequences (no secondary clustering).

- 18: $h_1 = 5 \rightarrow$ slot 5.
- 41: $h_1 = 2 \rightarrow$ slot 2.
- 25: $h_1 = 12 \rightarrow$ slot 12.
- 57: $h_1 = 5$ (full); $h_2 = 1 + 2 = 3 \rightarrow 8 \rightarrow$ slot 8.
- 70: $h_1 = 5$ (full); $h_2 = 1 + 4 = 5 \rightarrow 10 \rightarrow$ slot 10.
- 31: $h_1 = 5$ (full); $h_2 = 1 + 9 = 10 \rightarrow$ probes 5, $(5 + 10) \bmod 13 = 2$ (full), $(5 + 20) \bmod 13 = 12$ (full), $(5 + 30) \bmod 13 = 9 \rightarrow$ **slot 9**.

31 needed 4 probes even though 57 and 70 (same home slot) needed only 2 each.

Trap: using linear probing by mistake (31 would go to slot 6), or forgetting the '1 +' in h_2 (step 9 $\rightarrow$ probes 5, 1 $\rightarrow$ slot 1).

0	
1	
2	41
3	
4	
5	18
6	
7	
8	57
9	31
10	70
11	
12	25

Final table

Q.8 [MSQ · 1 mark · Binary trees — reconstruction from traversals]**Answer: (A), (B), (D)**

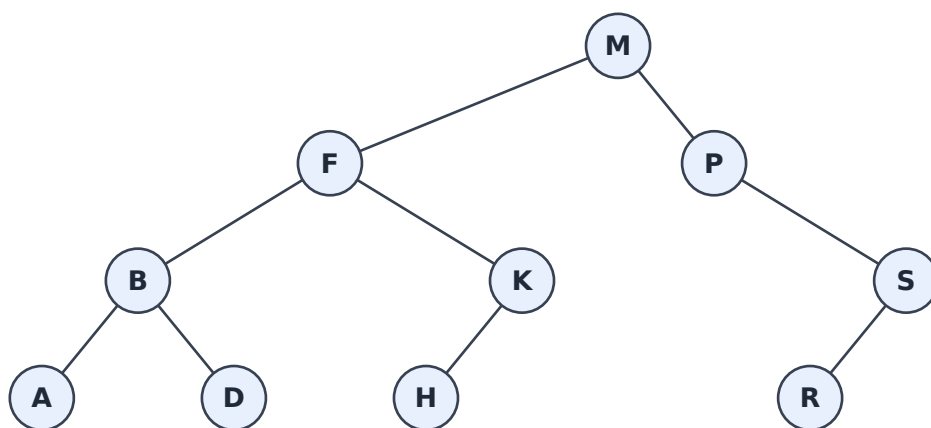
Concept: the first pre-order element is the root; its position in the in-order splits the remaining labels into the left and right subtrees. Recurse.

- Root M; in-order left {A, B, D, F, H, K}, right {P, R, S}.
- Left pre-order F, B, A, D, K, H → root F; in-order left {A, B, D}, right {H, K}.
- B, A, D → B with children A, D.
- K, H → K with left child H.
- Right pre-order P, S, R → root P; in-order {P | R, S} → P has no left child; right subtree pre-order S, R with in-order R, S → S with left child R.

Verdicts:

- (A) levels: M | F, P | B, K, S | A, D, H, R. **True.**
- (B) leaves A, D, H, R → 4. **True.**
- (C) longest paths M-F-B-A, M-F-K-H, M-P-S-R all have 3 edges → height 3. **False.**
- (D) post-order: A, D, B, H, K, F, R, S, P, M. **True.**

Trap: counting nodes instead of edges for height gives 4.



Reconstructed tree

Q.9 [MCQ · 1 mark · Binary search — lower bound]**Answer: (A)**

Concept: this is the half-open $[lo, hi)$ *lower-bound* search: it returns the first index i with $a[i] \geq x$ (or $len(a)$ if none). With duplicates it returns the **leftmost** occurrence.

- $lb(A, 4)$: first element ≥ 4 is $A[1] = 4 \rightarrow 1$. (Trace: $lo, hi = 0, 8 \rightarrow mid\ 4\ (7 \geq 4)\ hi = 4 \rightarrow mid\ 2\ (4)\ hi = 2 \rightarrow mid\ 1\ (4)\ hi = 1 \rightarrow mid\ 0\ (2 < 4)\ lo = 1 \rightarrow stop$.)
- $lb(A, 9)$: first element ≥ 9 is $A[5] \rightarrow 5$. $lb(A, 8)$: 8 is absent; first element ≥ 8 is also $A[5] \rightarrow 5$. Difference **0** — tells us 8 does not occur (count of $8 = lb(8 + 1) - lb(8) = 0$).
- $lb(A, 13)$: every element $< 13 \rightarrow$ returns $len(A) = 8$.

Output **1 0 8**.

Options: (C) returns the *last* occurrence of 4 (upper bound $- 1$) and counts 9s. (B) clamps to the last index 7. (D) returns the middle 4.

Tip: lower bound never reads $a[len(a)]$ because $mid < hi$ always holds.

Q.10 [MCQ · 1 mark · Selection sort — stability]**Answer: (D)**

Concept: swap-based selection sort is **not stable** — the long-distance swap can jump a record over another with the same key.

Trace (key-letter):

- start: 3a 1b 3c 2d 1e
- pass 0: min key 1, first at index 1 (1b) $\rightarrow$ swap with 3a $\rightarrow$ 1b 3a 3c 2d 1e
- pass 1: min of 3a 3c 2d 1e is 1e (index 4) $\rightarrow$ swap with 3a $\rightarrow$ 1b 1e 3c 2d 3a
- pass 2: min of 3c 2d 3a is 2d $\rightarrow$ swap with 3c $\rightarrow$ 1b 1e 2d 3c 3a
- pass 3: 3c vs 3a — not strictly smaller $\rightarrow$ no change.

Final letters: **b e d c a**.

The two 1s kept their order (b before e), but the 3s were reversed: a was originally before c, and pass 1 threw 3a to the end.

Options: (A) is the stable result (insertion sort or merge sort would give it). (C)/(B) reverse b and e, which never happens here.

Trap: concluding stability from the fact that *some* equal keys stayed in order.

Q.11 [NAT · 2 marks · Topological sorting — counting orders]**Answer: 21**

Concept: first identify vertices whose position is forced, then count the linear extensions of the remaining partial order by inserting the 'loose' vertices one at a time.

Forced tail. F needs D and E; D needs A, B, C; E needs C, G. So all of A, B, C, D, E, G precede F, and H (needing F and G) comes after F. Hence every order ends ..., **F, H**.

Orders of {A, B, C, D, E} without G. A is the only source; then $B < D, C < D, C < E$:

- A B C D E, A B C E D, A C B D E, A C B E D, A C E B D $\rightarrow$ 5 orders.

Insert G (only constraint $G < E$): G may go in any slot before E. If E is at position p (1-based) among the five, there are p slots.

- A B C D E ($p = 5$) $\rightarrow 5$
- A B C E D ($p = 4$) $\rightarrow 4$
- A C B D E ($p = 5$) $\rightarrow 5$
- A C B E D ($p = 4$) $\rightarrow 4$
- A C E B D ($p = 3$) $\rightarrow 3$

Total = $5 + 4 + 5 + 4 + 3 = 21$.

Trap: treating G as completely free (it is a source) and multiplying by 6 positions (giving 30) — G must still precede E.

Q.12 [MCQ · 2 marks · Topological sorting — Kahn's algorithm with a min-heap]

Answer: (A)

Concept: with a min-heap, Kahn's algorithm outputs the **lexicographically smallest** topological order: at each step it takes the smallest label among the current sources.

In-degrees: A 2 (H, B), C 1 (H), D 1 (A), E 2 (A, C), F 2 (D, C), G 2 (E, F); B, H 0.

Trace (heap contents → output):

- {B, H} → output **B**; A → 1, D → 0. Heap {D, H}.
- {D, H} → output **D**; F → 1. Heap {H}.
- {H} → output **H**; A → 0, C → 0. Heap {A, C}.
- output **A**; E → 1. Heap {C}.
- output **C**; E → 0, F → 0. Heap {E, F}.
- output **E**; G → 1. Then **F**; G → 0. Then **G**.

Order: **B, D, H, A, C, E, F, G** → (A).

Options:

- (B) is what a FIFO queue gives (B, H were both initially available, so H precedes the newly freed D).
- (C) is a valid topological order but not the smallest: after C, both E and F are available and $E < F$.
- (D) is also a valid topological order, but it starts with H although B is smaller.

Trap: confusing 'smallest available' with 'FIFO among sources'.

Q.13 [MSQ · 2 marks · DFS on a DAG — timestamps and topological order]

Answer: (A), (D)

Concept: in a DAG, DFS never finds a back edge, and for every edge $u \rightarrow v$ we have $f[u] > f[v]$; hence sorting by decreasing finish time is a topological order.

Trace:

- A(1) → B(2) → C(3), C finishes (4). B → E: E(5); E → C finished → cross; E finishes (6). B finishes (7).
- A → D(8); D → E finished, $d[D] = 8 > d[E] = 5$ → **cross**; D finishes (9). A finishes (10).
- Restart F(11): F → D finished → cross. F → G(12); G → E → cross; G finishes (13). F finishes (14).

Times: A 1/10, B 2/7, C 3/4, D 8/9, E 5/6, F 11/14, G 12/13.

Verdicts:

- (A) **True**.
- (B) **False** — E was already finished; a DAG has no back edges at all.
- (C) cross edges: $E \rightarrow C$, $D \rightarrow E$, $F \rightarrow D$, $G \rightarrow E$ → **4**, not 3. **False**. (Tree edges: $A \rightarrow B$, $B \rightarrow C$, $B \rightarrow E$, $A \rightarrow D$, $F \rightarrow G$.)
- (D) decreasing f: F 14, G 13, A 10, D 9, B 7, E 6, C 4 → F, G, A, D, B, E, C. Every edge goes left to right. **True**.

Trap: in (C), forgetting that edges from a later DFS tree into an earlier one ($F \rightarrow D$, $G \rightarrow E$) are cross edges too.

Discovery / finish times

	A	B	C	D	E	F	G
d	1	2	3	8	5	11	12
f	10	7	4	9	6	14	13

Q.14 [NAT · 2 marks · DAGs — longest (critical) path via topological order]**Answer: 14**

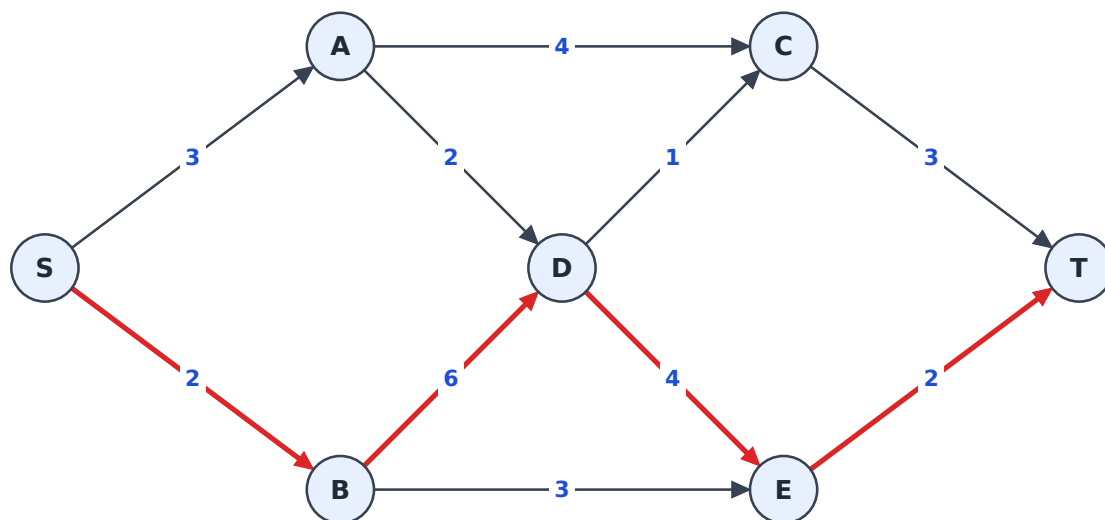
Concept: longest paths are NP-hard in general graphs, but in a DAG we can process vertices in topological order with $L(v) = \max \text{ over edges } u \rightarrow v \text{ of } L(u) + w(u, v)$.

Topological order: S, A, B, D, C, E, T.

- $L(S) = 0$
- $L(A) = 0 + 3 = 3$
- $L(B) = 0 + 2 = 2$
- $L(D) = \max(L(A) + 2, L(B) + 6) = \max(5, 8) = 8$
- $L(C) = \max(L(A) + 4, L(D) + 1) = \max(7, 9) = 9$
- $L(E) = \max(L(D) + 4, L(B) + 3) = \max(12, 5) = 12$
- $L(T) = \max(L(C) + 3, L(E) + 2) = \max(12, 14) = \mathbf{14}$

Critical path: $S \rightarrow B \rightarrow D \rightarrow E \rightarrow T$ ($2 + 6 + 4 + 2 = 14$).

Trap: following the locally heaviest first edge ($S \rightarrow A$, weight 3) leads at best to 11 ($S \rightarrow A \rightarrow D \rightarrow E \rightarrow T$); also, running a shortest-path algorithm gives the *earliest* arrival along the cheapest route ($S \rightarrow A \rightarrow C \rightarrow T = 10$), which is not the project completion time.



Critical path $S \rightarrow B \rightarrow D \rightarrow E \rightarrow T$

Q.15 [MCQ · 2 marks · Cycle detection — Kahn's algorithm in Python]**Answer: (D)**

Concept: Kahn's algorithm removes vertices of in-degree 0. If the graph has a cycle, the vertices on the cycle (and everything reachable only through it) never reach in-degree 0, so fewer than $|V|$ vertices are output — this is how Kahn detects cycles.

Initial in-degrees: a 0, b 1, c 1, d 2, e 2 (from c and f), f 2 (from d and e), g 1.

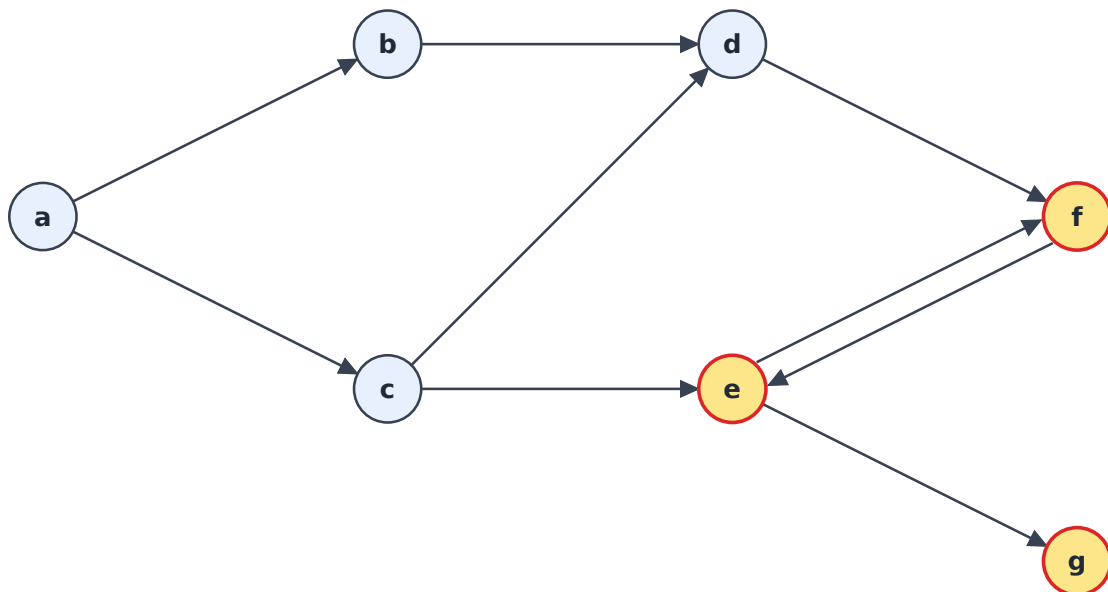
Trace:

- $q = [a] \rightarrow$ output a; $b \rightarrow 0$, $c \rightarrow 0 \rightarrow q = [b, c]$
- output b; $d \rightarrow 1$
- output c; $d \rightarrow 0$ (enqueue), $e \rightarrow 1$
- output d; $f \rightarrow 1$
- queue empty. e and f each still have in-degree 1 (they wait on each other: $e \rightarrow f \rightarrow e$ is a cycle) and g has in-degree 1 (waiting on e).

order = abcd; remaining in-degree sum = $1 + 1 + 1 = \mathbf{3}$. Output **abcd 3**.

Options: (A) ignores the cycle $e \rightleftharpoons f$. (B) forgets that g is blocked as well — it is not on the cycle but depends on it. (C) uses a stack-like order; the deque is FIFO.

Tip: $\text{len}(\text{order}) < \text{len}(G) \Leftrightarrow$ the directed graph contains a cycle.



Highlighted vertices are never output ($e \rightleftharpoons f$ is a cycle; g waits on e)

Q.16 [MCQ · 2 marks · Merge sort — worst-case comparison recurrence]

Answer: (C)

Concept: merging lists of total length m costs at most $m - 1$ comparisons; the recurrence adds this over the recursion tree. Closed form: $T(n) = n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$.

Bottom-up evaluation:

- $T(2) = T(1) + T(1) + 1 = 1$
- $T(3) = T(1) + T(2) + 2 = 3$
- $T(4) = T(2) + T(2) + 3 = 5$
- $T(6) = T(3) + T(3) + 5 = 11$
- $T(7) = T(3) + T(4) + 6 = 14$
- $T(13) = T(6) + T(7) + 12 = 11 + 14 + 12 = 37$

Check with the closed form: $\lceil \log_2 13 \rceil = 4 \rightarrow 13 \cdot 4 - 16 + 1 = 37. \checkmark$

Options:

- (A) 33 under-counts (e.g. uses $\lfloor n/2 \rfloor$ for both halves).
- (B) 49 uses n instead of $n - 1$ per merge.
- (D) $52 = n\lceil \log_2 n \rceil$ is only an upper bound.

Tip: for n a power of 2, $T(n) = n \log_2 n - n + 1$ (e.g. $T(8) = 17$).

Q.17 [MSQ · 2 marks · Shortest paths in a DAG with a negative edge]

Answer: (B), (D)

Concept: in a DAG, relaxing the out-edges of each vertex in topological order computes correct shortest paths in $\Theta(V + E)$ even with negative weights. Dijkstra's greedy choice fails with negative edges. Adding a constant to every edge penalises paths with more edges, so it does not preserve shortest paths.

- (A) New weights: $S \rightarrow A$ 6, $S \rightarrow B$ 9, $B \rightarrow A$ 0, $A \rightarrow C$ 8, $C \rightarrow T$ 5, $B \rightarrow T$ 10. Path costs: $S \rightarrow A \rightarrow C \rightarrow T$ 19, $S \rightarrow B \rightarrow T$ 19, $S \rightarrow B \rightarrow A \rightarrow C \rightarrow T$ 22. Dijkstra returns a 3-edge path or $S \rightarrow B \rightarrow T$, both of original cost 7, but the true

shortest is S-B-A-C-T (cost 6). **False.**

- (B) Topological order (unique): S, B, A, C, T. $d(S) = 0$; from S: $A = 2$, $B = 5$; from B: $A = \min(2, 5 - 4) = 1$, $T = 11$; from A: $C = 5$; from C: $T = \min(11, 6) = 6$. **True.**
- (C) S must be first; $B \rightarrow A$ forces B before A; then C, then T: only S, B, A, C, T. **False.**
- (D) Dijkstra: extract S $\rightarrow A = 2$, $B = 5$. Extract A (2) $\rightarrow C = 6$. Extract B (5): A is already final, so the improvement to 1 is lost; $T = 11$. Extract C (6) $\rightarrow T = 7$. Extract T $\rightarrow 7$. **True** (and wrong — the real distance is 6).

Trap: believing that 'shifting all weights to be non-negative' fixes Dijkstra.

Q.18 [NAT · 2 marks · Heaps — counting distinct max-heaps]

Answer: 210

Concept: the shape of an n -node heap is fixed. The root must hold the maximum; the remaining $n - 1$ keys are split between the left subtree (size L) and right subtree (size R) in $C(n - 1, L)$ ways, and each subtree is independently a heap:

$$H(n) = C(n - 1, L) \cdot H(L) \cdot H(R).$$

Subtree sizes for an 8-node complete tree: levels hold 1, 2, 4, 1 nodes; the single node on the last level is under the left child. So $L = 1 + 2 + 1 = 4$, $R = 3$.

- $H(1) = 1$, $H(2) = 1$ (the larger key must be the root).
- $H(3) = C(2, 1) \cdot 1 \cdot 1 = 2$.
- $H(4)$: $L = 2$, $R = 1 \rightarrow C(3, 2) \cdot H(2) \cdot H(1) = 3$.
- $H(8) = C(7, 4) \cdot H(4) \cdot H(3) = 35 \cdot 3 \cdot 2 = 210$.

Trap: taking $L = R = 3.5$ (impossible) or $L = 3$, $R = 4$ — in a complete binary tree the *left* subtree is filled first. With $L = 3$, $R = 4$ the formula gives the same number here by symmetry of $C(7, \cdot)$, but the sub-shapes matter in general (e.g. $H(6) = 20$, $H(7) = 80$).

Q.19 [MSQ · 2 marks · Python — closures, nonlocal and late binding]

Answer: (A), (D)

Concept: each call to `counter()` creates a fresh cell for `n`; `nonlocal` rebinds that cell. A lambda captures a *variable*, not its current value, so lambdas made in a loop see the variable's final value (late binding) — unless the value is frozen with a default argument `i=i`.

- `f`: $f() \rightarrow 1$, $f(5) \rightarrow 6$, then `f()` in the print $\rightarrow 7$.
- `g` has its own `n`: $g(2) \rightarrow 2$, then `g()` $\rightarrow 3$ (not shared with `f`).
- `adders[0](10)`: the comprehension's `i` ends at 2 $\rightarrow 10 + 2 = 12$.
- `mul[s][2](10)`: default `i = 2` was stored when the lambda was created $\rightarrow 10 \times 2 = 20$.

Output: 7 3 12 20.

Verdicts: (A) **True**. (B) **False** — 8 would require `f` and `g` to share `n`. (C) **False** — 10 assumes early binding of `i = 0`. (D) **True**.

Trap: the comprehension has its own scope in Python 3, but the closure still refers to that scope's single `i`, which ends at 2.

Q.20 [NAT · 2 marks · DAGs — counting paths with memoised recursion]

Answer: 24

Concept: the number of s - t paths in a DAG satisfies $P(t) = 1$ and $P(u) = \sum P(v)$ over successors v ; memoisation evaluates each vertex once (8 calls in all), turning an exponential enumeration into $O(V + E)$. Iterating over a string iterates over its characters, so `G['s'] = 'abc'` means successors `a`, `b`, `c`.

Evaluate in reverse topological order:

- $P(t) = 1$
- $P(f) = P(t) = 1$
- $P(d) = P(f) + P(t) = 2$
- $P(e) = P(d) + P(f) + P(t) = 2 + 1 + 1 = 4$
- $P(b) = P(d) + P(e) = 2 + 4 = 6$
- $P(a) = P(b) + P(d) = 6 + 2 = 8$
- $P(c) = P(b) + P(e) = 6 + 4 = 10$
- $P(s) = P(a) + P(b) + P(c) = 8 + 6 + 10 = \mathbf{24}$

Trap: counting *vertices* reachable or *edges* instead of paths, or forgetting the direct edges $d \rightarrow t$ and $e \rightarrow t$. Without `lru_cache` the answer is the same but $P(b)$, $P(d)$, ... are recomputed many times.

Mock Test 23

Hashing Analysis

Difficulty: GATE-level

Focus areas: load factor, expected probes, clustering, probability of collisions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — dict keys that hash and compare equal]

Consider the following Python program. What is printed?

Python

```
1 d = {1: 'a', True: 'b', 1.0: 'c'}
2 print(len(d), d[1], list(d.keys())[0])
```

- (A) 1 c 1
- (B) 3 a 1
- (C) 1 a 1.0
- (D) 1 c True

Q.2 [NAT · numerical answer | 1 mark | Simple uniform hashing — empty slot probability]

Ten keys are inserted into a hash table with 10 slots using separate chaining. Assume simple uniform hashing: each key independently hashes to each slot with probability 1/10. The probability that slot 0 is still empty after all insertions is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

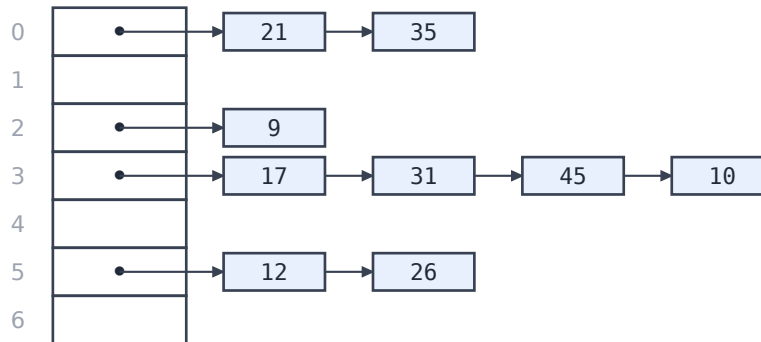
Q.3 [NAT · numerical answer | 1 mark | Expected number of collisions]

Twenty distinct keys are hashed into a table of 50 slots under simple uniform hashing. A *collision pair* is an unordered pair of distinct keys that hash to the same slot. The expected number of collision pairs is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.4 [MCQ · single correct | 1 mark | Chaining — average successful search cost]

The hash table below (size 7, $h(k) = k \bmod 7$, separate chaining) stores 9 keys; each chain is searched from its head. Each stored key is equally likely to be searched for. The expected number of **key comparisons** in a successful search is



Chains listed head first

- (A) 9/7
- (B) 2
- (C) 17/9
- (D) 13/9

Q.5 [MSQ · one or more correct | 1 mark | Python — hashability]

Which of the following statements about Python 3 is/are TRUE?

- (A) After `d = {(1, 2): 'x'}`, the expression `(2, 1) in d` evaluates to `True`
- (B) Evaluating `hash((1, [2]))` raises a `TypeError`
- (C) `len({0, 0.0, False, ''})` evaluates to 2
- (D) Evaluating `{[1, 2]: 3}` raises a `TypeError`

Q.6 [MCQ · single correct | 1 mark | Binary search — bisect boundaries]

Consider the following Python program. What is printed?

Python

```

1 from bisect import bisect_left, bisect_right
2 a = [1, 3, 3, 3, 5, 8, 8, 10]
3 print(bisect_right(a, 3) - bisect_left(a, 3),
4       bisect_left(a, 6), bisect_right(a, 8))
  
```

- (A) 3 5 7
- (B) 2 4 6
- (C) 3 4 7
- (D) 3 5 6

Q.7 [NAT · numerical answer | 1 mark | Stacks — postfix evaluation depth]

The postfix expression `2 3 4 * + 5 6 7 * + *` is evaluated with a stack (operands are pushed; an operator pops two operands and pushes the result). The maximum number of elements on the stack at any moment is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Python — linked-list pointer surgery]

Consider the following Python program. What is printed?

Python

```

1 class N:
2     def __init__(s, v, nx=None):
3         s.v, s.nx = v, nx
4
5 h = None
6 for v in [3, 1, 4, 1, 5]:
7     h = N(v, h)
8 p = h
9 while p.nx and p.nx.nx:
10    p.nx = p.nx.nx
11    p = p.nx
12 out = []
13 while h:
14     out.append(h.v)
15     h = h.nx
16 print(out)

```

- (A) [3, 4, 5]
- (B) [5, 4, 3]
- (C) [5, 1, 1]
- (D) [5, 4, 1, 3]

Q.9 [MCQ · single correct | 1 mark | Bubble sort with early termination]

Bubble sort with an early-exit flag (stop after the first pass that makes no swap; pass p compares adjacent pairs in $A[0..n-p]$) is applied to [2, 3, 9, 1, 4, 5, 8]. The number of passes executed (including the final swap-free pass) and the total number of swaps are, respectively,

- (A) 4 and 7
- (B) 3 and 6
- (C) 4 and 6
- (D) 6 and 6

Q.10 [MSQ · one or more correct | 1 mark | Adjacency matrix — reading graph properties]

An undirected simple graph G on vertices a, b, c, d, e has the adjacency matrix A shown. Which of the following statements is/are TRUE?

Adjacency matrix A

	a	b	c	d	e
a	0	1	1	0	0
b	1	0	1	1	0
c	1	1	0	0	1
d	0	1	0	0	1
e	0	0	1	1	0

- (A) G is bipartite
- (B) The (a, d) entry of A^2 is 2
- (C) G contains a cycle of length 5
- (D) The trace (sum of diagonal entries) of A^2 is 12

Q.11 [MSQ · one or more correct | 2 marks | Linear probing — reconstructing insertion order]

Six distinct keys were inserted, in some unknown order, into an initially empty hash table of size 10 using $h(k) = k \bmod 10$ and linear probing (step +1). No deletions occurred. The final table is shown below. Which of the following statements is/are TRUE?

0	
1	61
2	31
3	72
4	13
5	24
6	91
7	
8	
9	

Final table

- (A) 61 was inserted before 31
- (B) 24 could have been inserted before 72
- (C) 72 was inserted before 13
- (D) Exactly one insertion order produces this table

Q.12 [NAT · numerical answer | 2 marks | Linear probing — expected unsuccessful search cost]

The hash table below has 10 slots and uses linear probing (step +1, wrapping from 9 to 0). A search for a key **not** in the table starts at a home slot that is equally likely to be any of the 10 slots, and probes until it reaches an empty slot; every slot inspected (including the final empty one) counts as one probe. The expected number of probes is _____ (rounded off to two decimal places).

0	40
1	21
2	30
3	12
4	
5	55
6	15
7	
8	
9	19

Answer: _____ (enter a numerical value)

Q.13 [NAT · numerical answer | 2 marks | Double hashing — probe counting]

Keys 27, 53, 40, 66, 14, 79, 183 are inserted in that order into an initially empty table of size 13 using double hashing: probe i ($i = 0, 1, 2, \dots$) examines slot $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Each slot examined counts as one probe (including the slot finally used). The total number of probes for all seven insertions is _____.

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Probability of no collision]

Three distinct keys are inserted into a hash table with 7 slots. Each key independently hashes to each slot with probability $1/7$. The probability that no two keys hash to the same slot is

- (A) $1/7$
- (B) $36/49$
- (C) $6/7$
- (D) $30/49$

Q.15 [NAT · numerical answer | 2 marks | Linear probing with deletion (tombstones)]

A hash table of size 10 uses $h(k) = k \bmod 10$ and linear probing. Deletion replaces the key by a **tombstone**: searches continue past a tombstone, while an insertion places the new key in the first tombstone or empty slot it meets. Starting from an empty table, perform:

insert 15, 25, 35, 16, 45; delete 25; delete 16; insert 55.

Then search for 45 (present) and for 65 (absent). Counting every slot inspected (including the slot where the search ends), the total number of probes made by these two searches is _____.

Answer: _____ (enter a numerical value)

Q.16 [MCQ · single correct | 2 marks | Python — `__eq__` and `__hash__` in sets]

Consider the following Python program. What is printed?

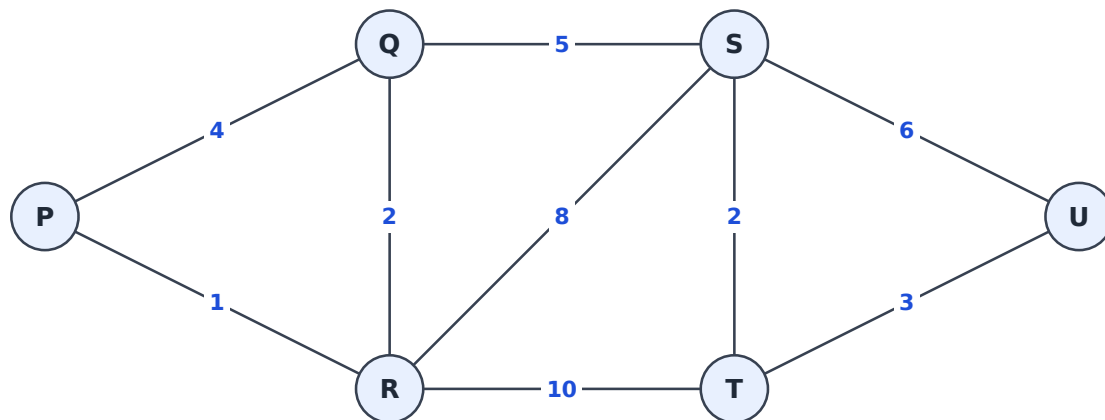
Python

```
1 class P:
2     def __init__(self, x, y):
3         self.x, self.y = x, y
4     def __eq__(self, o):
5         return self.x == o.x
6     def __hash__(self):
7         return hash(self.x % 3)
8
9 s = {P(1, 2), P(4, 2), P(1, 5), P(7, 0)}
10 t = {p.y for p in s}
11 print(len(s), sorted(t))
```

- (A) 4 [0, 2, 5]
- (B) 3 [0, 2]
- (C) 1 [2]
- (D) 3 [0, 2, 5]

Q.17 [NAT · numerical answer | 2 marks | Dijkstra — single-source distances]

Dijkstra's algorithm is run from vertex P on the weighted undirected graph below. The sum of the shortest-path distances from P to all the other five vertices is _____.



Answer: _____ (enter a numerical value)

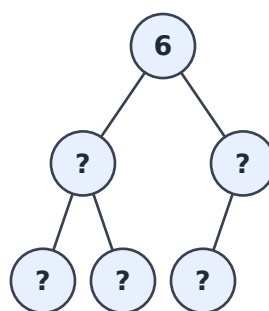
Q.18 [MSQ · one or more correct | 2 marks | Merge sort — comparison counts]

Top-down merge sort (split $[lo, hi]$ at $mid = (lo + hi) // 2$, sort the halves recursively, merge with the standard two-pointer merge; a comparison is one key-vs-key test) is applied to $[6, 2, 9, 4, 7, 1, 8, 3]$. Which of the following statements is/are TRUE?

- (A) Just before the final merge, the two sorted halves are $[2, 4, 6, 9]$ and $[1, 3, 7, 8]$
- (B) The final merge makes exactly 6 comparisons
- (C) No permutation of 8 distinct keys makes this algorithm perform more than 17 comparisons
- (D) The total number of comparisons is 17

Q.19 [MCQ · single correct | 2 marks | Heaps — counting distinct max-heaps]

How many distinct binary max-heaps (stored as arrays, i.e. complete binary trees) can be built using all of the keys 1, 2, 3, 4, 5, 6 exactly once?



Shape of every 6-node heap

- (A) 40
- (B) 16
- (C) 32
- (D) 20

Q.20 [MSQ · one or more correct | 2 marks | Open addressing — expected probe counts]

An open-addressing hash table has load factor $\alpha = 0.75$. Use the standard asymptotic estimates (uniform hashing for (A)–(B), Knuth's linear-probing estimates for (C)). Which of the following statements is/are TRUE?

- (A) Under linear probing, the expected number of probes in an unsuccessful search is about 8.5
- (B) Under uniform hashing, the expected number of probes in an unsuccessful search is at most 4
- (C) Under uniform hashing, the expected number of probes in a successful search is about 1.85
- (D) Separate chaining could not be used at this load factor, since chaining requires $\alpha \leq 0.5$

Answer Key — Mock Test 23

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — dict keys that hash and compare equal
2	NAT	1	0.34 to 0.36	Simple uniform hashing — empty slot probability
3	NAT	1	3.79 to 3.81	Expected number of collisions
4	MCQ	1	(C)	Chaining — average successful search cost
5	MSQ	1	(B), (C), (D)	Python — hashability
6	MCQ	1	(A)	Binary search — bisect boundaries
7	NAT	1	4	Stacks — postfix evaluation depth
8	MCQ	1	(B)	Python — linked-list pointer surgery
9	MCQ	1	(C)	Bubble sort with early termination
10	MSQ	1	(C), (D)	Adjacency matrix — reading graph properties
11	MSQ	2	(A), (C), (D)	Linear probing — reconstructing insertion order
12	NAT	2	2.79 to 2.81	Linear probing — expected unsuccessful search cost
13	NAT	2	15	Double hashing — probe counting
14	MCQ	2	(D)	Probability of no collision
15	NAT	2	11	Linear probing with deletion (tombstones)
16	MCQ	2	(B)	Python — <code>__eq__</code> and <code>__hash__</code> in sets
17	NAT	2	35	Dijkstra — single-source distances
18	MSQ	2	(A), (C), (D)	Merge sort — comparison counts
19	MCQ	2	(D)	Heaps — counting distinct max-heaps
20	MSQ	2	(A), (B), (C)	Open addressing — expected probe counts

Detailed Solutions — Mock Test 23

Q.1 [MCQ · 1 mark · Python — dict keys that hash and compare equal]

Answer: (A)

A dict locates a key by `hash(key)` and then confirms with `==`. In Python `hash(1) == hash(True) == hash(1.0)` and `1 == True == 1.0`, so all three literals denote the **same key**.

- `1: 'a'` creates the entry with key object `1`.
- `True: 'b'` finds the existing equal key; the **value** is replaced by 'b', but the original key object `1` is kept.
- `1.0: 'c'` again updates only the value $\rightarrow$ 'c'.

So `len(d) = 1`, `d[1] = 'c'`, and the stored key is still the int `1`. Output: **1 c 1**.

- (B) treats the three keys as different objects — they are equal, so they collide *and* match.
- (C) assumes the first value is kept (it is the first **key** that is kept).
- (D) assumes the key object is replaced by the last one written.

Trap: a dict update replaces the value but never the key object. **Tip:** equal objects must have equal hashes — this is the contract every hash table relies on.

Q.2 [NAT · 1 mark · Simple uniform hashing — empty slot probability]

Answer: 0.34 to 0.36

Slot 0 stays empty iff **every** key misses it. Each key misses slot 0 with probability $1 - 1/10 = 0.9$, independently, so

$$P(\text{slot 0 empty}) = 0.9^{10} \approx 0.3487 \approx \mathbf{0.35}.$$

Consequences:

- The expected number of empty slots is $10 \times 0.3487 \approx 3.49$ — even though $n = m$ (load factor $\alpha = 1$), about a third of the table is unused, so some chains must be longer than 1.
- For large m with $n = m$ this tends to $1/e \approx 0.368$.

Trap: computing $1 - 10 \cdot (1/10) = 0$ (union bound misuse), or $1/10$ (the chance a *single* key hits slot 0).

Q.3 [NAT · 1 mark · Expected number of collisions]

Answer: 3.79 to 3.81

Use **linearity of expectation** with an indicator for every pair $\{x, y\}$:

- $P(h(x) = h(y)) = 1/m = 1/50$ for each pair (whatever slot x picks, y must match it).
- Number of pairs $= C(20, 2) = 190$.

$$E[\text{collision pairs}] = 190 \times 1/50 = \mathbf{3.80}.$$

Note that linearity holds even though the indicators are **not** independent. This is the birthday-paradox calculation: with n keys and m slots, collisions become likely once $C(n, 2) \approx m$, i.e. $n \approx \sqrt{2m} = 10$ here.

Trap: using $n/m = 0.4$ (the load factor) or $n^2/m = 8$ (ordered pairs, counting each pair twice and including a key with itself).

Q.4 [MCQ · 1 mark · Chaining — average successful search cost]

Answer: (C)

Finding the key in position j of its chain (1-based) costs j comparisons. Average over all 9 stored keys.

- slot 0: 21 → 1, 35 → 2 (sum 3)
- slot 2: 9 → 1 (sum 1)
- slot 3: 17, 31, 45, 10 → 1 + 2 + 3 + 4 (sum 10)
- slot 5: 12 → 1, 26 → 2 (sum 3)

Total = 17 comparisons over 9 keys → **17/9 ≈ 1.89**.

- (A) $9/7 = \alpha$ is the expected cost of an **unsuccessful** search (comparisons only) when the home slot is uniform over all 7 slots.
- (B) 2 is a guess from the textbook estimate $1 + \alpha/2 \approx 1.64$ — such formulas give *expectations over random tables*, not the exact cost for this particular table.
- (D) 13/9 forgets that the 4-chain costs 10, not 6.

Tip: the one long chain at slot 3 contributes 10 of the 17 comparisons — uneven chains hurt more than the load factor suggests.

Q.5 [MSQ · 1 mark · Python — hashability]

Answer: (B), (C), (D)

Only hashable objects can be dict keys or set members. Lists are mutable and unhashable; a tuple is hashable only if **all** its elements are.

- (A) tuples are ordered: $(2, 1) \neq (1, 2)$. **False.**
- (B) hashing a tuple hashes its elements; the inner list fails. **True.**
- (C) 0, 0.0 and False are equal with equal hashes → one element; " is different → size 2. **True.**
- (D) a list as a key → **TypeError: unhashable type: 'list'**. **True.**

Trap: (B) — a tuple *looks* immutable but is only as hashable as its contents. **Tip:** for unordered pairs use **frozenset**.

Q.6 [MCQ · 1 mark · Binary search — bisect boundaries]

Answer: (A)

bisect_left(a, x) returns the first index i with $a[i] \geq x$; **bisect_right(a, x)** returns the first index with $a[i] > x$. Both use binary search: $\Theta(\log n)$.

- **bisect_left(a, 3) = 1, bisect_right(a, 3) = 4** → difference **3** = number of 3s.
- **bisect_left(a, 6)**: 6 is absent; the first element ≥ 6 is 8 at index **5** (the insertion point).
- **bisect_right(a, 8)**: the first element > 8 is 10 at index **7**.

Output **3 5 7**.

- (B) uses last-occurrence indices (**bisect_right** – 1) everywhere.
- (C) gives index 4 for 6 — that is where 5 lives; 6 goes *after* 5.
- (D) confuses **bisect_right(a, 8)** with **bisect_left(a, 8) + 1**.

Tip: **bisect_right(a, x) – bisect_left(a, x)** counts occurrences of x in $O(\log n)$.

Q.7 [NAT · 1 mark · Stacks — postfix evaluation depth]

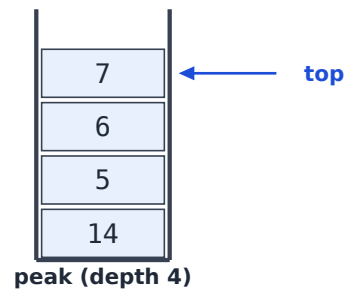
Answer: 4

Track the stack size after each token (operand: +1, binary operator: –2 then +1 = –1):

- 2 → 1, 3 → 2, 4 → 3, * → 2 ($3 \cdot 4 = 12$), + → 1 ($2 + 12 = 14$)
- 5 → 2, 6 → 3, 7 → 4, * → 3 (42), + → 2 (47), * → 1 ($14 \cdot 47 = 658$)

Maximum depth = **4**, reached with [14, 5, 6, 7] on the stack. The expression's value is $658 = (2 + 3 \cdot 4)(5 + 6 \cdot 7)$.

Trap: the depth is not the number of operands in a sub-expression; the earlier result 14 is still on the stack while 5, 6, 7 are pushed. **Tip:** max depth = maximum prefix sum of the ± 1 sequence.

**Q.8 [MCQ · 1 mark · Python — linked-list pointer surgery]****Answer: (B)**

$h = N(v, h)$ inserts at the **head**, so the list is built in reverse: $5 \rightarrow 1 \rightarrow 4 \rightarrow 1 \rightarrow 3$.

The loop removes every second node:

- $p = 5$: $p.nx$ (1) and $p.nx.nx$ (4) exist $\rightarrow 5.nx = 4$; $p = 4$.
- $p = 4$: $p.nx$ (1) and $p.nx.nx$ (3) exist $\rightarrow 4.nx = 3$; $p = 3$.
- $p = 3$: $p.nx$ is None $\rightarrow$ stop.

Remaining list: $5 \rightarrow 4 \rightarrow 3 \rightarrow$ output **[5, 4, 3]**.

- (A) forgets that head insertion reverses the input.
- (C) keeps the nodes at odd positions instead of deleting them.
- (D) stops after one deletion.

Tip: the guard $p.nx$ and $p.nx.nx$ relies on short-circuit evaluation — $p.nx.nx$ is never evaluated when $p.nx$ is None.

**Q.9 [MCQ · 1 mark · Bubble sort with early termination]****Answer: (C)**

Each bubble pass moves the largest remaining element to the end, but a small element moves left by only **one** position per pass. The 1 at index 3 needs 3 passes to reach index 0.

- Pass 1: 9 bubbles to the end (swaps $9 \leftrightarrow 1$, $9 \leftrightarrow 4$, $9 \leftrightarrow 5$, $9 \leftrightarrow 8 = 4$) $\rightarrow [2, 3, 1, 4, 5, 8, 9]$
- Pass 2: $3 \leftrightarrow 1$ (1 swap) $\rightarrow [2, 1, 3, 4, 5, 8, 9]$
- Pass 3: $2 \leftrightarrow 1$ (1 swap) $\rightarrow [1, 2, 3, 4, 5, 8, 9]$
- Pass 4: no swaps $\rightarrow$ stop

Passes = **4**, swaps = **6** (= number of inversions: 9 precedes 1, 4, 5, 8; 2 and 3 precede 1).

- (A) miscounts inversions.
- (B) forgets the verifying pass.
- (D) uses $n - 1 = 6$ passes, ignoring the early exit.

Tip: with the flag, passes = 1 + max over elements of (how far left it must move).

Q.10 [MSQ · 1 mark · Adjacency matrix — reading graph properties]**Answer: (C), (D)**

Edges: a-b, a-c, b-c, b-d, c-e, d-e (6 edges). $(A^2)[u][v]$ counts walks of length 2 from u to v, i.e. common neighbours.

- (A) a, b, c form a triangle (odd cycle). **False.**
- (B) $N(a) = \{b, c\}$, $N(d) = \{b, e\}$; common neighbour only b $\rightarrow$ entry 1. **False.**
- (C) a-b-d-e-c-a uses edges ab, bd, de, ec, ca — all present. **True.**

- (D) $(A^2)[v][v] = \deg(v)$, so $\text{trace}(A^2) = \sum \deg = 2|E| = 12$. **True.**

Tip: $\text{trace}(A^3) = 6 \times (\text{number of triangles})$; here it is 6, confirming exactly one triangle.

Q.11 [MSQ · 2 marks · Linear probing — reconstructing insertion order]

Answer: (A), (C), (D)

A key stored at distance d from its home slot proves that, when it was inserted, **all** of slots home ... home + $d - 1$ were already occupied.

Homes: 61→1, 31→1, 72→2, 13→3, 24→4, 91→1.

- 61 is at its home 1 — no constraint from itself.
- 31 (home 1) is at 2 → slot 1 (61) was full: **61 before 31.**
- 72 (home 2) is at 3 → slot 2 (31) was full: **31 before 72.**
- 13 (home 3) is at 4 → slot 3 (72) was full: **72 before 13.**
- 24 (home 4) is at 5 → slot 4 (13) was full: **13 before 24.**
- 91 (home 1) is at 6 → slots 1-5 all full: **91 last.**

The constraints form a chain $61 < 31 < 72 < 13 < 24 < 91$, so the order is forced.

- (A) **True.**
- (B) 24 needs 13 already in slot 4, and 13 needs 72 → **False.**
- (C) **True.**
- (D) the chain is total → exactly **1** order. **True.**

Trap: assuming the 'probe-chain' constraint applies only to keys with the same home slot; with linear probing a cluster is shared by all homes inside it (primary clustering).

Q.12 [NAT · 2 marks · Linear probing — expected unsuccessful search cost]

Answer: 2.79 to 2.81

An unsuccessful search from home h walks to the end of the cluster containing h . Occupied slots form clusters $\{9, 0, 1, 2, 3\}$ (wrapping) and $\{5, 6\}$; empty slots are 4, 7, 8.

Probes by home slot:

- $h = 9$: 9, 0, 1, 2, 3 full, 4 empty → 6
- $h = 0 \rightarrow 5$, $h = 1 \rightarrow 4$, $h = 2 \rightarrow 3$, $h = 3 \rightarrow 2$
- $h = 4 \rightarrow 1$ (empty immediately)
- $h = 5 \rightarrow 3$, $h = 6 \rightarrow 2$
- $h = 7 \rightarrow 1$, $h = 8 \rightarrow 1$

Sum = $6 + 5 + 4 + 3 + 2 + 1 + 3 + 2 + 1 + 1 = 28 \rightarrow \text{expected} = 28/10 = \mathbf{2.80}$.

Shortcut: a cluster of length L contributes $2 + 3 + \dots + (L + 1) = L(L + 3)/2$, and each empty slot contributes 1: $5 \cdot 8/2 + 2 \cdot 5/2 + 3 = 20 + 5 + 3 = 28$.

Trap: forgetting the wrap-around (treating slot 9 as costing 2) gives $24/10$. **Insight:** the long cluster makes the cost grow roughly with the *square* of cluster length — the essence of primary clustering. For comparison, uniform hashing at $\alpha = 0.7$ predicts $1/(1 - \alpha) \approx 3.33$.

Q.13 [NAT · 2 marks · Double hashing — probe counting]

Answer: 15

Every key here has $h_1 = 1$ (27, 53, 40, 66, 14, 79, 183 are all $\equiv 1 \pmod{13}$), yet double hashing sends them along **different** probe sequences because h_2 differs — this is how double hashing avoids secondary clustering.

- 27: $h_2 = 6$; slot 1 empty → 1 probe
- 53: $h_2 = 10$; 1 full → 11 → 2 probes
- 40: $h_2 = 8$; 1 full → 9 → 2 probes

- 66: $h_2 = 1$; 1 full $\rightarrow 2 \rightarrow 2$ probes
- 14: $h_2 = 4$; 1 full $\rightarrow 5 \rightarrow 2$ probes
- 79: $h_2 = 3$; 1 full $\rightarrow 4 \rightarrow 2$ probes
- 183: $h_2 = 1 + 7 = 8$; 1 full $\rightarrow 9$ full (40) $\rightarrow 17 \bmod 13 = 4$ full (79) $\rightarrow 25 \bmod 13 = \mathbf{12} \rightarrow 4$ probes

Total = $1 + 2 \cdot 5 + 4 = \mathbf{15}$.

Trap: computing h_2 as $k \bmod 11$ (forgetting the $+1$, which guarantees $h_2 \neq 0$). **Tip:** since 13 is prime, every $h_2 \in [1, 12]$ generates a full permutation of the slots.

0	
1	27
2	66
3	
4	79
5	14
6	
7	
8	
9	40
10	
11	53
12	183

Final table

Q.14 [MCQ · 2 marks · Probability of no collision]

Answer: (D)

Count favourable hash assignments: the first key may go anywhere (7 ways), the second must avoid it (6), the third must avoid both (5).

$$P(\text{no collision}) = (7 \cdot 6 \cdot 5) / 7^3 = 210/343 = \mathbf{30/49 \approx 0.612}.$$

Equivalently, $(6/7) \cdot (5/7) = 30/49$.

- (A) $1/7$ is the probability that a **particular** pair collides.
- (B) $36/49 = (6/7)^2$ assumes the third key only has to avoid one slot.
- (C) $6/7$ accounts only for the second key.

So with only 3 keys and 7 slots there is already a $19/49 \approx 39\%$ chance of at least one collision — the birthday paradox in miniature.

Tip: $P(\text{no collision}) = \prod_{i=0}^{n-1} (1 - i/m) \approx e^{-n(n-1)/(2m)} = e^{-3/7} \approx 0.65$.

Q.15 [NAT · 2 marks · Linear probing with deletion (tombstones)]

Answer: 11

Tombstones keep probe chains intact: emptying a slot outright would make later keys of the cluster unreachable.

Build the table:

- $15 \rightarrow 5$; $25 \rightarrow 6$; $35 \rightarrow 7$; $16 \rightarrow 6$, 7 full $\rightarrow 8$; $45 \rightarrow 5$, 6, 7, 8 full $\rightarrow 9$
- delete $25 \rightarrow$ slot 6 = tombstone; delete $16 \rightarrow$ slot 8 = tombstone
- insert 55: home 5 full, slot 6 is a tombstone $\rightarrow 55$ goes to **6**

Final: 5:15, 6:55, 7:35, 8:†, 9:45, others empty.

- search 45: 5 (15), 6 (55), 7 (35), 8 (†, keep going), 9 (found) → **5** probes
- search 65: 5, 6, 7, 8 (†), 9, 0 (empty → stop) → **6** probes

Total = 5 + 6 = **11**.

Trap: stopping at the tombstone in slot 8 would wrongly report 45 as absent (4 probes) — exactly the bug tombstones exist to prevent. **Note:** tombstones lengthen unsuccessful searches until the table is rebuilt.

0	
1	
2	
3	
4	
5	15
6	55
7	35
8	†
9	45

Table before the two searches († = tombstone)

Q.16 [MCQ · 2 marks · Python — `__eq__` and `__hash__` in sets]

Answer: (B)

A set first compares hashes, then uses `==` to decide whether an element is already present. Here every object has `hash(x % 3) = hash(1)` ($x = 1, 4, 1, 7$ are all $\equiv 1 \pmod{3}$), so all four land in the same bucket — but `__eq__` compares only `x`.

- add P(1, 2): new
- add P(4, 2): same hash, $4 \neq 1 \rightarrow$ new
- add P(1, 5): same hash and `x` equal to P(1, 2) → **duplicate; not added** (the first object, with `y = 2`, stays)
- add P(7, 0): new

`len(s) = 3`, and the stored `y`-values are `{2, 2, 0}` → set `{0, 2}`. Output **3 [0, 2]**.

- (A) assumes objects are distinct by identity (ignores the custom `__eq__`).
- (C) assumes equal hashes imply equal elements — a hash collision is not equality.
- (D) assumes the later P(1, 5) replaces P(1, 2); a set keeps the existing element.

Tip: a constant-ish hash is *correct* but degrades the set to a linear scan.

Q.17 [NAT · 2 marks · Dijkstra — single-source distances]

Answer: 35

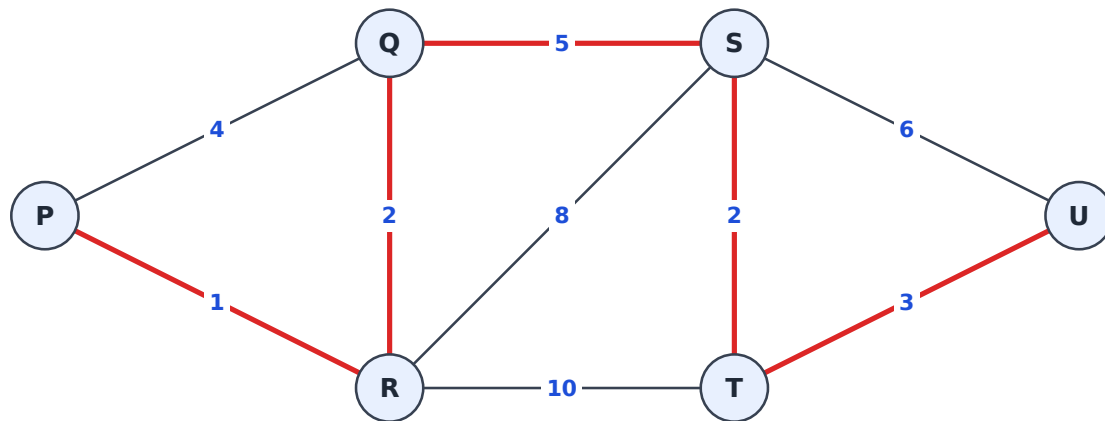
Dijkstra repeatedly finalises the unvisited vertex with the smallest tentative distance.

- Extract P (0): $Q = 4, R = 1$
- Extract R (1): $Q = \min(4, 1 + 2) = 3; S = 9; T = 11$
- Extract Q (3): $S = \min(9, 3 + 5) = 8$
- Extract S (8): $T = \min(11, 8 + 2) = 10; U = 14$
- Extract T (10): $U = \min(14, 10 + 3) = 13$
- Extract U (13)

Distances: Q 3, R 1, S 8, T 10, U 13 → sum = **35**.

Shortest-path tree: P-R, R-Q, Q-S, S-T, T-U (a single path P-R-Q-S-T-U).

Trap: taking the direct edges (P-Q = 4, R-T = 10 + 1 = 11, S-U = 8 + 6 = 14) without relaxing via cheaper detours gives 4 + 1 + 8 + 11 + 14 = 38.



Shortest-path tree from P

Q.18 [MSQ · 2 marks · Merge sort — comparison counts]

Answer: (A), (C), (D)

Merging runs of sizes p and q costs at most $p + q - 1$ comparisons (the maximum is reached when the runs interleave until the very end).

- Level 1: [6]+[2], [9]+[4], [7]+[1], [8]+[3] → 1 each = 4
- Level 2: [2 6]+[4 9]: 2|4, 6|4, 6|9 → 3; [1 7]+[3 8]: 1|3, 7|3, 7|8 → 3 → 6
- Level 3: [2 4 6 9]+[1 3 7 8]: 1, 2, 3, 4, 6, 7, then 9 vs 8 → 7 comparisons

Total = 4 + 6 + 7 = **17**.

- (A) **True** — the left half [6, 2, 9, 4] sorts to [2, 4, 6, 9], the right to [1, 3, 7, 8].
- (B) the final merge makes **7**. **False**.
- (C) every merge here already hit its maximum $p + q - 1$, so 17 is the worst case for $n = 8$ (formula $n\lceil\log_2 n\rceil - 2^{\lceil\log_2 n\rceil} + 1 = 24 - 8 + 1 = 17$). **True**.
- (D) **True**.

Tip: the best case for $n = 8$ is 12 comparisons (each merge stops after $\min(p, q)$ comparisons, e.g. on sorted input).

Q.19 [MCQ · 2 marks · Heaps — counting distinct max-heaps]

Answer: (D)

The shape of a 6-node complete tree is fixed: the root's left subtree has 3 nodes (indices 1, 3, 4) and its right subtree has 2 nodes (indices 2, 5).

- The root must be the maximum, 6.
- Choose which 3 of the remaining 5 keys go to the left subtree: $C(5, 3) = 10$ ways.
- A 3-node heap (root + 2 children) on fixed keys: the max is the root, the other two can be swapped → $H(3) = 2$.
- A 2-node heap: $H(2) = 1$.

$H(6) = C(5, 3) \cdot H(3) \cdot H(2) = 10 \cdot 2 \cdot 1 = \mathbf{20}$.

- (A) 40 uses $C(5, 2) \cdot 2 \cdot 2$, wrongly giving the 2-node subtree two arrangements.
- (B) $16 = 2^4$ — a guess treating every internal node as a free left/right choice.
- (C) 32 ignores that the 2-node right subtree has only one heap arrangement.

Tip: the recurrence $H(n) = C(n-1, L) \cdot H(L) \cdot H(R)$ gives 1, 1, 2, 3, 8, 20, 80 for $n = 1 \dots 7$.

Q.20 [MSQ · 2 marks · Open addressing — expected probe counts]

Answer: (A), (B), (C)

Standard results for open addressing ($\alpha = n/m < 1$):

- Uniform hashing, unsuccessful: $\leq 1/(1 - \alpha) = 1/0.25 = 4$. (B) **True.**
- Uniform hashing, successful: $\leq (1/\alpha) \ln(1/(1 - \alpha)) = (4/3) \ln 4 \approx (4/3)(1.386) \approx 1.85$. (C) **True.**
- Linear probing (Knuth): unsuccessful $\approx \frac{1}{2}(1 + 1/(1 - \alpha)^2) = \frac{1}{2}(1 + 16) = 8.5$; successful $\approx \frac{1}{2}(1 + 1/(1 - \alpha)) = 2.5$. (A) **True.**
- (D) Chaining works for **any** α (even $\alpha > 1$); the expected search cost is $\Theta(1 + \alpha)$. **False.**

Comparing (B) with (A): at the same load factor, primary clustering makes linear probing's unsuccessful search more than twice as expensive as ideal uniform hashing.

Trap: mixing up the successful and unsuccessful formulas — successful searches are always cheaper because they stop at the key, on average part-way along the probe sequence.

Mock Test 24

Sorting Properties

Difficulty: GATE-level

Focus areas: stability, inversions, adaptivity, in-place, comparison lower bounds

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — stable multi-pass sorting]

What is printed by the following Python program?

Python

```
1 recs = [("ravi", 3), ("asha", 1), ("dev", 3),
2         ("mina", 2), ("om", 1), ("lata", 2)]
3 r = sorted(recs, key=lambda t: t[0])
4 r = sorted(r, key=lambda t: -t[1])
5 print([name for name, _ in r])
```

- (A) ['ravi', 'dev', 'mina', 'lata', 'asha', 'om']
- (B) ['dev', 'ravi', 'lata', 'mina', 'asha', 'om']
- (C) ['asha', 'om', 'lata', 'mina', 'dev', 'ravi']
- (D) ['dev', 'ravi', 'mina', 'lata', 'om', 'asha']

Q.2 [NAT · numerical answer | 1 mark | Bubble sort — adaptivity (rabbits and turtles)]

Consider bubble sort with early termination:

for i in range(n-1): one left-to-right pass over A[0..n-2-i] swapping adjacent out-of-order pairs; if no swap was made in this pass: break.

Let p be the number of passes executed on X = [2, 3, 4, 5, 6, 1] and q the number of passes executed on Y = [6, 1, 2, 3, 4, 5]. The value of p + q is ____.

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Sorting — stability of standard algorithms]

Each algorithm below is implemented in its standard textbook form on an array and sorts records by key in ascending order. Which of them is/are **stable** for every input?

- (A) Selection sort that swaps the minimum of A[i..n-1] (first occurrence) with A[i]
- (B) Merge sort whose merge takes from the left half when the two front keys are equal
- (C) Quicksort using the Lomuto partition with the last element as pivot
- (D) Insertion sort that shifts A[j] right while A[j] > key

Q.4 [NAT · numerical answer | 1 mark | Binary search — lower bound with duplicates]

Consider the following Python program. It prints two integers p and q. The value of p + q is _____.

Python

```
1 def lb(A, x):
2     lo, hi, c = 0, len(A), 0
3     while lo < hi:
4         mid = (lo + hi) // 2
5         c += 1
6         if A[mid] < x:
7             lo = mid + 1
8         else:
9             hi = mid
10    return lo, c
11
12 A = [2, 5, 5, 5, 8, 8, 11, 14, 14, 14, 14, 20]
13 p, q = lb(A, 14)
14 print(p, q)
```

Answer: _____ (enter a numerical value)

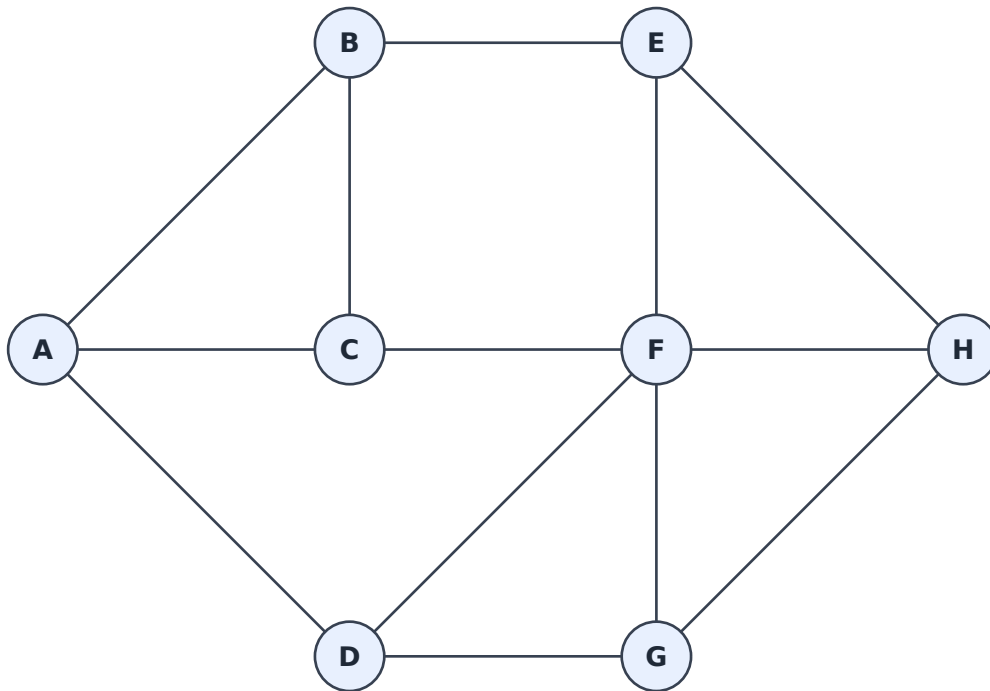
Q.5 [MCQ · single correct | 1 mark | Stacks — stack permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack in this order; pops may be performed at any time, and each popped value is printed. Which of the following output sequences is **not** possible?

- (A) 4 5 3 6 1 2
- (B) 3 2 5 6 4 1
- (C) 1 2 3 6 5 4
- (D) 4 3 5 2 6 1

Q.6 [NAT · numerical answer | 1 mark | BFS — levels and non-tree edges]

BFS is run from vertex A on the undirected graph below, enqueueing undiscovered neighbours in alphabetical order. The number of graph edges whose two endpoints lie at the **same** BFS level is _____.



Answer: _____ (enter a numerical value)

Q.7 [NAT · numerical answer | 1 mark | Python — sort keys with negative modulo]

What value is printed by the following Python program?

Python

```
1 L = [7, -4, 12, -9, 5, 3, -1, 10]
2 r = sorted(L, key=lambda x: (x % 3, -x))
3 print(sum(r[:4]))
```

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Sorting — auxiliary space]

Which of the following statements about the extra space used by sorting algorithms is/are TRUE?

- (A) Heapsort sorts an array using $O(1)$ auxiliary space
- (B) Merging two sorted singly linked lists of total length n by relinking nodes requires $\Theta(n)$ auxiliary space
- (C) Top-down merge sort on an array of n elements, with the usual merge, uses $\Theta(n)$ auxiliary space
- (D) Quicksort that recurses on the smaller part and loops on the larger part needs only $O(\log n)$ stack depth, even in the worst case

Q.9 [MCQ · single correct | 1 mark | Hashing — linear probing, unsuccessful search]

A hash table of size 11 uses $h(k) = k \bmod 11$ and linear probing. After inserting 22, 33, 14, 25, 47, 30, 41 (in that order) it looks as shown. Assuming the home slot of a key that is **not** in the table is equally likely to be any of the 11 slots, the expected number of slots probed by an unsuccessful search (counting the final empty slot) is

0	22
1	33
2	
3	14
4	25
5	47
6	
7	
8	30
9	41
10	

- (A) 12/11
- (B) 23/11
- (C) 18/11
- (D) 28/11

Q.10 [MCQ · single correct | 1 mark | BST counting]

How many structurally distinct binary search trees can be formed with the keys 1, 2, 3, 4, 5, 6 such that the root holds the key **3**?

- (A) 20
- (B) 14
- (C) 5
- (D) 10

Q.11 [MSQ · one or more correct | 2 marks | Inversions — insertion sort, bubble sort, reversal]

Let $A = [4, 9, 2, 7, 5, 1, 8, 3]$. An inversion is a pair of indices $i < j$ with $A[i] > A[j]$. Which of the following statements is/are TRUE?

- (A) Bubble sort with early termination (stop after a pass with no swap) executes all $n - 1 = 7$ passes on A
- (B) Insertion sort (shift while $A[j] > \text{key}$) performs exactly 16 element shifts on A
- (C) A has exactly 16 inversions
- (D) The reverse of A, $[3, 8, 1, 5, 7, 2, 9, 4]$, has exactly 12 inversions

Q.12 [NAT · numerical answer | 2 marks | Insertion sort — exact comparison count]

Insertion sort is applied to $A = [13, 6, 21, 2, 17, 9, 25, 4, 11]$:

```
for i in 1..n-1: x = A[i]; j = i - 1; while j ≥ 0 and A[j] > x: A[j+1] = A[j]; j -= 1; A[j+1] = x
```

Count one key comparison each time $A[j] > x$ is evaluated (it is not evaluated when $j < 0$). The total number of key comparisons is _____.

Answer: _____ (enter a numerical value)

Q.13 [MCQ · single correct | 2 marks | Selection sort — instability]

Records (key, tag) = [(2, a), (3, b), (2, c), (1, d), (3, e)] are sorted by key with selection sort: in pass i ($i = 0, 1, 2, 3$) the record with the smallest key in positions $i..4$ (the **leftmost** one in case of ties) is swapped with the record at position i . The final order of the tags is

- (A) d, a, c, b, e
- (B) d, c, a, e, b
- (C) d, c, a, b, e
- (D) d, a, c, e, b

Q.14 [NAT · numerical answer | 2 marks | Divide and conquer — counting split inversions]

Inversions in $A = [10, 3, 15, 8, 1, 12, 6, 14]$ are counted with the merge-sort method: each half is recursively sorted (counting its own inversions), and during the final merge of the two sorted halves, whenever an element is taken from the **right** half, the number of elements still remaining in the left half is added to the count. The amount added during the **final (top-level) merge only** is _____.

A =	10	3	15	8	1	12	6	14
	0	1	2	3	4	5	6	7

Left half = indices 0–3, right half = indices 4–7

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Comparison lower bounds — merging]

Two sorted lists, each containing 3 elements, are to be merged into one sorted list; all 6 elements are distinct. What is the smallest number k such that some comparison-based algorithm always merges them using at most k comparisons?

- (A) 4
- (B) 6
- (C) 5
- (D) 10

Q.16 [MSQ · one or more correct | 2 marks | Python — sorted(), reverse and stability]

Consider the following Python program (it prints nothing). Which of the following statements is/are TRUE after it runs?

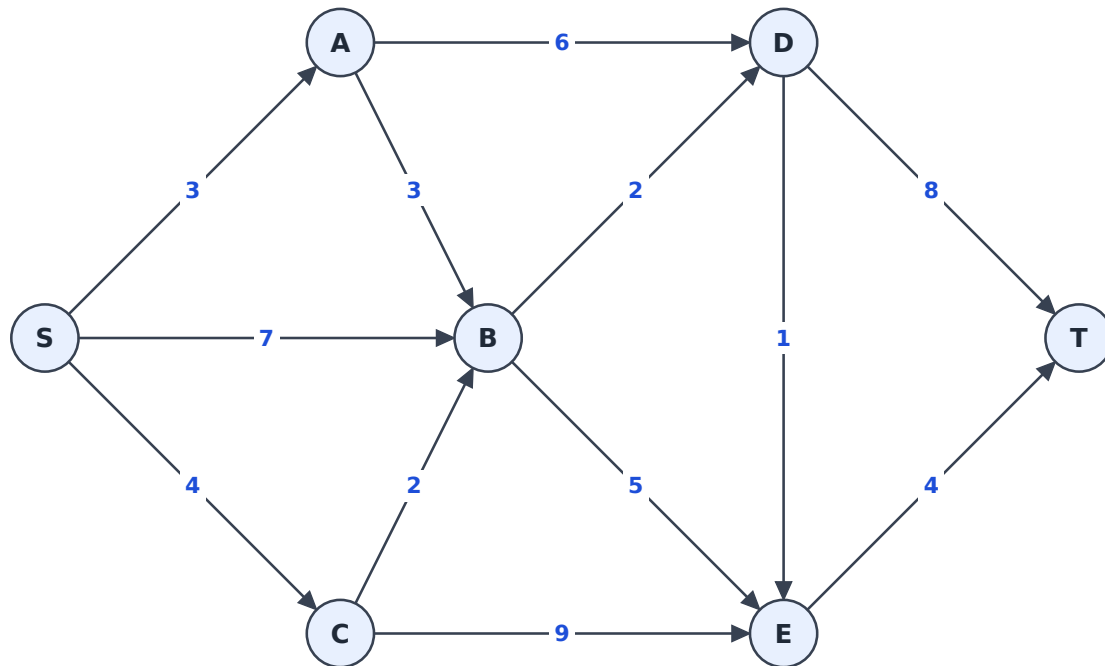
Python

```
1 data = [("p", 2), ("q", 1), ("r", 2), ("s", 1)]
2 a = sorted(data, key=lambda t: t[1], reverse=True)
3 b = sorted(data, key=lambda t: t[1])[:-1]
4 c = sorted(data, key=lambda t: -t[1])
5 d = data.sort(key=lambda t: t[1])
```

- (A) `a == c` evaluates to True
- (B) The names in `b`, in order, are r, p, s, q
- (C) `a == b` evaluates to True
- (D) `d` is None and `data` is now `[('q', 1), ('s', 1), ('p', 2), ('r', 2)]`

Q.17 [NAT · numerical answer | 2 marks | Shortest paths — tight edges]

For the weighted directed graph below, let $d(v)$ be the shortest-path distance from S. An edge (u, v) with weight w **lies on some shortest path from S** iff $d(u) + w = d(v)$. The number of edges of the graph that lie on some shortest path from S is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Linked lists — stable partition]

The function `part` rearranges a singly linked list around a value x . The list $7 \rightarrow 3 \rightarrow 9 \rightarrow 3 \rightarrow 5 \rightarrow 1 \rightarrow 8$ is partitioned with $x = 5$. What is printed?

Python

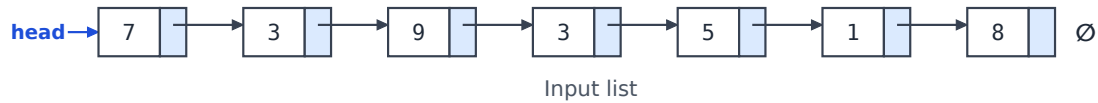
```

1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 def part(head, x):
6     lt = lt_t = Node(0)
7     ge = ge_t = Node(0)
8     while head:
9         if head.val < x:
10             lt_t.next = head
11             lt_t = head
12         else:
13             ge_t.next = head
14             ge_t = head
15         head = head.next
16     ge_t.next = None
17     lt_t.next = ge.next
18     return lt.next
19
20 head = None
21 for v in reversed([7, 3, 9, 3, 5, 1, 8]):
22     head = Node(v, head)
23 p, out = part(head, 5), []
24 while p:
25     out.append(p.val)
  
```

```

26     p = p.next
27     print(out)

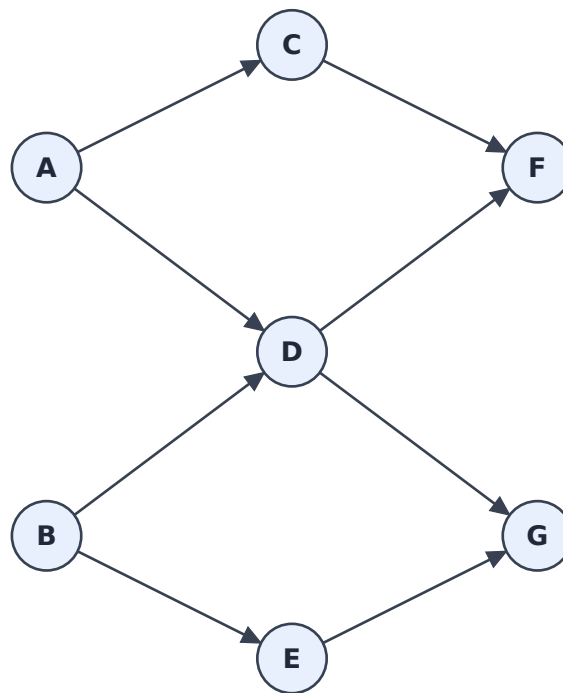
```



- (A) [1, 3, 3, 5, 7, 8, 9]
 (B) [3, 3, 1, 5, 7, 8, 9]
 (C) [1, 3, 3, 8, 5, 9, 7]
 (D) [3, 3, 1, 7, 9, 5, 8]

Q.19 [MSQ · one or more correct | 2 marks | Topological sorting — DFS and Kahn's algorithm]

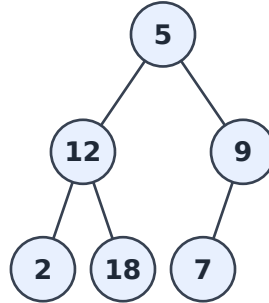
Consider the DAG below. Which of the following statements is/are TRUE?



- (A) D precedes E in every topological order
 (B) DFS with the outer loop over vertices in alphabetical order and out-neighbours explored alphabetically, outputting vertices in reverse order of finishing, gives B, E, A, D, G, C, F
 (C) C, A, B, D, E, F, G is a valid topological order
 (D) Kahn's algorithm that always removes the alphabetically smallest current source outputs A, B, C, D, E, F, G

Q.20 [MCQ · single correct | 2 marks | Heapsort — intermediate array state]

Heapsort (ascending) is applied to $A = [5, 12, 9, 2, 18, 7]$: first a max-heap is built bottom-up (sift-down at $i = 2, 1, 0$), then repeatedly the root is swapped with the last element of the heap, the heap size is decreased by one, and the new root is sifted down. What is the array after the build-heap phase **and** the first **two** root extractions?



Initial array as a complete binary tree

- (A) [9, 7, 5, 2, 12, 18]
- (B) [12, 7, 9, 2, 5, 18]
- (C) [9, 5, 7, 2, 12, 18]
- (D) [7, 9, 5, 2, 12, 18]

Answer Key — Mock Test 24

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — stable multi-pass sorting
2	NAT	1	7	Bubble sort — adaptivity (rabbits and turtles)
3	MSQ	1	(B), (D)	Sorting — stability of standard algorithms
4	NAT	1	11	Binary search — lower bound with duplicates
5	MCQ	1	(A)	Stacks — stack permutations
6	NAT	1	3	BFS — levels and non-tree edges
7	NAT	1	16	Python — sort keys with negative modulo
8	MSQ	1	(A), (C), (D)	Sorting — auxiliary space
9	MCQ	1	(B)	Hashing — linear probing, unsuccessful search
10	MCQ	1	(D)	BST counting
11	MSQ	2	(B), (C), (D)	Inversions — insertion sort, bubble sort, reversal
12	NAT	2	24	Insertion sort — exact comparison count
13	MCQ	2	(C)	Selection sort — instability
14	NAT	2	9	Divide and conquer — counting split inversions
15	MCQ	2	(C)	Comparison lower bounds — merging
16	MSQ	2	(A), (B), (D)	Python — sorted(), reverse and stability
17	NAT	2	7	Shortest paths — tight edges
18	MCQ	2	(D)	Linked lists — stable partition
19	MSQ	2	(B), (D)	Topological sorting — DFS and Kahn's algorithm
20	MCQ	2	(A)	Heapsort — intermediate array state

Detailed Solutions — Mock Test 24

Q.1 [MCQ · 1 mark · Python — stable multi-pass sorting]

Answer: (B)

Concept. Python's `sorted` is **stable**: records with equal keys keep their relative order from the input. Sorting first by the *secondary* key (name) and then by the *primary* key (score, descending via `-t[1]`) therefore yields 'score descending, ties by name ascending'.

Trace.

- After sort by name: asha(1), dev(3), lata(2), mina(2), om(1), ravi(3).
- Stable sort by `-score`: score 3 group keeps order dev, ravi; score 2 group lata, mina; score 1 group asha, om.
- Output: `['dev', 'ravi', 'lata', 'mina', 'asha', 'om']`.

Options.

- (A) keeps the *original* input order within ties — that would be the result of the second sort alone.
- (C) sorts by score ascending.
- (D) reverses the order inside each tie group, as an unstable sort might.

Tip: this 'sort by least significant key first' trick is exactly how LSD radix sort works, and it is correct only because each pass is stable. Equivalent one-liner: `sorted(recs, key=lambda t: (-t[1], t[0]))`.

Q.2 [NAT · 1 mark · Bubble sort — adaptivity (rabbits and turtles)]

Answer: 7

Concept. In one left-to-right pass a large element can travel any distance to the right (a *rabbit*), but a small element moves only **one** position to the left (a *turtle*). Bubble sort's pass count is governed by how far the worst element must move left.

X = [2, 3, 4, 5, 6, 1] — the turtle 1 must move 5 places left, one per pass:

- pass 1 → [2, 3, 4, 5, 1, 6], pass 2 → [2, 3, 4, 1, 5, 6], ..., pass 5 → [1, 2, 3, 4, 5, 6].
- Pass 5 still swapped, but the loop has only $n - 1 = 5$ iterations, so it ends. $p = 5$.

Y = [6, 1, 2, 3, 4, 5] — the rabbit 6 reaches the end in one pass:

- pass 1 → [1, 2, 3, 4, 5, 6] (5 swaps); pass 2 makes no swap → break. $q = 2$.

$p + q = 5 + 2 = 7$.

Trap: both arrays have exactly 5 inversions, so they need the same number of *swaps*, yet the number of *passes* differs greatly. Passes = 1 + max over elements of how far left they must move (capped at $n - 1$).

Q.3 [MSQ · 1 mark · Sorting — stability of standard algorithms]

Answer: (B), (D)

Concept. A sort is stable if equal keys keep their input order. Algorithms that move elements only between *adjacent* positions (or merge with a left-preference on ties) are stable; algorithms that make long-distance swaps usually are not.

- (A) **Not stable.** The long swap can jump an element over its equal twin. Example: [2a, 2b, 1] → swap 2a with 1 → [1, 2b, 2a].
- (B) **Stable.** On ties the left (earlier) record is output first.
- (C) **Not stable.** Example [1a, 1b, 0]: pivot 0; nothing ≤ 0 in the loop, final swap exchanges A[0] and A[2] → [0, 1b, 1a].
- (D) **Stable.** The strict test `A[j] > key` stops at an equal key, so a later equal key is placed after it.

Trap: stability is a property of the *implementation*: insertion sort with $\geq$ or merge sort preferring the right half on ties become unstable.

Q.4 [NAT · 1 mark · Binary search — lower bound with duplicates]

Answer: 11

Concept. This is the half-open **lower_bound** search: it returns the first index whose value is $\geq x$, and it never stops early on equality — it keeps shrinking until $lo = hi$.

Trace ($n = 12, x = 14$):

- $lo=0, hi=12 \rightarrow mid=6, A[6]=11 < 14 \rightarrow lo=7. (c=1)$
- $lo=7, hi=12 \rightarrow mid=9, A[9]=14 \geq 14 \rightarrow hi=9. (c=2)$
- $lo=7, hi=9 \rightarrow mid=8, A[8]=14 \rightarrow hi=8. (c=3)$
- $lo=7, hi=8 \rightarrow mid=7, A[7]=14 \rightarrow hi=7. (c=4)$
- $lo = hi = 7 \rightarrow \text{stop}.$

$p = 7$ (the first 14), $q = 4$ iterations; $p + q = \mathbf{11}$.

Trap: a classic binary search would stop at the first probe that hits a 14 (index 9) and return a *non-leftmost* occurrence. Lower-bound search always runs $\lceil \log_2(n+1) \rceil$ or $\lfloor \log_2(n+1) \rfloor$ iterations regardless of where x is found.



Probed indices 6, 9, 8, 7; answer index 7

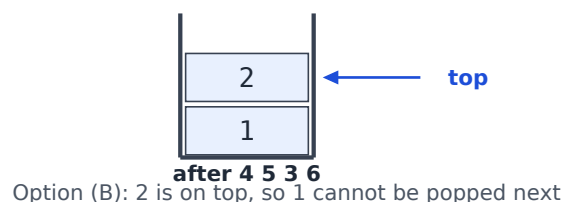
Q.5 [MCQ · 1 mark · Stacks — stack permutations]

Answer: (A)

Concept. A sequence is a valid stack permutation iff it never contains a pattern $\dots c \dots a \dots b \dots$ with $a < b < c$ (i.e. after printing c , smaller values still in the stack must come out in decreasing order).

- (A) push 1..4 pop 4; push 5 pop 5; pop 3; push 6 pop 6. Now the stack holds 1, 2 with 2 on top, so the next pop must be 2 — printing 1 is impossible. **Not possible** (pattern 4...1...2).
- (B) push 1,2,3 pop 3, pop 2; push 4,5 pop 5; push 6 pop 6; pop 4, pop 1. **Possible.**
- (C) pop 1, 2, 3 as they arrive; push 4,5,6 and pop 6, 5, 4. **Possible.**
- (D) push 1..4 pop 4, pop 3; push 5 pop 5; pop 2; push 6 pop 6; pop 1. **Possible.**

Tip: simulate greedily — push until the wanted value is on top; if it is buried below the top, the sequence is impossible.



Q.6 [NAT · 1 mark · BFS — levels and non-tree edges]**Answer: 3**

Concept. In an undirected BFS every edge joins vertices whose levels differ by 0 or 1. Tree edges and some non-tree edges go between consecutive levels; the remaining non-tree edges join two vertices of the same level (they close odd cycles).

Levels.

- Level 0: A.
- Level 1: B, C, D (neighbours of A).
- Level 2: E (from B), F (from C), G (from D).
- Level 3: H (from E).

Classifying the 13 edges.

- Tree edges (7): AB, AC, AD, BE, CF, DG, EH.
- Between consecutive levels, non-tree: DF, FH, GH (3).
- Same level: **BC** (level 1), **EF** and **FG** (level 2) → **3**.

Check: $7 + 3 + 3 = 13$ edges.

Tip: a graph is bipartite iff no BFS produces a same-level edge, so this count being non-zero proves the graph has an odd cycle (e.g. A-B-C).

Q.7 [NAT · 1 mark · Python — sort keys with negative modulo]**Answer: 16**

Concept. In Python, $x \% m$ with $m > 0$ is always in $[0, m - 1]$ (the result takes the sign of the divisor): $-4 \% 3 = 2$, $-1 \% 3 = 2$, $-9 \% 3 = 0$. Tuples compare lexicographically, so the key sorts by remainder first, then by value descending.

Remainders. $7 \rightarrow 1$, $-4 \rightarrow 2$, $12 \rightarrow 0$, $-9 \rightarrow 0$, $5 \rightarrow 2$, $3 \rightarrow 0$, $-1 \rightarrow 2$, $10 \rightarrow 1$.

Groups (each ordered by $-x$, i.e. larger first).

- remainder 0: 12, 3, -9
- remainder 1: 10, 7
- remainder 2: 5, -1, -4

$r = [12, 3, -9, 10, 7, 5, -1, -4]$; first four sum to $12 + 3 - 9 + 10 = \mathbf{16}$.

Trap: using C/Java semantics ($-4 \% 3 = -1$, $-1 \% 3 = -1$) puts -1 and -4 in a 'remainder -1' group at the very front and changes the answer completely.

Q.8 [MSQ · 1 mark · Sorting — auxiliary space]**Answer: (A), (C), (D)**

- (A) **True.** Build-heap and the repeated swap-root-with-last + sift-down work entirely inside the array; only a few index variables are needed.
- (B) **False.** Linked-list merge only changes **next** pointers using a constant number of extra pointer variables: $O(1)$ space. This is why merge sort is preferred for linked lists.
- (C) **True.** The array merge copies the two runs into a buffer (or writes into one); the top-level merge alone needs $n/2$ to n extra cells.
- (D) **True.** Each recursive call is on a part of size $\leq$ half of the current range, so the depth is $\leq \log_2 n$. The larger part is handled by the loop (tail-call elimination). The *time* can still be $\Theta(n^2)$ — only the stack is bounded.

Trap: confusing worst-case *time* of quicksort ($\Theta(n^2)$) with its worst-case *space*; with the smaller-side-first trick they are independent.

Q.9 [MCQ · 1 mark · Hashing — linear probing, unsuccessful search]**Answer: (B)**

Concept. An unsuccessful search starting at slot i probes every occupied slot of the run (cluster) starting at i , then the first empty slot. So $\text{cost}(i) = 1 + (\text{number of consecutive occupied slots from } i)$.

Occupied: 0, 1 | 3, 4, 5 | 8, 9. Empty: 2, 6, 7, 10.

- $i = 0$: 0, 1, 2 $\rightarrow$ 3
- $i = 1$: 1, 2 $\rightarrow$ 2
- $i = 2$: 1
- $i = 3$: 3, 4, 5, 6 $\rightarrow$ 4
- $i = 4$: 3
- $i = 5$: 2
- $i = 6$: 1
- $i = 7$: 1
- $i = 8$: 8, 9, 10 $\rightarrow$ 3
- $i = 9$: 2
- $i = 10$: 1

Sum = $3 + 2 + 1 + 4 + 3 + 2 + 1 + 1 + 3 + 2 + 1 = 23 \rightarrow$ expected **23/11 $\approx$ 2.09**.

Options. (A) 12/11 forgets to count the final empty slot ($23 - 11 = 12$); (C) $18/11 = 1 + 7/11$ counts each occupied slot once, ignoring clustering; (D) 28/11 charges every slot of a cluster the *whole* cluster length instead of the part from the home slot onwards.

Insight: a cluster of length L contributes $L(L + 1)/2$ extra probes — long clusters are quadratically costly, which is the essence of *primary clustering*.

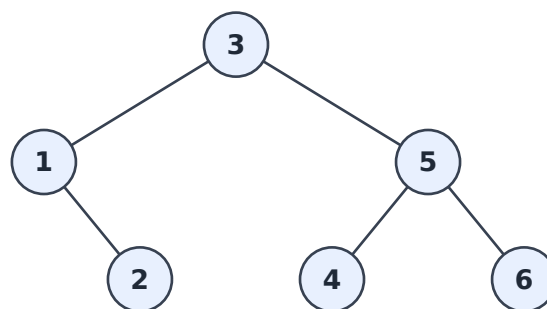
Q.10 [MCQ · 1 mark · BST counting]**Answer: (D)**

Concept. With root r , the left subtree is a BST on the $r - 1$ smaller keys and the right subtree a BST on the $n - r$ larger keys, chosen independently. The number of BSTs on k keys is the Catalan number C_k (1, 1, 2, 5, 14, 42, ...).

- Left subtree: keys {1, 2} $\rightarrow C_2 = 2$ shapes.
- Right subtree: keys {4, 5, 6} $\rightarrow C_3 = 5$ shapes.
- Total = $2 \times 5 = 10$.

Options. (B) $14 = C_4$ is the count for 4 keys, (C) 5 forgets the left subtree's two shapes, (A) 20 wrongly doubles the count, as if mirror images were extra trees.

Check: summing over all roots gives $C_0C_5 + C_1C_4 + C_2C_3 + C_3C_2 + C_4C_1 + C_5C_0 = 42 + 14 + 10 + 10 + 14 + 42 = 132 = C_6$.



One of the 10 trees

Q.11 [MSQ · 2 marks · Inversions — insertion sort, bubble sort, reversal]**Answer: (B), (C), (D)****Counting inversions** (for each element, how many later elements are smaller):

- 4: {2, 1, 3} → 3
- 9: {2, 7, 5, 1, 8, 3} → 6
- 2: {1} → 1
- 7: {5, 1, 3} → 3
- 5: {1, 3} → 2
- 1: 0
- 8: {3} → 1
- 3: 0

Total = 3 + 6 + 1 + 3 + 2 + 0 + 1 + 0 = **16**. (C) **True**.

- (A) **False**. Each pass moves an element at most one place to the left. The element that must travel farthest left is 1 (index 5 → 0, five places) — 3 travels from index 7 to 2, also five. So 5 passes sort the array and the 6th pass makes no swap and stops: 6 passes, not 7.
- (B) **True**. Every shift in insertion sort swaps one adjacent inverted pair, removing exactly one inversion, and the sort ends with 0 inversions → shifts = inversions = 16. (Bubble sort likewise performs exactly 16 swaps.)
- (D) **True**. Reversing turns every inverted pair into a non-inverted one and vice versa; there are $C(8, 2) = 28$ pairs, so the reverse has $28 - 16 = 12$ inversions.

Trap: the number of passes of bubble sort depends on the maximum leftward displacement, not on the number of inversions.**Q.12 [NAT · 2 marks · Insertion sort — exact comparison count]****Answer: 24****Concept.** When inserting x , every shifted element costs one comparison that succeeds; then there is one more *failing* comparison — unless x travels all the way to index 0 (then the loop stops on $j < 0$ and no failing comparison is made). Hencecomparisons = (inversions) + $(n - 1)$ – (number of elements that are inserted at index 0).**Per insertion** (shifts + failing comparison):

- 6: shifts 13, reaches index 0 → $1 + 0 = 1$
- 21: 0 shifts + 1 = 1
- 2: shifts 21, 13, 6, reaches index 0 → $3 + 0 = 3$
- 17: shifts 21 → $1 + 1 = 2$
- 9: shifts 21, 17, 13 → $3 + 1 = 4$
- 25: 0 + 1 = 1
- 4: shifts 25, 21, 17, 13, 9, 6 → $6 + 1 = 7$
- 11: shifts 25, 21, 17, 13 → $4 + 1 = 5$

Total = 1 + 1 + 3 + 2 + 4 + 1 + 7 + 5 = **24**.Formula check: inversions = 18, $n - 1 = 8$, elements reaching index 0: 6 and 2 → $18 + 8 - 2 = 24$.
✓**Trap:** answering 'inversions + $n - 1$ ' = 26, forgetting that new minima stop on the boundary check $j \geq 0$ rather than on a key comparison.**Q.13 [MCQ · 2 marks · Selection sort — instability]****Answer: (C)****Concept.** Selection sort's long-distance swap can carry a record past another record with the same key, so the algorithm is not stable — even with the leftmost-minimum rule.

Trace.

- $i = 0$: minimum key 1 at position 3 → swap positions 0, 3: [1d, 3b, 2c, 2a, 3e]. (2a has jumped behind 2c!)
- $i = 1$: minimum of positions 1..4 is key 2, leftmost at position 2 (2c) → swap 1, 2: [1d, 2c, 3b, 2a, 3e].
- $i = 2$: minimum of positions 2..4 is 2a at position 3 → swap 2, 3: [1d, 2c, 2a, 3b, 3e].
- $i = 3$: 3b vs 3e — no strictly smaller key, no change.

Final tags: **d, c, a, b, e**.

Options.

- (A) is the stable order (what insertion sort or merge sort would give).
- (B) also reverses b and e, but they never get swapped past each other.
- (D) is wrong for both tie groups.

Trap: assuming 'leftmost minimum' makes selection sort stable. It only controls which record is chosen; the *displaced* record $A[i]$ can still jump over its equals.

Records after each pass

start	2a	3b	2c	1d	3e
i=0	1d	3b	2c	2a	3e
i=1	1d	2c	3b	2a	3e
i=2	1d	2c	2a	3b	3e
i=3	1d	2c	2a	3b	3e

Q.14 [NAT · 2 marks · Divide and conquer — counting split inversions]

Answer: 9

Concept. Every inversion is either inside the left half, inside the right half, or a *split* inversion (i in left, j in right, $A[i] > A[j]$). When a right-half element y is output during the merge, all elements still waiting in the left half are larger than y — each forms a split inversion with y .

Sorted halves: $L = [3, 8, 10, 15]$, $R = [1, 6, 12, 14]$.

Merge trace (elements left in L when a right element is taken):

- take 1 (R): L has 3, 8, 10, 15 waiting → +4
- take 3 (L)
- take 6 (R): 8, 10, 15 waiting → +3
- take 8, 10 (L)
- take 12 (R): 15 waiting → +1
- take 14 (R): 15 waiting → +1
- take 15 (L)

Split inversions = $4 + 3 + 1 + 1 = 9$.

Cross-check: total inversions of $A = 13$; left-half inversions (10,3), (10,8), (15,8) = 3; right-half inversions (12,6) = 1; $13 - 3 - 1 = 9$. ✓

Trap: counting 'remaining in right' when a left element is taken counts the wrong direction, and counting during *all* merges gives the total (13), not the top-level share.

Q.15 [MCQ · 2 marks · Comparison lower bounds — merging]

Answer: (C)

Concept (decision-tree lower bound). A comparison-based algorithm is a binary decision tree; it must have a distinct leaf for every possible outcome. Its worst-case number of comparisons is the tree height h , and $2^h \geq \# \text{outcomes}$.

Outcomes. The merged order is determined by choosing which 3 of the 6 output positions belong to the first list: $C(6, 3) = 20$ possibilities. So $h \geq \lceil \log_2 20 \rceil = 5$.

Upper bound. The standard merge uses at most $m + n - 1 = 5$ comparisons. Hence $k = 5$ exactly — the lower bound is tight here.

Options.

- (A) 4 is impossible: $2^4 = 16 < 20$ leaves.
- (B) 6 is the total number of elements, not a comparison bound.
- (D) $10 = \lceil \log_2 6! \rceil$ is the lower bound for **sorting** 6 arbitrary elements; merging gets free information from the two lists already being sorted.

Tip: the same argument gives $\Omega(n \log n)$ for sorting ($\log_2 n!$ leaves) and $\Omega(\log n)$ for searching a sorted array ($n + 1$ outcomes).

Q.16 [MSQ · 2 marks · Python — `sorted()`, reverse and stability]

Answer: (A), (B), (D)

Concept. `sorted(..., reverse=True)` is still **stable**: it sorts as if each comparison were reversed, so records with equal keys keep their *original* order. Reversing an ascending result with `[::-1]` instead reverses the order of equal keys too. `list.sort()` sorts in place and returns `None`.

Values.

- a: key 2 first, ties in input order $\rightarrow p, r, q, s$.
- b: ascending is q, s, p, r ; reversed $\rightarrow r, p, s, q$.
- c: key $-2 < -1$, ties in input order $\rightarrow p, r, q, s$.
- d: None; `data` becomes q, s, p, r (stable ascending).

Statements.

- (A) **True** — both are p, r, q, s .
- (B) **True**.
- (C) **False** — b has the ties reversed (r before p , s before q).
- (D) **True** — note `a`, `b`, `c` were computed before the in-place sort, and `sorted` always returns a new list, so they are unaffected.

Trap: believing `reverse=True` is the same as reversing the sorted list. It is the same only when all keys are distinct.

Q.17 [NAT · 2 marks · Shortest paths — tight edges]

Answer: 7

Step 1 — distances (Dijkstra from S).

- Extract S (0): $A = 3, B = 7, C = 4$.
- Extract A (3): $B = \min(7, 6) = 6, D = 9$.
- Extract C (4): $B = \min(6, 6) = 6$ (tie), $E = 13$.
- Extract B (6): $D = \min(9, 8) = 8, E = \min(13, 11) = 11$.
- Extract D (8): $E = \min(11, 9) = 9, T = 16$.
- Extract E (9): $T = \min(16, 13) = 13$.

d: S 0, A 3, C 4, B 6, D 8, E 9, T 13.

Step 2 — test every edge for $d(u) + w = d(v)$.

- $S \rightarrow A$ $0+3=3$ ✓, $S \rightarrow B$ $0+7=7 \neq 6$ ✗, $S \rightarrow C$ $0+4=4$ ✓
- $A \rightarrow B$ $3+3=6$ ✓, $A \rightarrow D$ $3+6=9 \neq 8$ ✗
- $C \rightarrow B$ $4+2=6$ ✓, $C \rightarrow E$ $4+9=13 \neq 9$ ✗
- $B \rightarrow D$ $6+2=8$ ✓, $B \rightarrow E$ $6+5=11 \neq 9$ ✗
- $D \rightarrow E$ $8+1=9$ ✓, $D \rightarrow T$ $8+8=16 \neq 13$ ✗
- $E \rightarrow T$ $9+4=13$ ✓

Tight edges: SA, SC, AB, CB, BD, DE, ET → 7.

Trap: a shortest-path *tree* has only 6 edges (one parent per non-source vertex); B has two tight incoming edges (A→B and C→B), both of which lie on shortest paths — there are two shortest S-T paths: S→A→B→D→E→T and S→C→B→D→E→T, both of length 13.

Q.18 [MCQ · 2 marks · Linked lists — stable partition]

Answer: (D)

Concept. Two dummy-headed lists collect the nodes $< x$ and $\geq x$ by appending at their tails, so each sub-list keeps the original relative order — a **stable** partition (unlike Lomuto/Hoare array partitions). The lists are then concatenated.

Trace.

- 7 → ge; 3 → lt; 9 → ge; 3 → lt; 5 → ge (5 is not < 5); 1 → lt; 8 → ge.
- lt list: 3 → 3 → 1; ge list: 7 → 9 → 5 → 8.
- `ge_t.next = None` terminates the ge list (node 8's old `next` was None anyway, but in general the last $\geq$ -node may still point into the $<$ list).
- Concatenate: 3, 3, 1, 7, 9, 5, 8.

Options.

- (A) and (B) sort the values; partitioning does not sort either part.
- (C) is what you get if nodes were inserted at the *head* of each part (reversing them).

Trap: omitting `ge_t.next = None` can create a cycle when the last node of the original list belongs to the $<$ part — the traversal would then never end.



Q.19 [MSQ · 2 marks · Topological sorting — DFS and Kahn's algorithm]

Answer: (B), (D)

(B) DFS trace. Start A: A → C → F (F finishes 1st), C finishes; A → D → F seen, D → G (G finishes), D finishes; A finishes. Start B: B → D seen, B → E → G seen, E finishes, B finishes.

Finish order: F, C, G, D, A, E, B. Reverse: **B, E, A, D, G, C, F → True.**

(C) Edge A → C requires A before C. **False.**

(D) Kahn trace (current sources, choose smallest):

- {A, B} → A; now C is a source (D still waits for B).
- {B, C} → B; D and E become sources.
- {C, D, E} → C; F waits for D.
- {D, E} → D; F becomes a source, G waits for E.
- {E, F} → E; G becomes a source.
- {F, G} → F, then G.

Output **A, B, C, D, E, F, G → True.**

(A) There is no path between D and E, so they can appear in either order; e.g. the DFS order in (B) has E before D. **False.**

Trap: the DFS order and the Kahn order are both valid but generally different — a DAG usually has many topological orders (this one has 42).

Q.20 [MCQ · 2 marks · Heapsort — intermediate array state]

Answer: (A)

Build-heap.

- $i = 2$ (9): child 7 smaller $\rightarrow$ no change.
- $i = 1$ (12): children 2, 18 $\rightarrow$ swap with 18 $\rightarrow$ [5, 18, 9, 2, 12, 7].
- $i = 0$ (5): children 18, 9 $\rightarrow$ swap with 18 $\rightarrow$ [18, 5, 9, 2, 12, 7]; then at index 1, children 2, 12 $\rightarrow$ swap with 12 $\rightarrow$ [18, 12, 9, 2, 5, 7].

Extraction 1. Swap $A[0] \leftrightarrow A[5]$: [7, 12, 9, 2, 5, | 18]. Sift 7 in heap of size 5: children 12, 9 $\rightarrow$ swap with 12 $\rightarrow$ [12, 7, 9, 2, 5, | 18]; index 1 children 2, 5 $<$ 7 $\rightarrow$ stop.

Extraction 2. Swap $A[0] \leftrightarrow A[4]$: [5, 7, 9, 2, | 12, 18]. Sift 5 in heap of size 4: children 7, 9 $\rightarrow$ swap with 9 $\rightarrow$ [9, 7, 5, 2, | 12, 18]; index 2 has no children inside the heap $\rightarrow$ stop.

Result: **[9, 7, 5, 2, 12, 18]**.

Options.

- (B) is the state after only one extraction.
- (C) is a valid max-heap on the same keys, but not the one sift-down produces: 5 must move into 9's old slot (index 2), while 7 stays at index 1.
- (D) omits the sift-down entirely.

Note: heapsort is in-place but not stable, and its sorted suffix grows from the right.

9	7	5	2	12	18
0	1	2	3	4	5

Heap part $A[0..3]$ | sorted suffix $A[4..5]$

Mock Test 25

Binary Search Variants

Difficulty: GATE-level

Focus areas: lower/upper bound, rotated arrays, search on answer, off-by-one bugs

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [NAT · numerical answer | 1 mark | Binary search — lower bound]

Consider the following Python program, run on the sorted array A shown. The value printed is _____.

Python

```

1 def lb(A, x):
2     lo, hi = 0, len(A)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if A[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 A = [3, 7, 7, 7, 12, 15, 15, 21, 30]
12 print(lb(A, 15) + lb(A, 8) + lb(A, 31))

```

A =

3	7	7	7	12	15	15	21	30
0	1	2	3	4	5	6	7	8

Answer: _____ (enter a numerical value)

Q.2 [MCQ · single correct | 1 mark | Binary search — off-by-one and termination]

Consider the function below, intended to return the index of the last element $\leq x$. With $A = [2, 4, 6, 8]$, which of the following calls **never terminates**?

Python

```

1 def f(A, x):
2     lo, hi = 0, len(A) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if A[mid] <= x:
6             lo = mid
7         else:
8             hi = mid - 1
9     return lo

```

- (A) $f(A, 7)$
- (B) $f(A, 1)$
- (C) $f(A, 5)$
- (D) $f(A, 4)$

Q.3 [MCQ · single correct | 1 mark | Python — generator exhaustion]

Consider the following Python program. What is printed?

Python

```
1 g = (x * x for x in range(6) if x % 2)
2 print(sum(g), max(g, default=-1), list(g))
```

- (A) 35 25 [1, 9, 25]
- (B) ValueError is raised
- (C) 35 25 []
- (D) 35 -1 []

Q.4 [MSQ · one or more correct | 1 mark | Python — the bisect module]

Let $A = [1, 3, 3, 3, 7, 9]$ and assume `import bisect`. Which of the following expressions evaluate to True?

- (A) `bisect.bisect_left(A, 3) == 1`
- (B) `bisect.bisect_right(A, 3) - bisect.bisect_left(A, 3) == 3`
- (C) `bisect.bisect(A, 8) == 4`
- (D) `bisect.bisect_right(A, 9) == 5`

Q.5 [MCQ · single correct | 1 mark | Queues — queue using two stacks]

A queue is implemented with two stacks IN and OUT. `enqueue(x)` pushes x on IN. `dequeue()` pops from OUT; if OUT is empty it first pops every element of IN and pushes it on OUT. Each individual push or pop costs 1. Starting empty, the sequence

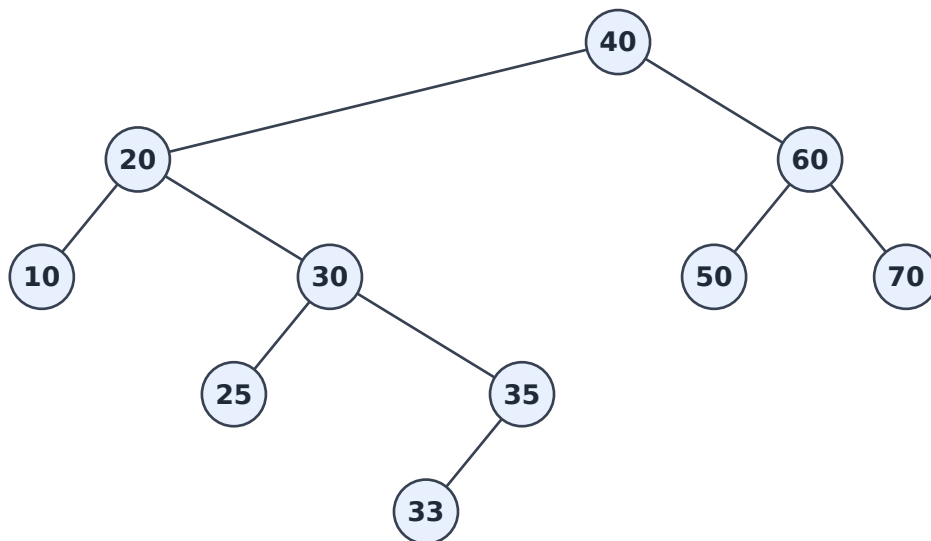
E1, E2, E3, D, E4, E5, D, D, D, E6, D

is executed ($E_k = \text{enqueue } k$, $D = \text{dequeue}$). The total cost is

- (A) 19
- (B) 21
- (C) 23
- (D) 25

Q.6 [MCQ · single correct | 1 mark | Binary search trees — deletion]

The keys 40, 20, 60, 10, 30, 50, 70, 25, 35, 33 are inserted in that order into an empty BST. Then 40 is deleted; a node with two children is replaced by its **in-order predecessor**. What is the pre-order traversal of the resulting tree?



BST before deleting 40

- (A) 35, 20, 10, 30, 25, 60, 50, 70, 33
- (B) 50, 20, 10, 30, 25, 35, 33, 60, 70
- (C) 35, 20, 10, 30, 25, 33, 60, 50, 70
- (D) 33, 20, 10, 30, 25, 35, 60, 50, 70

Q.7 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

The keys 22, 34, 11, 45, 56, 23 are inserted in that order into an empty hash table of size 11 using quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) for key k examines slot $(k \bmod 11 + i^2) \bmod 11$. The slot in which key 23 is stored is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Sorting — binary insertion sort]

In *binary insertion sort*, insertion sort on an array is modified so that the position of $A[i]$ in the sorted prefix $A[0..i-1]$ is found by binary search; the larger elements are then shifted one place right. For an input of n distinct keys in **decreasing** order, which statement is correct?

- (A) It makes $\Theta(n \log n)$ comparisons and runs in $\Theta(n \log n)$ time
- (B) It makes $\Theta(n \log n)$ comparisons but runs in $\Theta(n^2)$ time
- (C) It makes $\Theta(n^2)$ comparisons and runs in $\Theta(n^2)$ time
- (D) It makes $\Theta(n)$ comparisons and runs in $\Theta(n^2)$ time

Q.9 [NAT · numerical answer | 1 mark | Graph theory — components and edge count]

A simple undirected graph has 12 vertices and exactly 4 connected components. The maximum possible number of edges in such a graph is _____.

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Binary search trees — valid search sequences]

A BST with integer keys is searched for the key 55, and the keys of the nodes examined are recorded in order. Which of the following sequences **cannot** be such a record?

- (A) 90, 20, 80, 30, 70, 40, 60, 55
- (B) 10, 75, 64, 43, 60, 57, 55
- (C) 30, 70, 45, 62, 42, 55
- (D) 50, 65, 52, 60, 58, 55

Q.11 [NAT · numerical answer | 2 marks | Binary search — rotated sorted array]

The function below searches a rotated sorted array of distinct keys. For the array A shown, let p_1 be the number of probes returned by `search(A, 9)` and p_2 the number returned by `search(A, 50)`. The value of $p_1 + p_2$ is _____.

Python

```

1 def search(A, x):
2     lo, hi, probes = 0, len(A) - 1, 0
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         probes += 1
6         if A[mid] == x:
7             return mid, probes
8         if A[lo] <= A[mid]:           # left half sorted
9             if A[lo] <= x < A[mid]:
10                hi = mid - 1
11            else:
12                lo = mid + 1
13        else:                         # right half sorted
14            if A[mid] < x <= A[hi]:
15                lo = mid + 1
16            else:
17                hi = mid - 1
18    return -1, probes

```

A =	41	47	53	60	4	9	17	23	30	36
	0	1	2	3	4	5	6	7	8	9

Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | Binary search — correctness of first-occurrence variants]

Each function below is meant to return the index of the **first** occurrence of x in a sorted list A (possibly with duplicates, possibly empty), or -1 if x does not occur. Which of the functions is/are correct (terminate and return the right value) for **every** such input?

Python

```

1 def f1(A, x):
2     lo, hi, ans = 0, len(A) - 1, -1
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         if A[mid] >= x:
6             if A[mid] == x:
7                 ans = mid
8                 hi = mid - 1
9             else:
10                lo = mid + 1
11    return ans
12
13 def f2(A, x):

```

```

14     lo, hi = 0, len(A)
15     while lo < hi:
16         mid = (lo + hi) // 2
17         if A[mid] < x: lo = mid + 1
18         else: hi = mid
19     return lo if A[lo] == x else -1
20
21 def f3(A, x):
22     lo, hi = -1, len(A)
23     while hi - lo > 1:
24         mid = (lo + hi) // 2
25         if A[mid] < x: lo = mid
26         else: hi = mid
27     return hi if hi < len(A) and A[hi] == x else -1
28
29 def f4(A, x):
30     lo, hi = 0, len(A) - 1
31     while lo < hi:
32         mid = (lo + hi + 1) // 2
33         if A[mid] <= x: lo = mid
34         else: hi = mid - 1
35     return lo if A and A[lo] == x else -1

```

- (A) f2
(B) f3
(C) f1
(D) f4

Q.13 [NAT · numerical answer | 2 marks | Binary search — range counting with bisect]

Consider the following Python program. The value printed is ____.

Python

```

1 from bisect import bisect_left, bisect_right
2
3 A = sorted([12, 5, 19, 5, 8, 12, 30, 12, 26, 8, 19, 41])
4 Q = [(5, 12), (13, 25), (8, 8), (20, 50)]
5 print(sum(bisect_right(A, b) - bisect_left(A, a) for a, b in Q))

```

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Complexity — recursive binary search with slicing]

Consider the recursive Python binary search below. In the worst case (x absent) on a sorted list of n elements, its running time is

Python

```

1 def bs(A, x):
2     if not A:
3         return False
4     mid = len(A) // 2
5     if A[mid] == x:
6         return True
7     if A[mid] < x:
8         return bs(A[mid + 1:], x)
9     return bs(A[:mid], x)

```

- (A) $\Theta(n)$
(B) $\Theta(n \log n)$
(C) $\Theta(\log n)$
(D) $\Theta(\log^2 n)$

Q.15 [MSQ · one or more correct | 2 marks | Binary search on the answer — integer square root]

Consider the function below, intended to compute $\lfloor \sqrt{n} \rfloor$ for integers $n \geq 0$. Which of the following statements is/are TRUE?

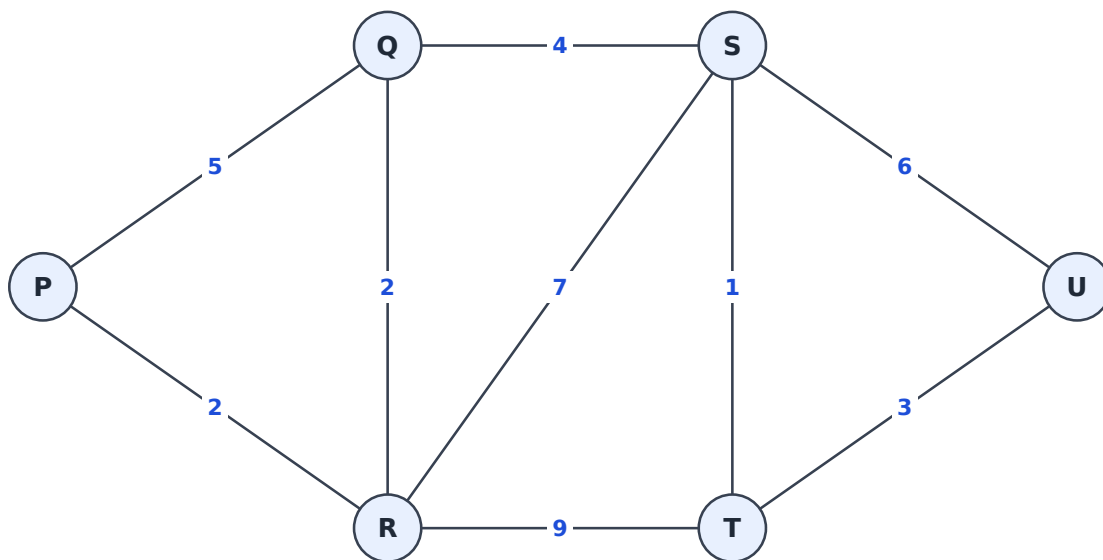
Python

```
1 def isqrt(n):
2     lo, hi = 0, n
3     while lo < hi:
4         mid = (lo + hi + 1) // 2
5         if mid * mid <= n:
6             lo = mid
7         else:
8             hi = mid - 1
9     return lo
```

- (A) For every integer $n \geq 0$ the function returns $\lfloor \sqrt{n} \rfloor$
- (B) `isqrt(99)` returns 9
- (C) For $n = 1000$ the loop body executes exactly 10 times
- (D) If `mid` were computed as `(lo + hi) // 2`, the call `isqrt(2)` would never terminate

Q.16 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra]

Dijkstra's algorithm is run from P on the weighted undirected graph shown. Which of the following statements is/are TRUE?



- (A) The edge R-S belongs to the shortest-path tree rooted at P
- (B) A shortest path from P to U passes through Q
- (C) The shortest-path distance from P to U is 12
- (D) Vertices are finalised in the order P, R, Q, S, T, U

Q.17 [MCQ · single correct | 2 marks | Merge sort — counting comparisons]

Top-down merge sort (split a list of length n into the first $\lfloor n/2 \rfloor$ and the remaining elements; merge by repeatedly comparing the two front elements, and copy the leftover without comparisons once a side is empty) is applied to $[5, 2, 8, 1, 9, 3, 7, 4]$. The total number of key comparisons is

- (A) 24
- (B) 16
- (C) 18
- (D) 17

Q.18 [NAT · numerical answer | 2 marks | Linked lists — binary search on a linked list]

A sorted singly linked list has 16 nodes holding 10, 20, 30, ..., 160 (index 0 is the head). 'Binary search' is applied with $lo = 0$, $hi = 15$ and $mid = \lfloor (lo + hi)/2 \rfloor$, exactly as on an array (go right if the key at mid is smaller than x, left if larger, stop when found or $lo > hi$). Reading the key at index i is done by starting at the head and following i **next** pointers. The total number of **next** pointers followed while searching for $x = 125$ is _____.



Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Graph theory — graphical degree sequences]

Which of the following sequences is/are the degree sequence of some **simple** undirected graph?

- (A) 5, 5, 4, 3, 2, 1
- (B) 3, 3, 3, 3, 3, 3
- (C) 3, 3, 3, 1
- (D) 4, 4, 3, 2, 2, 1

Q.20 [NAT · numerical answer | 2 marks | Binary search on the answer — minimum capacity]

Packages with the weights below must be shipped **in the given order**, at most $D = 3$ days, each day shipping a contiguous block whose total weight does not exceed the capacity. Consider the following Python program. The value printed is _____.

Python

```

1 def days_needed(w, cap):
2     d, load = 1, 0
3     for x in w:
4         if load + x > cap:
5             d, load = d + 1, 0
6         load += x
7     return d
8
9 def min_cap(w, D):
10    lo, hi = max(w), sum(w)
11    while lo < hi:
12        mid = (lo + hi) // 2
13        if days_needed(w, mid) <= D:
14            hi = mid
15        else:
16            lo = mid + 1
17    return lo
18
19 print(min_cap([7, 2, 5, 10, 8, 3, 6, 4], 3))

```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 25

Q	Type	Marks	Key	Topic
1	NAT	1	18	Binary search — lower bound
2	MCQ	1	(A)	Binary search — off-by-one and termination
3	MCQ	1	(D)	Python — generator exhaustion
4	MSQ	1	(A), (B)	Python — the bisect module
5	MCQ	1	(B)	Queues — queue using two stacks
6	MCQ	1	(C)	Binary search trees — deletion
7	NAT	1	10	Hashing — quadratic probing
8	MCQ	1	(B)	Sorting — binary insertion sort
9	NAT	1	36	Graph theory — components and edge count
10	MCQ	1	(C)	Binary search trees — valid search sequences
11	NAT	2	7	Binary search — rotated sorted array
12	MSQ	2	(B), (C)	Binary search — correctness of first-occurrence variants
13	NAT	2	14	Binary search — range counting with bisect
14	MCQ	2	(A)	Complexity — recursive binary search with slicing
15	MSQ	2	(A), (B), (D)	Binary search on the answer — integer square root
16	MSQ	2	(B), (C), (D)	Shortest paths — Dijkstra
17	MCQ	2	(D)	Merge sort — counting comparisons
18	NAT	2	43	Linked lists — binary search on a linked list
19	MSQ	2	(B), (D)	Graph theory — graphical degree sequences
20	NAT	2	18	Binary search on the answer — minimum capacity

Detailed Solutions — Mock Test 25

Q.1 [NAT · 1 mark · Binary search — lower bound]

Answer: 18

Concept. `lb` is the classic *lower bound*: it returns the smallest index i with $A[i] \geq x$ (or $\text{len}(A)$ if no such index). The half-open range $[lo, hi)$ always contains the answer, and `hi = mid` keeps `mid` as a candidate.

- `lb(A, 15)`: first element ≥ 15 is $A[5] = 15 \rightarrow 5$. (Trace: $[0,9)$ mid 4 ($12 < 15$) $\rightarrow lo\ 5$; $[5,9)$ mid 7 (21) $\rightarrow hi\ 7$; $[5,7)$ mid 6 (15) $\rightarrow hi\ 6$; $[5,6)$ mid 5 (15) $\rightarrow hi\ 5$.)
- `lb(A, 8)`: 8 is absent; first element ≥ 8 is $A[4] = 12 \rightarrow 4$.
- `lb(A, 31)`: every element is $< 31 \rightarrow$ returns $\text{len}(A) = 9$.

Sum = $5 + 4 + 9 = 18$.

Trap: returning the index of the *last* 15 (6) or treating the 'not found' case as -1 . Lower bound never returns -1 — it returns an insertion point.

Q.2 [MCQ · 1 mark · Binary search — off-by-one and termination]

Answer: (A)

Concept. With `mid = (lo + hi) // 2` (floor) and the update `lo = mid`, the range does not shrink when $hi = lo + 1$ and $A[lo] \leq x$: then `mid = lo` and `lo` stays the same forever. A 'move `lo` to `mid`' loop needs the **ceiling** `mid = (lo + hi + 1) // 2`.

Traces on $A = [2, 4, 6, 8]$:

- `f(A, 5)`: $(0,3)$ mid 1, $4 \leq 5 \rightarrow lo\ 1$; $(1,3)$ mid 2, $6 > 5 \rightarrow hi\ 1$; stop $\rightarrow 1$.
- `f(A, 1)`: $(0,3)$ mid 1, $4 > 1 \rightarrow hi\ 0$; stop $\rightarrow 0$.
- `f(A, 7)`: $(0,3)$ mid 1 $\rightarrow lo\ 1$; $(1,3)$ mid 2, $6 \leq 7 \rightarrow lo\ 2$; $(2,3)$ mid 2 $\rightarrow lo\ 2$ again ... **infinite loop**.
- `f(A, 4)`: $(0,3)$ mid 1 $\rightarrow lo\ 1$; $(1,3)$ mid 2, $6 > 4 \rightarrow hi\ 1$; stop $\rightarrow 1$.

Answer (A).

Tip: a quick termination test: check the two-element range $(lo, lo + 1)$. Each branch must strictly shrink it.

Q.3 [MCQ · 1 mark · Python — generator exhaustion]

Answer: (D)

Concept. A generator expression is a one-shot iterator. Once consumed, further iteration yields nothing. The arguments of `print` are evaluated left to right.

- `x % 2` is truthy for odd $x \rightarrow$ the generator yields 1, 9, 25.
- `sum(g)` consumes all of it $\rightarrow 35$.
- `max(g, default=-1)` sees an empty iterator $\rightarrow$ returns the default -1 (without `default`, it would raise `ValueError`).
- `list(g) \rightarrow []`.

Output: `35 -1 []` $\rightarrow$ (D).

- (A) and (C) assume the generator restarts like a list.
- (B) would be right only if `default` were omitted.

Trap: a list comprehension `[x*x for ...]` would give `35 25 [1, 9, 25]`; parentheses instead of brackets change everything.

Q.4 [MSQ · 1 mark · Python — the bisect module]**Answer: (A), (B)**

Concept. `bisect_left(A, x)` = first index i with $A[i] \geq x$ (lower bound); `bisect_right(A, x)` = first index i with $A[i] > x$ (upper bound). `bisect` is an alias of `bisect_right`. Their difference counts occurrences of x .

- (A) First element ≥ 3 is at index 1. **True.**
- (B) `bisect_right(A, 3) = 4` (first element > 3 is 7 at index 4); $4 - 1 = 3$ occurrences. **True.**
- (C) First element > 8 is 9 at index 5, so the result is 5. **False.**
- (D) No element is > 9 , so `bisect_right` returns `len(A) = 6`. **False.**

Trap: treating `bisect` as 'index of x ' — it returns an insertion point, which may equal `len(A)`.

Q.5 [MCQ · 1 mark · Queues — queue using two stacks]**Answer: (B)**

Concept. Each element is pushed on IN once, moved to OUT at most once (1 pop + 1 push) and popped from OUT once — at most 4 operations per element, giving $O(1)$ amortised cost.

Trace:

- E1, E2, E3: 3 pushes $\rightarrow$ cost 3. IN = [1, 2, 3].
- D: OUT empty $\rightarrow$ move 3 elements (6) + pop 1 $\rightarrow$ cost 7. OUT = [3, 2].
- E4, E5: cost 2. IN = [4, 5].
- D: pop 2 $\rightarrow$ 1. D: pop 3 $\rightarrow$ 1 (OUT now empty).
- D: move 4, 5 (4) + pop 4 $\rightarrow$ 5. OUT = [5].
- E6: 1. D: pop 5 $\rightarrow$ 1.

Total = $3 + 7 + 2 + 1 + 1 + 5 + 1 + 1 = 21 \rightarrow$ (B).

Check with the per-element rule: elements 1–5 each cost 4 (20); 6 is pushed only (1) $\rightarrow$ 21.

Trap: moving IN to OUT on every dequeue (not only when OUT is empty) — that breaks FIFO order and inflates the cost.

Q.6 [MCQ · 1 mark · Binary search trees — deletion]**Answer: (C)**

Concept. The in-order predecessor of a node with two children is the maximum of its left subtree (rightmost node there). It has no right child, so it can be unlinked by attaching its left child to its parent.

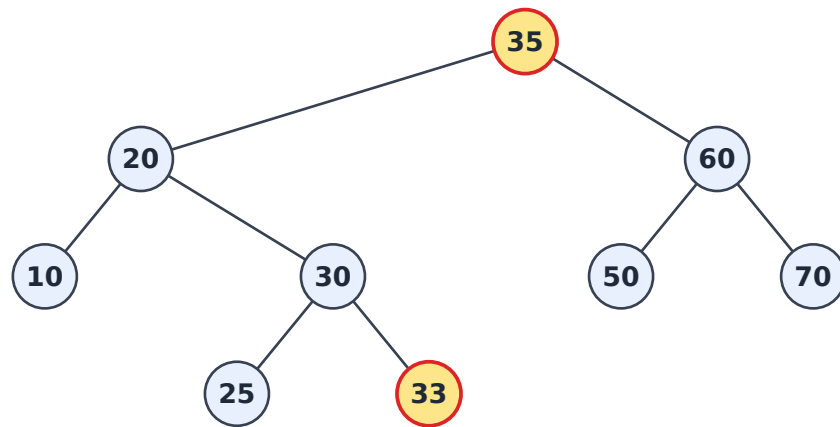
Tree before deletion: $40 \rightarrow (20 \rightarrow 10, 30 \rightarrow (25, 35 \rightarrow (33, \cdot)))$, $(60 \rightarrow 50, 70)$.

- Predecessor of 40 = rightmost of the left subtree: $20 \rightarrow 30 \rightarrow 35 \rightarrow$ **35**.
- Copy 35 into the root, and replace node 35 by its left child 33 (so 33 becomes 30's right child).

New tree: $35 \rightarrow (20 \rightarrow 10, 30 \rightarrow (25, 33))$, $(60 \rightarrow 50, 70)$. Pre-order: 35, 20, 10, 30, 25, 33, 60, 50, 70 $\rightarrow$ (C).

- (A) drops 33 into the wrong subtree.
- (B) uses the in-order successor (50).
- (D) picks 33 — the predecessor of 35, not of 40.

Trap: forgetting to re-attach the predecessor's left child (33).



After deleting 40

Q.7 [NAT · 1 mark · Hashing — quadratic probing]

Answer: 10

Concept. Quadratic probing jumps by offsets 0, 1, 4, 9, 16, ... from the home slot, which avoids primary clustering, but keys with the same home slot still follow the same sequence (secondary clustering).

Trace:

- 22: home 0 → slot 0.
- 34: home 1 → slot 1.
- 11: home 0 (full) → $0 + 1 = 1$ (full) → $0 + 4 = 4$.
- 45: home 1 (full) → $1 + 1 = 2$.
- 56: home 1 (full) → 2 (full) → $1 + 4 = 5$.
- 23: home 1 (full) → 2 (full) → 5 (full) → $1 + 9 = 10$.

Key 23 goes to slot **10**.

Trap: using linear offsets i instead of i^2 gives slot 3; using the *previous* probe + i^2 (cumulative) also gives a different slot.

0	22
1	34
2	45
3	
4	11
5	56
6	
7	
8	
9	
10	23

Final table

Q.8 [MCQ · 1 mark · Sorting — binary insertion sort]**Answer: (B)**

Concept. Binary search reduces the *comparisons* per insertion to $O(\log i)$, but the *data movement* in an array is unchanged: inserting at position p requires shifting $i - p$ elements.

Decreasing input: every new key is the smallest so far.

- Comparisons: $\sum \lceil \log_2(i + 1) \rceil$ over $i = 1 \dots n - 1 = \Theta(n \log n)$.
- Shifts: key i moves past all i previous keys $\rightarrow \sum i = n(n - 1)/2 = \Theta(n^2)$.
- Running time is dominated by shifts $\rightarrow \Theta(n^2)$.

Hence (B).

- (A) ignores the shifting cost.
- (C) is ordinary insertion sort's comparison count, not binary insertion sort's.
- (D) under-counts the binary searches.

Trap: 'binary search makes insertion sort $O(n \log n)$ ' — only the comparison count improves. On a linked list shifting is cheap, but binary search is then impossible in $O(\log n)$.

Q.9 [NAT · 1 mark · Graph theory — components and edge count]**Answer: 36**

Concept. For a fixed number of vertices and components, the edge count is maximised by making every component complete and putting as many vertices as possible into one component: $C(a, 2) + C(b, 2)$ is larger when the sizes are unbalanced (convexity of $C(x, 2)$).

- Use three isolated vertices and one component with $12 - 3 = 9$ vertices.
- Make that component K_9 : $C(9, 2) = 9 \cdot 8 / 2 = \mathbf{36}$ edges.

General formula: $C(n - k + 1, 2)$ for n vertices and k components.

Comparison: a balanced split $3 + 3 + 3 + 3$ gives only $4 \cdot C(3, 2) = 12$ edges.

Trap: computing $C(12, 2) - (\text{something})$ or splitting evenly. Also, the minimum number of edges with 4 components is $n - k = 8$ (a forest).

Q.10 [MCQ · 1 mark · Binary search trees — valid search sequences]**Answer: (C)**

Concept. While searching for x , each examined key narrows the open interval in which every later key must lie: after visiting $k < x$ we need later keys $> k$; after $k > x$, later keys $< k$. A sequence is valid iff every key falls in the current interval.

- (A) $(-\infty, \infty) \rightarrow 90: (-\infty, 90) \rightarrow 20: (20, 90) \rightarrow 80: (20, 80) \rightarrow 30: (30, 80) \rightarrow 70: (30, 70) \rightarrow 40: (40, 70) \rightarrow 60: (40, 60) \rightarrow 55 \checkmark$. Valid.
- (B) $10: (10, \infty) \rightarrow 75: (10, 75) \rightarrow 64: (10, 64) \rightarrow 43: (43, 64) \rightarrow 60: (43, 60) \rightarrow 57: (43, 57) \rightarrow 55 \checkmark$. Valid.
- (C) $30: (30, \infty) \rightarrow 70: (30, 70) \rightarrow 45: (45, 70) \rightarrow 62: (45, 62) \rightarrow \mathbf{42}$ is not in $(45, 62)$. Invalid $\rightarrow$ (C).
- (D) $50: (50, \infty) \rightarrow 65: (50, 65) \rightarrow 52: (52, 65) \rightarrow 60: (52, 60) \rightarrow 58: (52, 58) \rightarrow 55 \checkmark$. Valid.

Trap: checking only adjacent pairs. 42 is between 30 and 62, but it violates the lower bound set earlier by 45.

Q.11 [NAT · 2 marks · Binary search — rotated sorted array]**Answer: 7**

Concept. In a rotated sorted array, at least one of the halves $A[\text{lo}..\text{mid}]$ and $A[\text{mid}..\text{hi}]$ is sorted. Test whether x lies inside the sorted half's range; if so, search there, otherwise search the other half. Each probe still halves the range.

search(A, 9):

- lo 0, hi 9 → mid 4 (A = 4). $A[0] = 41 > 4$, so the right half [4 ... 36] is sorted; $4 < 9 \leq 36 \rightarrow \text{lo} = 5$.
- lo 5, hi 9 → mid 7 (23). $A[5] = 9 \leq 23 \rightarrow$ left half [9 ... 23] sorted; $9 \leq 9 < 23 \rightarrow \text{hi} = 6$.
- lo 5, hi 6 → mid 5 (9) → found. $p_1 = 3$.

search(A, 50):

- mid 4 (4): right half sorted, $50 \notin (4, 36) \rightarrow \text{hi} = 3$.
- lo 0, hi 3 → mid 1 (47): left half [41, 47] sorted, $50 \notin [41, 47) \rightarrow \text{lo} = 2$.
- lo 2, hi 3 → mid 2 (53): left half [53] sorted, $50 \notin [53, 53) \rightarrow \text{lo} = 3$.
- lo 3, hi 3 → mid 3 (60): $50 \notin [60, 60) \rightarrow \text{lo} = 4$; loop ends. $p_2 = 4$.

$p_1 + p_2 = 3 + 4 = 7$.

Trap: using $A[\text{lo}] < A[\text{mid}]$ (strict) misclassifies the one-element left half when $\text{lo} = \text{mid}$; the $\leq$ test is essential.

Q.12 [MSQ · 2 marks · Binary search — correctness of first-occurrence variants]

Answer: (B), (C)

Concept. A first-occurrence search must keep moving **left** after a match, must make progress in every branch, and must not index outside the list.

- **f1** — closed interval [lo, hi]; on $A[\text{mid}] \geq x$ it records a match (if equal) and continues left with $\text{hi} = \text{mid} - 1$, else goes right. Every branch shrinks the range; the last recorded match is the leftmost. Empty list: loop skipped, returns -1. **Correct.**
- **f2** — computes the lower bound correctly, but when x is larger than every element (or A is empty) $\text{lo} = \text{len}(A)$ and $A[\text{lo}]$ raises **IndexError**. E.g. $\text{f2}([1, 2], 5)$. **Incorrect.**
- **f3** — invariant $A[\text{lo}] < x \leq A[\text{hi}]$ with virtual sentinels $A[-1] = -\infty$ and $A[n] = +\infty$; the loop stops when $\text{hi} = \text{lo} + 1$, so hi is the lower bound. The guard $\text{hi} < \text{len}(A)$ prevents out-of-range access. **Correct.**
- **f4** — ceiling mid with $\text{lo} = \text{mid}$ on $A[\text{mid}] \leq x$ terminates, but it finds the **last** element $\leq x$, i.e. the *last* occurrence. $\text{f4}([3, 3, 3], 3)$ returns 2, not 0. **Incorrect.**

Answer: f1 and f3.

Trap: f2 passes every test where x is present — bugs at the boundaries (x beyond the maximum, empty list) are exactly what GATE questions probe.

Q.13 [NAT · 2 marks · Binary search — range counting with bisect]

Answer: 14

Concept. In a sorted list, the number of elements in the closed interval $[a, b]$ is $\text{bisect_right}(A, b) - \text{bisect_left}(A, a)$: everything before the upper bound of b minus everything strictly before a . Each query costs $O(\log n)$.

Sorted A (indices 0-11): 5, 5, 8, 8, 12, 12, 12, 19, 19, 26, 30, 41.

- [5, 12]: $\text{bisect_right}(12) = 7$, $\text{bisect_left}(5) = 0 \rightarrow 7$.
- [13, 25]: $\text{bisect_right}(25) = 9$, $\text{bisect_left}(13) = 7 \rightarrow 2$ (the two 19s).
- [8, 8]: $\text{bisect_right}(8) = 4$, $\text{bisect_left}(8) = 2 \rightarrow 2$.
- [20, 50]: $\text{bisect_right}(50) = 12$, $\text{bisect_left}(20) = 9 \rightarrow 3$ (26, 30, 41).

Total = $7 + 2 + 2 + 3 = 14$.

Trap: using bisect_left for the upper end excludes copies of b (would give $4 + 2 + 0 + 3 = 9$); using bisect_right for the lower end excludes copies of a .

5	5	8	8	12	12	12	19	19	26	30	41
0	1	2	3	4	5	6	7	8	9	10	11

$\uparrow$
 $lo(5)$
 $\uparrow$
 $hi(12)$

Query [5, 12]: indices 0 ... 6 counted

Q.14 [MCQ · 2 marks · Complexity — recursive binary search with slicing]

Answer: (A)

Concept. A Python slice `A[i:j]` creates a **new list**, copying $j - i$ references, so it costs $\Theta(j - i)$ time, not $O(1)$.

Recurrence: each call does $O(1)$ comparison work plus a slice of about $n/2$ elements, then recurses on that slice:

$$T(n) = T(n/2) + \Theta(n).$$

Unrolling: $c \cdot n + c \cdot n/2 + c \cdot n/4 + \dots \leq 2c \cdot n \rightarrow \Theta(n)$. (Master theorem: $a = 1$, $b = 2$, $f(n) = n = \Omega(n^{0+\epsilon})$ and regularity holds $\rightarrow \Theta(f(n)) = \Theta(n)$.)

- (B) would need $\log n$ levels each costing n — but the copy sizes shrink geometrically.
- (C) would be correct if the function passed indices lo , hi instead of slicing.
- (D) has no basis here.

Trap: the algorithm *looks* logarithmic. Extra memory is also $\Theta(n)$ in total across the recursion. Use `bisect` or index bounds for true $O(\log n)$.

Q.15 [MSQ · 2 marks · Binary search on the answer — integer square root]

Answer: (A), (B), (D)

Concept. 'Binary search on the answer': the predicate $P(m) = (m^2 \leq n)$ is true for $m = 0 \dots \lfloor \sqrt{n} \rfloor$ and false afterwards. The loop keeps the invariant $P(lo)$ is true and $P(hi + 1)$ is false, and finds the **last** true m . Because the true branch sets $lo = mid$, mid must be rounded **up** to guarantee progress.

- (A) The invariant holds initially ($P(0)$ true; $P(n + 1)$ false since $(n+1)^2 > n$), each branch preserves it, and every iteration shrinks $hi - lo$ because $lo < mid \leq hi$. **True** ($n = 0$ and $n = 1$ return immediately/correctly).
- (B) $9^2 = 81 \leq 99 < 100 = 10^2$. Trace: (0,99) mid 50 $\rightarrow$ hi 49; mid 25 $\rightarrow$ hi 24; mid 12 $\rightarrow$ hi 11; mid 6 $\rightarrow$ lo 6; (6,11) mid 9 $\rightarrow$ lo 9; (9,11) mid 10 $\rightarrow$ hi 9; stop $\rightarrow$ 9. **True.**
- (C) For $n = 1000$ the range shrinks $1000 \rightarrow 499 \rightarrow 249 \rightarrow \dots$; tracing gives **9** iterations, not 10 (result 31). **False.**
- (D) With floor mid: (0,2) mid 1, $1 \leq 2 \rightarrow$ lo 1; (1,2) mid 1 $\rightarrow$ lo 1 again ... infinite loop. **True.**

Trap: with the floor mid the code looks fine and works for many n (e.g. $n = 0, 1$), but hangs whenever the final two-element range has a true lower end.

Q.16 [MSQ · 2 marks · Shortest paths — Dijkstra]

Answer: (B), (C), (D)

Concept. Dijkstra finalises vertices in increasing distance; each vertex's parent in the shortest-path tree is the vertex whose relaxation last lowered it.

Trace:

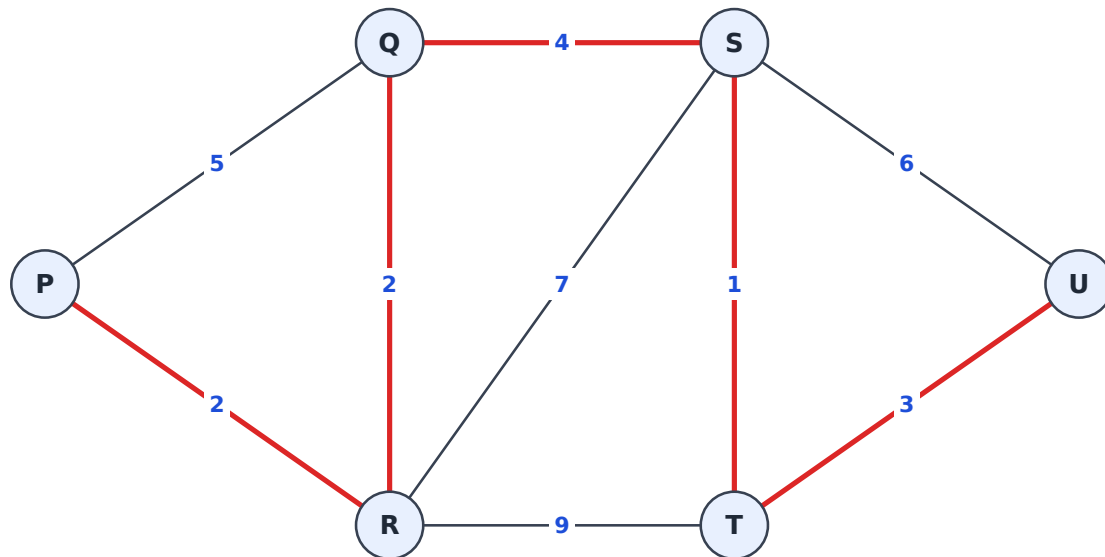
- P (0): $Q = 5$, $R = 2$.
- R (2): $Q = \min(5, 4) = 4$ (parent R); $S = 9$ (parent R); $T = 11$ (parent R).
- Q (4): $S = \min(9, 8) = 8$ (parent Q).
- S (8): $T = \min(11, 9) = 9$ (parent S); $U = 14$ (parent S).
- T (9): $U = \min(14, 12) = 12$ (parent T).

- U (12).

Option analysis:

- (A) **False** — S's parent became Q ($8 < 9$), so R-S is not a tree edge.
- (B) **True** — the unique shortest path uses Q.
- (C) **True** — $d(U) = 12$ via P-R-Q-S-T-U ($2 + 2 + 4 + 1 + 3$).
- (D) **True** — distances 0, 2, 4, 8, 9, 12 in that order.

Trap: R-S was used *temporarily* ($S = 9$) but a later relaxation replaced it.



Shortest-path tree (red)

Q.17 [MCQ · 2 marks · Merge sort — counting comparisons]

Answer: (D)

Concept. Merging lists of sizes p and q costs between $\min(p, q)$ and $p + q - 1$ comparisons; total = sum over all merges.

Level 1 (pairs):

- $[5] + [2] \rightarrow [2, 5]$: 1
- $[8] + [1] \rightarrow [1, 8]$: 1
- $[9] + [3] \rightarrow [3, 9]$: 1
- $[7] + [4] \rightarrow [4, 7]$: 1 (subtotal 4)

Level 2:

- $[2, 5] + [1, 8]$: $2v1 \rightarrow 1$, $2v8 \rightarrow 2$, $5v8 \rightarrow 5$, then copy 8 $\rightarrow$ 3 comparisons.
- $[3, 9] + [4, 7]$: $3v4 \rightarrow 3$, $9v4 \rightarrow 4$, $9v7 \rightarrow 7$, copy 9 $\rightarrow$ 3 comparisons. (subtotal 6)

Level 3: $[1, 2, 5, 8] + [3, 4, 7, 9]$:

- $1v3 \rightarrow 1$, $2v3 \rightarrow 2$, $5v3 \rightarrow 3$, $5v4 \rightarrow 4$, $5v7 \rightarrow 5$, $8v7 \rightarrow 7$, $8v9 \rightarrow 8$, copy 9 $\rightarrow$ 7 comparisons.

Total = $4 + 6 + 7 = 17 \rightarrow$ (D).

- (A) $24 = n \log_2 n$ is only an upper-bound estimate; the exact worst case for $n = 8$ is 17 as well — this input happens to be a worst case.
- (B) 16 and (C) 18 come from mis-counting one merge.

Tip: worst case for $n = 2^k$ is $n \cdot k - n + 1 = 8 \cdot 3 - 8 + 1 = 17$.

Q.18 [NAT · 2 marks · Linked lists — binary search on a linked list]**Answer: 43**

Concept. On a linked list there is no random access: reaching index i costs i pointer moves. Binary search still makes only $O(\log n)$ comparisons, but its pointer cost is $\Theta(n)$ per search — no better (here worse) than a linear scan.

Trace (key at index i is $10(i + 1)$):

- lo 0, hi 15 $\rightarrow$ mid 7 (80) $<$ 125 $\rightarrow$ lo 8. Cost 7.
- lo 8, hi 15 $\rightarrow$ mid 11 (120) $<$ 125 $\rightarrow$ lo 12. Cost 11.
- lo 12, hi 15 $\rightarrow$ mid 13 (140) $>$ 125 $\rightarrow$ hi 12. Cost 13.
- lo 12, hi 12 $\rightarrow$ mid 12 (130) $>$ 125 $\rightarrow$ hi 11. Cost 12. Stop (lo $>$ hi).

Total = 7 + 11 + 13 + 12 = **43** pointer moves.

A linear scan would stop at 130 (index 12) after only 12 moves.

Trap: counting comparisons (4) instead of pointer moves, or assuming each walk can continue from the previous position (that would need a backward pointer for the left moves).

Q.19 [MSQ · 2 marks · Graph theory — graphical degree sequences]**Answer: (B), (D)**

Concept. Havel-Hakimi: a non-increasing sequence $d_1 \geq d_2 \geq \dots$ is graphical iff the sequence obtained by deleting d_1 and subtracting 1 from the next d_1 terms is graphical. (Necessary first check: the sum must be even.)

- (A) 5,5,4,3,2,1 (sum 20): $\rightarrow$ 4,3,2,1,0 $\rightarrow$ 2,1,0,-1 $\times$. **Not graphical:** two vertices of degree 5 among 6 vertices are adjacent to everyone, so no vertex can have degree 1.
- (B) 3,3,3,3,3,3 (sum 18): $\rightarrow$ 2,2,2,3,3 $\rightarrow$ sort 3,3,2,2,2 $\rightarrow$ 2,1,1,2 $\rightarrow$ sort 2,2,1,1 $\rightarrow$ 1,0,1 $\rightarrow$ 1,1,0 $\rightarrow$ 0,0 $\checkmark$. **Graphical** (e.g. $K_{3,3}$ or the triangular prism).
- (C) 3,3,3,1 (sum 10): with 4 vertices a degree-3 vertex is adjacent to all others, so three such vertices force the last vertex to degree ≥ 3 . HH: $\rightarrow$ 2,2,0 $\rightarrow$ 1,-1 $\times$. **Not graphical.**
- (D) 4,4,3,2,2,1 (sum 16): $\rightarrow$ 3,2,1,1,1 $\rightarrow$ 1,0,0,1 $\rightarrow$ 1,1,0,0 $\rightarrow$ 0,0,0 $\checkmark$. **Graphical.**

Trap: an even degree sum is necessary but not sufficient — (A) and (C) both have even sums.

Q.20 [NAT · 2 marks · Binary search on the answer — minimum capacity]**Answer: 18**

Concept. `days_needed(cap)` is non-increasing in `cap`, so the predicate '`days_needed(cap)  $\leq$  D`' is false ... false true ... true. Binary search for the first true value (lower bound) between `max(w)` (every package must fit) and `sum(w)` (one day).

Search on [10, 45]:

- mid 27: greedy days [7,2,5,10] (24; +8 $>$ 27) [8,3,6,4] (21) $\rightarrow$ 2 days $\leq$ 3 $\rightarrow$ hi = 27.
- mid 18: [7,2,5] (14; +10 $>$ 18) [10,8] (18) [3,6,4] (13) $\rightarrow$ 3 days $\rightarrow$ hi = 18.
- mid 14: [7,2,5] [10] [8,3] [6,4] $\rightarrow$ 4 days $\rightarrow$ lo = 15.
- mid 16: [7,2,5] [10] [8,3] ... $\rightarrow$ 4 days $\rightarrow$ lo = 17.
- mid 17: [7,2,5] [10] [8,3,6] [4] $\rightarrow$ 4 days $\rightarrow$ lo = 18. Stop: lo = hi = **18**.

Check optimality: greedy packing (fill each day as far as possible) is optimal for contiguous blocks. With capacity 17 it gives [7,2,5] [10] [8,3,6] [4] = 4 days, because 10 fits neither with the first block (24) nor with 8 (18). With 18 the pair 10 + 8 fits exactly, giving 3 days.

Trap: starting lo at 0 or 1 still works but wastes iterations; starting hi below `sum(w)` can miss feasible answers. Also, `days_needed` must start counting at 1, not 0.

Mock Test 26

Linked-List Algorithms

Difficulty: GATE-level

Focus areas: reversal, cycle detection, merging, middle node, list surgery

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Linked lists — middle node with slow/fast pointers]

h is the head of the singly linked list $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60 \rightarrow 70 \rightarrow 80$. Consider the following Python fragment. What is printed?

Python

```

1 s = f = h
2 while f and f.nxt:
3     s, f = s.nxt, f.nxt.nxt
4 x = s.v
5
6 s = f = h
7 while f.nxt and f.nxt.nxt:
8     s, f = s.nxt, f.nxt.nxt
9 print(x, s.v)
```

(A) 50 40

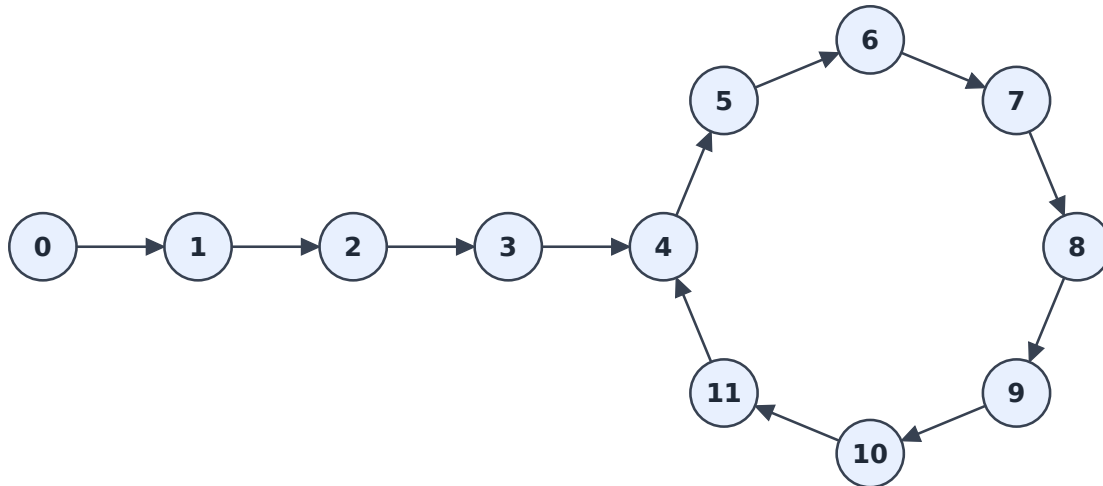
(B) 40 40

(C) 50 50

(D) 40 50

Q.2 [NAT · numerical answer | 1 mark | Cycle detection — Floyd's algorithm]

A singly linked list has nodes labelled 0, 1, ..., 11, where node i points to node $i + 1$ for $i < 11$ and node 11 points back to node 4 (see figure). Floyd's algorithm starts with `slow = fast = node 0` and repeats `slow = slow.next; fast = fast.next.next` until `slow is fast`. The number of iterations of this loop is _____.



Tail of length 4 (nodes 0-3) and a cycle of length 8 (nodes 4-11)

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Linked lists — cost of operations]

A singly linked list of n nodes is maintained with two references, `head` (first node) and `tail` (last node); nodes have only `val` and `next` fields. Which of the following operations can be performed in **O(1)** worst-case time (keeping `head/tail` correct)?

- (A) Appending a new node after the last node
- (B) Deleting the last node
- (C) Deleting the node referenced by a given pointer p , where p is known not to be the last node (node identity need not be preserved)
- (D) Finding the middle node

Q.4 [NAT · numerical answer | 1 mark | Stacks — bracket matching]

The string `{[(())]([{}(]))}{[()]}` is checked for balanced brackets using a stack: every opening bracket is pushed, and every closing bracket pops the top (which must match). The maximum number of elements on the stack at any moment is _____.

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Python — `+=` versus `+` on lists]

Consider the following Python program. What is printed?

Python

```
1 a = [1, 2]
2 b = a
3 a += [3]
4 c = a
5 a = a + [4]
6 print(b, c is b, len(a))
```

- (A) [1, 2] False 4
- (B) [1, 2, 3] True 4
- (C) [1, 2, 3, 4] True 4
- (D) [1, 2] True 3

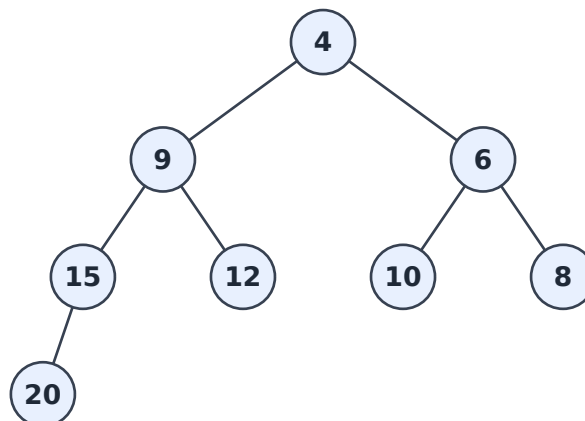
Q.6 [MSQ · one or more correct | 1 mark | Hashing — linear probing with deletion]

Keys 31, 41, 51, 22, 62, 9, 19 are inserted in this order into an empty table of size 10 using $h(k) = k \bmod 10$ and linear probing. Then 41 and 22 are deleted by marking their slots with a **tombstone** (searches continue past a tombstone; an insertion of a key not present uses the first tombstone or empty slot on its probe path). Which statements is/are TRUE?

- (A) Had the deletions simply emptied the slots (no tombstones), a search for 62 would fail
- (B) Inserting 72 now places it in slot 2
- (C) A search for 62 examines exactly 4 slots
- (D) An unsuccessful search for 12 examines exactly 6 slots

Q.7 [MCQ · single correct | 1 mark | Heaps — insertion with sift-up]

The min-heap shown below is stored in an array. The keys 5 and then 3 are inserted using the standard insert (append at the end, then sift up). The resulting array is



Min-heap [4, 9, 6, 15, 12, 10, 8, 20]

- (A) [3, 4, 6, 9, 12, 10, 8, 20, 15, 5]
- (B) [3, 5, 6, 9, 4, 10, 8, 20, 15, 12]
- (C) [3, 4, 6, 5, 9, 10, 8, 20, 15, 12]
- (D) [3, 4, 6, 9, 5, 10, 8, 20, 15, 12]

Q.8 [NAT · numerical answer | 1 mark | Selection sort — counting swaps]

Selection sort (ascending) is applied to [5, 2, 8, 1, 9, 3, 7]. In pass i ($i = 0, 1, \dots, n-2$) it finds the index m of the minimum of $A[i..n-1]$ and swaps $A[i]$ with $A[m]$ **only if** $m \neq i$. The number of swaps performed is _____.

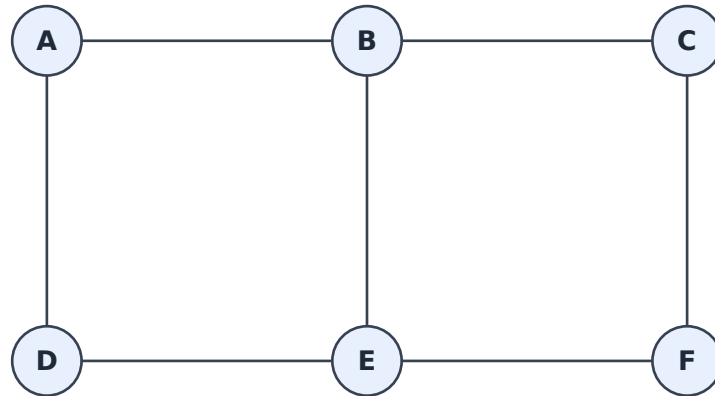
Answer: _____ (enter a numerical value)

Q.9 [MCQ · single correct | 1 mark | BFS with adjacency lists]

The undirected graph shown below is stored as adjacency **linked lists**, in which the neighbours appear in exactly this order:

A: D, B B: E, A, C C: B, F D: A, E E: D, B, F F: E, C

BFS from A visits neighbours in adjacency-list order. The BFS visit order is



(A) A, B, D, C, E, F

(B) A, D, B, E, C, F

(C) A, D, E, B, F, C

(D) A, D, B, C, E, F

Q.10 [MCQ · single correct | 1 mark | Binary search — lower bound]

Consider the following Python program. What is printed?

Python

```

1 def lb(A, x):
2     lo, hi = 0, len(A)
3     while lo < hi:
4         m = (lo + hi) // 2
5         if A[m] < x:
6             lo = m + 1
7         else:
8             hi = m
9     return lo
10
11 A = [2, 4, 4, 4, 7, 9, 9, 12]
12 print(lb(A, 4), lb(A, 9) - lb(A, 8), lb(A, 13))
  
```

(A) 1 2 8

(B) 3 2 7

(C) 1 0 8

(D) 1 0 7

Q.11 [MCQ · single correct | 2 marks | Linked lists — reversal in groups of k]

Consider the following Python program. What is printed?

Python

```

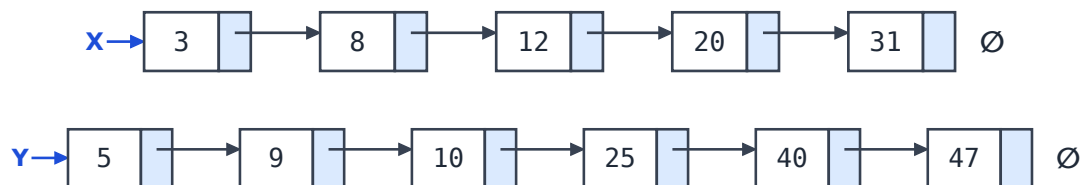
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4 def mk(vals):
5     h = None
6     for v in reversed(vals): h = Node(v, h)
7     return h
8 def show(h):
9     out = []
10    while h: out.append(h.v); h = h.nxt
11    return out
12
13 def rev_k(h, k):
14     p, n = h, 0
15     while p and n < k:
16         p, n = p.nxt, n + 1
17     if n < k:
18         return h
19     prev, cur = rev_k(p, k), h
20     for _ in range(k):
21         cur.nxt, prev, cur = prev, cur, cur.nxt
22     return prev
23
24 print(show(rev_k(mk([1, 2, 3, 4, 5, 6, 7, 8]), 3)))

```

- (A) [8, 7, 6, 5, 4, 3, 2, 1]
- (B) [3, 2, 1, 6, 5, 4, 8, 7]
- (C) [3, 2, 1, 6, 5, 4, 7, 8]
- (D) [3, 2, 1, 4, 5, 6, 7, 8]

Q.12 [MSQ · one or more correct | 2 marks | Merging sorted linked lists]

Two sorted singly linked lists $X = 3 \rightarrow 8 \rightarrow 12 \rightarrow 20 \rightarrow 31$ and $Y = 5 \rightarrow 9 \rightarrow 10 \rightarrow 25 \rightarrow 40 \rightarrow 47$ are merged into one sorted list by the standard method: repeatedly compare the heads, detach the smaller and append it to the result; when one list becomes empty the rest of the other is attached without comparisons. Which of the following statements is/are TRUE?



- (A) For some two sorted lists of lengths 5 and 6, this method makes 11 comparisons
- (B) For some two sorted lists of lengths 5 and 6, this method makes only 5 comparisons
- (C) Done by relinking the existing nodes, the merge needs only $O(1)$ extra space
- (D) Exactly 9 key comparisons are made for these two lists

Q.13 [NAT · numerical answer | 2 marks | Circular linked list — elimination game]

Consider the following Python program. The value printed is _____.

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 n, k = 10, 4
6 head = Node(1)
7 cur = head
8 for v in range(2, n + 1):
9     cur.nxt = Node(v)
10    cur = cur.nxt
11 cur.nxt = head          # close the circle; cur = node 10
12
13 while cur.nxt is not cur:
14     for _ in range(k - 1):
15         cur = cur.nxt
16     cur.nxt = cur.nxt.nxt  # remove the node after cur
17 print(cur.v)

```

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Linked lists — rotation by k]

Consider the following Python program. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4 def mk(vals):
5     h = None
6     for v in reversed(vals): h = Node(v, h)
7     return h
8 def show(h):
9     out = []
10    while h: out.append(h.v); h = h.nxt
11    return out
12
13 def rotate(h, k):
14     if h is None:
15         return h
16     n, tail = 1, h
17     while tail.nxt:
18         tail, n = tail.nxt, n + 1
19     k %= n
20     if k == 0:
21         return h
22     tail.nxt = h
23     for _ in range(n - k):
24         tail = tail.nxt
25     h, tail.nxt = tail.nxt, None
26     return h
27
28 print(show(rotate(mk([1, 2, 3, 4, 5]), 13)))

```

- (A) [2, 3, 4, 5, 1]
- (B) [4, 5, 1, 2, 3]
- (C) [1, 2, 3, 4, 5]
- (D) [3, 4, 5, 1, 2]

Q.15 [MSQ · one or more correct | 2 marks | Intersection of two linked lists — two-pointer method]

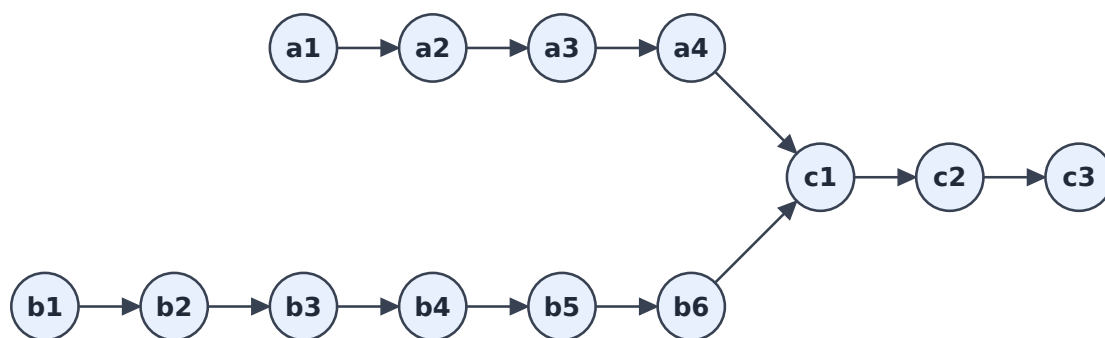
Two singly linked lists share a common tail as shown: list A is $a1 \rightarrow a2 \rightarrow a3 \rightarrow a4 \rightarrow c1 \rightarrow c2 \rightarrow c3$ and list B is $b1 \rightarrow \dots \rightarrow b6 \rightarrow c1 \rightarrow c2 \rightarrow c3$ (the c-nodes are the **same** objects). The fragment below is run with $hA = a1$ and $hB = b1$. Which of the following statements is/are TRUE?

Python

```

1 p, q, steps = hA, hB, 0
2 while p is not q:
3     p = p.nxt if p else hB
4     q = q.nxt if q else hA
5     steps += 1

```



- (A) The loop body executes exactly 14 times
- (B) When the loop ends, p and q both refer to c1
- (C) The fragment uses $O(1)$ extra space
- (D) If the two lists had no common node (lengths 7 and 9), the loop would end after 16 iterations with $p = q = \text{None}$

Q.16 [NAT · numerical answer | 2 marks | Recursion on linked lists]

Consider the following Python program. The value printed is _____.

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4 def mk(vals):
5     h = None
6     for v in reversed(vals): h = Node(v, h)
7     return h
8 def show(h):
9     out = []
10    while h: out.append(h.v); h = h.nxt
11    return out
12
13 def f(h):
14     if h is None:
15         return 0
16     r = f(h.nxt)
17     return r * 2 + h.v
18
19 print(f(mk([1, 1, 0, 1, 0, 0, 1])))

```

Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Generators over a linked list mutated during iteration]

Consider the following Python program. What is printed?

Python

```

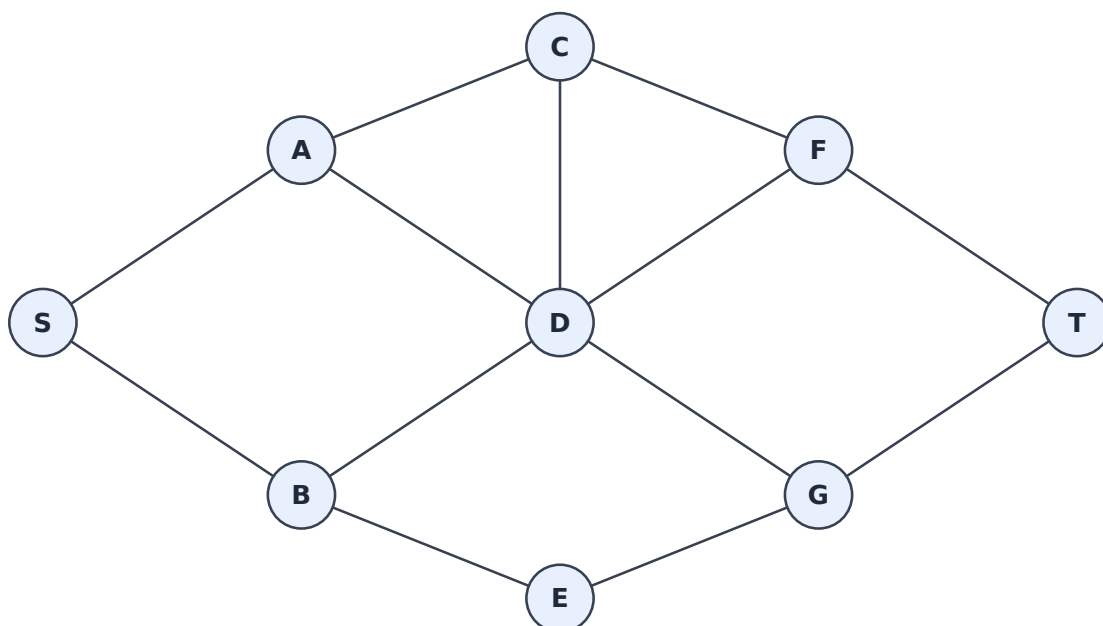
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 class LL:
6     def __init__(self, vals):
7         self.head = None
8         for v in reversed(vals):
9             self.head = Node(v, self.head)
10    def __iter__(self):
11        n = self.head
12        while n:
13            yield n
14            n = n.nxt
15
16 L = LL([3, 6, 7, 9, 10, 12, 13])
17 seen = []
18 for node in L:
19     seen.append(node.v)
20     if node.v % 3 == 0 and node.nxt:
21         node.nxt = node.nxt.nxt
22 print(seen, [n.v for n in L])

```

- (A) [3, 7, 9, 12] [3, 7, 9, 12]
 (B) [3, 6, 7, 9, 10, 12, 13] [3, 7, 9, 12]
 (C) [3, 6, 9, 12] [3, 7, 9, 12]
 (D) [3, 7, 9, 12] [3, 6, 7, 9, 10, 12, 13]

Q.18 [NAT · numerical answer | 2 marks | BFS — counting shortest paths]

In the unweighted undirected graph shown below, the number of distinct **shortest** paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Sorted list to balanced BST]

A sorted linked list $1 \rightarrow 2 \rightarrow \dots \rightarrow 10$ is converted into a BST recursively: for the sub-list at positions $lo..hi$ (0-based), the node at position $m = \lfloor (lo + hi)/2 \rfloor$ becomes the root, and positions $lo..m-1$ and $m+1..hi$ build the left and right subtrees. Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) The BST has exactly 5 leaves
- (B) The height of the BST is 4
- (C) The root of the BST is 5
- (D) The pre-order traversal is 5, 2, 1, 3, 4, 8, 6, 7, 9, 10

Q.20 [NAT · numerical answer | 2 marks | Merge sort on a linked list — comparisons]

Merge sort is applied to the linked list $21 \rightarrow 7 \rightarrow 33 \rightarrow 14 \rightarrow 2 \rightarrow 40 \rightarrow 19 \rightarrow 11 \rightarrow 28$. A list of length $L > 1$ is split into its first $\lfloor L/2 \rfloor$ nodes and the remaining $\lfloor L/2 \rfloor$ nodes; both halves are sorted recursively and merged by the standard merge (compare heads; once one list is empty, attach the rest without comparisons). The total number of key comparisons is _____.

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 26

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Linked lists — middle node with slow/fast pointers
2	NAT	1	8	Cycle detection — Floyd's algorithm
3	MSQ	1	(A), (C)	Linked lists — cost of operations
4	NAT	1	4	Stacks — bracket matching
5	MCQ	1	(B)	Python — += versus + on lists
6	MSQ	1	(A), (B), (C)	Hashing — linear probing with deletion
7	MCQ	1	(D)	Heaps — insertion with sift-up
8	NAT	1	3	Selection sort — counting swaps
9	MCQ	1	(B)	BFS with adjacency lists
10	MCQ	1	(C)	Binary search — lower bound
11	MCQ	2	(C)	Linked lists — reversal in groups of k
12	MSQ	2	(B), (C), (D)	Merging sorted linked lists
13	NAT	2	5	Circular linked list — elimination game
14	MCQ	2	(D)	Linked lists — rotation by k
15	MSQ	2	(A), (B), (C)	Intersection of two linked lists — two-pointer method
16	NAT	2	75	Recursion on linked lists
17	MCQ	2	(A)	Generators over a linked list mutated during iteration
18	NAT	2	6	BFS — counting shortest paths
19	MSQ	2	(C), (D)	Sorted list to balanced BST
20	NAT	2	20	Merge sort on a linked list — comparisons

Detailed Solutions — Mock Test 26

Q.1 [MCQ · 1 mark · Linked lists — middle node with slow/fast pointers]

Answer: (A)

The fast pointer moves two nodes per step and the slow pointer one, so slow ends near the middle. For an **even** length, the exact stopping condition decides which of the two middle nodes is returned.

Loop 1 (`while f and f.next`): positions (slow, fast) = (10, 10) → (20, 30) → (30, 50) → (40, 70) → (50, None). At (40, 70), `f = 70` and `f.next = 80`, so one more step: `f` becomes None. `x = 50` (the second middle).

Loop 2 (`while f.next and f.next.next`): (10, 10) → (20, 30) → (30, 50) → (40, 70); now `f.next = 80` but `f.next.next = None`, so stop. `s.v = 40` (the first middle).

Output: 50 40.

- (B), (C), (D) mix up which condition gives which middle.

Tip: for even n , `while f and f.next` returns node $n/2 + 1$ (1-based); `while f.next and f.next.next` returns node $n/2$ — the latter is what merge sort on lists uses to split into equal halves.



The two middles: 40 (loop 2) and 50 (loop 1)

Q.2 [NAT · 1 mark · Cycle detection — Floyd's algorithm]

Answer: 8

Let $\mu = 4$ be the tail length and $\lambda = 8$ the cycle length. After t iterations slow is at position t and fast at $2t$ along the 'unrolled' path. They meet at the first $t \geq \mu$ with $2t - t = t \equiv 0 \pmod{\lambda}$, i.e. the smallest multiple of λ that is $\geq \mu$.

Here $\lambda = 8 \geq \mu = 4$, so **$t = 8$** . Check by tracing (slow, fast):

(1, 2), (2, 4), (3, 6), (4, 8), (5, 10), (6, 4), (7, 6), (8, 8) → meet at node 8 after 8 iterations. (Fast at step 6: position 12 → node 4, since $12 - 4 = 8 \equiv 0$.)

Tip: the meeting point is μ steps $\pmod{\lambda}$ before the cycle start along the cycle, so restarting one pointer at the head and moving both one step at a time finds the cycle start (node 4) after $\mu = 4$ more steps.

Trap: the answer is not always λ : if $\mu > \lambda$, t is the first multiple of λ that is $\geq \mu$.

Q.3 [MSQ · 1 mark · Linked lists — cost of operations]

Answer: (A), (C)

$O(1)$ is possible only when every reference that must change is reachable in constant steps.

- (A) `tail.next = new; tail = new` — **$O(1)$** . True.
- (B) After deletion `tail` must point to the *second-to-last* node, which can only be found by walking from `head` (no back pointers) — $\Theta(n)$. False.
- (C) Copy the successor into `p` and bypass it: `p.val = p.next.val; p.next = p.next.next` (update `tail` if the bypassed node was the tail). **$O(1)$** . True — this is why the question says node identity need not be preserved.
- (D) Needs a traversal (e.g. slow/fast pointers) — $\Theta(n)$ unless the length and middle are maintained separately. False.

Trap: having a `tail` pointer makes *insertion* at the end $O(1)$ but not *deletion* at the end; that needs a doubly linked list.

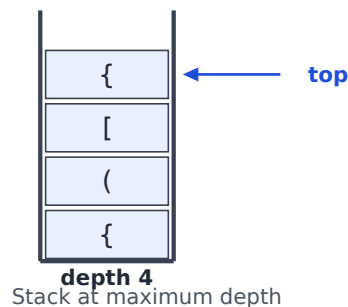
Q.4 [NAT · 1 mark · Stacks — bracket matching]**Answer: 4**

The stack size equals the current nesting depth. Track it character by character:

- { 1, [2, (3,) 2, (3,) 2,] 1
- (2, [3, { 4, } 3, (4,) 3,] 2,) 1, } 0
- [1, (2,) 1,] 0

Every closing bracket matches the top, and the stack is empty at the end, so the string is balanced. Maximum depth = **4**, reached at { inside ([{ and again at the (after }.

Trap: counting the total number of pushes (10) or the number of bracket *pairs* instead of the maximum simultaneous nesting.

**Q.5 [MCQ · 1 mark · Python — `+=` versus `+` on lists]****Answer: (B)**

For lists, `a += x` calls `list.__iadd__`, which **extends the same object** in place. `a = a + x` builds a **new** list and rebinds `a`.

- `b = a` → both names refer to list `L = [1, 2]`.
- `a += [3]` → `L` becomes `[1, 2, 3]`; `a` and `b` still refer to `L`.
- `c = a` → `c` refers to `L`.
- `a = a + [4]` → new list `[1, 2, 3, 4]`; only `a` is rebound.

So `b = [1, 2, 3]`, `c is b` is `True` (both are `L`), `len(a) = 4`.

- (A) treats `+=` as creating a new list.
- (C) treats `a = a + [4]` as modifying `L`.
- (D) misses the in-place extension *and* the final concatenation.

Trap: for immutable types (tuples, strings, ints) `+=` does rebound; for lists it mutates.

Q.6 [MSQ · 1 mark · Hashing — linear probing with deletion]**Answer: (A), (B), (C)**

Build the table: 31→1, 41→2, 51→3, 22→4 (2, 3 full), 62→5 (2, 3, 4 full), 9→9, 19→0 (9 full, wraps to 0). After deletions: slot 2 = T (tombstone), slot 4 = T.

Table: 0:19, 1:31, 2:T, 3:51, 4:T, 5:62, 9:9 (slots 6, 7, 8 empty).

- (A) Without tombstones slot 2 would be empty, so the search for 62 stops at its home slot and reports 'absent' although 62 is in slot 5. **True.**
- (B) 72 (home 2): slot 2 is the first tombstone → stored in slot **2**. **True.**
- (C) 62: probe 2 (T, continue), 3 (51), 4 (T), 5 (62 found) → **4 slots**. **True.**
- (D) 12 (home 2): 2 (T), 3, 4 (T), 5, 6 (empty → stop) → **5 slots**, not 6. **False.**

Trap: deletion in open addressing must not break probe chains; tombstones preserve them.

0	19
1	31
2	T
3	51
4	T
5	62
6	
7	
8	
9	9

Table after deleting 41 and 22 (T = tombstone)

Q.7 [MCQ · 1 mark · Heaps — insertion with sift-up]**Answer: (D)**

Insertion appends at index n and swaps with the parent (index $(i-1)//2$) while the parent is larger.

Insert 5 at index 8: parent 3 holds 15 $\rightarrow$ swap (5 at 3); parent 1 holds 9 $\rightarrow$ swap (5 at 1); parent 0 holds 4 $<$ 5 $\rightarrow$ stop.

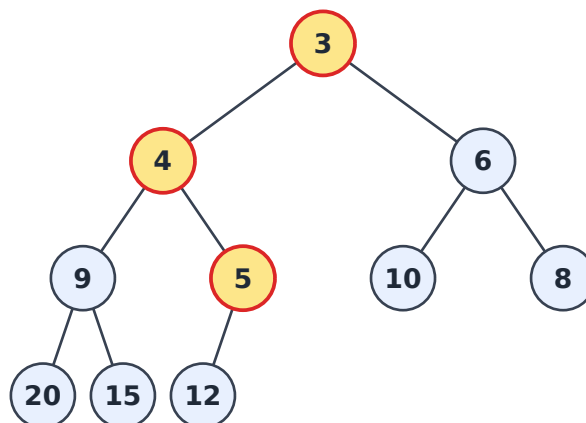
Array: [4, 5, 6, 9, 12, 10, 8, 20, 15].

Insert 3 at index 9: parent 4 holds 12 $\rightarrow$ swap (3 at 4); parent 1 holds 5 $\rightarrow$ swap (3 at 1); parent 0 holds 4 $\rightarrow$ swap (3 at 0).

Array: [3, 4, 6, 9, 5, 10, 8, 20, 15, 12].

- (A) forgets to sift 3 up beyond its parent 12 and mis-places 5.
- (B) leaves 4 at index 4 — the swaps move each displaced parent *down one level only*.
- (C) places 5 under the wrong parent (index 3's child is index 7/8, not index 4).

Tip: the sift-up path of index i is $i, (i-1)//2, \dots, 0$ — for $i = 9$: $9 \rightarrow 4 \rightarrow 1 \rightarrow 0$.



After inserting 5 and 3 (sift-up path of 3 highlighted)

Q.8 [NAT · 1 mark · Selection sort — counting swaps]**Answer: 3**

Selection sort places the i -th smallest element at position i in pass i ; a swap is skipped when that element is already in place.

- $i=0$: min of whole array is 1 (index 3) $\rightarrow$ swap $\rightarrow$ [1, 2, 8, 5, 9, 3, 7] (1)
- $i=1$: min of $A[1..]$ is 2, already at index 1 $\rightarrow$ no swap

- $i=2$: min is 3 (index 5) $\rightarrow$ swap $\rightarrow$ [1, 2, 3, 5, 9, 8, 7] (2)
- $i=3$: min is 5, already in place $\rightarrow$ no swap
- $i=4$: min is 7 (index 6) $\rightarrow$ swap $\rightarrow$ [1, 2, 3, 5, 7, 8, 9] (3)
- $i=5$: min is 8, in place $\rightarrow$ no swap

Total = **3** swaps.

Tip: the number of (non-trivial) swaps equals n minus the number of cycles of the permutation as modified by earlier passes — easiest is just to trace. Selection sort never needs more than $n - 1$ swaps.

Q.9 [MCQ · 1 mark · BFS with adjacency lists]

Answer: (B)

BFS dequeues a vertex and enqueues its unvisited neighbours **in the order of its adjacency list**, which need not be alphabetical.

- Dequeue A: enqueue D, B $\rightarrow$ queue [D, B]
- Dequeue D: neighbours A (seen), E $\rightarrow$ queue [B, E]
- Dequeue B: E (seen), A (seen), C $\rightarrow$ queue [E, C]
- Dequeue E: D, B seen; F $\rightarrow$ queue [C, F]
- Dequeue C, then F.

Order: **A, D, B, E, C, F**.

- (A) assumes alphabetical neighbour order.
- (C) is a DFS-like order (going deep through D $\rightarrow$ E).
- (D) processes B's neighbours before D's — but D was enqueued first.

Trap: the representation (order inside adjacency lists) determines the traversal order.

Q.10 [MCQ · 1 mark · Binary search — lower bound]

Answer: (C)

`lb(A, x)` returns the **first index i with $A[i] \geq x$** (lower bound) — $\text{len}(A)$ if no such index. The invariant: everything left of `lo` is $< x$, everything from `hi` on is $\geq x$.

- `lb(A, 4)`: first element ≥ 4 is at index **1** (the leftmost 4). Trace: (0,8) $m=4$ ($7 \geq 4$) $\rightarrow hi=4$; $m=2$ (4) $\rightarrow hi=2$; $m=1$ (4) $\rightarrow hi=1$; $m=0$ ($2 < 4$) $\rightarrow lo=1$.
- `lb(A, 9) = 5` and `lb(A, 8) = 5` (8 is absent; the first element ≥ 8 is 9 at index 5) $\rightarrow$ difference **0**.
- `lb(A, 13)`: every element is $< 13 \rightarrow$ returns $\text{len}(A) = \mathbf{8}$.

Output **1 0 8**.

- (A) treats `lb(9) - lb(8)` as the count of 9s (that would be `lb(A, 10) - lb(A, 9) = 2`).
- (B) returns the last occurrence of 4 and counts 9s instead.
- (D) thinks an absent larger key returns the last index.

Tip: the count of x is `lb(A, x+1) - lb(A, x)` for integers.

Q.11 [MCQ · 2 marks · Linked lists — reversal in groups of k]

Answer: (C)

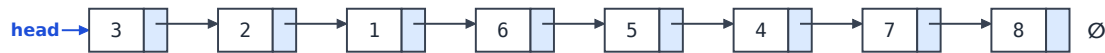
`rev_k` first checks that at least k nodes remain; if not, it returns the remainder **unchanged**. Otherwise it recursively processes the list after the first k nodes (`p`), then reverses the first k nodes so that the old first node points to that processed remainder.

The tuple assignment `cur.next, prev, cur = prev, cur, cur.next` evaluates the right-hand side first (old `prev`, old `cur`, old `cur.next`) and then assigns left to right: the target `cur.next` still refers to the old `cur`, so this is a correct one-line pointer reversal.

- `rev_k(7  $\rightarrow$  8, 3)`: only 2 nodes $\rightarrow$ returned unchanged: $7 \rightarrow 8$.

- $\text{rev}_k(4 \rightarrow 5 \rightarrow 6 \rightarrow \dots)$: reverse 4, 5, 6 onto $7 \rightarrow 8 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 7 \rightarrow 8$.
- $\text{rev}_k(1 \rightarrow \dots)$: reverse 1, 2, 3 onto that $\rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 7 \rightarrow 8$.
- (A) is a full reversal.
- (B) reverses the incomplete last group too.
- (D) reverses only the first group.

Trap: in a multiple assignment the right side is evaluated completely before any binding; but the *left-side targets* are assigned left to right, so `cur, cur.next = ...` (other order) would set the `next` of the **new** `cur` — a classic bug.



Result: two full groups reversed, last group of 2 untouched

Q.12 [MSQ · 2 marks · Merging sorted linked lists]

Answer: (B), (C), (D)

Each comparison outputs exactly one node; comparisons stop as soon as one list is exhausted.

Trace: 3 vs 5 $\rightarrow$ 3; 8 vs 5 $\rightarrow$ 5; 8 vs 9 $\rightarrow$ 8; 12 vs 9 $\rightarrow$ 9; 12 vs 10 $\rightarrow$ 10; 12 vs 25 $\rightarrow$ 12; 20 vs 25 $\rightarrow$ 20; 31 vs 25 $\rightarrow$ 25; 31 vs 40 $\rightarrow$ 31. X is now empty; attach 40 $\rightarrow$ 47. **9 comparisons.**

- (A) At most $m + n - 1 = 10$ comparisons: after 10 outputs, at least one list is empty (the last node is never compared). **False.**
- (B) If all of the 5-element list is smaller than the first element of the other, each comparison outputs a node of the shorter list: $\min(m, n) = 5$ comparisons. **True.**
- (C) Relinking uses a constant number of pointers (a dummy head and a tail pointer). **True.**
- (D) **True.**

Tip: the number of comparisons is $(m + n) - (\text{number of nodes appended after one list runs out})$; here $11 - 2 = 9$.



Merged list (40, 47 attached without comparisons)

Q.13 [NAT · 2 marks · Circular linked list — elimination game]

Answer: 5

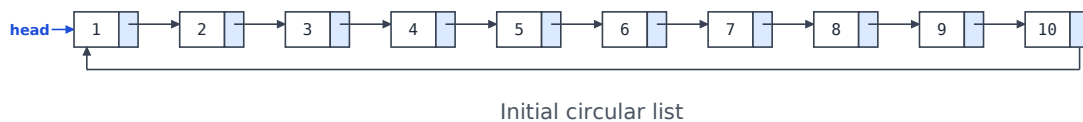
`cur` always sits **just before** the next person to be counted, so after $k - 1$ moves `cur.next` is the k -th person, who is unlinked. This is the Josephus process with every 4th person removed, starting the count at 1.

Circle 1 ... 10, `cur` = 10.

- Count 1, 2, 3, **4** $\rightarrow$ remove 4; circle 1 2 3 5 6 7 8 9 10, `cur` = 3.
- 5, 6, 7, **8** $\rightarrow$ remove 8; `cur` = 7.
- 9, 10, 1, **2** $\rightarrow$ remove 2; `cur` = 1.
- 3, 5, 6, **7** $\rightarrow$ remove 7; `cur` = 6.
- 9, 10, 1, **3** $\rightarrow$ remove 3; `cur` = 1.
- 5, 6, 9, **10** $\rightarrow$ remove 10; `cur` = 9.
- 1, 5, 6, **9** $\rightarrow$ remove 9; `cur` = 6.
- 1, 5, 6, **1** $\rightarrow$ remove 1; `cur` = 6.
- 5, 6, 5, **6** $\rightarrow$ remove 6; only 5 remains.

Removal order: 4, 8, 2, 7, 3, 10, 9, 1, 6. Survivor **5**.

Tip: the Josephus recurrence $J(1) = 0$, $J(n) = (J(n-1) + k) \bmod n$ (0-based) gives $J(10) = 4$, i.e. person 5. **Trap:** forgetting that counting skips removed nodes and wraps around.

**Q.14 [MCQ · 2 marks · Linked lists — rotation by k]****Answer: (D)**

The function rotates the list **right** by k : it finds the length n and the tail, reduces $k \bmod n$, makes the list circular, walks $n - k$ steps from the tail to reach the new tail, and breaks the circle after it.

- $n = 5$, tail = node 5; $k = 13 \bmod 5 = 3$.
- tail.next = head (circle $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$).
- Walk $n - k = 2$ steps from node 5: $\rightarrow 1 \rightarrow 2$. New tail = node 2.
- `h, tail.next = tail.next, None`: RHS evaluates to (node 3, None); then $h = \text{node 3}$ and node 2's next = None.

Result: **$3 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 2$** .

- (A) is a left rotation by 1.
- (B) is a rotation *left* by 3 (equivalently right by 2).
- (C) assumes 13 is a multiple of 5 or that the circle is never broken.

Tip: rotating right by k moves the last k nodes to the front; check: the last three nodes 3, 4, 5 come first.

Q.15 [MSQ · 2 marks · Intersection of two linked lists — two-pointer method]**Answer: (A), (B), (C)**

Each pointer walks its own list, then (after stepping onto **None** once) switches to the head of the other list. Both therefore walk $(\text{length A}) + 1 + (\text{length B before the junction})$ steps before arriving at the junction, which equalises the head-start difference.

- p's route: $a_1 \dots a_4, c_1, c_2, c_3$ (indices 0–6), None (7), $b_1 \dots b_6$ (8–13), **c1 (14)**.
- q's route: $b_1 \dots b_6, c_1, c_2, c_3$ (0–8), None (9), $a_1 \dots a_4$ (10–13), **c1 (14)**.
- They never coincide before step 14 (check: when p is at c_1 (step 4), q is at b_5 ; when q is at c_1 (step 6), p is at c_3).
- (A) **True** — 14 iterations.
- (B) **True** — they meet at the first common node c_1 .
- (C) Only two pointers and a counter. **True**.
- (D) With no common node, both reach **None** the second time after $7 + 1 + 9 = 9 + 1 + 7 = 17$ iterations, not 16. **False**.

Trap: because the **None** itself is a step in this code, the counts are one larger per list than the plain lengths suggest.

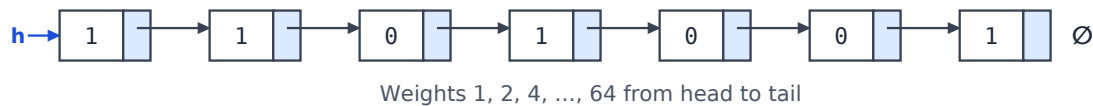
Q.16 [NAT · 2 marks · Recursion on linked lists]**Answer: 75**

Unfold the recursion: $f(h) = h.v + 2 \cdot f(h.\text{next})$. So for values $v_0, v_1, \dots, v_6$ (head first), $f = v_0 + 2v_1 + 4v_2 + 8v_3 + 16v_4 + 32v_5 + 64v_6$ — the list is read as a binary number whose **least significant bit is at the head**.

Values 1, 1, 0, 1, 0, 0, 1 $\rightarrow 1 + 2 + 0 + 8 + 0 + 0 + 64 = 75$.

Trace of the return values, from the tail back to the head ($f(\text{None}) = 0$): $0 \cdot 2 + 1 = 1 \rightarrow 2 \cdot 1 + 0 = 2 \rightarrow 2 \cdot 2 + 0 = 4 \rightarrow 2 \cdot 4 + 1 = 9 \rightarrow 2 \cdot 9 + 0 = 18 \rightarrow 2 \cdot 18 + 1 = 37 \rightarrow 2 \cdot 37 + 1 = 75$.

Trap: reading the list head-first as binary 1101001_2 gives 105 — that would require **f** to pass an accumulator *down* (Horner's rule from the head), not combine results on the way *up*.

**Q.17 [MCQ · 2 marks · Generators over a linked list mutated during iteration]****Answer: (A)**

The generator is **lazy**: after yielding node n it pauses, and only when resumed does it read $n.next$. Any change the loop body makes to $node.next$ is therefore seen by the iteration.

- Yield 3: $3 \% 3 == 0 \rightarrow 3.next = 7$ (6 unlinked). Generator resumes: $n = 3.next = 7$.
- Yield 7: not a multiple of 3. Next: 9.
- Yield 9: $9.next = 12$ (10 unlinked). Next: 12.
- Yield 12: $12.next = None$ (13 unlinked). Next: None $\rightarrow$ stop.

seen = [3, 7, 9, 12]; the list itself is now $3 \rightarrow 7 \rightarrow 9 \rightarrow 12$.

- (B) assumes the generator took a snapshot of the list.
- (C) assumes the removal takes effect one step late (visiting 6).
- (D) assumes the iteration worked on a copy and the list is unchanged.

Trap: generators do not copy their source; mutating a structure while iterating over it changes what the iterator produces.

Q.18 [NAT · 2 marks · BFS — counting shortest paths]**Answer: 6**

BFS assigns levels; the number of shortest paths to v is the sum of the counts of its neighbours on the **previous** level: $\sigma(v) = \sum \sigma(u)$ over edges $u-v$ with $d(u) = d(v) - 1$.

- Level 0: S ($\sigma = 1$)
- Level 1: A (1), B (1)
- Level 2: C (from A: 1), D (from A and B: 2), E (from B: 1)
- Level 3: F (from C and D: $1 + 2 = 3$), G (from D and E: $2 + 1 = 3$)
- Level 4: T (from F and G: $3 + 3 = 6$)

The edge C-D joins two vertices on the *same* level, so it lies on no shortest path.

Trap: adding paths that use C-D (e.g. S-A-C-D-G-T, length 5) or counting all simple paths instead of shortest ones.

BFS level d and path count σ

	S	A	B	C	D	E	F	G	T
d	0	1	1	2	2	2	3	3	4
σ	1	1	1	1	2	1	3	3	6

Q.19 [MSQ · 2 marks · Sorted list to balanced BST]**Answer: (C), (D)**

Choosing the (lower) middle as root at every level gives a height-balanced BST.

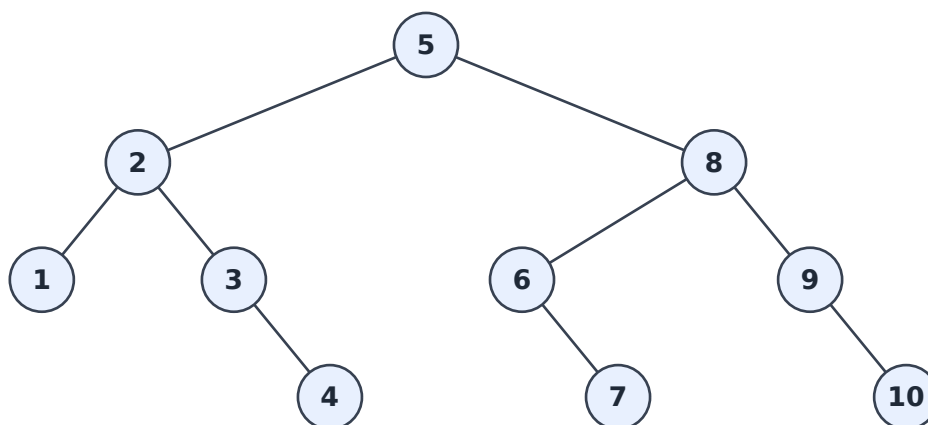
- Whole list, positions 0..9: $m = 4 \rightarrow$ root **5**. Left = {1..4}, right = {6..10}.
- {1, 2, 3, 4} (0..3): $m = 1 \rightarrow 2$; left {1}; right {3, 4} $\rightarrow m = 2 \rightarrow 3$ with right child 4.
- {6, ..., 10} (5..9): $m = 7 \rightarrow 8$; left {6, 7} $\rightarrow 6$ with right child 7; right {9, 10} $\rightarrow 9$ with right child 10.

Tree: $5 \rightarrow (2 \rightarrow 1, (3 \rightarrow -, 4)), (8 \rightarrow (6 \rightarrow -, 7), (9 \rightarrow -, 10))$.

- (A) Leaves are 1, 4, 7, 10 — four. **False.**

- (B) Longest paths, e.g. $5 \rightarrow 2 \rightarrow 3 \rightarrow 4$, have 3 edges; height = 3. **False.**
- (C) **True.**
- (D) Pre-order 5, 2, 1, 3, 4, 8, 6, 7, 9, 10. **True.**

Trap: using the upper middle $\lceil (\text{lo} + \text{hi}) / 2 \rceil$ would make 6 the root; read the rounding rule. Note the height $\lceil \log_2 10 \rceil = 3$ is optimal for 10 keys.



Balanced BST built from the sorted list

Q.20 [NAT · 2 marks · Merge sort on a linked list — comparisons]

Answer: 20

Split tree: $[21\ 7\ 33\ 14\ 2] + [40\ 19\ 11\ 28]$; $[21\ 7\ 33] + [14\ 2]$; $[21\ 7] + [33]$; $[40\ 19] + [11\ 28]$.

Merges (bottom-up) with comparison counts:

- $[21] + [7] \rightarrow [7, 21]$: 1
- $[7, 21] + [33] \rightarrow 7\ \text{vs}\ 33, 21\ \text{vs}\ 33$, then attach 33: 2
- $[14] + [2] \rightarrow [2, 14]$: 1
- $[7, 21, 33] + [2, 14] \rightarrow 7\ \text{vs}\ 2\ (2), 7\ \text{vs}\ 14\ (7), 21\ \text{vs}\ 14\ (14)$, then attach: 3
- $[40] + [19] \rightarrow 1$; $[11] + [28] \rightarrow 1$
- $[19, 40] + [11, 28] \rightarrow 19\ \text{vs}\ 11, 19\ \text{vs}\ 28, 40\ \text{vs}\ 28$, then attach 40: 3
- $[2, 7, 14, 21, 33] + [11, 19, 28, 40] \rightarrow 2/11, 7/11, 14/11, 14/19, 21/19, 21/28, 33/28, 33/40$, then attach 40: 8

Total = $1 + 2 + 1 + 3 + 1 + 1 + 3 + 8 = 20$.

Tip: each merge of sizes a, b costs between $\min(a, b)$ and $a + b - 1$ comparisons, so the total for $n = 9$ lies between 13 and 25 — a quick sanity check on any trace.

Merges performed

	left	right	result	comparisons
1	21	7	7 21	1
2	7 21	33	7 21 33	2
3	14	2	2 14	1
4	7 21 33	2 14	2 7 14 21 33	3
5	40	19	19 40	1
6	11	28	11 28	1
7	19 40	11 28	11 19 28 40	3
8	2 7 14 21 33	11 19 28 40	all 9 keys	8

Mock Test 27

Graph Representations

Difficulty: GATE-level

Focus areas: adjacency matrix/list, matrix powers & walks, degree sums, storage

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — aliasing when building adjacency lists]

A programmer builds adjacency lists for the path 0-1-2-3 as follows. What is printed?

Python

```

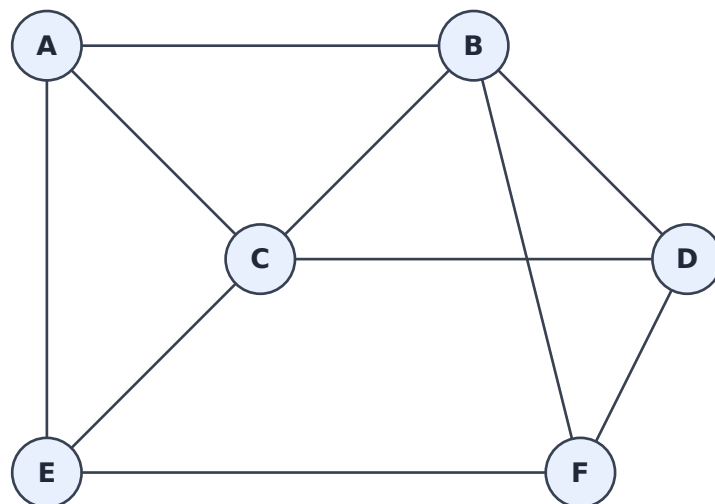
1 n = 4
2 adj = [[]] * n
3 for u, v in [(0, 1), (1, 2), (2, 3)]:
4     adj[u].append(v)
5     adj[v].append(u)
6 print(len(adj[0]), adj[3][:3])

```

- (A) 1 [2]
- (B) 6 [1, 0, 2]
- (C) 2 [2, 3]
- (D) 6 [2, 1, 0]

Q.2 [NAT · numerical answer | 1 mark | Adjacency matrix squared — common neighbours]

Let A be the adjacency matrix of the simple undirected graph G shown (rows and columns in the order A, B, C, D, E, F). The entry of A^2 in row B, column E is _____.



Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Graphical degree sequences]

Which one of the following sequences is the degree sequence of some **simple** undirected graph?

- (A) 5, 5, 4, 3, 2, 1
- (B) 6, 4, 3, 3, 2, 2
- (C) 4, 4, 3, 3, 2, 2
- (D) 3, 3, 3, 3, 3, 2

Q.4 [MSQ · one or more correct | 1 mark | Circular queue arithmetic]

A circular queue of capacity 8 is stored in an array $Q[0..7]$ with two variables: **front** (index of the first element) and **count**. Enqueue stores at $Q[(\text{front} + \text{count}) \bmod 8]$; dequeue returns $Q[\text{front}]$ and sets $\text{front} = (\text{front} + 1) \bmod 8$. Initially $\text{front} = 6$ and $\text{count} = 0$. The operations are: enqueue 10, 20, 30, 40, 50; dequeue twice; enqueue 60, 70, 80, 90. Which of the following statements is/are TRUE afterwards?

- (A) 90 is stored at $Q[6]$
- (B) $\text{front} = 0$
- (C) One more enqueue would succeed and store its value at $Q[7]$
- (D) The next dequeue returns 40

Q.5 [NAT · numerical answer | 1 mark | Hashing — quadratic probing]

Keys 25, 36, 47, 14, 58 are inserted in this order into an initially empty hash table of size 11 (indices 0–10) using quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) for key k examines slot $(k \bmod 11 + i^2) \bmod 11$. The index of the slot in which 58 is stored is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Binary search — lower-bound loop]

Consider the following Python program. What is printed?

Python

```
1 def search(a, x):
2     lo, hi = 0, len(a) - 1
3     steps = 0
4     while lo < hi:
5         mid = (lo + hi) // 2
6         steps += 1
7         if a[mid] < x:
8             lo = mid + 1
9         else:
10            hi = mid
11    return lo, steps
12
13 a = [3, 8, 8, 8, 15, 21, 27, 34, 40]
14 print(search(a, 8), search(a, 30))
```

- (A) (1, 4) (7, 3)
- (B) (2, 2) (7, 3)
- (C) (1, 3) (6, 3)
- (D) (3, 4) (7, 4)

Q.7 [NAT · numerical answer | 1 mark | Trees — parent-array representation]

A rooted tree on nodes 0..10 is stored as a parent array ($P[i]$ is the parent of node i , and -1 marks the root):

$P = [-1, 0, 0, 1, 1, 2, 4, 4, 6, 3, 8]$

The height of the tree (number of edges on the longest root-to-leaf path) is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Insertion sort — shifts]

Insertion sort sorts $[6, 2, 9, 4, 1, 8, 3]$ into ascending order. A *shift* is one execution of $A[j + 1] = A[j]$ (moving a larger element one place right). The total number of shifts is

- (A) 9
- (B) 11
- (C) 21
- (D) 12

Q.9 [MSQ · one or more correct | 1 mark | Adjacency matrix identities]

Let A be the adjacency matrix of a simple undirected graph G with m edges and t triangles. Which of the following statements is/are TRUE for every such G ?

- (A) The sum of all entries of A equals m
- (B) $\text{trace}(A^2) = 2m$
- (C) $\text{trace}(A^3) = 6t$
- (D) A is symmetric and every diagonal entry of A is 0

Q.10 [NAT · numerical answer | 1 mark | Directed adjacency matrix — in/out degrees]

The adjacency matrix M of a directed graph on vertices 1..6 is shown ($M[i][j] = 1$ means an edge $i \rightarrow j$). The number of vertices whose in-degree is **different** from their out-degree is _____.

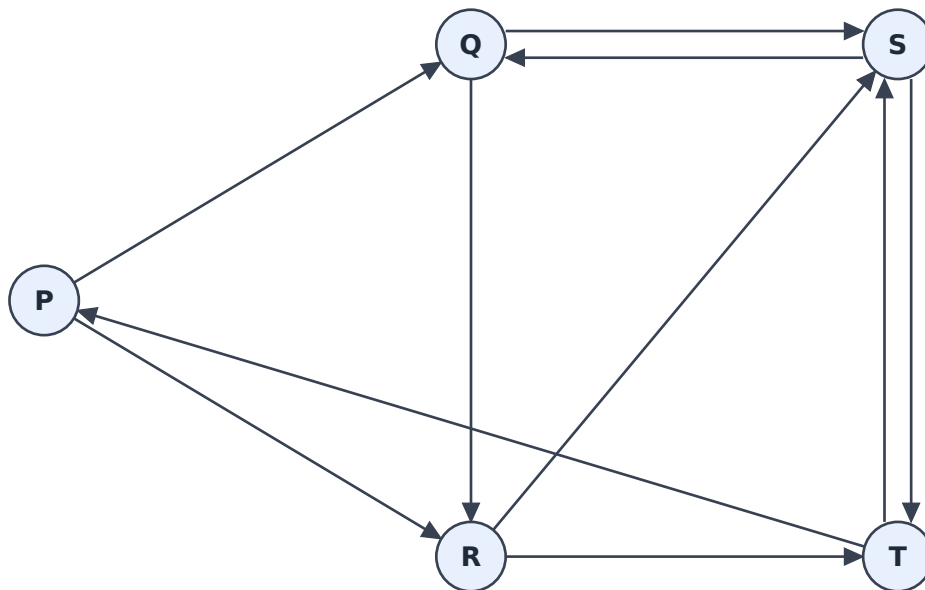
M (row = from, column = to)

	1	2	3	4	5	6
1	0	1	1	0	0	1
2	0	0	1	1	0	0
3	1	0	0	1	1	0
4	0	0	0	0	1	1
5	0	1	0	0	0	1
6	1	0	0	0	0	0

Answer: _____ (enter a numerical value)

Q.11 [NAT · numerical answer | 2 marks | Matrix powers — counting walks]

Let A be the adjacency matrix of the directed graph shown (order P, Q, R, S, T). The entry of A^3 in row P, column T — the number of directed walks of length exactly 3 from P to T — is ____.



Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Adjacency lists with head insertion + DFS]

Adjacency lists are built by inserting each new neighbour at the **front** of the list, and then a recursive DFS is run from vertex 1. What is printed?

Python

```

1 edges = [(1, 2), (1, 3), (2, 4), (3, 4),
2           (1, 5), (4, 6), (5, 6), (2, 5)]
3 adj = {v: [] for v in range(1, 7)}
4 for u, v in edges:
5     adj[u].insert(0, v)
6     adj[v].insert(0, u)
7
8 order = []
9 def dfs(u):
10     order.append(u)
11     for w in adj[u]:
12         if w not in order:
13             dfs(w)
14
15 dfs(1)
16 print(order)

```

- (A) [1, 2, 4, 3, 6, 5]
- (B) [1, 5, 2, 4, 6, 3]
- (C) [1, 5, 6, 4, 3, 2]
- (D) [1, 3, 4, 6, 5, 2]

Q.13 [MSQ · one or more correct | 2 marks | Matrix vs list — space and time]

A directed graph has $n = 2000$ vertices and $m = 10\,000$ edges. Representation X is an adjacency matrix with **1 bit** per entry. Representation Y is an array of n list-head pointers (32 bits each) plus one list node per edge (32-bit vertex id + 32-bit next pointer). Which of the following statements is/are TRUE?

- (A) Y uses fewer bits than X
- (B) Testing whether the edge (u, v) exists takes $O(1)$ time in X but may take $\Theta(\text{out-deg}(u))$ time in Y
- (C) Computing the transpose (reverse) graph takes $\Theta(n + m)$ time in X
- (D) Computing the in-degrees of all vertices takes $\Theta(n + m)$ time in Y

Q.14 [NAT · numerical answer | 2 marks | Triangles from the adjacency matrix]

The adjacency matrix of a simple undirected graph H on vertices 1..6 is shown. The value of $\text{trace}(A^3)$ is _____.

Adjacency matrix A of H						
	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	1	0	1	0
3	1	1	0	1	1	1
4	1	0	1	0	0	1
5	0	1	1	0	0	1
6	0	0	1	1	1	0

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Reverse graph with defaultdict]

Consider the following Python program, which builds the reverse of a directed graph. What is printed?

Python

```
1 from collections import defaultdict
2 G = {'a': ['b', 'c'], 'b': ['c'], 'c': ['a', 'd'], 'd': []}
3 R = defaultdict(list)
4 for u in G:
5     for v in G[u]:
6         R[v].append(u)
7 print(len(R), R['x'], len(R), R['c'])
```

- (A) 4 [] 4 ['a', 'b']
- (B) A **KeyError** is raised
- (C) 4 [] 5 ['a', 'b']
- (D) 3 [] 4 ['b', 'a']

Q.16 [NAT · numerical answer | 2 marks | Dijkstra on a weight matrix]

A weighted directed graph is given by the weight matrix W shown ($W[i][j]$ is the weight of edge $i \rightarrow j$; ∞ means no edge). Dijkstra's algorithm is run from S . The sum of the shortest-path distances from S to all five vertices (including S itself) is _____.

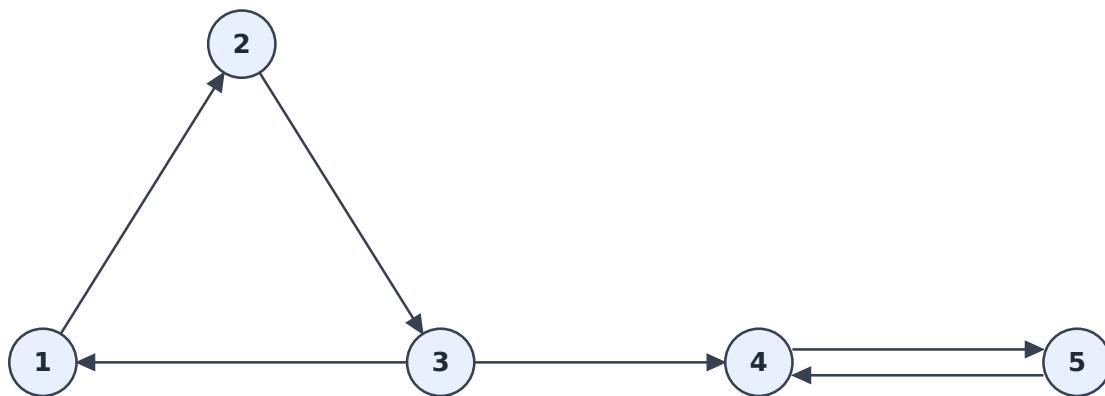
W (row = from, column = to)

	S	A	B	C	D
S	0	4	1	∞	∞
A	∞	0	∞	3	8
B	∞	2	0	6	9
C	∞	∞	∞	0	2
D	1	∞	∞	∞	0

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Reachability and walks in a digraph]

Let A be the adjacency matrix of the directed graph shown (vertices 1..5 in order). Which of the following statements is/are TRUE?



- (A) $(A^3)[1][1] = 1$
- (B) The graph is strongly connected
- (C) $(A^2)[3][5] = 1$
- (D) The sum of all entries of A^2 equals $\sum_v \text{in-deg}(v) \cdot \text{out-deg}(v)$

Q.18 [NAT · numerical answer | 2 marks | Quicksort on equal keys]

Quicksort with Lomuto partitioning (pivot = last element; an element goes to the left part if $a[j] \leq \text{pivot}$) is run on an array of 8 **identical** keys, as in the program below. The value printed is _____.

Python

```

1 comps = 0
2 def partition(a, lo, hi):
3     global comps
4     p, i = a[hi], lo - 1
5     for j in range(lo, hi):
6         comps += 1
7         if a[j] <= p:
8             i += 1
9             a[i], a[j] = a[j], a[i]
10    a[i + 1], a[hi] = a[hi], a[i + 1]
11    return i + 1
12
13 def qsort(a, lo, hi):
14     if lo < hi:
15         q = partition(a, lo, hi)
16         qsort(a, lo, q - 1)
17         qsort(a, q + 1, hi)
18
19 a = [7] * 8
20 qsort(a, 0, 7)
21 print(comps)

```

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | BST reconstruction from post-order]

The post-order traversal of a binary search tree with distinct keys is

5, 11, 9, 17, 14, 8, 26, 23, 35, 31, 20.

Its pre-order traversal is

- (A) 20, 8, 5, 14, 9, 11, 17, 31, 23, 26, 35
- (B) 20, 8, 5, 14, 11, 9, 17, 31, 23, 26, 35
- (C) 20, 8, 5, 9, 11, 14, 17, 31, 26, 23, 35
- (D) 20, 8, 14, 5, 9, 11, 17, 23, 26, 31, 35

Q.20 [MSQ · one or more correct | 2 marks | Reading structure from A^2]

G is a simple undirected graph on 7 vertices whose adjacency matrix A satisfies $(A^2)[i][i] = 2$ for every vertex i . Which of the following statements is/are **necessarily** TRUE?

- (A) G is connected
- (B) G has exactly 7 edges
- (C) G contains a cycle of odd length
- (D) G is bipartite

Answer Key — Mock Test 27

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — aliasing when building adjacency lists
2	NAT	1	3	Adjacency matrix squared — common neighbours
3	MCQ	1	(C)	Graphical degree sequences
4	MSQ	1	(A), (B), (C)	Circular queue arithmetic
5	NAT	1	8	Hashing — quadratic probing
6	MCQ	1	(A)	Binary search — lower-bound loop
7	NAT	1	5	Trees — parent-array representation
8	MCQ	1	(D)	Insertion sort — shifts
9	MSQ	1	(B), (C), (D)	Adjacency matrix identities
10	NAT	1	3	Directed adjacency matrix — in/out degrees
11	NAT	2	3	Matrix powers — counting walks
12	MCQ	2	(B)	Adjacency lists with head insertion + DFS
13	MSQ	2	(A), (B), (D)	Matrix vs list — space and time
14	NAT	2	30	Triangles from the adjacency matrix
15	MCQ	2	(C)	Reverse graph with defaultdict
16	NAT	2	18	Dijkstra on a weight matrix
17	MSQ	2	(A), (C), (D)	Reachability and walks in a digraph
18	NAT	2	28	Quicksort on equal keys
19	MCQ	2	(A)	BST reconstruction from post-order
20	MSQ	2	(B), (C)	Reading structure from A^2

Detailed Solutions — Mock Test 27

Q.1 [MCQ · 1 mark · Python — aliasing when building adjacency lists]

Answer: (B)

`[[] ] * n` creates a list containing **n references to one and the same** empty list. So `adj[0]`, `adj[1]`, `adj[2]`, `adj[3]` are all aliases of a single list object.

Every `append` goes to that shared list, in program order:

- edge (0,1): `append 1`, `append 0` → `[1, 0]`
- edge (1,2): `append 2`, `append 1` → `[1, 0, 2, 1]`
- edge (2,3): `append 3`, `append 2` → `[1, 0, 2, 1, 3, 2]`

`len(adj[0]) = 6` and `adj[3][:3] = [1, 0, 2]`. Output: `6 [1, 0, 2]`.

- (A) is what the *intended* adjacency lists would give (`adj[0] = [1]`, `adj[3] = [2]`).
- (C) mixes up which endpoint is appended.
- (D) reverses the order of appends.

Fix: `adj = [[] for _ in range(n)]` evaluates `[]` *n* times, giving *n* distinct lists.

Trap: `*` on a list copies references, not objects — the same trap as `[[0] * n] * n` for an adjacency *matrix*.

Q.2 [NAT · 1 mark · Adjacency matrix squared — common neighbours]

Answer: 3

$(A^2)[u][v] = \sum_w A[u][w] \cdot A[w][v]$ counts the walks of length 2 from *u* to *v*, i.e. the number of **common neighbours** of *u* and *v* (for $u \neq v$ in a simple graph).

- $N(B) = \{A, C, D, F\}$
- $N(E) = \{A, C, F\}$
- $N(B) \cap N(E) = \{A, C, F\}$

So $(A^2)[B][E] = 3$: the walks B-A-E, B-C-E, B-F-E.

Note that B and E are *not* adjacent, yet the entry is non-zero — A^2 measures 2-step connectivity, not adjacency.

Tip: the diagonal entry $(A^2)[v][v]$ equals $\deg(v)$ — e.g. $(A^2)[B][B] = 4$.

Trap: counting the edge B-E itself (it does not exist) or counting D, which is adjacent to B but not to E.

Q.3 [MCQ · 1 mark · Graphical degree sequences]

Answer: (C)

Necessary conditions: the degree sum is even (handshake lemma) and no degree exceeds $n - 1$. Havel-Hakimi decides the rest: remove the largest degree *d* and subtract 1 from the next *d* largest; repeat. The sequence is graphical iff we reach all zeros.

- (A) Sum 20 (even). `5,5,4,3,2,1` → remove 5: `4,3,2,1,0` → remove 4: `2,1,0,-1` → negative → **not graphical**. (Two vertices of degree 5 among 6 vertices would both be adjacent to everyone, forcing every degree ≥ 2 , but one vertex has degree 1.)
- (B) A vertex of degree 6 needs 6 distinct neighbours but only 5 other vertices exist → **not graphical**.
- (C) Sum 18. `4,4,3,3,2,2` → `3,2,2,1,2` → sort `3,2,2,2,1` → `1,1,1,1` → `0,1,1` → `0,0` → **graphical**.
- (D) Sum 17 is odd → **not graphical**.

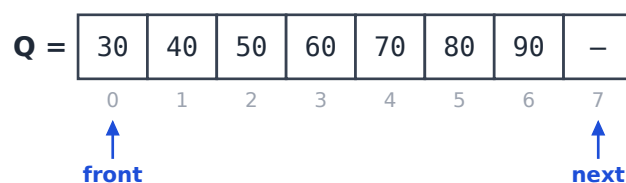
Trap: checking only the parity — (A) has an even sum but still fails.

Q.4 [MSQ · 1 mark · Circular queue arithmetic]**Answer: (A), (B), (C)**

Position of an enqueued item = $(\text{front} + \text{count}) \bmod 8$ at the moment of insertion.

- Enqueue 10, 20, 30, 40, 50 → indices 6, 7, 0, 1, 2; count = 5.
- Dequeue twice → returns 10 then 20; front = $(6 + 2) \bmod 8 = 0$; count = 3 (30, 40, 50).
- Enqueue 60, 70, 80, 90 → indices $(0 + 3) = 3, 4, 5, 6$; count = 7.
- (A) **True** — 90 is at index 6.
- (B) **True** — front = 0.
- (C) **True** — count = $7 < 8$, the next slot is $(0 + 7) \bmod 8 = 7$ (it still holds the stale value 20, which is simply overwritten).
- (D) **False** — $Q[\text{front}] = Q[0] = 30$ is returned next.

Trap: with a (front, count) design all 8 slots are usable; the “one slot wasted” rule applies only to the (front, rear) design that distinguishes full from empty by position.



Array after all operations (index 7 free)

Q.5 [NAT · 1 mark · Hashing — quadratic probing]**Answer: 8**

All five keys have the same home slot: 25, 36, 47, 14, 58 all leave remainder 3 mod 11. Quadratic probing visits 3, 3+1, 3+4, 3+9, 3+16, ... $(\bmod 11) = 3, 4, 7, 1, 8, \dots$

- 25: slot 3 ($i = 0$)
- 36: 3 full → slot 4 ($i = 1$)
- 47: 3, 4 full → slot 7 ($i = 2$)
- 14: 3, 4, 7 full → $(3 + 9) \bmod 11 = \text{slot } 1$ ($i = 3$)
- 58: 3, 4, 7, 1 full → $(3 + 16) \bmod 11 = 19 \bmod 11 = \text{slot } 8$ ($i = 4$)

Answer: **8**.

Keys with the same home slot follow the identical probe sequence — this is *secondary clustering*, which quadratic probing does not remove (double hashing does).

Trap: using $(h + i)$ or adding i^2 to the *previous* probe position instead of to the home slot.

0	
1	14
2	
3	25
4	36
5	
6	
7	47
8	58
9	
10	

Final table

Q.6 [MCQ · 1 mark · Binary search — lower-bound loop]**Answer: (A)**

This loop keeps the invariant “the answer lies in $[lo, hi]$ ” and returns the **first index whose value is $\geq x$** (a lower bound). It always halves the range, so it never stops early on a match.

x = 8 ($lo, hi = 0, 8$):

- mid 4: $15 \geq 8 \rightarrow hi = 4$
- mid 2: $8 \geq 8 \rightarrow hi = 2$
- mid 1: $8 \geq 8 \rightarrow hi = 1$
- mid 0: $3 < 8 \rightarrow lo = 1 \rightarrow$ stop. Result (1, 4).

x = 30:

- mid 4: $15 < 30 \rightarrow lo = 5$
- mid 6: $27 < 30 \rightarrow lo = 7$
- mid 7: $34 \geq 30 \rightarrow hi = 7 \rightarrow$ stop. Result (7, 3).

Output: (1, 4) (7, 3).

- (B) assumes the search stops at the first mid that matches (index 2).
- (C) miscounts steps and returns the last element < 30 .
- (D) returns the *last* occurrence of 8.

Trap: with $hi = len(a) - 1$ this function can never return $len(a)$; searching for 50 would wrongly return 8 — a classic off-by-one.

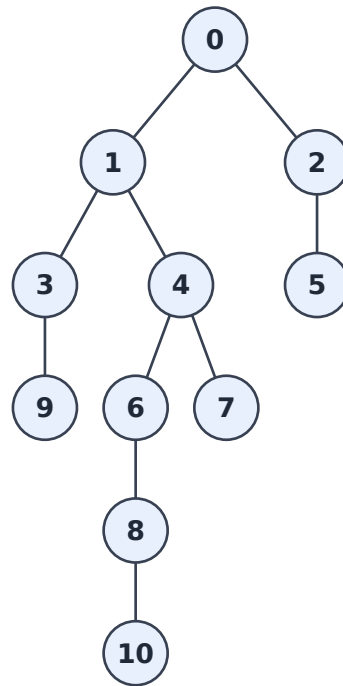
Q.7 [NAT · 1 mark · Trees — parent-array representation]**Answer: 5**

A parent array stores each edge once (child $\rightarrow$ parent); $depth(i) = 1 + depth(P[i])$ with $depth(root) = 0$. Compute depths in index order:

- node 0: root, depth 0
- nodes 1, 2: parent 0 $\rightarrow$ depth 1
- nodes 3, 4 (parent 1), 5 (parent 2) $\rightarrow$ depth 2
- nodes 6, 7 (parent 4), 9 (parent 3) $\rightarrow$ depth 3
- node 8 (parent 6) $\rightarrow$ depth 4
- node 10 (parent 8) $\rightarrow$ depth 5

Longest path $0 \rightarrow 1 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 10$ has **5** edges.

Tip: computing all depths naively costs $O(n \cdot \text{height})$; memoising depths makes it $O(n)$. **Trap:** counting nodes on the path (6) instead of edges.



Tree encoded by P

Q.8 [MCQ · 1 mark · Insertion sort — shifts]**Answer: (D)**

Each shift removes exactly one **inversion** (a pair $i < j$ with $A[i] > A[j]$), so the number of shifts equals the number of inversions.

Count, for every element, the larger elements to its **left**:

- 6: 0
- 2: 1 (6)
- 9: 0
- 4: 2 (6, 9)
- 1: 4 (6, 2, 9, 4)
- 8: 1 (9)
- 3: 4 (6, 9, 4, 8)

Total = $0 + 1 + 0 + 2 + 4 + 1 + 4 = 12$.

These are exactly the shifts made when inserting each element (e.g. inserting 1 shifts 9, 6, 4, 2).

- (A) 9 and (B) 11 miss some inversions (typically those of 3).
- (C) $21 = n(n - 1)/2$ is the worst case (reverse-sorted input).

Tip: comparisons = shifts + (number of insertions that stop on a comparison) — always between inversions and inversions + $n - 1$.

Q.9 [MSQ · 1 mark · Adjacency matrix identities]**Answer: (B), (C), (D)**

$(A^k)[i][j]$ counts walks of length k from i to j .

- (A) **False.** Each edge contributes two 1s ($A[u][v]$ and $A[v][u]$), so the sum is $2m$.
- (B) **True.** $(A^2)[i][i] = \text{number of closed walks of length 2 from } i = \text{deg}(i)$. Summing: $\sum \text{deg}(i) = 2m$ (handshake lemma).
- (C) **True.** A closed walk of length 3 is a triangle traversed from one of its 3 vertices in one of 2 directions; each triangle is counted $3 \times 2 = 6$ times.

- (D) **True.** Undirected $\rightarrow A[i][j] = A[j][i]$; simple (no self-loops) $\rightarrow$ zero diagonal.

Trap: for (C) dividing by 3 only (forgetting direction) or by 2 only. For a *directed* graph the analogues differ: the sum of entries is m and $\text{trace}(A^2)$ counts 2-cycles.

Q.10 [NAT · 1 mark · Directed adjacency matrix — in/out degrees]

Answer: 3

Out-degree of i = sum of **row** i ; in-degree of j = sum of **column** j .

- Row sums (out): 3, 2, 3, 2, 2, 1
- Column sums (in): 2, 2, 2, 2, 2, 3

They differ at vertex 1 (3 vs 2), vertex 3 (3 vs 2) and vertex 6 (1 vs 3) $\rightarrow$ **3** vertices.

Check: both sums total 13 = number of edges — in a directed graph $\sum \text{in} = \sum \text{out} = m$, so the surpluses (+1, +1) and the deficit (−2) must cancel.

Trap: reading rows as in-degrees. With an adjacency *list* representation the out-degree is the list length ($O(1)$ if stored), but every in-degree needs a full $\Theta(n + m)$ scan.

Q.11 [NAT · 2 marks · Matrix powers — counting walks]

Answer: 3

$(A^3)[P][T] = \sum_{x,y} A[P][x] \cdot A[x][y] \cdot A[y][T]$ = number of walks $P \rightarrow x \rightarrow y \rightarrow T$.

Out-neighbours: P: {Q, R}; Q: {R, S}; R: {S, T}; S: {T, Q}; T: {P, S}. In-neighbours of T: {R, S}.

Enumerate $x \in \text{out}(P)$, $y \in \text{out}(x)$ with $y \rightarrow T$:

- $x = Q$: $y = R$ ($R \rightarrow T \checkmark$) $\rightarrow$ P-Q-R-T; $y = S$ ($S \rightarrow T \checkmark$) $\rightarrow$ P-Q-S-T.
- $x = R$: $y = S$ ($S \rightarrow T \checkmark$) $\rightarrow$ P-R-S-T; $y = T$ ($T \rightarrow T?$ no self-loop) $\times$.

Total = **3** walks.

Equivalently, row P of A^2 is (P:0, Q:0, R:1, S:2, T:1) and multiplying by column T of A (1 for R and S) gives $1 + 2 = 3$.

Trap: counting *paths* vs *walks* does not matter here (all three walks have distinct vertices), but in general A^k counts walks, which may repeat vertices and edges.

Row P of A, A ² , A ³					
	P	Q	R	S	T
A	0	1	1	0	0
A ²	0	0	1	2	1
A ³	1	2	0	2	3

Q.12 [MCQ · 2 marks · Adjacency lists with head insertion + DFS]

Answer: (B)

The DFS order depends on the **representation**: with front insertion each list holds the neighbours in *reverse* order of edge insertion.

Lists: 1: [5, 3, 2]; 2: [5, 4, 1]; 3: [4, 1]; 4: [6, 3, 2]; 5: [2, 6, 1]; 6: [5, 4].

DFS trace:

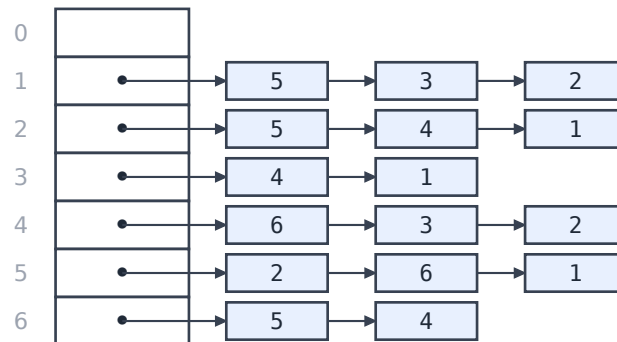
- visit 1 $\rightarrow$ first neighbour 5
- visit 5 $\rightarrow$ first neighbour 2
- visit 2 $\rightarrow$ 5 seen $\rightarrow$ 4
- visit 4 $\rightarrow$ 6
- visit 6 $\rightarrow$ 5, 4 seen $\rightarrow$ return to 4 $\rightarrow$ next 3

- visit 3 → 4, 1 seen → return ... all remaining neighbours seen.

Order: [1, 5, 2, 4, 6, 3].

- (A) is the order with *append* (lists in insertion order: 1: [2, 3, 5], ...).
- (C) goes 5 → 6 (but 2 precedes 6 in 5's list).
- (D) assumes 1's list starts with 3.

Note: *w not in order* is an $O(n)$ list scan, making this DFS $O(n \cdot (n + m))$; a *set* of visited vertices restores $O(n + m)$.



Adjacency lists (row = vertex; row 0 unused)

Q.13 [MSQ · 2 marks · Matrix vs list — space and time]

Answer: (A), (B), (D)

Space. X: $n^2 = 4\,000\,000$ bits. Y: $32n + 64m = 64\,000 + 640\,000 = 704\,000$ bits. So Y is about 5.7 times smaller → (A) **True**. (The break-even is $32n + 64m = n^2$, i.e. $m \approx 62\,000$ edges; denser graphs favour the bit-matrix.)

Edge test. X: read one bit $M[u][v] \rightarrow O(1)$. Y: scan u's list $\rightarrow \Theta(\text{out-deg}(u))$ in the worst case → (B) **True**.

All in-degrees in Y. One pass over all n lists and m nodes, incrementing $\text{in}[v]$ for each node $\rightarrow \Theta(n + m) \rightarrow$ (D) **True**.

Transpose in X. Every one of the n^2 entries must be read/written ($M^T[i][j] = M[j][i] \rightarrow \Theta(n^2)$, not $\Theta(n + m) \rightarrow$ (C) **False**. (In Y, transposing takes $\Theta(n + m)$.)

Trap: assuming the matrix is always larger. With 1-bit entries and sparse lists the comparison depends on m relative to $n^2/64$.

Q.14 [NAT · 2 marks · Triangles from the adjacency matrix]

Answer: 30

$\text{trace}(A^3) = \sum_i (A^3)[i][i]$ counts closed walks of length 3; in a simple graph each one is a triangle, and each triangle is counted 6 times (3 starting vertices $\times$ 2 directions). So $\text{trace}(A^3) = 6 \times (\text{number of triangles})$.

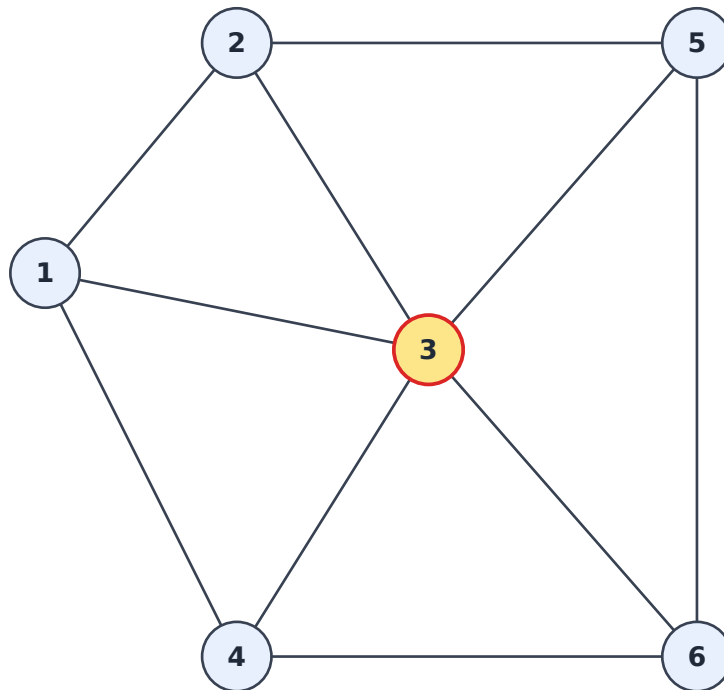
Edges read from the upper triangle: 1-2, 1-3, 1-4, 2-3, 2-5, 3-4, 3-5, 3-6, 4-6, 5-6 (10 edges).

Triangles (check every pair of neighbours for adjacency):

- {1, 2, 3}: 1-2, 2-3, 1-3 ✓
- {1, 3, 4}: ✓
- {2, 3, 5}: ✓
- {3, 4, 6}: ✓
- {3, 5, 6}: ✓
- others such as {1, 2, 5} (1-5 missing) or {4, 5, 6} (4-5 missing) fail.

5 triangles $\rightarrow \text{trace}(A^3) = 6 \times 5 = \mathbf{30}$. Note that vertex 3 lies in all five triangles ($(A^3)[3][3] = 10$).

Trap: answering 5 (the triangle count) or 15 (dividing by 2 or by 3 only).



H drawn from the matrix

Q.15 [MCQ · 2 marks · Reverse graph with defaultdict]**Answer: (C)**

The reverse graph has an edge $v \rightarrow u$ for every edge $u \rightarrow v$. `defaultdict(list)` creates an empty list the **first time a missing key is accessed** — even just for reading.

Building R (in the iteration order of G, which is insertion order):

- $u = a$: $R[b] = [a]$, $R[c] = [a]$
- $u = b$: $R[c] = [a, b]$
- $u = c$: $R[a] = [c]$, $R[d] = [c]$
- $u = d$: no edges

R has keys b, c, a, d $\rightarrow \text{len}(R) = 4$. (Vertex d appears as a key because it has an incoming edge.)

Arguments are evaluated left to right: $R['x']$ inserts key 'x' with value [] and returns [], so the next $\text{len}(R)$ is **5**. $R['c'] = ['a', 'b']$.

Output: 4 [] 5 ['a', 'b'].

- (A) assumes reading a missing key does not insert it.
- (B) is the behaviour of a plain `dict`.
- (D) forgets that d gets a key, and reverses append order.

Trap: probing a defaultdict silently adds vertices — use `'x' in R` or `R.get('x')` to test membership.

Q.16 [NAT · 2 marks · Dijkstra on a weight matrix]**Answer: 18**

With a matrix, Dijkstra scans a whole row to relax the extracted vertex's edges ($\Theta(V^2)$ overall with linear-scan extraction).

- Init: S 0, others ∞ .
- Extract S (0): A = 4, B = 1.
- Extract B (1): A = $\min(4, 1 + 2) = 3$; C = $1 + 6 = 7$; D = $1 + 9 = 10$.

- Extract A (3): $C = \min(7, 3 + 3) = 6$; $D = \min(10, 3 + 8) = 10$ (unchanged).
- Extract C (6): $D = \min(10, 6 + 2) = 8$.
- Extract D (8): edge $D \rightarrow S$ does not improve S.

Distances: S 0, A 3, B 1, C 6, D 8. Sum = $0 + 3 + 1 + 6 + 8 = 18$.

Trap: reading the matrix column-wise (treating $W[i][j]$ as $j \rightarrow i$) gives a different graph; and the back edge $D \rightarrow S$ is irrelevant for distances from S.

dist[] after each extraction

	S	A	B	C	D
init	0	∞	∞	∞	∞
S	0	4	1	∞	∞
B	0	3	1	7	10
A	0	3	1	6	10
C	0	3	1	6	8
D	0	3	1	6	8

Q.17 [MSQ · 2 marks · Reachability and walks in a digraph]

Answer: (A), (C), (D)

- (A) **True.** Closed walks of length 3 from 1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ is the only one (1 has the single out-edge to 2, 2 only to 3, and $3 \rightarrow 1$ closes it).
- (B) **False.** From 4 or 5 the only edges are $4 \leftrightarrow 5$, so vertices 1, 2, 3 are unreachable from 4. The strongly connected components are $\{1, 2, 3\}$ and $\{4, 5\}$.
- (C) **True.** Walks of length 2 from 3 to 5 must be $3 \rightarrow x \rightarrow 5$ with $x \in \text{out}(3) = \{1, 4\}$; only $x = 4$ has $4 \rightarrow 5$. Exactly one walk.
- (D) **True.** The sum of all entries of A^2 counts all length-2 walks $u \rightarrow v \rightarrow w$. Grouping by the middle vertex v gives $\text{in-deg}(v)$ choices for u and $\text{out-deg}(v)$ choices for w . Here: $v=1$: $1 \cdot 1$, $v=2$: $1 \cdot 1$, $v=3$: $1 \cdot 2$, $v=4$: $2 \cdot 1$, $v=5$: $1 \cdot 1 \rightarrow$ total 7.

Trap: in (B), a graph can contain a cycle through every vertex of one part yet fail to be strongly connected — the edge $3 \rightarrow 4$ has no return path.

Q.18 [NAT · 2 marks · Quicksort on equal keys]

Answer: 28

With $\leq$, every element equals the pivot and goes to the left part, so the pivot ends at the **last** position: the split is $(n - 1, 0)$ — the worst case, exactly as for sorted input.

- Range size 8: 7 comparisons, pivot at index 7 $\rightarrow$ recurse on size 7 (and an empty right part)
- size 7: 6 comparisons $\rightarrow$ size 6
- ... $\rightarrow$ size 2: 1 comparison $\rightarrow$ size 1 (no call work)

Total = $7 + 6 + 5 + 4 + 3 + 2 + 1 = 8 \cdot 7 / 2 = 28$ comparisons.

In general Lomuto quicksort costs $\Theta(n^2)$ on all-equal input, with recurrence $T(n) = T(n - 1) + (n - 1)$.

Trap: expecting equal keys to be an easy/best case. Hoare partitioning or 3-way (Dutch-flag) partitioning splits equal keys evenly or groups them, restoring $O(n \log n)$ or $O(n)$.

Q.19 [MCQ · 2 marks · BST reconstruction from post-order]**Answer: (A)**

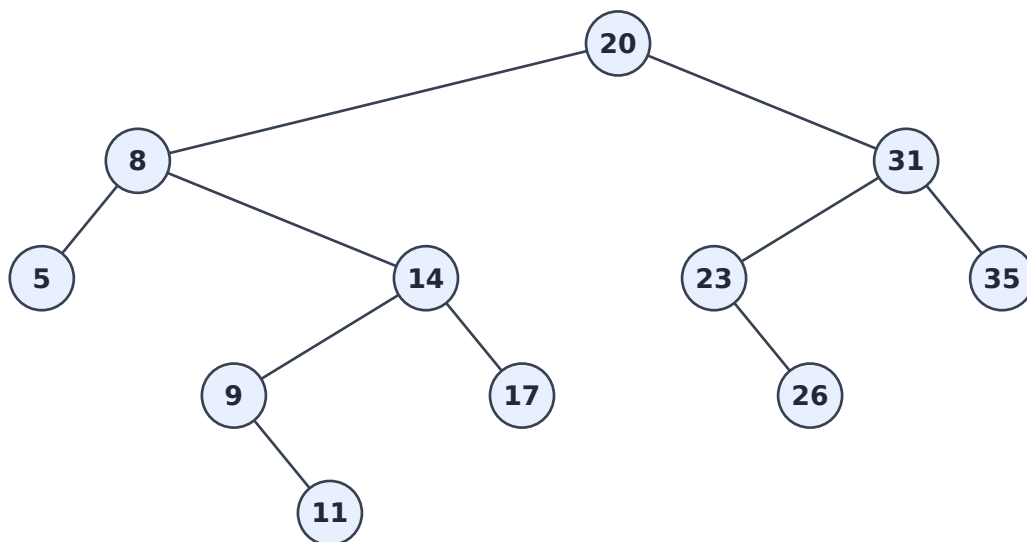
In post-order the **last** key is the root; the keys smaller than the root form the left subtree's post-order and the larger ones the right subtree's. Recurse.

- Root 20. Left: 5, 11, 9, 17, 14, 8 → root 8. Right: 26, 23, 35, 31 → root 31.
- Under 8: left {5} → 5; right 11, 9, 17, 14 → root 14.
- Under 14: left 11, 9 → root 9 with right child 11; right {17}.
- Under 31: left 26, 23 → root 23 with right child 26; right {35}.

Pre-order (root, left, right): 20, 8, 5, 14, 9, 11, 17, 31, 23, 26, 35.

- (A) Correct.
- (B) Makes 11 the parent of 9 — but 9 comes *after* 11 in post-order, so 9 is the ancestor.
- (C) Lists the left subtree of 8 in sorted (in-order) order, and puts 26 above 23.
- (D) Is not even consistent with the BST: 14 precedes 5 although $5 < 8 < 14$.

Tip: for a BST, post-order (or pre-order) alone determines the tree, because the in-order is just the sorted keys.



Reconstructed BST

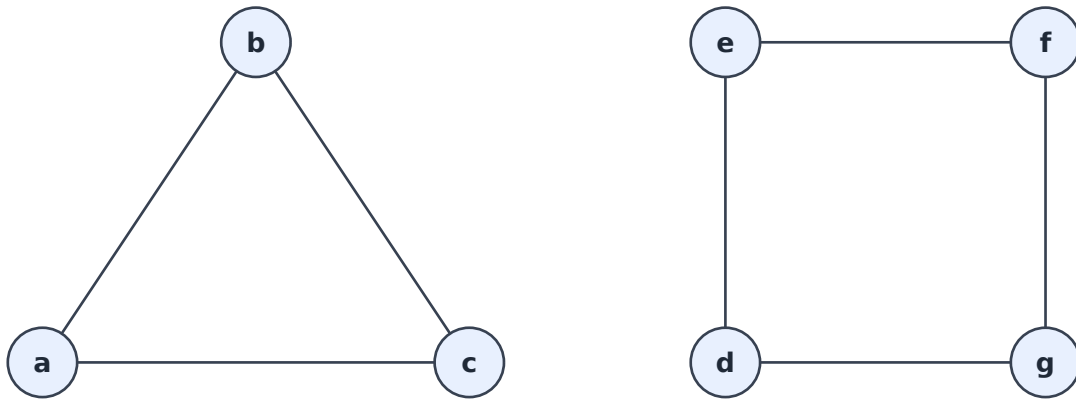
Q.20 [MSQ · 2 marks · Reading structure from A^2]**Answer: (B), (C)**

$(A^2)[i][i] = \deg(i)$, so G is **2-regular**. A 2-regular graph is a disjoint union of cycles, each of length ≥ 3 , whose lengths add up to 7.

Possible cycle structures: a single C_7 , or $C_3 \cup C_4$ (lengths ≥ 3 summing to 7).

- (A) **False.** $C_3 \cup C_4$ is 2-regular on 7 vertices but disconnected.
- (B) **True.** $\sum \deg = 7 \times 2 = 14 = 2m \rightarrow m = 7$ (also: a cycle on k vertices has k edges).
- (C) **True.** The cycle lengths sum to 7 (odd), so at least one cycle has odd length (C_7 itself, or the C_3).
- (D) **False.** A graph with an odd cycle is never bipartite — in fact (C) shows G is *never* bipartite.

Trap: assuming “2-regular” means “a single cycle”. Only connectivity would force C_7 .



$C_3 \cup C_4$: 2-regular on 7 vertices but disconnected

Mock Test 28

Connectivity & Bipartiteness

Difficulty: GATE-level

Focus areas: components, bridges/cut vertices intuition, bipartite checks, cycles

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [NAT · numerical answer | 1 mark | Graph theory — connected components]

A simple undirected graph G has vertex set $\{0, 1, 2, \dots, 29\}$. Two distinct vertices i and j are adjacent if and only if $|i - j| = 12$ or $|i - j| = 18$. The number of connected components of G is _____.

Answer: _____ (enter a numerical value)

Q.2 [MCQ · single correct | 1 mark | Union-find with path halving]

Consider the following Python program. What is printed?

Python

```

1 parent = list(range(8))
2
3 def find(x):
4     while parent[x] != x:
5         parent[x] = parent[parent[x]]
6         x = parent[x]
7     return x
8
9 def union(a, b):
10    ra, rb = find(a), find(b)
11    if ra != rb:
12        parent[ra] = rb
13
14 for a, b in [(0, 1), (2, 3), (1, 3), (4, 5), (6, 5), (0, 2)]:
15     union(a, b)
16 print(parent[:4], len({find(i) for i in range(8)}))

```

- (A) [3, 3, 3, 3] 3
- (B) [1, 3, 3, 3] 3
- (C) [3, 3, 3, 3] 4
- (D) [1, 3, 3, 3] 4

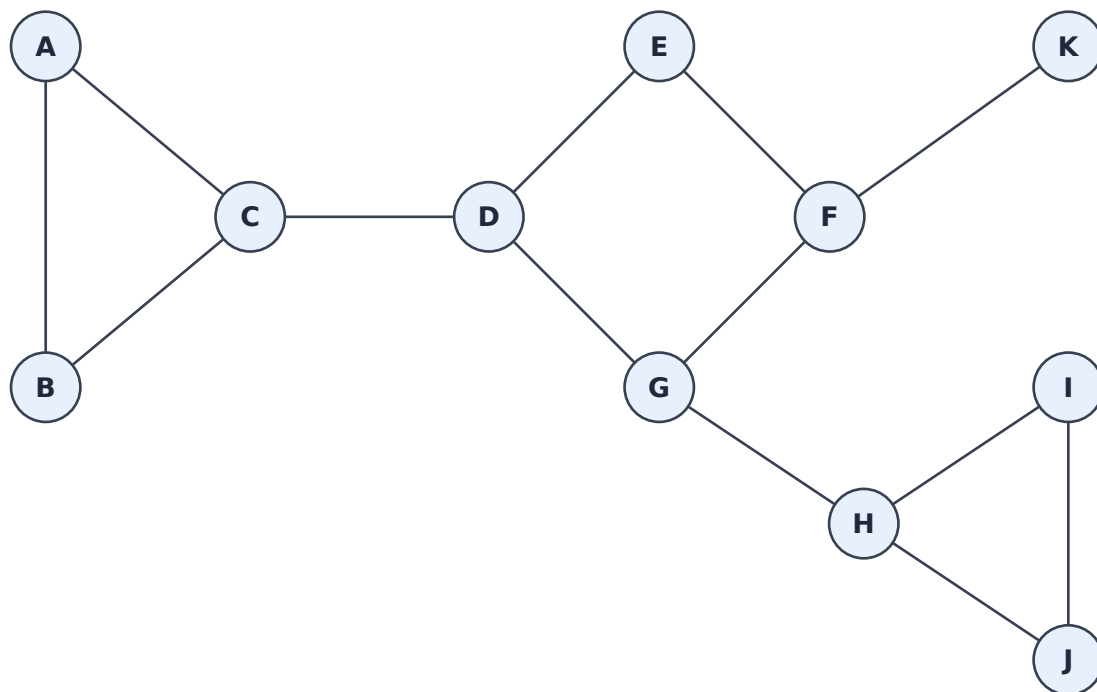
Q.3 [MSQ · one or more correct | 1 mark | Bipartite graphs — parity argument]

For a set D of positive integers, let G_D be the graph on vertices $\{1, 2, \dots, 12\}$ in which distinct i and j are adjacent iff $|i - j| \in D$. For which of the following choices of D is G_D bipartite?

- (A) $D = \{2, 4\}$
- (B) $D = \{1, 3\}$
- (C) $D = \{2, 3\}$
- (D) $D = \{3, 5\}$

Q.4 [NAT · numerical answer | 1 mark | Connectivity — bridges]

A *bridge* is an edge whose removal increases the number of connected components. The number of bridges in the undirected graph shown is _____.



Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Stacks — permutations]

The integers 1, 2, 3, 4, 5, 6 are pushed onto an initially empty stack in this order; pops may be interleaved arbitrarily with the pushes and each popped value is printed. Which of the following output sequences is **NOT** possible?

- (A) 3 2 4 1 6 5
- (B) 2 4 3 6 5 1
- (C) 4 3 5 1 2 6
- (D) 1 2 3 4 5 6

Q.6 [NAT · numerical answer | 1 mark | BST — number of insertion orders]

The binary search tree shown is obtained by inserting its 8 keys, one at a time, into an empty BST. The number of different insertion orders (permutations of the 8 keys) that produce exactly this tree is _____.

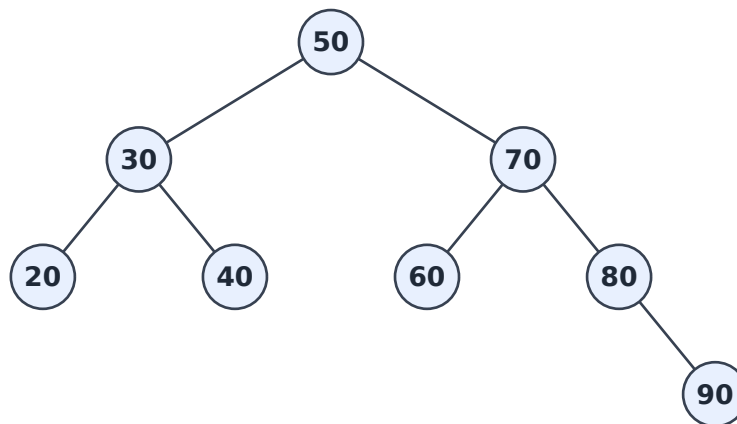


Figure: target BST

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Hashing — chaining]

The keys 5, 14, 22, 9, 31, 40, 17, 2 are inserted into a hash table with 9 slots (0-8) using $h(k) = (2k + 3) \bmod 9$ and separate chaining. What are the length of the longest chain and the number of empty slots, respectively?

- (A) 2 and 4
- (B) 3 and 4
- (C) 3 and 3
- (D) 3 and 5

Q.8 [MCQ · single correct | 1 mark | Bubble sort — passes]

Bubble sort (ascending) makes left-to-right passes; in each pass adjacent elements $a[j]$, $a[j+1]$ are swapped if $a[j] > a[j+1]$, and after pass p the last p positions are not examined again. Starting from [26, 11, 39, 4, 18, 33, 7], what is the array after **two** complete passes?

- (A) [11, 26, 4, 18, 33, 7, 39]
- (B) [11, 4, 18, 26, 7, 33, 39]
- (C) [4, 7, 11, 18, 26, 33, 39]
- (D) [11, 4, 18, 7, 26, 33, 39]

Q.9 [MCQ · single correct | 1 mark | Python — aliasing in adjacency lists]

A student builds the adjacency lists of the path graph 0 - 1 - 2 - 3 as follows. What is printed?

Python

```

1 adj = [[]] * 4
2 for u, v in [(0, 1), (1, 2), (2, 3)]:
3     adj[u].append(v)
4     adj[v].append(u)
5 print(len(adj[0]), adj[3][:3])
  
```

- (A) 1 [2]
- (B) 2 [2]
- (C) 6 [1, 0, 2]
- (D) 6 [2]

Q.10 [NAT · numerical answer | 1 mark | Binary search — lower bound]

Consider the following Python program. The value printed is ____.

Python

```

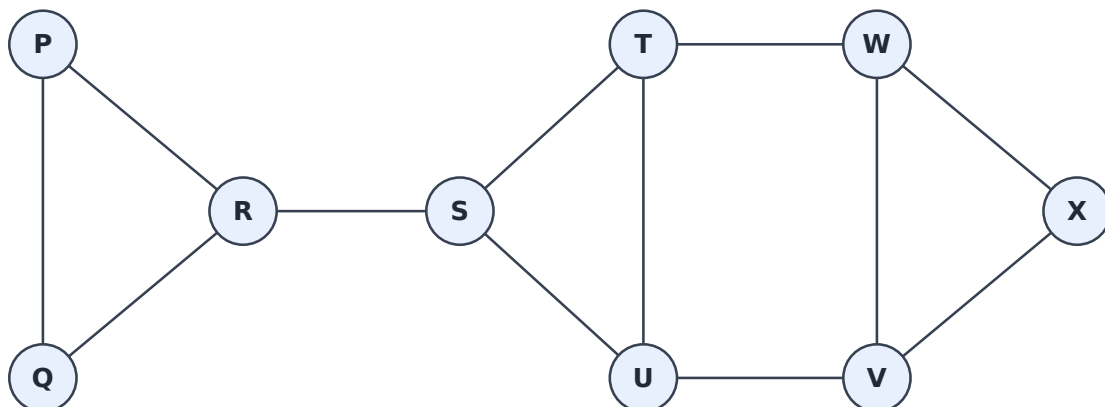
1 def lb(a, x):
2     lo, hi = 0, len(a)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 a = [2, 4, 4, 4, 7, 9, 9, 12, 15]
12 print(lb(a, 4) + lb(a, 10) + lb(a, 1))

```

Answer: _____ (enter a numerical value)

Q.11 [MSQ · one or more correct | 2 marks | Connectivity — cut vertices and bridges]

Consider the undirected graph shown. A *cut vertex* is a vertex whose removal (with its incident edges) increases the number of connected components; a *bridge* is an edge whose removal does so. Which of the following statements is/are TRUE?



- (A) The edge U-V is a bridge
- (B) R is a cut vertex
- (C) S is a cut vertex
- (D) U is a cut vertex

Q.12 [NAT · numerical answer | 2 marks | Graph theory — edges vs components]

The maximum number of edges in a simple undirected graph with 12 vertices and **exactly** 4 connected components is ____.

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Bipartiteness — BFS two-colouring]

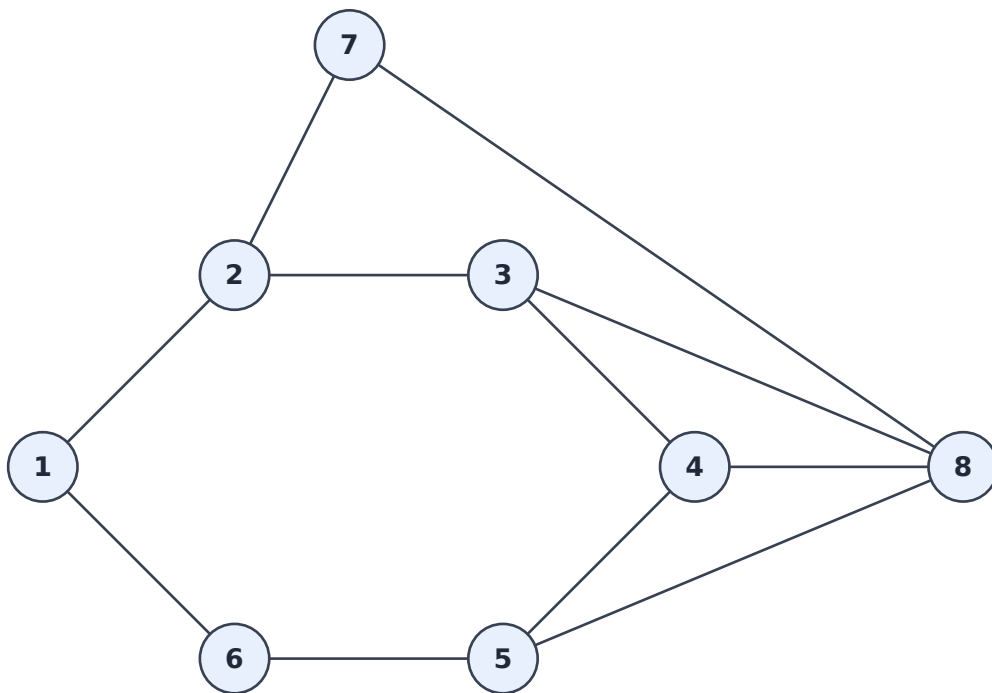
The function below tries to 2-colour the graph G shown, starting at vertex 1. Adjacency lists are sorted in increasing order, so `adj[u]` is visited in increasing vertex order. Which of the following statements is/are TRUE?

Python

```

1 from collections import deque
2
3 def two_colour(adj, s):
4     col = {s: 0}
5     q = deque([s])
6     while q:
7         u = q.popleft()
8         for v in adj[u]:
9             if v not in col:
10                col[v] = 1 - col[u]
11                q.append(v)
12            elif col[v] == col[u]:
13                return None, (u, v)
14     return col, None

```



- (A) The conflicting edge it reports is (4, 8)
- (B) Deleting only the edge 4-8 makes G bipartite
- (C) Deleting only the edge 3-8 makes G bipartite
- (D) `two_colour(adj, 1)` returns `None` as its first component

Q.14 [NAT · numerical answer | 2 marks | Connected components — iterative DFS in Python]

Consider the following Python program. The value printed is _____.

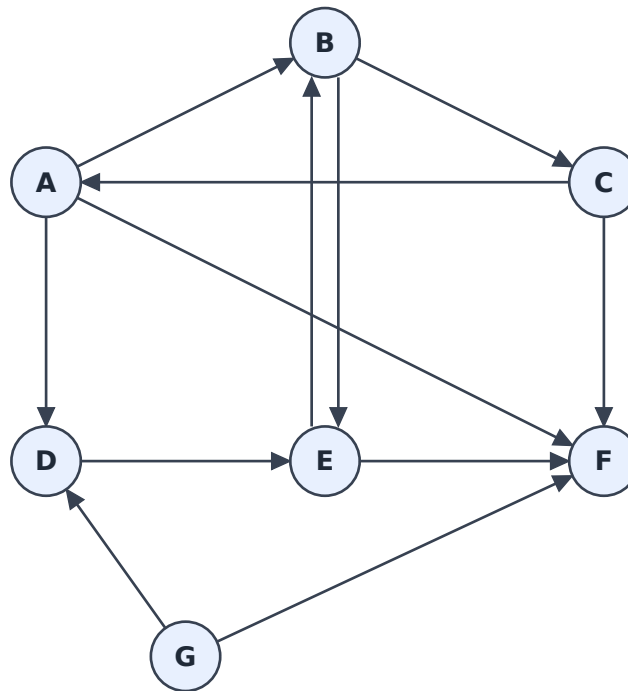
Python

```
1 def pairs(n, edges):
2     adj = [[] for _ in range(n)]
3     for u, v in edges:
4         adj[u].append(v)
5         adj[v].append(u)
6     seen = [False] * n
7     total = done = 0
8     for s in range(n):
9         if seen[s]:
10            continue
11        size, st = 0, [s]
12        seen[s] = True
13        while st:
14            u = st.pop()
15            size += 1
16            for v in adj[u]:
17                if not seen[v]:
18                    seen[v] = True
19                    st.append(v)
20        total += size * done
21        done += size
22    return total
23
24 E = [(0, 5), (5, 9), (1, 4), (4, 11), (11, 1),
25      (2, 7), (3, 8), (8, 10), (6, 6)]
26 print(pairs(12, E))
```

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | DFS — edge classification]

Depth-first search is run on the directed graph shown. The outer loop considers start vertices in alphabetical order, and each adjacency list is explored in alphabetical order. Let T, B, F, C be the numbers of tree, back, forward and cross edges respectively. What is (T, B, F, C)?



- (A) (5, 3, 1, 3)
- (B) (5, 2, 0, 5)
- (C) (6, 2, 1, 3)
- (D) (5, 2, 1, 4)

Q.16 [MSQ · one or more correct | 2 marks | Sorting — properties and counts]

Which of the following statements is/are TRUE?

- (A) Selection sort (swap $a[i]$ with the minimum of $a[i..n-1]$) is stable
- (B) Quicksort with the Lomuto partition (pivot = last element, test $a[j] \leq \text{pivot}$) makes exactly 15 element comparisons on the array $[7, 7, 7, 7, 7, 7, 7]$
- (C) Insertion sort makes exactly 28 element comparisons on an array of 8 distinct elements in strictly decreasing order
- (D) Merge sort whose merge takes the element from the **left** run when the two front elements are equal is a stable sort

Q.17 [NAT · numerical answer | 2 marks | Linked lists — Floyd's cycle detection]

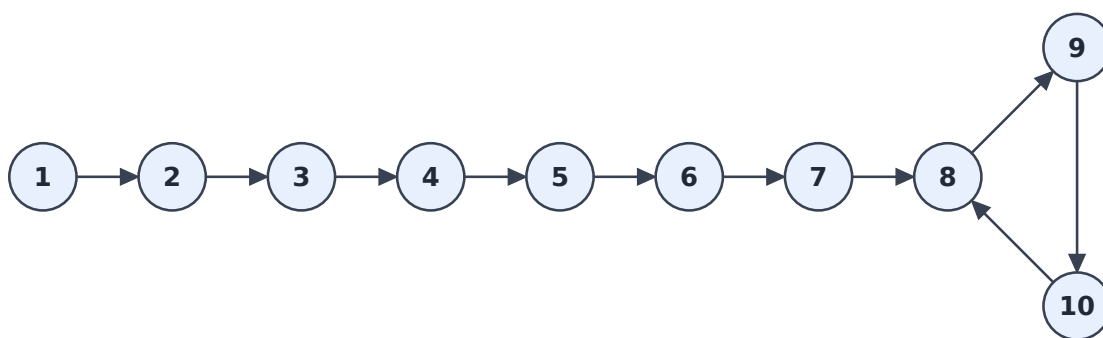
The following program builds the singly linked list shown (node 10 points back to node 8) and runs the meeting phase of Floyd's tortoise-and-hare algorithm. The value printed is _____.

Python

```

1 class Node:
2     def __init__(self, v):
3         self.v, self.nxt = v, None
4
5 nodes = [Node(i) for i in range(1, 11)]
6 for a, b in zip(nodes, nodes[1:]):
7     a.nxt = b
8 nodes[-1].nxt = nodes[7]
9
10 slow = fast = nodes[0]
11 steps = 0
12 while True:
13     slow = slow.nxt
14     fast = fast.nxt.nxt
15     steps += 1
16     if slow is fast:
17         break
18 print(steps)

```



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Heaps — bottom-up build-heap]

The array [35, 12, 47, 8, 29, 3, 61, 17, 5] is converted into a binary **min**-heap (0-based; children of i are $2i+1$ and $2i+2$) using the bottom-up method: for $i = \lfloor n/2 \rfloor - 1$ down to 0, sift $a[i]$ down, always swapping with the smaller child while that child is smaller. What is the resulting array?

- (A) [3, 5, 35, 8, 29, 47, 61, 17, 12]
- (B) [3, 5, 8, 12, 29, 47, 61, 35, 17]
- (C) [3, 5, 35, 12, 29, 47, 61, 17, 8]
- (D) [3, 8, 5, 12, 29, 47, 61, 17, 35]

Q.19 [MCQ · single correct | 2 marks | Hashing — unsuccessful search with linear probing]

A hash table of size 10 with linear probing ($h(k) = k \bmod 10$, step +1 with wrap-around) currently holds the keys shown. An unsuccessful search starts at the home slot of the sought key and stops at the first empty slot; every slot inspected, including that empty one, counts as a probe. If the home slot is equally likely to be any of the 10 slots, the expected number of probes for an unsuccessful search is:

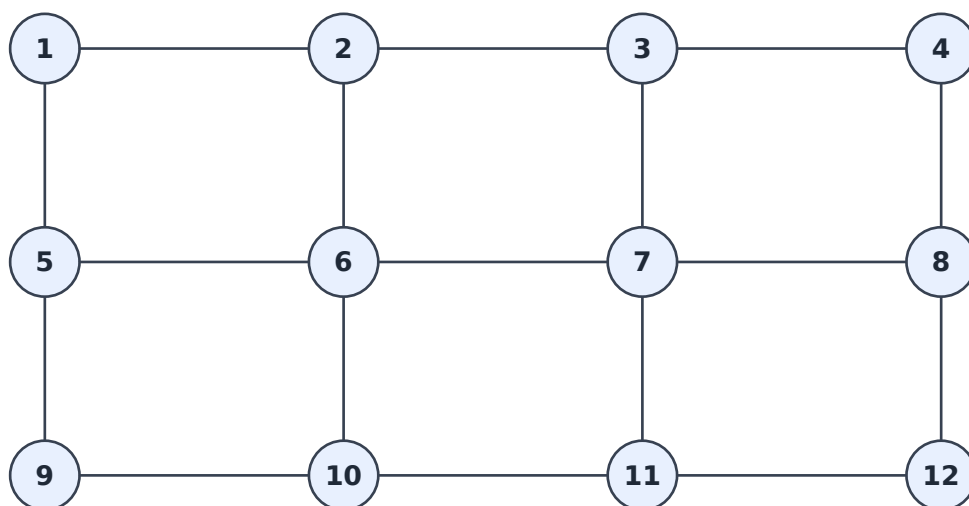
0	39
1	21
2	
3	
4	64
5	44
6	
7	
8	18
9	28

Figure: current table

- (A) 2.5
- (B) 1.7
- (C) 1.6
- (D) 2.3

Q.20 [MSQ · one or more correct | 2 marks | Bipartiteness and connectivity of a grid graph]

G is the 3×4 grid graph shown (vertex labels 1–12 row by row; edges join horizontally or vertically adjacent vertices). Which of the following statements is/are TRUE?



- (A) G has at least one cut vertex
- (B) G is bipartite and both colour classes have 6 vertices
- (C) Deleting the two vertices 2 and 5 disconnects G
- (D) Adding the single edge 1-6 makes G non-bipartite

Answer Key — Mock Test 28

Q	Type	Marks	Key	Topic
1	NAT	1	6	Graph theory — connected components
2	MCQ	1	(A)	Union-find with path halving
3	MSQ	1	(B), (D)	Bipartite graphs — parity argument
4	NAT	1	3	Connectivity — bridges
5	MCQ	1	(C)	Stacks — permutations
6	NAT	1	210	BST — number of insertion orders
7	MCQ	1	(B)	Hashing — chaining
8	MCQ	1	(B)	Bubble sort — passes
9	MCQ	1	(C)	Python — aliasing in adjacency lists
10	NAT	1	8	Binary search — lower bound
11	MSQ	2	(B), (C)	Connectivity — cut vertices and bridges
12	NAT	2	36	Graph theory — edges vs components
13	MSQ	2	(A), (B), (D)	Bipartiteness — BFS two-colouring
14	NAT	2	56	Connected components — iterative DFS in Python
15	MCQ	2	(D)	DFS — edge classification
16	MSQ	2	(B), (C), (D)	Sorting — properties and counts
17	NAT	2	9	Linked lists — Floyd's cycle detection
18	MCQ	2	(A)	Heaps — bottom-up build-heap
19	MCQ	2	(D)	Hashing — unsuccessful search with linear probing
20	MSQ	2	(B), (C), (D)	Bipartiteness and connectivity of a grid graph

Detailed Solutions — Mock Test 28

Q.1 [NAT · 1 mark · Graph theory — connected components]

Answer: 6

Every edge changes a vertex label by 12 or 18, both multiples of $\gcd(12, 18) = 6$. So $i \bmod 6$ is **invariant** along any path: vertices with different residues mod 6 can never be connected $\rightarrow$ at least 6 components.

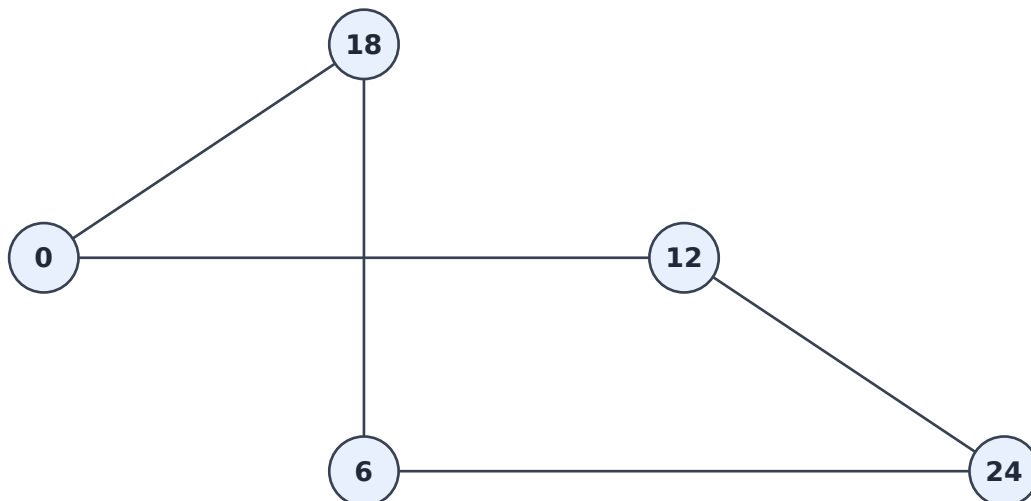
Now check that each residue class is connected. Class $r = \{r, r+6, r+12, r+18, r+24\}$ ($r = 0..5$, five vertices each):

- $r \rightarrow r+12 \rightarrow r+24$ (steps of 12)
- $r \rightarrow r+18$ (step 18) and $r+6 \rightarrow r+18$ (step 12), $r+6 \rightarrow r+24$ (step 18)

So $r, r+6, r+12, r+18, r+24$ are all linked: e.g. $r \rightarrow r+18 \rightarrow r+6 \rightarrow r+24 \rightarrow r+12$.

Hence exactly **6** components, each with 5 vertices.

Trap: answering gcd-based counts blindly. On a *finite path* of labels (not a cycle) a class could split if the steps overshoot the range — here 30 labels are enough for every class to stay connected, but always verify one class explicitly.



Component of residue class 0 (others are identical shifts)

Q.2 [MCQ · 1 mark · Union-find with path halving]

Answer: (A)

find uses **path halving**: each visited node is re-pointed to its grandparent, flattening the tree as a side-effect of a query.

- $\text{union}(0,1)$: roots 0, 1 $\rightarrow \text{parent}[0] = 1$.
- $\text{union}(2,3)$: $\text{parent}[2] = 3$.
- $\text{union}(1,3)$: roots 1, 3 $\rightarrow \text{parent}[1] = 3$. Now $0 \rightarrow 1 \rightarrow 3$.
- $\text{union}(4,5)$: $\text{parent}[4] = 5$; $\text{union}(6,5)$: $\text{parent}[6] = 5$.
- $\text{union}(0,2)$: $\text{find}(0)$: $\text{parent}[0] \neq 0$, so $\text{parent}[0] = \text{parent}[\text{parent}[0]] = \text{parent}[1] = 3$, $x = 3 \rightarrow$ root 3. $\text{find}(2) = 3$. Same root $\rightarrow$ no link, but $\text{parent}[0]$ has been **changed to 3** by the halving step.

$\text{parent}[:4] = [3, 3, 3, 3]$. The sets are $\{0,1,2,3\}$, $\{4,5,6\}$, $\{7\} \rightarrow 3$ components.

Output **[3, 3, 3, 3]** $\rightarrow$ option **(A)**.

- (B) ignores the compression done by the redundant $\text{union}(0,2)$.
- (C)/(D) miscount components (vertex 7 is a singleton, 4-5-6 is one set).

Trap: a **find** call is not read-only — even a union that does nothing can modify **parent**.

Q.3 [MSQ · 1 mark · Bipartite graphs — parity argument]**Answer: (B), (D)**

A graph is bipartite iff it has no odd cycle.

- If every $d \in D$ is **odd**, each edge joins an odd label to an even label, so the parity of the label is a valid 2-colouring $\rightarrow$ bipartite.
- Otherwise look for an odd cycle: a closed walk uses steps $\pm d$ whose signed sum is 0; an odd number of steps with sum 0 gives an odd closed walk, hence an odd cycle.

Options:

- (A) $\{2, 4\}$: $2 + 2 - 4 = 0$ uses 3 steps: $1 \rightarrow 3 \rightarrow 5 \rightarrow 1$ is a triangle $\rightarrow$ **not bipartite** (all edges stay within one parity class, but that does not help).
- (B) $\{1, 3\}$: both odd $\rightarrow$ **bipartite**.
- (C) $\{2, 3\}$: $2 + 2 + 2 - 3 - 3 = 0$ uses 5 steps: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 4 \rightarrow 1$ is a 5-cycle $\rightarrow$ **not bipartite**.
- (D) $\{3, 5\}$: both odd $\rightarrow$ **bipartite**.

Trap: in (A) the graph splits into an 'odd' and an 'even' component, which looks bipartite-like, but each component contains triangles.

Q.4 [NAT · 1 mark · Connectivity — bridges]**Answer: 3**

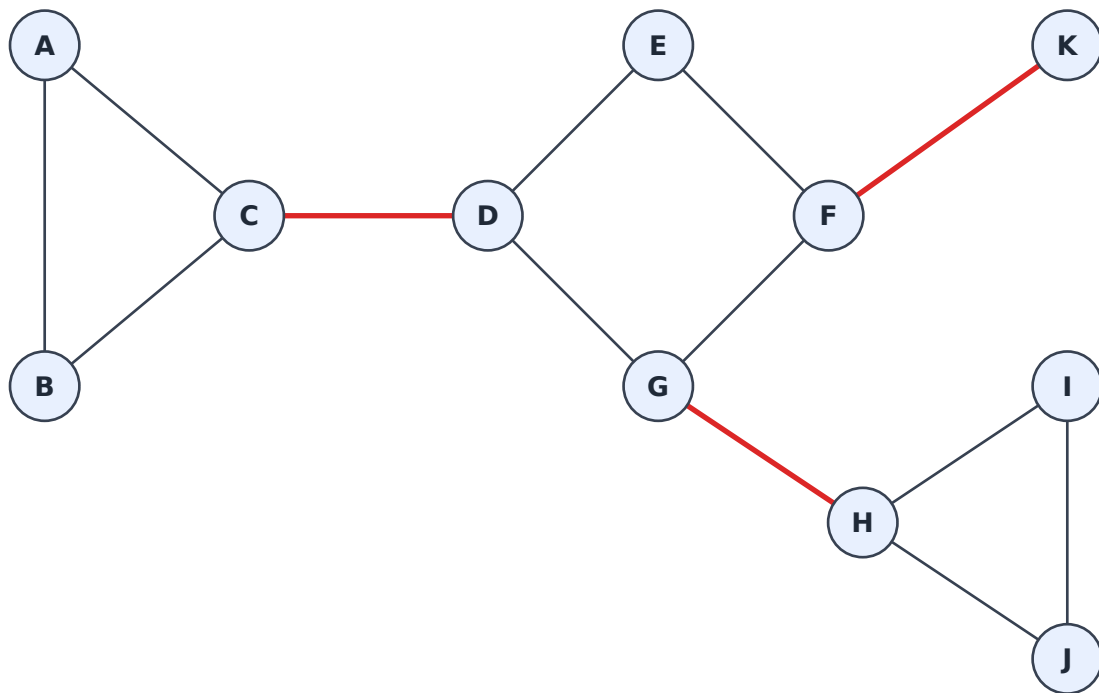
An edge is a bridge **iff it lies on no cycle**. So identify the cycles and the edges outside them.

- Triangle A-B-C: its 3 edges are on a cycle $\rightarrow$ not bridges.
- 4-cycle D-E-F-G-D: 4 edges, not bridges.
- Triangle H-I-J: 3 edges, not bridges.
- Remaining edges: **C-D** (joins the triangle to the square), **G-H** (joins the square to the second triangle), **F-K** (pendant edge to the leaf K). None of these lies on a cycle.

Number of bridges = **3**.

Check: the graph has 11 vertices and 13 edges; contracting each of the three cycles to a single node leaves a tree on 4 nodes (the three cycles plus K), whose 3 edges are exactly the bridges.

Trap: counting cut **vertices** (C, D, F, G, H — five of them) instead of bridges. Every bridge with a non-leaf endpoint gives cut vertices, but the counts differ.



Bridges highlighted

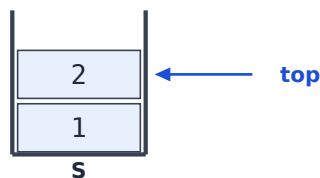
Q.5 [MCQ · 1 mark · Stacks — permutations]**Answer: (C)**

Simulate greedily: to output x , push until x is on top (if x has not been pushed yet), then pop; if x was pushed earlier it must be exactly the top, otherwise the sequence is impossible.

- (A) push 1,2,3 → pop 3, pop 2; push 4 → pop 4; pop 1; push 5,6 → pop 6, pop 5. Valid.
- (B) push 1,2 → pop 2; push 3,4 → pop 4, pop 3; push 5,6 → pop 6, pop 5; pop 1. Valid.
- (C) push 1–4 → pop 4, pop 3; stack is [1, 2] (top 2). push 5 → pop 5. Next output must be 1, but the top is **2** → impossible.
- (D) push/pop alternately. Valid.

Answer **(C)**.

Tip: a sequence is a valid stack permutation iff it has no pattern $i < j < k$ printed in the order k, i, j (a '312' pattern). In (C): 3 ... 1 ... 2 is such a pattern.



Stack in (C) when 1 is required — 2 blocks it

Q.6 [NAT · 1 mark · BST — number of insertion orders]**Answer: 210**

A permutation builds the tree iff every node is inserted **before all its descendants**. Equivalently, count recursively:

$$\text{ways}(T) = C(|L| + |R|, |L|) \times \text{ways}(L) \times \text{ways}(R)$$

(interleave any valid order of the left subtree with any valid order of the right subtree).

- Subtree 30 {20, 40}: 30 first, then 20 and 40 in any order → $C(2,1) = 2$.
- Subtree 80 {90}: 80 then 90 → 1.

- Subtree 70: left {60} (1 node), right {80, 90} (2 nodes) $\rightarrow C(3,1) \times 1 \times 1 = 3$.
- Root 50: left has 3 nodes, right has 4 nodes $\rightarrow C(7,3) \times 2 \times 3 = 35 \times 6 = \mathbf{210}$.

Sanity check: $8! = 40320$ permutations in total, and only 210 of them produce this shape.

Trap: multiplying only the subtree counts ($2 \times 3 = 6$) and forgetting the interleavings between left and right subtrees.

Q.7 [MCQ · 1 mark · Hashing — chaining]

Answer: (B)

Compute $h(k) = (2k + 3) \bmod 9$ for each key:

- $5 \rightarrow 13 \bmod 9 = 4$
- $14 \rightarrow 31 \bmod 9 = 4$
- $22 \rightarrow 47 \bmod 9 = 2$
- $9 \rightarrow 21 \bmod 9 = 3$
- $31 \rightarrow 65 \bmod 9 = 2$
- $40 \rightarrow 83 \bmod 9 = 2$
- $17 \rightarrow 37 \bmod 9 = 1$
- $2 \rightarrow 7 \bmod 9 = 7$

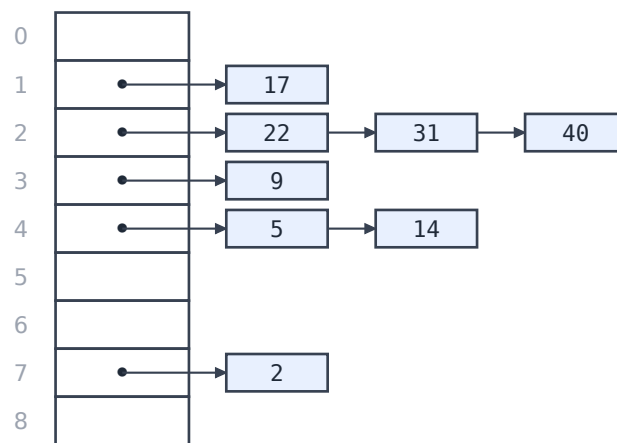
Chains: slot 1: [17]; slot 2: [22, 31, 40]; slot 3: [9]; slot 4: [5, 14]; slot 7: [2].

Occupied slots = 5 $\rightarrow$ empty slots = $9 - 5 = 4$. Longest chain = 3 (slot 2).

Answer **(B)** 3 and 4.

- (A) misses that 40 also maps to 2 ($2 \cdot 40 + 3 = 83 = 9 \cdot 9 + 2$).
- (C)/(D) miscount occupied slots.

Tip: keys that differ by a multiple of 9 always collide here, since $2(k + 9) + 3 \equiv 2k + 3 \pmod{9}$: 5/14, 22/31/40.



Chained table

Q.8 [MCQ · 1 mark · Bubble sort — passes]

Answer: (B)

Each pass 'bubbles' the largest remaining element to the end of the unsorted part.

Pass 1 on [26, 11, 39, 4, 18, 33, 7]:

- $26 > 11$ swap $\rightarrow$ [11, 26, 39, 4, 18, 33, 7]; $26 < 39$; $39 > 4$ swap; $39 > 18$ swap; $39 > 33$ swap; $39 > 7$ swap $\rightarrow$ [11, 26, 4, 18, 33, 7, **39**].

Pass 2 on the first six:

- $11 < 26$; $26 > 4$ swap $\rightarrow$ [11, 4, 26, 18, 33, 7, 39]; $26 > 18$ swap $\rightarrow$ [11, 4, 18, 26, 33, 7, 39]; $26 < 33$; $33 > 7$ swap $\rightarrow$ [11, 4, 18, 26, 7, **33**, 39].

Answer **(B)**.

- (A) is the state after only one pass.
- (C) is the fully sorted array.
- (D) moves 7 too far — a single pass moves a small element left by at most one position (it moved from index 6 to 5 to 4 over two passes).

Tip: small elements ('turtles') move left only one step per pass, which is why bubble sort needs many passes when a small element sits at the end.

Array after each pass							
	0	1	2	3	4	5	6
start	26	11	39	4	18	33	7
pass 1	11	26	4	18	33	7	39
pass 2	11	4	18	26	7	33	39

Q.9 [MCQ · 1 mark · Python — aliasing in adjacency lists]

Answer: (C)

`[[[]] * 4]` creates a list containing **four references to the same inner list**. Every `adj[x].append(...)` mutates that one shared list.

Appends in order: (0,1) → 1, 0; (1,2) → 2, 1; (2,3) → 3, 2. The shared list is [1, 0, 2, 1, 3, 2].

- `len(adj[0]) = 6`.
- `adj[3][:3] = [1, 0, 2]`.

Output 6 [1, 0, 2] → option **(C)**.

- (A) is the intended result with independent lists (`deg(0) = 1`, `adj[3] = [2]`).
- (B)/(D) mix the two behaviours.

Fix: `adj = [[] for _ in range(4)]` evaluates `[]` four times. With the buggy version a BFS would see every vertex adjacent to everything in the shared list — connectivity and bipartiteness checks silently give wrong answers.

Q.10 [NAT · 1 mark · Binary search — lower bound]

Answer: 8

`lb(a, x)` is the classic **lower bound**: it returns the first index i with $a[i] \geq x$ (or `len(a)` if none). The invariant is $a[0..lo-1] < x \leq a[hi..n-1]$.

- `lb(a, 4)`: first element ≥ 4 is the first 4 at index **1** (trace: $lo, hi = 0, 9 \rightarrow mid\ 4\ (7 \geq 4)\ hi=4 \rightarrow mid\ 2\ (4 \geq 4)\ hi=2 \rightarrow mid\ 1\ (4)\ hi=1 \rightarrow mid\ 0\ (2 < 4)\ lo=1$).
- `lb(a, 10)`: first element ≥ 10 is 12 at index **7**.
- `lb(a, 1)`: every element is $\geq 1 \rightarrow$ index **0**.

Sum = 1 + 7 + 0 = **8**.

Trap: a standard 'found / not found' binary search could return any of the indices 1, 2, 3 for $x = 4$; the lower-bound variant always converges to the *leftmost* one because it never stops on equality.

a =	2	4	4	4	7	9	9	12	15
	0	1	2	3	4	5	6	7	8
	↑	↑						↑	
	lb(1)	lb(4)						lb(10)	

Q.11 [MSQ · 2 marks · Connectivity — cut vertices and bridges]**Answer: (B), (C)**

Group the graph into *blocks* (maximal pieces with no cut vertex). A vertex shared by two blocks is a cut vertex; a block consisting of a single edge is a bridge.

Cycles present:

- P-Q-R (triangle);
- S-T-U (triangle), U-V-X-W-T-U and V-W-X (triangle). Because of the edge T-W, the vertices S, T, U, V, W, X all lie on common cycles → they form **one block**.
- R-S lies on no cycle → it is a bridge (its own block).

Blocks: {P, Q, R}, {R, S}, {S, T, U, V, W, X}.

- (A) U-V lies on the cycle U-V-W-T-U. Not a bridge. **False**.
- (B) R joins blocks {P,Q,R} and {R,S}: removing R isolates {P, Q}. **True**.
- (C) S joins {R,S} and the big block: removing S separates {P,Q,R} from the rest. **True**.
- (D) Removing U: S still reaches T, T reaches W via the edge T-W, and W reaches V, X. Graph stays connected. **False**.

Trap: without the chord T-W, U and V *would* be cut vertices and U-V a bridge; one extra edge merges three blocks into one.

Q.12 [NAT · 2 marks · Graph theory — edges vs components]**Answer: 36**

Each component must be a complete graph to maximise edges, so the question is how to split 12 vertices into 4 non-empty parts $n_1 + n_2 + n_3 + n_4 = 12$ maximising $\sum C(n_i, 2)$.

Since $C(n, 2)$ is convex, moving a vertex from a smaller part to a larger part never decreases the sum: $C(a+1, 2) + C(b-1, 2) - C(a, 2) - C(b, 2) = a - b + 1 > 0$ when $a \geq b$. So the optimum is as **unbalanced** as possible: three isolated vertices and one clique on $12 - 3 = 9$ vertices.

Maximum = $C(9, 2) = 36$.

Comparison with other splits:

- $3+3+3+3 \rightarrow 4 \times 3 = 12$ edges
- $6+2+2+2 \rightarrow 15 + 3 = 18$ edges
- $8+2+1+1 \rightarrow 28 + 1 = 29$ edges
- $9+1+1+1 \rightarrow \mathbf{36}$ edges

General formula: max edges with n vertices and k components = $C(n - k + 1, 2)$.

Trap: balancing the components (the instinct from 'fair splitting') gives the *minimum* among clique unions, not the maximum.

Q.13 [MSQ · 2 marks · Bipartiteness — BFS two-colouring]**Answer: (A), (B), (D)**

BFS colours each newly discovered vertex opposite to its parent; a non-tree edge between two vertices of the same colour proves an odd cycle.

Trace (colour in brackets):

- $u=1$ [0]: 2 [1], 6 [1].
- $u=2$: 1 ok, 3 [0], 7 [0].
- $u=6$: 1 ok, 5 [0].
- $u=3$: 2 ok, 4 [1], 8 [1].
- $u=7$ [0]: 2 ok, 8 [1] ok.
- $u=5$ [0]: 4 [1] ok, 6 ok, 8 [1] ok.
- $u=4$ [1]: 3 ok, 5 ok, **8 [1] — same colour** → return (None, (4, 8)).

- (A) Reported pair is (4, 8). **True.**
- (B) Every odd cycle uses 4-8 (triangles 3-4-8 and 4-5-8, etc.). Without it the colouring {1,3,5,7} vs {2,4,6,8} is proper. **True.**
- (C) Removing 3-8 leaves the triangle 4-5-8. **False.**
- (D) G is not bipartite and the function detects it. **True.**

Trap: the reported edge depends on the visiting order; it is *an* edge of an odd cycle, not necessarily the unique culprit — here it happens to be unique.

Q.14 [NAT · 2 marks · Connected components — iterative DFS in Python]

Answer: 56

The function finds each connected component (iterative DFS with an explicit stack; a vertex is marked when pushed, so it is counted exactly once) and accumulates size $\times$ (number of vertices in earlier components). That is the number of unordered pairs of vertices lying in **different** components.

Components of the 12 vertices:

- {0, 5, 9} (size 3)
- {1, 4, 11} (size 3; edges form a triangle)
- {2, 7} (size 2)
- {3, 8, 10} (size 3)
- {6} (size 1; the self-loop (6, 6) adds 6 to its own list twice but connects nothing)

Accumulation in order of the smallest vertex 0, 1, 2, 3, 6:

- $3 \times 0 = 0 \rightarrow$ done 3
- $3 \times 3 = 9 \rightarrow$ done 6
- $2 \times 6 = 12 \rightarrow$ done 8
- $3 \times 8 = 24 \rightarrow$ done 11
- $1 \times 11 = 11 \rightarrow$ done 12

Total = $0 + 9 + 12 + 24 + 11 = 56$.

Check: $C(12, 2) - (3 + 3 + 1 + 3 + 0) = 66 - 10 = 56$.

Trap: forgetting the isolated vertex 6 (11 pairs) or treating the self-loop as creating an extra component.

Q.15 [MCQ · 2 marks · DFS — edge classification]

Answer: (D)

Classify edge (u, v) when it is examined: v undiscovered $\rightarrow$ **tree**; v discovered but unfinished (an ancestor) $\rightarrow$ **back**; v finished with $d[u] < d[v]$ (a descendant) $\rightarrow$ **forward**; v finished with $d[v] < d[u]$ $\rightarrow$ **cross**.

DFS trace with (discovery/finish) times:

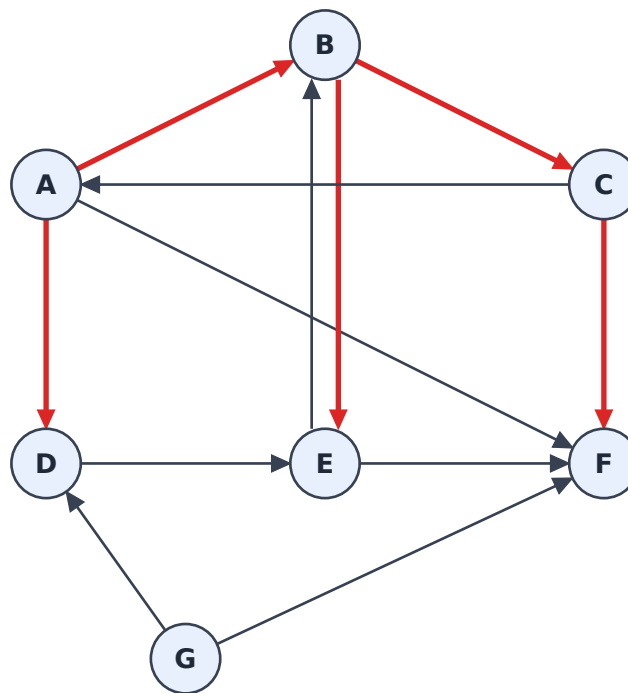
- A (1): A $\rightarrow$ B tree. B (2): B $\rightarrow$ C tree. C (3): C $\rightarrow$ A **back** (A is active); C $\rightarrow$ F tree. F (4/5). C finishes (6).
- Back in B: B $\rightarrow$ E tree. E (7): E $\rightarrow$ B **back**; E $\rightarrow$ F **cross** (F finished, $d[F]=4 < 7$). E (8). B (9).
- Back in A: A $\rightarrow$ D tree. D (10): D $\rightarrow$ E **cross**. D (11). A $\rightarrow$ F: F finished and $d[A]=1 < d[F]=4 \rightarrow$ **forward**. A (12).
- New tree from G (13): G $\rightarrow$ D **cross**, G $\rightarrow$ F **cross**. G (14).

Tree: A $\rightarrow$ B, B $\rightarrow$ C, C $\rightarrow$ F, B $\rightarrow$ E, A $\rightarrow$ D (5). Back: C $\rightarrow$ A, E $\rightarrow$ B (2). Forward: A $\rightarrow$ F (1). Cross: E $\rightarrow$ F, D $\rightarrow$ E, G $\rightarrow$ D, G $\rightarrow$ F (4). Total 12 edges ✓.

Answer **(D)** (5, 2, 1, 4).

- (A) treats E $\rightarrow$ F as back.
- (B) classifies A $\rightarrow$ F as cross — but F is a descendant of A.
- (C) counts A $\rightarrow$ F as a tree edge — F was already discovered via C.

Tip: the graph has a cycle iff DFS finds a back edge (here $A \rightarrow B \rightarrow C \rightarrow A$).



DFS forest (tree edges highlighted)

Q.16 [MSQ · 2 marks · Sorting — properties and counts]

Answer: (B), (C), (D)

- (B) With all keys equal, every $a[j] \leq \text{pivot}$, so the partition places the pivot at the **end** and recurses on $n - 1$ elements. Comparisons: $5 + 4 + 3 + 2 + 1 = 15$ — the $\Theta(n^2)$ worst case of Lomuto on duplicates. **True.**
- (D) Taking from the left run on ties keeps equal keys in their original relative order (left-run elements were earlier in the input). **True.**
- (C) In reverse-sorted input, inserting the i -th element compares it with all i earlier elements (the loop stops only at index 0, with no failing comparison). Total $1 + 2 + \dots + 7 = 28 = n(n-1)/2$. **True.**
- (A) The long-distance swap can jump an element over an equal one. Example $[2(a), 2(b), 1]$: pass 0 swaps $2(a)$ with $1 \rightarrow [1, 2(b), 2(a)]$ — the two 2s are reversed. **False.**

Trap: in (C) one might add a final failing comparison per pass (giving 35); in a strictly decreasing array every comparison succeeds until j falls below 0, where the $j \geq 0$ test (an index check, not an element comparison) stops the loop.

Q.17 [NAT · 2 marks · Linked lists — Floyd's cycle detection]

Answer: 9

Let μ = number of nodes before the cycle and λ = cycle length. Here the tail is $1 \rightarrow \dots \rightarrow 7$, so $\mu = 7$, and the cycle $8 \rightarrow 9 \rightarrow 10 \rightarrow 8$ has $\lambda = 3$.

After k steps the tortoise is k nodes from the head and the hare $2k$ nodes. Both are inside the cycle once $k \geq \mu$, and they coincide when $2k - k = k$ is a multiple of λ . So the first meeting is at the smallest $k \geq \mu$ with $k \equiv 0 \pmod{\lambda}$: $k \geq 7$, k multiple of 3 $\rightarrow k = 9$.

Explicit check (position = node value):

- $k=7$: slow 8, fast at 14 steps $\rightarrow 8 + (14-7) \bmod 3 = 8 + 1 = 9 \rightarrow$ different.

- $k=8$: slow 9, fast at 16 $\rightarrow 8 + 9 \bmod 3 = 8 \rightarrow$ different.
- $k=9$: slow 10, fast at 18 $\rightarrow 8 + 11 \bmod 3 = 10 \rightarrow$ **meet** at node 10.

Printed value: **9**.

Trap: assuming they meet at the cycle entry (node 8) or after λ steps. The meeting point is generally *inside* the cycle; finding the entry needs the second phase (restart one pointer at the head and move both one step at a time).

Q.18 [MCQ · 2 marks · Heaps — bottom-up build-heap]

Answer: (A)

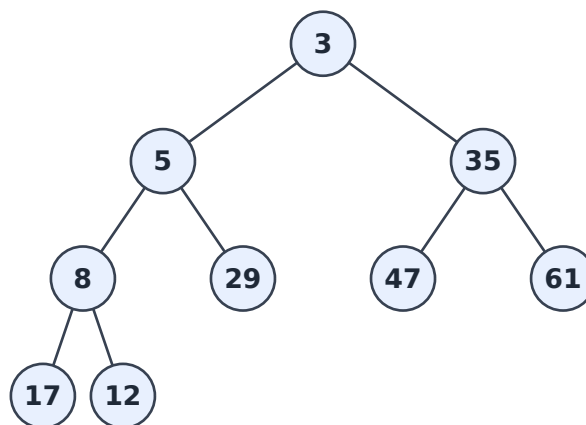
Bottom-up heap construction sifts down every internal node, from the last one (index $\lfloor 9/2 \rfloor - 1 = 3$) back to the root; total cost is $O(n)$.

- $i=3$ (8): children 17, 5 $\rightarrow$ swap with 5 $\rightarrow$ [35, 12, 47, **5**, 29, 3, 61, 17, **8**].
- $i=2$ (47): children 3, 61 $\rightarrow$ swap with 3 $\rightarrow$ [35, 12, **3**, 5, 29, **47**, 61, 17, 8].
- $i=1$ (12): children 5, 29 $\rightarrow$ swap with 5 (index 3); now 12 at index 3 has children 17, 8 $\rightarrow$ swap with 8 $\rightarrow$ [35, **5**, 3, **8**, 29, 47, 61, 17, **12**].
- $i=0$ (35): children 5, 3 $\rightarrow$ swap with 3 (index 2); children of index 2 are 47, 61 $\rightarrow$ stop $\rightarrow$ [**3**, 5, **35**, 8, 29, 47, 61, 17, 12].

Answer **(A)** (5 swaps in total).

- (B) is the heap obtained by inserting the keys one by one — a different (also valid) heap.
- (C) stops sifting 12 after one level ($12 > 8$ violates the heap property).
- (D) is also a valid min-heap of the same keys, but not the one this procedure builds: 35 only sinks one level (to index 2), it never reaches the last leaf.

Trap: build-heap and repeated insertion generally produce *different* heaps.



Min-heap after bottom-up construction

Q.19 [MCQ · 2 marks · Hashing — unsuccessful search with linear probing]

Answer: (D)

Occupied slots form two clusters: {8, 9, 0, 1} (wrapping around the end) and {4, 5}. An unsuccessful search from home slot h probes the rest of h 's cluster plus one empty slot.

Probes per home slot:

- $h=0$: 0, 1, 2 $\rightarrow$ 3
- $h=1$: 1, 2 $\rightarrow$ 2
- $h=2$: 1; $h=3$: 1
- $h=4$: 4, 5, 6 $\rightarrow$ 3; $h=5$: 5, 6 $\rightarrow$ 2
- $h=6$: 1; $h=7$: 1
- $h=8$: 8, 9, 0, 1, 2 $\rightarrow$ 5

- $h=9$: 9, 0, 1, 2 $\rightarrow$ 4

Sum = 3 + 2 + 1 + 1 + 3 + 2 + 1 + 1 + 5 + 4 = 23 $\rightarrow$ expected **2.3** probes $\rightarrow$ option **(D)**.

- (A) 2.5 is $1/(1 - \alpha)$ — the open-addressing estimate under *uniform* probing, again not this concrete table.
- (B) 1.7 forgets the wrap-around: it stops the searches from slots 8 and 9 at the end of the array (2 and 1 probes instead of 5 and 4).
- (C) $1.6 = 1 + \text{load factor } 0.6$ — the uniform-hashing estimate, which ignores clustering.

Tip: a cluster of length L contributes $(L+1) + L + \dots + 2 = (L+1)(L+2)/2 - 1$ probes; long clusters dominate the average (primary clustering).

Q.20 [MSQ · 2 marks · Bipartiteness and connectivity of a grid graph]

Answer: (B), (C), (D)

Colour vertex (r, c) by the parity of $r + c$ — every grid edge changes $r + c$ by 1, so this is a proper 2-colouring (a 'chessboard' colouring).

- (A) Every vertex lies on a 4-cycle (unit square) and removing any one vertex leaves the rest connected around it; the grid is 2-connected. **False.**
- (B) Each row of 4 has 2 vertices of each colour $\rightarrow$ 6 and 6. **True.**
- (C) Vertex 1's only neighbours are 2 and 5. Removing both isolates vertex 1 $\rightarrow$ disconnected. **True.**
- (D) Vertex 1 = (0,0) and 6 = (1,1) both have even $r + c$, i.e. the same colour. An edge between them closes the odd cycle 1-2-6-1 (length 3). **True.**

Extra facts: $|E| = 3 \cdot 3 + 2 \cdot 4 = 17$; minimum degree 2 (corners) — so the vertex connectivity is exactly 2.

Trap: concluding from (C) that the grid has a cut vertex — needing **two** vertices to disconnect it means it has none.

Mock Test 29

Weighted Shortest Paths

Difficulty: GATE-level

Focus areas: Dijkstra with ties, negative edges, Bellman-Ford rounds, path trees

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — heapq with tuple priorities]

Dijkstra implementations in Python usually store `(distance, vertex)` tuples in a `heapq`. Consider the following program. What is printed?

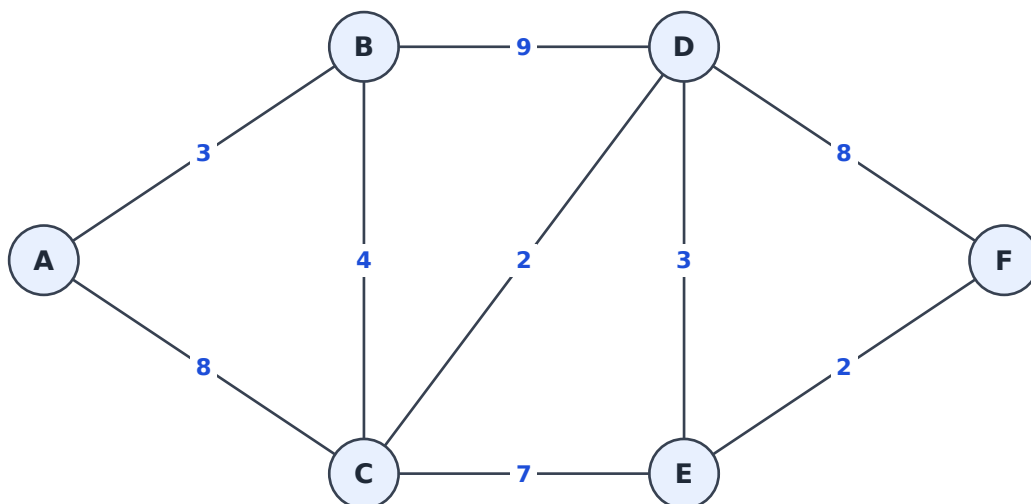
Python

```
1 import heapq
2 pq = []
3 for item in [(4, 'D'), (2, 'B'), (4, 'A'), (2, 'C'), (1, 'E')]:
4     heapq.heappush(pq, item)
5 print([heapq.heappop(pq)[1] for _ in range(len(pq))])
```

- (A) ['E', 'B', 'C', 'A', 'D']
- (B) ['E', 'C', 'B', 'A', 'D']
- (C) ['E', 'B', 'C', 'D', 'A']
- (D) ['D', 'A', 'B', 'C', 'E']

Q.2 [NAT · numerical answer | 1 mark | Dijkstra — undirected graph]

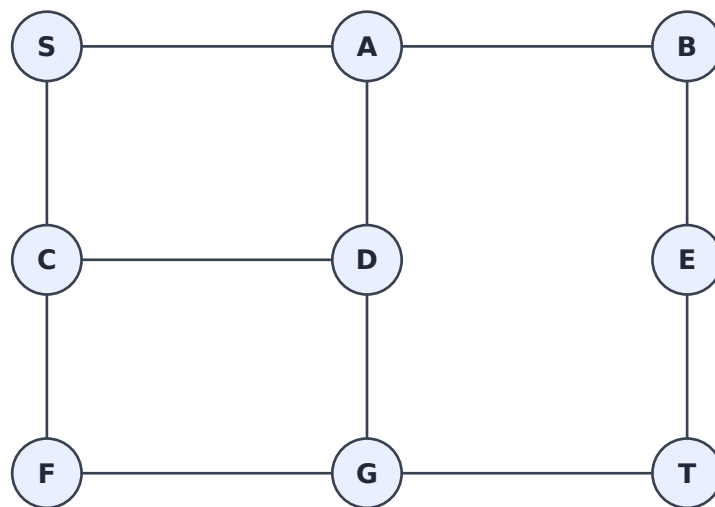
Dijkstra's algorithm is run from vertex **A** on the undirected weighted graph below. The shortest-path distance from A to F is _____.



Answer: _____ (enter a numerical value)

Q.3 [NAT · numerical answer | 1 mark | Unweighted shortest paths — counting]

In the unweighted undirected graph below (a 3×3 grid from which the edge D-E has been removed), the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.4 [MSQ · one or more correct | 1 mark | Shortest paths — negative edges and reweighting]

Consider single-source shortest paths in directed graphs with real edge weights. Dijkstra's algorithm here never re-extracts a vertex. Which of the following statements is/are TRUE?

- (A) On the graph with edges $S \rightarrow A$ (2), $S \rightarrow B$ (3), $B \rightarrow A$ (-2), Dijkstra from S reports $d(A) = 2$ while the true distance is 1
- (B) Multiplying every edge weight by the same constant $c > 0$ never changes which S-T path is shortest
- (C) If the graph has negative edges but no negative-weight cycle, Bellman-Ford from S computes correct distances to all vertices reachable from S
- (D) Adding the same constant $c > 0$ to every edge weight never changes which S-T path is shortest

Q.5 [MCQ · single correct | 1 mark | Queues and stacks — reversing a prefix]

A queue holds 10, 20, 30, 40, 50, 60 (front on the left). The following is done:

1. Dequeue 4 elements, pushing each onto an empty stack.
2. Pop the stack until empty, enqueueing each popped element.
3. Twice: dequeue an element and enqueue it again.

The final queue (front to rear) is

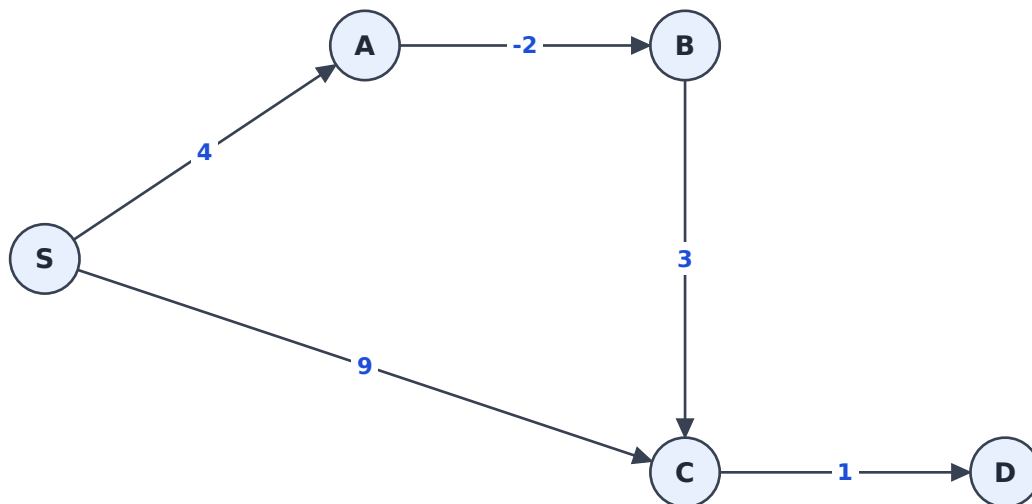
- (A) 60, 50, 40, 30, 20, 10
- (B) 50, 60, 40, 30, 20, 10
- (C) 10, 20, 30, 40, 50, 60
- (D) 40, 30, 20, 10, 50, 60

Q.6 [NAT · numerical answer | 1 mark | Bellman-Ford — rounds depend on edge order]

Bellman-Ford is run from S on the directed graph below. In every round the edges are relaxed in this fixed order:

C→D, B→C, A→B, S→A, S→C.

Relaxations use the current (most recently updated) distances. Initially $d(S) = 0$ and all other distances are ∞ . The number of the **last** round in which some distance changes is ____.



Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Hashing — double hashing]

A hash table has 13 slots (0–12). Keys are inserted with double hashing: the i -th probe ($i = 0, 1, 2, \dots$) for key k examines slot $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$.

The keys 27, 40, 53, 14, 66, 92, 48 are inserted in this order into an empty table. In which slot is 48 stored?

- (A) 10
- (B) 0
- (C) 3
- (D) 6

Q.8 [MCQ · single correct | 1 mark | Binary trees — reconstruction from traversals]

A binary tree with distinct labels has

pre-order: G C A B E D F K I H L

in-order: A B C D E F G H I K L

Its post-order traversal is

- (A) B A D F E C I H L K G
- (B) B A D F E C H I L K G
- (C) A B D F E C H I L K G
- (D) B A D E F C H I L K G

Q.9 [MSQ · one or more correct | 1 mark | Elementary sorts — swaps and passes]

The array [5, 2, 8, 1, 9, 3] is sorted into increasing order. Bubble sort compares adjacent elements left to right in each pass; selection sort swaps $A[i]$ with the minimum of $A[i..n-1]$ **only if** they are different positions; insertion sort shifts larger elements one place right. Which of the following statements is/are TRUE?

- (A) After the first pass of bubble sort the array is [2, 5, 1, 8, 3, 9]
- (B) Insertion sort performs exactly 7 element shifts
- (C) Selection sort performs exactly 4 swaps
- (D) Bubble sort performs exactly 7 swaps

Q.10 [MCQ · single correct | 1 mark | Python — aliasing in adjacency lists]

A programmer builds adjacency lists as follows. What is printed?

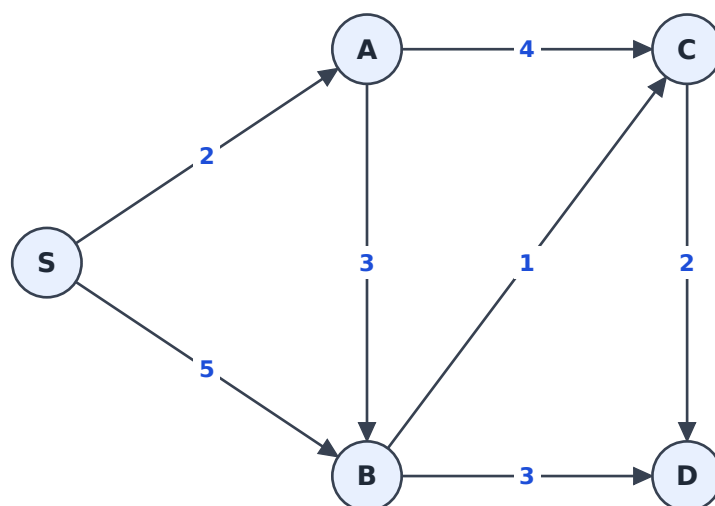
Python

```
1 G = dict.fromkeys("ABC", [])
2 G["A"].append("B")
3 G["B"] = G["B"] + ["C"]
4 G["C"] += ["A"]
5 print(G)
```

- (A) {'A': ['B'], 'B': ['C'], 'C': ['A']}
- (B) {'A': ['B', 'C', 'A'], 'B': ['B', 'C', 'A'], 'C': ['B', 'C', 'A']}
- (C) {'A': ['B', 'A'], 'B': ['B', 'C'], 'C': ['B', 'A']}
- (D) {'A': ['B'], 'B': ['B', 'C'], 'C': ['B', 'A']}

Q.11 [MCQ · single correct | 2 marks | Dijkstra — shortest-path tree with ties]

Dijkstra's algorithm is run from S on the directed graph below. A tentative distance $d[v]$ (and the parent of v) is changed **only on a strict improvement**, and when several vertices have the same smallest tentative distance the alphabetically smallest is extracted first. Which set of edges forms the resulting shortest-path tree?



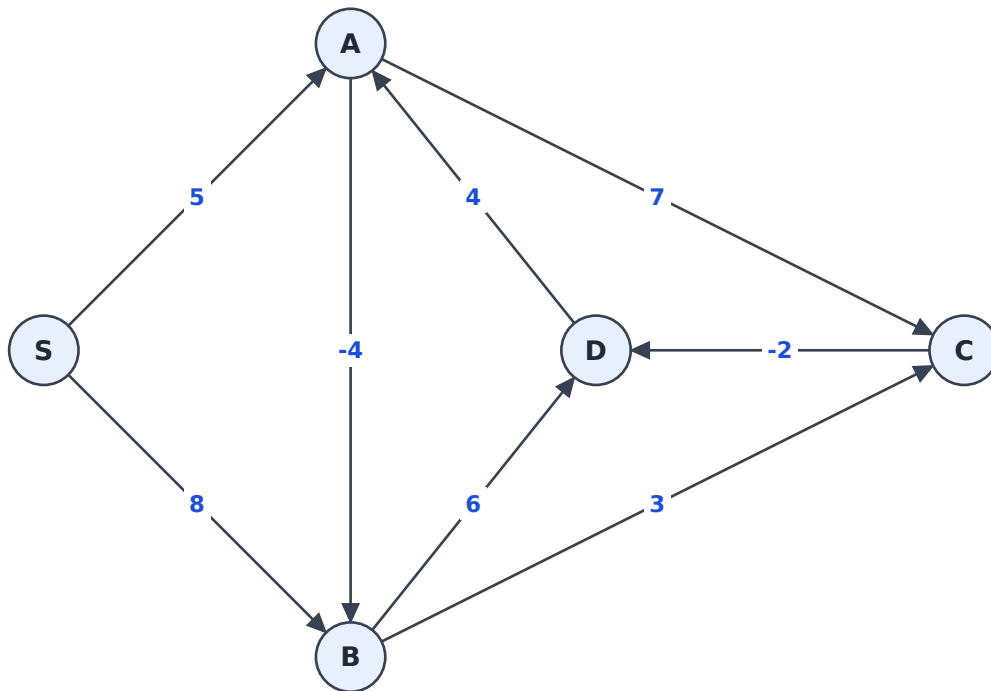
- (A) {S→A, A→B, B→C, C→D}
- (B) {S→A, A→B, A→C, C→D}
- (C) {S→A, S→B, A→C, C→D}
- (D) {S→A, S→B, A→C, B→D}

Q.12 [MSQ · one or more correct | 2 marks | Bellman-Ford — round-by-round trace]

Bellman-Ford is run from S on the directed graph below. In every round the edges are relaxed in the order

C→D, B→D, B→C, A→C, A→B, S→B, S→A, D→A

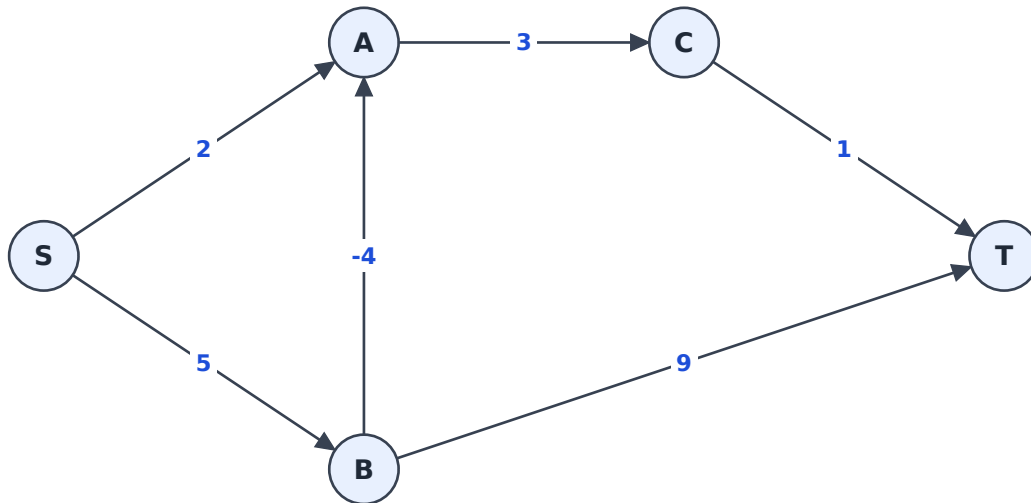
using the current distance values. Initially $d(S) = 0$ and all others are ∞ . Which of the following statements is/are TRUE?



- (A) After round 1, $d(B) = 8$
- (B) The final value of $d(D)$ is 2
- (C) The algorithm reports a negative-weight cycle
- (D) After round 2, $d(D) = 7$

Q.13 [NAT · numerical answer | 2 marks | Dijkstra on a negative edge — error analysis]

The textbook Dijkstra algorithm is run from S on the directed graph below, which has one negative edge. Once a vertex is extracted its distance is final and is **never** changed again; among equal tentative distances the alphabetically smaller vertex is extracted first. Let $r(v)$ be the distance reported by Dijkstra and $\delta(v)$ the true shortest distance. The value of $\sum (r(v) - \delta(v))$ over $v \in \{A, B, C, T\}$ is _____.



Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Python — a subtly wrong Dijkstra]

The following program tries to compute shortest-path distances, but marks a vertex as **seen** when it is **first pushed** instead of when it is popped. What is printed?

Python

```

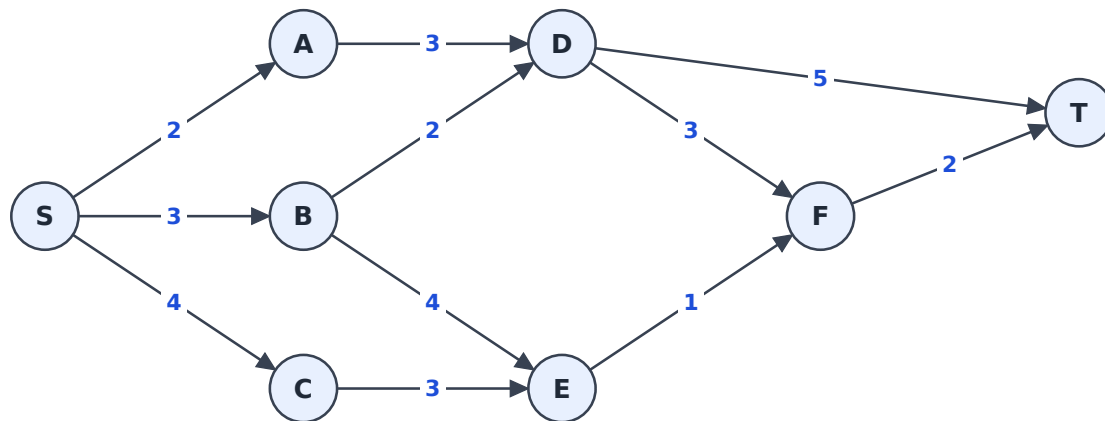
1 import heapq
2 G = {'S': [('A', 1), ('B', 4)], 'A': [('B', 2), ('C', 6)],
3      'B': [('C', 1)], 'C': []}
4
5 def sp(src):
6     dist = {src: 0}
7     pq = [(0, src)]
8     seen = {src}
9     while pq:
10        d, u = heapq.heappop(pq)
11        for v, w in G[u]:
12            if v not in seen:
13                seen.add(v)
14                dist[v] = d + w
15                heapq.heappush(pq, (dist[v], v))
16    return dist
17
18 print(sp('S'))

```

- (A) {'S': 0, 'A': 1, 'B': 3, 'C': 4}
- (B) {'S': 0, 'A': 1, 'B': 4, 'C': 7}
- (C) {'S': 0, 'A': 1, 'B': 4, 'C': 5}
- (D) {'S': 0, 'A': 1, 'B': 3, 'C': 7}

Q.15 [NAT · numerical answer | 2 marks | Counting shortest paths in a weighted graph]

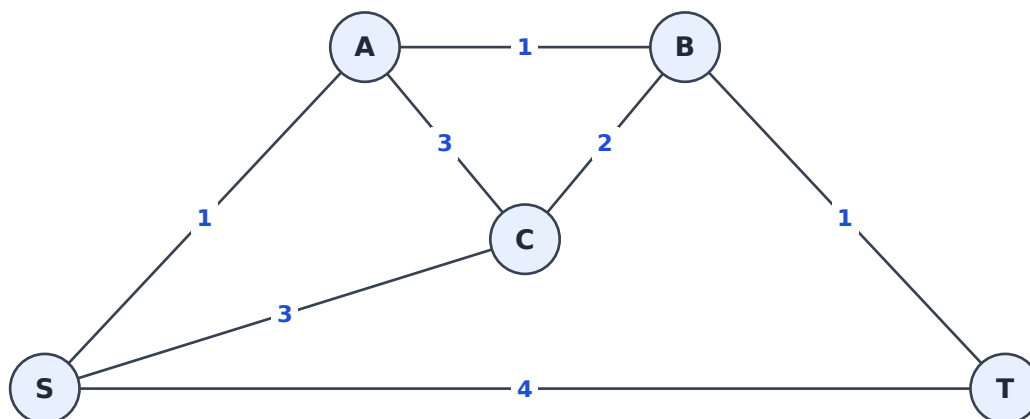
In the weighted directed graph below, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Shortest paths vs MST; transforming weights]

Consider the undirected weighted graph below. Which of the following statements is/are TRUE?



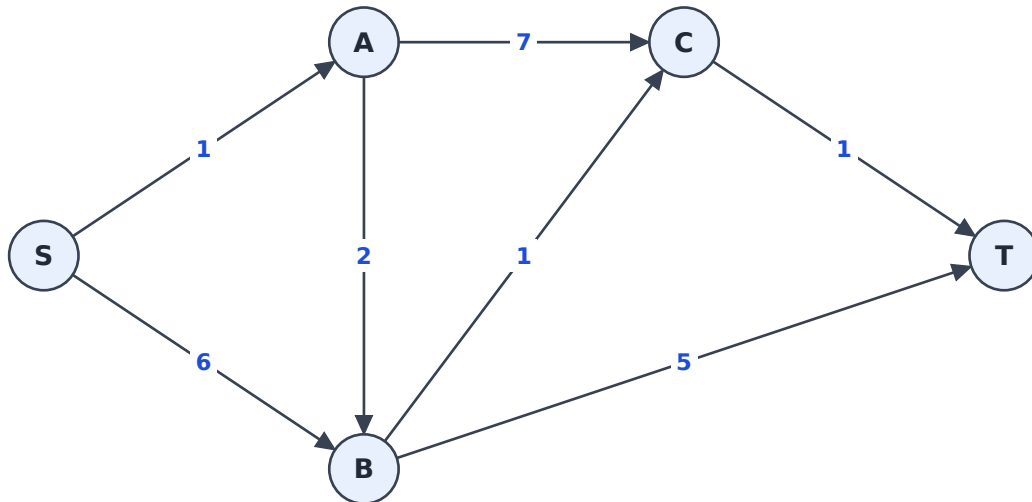
- (A) If every edge weight is increased by 1, the shortest S-T path changes
- (B) The shortest-path tree from S is a minimum spanning tree of the graph
- (C) If every edge weight is squared, the shortest S-T path does not change
- (D) If every edge weight is doubled, the shortest S-T path changes

Q.17 [NAT · numerical answer | 2 marks | Bellman-Ford — synchronous rounds = hop-bounded paths]

A *synchronous* Bellman-Ford is run from S on the directed graph below: in round k every vertex computes

$$d_k(v) = \min(d_{k-1}(v), \min \text{ over edges } u \rightarrow v \text{ of } d_{k-1}(u) + w(u, v))$$

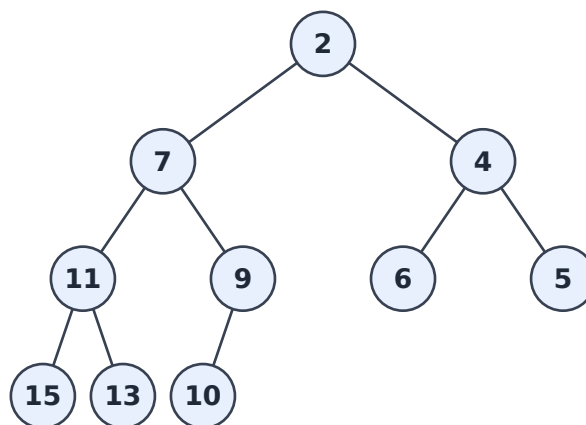
using **only the values of the previous round**; $d_0(S) = 0$ and $d_0(v) = \infty$ otherwise. The value of $d_3(T)$ is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Heaps — decrease-key (as used by Dijkstra)]

Dijkstra's priority queue is the binary min-heap below (array, root at index 0). The key at index 8 (value 13) is **decreased to 3** and the heap property is restored by sifting it up. The resulting array is

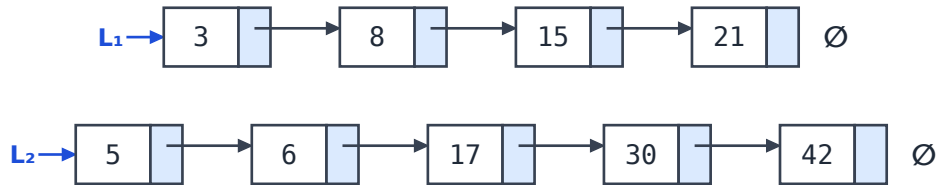


Min-heap before decrease-key

- (A) [2, 3, 4, 7, 9, 6, 5, 15, 11, 10]
- (B) [2, 3, 4, 11, 9, 6, 5, 15, 7, 10]
- (C) [3, 2, 4, 7, 9, 6, 5, 15, 11, 10]
- (D) [2, 7, 4, 3, 9, 6, 5, 15, 11, 10]

Q.19 [MSQ · one or more correct | 2 marks | Linked lists — merging sorted lists (merge sort step)]

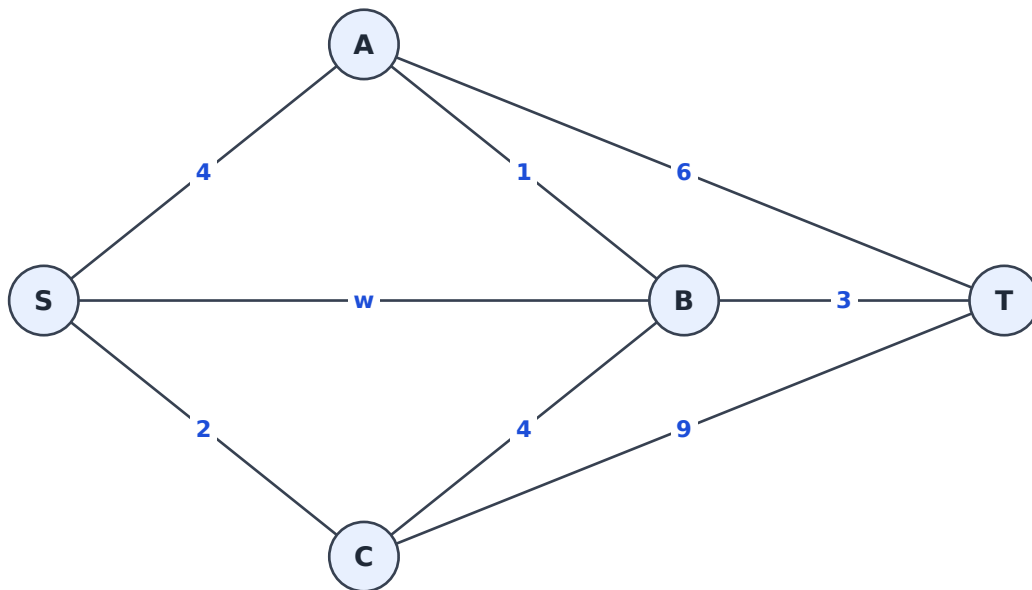
The two sorted singly linked lists L_1 and L_2 shown are merged into one sorted list by the standard merge (repeatedly compare the two front nodes, unlink the smaller one and append it to the result; when one list becomes empty, attach the rest of the other without comparisons). Which of the following statements is/are TRUE?



- (A) Merging **any** two sorted lists of lengths 4 and 5 needs at most 8 key comparisons
- (B) Merging **any** two sorted lists of lengths 4 and 5 needs at least 5 key comparisons
- (C) Merging L_1 and L_2 performs exactly 7 key comparisons
- (D) The 5th node of the merged list holds 8

Q.20 [NAT · numerical answer | 2 marks | Shortest paths — parametric edge weight]

In the undirected graph below, the edge S-B has an unknown positive integer weight w ; all other weights are as shown. The **largest** value of w for which the shortest S-T path is unique and uses the edge S-B is _____.



Answer: _____ (enter a numerical value)

Answer Key — Mock Test 29

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — heapq with tuple priorities
2	NAT	1	14	Dijkstra — undirected graph
3	NAT	1	4	Unweighted shortest paths — counting
4	MSQ	1	(A), (B), (C)	Shortest paths — negative edges and reweighting
5	MCQ	1	(D)	Queues and stacks — reversing a prefix
6	NAT	1	4	Bellman-Ford — rounds depend on edge order
7	MCQ	1	(C)	Hashing — double hashing
8	MCQ	1	(B)	Binary trees — reconstruction from traversals
9	MSQ	1	(A), (B), (D)	Elementary sorts — swaps and passes
10	MCQ	1	(C)	Python — aliasing in adjacency lists
11	MCQ	2	(D)	Dijkstra — shortest-path tree with ties
12	MSQ	2	(A), (B)	Bellman-Ford — round-by-round trace
13	NAT	2	3	Dijkstra on a negative edge — error analysis
14	MCQ	2	(B)	Python — a subtly wrong Dijkstra
15	NAT	2	6	Counting shortest paths in a weighted graph
16	MSQ	2	(A), (C)	Shortest paths vs MST; transforming weights
17	NAT	2	8	Bellman-Ford — synchronous rounds = hop-bounded paths
18	MCQ	2	(A)	Heaps — decrease-key (as used by Dijkstra)
19	MSQ	2	(A), (C)	Linked lists — merging sorted lists (merge sort step)
20	NAT	2	4	Shortest paths — parametric edge weight

Detailed Solutions — Mock Test 29

Q.1 [MCQ · 1 mark · Python — heapq with tuple priorities]

Answer: (A)

`heapq` is a binary **min**-heap, and tuples are compared **lexicographically**: first by distance, and only on a tie by the second component (here the vertex name, compared as strings).

Sorted order of the five tuples: $(1, 'E') < (2, 'B') < (2, 'C') < (4, 'A') < (4, 'D')$.

- `range(len(pq))` is evaluated **once**, when the comprehension starts (`len` = 5), so exactly five pops occur even though the heap shrinks.
- Each `heappop` returns the current minimum, so the names come out as E, B, C, A, D.

Output `['E', 'B', 'C', 'A', 'D']` → option (A).

- (B) reverses the tie between B and C.
- (C) assumes ties keep insertion order (D was pushed before A) — a heap is **not** stable, and here the tuple comparison decides anyway.
- (D) treats `heapq` as a max-heap.

Tip: this is why ‘ties broken alphabetically’ in Dijkstra questions matches a heap of (distance, name) tuples exactly. If the second component were not comparable (e.g. a dict), a tie would raise `TypeError` — a common bug fixed by inserting a counter.

Q.2 [NAT · 1 mark · Dijkstra — undirected graph]

Answer: 14

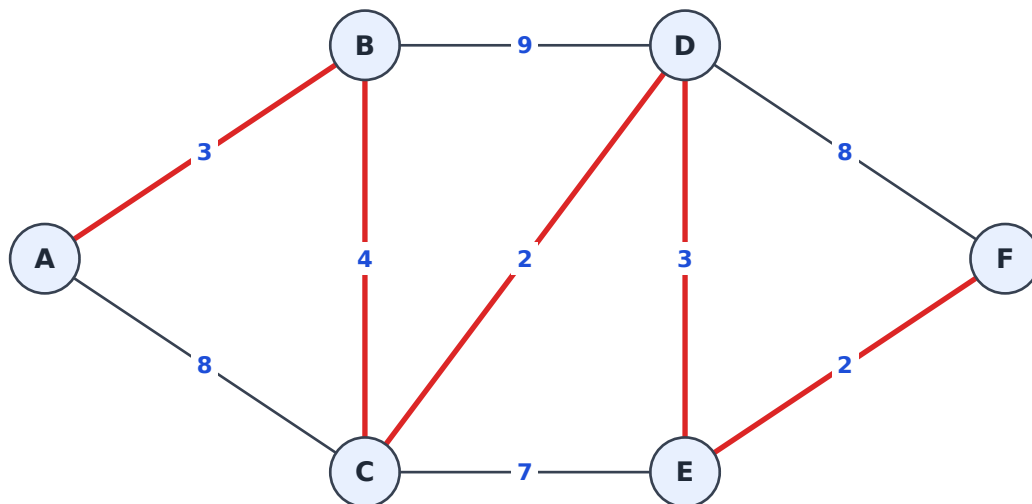
Extract the closest unfinished vertex each time and relax its edges.

- A (0): B = 3, C = 8.
- B (3): C = $\min(8, 3+4) = 7$; D = $3+9 = 12$.
- C (7): D = $\min(12, 7+2) = 9$; E = $7+7 = 14$.
- D (9): E = $\min(14, 9+3) = 12$; F = $9+8 = 17$.
- E (12): F = $\min(17, 12+2) = 14$.
- F (14).

$d(F) = 14$ along $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ ($3 + 4 + 2 + 3 + 2$).

Notice that the shortest path uses **five** edges although F is only three edges from A (e.g. A-B-D-F costs 20). Every ‘direct’ edge (A-C, B-D, C-E, D-F) is beaten by a detour.

Trap: stopping at the first finite value seen for a vertex (D = 12, F = 17). A label becomes final only when its vertex is extracted.



Shortest-path tree from A (a single path here)

Q.3 [NAT · 1 mark · Unweighted shortest paths — counting]**Answer: 4**

BFS gives levels and the number of shortest paths: $\text{paths}(v) = \sum \text{paths}(u)$ over neighbours u one level closer to S .

- Level 0: S (1 path).
- Level 1: A (1), C (1).
- Level 2: B (from A: 1), D (from A and C: 2), F (from C: 1).
- Level 3: E (from B only, since D-E is missing: 1), G (from D and F: $2 + 1 = 3$).
- Level 4: T (from E and G: $1 + 3 = 4$).

The four paths: S-A-B-E-T, S-A-D-G-T, S-C-D-G-T, S-C-F-G-T.

In the complete 3×3 grid there would be $C(4, 2) = 6$ monotone paths; removing D-E kills exactly the two that use it (S-A-D-E-T and S-C-D-E-T).

Trap: counting all simple paths (there are longer ones, e.g. S-C-D-A-B-E-T) — only paths of the minimum length 4 count.

Q.4 [MSQ · 1 mark · Shortest paths — negative edges and reweighting]**Answer: (A), (B), (C)**

- (A) **True.** Dijkstra extracts S (0), sets $A = 2$, $B = 3$, then extracts A (2) and finalises it. When B (3) is extracted, the edge $B \rightarrow A$ would give $3 - 2 = 1$, but A is already final, so the reported value stays 2. Dijkstra's greedy proof needs non-negative weights.
- (B) **True.** Every path cost is multiplied by the same $c > 0$, which preserves the order of path costs.
- (C) **True.** After $|V| - 1$ rounds, Bellman-Ford has found every shortest path with at most $|V| - 1$ edges, and without negative cycles shortest paths are simple.
- (D) **False.** A path with k edges gains $k \cdot c$, so paths with *more* edges are penalised more. E.g. $S \rightarrow X \rightarrow T$ with weights 1, 1 (total 2) vs $S \rightarrow T$ with weight 3: after adding 2 the costs become 6 and 5 — the shortest path changes.

Answer: (A), (B), (C).

Trap: 'just add a constant to make all weights non-negative, then run Dijkstra' is wrong — that is exactly statement (D). (Johnson's algorithm uses vertex potentials, which shift every S-T path by the *same* amount.)

Q.5 [MCQ · 1 mark · Queues and stacks — reversing a prefix]**Answer: (D)**

This is the standard 'reverse the first k elements of a queue' routine with $k = 4$.

- Step 1: stack (bottom→top) = 10, 20, 30, 40; queue = 50, 60.
- Step 2: pops come out 40, 30, 20, 10 (LIFO) and are enqueued: queue = 50, 60, 40, 30, 20, 10.
- Step 3: rotate the $n - k = 2$ elements that were originally behind the prefix: dequeue 50 → enqueue; dequeue 60 → enqueue. Queue = **40, 30, 20, 10, 50, 60**.

Answer (D).

- (A) would need all six elements to pass through the stack.
- (B) is the state after step 2 (forgetting step 3).
- (C) would result if a queue (FIFO) had been used instead of the stack.

Cost: k pushes, k pops and $n + (n - k)$ queue operations → $\Theta(n)$.

Tip: a stack reverses order; a queue preserves it. Rotations of a queue never change the cyclic order.

Q.6 [NAT · 1 mark · Bellman-Ford — rounds depend on edge order]**Answer: 4**

Bellman-Ford's guarantee is that after round i every shortest path with $\leq i$ edges is found; how *fast* it converges in practice depends on the edge order. Here the order is the reverse of the path $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$, the worst case.

- Round 1: $C \rightarrow D$, $B \rightarrow C$, $A \rightarrow B$ skipped (tails at ∞); $S \rightarrow A$: $A = 4$; $S \rightarrow C$: $C = 9$.
- Round 2: $C \rightarrow D$: $D = 10$; $B \rightarrow C$ skipped ($B = \infty$); $A \rightarrow B$: $B = 2$.
- Round 3: $B \rightarrow C$: $C = \min(9, 2+3) = 5$ ($C \rightarrow D$ was already processed this round).
- Round 4: $C \rightarrow D$: $D = \min(10, 5+1) = 6$.
- Round 5: no change → converged.

Final: $A = 4$, $B = 2$, $C = 5$, $D = 6$. The last change happens in round **4** = $|V| - 1$, the maximum possible.

With the order $S \rightarrow A$, $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, $S \rightarrow C$, everything would be final after round 1.

Trap: thinking the negative edge $A \rightarrow B$ forces extra rounds — what matters is how many edges the shortest path has and whether they are relaxed in path order.

Distances at the end of each round

	S	A	B	C	D
init	0	∞	∞	∞	∞
round 1	0	4	∞	9	∞
round 2	0	4	2	9	10
round 3	0	4	2	5	10
round 4	0	4	2	5	6

Q.7 [MCQ · 1 mark · Hashing — double hashing]**Answer: (C)**

Double hashing uses a key-dependent step $h_2(k)$, so keys with the same home slot follow different probe sequences (no secondary clustering).

- 27: $h_1 = 1 \rightarrow$ slot **1**.
- 40: $h_1 = 1$ (taken), $h_2 = 1 + 7 = 8 \rightarrow 9 \rightarrow$ slot **9**.
- 53: $h_1 = 1$, $h_2 = 1 + 9 = 10 \rightarrow 11 \rightarrow$ slot **11**.

- 14: $h_1 = 1$, $h_2 = 1 + 3 = 4 \rightarrow 5 \rightarrow$ slot **5**.
- 66: $h_1 = 1$, $h_2 = 1 + 0 = 1 \rightarrow 2 \rightarrow$ slot **2**.
- 92: $h_1 = 1$, $h_2 = 1 + 4 = 5 \rightarrow 6 \rightarrow$ slot **6**.
- 48: $h_1 = 48 \bmod 13 = 9$ (taken by 40); $h_2 = 1 + (48 \bmod 11) = 5$. Probes: $9, 14 \bmod 13 = 1$ (27), $19 \bmod 13 = 6$ (92), $24 \bmod 13 = 11$ (53), $29 \bmod 13 = 3 \rightarrow$ empty.

48 is stored in slot **3** after 5 probes $\rightarrow$ option (C).

- (A) 10 is what linear probing (step 1) would give from slot 9.
- (B) 0 comes from forgetting the '1 +' in h_2 (step 4): $9 \rightarrow 13 \bmod 13 = 0$.
- (D) 6 overlooks that 92 already occupies slot 6.

Tip: since 13 is prime, every step 1 ... 12 visits all slots, so double hashing always finds an empty slot when one exists.

0	
1	27
2	66
3	48
4	
5	14
6	92
7	
8	
9	40
10	
11	53
12	

Final table (48 lands in slot 3)

Q.8 [MCQ · 1 mark · Binary trees — reconstruction from traversals]

Answer: (B)

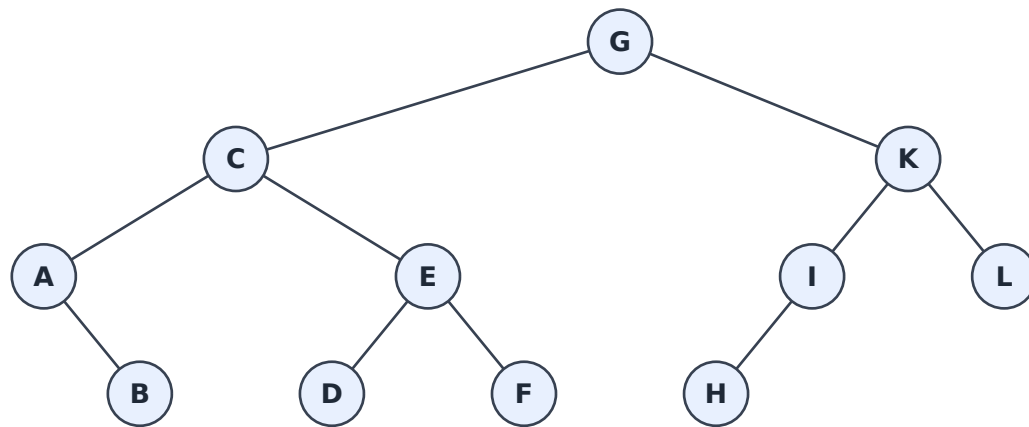
The first pre-order symbol is the root; its position in the in-order sequence splits the left and right subtrees. Recurse.

- Root G; in-order left = A B C D E F, right = H I K L.
- Left subtree: pre-order C A B E D F $\rightarrow$ root C; left {A, B}, right {D, E, F}.
- {A, B}: pre-order A B $\rightarrow$ root A, B is its **right** child (B follows A in-order).
- {D, E, F}: pre-order E D F $\rightarrow$ root E with children D and F.
- Right subtree: pre-order K I H L $\rightarrow$ root K; left {H, I}, right {L}.
- {H, I}: pre-order I H $\rightarrow$ root I, H its **left** child.

Post-order (left, right, root): B A | D F E | C | H I | L | K | G $\rightarrow$ **B A D F E C H I L K G** $\rightarrow$ option (B).

- (A) swaps H and I (treats H as I's parent).
- (C) puts A before B (treats B as A's parent).
- (D) visits E before F (in-order inside E's subtree).

Tip: pre-order + in-order (or post-order + in-order) determine a binary tree uniquely; pre-order + post-order alone do not when a node has a single child (A and I here).



Reconstructed tree

Q.9 [MSQ · 1 mark · Elementary sorts — swaps and passes]**Answer: (A), (B), (D)**

Bubble-sort swaps and insertion-sort shifts each remove exactly **one inversion**, so both equal the inversion count.

Inversions of [5, 2, 8, 1, 9, 3]: (5,2), (5,1), (5,3), (2,1), (8,1), (8,3), (9,3) → **7**.

- (A) **True** — pass 1: (5,2) swap → 2 5 8 1 9 3; (5,8) ok; (8,1) swap → 2 5 1 8 9 3; (8,9) ok; (9,3) swap → 2 5 1 8 3 9. The largest element bubbles to the end.
- (B) **True** — 7 shifts (one per inversion).
- (C) **False** — selection sort: i=0 min 1 at index 3 → swap → [1, 2, 8, 5, 9, 3]; i=1 min 2 already in place; i=2 min 3 at index 5 → swap → [1, 2, 3, 5, 9, 8]; i=3 5 in place; i=4 min 8 at index 5 → swap → [1, 2, 3, 5, 8, 9]. Only **3** swaps.
- (D) **True** — 7 swaps.

Trap: selection sort's swap count is at most $n - 1$ and is *not* related to inversions; it is the number of elements not in place minus the number of cycles in the permutation.

Q.10 [MCQ · 1 mark · Python — aliasing in adjacency lists]**Answer: (C)**

`dict.fromkeys(keys, value)` stores the **same** object as the value of every key. Call the shared list L.

- `G["A"].append("B")` mutates `L` → `L = ['B']`, seen through A, B and C.
- `G["B"] = G["B"] + ["C"]`: `+` builds a **new** list `['B', 'C']` and rebinds only key B. L is unchanged.
- `G["C"] += ["A"]`: for lists `+=` is in-place **extend**, so L itself becomes `['B', 'A']`; then `G['C']` is re-assigned to the same L.

A and C still share `L = ['B', 'A']`; B has its own `['B', 'C']`. Output → option (C).

- (A) assumes each key got its own empty list.
- (B) assumes `+` also mutates in place.
- (D) forgets that A aliases the list extended through C.

Fix: `G = {v: [] for v in "ABC"}` creates a fresh list per key (or use `collections.defaultdict(list)`).

Trap: `x = x + y` and `x += y` are *not* equivalent for mutable sequences.

Q.11 [MCQ · 2 marks · Dijkstra — shortest-path tree with ties]**Answer: (D)**

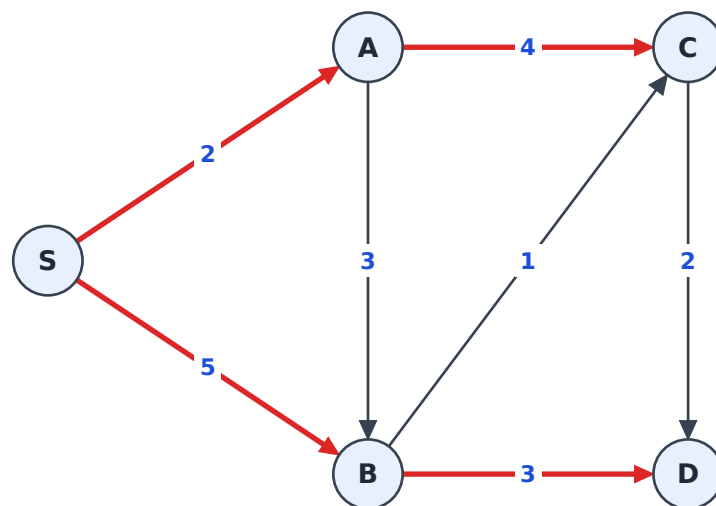
Every vertex here has **two** shortest paths, so the tree is decided purely by the update rule and the extraction order.

- Extract S (0): A = 2 (parent S), B = 5 (parent S).
- Extract A (2): B via A = 2 + 3 = 5 — **not** < 5, so B keeps parent S. C = 2 + 4 = 6 (parent A).
- Extract B (5): C via B = 5 + 1 = 6 — not < 6, keeps parent A. D = 5 + 3 = 8 (parent B).
- Extract C (6): D via C = 6 + 2 = 8 — not < 8, keeps parent B.
- Extract D (8).

Tree: {S→A, S→B, A→C, B→D} → option (D). Distances: A 2, B 5, C 6, D 8.

- (A) is the tree produced by a $\leq$ rule, where every later equal-cost offer overwrites the parent: B←A, then C←B, then D←C.
- (B) mixes both mistakes.
- (C) assumes D is first reached from C — but B is extracted before C.

Tip: with ties, 'the' shortest-path tree is not unique; the exam must specify the update rule. With strict '<' the parent is the **first** vertex extracted that offers the optimal distance.



Shortest-path tree (strict-improvement rule)

Q.12 [MSQ · 2 marks · Bellman-Ford — round-by-round trace]**Answer: (A), (B)**

Trace (only changes are listed):

- **Round 1:** C→D, B→D, B→C, A→C, A→B skipped (tails at ∞); S→B: B = 8; S→A: A = 5; D→A skipped. End: A 5, B 8, C ∞ , D ∞ .
- **Round 2:** C→D skipped; B→D: D = 14; B→C: C = 11; A→C: 12 > 11; A→B: B = 5 - 4 = 1. End: A 5, B 1, C 11, D 14.
- **Round 3:** C→D: D = 9; B→D: D = 1 + 6 = 7; B→C: C = 1 + 3 = 4. End: A 5, B 1, C 4, D 7.
- **Round 4:** C→D: D = 4 - 2 = 2. End: A 5, B 1, C 4, D 2.
- **Check round:** D→A gives 2 + 4 = 6 > 5; nothing improves → no negative cycle.

Verdicts:

- (A) **True** — B is first set to 8 in round 1 (the improvement A→B comes later in the order than S→A, so it must wait for round 2).
- (B) **True** — shortest path S→A→B→C→D = 5 - 4 + 3 - 2 = 2.

- (C) **False** — the only cycles $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ (+1), $A \rightarrow C \rightarrow D \rightarrow A$ (+9) and $A \rightarrow B \rightarrow D \rightarrow A$ (+6) are all positive.
- (D) **False** — $d(D) = 14$ after round 2; 7 appears only in round 3.

Trap: relaxing in a 'natural' order (S's edges first) would converge in one or two rounds. The given order relaxes S's edges *last*, so the improvement travels only one edge of $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$ per round — always trace with the order stated.

Distances at the end of each round

	S	A	B	C	D
init	0	∞	∞	∞	∞
round 1	0	5	8	∞	∞
round 2	0	5	1	11	14
round 3	0	5	1	4	7
round 4	0	5	1	4	2

Q.13 [NAT · 2 marks · Dijkstra on a negative edge — error analysis]

Answer: 3

Dijkstra's run:

- Extract S (0): $A = 2$, $B = 5$.
- Extract A (2): $C = 2 + 3 = 5$.
- Tie $B = C = 5 \rightarrow$ extract B (alphabetical): $B \rightarrow A$ would give 1, but A is already final $\rightarrow$ ignored. $T = 5 + 9 = 14$.
- Extract C (5): $T = \min(14, 5 + 1) = 6$.
- Extract T (6).

Reported: $r(A) = 2$, $r(B) = 5$, $r(C) = 5$, $r(T) = 6$.

True distances (the graph has no cycle, so just take the best path):

- $\delta(B) = 5$; $\delta(A) = \min(2, 5 - 4) = 1$ ($S \rightarrow B \rightarrow A$);
- $\delta(C) = 1 + 3 = 4$; $\delta(T) = \min(4 + 1, 5 + 9) = 5$.

Differences: A 1, B 0, C 1, T 1 $\rightarrow$ sum = **3**.

The single wrong label at A propagates to every vertex whose shortest path passes through A — Dijkstra never revisits A's out-edges after the improvement.

Trap: fixing only A (sum 1). Even an implementation that *did* lower $d(A)$ to 1 when relaxing $B \rightarrow A$ would leave C and T wrong, because A's edges are not relaxed again.

Q.14 [MCQ · 2 marks · Python — a subtly wrong Dijkstra]

Answer: (B)

Because a vertex is frozen the first time it is *discovered*, each vertex keeps the distance of its **first discovery**, never improved — the code behaves like BFS-order labelling with weights, not like Dijkstra.

- Pop (0, S): A unseen $\rightarrow$ dist A = 1; B unseen $\rightarrow$ dist B = 4. Both pushed.
- Pop (1, A): B already seen $\rightarrow$ the better value $1 + 2 = 3$ is **ignored**; C unseen $\rightarrow$ dist C = $1 + 6 = 7$.
- Pop (4, B): C already seen $\rightarrow 4 + 1 = 5$ ignored.
- Pop (7, C): no edges.

Output: `{'S': 0, 'A': 1, 'B': 4, 'C': 7}` $\rightarrow$ option (B). Dicts print in insertion order (S, A, B, C).

- (A) is the **correct** answer of real Dijkstra ($S \rightarrow A \rightarrow B \rightarrow C = 1 + 2 + 1 = 4$).
- (C) assumes C is improved from B but B itself is not.
- (D) assumes B is fixed but C is not.

Fix: test `if d + w < dist.get(v, inf)` when relaxing, and skip stale heap entries on pop (`if d > dist[u]: continue`) — finalise on *pop*, not on push.

Q.15 [NAT · 2 marks · Counting shortest paths in a weighted graph]

Answer: 6

Run Dijkstra and keep, for each vertex, the number of shortest paths: when a relaxation gives a strictly smaller distance, *replace* the count by the predecessor's count; when it gives an equal distance, *add* the predecessor's count.

- S: d 0, count 1.
- A: d 2 (1). B: d 3 (1). C: d 4 (1).
- D: via A $2 + 3 = 5$, via B $3 + 2 = 5 \rightarrow$ d 5, count $1 + 1 = 2$.
- E: via B $3 + 4 = 7$, via C $4 + 3 = 7 \rightarrow$ d 7, count **2**.
- F: via D $5 + 3 = 8$, via E $7 + 1 = 8 \rightarrow$ d 8, count $2 + 2 = 4$.
- T: via F $8 + 2 = 10$, via D $5 + 5 = 10 \rightarrow$ d 10, count $4 + 2 = 6$.

The six paths: S-A-D-F-T, S-B-D-F-T, S-B-E-F-T, S-C-E-F-T, S-A-D-T, S-B-D-T (all cost 10).

Trap: forgetting the direct edge D→T (answer 4) or counting B→D→F and B→E→F as one because both start with S→B. Paths are different if their edge sequences differ.

Q.16 [MSQ · 2 marks · Shortest paths vs MST; transforming weights]

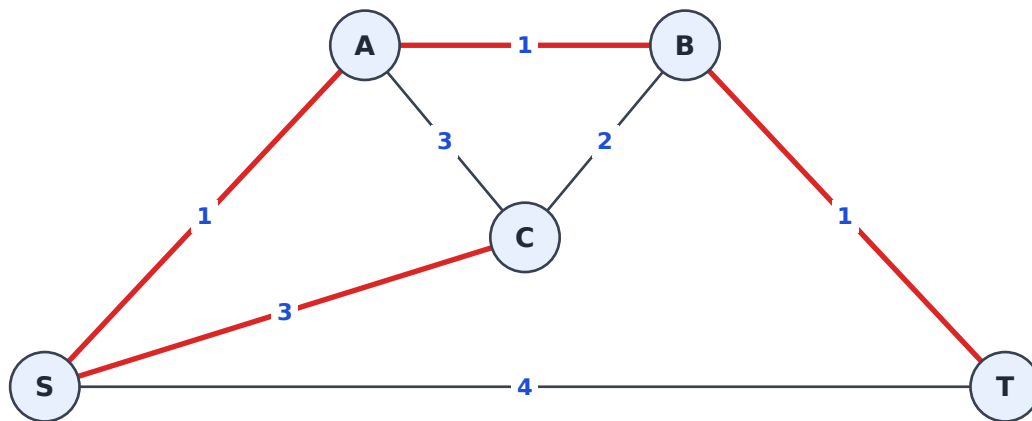
Answer: (A), (C)

Original shortest S-T path: S-A-B-T with cost $1 + 1 + 1 = 3$ (S-T costs 4, S-C-B-T costs $3 + 2 + 1 = 6$).

- (A) **True.** +1 per edge: S-A-B-T (3 edges) $\rightarrow 6$, S-T (1 edge) $\rightarrow 5$, S-C-B-T $\rightarrow 9$. The single-edge path now wins — adding a constant favours paths with fewer edges.
- (B) **False.** Distances from S: A 1, B 2, T 3, C 3 (S-C directly; S-A-C and S-A-B-C both cost 4). Shortest-path tree = {S-A, A-B, B-T, S-C}, weight $1 + 1 + 1 + 3 = 6$. Kruskal picks S-A, A-B, B-T (1 each), then B-C (2) $\rightarrow$ MST weight **5**. The SPT is not an MST: it pays 3 for S-C because it optimises distance *from S*, not total weight.
- (C) **True.** Squared weights: S-A-B-T = $1 + 1 + 1 = 3$, S-T = 16, S-C-B-T = $9 + 4 + 1 = 14$, S-A-C-... larger $\rightarrow$ S-A-B-T remains shortest. (Squaring is not order-preserving in general — it penalises heavy edges — but here it does not change the winner.)
- (D) **False.** Doubling scales every path cost by 2 $\rightarrow$ order unchanged (6 vs 8 vs 12).

Answer: (A), (C).

Trap: believing the SPT and MST coincide; they share the edges incident to S only in special cases.



Shortest-path tree from S (weight 6); MST uses B-C instead

Q.17 [NAT · 2 marks · Bellman-Ford — synchronous rounds = hop-bounded paths]**Answer: 8**

With synchronous updates, $d_k(v)$ is exactly the length of the shortest $S \rightarrow v$ walk using **at most k edges** — so the answer is the best $S \rightarrow T$ path with ≤ 3 edges.

- $k = 1$: A 1, B 6.
- $k = 2$: $B = \min(6, 1 + 2) = 3$; $C = \min(1 + 7, 6 + 1) = 7$; $T = 6 + 5 = 11$.
- $k = 3$: $C = \min(7, 3 + 1, 1 + 7) = 4$; $T = \min(11, d_2(B) + 5 = 8, d_2(C) + 1 = 8) = \mathbf{8}$.
- ($k = 4$: $T = \min(8, 4 + 1) = 5$ — the true distance, via the 4-edge path $S \rightarrow A \rightarrow B \rightarrow C \rightarrow T$.)

Check by enumeration of paths with ≤ 3 edges: S-B-T 11, S-A-B-T 8, S-B-C-T 8, S-A-C-T 9 $\rightarrow$ minimum 8.

Trap: using freshly updated values within a round (in-place Bellman-Ford). With a favourable edge order that version could already reach 5 in round 1 or 2; the synchronous version cannot ‘skip ahead’.

 $d_k(v)$ for synchronous rounds

	S	A	B	C	T
$k = 0$	0	∞	∞	∞	∞
$k = 1$	0	1	6	∞	∞
$k = 2$	0	1	3	7	11
$k = 3$	0	1	3	4	8
$k = 4$	0	1	3	4	5

Q.18 [MCQ · 2 marks · Heaps — decrease-key (as used by Dijkstra)]**Answer: (A)**

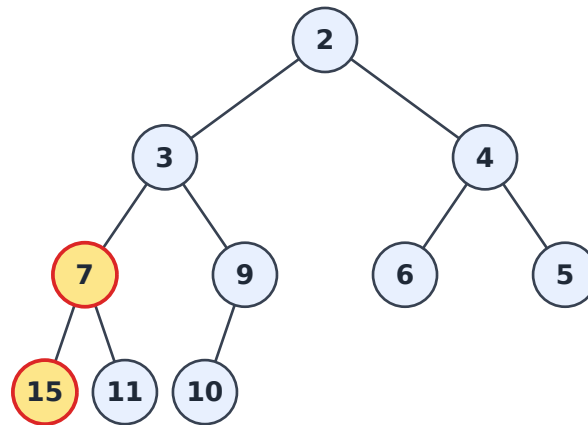
Decrease-key lowers a key, which can only violate heap order with its **ancestors**, so the key moves up: swap with the parent ($((i - 1) // 2)$) while it is smaller.

- Index 8 now holds 3; parent index 3 holds 11 $\rightarrow 3 < 11 \rightarrow$ swap. Array: [2, 7, 4, 3, 9, 6, 5, 15, 11, 10].
- Index 3 holds 3; parent index 1 holds 7 $\rightarrow$ swap. Array: [2, 3, 4, 7, 9, 6, 5, 15, 11, 10].
- Index 1 holds 3; parent index 0 holds 2 $\rightarrow 3 > 2 \rightarrow$ stop.

Result **[2, 3, 4, 7, 9, 6, 5, 15, 11, 10]** $\rightarrow$ option (A). Two swaps — $O(\log n)$ in general, which is why each relaxation in a binary-heap Dijkstra costs $O(\log V)$.

- (B) moves 7 to index 8, which is not a child of index 1 (children of 1 are 3 and 4); the elements on the path must shift down one level each.
- (C) swaps past the root although $2 < 3$.
- (D) stops after the first swap.

Tip: the path traversed is $8 \rightarrow 3 \rightarrow 1 \rightarrow 0$; only elements on this path can move.



After decrease-key (path $8 \rightarrow 3 \rightarrow 1$)

Q.19 [MSQ · 2 marks · Linked lists — merging sorted lists (merge sort step)]

Answer: (A), (C)

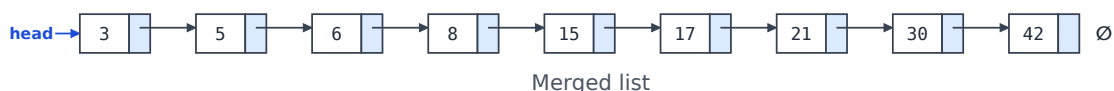
Each comparison outputs exactly one node, and comparisons stop when one list empties. So $\# \text{comparisons} = (m + n) - (\text{number of nodes left in the other list at that moment})$.

Trace:

- $3 \text{ vs } 5 \rightarrow 3$; $8 \text{ vs } 5 \rightarrow 5$; $8 \text{ vs } 6 \rightarrow 6$; $8 \text{ vs } 17 \rightarrow 8$; $15 \text{ vs } 17 \rightarrow 15$; $21 \text{ vs } 17 \rightarrow 17$; $21 \text{ vs } 30 \rightarrow 21$. L_1 is now empty $\rightarrow$ attach $30 \rightarrow 42$.
- Merged: 3, 5, 6, 8, 15, 17, 21, 30, 42.
- (A) **True** — the maximum is $m + n - 1 = 8$ (when the two lists interleave until the last element).
- (B) **False** — the minimum is $\min(m, n) = 4$, e.g. $L_1 = 1, 2, 3, 4$ and $L_2 = 5, \dots, 9$: four comparisons empty L_1 .
- (C) **True** — 7 comparisons (9 nodes – 2 left over).
- (D) **False** — the 5th node is 15 (8 is the 4th).

Merging linked lists needs only $O(1)$ extra space (just pointer relinking), which is why merge sort is the method of choice for sorting linked lists: $\Theta(n \log n)$ time with middle-finding by slow/fast pointers.

Trap: assuming every merge costs $m + n - 1$ comparisons.



Q.20 [NAT · 2 marks · Shortest paths — parametric edge weight]

Answer: 4

Find the best S-T path that **avoids** S-B; the path through S-B must beat it strictly.

Paths avoiding S-B:

- S-A-B-T = $4 + 1 + 3 = 8$ (best)
- S-C-B-T = $2 + 4 + 3 = 9$
- S-A-T = $4 + 6 = 10$
- S-C-T = $2 + 9 = 11$

Paths using S-B: the best is clearly $S-B-T = w + 3$ ($S-B-A-T = w + 7$ and $S-B-C-T = w + 13$ are longer).

Unique shortest path through S-B $\Leftrightarrow w + 3 < 8 \Leftrightarrow w < 5 \Leftrightarrow w \leq 4$ (w integer).

At $w = 5$ the paths S-B-T and S-A-B-T tie at 8 (not unique); for $w \geq 6$ the shortest path avoids S-B.

Trap: comparing only with the obvious two-edge alternatives S-A-T (10) or S-C-T (11), which would give $w \leq 6$. The cheap edge A-B creates the real competitor S-A-B-T. (Running Dijkstra from S with w left symbolic gives the same conclusion: $d(B) = \min(w, 5, 6)$.)

Mock Test 30

Divide & Conquer Applications

Difficulty: GATE-level

Focus areas: inversion counting, max subarray, fast power, D&C recurrences

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — recursive fast exponentiation]

Consider the following Python program. What is printed?

Python

```

1 calls = 0
2 def power(b, e):
3     global calls
4     calls += 1
5     if e == 0:
6         return 1
7     h = power(b, e // 2)
8     return h * h * (b if e % 2 else 1)
9
10 print(power(3, 13) % 100, calls)

```

- (A) 23 4
- (B) 23 5
- (C) 23 13
- (D) 63 5

Q.2 [NAT · numerical answer | 1 mark | Recursion — evaluating a D&C recurrence]

Consider the function below. The value returned by $f(20)$ is _____.

Python

```

1 def f(n):
2     if n <= 1:
3         return 1
4     return f(n // 2) + f(n // 2) + n

```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Recurrences — master theorem (extended case 2)]

The solution of the recurrence $T(n) = 4T(n/2) + n^2 \log n$, $T(1) = 1$, is

- (A) $\Theta(n^2 \log n)$
- (B) $\Theta(n^3)$
- (C) $\Theta(n^2 \log^2 n)$
- (D) $\Theta(n^2 \log \log n)$

Q.4 [MSQ · one or more correct | 1 mark | Maximum subarray — divide and conquer]

The divide-and-conquer maximum-subarray algorithm splits A into the left half $A[0..3]$ and the right half $A[4..7]$, solves both recursively, and also computes the best subarray *crossing* the middle (best suffix of the left half + best prefix of the right half). Which of the following statements is/are TRUE?

$A =$	-3	5	-2	6	-9	4	3	-1
	0	1	2	3	4	5	6	7

Input array

- (A) The maximum subarray sum of A is 9
- (B) The crossing sum computed at the top level is 9
- (C) The recursive call on the left half returns 9
- (D) If every element of A is negated, the maximum subarray sum becomes 9

Q.5 [NAT · numerical answer | 1 mark | Inversion counting]

An inversion in an array A is a pair of indices (i, j) with $i < j$ and $A[i] > A[j]$. The number of inversions in the array below is _____.

$A =$	3	8	1	6	2	7	5	4
	0	1	2	3	4	5	6	7

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Stacks — bracket matching]

Consider the following Python program. What is printed?

Python

```

1 def check(s):
2     st = []
3     pairs = {'(': ')', '[': ']', '{': '}'
4     for i, c in enumerate(s):
5         if c in '([{':
6             st.append(c)
7         elif not st or st.pop() != pairs[c]:
8             return i
9     return -1 if not st else len(s)
10
11 print(check("{[()]}" ), check("([)]" ), check("([)]{}"))
```

- (A) -1 3 6
- (B) -1 2 5
- (C) -1 2 6
- (D) 0 2 6

Q.7 [MSQ · one or more correct | 1 mark | Hashing — quadratic probing coverage]

A hash table of size $m = 8$ uses quadratic probing: probe i ($i = 0, 1, 2, \dots$) for key k examines slot $(h(k) + i^2) \bmod 8$. Which of the following statements is/are TRUE?

- (A) If m were 7 instead, the probe sequence from any home slot would reach exactly 4 distinct slots
- (B) For a key whose home slot is 3, the first four probes examine slots 3, 4, 7, 6
- (C) For a key whose home slot is 0, the slots 2, 3, 5, 6 and 7 are never examined
- (D) An insertion can fail even when only 3 of the 8 slots are occupied

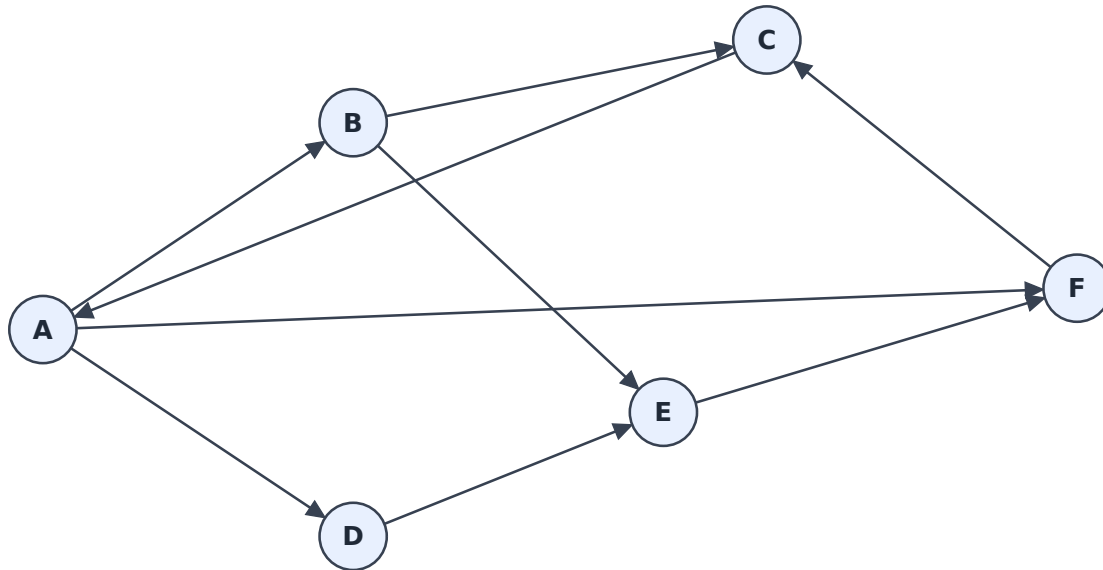
Q.8 [NAT · numerical answer | 1 mark | Counting BSTs (Catalan recurrence)]

The number of structurally distinct binary search trees on the keys 1, 2, 3, 4, 5, 6 whose **root is 3** is _____.

Answer: _____ (enter a numerical value)

Q.9 [MCQ · single correct | 1 mark | DFS — edge classification]

DFS is run on the directed graph below, starting at A; whenever there is a choice, vertices are explored in alphabetical order (all vertices are reachable from A). How are the edges $F \rightarrow C$, $D \rightarrow E$ and $A \rightarrow F$ classified (tree / back / forward / cross)?



- (A) back, cross, forward
- (B) cross, cross, forward
- (C) cross, back, tree
- (D) cross, forward, forward

Q.10 [NAT · numerical answer | 1 mark | Linked lists — recursive merge]

The recursive function below merges two sorted singly linked lists (as in the merge step of merge sort). It is called on the lists $2 \rightarrow 5 \rightarrow 9 \rightarrow 12$ and $3 \rightarrow 4 \rightarrow 10 \rightarrow 15 \rightarrow 20$. The total number of calls made to **merge** (including the first call) is _____.

Python

```

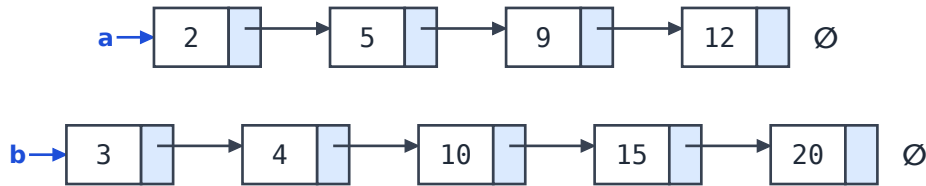
1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 calls = 0
6 def merge(a, b):
7     global calls
8     calls += 1
9     if a is None: return b
10    if b is None: return a
11    if a.v <= b.v:
12        a.nxt = merge(a.nxt, b)
13        return a
14    b.nxt = merge(a, b.nxt)
15    return b

```

```

17 def build(xs):
18     h = None
19     for x in reversed(xs): h = Node(x, h)
20     return h
21
22 h = merge(build([2, 5, 9, 12]), build([3, 4, 10, 15, 20]))

```



Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Inversion counting via merge sort — a buggy count]

The following function is intended to count inversions while merge-sorting, but the line marked (*) is wrong. What does the program print?

Python

```

1 def sort_count(a):
2     if len(a) <= 1:
3         return a, 0
4     m = len(a) // 2
5     L, x = sort_count(a[:m])
6     R, y = sort_count(a[m:])
7     out, i, j, c = [], 0, 0, x + y
8     while i < len(L) and j < len(R):
9         if L[i] <= R[j]:
10            out.append(L[i]); i += 1
11        else:
12            out.append(R[j]); j += 1
13            c += 1 # (*)
14    out += L[i:] + R[j:]
15    return out, c
16
17 print(sort_count([4, 1, 7, 3, 6, 2, 5])[1])

```

- (A) 7
- (B) 6
- (C) 10
- (D) 21

Q.12 [NAT · numerical answer | 2 marks | Quicksort — comparison count]

The functional quicksort below uses the first element as pivot. Assume each call on a list of length $n \geq 2$ compares every one of the other $n - 1$ elements with the pivot exactly once (count only the $x < p$ tests). For the input $[6, 3, 9, 1, 8, 2, 7, 4, 5]$, the total number of such comparisons is _____.

Python

```

1 def qs(a):
2     if len(a) <= 1:
3         return a
4     p = a[0]
5     left = [x for x in a[1:] if x < p]
6     right = [x for x in a[1:] if x >= p]
7     return qs(left) + [p] + qs(right)

```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Recurrences — asymptotic solutions]

Which of the following recurrence/solution pairs is/are correct? ($T(1) = \Theta(1)$ in each case.)

- (A) $T(n) = T(\sqrt{n}) + 1 \Rightarrow T(n) = \Theta(\log \log n)$
- (B) $T(n) = 7T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^3)$
- (C) $T(n) = 2T(n/2) + n \log n \Rightarrow T(n) = \Theta(n \log^2 n)$
- (D) $T(n) = T(n/2) + T(n/4) + n \Rightarrow T(n) = \Theta(n)$

Q.14 [NAT · numerical answer | 2 marks | D&C counting — reverse pairs]

A *reverse pair* in an array A is a pair of indices (i, j) with $i < j$ and $A[i] > 2 \cdot A[j]$. Such pairs can be counted in $\Theta(n \log n)$ by a merge-sort-style divide and conquer. The number of reverse pairs in the array below is _____.

$A =$	7	2	9	3	1	8	4	6
	0	1	2	3	4	5	6	7

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Fast exponentiation — counting multiplications]

Consider the following Python program. What is printed?

Python

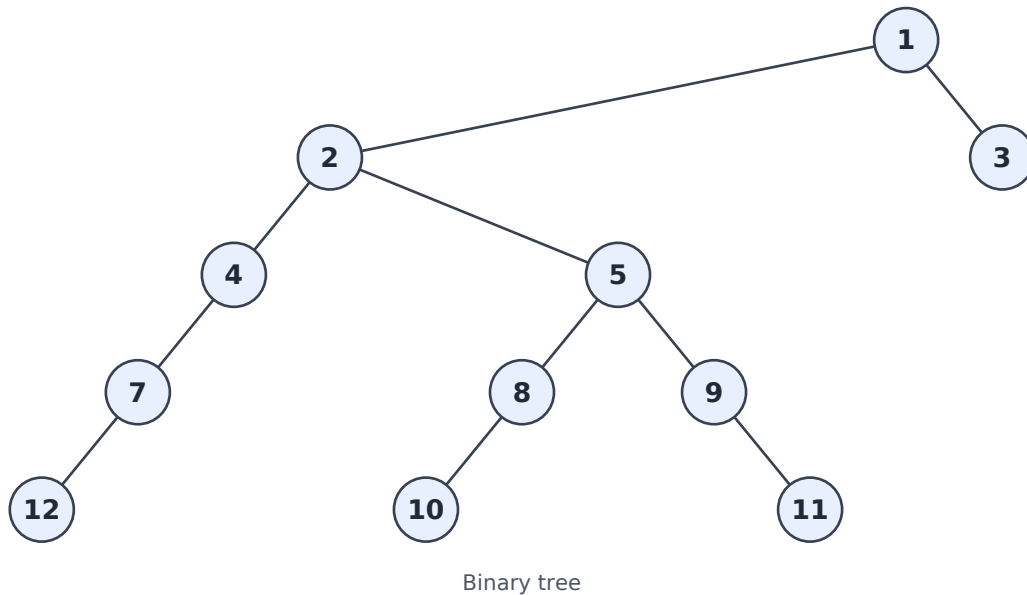
```

1 m = 0
2 def pw(x, n):
3     global m
4     r = 1
5     while n:
6         if n & 1:
7             r *= x; m += 1
8         x *= x; m += 1
9         n >>= 1
10    return r
11
12 v = pw(2, 55)
13 print(m, v == 2 ** 55)
```

- (A) 54 True
- (B) 10 True
- (C) 9 True
- (D) 11 True

Q.16 [NAT · numerical answer | 2 marks | Trees — diameter by divide and conquer]

The diameter of a binary tree is the number of **edges** on the longest path between any two nodes. It can be computed recursively as $\max(\text{diam}(L), \text{diam}(R), h(L) + h(R) + 2)$, where $h(\text{empty}) = -1$. The diameter of the tree below is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Quickselect — Lomuto partition]

The function below returns the k-th smallest element (k is 1-based) using repeated Lomuto partitioning. It is called as `select(A, 4)` with `A = [13, 4, 21, 9, 17, 2, 25, 11, 6, 15]`. Which of the following statements is/are TRUE?

Python

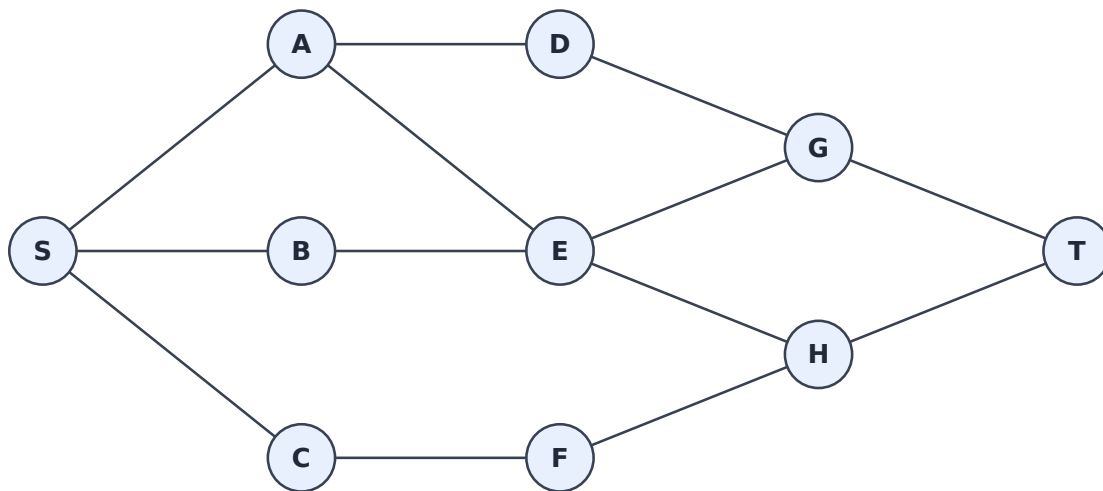
```

1 def select(A, k):
2     lo, hi = 0, len(A) - 1
3     while True:
4         p, i = A[hi], lo - 1
5         for j in range(lo, hi):
6             if A[j] <= p:
7                 i += 1
8                 A[i], A[j] = A[j], A[i]
9         A[i + 1], A[hi] = A[hi], A[i + 1]
10        q = i + 1
11        if q == k - 1:
12            return A[q]
13        if q < k - 1:
14            lo = q + 1
15        else:
16            hi = q - 1
  
```

- (A) After the first partitioning pass, 15 is at index 7
- (B) When the function returns, `A[0..2]` is in ascending order
- (C) The function returns 9
- (D) Exactly 3 partitioning passes are performed

Q.18 [NAT · numerical answer | 2 marks | BFS — counting shortest paths]

In the undirected, unweighted graph below, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Complexity of a D&C function]

What is the asymptotic running time of $g(a, 0, n)$ for a list a of length n ?

Python

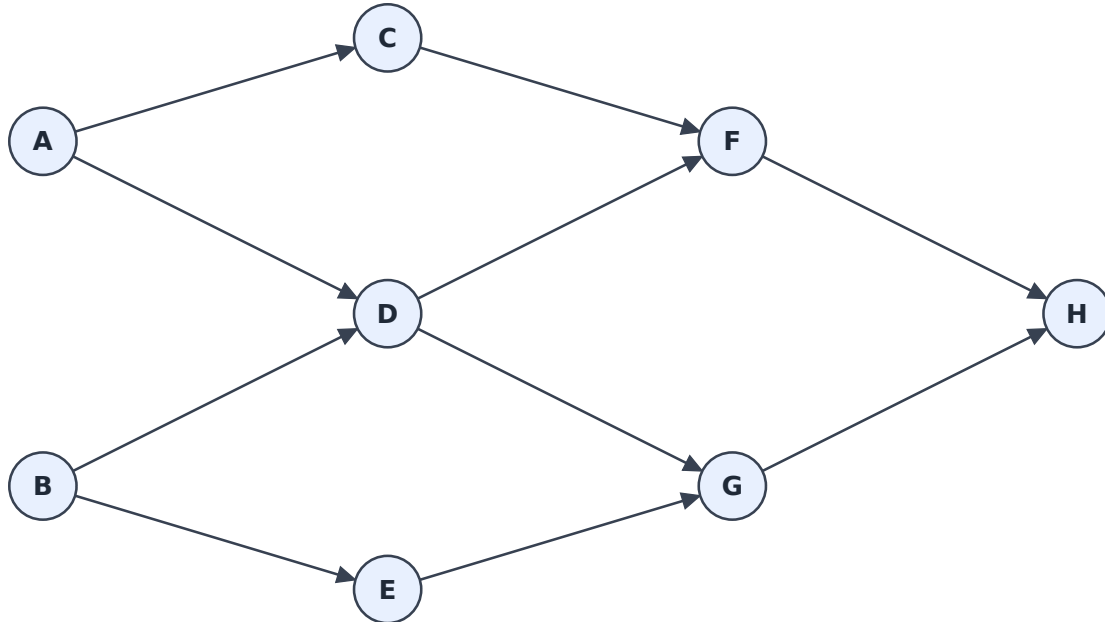
```

1 def g(a, lo, hi):
2     if hi - lo < 4:
3         return sum(a[lo:hi])
4     s = 0
5     for i in range(lo, hi):
6         s += a[i]
7     m = (lo + hi) // 2
8     q = (hi - lo) // 4
9     return (s + g(a, lo, m) + g(a, m, hi)
10            + g(a, lo + q, hi - q))
  
```

- (A) $\Theta(n^{\log_2 3})$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n^2)$
- (D) $\Theta(n \log^2 n)$

Q.20 [MSQ · one or more correct | 2 marks | DFS on a DAG — finish times and topological order]

DFS is run on the DAG below. The outer loop tries start vertices in alphabetical order, and each vertex's out-neighbours are explored in alphabetical order. A global clock starts at 1 and is incremented at every discovery and every finish (so A is discovered at time 1). Let $f(v)$ denote the finish time of v . Which of the following statements is/are TRUE?



- (A) Listing the vertices in decreasing order of f gives B, E, A, D, G, C, F, H
- (B) The edge $B \rightarrow D$ is a cross edge
- (C) The edge $A \rightarrow D$ is a forward edge
- (D) $f(A) = 11$

Answer Key — Mock Test 30

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — recursive fast exponentiation
2	NAT	1	92	Recursion — evaluating a D&C recurrence
3	MCQ	1	(C)	Recurrences — master theorem (extended case 2)
4	MSQ	1	(A), (C), (D)	Maximum subarray — divide and conquer
5	NAT	1	14	Inversion counting
6	MCQ	1	(C)	Stacks — bracket matching
7	MSQ	1	(A), (C), (D)	Hashing — quadratic probing coverage
8	NAT	1	10	Counting BSTs (Catalan recurrence)
9	MCQ	1	(B)	DFS — edge classification
10	NAT	1	8	Linked lists — recursive merge
11	MCQ	2	(A)	Inversion counting via merge sort — a buggy count
12	NAT	2	17	Quicksort — comparison count
13	MSQ	2	(A), (C), (D)	Recurrences — asymptotic solutions
14	NAT	2	7	D&C counting — reverse pairs
15	MCQ	2	(D)	Fast exponentiation — counting multiplications
16	NAT	2	6	Trees — diameter by divide and conquer

Q	Type	Marks	Key	Topic
17	MSQ	2	(C), (D)	Quickselect — Lomuto partition
18	NAT	2	6	BFS — counting shortest paths
19	MCQ	2	(A)	Complexity of a D&C function
20	MSQ	2	(A), (B)	DFS on a DAG — finish times and topological order

Detailed Solutions — Mock Test 30

Q.1 [MCQ · 1 mark · Python — recursive fast exponentiation]

Answer: (B)

Concept. Exponentiation by squaring halves the exponent at every call: $b^e = (b^{\lfloor e/2 \rfloor})^2 \cdot b^{e \bmod 2}$. The number of calls is the number of halvings to reach 0, i.e. $\lfloor \log_2 e \rfloor + 2$ for $e \geq 1$.

Call chain for $e = 13$: $13 \rightarrow 6 \rightarrow 3 \rightarrow 1 \rightarrow 0$ — that is **5** calls (the call with $e = 0$ counts).

Values on the way back up:

- $e=0$: 1
- $e=1$: $1 \cdot 1 \cdot 3 = 3$
- $e=3$: $3 \cdot 3 \cdot 3 = 27$
- $e=6$: $27 \cdot 27 = 729$
- $e=13$: $729 \cdot 729 \cdot 3 = 1\,594\,323 \rightarrow \bmod 100 = \mathbf{23}$.

Output **23 5** $\rightarrow$ (B).

Options. (A) forgets the base-case call with $e = 0$. (D) gets the remainder wrong (63 would come from 3^{12} ... mixing up the odd factor). (C) confuses the call count with the exponent — that is the *naïve* method's cost.

Tip: the recursion depth, and the number of multiplications, are $\Theta(\log e)$.

Q.2 [NAT · 1 mark · Recursion — evaluating a D&C recurrence]

Answer: 92

Concept. f satisfies $f(n) = 2 f(\lfloor n/2 \rfloor) + n$ with $f(1) = 1$. For powers of two this gives $n \log_2 n + n$, but for other n the floors matter, so evaluate bottom-up along the chain $20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$.

- $f(1) = 1$
- $f(2) = 2 \cdot f(1) + 2 = 4$
- $f(5) = 2 \cdot f(2) + 5 = 13$
- $f(10) = 2 \cdot f(5) + 10 = 36$
- $f(20) = 2 \cdot f(10) + 20 = \mathbf{92}$

Trap: plugging into the closed form $n \log_2 n + n$ with $\log_2 20 \approx 4.32$ gives ≈ 106.4 — the closed form only holds exactly for powers of two. Another slip is $f(5) = 2 \cdot f(2.5)$: integer division gives $f(2)$.

Q.3 [MCQ · 1 mark · Recurrences — master theorem (extended case 2)]

Answer: (C)

Concept. Compare $f(n)$ with $n^{\log_b a}$. Here $a = 4$, $b = 2$, so $n^{\log_2 4} = n^2$. $f(n) = n^2 \log n = \Theta(n^{\log_b a} \cdot \log^k n)$ with $k = 1$ — the *extended case 2* of the master theorem, giving $\Theta(n^{\log_b a} \cdot \log^{k+1} n)$.

Direct check by unrolling: level i of the recursion tree has 4^i subproblems of size $n/2^i$, each costing $(n/2^i)^2 \log(n/2^i)$; the level total is $n^2 (\log n - i)$. Summing over $i = 0 \dots \log n$ gives $n^2 \cdot \sum (\log n - i) = n^2 \cdot \Theta(\log^2 n)$.

Hence $T(n) = \mathbf{\Theta(n^2 \log^2 n)} \rightarrow$ (C).

Options. (A) is what you get by ignoring the log factor in f (case 2 with $k = 0$). (B) would need f to be polynomially larger than n^2 . (D) arises for $f(n) = n^2/\log n$ ($k = -1$), not $n^2 \log n$.

Trap: $n^2 \log n$ is *not* polynomially larger than n^2 , so case 3 does not apply.

Q.4 [MSQ · 1 mark · Maximum subarray — divide and conquer]

Answer: (A), (C), (D)

Concept. answer = max(left best, right best, crossing best).

- Left half $[-3, 5, -2, 6]$: best is $5 - 2 + 6 = 9$.
- Right half $[-9, 4, 3, -1]$: best is $4 + 3 = 7$.
- Crossing: suffix sums of the left half (ending at index 3) are 6, 4, 9, 6 $\rightarrow$ best 9; prefix sums of the right half (starting at index 4) are $-9, -5, -2, -3 \rightarrow$ best -2 . Crossing $= 9 - 2 = 7$.
- Overall $= \max(9, 7, 7) = 9$.

Option by option:

- (A) **TRUE** (subarray 5, -2 , 6).
- (B) The crossing sum is 7, not 9 — the crossing subarray *must* include $A[4] = -9$. **FALSE**.
- (C) The left call returns 9. **TRUE**.
- (D) Negated array: 3, -5 , 2, -6 , 9, -4 , -3 , 1. The single element 9 is best ($9 - 4 < 9$, and extending left adds -6). **TRUE**.

Trap: the crossing part must contain *at least one* element from each side, so its best prefix can be negative.

Q.5 [NAT · 1 mark · Inversion counting]

Answer: 14

Concept. Count, for each element, how many *later* elements are smaller (or, equivalently, how many earlier elements are larger). Merge-sort counting does this in $\Theta(n \log n)$; for $n = 8$ a direct count is quick.

Smaller elements to the right of each position:

- 3: $\{1, 2\} \rightarrow 2$
- 8: $\{1, 6, 2, 7, 5, 4\} \rightarrow 6$
- 1: none $\rightarrow 0$
- 6: $\{2, 5, 4\} \rightarrow 3$
- 2: none $\rightarrow 0$
- 7: $\{5, 4\} \rightarrow 2$
- 5: $\{4\} \rightarrow 1$
- 4: $\rightarrow 0$

Total $= 2 + 6 + 0 + 3 + 0 + 2 + 1 + 0 = 14$.

Cross-check: insertion sort on this array would perform exactly 14 shifts, and bubble sort exactly 14 swaps.

Trap: counting only *adjacent* out-of-order pairs (here 4) — inversions include all pairs.

Q.6 [MCQ · 1 mark · Stacks — bracket matching]

Answer: (C)

Concept. The function returns -1 for a balanced string, the index of the first closing bracket that does not match, or $\text{len}(s)$ if openers remain unmatched at the end.

- $\{[(())]\}$: every closer matches the most recent opener; stack ends empty $\rightarrow -1$.
- $\{([)]\}$: push $($, $[$; at $i = 2$ the closer $)$ pops $[\neq (\rightarrow$ return **2**.
- $\{(()[]\}$: pairs $()$ and $[]$ match, but $($ (index 0) and $\{$ remain on the stack at the end $\rightarrow$ return $\text{len}(s) = 6$.

Output $-1 \ 2 \ 6 \rightarrow$ (C).

Options. (A) reports the index of the *second* mismatching bracket. (B) returns the index of the last character instead of $\text{len}(s)$. (D) treats the first string as unbalanced.

Trap: in `not st or st.pop() != ...`, short-circuiting matters: when the stack is empty `pop()` is never called, so no `IndexError` occurs.

Q.7 [MSQ · 1 mark · Hashing — quadratic probing coverage]**Answer: (A), (C), (D)**

Concept. The slots reachable from home h are $h + (i^2 \bmod m)$. The set of squares modulo m decides how much of the table is reachable.

Squares mod 8: 0, 1, 4, 1, 0, 1, 4, 1, ... $\rightarrow$ only offsets **{0, 1, 4}**.

- (A) Squares mod 7: 0, 1, 4, 2, 2, 4, 1 $\rightarrow$ {0, 1, 2, 4}, i.e. $(7 + 1)/2 = 4$ distinct offsets. **TRUE.**
- (B) Offsets for $i = 0, 1, 2, 3$ are 0, 1, 4, $9 \bmod 8 = 1 \rightarrow$ slots 3, 4, 7, **4**, not 6. **FALSE.**
- (C) From home 0 only slots 0, 1, 4 are ever probed; 2, 3, 5, 6, 7 never. **TRUE.**
- (D) If slots 0, 1, 4 are occupied, a key hashing to 0 can never be placed, although 5 slots are free. **TRUE.**

Tip: for a prime m , quadratic probing reaches exactly $\lceil m/2 \rceil$ slots, which guarantees an insertion succeeds whenever the table is at most half full.

Q.8 [NAT · 1 mark · Counting BSTs (Catalan recurrence)]**Answer: 10**

Concept. Fixing the root splits the problem (divide and conquer): keys smaller than the root form the left subtree, larger keys the right subtree, independently. The number of BSTs on k keys is the Catalan number C_k : 1, 1, 2, 5, 14, ...

- Left subtree: keys {1, 2} $\rightarrow C_2 = 2$ shapes.
- Right subtree: keys {4, 5, 6} $\rightarrow C_3 = 5$ shapes.

Total = $2 \times 5 = 10$.

Summing $C_{r-1} \cdot C_{6-r}$ over all six roots r gives $42 + 14 + 10 + 10 + 14 + 42 = 132 = C_6$, the total number of BSTs on six keys.

Trap: adding instead of multiplying ($2 + 5 = 7$); the two subtrees are chosen independently, so the counts multiply.

Q.9 [MCQ · 1 mark · DFS — edge classification]**Answer: (B)**

Concept. For an edge $u \rightarrow v$ met while scanning u : v undiscovered $\rightarrow$ tree; v discovered but unfinished (on the recursion stack) $\rightarrow$ back; v finished and discovered *after* $u \rightarrow$ forward; v finished and discovered *before* $u \rightarrow$ cross.

DFS with (discovery/finish) times:

- A 1 $\rightarrow$ B 2 $\rightarrow$ C 3: C $\rightarrow$ A is a back edge; C finishes 4.
- B $\rightarrow$ E 5 $\rightarrow$ F 6: F $\rightarrow$ C — C is already finished and was discovered (3) before F (6) $\rightarrow$ **cross**. F finishes 7, E 8, B 9.
- A $\rightarrow$ D 10: D $\rightarrow$ E — E finished (8), discovered at $5 < 10 \rightarrow$ **cross**. D finishes 11.
- A $\rightarrow$ F: F is finished and was discovered (6) *after* A (1) $\rightarrow$ **forward**. A finishes 12.

Answer: cross, cross, forward $\rightarrow$ (B).

Options. (A) mislabels F $\rightarrow$ C as back, but C had already finished. (C) and (D) mislabel D $\rightarrow$ E: E is not an ancestor of D (back) nor a descendant (forward).

Trap: an edge into an *already finished* vertex is never a back edge; back edges exist iff the graph has a cycle (here C $\rightarrow$ A closes A $\rightarrow$ B $\rightarrow$ C).

Q.10 [NAT · 1 mark · Linked lists — recursive merge]**Answer: 8**

Concept. Each non-base call places exactly one node (the smaller front) and recurses on the rest; the recursion stops at the first call where one list is empty, and the remainder of the other list is attached in $O(1)$.

Nodes placed, one per call: 2, 3, 4, 5, 9, 10, 12 — that is 7 calls. After 12 is placed, the next call is `merge(None, 15 → 20)`, which returns immediately: call **8**.

Total calls = $7 + 1 = 8$. The merged list is 2 3 4 5 9 10 12 15 20.

In general, calls = (number of nodes placed before one list runs out) + 1, which is between $\min(p, q) + 1$ and $p + q$.

Trap: answering $p + q = 9$ (assuming every node triggers a call) or forgetting the final base-case call (7).

Q.11 [MCQ · 2 marks · Inversion counting via merge sort — a buggy count]**Answer: (A)**

Concept. When $R[j]$ is output before $L[i]$, $R[j]$ is smaller than *all* remaining left elements $L[i..]$, so the correct update is `c += len(L) - i`. The buggy line adds only 1, i.e. it counts how many right-run elements are output while the left run is non-empty.

Recursion on [4, 1, 7, 3, 6, 2, 5] (split 3 | 4):

- [4] | [1, 7] (inner merge [1][7]: 0): merging [4] with [1, 7] outputs 1 before 4 → +1. Left total **1** (true inversions 1).
- [3, 6] (0) | [2, 5] (0): merge outputs 2 (+1), 3, 5 (+1), 6 → **2** (true: (3,2), (6,2), (6,5) = 3).
- Top merge [1, 4, 7] with [2, 3, 5, 6]: outputs 1, 2 (+1), 3 (+1), 4, 5 (+1), 6 (+1), then 7 → **4** (true cross inversions: 2 and 3 are each below 4 and 7, 5 and 6 below 7 → 6).

Printed value = $1 + 2 + 4 = 7$ → (A). The true inversion count is $1 + 3 + 6 = 10$.

Option by option:

- (A) Correct.
- (B) 6 counts only the top-level cross inversions.
- (C) 10 is what the *correct* code prints.
- (D) $21 = C(7, 2)$ is the total number of pairs.

Trap: the bug is invisible on inputs where each left run has a single remaining element when a right element jumps ahead — test on richer inputs.

Q.12 [NAT · 2 marks · Quicksort — comparison count]**Answer: 17**

Concept. A call on n elements costs $n - 1$ comparisons; the sublists keep the *original relative order*, so each sublist's first element is its pivot.

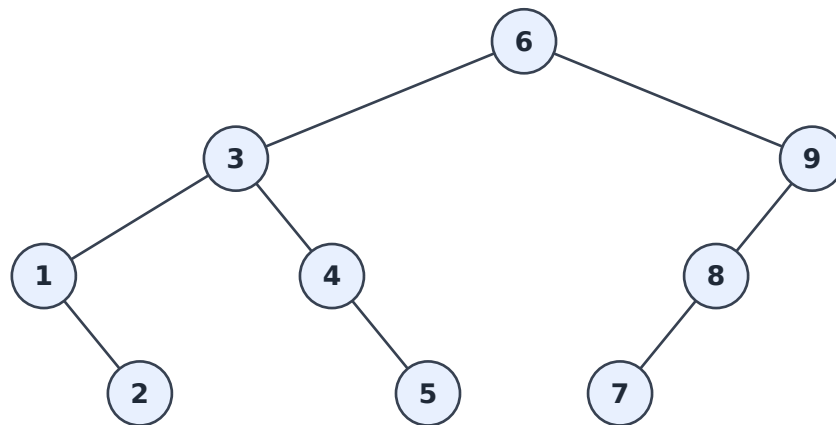
Recursion tree (list → cost):

- [6, 3, 9, 1, 8, 2, 7, 4, 5] → 8; left [3, 1, 2, 4, 5], right [9, 8, 7].
- [3, 1, 2, 4, 5] → 4; left [1, 2], right [4, 5].
- [1, 2] → 1 (right [2]); [4, 5] → 1 (right [5]).
- [9, 8, 7] → 2; left [8, 7] → 1 (left [7]).

Total = $8 + 4 + 1 + 1 + 2 + 1 = 17$.

For comparison: the best case for $n = 9$ (perfectly balanced splits 4 | 4, then 1 | 2) is $8 + 2 \cdot (3 + 1) = 16$, the worst case (sorted input) is $8 + 7 + \dots + 1 = 36$.

Trap: this code actually scans `a[1:]` *twice* (once per comprehension); if both tests were counted the answer would double to 34 — read what is being counted.



Pivots form a BST: the recursion tree of quicksort

Q.13 [MSQ · 2 marks · Recurrences — asymptotic solutions]**Answer: (A), (C), (D)**

- (A) Substitute $n = 2^m$: $S(m) = T(2^m)$ satisfies $S(m) = S(m/2) + 1 \rightarrow S(m) = \Theta(\log m) = \Theta(\log \log n)$. **Correct.**
- (B) $a = 7, b = 2$: $n^{\log_2 7} \approx n^{2.807}$, which dominates $f(n) = n^2$ polynomially $\rightarrow$ case 1 $\rightarrow \Theta(n^{\log_2 7})$, **not** $\Theta(n^3)$. (This is Strassen's recurrence.) **Incorrect.**
- (C) $a = 2, b = 2$, $n^{\log_b a} = n$; $f(n) = n \log n = \Theta(n \cdot \log^1 n) \rightarrow$ extended case 2 $\rightarrow \Theta(n \log^2 n)$. (Each of the $\log n$ levels costs $n \cdot \log(n/2^i)$; the sum is $n \cdot \sum (\log n - i) = \Theta(n \log^2 n)$.) **Correct.**
- (D) Recursion-tree: level costs are $n, (3/4)n, (3/4)^2 n, \dots$ because the subproblem sizes sum to $n/2 + n/4 = 3n/4 < n$. A decreasing geometric series $\rightarrow \Theta(n)$. **Correct.**

Trap: in (D), seeing two recursive calls and guessing $\Theta(n \log n)$ — that needs the sizes to add up to n (as in $T(n/2) + T(n/2)$). In (B), $7 < 8 = 2^3$ is exactly why Strassen beats the cubic algorithm.

Q.14 [NAT · 2 marks · D&C counting — reverse pairs]**Answer: 7**

Concept (D&C). Split into halves; count pairs inside each half recursively; count cross pairs (i in left, j in right) with two pointers over the *sorted* halves; then merge. Because the halves are sorted, the cross count takes linear time.

Top level: left $[7, 2, 9, 3]$, right $[1, 8, 4, 6]$.

- Inside the left half: $(7, 2)? 7 > 4 \checkmark$; $(7, 3)? 7 > 6 \checkmark$; $(9, 3)? 9 > 6 \checkmark$; $(2, \dots), (9, \dots)$ no others $\rightarrow 3$.
- Inside the right half: $(8, 4)? 8 > 8 \times$; $(8, 6)? \times$; 1 is never on the right of a pair since nothing precedes it in this half $\rightarrow 0$.
- Cross pairs with sorted left $[2, 3, 7, 9]$ and sorted right $[1, 4, 6, 8]$: $2 \cdot 1 = 2$ is exceeded by 3, 7, 9 $\rightarrow 3$ pairs; $2 \cdot 4 = 8$ is exceeded by 9 $\rightarrow 1$ pair; $2 \cdot 6 = 12$ and $2 \cdot 8 = 16 \rightarrow 0$. Cross = 4.

Total = $3 + 0 + 4 = 7$: $(7,2), (7,3), (9,3), (7,1), (9,1), (3,1), (9,4)$.

Trap: counting ordinary inversions ($A[i] > A[j]$) instead — that gives a much larger number — or using $\geq$ instead of $>$ (8 vs $2 \cdot 4$ is *not* a reverse pair).

Q.15 [MCQ · 2 marks · Fast exponentiation — counting multiplications]**Answer: (D)**

Concept. Right-to-left binary exponentiation squares x once per bit of n and multiplies it into r once per **1-bit**. Multiplications = (number of bits) + (number of 1-bits).

$55 = 110111_2$: 6 bits, 5 of them are 1.

Iteration trace (n in binary, ops):

- $n=110111$: bit 1 $\rightarrow r*=x, x*=x$ (2)
- $n=11011$: bit 1 $\rightarrow 2$
- $n=1101$: bit 1 $\rightarrow 2$
- $n=110$: bit 0 $\rightarrow$ only $x*=x$ (1)
- $n=11$: bit 1 $\rightarrow 2$
- $n=1$: bit 1 $\rightarrow 2$, then n becomes 0.

Total = $2+2+2+1+2+2 = 11$; and the result is correct, so the program prints **11 True** $\rightarrow$ (D).

Options. (B) and (C) subtract the “wasted” operations (the first $r *= x$ multiplies by 1, and the last squaring is never used) — a smarter implementation would save them, but this code counts them. (A) is the naive method’s $n - 1$.

Tip: the cost is $\Theta(\log n)$ — between $\lfloor \log_2 n \rfloor + 1$ and $2(\lfloor \log_2 n \rfloor + 1)$.

Q.16 [NAT · 2 marks · Trees — diameter by divide and conquer]

Answer: 6

Concept. The longest path bends at some node v , using the deepest leaf of each subtree of v . The recursion returns the height and keeps the best $h(L) + h(R) + 2$ over all nodes.

Heights (edges) bottom-up:

- $h(12) = 0, h(7) = 1, h(4) = 2$.
- $h(10) = 0, h(8) = 1; h(11) = 0, h(9) = 1; h(5) = 2$.
- $h(2) = 1 + \max(2, 2) = 3; h(3) = 0$.

Bend values $h(L) + h(R) + 2$:

- at 5: $1 + 1 + 2 = 4$ (path 10–8–5–9–11)
- at 2: $2 + 2 + 2 = 6$ (path 12–7–4–2–5–8–10)
- at 1 (root): $3 + 0 + 2 = 5$
- others are smaller.

Diameter = **6**, and it does **not** pass through the root.

Trap: computing only $h(\text{left}) + h(\text{right}) + 2$ at the root (5), or counting nodes instead of edges (7).

Q.17 [MSQ · 2 marks · Quickselect — Lomuto partition]

Answer: (C), (D)

Concept. Quickselect partitions once, then continues on **one** side only — the side containing index $k - 1 = 3$. Expected time $\Theta(n)$, worst case $\Theta(n^2)$.

- Pass 1 on $A[0..9]$, pivot 15: elements ≤ 15 are 13, 4, 9, 2, 11, 6 (6 of them) $\rightarrow$ pivot goes to index **6**. $A = [13, 4, 9, 2, 11, 6, \mathbf{15}, 17, 21, 25]$. $6 > 3 \rightarrow hi = 5$.
- Pass 2 on $A[0..5]$, pivot 6: elements ≤ 6 are 4, 2 $\rightarrow$ pivot to index 2. $A[0..5] = [4, 2, \mathbf{6}, 13, 11, 9]$. $2 < 3 \rightarrow lo = 3$.
- Pass 3 on $A[3..5] = [13, 11, 9]$, pivot 9: nothing $\leq 9 \rightarrow$ pivot to index 3 $\rightarrow A[3..5] = [\mathbf{9}, 11, 13]$. $q = 3 = k - 1 \rightarrow$ return **9**.

Option by option:

- (A) 15 lands at index 6 (six smaller elements). **FALSE**.
- (B) $A[0..2] = [4, 2, 6]$ — quickselect only guarantees that these are the elements *smaller* than $A[3]$, not that they are sorted. **FALSE**.
- (C) The 4th smallest of $\{2, 4, 6, 9, 11, \dots\}$ is 9. **TRUE**.
- (D) Three passes. **TRUE**.

Trap: expecting a sorted prefix — selection does strictly less work than sorting.

Q.18 [NAT · 2 marks · BFS — counting shortest paths]**Answer: 6**

Concept. Run BFS from S; the number of shortest paths to v is the sum of the counts of its neighbours one level closer: $\text{cnt}(v) = \sum \text{cnt}(u)$ over edges $u-v$ with $\text{dist}(u) = \text{dist}(v) - 1$.

- Level 0: S (1).
- Level 1: A, B, C — each 1.
- Level 2: $D \leftarrow A: 1$; $E \leftarrow A, B: 1 + 1 = 2$; $F \leftarrow C: 1$.
- Level 3: $G \leftarrow D, E: 1 + 2 = 3$; $H \leftarrow E, F: 2 + 1 = 3$.
- Level 4: $T \leftarrow G, H: 3 + 3 = 6$.

So $\text{dist}(S, T) = 4$ and there are **6** shortest paths, e.g. S-A-D-G-T, S-A-E-G-T, S-B-E-G-T, S-A-E-H-T, S-B-E-H-T, S-C-F-H-T.

Trap: adding counts from *same-level* neighbours, or counting every simple path (which would include longer ones). Note E contributes to both G and H — a classic double-counting check.

BFS levels and path counts

	S	A	B	C	D	E	F	G	H	T
dist	0	1	1	1	2	2	2	3	3	4
count	1	1	1	1	1	2	1	3	3	6

Q.19 [MCQ · 2 marks · Complexity of a D&C function]**Answer: (A)**

Concept. Write the recurrence from the code: the loop costs $\Theta(n)$; there are **three** recursive calls, each on a range of size $\approx n/2$ (the third covers the middle half $[lo + n/4, hi - n/4)$).

$$T(n) = 3T(n/2) + \Theta(n).$$

Master theorem: $a = 3$, $b = 2$, $n^{\log_2 3} \approx n^{1.585}$ dominates $f(n) = n$ polynomially $\rightarrow$ case 1 $\rightarrow \Theta(n^{\log_2 3}) \rightarrow$ (A).

Recursion-tree check: level i has 3^i calls of size $n/2^i$, costing $(3/2)^i \cdot n$ — an increasing geometric series dominated by the leaves: $3^{\log_2 n} = n^{\log_2 3}$.

Options. (B) assumes two calls (merge-sort shape). (C) over-estimates: the leaves are only $n^{1.585}$. (D) has no basis here.

Trap: the third call overlaps the other two, but overlap does not reduce cost — each call re-scans its range. (The base case $hi - lo < 4$ is what guarantees that $q \geq 1$, so the middle call is strictly smaller.) This is the same recurrence as Karatsuba multiplication.

Q.20 [MSQ · 2 marks · DFS on a DAG — finish times and topological order]**Answer: (A), (B)**

Concept. Decreasing finish time of a DFS on a DAG is a topological order. Edge types: tree (first discovery), forward (to an already-finished descendant), cross (to a finished vertex in another branch/tree).

Trace (discovery/finish):

- A 1 $\rightarrow$ C 2 $\rightarrow$ F 3 $\rightarrow$ H 4; H finishes 5, F 6, C 7.
- A $\rightarrow$ D 8; D $\rightarrow$ F (F finished); D $\rightarrow$ G 9; G $\rightarrow$ H (finished); G finishes 10, D 11. A finishes **12**.
- B 13; B $\rightarrow$ D (finished, discovered earlier in another tree); B $\rightarrow$ E 14; E $\rightarrow$ G finished; E finishes 15, B 16.

Finish times: H 5, F 6, C 7, G 10, D 11, A 12, E 15, B 16.

- (A) Decreasing f: B(16), E(15), A(12), D(11), G(10), C(7), F(6), H(5). **TRUE.**

- (B) D was discovered (8) and finished (11) before B was discovered (13) → **cross** edge. **TRUE.**
- (C) When A scans D, D is still undiscovered (A explored C's branch first, which never reaches D) → A→D is a **tree** edge. **FALSE.**
- (D) $f(A) = 12$, not 11 (11 is $f(D)$). **FALSE.**

Trap: assuming that an edge reaching a vertex “late” must be forward; it is forward only if the target was already *discovered from u 's subtree* when u examines it.

Mock Test 31

Full-Syllabus Mock — Paper 1

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — split, join and reversal]

Consider the following Python program. What is printed?

Python

```
1 s = " GATE DA 2027 "
2 parts = s.split()
3 t = "-".join(p[::-1] for p in parts)
4 print(t, len(s.split(" ")))
```

- (A) ETAG-AD-7202 3
- (B) ETAG--AD-7202 6
- (C) 7202-AD-ETAG 7
- (D) ETAG-AD-7202 7

Q.2 [NAT · numerical answer | 1 mark | Python — generator exhaustion]

Consider the following Python program. The value printed is ____.

Python

```
1 g = (x * x for x in range(6) if x % 2)
2 print(sum(g) + sum(g) + max(g, default=-1))
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Queues — queue using two stacks]

A queue is implemented with two stacks IN and OUT. ENQUEUE(x) pushes x onto IN. DEQUEUE pops from OUT; if OUT is empty it first pops **every** element of IN and pushes it onto OUT (each such move is one *transfer*). Starting empty, the operations are:

ENQ 1, ENQ 2, ENQ 3, DEQ, ENQ 4, ENQ 5, DEQ, DEQ, ENQ 6, DEQ.

Which option gives the values returned by the four DEQs and the total number of transfers?

- (A) 1, 2, 3, 4 and 6 transfers
- (B) 1, 2, 3, 4 and 5 transfers
- (C) 3, 2, 1, 6 and 6 transfers
- (D) 1, 2, 4, 5 and 4 transfers

Q.4 [NAT · numerical answer | 1 mark | Circular linked lists — repeated deletion]

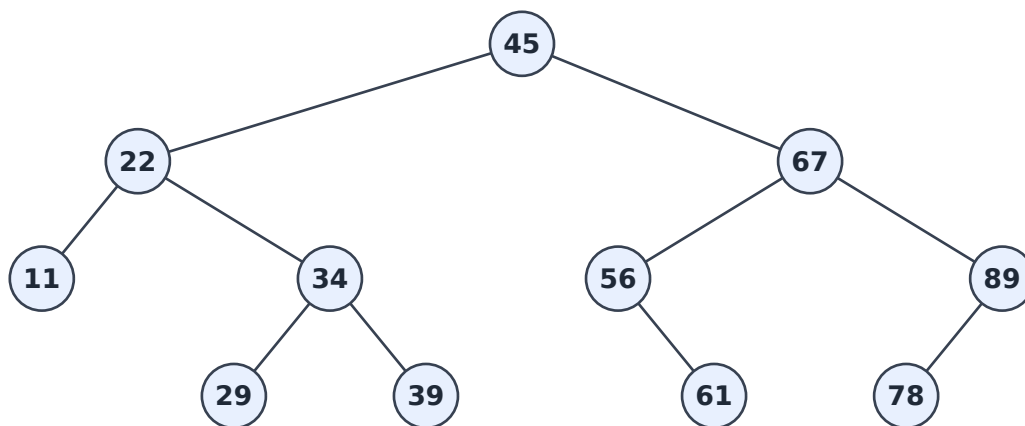
Nine nodes labelled 1 to 9 form the circular singly linked list shown. Starting the count at node 1, the process repeatedly counts 4 nodes (the node where the count starts is counted as 1), deletes the 4th node, and restarts the count at the node following the deleted one, until one node remains. The label of the **fifth** node deleted is _____.



Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | Binary search trees — deletion and successors]

Consider the binary search tree T shown below. Which of the following statements is/are TRUE?



BST T

- (A) If 45 is deleted by replacing it with its in-order predecessor, the new root is 39
- (B) The in-order successor of 61 in T is 78
- (C) After that deletion, node 34 has exactly one child
- (D) T has exactly 4 leaves

Q.6 [MCQ · single correct | 1 mark | Hashing — probability of a collision]

Three keys are inserted into an initially empty hash table with 10 slots. Assume simple uniform hashing (each key independently hashes to each slot with probability $1/10$). The probability that **at least one** collision occurs (i.e. some two keys hash to the same slot) is

- (A) 0.72
- (B) 0.28
- (C) 0.30
- (D) 0.03

Q.7 [MCQ · single correct | 1 mark | Binary search — lower-bound variant]

Consider the following Python program. What is printed?

Python

```
1 def lb(a, x):
2     lo, hi = 0, len(a)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 a = [2, 4, 4, 4, 7, 9, 9, 12]
12 print(lb(a, 4), lb(a, 9) - lb(a, 8), lb(a, 13), lb(a, 4.5))
```

- (A) 1 2 8 4
- (B) 3 0 7 4
- (C) 1 0 8 4
- (D) 1 0 -1 3

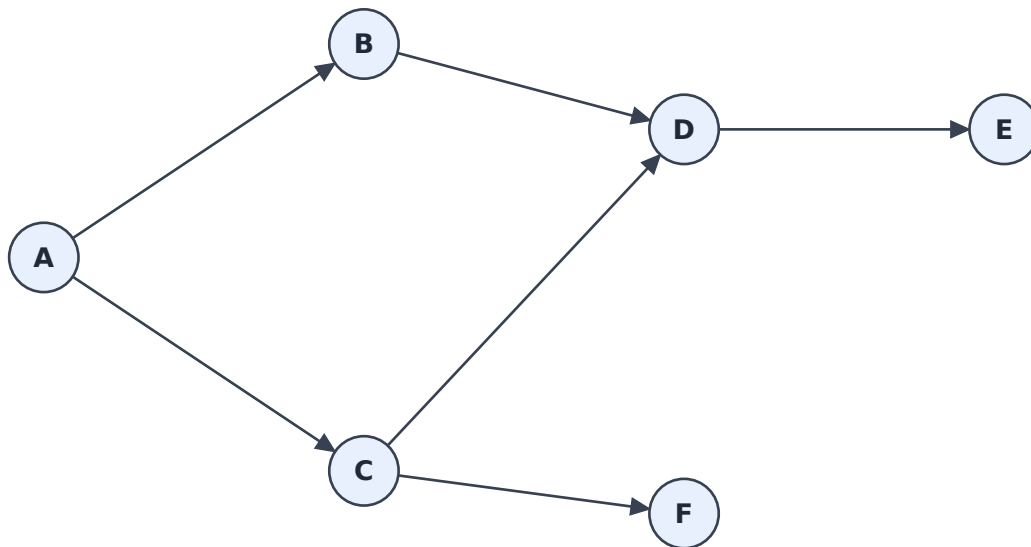
Q.8 [MCQ · single correct | 1 mark | Elementary sorts — intermediate states]

The array [52, 17, 33, 8, 41, 25] is sorted into ascending order by standard **insertion sort**. What is the array after the first three iterations of the outer loop (i.e. after A[1], A[2], A[3] have been inserted)?

- (A) [8, 17, 25, 52, 41, 33]
- (B) [8, 17, 33, 52, 41, 25]
- (C) [17, 33, 8, 41, 25, 52]
- (D) [17, 33, 52, 8, 41, 25]

Q.9 [MSQ · one or more correct | 1 mark | DAGs — topological orders and DFS]

Consider the directed acyclic graph G below. Which of the following statements is/are TRUE?



- (A) Running DFS from A (neighbours in alphabetical order) and listing vertices in decreasing order of finishing time gives A, C, F, B, D, E
- (B) G has exactly 6 topological orderings
- (C) Deleting the edge $C \rightarrow D$ increases the number of topological orderings
- (D) A is the only vertex with in-degree 0

Q.10 [NAT · numerical answer | 1 mark | Complexity — counting loop iterations]

Consider the following Python fragment with $n = 16$. The final value of `count` is _____.

Python

```
1 n = 16
2 count = 0
3 for i in range(1, n + 1):
4     j = i
5     while j <= n:
6         count += 1
7         j *= 2
```

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — class vs instance attributes]

Consider the following Python program. What is printed?

Python

```
1 class Tally:
2     count = 0
3     items = []
4
5     def __init__(self, x):
6         self.count += 1
7         self.items.append(x)
8
9 a = Tally(1)
10 b = Tally(2)
11 Tally.count += 5
12 print(a.count, b.count, Tally.count, len(a.items))
```

- (A) 2 2 7 2
- (B) 1 1 5 1
- (C) 1 1 5 2
- (D) 6 6 5 2

Q.12 [NAT · numerical answer | 2 marks | Python — recursion with try/except/finally]

Consider the following Python program. The value printed is _____.

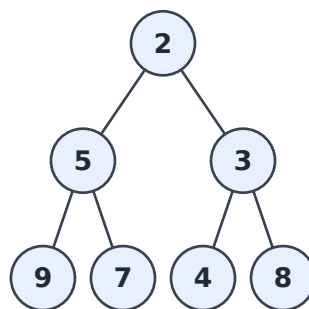
Python

```
1 def f(n):
2     try:
3         if n == 0:
4             raise ValueError
5         return n + f(n - 1)
6     except ValueError:
7         return 100
8     finally:
9         if n == 2:
10            return -n
11
12 print(f(4))
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Heaps — where can the k-th smallest be?]

A binary **min-heap** stores 31 distinct keys (a perfect binary tree of height 4; the root has depth 0). Which of the following statements is/are TRUE?



Example: the top three levels of one such heap

- (A) The largest key is always at a leaf
- (B) The 4th smallest key can be at depth 4
- (C) The 3rd smallest key can be at depth 3
- (D) The 3rd smallest key can be at depth 2

Q.14 [NAT · numerical answer | 2 marks | Quicksort — counting swaps (Lomuto)]

Quicksort with the Lomuto partition below sorts $A = [4, 8, 2, 7, 1, 5, 3, 6]$. Every execution of a swap statement counts as one swap, **including** swaps of an element with itself. The total number of swaps is _____.

Python

```
1 def partition(A, lo, hi):
2     p = A[hi]
3     i = lo - 1
4     for j in range(lo, hi):
5         if A[j] <= p:
6             i += 1
7             A[i], A[j] = A[j], A[i]
8     A[i + 1], A[hi] = A[hi], A[i + 1]
9     return i + 1
10
11 def quicksort(A, lo, hi):
12     if lo < hi:
13         q = partition(A, lo, hi)
14         quicksort(A, lo, q - 1)
15         quicksort(A, q + 1, hi)
```

Answer: _____ (enter a numerical value)

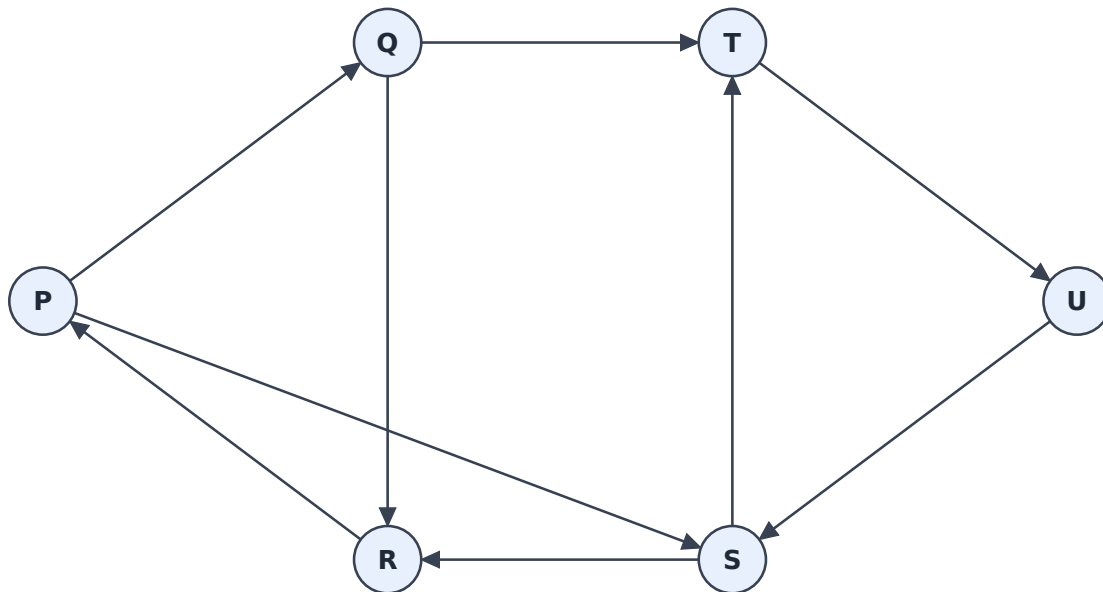
Q.15 [MCQ · single correct | 2 marks | Merging — optimal merge order]

Four sorted lists of lengths 10, 20, 30 and 40 must be merged into one sorted list by repeatedly merging **two** lists at a time. Merging lists of lengths p and q costs $p + q - 1$ comparisons in the worst case. The minimum possible total worst-case number of comparisons, over all merge orders, is

- (A) 190
- (B) 197
- (C) 200
- (D) 187

Q.16 [NAT · numerical answer | 2 marks | DFS — discovery and finishing times]

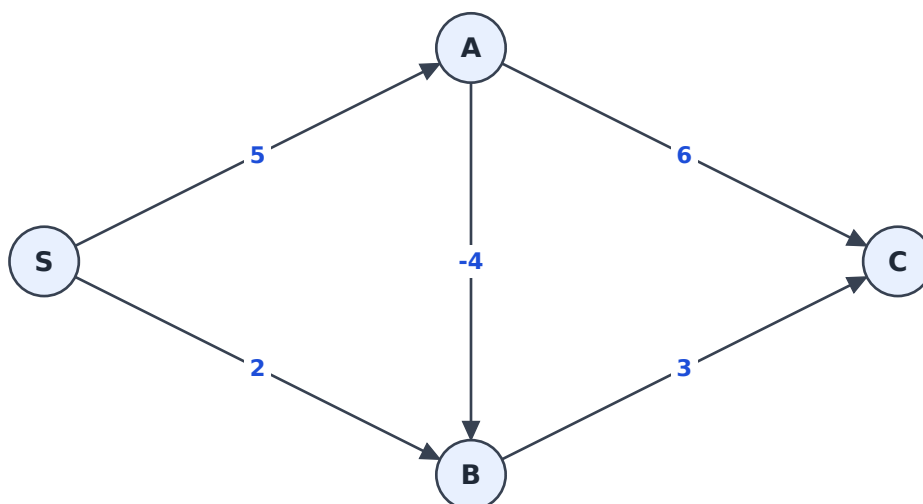
Depth-first search is run on the directed graph below starting from P; neighbours are explored in alphabetical order, and a single clock starting at 1 is incremented at every discovery and every finish (so $d(P) = 1$). The finishing time of vertex T is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Shortest paths — negative edge weights]

Consider the weighted directed graph below with source S. Dijkstra's algorithm is run in its standard form: once a vertex is extracted from the priority queue its distance is never changed again. Which of the following statements is/are TRUE?



- (A) The true shortest-path distance from S to C is 4
- (B) Adding 4 to every edge weight and then running Dijkstra yields the correct shortest paths of the original graph
- (C) The Bellman-Ford algorithm reports a negative-weight cycle in this graph
- (D) Dijkstra's algorithm reports the distance of C as 5

Q.18 [MCQ · single correct | 2 marks | BST — valid search paths]

The keys of a binary search tree are integers in the range 1 to 100. A search for the key 63 examines a sequence of nodes ending at 63. Which of the following could be the sequence of keys examined?

- (A) 90, 40, 80, 50, 70, 60, 63
- (B) 20, 75, 30, 60, 70, 58, 63
- (C) 10, 90, 85, 40, 88, 63
- (D) 50, 66, 55, 68, 63

Q.19 [NAT · numerical answer | 2 marks | Hashing — double hashing]

The keys 18, 41, 22, 44, 59, 32, 31, 73 are inserted in that order into an initially empty table with 13 slots (0 ... 12) using double hashing: the i -th probe for key k is $(h_1(k) + i \cdot h_2(k)) \bmod 13$, $i = 0, 1, 2, \dots$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. The index of the slot in which 31 is stored is _____.

Answer: _____ (enter a numerical value)

Q.20 [MSQ · one or more correct | 2 marks | Python — mutability, tuples and closures]

Each statement below refers to a separate fresh Python 3 session. Which of the statements is/are TRUE?

- (A) After `t = (1, [2])` and running `t[1] += [3]` inside `try: ... except TypeError: pass`, `t == (1, [2, 3])` is True
- (B) `sorted("bca") == "abc"` is True
- (C) After `x = 5`, `f = lambda: x`, `x = 7`, the call `f()` returns 7
- (D) After `m = [[0] * 2 for _ in range(2)]` and `m[0][0] = 1`, `m == [[1, 0], [1, 0]]` is True

Answer Key — Mock Test 31

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Python — split, join and reversal
2	NAT	1	34	Python — generator exhaustion
3	MCQ	1	(A)	Queues — queue using two stacks
4	NAT	1	6	Circular linked lists — repeated deletion
5	MSQ	1	(A), (C)	Binary search trees — deletion and successors
6	MCQ	1	(B)	Hashing — probability of a collision
7	MCQ	1	(C)	Binary search — lower-bound variant
8	MCQ	1	(B)	Elementary sorts — intermediate states
9	MSQ	1	(A), (C), (D)	DAGs — topological orders and DFS
10	NAT	1	31	Complexity — counting loop iterations
11	MCQ	2	(C)	Python — class vs instance attributes
12	NAT	2	5	Python — recursion with try/except/finally
13	MSQ	2	(A), (D)	Heaps — where can the k -th smallest be?
14	NAT	2	12	Quicksort — counting swaps (Lomuto)
15	MCQ	2	(D)	Merging — optimal merge order
16	NAT	2	10	DFS — discovery and finishing times
17	MSQ	2	(A), (D)	Shortest paths — negative edge weights
18	MCQ	2	(A)	BST — valid search paths

Q	Type	Marks	Key	Topic
19	NAT	2	12	Hashing — double hashing
20	MSQ	2	(A), (C)	Python — mutability, tuples and closures

Detailed Solutions — Mock Test 31

Q.1 [MCQ · 1 mark · Python — split, join and reversal]

Answer: (D)

`split()` with no argument splits on **runs** of whitespace and discards leading and trailing whitespace; `split(" ")` splits on **every single space**, producing empty strings between consecutive separators and at the ends.

- `s.split()` → ['GATE', 'DA', '2027'].
- Each part reversed: 'ETAG', 'AD', '7202'; joined with '-' → **ETAG-AD-7202**.
- `s.split(" ")`: `s = "GATE DA 2027"` has 6 spaces, so it yields 7 pieces: ['', '', 'GATE', '', 'DA', '2027', ''] → length 7.

Output **ETAG-AD-7202 7** → (D).

- (A) assumes `split(" ")` behaves like `split()`.
- (B) keeps an empty word in the join and miscounts the pieces.
- (C) reverses the order of the words instead of each word.

Tip: `len(s.split(sep))` = (number of occurrences of sep) + 1 for any explicit separator.

Q.2 [NAT · 1 mark · Python — generator exhaustion]

Answer: 34

A generator expression produces its values **once**. After it has been consumed, further iteration yields nothing.

- The generator yields squares of the odd numbers in `range(6)`: 1, 9, 25.
- First `sum(g)` consumes them all → 35.
- Second `sum(g)` sees an exhausted generator → sum of nothing = 0.
- `max(g, default=-1)` also sees nothing → returns the default -1 (without `default` it would raise `ValueError`).

Total = 35 + 0 + (-1) = **34**.

If `g` were a list comprehension `[x * x for ...]`, the result would be 35 + 35 + 25 = 95.

Trap: treating the generator like a list. Note also that the operands of `+` are evaluated left to right, so the first `sum` is the one that sees the values.

Q.3 [MCQ · 1 mark · Queues — queue using two stacks]

Answer: (A)

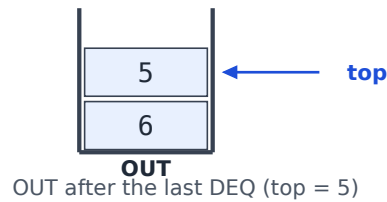
Transfers reverse the order of IN, so the oldest element ends on top of OUT; the structure is a correct FIFO queue.

- ENQ 1, 2, 3 → IN = [1, 2, 3], OUT = [].
- DEQ: OUT empty → move 3, 2, 1 (3 transfers) → OUT = [3, 2, 1] (top 1); pop → **1**.
- ENQ 4, 5 → IN = [4, 5].
- DEQ → OUT non-empty, pop → **2**.
- DEQ → pop → **3**; OUT now empty.
- ENQ 6 → IN = [4, 5, 6].
- DEQ: OUT empty → move 6, 5, 4 (3 transfers) → pop → **4**.

Returned values 1, 2, 3, 4 with 3 + 3 = **6** transfers → (A).

- (B) forgets that the second refill moves 6 as well.
- (C) treats the structure as a stack.
- (D) assumes 4 and 5 jump ahead after the first refill.

Tip: each element is transferred at most once, so any sequence of m operations costs $O(m)$ in total — amortised $O(1)$ per operation.



Q.4 [NAT · 1 mark · Circular linked lists — repeated deletion]

Answer: 6

Simulate, writing the remaining circle starting from where the count begins:

- Count 1, 2, 3, 4 → delete **4**; restart at 5. Remaining: 5 6 7 8 9 1 2 3.
- Count 5, 6, 7, 8 → delete **8**; restart at 9. Remaining: 9 1 2 3 5 6 7.
- Count 9, 1, 2, 3 → delete **3**; restart at 5. Remaining: 5 6 7 9 1 2.
- Count 5, 6, 7, 9 → delete **9**; restart at 1. Remaining: 1 2 5 6 7.
- Count 1, 2, 5, 6 → delete **6**; restart at 7. Remaining: 7 1 2 5.

The fifth deletion removes node **6**. (Continuing: 5, 7, 2 are deleted and node 1 survives.)

In a circular list, deletion needs a pointer to the node **before** the victim, so an implementation walks $k - 1 = 3$ links from the predecessor each round; total work is $O(nk)$.

Trap: restarting the count at the deleted node's predecessor, or counting the start node as 0, shifts every later deletion.

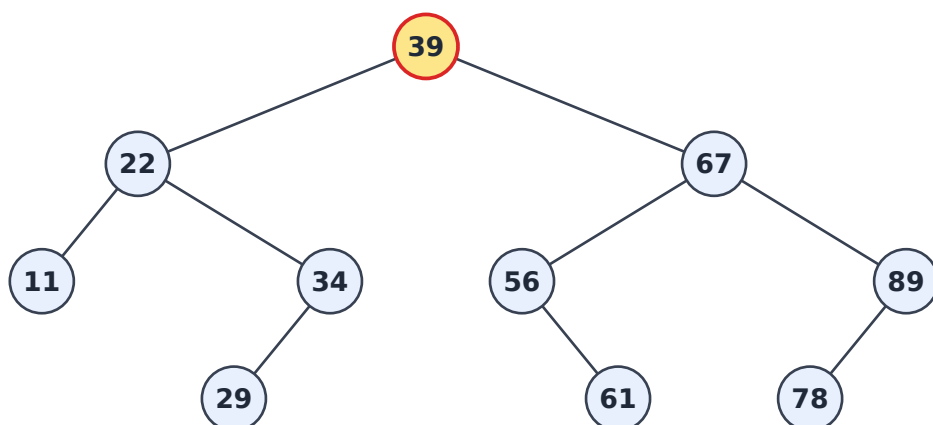
Q.5 [MSQ · 1 mark · Binary search trees — deletion and successors]

Answer: (A), (C)

The in-order predecessor of a node with a left subtree is the **maximum** of that left subtree (go left once, then right as far as possible). The in-order successor of a node **without** a right subtree is the nearest ancestor of which it lies in the left subtree.

- (A) Left subtree of 45 = {22, 11, 34, 29, 39}; its maximum is 39 (22 → 34 → 39). 39 replaces 45. **TRUE.**
- (B) 61 has no right child; walking up, 61 is in the right subtree of 56 and in the **left** subtree of 67 → successor 67 (in-order ..., 56, 61, 67, 78, 89). **FALSE.**
- (C) 39 was the right child (a leaf) of 34, so after removing it 34 keeps only its left child 29. **TRUE.**
- (D) Leaves: 11, 29, 39, 61, 78 → 5 leaves. **FALSE.**

Trap: in (B), picking the smallest key in some right subtree (78) — the successor is found upward when the node has no right child.



After deleting 45 (predecessor 39 promoted)

Q.6 [MCQ · 1 mark · Hashing — probability of a collision]

Answer: (B)

Compute the complement: the probability that all three keys land in **distinct** slots.

- Key 1: any slot $\rightarrow 10/10$.
- Key 2: must avoid key 1's slot $\rightarrow 9/10$.
- Key 3: must avoid two slots $\rightarrow 8/10$.

$P(\text{no collision}) = (10 \cdot 9 \cdot 8) / 10^3 = 720 / 1000 = 0.72$, so $P(\text{at least one collision}) = 1 - 0.72 = \mathbf{0.28}$
 $\rightarrow$ (B).

Option analysis:

- (A) 0.72 is the probability of **no** collision.
- (C) 0.30 adds the three pairwise probabilities $3 \times 1/10$, double-counting the case where all three collide (it is only an upper bound).
- (D) $0.03 = 3/100$ counts only one specific arrangement.

Tip: this is the birthday problem; with m slots, collisions become likely after about $\sqrt{m}$ keys.

Q.7 [MCQ · 1 mark · Binary search — lower-bound variant]

Answer: (C)

`lb(a, x)` maintains $a[0 \dots lo-1] < x \leq a[hi \dots n-1]$ and returns the **first index whose value is $\geq x$** (`len(a)` if none) — Python's `bisect_left`.

- `lb(a, 4)`: first element ≥ 4 is `a[1]` $\rightarrow 1$ (the leftmost of the duplicates).
- `lb(a, 9) = 5` and `lb(a, 8) = 5` (the first element ≥ 8 is also 9 at index 5) $\rightarrow$ difference 0.
- `lb(a, 13)`: no element $\geq 13 \rightarrow \text{len}(a) = 8$.
- `lb(a, 4.5)`: first element ≥ 4.5 is 7 at index 4 $\rightarrow 4$.

Output **1 0 8 4** $\rightarrow$ (C).

- (A) computes the count of 9s (`lb(a, 10) - lb(a, 9) = 2`) instead of `lb(9) - lb(8)`.
- (B) returns the last 4 (upper-bound - 1) and $n - 1$ for the missing key.
- (D) returns -1 for “not found”, which this half-open version never does.

Tip: the number of occurrences of x is `lb(a, x +  $\epsilon$ ) - lb(a, x)`, e.g. `lb(a, 4.5) - lb(a, 4) = 3` copies of 4.

Q.8 [MCQ · 1 mark · Elementary sorts — intermediate states]

Answer: (B)

After iteration i of insertion sort, the **prefix** `A[0 ... i]` holds the original first $i + 1$ elements in sorted order and the suffix is untouched.

- $i = 1$: insert 17 $\rightarrow [17, 52, 33, 8, 41, 25]$
- $i = 2$: insert 33 $\rightarrow [17, 33, 52, 8, 41, 25]$
- $i = 3$: insert 8 $\rightarrow [8, 17, 33, 52, 41, 25]$

$\rightarrow$ (B).

Option analysis:

- (A) is the state after three passes of **selection** sort (8, 17, 25 selected; the suffix is permuted by the swaps).
- (B) **Correct.**
- (C) is the state after one pass of **bubble** sort (the maximum 52 bubbles to the end).
- (D) is the state after only two iterations.

Tip: recognise the algorithm from the state: insertion sort → sorted prefix made of the *first* elements, untouched suffix; selection sort → prefix of the *smallest* elements; bubble sort → suffix of the *largest* elements.

Insertion sort, state after each iteration

	0	1	2	3	4	5
start	52	17	33	8	41	25
i = 1	17	52	33	8	41	25
i = 2	17	33	52	8	41	25
i = 3	8	17	33	52	41	25

Q.9 [MSQ · 1 mark · DAGs — topological orders and DFS]

Answer: (A), (C), (D)

- (A) DFS(A): $A \rightarrow B \rightarrow D \rightarrow E$; E finishes, D finishes, B finishes; then $A \rightarrow C \rightarrow (D \text{ done}) \rightarrow F$; F, C, A finish. Finish order E, D, B, F, C, A; reversed: A, C, F, B, D, E. **TRUE.**
- (B) Count orderings: A is first. Constraints left: B, C before D; D before E; C before F. Place F relative to the chain: sequences of {B, C, D, E} with B, C before D before E are B C D E and C B D E (2). F must come after C: in B C D E, F can go after C in 3 gaps; in C B D E, F can go after C in 4 gaps. Total $3 + 4 = 7$, not 6. **FALSE.**
- (C) Without $C \rightarrow D$ the constraints are A first, B before D before E, C before F. The number of interleavings of chain B D E with chain C F is $C(5, 2) = 10 > 7$. **TRUE.**
- (D) In-degrees: A 0, B 1, C 1, D 2, E 1, F 1. **TRUE.**

Tip: reverse post-order of DFS is always a valid topological order; removing constraints can never decrease the count.

Q.10 [NAT · 1 mark · Complexity — counting loop iterations]

Answer: 31

For a fixed i , j takes the values $i, 2i, 4i, \dots$ while $\leq n$, so the inner loop runs $\lfloor \log_2(n/i) \rfloor + 1$ times.

- $i = 1$: 1, 2, 4, 8, 16 → 5
- $i = 2$: 2, 4, 8, 16 → 4
- $i = 3$: 3, 6, 12 → 3
- $i = 4$: 4, 8, 16 → 3
- $i = 5, 6, 7, 8$: two values each ($i, 2i \leq 16$) → $4 \times 2 = 8$
- $i = 9 \dots 16$: one value each → 8

count = $5 + 4 + 3 + 3 + 8 + 8 = 31$.

Asymptotically $\sum_i (\log_2(n/i) + 1) = n + \log_2(n^n/n!) \approx n + n \log_2 e = \Theta(n)$: most values of i contribute only one or two iterations, so the fragment is linear, not $\Theta(n \log n)$.

Trap: multiplying n by the worst-case inner count ($16 \times 5 = 80$).

Q.11 [MCQ · 2 marks · Python — class vs instance attributes]

Answer: (C)

Attribute **lookup** on an instance checks the instance first, then the class. Attribute **assignment** through **self** always creates/updates an attribute on the instance. Mutating a class-level object through **self** mutates the shared object.

- **self.count += 1** means **self.count = self.count + 1**: the read finds the class value 0, and the write creates an **instance** attribute count = 1. This happens separately for **a** and **b**, so

`a.count = b.count = 1`, and `Tally.count` stays 0.

- `self.items.append(x)` reads the class list (no assignment) and mutates it: the single shared list becomes `[1, 2]`.
- `Tally.count += 5` changes only the class attribute $\rightarrow$ 5. It does not affect `a` or `b`, whose own instance attributes now shadow it.

Output `1 1 5 2` $\rightarrow$ (C).

- (A) assumes `self.count += 1` increments the shared class attribute.
- (B) assumes each instance has its own `items` list.
- (D) assumes the instances still see the class attribute after the change.

Trap: immutable class attributes behave “per instance” after the first augmented assignment, while mutable ones (lists) stay shared.

Q.12 [NAT · 2 marks · Python — recursion with try/except/finally]

Answer: 5

Two rules matter: (1) an exception raised inside a *callee* that the callee handles never reaches the caller; (2) a `return` inside **finally overrides** whatever the `try/except` block was about to return (or raise).

Evaluate from the bottom of the recursion:

- `f(0)`: raises `ValueError` inside its own `try` $\rightarrow$ its `except` returns 100; `finally` does nothing ($n \neq 2$). `f(0) = 100`.
- `f(1)`: returns `1 + 100 = 101`.
- `f(2)`: the `try` computes `2 + 101 = 103` and starts returning it, but **finally** executes `return -2`, which replaces it. `f(2) = -2`.
- `f(3)`: `3 + (-2) = 1`.
- `f(4)`: `4 + 1 = 5`.

Without the **finally** clause the answer would be `4 + 3 + 2 + 1 + 100 = 110`.

Traps:

- Thinking the `ValueError` from `f(0)` propagates up and is caught by every level (it is handled once, in `f(0)`).
- Ignoring the overriding `return` in **finally** (answer 110).

Q.13 [MSQ · 2 marks · Heaps — where can the k-th smallest be?]

Answer: (A), (D)

In a min-heap every ancestor of a key is smaller than it. So the k -th smallest key has at most $k - 1$ ancestors, i.e. its depth is at most **$k - 1$** .

- (A) Every internal node has a child, which must be larger; the largest key has nothing larger, so it can have no children $\rightarrow$ it is a leaf (depth 4 here). **TRUE.**
- (B) Depth 4 needs 4 smaller ancestors; the 4th smallest has only 3 smaller keys. Its depth is at most 3. **FALSE.**
- (C) Depth 3 would require 3 smaller ancestors, but only 2 keys are smaller. **FALSE.**
- (D) The 3rd smallest may have the 2nd smallest as parent and the minimum as grandparent. In the example figure, the keys 2, 3, 4 lie on a root path: 4 (the 3rd smallest) is at depth 2. **TRUE.**

Consequence: finding the k -th smallest needs to inspect only the top k levels (and in fact only $O(k)$ nodes with a priority-queue walk).

Trap: confusing “can be at depth d ” with “must be at depth d ”: the 3rd smallest can be at depth 1 or 2.

Q.14 [NAT · 2 marks · Quicksort — counting swaps (Lomuto)]**Answer: 12**

Each partition call on k elements performs (number of non-pivot elements $\leq$ pivot) swaps in the loop plus 1 final pivot swap.

- partition(0, 7), pivot 6: elements ≤ 6 among 4, 8, 2, 7, 1, 5, 3 are 4, 2, 1, 5, 3 $\rightarrow 5 + 1 = 6$ swaps. Result [4, 2, 1, 5, 3, **6**, 8, 7], $q = 5$.
- partition(0, 4) on [4, 2, 1, 5, 3], pivot 3: elements ≤ 3 are 2, 1 $\rightarrow 2 + 1 = 3$ swaps. Result [2, 1, **3**, 5, 4], $q = 2$.
- partition(0, 1) on [2, 1], pivot 1: none $\leq 1 \rightarrow 0 + 1 = 1$ swap $\rightarrow$ [1, 2].
- partition(3, 4) on [5, 4], pivot 4: $0 + 1 = 1$ swap $\rightarrow$ [4, 5].
- partition(6, 7) on [8, 7], pivot 7: $0 + 1 = 1$ swap $\rightarrow$ [7, 8].
- Remaining calls have $lo \geq hi$ and do nothing.

Total = $6 + 3 + 1 + 1 + 1 = \mathbf{12}$.

Note the first loop swap ($j = 0$, element 4) swaps $A[0]$ with itself — it is counted, as the question specifies.

Trap: forgetting the final pivot swap in the 2-element partitions, or skipping self-swaps (that would give $12 - 1 = 11$, as only one self-swap occurs: $A[0] \leftrightarrow A[0]$ in the first call).

Array after each partition call

	0	1	2	3	4	5	6	7
start	4	8	2	7	1	5	3	6
(0,7)	4	2	1	5	3	6	8	7
(0,4)	2	1	3	5	4	6	8	7
(0,1)	1	2	3	5	4	6	8	7
(3,4)	1	2	3	4	5	6	8	7
(6,7)	1	2	3	4	5	6	7	8

Q.15 [MCQ · 2 marks · Merging — optimal merge order]**Answer: (D)**

Each merge costs (size of the result) $- 1$, and there are always exactly 3 merges, so total = (sum of all intermediate result sizes) $- 3$. Minimising the sum of result sizes is the *optimal merge pattern*: always merge the two **shortest** lists (Huffman's greedy rule).

- Merge $10 + 20 \rightarrow 30$, cost 29. Lists: 30, 30, 40.
- Merge $30 + 30 \rightarrow 60$, cost 59. Lists: 40, 60.
- Merge $40 + 60 \rightarrow 100$, cost 99.

Total = $29 + 59 + 99 = \mathbf{187} \rightarrow$ (D).

Other orders for comparison:

- Left to right $((10+20)+30)+40$: $29 + 59 + 99 = 187$ (here it happens to coincide).
- $(10+20)$ then $(30+40)$, then together: $29 + 69 + 99 = 197 \rightarrow$ option (B).
- $(40+30)$ first: $69 + 89 + 99 = 257$.
- (A) $190 = 30 + 60 + 100$ forgets the “ $- 1$ ” per merge.
- (C) $200 = 2 \times 100$ is not a valid count for any order.

Tip: the final merge always costs $n - 1 = 99$; the choice only affects the earlier merges.

Q.16 [NAT · 2 marks · DFS — discovery and finishing times]**Answer: 10**

DFS(u): set d(u), recursively visit each undiscovered neighbour in order, then set f(u).

- P discovered (1). First neighbour Q.
- Q discovered (2). First neighbour R.
- R discovered (3). Its neighbour P is on the stack (grey) → back edge. R finishes (4).
- Back at Q, next neighbour T → T discovered (5).
- T → U discovered (6).
- U → S discovered (7).
- S: neighbour R is finished (black) → cross edge; neighbour T is grey → back edge. S finishes (8).
- U finishes (9). **T finishes (10)**. Q finishes (11).
- Back at P: neighbour S already finished → forward edge (S is a descendant of P). P finishes (12).

f(T) = **10**.

Edge classification: tree P→Q, Q→R, Q→T, T→U, U→S; back R→P, S→T; cross S→R; forward P→S. Since back edges exist, the graph has cycles (P→Q→R→P and S→T→U→S).

Trap: visiting S from P before exploring Q's subtree completely — DFS finishes Q's entire subtree first, which is why S is discovered through U.

Discovery / finishing times						
	P	Q	R	S	T	U
d	1	2	3	7	5	6
f	12	11	4	8	10	9

Q.17 [MSQ · 2 marks · Shortest paths — negative edge weights]**Answer: (A), (D)**

Dijkstra's correctness proof needs non-negative weights: it assumes no path through an unfinished vertex can later improve a finished one.

Dijkstra trace:

- Extract S (0): A = 5, B = 2.
- Extract B (2): C = 2 + 3 = 5.
- Extract A (5): A → B would give 1, but B is already finalised → ignored. A → C gives 11 > 5.
- Extract C (5).

True distances: B = min(2, 5 - 4) = 1 via S→A→B, and C = 1 + 3 = 4 via S→A→B→C.

- (A) **TRUE** — S→A→B→C costs 5 - 4 + 3 = 4.
- (B) **FALSE** — adding a constant penalises paths with more edges. New weights: S→A 9, A→B 0, B→C 7, S→B 6; S→B→C = 13 < S→A→B→C = 16, so the re-weighted shortest path to C is S→B→C, which is wrong for the original graph.
- (C) **FALSE** — the graph is acyclic, so it has no cycle at all, let alone a negative one; Bellman-Ford returns the correct distances.
- (D) **TRUE** — Dijkstra outputs 5 for C (and 2 for B).

Trap: (B) looks like a fix, but uniform shifting does not preserve shortest paths (Johnson's algorithm uses vertex potentials instead).

Q.18 [MCQ · 2 marks · BST — valid search paths]**Answer: (A)**

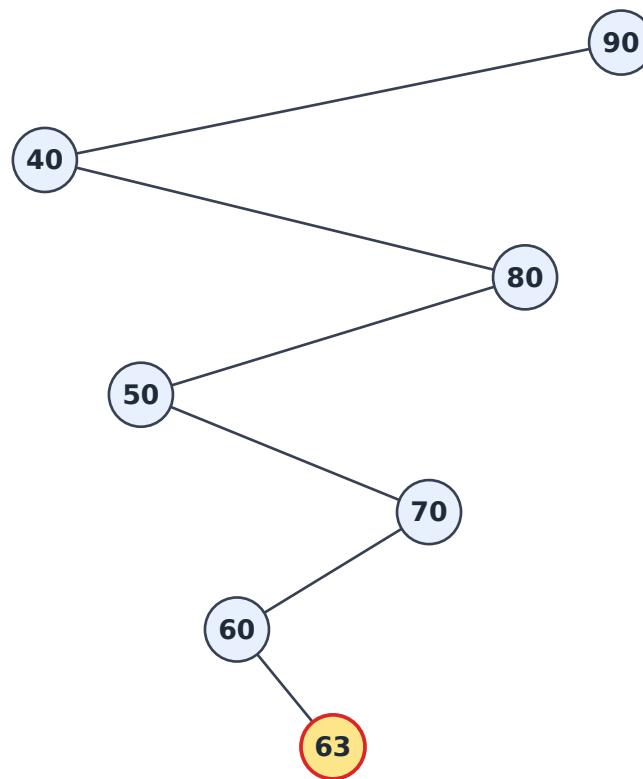
As the search descends, each examined key narrows an open interval (low, high) that must contain 63: going left from key k sets $\text{high} = k$, going right sets $\text{low} = k$. Every later key must lie strictly inside the current interval.

- (A) $90 \rightarrow (-\infty, 90)$; $40 \rightarrow (40, 90)$; $80 \rightarrow (40, 80)$; $50 \rightarrow (50, 80)$; $70 \rightarrow (50, 70)$; $60 \rightarrow (60, 70)$; 63 ✓. **Valid.**
- (B) ... $60 \rightarrow (60, \infty)$; $75 \rightarrow (60, 75)$; $70 \rightarrow (60, 70)$; then 58 is not in $(60, 70)$. **Invalid.**
- (C) $10 \rightarrow (10, \infty)$; $90 \rightarrow (10, 90)$; $85 \rightarrow (10, 85)$; $40 \rightarrow (40, 85)$; 88 is not < 85 . **Invalid.**
- (D) $50 \rightarrow (50, \infty)$; $66 \rightarrow (50, 66)$; $55 \rightarrow (55, 66)$; 68 is not < 66 . **Invalid.**

Answer (A).

Quick test: in a valid sequence, the keys greater than the target must appear in **decreasing** order and those smaller in **increasing** order. In (A): larger keys 90, 80, 70 decrease; smaller keys 40, 50, 60 increase ✓. In (B), smaller keys 20, 30, 60, 58 are not increasing ✗.

Trap: checking only that each step moves in the right direction relative to 63, while ignoring earlier bounds.



Search path (A) drawn as a BST path

Q.19 [NAT · 2 marks · Hashing — double hashing]**Answer: 12**

Double hashing uses a key-dependent step $h_2(k)$, so keys sharing a home slot usually follow different probe sequences.

- 18: $h_1 = 5 \rightarrow \text{slot } 5$.
- 41: $h_1 = 2 \rightarrow \text{slot } 2$.
- 22: $h_1 = 9 \rightarrow \text{slot } 9$.
- 44: $h_1 = 5$ (taken), $h_2 = 1 + 0 = 1 \rightarrow 6 \rightarrow \text{slot } 6$.
- 59: $h_1 = 7 \rightarrow \text{slot } 7$.

- 32: $h_1 = 6$ (taken), $h_2 = 1 + 10 = 11 \rightarrow 6 + 11 = 17 \equiv 4 \rightarrow$ slot 4.
- 31: $h_1 = 5$ (taken), $h_2 = 1 + 9 = 10 \rightarrow 15 \equiv 2$ (taken by 41) $\rightarrow 5 + 20 = 25 \equiv 12 \rightarrow$ slot 12.
- 73: $h_1 = 8 \rightarrow$ slot 8.

Key 31 is stored in slot **12** after two collisions.

Since 13 is prime and $1 \leq h_2 \leq 11 < 13$, every probe sequence visits all 13 slots, so insertion succeeds whenever a free slot exists.

Trap: using $h_2(k) = k \bmod 11$ (forgetting the “+”) gives step 9 for 31 and a different slot; h_2 must never be 0.

0	
1	
2	41
3	
4	32
5	18
6	44
7	59
8	73
9	22
10	
11	
12	31

Final table

Q.20 [MSQ · 2 marks · Python — mutability, tuples and closures]

Answer: (A), (C)

- (A) `t[1] += [3]` runs in two steps: `t[1].__iadd__([3])` mutates the list in place (succeeds), then Python tries `t[1] = <result>`, which raises `TypeError` because tuples do not support item assignment. The exception is swallowed, but the mutation has already happened: `t == (1, [2, 3])`. **TRUE.**
- (B) `sorted` always returns a **list**: `['a', 'b', 'c'] != 'abc'`. `(''.join(sorted("bca")))` would equal `'abc'`. **FALSE.**
- (C) The lambda looks up the global name `x` when it is **called**, by which time `x` is 7. **TRUE.**
- (D) The comprehension creates a **new** inner list on each iteration, so the rows are independent: `m = [[1, 0], [0, 0]]`. **FALSE.** (The aliasing trap applies to `[[0] * 2] * 2`.)

Trap: in (A), most people assume an operation that raises has no effect; augmented assignment on a mutable element of an immutable container is the classic counter-example. Similarly in (C), a closure captures *variables*, not values; to freeze the value write `lambda x=x: x`.

Mock Test 32

Full-Syllabus Mock — Paper 2

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — augmented assignment on a tuple element]

Consider the following Python program. What is printed?

Python

```

1 t = ([1], 2)
2 try:
3     t[0] += [3]
4 except TypeError:
5     print("E", end=" ")
6 print(t)
```

- (A) ([1, 3], 2)
- (B) E ([1], 2)
- (C) E ([1, 3], 2)
- (D) ([1], 2, 3)

Q.2 [NAT · numerical answer | 1 mark | Queues — round-robin simulation]

Five jobs are scheduled round-robin with time quantum 3 using a FIFO ready queue. P1, P2, P3, P4 (bursts 5, 3, 8, 2) are in the queue at time 0 in that order; P5 (burst 4) arrives at time 4. A job whose quantum expires is put at the rear of the queue, after any jobs that arrived while it was running. Context switches take no time. The turnaround time (completion time – arrival time) of P5 is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Linked lists — recursive pairwise swap]

Consider the following Python program. What is printed?

Python

```

1 class N:
2     def __init__(s, v, nx=None):
3         s.v, s.nx = v, nx
4
5     def build(xs):
6         h = None
7         for x in reversed(xs):
8             h = N(x, h)
9         return h
10
11    def swap_pairs(h):
12        if h is None or h.nx is None:
13            return h
14        second = h.nx
15        h.nx = swap_pairs(second.nx)
16        second.nx = h
17        return second
18
19    h = build([1, 2, 3, 4, 5, 6, 7])
20    h.nx = swap_pairs(h.nx)
21    out = []
22    while h:
23        out.append(h.v)
24        h = h.nx
25    print(out)

```

- (A) [2, 1, 4, 3, 6, 5, 7]
- (B) [1, 3, 2, 5, 4, 7, 6]
- (C) [1, 2, 4, 3, 6, 5, 7]
- (D) [7, 6, 5, 4, 3, 2, 1]

Q.4 [NAT · numerical answer | 1 mark | Binary search trees — internal path length]

The keys 40, 25, 60, 10, 30, 50, 70, 5, 35, 65, 80, 62 are inserted in this order into an initially empty binary search tree. The sum of the depths of all nodes (the root has depth 0) is ____.

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Hashing — linear probing with lazy deletion]

A hash table of size 10 uses $h(k) = k \bmod 10$ with linear probing. Keys 21, 31, 12, 41, 15, 25 are inserted in that order. Then 31 is deleted by marking its slot DELETED (a tombstone). A search skips over DELETED slots and stops at an EMPTY slot; an insertion places the key in the first DELETED or EMPTY slot on its probe sequence.

Let p be the number of slots examined when searching for 41 after the deletion, and let s be the slot where 51 is stored if it is inserted next. Then (p, s) is

- (A) (4, 2)
- (B) (3, 2)
- (C) (4, 7)
- (D) (2, 2)

Q.6 [NAT · numerical answer | 1 mark | Binary search — average successful comparisons]

Binary search ($\text{mid} = \lfloor (\text{lo} + \text{hi})/2 \rfloor$, one three-way comparison per probe) is performed on a sorted array of 15 distinct keys. Each key is equally likely to be searched for, and every search is successful. The average number of probes, rounded off to two decimal places, is ____.

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Bubble sort — state after k passes]

Bubble sort (each pass compares adjacent pairs from left to right and swaps if the left element is larger) is applied to [6, 2, 9, 1, 5, 8, 3]. The array after **two** complete passes is

- (A) [2, 6, 1, 5, 8, 3, 9]
- (B) [1, 2, 6, 5, 3, 8, 9]
- (C) [2, 1, 5, 6, 3, 8, 9]
- (D) [1, 2, 3, 6, 9, 5, 8]

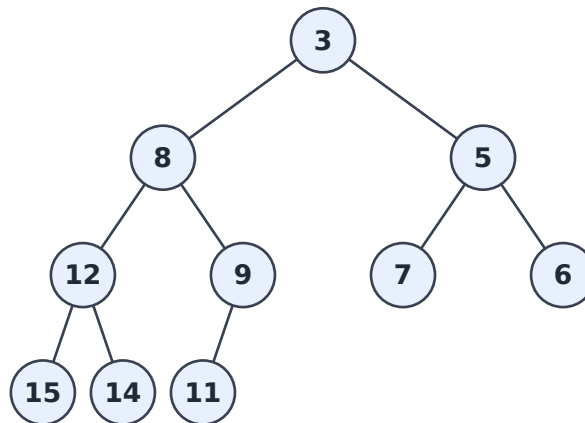
Q.8 [NAT · numerical answer | 1 mark | Graph theory — leaves of a tree]

A tree has exactly three vertices of degree 4, two vertices of degree 3, one vertex of degree 2, and every other vertex has degree 1. The number of vertices of degree 1 is _____.

Answer: _____ (enter a numerical value)

Q.9 [MSQ · one or more correct | 1 mark | Heaps — min-heap insert and delete]

The array $H = [3, 8, 5, 12, 9, 7, 6, 15, 14, 11]$ (0-based) is a binary min-heap. Which of the following statements is/are TRUE? (Each refers to the original H.)



Min-heap H

- (A) Inserting 4 yields [3, 4, 5, 12, 8, 7, 6, 15, 14, 11, 9]
- (B) H has exactly 5 leaves
- (C) Deleting the minimum yields [5, 8, 6, 12, 9, 7, 11, 15, 14]
- (D) The third-smallest key, 6, is a child of the root

Q.10 [MSQ · one or more correct | 1 mark | Python — built-in semantics]

Which of the following Python 3 expressions evaluate to **True**?

- (A) `{1, 2, 3} ^ {2, 3, 4} == {1, 4}`
- (B) `round(2.5) == 3`
- (C) `sorted("BaNaNa", key=str.lower) == ['a', 'a', 'a', 'B', 'N', 'N']`
- (D) `[1, 2, 3] * 2 == [2, 4, 6]`

Q.11 [NAT · numerical answer | 2 marks | Python — counting recursive calls]

Consider the following Python program. The value printed is _____.

Python

```
1 calls = 0
2
3 def f(n):
4     global calls
5     calls += 1
6     if n <= 1:
7         return 1
8     return f(n - 1) + f(n // 2)
9
10 f(10)
11 print(calls)
```

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Merge sort — bottom-up passes]

Bottom-up merge sort sorts [9, 4, 7, 1, 8, 2, 5]. In the pass with run width w ($w = 1, 2, 4, \dots$), the array is cut into consecutive blocks of $2w$ elements starting at index 0, and in each block the first w elements are merged with the rest of the block (which may be shorter than w , or empty). The array after the passes with $w = 1$ and $w = 2$ is

- (A) [1, 4, 7, 9, 2, 5, 8]
- (B) [4, 9, 1, 7, 2, 8, 5]
- (C) [1, 4, 7, 9, 2, 8, 5]
- (D) [1, 2, 4, 5, 7, 8, 9]

Q.13 [MSQ · one or more correct | 2 marks | Binary search trees — reconstruction from post-order]

The post-order traversal of a binary search tree with distinct keys is 8, 15, 12, 22, 20, 35, 45, 40, 30.

Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) If 30 is deleted by replacing it with its in-order predecessor, the new root is 22
- (B) The tree has exactly 4 leaves
- (C) The pre-order traversal is 30, 20, 12, 8, 15, 22, 40, 35, 45
- (D) The height of the tree is 3

Q.14 [NAT · numerical answer | 2 marks | BFS — shortest path in a grid maze]

In the 5×6 grid below, # cells are walls. From any open cell one may move up, down, left or right to an adjacent open cell (each move costs 1). The minimum number of moves needed to go from S (row 0, column 0) to T (row 0, column 5) is _____.

	0	1	2	3	4	5
0	S				#	T
1	#	#	#		#	
2					#	
3		#		#	#	
4						

Maze (highlighted cells are walls)

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Python — class vs instance attributes]

Consider the following Python program. What is printed?

Python

```

1 class Node:
2     count = 0
3     kids = []
4     def __init__(self, v):
5         self.v = v
6         Node.count += 1
7         self.count = Node.count * 10
8     def add(self, c):
9         self.kids.append(c)
10        return self
11
12 a = Node(1)
13 b = Node(2).add(a)
14 c = Node(3)
15 c.add(b)
16 c.kids = []
17 print(Node.count, a.count, len(a.kids), len(c.kids))

```

- (A) 3 30 2 0
- (B) 3 10 2 0
- (C) 3 10 0 0
- (D) 3 10 1 0

Q.16 [NAT · numerical answer | 2 marks | Heaps — streaming k-th largest with a min-heap]

To maintain the 3 largest values of a stream, a min-heap of capacity 3 is used: the first 3 values are simply inserted; afterwards, each new value x **replaces the root** (pop-min then push x) if x is greater than the current root, and is discarded otherwise. The stream is

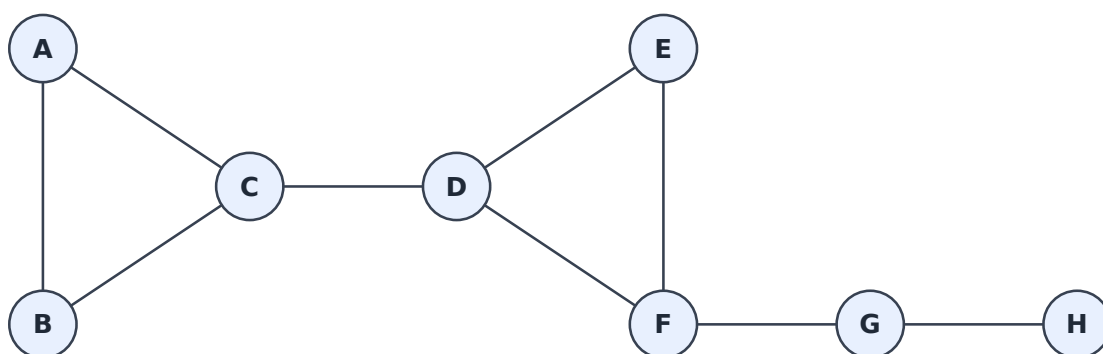
5, 12, 3, 8, 15, 7, 10, 20, 1, 11

The number of root replacements performed is _____.

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Graph theory — bridges, cut vertices, bipartiteness]

Consider the undirected graph G shown below. Which of the following statements is/are TRUE?



- (A) G is bipartite
- (B) Removing vertex D (with its edges) leaves exactly 2 connected components
- (C) G has exactly 3 bridges (edges whose removal disconnects the graph)
- (D) G has exactly 3 articulation points (cut vertices)

Q.18 [NAT · numerical answer | 2 marks | Hashing — expected number of empty slots]

Six keys are inserted into a hash table with 8 slots using separate chaining. Assume simple uniform hashing: each key independently hashes to each slot with probability $1/8$. The expected number of slots that remain **empty**, rounded off to two decimal places, is _____.

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Complexity — nested loops with halving and doubling]

The function below returns c. As a function of n, the returned value is

Python

```
1 def work(n):
2     c = 0
3     i = n
4     while i > 1:
5         j = 1
6         while j < i:
7             c += 1
8             j *= 2
9         i //= 2
10    return c
```

- (A) $\Theta(\log n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n)$
- (D) $\Theta(\log^2 n)$

Q.20 [MSQ · one or more correct | 2 marks | Linked lists + stacks — palindrome check]

The function below checks whether a singly linked list is a palindrome, using a slow/fast pointer pair and a stack. Which of the following statements is/are TRUE?

Python

```
1 def is_pal(head):
2     st, slow, fast = [], head, head
3     while fast and fast.nx:
4         st.append(slow.v)
5         slow = slow.nx
6         fast = fast.nx.nx
7     if fast:
8         slow = slow.nx
9     while slow:
10        if st.pop() != slow.v:
11            return False
12        slow = slow.nx
13    return True
```

- (A) It returns True for the list $4 \rightarrow 5 \rightarrow 5 \rightarrow 4 \rightarrow 4$
- (B) For a list of 9 nodes, the stack never holds more than 4 values
- (C) It returns True for the list $1 \rightarrow 2 \rightarrow 1 \rightarrow 2$
- (D) It returns True for the list $1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 1$

Answer Key — Mock Test 32

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — augmented assignment on a tuple element
2	NAT	1	16	Queues — round-robin simulation
3	MCQ	1	(B)	Linked lists — recursive pairwise swap
4	NAT	1	26	Binary search trees — internal path length
5	MCQ	1	(A)	Hashing — linear probing with lazy deletion
6	NAT	1	3.26 to 3.28	Binary search — average successful comparisons
7	MCQ	1	(C)	Bubble sort — state after k passes
8	NAT	1	10	Graph theory — leaves of a tree
9	MSQ	1	(A), (B), (C)	Heaps — min-heap insert and delete
10	MSQ	1	(A), (C)	Python — built-in semantics
11	NAT	2	59	Python — counting recursive calls
12	MCQ	2	(A)	Merge sort — bottom-up passes
13	MSQ	2	(A), (C), (D)	Binary search trees — reconstruction from post-order
14	NAT	2	15	BFS — shortest path in a grid maze
15	MCQ	2	(B)	Python — class vs instance attributes
16	NAT	2	4	Heaps — streaming k-th largest with a min-heap
17	MSQ	2	(B), (C)	Graph theory — bridges, cut vertices, bipartiteness
18	NAT	2	3.58 to 3.60	Hashing — expected number of empty slots
19	MCQ	2	(D)	Complexity — nested loops with halving and doubling
20	MSQ	2	(B), (D)	Linked lists + stacks — palindrome check

Detailed Solutions — Mock Test 32

Q.1 [MCQ · 1 mark · Python — augmented assignment on a tuple element]

Answer: (C)

Concept: `x[i] += y` is executed as two steps: (1) `tmp = x[i].__iadd__(y)` and (2) `x[i] = tmp`. For a list, step 1 extends the list **in place**. Step 2 then tries to assign into the tuple, which is immutable, and raises `TypeError`.

- Step 1 succeeds: the list inside the tuple becomes `[1, 3]`.
- Step 2 fails: `'tuple' object does not support item assignment` → the `except` block prints `E` (with `end=" "`, no newline).
- The tuple still holds the *same* list object, now mutated → `print(t)` shows `([1, 3], 2)`.

Output: `E ([1, 3], 2)`.

Options:

- (A) assumes no exception is raised.
- (B) assumes the failed statement is rolled back — Python has no such rollback; the in-place extension already happened.
- (D) confuses `t[0] += [3]` with tuple concatenation.

Trap: 'immutable tuple' means its *references* cannot change, not that the objects it refers to cannot be mutated. `t[0].append(3)` would succeed silently.

Q.2 [NAT · 1 mark · Queues — round-robin simulation]

Answer: 16

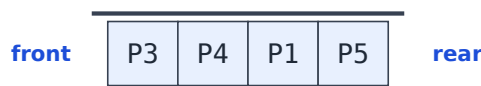
Concept: round-robin is a queue simulation: dequeue the front job, run it for `min(quantum, remaining)`, enqueue new arrivals, then re-enqueue the job if unfinished.

Timeline (queue after each slice, front first):

- 0-3 P1 (rem 2) → [P2, P3, P4, P1]
- 3-6 P2 done at 6; P5 arrived at 4 → [P3, P4, P1, P5]
- 6-9 P3 (rem 5) → [P4, P1, P5, P3]
- 9-11 P4 done at 11 → [P1, P5, P3]
- 11-13 P1 done at 13 → [P5, P3]
- 13-16 P5 (rem 1) → [P3, P5]
- 16-19 P3 (rem 2) → [P5, P3]
- 19-20 P5 **done at 20** → [P3]
- 20-22 P3 done at 22

Turnaround of P5 = 20 − 4 = **16**.

Trap: reporting the completion time (20) instead of the turnaround time, or enqueueing P5 at time 0.



Ready queue at time 6 (front on the left)

Q.3 [MCQ · 1 mark · Linked lists — recursive pairwise swap]**Answer: (B)**

Concept: `build` inserts at the head while iterating over the *reversed* input, so the list keeps the original order $1 \rightarrow 2 \rightarrow \dots \rightarrow 7$. `swap_pairs` swaps nodes in adjacent pairs and returns the new head; an odd node at the end stays put.

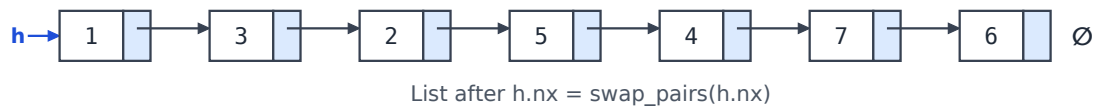
Here the swap starts at the **second** node (`h.nx`), so node 1 is untouched and the sublist $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ is processed:

- pair $(2, 3) \rightarrow 3, 2$
- pair $(4, 5) \rightarrow 5, 4$
- pair $(6, 7) \rightarrow 7, 6$
- Sublist becomes $3 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 7 \rightarrow 6$ and `h.nx` is relinked to its head 3.

Output: `[1, 3, 2, 5, 4, 7, 6]`.

Options: (A) swaps from the head. (C) shifts the pairing by one more node. (D) is a full reversal.

Trap: forgetting to store the returned head — calling `swap_pairs(h.nx)` without assigning it to `h.nx` would leave node 1 pointing at node 2, which is now *second* in its pair, losing node 3 from the traversal.

**Q.4 [NAT · 1 mark · Binary search trees — internal path length]****Answer: 26**

Concept: the depth of a key is the number of comparisons-with-descent made when it was inserted. The sum of depths (internal path length) determines the average cost of a successful search: $(\text{sum} + n)/n$ comparisons.

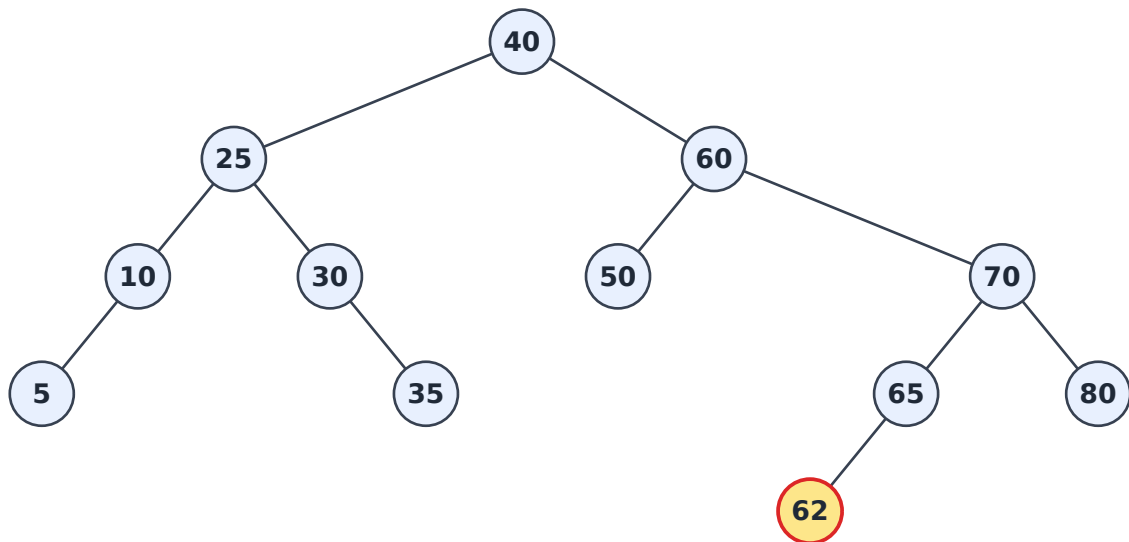
Insert and record depth:

- 40 → depth 0 (root)
- 25, 60 → depth 1 (children of 40)
- 10, 30 (under 25), 50, 70 (under 60) → depth 2
- 5 (left of 10), 35 (right of 30), 65 (left of 70), 80 (right of 70) → depth 3
- 62 → 40 → 60 → 70 → 65 → left of 65 → depth 4

Sum = $0 + 2 \cdot 1 + 4 \cdot 2 + 4 \cdot 3 + 1 \cdot 4 = 0 + 2 + 8 + 12 + 4 = 26$.

(Average successful search = $(26 + 12)/12 \approx 3.17$ comparisons.)

Trap: taking the root's depth as 1 (which adds $n = 12$ and gives 38).



BST after all insertions; 62 is the only depth-4 node

Q.5 [MCQ · 1 mark · Hashing — linear probing with lazy deletion]

Answer: (A)

Concept: with open addressing, a deleted slot cannot simply be emptied — that would cut probe chains that pass through it. A tombstone keeps searches going but can be reused by insertions.

Insertions: 21 → 1; 31 → 1 full → **2**; 12 → 2 full → **3**; 41 → 1, 2, 3 full → **4**; 15 → 5; 25 → 5 full → **6**.

Delete 31: slot 2 becomes DELETED.

Search 41: slot 1 (21, no), slot 2 (DELETED, continue), slot 3 (12, no), slot 4 (41, found) → **p = 4** slots examined.

Insert 51: slot 1 occupied, slot 2 DELETED → reuse → **s = 2**.

Answer (4, 2).

Options: (D) is what happens if deletion *empties* slot 2: the search for 41 stops at the empty slot 2 after 2 probes and wrongly reports 'absent'. (B) miscounts the probes. (C) ignores tombstone reuse and places 51 in the first truly empty slot, 7.

Tip: a common refinement first finishes the search (to avoid duplicates) and then inserts at the first tombstone seen.

0	
1	21
2	DEL
3	12
4	41
5	15
6	25
7	
8	
9	

Table after deleting 31 (before inserting 51)

Q.6 [NAT · 1 mark · Binary search — average successful comparisons]**Answer: 3.26 to 3.28**

Concept: the probes of binary search on $n = 2^k - 1$ keys form a perfectly balanced decision tree: 1 key is found with 1 probe (the middle), 2 keys with 2 probes, 4 with 3, 8 with 4.

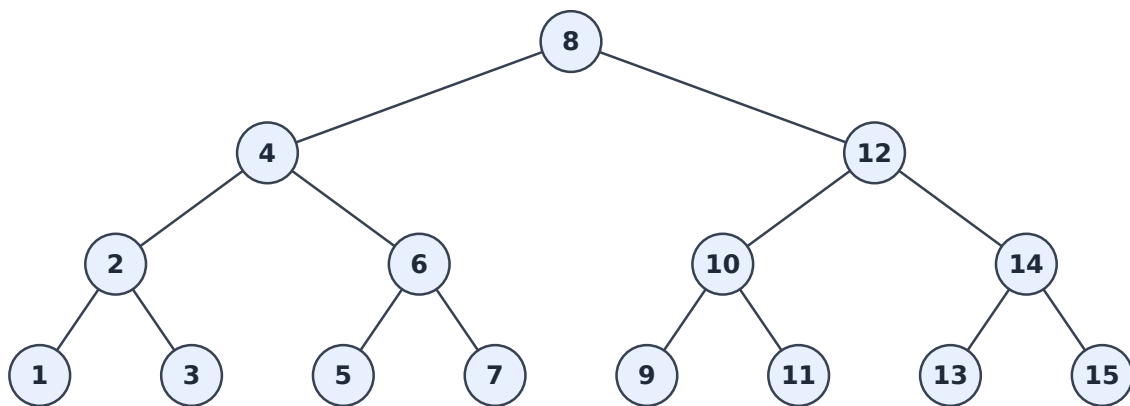
For $n = 15$ ($k = 4$):

- level 1: 1 key $\times$ 1 probe = 1
- level 2: 2 keys $\times$ 2 = 4
- level 3: 4 keys $\times$ 3 = 12
- level 4: 8 keys $\times$ 4 = 32

Total = 49 probes over 15 keys $\rightarrow 49/15 = 3.27$.

Compare: the worst case is $\lfloor \log_2 15 \rfloor + 1 = 4$ probes, so the average is close to the worst case because more than half the keys sit on the deepest level.

Trap: averaging the levels $(1 + 2 + 3 + 4)/4 = 2.5$ ignores that deeper levels hold exponentially more keys.



Decision tree of probe positions (1-based) for $n = 15$

Q.7 [MCQ · 1 mark · Bubble sort — state after k passes]**Answer: (C)**

Concept: each bubble-sort pass carries the largest remaining element to the end of the unsorted part; small elements move left by at most one position per pass.

Pass 1 on [6, 2, 9, 1, 5, 8, 3]:

- $6 > 2$ swap $\rightarrow$ [2, 6, 9, 1, 5, 8, 3]; $6 < 9$; $9 > 1$ swap $\rightarrow$ [2, 6, 1, 9, 5, 8, 3]; $9 > 5$ swap; $9 > 8$ swap; $9 > 3$ swap $\rightarrow$ [2, 6, 1, 5, 8, 3, 9]

Pass 2 (up to index 5):

- $2 < 6$; $6 > 1$ swap $\rightarrow$ [2, 1, 6, 5, 8, 3, 9]; $6 > 5$ swap $\rightarrow$ [2, 1, 5, 6, 8, 3, 9]; $6 < 8$; $8 > 3$ swap $\rightarrow$ [2, 1, 5, 6, 3, 8, 9]

Answer (C).

Options: (B) moves 1 from index 3 to index 0 in two passes — impossible, as a small element moves left by at most one position per pass. (A) is the state after one pass. (D) looks like a selection-sort state (smallest elements fixed at the front).

Tip: after k passes the last k positions hold the k largest values in order (8, 9 here).

Bubble sort passes							
	0	1	2	3	4	5	6
start	6	2	9	1	5	8	3
pass 1	2	6	1	5	8	3	9
pass 2	2	1	5	6	3	8	9

Q.8 [NAT · 1 mark · Graph theory — leaves of a tree]**Answer: 10**

Concept: a tree with n vertices has $n - 1$ edges, and by the handshaking lemma $\sum \deg = 2(n - 1)$. Equivalently, $\# \text{leaves} = 2 + \sum \text{over vertices of degree } \geq 2 \text{ of } (\deg - 2)$.

Let L be the number of leaves. Then $n = 3 + 2 + 1 + L = 6 + L$.

Degree sum: $3 \cdot 4 + 2 \cdot 3 + 1 \cdot 2 + L \cdot 1 = 20 + L$.

Handshaking: $20 + L = 2(n - 1) = 2(5 + L) = 10 + 2L \rightarrow L = 10$.

Shortcut check: $2 + 3 \cdot (4 - 2) + 2 \cdot (3 - 2) + 1 \cdot (2 - 2) = 2 + 6 + 2 + 0 = 10$. ✓

The tree has 16 vertices and 15 edges.

Trap: forgetting the degree-2 vertex when computing n (giving 9). Note that degree-2 vertices never change the leaf count — they just subdivide edges.

Q.9 [MSQ · 1 mark · Heaps — min-heap insert and delete]**Answer: (A), (B), (C)**

Concept: insertion appends and sifts up; delete-min moves the last element to the root and sifts down towards the **smaller** child.

- (A) 4 goes to index 10; parent index 4 (9) > 4 → swap; parent index 1 (8) > 4 → swap; parent index 0 (3) < 4 → stop. Result [3, 4, 5, 12, 8, 7, 6, 15, 14, 11, 9]. **True.**
- (B) A heap with $n = 10$ nodes has $n - \lfloor n/2 \rfloor = 5$ leaves (indices 5..9). **True.**
- (C) Move 11 to the root: [11, 8, 5, 12, 9, 7, 6, 15, 14]. Smaller child 5 → swap → 11 at index 2; children 7, 6 → swap with 6 → 11 at index 6 (leaf). Result [5, 8, 6, 12, 9, 7, 11, 15, 14]. **True.**
- (D) The root's children are 8 and 5; 6 is at index 6, a child of 5. **False.**

Trap: in (C), sifting towards the *left* child by default (8) would produce an invalid heap.

Q.10 [MSQ · 1 mark · Python — built-in semantics]**Answer: (A), (C)**

Concept checks: key functions and stability of **sorted**, sequence repetition, set symmetric difference, and Python's rounding rule.

- (A) $\wedge$ is symmetric difference: elements in exactly one set → {1, 4}. **True.**
- (B) Python 3 uses round-half-to-even ('banker's rounding'): $\text{round}(2.5) = 2$ (and $\text{round}(3.5) = 4$). **False.**
- (C) Keys: $B \rightarrow b$, $a \rightarrow a$, $N \rightarrow n$, a , N , a . Sorting by key puts the three 'a's first (in original order), then 'B', then the two 'N's. Result ['a', 'a', 'a', 'B', 'N', 'N']. **True.** (Without the key, uppercase letters sort before lowercase: ['B', 'N', 'N', 'a', 'a', 'a'].)
- (D) $*$ on a list means repetition: [1, 2, 3, 1, 2, 3], not element-wise multiplication. **False.**

Trap: (B) — most people expect 3 from school rounding.

Q.11 [NAT · 2 marks · Python — counting recursive calls]**Answer: 59**

Concept: without memoisation every call re-expands its whole recursion tree. The number of calls satisfies $C(n) = 1 + C(n-1) + C(\lfloor n/2 \rfloor)$ for $n \geq 2$ and $C(0) = C(1) = 1$.

Bottom-up table:

- $C(0) = C(1) = 1$
- $C(2) = 1 + C(1) + C(1) = 3$
- $C(3) = 1 + C(2) + C(1) = 5$
- $C(4) = 1 + C(3) + C(2) = 9$
- $C(5) = 1 + C(4) + C(2) = 13$
- $C(6) = 1 + C(5) + C(3) = 19$
- $C(7) = 1 + C(6) + C(3) = 25$
- $C(8) = 1 + C(7) + C(4) = 35$
- $C(9) = 1 + C(8) + C(4) = 45$
- $C(10) = 1 + C(9) + C(5) = 59$

`global calls` makes the increment update the module-level counter (without it, `calls += 1` would raise `UnboundLocalError`).

(The returned value `f(10)` is 30, but only the call count is printed.)

Trap: forgetting to count the base-case calls ($n \leq 1$), which are leaves of the recursion tree; with `lru_cache` only 10 distinct calls ($n = 1..10$) would remain.

Q.12 [MCQ · 2 marks · Merge sort — bottom-up passes]**Answer: (A)**

Concept: bottom-up merge sort doubles the length of sorted runs each pass; a leftover short run at the end is merged with whatever partner exists (possibly an empty one).

Pass $w = 1$ (blocks of 2): $[9, 4] \rightarrow [4, 9]$; $[7, 1] \rightarrow [1, 7]$; $[8, 2] \rightarrow [2, 8]$; $[5]$ (alone) $\rightarrow [5]$. Array: $[4, 9, 1, 7, 2, 8, 5]$.

Pass $w = 2$ (blocks of 4): $[4, 9] + [1, 7] \rightarrow [1, 4, 7, 9]$; block starting at index 4 is $[2, 8] + [5] \rightarrow [2, 5, 8]$. Array: **$[1, 4, 7, 9, 2, 5, 8]$** .

(Pass $w = 4$ then merges $[1, 4, 7, 9]$ with $[2, 5, 8]$ to finish.)

Options:

- (B) is the state after only the $w = 1$ pass.
- (C) fails to merge the short trailing run $[5]$ with $[2, 8]$.
- (D) is the fully sorted array (after $w = 4$).

Tip: bottom-up merge sort needs $\lceil \log_2 n \rceil$ passes — 3 for $n = 7$.

Bottom-up merge sort passes

	0	1	2	3	4	5	6
start	9	4	7	1	8	2	5
$w = 1$	4	9	1	7	2	8	5
$w = 2$	1	4	7	9	2	5	8
$w = 4$	1	2	4	5	7	8	9

Q.13 [MSQ · 2 marks · Binary search trees — reconstruction from post-order]**Answer: (A), (C), (D)**

Concept: in a BST, the in-order traversal is the sorted key list, so post-order alone determines the tree: the last element is the root, keys smaller than it form the left subtree (a contiguous prefix of the post-order), larger keys the right subtree.

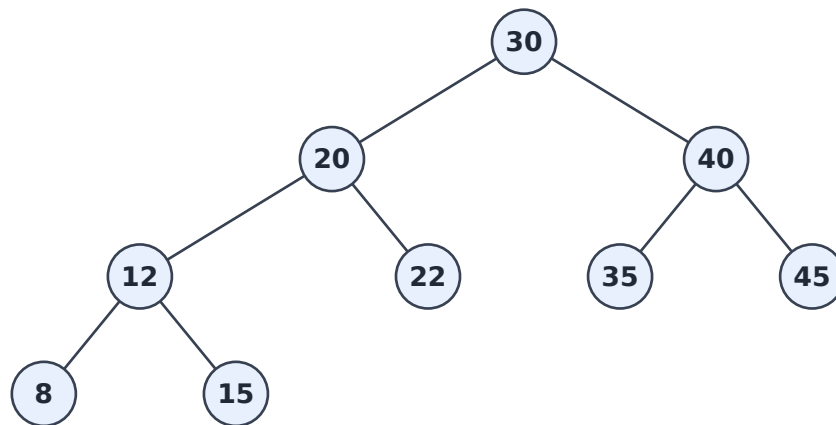
Reconstruction:

- Root 30. Left part (< 30): 8, 15, 12, 22, 20 $\rightarrow$ root 20; its left part 8, 15, 12 $\rightarrow$ root 12 with children 8 and 15; its right part 22 $\rightarrow$ leaf.
- Right part (> 30): 35, 45, 40 $\rightarrow$ root 40 with children 35 and 45.

Verdicts:

- (A) in-order predecessor of 30 = maximum of the left subtree = 22 (a leaf). It is copied into the root and removed. **True.**
- (B) leaves: 8, 15, 22, 35, 45 $\rightarrow$ 5, not 4. **False.**
- (C) pre-order: 30, 20, 12, 8, 15, 22, 40, 35, 45. **True.**
- (D) longest path 30 $\rightarrow$ 20 $\rightarrow$ 12 $\rightarrow$ 8 (or $\rightarrow$ 15) has 3 edges. **True.**

Trap: in (A), taking the predecessor to be the left child (20) — it is the *rightmost* node of the left subtree.



BST reconstructed from the post-order

Q.14 [NAT · 2 marks · BFS — shortest path in a grid maze]**Answer: 15**

Concept: on an unweighted grid, BFS from S labels every reachable cell with its shortest distance; the first time T is dequeued its label is the answer.

The wall column 4 (rows 0–3) separates S from T, so any path must go round through row 4.

BFS distances (see the solution table):

- Row 0: 0, 1, 2, 3 $\rightarrow$ the only way down is column 3: (1,3) = 4, (2,3) = 5.
- From (2,3) go left: (2,2) = 6, (2,1) = 7, (2,0) = 8.
- Two ways down to row 4: via (3,2) = 7 $\rightarrow$ (4,2) = 8, or via (3,0) = 9 $\rightarrow$ (4,0) = 10.
- The shorter one continues (4,3) = 9, (4,4) = 10, (4,5) = 11, then up column 5: (3,5) = 12, (2,5) = 13, (1,5) = 14, (0,5) = **15**.

Path: S $\rightarrow$ (0,3) $\rightarrow$ (2,3) $\rightarrow$ (2,2) $\rightarrow$ (4,2) $\rightarrow$ (4,5) $\rightarrow$ T: 3 + 2 + 1 + 2 + 3 + 4 = 15 moves.

Trap: missing the gap at (3,2) and going down at column 0 (19 moves), or computing the Manhattan distance 5, which ignores the walls.

BFS distance of every open cell

	0	1	2	3	4	5
0	0	1	2	3	#	15
1	#	#	#	4	#	14
2	8	7	6	5	#	13
3	9	#	7	#	#	12
4	10	9	8	9	10	11

Q.15 [MCQ · 2 marks · Python — class vs instance attributes]**Answer: (B)**

Concept: attribute *lookup* (`self.kids`) searches the instance first and then the class; attribute *assignment* (`self.count = ...`, `c.kids = []`) always creates or rebinds an **instance** attribute and never touches the class attribute.

Trace:

- **Node(1):** `Node.count = 1`; `a.count = 10` (instance attribute shadows the class one).
- **Node(2):** `Node.count = 2`; its `count = 20`. `.add(a)`: the node has no own `kids`, so `self.kids` is the class list $\rightarrow$ class list = `[a]`. `add` returns the `Node(2)` object $\rightarrow$ `b`.
- **Node(3):** `Node.count = 3`; `c.count = 30`. `c.add(b)` $\rightarrow$ class list = `[a, b]`.
- `c.kids = []` gives `c` its own empty list; the class list is unchanged.

Printed: `Node.count = 3`; `a.count = 10`; `a.kids` (class list) has length 2; `c.kids` (instance list) has length 0 $\rightarrow$ `3 10 2 0`.

Options: (A) reads `a.count` from the class-level counter. (C) assumes each node has its own `kids` list. (D) forgets `c.add(b)` also appended to the shared list.

Trap: a mutable *class* attribute used as if it were per-instance — the classic shared-list bug.

Q.16 [NAT · 2 marks · Heaps — streaming k-th largest with a min-heap]**Answer: 4**

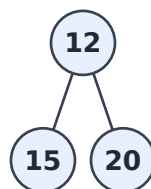
Concept: a min-heap of size `k` holds the `k` largest values seen so far, and its root is the current `k`-th largest. A new value enters only if it beats the root. Cost: $O(n \log k)$.

Trace (heap contents as a set; root = minimum):

- `5, 12, 3` $\rightarrow$ `{3, 5, 12}`, root 3
- `8 > 3` $\rightarrow$ replace (1) $\rightarrow$ `{5, 8, 12}`, root 5
- `15 > 5` $\rightarrow$ replace (2) $\rightarrow$ `{8, 12, 15}`, root 8
- `7 < 8` $\rightarrow$ discard
- `10 > 8` $\rightarrow$ replace (3) $\rightarrow$ `{10, 12, 15}`, root 10
- `20 > 10` $\rightarrow$ replace (4) $\rightarrow$ `{12, 15, 20}`, root 12
- `1` $\rightarrow$ discard; `11 < 12` $\rightarrow$ discard

Replacements = **4**; the final root 12 is the 3rd largest value of the stream (20, 15, 12).

Trap: using a *max*-heap here — its root is the largest value, which tells you nothing about whether a new value belongs in the top 3.



Final min-heap: root 12 is the 3rd largest

Q.17 [MSQ · 2 marks · Graph theory — bridges, cut vertices, bipartiteness]

Answer: (B), (C)

Concept: an edge is a bridge iff it lies on no cycle; a vertex is a cut vertex iff removing it increases the number of components. A graph is bipartite iff it has no odd cycle.

Cycles: triangle A–B–C and triangle D–E–F. Edges outside every cycle: C–D, F–G, G–H.

- (A) Triangles are odd cycles → not bipartite. **False.**
- (B) Without D: {A, B, C} and {E, F, G, H} (E–F–G–H still connected). Exactly 2. **True.**
- (C) Bridges are exactly C–D, F–G, G–H → 3. **True.**
- (D) Cut vertices: C (separates A, B), D, F (separates G, H), G (separates H) → 4. **False.**

Trap: in (D) forgetting G — an internal vertex of a pendant path is always a cut vertex. Also note: an endpoint of a bridge is a cut vertex unless it has degree 1 (H is not a cut vertex).

Method tip: for small graphs, first mark every edge that lies on a cycle (here the six triangle edges); the remaining edges are exactly the bridges. Here every endpoint of a bridge that has degree ≥ 2 (C, D, F, G) is a cut vertex; in general you can also confirm a cut vertex by deleting it and checking connectivity.

Q.18 [NAT · 2 marks · Hashing — expected number of empty slots]

Answer: 3.58 to 3.60

Concept: use linearity of expectation with indicator variables — no need to work out the distribution of empty slots.

Let $X_j = 1$ if slot j is empty after all insertions. A given key misses slot j with probability $7/8$, and the keys are independent, so

$$P(X_j = 1) = (7/8)^6.$$

$$E[\text{\#empty}] = \sum_{j=1}^8 E[X_j] = 8 \cdot (7/8)^6.$$

$$\text{Compute: } (7/8)^2 = 0.765625; (7/8)^3 = 0.669922; (7/8)^6 = 0.669922^2 \approx 0.448795.$$

$$8 \times 0.448795 \approx \mathbf{3.59}.$$

So on average fewer than half of the 8 slots are used even though there are 6 keys — collisions are common long before the table is 'full' (birthday paradox).

Why not count directly? The number of empty slots is a dependent sum (if one slot is empty, the others are slightly more likely to be hit), but linearity of expectation holds regardless of dependence, so the indicator method is exact.

Related quantity: the expected number of *occupied* slots is $8 - 3.59 \approx 4.41$, and the expected number of colliding key pairs is $C(6, 2)/8 = 1.875$.

Trap: answering $8 - 6 = 2$ (that assumes no collisions at all), or using $(1/8)^6$.

Q.19 [MCQ · 2 marks · Complexity — nested loops with halving and doubling]

Answer: (D)

Concept: the outer loop halves i , so it runs about $\log_2 n$ times; for a given i , the inner loop doubles j from 1 until it reaches i , i.e. $\lceil \log_2 i \rceil$ iterations.

Take $n = 2^k$. The outer loop sees $i = 2^k, 2^{k-1}, \dots, 2$ and the inner loop runs $k, k-1, \dots, 1$ times respectively:

$$c = k + (k-1) + \dots + 1 = k(k+1)/2 = \Theta(k^2) = \mathbf{\Theta(\log^2 n)}.$$

$$\text{Check: } \text{work}(2) = 1, \text{work}(4) = 3, \text{work}(8) = 6, \text{work}(16) = 10, \text{work}(64) = 21.$$

Options:

- (A) counts only the outer loop.

- (B) would need a linear inner loop with a linear outer loop.
- (C) would hold if the inner loop ran i times ($j += 1$) — then $c = n + n/2 + \dots \approx 2n$.

Trap: multiplying 'log n outer $\times$ log n inner' gives the right order here, but the careful sum shows the constant is $1/2$: $c \approx (\log_2 n)^2/2$.

Q.20 [MSQ · 2 marks · Linked lists + stacks — palindrome check]

Answer: (B), (D)

Concept: the fast pointer moves two steps per iteration, so when it stops, slow is at the middle and the stack holds the first $\lfloor n/2 \rfloor$ values (in reverse order). For odd n (**fast** not None) the middle node is skipped. The second half is then compared with the popped values.

- (A) $n = 5$: stack [4, 5]; skip middle 5; compare 4 with pop 5 $\rightarrow$ mismatch $\rightarrow$ False. **False.** (Reversed it reads 4 4 5 5 4.)
- (B) $n = 9$: the loop runs while fast and fast.next exist: fast visits nodes 1, 3, 5, 7, 9 $\rightarrow$ 4 iterations $\rightarrow$ 4 pushes; afterwards the stack only shrinks. **True.**
- (C) $n = 4$: stack [1, 2]; fast becomes None $\rightarrow$ no skip; compare 1 with pop 2 $\rightarrow$ False. **False.**
- (D) $n = 5$: stack [1, 2]; slow at 3, fast at the last node $\rightarrow$ skip 3. Compare 2 = pop 2, 1 = pop 1 $\rightarrow$ True. **True.**

Trap: forgetting to skip the middle element for odd lengths — then (D) would compare 3 against 2 and fail. The **if fast:** line handles exactly that.

Mock Test 33

Full-Syllabus Mock — Paper 3

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — closures and late binding]

Consider the following Python program. What is printed?

Python

```
1 fs = []
2 for i in range(3):
3     fs.append(lambda x, i=i: x + i)
4     fs.append(lambda x: x * i)
5 print([f(10) for f in fs])
```

- (A) [10, 0, 11, 10, 12, 20]
- (B) [10, 20, 11, 20, 12, 20]
- (C) [12, 20, 12, 20, 12, 20]
- (D) [10, 20, 11, 10, 12, 0]

Q.2 [NAT · numerical answer | 1 mark | Python — floor division and modulo with negatives]

The value printed by the Python statement `print(-17 // 5 + -17 % 5 * 2 + 17 % -5)` is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — infix to postfix]

Using the usual precedences (^ highest and **right**-associative; *, / next, left-associative; +, - lowest, left-associative), the postfix form of

$a + b * (c - d) / e ^ f ^ g$

is

- (A) $a \ b \ c \ d \ - \ * \ e \ f \ g \ ^ \ ^ \ / \ +$
- (B) $a \ b \ c \ d \ - \ * \ e \ f \ ^ \ g \ ^ \ / \ +$
- (C) $a \ b \ c \ d \ - \ e \ f \ g \ ^ \ ^ \ / \ * \ +$
- (D) $a \ b \ c \ d \ - \ * \ / \ e \ f \ g \ ^ \ ^ \ +$

Q.4 [NAT · numerical answer | 1 mark | Python — queue and stack interplay]

Consider the following Python program. The value printed is _____.

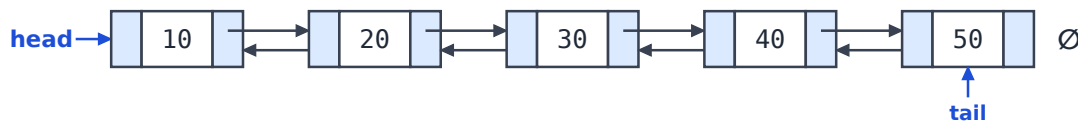
Python

```
1 from collections import deque
2 q = deque([1, 2, 3, 4, 5, 6])
3 st = []
4 for _ in range(3):
5     st.append(q.popleft())
6 while st:
7     q.append(st.pop())
8 for _ in range(3):
9     q.append(q.popleft())
10 print(int(''.join(map(str, q))))
```

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Doubly linked list — deletion]

In the doubly linked list below every node has fields **prev**, **val**, **next**. The pointer **p** refers to the node with value 30 (neither first nor last). Which code fragment correctly unlinks **p**, so that the list reads 10, 20, 40, 50 in both directions?



- (A) `p.next.prev = p.prev; p.prev = p.next`
- (B) `p.prev.next = p.next; p.next.prev = p.prev.next`
- (C) `p.prev.next = p.next; p.next.prev = p.prev`
- (D) `p.prev.next = p.next.next; p.next.prev = p.prev`

Q.6 [NAT · numerical answer | 1 mark | Binary trees — node counting]

A binary tree has exactly 20 leaves and exactly 9 nodes that have exactly one child. The total number of nodes in the tree is _____.

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Hashing — choice of table size]

The 20 keys 4, 8, 12, ..., 80 (all multiples of 4) are inserted into a hash table of size $m = 12$ with $h(k) = k \bmod 12$ and separate chaining. The number of non-empty slots and the length of the longest chain are, respectively,

- (A) 3 and 7
- (B) 12 and 2
- (C) 3 and 6
- (D) 4 and 5

Q.8 [MSQ · one or more correct | 1 mark | Binary search — termination bug]

Consider the following function on the sorted list $a = [2, 4, 6, 8, 10]$. For which of the following calls does the **while** loop run **forever**?

Python

```
1 def bs(a, x):
2     lo, hi = 0, len(a) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid
7         else:
8             hi = mid
9     return lo
```

- (A) $bs(a, 10)$
- (B) $bs(a, 7)$
- (C) $bs(a, 2)$
- (D) $bs(a, 6)$

Q.9 [MSQ · one or more correct | 1 mark | Sorting — stability]

The records (3,a), (1,b), (3,c), (2,d), (1,e) are sorted by their **numeric key only** (letters are just labels). Selection sort is the standard version (find the minimum of $A[i..n-1]$ with strict $<$, then swap it with $A[i]$); insertion sort shifts while the key is strictly greater. Which of the following statements is/are TRUE?

- (A) Selection sort outputs (1,b) before (1,e)
- (B) Selection sort outputs (3,c) before (3,a)
- (C) Insertion sort outputs (3,a) before (3,c)
- (D) Selection sort is stable on this input

Q.10 [MCQ · single correct | 1 mark | Graph traversal — tree and non-tree edges]

An undirected simple graph G has 12 vertices, 20 edges and exactly 3 connected components. A complete DFS (restarting from an unvisited vertex until every vertex is visited) is run on G . The number of edges of G that are **not** tree edges of the resulting DFS forest is

- (A) 12
- (B) 9
- (C) 8
- (D) 11

Q.11 [NAT · numerical answer | 2 marks | Python — counting recursive calls]

Consider the following Python program. The value printed is _____.

Python

```
1 calls = 0
2
3 def f(n):
4     global calls
5     calls += 1
6     if n <= 2:
7         return n
8     return f(n - 1) + f(n - 3)
9
10 f(9)
11 print(calls)
```

Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | BST — reconstruction from pre-order]

The pre-order traversal of a binary search tree with distinct keys is

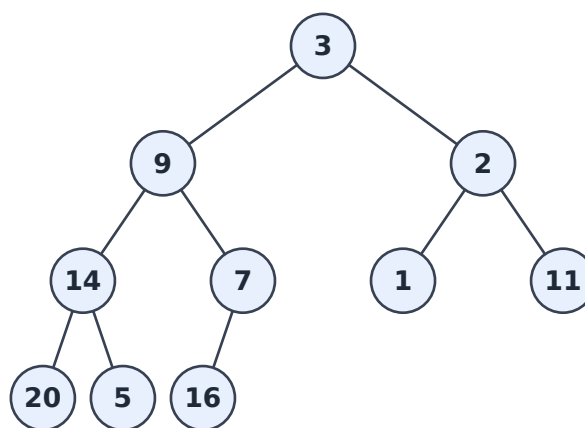
30, 20, 10, 25, 22, 28, 40, 35, 50, 45

Which of the following statements is/are TRUE?

- (A) The in-order successor of 28 is 35
- (B) The tree has exactly 4 leaves
- (C) The post-order traversal ends with 45, 50, 40, 30
- (D) The level-order traversal is 30, 20, 40, 10, 25, 35, 50, 22, 28, 45

Q.13 [NAT · numerical answer | 2 marks | Heaps — bottom-up build-heap]

The array [3, 9, 2, 14, 7, 1, 11, 20, 5, 16] (0-based) is converted into a **max**-heap with the bottom-up build-heap algorithm: for $i = \lfloor n/2 \rfloor - 1$ down to 0, sift $A[i]$ down, each step swapping it with its larger child while that child is larger. The total number of swaps performed is _____.



Input array viewed as a complete binary tree

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Quicksort — Hoare partition]

The Hoare partition scheme below (pivot = $A[\text{lo}]$) is called as **hoare**(A , 0, 7) on $A = [6, 3, 9, 6, 1, 8, 2, 7]$. What are the contents of A and the returned value afterwards?

Python

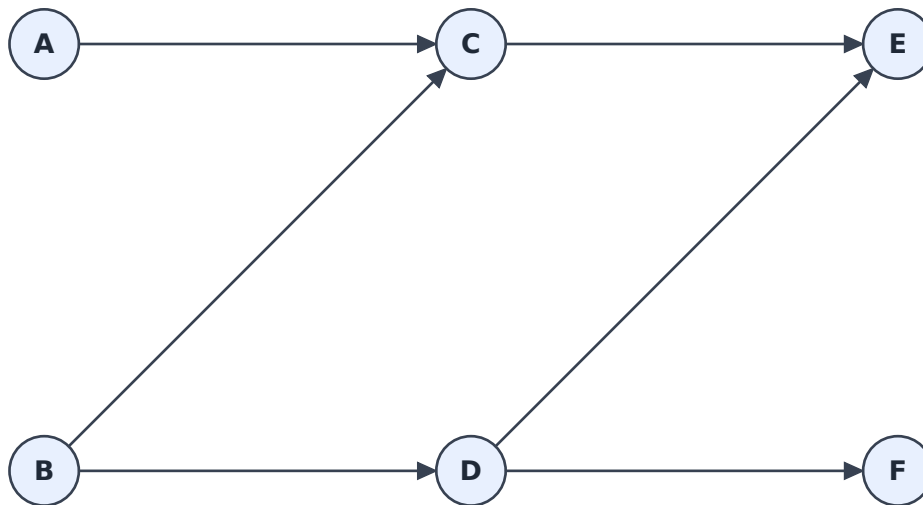
```

1 def hoare(A, lo, hi):
2     p = A[lo]
3     i, j = lo - 1, hi + 1
4     while True:
5         i += 1
6         while A[i] < p:
7             i += 1
8         j -= 1
9         while A[j] > p:
10            j -= 1
11        if i >= j:
12            return j
13        A[i], A[j] = A[j], A[i]
```

- (A) $A = [1, 3, 2, 6, 9, 8, 6, 7]$, returns 3
- (B) $A = [2, 3, 1, 6, 9, 8, 6, 7]$, returns 4
- (C) $A = [2, 3, 1, 6, 9, 8, 6, 7]$, returns 3
- (D) $A = [2, 3, 1, 6, 6, 8, 9, 7]$, returns 3

Q.15 [NAT · numerical answer | 2 marks | DAGs — counting topological orders]

The number of distinct topological orderings of the directed acyclic graph below is _____.



Answer: _____ (enter a numerical value)

Q.16 [MCQ · single correct | 2 marks | Complexity of a loop nest]

What is the time complexity of `work(n)` as a function of n (count the `c += 1` steps)?

Python

```

1 def work(n):
2     c = 0
3     i = 1
4     while i <= n:
5         j = 1
6         while j <= n:
7             c += 1
8             j += i
9         i *= 2
10    return c
  
```

- (A) $\Theta(n \log n)$
- (B) $\Theta(n)$
- (C) $\Theta(n^2)$
- (D) $\Theta(\log^2 n)$

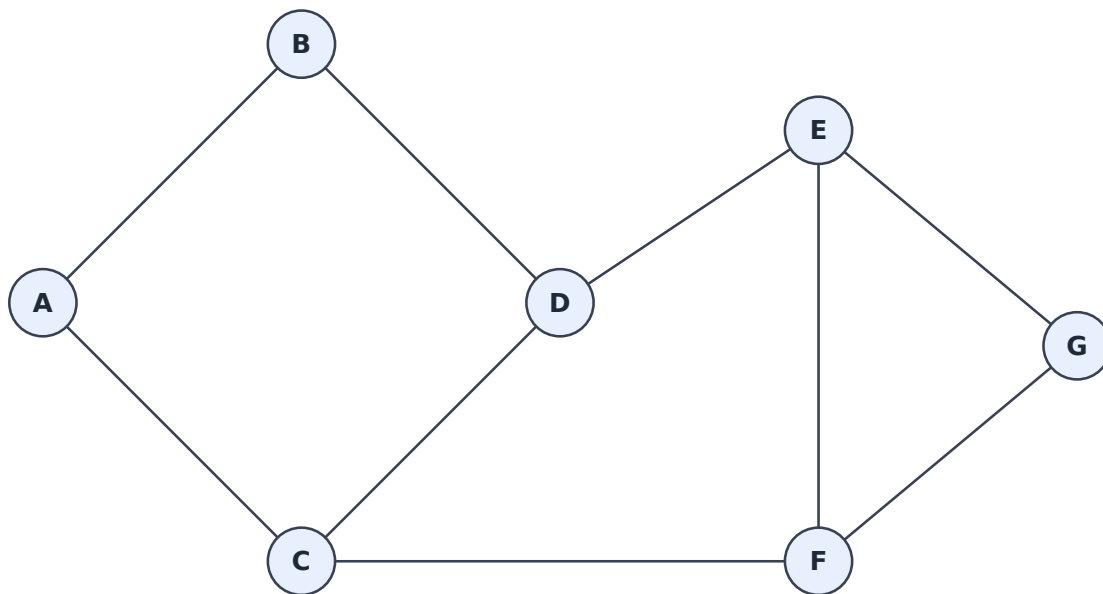
Q.17 [NAT · numerical answer | 2 marks | Merge sort — counting inversions]

Inversions of $A = [5, 9, 2, 7, 1, 8, 3, 6]$ are counted with the divide-and-conquer merge-sort method: inversions inside the left half + inside the right half + *split* inversions (pairs with one element in each half) counted during the final merge. The number of split inversions counted during the **final** merge (left half $[5, 9, 2, 7]$, right half $[1, 8, 3, 6]$) is _____.

Answer: _____ (enter a numerical value)

Q.18 [MSQ · one or more correct | 2 marks | DFS and BFS trees of an undirected graph]

DFS and BFS are each run from vertex A on the undirected graph below, visiting neighbours in alphabetical order. Which of the following statements is/are TRUE?



- (A) In the DFS, G is discovered before E
- (B) The DFS produces exactly 3 back edges
- (C) Every vertex of the DFS tree has at most one child (the DFS tree is a path)
- (D) In the BFS tree the largest distance (level) of any vertex from A is 3

Q.19 [MSQ · one or more correct | 2 marks | Linear probing — clustering effects]

Keys 25, 47, 14, 36, 58, 70, 3 are inserted in that order into an initially empty hash table of size 11 with $h(k) = k \bmod 11$ and linear probing (step +1). Probes are counted as slots inspected, including the final one. Which of the following statements is/are TRUE?

- (A) An unsuccessful search for a key whose home slot is 3 makes 8 probes
- (B) The average number of probes over the seven successful searches is 27/7
- (C) Key 70 is stored in slot 8
- (D) The table contains a run of 8 consecutive occupied slots

Q.20 [MCQ · single correct | 2 marks | Python — generator and iterator exhaustion]

Consider the following Python program. What is printed?

Python

```
1 g = (x * x for x in range(5))
2 a = sum(g)
3 b = sum(g)
4 it = iter([1, 2, 3, 4])
5 pairs = list(zip(it, it))
6 print(a, b, pairs)
```

- (A) 30 0 [(1, 1), (2, 2), (3, 3), (4, 4)]
- (B) 30 30 [(1, 1), (2, 2), (3, 3), (4, 4)]
- (C) 30 30 [(1, 2), (3, 4)]
- (D) 30 0 [(1, 2), (3, 4)]

Answer Key — Mock Test 33

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — closures and late binding
2	NAT	1	-1	Python — floor division and modulo with negatives
3	MCQ	1	(A)	Stacks — infix to postfix
4	NAT	1	321456	Python — queue and stack interplay
5	MCQ	1	(C)	Doubly linked list — deletion
6	NAT	1	48	Binary trees — node counting
7	MCQ	1	(A)	Hashing — choice of table size
8	MSQ	1	(A), (B), (D)	Binary search — termination bug
9	MSQ	1	(A), (B), (C)	Sorting — stability
10	MCQ	1	(D)	Graph traversal — tree and non-tree edges
11	NAT	2	37	Python — counting recursive calls
12	MSQ	2	(C), (D)	BST — reconstruction from pre-order
13	NAT	2	8	Heaps — bottom-up build-heap
14	MCQ	2	(C)	Quicksort — Hoare partition
15	NAT	2	14	DAGs — counting topological orders
16	MCQ	2	(B)	Complexity of a loop nest
17	NAT	2	10	Merge sort — counting inversions
18	MSQ	2	(B), (C), (D)	DFS and BFS trees of an undirected graph
19	MSQ	2	(A), (B), (C)	Linear probing — clustering effects
20	MCQ	2	(D)	Python — generator and iterator exhaustion

Detailed Solutions — Mock Test 33

Q.1 [MCQ · 1 mark · Python — closures and late binding]

Answer: (B)

A lambda body looks up free variables **when it is called** (late binding), but a default argument is evaluated **when the lambda is created**.

- `lambda x, i=i: x + i` freezes the current `i` (0, 1, 2) → 10, 11, 12.
- `lambda x: x * i` reads the global `i` at call time; after the loop `i = 2` → every one gives $10 \times 2 = 20$.

Interleaved: **[10, 20, 11, 20, 12, 20]**.

- (A) assumes the multiplying lambdas captured `i = 0, 1, 2` at creation time.
- (C) assumes the default-argument lambdas are also late-bound.
- (D) mixes the two models in reverse order.

Trap: closures capture *variables*, not values. **Fix:** use a default argument or `functools.partial` to freeze a value.

Q.2 [NAT · 1 mark · Python — floor division and modulo with negatives]

Answer: -1

Python's `//` rounds toward $-\infty$ and `%` satisfies $a == (a // b) \cdot b + a \% b$, so the remainder takes the **sign of the divisor**. Unary minus binds tighter than `//`, `%`, `*`; and `%`, `*`, `//` share one precedence level (left to right).

- `-17 // 5 = [-3.4] = -4`
- `-17 % 5 = -17 - (-4)(5) = 3`; then `3 * 2 = 6`
- `17 % -5: 17 // -5 = [-3.4] = -4`, remainder $17 - (-4)(-5) = -3$

Sum: $-4 + 6 - 3 = -1$.

Trap: C/Java truncate toward zero, which would give $-3 + (-2) \cdot 2 + 2 = -5$. In Python, `a % b` always has the sign of `b`.

Q.3 [MCQ · 1 mark · Stacks — infix to postfix]

Answer: (A)

Fully parenthesise first: $\wedge$ is right-associative, so $e \wedge f \wedge g = e \wedge (f \wedge g)$; $*$ and $/$ are left-associative, so $b * (c - d) / X = (b * (c - d)) / X$.

Expression: $a + ((b * (c - d)) / (e \wedge (f \wedge g)))$.

- $b * (c - d) \rightarrow b \ c \ d \ - \ *$
- $e \wedge (f \wedge g) \rightarrow e \ f \ g \ \wedge \ \wedge$
- division $\rightarrow b \ c \ d \ - \ * \ e \ f \ g \ \wedge \ \wedge \ /$
- plus $a \rightarrow a \ b \ c \ d \ - \ * \ e \ f \ g \ \wedge \ \wedge \ / \ +$
- (B) `e f ^ g ^` treats $\wedge$ as left-associative $((e \wedge f) \wedge g)$.
- (C) groups $b * ((c - d) / \dots)$, i.e. treats $*$ and $/$ as right-associative.
- (D) places $/$ before its right operand is complete — not a valid postfix of this expression.

Tip: in the shunting-yard algorithm, an incoming $\wedge$ does **not** pop a $\wedge$ on the stack (right associativity), while an incoming $/$ does pop a $*$ (equal precedence, left).

Q.4 [NAT · 1 mark · Python — queue and stack interplay]

Answer: 321456

A classic 'reverse the first `k` elements of a queue' routine ($k = 3$).

- Dequeue 1, 2, 3 and push them: queue [4, 5, 6], stack [1, 2, 3] (top 3).
- Pop the stack into the queue (LIFO reverses): queue [4, 5, 6, 3, 2, 1].
- Rotate $n - k = 3$ times (dequeue + enqueue): [3, 2, 1, 4, 5, 6].

Joined as digits: **321456**.

Trap: forgetting that the final rotation brings the reversed block to the **front** (answer 456321), or reversing twice (123456). **Tip:** the routine costs $\Theta(n)$ with one auxiliary stack of size k .

Q.5 [MCQ · 1 mark · Doubly linked list — deletion]

Answer: (C)

To unlink p , its predecessor must point forward to p 's successor and its successor must point back to p 's predecessor. The two assignments are independent, so either order works.

- (A) $40.\text{prev} = 20$ is right, but $p.\text{prev} = p.\text{next}$ only changes p 's own field; $20.\text{next}$ still points to 30, so the forward list is unchanged.
- (B) after the first statement $p.\text{prev}.\text{next}$ is 40, so the second sets $40.\text{prev} = 40$ — a self-loop; backward traversal breaks.
- (C) $20.\text{next} = 40$ and $40.\text{prev} = 20$. **Correct.**
- (D) $20.\text{next} = 50$ skips 40 in the forward direction.

Trap: (B) — after modifying a link, an expression that goes *through* that link no longer means what it did. **Tip:** deletion is $\Theta(1)$ in a doubly linked list given p ; a singly linked list needs the predecessor.

Q.6 [NAT · 1 mark · Binary trees — node counting]

Answer: 48

Let n_0, n_1, n_2 be the numbers of nodes with 0, 1, 2 children. Counting edges two ways:

- edges = $n - 1 = n_0 + n_1 + n_2 - 1$ (every node except the root has a parent edge)
- edges = $n_1 + 2n_2$ (child pointers)

Equating gives $n_2 = n_0 - 1$ — independent of n_1 .

So $n_2 = 19$ and $n = 20 + 9 + 19 = \mathbf{48}$.

Trap: thinking the one-child nodes change the $n_2 = n_0 - 1$ relation, or adding 1 for the root separately. **Tip:** a *full* binary tree ($n_1 = 0$) with L leaves has exactly $2L - 1$ nodes.

Q.7 [MCQ · 1 mark · Hashing — choice of table size]

Answer: (A)

$4k \bmod 12 = 4 \cdot (k \bmod 3)$, so only slots **0, 4, 8** can ever be used — $\gcd(4, 12) = 4$ means the stride shares a factor with m and only $m / \gcd = 3$ slots are reachable.

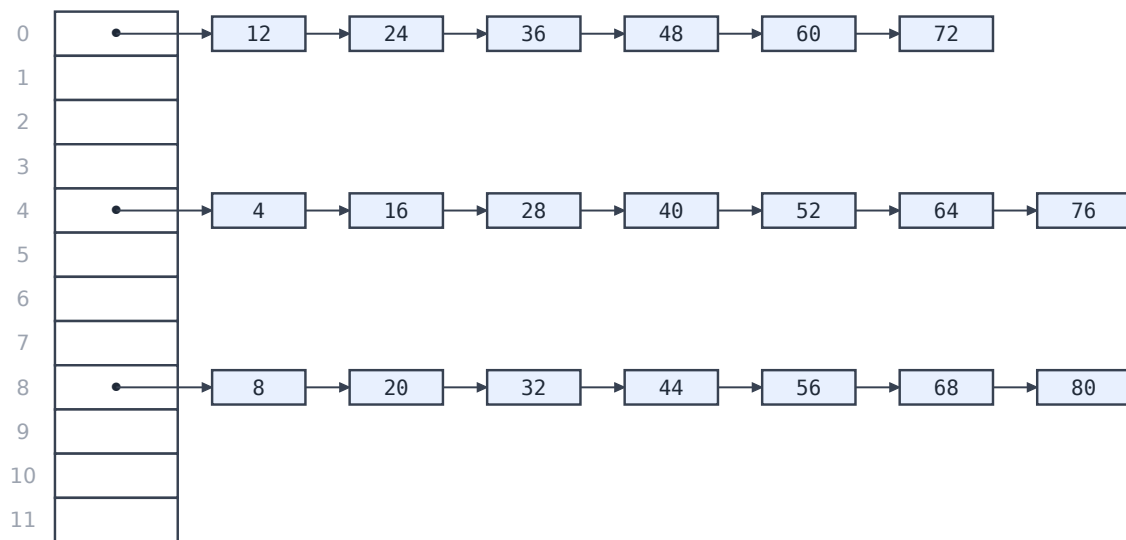
For $k = 1 \dots 20$:

- $k \equiv 1 \pmod{3}$: $k = 1, 4, \dots, 19 \rightarrow 7$ keys $\rightarrow$ slot 4
- $k \equiv 2 \pmod{3}$: $k = 2, 5, \dots, 20 \rightarrow 7$ keys $\rightarrow$ slot 8
- $k \equiv 0 \pmod{3}$: $k = 3, 6, \dots, 18 \rightarrow 6$ keys $\rightarrow$ slot 0

Non-empty slots = **3**, longest chain = **7**.

- (B) is what a well-spread hash would give.
- (C) gets the slot count right but miscounts the chains.
- (D) assumes $m / \gcd = 4$ reachable slots.

Tip: choose m prime (e.g. 13) so that arithmetic patterns in the keys do not collapse onto a few slots; with $m = 13$ these keys would occupy all 13 slots.



Only three chains are used

Q.8 [MSQ · 1 mark · Binary search — termination bug]

Answer: (A), (B), (D)

The bug: $lo = mid$ instead of $lo = mid + 1$. When $hi = lo + 1$, $mid = lo$ (floor), and if $a[lo] < x$ the assignment $lo = mid$ changes nothing → infinite loop.

- (A) $x = 10$: $mid\ 2 \rightarrow lo\ 2$; $mid\ 3\ (8 < 10) \rightarrow lo\ 3$; $lo\ 3, hi\ 4 \rightarrow mid\ 3$ forever. **Loops.**
- (B) $x = 7$: $mid\ 2\ (6 < 7) \rightarrow lo\ 2$; $mid\ 3\ (8 \geq 7) \rightarrow hi\ 3$; $lo\ 2, hi\ 3 \rightarrow mid\ 2 \rightarrow lo\ 2$ forever. **Loops.**
- (C) $x = 2$: $mid\ 2 \rightarrow hi\ 2$; $mid\ 1\ (4 \geq 2) \rightarrow hi\ 1$; $mid\ 0\ (2 \geq 2) \rightarrow hi\ 0 \rightarrow stop$. Terminates.
- (D) $x = 6$: $mid\ 2\ (6 \geq 6) \rightarrow hi\ 2$; $mid\ 1\ (4 < 6) \rightarrow lo\ 1$; now $lo\ 1, hi\ 2 \rightarrow mid\ 1$ again → $lo\ 1$ forever. **Loops.**

Rule: with $mid = \lfloor (lo + hi)/2 \rfloor$ the lower bound must move to $mid + 1$; with $mid = \lceil (lo + hi)/2 \rceil$ the upper bound must move to $mid - 1$. Otherwise a 2-element range can stall.

Q.9 [MSQ · 1 mark · Sorting — stability]

Answer: (A), (B), (C)

A sort is *stable* if equal keys keep their input order. Insertion sort (strict $>$) is stable; selection sort's long-distance swap can jump an element over an equal one.

Selection sort trace:

- $i=0$: min key 1 is (1,b) at index 1 → swap with (3,a): (1,b) (3,a) (3,c) (2,d) (1,e)
- $i=1$: min of the rest is (1,e) at index 4 → swap with (3,a): (1,b) (1,e) (3,c) (2,d) (3,a)
- $i=2$: min (2,d) → swap with (3,c): (1,b) (1,e) (2,d) (3,c) (3,a)
- $i=3$: (3,c) vs (3,a): no strictly smaller → stays

Result: (1,b) (1,e) (2,d) **(3,c) (3,a)**.

- (A) **True** — the 1-keys happen to keep their order.
- (B) **True** — (3,a) was carried to the end by the swap at $i=1$.
- (C) insertion sort gives (1,b) (1,e) (2,d) (3,a) (3,c). **True.**
- (D) the 3-keys are reordered. **False.**

Trap: concluding stability from one pair that happens to stay in order (A).

Q.10 [MCQ · 1 mark · Graph traversal — tree and non-tree edges]**Answer: (D)**

A DFS (or BFS) forest contains one spanning tree per connected component. A spanning tree on c vertices has $c - 1$ edges, so the forest has $\sum(c_i - 1) = n - k$ edges.

- Tree edges = $12 - 3 = 9$
- Non-tree edges = $20 - 9 = 11$

(In an undirected DFS every non-tree edge is a back edge.)

- (A) $12 = 20 - 8$ follows from wrongly taking $n - k - 1 = 8$ tree edges.
- (B) 9 is the number of tree edges.
- (C) 8 uses $n - 1 = 11$ tree edges *and* subtracts one more.

Tip: the answer does not depend on the traversal order or on BFS vs DFS — only on n , $|E|$ and the number of components.

Q.11 [NAT · 2 marks · Python — counting recursive calls]**Answer: 37**

Let $C(n)$ be the number of calls made by $f(n)$, including itself. Then $C(n) = 1$ for $n \leq 2$ and $C(n) = 1 + C(n - 1) + C(n - 3)$ for $n \geq 3$ ($f(0)$ is reached from $f(3)$, so n never goes negative).

- $C(0) = C(1) = C(2) = 1$
- $C(3) = 1 + C(2) + C(0) = 3$
- $C(4) = 1 + 3 + 1 = 5$
- $C(5) = 1 + 5 + 1 = 7$
- $C(6) = 1 + 7 + 3 = 11$
- $C(7) = 1 + 11 + 5 = 17$
- $C(8) = 1 + 17 + 7 = 25$
- $C(9) = 1 + 25 + 11 = 37$

(The returned value $f(9) = 22$ is not printed.)

Trap: writing $C(n) = C(n - 1) + C(n - 3)$ without the $+1$ for the current call, or confusing the call count with the return value. `global calls` is required — without it `calls += 1` raises `UnboundLocalError`.

Q.12 [MSQ · 2 marks · BST — reconstruction from pre-order]**Answer: (C), (D)**

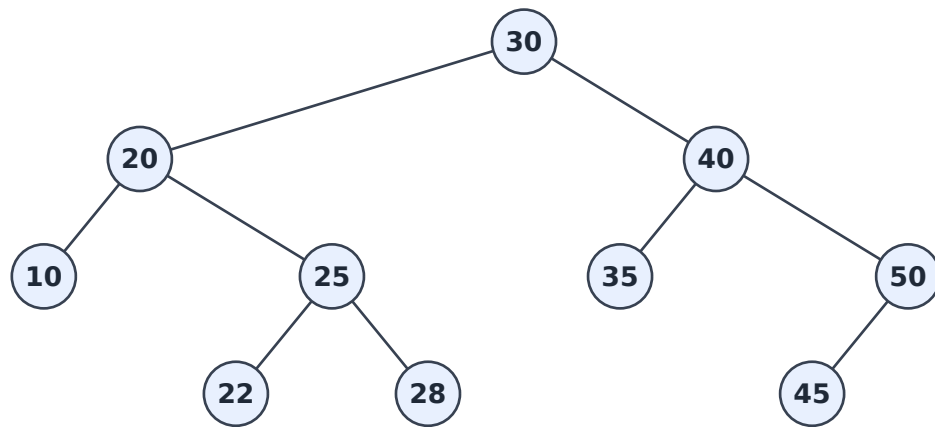
In a BST the pre-order sequence determines the tree: the first key is the root; the following keys smaller than it form the left subtree's pre-order, the larger ones the right subtree's. (Equivalently, insert the keys in pre-order sequence into an empty BST.)

- Root 30; left part 20, 10, 25, 22, 28; right part 40, 35, 50, 45.
- Left: 20 with left 10 and right 25 (whose children are 22 and 28).
- Right: 40 with left 35 and right 50 (whose left child is 45).

Post-order: 10, 22, 28, 25, 20, 35, 45, 50, 40, 30.

- (A) 28 has no right child, so its successor is the nearest ancestor for which 28 lies in the left subtree: **30**, not 35. **False.**
- (B) leaves: 10, 22, 28, 35, 45 $\rightarrow$ **5**. **False.**
- (C) **True.**
- (D) levels: [30] [20, 40] [10, 25, 35, 50] [22, 28, 45]. **True.**

Tip: the in-order sequence of a BST is just the sorted keys — 28's successor is the next larger key, 30.



Reconstructed BST

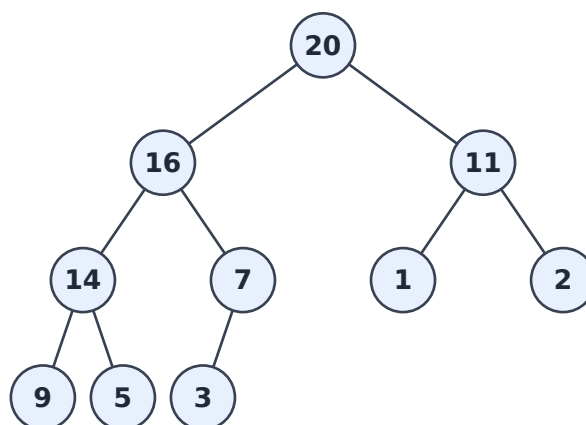
Q.13 [NAT · 2 marks · Heaps — bottom-up build-heap]**Answer: 8**

Build-heap processes internal nodes from the last one (index 4) back to the root; each sift-down may cascade several levels.

- $i=4$ (7): child 16 $\rightarrow$ swap (1) $\rightarrow$ [3, 9, 2, 14, 16, 1, 11, 20, 5, 7]
- $i=3$ (14): children 20, 5 $\rightarrow$ swap with 20 (2) $\rightarrow$ [3, 9, 2, 20, 16, 1, 11, 14, 5, 7]
- $i=2$ (2): children 1, 11 $\rightarrow$ swap with 11 (3) $\rightarrow$ [3, 9, 11, 20, 16, 1, 2, 14, 5, 7]
- $i=1$ (9): children 20, 16 $\rightarrow$ swap with 20 (4); now at index 3, children 14, 5 $\rightarrow$ swap with 14 (5) $\rightarrow$ [3, 20, 11, 14, 16, 1, 2, 9, 5, 7]
- $i=0$ (3): swap with 20 (6); at index 1 children 14, 16 $\rightarrow$ swap with 16 (7); at index 4 child 7 $\rightarrow$ swap (8) $\rightarrow$ [20, 16, 11, 14, 7, 1, 2, 9, 5, 3]

Total = **8** swaps.

Trap: stopping each sift-down after one level (that gives 5), or using repeated insertion (sift-up) instead, which performs a different set of swaps. **Tip:** build-heap is $\Theta(n)$ overall because most nodes are near the bottom.



Resulting max-heap

Q.14 [MCQ · 2 marks · Quicksort — Hoare partition]**Answer: (C)**

Hoare's scheme moves i right past elements $< p$ and j left past elements $> p$, then swaps. It returns j such that every element of $A[lo..j] \leq p \leq$ every element of $A[j+1..hi]$; the pivot is **not** necessarily at position j .

$p = 6$.

- Round 1: i stops at 0 (6 is not < 6); j : 7 ($7 > 6$) $\rightarrow$ 6 (2) stops. Swap $\rightarrow$ [2, 3, 9, 6, 1, 8, 6, 7]
- Round 2: i : 1 ($3 < 6$) $\rightarrow$ 2 (9) stops; j : 5 ($8 > 6$) $\rightarrow$ 4 (1) stops. Swap $\rightarrow$ [2, 3, 1, 6, 9, 8, 6, 7]
- Round 3: i : 3 (6) stops; j : 3 (6) stops. $i \geq j \rightarrow$ **return 3**.

Left part $A[0..3] = 2, 3, 1, 6$ (all ≤ 6); right part $A[4..7] = 9, 8, 6, 7$ (all ≥ 6).

- (A) gets the first swap wrong (1 is not reached by j in round 1).
- (B) is right about A but returns $i + 1$ -style index 4 (that is Lomuto thinking).
- (D) moves the second 6 next to the first, as if the pivot were placed in final position.

Trap: with Hoare partition, recursion must be on $[lo, j]$ and $[j+1, hi]$ — **not** $[lo, j-1]$ — because the pivot may sit anywhere in the left part.



After partition: $A[0..3] \leq 6 \leq A[4..7]$

Q.15 [NAT · 2 marks · DAGs — counting topological orders]

Answer: 14

A topological order lists each vertex after all its predecessors. Count by branching on which source is output next (only vertices with no remaining in-edges may be chosen).

Constraints: $A \rightarrow C$, $B \rightarrow C$, $B \rightarrow D$, $C \rightarrow E$, $D \rightarrow E$, $D \rightarrow F$. Initial sources: A, B.

Case 1 — A first. Now only B is a source $\rightarrow$ B second. Remaining C, D, E, F with $C \rightarrow E$, $D \rightarrow E$, $D \rightarrow F$:

- C next: then D, then E and F in either order $\rightarrow 2$
- D next: remaining C, E, F with only $C \rightarrow E \rightarrow 3$ orders (F can be in any of 3 slots)
- Case 1 total = 5

Case 2 — B first. Sources now A, D:

- A next: remaining C, D, E, F with $C \rightarrow E$, $D \rightarrow E$, $D \rightarrow F \rightarrow 5$ (same as above)
- D next: remaining A, C, E, F with $A \rightarrow C \rightarrow E$ and F free $\rightarrow$ F can go in any of 4 slots $\rightarrow 4$
- Case 2 total = 9

Total = $5 + 9 = 14$.

Trap: multiplying independent-looking choices (e.g. $2 \times 2 \times \dots$) — the choices interact, so systematic case analysis (or DP over subsets) is needed.

Q.16 [MCQ · 2 marks · Complexity of a loop nest]

Answer: (B)

The outer loop takes $i = 1, 2, 4, \dots, 2^k \leq n$ (about $\log_2 n + 1$ iterations). For a given i the inner loop steps j by i from 1 to n , so it runs $\lceil n/i \rceil$ times.

Total $\approx n/1 + n/2 + n/4 + \dots \leq 2n$ — a **geometric** series, so $\text{work}(n) = \Theta(n)$.

Concretely, $\text{work}(2^{10}) = 2047$ and $\text{work}(2^{14}) = 32767$, i.e. $2n - 1$ for powers of two.

- (A) $\Theta(n \log n)$ is what you get if i ran over **all** integers $1 \dots n$ (harmonic series $n \cdot H_n$) — but here i doubles.
- (C) $\Theta(n^2)$ would need the inner loop to do n steps for each of n outer values.
- (D) $\Theta(\log^2 n)$ ignores that the inner loop does n/i steps, not $\log n$.

Trap: seeing two nested loops plus a log-style outer loop and multiplying $n \times \log n$ without checking how the inner bound depends on i .

Q.17 [NAT · 2 marks · Merge sort — counting inversions]**Answer: 10**

A split inversion is a pair (x, y) with x in the left half, y in the right half and $x > y$. During the merge of the **sorted** halves $L = [2, 5, 7, 9]$ and $R = [1, 3, 6, 8]$, each time an element of R is output, it forms an inversion with every element still remaining in L .

- output 1 (from R): L remaining = 4 $\rightarrow +4$
- output 2 (L), then 3 (R): L remaining $\{5, 7, 9\} \rightarrow +3$
- output 5 (L), then 6 (R): L remaining $\{7, 9\} \rightarrow +2$
- output 7 (L), then 8 (R): L remaining $\{9\} \rightarrow +1$
- output 9

Split inversions = $4 + 3 + 2 + 1 = 10$.

Check directly: $5 > 1, 3$; $9 > 1, 8, 3, 6$; $2 > 1$; $7 > 1, 3, 6 \rightarrow 2 + 4 + 1 + 3 = 10$. (Total inversions = 3 in the left half + 2 in the right half + 10 = 15.)

Trap: adding 1 per R -element output instead of 'number of remaining L elements'. **Tip:** the whole count runs in $\Theta(n \log n)$.

Q.18 [MSQ · 2 marks · DFS and BFS trees of an undirected graph]**Answer: (B), (C), (D)**

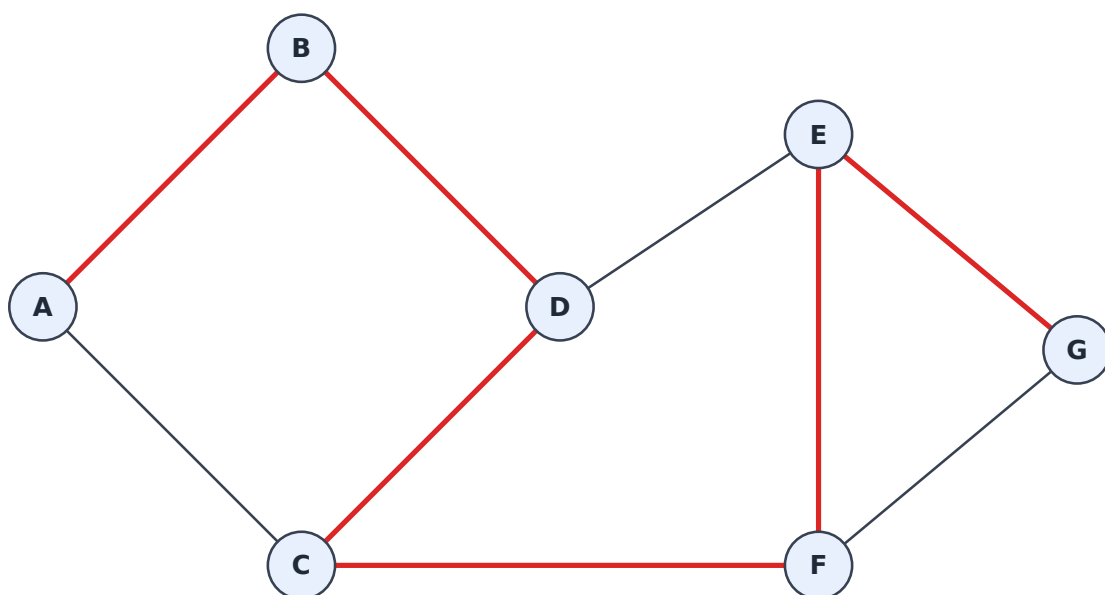
DFS (recursive, alphabetical): $A \rightarrow B$ (A 's first neighbour) $\rightarrow D$ (B 's next unvisited) $\rightarrow C$ (D 's neighbours B, C, E : B visited, so C) $\rightarrow F$ (C 's neighbours A, D visited, F new) $\rightarrow E$ (F 's neighbours $C, E, G \rightarrow E$ first) $\rightarrow G$ (E 's neighbours D, F visited, G new). Then everything backtracks.

Tree edges: AB, BD, DC, CF, FE, EG — the tree is the path $A-B-D-C-F-E-G$. The other $9 - 6 = 3$ edges (AC, DE, FG) are back edges.

BFS from A: level 1 $\{B, C\}$; level 2 $\{D \text{ (via } B), F \text{ (via } C)\}$; level 3 $\{E \text{ (via } D), G \text{ (via } F)\}$.

- (A) E is discovered from F before G . **False.**
- (B) **True** — in an undirected DFS every non-tree edge is a back edge; $9 - (7 - 1) = 3$.
- (C) **True.**
- (D) **True** — E and G are at distance 3.

Contrast: the DFS tree has depth 6 while the BFS tree has depth 3 — BFS trees are shortest-path trees; DFS trees can be as deep as $n - 1$.



DFS tree (highlighted) is a Hamiltonian path

Q.19 [MSQ · 2 marks · Linear probing — clustering effects]

Answer: (A), (B), (C)

Homes: 25→3, 47→3, 14→3, 36→3, 58→3, 70→4, 3→3 — six keys share home 3!

- 25 → 3 (1 probe); 47 → 4 (2); 14 → 5 (3); 36 → 6 (4); 58 → 7 (5)
- 70: home 4 is inside the cluster → 4, 5, 6, 7 full → **8** (5 probes)
- 3: 3 ... 8 full → 9 (7 probes)

Final: slots 3–9 occupied (a run of 7), slots 0, 1, 2, 10 empty.

- (A) from slot 3: slots 3 ... 9 full (7 probes), slot 10 empty (8th probe). **True.**
- (B) a successful search for a key repeats its insertion probes: $1 + 2 + 3 + 4 + 5 + 5 + 7 = 27 \rightarrow 27/7 \approx 3.86$. **True.**
- (C) **True.**
- (D) the run is slots 3–9, length **7**. **False.**

Insight: key 70 had a *different* home slot yet paid 5 probes — primary clustering penalises every key whose home falls inside a cluster, not just keys with the same home.

0	
1	
2	
3	25
4	47
5	14
6	36
7	58
8	70
9	3
10	

Final table: one cluster, slots 3–9

Q.20 [MCQ · 2 marks · Python — generator and iterator exhaustion]

Answer: (D)

Generators and iterators are **single-pass**: once consumed they yield nothing more.

- `a = sum(g)` consumes the generator: $0 + 1 + 4 + 9 + 16 = 30$.
- `b = sum(g)`: `g` is exhausted → sum of nothing = **0**.
- `zip(it, it)` pulls from the **same** iterator for both positions: first pair takes 1 then 2, the next pair 3 then 4 → **[(1, 2), (3, 4)]**.

Output: **30 0 [(1, 2), (3, 4)]**.

- (A) gets exhaustion right but treats the two `it` arguments as independent copies.
- (B) treats `g` as re-iterable and `it` as two independent iterators.
- (C) gets `zip` right but assumes the generator restarts.

Tip: `zip(*[iter(seq)] * k)` is the standard idiom for chunking a sequence into `k`-tuples — it works precisely because all `k` arguments are the same iterator.

Mock Test 34

Full-Syllabus Mock — Paper 4

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — closures and late binding]

What is printed by the following Python program?

Python

```
1 fs = [lambda: i * 2 for i in range(3)]
2 gs = [lambda i=i: i * 2 for i in range(3)]
3 print(sum(f() for f in fs), sum(g() for g in gs))
```

- (A) 6 6
- (B) 12 12
- (C) 12 6
- (D) 0 6

Q.2 [NAT · numerical answer | 1 mark | Python — floor division and modulo with negatives]

What value is printed by the following Python statement?

Python

```
1 print(-17 // 5 + -17 % 5 + 17 // -5)
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — infix to postfix]

The infix expression

$a - b * (c ^ d ^ e - f) / g + h$

is converted to postfix with the usual operator-stack algorithm, where $^$ has the highest precedence and is **right-associative**, $*$ and $/$ come next, $+$ and $-$ are lowest, and all binary operators other than $^$ are left-associative. The postfix expression is

- (A) $a b c d e ^ ^ f - * g / h + -$
- (B) $a b c d ^ e ^ f - * g / - h +$
- (C) $a b c d e ^ ^ f - g / * - h +$
- (D) $a b c d e ^ ^ f - * g / - h +$

Q.4 [NAT · numerical answer | 1 mark | Queues — queue using two stacks]

A queue is implemented with two stacks S1 and S2. ENQUEUE(x) pushes x onto S1. DEQUEUE pops from S2; if S2 is empty, it first pops **every** element of S1 and pushes it onto S2. Starting with both stacks empty, the operations

E E E D E E D D D E D D

are performed (E = enqueue of a new element, D = dequeue). The total number of elements moved from S1 to S2 is _____.

Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | Recurrences — master theorem]

Which of the following recurrences has/have the solution $T(n) = \Theta(n \log n)$? ($T(1) = 1$ in each case.)

- (A) $T(n) = 4T(n/2) + n$
- (B) $T(n) = 3T(n/3) + 5n$
- (C) $T(n) = 2T(n/2) + n$
- (D) $T(n) = 2T(n/4) + \sqrt{n}$

Q.6 [MCQ · single correct | 1 mark | Binary trees — reconstruction from traversals]

A binary tree with distinct keys has

pre-order: 8, 3, 5, 1, 9, 2, 6, 4, 7

in-order: 5, 1, 3, 2, 9, 8, 6, 7, 4

Its post-order traversal is

- (A) 1, 5, 9, 2, 3, 7, 4, 6, 8
- (B) 1, 5, 2, 9, 3, 7, 4, 6, 8
- (C) 5, 1, 2, 9, 3, 4, 7, 6, 8
- (D) 1, 5, 2, 9, 3, 4, 7, 6, 8

Q.7 [MCQ · single correct | 1 mark | Hashing — double hashing]

A hash table has 13 slots (0-12) and uses double hashing: the i -th probe for key k ($i = 0, 1, 2, \dots$) is $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Keys 20, 25, 30, 46, 59 are inserted in that order into an empty table. In which slot is 59 stored?

- (A) 9
- (B) 8
- (C) 11
- (D) 2

Q.8 [NAT · numerical answer | 1 mark | Quicksort — counting comparisons]

Quicksort with the Lomuto partition (pivot = last element of the sub-array; one comparison

$A[j] \leq \text{pivot}$ for each other element of the sub-array) sorts $A = [3, 7, 1, 9, 5, 2]$. Sub-arrays of size 0 or 1 are not partitioned. The total number of comparisons is _____.

Answer: _____ (enter a numerical value)

Q.9 [MSQ · one or more correct | 1 mark | Graph theory — degree sequences]

Which of the following sequences is/are the degree sequence of some **simple** undirected graph (no loops, no multiple edges)?

- (A) 3, 3, 3, 3, 2, 2
- (B) 5, 5, 4, 2, 1, 1
- (C) 4, 4, 3, 1, 1, 1
- (D) 5, 3, 3, 2, 2, 1

Q.10 [NAT · numerical answer | 1 mark | Binary search — worst-case probes]

Standard binary search ($\text{mid} = \lfloor (\text{lo} + \text{hi})/2 \rfloor$, three-way comparison at each probe, stop when found or when $\text{lo} > \text{hi}$) is performed on a sorted array of **200** distinct elements. The maximum number of array elements probed over all possible search keys (present or absent) is _____.

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — generator exhaustion and `in`]

What is printed by the following Python program?

Python

```
1 g = (x * x for x in range(6))
2 a = 4 in g
3 b = list(g)
4 c = 16 in g
5 print(a, b, c)
```

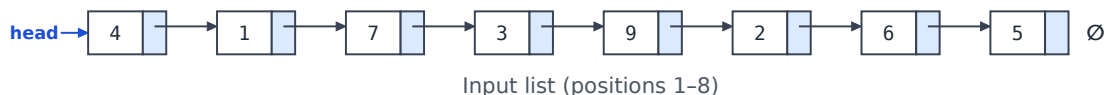
- (A) True [0, 1, 4, 9, 16, 25] True
- (B) True [9, 16, 25] True
- (C) True [9, 16, 25] False
- (D) True [16, 25] False

Q.12 [NAT · numerical answer | 2 marks | Linked lists — reversing a sub-list in place]

The function below reverses the nodes at positions m..n (1-based) of a singly linked list by repeatedly moving the node after **cur** to the front of the sub-list. The program prints the final list as one integer. What is printed?

Python

```
1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 def rev_between(head, m, n):
6     dummy = Node(0, head)
7     pre = dummy
8     for _ in range(m - 1):
9         pre = pre.next
10    cur = pre.next
11    for _ in range(n - m):
12        nxt = cur.next
13        cur.next = nxt.next
14        nxt.next = pre.next
15        pre.next = nxt
16    return dummy.next
17
18 head = None
19 for v in reversed([4, 1, 7, 3, 9, 2, 6, 5]):
20     head = Node(v, head)
21 p, s = rev_between(head, 3, 6), ""
22 while p:
23     s, p = s + str(p.val), p.next
24 print(int(s))
```



Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | BST — reconstruction from post-order]

The post-order traversal of a binary search tree T with distinct keys is

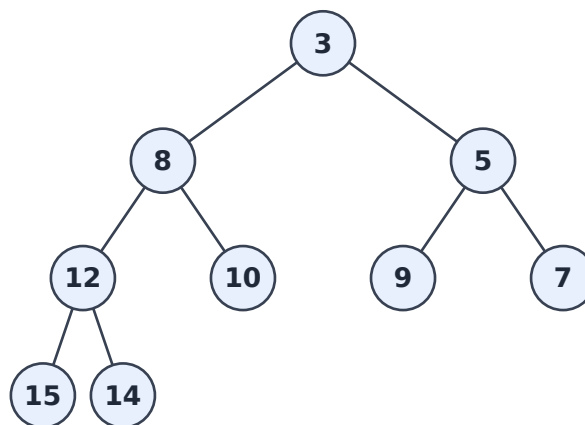
12, 20, 18, 31, 27, 15, 44, 52, 48, 40

Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) The in-order successor of 27 in T is 40
- (B) The pre-order traversal of T is 40, 15, 12, 27, 18, 20, 31, 48, 44, 52
- (C) The height of T is 4
- (D) T has exactly 5 leaves

Q.14 [MCQ · single correct | 2 marks | Binary heaps — mixed operations]

The min-heap shown below is stored in an array. The operations DELETE-MIN, INSERT(4), DELETE-MIN are performed in this order (DELETE-MIN moves the last element to the root and sifts it down, swapping with the smaller child; INSERT appends and sifts up). The final array is



Initial min-heap [3, 8, 5, 12, 10, 9, 7, 15, 14]

- (A) [4, 5, 7, 8, 10, 9, 14, 15]
- (B) [5, 7, 8, 12, 10, 9, 14, 15]
- (C) [5, 8, 7, 12, 10, 9, 15, 14]
- (D) [5, 8, 7, 12, 10, 9, 14, 15]

Q.15 [NAT · numerical answer | 2 marks | Complexity — exact loop counting]

What value is printed by the following Python program?

Python

```

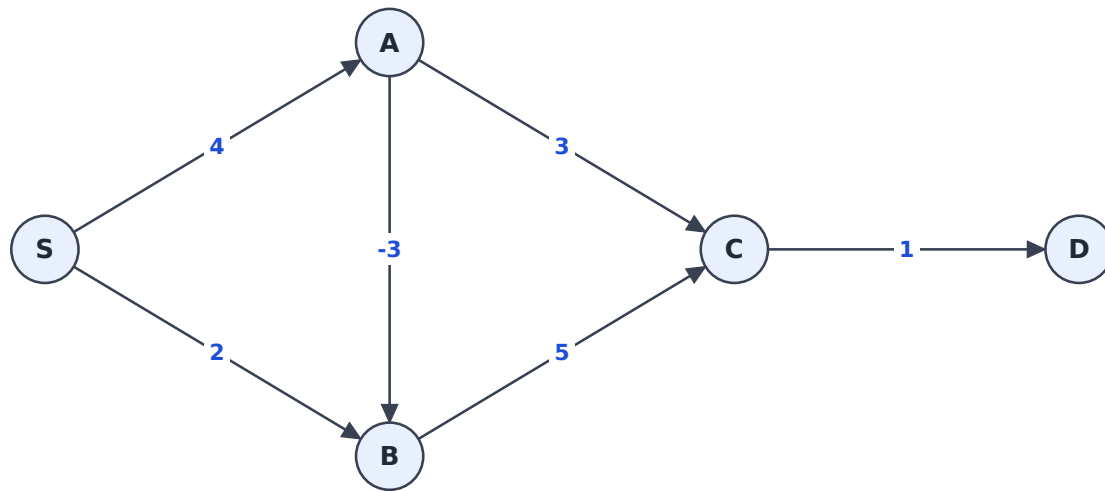
1 cnt = 0
2 for i in range(1, 65):
3     j = i
4     while j < 64:
5         j *= 2
6         cnt += 1
7 print(cnt)

```

Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra with a negative edge]

The standard Dijkstra algorithm (a vertex's distance is never changed after it is extracted from the priority queue) is run from S on the directed graph below, which has one negative edge. Which of the following statements is/are TRUE?



- (A) The distance Dijkstra reports for A is the true shortest distance
- (B) Dijkstra reports a distance of 7 for C
- (C) The graph contains a negative-weight cycle
- (D) The true shortest distance from S to D is 7

Q.17 [MCQ · single correct | 2 marks | Graph theory — edges vs components]

A simple undirected graph has 10 vertices and exactly 3 connected components. What is the maximum possible number of edges?

- (A) 36
- (B) 28
- (C) 21
- (D) 33

Q.18 [NAT · numerical answer | 2 marks | Hashing — deletion with tombstones]

A hash table of size 10 uses $h(k) = k \bmod 10$ and linear probing. Deletion marks a slot as DELETED (a tombstone): searches continue past DELETED slots, while an insertion stores the new key in the **first** DELETED or empty slot on its probe path. Starting from an empty table:

insert 31, 41, 51, 22, 61; delete 41; insert 71.

The number of slots probed by an unsuccessful search for key 81 (counting the empty slot at which it stops) is ____.

Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Python — exceptions and try/finally]

Consider the following Python definitions. Which of the following statements is/are TRUE?

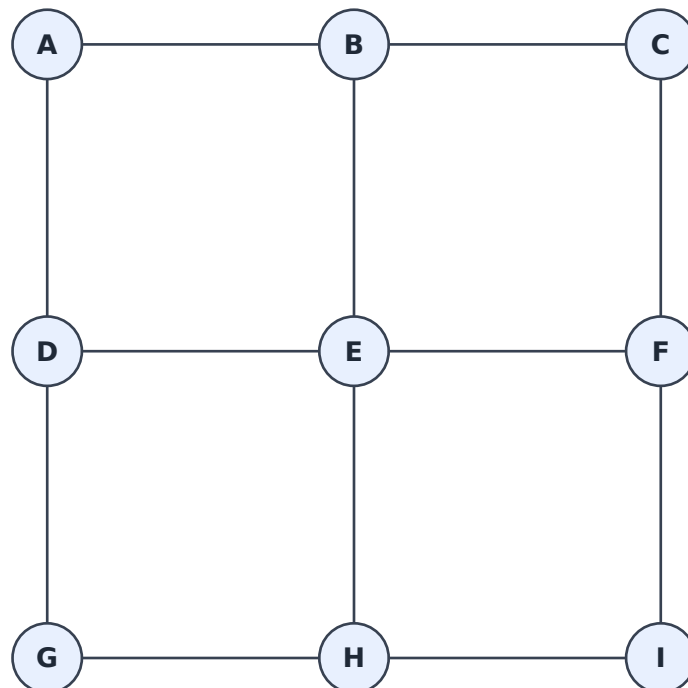
Python

```
1 log = []
2
3 def f(x):
4     try:
5         log.append("t")
6         if x < 0:
7             raise ValueError
8         return 10 // x
9     except ValueError:
10        log.append("e")
11        return -1
12    finally:
13        log.append("f")
14        if x == 0:
15            return 0
```

- (A) `f(4)` returns 2 and appends exactly "t", "f" to `log`
- (B) If `log` is empty and `f(4)`, `f(-3)`, `f(0)` are called in this order, `len(log)` is 7 afterwards
- (C) `f(-3)` returns -1
- (D) `f(0)` raises `ZeroDivisionError`

Q.20 [MCQ · single correct | 2 marks | DFS vs BFS — tree heights on a grid]

Consider the 3×3 grid graph below. DFS (recursive, neighbours explored in **alphabetical** order) is run from A, producing a DFS tree. What is the height of this DFS tree (number of edges on its longest root-to-leaf path)?



- (A) 8
- (B) 6
- (C) 7
- (D) 4

Answer Key — Mock Test 34

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — closures and late binding
2	NAT	1	-5	Python — floor division and modulo with negatives
3	MCQ	1	(D)	Stacks — infix to postfix
4	NAT	1	6	Queues — queue using two stacks
5	MSQ	1	(B), (C)	Recurrences — master theorem
6	MCQ	1	(B)	Binary trees — reconstruction from traversals
7	MCQ	1	(A)	Hashing — double hashing
8	NAT	1	9	Quicksort — counting comparisons
9	MSQ	1	(A), (D)	Graph theory — degree sequences
10	NAT	1	8	Binary search — worst-case probes
11	MCQ	2	(C)	Python — generator exhaustion and in
12	NAT	2	41293765	Linked lists — reversing a sub-list in place
13	MSQ	2	(B), (C), (D)	BST — reconstruction from post-order
14	MCQ	2	(D)	Binary heaps — mixed operations
15	NAT	2	120	Complexity — exact loop counting
16	MSQ	2	(A), (B), (D)	Shortest paths — Dijkstra with a negative edge
17	MCQ	2	(B)	Graph theory — edges vs components
18	NAT	2	6	Hashing — deletion with tombstones
19	MSQ	2	(A), (B), (C)	Python — exceptions and try/finally
20	MCQ	2	(A)	DFS vs BFS — tree heights on a grid

Detailed Solutions — Mock Test 34

Q.1 [MCQ · 1 mark · Python — closures and late binding]

Answer: (C)

Concept. A lambda looks up a free variable when it is *called*, not when it is created (late binding). All closures created in one comprehension share the comprehension's single variable `i`, whose final value is 2. A default argument, in contrast, is evaluated once at definition time and stored with each function.

Trace.

- `fs`: three lambdas, each computing `i * 2` with the *final* `i = 2` → `4 + 4 + 4 = 12`.
- `gs`: defaults capture 0, 1, 2 respectively → `0 + 2 + 4 = 6`.

Output: `12 6`.

Options.

- (A) assumes the plain lambdas also capture the current `i`.
- (B) assumes the default-argument trick does not help.
- (D) assumes the free `i` is 0 (its first value).

Tip: `i=i` (or `functools.partial`) is the idiomatic fix for late binding in loops.

Q.2 [NAT · 1 mark · Python — floor division and modulo with negatives]

Answer: -5

Concept. Python's `//` is *floor* division (rounds towards $-\infty$), and `%` is defined so that `a == (a // b) * b + a % b` always holds; hence the remainder has the sign of the divisor.

Evaluation.

- `-17 // 5 = ⌊-3.4⌋ = -4` (not `-3`).
- `-17 % 5 = -17 - (-4)(5) = -17 + 20 = 3`.
- `17 // -5 = ⌊-3.4⌋ = -4`.
- `Sum = -4 + 3 - 4 = -5`.

Note the unary minus binds tighter than `//` and `%`, so `-17 // 5` is `(-17) // 5`.

Trap: C/Java truncate towards zero, giving `-3`, `-2` and `-3` (sum `-8`). Python never truncates with `//`.

Q.3 [MCQ · 1 mark · Stacks — infix to postfix]

Answer: (D)

Concept. On reading an operator, pop operators of higher precedence, and of *equal* precedence only if the incoming operator is left-associative. For right-associative `^`, an incoming `^` does not pop a `^` already on the stack.

Structure. `c ^ d ^ e = c ^ (d ^ e) → c d e ^ ^`; then `- f → c d e ^ ^ f -`. `b * ( ... ) / g` is `((b * ( ... )) / g)` by left associativity → `b ... * g /`. Then `a - (that)` and finally `+ h`, both left-associative: `((a - X) + h)`.

Result: `a b c d e ^ ^ f - * g / - h +`.

Options.

- (A) computes `a - (X + h)`, as if `+` had higher precedence than `-`.
- (B) treats `^` as left-associative `((c ^ d) ^ e)`.
- (C) applies `/` before `*`, i.e. `b * (... / g)` — wrong for left-associative `*//`.

Tip: check the *last* operator of the postfix — it must be the operator applied last (here the final `+`).

Q.4 [NAT · 1 mark · Queues — queue using two stacks]**Answer: 6**

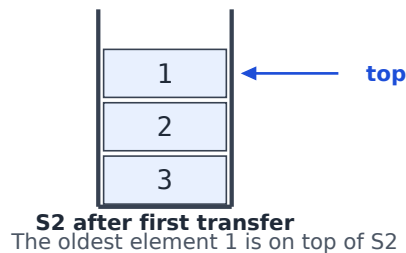
Concept. Each element is moved from S1 to S2 at most once in its lifetime, so the total transfer cost is at most the number of enqueues — giving $O(1)$ amortised cost per operation. Transfers happen only when S2 is empty at a dequeue.

Trace (elements numbered 1, 2, ... in enqueue order; stacks shown bottom → top):

- E1 E2 E3: S1 = [1, 2, 3].
- D: S2 empty → move 3 elements (total 3): S2 = [3, 2, 1]; pop 1.
- E4 E5: S1 = [4, 5].
- D: pop 2. D: pop 3. S2 now empty.
- D: move 2 elements (total 5): S2 = [5, 4]; pop 4.
- E6: S1 = [6].
- D: pop 5.
- D: S2 empty → move 1 element (total 6); pop 6.

Total moves = 3 + 2 + 1 = **6** (= number of enqueues, since every element was dequeued).

Trap: transferring on *every* dequeue (even when S2 is non-empty) would break FIFO order as well as the cost bound.

**Q.5 [MSQ · 1 mark · Recurrences — master theorem]****Answer: (B), (C)**

Master theorem for $T(n) = aT(n/b) + f(n)$: compare $f(n)$ with $n^{\log_b a}$.

- (A) $a = 4, b = 2$: $n^{\log_2 4} = n^2$ dominates $f(n) = n \rightarrow$ case 1 $\rightarrow \Theta(n^2)$. **No.**
- (B) $a = 3, b = 3$: $n^{\log_3 3} = n$, $f(n) = 5n = \Theta(n) \rightarrow$ case 2 $\rightarrow \Theta(n \log n)$. **Yes** (the constant 5 does not matter).
- (C) $a = 2, b = 2$: $n^{\log_2 2} = n = f(n) \rightarrow$ case 2 $\rightarrow \Theta(n \log n)$. **Yes.**
- (D) $a = 2, b = 4$: $n^{\log_4 2} = n^{1/2} = \sqrt{n} = f(n) \rightarrow$ case 2 $\rightarrow \Theta(\sqrt{n} \log n)$. **No** — it has a log factor but not $n \log n$.

Trap: (D) looks like merge sort's 'balanced' case and indeed falls in case 2, but the critical exponent is $\frac{1}{2}$, so the answer is $\sqrt{n} \log n$.

Q.6 [MCQ · 1 mark · Binary trees — reconstruction from traversals]**Answer: (B)**

Concept. The first pre-order key is the root; its position in the in-order splits the remaining keys into left and right subtrees. Recurse.

Reconstruction.

- Root 8. In-order left of 8: {5, 1, 3, 2, 9}; right: {6, 7, 4}.
- Left part, pre-order 3, 5, 1, 9, 2 $\rightarrow$ root 3; in-order 5, 1 | 3 | 2, 9.
- {5, 1}: pre 5, 1 $\rightarrow$ root 5, in-order 5, 1 $\rightarrow$ 1 is 5's right child.
- {2, 9}: pre 9, 2 $\rightarrow$ root 9, in-order 2, 9 $\rightarrow$ 2 is 9's left child.

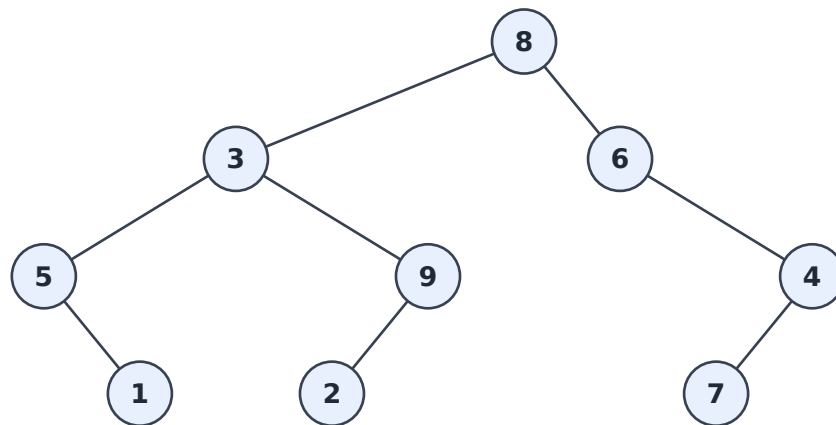
- Right part, pre-order 6, 4, 7 → root 6; in-order 6 | 7, 4 → right subtree {7, 4}: root 4 with left child 7.

Post-order: 1, 5, 2, 9, 3, 7, 4, 6, 8.

Options.

- (A) makes 2 the *right* child of 9.
- (C) makes 1 the left child of 5 (in-order 5, 1 says 1 comes after 5, i.e. right).
- (D) makes 7 the right child of 4.

Tip: pre-order + in-order (or post-order + in-order) determines a binary tree uniquely; pre-order + post-order alone does not.



Reconstructed tree

Q.7 [MCQ · 1 mark · Hashing — double hashing]

Answer: (A)

Concept. In double hashing the step size depends on the key, so keys with the same home slot follow different probe sequences (no secondary clustering).

Trace.

- 20: $h_1 = 7 \rightarrow$ slot 7.
- 25: $h_1 = 12 \rightarrow$ slot 12.
- 30: $h_1 = 4 \rightarrow$ slot 4.
- 46: $h_1 = 7$ (full); $h_2 = 1 + 46 \bmod 11 = 1 + 2 = 3 \rightarrow 7 + 3 = 10$.
- 59: $h_1 = 59 \bmod 13 = 7$, $h_2 = 1 + 59 \bmod 11 = 1 + 4 = 5$. Probes: 7 (full), 12 (full), $17 \bmod 13 = 4$ (full), $22 \bmod 13 = 9$ (free).

59 is stored in slot **9** after 4 probes.

Options.

- (B) 8 is what linear probing would give (7 full → 8).
- (C) 11 uses $h_2 = k \bmod 11 = 4$ (forgetting the '1 +'): $7 \rightarrow 11$.
- (D) 2 uses $h_2 = 1 + (k \bmod 13) = 8$: $7 \rightarrow 15 \bmod 13 = 2$.

Tip: h_2 must never be 0 (hence the '1 +'), and should be coprime to the table size — with a prime size like 13 every probe sequence visits all slots.

0	
1	
2	
3	
4	30
5	
6	
7	20
8	
9	59
10	46
11	
12	25

Final table

Q.8 [NAT · 1 mark · Quicksort — counting comparisons]**Answer: 9**

Concept. Partitioning a sub-array of size s costs $s - 1$ comparisons. Total cost depends on how balanced the pivots are.

Trace.

- Partition [3, 7, 1, 9, 5, 2], pivot 2 (5 comparisons): only $1 \leq 2 \rightarrow [1, 2, 3, 9, 5, 7]$, pivot at index 1. Left part [1] (size 1), right part [3, 9, 5, 7].
- Partition [3, 9, 5, 7], pivot 7 (3 comparisons): 3 and 5 $\leq 7 \rightarrow [3, 5, 7, 9]$. Left part [3, 5], right part [9].
- Partition [3, 5], pivot 5 (1 comparison) $\rightarrow [3, 5]$; parts [3] and [].

Total = $5 + 3 + 1 = 9$.

Trap: the pivot 2 is the second smallest — a poor split — so the right part of size 4 still needs work; a perfectly balanced run on 6 keys would use fewer comparisons, a sorted input would use the maximum $5 + 4 + 3 + 2 + 1 = 15$.

Q.9 [MSQ · 1 mark · Graph theory — degree sequences]**Answer: (A), (D)**

Tools. The degree sum must be even (handshaking lemma), and the Havel–Hakimi test: remove the largest degree d , subtract 1 from the next d largest, repeat; the sequence is graphical iff this ends with all zeros.

- (A) Sum 16. 3,3,3,3,2,2 $\rightarrow$ remove 3: 2,2,2,2,2 $\rightarrow$ remove 2: 1,1,2,2 $\rightarrow$ sort 2,2,1,1 $\rightarrow$ remove 2: 1,0,1 $\rightarrow$ 1,1,0 $\rightarrow$ remove 1: 0,0. **Graphical.**
- (B) Sum 18 (even), but two vertices of degree 5 in a 6-vertex graph are adjacent to *all* other vertices, forcing every vertex to have degree ≥ 2 — yet two vertices have degree 1. **Not graphical.**
- (C) Sum 14 (even), but: the two vertices of degree 4 each need 4 neighbours among the other 5 vertices. Apart from each other they each need 3 more from $\{3, 1, 1, 1\}$; the three degree-1 vertices can each serve only one of them, so at most $1 + 3 = 4$ slots are available for 6 demands. Havel–Hakimi: 4,4,3,1,1,1 $\rightarrow$ 3,2,0,0,1 $\rightarrow$ sort 3,2,1,0,0 $\rightarrow$ 1,0,-1 $\times$. **Not graphical.**
- (D) Sum 16. 5,3,3,2,2,1 $\rightarrow$ 2,2,1,1,0 $\rightarrow$ 1,0,1,0 $\rightarrow$ 1,1,0,0 $\rightarrow$ 0,0,0. **Graphical.**

Trap: an even sum is necessary but not sufficient.

Q.10 [NAT · 1 mark · Binary search — worst-case probes]

Answer: 8

Concept. Binary search follows a root-to-node path in an implicit balanced BST of n nodes. The worst case equals the height of that tree in nodes: $\lfloor \log_2 n \rfloor + 1$.

Computation. $2^7 = 128 \leq 200 < 256 = 2^8$, so $\lfloor \log_2 200 \rfloor = 7$ and the maximum number of probes is $7 + 1 = 8$.

Intuition. After k probes at most $2^k - 1$ elements can have been 'covered'. With 7 probes only $127 < 200$ elements can be distinguished, so 8 are needed; 8 probes cover up to 255.

Trap: answering $\lfloor \log_2 200 \rfloor = 8$ happens to coincide here, but the correct general formula is $\lfloor \log_2 n \rfloor + 1$ (for $n = 256$ it gives 9, not 8).

Q.11 [MCQ · 2 marks · Python — generator exhaustion and `in`]

Answer: (C)

Concept. A generator is an iterator: it produces each value once. The `in` operator on an iterator consumes values until it finds a match (or exhausts it), and `list(g)` consumes everything that is left.

Trace. `g` will yield 0, 1, 4, 9, 16, 25.

- `4 in g` consumes 0, 1, 4 and stops at the match $\rightarrow a = \text{True}$.
- `list(g)` takes the remaining 9, 16, 25 $\rightarrow b = [9, 16, 25]$; `g` is now exhausted.
- `16 in g` finds nothing left $\rightarrow c = \text{False}$ (even though 16 was produced earlier).

Output: `True [9, 16, 25] False`.

Options.

- (A) treats `g` like a list that can be re-scanned.
- (B) forgets that `list(g)` exhausted the generator.
- (D) assumes `in` also consumed the element *after* the match.

Trap: membership tests on generators/iterators have side effects; convert to a list or set first if you need repeated lookups.

Q.12 [NAT · 2 marks · Linked lists — reversing a sub-list in place]

Answer: 41293765

Concept. `pre` stays fixed on the node just before position m , and `cur` stays on the node that was originally at position m (it drifts towards the end of the reversed block). Each iteration unlinks `cur.next` and inserts it right after `pre` — head insertion inside the block — so $n - m = 3$ iterations reverse positions 3..6.

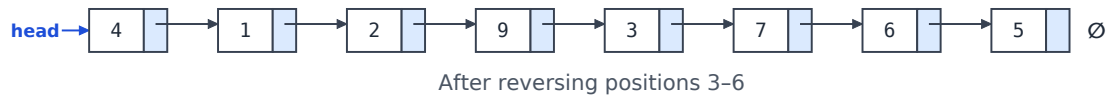
Trace. `pre` = node 1 (value 1), `cur` = node 7.

- start: 4 1 [7 3 9 2] 6 5
- iter 1: move 3 to front of block $\rightarrow$ 4 1 [3 7 9 2] 6 5
- iter 2: move 9 $\rightarrow$ 4 1 [9 3 7 2] 6 5
- iter 3: move 2 $\rightarrow$ 4 1 [2 9 3 7] 6 5

The printed integer is **41293765**.

Why a dummy node? If $m = 1$ there is no real predecessor; the dummy makes that case identical to the others and `dummy.next` is the (possibly new) head.

Trap: advancing `cur` in the loop (as in ordinary reversal) would break the algorithm — here `cur` must stay put while nodes are pulled past it.

**Q.13 [MSQ · 2 marks · BST — reconstruction from post-order]****Answer: (B), (C), (D)**

Concept. In a BST's post-order, the last key is the root; the keys before it split into a prefix of keys smaller than the root (left subtree) and the following keys larger than it (right subtree). Recurse.

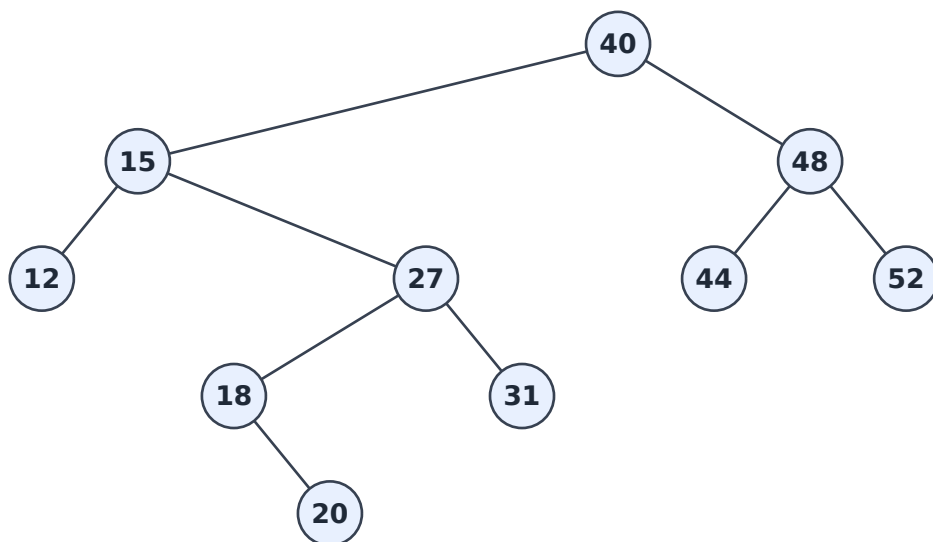
Reconstruction.

- Root 40; left post-order 12, 20, 18, 31, 27, 15; right 44, 52, 48.
- Left: root 15; smaller {12}; larger 20, 18, 31, 27 → root 27 with left post 20, 18 (root 18, right child 20) and right 31.
- Right: root 48 with children 44 and 52.

Statements.

- (A) 27 has a right subtree {31}, so its successor is the minimum there, **31. False.**
- (B) Pre-order: 40, 15, 12, 27, 18, 20, 31, 48, 44, 52. **True.**
- (C) Longest path 40 → 15 → 27 → 18 → 20 has 4 edges. **True.**
- (D) Leaves: 12, 20, 31, 44, 52 → 5. **True.**

Trap: the successor of a node is an ancestor only when the node has *no* right subtree.



The BST T

Q.14 [MCQ · 2 marks · Binary heaps — mixed operations]**Answer: (D)**

DELETE-MIN 1. Remove 3; last element 14 goes to the root: [14, 8, 5, 12, 10, 9, 7, 15]. Sift down: children 8, 5 → swap with 5 → [5, 8, 14, 12, 10, 9, 7, 15]; children of index 2 are 9, 7 → swap with 7 → [5, 8, 7, 12, 10, 9, 14, 15].

INSERT(4). Append at index 8 (parent index 3 = 12): swap → index 3; parent index 1 = 8: swap → index 1; parent index 0 = 5: swap → root. Array [4, 5, 7, 8, 10, 9, 14, 15, 12].

DELETE-MIN 2. Remove 4; last element 12 to root: [12, 5, 7, 8, 10, 9, 14, 15]. Children 5, 7 → swap with 5 → index 1: children 8, 10 → swap with 8 → index 3: child 15 > 12 → stop. Array **[5, 8, 7, 12, 10, 9, 14, 15]**.

Interesting: the final array equals the array after the first DELETE-MIN — inserting a new minimum and deleting it retraced the same path here.

Options.

- (A) is the state before the second DELETE-MIN, with 12 dropped.
- (B) swaps with the left child instead of the smaller child at the first step of sift-down.
- (C) puts 15 and 14 in the wrong leaf positions (14 must sink to index 6 in the first deletion).

Q.15 [NAT · 2 marks · Complexity — exact loop counting]**Answer: 120**

Concept. For a fixed i , the inner loop doubles j until $j \geq 64$, i.e. it runs $\lceil \log_2(64/i) \rceil$ times. Group the values of i by that count.

- $i = 1$: $1 \rightarrow 2 \rightarrow \dots \rightarrow 64$: 6 iterations $\rightarrow 6$
- $i = 2, 3$: 5 iterations each $\rightarrow 10$
- $i = 4 \dots 7$: 4 each $\rightarrow 16$
- $i = 8 \dots 15$: 3 each $\rightarrow 24$
- $i = 16 \dots 31$: 2 each $\rightarrow 32$
- $i = 32 \dots 63$: 1 each $\rightarrow 32$
- $i = 64$: 0

Total = $6 + 10 + 16 + 24 + 32 + 32 = 120$.

Asymptotics. For general $n = 2^k$ the total is $\sum_{t=1}^k t \cdot 2^{k-t} = 2^{k+1} - k - 2 = \Theta(n)$ — not $\Theta(n \log n)$, because most i need only a few doublings. (Check: $128 - 6 - 2 = 120$.)

Trap: multiplying the outer count by the worst-case inner count ($64 \times 6 = 384$) gives only an upper bound.

Q.16 [MSQ · 2 marks · Shortest paths — Dijkstra with a negative edge]**Answer: (A), (B), (D)****Dijkstra trace.**

- Extract $S(0)$: $A = 4$, $B = 2$.
- Extract $B(2)$: $C = 2 + 5 = 7$.
- Extract $A(4)$: $A \rightarrow B$ would give 1, but B is already extracted — no change. $A \rightarrow C$ gives 7, not smaller.
- Extract $C(7)$: $D = 8$. Extract $D(8)$.

Reported: $A\ 4$, $B\ 2$, $C\ 7$, $D\ 8$.

True distances. B : $S \rightarrow A \rightarrow B = 4 - 3 = 1$. C : $\min(S \rightarrow A \rightarrow C = 7, S \rightarrow A \rightarrow B \rightarrow C = 1 + 5 = 6) = 6$. $D = 6 + 1 = 7$. $A = 4$.

- (A) **True** — 4 is correct (no path into A other than $S \rightarrow A$).
- (B) **True** — Dijkstra reports 7 (wrong; the true value is 6).
- (C) **False** — there is no cycle at all; the only negative edge $A \rightarrow B$ lies on no cycle.
- (D) **True** — $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = 4 - 3 + 5 + 1 = 7$.

Concept/Trap. Dijkstra's greedy choice assumes that a path can only get longer when extended. A negative edge breaks this even without negative cycles; B was finalised at 2 before the cheaper route through A was discovered, and the error propagated to C and D . Bellman-Ford handles this graph correctly.

Q.17 [MCQ · 2 marks · Graph theory — edges vs components]**Answer: (B)**

Concept. For fixed n and k components, the edge count is maximised by making each component complete, and among complete components by making one as large as possible: $k - 1$ isolated vertices and one K_{n-k+1} . (Moving a vertex from a smaller clique of size a to a larger one of size $b \geq a$ changes the edge count by $b - (a - 1) > 0$.)

Computation. Sizes 8, 1, 1 $\rightarrow C(8, 2) = 28$ edges.

Compare other splits: $6 + 2 + 2 \rightarrow 15 + 1 + 1 = 17$; $4 + 3 + 3 \rightarrow 6 + 3 + 3 = 12$; $7 + 2 + 1 \rightarrow 21 + 1 = 22$. The maximum is **28**.

Options.

- (A) $36 = C(9, 2)$ is the maximum with 2 components.
- (C) $21 = C(7, 2)$ comes from wrongly using $n - k = 7$ as the clique size.
- (D) 33 has no valid construction.

Related bound: the *minimum* number of edges with k components is $n - k$ (a forest).

Q.18 [NAT · 2 marks · Hashing — deletion with tombstones]

Answer: 6

Concept. Tombstones keep probe chains intact: an empty slot means 'the key cannot be further along', a DELETED slot does not. Insertions may recycle tombstones.

Trace.

- $31 \rightarrow 1$; $41 \rightarrow 1$ full $\rightarrow 2$; $51 \rightarrow 1$, 2 full $\rightarrow 3$; $22 \rightarrow 2$, 3 full $\rightarrow 4$; $61 \rightarrow 1$, 2, 3, 4 full $\rightarrow 5$.
- delete 41: slot 2 becomes DELETED.
- insert 71: home 1 (31), slot 2 DELETED $\rightarrow$ store 71 in slot 2.
- Table: 1:31, 2:71, 3:51, 4:22, 5:61, others empty.
- search 81: home 1 $\rightarrow$ probes 1, 2, 3, 4, 5 (all occupied, none is 81), 6 empty $\rightarrow$ stop.

Probes = **6**.

Trap: if deletion simply emptied slot 2, a later search for 61 would stop at slot 2 and wrongly report 'not found' — that is why tombstones exist. Note too that the cluster 1-5 still makes every search hashing to slot 1 expensive.

0	
1	31
2	71
3	51
4	22
5	61
6	
7	
8	
9	

Table before the search for 81

Q.19 [MSQ · 2 marks · Python — exceptions and try/finally]

Answer: (A), (B), (C)

Concept. A **finally** block always runs — after a **return**, after an exception is handled, and even while an unhandled exception is propagating. If the **finally** block itself executes **return**, that return value replaces whatever was pending, *including a pending exception*, which is silently discarded.

- $f(4)$: log gets "t"; returns $10 // 4 = 2 \rightarrow$ finally appends "f"; $x \neq 0$ so the pending return 2 stands.
(A) **True**.
- $f(-3)$: "t"; ValueError raised and caught: "e", return -1 pending; finally appends "f". Returns -1 .
(C) **True**.

- `f(0)`: `"t"; 10 // 0` raises `ZeroDivisionError`, which the `except ValueError` clause does not catch; on the way out, finally appends `"f"` and executes `return 0`, which swallows the exception. So `f(0)` **returns 0**. (D) **False**.
- Log lengths: $2 + 3 + 2 = 7$. (B) **True**.

Trap: a `return` (or `break`) inside `finally` hides errors — a notorious source of silent bugs.

Q.20 [MCQ · 2 marks · DFS vs BFS — tree heights on a grid]

Answer: (A)

Concept. DFS keeps going deeper as long as an undiscovered neighbour exists, so on a grid it tends to produce long, path-like trees; BFS produces the shallowest possible tree (height = eccentricity of the source).

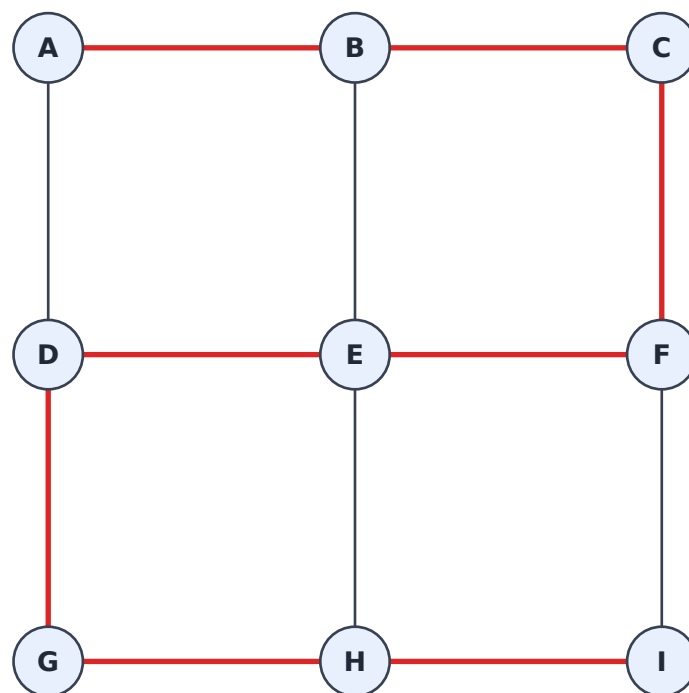
DFS trace (alphabetical neighbours):

- A: neighbours B, D → go to B.
- B: A seen, C → go to C.
- C: B seen, F → F.
- F: C seen, E → E.
- E: B seen, D → D.
- D: A, E seen, G → G.
- G: D seen, H → H.
- H: E, G seen, I → I.
- I: all seen. Backtrack all the way; every vertex is already discovered.

The DFS tree is the Hamiltonian path A-B-C-F-E-D-G-H-I with **8** edges, so its height is **8**.

Options. (D) 4 is the height of the **BFS** tree from A (I is at distance 4). (B) and (C) come from backtracking too early.

Tip: a DFS tree of a connected graph with n vertices has height between the BFS height and $n - 1$; here it reaches the maximum $n - 1 = 8$.



DFS tree edges (highlighted)

Mock Test 35

Full-Syllabus Mock — Paper 5

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — try / except / finally]

Consider the following Python program. What is printed?

Python

```

1 def f(n):
2     try:
3         if n < 0:
4             raise ValueError
5         return n * 2
6     except ValueError:
7         return -1
8     finally:
9         if n == 3:
10            return 0
11
12 print(f(3), f(-2), f(5))

```

- (A) 6 -1 10
- (B) 0 -1 10
- (C) 0 0 10
- (D) 0 -1 0

Q.2 [NAT · numerical answer | 1 mark | Stacks — infix to postfix conversion]

The infix expression

$a + b * (c - d) / e ^ f ^ g - h$

is converted to postfix with the standard operator-stack algorithm. Precedence: ^ (highest, right-associative) > *, / (left-associative) > +, - (left-associative). '(' is pushed on the stack and popped at the matching ')'. The maximum number of symbols (operators and parentheses) on the stack at any moment is ____.

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Trees — complete binary tree in an array]

A complete binary tree with 50 nodes is stored in an array $T[1..50]$ in level order (the children of $T[i]$ are $T[2i]$ and $T[2i + 1]$). Height is the number of edges on the longest root-to-leaf path. Which of the following statements is/are TRUE?

- (A) $T[26]$ is an internal node
- (B) The tree has exactly 25 leaves
- (C) The parent of $T[37]$ is $T[18]$
- (D) The height of the tree is 5

Q.4 [NAT · numerical answer | 1 mark | Hashing — double hashing]

The keys 18, 41, 22, 44, 59, 32, 31, 73 are inserted in that order into an empty table of size 13 using double hashing: the i -th probe ($i = 0, 1, 2, \dots$) for key k is $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. The index of the slot where key 31 is stored is _____.

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Bubble sort — passes with early exit]

Consider the following Python program. What is printed?

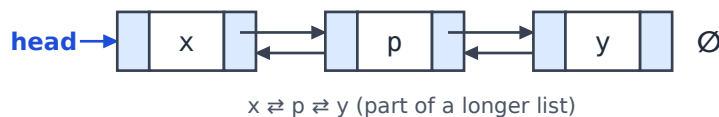
Python

```
1 def bubble(a):
2     a, passes = a[:], 0
3     for i in range(len(a) - 1):
4         passes += 1
5         swapped = False
6         for j in range(len(a) - 1 - i):
7             if a[j] > a[j + 1]:
8                 a[j], a[j + 1] = a[j + 1], a[j]
9                 swapped = True
10        if not swapped:
11            break
12    return passes
13
14 print(bubble([2, 3, 4, 5, 6, 1]), bubble([6, 1, 2, 3, 4, 5]))
```

- (A) 5 2
- (B) 2 5
- (C) 1 1
- (D) 5 1

Q.6 [MCQ · single correct | 1 mark | Linked lists — deleting from a doubly linked list]

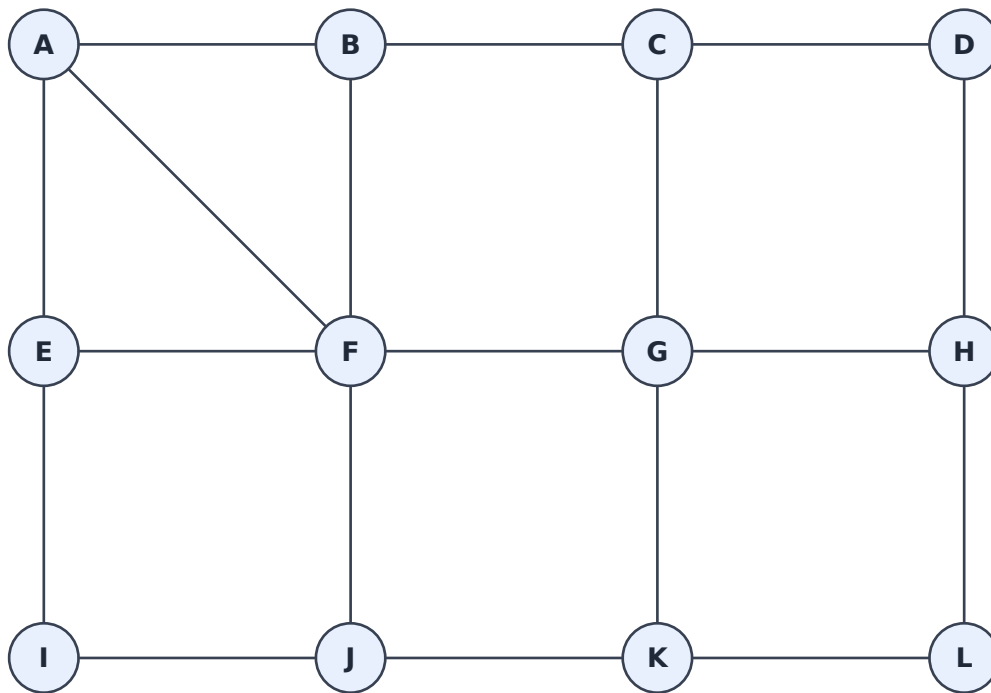
In a doubly linked list each node has fields **prev** and **next**. Node **p** is neither the first nor the last node. Which statement sequence correctly unlinks **p** so that forward and backward traversals both skip it?



- (A) `p.prev.next = p.next; p.next = p.prev.next`
- (B) `p.prev.next = p.next; p.next.prev = p.next`
- (C) `p.next.prev = p.prev; p.prev = p.next`
- (D) `p.prev.next = p.next; p.next.prev = p.prev`

Q.7 [NAT · numerical answer | 1 mark | Graphs — BFS distances]

The undirected graph shown is a 3×4 grid with one extra diagonal edge A-F. The number of vertices whose shortest-path distance (in edges) from A is exactly 2 is _____.



Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Quicksort — worst-case comparisons]

Quicksort always picks the **first** element of the current sub-array as the pivot, and partitioning a sub-array of m elements makes exactly $m - 1$ key comparisons. Sub-arrays of size 0 or 1 are not partitioned. The total number of key comparisons made when sorting the already-sorted array [1, 2, 3, 4, 5, 6, 7, 8] is

- (A) 28
- (B) 36
- (C) 24
- (D) 17

Q.9 [NAT · numerical answer | 1 mark | Python — nested comprehensions and zip]

Consider the following Python program. The value printed is _____.

Python

```

1 M = [[i * j for j in range(1, 5)] for i in range(1, 4)]
2 T = list(zip(*M))
3 print(sum(T[2]) - sum(M[1]))

```

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Binary search — worst-case probes]

Standard binary search (each probe compares x with $A[mid]$, $mid = \lfloor (lo + hi)/2 \rfloor$, and either stops or discards the half that cannot contain x) is run on a sorted array of 1000 distinct keys. The maximum number of probes over all possible search keys x (present or absent) is

- (A) 9
- (B) 11
- (C) 10
- (D) 500

Q.11 [MCQ · single correct | 2 marks | Python — class vs instance attributes]

Consider the following Python program. What is printed?

Python

```
1 class Counter:
2     hits = []
3     total = 0
4
5     def __init__(self, name):
6         self.name = name
7
8     def hit(self, k):
9         self.hits.append(k)
10        self.total += k
11
12 a, b = Counter("a"), Counter("b")
13 a.hit(3)
14 b.hit(4)
15 a.hit(5)
16 print(len(a.hits), a.total, b.total, Counter.total)
```

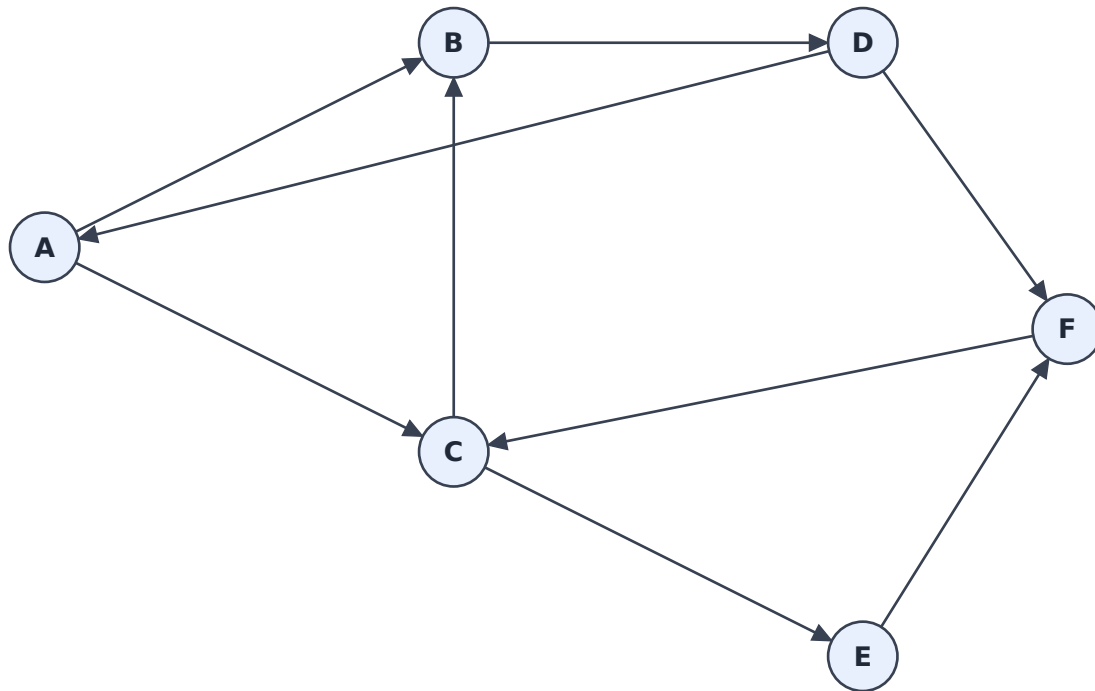
- (A) 3 8 4 12
- (B) 2 8 4 0
- (C) 3 8 4 0
- (D) 3 12 12 12

Q.12 [MSQ · one or more correct | 2 marks | Heaps — successive insertions into a max-heap]

The keys 15, 22, 9, 40, 31, 27, 50, 12 are inserted one at a time, in this order, into an initially empty binary max-heap stored in an array (0-based). Each insertion appends the key and sifts it up, swapping with its parent while the parent is smaller. Which of the following statements is/are TRUE?

- (A) Inserting 50 causes exactly 2 swaps
- (B) The final array is [50, 31, 40, 15, 22, 9, 27, 12]
- (C) In the final heap, 40 is the left child of the root
- (D) The total number of swaps over all insertions is 7

Q.13 [MSQ · one or more correct | 2 marks | Graphs — DFS timestamps and edge classification]
 DFS is run on the directed graph shown, starting at A; adjacency lists are in alphabetical order, and a single clock is incremented at every discovery and every finish (so A is discovered at time 1). Which of the following statements is/are TRUE?



- (A) Exactly 3 edges are back edges
- (B) C is discovered at time 5 and finished at time 8
- (C) A → C is a forward edge
- (D) C → B is a cross edge

Q.14 [NAT · numerical answer | 2 marks | Recurrences — exact evaluation with floors]

Let $T(1) = 1$ and $T(n) = 2 \cdot T(\lfloor n/2 \rfloor) + n$ for $n \geq 2$. The value of $T(20)$ is _____.

Answer: _____ (enter a numerical value)

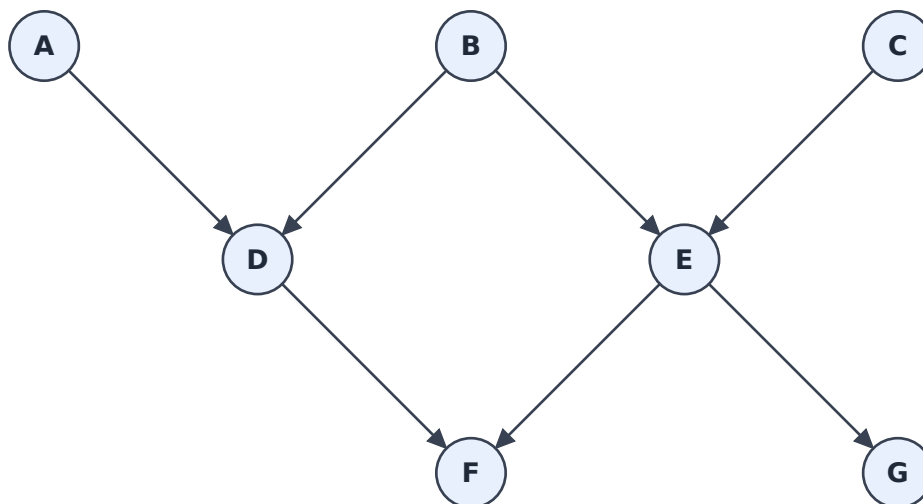
Q.15 [MSQ · one or more correct | 2 marks | Hashing — deletion with tombstones]

A hash table of size 10 uses $h(k) = k \bmod 10$ with linear probing. Keys 31, 41, 51, 22, 62 are inserted in that order. Then 41 is deleted by marking its slot DELETED (a tombstone). A search probes until it finds the key or an EMPTY slot (tombstones are skipped over). An insertion places the key in the first slot of its probe sequence that is EMPTY or DELETED. Which of the following statements is/are TRUE? (A probe = one slot examined.)

- (A) If 71 is then inserted, it is placed in slot 2
- (B) Searching for 51 after the deletion examines exactly 3 slots
- (C) If slot 2 had instead been simply set to EMPTY, a search for 22 would still succeed
- (D) An unsuccessful search for 72 after the deletion examines exactly 5 slots

Q.16 [NAT · numerical answer | 2 marks | Graphs — counting topological orderings]

The number of distinct topological orderings of the directed acyclic graph shown is _____.



Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Stacks — monotonic stack (next greater element)]

Consider the following Python program. What is printed?

Python

```

1 def nge(a):
2     res, st = [-1] * len(a), []
3     for i, x in enumerate(a):
4         while st and a[st[-1]] < x:
5             res[st.pop()] = x
6             st.append(i)
7     return res
8
9 print(nge([4, 9, 2, 5, 3, 8, 1, 7]))

```

- (A) [9, -1, 5, 8, 8, -1, 7, 7]
- (B) [9, -1, 5, 8, 8, -1, 7, -1]
- (C) [9, -1, 5, 9, 8, -1, 7, -1]
- (D) [9, -1, 5, 8, 5, -1, 7, -1]

Q.18 [MSQ · one or more correct | 2 marks | Merge sort and quicksort — exact comparison counts]

Assume quicksort uses the Lomuto partition with the **last** element as pivot (partitioning m elements costs $m - 1$ comparisons, and elements equal to the pivot go to the left part), and merge sort splits into $\lfloor n/2 \rfloor$ and $\lfloor n/2 \rfloor$ elements and stops comparing once a side is exhausted. Which of the following statements is/are TRUE?

- (A) Quicksort on 5 equal keys [4, 4, 4, 4, 4] makes exactly 10 comparisons
- (B) If merge sort's merge takes from the right list whenever $L[i] == R[j]$, it is still stable
- (C) Merge sort on the sorted array [1, 2, ..., 8] makes exactly 12 comparisons
- (D) Quicksort on [3, 1, 2] makes exactly 3 comparisons

Q.19 [NAT · numerical answer | 2 marks | Linked lists — Floyd's cycle detection]

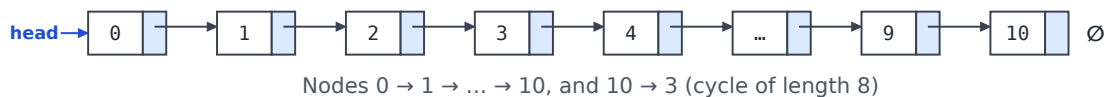
Consider the following Python program, which builds an 11-node list whose last node points back to node 3, and then runs Floyd's cycle-finding algorithm. The program prints two numbers. The **first** number printed is _____.

Python

```

1 class Node:
2     def __init__(self, v):
3         self.v, self.nxt = v, None
4
5 nodes = [Node(i) for i in range(11)]
6 for a, b in zip(nodes, nodes[1:]):
7     a.nxt = b
8 nodes[-1].nxt = nodes[3]
9
10 slow = fast = nodes[0]
11 steps = 0
12 while True:
13     slow, fast = slow.nxt, fast.nxt.nxt
14     steps += 1
15     if slow is fast:
16         break
17 p = nodes[0]
18 while p is not slow:
19     p, slow = p.nxt, slow.nxt
20     steps += 1
21 print(steps, p.v)

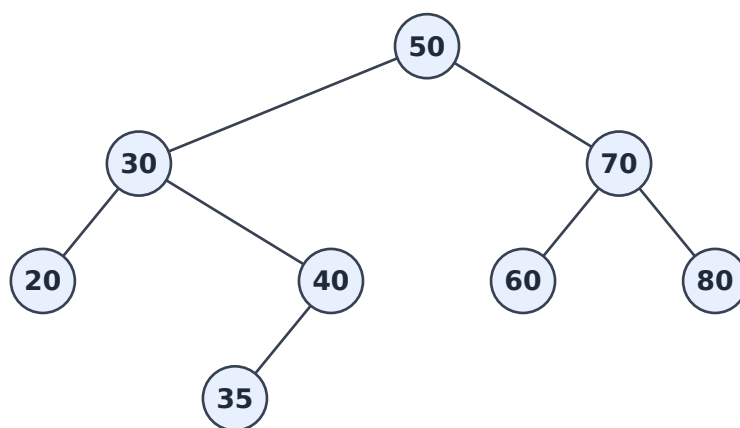
```



Answer: _____ (enter a numerical value)

Q.20 [MCQ · single correct | 2 marks | Binary search trees — insertion orders giving the same tree]

How many permutations of the keys {20, 30, 35, 40, 50, 60, 70, 80}, when inserted in that order into an initially empty BST, produce exactly the BST shown?



Target BST

- (A) 35
- (B) 105
- (C) 420
- (D) 210

Answer Key — Mock Test 35

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — try / except / finally
2	NAT	1	4	Stacks — infix to postfix conversion
3	MSQ	1	(B), (C), (D)	Trees — complete binary tree in an array
4	NAT	1	12	Hashing — double hashing
5	MCQ	1	(A)	Bubble sort — passes with early exit
6	MCQ	1	(D)	Linked lists — deleting from a doubly linked list
7	NAT	1	4	Graphs — BFS distances
8	MCQ	1	(A)	Quicksort — worst-case comparisons
9	NAT	1	-2	Python — nested comprehensions and zip
10	MCQ	1	(C)	Binary search — worst-case probes
11	MCQ	2	(C)	Python — class vs instance attributes
12	MSQ	2	(A), (B), (D)	Heaps — successive insertions into a max-heap
13	MSQ	2	(A), (B), (C)	Graphs — DFS timestamps and edge classification
14	NAT	2	92	Recurrences — exact evaluation with floors
15	MSQ	2	(A), (B), (D)	Hashing — deletion with tombstones
16	NAT	2	42	Graphs — counting topological orderings
17	MCQ	2	(B)	Stacks — monotonic stack (next greater element)
18	MSQ	2	(A), (C)	Merge sort and quicksort — exact comparison counts
19	NAT	2	11	Linked lists — Floyd's cycle detection
20	MCQ	2	(D)	Binary search trees — insertion orders giving the same tree

Detailed Solutions — Mock Test 35

Q.1 [MCQ · 1 mark · Python — try / except / finally]

Answer: (B)

Concept. A **finally** block always runs as the function leaves the **try** statement, even after a **return**. If **finally** itself executes a **return**, that value **replaces** the pending return value.

- $f(3)$: **try** evaluates **return 6**, but **finally** runs; $n == 3$, so it returns 0. $\rightarrow 0$
- $f(-2)$: **ValueError** is raised and caught $\rightarrow$ pending **return -1**; **finally** runs, $n \neq 3$, falls through $\rightarrow -1$.
- $f(5)$: pending **return 10**; **finally** does nothing $\rightarrow 10$.

Output 0 -1 10 $\rightarrow$ (B).

- (A) ignores the overriding **return** in **finally**.
- (C) assumes **finally** returns 0 for every negative n as well.
- (D) applies the override to $f(5)$ too.

Trap: a **return** inside **finally** also silently swallows any exception that was propagating — a classic code-review red flag.

Q.2 [NAT · 1 mark · Stacks — infix to postfix conversion]

Answer: 4

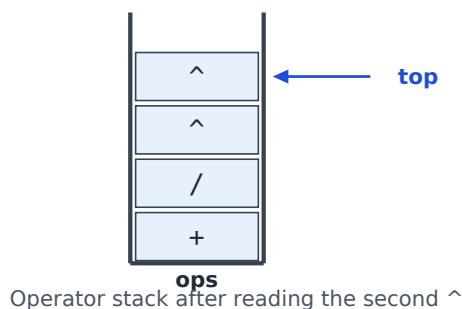
Concept. An incoming operator pops stack operators of higher precedence, or of equal precedence when it is left-associative; a right-associative $\wedge$ does not pop an equal $\wedge$. Operands go straight to the output.

Stack after each operator/parenthesis (bottom $\rightarrow$ top):

- $+$ $\rightarrow$ [$+$]
- $*$ $\rightarrow$ [$+$ $*$]
- $($ $\rightarrow$ [$+$ $*$ $($]
- $-$ $\rightarrow$ [$+$ $*$ $($ $-$] (4)
- $)$ $\rightarrow$ pops $-$ and $($ $\rightarrow$ [$+$ $*$]
- $/$ $\rightarrow$ pops $*$ (equal, left-*assoc*) $\rightarrow$ [$+$ $/$]
- $\wedge$ $\rightarrow$ [$+$ $/$ $\wedge$]
- $\wedge$ $\rightarrow$ right-*assoc*, no pop $\rightarrow$ [$+$ $/$ $\wedge$ $\wedge$] (4)
- $-$ $\rightarrow$ pops $\wedge$, $\wedge$, $/$, $+$ $\rightarrow$ [$-$]
- end $\rightarrow$ pop $-$.

Postfix: $a \ b \ c \ d \ - \ * \ e \ f \ g \ \wedge \ \wedge \ / \ + \ h \ -$. Maximum stack size = 4.

Trap: treating $\wedge$ as left-associative pops the first $\wedge$ before pushing the second, so the maximum would still be reached inside the parentheses — but the postfix would wrongly become ... $e \ f \ \wedge \ g \ \wedge \ ...$



Q.3 [MSQ · 1 mark · Trees — complete binary tree in an array]**Answer: (B), (C), (D)**

Concept. In a 1-based array heap layout, node i has children $2i$ and $2i + 1$ and parent $\lfloor i/2 \rfloor$. Node i is internal iff $2i \leq n$.

- (A) $2 \times 26 = 52 > 50$, so $T[26]$ has no children — it is a leaf. **False.**
- (B) Internal nodes are $i = 1 \dots \lfloor 50/2 \rfloor = 25$, so leaves are $26 \dots 50 \rightarrow 25$ leaves. **True.**
- (C) $\text{parent}(37) = \lfloor 37/2 \rfloor = 18$. **True.**
- (D) Height = $\lfloor \log_2 50 \rfloor = 5$ (levels hold 1, 2, 4, 8, 16 and then 19 nodes; $2^5 = 32 \leq 50 < 64$). **True.**

Note $T[25]$ is the only node with exactly one child ($T[50]$), because n is even.

Trap: using 0-based formulas (children $2i + 1$, $2i + 2$) on a 1-based array gives $\text{parent}(37) = 18$ by coincidence but breaks the leaf test.

Q.4 [NAT · 1 mark · Hashing — double hashing]**Answer: 12**

Concept. In double hashing the step size $h_2(k)$ depends on the key, so keys sharing a home slot follow different probe sequences.

Trace ($h_1, h_2 \rightarrow \text{slot}$):

- 18: $h_1 \ 5 \rightarrow \text{slot } 5$.
- 41: $h_1 \ 2 \rightarrow \text{slot } 2$.
- 22: $h_1 \ 9 \rightarrow \text{slot } 9$.
- 44: $h_1 \ 5$ (full), $h_2 = 1 + 0 = 1 \rightarrow 6$.
- 59: $h_1 \ 7 \rightarrow \text{slot } 7$.
- 32: $h_1 \ 6$ (full), $h_2 = 1 + 10 = 11 \rightarrow (6 + 11) \bmod 13 = 4$.
- 31: $h_1 \ 5$ (full), $h_2 = 1 + 9 = 10 \rightarrow (5 + 10) \bmod 13 = 2$ (full) $\rightarrow (5 + 20) \bmod 13 = \mathbf{12}$.
- 73: $h_1 \ 8 \rightarrow \text{slot } 8$.

Key 31 is stored in slot **12**.

Trap: using $h_2 = k \bmod 11$ (without the +1) would give step 9 for 31; and forgetting the mod 13 after adding $2 \cdot h_2 = 20$ gives 25.

0	
1	
2	41
3	
4	32
5	18
6	44
7	59
8	73
9	22
10	
11	
12	31

Final table

Q.5 [MCQ · 1 mark · Bubble sort — passes with early exit]**Answer: (A)**

Concept. Each bubble pass carries the largest unsorted element all the way to the right ('rabbits' move fast), but a small element moves only **one** position left per pass ('turtles' move slowly). The early-exit flag stops after the first pass with no swaps.

- [2, 3, 4, 5, 6, 1]: the 1 is a turtle at index 5; it reaches index 0 only after 5 passes. The loop allows at most $n - 1 = 5$ passes, so it ends after pass 5 (no extra check pass). → 5.
- [6, 1, 2, 3, 4, 5]: pass 1 carries 6 to the end → sorted; pass 2 makes no swap and breaks. → 2.

Output 5 2 → (A).

- (B) swaps the two cases.
- (D) forgets that the verifying pass (pass 2) is counted.

Trap: both arrays have exactly 5 inversions, so both need 5 swaps — but the number of *passes* depends on how far elements must move left.

Q.6 [MCQ · 1 mark · Linked lists — deleting from a doubly linked list]**Answer: (D)**

Concept. To unlink p from $x \rightleftharpoons p \rightleftharpoons y$, two pointers outside p must change: $x.next$ must become y and $y.prev$ must become x . Pointers inside p may stay as they are.

- (D) $x.next = y$; $y.prev = x$. Both directions skip p . **Correct.**
- (B) sets $y.prev = y$ (a self-loop) — backward traversal from y loops forever.
- (C) fixes $y.prev = x$ but never changes $x.next$, so forward traversal still reaches p ; then it rewires p 's own pointer, which does not help.
- (A) fixes $x.next = y$ but then only changes $p.next$ (to y again); $y.prev$ still points to p .

Trap: in (A) it *looks* like two pointers are updated, but only one of them belongs to a neighbour. Always check that both $x.next$ and $y.prev$ change.

Cost: $O(1)$ given p — the key advantage of a doubly linked list over a singly linked one, which needs the predecessor.

Q.7 [NAT · 1 mark · Graphs — BFS distances]**Answer: 4**

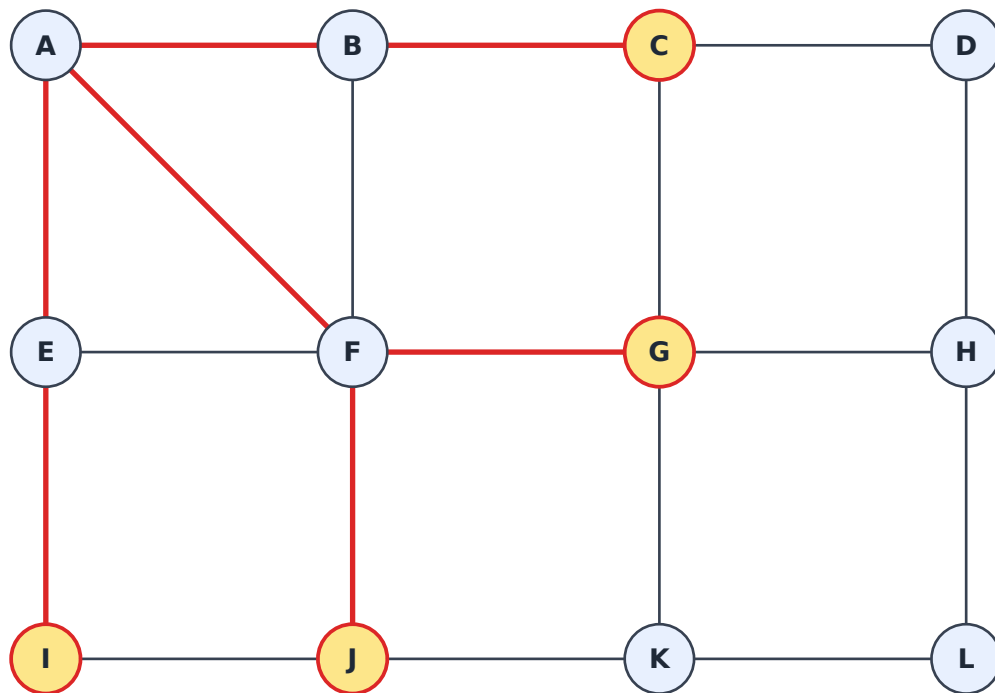
Concept. BFS from A labels each vertex with its hop distance. The diagonal A-F pulls F one step closer, which in turn pulls F's neighbours closer.

Level by level:

- Distance 0: A.
- Distance 1: neighbours of A → B, E, F (F via the diagonal).
- Distance 2: new neighbours of B (C), of E (I) and of F (G, J) → C, G, I, J.
- Distance 3: D (from C), H (from G), K (from G or J).
- Distance 4: L.

Exactly **4** vertices (C, G, I, J) are at distance 2. Without the diagonal, the distance-2 set would be {C, F, I} — 3 vertices — and G, J would be at distance 3.

Trap: K looks 'two squares away' diagonally from F, but grid moves are only along drawn edges: K's neighbours G, J, L are all at distance ≥ 2 , so K is at 3.



BFS tree edges (red) up to level 2; level-2 vertices highlighted

Q.8 [MCQ · 1 mark · Quicksort — worst-case comparisons]**Answer: (A)**

Concept. With the first element as pivot, a sorted array gives the most unbalanced split possible: the pivot is the minimum, the left part is empty and the right part has $m - 1$ elements. $T(m) = T(m - 1) + (m - 1)$.

Partition sizes: 8, 7, 6, 5, 4, 3, 2 (the final size-1 sub-array is not partitioned).

Comparisons: $7 + 6 + 5 + 4 + 3 + 2 + 1 = 28 = 8 \cdot 7/2 \rightarrow (A)$.

- (B) $36 = 9 \cdot 8/2$ counts one extra level.
- (C) $24 \approx n \log_2 n$ is the balanced (best-case-like) estimate.
- (D) 17 is merge sort's worst case for $n = 8$, not quicksort's.

Trap: sorted (or reverse-sorted) input is the *worst* case for naive pivot choice; randomised or median-of-three pivots avoid it.

Q.9 [NAT · 1 mark · Python — nested comprehensions and zip]**Answer: -2**

Concept. The outer comprehension variable (i) selects the row; the inner one (j) the column. `zip(*M)` transposes M: $T[c]$ is the tuple of column c.

- $M = [[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12]]$ (3 rows $\times$ 4 columns).
- $T = [(1, 2, 3), (2, 4, 6), (3, 6, 9), (4, 8, 12)]$.
- $T[2] = (3, 6, 9) \rightarrow \text{sum } 18$.
- $M[1] = [2, 4, 6, 8] \rightarrow \text{sum } 20$.

Printed value: $18 - 20 = -2$.

Trap: reading the comprehension 'inside out' gives a 4×3 matrix, and then $T[2]$ would be a row of length 4. Remember: the **leftmost** `for` in a nested comprehension of lists builds the outer list.

Q.10 [MCQ · 1 mark · Binary search — worst-case probes]**Answer: (C)**

Concept. After a probe on a range of size m , the larger remaining part has size $\lfloor m/2 \rfloor$. The worst-case number of probes $W(m)$ satisfies $W(m) = 1 + W(\lfloor m/2 \rfloor)$, $W(0) = 0$, giving $W(m) = \lfloor \log_2 m \rfloor + 1$.

For $m = 1000$: sizes $1000 \rightarrow 500 \rightarrow 250 \rightarrow 125 \rightarrow 62 \rightarrow 31 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 0$, i.e. 10 probes. Equivalently $\lfloor \log_2 1000 \rfloor + 1 = 9 + 1 = \mathbf{10} \rightarrow (C)$.

- (A) $9 = \lfloor \log_2 1000 \rfloor$ forgets the final probe on a one-element range.
- (B) $11 = \lfloor \log_2 1000 \rfloor + 1$ over-counts.
- (D) 500 is linear search's average, not binary search.

Tip: the decision tree of binary search on n keys has height $\lfloor \log_2 n \rfloor$, and an unsuccessful search needs at most $\lfloor \log_2 n \rfloor + 1$ probes.

Q.11 [MCQ · 2 marks · Python — class vs instance attributes]**Answer: (C)**

Concept. Attribute *lookup* through an instance falls back to the class, but attribute *assignment* through an instance creates/updates an **instance** attribute. Mutating a class-level list through **self** mutates the one shared list.

- `self.hits.append(k)`: looks up `hits` → finds the class list (no instance attribute) → appends to the **shared** list. After three calls it holds `[3, 4, 5]`; `len(a.hits) = 3`.
- `self.total += k` means `self.total = self.total + k`. The right side reads the class value 0 the first time; the assignment then creates an instance attribute.
- `a.hit(3)`: `a.total = 0 + 3 = 3`.
- `b.hit(4)`: `b.total = 0 + 4 = 4`.
- `a.hit(5)`: `a.total = 3 + 5 = 8` (now read from the instance).
- `Counter.total` was never assigned → still 0.

Output: `3 8 4 0` → (C).

- (A) assumes `+=` writes back to the class.
- (B) treats `hits` as per-instance.
- (D) treats `total` as shared like `hits`.

Trap: `+=` on an immutable int rebinds; `.append` on a list mutates. The same `self.x` syntax behaves differently.

Q.12 [MSQ · 2 marks · Heaps — successive insertions into a max-heap]**Answer: (A), (B), (D)**

Concept. Insertion places the key at the next free leaf and sifts it up; the number of swaps is at most the depth of that leaf.

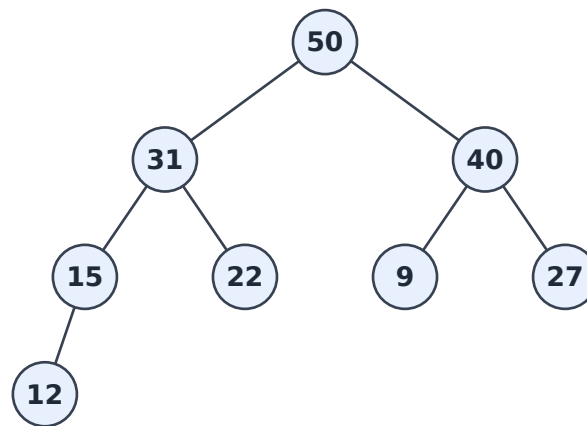
Trace (array after each insertion, swaps in brackets):

- `15` → `[15]` (0)
- `22` → `[15, 22]` → swap with `15` → `[22, 15]` (1)
- `9` → `[22, 15, 9]` (0)
- `40` → index 3, parent 15 → swap → index 1, parent 22 → swap → `[40, 22, 9, 15]` (2)
- `31` → index 4, parent 22 → swap → index 1, parent 40 stop → `[40, 31, 9, 15, 22]` (1)
- `27` → index 5, parent 9 → swap → index 2, parent 40 stop → `[40, 31, 27, 15, 22, 9]` (1)
- `50` → index 6, parent 27 → swap → index 2, parent 40 → swap → `[50, 31, 40, 15, 22, 9, 27]` (2)
- `12` → index 7, parent 15 → stop → `[50, 31, 40, 15, 22, 9, 27, 12]` (0)

Total swaps = $1 + 2 + 1 + 1 + 2 = 7$.

- (A) **True.**
- (B) **True.**
- (C) **False** — 40 is at index 2, the **right** child of the root; 31 is the left child.
- (D) **True.**

Trap: repeated insertion and bottom-up build-heap generally produce *different* heaps from the same keys; do not mix the two procedures.



Final max-heap

Q.13 [MSQ · 2 marks · Graphs — DFS timestamps and edge classification]

Answer: (A), (B), (C)

Concept. For an edge $u \rightarrow v$ met during DFS: v undiscovered $\rightarrow$ tree edge; v discovered but not finished (on the recursion stack) $\rightarrow$ back edge; v finished and $d[u] < d[v] \rightarrow$ forward edge; v finished and $d[u] > d[v] \rightarrow$ cross edge.

Trace:

- A (d 1) $\rightarrow$ B (d 2) $\rightarrow$ D (d 3). D $\rightarrow$ A: A active $\rightarrow$ **back**. D $\rightarrow$ F: F (d 4).
- F $\rightarrow$ C: C (d 5). C $\rightarrow$ B: B active $\rightarrow$ **back**. C $\rightarrow$ E: E (d 6). E $\rightarrow$ F: F active $\rightarrow$ **back**. E finishes (7). C finishes (8). F finishes (9). D finishes (10). B finishes (11).
- A $\rightarrow$ C: C already finished and $d[A] = 1 < d[C] = 5 \rightarrow$ **forward**. A finishes (12).

Tree edges: A $\rightarrow$ B, B $\rightarrow$ D, D $\rightarrow$ F, F $\rightarrow$ C, C $\rightarrow$ E. Back: D $\rightarrow$ A, C $\rightarrow$ B, E $\rightarrow$ F. Forward: A $\rightarrow$ C. No cross edges.

- (A) **True** — three back edges.
- (B) **True** — $d[C] = 5$, $f[C] = 8$.
- (C) **True.**
- (D) **False** — when C $\rightarrow$ B is examined, B is still on the stack (it finishes at 11), so it is a back edge.

Trap: B was discovered long before C, which tempts one to call C $\rightarrow$ B a cross edge; the deciding test is whether B is still active.

Q.14 [NAT · 2 marks · Recurrences — exact evaluation with floors]

Answer: 92

Concept. This is the merge-sort-style recurrence; asymptotically $T(n) = \Theta(n \log n)$, but with floors the exact value must be computed by unwinding.

- $T(2) = 2 \cdot T(1) + 2 = 4$
- $T(5) = 2 \cdot T(2) + 5 = 13$
- $T(10) = 2 \cdot T(5) + 10 = 36$
- $T(20) = 2 \cdot T(10) + 20 = \mathbf{92}$

Sanity check against the power-of-two closed form $T(2^k) = 2^k(k + 1)$: $T(16) = 16 \times 5 = 80$ and $T(32) = 32 \times 6 = 192$, and $80 < 92 < 192$ as expected for a non-decreasing function.

Trap: using $n \log_2 n + n = 20 \times 4.32 + 20 \approx 106$ — the closed form holds only for powers of two. Another slip is $T(5) = 2 \cdot T(2.5)$: floors must be applied.

Q.15 [MSQ · 2 marks · Hashing — deletion with tombstones]

Answer: (A), (B), (D)

Concept. In open addressing, deleting by emptying a slot can break the probe chains of keys inserted after it. A tombstone keeps chains intact for searches and can be reused by insertions.

Insertions: $31 \rightarrow 1$; $41 \rightarrow 1$ full $\rightarrow 2$; $51 \rightarrow 1$, 2 full $\rightarrow 3$; $22 \rightarrow 2$, 3 full $\rightarrow 4$; $62 \rightarrow 2$, 3, 4 full $\rightarrow 5$. Delete $41 \rightarrow$ slot 2 = DEL.

- (A) Insert 71: home 1 occupied, slot 2 is DELETED $\rightarrow$ 71 goes into slot 2. **True.**
- (B) Search 51: slot 1 (31), slot 2 (DEL, continue), slot 3 (51) $\rightarrow$ 3 probes. **True.**
- (C) Search 22 with slot 2 EMPTY: the home slot 2 is empty $\rightarrow$ search stops and **fails**, although 22 is in slot 4. **False.**
- (D) Search 72: home 2 (DEL), 3 (51), 4 (22), 5 (62), 6 (EMPTY) $\rightarrow$ 5 probes. **True.**

Trap: (C) is exactly why tombstones exist; many students assume emptying the slot is harmless.

0	
1	31
2	DEL
3	51
4	22
5	62
6	
7	
8	
9	

Table after deleting 41 (tombstone in slot 2)

Q.16 [NAT · 2 marks · Graphs — counting topological orderings]

Answer: 42

Concept. Count linear extensions by splitting on structure: here F must come after D and E; G only after E; D after A, B; E after B, C.

Step 1 — orders of A, B, C, D, E (F comes after all of them except possibly G, so place F last among these six). Constraints: A, B before D; B, C before E. The last of the five must be D or E:

- Last = E: arrange A, B, D with D after A and B (2 ways: ABD, BAD), then insert C in any of 4 gaps $\rightarrow$ 8 orders. Here E is in position 5.
- Last = D: symmetric — arrange B, C, E with E last (2 ways), insert A in any of 4 gaps $\rightarrow$ 8 orders. In 6 of them A is inserted before E (E in position 4); in 2 of them A is inserted after E (B, C, E, A, D and C, B, E, A, D: E in position 3).
- Total 16 orders; appending F gives 16 orders of the six vertices A-F.

Step 2 — insert G. G's only constraint is that it follows E (it has no successors), so in a six-vertex order with E at position p it can go into any of the 7 - p gaps after E:

- E at position 5 (8 orders): 2 gaps $\rightarrow$ 16.
- E at position 4 (6 orders): 3 gaps $\rightarrow$ 18.

- E at position 3 (2 orders): 4 gaps $\rightarrow$ 8.
- Total = 16 + 18 + 8 = **42**.

Trap: forgetting that G is a sink with a single predecessor, so it can float anywhere after E — including after F.

Q.17 [MCQ · 2 marks · Stacks — monotonic stack (next greater element)]

Answer: (B)

Concept. The stack holds indices whose next greater element has not been seen; their values are non-increasing from bottom to top. A new x pops (and resolves) every smaller value on top. Each index is pushed and popped at most once $\rightarrow O(n)$.

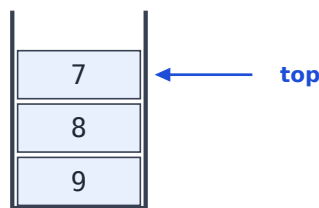
Trace (stack shows values):

- 4: push $\rightarrow [4]$
- 9: pops 4 ($\text{res}[0] = 9$) $\rightarrow [9]$
- 2: push $\rightarrow [9, 2]$
- 5: pops 2 ($\text{res}[2] = 5$) $\rightarrow [9, 5]$
- 3: push $\rightarrow [9, 5, 3]$
- 8: pops 3 ($\text{res}[4] = 8$), pops 5 ($\text{res}[3] = 8$) $\rightarrow [9, 8]$
- 1: push $\rightarrow [9, 8, 1]$
- 7: pops 1 ($\text{res}[6] = 7$); $8 > 7$ stops $\rightarrow [9, 8, 7]$
- End: 9, 8, 7 remain $\rightarrow -1$.

Result $[9, -1, 5, 8, 8, -1, 7, -1] \rightarrow$ (B).

- (A) gives the last element a next-greater of itself.
- (C) assigns 9 to the 5 — but 8 appears first to its right.
- (D) gives 3 the value 5, which is to its **left**.

Trap: the answer for 5 is the *first* greater element to the right (8), not the maximum to the right (9).



Values left on the stack at the end (answer -1)

Q.18 [MSQ · 2 marks · Merge sort and quicksort — exact comparison counts]

Answer: (A), (C)

Concept. Exact counts depend on how the splits fall. All-equal keys are a worst case for Lomuto (every key $\leq$ pivot), while sorted input is a best case for merge sort's merging.

- (A) Equal keys: the pivot ends up at the far right every time, leaving $m - 1$ keys on the left. Comparisons $4 + 3 + 2 + 1 = 10$. **True.**
- (B) Taking the right element on ties puts later equal keys before earlier ones $\rightarrow$ **not stable. False.**
- (C) Sorted input: every merge of sizes $p = q$ stops after p comparisons (left list empties first). Level 1: $4 \text{ merges} \times 1 = 4$; level 2: $2 \times 2 = 4$; level 3: $1 \times 4 = 4 \rightarrow 12$. **True.**
- (D) $[3, 1, 2]$, pivot 2: 2 comparisons $\rightarrow [1, 2, 3]$, pivot index 1; the left part $[1]$ and right part $[3]$ have size 1 $\rightarrow$ no more comparisons. Total **2. False.**

Trap: for (D), counting one comparison for the pivot with itself. For (B), stability lives entirely in the tie-breaking rule of the merge.

Q.19 [NAT · 2 marks · Linked lists — Floyd's cycle detection]**Answer: 11**

Concept. Let μ be the tail length (nodes before the cycle) and λ the cycle length. Here $\mu = 3$ (nodes 0, 1, 2) and $\lambda = 8$ (nodes 3 ... 10). After k steps slow is at position k and fast at $2k$; they meet at the first $k \geq \mu$ with $k \equiv 0 \pmod{\lambda}$. Then a pointer from the head and one from the meeting point, moving one step at a time, meet exactly at the cycle start after μ steps.

Phase 1: smallest $k \geq 3$ divisible by 8 $\rightarrow k = 8$. Check: slow at node 8; fast has moved 16 steps: $0 \rightarrow 10$ is 10 steps, then 6 more around the cycle $3, 4, \dots: 10 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$. ✓ steps = 8.

Phase 2: p from node 0 and slow from node 8 advance together: (1, 9), (2, 10), (3, 3) $\rightarrow$ 3 more steps; they meet at node 3, the cycle start.

Total steps = $8 + 3 = 11$; the program prints **11 3**.

Trap: assuming the pointers meet at the cycle start in phase 1, or that phase 1 takes exactly $\lambda - \mu$ steps. In general phase 1 takes $\lambda \cdot \lceil \mu / \lambda \rceil$ steps (or λ if $\mu = 0$).

Q.20 [MCQ · 2 marks · Binary search trees — insertion orders giving the same tree]**Answer: (D)**

Concept. A permutation yields the given BST iff the root comes first and, recursively, the keys of each subtree appear in an order that builds that subtree. Keys of the left and right subtrees may be **interleaved** arbitrarily. So $N(T) = C(|L| + |R|, |L|) \cdot N(L) \cdot N(R)$.

- Right subtree {70, 60, 80}: 70 first, then 60 and 80 in either order $\rightarrow N = 2$.
- Left subtree {30, 20, 40, 35}: 30 first; then interleave {20} with the sequence (40 before 35): $C(3, 1) \cdot 1 \cdot 1 = 3 \rightarrow N = 3$.
- Whole tree: 50 first, then interleave 4 left keys with 3 right keys: $C(7, 4) = 35$.

$N = 35 \times 3 \times 2 = 210 \rightarrow (D)$.

- (A) 35 counts only the interleavings at the root.
- (B) 105 forgets the factor 2 for the right subtree.
- (C) 420 also lets 35 precede 40 (impossible: 35 must hang below 40).

Trap: the subtree root must precede all its descendants, but keys from different subtrees impose no order on each other.

Mock Test 36

Full-Syllabus Mock — Paper 6

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — floor division and modulo with negatives]

Consider the following Python program. What is printed?

Python

```
1 print(-17 // 5, -17 % 5, 17 // -5, int(-17 / 5))
```

- (A) -3 -2 -3 -3
- (B) -4 3 -4 -3
- (C) -4 -2 -4 -4
- (D) -3 3 -4 -3

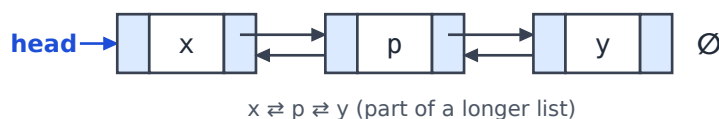
Q.2 [NAT · numerical answer | 1 mark | Circular queue — capacity and wrap-around]

A circular queue is implemented in an array of size 8 with indices **front** and **rear**, both initially 0. **enqueue(x)** stores x at **rear** and sets $\text{rear} = (\text{rear} + 1) \bmod 8$, but is **ignored** if $(\text{rear} + 1) \bmod 8 = \text{front}$ (queue full). **dequeue()** returns the element at **front** and sets $\text{front} = (\text{front} + 1) \bmod 8$. The operations performed are: enqueue 1, 2, ..., 9 (in order); then 3 dequeues; then enqueue 10, 11, 12, 13. The number of enqueue operations that were ignored is ____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Doubly linked lists — deleting a node]

In a doubly linked list each node has fields **prev** and **nxt**. Node **p** is neither the first nor the last node. Which of the following two-statement fragments correctly unlinks **p** from the list (leaving all other links consistent)?



- (A) $p.\text{nxt}.\text{prev} = p.\text{prev}; p.\text{prev}.\text{nxt} = p.\text{nxt}.\text{prev}$
- (B) $p.\text{prev}.\text{nxt} = p.\text{nxt}; p.\text{nxt}.\text{prev} = p.\text{prev}.\text{nxt}$
- (C) $p.\text{prev}.\text{nxt} = p.\text{nxt}; p.\text{prev}.\text{nxt}.\text{prev} = p.\text{prev}$
- (D) $p.\text{nxt} = p.\text{nxt}.\text{nxt}; p.\text{nxt}.\text{prev} = p$

Q.4 [NAT · numerical answer | 1 mark | Heaps — possible positions of a key]

A max-heap with 25 **distinct** keys is stored in an array $A[0..24]$ (children of i at $2i+1$ and $2i+2$). The number of distinct indices at which the **third largest** key could be located is ____.

Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | Hashing — probe sequences that cover the table]

For a key with home slot h , an open-addressing scheme examines slots $s_i = (h + g(i)) \bmod m$ for $i = 0, 1, \dots, m - 1$. For which of the following choices are **all m slots** guaranteed to be examined?

- (A) $m = 12, g(i) = 4i$
- (B) $m = 11, g(i) = i^2$
- (C) $m = 16, g(i) = i(i + 1)/2$
- (D) $m = 12, g(i) = 5i$

Q.6 [MCQ · single correct | 1 mark | Binary search — worst-case probes]

The iterative binary search below is run on a sorted list of **100** distinct integers. The maximum possible value returned (over all search keys x , present or absent) is

Python

```
1 def bs(A, x):
2     lo, hi, c = 0, len(A) - 1, 0
3     while lo <= hi:
4         m = (lo + hi) // 2
5         c += 1
6         if A[m] == x:
7             return c
8         if A[m] < x:
9             lo = m + 1
10        else:
11            hi = m - 1
12    return c
```

- (A) 6
- (B) 10
- (C) 8
- (D) 7

Q.7 [NAT · numerical answer | 1 mark | Insertion sort — counting comparisons]

Insertion sort (ascending) is run on $[6, 2, 9, 4, 4, 1, 8]$. In each pass, the key $A[i]$ is compared with $A[i-1], A[i-2], \dots$; each such check $A[j] > \text{key}$ counts as **one** comparison, and the pass stops at the first $A[j] \leq \text{key}$ or when $j < 0$. The total number of comparisons is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Graph theory — edges versus components]

A simple undirected graph has 10 vertices and exactly 3 connected components. The maximum possible number of edges is

- (A) 28
- (B) 36
- (C) 21
- (D) 27

Q.9 [MSQ · one or more correct | 1 mark | Python — dictionaries, sets and stable sorting]

Consider the following Python program. After it runs, which of the following expressions evaluate to **True**?

Python

```
1 d = {}
2 for w in "the cat sat on the mat the end".split():
3     d[w] = d.get(w, 0) + 1
```

- (A) `sorted(d.items(), key=lambda kv: -kv[1])[1][0] == 'cat'`
- (B) `len(set('banana') - set('ban')) > 0`
- (C) `max(d, key=d.get) == 'the'`
- (D) `list(d)[0] == 'the'`

Q.10 [MCQ · single correct | 1 mark | Recurrences — master theorem]

The solution of the recurrence $T(n) = 2T(n/4) + \sqrt{n}$, with $T(1) = 1$, is

- (A) $\Theta(\sqrt{n} \log n)$
- (B) $\Theta(\sqrt{n})$
- (C) $\Theta(n)$
- (D) $\Theta(n^{1/2} \log^2 n)$

Q.11 [NAT · numerical answer | 2 marks | Stacks — infix to postfix conversion]

The infix expression

`a - b * c ^ d ^ ( e + f * g ) / h`

is converted to postfix with the standard operator-stack algorithm. Precedence: $\wedge$ (highest, **right**-associative) $>$ $*$, $/$ (left-associative) $>$ $+$, $-$ (left-associative). A left parenthesis is pushed onto the same stack. The maximum number of symbols (operators and parentheses) on the stack at any time is _____.

Answer: _____ (enter a numerical value)

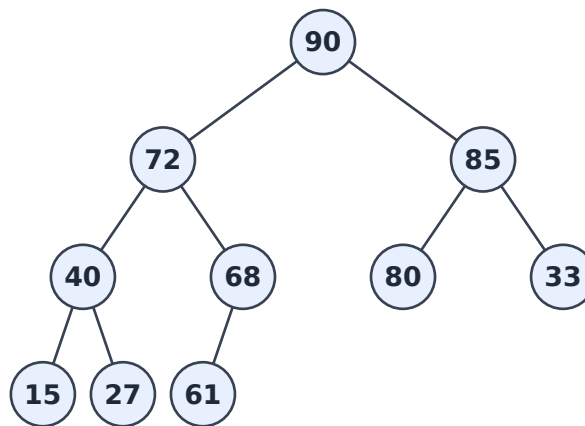
Q.12 [MCQ · single correct | 2 marks | BST — insertion orders giving the same tree]

The keys 46, 23, 71, 12, 35, 58, 89, 30, 64 are inserted in this order into an empty BST. Which of the following insertion orders produces **exactly the same** BST?

- (A) 46, 23, 71, 30, 12, 35, 58, 89, 64
- (B) 46, 71, 23, 89, 64, 58, 12, 35, 30
- (C) 46, 23, 12, 30, 35, 71, 58, 64, 89
- (D) 46, 71, 58, 64, 23, 35, 12, 89, 30

Q.13 [MSQ · one or more correct | 2 marks | Heaps — delete-max]

Two delete-max operations are performed on the max-heap [90, 72, 85, 40, 68, 80, 33, 15, 27, 61] (shown below). Each delete-max moves the last element to the root and sifts it down, always swapping with the larger child. Which of the following statements is/are TRUE?



Initial max-heap

- (A) After both deletions the key 27 is at index 5
- (B) After both deletions the children of the root are 72 and 61
- (C) The two deletions perform 3 swaps in total
- (D) After the first delete-max the array is [85, 72, 80, 40, 68, 61, 33, 15, 27]

Q.14 [NAT · numerical answer | 2 marks | Quicksort — counting comparisons]

Quicksort using the Lomuto partition (pivot = **last** element of the sub-array; every test $A[j] \leq \text{pivot}$ is one comparison) sorts [4, 7, 2, 6, 1, 8, 3, 5]. Sub-arrays of size 0 or 1 are not partitioned. The total number of comparisons is _____.

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Python — generator exhaustion with zip]

Consider the following Python program. What is printed?

Python

```

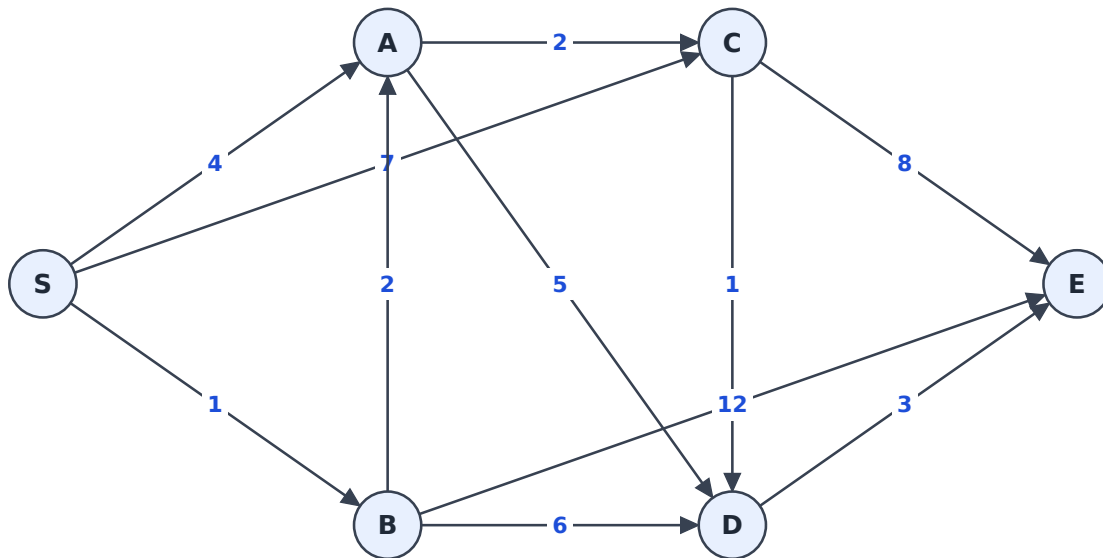
1 g = (x * x for x in range(6))
2 a = list(zip(g, "abc"))
3 b = list(g)
4 print(a, b, sum(g))

```

- (A) [(0, 'a'), (1, 'b'), (4, 'c')] [9, 16, 25] 50
- (B) [(0, 'a'), (1, 'b'), (4, 'c')] [16, 25] 0
- (C) [(0, 'a'), (1, 'b'), (4, 'c')] [9, 16, 25] 0
- (D) [(0, 'a'), (1, 'b'), (4, 'c')] [0, 1, 4, 9, 16, 25] 55

Q.16 [NAT · numerical answer | 2 marks | Dijkstra — counting successful relaxations]

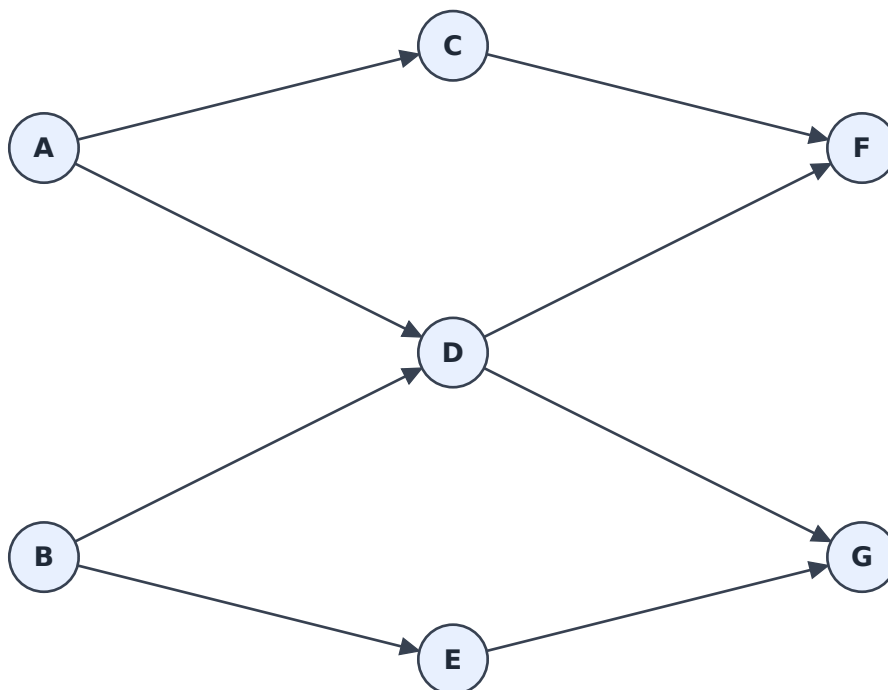
Dijkstra's algorithm is run from **S** on the directed graph shown below. A relaxation of edge (u, v) is *successful* if it strictly decreases $d[v]$ (including a decrease from ∞). Edges into already-extracted vertices are ignored. The total number of successful relaxations is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Topological sorting]

Which of the following are valid topological orderings of the DAG shown below?



- (A) A, B, D, C, E, G, F
- (B) A, E, B, D, C, F, G
- (C) B, E, A, D, C, G, F
- (D) A, C, F, B, D, E, G

Q.18 [NAT · numerical answer | 2 marks | Hashing — double hashing]

Keys 27, 40, 14, 53, 66, 79, 1 are inserted in this order into an empty hash table of size 13 using double hashing: probe i ($i = 0, 1, 2, \dots$) examines slot $(h_1(k) + i \cdot h_2(k)) \bmod 13$, where $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Counting every slot examined, the total number of probes over all seven insertions is _____.

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Bottom-up merge sort — intermediate state]

Bottom-up (iterative) merge sort is applied to [38, 16, 27, 39, 12, 27, 5, 44]. Pass 1 merges adjacent runs of length 1, pass 2 merges adjacent runs of length 2, and so on. The array after **pass 2** is

- (A) [5, 12, 16, 27, 27, 38, 39, 44]
- (B) [16, 38, 27, 39, 12, 27, 5, 44]
- (C) [16, 27, 38, 39, 5, 12, 27, 44]
- (D) [16, 27, 38, 39, 12, 27, 5, 44]

Q.20 [MSQ · one or more correct | 2 marks | Python — class versus instance attributes]

Consider the following Python program. Which of the following statements about its output is/are TRUE?

Python

```

1 class Counter:
2     total = 0
3     tags = []
4
5     def __init__(self, name):
6         self.name = name
7         Counter.total += 1
8         self.tags.append(name)
9
10    def bump(self):
11        self.total += 1
12
13    a = Counter("x")
14    b = Counter("y")
15    a.bump(); a.bump()
16    b.tags = ["z"]
17    print(Counter.total, a.total, b.total,
18          len(a.tags), len(Counter.tags), b.tags)

```

- (A) The fourth value printed is 1
- (B) The last value printed is ['z'], and Counter.tags is not affected by that assignment
- (C) The second value printed is 4
- (D) The first value printed is 2

Answer Key — Mock Test 36

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — floor division and modulo with negatives
2	NAT	1	3	Circular queue — capacity and wrap-around
3	MCQ	1	(C)	Doubly linked lists — deleting a node
4	NAT	1	6	Heaps — possible positions of a key
5	MSQ	1	(C), (D)	Hashing — probe sequences that cover the table
6	MCQ	1	(D)	Binary search — worst-case probes
7	NAT	1	15	Insertion sort — counting comparisons
8	MCQ	1	(A)	Graph theory — edges versus components
9	MSQ	1	(A), (C), (D)	Python — dictionaries, sets and stable sorting
10	MCQ	1	(A)	Recurrences — master theorem
11	NAT	2	7	Stacks — infix to postfix conversion
12	MCQ	2	(D)	BST — insertion orders giving the same tree
13	MSQ	2	(A), (B), (D)	Heaps — delete-max
14	NAT	2	13	Quicksort — counting comparisons
15	MCQ	2	(B)	Python — generator exhaustion with zip
16	NAT	2	9	Dijkstra — counting successful relaxations
17	MSQ	2	(A), (C)	Topological sorting
18	NAT	2	13	Hashing — double hashing
19	MCQ	2	(C)	Bottom-up merge sort — intermediate state
20	MSQ	2	(B), (C), (D)	Python — class versus instance attributes

Detailed Solutions — Mock Test 36

Q.1 [MCQ · 1 mark · Python — floor division and modulo with negatives]

Answer: (B)

Python's `//` is **floor** division (rounds toward $-\infty$) and `%` returns a result with the sign of the **divisor**, so that `a == (a // b) * b + a % b` always holds. `int()` of a float **truncates** toward 0.

- `-17 // 5`: -3.4 floored $\rightarrow -4$.
- `-17 % 5`: $-17 - (-4)(5) = -17 + 20 = 3$.
- `17 // -5`: -3.4 floored $\rightarrow -4$.
- `int(-17 / 5)`: -3.4 truncated $\rightarrow -3$.

Output: `-4 3 -4 -3`.

- (A) is C-style truncation for `//` and a negative remainder.
- (C) gets `//` right but uses C's remainder sign and floors in `int()`.
- (D) truncates the first division but floors the third — inconsistent.

Trap: `//` and `int(a / b)` differ exactly when the quotient is negative and inexact.

Q.2 [NAT · 1 mark · Circular queue — capacity and wrap-around]

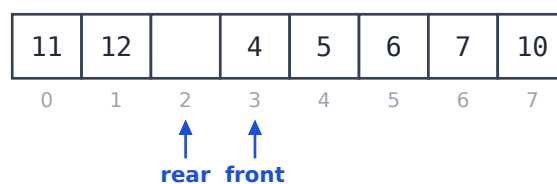
Answer: 3

With the 'one slot always empty' convention, an array of size 8 holds at most **7** elements; full means $(\text{rear} + 1) \bmod 8 = \text{front}$.

- Enqueue 1 ... 7 into slots 0 ... 6; rear = 7. Enqueue 8: $(7+1) \bmod 8 = 0 = \text{front} \rightarrow$ **ignored**. Enqueue 9 $\rightarrow$ **ignored**.
- 3 dequeues remove 1, 2, 3 $\rightarrow \text{front} = 3$; size 4.
- Enqueue 10 at slot 7 (rear $\rightarrow 0$), 11 at slot 0 (rear $\rightarrow 1$), 12 at slot 1 (rear $\rightarrow 2$).
- Enqueue 13: $(2+1) \bmod 8 = 3 = \text{front} \rightarrow$ **ignored**.

Ignored operations: 8, 9, 13 $\rightarrow$ **3**. Final state: front = 3, rear = 2, size 7.

Trap: assuming the capacity is 8 (then only 9 and none of the later ones would be rejected). The empty slot is what distinguishes 'full' from 'empty' (front = rear).



Final array (slot 2 is the reserved empty slot)

Q.3 [MCQ · 1 mark · Doubly linked lists — deleting a node]

Answer: (C)

We need `x.next = y` and `y.prev = x` (with `x = p.prev`, `y = p.next`). The catch is that the first statement changes a link that the second statement reads.

- (A) First: `y.prev = x`. Second: `x.next = p.next.prev = y.prev = x` $\rightarrow$ x points to itself. Wrong.
- (B) First: `x.next = y`. Second: `y.prev = p.prev.next = x.next = y` $\rightarrow$ y points back to itself. Wrong.
- (C) First: `x.next = y`. Second: `p.prev.next` is now y, so `y.prev = p.prev = x`. **Correct**.
- (D) Sets `p.next` to y's successor and that node's prev to p — this removes y, not p.

Trap: sequential pointer updates are not simultaneous; always re-evaluate each expression after the previous assignment (or save x and y in temporaries first).

Q.4 [NAT · 1 mark · Heaps — possible positions of a key]**Answer: 6**

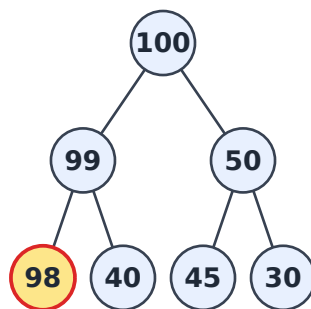
In a max-heap every key is smaller than all its ancestors. So a key with exactly two larger keys has **at most 2 ancestors** $\rightarrow \text{depth} \leq 2 \rightarrow \text{index} \in \{1, \dots, 6\}$ (it cannot be the root, which is the maximum).

All of these are achievable:

- depth 1 (index 1 or 2): e.g. largest at root, second and third largest as its two children.
- depth 2 (indices 3–6): the second largest is at index 1 (or 2) and the third largest is its child — choose either side, and either child.

Hence **6** possible indices.

Trap: answering 2 (only the root's children) forgets that the third largest can be a child of the second largest. In general the k -th largest can be anywhere at $\text{depth} \leq k - 1$.



Example: third largest (98) at index 3, a child of 99

Q.5 [MSQ · 1 mark · Hashing — probe sequences that cover the table]**Answer: (C), (D)**

A linear step c covers the whole table iff $\text{gcd}(c, m) = 1$. Quadratic sequences are subtler.

- (A) $\text{gcd}(4, 12) = 4 \rightarrow$ only offsets 0, 4, 8 are reached (3 slots). **False.**
- (B) $i^2 \bmod 11$ takes only the quadratic residues $\{0, 1, 3, 4, 5, 9\}$: $6 = (m+1)/2$ distinct slots. **False.** (Quadratic probing with prime m is guaranteed to find a free slot only when the table is at most half full.)
- (C) Triangular numbers $i(i+1)/2 \bmod 2^k$ are a permutation of $0 \dots 2^k - 1$; for $m = 16$ the offsets are 0, 1, 3, 6, 10, 15, 5, 12, 4, 13, 7, 2, 14, 11, 9, 8 — all 16. **True.**
- (D) $\text{gcd}(5, 12) = 1 \rightarrow 5i \bmod 12$ runs through all 12 residues. **True.**

Tip: this is why practical quadratic probing often uses triangular offsets with a power-of-two table size.

Q.6 [MCQ · 1 mark · Binary search — worst-case probes]**Answer: (D)**

Each iteration (probe) roughly halves the range; the worst case is $\lfloor \log_2 n \rfloor + 1$ probes, the height (in nodes) of the implicit decision tree.

For $n = 100$: $\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$.

Concretely, a search for a key larger than everything shrinks the range sizes $100 \rightarrow 50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1 \rightarrow 0$: 7 probes. The search for the smallest elements goes $100 \rightarrow 49 \rightarrow 24 \rightarrow 11 \rightarrow 5 \rightarrow 2 \rightarrow 0/1$ and can also need 7.

- (A) $6 = \lfloor \log_2 100 \rfloor$ forgets the $+1$.
- (B) 10 confuses with $\lfloor \log_2 1000 \rfloor$.
- (C) 8 would be needed only for $n \geq 128$.

Tip: worst case = $\lceil \log_2(n + 1) \rceil = 7$ for $64 \leq n \leq 127$.

Q.7 [NAT · 1 mark · Insertion sort — counting comparisons]

Answer: 15

A pass for key $A[i]$ costs (number of larger elements to its left, which get shifted) + 1, except that the +1 is missing when the key travels all the way to index 0 (the loop ends because $j < 0$, not because of a comparison).

- $i=1$, key 2: $6 > 2$ shift; $j < 0 \rightarrow 1$ comparison $\rightarrow [2, 6, 9, 4, 4, 1, 8]$
- $i=2$, key 9: $6 \leq 9 \rightarrow 1 \rightarrow$ unchanged
- $i=3$, key 4: $9 >, 6 >, 2 \leq \rightarrow 3 \rightarrow [2, 4, 6, 9, 4, 1, 8]$
- $i=4$, key 4: $9 >, 6 >, 4 \leq$ (equal stops — keeps stability) $\rightarrow 3 \rightarrow [2, 4, 4, 6, 9, 1, 8]$
- $i=5$, key 1: $9, 6, 4, 4, 2$ all > 1 ; $j < 0 \rightarrow 5 \rightarrow [1, 2, 4, 4, 6, 9, 8]$
- $i=6$, key 8: $9 >, 6 \leq \rightarrow 2 \rightarrow [1, 2, 4, 4, 6, 8, 9]$

Total = $1 + 1 + 3 + 3 + 5 + 2 = 15$.

Trap: adding 1 for every pass (giving 17) double-counts the two passes whose key reached the front; and treating the equal 4 as 'greater' (giving 16) breaks stability.

Q.8 [MCQ · 1 mark · Graph theory — edges versus components]

Answer: (A)

To maximise edges with k components, make each component complete and make the sizes as **unequal** as possible: one big clique plus $k - 1$ isolated vertices, because $C(a, 2) + C(b, 2)$ grows when we move a vertex from the smaller to the larger part.

Sizes 8, 1, 1 $\rightarrow C(8, 2) + 0 + 0 = 28$ edges.

Compare with balanced sizes 4, 3, 3 $\rightarrow 6 + 3 + 3 = 12$, or 6, 2, 2 $\rightarrow 15 + 1 + 1 = 17$.

General formula: $C(n - k + 1, 2)$.

- (B) $36 = C(9, 2)$ is the maximum for 2 components.
- (C) $21 = C(7, 2)$ would be for 4 components.
- (D) 27 has no justification ($28 - 1$).

Trap: spreading vertices evenly *minimises*, not maximises, the edge count.

Q.9 [MSQ · 1 mark · Python — dictionaries, sets and stable sorting]

Answer: (A), (C), (D)

Since Python 3.7 dictionaries preserve **insertion order**; **sorted** is **stable**.

The dictionary is `{'the': 3, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1, 'end': 1}` (in this order).

- (A) Sorting by `-count` puts 'the' first; all others tie at 1 and keep their insertion order, so the second item is ('cat', 1). **True.**
- (B) `set('banana') = {'b', 'a', 'n'} = set('ban')`; the difference is empty $\rightarrow$ length 0. **False.**
- (C) 'the' has the largest count, 3. **True.**
- (D) The first key inserted is 'the'. **True.**

Trap: for (A), one might think ties are broken alphabetically ('end') — they are not; a stable sort keeps the original order.

Q.10 [MCQ · 1 mark · Recurrences — master theorem]

Answer: (A)

Master theorem with $a = 2$, $b = 4$: $n^{\log_b a} = n^{\log_4 2} = n^{1/2}$. The driving function $f(n) = \sqrt{n}$ is of the **same order**, which is case 2 $\rightarrow T(n) = \Theta(\sqrt{n} \log n)$.

Recursion-tree view: level i has 2^i subproblems of size $n/4^i$, each costing $\sqrt{n/4^i} = \sqrt{n} / 2^i$; total per level $= 2^i \cdot \sqrt{n} / 2^i = \sqrt{n}$. There are $\log_4 n + 1$ levels, so $T(n) \approx \sqrt{n} (\log_4 n + 1) = \Theta(\sqrt{n} \log n)$.

- (B) would hold if $f(n)$ were polynomially smaller than $\sqrt{n}$ (case 1).
- (C) would need $f(n) = n$ (case 3).
- (D) would need $f(n) = \sqrt{n} \log n$.

Trap: concluding $\Theta(\sqrt{n})$ because 'the leaves and the root both cost $\sqrt{n}$ ' — in case 2 every level costs the same, so the level count multiplies in.

Q.11 [NAT · 2 marks · Stacks — infix to postfix conversion]

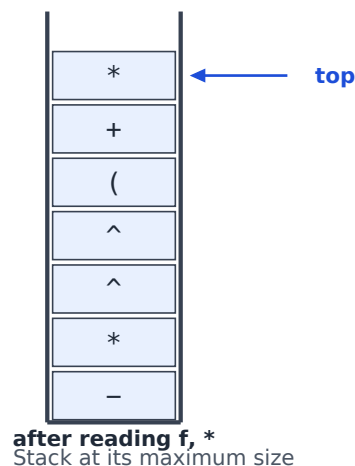
Answer: 7

Rules: operands go straight to the output; an incoming operator first pops operators of **higher** precedence, or **equal** precedence if it is left-associative; (is pushed;) pops until the matching (.

- a $\rightarrow$ out; - $\rightarrow$ stack [-]
- b; * (higher than -) $\rightarrow$ [-, *]
- c; ^ $\rightarrow$ [-, *, ^]
- d; ^ is right-associative, so the ^ on top is **not** popped $\rightarrow$ [-, *, ^, ^]
- ($\rightarrow$ [-, *, ^, ^, (] (5)
- e; + $\rightarrow$ [-, *, ^, ^, (, +] (6)
- f; * (higher than +) $\rightarrow$ [-, *, ^, ^, (, +, *] (7)
- g;) pops *, + and discards ($\rightarrow$ [-, *, ^, ^]
- / pops ^, ^, * (all $\geq$ its precedence) $\rightarrow$ [-, /]; h; end pops / and -.

Postfix: a b c d e f g * + ^ ^ * h / -. Maximum stack size = 7.

Trap: treating ^ as left-associative pops the first ^ when the second arrives, giving a maximum of 6 and a different (wrong) postfix.



Q.12 [MCQ · 2 marks · BST — insertion orders giving the same tree]

Answer: (D)

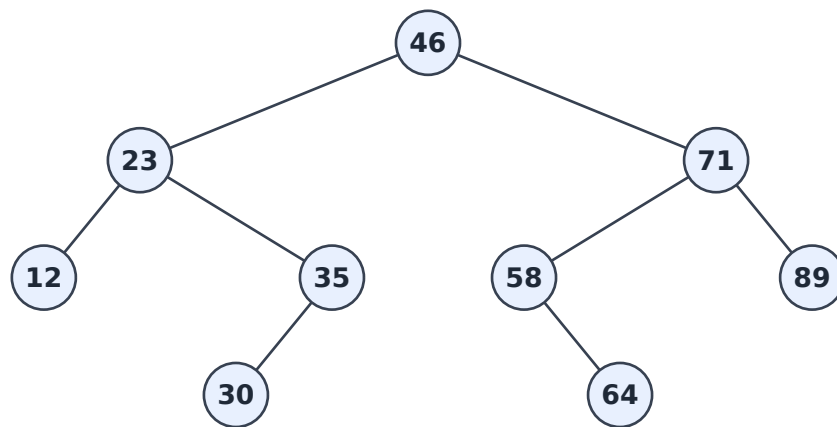
Two orders give the same BST iff, for every node, it is inserted **after all of its ancestors**. So list the ancestor constraints of the target tree and check each option.

Target tree: 46 $\rightarrow$ (23 $\rightarrow$ 12, (35 $\rightarrow$ 30, -)), (71 $\rightarrow$ (58 $\rightarrow$ -, 64), 89). Constraints: 46 first; 23 before 12 and 35; **35 before 30**; 71 before 58 and 89; **58 before 64**.

- (A) 30 comes before 35 $\rightarrow$ 30 becomes 23's right child and 35 hangs below 30. **Different.**
- (B) 64 precedes 58 $\rightarrow$ 64 becomes 71's left child and 58 its left child. **Different.**
- (C) 30 again precedes 35. **Different.**

- (D) 46, 71, 58, 64 (58 before 64 ✓), 23, 35, 12, 89, 30 (35 before 30 ✓). All constraints hold.
Same tree.

Tip: you never need to build four trees — just check 'ancestor before descendant' for the few deep nodes.



Target BST

Q.13 [MSQ · 2 marks · Heaps — delete-max]

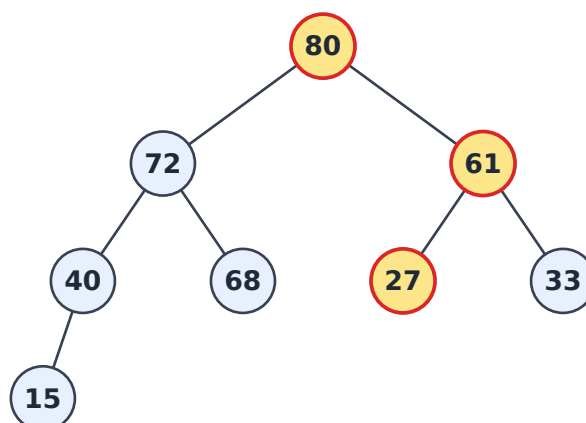
Answer: (A), (B), (D)

Delete 90: move 61 (last) to the root → [61, 72, 85, 40, 68, 80, 33, 15, 27]. 61 vs children 72, 85 → swap with 85 (index 2). Children of index 2: 80, 33 → swap with 80 (index 5), a leaf. Result [85, 72, 80, 40, 68, 61, 33, 15, 27] (2 swaps).

Delete 85: move 27 to the root → [27, 72, 80, 40, 68, 61, 33, 15]. Swap with 80 (index 2); children 61, 33 → swap with 61 (index 5); leaf. Result [80, 72, 61, 40, 68, 27, 33, 15] (2 swaps).

- (A) 27 ends at index 5. **True.**
- (B) Root 80 has children 72 (index 1) and 61 (index 2). **True.**
- (C) $2 + 2 = 4$ swaps. **False.**
- (D) **True.**

Trap: swapping with the *left* child by default (72) would violate the heap property because $85 > 72$.



After two delete-max operations

Q.14 [NAT · 2 marks · Quicksort — counting comparisons]

Answer: 13

A Lomuto partition of a sub-array of size s makes exactly $s - 1$ comparisons. So we only need the sizes of all partitioned sub-arrays.

- [4, 7, 2, 6, 1, 8, 3, 5], pivot 5: **7** comparisons → [4, 2, 1, 3, **5**, 8, 6, 7].
- Left [4, 2, 1, 3], pivot 3: **3** → [2, 1, **3**, 4].
- [2, 1], pivot 1: **1** → [**1**, 2]. ([4] and [2] are size 1.)
- Right [8, 6, 7], pivot 7: **2** → [6, **7**, 8].

Total = 7 + 3 + 1 + 2 = **13**.

Tip: with $C(s) = s - 1 + C(\text{left}) + C(\text{right})$, the minimum for $n = 8$ is $7 + C(3) + C(4) = 7 + 2 + 4 = 13$ (perfectly balanced splits) and the maximum is $n(n-1)/2 = 28$ (e.g. already sorted input). Here every pivot splits evenly, so this input hits the best case.

Partition calls

	sub-array	pivot	result	comparisons
1	4 7 2 6 1 8 3 5	5	4 2 1 3 5 8 6 7	7
2	4 2 1 3	3	2 1 3 4	3
3	2 1	1	1 2	1
4	8 6 7	7	6 7 8	2

Q.15 [MCQ · 2 marks · Python — generator exhaustion with zip]

Answer: (B)

A generator can be consumed only once. `zip` pulls from its arguments **left to right** and stops when any argument is exhausted.

- Rounds 1-3: `zip` takes 0, 1, 4 from `g` and 'a', 'b', 'c'.
- Round 4: `zip` first takes **9 from g**, then finds "abc" exhausted and stops. The 9 is discarded.
- `list(g)` gets the remaining 16, 25.
- `sum(g)`: `g` is exhausted → 0.

Output: [(0, 'a'), (1, 'b'), (4, 'c')] [16, 25] 0.

- (A) and (C) assume `zip` stops *before* pulling the 4th element from `g` (it would, only if `g` were the shorter, second argument).
- (D) assumes generators restart.

Trap: with `zip("abc", g)` (string first) no element of `g` would be lost — argument order matters.

Q.16 [NAT · 2 marks · Dijkstra — counting successful relaxations]

Answer: 9

Every vertex other than `S` gets exactly one 'first' successful relaxation (from ∞); the rest are improvements.

- Extract `S` (0): `A` = 4 ✓, `B` = 1 ✓, `C` = 7 ✓ (3 successful)
- Extract `B` (1): `A` = $\min(4, 3) = 3$ ✓; `D` = 7 ✓; `E` = 13 ✓ (3)
- Extract `A` (3): `C` = $\min(7, 5) = 5$ ✓; `D`: $3 + 5 = 8 > 7$ ✗ (1)
- Extract `C` (5): `D` = $\min(7, 6) = 6$ ✓; `E`: $5 + 8 = 13$, not < 13 ✗ (1)
- Extract `D` (6): `E` = $\min(13, 9) = 9$ ✓ (1)
- Extract `E` (9): no out-edges.

Total = 3 + 3 + 1 + 1 + 1 = **9**. Final distances: `A` 3, `B` 1, `C` 5, `D` 6, `E` 9.

Trap: `C` → `E` gives 13, which *equals* the current value — a tie is not a successful relaxation.

d[.] after each extraction (✓ = changed)

	S	A	B	C	D	E
S	0	4 ✓	1 ✓	7 ✓	∞	∞
B	0	3 ✓	1	7	7 ✓	13 ✓
A	0	3	1	5 ✓	7	13
C	0	3	1	5	6 ✓	13
D	0	3	1	5	6	9 ✓

Q.17 [MSQ · 2 marks · Topological sorting]**Answer: (A), (C)**

An ordering is topological iff for **every** edge $u \rightarrow v$, u appears before v . Check each option against the 8 edges $A \rightarrow C$, $A \rightarrow D$, $B \rightarrow D$, $B \rightarrow E$, $C \rightarrow F$, $D \rightarrow F$, $D \rightarrow G$, $E \rightarrow G$.

- (A) A, B, D, C, E, G, F: all hold (E before G, C and D before F). **Valid.**
- (B) E comes before B, violating $B \rightarrow E$. **Invalid.**
- (C) B, E, A, D, C, G, F: $B < D$, $B < E$, $A < C$, $A < D$, $C < F$, $D < F$, $D < G$, $E < G$ all hold. **Valid.**
- (D) F comes before D, violating $D \rightarrow F$. **Invalid.**

(In total this DAG has 42 topological orderings.)

Tip: scan each option once, maintaining the set of already-placed vertices, and verify that all in-neighbours of the current vertex are already placed.

Q.18 [NAT · 2 marks · Hashing — double hashing]**Answer: 13**

Double hashing uses a key-dependent step $h_2(k)$, so keys colliding at the same home slot follow **different** probe sequences.

- 27: $h_1 = 1 \rightarrow$ slot 1 (1 probe)
- 40: $h_1 = 1$ (full), $h_2 = 1 + 7 = 8 \rightarrow$ slot 9 (2 probes)
- 14: $h_1 = 1$ (full), $h_2 = 1 + 3 = 4 \rightarrow$ slot 5 (2)
- 53: $h_1 = 1$ (full), $h_2 = 1 + 9 = 10 \rightarrow$ slot 11 (2)
- 66: $h_1 = 1$ (full), $h_2 = 1 + 0 = 1 \rightarrow$ slot 2 (2)
- 79: $h_1 = 1$ (full), $h_2 = 1 + 2 = 3 \rightarrow$ slot 4 (2)
- 1: $h_1 = 1$ (full), $h_2 = 1 + 1 = 2 \rightarrow$ slot 3 (2)

Total = $1 + 6 \times 2 = 13$ probes.

Trap: all seven keys are $\equiv 1 \pmod{13}$. With linear probing they would form one long cluster costing $1 + 2 + \dots + 7 = 28$ probes; double hashing spreads them out.

0	
1	27
2	66
3	1
4	79
5	14
6	
7	
8	
9	40
10	
11	53
12	

Final table

Q.19 [MCQ · 2 marks · Bottom-up merge sort — intermediate state]**Answer: (C)**

Bottom-up merge sort merges runs of width 1, 2, 4, ... across the whole array in each pass.

- Pass 1 (pairs): [38,16]→[16,38]; [27,39]→[27,39]; [12,27]→[12,27]; [5,44]→[5,44]. Array: [16, 38, 27, 39, 12, 27, 5, 44].
- Pass 2 (merge runs of 2): [16,38]+[27,39] → [16, 27, 38, 39]; [12,27]+[5,44] → [5, 12, 27, 44]. Array: **[16, 27, 38, 39, 5, 12, 27, 44]**.
- Pass 3 would give the sorted array.
- (A) is after pass 3.
- (B) is the array after pass 1.
- (D) merges only the left half in pass 2 — a top-down order of work, not bottom-up.

Tip: after pass p , every aligned block of length 2^p is sorted.

Array after each pass

	0	1	2	3	4	5	6	7
start	38	16	27	39	12	27	5	44
pass 1	16	38	27	39	12	27	5	44
pass 2	16	27	38	39	5	12	27	44
pass 3	5	12	16	27	27	38	39	44

Q.20 [MSQ · 2 marks · Python — class versus instance attributes]**Answer: (B), (C), (D)**

Attribute **lookup** on an instance falls back to the class; attribute **assignment** on an instance always creates/updates an instance attribute. Mutating a class-level list through an instance changes the shared list.

- Two constructions: `Counter.total = 2`; `self.tags.append` mutates the **shared** class list → `Counter.tags = ['x', 'y']`.

- `a.bump(): self.total += 1` reads 2 (class), then assigns `a.total = 3` (new instance attribute); second bump → `a.total = 4`. `Counter.total` stays 2.
- `b.total` has no instance attribute → class value 2.
- `b.tags = ['z']` creates an instance attribute on b only.

Output: 2 4 2 2 2 ['z'].

- (A) `a.tags` is the shared class list of length 2. **False.**
- (B) **True** — rebinding `b.tags` does not touch `Counter.tags`.
- (D) **True.** (C) **True.**

Trap: `self.x += 1` on an immutable class attribute *shadows* it, while `self.lst.append()` on a mutable class attribute *shares* it.

Mock Test 37

Full-Syllabus Mock — Paper 7

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — floor division, modulo, rounding]

Consider the following Python program. What is printed?

Python

```
1 print(-17 // 5, -17 % 5, 17 // -5, round(-2.5))
```

- (A) -3 -2 -3 -3
- (B) -4 3 -4 -2
- (C) -4 -2 -4 -3
- (D) -3 3 -4 -2

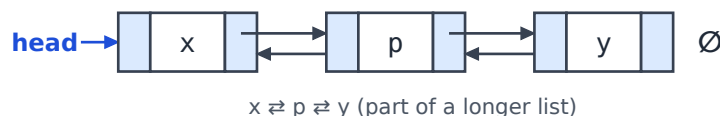
Q.2 [NAT · numerical answer | 1 mark | Stacks — infix to postfix]

The infix expression $a - b * (c + d / e ^ f) * g + h$ is converted to postfix with the standard operator-stack algorithm. Precedence: $^$ (highest, right-associative), then $*$ /, then $+$ - (left-associative). Left parentheses are pushed onto the same stack. The maximum number of symbols (operators and parentheses) on the stack at any moment is ____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Doubly linked lists — deletion]

In a doubly linked list, node p is neither the first nor the last node. Each node has fields `prev` and `next`. Which one of the following statement sequences correctly removes p from the list (so that forward and backward traversals both skip p)?



- (A) `p.next.prev = p.prev; p.prev.next = p.next.prev`
- (B) `p.prev.next = p.next; p.next.prev = p.prev.next`
- (C) `p.prev.next = p.next; p.next.prev = p.prev`
- (D) `p.next = p.next.next; p.next.prev = p`

Q.4 [NAT · numerical answer | 1 mark | Hashing — double hashing]

Keys 18, 41, 44, 31, 57, 83 are inserted in this order into an empty table of size 13 using double hashing: the i -th probe ($i = 0, 1, 2, \dots$) for key k is $(h_1(k) + i \cdot h_2(k)) \bmod 13$, with $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. The index of the slot in which 83 is stored is ____.

Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | BST search paths]

A binary search tree contains distinct integer keys, including 55. Which of the following could be the sequence of keys examined, in order, during a search for 55?

- (A) 90, 20, 70, 30, 60, 50, 55
- (B) 10, 80, 35, 75, 40, 70, 55
- (C) 52, 70, 54, 65, 53, 55
- (D) 60, 25, 58, 40, 62, 55

Q.6 [MCQ · single correct | 1 mark | Selection sort is not stable]

Consider the following Python program, which sorts records by their numeric key. What is printed?

Python

```
1 recs = [(3, 'a'), (1, 'b'), (3, 'c'), (2, 'd'), (1, 'e')]
2 for i in range(len(recs) - 1):
3     m = i
4     for j in range(i + 1, len(recs)):
5         if recs[j][0] < recs[m][0]:
6             m = j
7     recs[i], recs[m] = recs[m], recs[i]
8 print(''.join(r[1] for r in recs))
```

- (A) bedca
- (B) bedac
- (C) ebdac
- (D) ebdca

Q.7 [NAT · numerical answer | 1 mark | Recurrences — exact value]

Let $T(1) = 1$ and $T(n) = 3 \cdot T(n/3) + n$ for n a power of 3 greater than 1. The value of $T(81)$ is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Graph theory — edges vs components]

The maximum number of edges in a simple undirected graph with 10 vertices and exactly 3 connected components is

- (A) 21
- (B) 27
- (C) 28
- (D) 36

Q.9 [NAT · numerical answer | 1 mark | Python — dict accumulation with enumerate]

The value printed by the following Python program is _____.

Python

```
1 s = "abracadabra"
2 d = {}
3 for i, c in enumerate(s):
4     d[c] = d.get(c, 0) + i
5 print(d['a'] - d['b'])
```

Answer: _____ (enter a numerical value)

Q.10 [MSQ · one or more correct | 1 mark | Queue using two stacks]

A queue is implemented with two stacks S1 and S2. Enqueue pushes onto S1. Dequeue pops from S2; if S2 is empty it first pops **every** element of S1 and pushes it onto S2. Starting empty, the operations are: enqueue 1, 2, 3, 4; dequeue; enqueue 5, 6; dequeue; enqueue 7. Which of the following statements is/are TRUE afterwards?

- (A) S1 holds 5, 6, 7 with 7 on top
- (B) The next dequeue will trigger a transfer from S1 to S2
- (C) Exactly 4 element transfers from S1 to S2 have taken place so far
- (D) S2 holds exactly 2 elements and 3 is on its top

Q.11 [MCQ · single correct | 2 marks | Python — closures with nonlocal]

Consider the following Python program. What is printed?

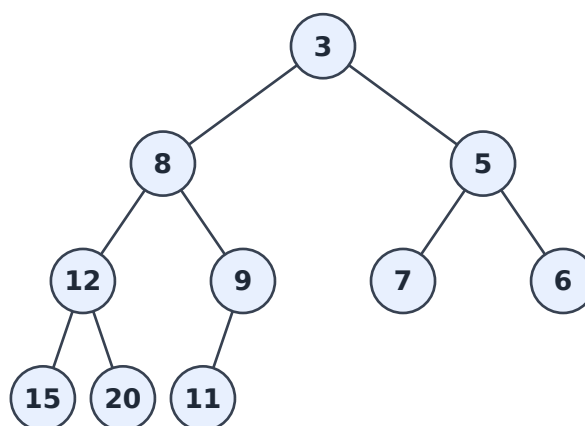
Python

```
1 def make():
2     total = 0
3     def add(x):
4         nonlocal total
5         total += x
6         return total
7     return add
8
9 a = make()
10 b = make()
11 print(a(3), a(4), b(5), a(b(1)))
```

- (A) 3 7 12 26
- (B) 3 7 5 13
- (C) 3 7 5 8
- (D) 3 4 5 7

Q.12 [NAT · numerical answer | 2 marks | Min-heap insert and extract-min]

The binary min-heap shown is stored (0-indexed) as [3, 8, 5, 12, 9, 7, 6, 15, 20, 11]. The following operations are performed in order: insert 4; extract-min; extract-min. Insertion appends and sifts up; extract-min moves the last element to the root and sifts it down, exchanging with the smaller child. After these operations, the key at index 6 of the array is ____.

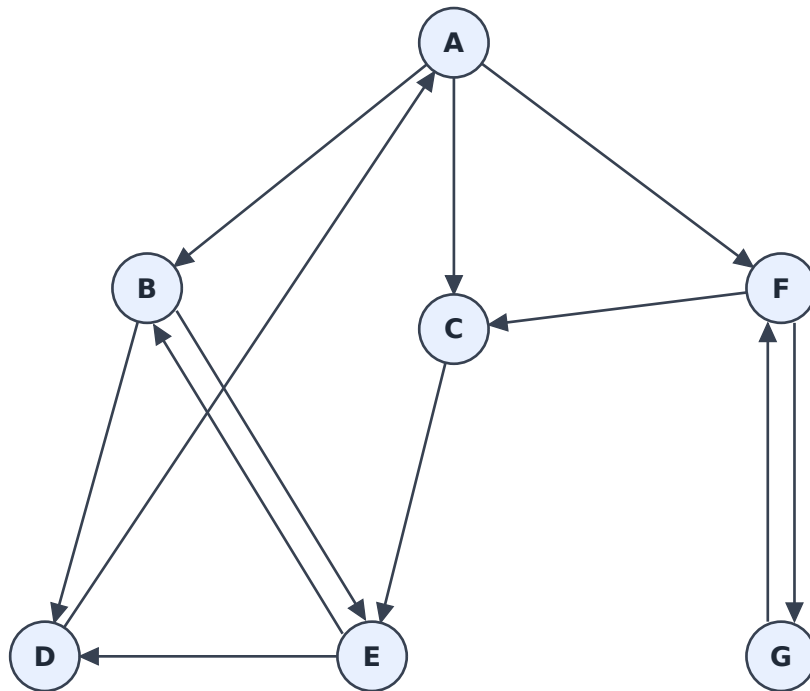


Initial min-heap

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | DFS edge classification]

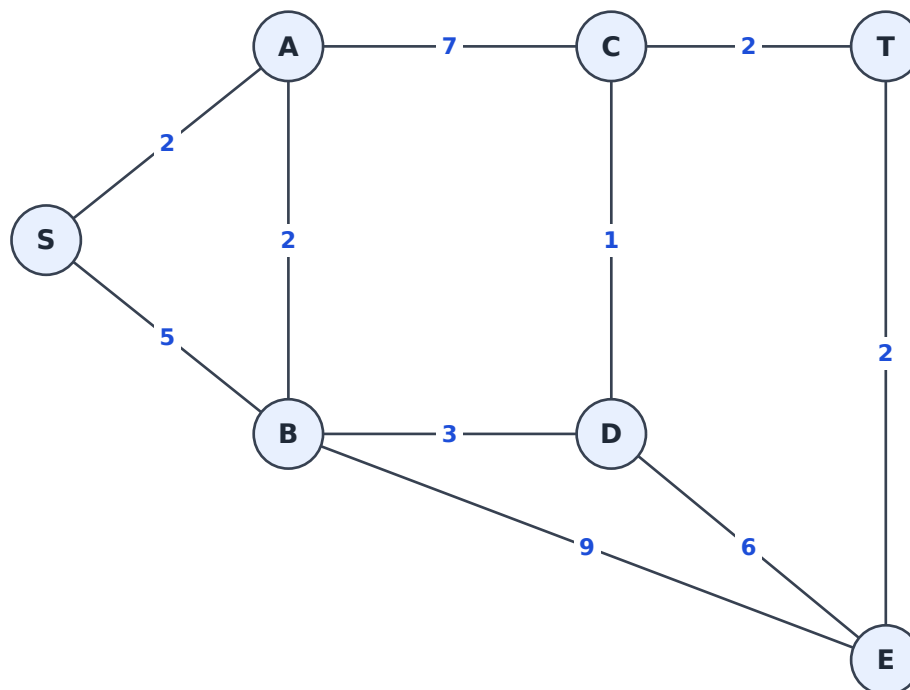
DFS is run on the directed graph shown, starting at A; neighbours are explored in alphabetical order, and if unvisited vertices remained a new DFS would start at the alphabetically smallest one. Which of the following statements is/are TRUE?



- (A) $E \rightarrow D$ is a back edge
- (B) Exactly 3 edges are back edges
- (C) The DFS has at least one forward edge
- (D) $F \rightarrow C$ is a cross edge

Q.14 [NAT · numerical answer | 2 marks | Dijkstra — undirected weighted graph]

Dijkstra's algorithm is run from S on the undirected weighted graph shown. The shortest-path distance from S to E is _____.



Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Quicksort — Hoare partition]

Hoare's partition (pivot $p = A[lo]$) is applied once to

$A = [15, 22, 7, 31, 15, 4, 18, 10, 26, 3]$ with $lo = 0$, $hi = 9$:

$i = lo - 1$, $j = hi + 1$; repeat { decrement j until $A[j] \leq p$; increment i until $A[i] \geq p$; if $i < j$ swap $A[i]$, $A[j]$ else return j }.

Which option gives the array after the call, and the returned index?

- (A) $[3, 10, 7, 15, 4, 31, 18, 22, 26, 15]$, returns 3
- (B) $[3, 10, 7, 4, 15, 31, 18, 22, 26, 15]$, returns 5
- (C) $[10, 3, 7, 4, 15, 15, 18, 22, 26, 31]$, returns 4
- (D) $[3, 10, 7, 4, 15, 31, 18, 22, 26, 15]$, returns 4

Q.16 [NAT · numerical answer | 2 marks | Binary search — iterations per key]

The sorted array $A[0..49]$ holds 50 distinct keys. The following search is run once for **each** of the 50 keys:

$lo = 0$, $hi = 49$; while $lo \leq hi$: $mid = \lfloor (lo + hi)/2 \rfloor$; if $A[mid] = x$ return; elif $A[mid] < x$: $lo = mid + 1$ else $hi = mid - 1$.

The number of keys for which the loop body executes exactly 6 times is _____.

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Tree reconstruction from traversals]

A binary tree with distinct labels has

pre-order: M D B A H F K L T W U

in-order: A B D F H K L M T U W

Which of the following statements is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)

- (A) It has exactly 4 leaves
- (B) Its post-order is A B F L K H D U W T M
- (C) Exactly 3 nodes have exactly one child
- (D) Its height is 4

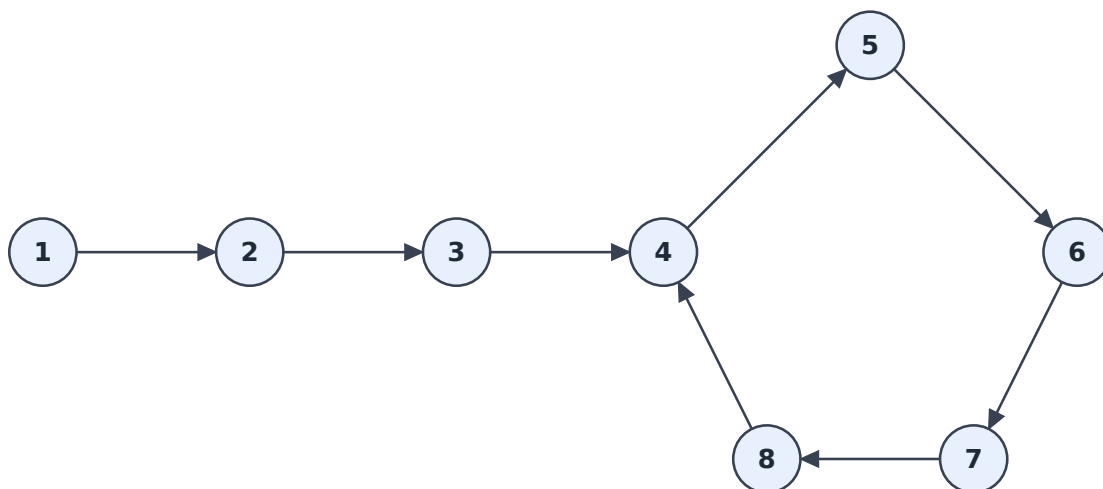
Q.18 [NAT · numerical answer | 2 marks | Insertion sort — comparisons]

Insertion sort (inner loop: `while j >= 0 and A[j] > key`) sorts `[5, 9, 2, 7, 3, 8, 1]` in ascending order. A *comparison* is one evaluation of `A[j] > key` (the test `j >= 0` is not counted, and when it is false `A[j] > key` is not evaluated). The total number of comparisons is ____.

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — Floyd's cycle detection]

A singly linked list has nodes `1 → 2 → 3 → 4 → 5 → 6 → 7 → 8`, and node 8's `next` points back to node 4. Floyd's algorithm starts with `slow = fast = head` (node 1) and repeats `slow = slow.next; fast = fast.next.next` until `slow is fast`. Where do they first meet, and after how many iterations?



- (A) At node 6 after 5 iterations
- (B) At node 4 after 4 iterations
- (C) At node 8 after 5 iterations
- (D) At node 5 after 6 iterations

Q.20 [MSQ · one or more correct | 2 marks | Solving recurrences]

Which of the following recurrences (with $T(1) = 1$) are solved correctly?

- (A) $T(n) = T(n - 1) + n \Rightarrow T(n) = \Theta(n^2)$
- (B) $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$
- (C) $T(n) = 3T(n/2) + n \Rightarrow T(n) = \Theta(n^{\log_2 3})$
- (D) $T(n) = 2T(n/2) + n \log n \Rightarrow T(n) = \Theta(n \log^2 n)$

Answer Key — Mock Test 37

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — floor division, modulo, rounding
2	NAT	1	6	Stacks — infix to postfix
3	MCQ	1	(C)	Doubly linked lists — deletion
4	NAT	1	0	Hashing — double hashing
5	MSQ	1	(A), (B)	BST search paths
6	MCQ	1	(A)	Selection sort is not stable
7	NAT	1	405	Recurrences — exact value
8	MCQ	1	(C)	Graph theory — edges vs components
9	NAT	1	16	Python — dict accumulation with enumerate
10	MSQ	1	(A), (C), (D)	Queue using two stacks
11	MCQ	2	(B)	Python — closures with nonlocal
12	NAT	2	11	Min-heap insert and extract-min
13	MSQ	2	(B), (D)	DFS edge classification
14	NAT	2	12	Dijkstra — undirected weighted graph
15	MCQ	2	(D)	Quicksort — Hoare partition
16	NAT	2	19	Binary search — iterations per key
17	MSQ	2	(A), (B), (D)	Tree reconstruction from traversals
18	NAT	2	17	Insertion sort — comparisons
19	MCQ	2	(A)	Linked lists — Floyd's cycle detection
20	MSQ	2	(A), (C), (D)	Solving recurrences

Detailed Solutions — Mock Test 37

Q.1 [MCQ · 1 mark · Python — floor division, modulo, rounding]

Answer: (B)

Python's `//` is **floor** division (rounds toward $-\infty$), and `%` is defined so that $a == (a // b) * b + a \% b$, which makes the remainder take the sign of the **divisor**. `round` uses round-half-to-even (banker's rounding).

- $-17 / 5 = -3.4 \rightarrow \text{floor} \rightarrow -4$.
- $-17 \% 5 = -17 - (-4)(5) = -17 + 20 = 3$.
- $17 / -5 = -3.4 \rightarrow \text{floor} \rightarrow -4$.
- `round(-2.5)`: halfway between -3 and $-2 \rightarrow$ the even one, -2 .

Output: `-4 3 -4 -2`.

- (A) truncates toward zero (C/Java semantics) and rounds half away from zero.
- (C) gets floor right but uses a C-style negative remainder and the wrong rounding.
- (D) truncates in the first expression only — inconsistent.

Trap: in C, $-17 / 5 = -3$ and $-17 \% 5 = -2$; Python differs on both.

Q.2 [NAT · 1 mark · Stacks — infix to postfix]

Answer: 6

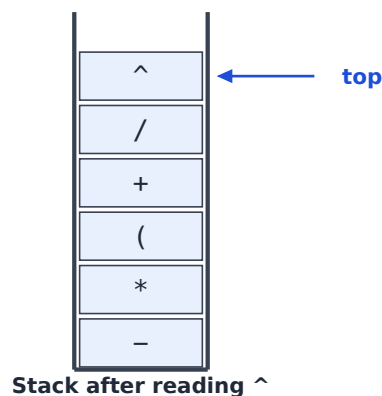
Rules: operands go straight to the output; `(` is pushed; `)` pops to the matching `(`; an operator first pops operators of higher precedence (or equal precedence if left-associative), then is pushed.

Stack after each token (bottom $\rightarrow$ top):

- `-` $\rightarrow$ `[-]`
- `*` $\rightarrow$ `[-, *]` (higher than `-`, no pop)
- `(` $\rightarrow$ `[-, *, (]`
- `+` $\rightarrow$ `[-, *, (, +]`
- `/` $\rightarrow$ `[-, *, (, +, /]`
- `^` $\rightarrow$ `[-, *, (, +, /, ^]` $\leftarrow$ **6 symbols**
- `)` $\rightarrow$ pops `^`, `/`, `+` and the `(` $\rightarrow$ `[-, *]`
- `*` $\rightarrow$ pops the equal-precedence `*` (left-assoc), pushes `*` $\rightarrow$ `[-, *]`
- `+` $\rightarrow$ pops `*`, then `-` $\rightarrow$ `[+]`
- end $\rightarrow$ pop `+`

Maximum = **6**. Postfix: `a b c d e f ^ / + * g * - h +`.

Trap: forgetting to count the `(`, which occupies a stack slot (answer 5).



Q.3 [MCQ · 1 mark · Doubly linked lists — deletion]**Answer: (C)**

Removing p requires exactly two pointer changes: $x.next$ must become y , and $y.prev$ must become x (where $x = p.prev$ and $y = p.next$). p 's own fields can stay as they are.

- (A) After the first statement, $p.next.prev$ is x , so the second sets $x.next = x$ — forward traversal loops at x .
- (B) After the first statement, $p.prev.next$ is y , so the second sets $y.prev = y$ — a self-loop; backward traversal breaks.
- (C) $x.next = y$; $y.prev = x$. **Correct.** (The two statements are independent, so their order does not matter.)
- (D) Deletes y (the node *after* p), not p .

Trap: an expression like $p.prev.next$ must be re-evaluated *after* earlier assignments — chained pointer expressions change meaning once a link is redirected.

Q.4 [NAT · 1 mark · Hashing — double hashing]**Answer: 0**

Unlike linear/quadratic probing, keys with the same home slot get *different* step sizes h_2 , so they follow different probe sequences.

- 18: $h_1 = 5 \rightarrow$ slot 5.
- 41: $h_1 = 2 \rightarrow$ slot 2.
- 44: $h_1 = 5$ (full), $h_2 = 1 + 0 = 1 \rightarrow 6 \rightarrow$ slot 6.
- 31: $h_1 = 5$, $h_2 = 1 + 9 = 10 \rightarrow 15 \bmod 13 = 2$ (full) $\rightarrow 25 \bmod 13 = 12 \rightarrow$ slot 12.
- 57: $h_1 = 5$, $h_2 = 1 + 2 = 3 \rightarrow 8 \rightarrow$ slot 8.
- 83: $h_1 = 83 \bmod 13 = 5$, $h_2 = 1 + (83 \bmod 11) = 1 + 6 = 7$. Probes: 5 (full), 12 (full, 31), $19 \bmod 13 = 6$ (full, 44), $26 \bmod 13 = 0$ (empty) $\rightarrow$ slot 0.

Trap: computing the probe as $(\text{previous slot} + i \cdot h_2)$ instead of $(h_1 + i \cdot h_2)$, or using $h_2 = k \bmod 11$ without the $+1$ (which could be 0 and loop forever).

0	83
1	
2	41
3	
4	
5	18
6	44
7	
8	57
9	
10	
11	
12	31

Final table

Q.5 [MSQ · 1 mark · BST search paths]**Answer: (A), (B)**

Each examined key narrows the interval in which all later keys must lie: going left after key k sets the upper bound to k , going right sets the lower bound to k . A sequence is valid iff every key lies inside the current interval.

- (A) $(-\infty, \infty) \rightarrow 90 \text{ L } (-\infty, 90) \rightarrow 20 \text{ R } (20, 90) \rightarrow 70 \text{ L } (20, 70) \rightarrow 30 \text{ R } (30, 70) \rightarrow 60 \text{ L } (30, 60) \rightarrow 50 \text{ R } (50, 60) \rightarrow 55 \checkmark$. **Possible.**
- (B) $10 \text{ R } (10, \infty) \rightarrow 80 \text{ L } (10, 80) \rightarrow 35 \text{ R } (35, 80) \rightarrow 75 \text{ L } (35, 75) \rightarrow 40 \text{ R } (40, 75) \rightarrow 70 \text{ L } (40, 70) \rightarrow 55 \checkmark$. **Possible.**
- (C) $52 \text{ R } (52, \infty) \rightarrow 70 \text{ L } (52, 70) \rightarrow 54 \text{ R } (54, 70) \rightarrow 65 \text{ L } (54, 65) \rightarrow 53 \notin (54, 65)$. **Impossible.**
- (D) $60 \text{ L } (-\infty, 60) \rightarrow 25 \text{ R } (25, 60) \rightarrow 58 \text{ L } (25, 58) \rightarrow 40 \text{ R } (40, 58) \rightarrow 62 \notin (40, 58)$. **Impossible** — 62 cannot lie in the left subtree of 60.

Trap: checking only consecutive pairs (each step “moves toward 55”) is not enough — all earlier bounds still apply.

Q.6 [MCQ · 1 mark · Selection sort is not stable]**Answer: (A)**

Selection sort swaps the minimum into position i ; the long-distance swap can jump an element over another with an equal key, so the algorithm is **not stable**.

- $i = 0$: minimum key 1 first found at index 1 ('b') $\rightarrow$ swap with (3, 'a') $\rightarrow$ b, a, c, d, e (keys 1, 3, 3, 2, 1)
- $i = 1$: minimum key 1 at index 4 ('e') $\rightarrow$ swap with (3, 'a') $\rightarrow$ b, e, c, d, a
- $i = 2$: minimum key 2 at index 3 ('d') $\rightarrow$ swap with (3, 'c') $\rightarrow$ b, e, d, c, a
- $i = 3$: (3, 'c') vs (3, 'a') — strict $<$ finds no smaller $\rightarrow$ no change

Output: **bedca**. The two key-3 records ended in the order c, a — reversed relative to the input.

- (B) **bedac** is the **stable** order (what `sorted(recs, key=lambda r: r[0])` gives).
- (C) **ebdac** is what the code would print with `<=` (choosing the *last* minimum).
- (D) **ebdca** mixes the two rules and is produced by neither version.

Trap: strict $<$ keeps the *first* minimum, which preserves order among the minima, but the element displaced by the swap ('a') can still leap past its equal ('c').

Q.7 [NAT · 1 mark · Recurrences — exact value]**Answer: 405**

Unroll: each level of the recursion tree does total work n (3^i subproblems of size $n/3^i$), and there are $\log_3 n$ levels of this kind plus the n leaves costing $T(1) = 1$ each.

$$T(n) = n \cdot \log_3 n + n \cdot T(1) = n(\log_3 n + 1).$$

$$\text{For } n = 81 = 3^4: T(81) = 81 \times (4 + 1) = \mathbf{405}.$$

Step-by-step check:

- $T(3) = 3 \cdot 1 + 3 = 6$
- $T(9) = 3 \cdot 6 + 9 = 27$
- $T(27) = 3 \cdot 27 + 27 = 108$
- $T(81) = 3 \cdot 108 + 81 = 405$

Asymptotically $T(n) = \Theta(n \log n)$ (Master theorem case 2: $a = b = 3$, $f(n) = n = n^{\log_3 3}$).

Trap: forgetting the leaf level — $n \cdot \log_3 n$ alone gives 324.

Q.8 [MCQ · 1 mark · Graph theory — edges vs components]**Answer: (C)**

For a fixed number of components, edges are maximised by making each component complete. With component sizes $n_1 + n_2 + n_3 = 10$ the edge count is $\sum C(n_i, 2)$, and because $C(x, 2)$ is convex, the sum is largest when the sizes are as **unbalanced** as possible: two isolated vertices and one K_8 .

- Sizes (8, 1, 1): $C(8, 2) = \mathbf{28}$
- Sizes (7, 2, 1): $21 + 1 = 22$
- Sizes (4, 3, 3): $6 + 3 + 3 = 12$

General formula: $C(n - k + 1, 2) = C(8, 2) = 28$ for $n = 10, k = 3$.

- (A) $21 = C(7, 2)$ uses a K_7 (off by one).
- (B) 27 is not achievable by any split.
- (D) $36 = C(9, 2)$ is the maximum with only **2** components.

Trap: spreading vertices evenly *minimises* the edge count among complete components.

Q.9 [NAT · 1 mark · Python — dict accumulation with enumerate]**Answer: 16**

`enumerate` yields (index, character) pairs starting at 0, and `d.get(c, 0)` returns 0 for a new key, so `d[c]` ends up as the **sum of the indices** at which `c` occurs.

Indices: a b r a c a d a b r a → 0 1 2 3 4 5 6 7 8 9 10.

- 'a' occurs at 0, 3, 5, 7, 10 → sum 25.
- 'b' occurs at 1, 8 → sum 9.

Printed value = $25 - 9 = \mathbf{16}$.

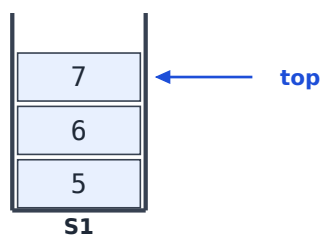
Trap: counting occurrences ($5 - 2 = 3$) instead of summing indices, or indexing from 1 (which would give $30 - 11 = 19$).

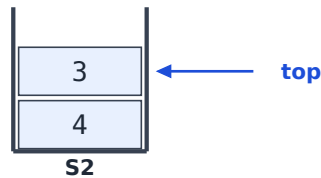
Q.10 [MSQ · 1 mark · Queue using two stacks]**Answer: (A), (C), (D)**

Trace (stacks written bottom → top):

- enqueue 1..4 → $S1 = [1, 2, 3, 4]$, $S2 = []$
- dequeue: $S2$ empty → transfer 4 elements → $S2 = [4, 3, 2, 1]$; pop → returns 1; $S2 = [4, 3, 2]$
- enqueue 5, 6 → $S1 = [5, 6]$
- dequeue: $S2$ non-empty → pop → returns 2; $S2 = [4, 3]$
- enqueue 7 → $S1 = [5, 6, 7]$
- (A) **True**.
- (B) **False** — $S2$ is non-empty, so the next dequeue just pops 3.
- (C) **True** — only the first dequeue transferred, moving 4 elements.
- (D) **True** — $S2 = [4, 3]$, top 3 (the next element to leave the queue).

Tip: each element is pushed and popped at most twice in total, so any sequence of n operations costs $O(n)$ — amortised $O(1)$ per operation, although one dequeue can cost $\Theta(n)$.



**Q.11 [MCQ · 2 marks · Python — closures with nonlocal]****Answer: (B)**

Each call to `make()` creates a **new** local variable `total` and a new closure `add` that captures it. `nonlocal` lets `add` rebind that captured variable, so each closure keeps its own running sum.

Arguments are evaluated left to right:

- `a(3)`: a's total 0 → 3 → prints 3
- `a(4)`: a's total 3 → 7 → prints 7
- `b(5)`: b's total (separate) 0 → 5 → prints 5
- `a(b(1))`: first `b(1)`: 5 → 6; then `a(6)`: 7 → 13 → prints 13

Output: 3 7 5 13.

- (A) assumes a and b share one `total` (as if it were a global).
- (C) adds only `b(1) = 1` inside a (forgets b's accumulated 5).
- (D) assumes `total` resets on every call (no accumulation).

Trap: without `nonlocal`, `total += x` would raise `UnboundLocalError`, because the assignment makes `total` local to `add`.

Q.12 [NAT · 2 marks · Min-heap insert and extract-min]**Answer: 11**

Insert 4 at index 10. Parent index 4 holds $9 > 4$ → swap; parent index 1 holds $8 > 4$ → swap; parent index 0 holds $3 < 4$ → stop.

[3, 4, 5, 12, 8, 7, 6, 15, 20, 11, 9]

Extract-min #1 (removes 3): last key 9 moves to the root → [9, 4, 5, 12, 8, 7, 6, 15, 20, 11].

- 9 vs children 4, 5 → swap with 4 (index 1)
- 9 vs children 12, 8 → swap with 8 (index 4)
- 9 vs child 11 (index 9) → stop.

[4, 8, 5, 12, 9, 7, 6, 15, 20, 11]

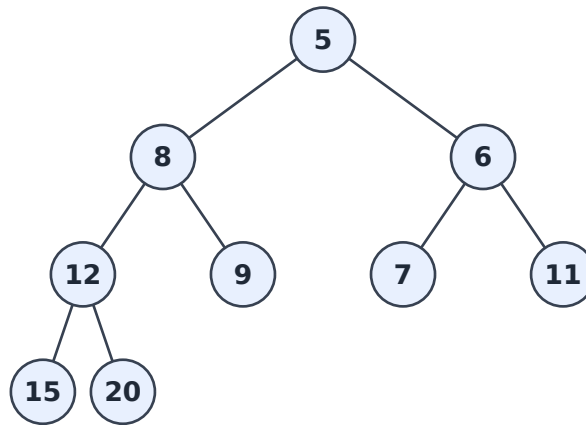
Extract-min #2 (removes 4): last key 11 to the root → [11, 8, 5, 12, 9, 7, 6, 15, 20].

- 11 vs 8, 5 → swap with 5 (index 2)
- 11 vs 7, 6 → swap with 6 (index 6)
- index 6 has no children (13, 14 out of range) → stop.

[5, 8, 6, 12, 9, 7, 11, 15, 20]

Key at index 6 = **11**.

Trap: in the second extraction, exchanging 11 with the *left* child 8 instead of the smaller child 5 would put 8 above 5 — not a heap. Always pick the smaller child in a min-heap.



Final heap

Q.13 [MSQ · 2 marks · DFS edge classification]**Answer: (B), (D)**

Classify edge $u \rightarrow v$ using discovery/finish times: **tree** (v first discovered via it), **back** (v is an ancestor still active), **forward** (v is a finished descendant), **cross** (v finished, not a descendant).

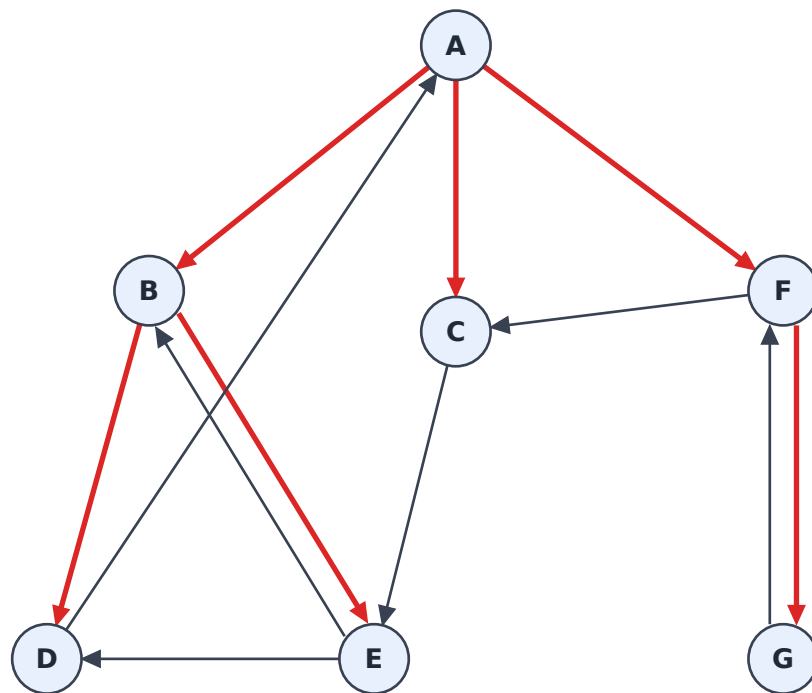
Trace (time: discover/finish):

- A 1 $\rightarrow$ B 2 $\rightarrow$ D 3: D $\rightarrow$ A is to an active ancestor (back). D finishes 4.
- B $\rightarrow$ E: E 5. E $\rightarrow$ B back (B active); E $\rightarrow$ D: D finished, not a descendant of E (cross). E finishes 6.
- B finishes 7. A $\rightarrow$ C: C 8; C $\rightarrow$ E: E finished in B's subtree (cross). C finishes 9.
- A $\rightarrow$ F: F 10; F $\rightarrow$ C: finished, not a descendant (cross); F $\rightarrow$ G: G 11; G $\rightarrow$ F back. G 12, F 13, A 14.

Tree: A $\rightarrow$ B, B $\rightarrow$ D, B $\rightarrow$ E, A $\rightarrow$ C, A $\rightarrow$ F, F $\rightarrow$ G (6). Back: D $\rightarrow$ A, E $\rightarrow$ B, G $\rightarrow$ F (3). Cross: E $\rightarrow$ D, C $\rightarrow$ E, F $\rightarrow$ C (3). Forward: none.

- (A) **False** — D had already finished, so E $\rightarrow$ D is a cross edge.
- (B) **True.**
- (C) **False** — no edge goes from an ancestor to an already-finished descendant.
- (D) **True.**

Trap: calling E $\rightarrow$ D a back edge because D was discovered earlier; a back edge requires the target to be an *active ancestor* (discovered but not finished).



DFS tree edges highlighted

Q.14 [NAT · 2 marks · Dijkstra — undirected weighted graph]**Answer: 12**

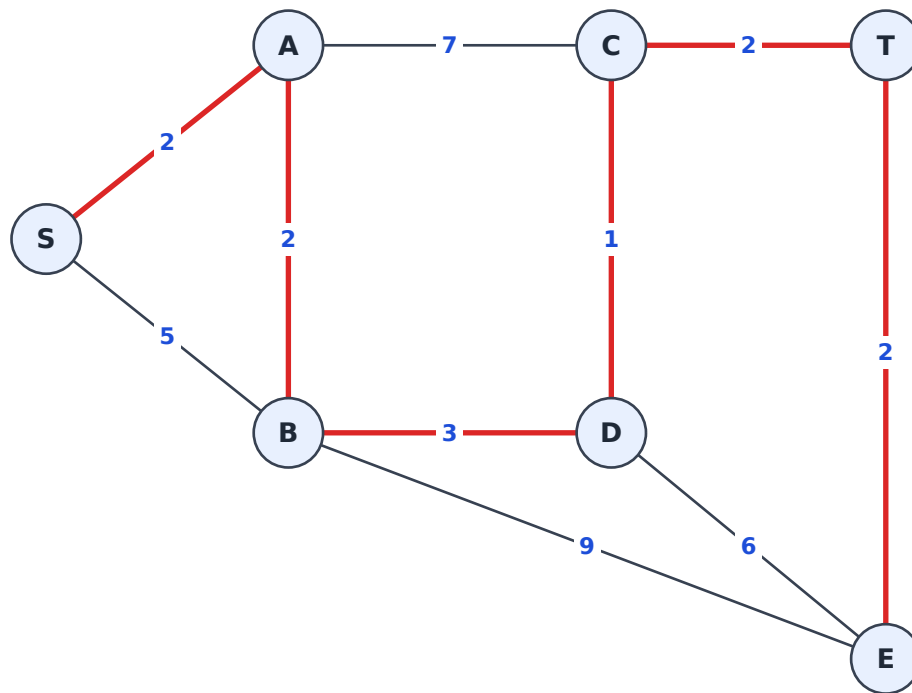
Extraction order with distances:

- S (0): A = 2, B = 5
- A (2): B = $\min(5, 4) = 4$; C = 9
- B (4): D = 7; E = $4 + 9 = 13$
- D (7): C = $\min(9, 8) = 8$; E = $\min(13, 7 + 6) = 13$
- C (8): T = 10
- T (10): E = $\min(13, 10 + 2) = \mathbf{12}$
- E (12)

Shortest path $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow T \rightarrow E$ with cost $2 + 2 + 3 + 1 + 2 + 2 = \mathbf{12}$.

Both “direct” routes into E (via B: 13, via D: 13) are beaten by the six-edge detour through T — the number of edges is irrelevant, only total weight counts.

Trap: stopping as soon as E receives a finite label (13). A label is final only when the vertex is **extracted**.



Shortest-path tree

Q.15 [MCQ · 2 marks · Quicksort — Hoare partition]**Answer: (D)**

Hoare's scheme moves i right past keys $< p$ and j left past keys $> p$, swapping out-of-place pairs. Keys equal to the pivot stop **both** scans. Unlike Lomuto, the pivot need not end in its final position; the call guarantees only $A[\text{lo}..j] \leq p \leq A[j+1..hi]$.

$p = 15$.

- Round 1: j from 10 $\rightarrow$ 9 ($A[9] = 3 \leq 15$); i from $-1 \rightarrow 0$ ($A[0] = 15 \geq 15$). Swap $\rightarrow$ [3, 22, 7, 31, 15, 4, 18, 10, 26, 15]
- Round 2: $j \rightarrow 8$ (26), 7 ($10 \leq 15$) stop; $i \rightarrow 1$ ($22 \geq 15$) stop. Swap $\rightarrow$ [3, 10, 7, 31, 15, 4, 18, 22, 26, 15]
- Round 3: $j \rightarrow 6$ (18), 5 (4) stop; $i \rightarrow 2$ (7), 3 (31) stop. Swap $\rightarrow$ [3, 10, 7, 4, 15, 31, 18, 22, 26, 15]
- Round 4: $j \rightarrow 4$ ($15 \leq 15$) stop; $i \rightarrow 4$ ($15 \geq 15$) stop. $i = j = 4 \rightarrow$ **return 4**.
- (A) returns 3 by stopping j at the 4 rather than at 15.
- (B) has the right array but returns $i + 1$ (Lomuto-style pivot index).
- (C) is not produced by any of the swaps above.
- (D) Correct: left part $A[0..4] = \{3, 10, 7, 4, 15\} \leq 15$, right part ≥ 15 .

Trap: expecting the pivot 15 to be at index j and the second 15 to be on the left — with Hoare, equal keys can end up on either side.

Q.16 [NAT · 2 marks · Binary search — iterations per key]**Answer: 19**

The sequence of mids forms an implicit **decision tree** (a BST over the indices): the root is index 24, its children are the mids of $[0..23]$ and $[25..49]$, and so on. A key found at depth d (root depth 0) needs $d + 1$ iterations.

Count nodes level by level. A balanced split of n elements puts $\lfloor (n-1)/2 \rfloor$ on the left and $\lceil (n-1)/2 \rceil$ on the right:

- Level 0 (1 iteration): 1 node (index 24); subranges sizes 24 and 25
- Level 1 (2): 2 nodes; subranges 11, 12, 12, 12
- Level 2 (3): 4 nodes; subranges 5, 5, 5, 6, 5, 6, 5, 6
- Level 3 (4): 8 nodes; remaining $50 - 15 = 35$ keys below
- Level 4 (5): 16 nodes
- Level 5 (6): the rest, $50 - (1 + 2 + 4 + 8 + 16) = 19$ nodes

Since levels 0–4 are full (31 nodes) and the tree has height $\lceil \log_2 50 \rceil = 5$, exactly 19 keys need 6 iterations. (The average successful search therefore costs $(1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 8 + 5 \cdot 16 + 6 \cdot 19)/50 = 243/50 = 4.86$ iterations.)

Trap: answering 32 (a full last level) — the last level of a 50-node tree is only partly filled.

Q.17 [MSQ · 2 marks · Tree reconstruction from traversals]

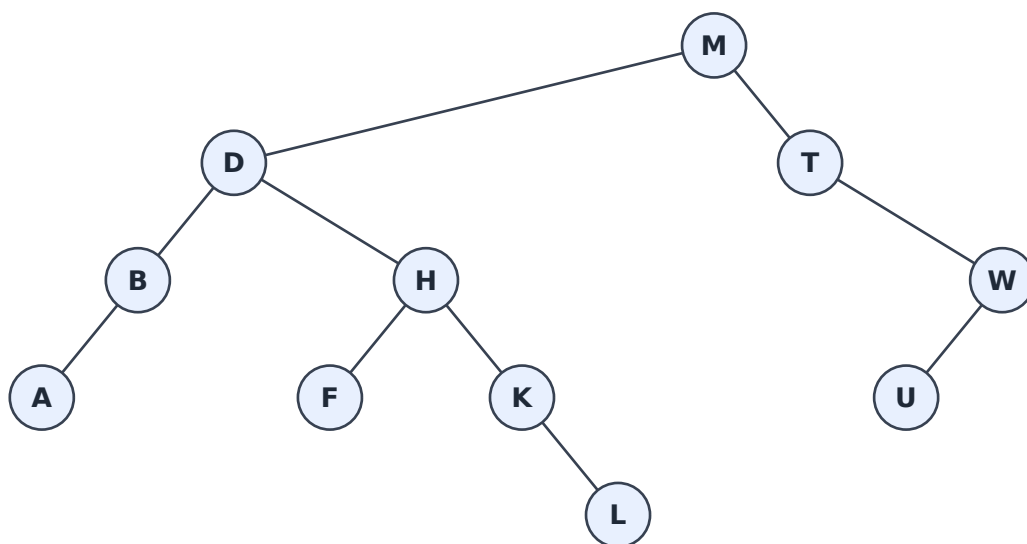
Answer: (A), (B), (D)

Pre-order gives the root first; its position in the in-order splits the remaining labels into left and right subtrees. Recurse.

- Root M. In-order left of M: A B D F H K L (7 nodes); right: T U W.
- Left subtree pre-order D B A H F K L → root D; in-order left {A, B}, right {F, H, K, L}.
- B with left child A. H with left child F and right child K; K has right child L.
- Right subtree pre-order T W U → root T; in-order T is first → no left child; right {U, W} with pre-order W U → W with left child U.
- (A) Leaves: A, F, L, U → 4 → **True**.
- (B) Post-order = A B | F L K H | D | U W T | M → **True**.
- (C) One-child nodes: B (left only), K (right only), T (right only), W (left only) → 4, not 3 → **False**.
- (D) Longest path M → D → H → K → L has 4 edges → **True**.

Check: in a binary tree, leaves = (nodes with two children) + 1. Two-child nodes are M, D, H (3), so leaves = 4 ✓.

Trap: pre-order + post-order would *not* determine a tree with one-child nodes; pre-order + in-order always does.



Reconstructed tree

Q.18 [NAT · 2 marks · Insertion sort — comparisons]**Answer: 17**

Every comparison that returns True causes a shift (removes one inversion). A comparison that returns False ends the insertion; that happens once per insertion **unless** the key travels all the way to index 0 (then the loop ends on $j \geq 0$).

So comparisons = inversions + (insertions that stop early).

Insertions ($i = 1..6$):

- 9: compare with 5 → False. 1 comparison, 0 shifts.
- 2: shifts 9, 5 → reaches front. 2 comparisons.
- 7: shifts 9; compare with 5 → False. 2 comparisons.
- 3: shifts 9, 7, 5; compare with 2 → False. 4 comparisons.
- 8: shifts 9; compare with 7 → False. 2 comparisons.
- 1: shifts 9, 8, 7, 5, 3, 2 → reaches front. 6 comparisons.

Total = 1 + 2 + 2 + 4 + 2 + 6 = **17** (= 13 inversions + 4 early stops).

Trap: answering 13 (shifts only) or 21 (= $n(n-1)/2$, the worst case).

Array after each insertion							
	0	1	2	3	4	5	6
start	5	9	2	7	3	8	1
i=1	5	9	2	7	3	8	1
i=2	2	5	9	7	3	8	1
i=3	2	5	7	9	3	8	1
i=4	2	3	5	7	9	8	1
i=5	2	3	5	7	8	9	1
i=6	1	2	3	5	7	8	9

Q.19 [MCQ · 2 marks · Linked lists — Floyd's cycle detection]**Answer: (A)**

Tail length $\mu = 3$ (nodes 1, 2, 3 precede the cycle), cycle length $\lambda = 5$ ($4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 4$).

Positions after each iteration:

- 1: slow 2, fast 3
- 2: slow 3, fast 5
- 3: slow 4, fast 7
- 4: slow 5, fast 4 ($7 \rightarrow 8 \rightarrow 4$)
- 5: slow 6, fast 6 ($4 \rightarrow 5 \rightarrow 6$) → **meet at node 6**

Check with theory: when slow enters the cycle (after $\mu = 3$ steps, at node 4) fast is 3 steps ahead along the cycle (at node 7). The gap closes by 1 per iteration, so they meet after $(\lambda - 3) \bmod \lambda = 2$ more iterations: total 5, at 2 steps past node 4 = node 6.

- (B) Node 4 is the cycle entry, found only in phase 2 (reset one pointer to head and move both by 1: from 1 and from 6 they meet at 4 after 3 steps = μ).
- (C), (D) result from mis-stepping fast around $8 \rightarrow 4$.

Trap: assuming the meeting point is the start of the cycle.

Q.20 [MSQ · 2 marks · Solving recurrences]**Answer: (A), (C), (D)**

- (A) **Correct.** $T(n) = 1 + 2 + \dots + n = n(n + 1)/2 = \Theta(n^2)$.
- (B) **Incorrect.** $a = 4, b = 2 \rightarrow n^{\log_2 4} = n^2$, which matches $f(n) = n^2$ (case 2) $\rightarrow \Theta(n^2 \log n)$, not $\Theta(n^2)$.
- (C) **Correct.** $n^{\log_2 3} \approx n^{1.585}$ dominates $f(n) = n$ polynomially (case 1) $\rightarrow \Theta(n^{\log_2 3})$.
- (D) **Correct.** Recursion tree: level i has 2^i nodes each costing $(n/2^i) \cdot \log(n/2^i)$, level total $n(\log n - i)$. Summing $i = 0 \dots \log n$ gives $n \cdot \Theta(\log^2 n)$. (The basic Master theorem does not apply since $n \log n$ is not polynomially larger than n ; the extended case 2 gives $\Theta(n \log^{k+1} n)$ with $k = 1$.)

Trap: in case 2 the answer always picks up an extra log factor; equating it with $f(n)$ is the most common error.

Mock Test 38

Full-Syllabus Mock — Paper 8

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — string slicing with negative steps]

Consider the following Python program. What is printed?

Python

```
1 s = "DATASCIENCE"
2 print(s[-2:2:-3], s[::4], s[3:-12:-1])
```

- (A) CIA DSN followed by an empty string
- (B) CIAT DSN ATAD
- (C) CIA DSNE ATA
- (D) CIA DSN ATAD

Q.2 [NAT · numerical answer | 1 mark | Python — floor division and modulo with negatives]

Consider the following Python program. The value printed is _____.

Python

```
1 print((-7) // 2 + (-7) % 3 + 7 // -2 + int(-7 / 2) + (-9) % 4)
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — infix to postfix]

Operators have the usual precedence: ^ (highest, **right**-associative), then * and / (left-associative), then + and - (left-associative). The postfix form of

$a + b * (c - d) ^ e ^ f / g$

is:

- (A) $a \ b \ c \ d \ - \ e \ ^ \ f \ ^ \ * \ g \ / \ +$
- (B) $a \ b \ c \ d \ - \ e \ f \ ^ \ ^ \ * \ g \ / \ +$
- (C) $a \ b \ c \ d \ - \ e \ f \ ^ \ ^ \ g \ / \ * \ +$
- (D) $a \ b \ c \ d \ - \ e \ f \ ^ \ ^ \ * \ + \ g \ /$

Q.4 [NAT · numerical answer | 1 mark | Queues — elimination simulation]

People numbered 1 to 9 stand in a queue (1 at the front). Repeatedly: the person at the front is dequeued and enqueued at the rear **three** times, and then the person now at the front is dequeued and eliminated. This continues until one person remains. The number of the last remaining person is _____.



Initial queue (front at left)

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Linked lists — doubly linked list surgery]

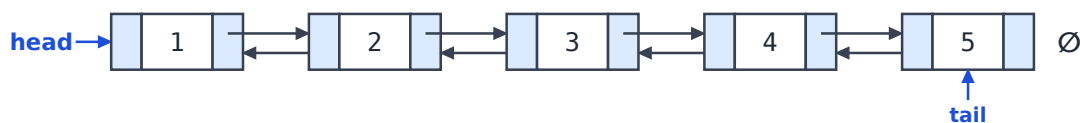
The program below tries to move node 2 of a doubly linked list to the end. What is printed? (*walk* follows the named pointer for at most k nodes.)

Python

```

1 class N:
2     def __init__(self, v):
3         self.v, self.prev, self.next = v, None, None
4
5 nodes = [N(i) for i in range(1, 6)]
6 for a, b in zip(nodes, nodes[1:]):
7     a.next, b.prev = b, a
8 head, tail = nodes[0], nodes[-1]
9
10 x = nodes[1]
11 x.prev.next = x.next
12 x.next.prev = x.prev
13 tail.next, x.prev = x, tail
14 tail = x
15
16 def walk(p, step, k):
17     out = []
18     while p and len(out) < k:
19         out.append(p.v)
20         p = getattr(p, step)
21     return out
22
23 print(walk(head, 'next', 7), walk(tail, 'prev', 7))

```



List before the move

- (A) [1, 3, 4, 5, 2, 3, 4] [2, 5, 4, 3, 1]
 (B) [1, 3, 4, 5, 2] [2, 5, 4, 3, 1]
 (C) [1, 3, 4, 5, 2, 3, 4] [2, 5, 4, 3, 2, 5, 4]
 (D) [1, 2, 3, 4, 5] [5, 4, 3, 2, 1]

Q.6 [MCQ · single correct | 1 mark | Binary trees — node counting]

A binary tree has 20 leaves and 13 nodes that have exactly one child. The total number of nodes in the tree is:

- (A) 45
- (B) 53
- (C) 52
- (D) 33

Q.7 [MSQ · one or more correct | 1 mark | Heaps — recognising max-heaps]

Which of the following arrays (0-based; children of index i at $2i+1$ and $2i+2$) represent a binary **max**-heap?

- (A) [92, 80, 90, 81, 70, 60, 30]
- (B) [92, 88, 90, 50, 88, 89, 20, 49, 50]
- (C) [92, 70, 85, 75, 60, 80, 50]
- (D) [92, 85, 70, 60, 85, 40, 65, 10]

Q.8 [MCQ · single correct | 1 mark | Complexity of a loop nest]

What is the time complexity of the following fragment as a function of n ($n \geq 2$)?

Python

```
1 count = 0
2 for i in range(1, n + 1):
3     j = i
4     while j < n:
5         j *= 2
6         count += 1
```

- (A) $\Theta(n \log n)$
- (B) $\Theta(n \log \log n)$
- (C) $\Theta(\log^2 n)$
- (D) $\Theta(n)$

Q.9 [NAT · numerical answer | 1 mark | Bubble sort — passes with early exit]

Bubble sort with early termination makes left-to-right passes over [12, 3, 25, 18, 30, 41, 7, 50]; pass p compares $a[j]$, $a[j+1]$ for $j = 0 \dots n-1-p$ and swaps if $a[j] > a[j+1]$. The algorithm stops after the first pass in which **no swap** occurs. The total number of passes made (including that final pass) is _____.

Answer: _____ (enter a numerical value)

Q.10 [MSQ · one or more correct | 1 mark | Graph theory — degree sequences]

Which of the following sequences is/are the degree sequence of some **simple** undirected graph?

- (A) (3, 3, 3, 3, 3, 3, 3)
- (B) (6, 1, 1, 1, 1, 1, 1)
- (C) (5, 5, 4, 3, 2, 1)
- (D) (4, 4, 3, 3, 2, 2)

Q.11 [MCQ · single correct | 2 marks | Python — sort stability with reverse=True]

Consider the following Python program. What is printed?

Python

```
1 words = ["kiwi", "fig", "plum", "date", "pear", "lime", "apple"]
2 r1 = sorted(words, key=len, reverse=True)
3 r2 = sorted(words, key=len)[::-1]
4 print(r1[:3], r2[:3])
```

- (A) ['apple', 'kiwi', 'plum'] ['apple', 'kiwi', 'plum']
- (B) ['apple', 'lime', 'pear'] ['apple', 'lime', 'pear']
- (C) ['apple', 'kiwi', 'plum'] ['apple', 'lime', 'pear']
- (D) ['apple', 'plum', 'pear'] ['apple', 'lime', 'pear']

Q.12 [NAT · numerical answer | 2 marks | Merge sort — counting inversions]

The following function counts inversions (pairs $i < j$ with $a[i] > a[j]$) while merge-sorting. The value printed is _____.

Python

```
1 def sort_count(a):
2     if len(a) <= 1:
3         return a, 0
4     m = len(a) // 2
5     L, x = sort_count(a[:m])
6     R, y = sort_count(a[m:])
7     out, i, j, c = [], 0, 0, x + y
8     while i < len(L) and j < len(R):
9         if L[i] <= R[j]:
10            out.append(L[i]); i += 1
11        else:
12            out.append(R[j]); j += 1
13            c += len(L) - i
14    return out + L[i:] + R[j:], c
15
16 print(sort_count([8, 3, 9, 1, 6, 2, 7, 4])[1])
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Quicksort — Hoare partition]

The Hoare partition scheme below is applied once to $A = [24, 31, 8, 47, 15, 24, 3, 52, 19]$ with $lo = 0$, $hi = 8$.

$pivot = A[lo]$; $i = lo - 1$; $j = hi + 1$

repeat forever:

- do $j = j - 1$ while $A[j] > pivot$
- do $i = i + 1$ while $A[i] < pivot$
- if $i < j$: swap $A[i]$, $A[j]$ else: return j

Which of the following statements is/are TRUE?

- (A) Exactly 3 swaps are performed
- (B) After the call, $A[4] = 24$, i.e. the pivot is in its final sorted position
- (C) After the call, every element of $A[0..4]$ is $\leq$ every element of $A[5..8]$
- (D) The value returned is 4

Q.14 [NAT · numerical answer | 2 marks | Recurrences — exact evaluation]

Let $T(1) = 1$ and $T(n) = 3T(n/3) + n$ for n a power of 3, $n > 1$. The value of $T(81)$ is _____.

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | BST — successive deletions]

The keys 47, 23, 71, 12, 35, 58, 84, 29, 41, 63, 90, 38 are inserted in that order into an empty BST (shown). Then 23, 71 and 47 are deleted, in that order; a node with two children is replaced by its **in-order successor**, and a node with at most one child is spliced out. What is the pre-order traversal of the final tree?

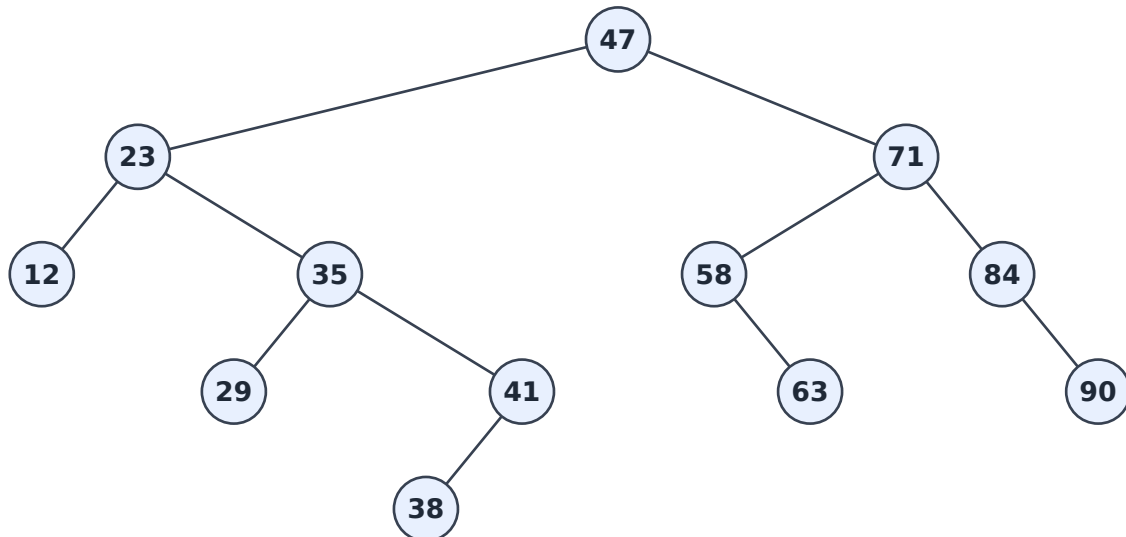


Figure: BST before deletions

- (A) 58, 29, 12, 35, 41, 38, 84, 63, 90
- (B) 41, 12, 35, 29, 38, 63, 58, 84, 90
- (C) 58, 29, 12, 35, 38, 41, 84, 63, 90
- (D) 58, 35, 12, 29, 41, 38, 84, 63, 90

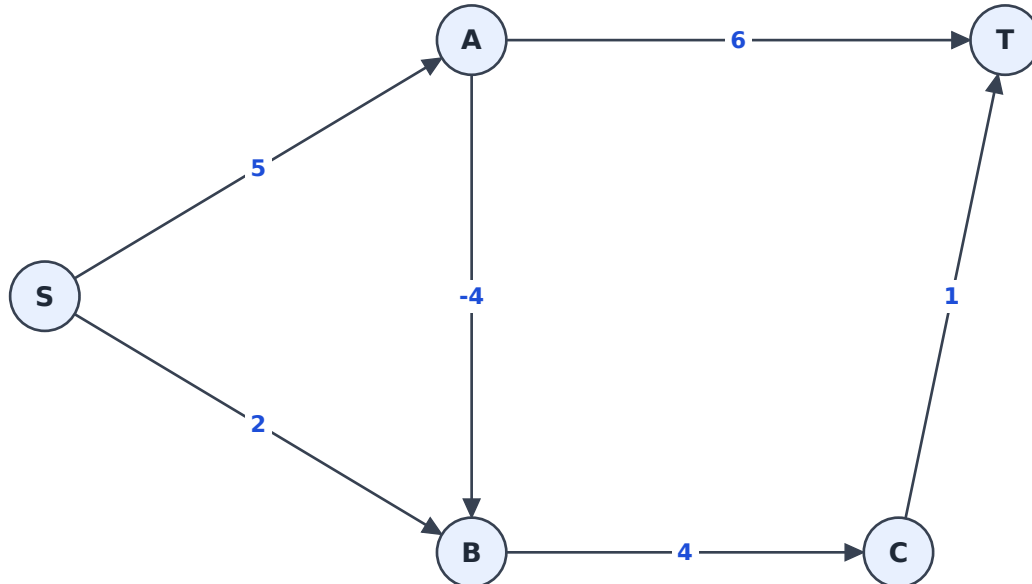
Q.16 [MSQ · one or more correct | 2 marks | Hashing — limits of quadratic probing]

Keys 14, 21, 28, 35 are inserted in that order into an empty table of size 7 (slots 0-6) using quadratic probing $h(k, i) = (k \bmod 7 + i^2) \bmod 7$, $i = 0, 1, 2, \dots$. Next, key 42 is to be inserted. Which of the following statements is/are TRUE?

- (A) If double hashing $h(k, i) = (k \bmod 7 + i \cdot (5 - k \bmod 5)) \bmod 7$ had been used for all five keys, every key would have been placed
- (B) If linear probing (step +1) had been used for all five keys, 42 would be stored in slot 3
- (C) Key 42 cannot be placed by this probe sequence, even though 3 slots are empty
- (D) Key 35 is stored in slot 2

Q.17 [MCQ · single correct | 2 marks | Dijkstra with a negative edge]

Dijkstra's algorithm is run from S on the directed graph shown, which has one negative edge. A vertex is **finalised** when it is extracted from the priority queue, and relaxations into finalised vertices are ignored. Let $d(C)$, $d(T)$ be the distances it reports and $\delta(T)$ the true shortest distance from S to T. Then $(d(C), d(T), \delta(T))$ is:



- (A) (5, 6, 6)
- (B) (6, 7, 6)
- (C) (6, 7, 7)
- (D) (6, 11, 6)

Q.18 [NAT · numerical answer | 2 marks | BFS — counting shortest paths in a grid]

A robot moves on the 5×6 grid shown, one step at a time up, down, left or right, never entering a blocked cell (#). The number of distinct shortest paths from the top-left cell (row 0, column 0) to the bottom-right cell (row 4, column 5) is _____.

Grid (# = blocked)

	0	1	2	3	4	5
0	S					
1		#			#	
2			#			
3					#	
4						T

Answer: _____ (enter a numerical value)

Q.19 [MSQ · one or more correct | 2 marks | Python — shallow copy, deep copy and aliasing]

Consider the following Python code. Which of the following statements is/are TRUE after it runs?

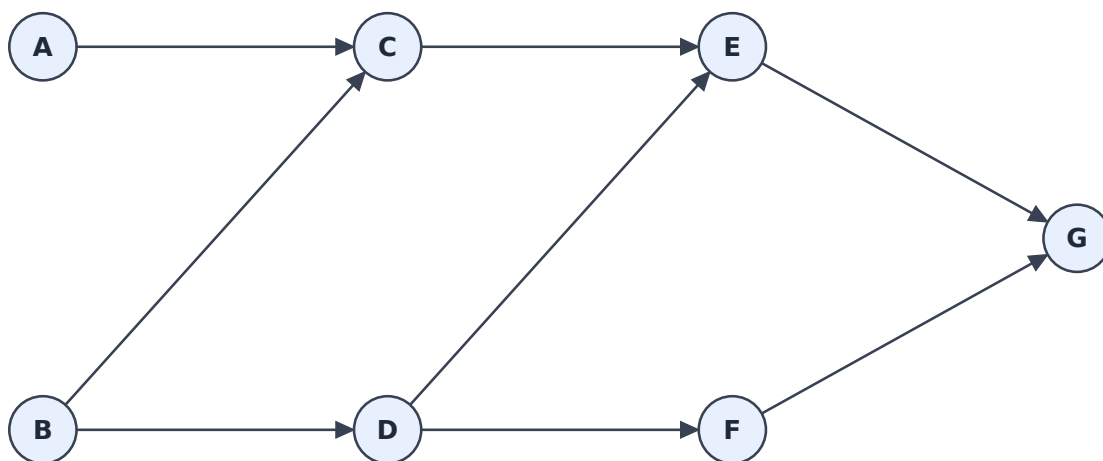
Python

```
1 import copy
2 a = [[1, 2], [3, 4]]
3 b = a[:]
4 c = copy.deepcopy(a)
5 d = a
6 b[0].append(9)
7 b[1] = [7]
8 d.append([5])
9 c[0][0] = 0
```

- (A) `a == [[1, 2, 9], [3, 4], [5]]`
- (B) `c[0] == [0, 2, 9]`
- (C) `a[0] is b[0]`
- (D) `b == [[1, 2, 9], [7]]`

Q.20 [NAT · numerical answer | 2 marks | DAGs — counting topological orders]

The number of distinct topological orderings of the directed acyclic graph shown is _____.



Answer: _____ (enter a numerical value)

Answer Key — Mock Test 38

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Python — string slicing with negative steps
2	NAT	1	-6	Python — floor division and modulo with negatives
3	MCQ	1	(B)	Stacks — infix to postfix
4	NAT	1	1	Queues — elimination simulation
5	MCQ	1	(A)	Linked lists — doubly linked list surgery
6	MCQ	1	(C)	Binary trees — node counting
7	MSQ	1	(B), (D)	Heaps — recognising max-heaps
8	MCQ	1	(D)	Complexity of a loop nest
9	NAT	1	6	Bubble sort — passes with early exit
10	MSQ	1	(B), (D)	Graph theory — degree sequences
11	MCQ	2	(C)	Python — sort stability with reverse=True
12	NAT	2	16	Merge sort — counting inversions
13	MSQ	2	(A), (C), (D)	Quicksort — Hoare partition
14	NAT	2	405	Recurrences — exact evaluation
15	MCQ	2	(A)	BST — successive deletions
16	MSQ	2	(A), (C), (D)	Hashing — limits of quadratic probing
17	MCQ	2	(B)	Dijkstra with a negative edge
18	NAT	2	12	BFS — counting shortest paths in a grid
19	MSQ	2	(A), (C), (D)	Python — shallow copy, deep copy and aliasing
20	NAT	2	14	DAGs — counting topological orders

Detailed Solutions — Mock Test 38

Q.1 [MCQ · 1 mark · Python — string slicing with negative steps]

Answer: (D)

Index the 11 characters: D0 A1 T2 A3 S4 C5 I6 E7 N8 C9 E10. A slice `s[a:b:k]` starts at `a` and moves by `k`, stopping **before** `b`; negative indices are first converted by adding `len(s) = 11`.

- `s[-2:2:-3]`: start $-2 \rightarrow 9$. Indices 9, 6, 3 (next would be 0, which is past the stop index 2) $\rightarrow$ C, I, A $\rightarrow$ CIA.
- `s[:4]`: indices 0, 4, 8 $\rightarrow$ D, S, N $\rightarrow$ DSN.
- `s[3:-12:-1]`: $-12 + 11 = -1$, which for a negative step is clamped to 'before index 0', so the slice runs 3, 2, 1, 0 $\rightarrow$ A, T, A, D $\rightarrow$ ATAD.

Output CIA DSN ATAD $\rightarrow$ option **(D)**.

- (A) treats -12 as index $-1 = 10$, which would give an empty slice; but -12 is out of range on the left and is clamped, not wrapped.
- (B) includes the stop index 2 (T) — the stop is exclusive.
- (C) takes index 10 (E) in the second slice and drops D in the third.

Trap: `s[3:-1:-1]` really is empty (-1 means index 10), while `s[3:-12:-1]` is not — out-of-range bounds are clamped.

Q.2 [NAT · 1 mark · Python — floor division and modulo with negatives]

Answer: -6

Python's `//` rounds **towards** $-\infty$ (floor), and `%` returns a result with the sign of the **divisor**, so that `a == (a // b) * b + a % b` always holds. `int()` of a float truncates **towards 0**.

- $(-7) // 2 = \text{floor}(-3.5) = -4$
- $(-7) \% 3 = -7 - 3 \cdot \text{floor}(-7/3) = -7 - 3 \cdot (-3) = 2$
- $7 // -2 = \text{floor}(-3.5) = -4$
- $\text{int}(-7 / 2) = \text{int}(-3.5) = -3$ (truncation)
- $(-9) \% 4 = -9 - 4 \cdot (-3) = 3$

Sum $= -4 + 2 - 4 - 3 + 3 = -6$.

Trap: C/Java semantics (truncate towards zero, remainder takes the sign of the dividend) would give $-3 - 1 - 3 - 3 - 1 = -11$.

Q.3 [MCQ · 1 mark · Stacks — infix to postfix]

Answer: (B)

First fully parenthesise using precedence and associativity:

- $\wedge$ is right-associative: $(c - d) \wedge e \wedge f = (c - d) \wedge (e \wedge f)$.
- $*$ and $/$ are left-associative: $b * X / g = (b * X) / g$.
- Then $+$: $a + ((b * ((c - d) \wedge (e \wedge f))) / g)$.

Convert bottom-up:

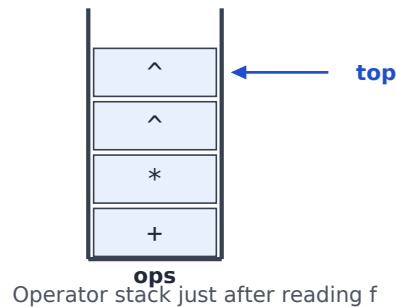
- $c - d \rightarrow c \ d \ -$
- $e \wedge f \rightarrow e \ f \ \wedge$; $(c - d) \wedge (e \wedge f) \rightarrow c \ d \ - \ e \ f \ \wedge \ \wedge$
- $b * \dots \rightarrow b \ c \ d \ - \ e \ f \ \wedge \ \wedge \ *$
- $\dots / g \rightarrow b \ c \ d \ - \ e \ f \ \wedge \ \wedge \ * \ g \ /$
- $a + \dots \rightarrow a \ b \ c \ d \ - \ e \ f \ \wedge \ \wedge \ * \ g \ / \ +$

Answer **(B)**.

- (A) treats $\wedge$ as left-associative $((c - d) \wedge e) \wedge f$.
- (C) groups $b * (X / g)$ — wrong for left-associative $*, /$.

- (D) applies $+$ before $/$.

Tip (stack algorithm): when an incoming operator has the same precedence as the stack top, pop first if it is left-associative, but push on top for right-associative $^$.



Q.4 [NAT · 1 mark · Queues — elimination simulation]

Answer: 1

Each round moves 3 people to the back and eliminates the 4th — the Josephus problem with $k = 4$.

Queue states (front first), eliminated person in bold:

- [1..9] → rotate 1,2,3 → eliminate **4** → [5,6,7,8,9,1,2,3]
- rotate 5,6,7 → eliminate **8** → [9,1,2,3,5,6,7]
- rotate 9,1,2 → eliminate **3** → [5,6,7,9,1,2]
- rotate 5,6,7 → eliminate **9** → [1,2,5,6,7]
- rotate 1,2,5 → eliminate **6** → [7,1,2,5]
- rotate 7,1,2 → eliminate **5** → [7,1,2]
- rotate 7,1,2 → eliminate **7** → [1,2]
- rotate 1,2,1 → front is 2 → eliminate **2** → [1]

Survivor: **1**.

Check with the Josephus recurrence $J(1) = 0$, $J(n) = (J(n-1) + 4) \bmod n$ (0-based): 0, 0, 1, 1, 0, 4, 1, 5, 0 → position 0 → person 1.

Trap: when fewer than 4 people remain the rotation wraps around more than once — with [1, 2], three rotations leave 2 at the front.

Q.5 [MCQ · 1 mark · Linked lists — doubly linked list surgery]

Answer: (A)

Moving a node needs **four** pointer fields to be right: the neighbours' links around the hole, and both **prev** and **next** of the moved node.

- Unlink: $1.\text{next} = 3$ and $3.\text{prev} = 1$. Correct.
- Append: $5.\text{next} = 2$ and $2.\text{prev} = 5$; $\text{tail} = 2$.
- **Missing:** $x.\text{next} = \text{None}$. Node 2 still has **next** pointing to 3.

Forward walk from head: $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \dots$ a cycle; **walk** stops after 7 values: [1, 3, 4, 5, 2, 3, 4].

Backward walk from tail: $2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow \text{None}$ (all **prev** links are correct): [2, 5, 4, 3, 1].

Answer **(A)**.

- (B) is what a correct move would print.
- (C) assumes the **prev** chain is also cyclic — $1.\text{prev}$ is still None.
- (D) assumes nothing changed.

Trap: stale pointers in the moved node silently create cycles; a forward traversal without the length guard would never terminate.

Q.6 [MCQ · 1 mark · Binary trees — node counting]**Answer: (C)**

Let n_0, n_1, n_2 be the numbers of nodes with 0, 1, 2 children, and n the total.

- Every node except the root has exactly one parent, so the number of edges is $n - 1$.
- Counting edges from the parent side: edges = $n_1 + 2n_2$.
- So $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \Rightarrow n_0 = n_2 + 1$.

Here $n_0 = 20 \Rightarrow n_2 = 19$. Total $n = 20 + 13 + 19 = 52 \rightarrow$ option **(C)**.

- (A) uses $n_2 = n_1 - 1$ (confusing the two counts).
- (B) uses $n_2 = n_0$ (forgetting the -1).
- (D) ignores the two-child nodes.

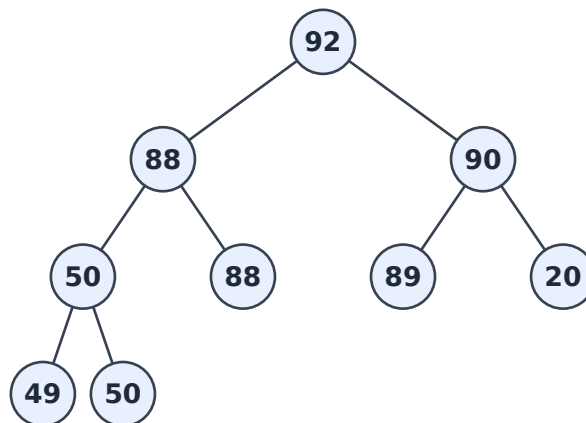
Tip: the number of one-child nodes n_1 does not affect n_2 at all — any number of single-child nodes can be inserted along edges without changing the leaf count.

Q.7 [MSQ · 1 mark · Heaps — recognising max-heaps]**Answer: (B), (D)**

Check $a[\text{parent}] \geq a[\text{child}]$ for every internal index $i = 0 \dots \lfloor n/2 \rfloor - 1$. Equal keys are allowed.

- (A) $i=1$: 80 has child $a[3] = 81$. **Not a heap.**
- (B) $i=0$: $92 \geq 88, 90$ · $i=1$: $88 \geq 50, 88$ · $i=2$: $90 \geq 89, 20$ · $i=3$: $50 \geq 49, 50$. **Max-heap.**
- (C) $i=1$: 70 has child $a[3] = 75 > 70$. **Not a heap.**
- (D) $i=0$: $92 \geq 85, 70$ · $i=1$: $85 \geq 60, 85$ · $i=2$: $70 \geq 40, 65$ · $i=3$: $60 \geq 10$. **Max-heap.**

Trap: checking only that the array is 'roughly decreasing' or that each level's values are smaller than the previous level's — the heap property is only between a parent and its own children (in (B), 89 at depth 2 exceeds 88 at depth 1, which is fine).



Array (C) drawn as a tree

Q.8 [MCQ · 1 mark · Complexity of a loop nest]**Answer: (D)**

For a fixed i the inner loop doubles j from i until it reaches n , so it runs $\lceil \log_2(n/i) \rceil$ times — **not** $\log_2 n$ times.

Total $\approx \sum_{i=1}^n \log_2(n/i) = n \log_2 n - \log_2(n!)$.

By Stirling, $\log_2(n!) = n \log_2 n - n \log_2 e + O(\log n)$, so the total is $\approx n \log_2 e \approx 1.44 n = \Theta(n)$.

Intuition: half of the i values ($i > n/2$) need ≤ 1 iteration, a quarter need ≤ 2 , an eighth need ≤ 3 , ...
 $\rightarrow n(1/2 + 2/4 + 3/8 + \dots) = 2n$.

Answer **(D)**.

- (A) bounds every inner loop by $\log n$ — a valid upper bound, but not tight.
- (C), (B) have no basis in the loop structure.

Trap: multiplying 'outer $\times$ worst inner' gives $O(n \log n)$; GATE asks for the tight Θ bound.

Q.9 [NAT · 1 mark · Bubble sort — passes with early exit]

Answer: 6

Large elements sink to the end quickly, but a small element moves **left by at most one position per pass**. The 7 at index 6 must reach index 1, which needs 5 passes; one more pass confirms sortedness.

- Pass 1: [3, 12, 18, 25, 30, 7, 41, 50]
- Pass 2: [3, 12, 18, 25, 7, 30, 41, 50]
- Pass 3: [3, 12, 18, 7, 25, 30, 41, 50]
- Pass 4: [3, 12, 7, 18, 25, 30, 41, 50]
- Pass 5: [3, 7, 12, 18, 25, 30, 41, 50]
- Pass 6: no swap $\rightarrow$ stop.

Total passes = 6.

Trap: answering 5 (forgetting the confirming pass) or 2 (thinking 'the array is almost sorted, so bubble sort finishes fast' — bubble sort is adaptive only to elements that are out of place to the *left*).

Array after each pass								
	0	1	2	3	4	5	6	7
start	12	3	25	18	30	41	7	50
p1	3	12	18	25	30	7	41	50
p2	3	12	18	25	7	30	41	50
p3	3	12	18	7	25	30	41	50
p4	3	12	7	18	25	30	41	50
p5	3	7	12	18	25	30	41	50
p6	3	7	12	18	25	30	41	50

Q.10 [MSQ · 1 mark · Graph theory — degree sequences]

Answer: (B), (D)

Necessary: the degree sum is even (handshaking). Sufficient test: **Havel-Hakimi** — remove the largest degree d , subtract 1 from the next d degrees, re-sort, repeat; the sequence is graphical iff this ends with all zeros and never goes negative.

- (A) seven vertices of odd degree $\rightarrow$ odd sum 21. **Not graphical.**
- (B) the star $K_{1,6}$. **Graphical.**
- (C) sum 20 (even), but: remove 5 $\rightarrow$ (4,3,2,1,0) $\rightarrow$ remove 4 $\rightarrow$ (2,1,0,-1) **negative** $\rightarrow$ not graphical. Intuition: two vertices of degree 5 in a 6-vertex simple graph are adjacent to every other vertex, so no vertex can have degree 1. **Not graphical.**
- (D) sum 18. (4,4,3,3,2,2) $\rightarrow$ remove 4: (3,2,2,1,2) $\rightarrow$ sort (3,2,2,2,1) $\rightarrow$ remove 3: (1,1,1,1) $\rightarrow$ remove 1: (0,1,1) $\rightarrow$ (1,1,0) $\rightarrow$ (0,0). **Graphical.**

Trap: stopping at the parity check — (C) has an even sum yet is impossible.

Q.11 [MCQ · 2 marks · Python — sort stability with reverse=True]**Answer: (C)**

Python's `sorted` is **stable**, and `reverse=True` is implemented so that stability is preserved: elements with equal keys keep their *original* relative order. Reversing a sorted list afterwards, however, flips the order of equal keys.

Lengths: kiwi 4, fig 3, plum 4, date 4, pear 4, lime 4, apple 5.

- r1: descending by length, ties in input order → apple, kiwi, plum, date, pear, lime, fig → first three ['apple', 'kiwi', 'plum'].
- r2: ascending sort gives fig, kiwi, plum, date, pear, lime, apple; reversing it gives apple, lime, pear, date, plum, kiwi, fig → ['apple', 'lime', 'pear'].

Answer **(C)**.

- (A) assumes `[::-1]` preserves tie order.
- (B) assumes `reverse=True` is the same as sorting and then reversing.
- (D) mixes the two orders arbitrarily.

Tip: to sort descending by one key while keeping stability, use `reverse=True` (or a negated key), never `[::-1]` after an ascending sort.

Q.12 [NAT · 2 marks · Merge sort — counting inversions]**Answer: 16**

When $R[j]$ is output before the remaining $L[i..]$, it is smaller than **all** $\text{len}(L) - i$ of them (L is sorted), so each such step adds $\text{len}(L) - i$ inversions that cross the two halves. Inversions inside the halves are counted recursively.

Level 1 (pairs): $[8,3] \rightarrow 1$, $[9,1] \rightarrow 1$, $[6,2] \rightarrow 1$, $[7,4] \rightarrow 1 \rightarrow 4$.

Level 2:

- $[3,8] + [1,9]$: 1 beats 3 and 8 → +2 → $[1,3,8,9]$.
- $[2,6] + [4,7]$: 4 beats 6 → +1 → $[2,4,6,7]$.

Level 3: $[1,3,8,9] + [2,4,6,7]$:

- 2 beats 3, 8, 9 → +3; 4 beats 8, 9 → +2; 6 → +2; 7 → +2 → +9.

Total = $4 + 3 + 9 = 16$.

Brute-force check: larger elements to the left of each element: $3:1$, $9:0$, $1:3$, $6:2$, $2:4$, $7:2$, $4:4 \rightarrow 1+0+3+2+4+2+4 = 16$.

Trap: adding 1 per out-of-order merge step instead of $\text{len}(L) - i$, or using $<$ instead of $<=$ (which would also count equal pairs if duplicates were present).

Cross inversions found at each merge

	merge	result	added
L1	8 3	3 8	1
L1	9 1	1 9	1
L1	6 2	2 6	1
L1	7 4	4 7	1
L2	3 8 1 9	1 3 8 9	2
L2	2 6 4 7	2 4 6 7	1
L3	1 3 8 9 2 4 6 7	1 2 3 4 6 7 8 9	9

Q.13 [MSQ · 2 marks · Quicksort — Hoare partition]**Answer: (A), (C), (D)**

Hoare's scheme moves two pointers towards each other and swaps out-of-place pairs. It guarantees $A[lo..j] \leq \text{pivot} \leq A[j+1..hi]$ but does **not** put the pivot in its final place. Pivot = 24.

- Round 1: j stops at 8 ($19 \leq 24$); i stops at 0 (24 is not < 24). Swap $\rightarrow [19, 31, 8, 47, 15, 24, 3, 52, 24]$.
- Round 2: j: 7 ($52 > 24$) $\rightarrow$ 6 (3) stop; i: 1 (31) stop. Swap $\rightarrow [19, 3, 8, 47, 15, 24, 31, 52, 24]$.
- Round 3: j: 5 (24, not > 24) stop; i: 2 ($8 < 24$), 3 (47) stop. Swap $\rightarrow [19, 3, 8, 24, 15, 47, 31, 52, 24]$.
- Round 4: j: 4 (15) stop; i: 4 ($15 < 24$), 5 (47) stop. $i = 5 \geq j = 4 \rightarrow$ return 4.
- (A) Three swaps. **True.**
- (B) $A[4] = 15$; the original pivot ended at index 8. **False.**
- (C) $A[0..4] = \{19, 3, 8, 24, 15\} \leq 24 \leq A[5..8] = \{47, 31, 52, 24\}$. **True.**
- (D) Returns 4. **True.**

Trap: applying Lomuto intuition ('the pivot lands at the returned index'). With Hoare, the next recursive calls are on $[lo..j]$ and $[j+1..hi]$ — the pivot is not excluded.

Array after each swap (pivot 24)									
	0	1	2	3	4	5	6	7	8
start	24	31	8	47	15	24	3	52	19
swap 1	19	31	8	47	15	24	3	52	24
swap 2	19	3	8	47	15	24	31	52	24
swap 3	19	3	8	24	15	47	31	52	24

Q.14 [NAT · 2 marks · Recurrences — exact evaluation]**Answer: 405**

Unroll the recurrence (recursion-tree method). With $n = 3^k$:

$$T(n) = 3T(n/3) + n = 9T(n/9) + n + n = \dots = 3^k \cdot T(1) + k \cdot n = n + n \log_3 n.$$

Each of the k levels of the recursion tree contributes exactly n (3^i subproblems of size $n/3^i$), and the $3^k = n$ leaves contribute $T(1) = 1$ each.

For $n = 81 = 3^4$: $T(81) = 81 + 81 \times 4 = \mathbf{405}$.

Step-by-step check:

- $T(3) = 3 \cdot 1 + 3 = 6$
- $T(9) = 3 \cdot 6 + 9 = 27$
- $T(27) = 3 \cdot 27 + 27 = 108$
- $T(81) = 3 \cdot 108 + 81 = 405 \checkmark$

Asymptotically this is case 2 of the master theorem ($a = b = 3$, $f(n) = \Theta(n)$) $\rightarrow \Theta(n \log n)$, the same shape as merge sort's recurrence.

Trap: forgetting the leaf contribution $n \cdot T(1)$ and answering $324 = n \log_3 n$.

Q.15 [MCQ · 2 marks · BST — successive deletions]**Answer: (A)**

In-order successor of a node with two children = minimum of its right subtree; that node has no left child, so removing it from its old place is a simple splice.

- Delete 23 (children 12, 35): successor = min of $\{35, 29, 41, 38\} = 29$. 29 replaces 23; 29 was a leaf $\rightarrow$ removed. Left subtree becomes 29 [12, 35 [–, 41 [38, –]]].

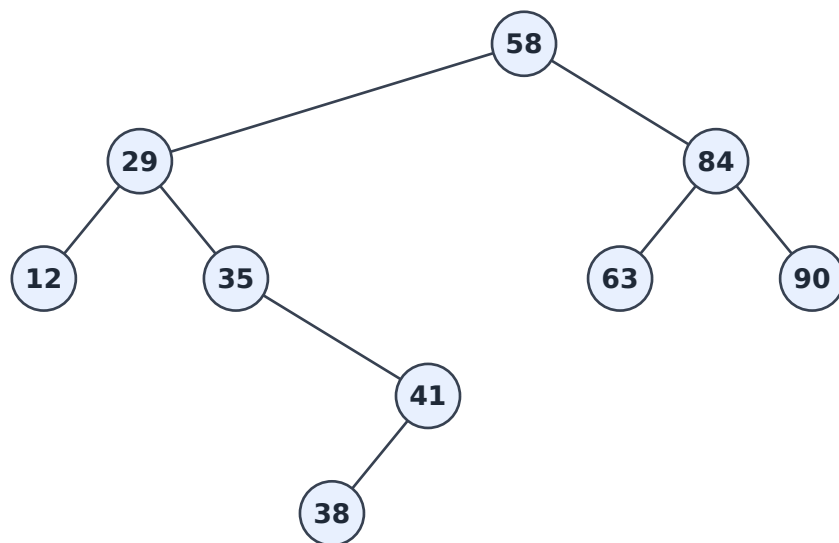
- Delete 71 (children 58, 84): successor = 84 (84 has no left child). 84 replaces 71 and 84's right child 90 moves up. Right subtree: 84 [58 [–, 63], 90].
- Delete 47 (root): successor = min of right subtree = 58. 58 replaces 47; 58's right child 63 takes its place. Right subtree: 84 [63, 90].

Final tree: 58 [29 [12, 35 [–, 41 [38, –]], 84 [63, 90]].

Pre-order: 58, 29, 12, 35, 41, 38, 84, 63, 90 → option **(A)**.

- (B) uses the in-order **predecessor** instead.
- (C) lists 38 before 41, but 38 is 41's left child, so 41 comes first in pre-order.
- (D) puts 35 at 29's position — 35 is not the successor of 23.

Trap: when deleting the root last, use the tree *after* the earlier deletions — 58's right child is now 63.



Final BST

Q.16 [MSQ · 2 marks · Hashing — limits of quadratic probing]

Answer: (A), (C), (D)

All five keys are multiples of 7, so all have home slot 0. Quadratic probing visits $0 + i^2 \bmod 7$ for $i = 0, 1, 2, \dots$: the squares mod 7 are 0, 1, 4, 2, 2, 4, 1, 0, ... — only the **4** quadratic residues {0, 1, 2, 4}.

- $14 \rightarrow \text{slot } 0$; $21 \rightarrow 0, \mathbf{1}$; $28 \rightarrow 0, 1, \mathbf{4}$; $35 \rightarrow 0, 1, 4, 9 \bmod 7 = \mathbf{2}$.
- (A) $h_2(k) = 5 - (k \bmod 5) \in \{1, \dots, 5\}$ is never 0 and 7 is prime, so each probe sequence is a permutation of all 7 slots; with only 5 keys every insertion succeeds. **True.**
- (B) With linear probing: $14 \rightarrow 0$, $21 \rightarrow 1$, $28 \rightarrow 2$, $35 \rightarrow 3$, $42 \rightarrow \mathbf{4}$, not 3. **False.**
- (C) 42 can only probe {0, 1, 2, 4}, all full; slots 3, 5, 6 are empty but unreachable. **True.** (Quadratic probing is guaranteed to succeed only when the load factor is $\leq 1/2$ for prime m .)
- (D) 35 lands in slot 2. **True.**

Trap: assuming 'table not full $\Rightarrow$ insertion succeeds' for every open-addressing scheme.

0	14
1	21
2	35
3	
4	28
5	
6	

After four insertions; 42 can only probe slots 0, 1, 4, 2

Q.17 [MCQ · 2 marks · Dijkstra with a negative edge]

Answer: (B)

Dijkstra's correctness proof assumes that a vertex's distance can only grow along a path; a negative edge breaks this.

Trace:

- Extract S (0): $A = 5$, $B = 2$.
- Extract B (2) — **finalised at 2**: $C = 2 + 4 = 6$.
- Extract A (5): $A \rightarrow B$ would give 1, but B is finalised $\rightarrow$ ignored; $T = 5 + 6 = 11$.
- Extract C (6): $T = \min(11, 7) = 7$.
- Extract T (7).

Reported $d(C) = 6$, $d(T) = 7$.

True distances (e.g. Bellman-Ford): $B = 5 - 4 = 1$, $C = 1 + 4 = 5$, $T = 5 + 1 = 6$ via $S \rightarrow A \rightarrow B \rightarrow C \rightarrow T$.

Answer **(B)** (6, 7, 6).

- (A) gives the true values for C and T — not what this Dijkstra reports.
- (C) assumes Dijkstra is still correct.
- (D) forgets the relaxation of $C \rightarrow T$.

Trap: adding a constant to all edge weights to remove negatives does not work either — it penalises paths with more edges.

Q.18 [NAT · 2 marks · BFS — counting shortest paths in a grid]

Answer: 12

BFS from S gives every cell its distance; the number of shortest paths to a cell is the sum of the counts of its neighbours that are exactly one step closer. Here a monotone (right/down only) path of length $4 + 5 = 9$ exists, so every shortest path is monotone and the count obeys $N(r, c) = N(r-1, c) + N(r, c-1)$ (0 for blocked cells).

Row by row:

- row 0: 1, 1, 1, 1, 1, 1
- row 1: 1, #, 1, 2, #, 1
- row 2: 1, 1, #, 2, 2, 3
- row 3: 1, 2, 2, 4, #, 3
- row 4: 1, 3, 5, 9, 9, **12**

e.g. $N(4,3) = N(3,3) + N(4,2) = 4 + 5 = 9$, $N(4,5) = N(3,5) + N(4,4) = 3 + 9 = 12$.

Answer: **12** shortest paths, each of length 9.

Trap: using the unobstructed count $C(9, 4) = 126$, or subtracting paths through each blocked cell separately (inclusion-exclusion errors). The DP table is quicker and safer.

Number of shortest paths to each cell

	0	1	2	3	4	5
0	1	1	1	1	1	1
1	1	#	1	2	#	1
2	1	1	#	2	2	3
3	1	2	2	4	#	3
4	1	3	5	9	9	12

Q.19 [MSQ · 2 marks · Python — shallow copy, deep copy and aliasing]**Answer: (A), (C), (D)**

- `b = a[:]` is a **shallow** copy: a new outer list whose elements are the *same* inner list objects.
- `c = copy.deepcopy(a)` copies the inner lists too.
- `d = a` is just another name for the same object.

Effects:

- `b[0].append(9)` mutates the shared inner list → visible through a, b, d (not c).
- `b[1] = [7]` rebinds slot 1 of **b's** outer list only.
- `d.append([5])` appends to the outer list of a (= d), not to b.
- `c[0][0] = 0` changes only c's private inner list.

Final values: `a = d = [[1, 2, 9], [3, 4], [5]]`, `b = [[1, 2, 9], [7]]`, `c = [[0, 2], [3, 4]]`.

- (A) **True.**
- (B) c was copied *before* 9 was appended and does not share inner lists → `c[0] = [0, 2]`. **False.**
- (C) Shallow copy shares inner objects. **True.**
- (D) **True.**

Trap: thinking `b[1] = [7]` affects a (assignment to an index of the copy only touches the copy), or that `d.append` affects b.**Q.20** [NAT · 2 marks · DAGs — counting topological orders]**Answer: 14**

G must be last (it depends on E and F, which depend on everything else). E needs C and D; C needs A and B; D needs B; F needs D. Count orderings of the first six vertices by case analysis on where F goes.

Step 1 — orderings of {A, B, C, D, E} (ignoring F): A and B before C; B before D; C and D before E. So E is 5th; the first four are a topological order of $A \rightarrow C$, $B \rightarrow C$, $B \rightarrow D$: B first then A, C, D with A before $C \rightarrow (B, A, C, D)$, (B, A, D, C) , (B, D, A, C) ; or A first: (A, B, C, D) , (A, B, D, C) . That is **5** orders.

Step 2 — insert F. F must come after D (and before G); it is independent of A, C, E. For each of the 5 orders, F can go in any gap after D among the six positions:

- D in position 4 (orders B, A, C, D and A, B, C, D): gaps after D → 2 choices each (before E or after E) → 4.
- D in position 3 (B, A, D, C and A, B, D, C): 3 choices each → 6.
- D in position 2 (B, D, A, C): 4 choices → 4.

Total = $4 + 6 + 4 = \mathbf{14}$.**Trap:** multiplying independent-looking counts (e.g. 5×3) — the number of slots for F depends on where D sits in each ordering.

Mock Test 39

Full-Syllabus Mock — Paper 9

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — closures and late binding]

Consider the following Python program. What is printed?

Python

```
1 fs = [lambda x: x * i for i in range(1, 4)]
2 gs = [lambda x, i=i: x * i for i in range(1, 4)]
3 print([f(10) for f in fs], [g(10) for g in gs])
```

- (A) [10, 20, 30] [10, 20, 30]
- (B) [30, 30, 30] [30, 30, 30]
- (C) [10, 20, 30] [30, 30, 30]
- (D) [30, 30, 30] [10, 20, 30]

Q.2 [NAT · numerical answer | 1 mark | Stacks — infix to postfix conversion]

The infix expression

$$a - b * (c + d ^ e ^ (f - g)) / h$$

is converted to postfix with the standard operator-stack algorithm. Precedence: ^ (highest, **right**-associative) > *, / > +, − (left-associative). Opening parentheses are pushed on the stack and popped (not output) at the matching ')'. The maximum number of symbols (operators and '(') on the stack at any instant is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Python — deque rotations]

Consider the following Python program. What is printed?

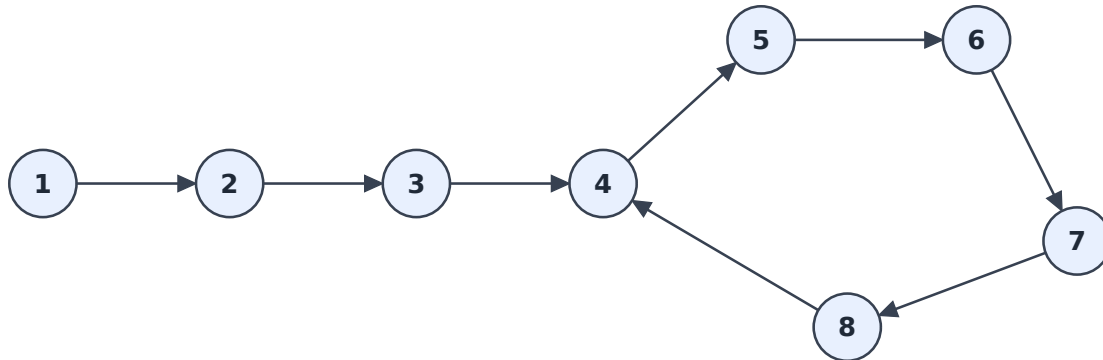
Python

```
1 from collections import deque
2 d = deque([1, 2, 3, 4, 5])
3 d.rotate(2)
4 d.appendleft(d.pop())
5 d.extend([6, 7])
6 d.rotate(-3)
7 print(list(d))
```

- (A) [3, 4, 5, 1, 2, 6, 7]
- (B) [2, 6, 7, 3, 4, 5, 1]
- (C) [1, 2, 6, 7, 3, 4, 5]
- (D) [5, 1, 6, 7, 2, 3, 4]

Q.4 [NAT · numerical answer | 1 mark | Linked lists — Floyd's cycle detection]

A singly linked list has nodes $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$ and node 8's **next** points back to node 4 (see figure). Floyd's algorithm starts **slow** and **fast** at node 1; in each iteration **slow** advances one node and **fast** advances two nodes, and then the two pointers are compared. The number of iterations performed until they first point to the same node is _____.



Answer: _____ (enter a numerical value)

Q.5 [MSQ · one or more correct | 1 mark | BST — valid search sequences]

A binary search tree contains integer keys including 55. A search for 55 compares it with the keys of the nodes on the search path, in order. Which of the following could be such a sequence of examined keys?

- (A) 79, 14, 72, 56, 16, 53, 55
- (B) 10, 75, 64, 43, 60, 57, 55
- (C) 90, 12, 68, 34, 62, 45, 55
- (D) 9, 85, 47, 68, 43, 57, 55

Q.6 [MSQ · one or more correct | 1 mark | Heaps — successive insertion vs build-heap]

The keys 15, 22, 9, 31, 40, 12, 27 are inserted **one at a time** (in this order) into an initially empty binary **max**-heap stored in an array (each insertion appends the key and sifts it up). Which of the following statements is/are TRUE?

- (A) In the final heap, the key 9 is a leaf
- (B) The final array is [40, 31, 27, 15, 22, 9, 12]
- (C) A total of 7 swaps are performed
- (D) Running bottom-up build-heap on the array [15, 22, 9, 31, 40, 12, 27] produces the same final array

Q.7 [MCQ · single correct | 1 mark | Hashing — quadratic probing failure]

A hash table has 10 slots (0-9). Key k is inserted using quadratic probing: the i -th probe ($i = 0, 1, 2, \dots$) examines slot $(k \bmod 10 + i^2) \bmod 10$. The keys 13, 23, 33, 43, 53, 63, 73 are inserted in this order into an empty table. What happens when 73 is inserted?

- (A) 73 is stored in slot 0
- (B) 73 is stored in slot 5
- (C) 73 is stored in slot 1
- (D) 73 cannot be inserted, although 4 slots are empty

Q.8 [NAT · numerical answer | 1 mark | Linear search — expected comparisons]

An unsorted array holds $n = 9$ distinct keys. A key x is searched by linear search from index 0. With probability $2/3$ the key is present, and if present it is equally likely to be at each of the 9 positions; with probability $1/3$ it is absent. The expected number of key comparisons (rounded off to two decimal places) is _____.

Answer: _____ (enter a numerical value)

Q.9 [MCQ · single correct | 1 mark | Insertion sort — intermediate state]

Insertion sort (increasing order) is applied to [31, 12, 47, 5, 26, 18]. One pass inserts the next element $A[i]$ ($i = 1, 2, \dots$) into the sorted prefix $A[0..i-1]$. The array after **three** passes is

- (A) [5, 12, 31, 47, 26, 18]
- (B) [12, 31, 47, 5, 26, 18]
- (C) [5, 12, 26, 31, 47, 18]
- (D) [5, 12, 18, 31, 26, 47]

Q.10 [MSQ · one or more correct | 1 mark | Graph theory — degrees and handshaking]

A simple undirected graph G has exactly 12 edges. Three of its vertices have degree 4 and every other vertex has degree 3. Which of the following statements is/are TRUE?

- (A) G must be connected
- (B) The complement of G has exactly 9 edges
- (C) G has exactly 7 vertices
- (D) G has exactly 3 vertices of odd degree

Q.11 [MCQ · single correct | 2 marks | Python — generators with try/finally]

Consider the following Python program. What is printed?

Python

```
1 def gen():
2     try:
3         for i in range(3):
4             yield i
5     finally:
6         print("F", end=" ")
7
8 g = gen()
9 print(next(g), end=" ")
10 print(sum(g), end=" ")
11 print(list(g))
```

- (A) 0 F 3 []
- (B) 0 3 F []
- (C) 0 3 [] F
- (D) 0 F 3 [0, 1, 2]

Q.12 [NAT · numerical answer | 2 marks | Counting BSTs by height]

Consider all binary search trees whose keys are exactly 1, 2, 3, 4, 5, 6 (there are 132 of them). The height of a tree is the number of **edges** on its longest root-to-leaf path. The number of these BSTs that have height exactly 4 is _____.

Answer: _____ (enter a numerical value)

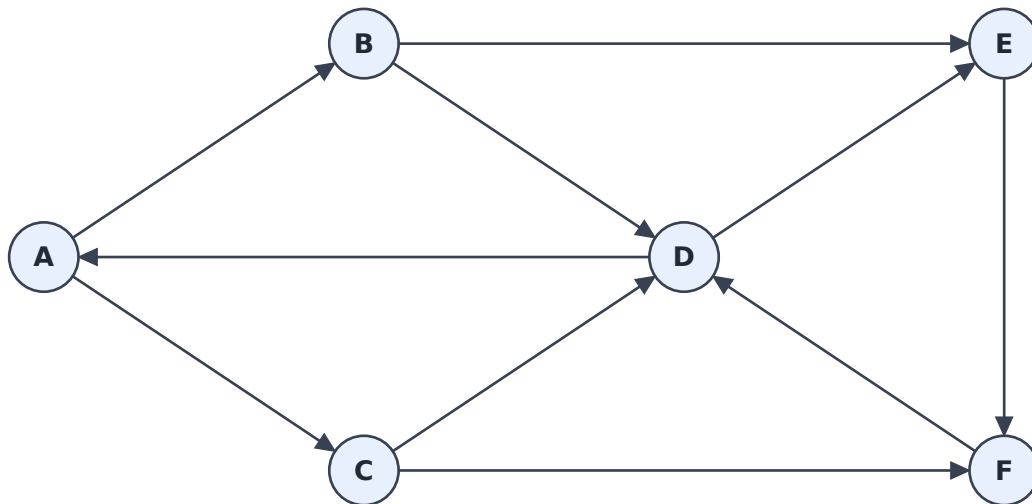
Q.13 [MSQ · one or more correct | 2 marks | Quicksort and merge sort — exact comparison counts]

Quicksort uses Lomuto partition with the **last** element as pivot (each partition of a sub-array of length m makes $m - 1$ comparisons). Merge sort is top-down and stops a merge as soon as one run is exhausted. Which of the following statements is/are TRUE for $n = 16$ distinct keys?

- (A) Quicksort on the decreasing array $[16, 15, \dots, 1]$ makes exactly 120 comparisons
- (B) Merge sort on the increasing array $[1, 2, \dots, 16]$ makes exactly 32 comparisons
- (C) Quicksort makes fewer comparisons on the increasing array $[1, 2, \dots, 16]$ than on the decreasing array
- (D) Merge sort on the decreasing array $[16, 15, \dots, 1]$ makes exactly 32 comparisons

Q.14 [MSQ · one or more correct | 2 marks | DFS — timestamps and edge classification]

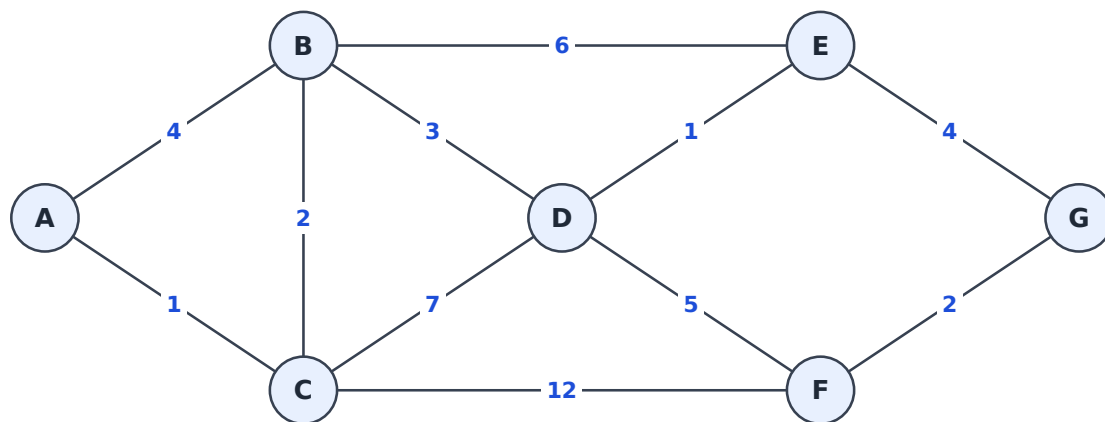
Depth-first search is run on the directed graph below starting at **A**; adjacency lists are scanned in **alphabetical order**, and a global clock starting at 1 is incremented at every discovery and every finish ($d(A) = 1$). Which of the following statements is/are TRUE?



- (A) The vertices of the graph can be topologically sorted
- (B) $B \rightarrow E$ is a cross edge
- (C) $d(C) = 10$ and $f(C) = 11$
- (D) Exactly two edges are back edges

Q.15 [MCQ · single correct | 2 marks | Dijkstra — extraction order]

Dijkstra's algorithm is run from **A** on the undirected weighted graph below. When two vertices have equal tentative distance, the alphabetically smaller is extracted first. In which order are the vertices extracted from the priority queue?



- (A) A, B, C, D, E, F, G
- (B) A, C, B, D, E, F, G
- (C) A, C, B, D, E, G, F
- (D) A, C, B, E, D, F, G

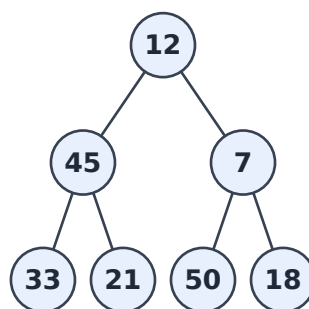
Q.16 [NAT · numerical answer | 2 marks | Hashing — expected number of empty slots]

Ten keys are inserted into a hash table with 10 slots using separate chaining. Assume simple uniform hashing: each key independently hashes to each slot with probability $1/10$. The expected number of slots that remain **empty** (rounded off to two decimal places) is _____.

Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Heapsort — trace]

Heapsort is applied to [12, 45, 7, 33, 21, 50, 18]: first bottom-up build-max-heap, then repeatedly swap the root with the last element of the heap, shrink the heap by one and sift the new root down (always toward the **larger** child). What is the array after build-max-heap and **two** such extraction steps?

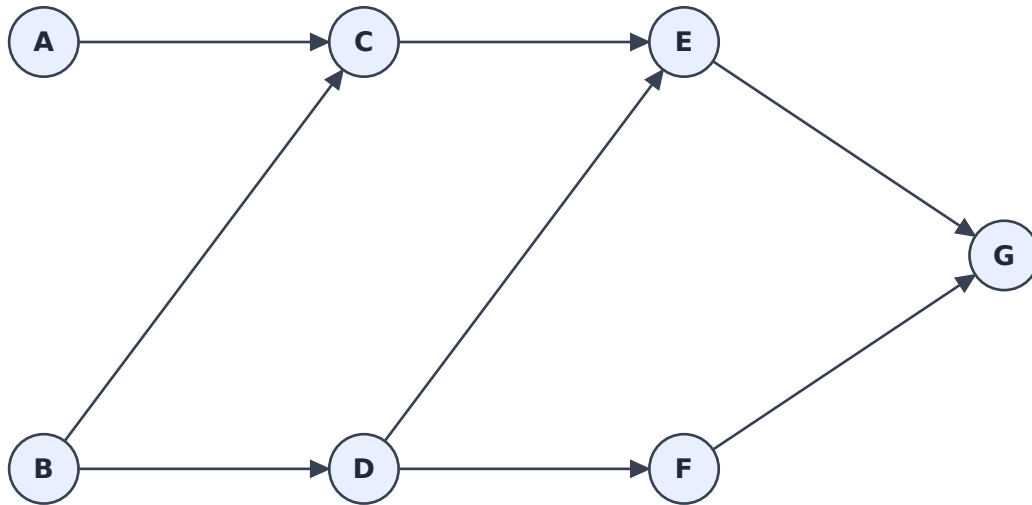


Input array viewed as a complete binary tree

- (A) [33, 21, 18, 7, 12, 45, 50]
- (B) [33, 21, 18, 12, 7, 45, 50]
- (C) [45, 33, 18, 12, 21, 7, 50]
- (D) [33, 12, 18, 21, 7, 45, 50]

Q.18 [NAT · numerical answer | 2 marks | DAGs — counting topological orders]

The number of distinct topological orderings of the directed acyclic graph below is _____.



Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Monotonic stack — next greater element]

Consider the following Python program. What is printed?

Python

```
1 def nge(a):
2     res = [-1] * len(a)
3     st = []          # stack of indices
4     for i, x in enumerate(a):
5         while st and a[st[-1]] < x:
6             res[st.pop()] = x
7             st.append(i)
8     return res
9
10 print(nge([4, 7, 3, 5, 9, 1, 6]))
```

- (A) [7, 9, 9, 9, -1, 6, -1]
- (B) [7, 9, 5, 9, -1, 6, 6]
- (C) [7, 9, 5, 9, -1, 6, -1]
- (D) [7, -1, 5, 9, -1, 6, -1]

Q.20 [NAT · numerical answer | 2 marks | Binary search on the answer]

Packages with weights w must be shipped **in the given order** within $D = 3$ days; each day a contiguous block of packages is loaded and its total weight may not exceed the ship's capacity c . The program below binary-searches for the minimum feasible c . The value printed is _____.

Python

```

1 w = [7, 2, 5, 10, 8, 3, 6]
2 D = 3
3
4 def days(c):
5     d, cur = 1, 0
6     for x in w:
7         if cur + x > c:
8             d, cur = d + 1, 0
9         cur += x
10    return d
11
12 lo, hi = max(w), sum(w)
13 while lo < hi:
14     mid = (lo + hi) // 2
15     if days(mid) <= D:
16         hi = mid
17     else:
18         lo = mid + 1
19 print(lo)

```

Answer: _____ (enter a numerical value)

Answer Key — Mock Test 39

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Python — closures and late binding
2	NAT	1	8	Stacks — infix to postfix conversion
3	MCQ	1	(C)	Python — deque rotations
4	NAT	1	5	Linked lists — Floyd's cycle detection
5	MSQ	1	(A), (B), (C)	BST — valid search sequences
6	MSQ	1	(A), (B), (C)	Heaps — successive insertion vs build-heap
7	MCQ	1	(D)	Hashing — quadratic probing failure
8	NAT	1	6.32 to 6.34	Linear search — expected comparisons
9	MCQ	1	(A)	Insertion sort — intermediate state
10	MSQ	1	(A), (B), (C)	Graph theory — degrees and handshaking
11	MCQ	2	(A)	Python — generators with try/finally
12	NAT	2	56	Counting BSTs by height
13	MSQ	2	(A), (B), (D)	Quicksort and merge sort — exact comparison counts
14	MSQ	2	(C), (D)	DFS — timestamps and edge classification
15	MCQ	2	(B)	Dijkstra — extraction order
16	NAT	2	3.48 to 3.50	Hashing — expected number of empty slots
17	MCQ	2	(B)	Heapsort — trace
18	NAT	2	14	DAGs — counting topological orders
19	MCQ	2	(C)	Monotonic stack — next greater element
20	NAT	2	17	Binary search on the answer

Detailed Solutions — Mock Test 39

Q.1 [MCQ · 1 mark · Python — closures and late binding]

Answer: (D)

A lambda's body looks up free variables **when it is called**, not when it is created (late binding). A default argument, by contrast, is evaluated **once, at definition time**.

- In `fs`, all three lambdas refer to the same variable `i` of the comprehension's scope. When they are finally called, the loop has finished and `i = 3`, so each returns $10 \times 3 = 30 \rightarrow [30, 30, 30]$.
- In `gs`, `i=i` copies the *current* value of `i` into each lambda's own default parameter (1, 2, 3) $\rightarrow [10, 20, 30]$.

Output `[30, 30, 30]` `[10, 20, 30]` $\rightarrow$ option (D).

- (A) assumes the closures captured values.
- (B) assumes the default-argument trick does not help.
- (C) swaps the two behaviours.

Trap: the comprehension has its own scope in Python 3, but that changes nothing — the closures share that scope's single `i`. `functools.partial` is another way to freeze values.

Q.2 [NAT · 1 mark · Stacks — infix to postfix conversion]

Answer: 8

Rule: on an operator, pop while the top has higher precedence, or equal precedence and the incoming operator is left-associative; then push. '(' is always pushed; ')' pops down to the matching '('.

- `a` $\rightarrow$ out; `-` $\rightarrow$ `[-]`; `b` $\rightarrow$ out.
- `*` (higher than `-`) $\rightarrow$ `[- *]`; `(` $\rightarrow$ `[- * (]`; `c` $\rightarrow$ out.
- `+` (top is '(') $\rightarrow$ `[- * ( +]`; `d` $\rightarrow$ out.
- `^` $\rightarrow$ `[- * ( + ^]`; `e` $\rightarrow$ out.
- `^` (equal, but right-associative $\rightarrow$ no pop) $\rightarrow$ `[- * ( + ^ ^]` (6).
- `(` $\rightarrow$ 7 symbols; `f` $\rightarrow$ out; `-` (top '(') $\rightarrow$ `[- * ( + ^ ^ ( -]` **$\rightarrow$ 8**.
- `g` $\rightarrow$ out; `)` pops `-` and the '(' $\rightarrow$ 6; `)` pops `^ ^ +` and '(' $\rightarrow$ `[- *]`.
- `/` pops `*` (equal, left-associative) $\rightarrow$ `[- /]`; `h` $\rightarrow$ out; end pops `/ -`.

Postfix: `a b c d e f g - ^ ^ + * h / -`. Maximum stack size = **8**.

Trap: treating `^` as left-associative pops the first `^` before pushing the second, giving a maximum of 7 (and a wrong postfix: `... d e ^ f g - ^ ...`).



operator stack at its peak
Stack contents just after pushing the inner `-`

Q.3 [MCQ · 1 mark · Python — deque rotations]**Answer: (C)**

`rotate(n)` with $n > 0$ moves the last n elements to the front (rotate **right**); $n < 0$ rotates left. `pop()` removes from the right end, `appendleft` adds at the left.

- Start: [1, 2, 3, 4, 5].
- `rotate(2)` → [4, 5, 1, 2, 3].
- `appendleft(pop())`: pop 3 from the right and put it in front → [3, 4, 5, 1, 2] (this is just one more right rotation).
- `extend([6, 7])` → [3, 4, 5, 1, 2, 6, 7].
- `rotate(-3)`: move the first three to the back → [1, 2, 6, 7, 3, 4, 5].

Output → option (C).

- (A) omits the final rotation.
- (B) rotates right by 3 at the end instead of left.
- (D) treats `rotate(2)` as a left rotation.

Tip: each deque operation used here is $O(1)$ except `rotate(k)`, which is $O(k)$ — far better than slicing a list, which is $O(n)$.

Q.4 [NAT · 1 mark · Linked lists — Floyd's cycle detection]**Answer: 5**

The list has a tail of $\mu = 3$ nodes (1, 2, 3) and a cycle of length $\lambda = 5$ ($4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$). After t iterations slow has made t moves and fast $2t$ moves; once both are in the cycle they coincide when the difference t is a multiple of λ . So the first meeting is at the smallest $t \geq \mu$ with $t \equiv 0 \pmod{\lambda}$, i.e. $t = 5$.

Trace (slow / fast):

- $t = 0$: 1 / 1 (start, not counted)
- $t = 1$: 2 / 3
- $t = 2$: 3 / 5
- $t = 3$: 4 / 7
- $t = 4$: 5 / 4 (fast went $7 \rightarrow 8 \rightarrow 4$)
- $t = 5$: 6 / 6 → **meet** after 5 iterations at node 6.

Floyd then finds the cycle start by moving one pointer back to the head and advancing both one step at a time: they meet after $\mu = 3$ more steps at node 4.

Trap: counting the initial equality at node 1 (both start there) as a meeting — the comparison happens only after moving.

Q.5 [MSQ · 1 mark · BST — valid search sequences]**Answer: (A), (B), (C)**

Maintain the open interval (low, high) the target must lie in: going right after key x sets $\text{low} = x$, going left sets $\text{high} = x$. Every later key on the path must lie inside the current interval.

- (A) $79 \rightarrow (-\infty, 79)$; $14 \rightarrow (14, 79)$; $72 \rightarrow (14, 72)$; $56 \rightarrow \text{left } (14, 56)$; $16 \rightarrow (16, 56)$; $53 \rightarrow (53, 56)$; $55 \checkmark$. **Valid.**
- (B) $10 \rightarrow \text{right } (10, \infty)$; $75 \rightarrow \text{left } (10, 75)$; $64 \rightarrow \text{left } (10, 64)$; $43 \rightarrow \text{right } (43, 64)$; $60 \rightarrow \text{left } (43, 60)$; $57 \rightarrow \text{left } (43, 57)$; $55 \checkmark$. **Valid.**
- (C) $90 \rightarrow \text{left } (-\infty, 90)$; $12 \rightarrow \text{right } (12, 90)$; $68 \rightarrow \text{left } (12, 68)$; $34 \rightarrow \text{right } (34, 68)$; $62 \rightarrow \text{left } (34, 62)$; $45 \rightarrow \text{right } (45, 62)$; $55 \checkmark$. **Valid.**
- (D) $9 \rightarrow (9, \infty)$; $85 \rightarrow (9, 85)$; $47 \rightarrow (47, 85)$; $68 \rightarrow (47, 68)$; next key 43 is **outside** (47, 68) — after going right at 47, no key below 47 can appear. **Invalid.**

Answer: (B), (C), (A).

Trap: checking only the direction at each step (55 vs current key) without tracking the interval inherited from all ancestors.

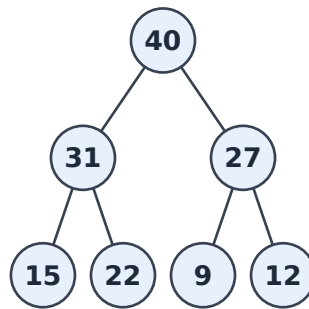
Q.6 [MSQ · 1 mark · Heaps — successive insertion vs build-heap]

Answer: (A), (B), (C)

Insertion = append at the end, then swap with the parent while larger.

- $15 \rightarrow [15]$.
- $22 \rightarrow$ swap with 15 $\rightarrow [22, 15]$ (1 swap).
- $9 \rightarrow [22, 15, 9]$.
- $31 \rightarrow$ idx 3, swap with 15, then with 22 $\rightarrow [31, 22, 9, 15]$ (2 swaps, total 3).
- $40 \rightarrow$ idx 4, swap with 22, then with 31 $\rightarrow [40, 31, 9, 15, 22]$ (2, total 5).
- $12 \rightarrow$ idx 5, swap with 9 $\rightarrow [40, 31, 12, 15, 22, 9]$ (1, total 6).
- $27 \rightarrow$ idx 6, swap with 12 $\rightarrow [40, 31, 27, 15, 22, 9, 12]$ (1, total 7).
- (A) **True.** 9 is at index $5 \geq \lfloor 7/2 \rfloor = 3$, so it is a leaf.
- (B) **True.** (C) **True.**
- (D) **False.** Build-heap sifts down from index 2: 9 vs children 12, 27 $\rightarrow$ swap with 27; index 1: 22 vs 31, 40 $\rightarrow$ swap with 40; index 0: 15 vs 40, 27 $\rightarrow$ 40, then 15 vs 31, 22 $\rightarrow$ 31. Result $[40, 31, 27, 15, 22, 12, 9]$ — 12 and 9 are in the opposite places.

Tip: the two construction methods give valid but generally different heaps; build-heap is $O(n)$ while n insertions are $O(n \log n)$ in the worst case.



Heap after the seven insertions

Q.7 [MCQ · 1 mark · Hashing — quadratic probing failure]

Answer: (D)

All keys have home slot 3. The offsets $i^2 \bmod 10$ for $i = 0, 1, 2, \dots$ are 0, 1, 4, 9, 6, 5, 6, 9, 4, 1, 0, ... — only **six** distinct values $\{0, 1, 4, 5, 6, 9\}$. So a key with home 3 can only ever probe slots 3, 4, 7, 2, 9, 8 ($= 3 + \text{offset} \bmod 10$).

- $13 \rightarrow 3$; $23 \rightarrow 3+1 = 4$; $33 \rightarrow 3+4 = 7$; $43 \rightarrow 3+9 = 12 \rightarrow 2$; $53 \rightarrow 3+16 = 19 \rightarrow 9$; $63 \rightarrow 3+25 = 28 \rightarrow 8$.
- 73: probes 3, 4, 7, 2, 9, 8, 8, 9, 2, 7, 3, ... — all six reachable slots are full, and the sequence repeats forever. Slots 0, 1, 5, 6 are empty but **unreachable**.

Hence the insertion fails although the table is only 60 % full $\rightarrow$ option (D).

Options (A), (B), (C) name empty slots that the probe sequence for home slot 3 never visits.

Tip: quadratic probing is guaranteed to find an empty slot only under conditions such as m prime and load factor $\leq 1/2$ (then the first $\lfloor m/2 \rfloor$ probes are distinct).

0	
1	
2	43
3	13
4	23
5	
6	
7	33
8	63
9	53

Table before inserting 73 (slots 0, 1, 5, 6 unreachable)

Q.8 [NAT · 1 mark · Linear search — expected comparisons]**Answer: 6.32 to 6.34**

Use the law of total expectation over the two cases.

- **Present** (probability $2/3$): if x is at position i (1-based), linear search makes i comparisons. Averaging over $i = 1 \dots 9$: $(1 + 2 + \dots + 9)/9 = 45/9 = 5 = (n + 1)/2$.
- **Absent** (probability $1/3$): all $n = 9$ elements are compared $\rightarrow 9$.

$$E = (2/3)(5) + (1/3)(9) = 10/3 + 3 = 19/3 \approx \mathbf{6.33}.$$

General formula: $E = p(n + 1)/2 + (1 - p)n$. For $p = 1$ this is $(n + 1)/2$, for $p = 0$ it is n .

Trap: averaging 5 and 9 with equal weights (7) — the probabilities are $2/3$ and $1/3$, not $1/2$ each. Another slip is using $n/2 = 4.5$ instead of $(n + 1)/2$ for the successful case.

Q.9 [MCQ · 1 mark · Insertion sort — intermediate state]**Answer: (A)**

Insertion sort keeps $A[0..i]$ sorted after pass i ; the suffix is untouched.

- Pass 1 ($i = 1$, insert 12): 31 shifts right $\rightarrow [12, 31, 47, 5, 26, 18]$.
- Pass 2 ($i = 2$, insert 47): already larger than 31 $\rightarrow$ no shift.
- Pass 3 ($i = 3$, insert 5): 47, 31, 12 shift right $\rightarrow [5, 12, 31, 47, 26, 18]$.

Answer (A).

- (B) is the state after only two passes.
- (C) is after four passes (26 inserted).
- (D) is **selection** sort after three passes (minimums 5, 12, 18 swapped into place) — selection sort fixes final positions, insertion sort only orders the prefix.

Shifts so far: $1 + 0 + 3 = 4 =$ the number of inversions among the first four elements.

Tip: after k passes, insertion sort's prefix holds the *first* $k + 1$ input elements in sorted order, whereas selection sort's prefix holds the k *smallest* elements overall.

Q.10 [MSQ · 1 mark · Graph theory — degrees and handshaking]**Answer: (A), (B), (C)**

Handshaking lemma: the sum of degrees equals $2|E|$.

- (A) **True.** In a simple graph a vertex of degree ≥ 3 needs at least 3 neighbours in its component, so every component has ≥ 4 vertices. Two components would need ≥ 8 vertices, but G has only 7 — so G is connected.

- (B) **True.** K_7 has $C(7, 2) = 21$ edges, so the complement has $21 - 12 = 9$ edges.
- (C) **True.** $3 \cdot 4 + 3(n - 3) = 24 \rightarrow 3n + 3 = 24 \rightarrow n = 7$.
- (D) **False.** The degree-3 vertices are the odd ones: $7 - 3 = 4$ of them (the number of odd-degree vertices is always even).

Answer: (C), (B), (A).

Trap: assuming (A) needs a specific drawing. The counting argument (minimum component size $\delta + 1$) settles it for every such graph; the verifier also checks all 19 355 labelled graphs with this degree sequence.

Q.11 [MCQ · 2 marks · Python — generators with try/finally]

Answer: (A)

A generator runs its body lazily, pausing at each **yield**. The **finally** block runs when the body actually finishes — here, when the loop ends after the last value is consumed.

- **next(g)** runs up to the first **yield** $\rightarrow 0$. Printed: **0**.
- **sum(g)** must evaluate completely **before** the outer **print** can print its result. It pulls 1 and 2; asking for the next value resumes the body, the loop ends, the **finally** block prints **F** and then StopIteration ends the sum. Only now is $1 + 2 = 3$ printed. Output so far: **0 F 3**.
- **list(g)** on an exhausted generator $\rightarrow []$.

Full output: **0 F 3 []** $\rightarrow$ option (A).

- (B) assumes **finally** runs after the value 3 is printed — but arguments are evaluated before **print** executes.
- (C) delays **finally** until the program ends (it would only do that if the generator were never exhausted and was garbage-collected late).
- (D) assumes the generator restarts.

Trap: side effects inside generators happen at *consumption* time, interleaved with the consumer's own output.

Q.12 [NAT · 2 marks · Counting BSTs by height]

Answer: 56

Let $N(n, h)$ = number of BSTs on n keys with height **at most** h ($N(0, h) = 1$ for the empty tree, $N(n, -1) = 0$ for $n \geq 1$). Choosing root r leaves $r - 1$ keys on the left and $n - r$ on the right, each of height $\leq h - 1$:

$$N(n, h) = \sum_{r=1}^n N(r-1, h-1) \cdot N(n-r, h-1).$$

Then (height exactly 4) = $N(6, 4) - N(6, 3)$.

- $N(6, 4)$: left/right sizes (0,5), (1,4), (2,3), (3,2), (4,1), (5,0) with $h \leq 3$: $26 + 14 + 10 + 10 + 14 + 26 = 100$.
- $N(6, 3)$: same splits with $h \leq 2$: $6 + 6 + 10 + 10 + 6 + 6 = 44$.

Answer = $100 - 44 = 56$.

Cross-check: the 132 trees split by height as 4 (height 2) + 40 (height 3) + 56 (height 4) + 32 (height 5). Height 5 means a path ('zig-zag chain'): each of the 5 non-root levels chooses left or right $\rightarrow 2^5 = 32$, and $132 - 32 - 44 = 56$. ✓

Trap: using the node-count definition of height, which shifts every value by one and would make 'height 4' mean edges = 3 (40 trees).

$N(n, h)$ = BSTs on n keys with height $\leq h$

	$h=0$	$h=1$	$h=2$	$h=3$	$h=4$	$h=5$
$n=1$	1	1	1	1	1	1
$n=2$	0	2	2	2	2	2
$n=3$	0	1	5	5	5	5
$n=4$	0	0	6	14	14	14
$n=5$	0	0	6	26	42	42
$n=6$	0	0	4	44	100	132

Q.13 [MSQ · 2 marks · Quicksort and merge sort — exact comparison counts]

Answer: (A), (B), (D)

- (A) **True.** Decreasing input: pivot 1 is the minimum $\rightarrow$ partition gives sizes 0 and 15; the 15 remaining keys [15, ..., 2, 16] — after the swaps the right part has pivot 16 (max) $\rightarrow$ sizes 14 and 0, and so on. Every partition is maximally unbalanced, so the total is $15 + 14 + \dots + 1 = 16 \cdot 15/2 = 120$.
- (B) **True.** On sorted input, every merge of two runs of length k compares the k elements of the left run with the first element of the right run and then stops: k comparisons. Each level costs $n/2 = 8$, and there are $\log_2 16 = 4$ levels $\rightarrow 32 = (n/2) \log_2 n$ — the best case.
- (C) **False.** With the last element as pivot, sorted input makes the pivot the maximum every time $\rightarrow$ also 120 comparisons, the same as (A).
- (D) **True.** On reversed input every right run is entirely smaller, so the right run is exhausted after k comparisons: again 8 per level $\rightarrow 32$.

Answer: (A), (B), (D).

Insight: merge sort's comparison count varies only between $(n/2)\log_2 n = 32$ and $n \log_2 n - n + 1 = 49$ for $n = 16$, whereas quicksort's count can climb all the way to $n(n-1)/2 = 120$.

Q.14 [MSQ · 2 marks · DFS — timestamps and edge classification]

Answer: (C), (D)

Classification for an edge $u \rightarrow v$ met while scanning u : v undiscovered $\rightarrow$ **tree**; v discovered but unfinished (on the recursion stack) $\rightarrow$ **back**; v finished with $d(u) < d(v) \rightarrow$ **forward**; v finished with $d(u) > d(v) \rightarrow$ **cross**.

Trace:

- A $d=1 \rightarrow$ B $d=2 \rightarrow$ (B's list: D, E) D $d=3$.
- D: A is on the stack $\rightarrow D \rightarrow A$ **back**; E undiscovered $\rightarrow$ E $d=4$.
- E $\rightarrow$ F $d=5$. F: D is on the stack $\rightarrow F \rightarrow D$ **back**. F finishes $f=6$; E $f=7$; D $f=8$.
- Back at B: E is finished and $d(B)=2 < d(E)=4 \rightarrow B \rightarrow E$ **forward**. B $f=9$.
- Back at A: C $d=10$. C $\rightarrow$ D: D finished, $d(C)=10 > 3 \rightarrow$ **cross**; C $\rightarrow$ F $\rightarrow$ **cross**. C $f=11$; A $f=12$.

Tree edges: A $\rightarrow$ B, B $\rightarrow$ D, D $\rightarrow$ E, E $\rightarrow$ F, A $\rightarrow$ C.

- (A) **False** — a back edge means a directed cycle (A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ A), so no topological order.
- (B) **False** — B $\rightarrow$ E is a forward edge (E is a descendant of B reached via D).
- (C) **True** — $d(C) = 10$, $f(C) = 11$.
- (D) **True** — D $\rightarrow$ A and F $\rightarrow$ D.

Trap: calling B $\rightarrow$ E a tree edge because it is drawn directly; with alphabetical order D is explored first and reaches E before B gets to scan E.

		Discovery / finish times					
		A	B	C	D	E	F
d		1	2	10	3	4	5
f		12	9	11	8	7	6

Q.15 [MCQ · 2 marks · Dijkstra — extraction order]**Answer: (B)**

Dijkstra extracts vertices in non-decreasing order of their final distances (ties by the stated rule).

- A (0): B = 4, C = 1.
- C (1): B = $\min(4, 1+2) = 3$; D = $1+7 = 8$; F = $1+12 = 13$.
- B (3): D = $\min(8, 3+3) = 6$; E = $3+6 = 9$.
- D (6): E = $\min(9, 6+1) = 7$; F = $\min(13, 6+5) = 11$.
- E (7): G = $7+4 = 11$.
- Now F = 11 and G = 11 tie → F first (alphabetical); F: G = $\min(11, 11+2)$ stays 11.
- G (11).

Order **A, C, B, D, E, F, G** → option (B). Final distances: A0 C1 B3 D6 E7 F11 G11.

- (A) extracts B before C — B's initial label 4 is not its final distance 3, and C (1) is smaller anyway.
- (C) breaks the F/G tie the wrong way.
- (D) extracts E (tentative 9 at that time) before D (6).

Trap: the direct edges A-B and C-F are both 'shortcuts' that lose to detours; never fix a label before its vertex is extracted.

Q.16 [NAT · 2 marks · Hashing — expected number of empty slots]**Answer: 3.48 to 3.50**

Use **linearity of expectation** with indicator variables. Let $X_j = 1$ if slot j is empty.

- A particular key misses slot j with probability $1 - 1/10 = 0.9$.
- All 10 keys miss slot j (independently) with probability 0.9^{10} .
- $E[X_j] = 0.9^{10} \approx 0.348678$.

$$E[\# \text{ empty slots}] = \sum_j E[X_j] = 10 \times 0.9^{10} \approx 10 \times 0.348678 = \mathbf{3.49}.$$

So even with load factor $\alpha = 1$, about 35 % of the slots are empty and the 10 keys crowd into ≈ 6.5 slots — collisions are unavoidable (for large m the fraction tends to $e^{-\alpha} \approx 0.368$).

Related quantities: expected number of colliding pairs = $C(10, 2)/10 = 4.5$; expected chain length seen by an unsuccessful search = $\alpha = 1$.

Trap: assuming that 10 keys in 10 slots 'fill' the table (0 empty), or computing $(1 - 1/10) \cdot 10 = 9$ by forgetting that all ten keys must miss the slot.

Q.17 [MCQ · 2 marks · Heapsort — trace]**Answer: (B)**

Build-max-heap (sift down indices 2, 1, 0):

- $i = 2$: 7 vs children 50, 18 → swap with 50 → [12, 45, 50, 33, 21, 7, 18].
- $i = 1$: 45 vs 33, 21 → already a heap.
- $i = 0$: 12 vs 45, 50 → swap with 50; at index 2, 12 vs 7, 18 → swap with 18 → [50, 45, 18, 33, 21, 7, 12].

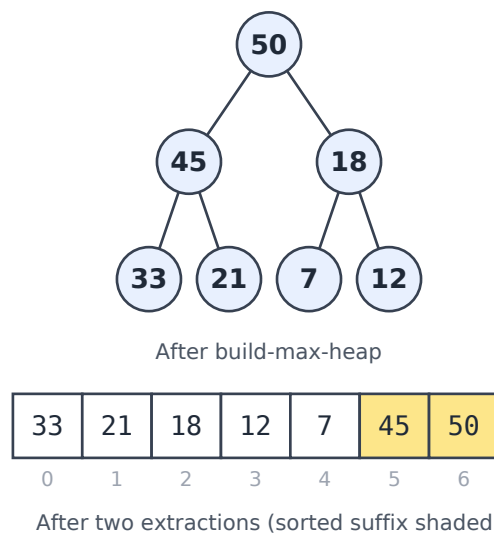
Extraction 1: swap 50 ↔ 12 → [12, 45, 18, 33, 21, 7 | 50]; sift 12: children 45, 18 → 45; then children 33, 21 → 33 → [45, 33, 18, 12, 21, 7 | 50].

Extraction 2: swap $45 \leftrightarrow 7 \rightarrow [7, 33, 18, 12, 21 \mid 45, 50]$; sift 7: children 33, 18 $\rightarrow 33$; then children 12, 21 $\rightarrow 21 \rightarrow [33, 21, 18, 12, 7 \mid 45, 50]$.

Answer (B).

- (A) sifts 7 toward the *smaller* child 12 in the second extraction.
- (C) is the state after only one extraction.
- (D) stops the second sift after one level and mis-orders 12 and 21.

Tip: after k extractions the last k cells hold the k largest keys in sorted order — a quick sanity check (45, 50 here).



Q.18 [NAT · 2 marks · DAGs — counting topological orders]

Answer: 14

Every vertex reaches G (via E or F), so G is always **last**; count orders of A-F.

Ignore A first. B must precede C and D, so B comes first among $\{B, C, D, E, F\}$. The remaining constraints are $C < E$, $D < E$, $D < F$. Orders of $\{C, D, E, F\}$:

- C D E F, C D F E, D C E F, D C F E, D F C E $\rightarrow$ **5** orders.

Now insert A; its only constraint is $A < C$. For each order (with B in front), count the slots before C:

- B C D E F $\rightarrow$ A can go before B or between B and C: 2.
- B C D F E $\rightarrow$ 2.
- B D C E F $\rightarrow$ before B, after B, after D: 3.
- B D C F E $\rightarrow$ 3.
- B D F C E $\rightarrow$ 4.

Total = $2 + 2 + 3 + 3 + 4 =$ **14**.

Trap: treating A and B symmetrically (both are sources) — B has more successors (C and D), so it is far more constrained than A.

Q.19 [MCQ · 2 marks · Monotonic stack — next greater element]

Answer: (C)

The stack holds indices whose next greater element is still unknown; their values are decreasing from bottom to top. A new x pops every smaller value — x is their answer.

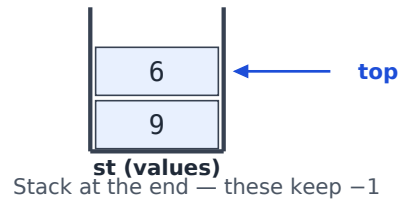
- $i=0$ (4): push $\rightarrow$ st = [4].
- $i=1$ (7): pop 4 $\rightarrow$ res[0] = 7; push $\rightarrow$ [7].
- $i=2$ (3): push $\rightarrow$ [7, 3].
- $i=3$ (5): pop 3 $\rightarrow$ res[2] = 5; $7 > 5$ stop; push $\rightarrow$ [7, 5].

- $i=4$ (9): $\text{pop } 5 \rightarrow \text{res}[3] = 9$; $\text{pop } 7 \rightarrow \text{res}[1] = 9$; $\text{push} \rightarrow [9]$.
- $i=5$ (1): $\text{push} \rightarrow [9, 1]$.
- $i=6$ (6): $\text{pop } 1 \rightarrow \text{res}[5] = 6$; $\text{push} \rightarrow [9, 6]$.
- End: 9 and 6 remain $\rightarrow -1$.

Output $[7, 9, 5, 9, -1, 6, -1] \rightarrow$ option (C).

- (A) gives the *maximum to the right* for index 2, not the next greater.
- (B) assigns the last element its own value.
- (D) misses that 7 is popped by 9.

Complexity: each index is pushed and popped at most once $\rightarrow \Theta(n)$ total, even though there is a nested **while** loop (amortised analysis).



Q.20 [NAT · 2 marks · Binary search on the answer]

Answer: 17

days(c) is non-increasing in c , so feasibility is monotone and the smallest feasible c can be found by a lower-bound binary search over $[\max(w), \text{sum}(w)] = [10, 41]$.

- $\text{lo}=10, \text{hi}=41$: $\text{mid}=25 \rightarrow \text{loads } 7+2+5+10 = 24 \mid 8+3+6 = 17 \rightarrow 2 \text{ days} \leq 3 \rightarrow \text{hi} = 25$.
- $\text{lo}=10, \text{hi}=25$: $\text{mid}=17 \rightarrow 7+2+5 = 14 \mid 10 \mid 8+3+6 = 17 \rightarrow 3 \text{ days} \rightarrow \text{hi} = 17$.
- $\text{lo}=10, \text{hi}=17$: $\text{mid}=13 \rightarrow 9 \mid 5 \mid 10 \mid 8+3 \mid 6 \rightarrow 5 \text{ days} \rightarrow \text{lo} = 14$.
- $\text{lo}=14, \text{hi}=17$: $\text{mid}=15 \rightarrow 14 \mid 10 \mid 11 \mid 6 \rightarrow 4 \text{ days} \rightarrow \text{lo} = 16$.
- $\text{lo}=16, \text{hi}=17$: $\text{mid}=16 \rightarrow 14 \mid 10 \mid 11 \mid 6 \rightarrow 4 \text{ days} \rightarrow \text{lo} = 17$.
- $\text{lo} = \text{hi} = 17 \rightarrow$ print **17**.

Check: $c = 17$ works ($14 \mid 10 \mid 17$) and $c = 16$ does not (the last block $8 + 3 + 6 = 17$ must be split).

Cost: $O(n \log(\sum w))$ — 5 iterations of an $O(n)$ check here.

Trap: starting with $\text{lo} = 1$ is still correct but slower; starting with $\text{lo} = \text{sum}/D \approx 13.7$ and *rounding down* is fine too, but returning **mid** instead of **lo** can be off by one.

Mock Test 40

Full-Syllabus Mock — Paper 10

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — floor division, modulo and rounding]

What does the following Python statement print?

Python

```
1 print(-17 // 5, -17 % 5, 17 // -5, round(-2.5), int(-2.5))
```

- (A) -4 3 -4 -2 -2
- (B) -3 -2 -3 -3 -2
- (C) -4 3 -3 -2 -3
- (D) -4 -2 -4 -3 -2

Q.2 [NAT · numerical answer | 1 mark | Python — nested list comprehensions]

What is printed by the following program?

Python

```
1 m = [[i * j for j in range(i)] for i in range(5)]
2 print(sum(map(sum, m)))
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks and queues — simulation]

Consider the following Python program. What is printed?

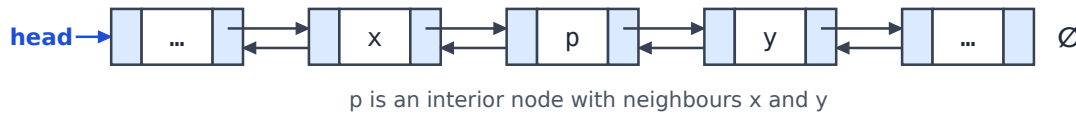
Python

```
1 from collections import deque
2 s, q = [], deque()
3 for x in range(1, 9):
4     s.append(x)
5     if x % 3 == 0:
6         q.append(s.pop()); q.append(s.pop())
7     if x % 4 == 0:
8         s.append(q.popleft())
9 print(list(q), s)
```

- (A) [5, 6] [1, 4, 3, 7, 8, 2]
- (B) [2, 6, 5] [1, 4, 3, 7, 8]
- (C) [6, 5] [1, 4, 2, 7, 8, 3]
- (D) [6, 5] [1, 4, 3, 7, 8, 2]

Q.4 [MSQ · one or more correct | 1 mark | Doubly linked lists — deleting a node]

In a doubly linked list each node has fields `prev` and `next`. Let `p` point to a node that is neither the first nor the last node. Which of the following code fragments correctly unlink `p` from the list (so that a forward and a backward traversal both skip `p`)?



- (A) `p.prev.next = p.next; p.prev.next.prev = p.prev`
- (B) `p.next.prev = p.prev; p.prev.next = p.next`
- (C) `p.prev = p.next; p.next.prev = p.prev`
- (D) `p.prev.next = p.next; p.next.prev = p.prev`

Q.5 [NAT · numerical answer | 1 mark | Binary heaps — insertion swaps]

The keys 30, 20, 25, 10, 15, 5, 22 are inserted one at a time, in this order, into an initially empty binary **min-heap** (array based; each insertion appends the key and sifts it up, swapping with its parent while the parent is larger). The total number of swaps performed is ____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Binary search — minimum of a rotated array]

Consider the following Python program. What is printed?

Python

```
1 def find_min(a):
2     lo, hi, it = 0, len(a) - 1, 0
3     while lo < hi:
4         it += 1
5         mid = (lo + hi) // 2
6         if a[mid] > a[hi]:
7             lo = mid + 1
8         else:
9             hi = mid
10    return lo, a[lo], it
11
12 print(find_min([24, 31, 2, 5, 9, 13, 17, 20]))
```

- (A) (1, 31, 3)
- (B) (2, 2, 4)
- (C) (2, 2, 3)
- (D) (2, 2, 2)

Q.7 [NAT · numerical answer | 1 mark | Hashing — linear probing with deletion markers]

Keys 21, 31, 41, 12, 52, 3 are inserted in this order into an empty table of size 10 with $h(k) = k \bmod 10$ and linear probing. Then 31 is deleted by placing a DELETED marker in its slot (searches continue past DELETED and stop at an EMPTY slot). Let `x` be the number of slots examined when searching for 52, and `y` the number examined in an (unsuccessful) search for 72. The value of `x + y` is ____.

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Elementary sorts on a reversed array]

The array 6, 5, 4, 3, 2, 1 is sorted into ascending order. Selection sort swaps only when the minimum is not already in place; bubble sort and insertion sort are the standard versions (insertion sort compares $A[j] > \text{key}$ while $j \geq 0$). Which of the following statements is/are TRUE?

- (A) Bubble sort performs exactly 15 swaps
- (B) Insertion sort performs exactly 20 key comparisons
- (C) Selection sort performs exactly 15 key comparisons
- (D) Selection sort performs exactly 3 swaps

Q.9 [NAT · numerical answer | 1 mark | Graph theory — handshaking lemma]

A simple undirected graph has 12 edges. Exactly three of its vertices have degree 4 and every other vertex has degree 3. The number of vertices in the graph is _____.

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Binary trees — reconstruction from traversals]

A binary tree with 12 nodes has
pre-order: G B Q A C K F P D E R H
in-order: Q B K C F A G P E D H R
Its post-order traversal is

- (A) Q K F C A B H R E D P G
- (B) Q K F C A B E H R D P G
- (C) K F C Q A B E H R D P G
- (D) Q K C F A B E H R D P G

Q.11 [MCQ · single correct | 2 marks | Python — generators and iterator exhaustion]

Consider the following Python program. What is printed?

Python

```
1 g = (x * x for x in range(5))
2 a = list(zip(g, g))
3 b = sum(g)
4 it = iter([1, 2, 3, 4])
5 c = [x for x in it if x % 2 == 0] + list(it)
6 print(a, b, c)
```

- (A) [(0, 1), (4, 9)] 0 [2, 4]
- (B) [(0, 1), (4, 9)] 16 [2, 4]
- (C) [(0, 0), (1, 1), (4, 4), (9, 9), (16, 16)] 30 [2, 4, 1, 2, 3, 4]
- (D) [(0, 1), (4, 9)] 0 [2, 4, 1, 2, 3, 4]

Q.12 [NAT · numerical answer | 2 marks | Python — try / except / finally with return]

What value does the following program print?

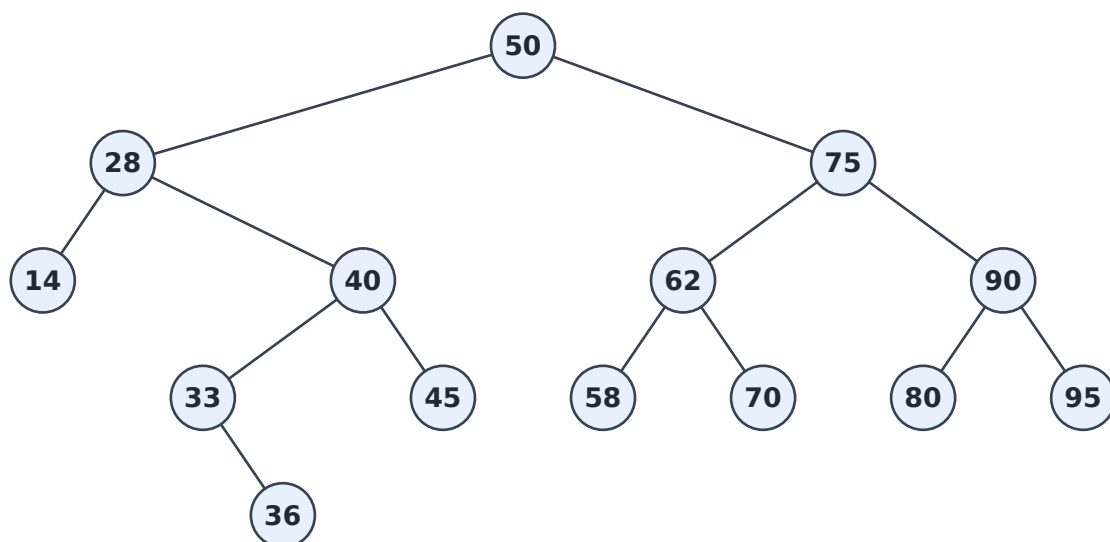
Python

```
1 def f(x):
2     try:
3         return 10 // x
4     except ZeroDivisionError:
5         return -1
6     finally:
7         if x == 2:
8             return 99
9
10 print(sum(f(x) for x in [0, 1, 2, 5]))
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | BST — deletion with in-order successor]

In the BST below, key 28 is deleted and then key 75 is deleted. A node with two children is deleted by copying its **in-order successor** into it and then deleting the successor from the right subtree. Which of the following statements about the final tree is/are TRUE? (Height = number of edges on the longest root-to-leaf path.)



Initial BST

- (A) 36 is the left child of 40
- (B) The left child of the root is 36
- (C) The height of the tree is 3
- (D) The pre-order traversal is 50, 33, 14, 40, 36, 45, 80, 62, 58, 70, 90, 95

Q.14 [NAT · numerical answer | 2 marks | Bottom-up merge sort — comparisons]

Bottom-up (iterative) merge sort is applied to the 11-element array below. In the pass with run width $w = 1, 2, 4, 8$ it merges $A[lo..lo+w-1]$ with $A[lo+w..lo+2w-1]$ for $lo = 0, 2w, 4w, \dots$ (a final run with no partner is left as it is). A merge stops comparing as soon as one run is exhausted. The total number of element comparisons is _____.

A =

12	7	3	18	9	1	15	4	11	6	2
0	1	2	3	4	5	6	7	8	9	10

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Quicksort — Hoare partition]

The Hoare partition below is called as `hoare(A, 0, 8)` on `A = [11, 4, 17, 8, 2, 19, 6, 14, 9]`. Which option gives the returned index and the array immediately after the call?

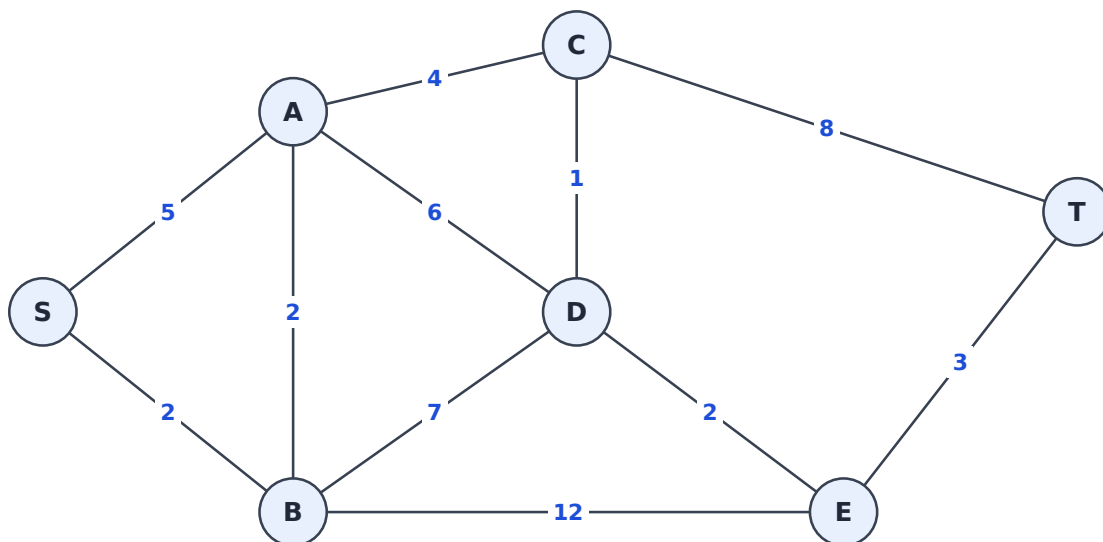
Python

```
1 def hoare(A, lo, hi):
2     p = A[lo]
3     i, j = lo - 1, hi + 1
4     while True:
5         i += 1
6         while A[i] < p:
7             i += 1
8         j -= 1
9         while A[j] > p:
10            j -= 1
11        if i >= j:
12            return j
13        A[i], A[j] = A[j], A[i]
```

- (A) 5 and [9, 4, 6, 8, 2, 19, 17, 14, 11]
- (B) 4 and [9, 4, 6, 8, 2, 19, 17, 14, 11]
- (C) 4 and [9, 4, 6, 8, 2, 11, 17, 14, 19]
- (D) 5 and [2, 4, 6, 8, 9, 11, 19, 17, 14]

Q.16 [NAT · numerical answer | 2 marks | Dijkstra — sum of shortest distances]

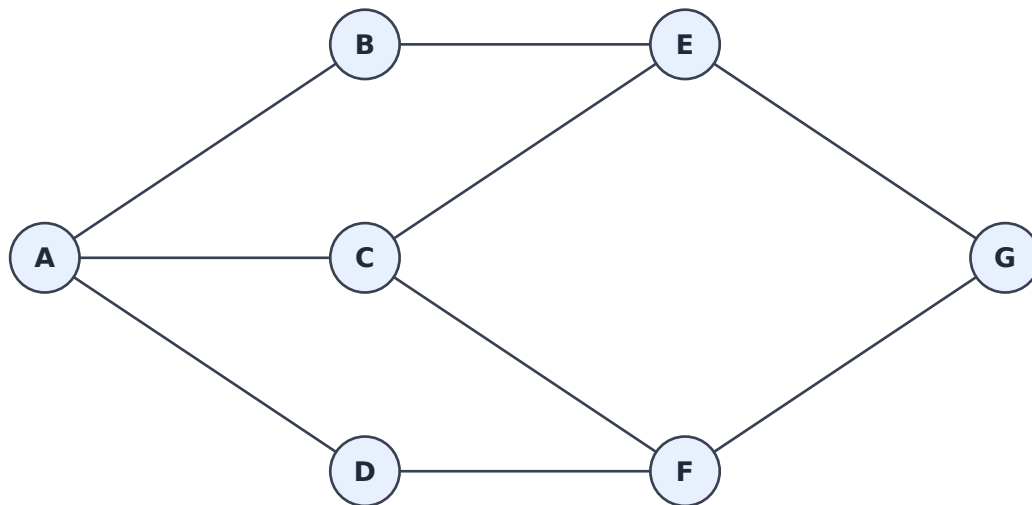
Dijkstra's algorithm is run from S on the undirected weighted graph below. The sum of the shortest-path distances from S to all seven vertices (including $d(S) = 0$) is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | DFS — valid traversal orders]

Depth-first search is started at A on the undirected graph below, with **no** fixed rule for the order in which neighbours are tried. Which of the following are possible DFS visiting (discovery) orders?



- (A) A B E C D F G
- (B) A C E B G F D
- (C) A B E G F C D
- (D) A D F G C E B

Q.18 [NAT · numerical answer | 2 marks | Queues — round-robin scheduling simulation]

Five processes are scheduled with round robin using a FIFO ready queue and time quantum 3:
 P1 (arrival 0, burst 7), P2 (1, 4), P3 (2, 9), P4 (4, 3), P5 (6, 5).

When a quantum ends, processes that have arrived up to and including that instant are enqueued **before** the preempted process is put back at the rear. Context switches take no time. The average turnaround time (completion – arrival) is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — reversing in groups of k]

Consider the following Python program. What is printed?

Python

```

1 class Node:
2     def __init__(self, v, nxt=None):
3         self.v, self.nxt = v, nxt
4
5 def rev_k(head, k):
6     cur, n = head, 0
7     while cur and n < k:
8         cur = cur.nxt; n += 1
9     if n < k:
10        return head
11    prev, cur = rev_k(cur, k), head
12    for _ in range(k):
13        cur.nxt, prev, cur = prev, cur, cur.nxt
14    return prev
15
16 h = None
17 for x in reversed(range(1, 9)):
18     h = Node(x, h)
19 h = rev_k(h, 3)
20 out = []
21 while h:
22     out.append(h.v); h = h.nxt
23 print(*out)

```



- (A) 3 2 1 6 5 4 8 7
- (B) 8 7 6 5 4 3 2 1
- (C) 3 2 1 6 5 4 7 8
- (D) 1 2 3 4 5 6 7 8

Q.20 [MSQ · one or more correct | 2 marks | Hashing — linear probing analysis]

Keys 8, 19, 13, 2, 25, 30, 6 are inserted in this order into an empty hash table with 11 slots (0–10) using $h(k) = (2k + 3) \bmod 11$ and linear probing (step +1, wrapping around). A *probe* is one slot examination; a successful first examination counts as 1 probe. Which of the following statements is/are TRUE?

- (A) The total number of probes over all seven insertions is 16
- (B) The longest run of consecutive occupied slots (with wrap-around) has length 6
- (C) If key 41 is inserted next, it is stored in slot 3
- (D) Key 30 is stored in slot 1

Answer Key — Mock Test 40

Q	Type	Marks	Key	Topic
1	MCQ	1	(A)	Python — floor division, modulo and rounding
2	NAT	1	35	Python — nested list comprehensions
3	MCQ	1	(D)	Stacks and queues — simulation
4	MSQ	1	(A), (B), (D)	Doubly linked lists — deleting a node
5	NAT	1	6	Binary heaps — insertion swaps
6	MCQ	1	(C)	Binary search — minimum of a rotated array
7	NAT	1	10	Hashing — linear probing with deletion markers
8	MSQ	1	(A), (C), (D)	Elementary sorts on a reversed array
9	NAT	1	7	Graph theory — handshaking lemma
10	MCQ	1	(B)	Binary trees — reconstruction from traversals
11	MCQ	2	(A)	Python — generators and iterator exhaustion
12	NAT	2	110	Python — try / except / finally with return
13	MSQ	2	(A), (C), (D)	BST — deletion with in-order successor
14	NAT	2	27	Bottom-up merge sort — comparisons
15	MCQ	2	(B)	Quicksort — Hoare partition
16	NAT	2	48	Dijkstra — sum of shortest distances
17	MSQ	2	(B), (C)	DFS — valid traversal orders
18	NAT	2	19.39 to 19.41	Queues — round-robin scheduling simulation
19	MCQ	2	(C)	Linked lists — reversing in groups of k
20	MSQ	2	(B), (D)	Hashing — linear probing analysis

Detailed Solutions — Mock Test 40

Q.1 [MCQ · 1 mark · Python — floor division, modulo and rounding]

Answer: (A)

Concept. Python's `//` floors toward $-\infty$ and `%` takes the sign of the divisor, so that `a == (a // b) * b + a % b` always holds. `round` uses banker's rounding (ties to even); `int` truncates toward 0.

- `-17 // 5 = ⌊-3.4⌋ = -4`.
- `-17 % 5 = -17 - (-4)(5) = 3`.
- `17 // -5 = ⌊-3.4⌋ = -4`.
- `round(-2.5)`: tie between -2 and -3 → the even one, -2.
- `int(-2.5)` truncates → -2.

Output `-4 3 -4 -2 -2` → (A).

Options. (B) is C/Java behaviour (truncating division, remainder with the sign of the dividend) plus round-half-away-from-zero. (C) truncates only the third division and floors `int`. (D) mixes a C-style remainder with half-away rounding.

Trap: `round(2.5)` is 2 and `round(3.5)` is 4 in Python 3.

Q.2 [NAT · 1 mark · Python — nested list comprehensions]

Answer: 35

Concept. The outer comprehension creates one row per `i`; the inner one ranges over `j = 0 ... i - 1`, so row `i` is a triangle-shaped list.

- `i = 0`: `[]` → 0
- `i = 1`: `[0]` → 0
- `i = 2`: `[0, 2]` → 2
- `i = 3`: `[0, 3, 6]` → 9
- `i = 4`: `[0, 4, 8, 12]` → 24

`map(sum, m)` gives the row sums 0, 0, 2, 9, 24, whose total is **35**.

Closed form: row `i` sums to `i·(0 + 1 + ... + (i-1)) = i²(i-1)/2`, giving 0, 0, 2, 9, 24.

Trap: reading `range(i)` as `range(5)` (a full 5×5 multiplication table, total 100), or as 1..`i` inclusive (total 65).

Q.3 [MCQ · 1 mark · Stacks and queues — simulation]

Answer: (D)

Concept. `s` is a LIFO stack (append/pop at the right) and `q` a FIFO queue (append right, popleft).

- `x = 1, 2, 3`: `s = [1, 2, 3]`; at 3, pop 3 then 2 into `q` → `q = [3, 2]`, `s = [1]`.
- `x = 4`: `s = [1, 4]`; `4 % 4 == 0` → move `q`'s front (3) to `s` → `s = [1, 4, 3]`, `q = [2]`.
- `x = 5`: `s = [1, 4, 3, 5]`.
- `x = 6`: `s = [..., 5, 6]`; pop 6 then 5 into `q` → `q = [2, 6, 5]`, `s = [1, 4, 3]`.
- `x = 7, 8`: `s = [1, 4, 3, 7, 8]`; at 8 move `q`'s front (2) → `s = [1, 4, 3, 7, 8, 2]`, `q = [6, 5]`.

Output `[6, 5] [1, 4, 3, 7, 8, 2]` → (D).

Options. (A) reverses the order of the two pops. (C) dequeues from the wrong end at `x = 4`. (B) forgets the second transfer at `x = 8`.

Trap: two consecutive pops from a stack deliver the elements in reverse order of pushing.

Q.4 [MSQ · 1 mark · Doubly linked lists — deleting a node]**Answer: (A), (B), (D)**

Concept. Unlinking p needs two updates: $x.next = y$ and $y.prev = x$, where $x = p.prev$ and $y = p.next$. Order does not matter as long as each right-hand side still refers to the intended node at the time it is evaluated.

- (A) After the first statement, $p.prev.next$ is y , so $p.prev.next.prev = p.prev$ sets $y.prev = x$. **Correct** (just an indirect way to reach y).
- (B) The same two assignments in the other order; p 's own fields are untouched, so both still refer to x and y . **Correct**.
- (C) The first statement overwrites $p.prev$ with y ; the second sets $y.prev = p.prev = y$ (a self-loop) and $x.next$ still points to p . **Wrong**.
- (D) $x.next = y$; $y.prev = x$. **Correct**.

Trap: modifying p 's own pointers before using them — as in (C) — loses the reference to a neighbour.

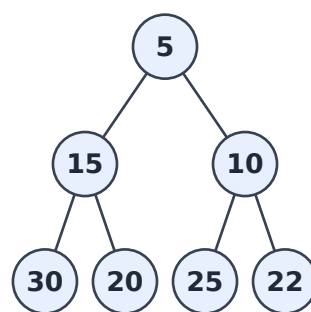
Q.5 [NAT · 1 mark · Binary heaps — insertion swaps]**Answer: 6**

Concept. Sift-up moves a new key upward past every larger ancestor; the number of swaps equals the number of levels it climbs.

- 30: [30] — 0 swaps.
- 20: child of 30 → swap → [20, 30] — 1.
- 25: child of 20 → no swap → [20, 30, 25] — 0.
- 10: index 3, parent 30 → swap; parent 20 → swap → [10, 20, 25, 30] — 2.
- 15: index 4, parent 20 → swap; parent 10 → stop → [10, 15, 25, 30, 20] — 1.
- 5: index 5, parent 25 → swap; parent 10 → swap → [5, 15, 10, 30, 20, 25] — 2.
- 22: index 6, parent 10 → stop — 0.

Total = 0 + 1 + 0 + 2 + 1 + 2 + 0 = **6**. Final heap: [5, 15, 10, 30, 20, 25, 22].

Trap: building with bottom-up heapify instead of repeated insertion gives a *different* heap and a different swap count — read which method is used.



Final min-heap

Q.6 [MCQ · 1 mark · Binary search — minimum of a rotated array]**Answer: (C)**

Concept. In a rotated sorted array, comparing $a[mid]$ with $a[hi]$ tells which half holds the rotation point: $a[mid] > a[hi]$ means the minimum is strictly right of mid ; otherwise it is at mid or to its left.

- $(lo, hi) = (0, 7)$: $mid\ 3$, $a[3] = 5 \leq a[7] = 20 \rightarrow hi = 3$.
- $(0, 3)$: $mid\ 1$, $a[1] = 31 > a[3] = 5 \rightarrow lo = 2$.
- $(2, 3)$: $mid\ 2$, $a[2] = 2 \leq 5 \rightarrow hi = 2$.

- $lo = hi = 2 \rightarrow$ stop after **3** iterations.

Returns (2, 2, 3) $\rightarrow$ (C).

Options. (B) counts the final failed loop test as an iteration. (A) returns the *maximum* (index of the rotation's last element). (D) undercounts the trace.

Trap: comparing with $a[lo]$ instead of $a[hi]$ fails when the array is not rotated at all; the $a[hi]$ comparison works in every case.

Q.7 [NAT · 1 mark · Hashing — linear probing with deletion markers]

Answer: 10

Concept. Lazy deletion keeps probe chains intact: a DELETED slot never stops a search.

Insertion:

- $21 \rightarrow 1, 31 \rightarrow 2, 41 \rightarrow 3$.
- 12: 2, 3 full $\rightarrow$ **4**.
- 52: 2, 3, 4 full $\rightarrow$ **5**.
- 3: 3, 4, 5 full $\rightarrow$ **6**.

Table: 1:21, 2:31, 3:41, 4:12, 5:52, 6:3. After deleting 31, slot 2 holds DELETED.

- Search 52: slots 2 (DELETED), 3, 4, 5 (found) $\rightarrow x = 4$.
- Search 72: slots 2 (DELETED), 3, 4, 5, 6, 7 (EMPTY $\rightarrow$ stop) $\rightarrow y = 6$.

$x + y = 10$.

Trap: if slot 2 were simply emptied, the search for 52 would stop immediately and wrongly report “not found” — the reason tombstones exist.

0	
1	21
2	DEL
3	41
4	12
5	52
6	3
7	
8	
9	

Table after deleting 31

Q.8 [MSQ · 1 mark · Elementary sorts on a reversed array]

Answer: (A), (C), (D)

- (A) Every adjacent swap removes exactly one inversion; a reversed array of 6 has $6 \cdot 5/2 = 15$ inversions. **TRUE.**
- (B) Each key 5, 4, 3, 2, 1 travels to index 0, comparing with every element to its left and then stopping because $j < 0$ (no extra comparison): $1 + 2 + 3 + 4 + 5 = 15$, not 20. **FALSE.**
- (C) Selection sort always compares $n(n-1)/2 = 15$ times, whatever the input. **TRUE.**
- (D) Pass 0 swaps $6 \leftrightarrow 1 \rightarrow 1\ 5\ 4\ 3\ 2\ 6$; pass 1 swaps $5 \leftrightarrow 2 \rightarrow 1\ 2\ 4\ 3\ 5\ 6$; pass 2 swaps $4 \leftrightarrow 3 \rightarrow$ sorted; passes 3 and 4 find the minimum already in place. **3 swaps — TRUE.**

Tip: for reversed input, selection sort does $\lfloor n/2 \rfloor$ swaps because each swap fixes *two* positions at once.

Trap: in (B), adding a “failing” comparison per pass — it never happens when the key reaches index 0.

Q.9 [NAT · 1 mark · Graph theory — handshaking lemma]

Answer: 7

Concept. Handshaking lemma: the sum of all vertex degrees equals twice the number of edges, because every edge contributes 1 to the degree of each of its two endpoints.

Let n be the number of vertices. Then

$$3 \cdot 4 + (n - 3) \cdot 3 = 2 \cdot 12$$

$$12 + 3n - 9 = 24$$

$$3n = 21 \rightarrow n = 7.$$

Sanity checks: degrees are at most $n - 1 = 6$, so a simple graph is possible; the degree sum 24 is even as required (and the number of odd-degree vertices, 4, is even).

Trap: forgetting the factor 2 ($\sum \deg = |E|$) gives $3n + 3 = 12 \rightarrow n = 3$, which is impossible since three vertices of degree 4 need at least 5 vertices.

Q.10 [MCQ · 1 mark · Binary trees — reconstruction from traversals]

Answer: (B)

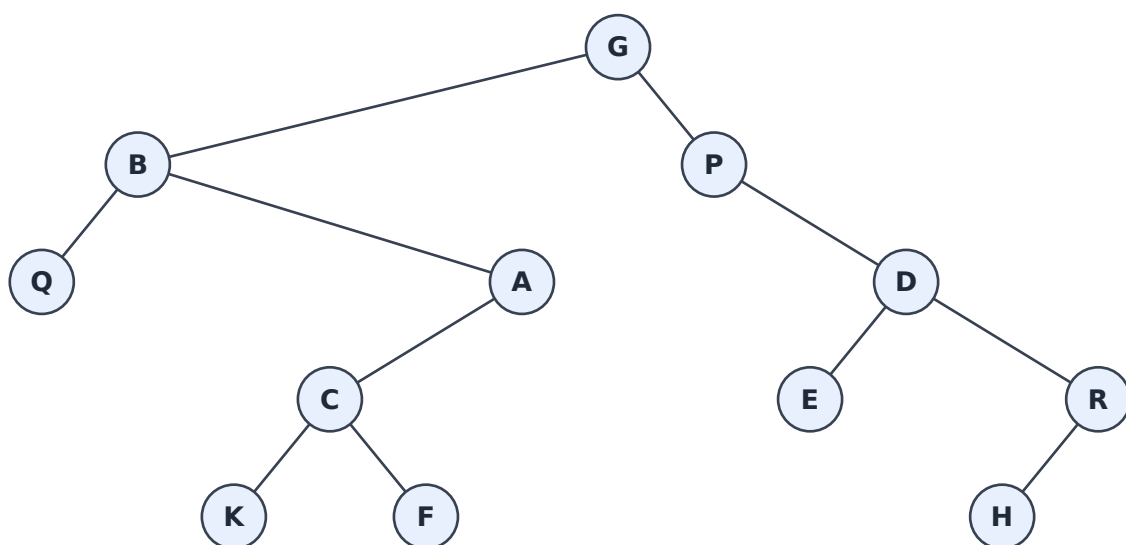
Concept. The first pre-order element is the root; its position in the in-order splits the remaining nodes into left and right subtrees. Recurse.

- Root G; in-order left {Q B K C F A}, right {P E D H R}.
- Left: pre-order B Q A C K F $\rightarrow$ root B; in-order Q | B | K C F A $\rightarrow$ left Q, right {K C F A} with root A (next in pre-order); A's left is {K C F} with root C (children K, F); A has no right child.
- Right: pre-order P D E R H $\rightarrow$ root P; in-order P | E D H R $\rightarrow$ P has no left child; right subtree root D with left E and right {H R} rooted at R (left child H).

Post-order (left, right, root): Q, K F C A, B, then E, H R, D, P, then G $\rightarrow$ **Q K F C A B E H R D P G** $\rightarrow$ (B).

Options. (A) places H R before E (R's subtree before D's left child). (C) puts Q after its cousins. (D) swaps the children K and F's order relative to C.

Tip: the last post-order element is always the root (G), and the post-order of each subtree is contiguous — a quick way to eliminate options.



Reconstructed tree

Q.11 [MCQ · 2 marks · Python — generators and iterator exhaustion]**Answer: (A)**

Concept. A generator (and any iterator) can be consumed only once; every `next()` call advances the *same* underlying object, whoever makes it.

- `zip(g, g)` pulls alternately from the same generator: (0, 1), (4, 9). For the third pair it pulls 16 for the first slot, then the generator is exhausted for the second slot, so `zip` stops — **16 is consumed and discarded**. `a = [(0, 1), (4, 9)]`.
- `sum(g)`: `g` is already exhausted → `b = 0`.
- The comprehension iterates `it` to the end, keeping 2 and 4; then `list(it)` is empty → `c = [2, 4]`.

Output `[(0, 1), (4, 9)] 0 [2, 4] → (A)`.

Option by option:

- (B) assumes `zip` “puts back” the unpaired 16.
- (C) treats `g` like a list that can be iterated twice in parallel.
- (D) assumes `list(it)` restarts the iterator.

Trap: `zip(it, it)` is the idiom for chunking into pairs — and an odd leftover element is silently lost.

Q.12 [NAT · 2 marks · Python — try / except / finally with return]**Answer: 110**

Concept. A `finally` block runs on every exit from the `try` statement — including a `return` from `try` or `except`. If `finally` itself executes `return`, that value **replaces** the pending return value.

- `f(0)`: `10 // 0` raises `ZeroDivisionError` → `except` returns `-1`; `finally` runs but `x ≠ 2` → result **-1**.
- `f(1)`: returns `10 // 1 = 10` (`finally` does nothing).
- `f(2)`: `try` prepares to return 5, but `finally` executes `return 99` → result **99**.
- `f(5)`: returns `10 // 5 = 2`.

Sum = `-1 + 10 + 99 + 2 = 110`.

Trap: assuming the `try` block’s `return` wins (giving 5 for `f(2)` and a sum of 16), or assuming the exception propagates for `x = 0`. A `return` inside `finally` also silently swallows any in-flight exception, which is why linters warn about it.

Order of events for `f(2)`: the expression `10 // 2` is evaluated and the value 5 is stashed as the pending return value; control then enters `finally`, where `return 99` discards the stashed 5. For `x = 1` and `x = 5` the `finally` block falls through, so the stashed value is returned unchanged.

Q.13 [MSQ · 2 marks · BST — deletion with in-order successor]**Answer: (A), (C), (D)**

Concept. The in-order successor of a node with two children is the leftmost node of its right subtree; it has no left child, so removing it is a one-child (or leaf) deletion.

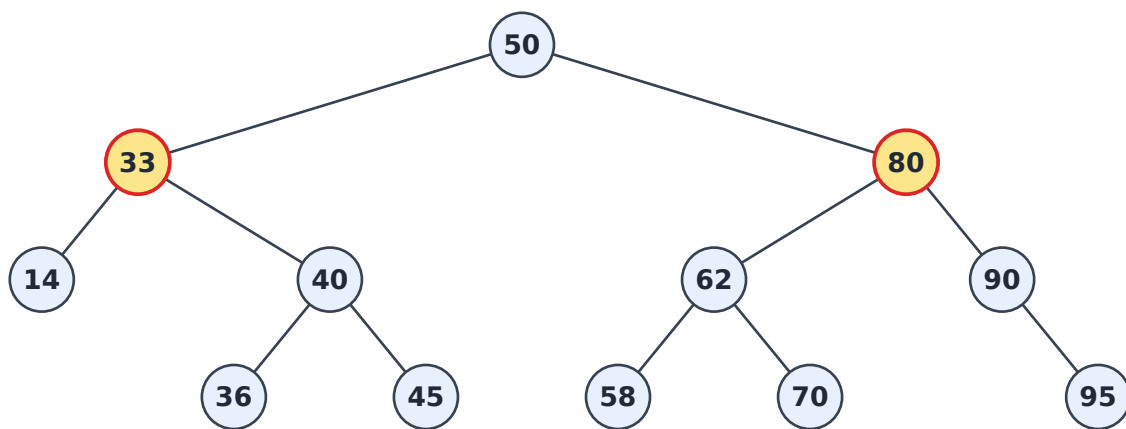
Delete 28: right subtree is rooted at 40; its leftmost node is **33** (33 has a right child 36). Copy 33 into the node of 28, then remove the old 33 by linking its child 36 to 40’s left. Left subtree becomes `33 → (14, 40)`, `40 → (36, 45)`.

Delete 75: successor = leftmost of the subtree at 90 = **80** (a leaf). Copy 80 up and remove the leaf. Right subtree becomes `80 → (62, 90)`, `62 → (58, 70)`, `90 → (-, 95)`.

- (A) 36 now hangs from 40 as its left child. **TRUE**.
- (B) The root’s left child is 33, not 36 (36 would be the answer only if we had promoted the successor’s *child*). **FALSE**.

- (C) Longest paths, e.g. $50 \rightarrow 33 \rightarrow 40 \rightarrow 36$, have 3 edges (before the deletions the height was 4 via $28 \rightarrow 40 \rightarrow 33 \rightarrow 36$). **TRUE.**
- (D) Pre-order: 50, 33, 14, 40, 36, 45, 80, 62, 58, 70, 90, 95. **TRUE.**

Trap: using the in-order predecessor for 28 would promote 14 instead of 33, giving a different tree.



After deleting 28 and 75

Q.14 [NAT · 2 marks · Bottom-up merge sort — comparisons]

Answer: 27

Concept. Bottom-up merge sort runs $\lceil \log_2 n \rceil$ passes; a merge of runs of length p and q costs $(p + q)$ minus the number of elements left over when one run empties.

- **w = 1:** pairs (12,7), (3,18), (9,1), (15,4), (11,6) → 5 comparisons; 2 is unpaired. A = 7 12 | 3 18 | 1 9 | 4 15 | 6 11 | 2.
- **w = 2:** [7,12]+[3,18] → 3, 7, 12 then copy 18: **3**; [1,9]+[4,15] → 1, 4, 9 then copy 15: **3**; [6,11]+[2] → 2 then copy: **1**. Pass total 7. A = 3 7 12 18 | 1 4 9 15 | 2 6 11.
- **w = 4:** [3,7,12,18]+[1,4,9,15] → outputs 1, 3, 4, 7, 9, 12, 15 with 7 comparisons, then copy 18: **7**. [2, 6, 11] has no partner.
- **w = 8:** [1,3,4,7,9,12,15,18]+[2,6,11] → outputs 1, 2, 3, 4, 6, 7, 9, 11 (8 comparisons), then copy 12, 15, 18: **8**.

Total = 5 + 7 + 7 + 8 = **27**.

Trap: assuming top-down and bottom-up merge sort split the same way — for $n = 11$ they do not (bottom-up merges 8 + 3 at the end), so their comparison counts can differ.

	Array after each pass											
	0	1	2	3	4	5	6	7	8	9	10	comps
w=1	7	12	3	18	1	9	4	15	6	11	2	5
w=2	3	7	12	18	1	4	9	15	2	6	11	7
w=4	1	3	4	7	9	12	15	18	2	6	11	7
w=8	1	2	3	4	6	7	9	11	12	15	18	8

Q.15 [MCQ · 2 marks · Quicksort — Hoare partition]

Answer: (B)

Concept. Hoare's scheme moves i right to an element $\geq$ pivot and j left to an element $\leq$ pivot, swaps them, and repeats until the pointers cross. It returns j such that every element of $A[\text{lo}..j]$ is $\leq$ pivot and every element of $A[j+1..hi]$ is $\geq$ pivot. The pivot is **not** necessarily in its final position.

Pivot $p = 11$.

- Round 1: i stops at 0 ($11 \geq 11$); j stops at 8 ($9 \leq 11$). Swap → [9, 4, 17, 8, 2, 19, 6, 14, 11].

- Round 2: i moves 1 (4), stops at 2 (17); j moves 7 ($14 > 11$), stops at 6 (6). Swap $\rightarrow [9, 4, 6, 8, 2, 19, 17, 14, 11]$.
- Round 3: i moves 3, 4, stops at 5 (19); j moves 5 ($19 > 11$), stops at 4 (2). Now $i = 5 \geq j = 4 \rightarrow$ return **4**.

Result: 4 and $[9, 4, 6, 8, 2, 19, 17, 14, 11] \rightarrow (B)$. Left part $\{9, 4, 6, 8, 2\} \leq 11$, right part $\{19, 17, 14, 11\} \geq 11$.

Options. (A) returns i instead of j . (C) places the pivot between the parts as Lomuto would. (D) is not produced by any single partition.

Trap: with Hoare partition, recurse on $A[\text{lo}..j]$ and $A[j+1..hi]$ — *not* on $j-1$, because the pivot may sit inside the right part (here 11 is at index 8).

Q.16 [NAT · 2 marks · Dijkstra — sum of shortest distances]

Answer: 48

Concept. Dijkstra finalises vertices in increasing order of distance; each extraction relaxes the edges of the extracted vertex.

- Extract S (0): $A = 5, B = 2$.
- Extract B (2): $A = \min(5, 2+2) = 4; D = 9; E = 14$.
- Extract A (4): $C = 8; D = \min(9, 10) = 9$.
- Extract C (8): $D = \min(9, 9)$ — tie, unchanged; $T = 16$.
- Extract D (9): $E = \min(14, 11) = 11$.
- Extract E (11): $T = \min(16, 14) = 14$.
- Extract T (14).

Distances: S 0, A 4, B 2, C 8, D 9, E 11, T 14 $\rightarrow$ sum = $0+4+2+8+9+11+14 = \mathbf{48}$.

Trap: taking the direct edges S-A (5), B-E (12) or C-T ($8 + 8 = 16$) at face value — each of them is beaten by a detour (S-B-A = 4, S-B-D-E = 11, ...-D-E-T = 14).

Tentative distances after each extraction

	S	A	B	C	D	E	T
S	0	5	2	∞	∞	∞	∞
B	0	4	2	∞	9	14	∞
A	0	4	2	8	9	14	∞
C	0	4	2	8	9	14	16
D	0	4	2	8	9	11	16
E	0	4	2	8	9	11	14

Q.17 [MSQ · 2 marks · DFS — valid traversal orders]

Answer: (B), (C)

Concept. In DFS, the next discovered vertex must be an unvisited neighbour of the **deepest** vertex on the current path that still has unvisited neighbours (backtracking only when the current vertex is exhausted).

- (A) $A \rightarrow B \rightarrow E \rightarrow C$; now C still has an unvisited neighbour F, so the next vertex must be F — D is not adjacent to C. **Invalid.**
- (B) $A \rightarrow C \rightarrow E \rightarrow B$; B is exhausted $\rightarrow$ back to E $\rightarrow G \rightarrow F$ ($G-F$) $\rightarrow D$ ($F-D$). **Valid.**
- (C) $A \rightarrow B \rightarrow E \rightarrow G \rightarrow F$ ($G-F$) $\rightarrow C$ ($F-C$) $\rightarrow D$: at C, its neighbours A, E, F are visited, so backtrack to F, whose unvisited neighbour D is next. **Valid.**
- (D) $A \rightarrow D \rightarrow F \rightarrow G$; G's unvisited neighbour E must come next, but C is not adjacent to G. **Invalid.**

Trap: checking only that each vertex is adjacent to *some* earlier vertex — that is the condition for a valid *BFS-like* or *arbitrary-search* order, not for DFS. The backtracking rule is what (A) and (D) violate.

Q.18 [NAT · 2 marks · Queues — round-robin scheduling simulation]

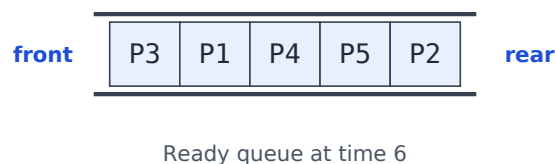
Answer: 19.39 to 19.41

Concept. Simulate the ready queue exactly; the tie rule at time 6 (P5 arrives as P2's quantum ends) matters.

- 0–3 P1 (rem 4). Arrived: P2, P3 → queue P2 P3 P1.
- 3–6 P2 (rem 1). Arrived: P4 (4), P5 (6) → queue P3 P1 P4 P5 P2.
- 6–9 P3 (rem 6) → P1 P4 P5 P2 P3.
- 9–12 P1 (rem 1) → P4 P5 P2 P3 P1.
- 12–15 P4 finishes (**15**).
- 15–18 P5 (rem 2) → P2 P3 P1 P5.
- 18–19 P2 finishes (**19**).
- 19–22 P3 (rem 3) → P1 P5 P3.
- 22–23 P1 finishes (**23**). 23–25 P5 finishes (**25**). 25–28 P3 finishes (**28**).

Turnaround: P1 $23 - 0 = 23$, P2 $19 - 1 = 18$, P3 $28 - 2 = 26$, P4 $15 - 4 = 11$, P5 $25 - 6 = 19$. Sum = 97 → average $97/5 = \mathbf{19.40}$.

Trap: re-queuing the preempted process *before* the new arrivals changes the order (e.g. P2 would precede P5 at time 6) and the answer. Also, the last completion is always the total burst time (28) when the CPU is never idle — a useful check.



Q.19 [MCQ · 2 marks · Linked lists — reversing in groups of k]

Answer: (C)

Concept. `rev_k` first checks that at least k nodes remain; if not, the tail is returned unchanged. Otherwise it recursively processes the rest, then reverses the first k nodes, pointing the old first node at the processed remainder.

- Call on 1...8: 3 nodes exist; recurse on 4...8.
- Call on 4...8: recurse on 7 → 8.
- Call on 7 → 8: only $2 < 3$ nodes → returned **unchanged**.
- Back in 4...8: reverse 4, 5, 6 with prev starting at node 7 → $6 \rightarrow 5 \rightarrow 4 \rightarrow 7 \rightarrow 8$.
- Back in 1...8: reverse 1, 2, 3 with prev starting at node 6 → $3 \rightarrow 2 \rightarrow 1 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 7 \rightarrow 8$.

Output **3 2 1 6 5 4 7 8** → (C).

Trap (tuple assignment). All right-hand values (prev, cur, cur.nxt) are evaluated first; then `cur.nxt` is assigned while `cur` still names the old node, and only afterwards is `cur` advanced — so the line is correct. (Writing `cur, cur.nxt, prev = ...` would assign `cur` first and corrupt the wrong node.)

Options. (A) also reverses the incomplete last group. (B) reverses the whole list. (D) assumes nothing changes.

Q.20 [MSQ · 2 marks · Hashing — linear probing analysis]

Answer: (B), (D)

Concept. Compute the home slot, then step +1 mod 11 until an empty slot is found.

- 8: $(16+3) \bmod 11 = 8 \rightarrow$ slot 8 (1 probe).
- 19: $41 \bmod 11 = 8 \rightarrow$ 8 full $\rightarrow$ **9** (2 probes).
- 13: $29 \bmod 11 = 7 \rightarrow$ slot 7 (1).
- 2: $7 \bmod 11 = 7 \rightarrow$ 7, 8, 9 full $\rightarrow$ **10** (4).
- 25: $53 \bmod 11 = 9 \rightarrow$ 9, 10 full $\rightarrow$ **0** (3).
- 30: $63 \bmod 11 = 8 \rightarrow$ 8, 9, 10, 0 full $\rightarrow$ **1** (5).
- 6: $15 \bmod 11 = 4 \rightarrow$ slot 4 (1).

Final table: 0:25, 1:30, 4:6, 7:13, 8:8, 9:19, 10:2.

- (A) Probes = $1+2+1+4+3+5+1 =$ **17**, not 16. **FALSE.**
- (B) Slots 7, 8, 9, 10, 0, 1 form one wrapped cluster of length **6**. **TRUE.**
- (C) $h(41) = 85 \bmod 11 = 8 \rightarrow$ 8, 9, 10, 0, 1 full $\rightarrow$ slot **2**, not 3. **FALSE.**
- (D) 30 is in slot 1. **TRUE.**

Trap: forgetting wrap-around — the cluster 7..10 continues into 0..1, which is exactly why 30 and 41 travel so far (primary clustering).

0	25
1	30
2	
3	
4	6
5	
6	
7	13
8	8
9	19
10	2

Final table

Mock Test 41

Full-Syllabus Mock — Paper 11

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — identity versus equality]

Consider the following Python program. What is printed?

Python

```
1 a = [1, 2]
2 b = a[:]
3 c = a
4 print(a == b, a is b, c is a, [a] * 2 == [b, c])
```

- (A) True True True True
- (B) True False True False
- (C) True False True True
- (D) False False True True

Q.2 [NAT · numerical answer | 1 mark | Python — dict comprehension with repeated keys]

Consider the following Python program. The value printed is ____.

Python

```
1 d = {x % 4: x for x in range(10)}
2 print(sum(d.values()))
```

Answer: _____ (enter a numerical value)

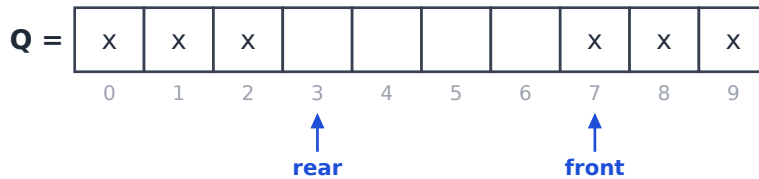
Q.3 [MSQ · one or more correct | 1 mark | Stacks — infix to postfix conversion]

The infix expression $a + b * (c - d) / e ^ f ^ g$ is converted to postfix with the usual operator-stack algorithm. Precedence: $^$ highest (right-associative), then $*$ and $/$ (left-associative), then $+$ and $-$ (left-associative). Which of the following statements is/are TRUE?

- (A) The operator stack never holds more than 3 symbols (counting $()$)
- (B) If $^$ were left-associative, the postfix form would not change
- (C) The prefix form is $+ a / * b - c d ^ e ^ f g$
- (D) The postfix form is $a b c d - * e f g ^ ^ / +$

Q.4 [MCQ · single correct | 1 mark | Queues — circular array arithmetic]

A circular queue is stored in an array $Q[0 \dots 9]$. **front** is the index of the first element and **rear** is the index of the next free slot; one slot is always left unused so that a full queue can be told apart from an empty one. If $\text{front} = 7$ and $\text{rear} = 3$, the number of elements in the queue is



- (A) 4
(B) 5
(C) 6
(D) 7

Q.5 [NAT · numerical answer | 1 mark | Linked lists — Floyd's cycle detection]

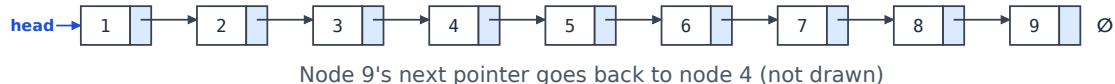
A singly linked list has nodes $1 \rightarrow 2 \rightarrow \dots \rightarrow 9$ and the **next** pointer of node 9 points back to node 4 (see figure). Floyd's algorithm is run from the head: in each step **slow** moves one node and **fast** moves two nodes, and the step counter is incremented; the loop stops as soon as **slow** and **fast** are on the same node. The number of steps executed is _____.

Python

```

1 slow = fast = head
2 steps = 0
3 while True:
4     slow = slow.next
5     fast = fast.next.next
6     steps += 1
7     if slow is fast:
8         break

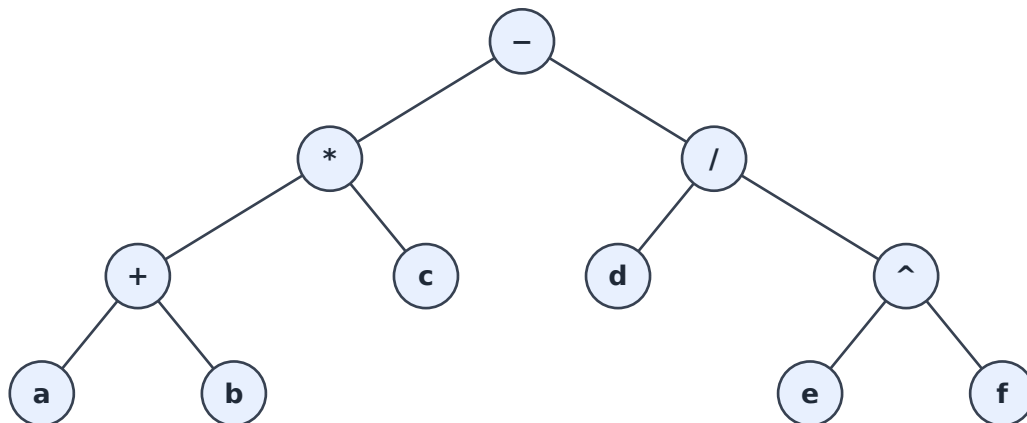
```



Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Expression trees — prefix form]

The prefix (Polish) form of the expression tree shown below is



Expression tree

- (A) $- * + a b c / d ^ e f$
- (B) $a b + c * d e f ^ / -$
- (C) $- + * a b c / ^ d e f$
- (D) $- * a + b c / d ^ e f$

Q.7 [NAT · numerical answer | 1 mark | Hashing — load factor and chaining cost]

3000 keys are stored in a hash table with 1024 slots using separate chaining. Under simple uniform hashing, the expected number of keys examined during an **unsuccessful** search (not counting the cost of computing the hash) is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Binary search — a buggy variant]

Consider the following (buggy) binary search with $a = [3, 6, 9, 12, 15]$. Which of the following statements is/are TRUE?

Python

```
1 def find(a, x):
2     lo, hi = 0, len(a) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid
7         else:
8             hi = mid
9     return lo
```

- (A) `find(a, 6)` terminates and returns 1
- (B) `find(a, 3)` terminates and returns 0
- (C) `find(a, 1)` terminates and returns 0
- (D) `find(a, 100)` terminates and returns 4

Q.9 [MCQ · single correct | 1 mark | Selection sort — counting index updates]

Selection sort is run on $A = [7, 3, 9, 1, 6, 2]$. In pass i , it sets $m = i$ and then, for $j = i+1 \dots n-1$, executes $m = j$ whenever $A[j] < A[m]$; finally it swaps $A[i]$ and $A[m]$. Over the whole sort, how many times is the statement $m = j$ executed?

- (A) 7
- (B) 4
- (C) 9
- (D) 15

Q.10 [MCQ · single correct | 1 mark | Graph theory — edges in a disconnected graph]

The maximum number of edges in a **disconnected** simple undirected graph on 10 vertices is

- (A) 45
- (B) 36
- (C) 40
- (D) 35

Q.11 [NAT · numerical answer | 2 marks | Python — closures with nonlocal state]

Consider the following Python program. The value printed is _____.

Python

```
1 def make():
2     c = 0
3     def inc(k=1):
4         nonlocal c
5         c += k
6         return c
7     return inc
8
9 f = make()
10 g = make()
11 f(); f(3); g(10)
12 h = f
13 print(f() + g() + h(2))
```

Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Python — iterators and zip]

Consider the following Python program. What is printed?

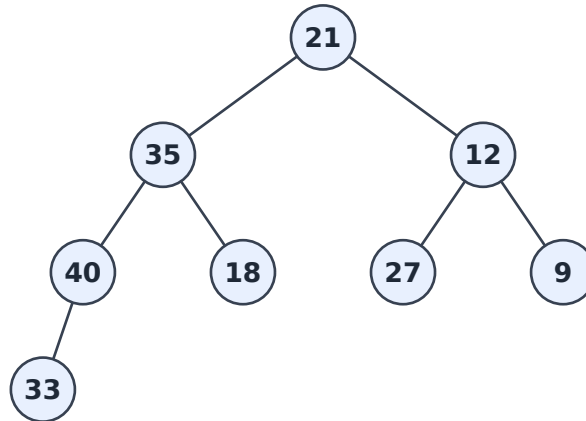
Python

```
1 a = [1, 2, 3, 4, 5, 6, 7]
2 it = iter(a)
3 p = list(zip(it, it))
4 q = list(it)
5 r = dict(zip("abc", range(5)))
6 print(p, q, r)
```

- (A) [(1, 2), (3, 4), (5, 6)] [7] {'a': 0, 'b': 1, 'c': 2}
- (B) [(1, 2), (3, 4), (5, 6), (7, None)] [] {'a': 0, 'b': 1, 'c': 2, 3: 4}
- (C) [(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6), (7, 7)] [] {'a': 0, 'b': 1, 'c': 2}
- (D) [(1, 2), (3, 4), (5, 6)] [] {'a': 0, 'b': 1, 'c': 2}

Q.13 [MSQ · one or more correct | 2 marks | Heapsort — build-heap and extraction steps]

Heapsort (ascending) is applied to $A = [21, 35, 12, 40, 18, 27, 9, 33]$. It first builds a max-heap bottom-up (sift-down at $i = 3, 2, 1, 0$) and then repeatedly swaps $A[0]$ with the last element of the heap, shrinks the heap by one and sifts down the new root. Which of the following statements is/are TRUE?



A as a complete binary tree before build-heap

- (A) After build-heap, $A = [40, 35, 27, 33, 18, 12, 9, 21]$
- (B) After the two extraction steps, $A[0 \dots 5]$ is a valid max-heap and $A[6 \dots 7]$ holds the two largest keys in increasing order
- (C) After two extraction steps, $A = [33, 21, 27, 9, 18, 12, 35, 40]$
- (D) Build-heap performs exactly 4 parent-child swaps

Q.14 [NAT · numerical answer | 2 marks | Merge sort — comparisons on structured input]

Top-down merge sort (split into halves of equal size; the merge stops comparing as soon as one run is exhausted) sorts the 16-element array

$[2, 4, 6, 8, 10, 12, 14, 16, 1, 3, 5, 7, 9, 11, 13, 15]$.

The total number of element comparisons is _____.

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Recurrences — change of variable]

Let $T(n) = 2T(\sqrt{n}) + \log_2 n$ for $n > 2$, with $T(2) = 1$. Which of the following is a tight bound for $T(n)$?

- (A) $\Theta(\log n)$
- (B) $\Theta(\log n \cdot \log \log n)$
- (C) $\Theta(\sqrt{n})$
- (D) $\Theta(\log^2 n)$

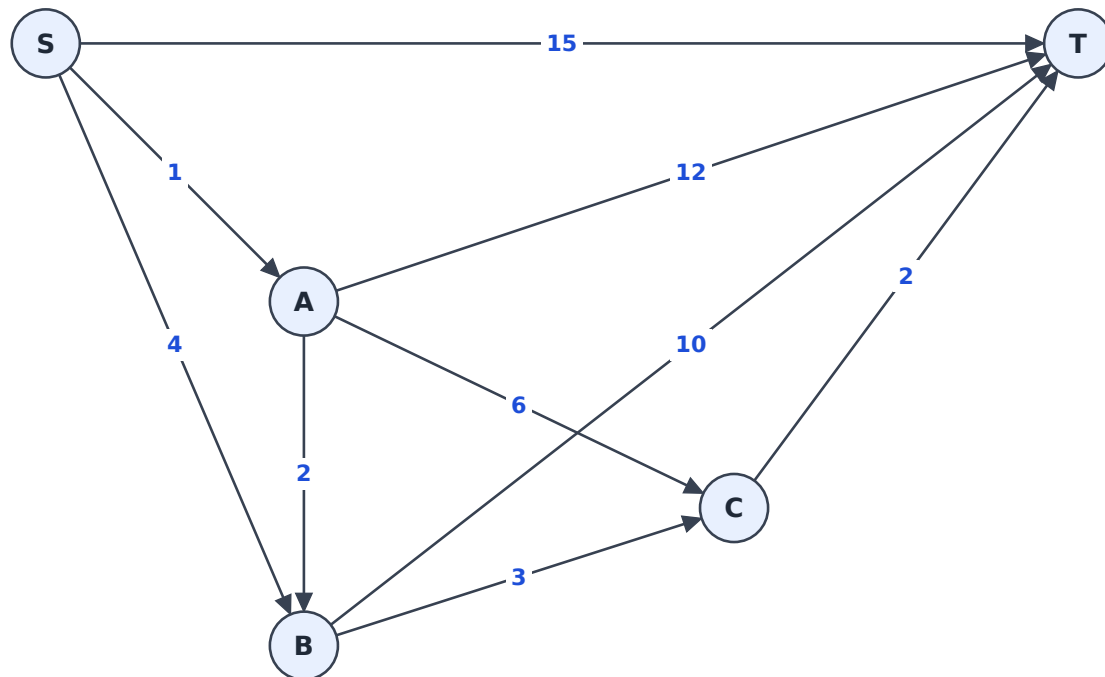
Q.16 [NAT · numerical answer | 2 marks | BFS — distances in a rule-defined graph]

Graph G has vertex set $\{0, 1, \dots, 11\}$, and vertices i and j are adjacent iff $|i - j| \in \{3, 5\}$. BFS is run from vertex 0. The sum of the BFS distances (number of edges) from 0 to all 12 vertices is _____.

Answer: _____ (enter a numerical value)

Q.17 [MCQ · single correct | 2 marks | Dijkstra — counting distance updates]

Dijkstra's algorithm is run from S on the weighted directed graph below. Initially $d[S] = 0$ and all other d values are ∞ ; an *update* of $d[T]$ is an assignment that makes $d[T]$ **strictly smaller** (the first assignment from ∞ counts). How many times is $d[T]$ updated?



- (A) 2
- (B) 4
- (C) 5
- (D) 3

Q.18 [MSQ · one or more correct | 2 marks | Hashing — deletion with tombstones]

A table with 10 slots uses $h(k) = k \bmod 10$ and linear probing. The keys 31, 41, 51, 22, 61 are inserted in that order, and then 41 is deleted by marking its slot DELETED (a tombstone). Searches skip over DELETED slots and stop at an EMPTY slot; an insertion places the key in the first EMPTY or DELETED slot on its probe sequence. Which of the following statements is/are TRUE?

- (A) Inserting 71 now stores it in slot 2
- (B) A search for 22 now examines exactly 2 slots
- (C) A search for 61 now examines exactly 5 slots
- (D) If 41's slot had been made EMPTY instead of DELETED, a search for 51 would wrongly report that 51 is absent

Q.19 [NAT · numerical answer | 2 marks | Stacks — counting output sequences]

The integers 1, 2, 3, 4, 5 are pushed onto an initially empty stack in this order, with pops interleaved arbitrarily; each popped value is output, and all five values are eventually output. The number of possible output sequences whose **first** element is 3 is _____.

Answer: _____ (enter a numerical value)

Q.20 [MSQ · one or more correct | 2 marks | Linked lists — moving the tail to the front]

Consider the following Python function on a singly linked list (nodes have fields **v** and **nxt**). Which of the following statements is/are TRUE?

Python

```
1 def f(h, k):
2     for _ in range(k):
3         prev, cur = None, h
4         while cur.nxt:
5             prev, cur = cur, cur.nxt
6         prev.nxt = None
7         cur.nxt = h
8         h = cur
9     return h
```



- (A) For a list with a single node, $f(h, 1)$ raises an `AttributeError`
- (B) For the list shown, $f(h, 8)$ returns the list $3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 1 \rightarrow 2$
- (C) For the list shown, $f(h, 2)$ returns the list $5 \rightarrow 6 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
- (D) For the list shown, $f(h, 2)$ executes the statement `prev, cur = cur, cur.nxt` exactly 12 times

Answer Key — Mock Test 41

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — identity versus equality
2	NAT	1	30	Python — dict comprehension with repeated keys
3	MSQ	1	(C), (D)	Stacks — infix to postfix conversion
4	MCQ	1	(C)	Queues — circular array arithmetic
5	NAT	1	6	Linked lists — Floyd's cycle detection
6	MCQ	1	(A)	Expression trees — prefix form
7	NAT	1	2.92 to 2.94	Hashing — load factor and chaining cost
8	MSQ	1	(B), (C)	Binary search — a buggy variant
9	MCQ	1	(A)	Selection sort — counting index updates
10	MCQ	1	(B)	Graph theory — edges in a disconnected graph
11	NAT	2	23	Python — closures with nonlocal state
12	MCQ	2	(D)	Python — iterators and zip
13	MSQ	2	(A), (B), (C)	Heapsort — build-heap and extraction steps
14	NAT	2	39	Merge sort — comparisons on structured input
15	MCQ	2	(B)	Recurrences — change of variable
16	NAT	2	26	BFS — distances in a rule-defined graph
17	MCQ	2	(D)	Dijkstra — counting distance updates
18	MSQ	2	(A), (C), (D)	Hashing — deletion with tombstones
19	NAT	2	9	Stacks — counting output sequences
20	MSQ	2	(A), (C)	Linked lists — moving the tail to the front

Detailed Solutions — Mock Test 41

Q.1 [MCQ · 1 mark · Python — identity versus equality]

Answer: (C)

`==` compares **values** (element by element for lists); `is` compares **object identity**.

- `a == b`: `b` is a slice copy with the same elements → True.
- `a is b`: the slice created a new list object → False.
- `c is a`: plain assignment binds another name to the same object → True.
- `[a] * 2 == [b, c]`: the left side is `[a, a]`; list equality compares element-wise with `==`: `a == b` (True) and `a == c` (True) → True. It does not matter that the left list holds the same object twice.

Output `True False True True` → (C).

- (A) thinks a slice copy is the same object.
- (B) thinks list equality requires identical element objects.
- (D) thinks a copy is not equal to the original.

Tip: use `is` only for identity checks such as `x is None`; never use it to compare values.

Q.2 [NAT · 1 mark · Python — dict comprehension with repeated keys]

Answer: 30

A dict comprehension inserts key-value pairs in order; when a key repeats, the **later** value overwrites the earlier one (the key keeps its original position).

For `x = 0 ... 9` the key is `x % 4`:

- key 0: `x = 0, 4, 8` → final value 8
- key 1: `x = 1, 5, 9` → final value 9
- key 2: `x = 2, 6` → final value 6
- key 3: `x = 3, 7` → final value 7

`d = {0: 8, 1: 9, 2: 6, 3: 7}`, so sum of values = `8 + 9 + 6 + 7 = 30`.

Traps:

- Keeping the **first** value per key gives `0 + 1 + 2 + 3 = 6`.
- Summing all of `range(10)` (45) ignores that only 4 keys survive.

Q.3 [MSQ · 1 mark · Stacks — infix to postfix conversion]

Answer: (C), (D)

Shunting-yard: operands go straight to the output; an incoming operator first pops operators of higher precedence (or equal precedence if it is left-associative); `(` is pushed and `)` pops back to it.

- `a` → out. `+` → stack `[+]`. `b` → out.
- `*` (higher than `+`) → `[+, *]`. `(` → `[+, *, (]`. `c` → out. `-` → `[+, *, (, -]`. `d` → out.
- `)` pops `-` → out; stack `[+, *]`.
- `/`: equal to `*` and left-associative → pop `*`; then push → `[+, /]`.
- `e` → out. `^` → `[+, /, ^]`. `f` → out.
- second `^`: equal precedence but right-associative → no pop → `[+, /, ^, ^]`. `g` → out.
- End: pop `^`, `^`, `/`, `+`.

Postfix: `a b c d - * e f g ^ ^ / +`.

- (A) The stack reaches 4 symbols twice (`[+, *, (, -]` and `[+, /, ^, ^]`). **FALSE.**
- (B) Left-associative `^` would pop the first `^` before pushing the second, giving `... e f ^ g ^ ....` **FALSE.**
- (C) The tree is `+(a, / (* (b, - (c, d)), ^ (e, ^ (f, g))))`; its pre-order is `+ a / * b - c d ^ e ^ f g`. **TRUE.**

- (D) **TRUE.**

Trap: $e \wedge f \wedge g$ means $e \wedge (f \wedge g)$ — right associativity is what keeps both $\wedge$ on the stack.

Q.4 [MCQ · 1 mark · Queues — circular array arithmetic]

Answer: (C)

With **rear** pointing at the next free slot, the occupied slots are front, front+1, ..., rear-1 taken modulo the capacity n, so

size = (rear - front + n) mod n.

Here: $(3 - 7 + 10) \bmod 10 = 6$. The occupied slots are 7, 8, 9, 0, 1, 2 → **6** elements → (C).

Option analysis:

- (A) $4 = \text{front} - \text{rear}$: the number of *free* slots (including the reserved one).
- (B) 5 assumes rear points at the last element and subtracts the reserved slot.
- (D) 7 counts slot 3 as occupied ($\text{rear} - \text{front} + n + 1$).

The queue can hold at most $n - 1 = 9$ elements in this scheme; it is full when $(\text{rear} + 1) \bmod n == \text{front}$ and empty when $\text{rear} == \text{front}$.

Trap: forgetting the “+ n” and getting a negative size, or treating **rear** as the last occupied index.

Q.5 [NAT · 1 mark · Linked lists — Floyd's cycle detection]

Answer: 6

The tail (1, 2, 3) has length $\mu = 3$ and the cycle $4 \rightarrow 5 \rightarrow \dots \rightarrow 9 \rightarrow 4$ has length $\lambda = 6$. Track positions after each step (slow moves 1, fast moves 2):

- Step 1: slow 2, fast 3
- Step 2: slow 3, fast 5
- Step 3: slow 4, fast 7
- Step 4: slow 5, fast 9
- Step 5: slow 6, fast 5 ($9 \rightarrow 4 \rightarrow 5$)
- Step 6: slow 7, fast 7 → **meet**

Steps = **6**.

General fact: they meet after t steps where t is the smallest multiple of λ with $t \geq \mu$; here $\mu = 3$, $\lambda = 6 \rightarrow t = 6$. Once slow enters the cycle, the gap closes by one node per step, so the meeting happens within λ steps of slow's entry.

To find the cycle's start, reset one pointer to the head and advance both one step at a time: they meet at node 4 after $\mu = 3$ steps.

Trap: stopping when fast first passes slow, or counting fast's moves (12) instead of loop iterations.

Q.6 [MCQ · 1 mark · Expression trees — prefix form]

Answer: (A)

Prefix form = **pre-order** traversal of the expression tree (operator, then left operand, then right operand). Post-order gives the postfix form; in-order (with parentheses) gives the infix form.

- Root —.
- Left subtree pre-order: *, then + a b, then c $\rightarrow * + a b c$.
- Right subtree pre-order: /, d, then ^ e f $\rightarrow / d ^ e f$.

Prefix: $- * + a b c / d ^ e f \rightarrow (A)$. Infix: $(a + b) * c - d / e^f$.

- (B) is the **postfix** (post-order) form.
- (C) swaps the order of * and + (as if + were the parent of *) and misplaces ^.
- (D) places + under the right operand of * — that is the tree for $a * (b + c)$.

Tip: a valid prefix expression scanned right to left with a stack evaluates correctly; check the number of operands: each binary operator needs exactly two.

Q.7 [NAT · 1 mark · Hashing — load factor and chaining cost]

Answer: 2.92 to 2.94

In an unsuccessful search the whole chain of the probed slot is scanned. Under simple uniform hashing each of the n keys lies in that chain with probability $1/m$, so by linearity of expectation the expected chain length is $n/m = \alpha$ (the **load factor**).

- $\alpha = 3000 / 1024 = 2.9296875 \approx \mathbf{2.93}$.

For comparison, a *successful* search examines on average $1 + (n - 1)/(2m) = 1 + 2999/2048 \approx 2.46$ keys: the target itself plus half of the other keys that were inserted before it in the same chain (for tail insertion).

With $1 + \alpha \approx 3.93$ one would be counting the slot access as well — the question explicitly excludes it.

Trap: applying the open-addressing formula $1/(1 - \alpha)$, which is meaningless here because $\alpha > 1$ — chaining allows load factors above 1.

Q.8 [MSQ · 1 mark · Binary search — a buggy variant]

Answer: (B), (C)

The bug is `lo = mid` (instead of `mid + 1`). When $hi = lo + 1$, $mid = lo$; if $a[lo] < x$ the assignment `lo = mid` changes nothing and the loop never ends.

Once `lo` moves at all, $a[lo] < x$ holds, so eventually the interval shrinks to $[lo, lo + 1]$ with $a[lo] < x \rightarrow$ infinite loop. The only way to terminate is for `lo` never to move *and* $a[0] \geq x$, so that `hi` keeps shrinking down to 0.

- (A) $x = 6$: `mid` 2 ($9 \geq 6$) $\rightarrow$ `hi` 2; `mid` 1 ($6 \geq 6$) $\rightarrow$ `hi` 1; `mid` 0 ($3 < 6$) $\rightarrow$ `lo` = 0 again with `hi` = 1 $\rightarrow$ loops forever. **FALSE**.
- (B) $x = 3$: `mids` 2, 1, 0 all have $a[mid] \geq 3 \rightarrow$ `hi` = 2, 1, 0 $\rightarrow$ returns 0. **TRUE**.
- (C) $x = 1$: like (B), returns 0. **TRUE**.
- (D) $x = 100$: `lo` moves to 2, then 3; with `lo` = 3, `hi` = 4, `mid` = 3 $\rightarrow$ stuck. **FALSE**.

Tip: with `mid = (lo + hi) // 2` (rounding down) the update that keeps `mid` must be `hi = mid`; the other must be `lo = mid + 1`.

Q.9 [MCQ · 1 mark · Selection sort — counting index updates]

Answer: (A)

The number of comparisons is fixed ($n(n-1)/2 = 15$), but the number of updates of `m` depends on how often a new running minimum appears in the scanned suffix.

- Pass 0, scan 3, 9, 1, 6, 2 from 7: new minima 3, 1 $\rightarrow$ 2 updates; swap $\rightarrow$ [1, 3, 9, 7, 6, 2].
- Pass 1, scan 9, 7, 6, 2 from 3: new minimum 2 $\rightarrow$ 1 update; $\rightarrow$ [1, 2, 9, 7, 6, 3].
- Pass 2, scan 7, 6, 3 from 9: 7, 6, 3 are each new minima $\rightarrow$ 3 updates; $\rightarrow$ [1, 2, 3, 7, 6, 9].
- Pass 3, scan 6, 9 from 7: 6 $\rightarrow$ 1 update; $\rightarrow$ [1, 2, 3, 6, 7, 9].
- Pass 4, scan 9 from 7: none.

Total = $2 + 1 + 3 + 1 + 0 = \mathbf{7} \rightarrow$ (A).

- (B) 4 is the number of swaps that actually move elements.
- (C) 9 is not obtainable from any natural count here.
- (D) 15 is the number of comparisons.

Trap: counting updates on the *original* array instead of the array as it is after the previous swaps (pass 2 sees 9, 7, 6, 3 only because of the earlier swaps).

Q.10 [MCQ · 1 mark · Graph theory — edges in a disconnected graph]**Answer: (B)**

A disconnected graph splits its vertices into at least two parts with no edges between them. If one part has k vertices and the other $10 - k$, the maximum edge count is $C(k, 2) + C(10 - k, 2)$, which is largest when the split is as **unbalanced** as possible: $k = 1$.

- $k = 1$: $C(1, 2) + C(9, 2) = 0 + 36 = 36$.
- $k = 2$: $1 + 28 = 29$.
- $k = 5$: $10 + 10 = 20$.

So the answer is **36**: K_9 plus one isolated vertex $\rightarrow$ (B). Adding any further edge would have to touch the isolated vertex and connect the graph.

- (A) $45 = C(10, 2)$ is the complete graph, which is connected.
- (C) 40 and (D) 35 do not correspond to an optimal split.

Corollary: any simple graph on n vertices with more than $C(n - 1, 2)$ edges is connected.

Trap: guessing that a balanced split maximises edges — it minimises them.

Q.11 [NAT · 2 marks · Python — closures with nonlocal state]**Answer: 23**

Each call of `make()` creates a **new** local variable `c` and a new closure that captures it; `nonlocal c` lets `inc` rebind that captured variable. `h = f` creates no new closure — `h` and `f` are the same function object sharing one `c`.

- `f()`: `f`'s `c = 1`.
- `f(3)`: `f`'s `c = 4`.
- `g(10)`: `g`'s `c = 10` (independent of `f`).
- In the print, operands are evaluated left to right: `f()` $\rightarrow$ `f`'s `c = 5`, value 5; `g()` $\rightarrow$ `g`'s `c = 11`, value 11; `h(2)` $\rightarrow$ same counter as `f`: $5 + 2 = 7$, value 7.

Sum = $5 + 11 + 7 = 23$.

Common wrong answers:

- 18 — treating `h` as a fresh counter (`h(2) = 2`).
- 29 — treating `f` and `g` as sharing one counter.
- An `UnboundLocalError` would occur only without `nonlocal`, because `c += k` makes `c` local to `inc`.

Tip: default arguments (`k=1`) are evaluated once at definition time, but that is harmless here because 1 is immutable.

Q.12 [MCQ · 2 marks · Python — iterators and zip]**Answer: (D)**

`zip(it, it)` pulls alternately from the **same** iterator, so it pairs consecutive elements. `zip` stops as soon as **any** input is exhausted — but it only discovers that by calling `next` on it.

- Pairs (1, 2), (3, 4), (5, 6) are produced.
- For the 4th pair, `zip` calls `next(it)` for the first slot and gets **7** (consumed!), then calls `next(it)` for the second slot $\rightarrow$ `StopIteration` $\rightarrow$ `zip` stops and the 7 is silently discarded.
- So `q = list(it)` is `[]`.
- `zip("abc", range(5))` stops at the shorter input $\rightarrow$ `{'a': 0, 'b': 1, 'c': 2}`.

Output: `[(1, 2), (3, 4), (5, 6)] [] {'a': 0, 'b': 1, 'c': 2}` $\rightarrow$ (D).

Option analysis:

- (A) assumes the leftover 7 is still available — it was consumed by `zip`.

- (B) behaves like `itertools.zip_longest` and invents a key 3.
- (C) would be `zip(a, a)` on the *list*, which creates two independent iterators.

Trap: iterator exhaustion inside `zip` silently drops an element; `itertools.zip_longest` or slicing (`a[0::2]`, `a[1::2]`) avoids it.

Q.13 [MSQ · 2 marks · Heapsort — build-heap and extraction steps]

Answer: (A), (B), (C)

Build-heap (sift-down swaps with the larger child):

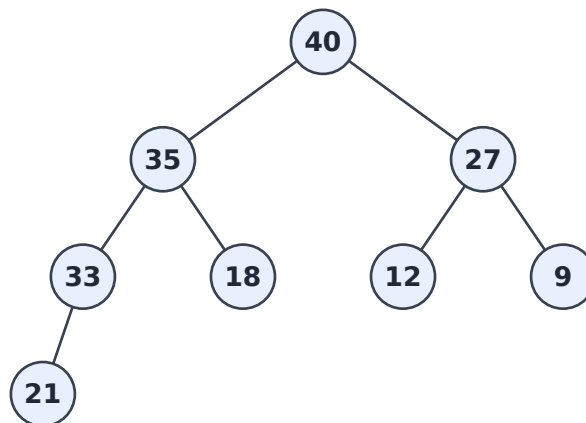
- $i = 3$ (40): child 33 is smaller $\rightarrow$ no swap.
- $i = 2$ (12): children 27, 9 $\rightarrow$ swap with 27 (1).
- $i = 1$ (35): children 40, 18 $\rightarrow$ swap with 40 (2); at index 3, 35 vs child 33 $\rightarrow$ stop.
- $i = 0$ (21): children 40, 27 $\rightarrow$ swap with 40 (3); at index 1, children 35, 18 $\rightarrow$ swap with 35 (4); at index 3, child 33 $>$ 21 $\rightarrow$ swap (5).
- Result [40, 35, 27, 33, 18, 12, 9, 21].

Extraction 1: swap 40 $\leftrightarrow$ 21 $\rightarrow$ [21, 35, 27, 33, 18, 12, 9 | 40]; sift 21: $\leftrightarrow$ 35, $\leftrightarrow$ 33 $\rightarrow$ [35, 33, 27, 21, 18, 12, 9 | 40].

Extraction 2: swap 35 $\leftrightarrow$ 9 $\rightarrow$ [9, 33, 27, 21, 18, 12 | 35, 40]; sift 9: $\leftrightarrow$ 33, $\leftrightarrow$ 21 $\rightarrow$ [33, 21, 27, 9, 18, 12 | 35, 40].

- (A) **TRUE.**
- (B) The heap part [33, 21, 27, 9, 18, 12] satisfies the heap property, and the sorted tail is 35, 40. **TRUE.**
- (C) **TRUE.**
- (D) Build-heap made **5** swaps (the root's 21 sank three levels). **FALSE.**

Trap: stopping sift-down after one swap; 21 must keep sinking while a child is larger.



Max-heap after build-heap

Q.14 [NAT · 2 marks · Merge sort — comparisons on structured input]

Answer: 39

Merging two runs where every element of one run is smaller than every element of the other costs only $\min(p, q)$ comparisons; a perfectly interleaved merge costs $p + q - 1$.

Left half [2, 4, ..., 16] is already sorted. At every merge inside it, the left run is entirely smaller than the right run, so each merge of two runs of length s costs s :

- 4 merges of $1+1 \rightarrow 4 \times 1 = 4$
- 2 merges of $2+2 \rightarrow 2 \times 2 = 4$
- 1 merge of $4+4 \rightarrow 4$
- Left half total = 12.

The right half [1, 3, ..., 15] is also sorted → another 12.

Final merge of [2, 4, ..., 16] with [1, 3, ..., 15]: the outputs alternate 1, 2, 3, ...; the right run is exhausted when 15 is output, i.e. after 15 elements, each needing one comparison, and 16 is copied → 15 comparisons.

Total = 12 + 12 + 15 = **39**.

For reference: best case for $n = 16$ is $(n/2) \log_2 n = 32$ and worst case is $n \log_2 n - n + 1 = 49$.

Trap: assuming the final merge is also a best case (8) because both halves were sorted — interleaved values force the maximum $p + q - 1$.

Comparisons per merge level

	1+1	2+2	4+4	8+8
left half	4	4	4	—
right half	4	4	4	—
final	—	—	—	15

Q.15 [MCQ · 2 marks · Recurrences — change of variable]

Answer: (B)

Substitute $m = \log_2 n$, i.e. $n = 2^m$, so $\sqrt{n} = 2^{m/2}$. Define $S(m) = T(2^m)$. Then $S(m) = 2S(m/2) + m$.

This is the merge-sort recurrence: by the master theorem ($a = 2$, $b = 2$, $f(m) = m = \Theta(m^{\log_2 2})$) $S(m) = \Theta(m \log m)$.

Translating back: $T(n) = S(\log n) = \Theta(\log n \cdot \log \log n) \rightarrow (B)$.

Recursion-tree view: the depth is $\log \log n$ (taking square roots repeatedly until the argument is constant); at depth i there are 2^i subproblems, each of size $n^{1/2^i}$, so each costs $(\log n)/2^i$; every level sums to $\log n$.

Option analysis:

- (A) would hold for $T(n) = T(\sqrt{n}) + \log n$ (one recursive call: geometric sum).
- (C) confuses $\sqrt{n}$ in the argument with the cost.
- (D) would need $\log n$ levels, but there are only $\log \log n$.

Tip: recurrences on $\sqrt{n}$ become ordinary divide-and-conquer recurrences after the substitution $m = \log n$.

Q.16 [NAT · 2 marks · BFS — distances in a rule-defined graph]

Answer: 26

Neighbours of i are $i \pm 3$ and $i \pm 5$ (within $0 \dots 11$). BFS level by level:

- Level 0: {0}.
- Level 1: neighbours of 0 → {3, 5}.
- Level 2: from 3 → 6, 8; from 5 → 2, 8, 10 → {2, 6, 8, 10}.
- Level 3: from 2 → 7; from 6 → 1, 9, 11; from 8 → 11; from 10 → 7 → {1, 7, 9, 11}.
- Level 4: from 1 → 4; (4 is not adjacent to anything earlier: $4 \pm 3 = 1, 7$ and $4 \pm 5 = 9$ are all level 3) → {4}.

All 12 vertices are reached (G is connected). Sum = $0 \cdot 1 + 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 4 + 4 \cdot 1 = 0 + 2 + 8 + 12 + 4 = \mathbf{26}$.

Observation: every edge changes the parity of the vertex by 3 or 5 (both odd), so G is **bipartite** (even vs odd vertices) — consistent with even vertices appearing only on even levels.

Trap: stopping at level 3 and missing vertex 4, or treating $0 + 5 + 3 = 8$ as two distinct routes (BFS counts each vertex once).

BFS distance from 0												
	0	1	2	3	4	5	6	7	8	9	10	11
dist	0	3	2	1	4	1	2	3	2	3	2	3

Q.17 [MCQ · 2 marks · Dijkstra — counting distance updates]

Answer: (D)

Dijkstra relaxes the out-edges of each vertex when it is extracted; $d[T]$ changes only when a strictly shorter route is found.

- Extract S (0): $d[A] = 1$, $d[B] = 4$, $d[T] = 15 \rightarrow$ **update 1**.
- Extract A (1): $d[B] = \min(4, 3) = 3$, $d[C] = 7$, $d[T] = \min(15, 1 + 12 = 13) \rightarrow$ **update 2**.
- Extract B (3): $d[C] = \min(7, 6) = 6$; $d[T]: 3 + 10 = 13$ is **not** smaller than 13 $\rightarrow$ no update.
- Extract C (6): $d[T] = \min(13, 6 + 2 = 8) \rightarrow$ **update 3**.
- Extract T (8).

$d[T]$ is updated **3** times ($15 \rightarrow 13 \rightarrow 8$) $\rightarrow$ (D). Final shortest path $S \rightarrow A \rightarrow B \rightarrow C \rightarrow T$ with cost $1 + 2 + 3 + 2 = 8$.

Option analysis:

- (A) 2 forgets the initial assignment from ∞ .
- (B) 4 counts the tie $13 = 13$ at B as an update.
- (C) 5 counts one update per incoming edge of T, plus one.

Trap: with $<$ in the relaxation test, equal-cost alternatives never trigger an update.

Q.18 [MSQ · 2 marks · Hashing — deletion with tombstones]

Answer: (A), (C), (D)

Insertions: $31 \rightarrow 1$; $41 \rightarrow 1$ taken $\rightarrow 2$; $51 \rightarrow 1$, 2 taken $\rightarrow 3$; $22 \rightarrow 2$, 3 taken $\rightarrow 4$; $61 \rightarrow 1$, 2, 3, 4 taken $\rightarrow 5$. Then slot 2 (41) becomes DELETED.

Table: 1: 31, 2: DEL, 3: 51, 4: 22, 5: 61, rest EMPTY.

- (A) Insert 71: slot 1 occupied, slot 2 DELETED $\rightarrow$ stored in slot 2 (reusing the tombstone). **TRUE**.
- (B) Search 22: slots 2 (DEL), 3 (51), 4 (22) $\rightarrow$ 3 slots, not 2. **FALSE**.
- (C) Search 61: slots 1 (31), 2 (DEL, skip), 3 (51), 4 (22), 5 (61, found) $\rightarrow$ 5 slots. **TRUE**.
- (D) Search 51 starts at slot 1 (31) and then reaches slot 2; if it were EMPTY, the search would stop and report “absent” although 51 sits in slot 3. This is exactly why tombstones are needed. **TRUE**.

Trap: in (B), forgetting that a tombstone still costs a probe. Tombstones keep searches correct but make them longer — tables with many deletions need periodic rehashing.

0	
1	31
2	DEL
3	51
4	22
5	61
6	
7	
8	
9	

Table after deleting 41

Q.19 [NAT · 2 marks · Stacks — counting output sequences]**Answer: 9**

For 3 to be output first, the operations must start push 1, push 2, push 3, pop. At that moment the stack holds 1, 2 (2 on top) and 4, 5 are still to be pushed.

Constraint: 2 must be output before 1. The values 4 and 5 can be pushed at any time; count by when 2 and 1 are popped relative to 4 and 5:

- 2 1 then 4, 5: 2 1 4 5, 2 1 5 4 → 2
- 2 first, then 1 only after 4 and/or 5 has been output: 2 4 1 5, 2 4 5 1, 2 5 4 1 → 3
- 4 first: 4 2 1 5, 4 2 5 1, 4 5 2 1 → 3
- 5 first (4 must be on the stack below it): 5 4 2 1 → 1

Total = 2 + 3 + 3 + 1 = **9**.

Check of an excluded case: 2 5 1 4 is impossible — after 5 is popped, 4 is on top of 1, so 1 cannot precede 4.

Cross-check: summing such counts over all possible first elements gives the total number of stack permutations of 5 elements, the Catalan number $C_5 = 42$ (first element 1: 14, 2: 14, 3: 9, 4: 4, 5: 1).

Trap: counting $4!/2 = 12$ by only enforcing “2 before 1” — the order of 4 and 5 relative to 1 and 2 is also constrained.

Q.20 [MSQ · 2 marks · Linked lists — moving the tail to the front]**Answer: (A), (C)**

Each iteration walks to the last node (keeping its predecessor), detaches it and makes it the new head: one **right rotation** by one position. k iterations rotate right by k .

- (A) With one node, the while-loop body never runs, so `prev` stays `None` and `prev.next = None` raises `AttributeError ('NoneType' object has no attribute 'next')`. **TRUE.**
- (B) 8 right rotations of a 6-node list = $8 \bmod 6 = 2$ rotations → 5 6 1 2 3 4. The stated list (3 4 5 6 1 2) is the result of rotating **left** by 2. **FALSE.**
- (C) Two right rotations of 1...6: 6 1 2 3 4 5 → 5 6 1 2 3 4. **TRUE.**
- (D) In each iteration the inner loop advances from the head to the last node: $n - 1 = 5$ steps. Two iterations → 10, not 12. **FALSE.**

Cost: $\Theta(nk)$ overall. A better approach computes $k \bmod n$, finds the new tail in one pass and re-links in $O(n)$.

Trap: confusing a right rotation (tail to front) with a left rotation (head to back) in (B).

Mock Test 42

Full-Syllabus Mock — Paper 12

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — list repetition and aliasing]

Consider the following Python program. What is printed?

Python

```
1 rows = [[]] * 3
2 rows[1].append(4)
3 rows += [[]]
4 rows[3].append(7)
5 rows[0] = rows[0] + [9]
6 print(rows)
```

- (A) `[[9], [4], [], [7]]`
- (B) `[[4, 9], [4], [4], [4, 7]]`
- (C) `[[4, 9], [4, 9], [4, 9], [7]]`
- (D) `[[4, 9], [4], [4], [7]]`

Q.2 [NAT · numerical answer | 1 mark | Stacks and queues — interleaved operations]

Consider the following Python program. The value printed is _____.

Python

```
1 from collections import deque
2
3 s, q = [], deque()
4 for x in range(1, 9):
5     if x % 3 == 0:
6         q.append(s.pop())
7     else:
8         s.append(x)
9         if x % 4 == 0:
10            s.append(q.popleft())
11 print(sum(s[:2]) - len(q))
```

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Linked lists — operation costs]

A singly linked list of n nodes is maintained with both a **head** and a **tail** pointer (each node stores only a value and a **next** pointer). Which of the following operations can be performed in $O(1)$ worst-case time, independent of n ?

- (A) Insert a new node at the front
- (B) Insert a new node at the end
- (C) Delete the last node (and update **tail**)
- (D) Concatenate a second such list (with its own head and tail) to the end of this one

Q.4 [NAT · numerical answer | 1 mark | Binary trees — counting shapes of maximum height]

The number of structurally distinct binary trees with 5 nodes whose height is 4 (height = number of edges on the longest root-to-leaf path) is _____.

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Hashing — reconstructing the insertion order]

A hash table of size 10 uses $h(k) = k \bmod 10$ and linear probing. After six insertions into an initially empty table, the table is as shown below. Which insertion order produces this table?

0	
1	41
2	52
3	13
4	31
5	22
6	64
7	
8	
9	

Final table

- (A) 52, 41, 13, 31, 22, 64
- (B) 41, 52, 31, 13, 22, 64
- (C) 13, 52, 41, 22, 31, 64
- (D) 52, 13, 41, 22, 31, 64

Q.6 [NAT · numerical answer | 1 mark | Binary search on the answer]

Packages with the given weights must be shipped **in order**, using one truck trip per day; a day's load is a contiguous group of packages whose total weight must not exceed the truck capacity. The program finds the minimum capacity that allows shipping within D days. The value printed is _____.

Python

```

1 W, D = [4, 9, 3, 6, 8, 2, 7], 3
2
3 def days(cap):
4     d, cur = 1, 0
5     for w in W:
6         if cur + w > cap:
7             d, cur = d + 1, 0
8         cur += w
9     return d
10
11 lo, hi = max(W), sum(W)
12 while lo < hi:
13     mid = (lo + hi) // 2
14     if days(mid) <= D:
15         hi = mid
16     else:
17         lo = mid + 1
18 print(lo)

```

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Insertion sort — partial passes]

Insertion sort processes $A = [8, 3, 6, 1, 9, 2]$ by inserting $A[1]$, then $A[2]$, then $A[3]$, ... into the sorted prefix. The array immediately after $A[3]$ has been inserted (three insertions) is

- (A) $[3, 6, 8, 1, 9, 2]$
- (B) $[1, 3, 6, 8, 9, 2]$
- (C) $[1, 3, 6, 8, 2, 9]$
- (D) $[1, 2, 3, 8, 9, 6]$

Q.8 [MSQ · one or more correct | 1 mark | Graph representations — adjacency matrix powers]

A is the adjacency matrix of a simple undirected graph G on vertices 1..5, shown below. Which of the following statements is/are TRUE?

Adjacency matrix A

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	1	1
4	0	1	1	0	1
5	0	0	1	1	0

- (A) G has 7 edges
- (B) $(A^3)[1][1] = 1$
- (C) G contains exactly 3 triangles
- (D) $(A^2)[1][4] = 2$

Q.9 [NAT · numerical answer | 1 mark | Heaps — 3-ary max-heap insertion]

A **3-ary** max-heap is stored in an array (0-based): the children of index i are $3i + 1$, $3i + 2$, $3i + 3$, and the parent of index $i > 0$ is $\lfloor (i - 1)/3 \rfloor$. The heap is

$[90, 70, 85, 60, 50, 40, 65, 30, 20, 45, 55]$

First 88 is inserted, then 95 is inserted (each by appending and sifting up). The index at which 88 finally resides is _____.

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Recurrences — exact evaluation]

Let $T(1) = 1$ and $T(n) = 2T(n/2) + n$ for n a power of 2. The value of $T(64)$ is

- (A) 448
- (B) 512
- (C) 384
- (D) 320

Q.11 [MCQ · single correct | 2 marks | Python — lazy generator pipelines]

Consider the following Python program. What is printed?

Python

```
1 def gen(xs):
2     for x in xs:
3         print("g", x, end=" ")
4         yield x * x
5
6 it = gen([1, 2, 3, 4])
7 ev = (y for y in it if y % 2 == 0)
8 print(next(ev), end=" ")
9 print(sum(it))
```

- (A) g 1 g 2 g 3 g 4 4 25
- (B) g 1 g 2 4 g 3 g 4 29
- (C) g 1 g 2 4 g 3 g 4 25
- (D) g 1 g 2 4 g 3 g 4 16

Q.12 [NAT · numerical answer | 2 marks | Quicksort — comparisons and the BST connection]

The quicksort below always uses the **first** element as pivot and makes exactly one comparison per non-pivot element in each call. It is run on [5, 8, 2, 9, 1, 7, 3, 6, 4]. The value printed is _____.

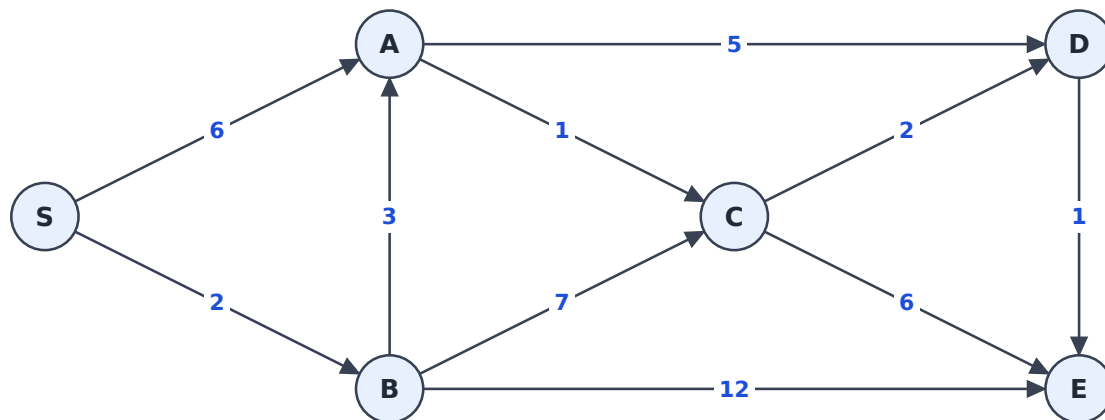
Python

```
1 comps = 0
2
3 def qs(a):
4     global comps
5     if len(a) <= 1:
6         return a
7     p, L, R = a[0], [], []
8     for x in a[1:]:
9         comps += 1
10        (L if x < p else R).append(x)
11    return qs(L) + [p] + qs(R)
12
13 qs([5, 8, 2, 9, 1, 7, 3, 6, 4])
14 print(comps)
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Shortest paths — Dijkstra relaxations]

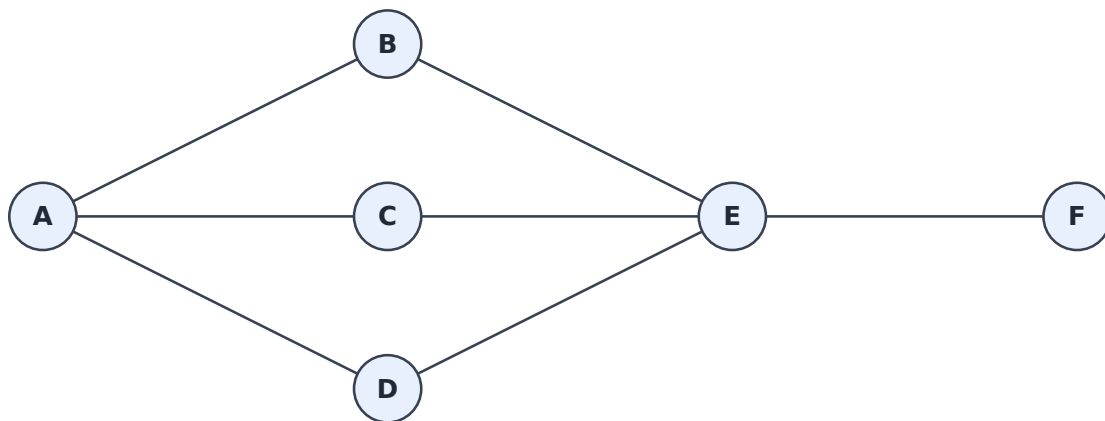
Dijkstra's algorithm is run from S on the weighted directed graph below; a tentative distance is changed only when it strictly decreases. Which of the following statements is/are TRUE?



- (A) The shortest distance from S to E is 9
- (B) The shortest path from S to E uses exactly 4 edges
- (C) The vertices are finalised in the order S, B, A, C, D, E
- (D) Before E is finalised, its tentative distance takes exactly two distinct finite values

Q.14 [NAT · numerical answer | 2 marks | DFS — counting possible visiting orders]

Depth-first search is started at vertex A of the undirected graph below. At each step the algorithm may move to **any** unvisited neighbour of the current vertex (backtracking when none exists). The number of distinct orders in which the six vertices can be visited (DFS pre-orders) is _____.



Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Python — dictionary keys, hashing and equality]

Consider the following Python program. What is printed?

Python

```
1 d = {1: 'a', True: 'b', 1.0: 'c', 2: 'd'}
2 d[2.0] = 'e'
3 print(len(d), d[1], list(d)[0])
```

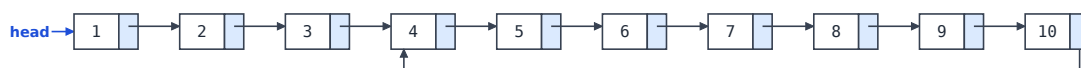
- (A) 4 c 1
- (B) 2 a 1
- (C) 2 c 1
- (D) 2 c True

Q.16 [NAT · numerical answer | 2 marks | Linked lists — Floyd's cycle detection]

Nodes with values 1, 2, ..., 10 are linked in this order, and the **next** pointer of the node with value 10 points back to the node with value 4 (see figure). The program below runs Floyd's tortoise-and-hare loop. The value printed is _____.

Python

```
1 slow = fast = head
2 while True:
3     slow = slow.nx
4     fast = fast.nx.nx
5     if slow is fast:
6         break
7 print(slow.v)
```



Node 10's next pointer goes back to node 4

Answer: _____ (enter a numerical value)

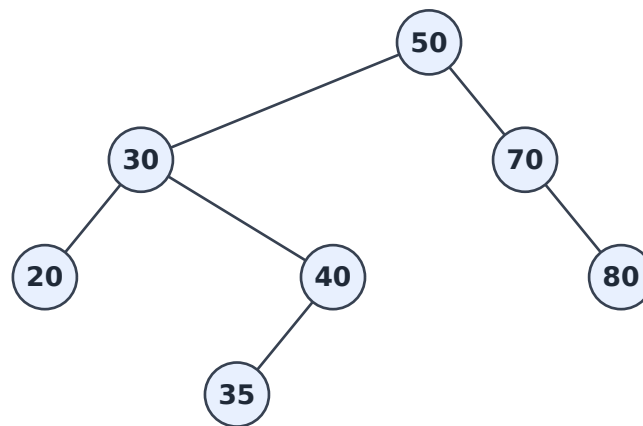
Q.17 [MSQ · one or more correct | 2 marks | Elementary sorts — exact comparison counts]

For an array of n distinct keys, consider these standard implementations: insertion sort (inner loop **while $j \geq 0$ and $a[j] > \text{key}$**), selection sort (scan the unsorted suffix for the minimum), bubble sort with early exit (stop after a pass with no swap), and top-down merge sort. Which of the following statements is/are TRUE?

- (A) Selection sort makes exactly $n(n - 1)/2$ key comparisons on every input
- (B) Merge sort, as usually implemented on arrays, sorts in place using $O(1)$ extra memory
- (C) Insertion sort makes exactly $n - 1$ key comparisons on an already-sorted array
- (D) Bubble sort with early exit makes exactly $n(n - 1)/2$ key comparisons on a reverse-sorted array

Q.18 [NAT · numerical answer | 2 marks | Binary search trees — insertion orders producing a tree]

The number of permutations of the keys {20, 30, 35, 40, 50, 70, 80} which, when inserted in that order into an initially empty BST, produce exactly the tree shown below is _____.



Target BST

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Stacks — infix to prefix conversion]

Operators have the usual precedence: ^ (highest, right-associative) > *, / (left-associative) > +, - (left-associative). The prefix (Polish) form of

$(a + b) * c - d / (e - f) ^ g$

is

- (A) - * + a b c ^ / d - e f g
- (B) - * + a b c / d ^ - e f g
- (C) * + a b - c / d ^ - e f g
- (D) - + a b * c / d ^ - e f g

Q.20 [MSQ · one or more correct | 2 marks | Graph theory — edges, components and connectivity]

Consider simple undirected graphs (no loops, no multi-edges). Which of the following statements is/are TRUE?

- (A) A forest (acyclic graph) with 10 vertices and 3 connected components has exactly 8 edges
- (B) Every graph with 6 vertices in which every vertex has degree at least 2 is connected
- (C) Every graph with 10 vertices and 37 edges is connected
- (D) The maximum number of edges in a graph with 10 vertices and exactly 3 connected components is 28

Answer Key — Mock Test 42

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Python — list repetition and aliasing
2	NAT	1	11	Stacks and queues — interleaved operations
3	MSQ	1	(A), (B), (D)	Linked lists — operation costs
4	NAT	1	16	Binary trees — counting shapes of maximum height
5	MCQ	1	(A)	Hashing — reconstructing the insertion order
6	NAT	1	16	Binary search on the answer
7	MCQ	1	(B)	Insertion sort — partial passes
8	MSQ	1	(A), (C), (D)	Graph representations — adjacency matrix powers
9	NAT	1	12	Heaps — 3-ary max-heap insertion
10	MCQ	1	(A)	Recurrences — exact evaluation
11	MCQ	2	(C)	Python — lazy generator pipelines
12	NAT	2	16	Quicksort — comparisons and the BST connection
13	MSQ	2	(A), (C)	Shortest paths — Dijkstra relaxations
14	NAT	2	18	DFS — counting possible visiting orders
15	MCQ	2	(C)	Python — dictionary keys, hashing and equality
16	NAT	2	8	Linked lists — Floyd's cycle detection
17	MSQ	2	(A), (C), (D)	Elementary sorts — exact comparison counts
18	NAT	2	45	Binary search trees — insertion orders producing a tree
19	MCQ	2	(B)	Stacks — infix to prefix conversion
20	MSQ	2	(C), (D)	Graph theory — edges, components and connectivity

Detailed Solutions — Mock Test 42

Q.1 [MCQ · 1 mark · Python — list repetition and aliasing]

Answer: (D)

Concept: `[[ ]] * 3` creates a list holding **three references to one** inner list. In-place mutation through any of them is visible through all; *rebinding* a slot (`rows[0] = ...`) affects only that slot.

Let `L` be the shared inner list.

- `rows[1].append(4)` → `L = [4]`; `rows = [L, L, L]`.
- `rows += [[]]` extends `rows` with a *new*, separate empty list `M` → `[L, L, L, M]`.
- `rows[3].append(7)` → `M = [7]`.
- `rows[0] = rows[0] + [9]` builds a **new** list `[4, 9]` (the `+` operator copies) and stores it in slot 0 only; `L` is still `[4]`.

Output: `[[4, 9], [4], [4], [7]]`.

Options:

- (A) assumes the three inner lists were independent.
- (B) assumes `+= [[]]` appended another reference to `L`.
- (C) treats `rows[0] + [9]` as an in-place change of `L` (that would be `rows[0] += [9]`).

Trap: `x = x + y` and `x += y` differ for lists — the first creates a new object, the second mutates in place.

Q.2 [NAT · 1 mark · Stacks and queues — interleaved operations]

Answer: 11

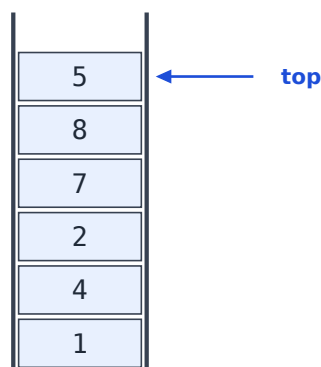
Concept: `s` is a stack (list: append/pop at the right end) and `q` a FIFO queue (append at the right, popleft at the left). Multiples of 3 move the stack top into the queue; multiples of 4 additionally move the queue front back onto the stack.

Trace (stack bottom → top | queue front → rear):

- `x = 1`: push → `[1] | []`
- `x = 2`: push → `[1, 2] | []`
- `x = 3`: pop 2 into `q` → `[1] | [2]`
- `x = 4`: push 4, then push `q.popleft() = 2` → `[1, 4, 2] | []`
- `x = 5`: push → `[1, 4, 2, 5] | []`
- `x = 6`: pop 5 into `q` → `[1, 4, 2] | [5]`
- `x = 7`: push → `[1, 4, 2, 7] | [5]`
- `x = 8`: push 8, then push 5 → `[1, 4, 2, 7, 8, 5] | []`

`s[::-2] = [1, 2, 8]` → sum 11; `len(q) = 0`. Printed: **11**.

Trap: popping from the wrong end — `q.pop()` vs `q.popleft()` coincide here only because `q` never holds two elements, but `s.pop(0)` instead of `s.pop()` would change everything.



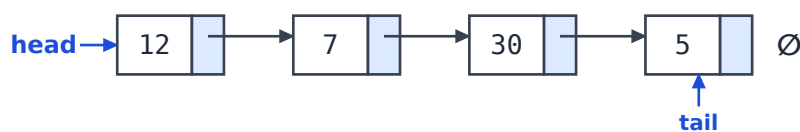
Final stack (queue is empty)

Q.3 [MSQ · 1 mark · Linked lists — operation costs]**Answer: (A), (B), (D)**

Concept: an $O(1)$ operation may only touch a constant number of nodes reachable from the stored pointers. A singly linked node cannot reach its predecessor.

- (A) `new.next = head; head = new.` **$O(1)$.**
- (B) `tail.next = new; tail = new.` **$O(1)$.**
- (C) After removing the last node, `tail` must point to the *second-to-last* node, and the only way to find it is to walk from `head` — $\Theta(n)$. **Not $O(1)$.**
- (D) `tail1.next = head2; tail1 = tail2.` **$O(1)$.** (This is why linked lists, unlike arrays, concatenate cheaply.)

Trap: assuming the tail pointer makes *deletion* at the tail cheap. It would require a doubly linked list (a `prev` pointer) — then all four operations are $O(1)$.



Deleting 5 needs the node 30, reachable only from head

Q.4 [NAT · 1 mark · Binary trees — counting shapes of maximum height]**Answer: 16**

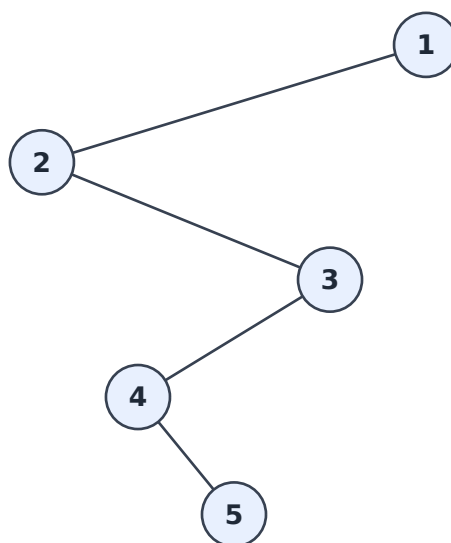
Concept: a binary tree with n nodes has height at most $n - 1$, attained exactly when the tree is a single path (every non-leaf node has exactly one child).

With 5 nodes and height 4, all 5 nodes lie on one root-to-leaf path, so each of the 4 non-leaf nodes has exactly one child, which can be either a **left** or a **right** child. These choices are independent and each gives a different structure:

$$2 \times 2 \times 2 \times 2 = 2^4 = \mathbf{16}.$$

For comparison, there are $\text{Catalan}(5) = 42$ binary trees with 5 nodes in total; 16 of them have height 4 (and 20 have height 3, 6 have height 2).

Trap: answering 1 or 2 (only the all-left / all-right chains) — zig-zag paths count as different structures.



One of the 16 shapes (choices: L, R, L, R)

Q.5 [MCQ · 1 mark · Hashing — reconstructing the insertion order]**Answer: (A)**

Concept: a key sitting in its home slot must have been inserted while that slot was free; a displaced key must have found every slot from its home up to its final position already occupied.

Home slots: 41→1, 52→2, 13→3, 31→1, 22→2, 64→4.

- 41, 52, 13 are at home.
- 31 is at 4, so slots 1, 2, 3 were full before 31 → 31 comes after 41, 52 **and 13**.
- 22 is at 5, so slots 2, 3, 4 were full → 22 comes after 52, 13 **and 31**.
- 64 is at 6, so 4 and 5 were full → 64 comes after 31 and 22.

Check options:

- (A) 52, 41, 13, 31 (→ 4), 22 (→ 5), 64 (→ 6). ✓
- (B) 31 is inserted before 13 → 31 lands in slot 3. ✗
- (C) 22 before 31 → 22 lands in slot 4. ✗
- (D) same problem as (C). ✗

Trap: checking only the home-slot keys; the order constraints come from the displaced keys.

Q.6 [NAT · 1 mark · Binary search on the answer]**Answer: 16**

Concept: `days(cap)` is non-increasing in `cap`, so the feasible capacities form a suffix $[c^*, \infty)$. Binary search over the *answer range* $[\max(W), \text{sum}(W)] = [9, 39]$ finds the smallest feasible c^* .

Greedy day counts for candidate capacities:

- cap 15: [4, 9] (13) | [3, 6] (9; adding 8 → 17) | [8, 2] (10; adding 7 → 17) | [7] → 4 days ✗
- cap 16: [4, 9, 3] (16) | [6, 8, 2] (16) | [7] → 3 days ✓

Binary search trace: (9, 39) mid 24 ✓ → hi 24; mid 16 ✓ → hi 16; mid 12 ✗ → lo 13; mid 14 ✗ → lo 15; mid 15 ✗ → lo 16 = hi. Prints **16**.

Lower bound check: total 39 over 3 days needs at least $\lceil 39/3 \rceil = 13$, but contiguity forces more.

Trap: answering 13 (ignoring that packages must stay in order) or 9 (the heaviest package only).

Q.7 [MCQ · 1 mark · Insertion sort — partial passes]**Answer: (B)**

Concept: after inserting `A[i]`, the prefix `A[0..i]` is sorted and contains exactly the *original* first $i + 1$ elements; the suffix is untouched.

- Insert 3: [3, 8, 6, 1, 9, 2]
- Insert 6: [3, 6, 8, 1, 9, 2]
- Insert 1: 1 shifts past 8, 6, 3 → [1, 3, 6, 8, 9, 2]

Answer **[1, 3, 6, 8, 9, 2]** (B).

Options:

- (A) is the state after only two insertions.
- (C) changes the suffix, which insertion sort never touches before reaching it.
- (D) is selection sort after three passes — it places the global minima 1, 2, 3 at the front, which insertion sort does not.

Tip: insertion sort's invariant is 'prefix sorted', selection sort's is 'prefix sorted **and** final'.

Q.8 [MSQ · 1 mark · Graph representations — adjacency matrix powers]**Answer: (A), (C), (D)**

Concept: $(A^k)[i][j]$ counts walks of length k from i to j . In particular $(A^2)[i][j]$ ($i \neq j$) counts common neighbours, and $(A^3)[i][i] = 2 \times (\text{number of triangles through } i)$, since each triangle can be walked in two directions.

Edges (upper triangle): 1-2, 1-3, 2-3, 2-4, 3-4, 3-5, 4-5.

- (A) 7 ones above the diagonal → **7 edges**. True.
- (B) Vertex 1 lies on one triangle (1-2-3), giving the closed walks $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ and $1 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow (A^3)[1][1] = 2$. False.
- (C) Triangles: $\{1, 2, 3\}$, $\{2, 3, 4\}$, $\{3, 4, 5\}$. ($\{1, 3, 4\}$? 1-4 missing. $\{2, 4, 5\}$? 2-5 missing.) Exactly **3**. True.
- (D) $N(1) = \{2, 3\}$, $N(4) = \{2, 3, 5\}$: common neighbours $\{2, 3\} \rightarrow 2$. True.

Trap: in (B), forgetting that each triangle yields two closed walks (one per direction). Total triangles = $\text{trace}(A^3)/6$.

Q.9 [NAT · 1 mark · Heaps — 3-ary max-heap insertion]**Answer: 12**

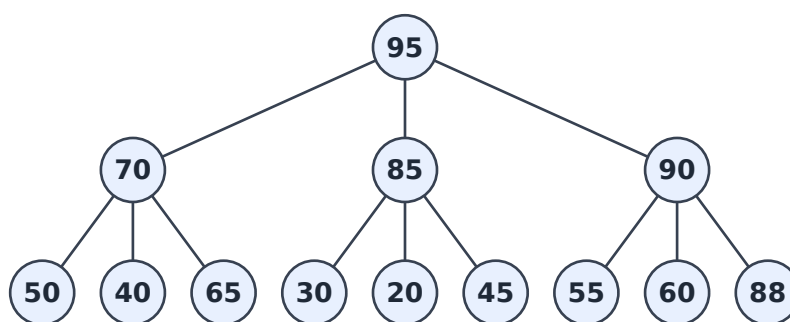
Concept: sift-up compares a node only with its parent $\lfloor (i-1)/3 \rfloor$ and swaps while the parent is smaller. A later insertion can push an earlier key back **down** its path.

Insert 88: appended at index 11. Parent $\lfloor 10/3 \rfloor = 3$ (60) < 88 → swap → 88 at index 3. Parent $\lfloor 2/3 \rfloor = 0$ (90) > 88 → stop. Heap: [90, 70, 85, 88, 50, 40, 65, 30, 20, 45, 55, 60].

Insert 95: appended at index 12. Parent $\lfloor 11/3 \rfloor = 3$ (88) < 95 → swap → 95 at index 3, 88 moves to index 12. Parent 0 (90) < 95 → swap → 95 at the root, 90 at index 3.

Final: [95, 70, 85, 90, 50, 40, 65, 30, 20, 45, 55, 60, 88]; 88 sits at index **12**.

Trap: using binary-heap parent formula $\lfloor (i-1)/2 \rfloor$, or answering 3 by forgetting that 95's sift-up path passes through 88's slot.



3-ary heap after both insertions (88 is the last leaf, index 12)

Q.10 [MCQ · 1 mark · Recurrences — exact evaluation]**Answer: (A)**

Concept: unrolling the merge-sort-type recurrence: there are $\log_2 n$ levels each contributing n , plus n leaves each costing $T(1) = 1$, so $T(n) = n \log_2 n + n$.

Direct evaluation:

- $T(2) = 2 \cdot 1 + 2 = 4$
- $T(4) = 2 \cdot 4 + 4 = 12$
- $T(8) = 2 \cdot 12 + 8 = 32$
- $T(16) = 2 \cdot 32 + 16 = 80$
- $T(32) = 2 \cdot 80 + 32 = 192$

- $T(64) = 2 \cdot 192 + 64 = \mathbf{448}$

Closed form: $64 \cdot 6 + 64 = 448$. ✓

Options: (C) $384 = n \log_2 n$ forgets the leaf level. (B) $512 = n(\log_2 n + 2)$ over-counts a level. (D) $320 = n(\log_2 n - 1)$ under-counts.

Tip: the leaf contribution $n \cdot T(1)$ matters for exact values even though it does not change $\Theta(n \log n)$.

Q.11 [MCQ · 2 marks · Python — lazy generator pipelines]

Answer: (C)

Concept: generators are lazy — code inside `gen` runs only when a value is requested. `ev` does not copy `it`; it pulls from the **same** generator, so whatever `ev` consumes is gone for later consumers.

Trace:

- Creating `it` and `ev` prints nothing.
- `next(ev)` pulls from `it`: prints `g 1`, yields 1 (odd, rejected by the filter); pulls again: prints `g 2`, yields 4 (even) → `next(ev)` returns 4 and `print(4, end=" ")`.
- `sum(it)` resumes the generator **after** `x = 2`: prints `g 3` (yields 9), `g 4` (yields 16); the sum is $9 + 16 = 25$.

Output: `g 1 g 2 4 g 3 g 4 25`.

Options:

- (A) assumes the whole generator runs eagerly before anything is printed.
- (B) adds the rejected value 4 again ($4 + 9 + 16 = 29$) — but it was already consumed.
- (D) thinks `sum(it)` sums only the even squares (16) — `it` itself is unfiltered.

Trap: believing that `ev` and `it` are independent iterators.

Q.12 [NAT · 2 marks · Quicksort — comparisons and the BST connection]

Answer: 16

Concept: this partition is stable, so each subarray keeps the original relative order and its first element becomes the next pivot. Hence the pivots form exactly the BST obtained by inserting the keys in the given order, and an element is compared with each of its BST **ancestors** exactly once. Total comparisons = sum of node depths (internal path length).

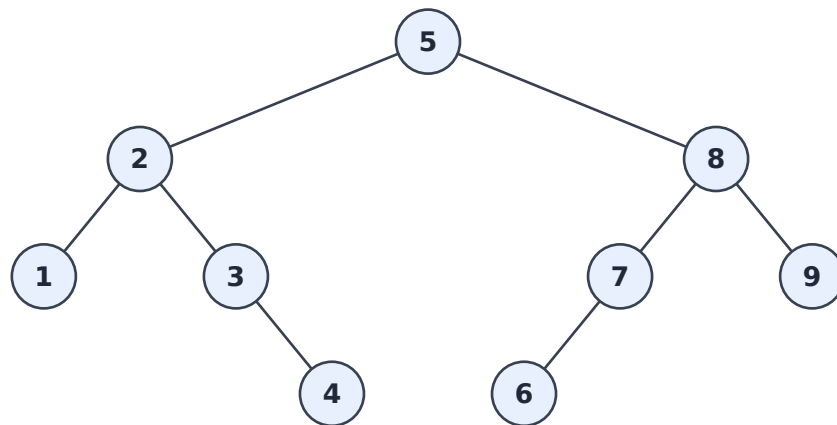
Insert 5, 8, 2, 9, 1, 7, 3, 6, 4 into a BST:

- depth 0: 5
- depth 1: 2, 8
- depth 2: 1, 3 (under 2); 7, 9 (under 8)
- depth 3: 4 (right of 3), 6 (left of 7)

Sum of depths = $2 \cdot 1 + 4 \cdot 2 + 2 \cdot 3 = 2 + 8 + 6 = \mathbf{16}$.

Check by calls: [5...] 8 comps; [2, 1, 3, 4] 3; [8, 9, 7, 6] 3; [3, 4] 1; [7, 6] 1 → $8 + 3 + 3 + 1 + 1 = 16$. ✓

Trap: using $n \log_2 n \approx 28.5$ or the worst case 36 instead of tracing; or counting two comparisons per element (as a two-comprehension partition would).



Pivot tree = BST of the input order

Q.13 [MSQ · 2 marks · Shortest paths — Dijkstra relaxations]

Answer: (A), (C)

Concept: each extraction finalises the closest unfinished vertex and may lower the tentative distances of its out-neighbours.

Trace (changes in tentative distances):

- Extract S (0): A = 6, B = 2.
- Extract B (2): A = $\min(6, 5) = 5$; C = 9; E = 14.
- Extract A (5): C = $\min(9, 6) = 6$; D = 10.
- Extract C (6): D = $\min(10, 8) = 8$; E = $\min(14, 12) = 12$.
- Extract D (8): E = $\min(12, 9) = 9$.
- Extract E (9).

Verdicts:

- (A) $d(E) = 9$. **True.**
- (B) Path: S → B → A → C → D → E ($2 + 3 + 1 + 2 + 1 = 9$) has **5** edges. **False.**
- (C) Order S, B, A, C, D, E. **True.**
- (D) E's tentative value goes $14 \rightarrow 12 \rightarrow 9$: **three** finite values. **False.**

Trap: the direct-looking edges (S → A = 6, B → E = 12, C → E = 6) are all beaten by detours; shortest paths minimise weight, not the number of edges.

Tentative distances after each extraction

	S	A	B	C	D	E
S	0	6	2	∞	∞	∞
B	0	5	2	9	∞	14
A	0	5	2	6	10	14
C	0	5	2	6	8	12
D	0	5	2	6	8	9
E	0	5	2	6	8	9

Q.14 [NAT · 2 marks · DFS — counting possible visiting orders]

Answer: 18

Concept: count the choices DFS can make, being careful that after backtracking the set of unvisited neighbours may have shrunk.

- From A, the first move goes to one of B, C, D: **3** choices. Call it X.

- X's neighbours are A and E; A is visited, so DFS is forced to go to **E**.
- At E the unvisited neighbours are F and the two vertices of {B, C, D} other than X — call them Y, Z. Each of F, Y, Z has no unvisited neighbour when reached (Y's neighbours A and E are both visited; F's only neighbour is E). So DFS visits them one after another, backtracking to E each time, in **any** order: $3! = 6$ orders.
- After E, backtracking to X and then A finds nothing new.

Total = $3 \times 6 = 18$.

Example orders: A B E F C D, A B E C D F, A D E C F B, ...

Trap: thinking DFS from A could visit B, then go back to A and visit C directly — that would skip E, but DFS must exhaust B's branch first, and $B \rightarrow E$ is available. Another error is BFS-style counting ($3!$ choices at A $\times$...).

Q.15 [MCQ · 2 marks · Python — dictionary keys, hashing and equality]

Answer: (C)

Concept: a dict (a hash table) treats two keys as the same if they have equal hashes **and** compare equal. In Python $1 == \text{True} == 1.0$ and $\text{hash}(1) == \text{hash}(\text{True}) == \text{hash}(1.0)$, so they are a single key. When an existing key is assigned again, the **value** is replaced but the **original key object** is kept.

Building the literal left to right:

- $1: 'a' \rightarrow \{1: 'a'\}$
- $\text{True}: 'b' \rightarrow$ same key $\rightarrow \{1: 'b'\}$ (key stays the int 1)
- $1.0: 'c' \rightarrow$ same key $\rightarrow \{1: 'c'\}$
- $2: 'd' \rightarrow \{1: 'c', 2: 'd'\}$
- $d[2.0] = 'e' \rightarrow 2.0 == 2 \rightarrow \{1: 'c', 2: 'e'\}$

So $\text{len}(d) = 2$, $d[1] = 'c'$, and the first key in insertion order is the int 1. Output **2 c 1**.

Options: (A) treats 1, True, 1.0 as distinct keys. (B) keeps the first value instead of the last. (D) assumes the key object is replaced too.

Trap: the same rule makes $\{1, \text{True}, 1.0\}$ a set of size 1.

Q.16 [NAT · 2 marks · Linked lists — Floyd's cycle detection]

Answer: 8

Concept: let μ = number of nodes before the cycle and λ = cycle length. After k steps the tortoise is at position k and the hare at $2k$ (positions counted from the head, 0-based); they coincide once both are in the cycle and $2k - k = k$ is a multiple of λ . So k = the smallest multiple of λ that is $\geq \mu$.

Here the cycle is $4 \rightarrow 5 \rightarrow \dots \rightarrow 10 \rightarrow 4$: $\lambda = 7$, and $\mu = 3$ (nodes 1, 2, 3 lead into it).

Smallest multiple of 7 that is ≥ 3 is $k = 7$.

After 7 steps the tortoise has moved 7 nodes from node 1 $\rightarrow$ it is at node **8**.

Step check (tortoise / hare values):

- $1: 2 / 3 \cdot 2: 3 / 5 \cdot 3: 4 / 7 \cdot 4: 5 / 9 \cdot 5: 6 / 4 \cdot 6: 7 / 6 \cdot 7: 8 / 8 \checkmark$

Trap: expecting them to meet at the cycle entrance (node 4). That needs the second phase of Floyd's algorithm: reset one pointer to the head and move both one step at a time; they meet at node 4 after $\mu = 3$ steps.

Q.17 [MSQ · 2 marks · Elementary sorts — exact comparison counts]

Answer: (A), (C), (D)

Concept: count comparisons per pass.

- (A) Pass i scans $n - 1 - i$ elements regardless of the data: $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$. **True** — selection sort is not adaptive.
- (B) The standard array merge needs an auxiliary array of $\Theta(n)$. **False.**
- (C) Sorted input: each key is compared once with its left neighbour, the test fails, and the loop exits $\rightarrow$ 1 comparison for each of $i = 1..n - 1 \rightarrow n - 1$ total. **True** (the best case, $\Theta(n)$).
- (D) Reverse-sorted input: every pass swaps, so all $n - 1$ passes are needed, with $n - 1, n - 2, \dots, 1$ comparisons $\rightarrow n(n - 1)/2$. Early exit never triggers. **True.**

For $n = 8$: insertion (sorted) 7; selection 28; bubble (reversed) 28.

Trap: in (D), thinking the early-exit flag saves a final pass — it only helps when the array becomes sorted before the last pass, which cannot happen with reversed input (the smallest element moves left only one position per pass).

Q.18 [NAT · 2 marks · Binary search trees — insertion orders producing a tree]

Answer: 45

Concept: a permutation produces the tree iff every node is inserted before all its descendants. The root must come first; the left and right subtrees' insertion sequences can be **interleaved** arbitrarily. Hence

$N(T) = C(|L| + |R|, |L|) \cdot N(L) \cdot N(R)$, equivalently $n! / \prod (\text{subtree size of every node})$.

Subtree sizes: $50 \rightarrow 7, 30 \rightarrow 4, 20 \rightarrow 1, 40 \rightarrow 2, 35 \rightarrow 1, 70 \rightarrow 2, 80 \rightarrow 1$.

Recursive count:

- Subtree 40 (with 35): 1 order (40, 35).
- Subtree 30: root 30 then interleave $\{20\}$ (size 1) with $\{40, 35\}$ (size 2): $C(3, 1) = 3$.
- Subtree 70: 1 order (70, 80).
- Root 50: interleave left (size 4, 3 orders) with right (size 2, 1 order): $C(6, 2) \cdot 3 \cdot 1 = 15 \cdot 3 = \mathbf{45}$.

Formula check: $7! / (7 \cdot 4 \cdot 1 \cdot 2 \cdot 1 \cdot 2 \cdot 1) = 5040 / 112 = 45$. ✓

Trap: multiplying only $N(L) \cdot N(R) = 3$ (forgetting the interleavings) or using the number of BST shapes (Catalan numbers), which answers a different question.

Q.19 [MCQ · 2 marks · Stacks — infix to prefix conversion]

Answer: (B)

Concept: build the expression tree from precedence and associativity, then read it in pre-order (operator before its operands). (Equivalently: reverse the infix, swap brackets, convert with a stack, reverse the result — taking care with associativity.)

Parsing:

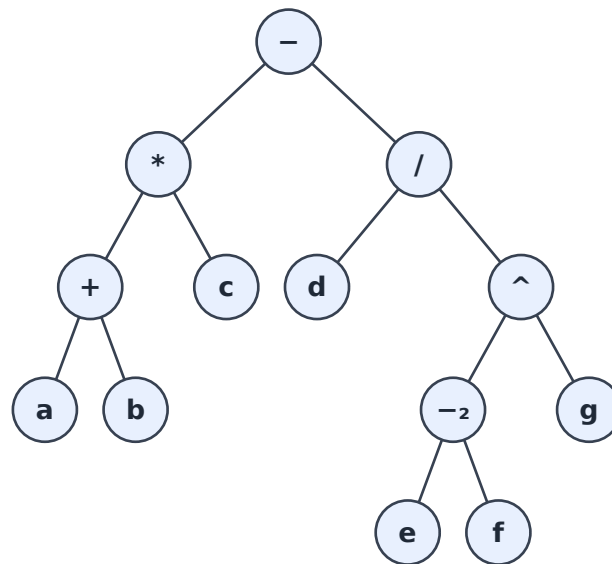
- $\wedge$ binds tightest: $(e - f) \wedge g$.
- $/$ next: $d / ((e - f) \wedge g)$.
- $*$: $(a + b) * c$.
- $-$ at the top: $[(a + b) * c] - [d / ((e - f) \wedge g)]$.

Pre-order: $-$, then left $* + a b c$, then right $/ d \wedge - e f g \rightarrow - * + a b c / d \wedge - e f g$, which is option (B).

Options:

- (A) groups $(d / (e - f)) \wedge g$, giving $/$ lower priority than it has relative to $\wedge$.
- (C) makes $*$ the root, i.e. $(a + b) * (c - \dots)$, ignoring that $-$ has lower precedence.
- (D) reads as $(a + b) - \dots$, dropping the multiplication by c into the right operand.

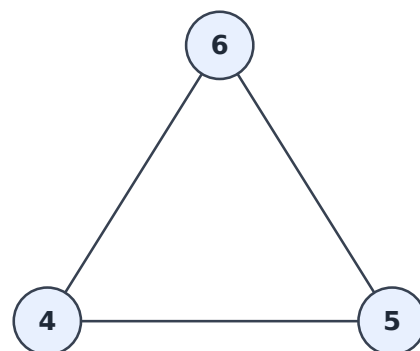
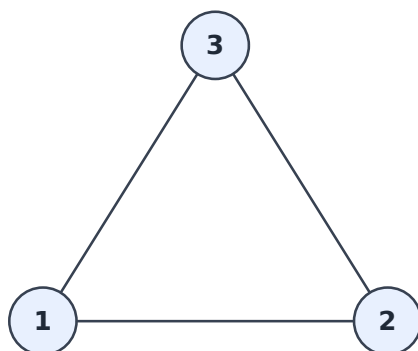
Tip: verify by evaluating with sample values, e.g. $a = 1, b = 2, c = 3, d = 8, e = 5, f = 3, g = 2$: infix value $9 - 8/4 = 7$, and the prefix in (B) gives 7 as well.

Expression tree ($-_2$ is the inner subtraction)**Q.20 [MSQ · 2 marks · Graph theory — edges, components and connectivity]****Answer: (C), (D)**

Concept: to maximise edges with k components on n vertices, make $k - 1$ components single vertices and the last one a complete graph K_{n-k+1} . A forest with c components on n vertices has $n - c$ edges.

- (A) A forest with 10 vertices and 3 trees has $10 - 3 = 7$ edges. **False.**
- (B) Two disjoint triangles: 6 vertices, all of degree 2, yet 2 components. **False.** (Minimum degree $\geq (n - 1)/2 = 2.5$, i.e. ≥ 3 , would guarantee connectivity.)
- (C) The maximum number of edges in a *disconnected* graph on 10 vertices is $C(9, 2) = 36$ (K_9 plus an isolated vertex). So 37 edges force connectivity. **True.**
- (D) K_8 plus two isolated vertices: $C(8, 2) = 28$ edges. Any more balanced split has fewer edges (e.g. $6 + 2 + 2$ vertices $\rightarrow 15 + 1 + 1$). **True.**

Trap: in (D), splitting the vertices evenly feels natural but minimises, not maximises, the edge count, because $C(m, 2)$ is convex.



Counterexample to (C): degree 2 everywhere, but disconnected

Mock Test 43

Full-Syllabus Mock — Paper 13

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — shallow vs deep copy]

Consider the following Python program. What is printed?

Python

```
1 import copy
2 a = [[1, 2], [3]]
3 b = a[:]
4 c = copy.deepcopy(a)
5 b[0].append(9)
6 b.append([7])
7 c[1].append(8)
8 print(a, len(b), c[1])
```

- (A) `[[1, 2, 9], [3], [7]] 3 [3, 8]`
- (B) `[[1, 2], [3]] 3 [3, 8]`
- (C) `[[1, 2, 9], [3]] 3 [3, 8]`
- (D) `[[1, 2, 9], [3, 8]] 3 [3, 8]`

Q.2 [NAT · numerical answer | 1 mark | Python — string slicing and searching]

Let `s = "datascience"`. The value of `len(set(s[::2])) + s.find('c') + s.rfind('e')` is _____.

Answer: _____ (enter a numerical value)

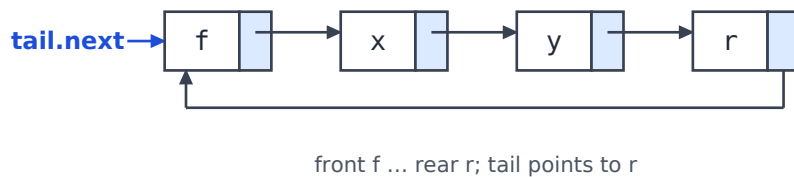
Q.3 [NAT · numerical answer | 1 mark | Stacks — counting pop sequences]

The integers 1, 2, 3, 4, 5 are pushed onto an initially empty stack in this order; pops may occur at any time and every popped value is printed, until all five are printed. The number of possible printed sequences whose **first** value is 3 is _____.

Answer: _____ (enter a numerical value)

Q.4 [MSQ · one or more correct | 1 mark | Queue as a circular linked list]

A queue is stored in a **circular** singly linked list, and only a pointer **tail** to the last (rear) node is kept; **tail.next** is the front node. Which of the following operations can be performed in $\Theta(1)$ worst-case time on a queue of n elements?



- (A) Dequeue the front element
- (B) Enqueue a new element at the rear
- (C) Delete the rear element (the node pointed to by tail)
- (D) Read the value of the front element

Q.5 [NAT · numerical answer | 1 mark | Counting BSTs with a fixed root]

The number of structurally distinct binary search trees that can store the keys 1, 2, 3, 4, 5 and have the key **3** at the root is _____.

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Hash functions for strings]

Strings are hashed into a table of size 11 by $h(s) = (\text{sum of the character codes of } s) \bmod 11$. Which of the following strings is **certain** to land in the same slot as "listen", without computing any character codes?

- (A) "silence"
- (B) "lister"
- (C) "listens"
- (D) "tinsel"

Q.7 [MCQ · single correct | 1 mark | Binary search — average successful cost]

Iterative binary search ($\text{mid} = \lfloor (\text{lo} + \text{hi})/2 \rfloor$, one three-way comparison per probe) is used on a sorted array of 15 distinct elements. Each element is equally likely to be the target, and the target is always present. The expected number of probes is

- (A) $49/15$
- (B) 4
- (C) $15/4$
- (D) 3

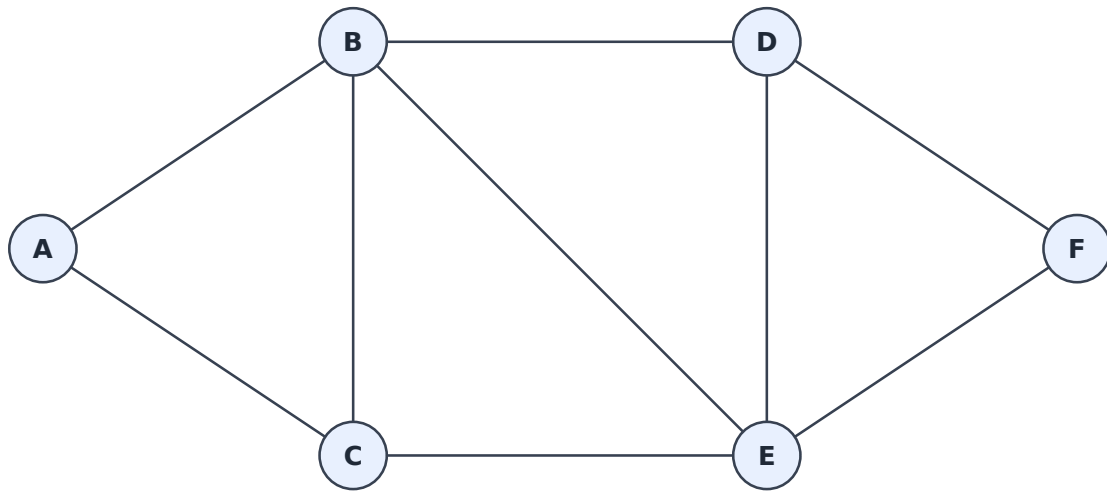
Q.8 [NAT · numerical answer | 1 mark | Insertion sort — comparisons]

Insertion sort (ascending) is run on $[4, 1, 3, 9, 7, 2]$. For each $i = 1, \dots, n - 1$ the key $A[i]$ is compared with $A[i-1], A[i-2], \dots$ moving left, shifting each larger element, and stopping at the first element $\leq$ key or at the left end. The total number of key comparisons is _____.

Answer: _____ (enter a numerical value)

Q.9 [MSQ · one or more correct | 1 mark | Degrees and Euler trails]

Consider the undirected graph G below. Which of the following statements is/are TRUE?



- (A) G has an Euler trail (a walk using every edge exactly once) but no Euler circuit
- (B) G is bipartite
- (C) G has exactly two vertices of odd degree
- (D) After deleting edge B-E, the resulting graph has an Euler circuit

Q.10 [MCQ · single correct | 1 mark | Recurrences — master theorem]

The solution of the recurrence $T(n) = 2T(n/4) + \sqrt{n}$, with $T(1) = 1$, is

- (A) $\Theta(\sqrt{n} \log n)$
- (B) $\Theta(\sqrt{n})$
- (C) $\Theta(n)$
- (D) $\Theta(n^{1/4} \log n)$

Q.11 [MCQ · single correct | 2 marks | Python — try / except / finally]

Consider the following Python program. What is printed?

Python

```

1 def f(x):
2     try:
3         if x > 0:
4             return 'pos'
5         raise ValueError
6     except ValueError:
7         return 'err'
8     finally:
9         print('F', end=' ')
10
11 def g():
12     try:
13         return 1
14     finally:
15         return 2
16
17 print(f(1), f(0), g())

```

- (A) pos F err F 1
- (B) F F pos err 2
- (C) F pos F err 2
- (D) F F pos err 1

Q.12 [NAT · numerical answer | 2 marks | Monotonic stack — next greater element]

Consider the following Python program. The value printed is _____.

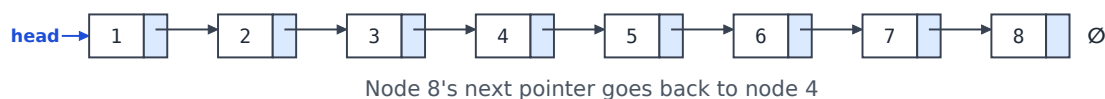
Python

```
1 def nge(a):
2     st, res = [], [-1] * len(a)
3     for i, x in enumerate(a):
4         while st and a[st[-1]] < x:
5             res[st.pop()] = x
6         st.append(i)
7     return res
8
9 print(sum(nge([5, 3, 8, 2, 4, 9, 1, 6])))
```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Linked lists — Floyd's cycle detection]

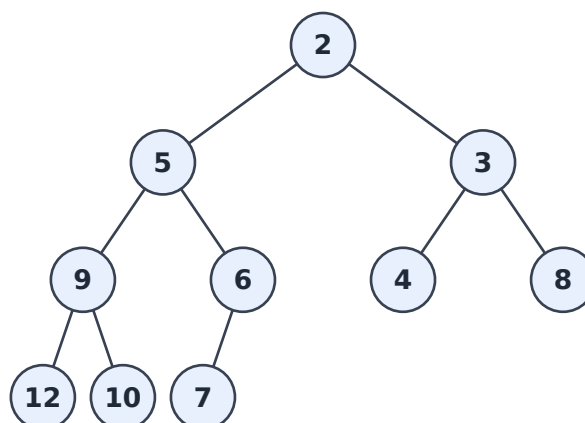
A singly linked list has nodes with values 1, 2, ..., 8 in order, but the **next** pointer of node 8 points back to node 4 (so the list ends in a cycle). Floyd's algorithm is run: **slow** and **fast** both start at node 1; in each iteration slow moves 1 step and fast moves 2 steps, and the loop stops as soon as they are at the same node. Which of the following statements is/are TRUE?



- (A) The cycle contains exactly 5 nodes
- (B) They first meet after exactly 4 iterations
- (C) If one pointer is then reset to node 1 and both advance one step at a time, they next meet at node 4 after 3 steps
- (D) slow and fast first meet at node 6

Q.14 [NAT · numerical answer | 2 marks | Min-heap — extract-min and insert]

The array below is a binary min-heap (0-based). Two **extract-min** operations are performed (move the last element to the root, then sift down swapping with the smaller child), followed by **insert(1)** (append, then sift up). In the resulting heap, the sum of the keys stored in the **leaves** is _____.

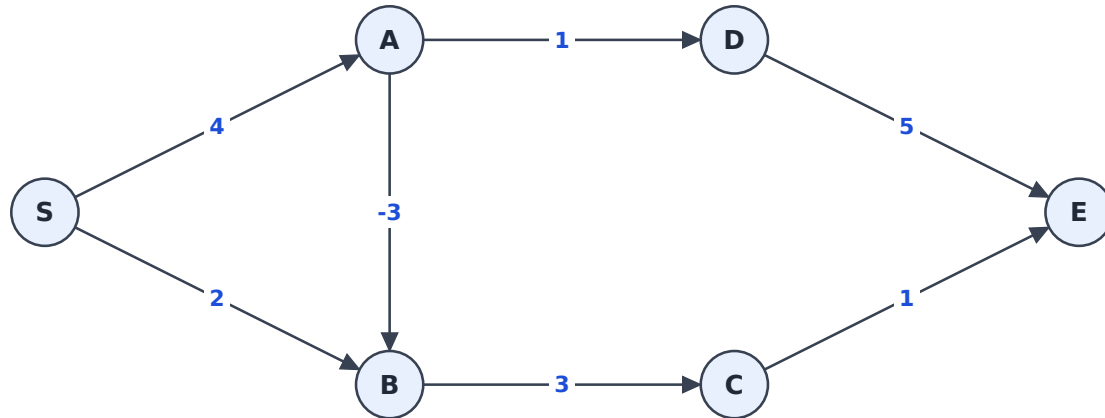


Initial min-heap

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Dijkstra with a negative edge]

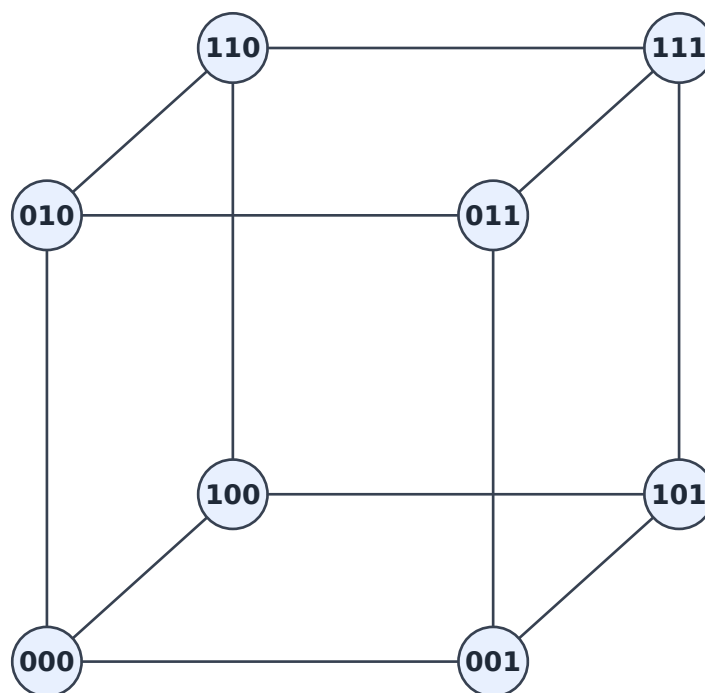
Dijkstra's algorithm is run from S on the directed graph below, which has one negative edge ($A \rightarrow B$ with weight -3). The implementation never changes the distance of a vertex after it has been extracted, and breaks ties alphabetically. Which set of vertices ends up with a final distance **different** from the true shortest-path distance?



- (A) {B, C, D, E}
- (B) {B}
- (C) {B, C}
- (D) {B, C, E}

Q.16 [NAT · numerical answer | 2 marks | BFS — shortest paths in a hypercube]

The 4-dimensional hypercube Q_4 has the 16 binary strings of length 4 as vertices; two vertices are adjacent iff they differ in exactly one bit. The number of distinct shortest paths from 0000 to 1111 is _____.



For intuition: the 3-cube Q_3

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Quicksort — input sensitivity]

Quicksort uses the Lomuto partition with the **last** element as pivot; a partition of a subarray of size s makes exactly $s - 1$ key comparisons, and subarrays of size ≤ 1 are not partitioned. Which of the following statements about sorting 6 distinct keys is/are TRUE?

- (A) On [6, 5, 4, 3, 2, 1] it makes 15 comparisons
- (B) On [2, 1, 4, 3, 6, 5] it makes 8 comparisons
- (C) On [1, 2, 3, 4, 5, 6] it makes 15 comparisons
- (D) On every permutation of 6 distinct keys it makes at least 8 comparisons

Q.18 [MCQ · single correct | 2 marks | Bubble sort vs selection sort — swaps]

Let X be the number of swaps made by bubble sort and Y the number of swaps made by selection sort (which swaps only when the minimum is not already in place) when sorting [8, 3, 6, 1, 7, 2, 5, 4] in ascending order. The value of $X - Y$ is

- (A) 17
- (B) 11
- (C) 12
- (D) 13

Q.19 [MCQ · single correct | 2 marks | Double hashing — choosing h_2]

A table of size $m = 12$ uses double hashing: probe i examines $(h_1(k) + i \cdot h_2(k)) \bmod 12$. Which choice of h_2 guarantees that, for **every** key $k \geq 0$, the probe sequence visits all 12 slots?

- (A) $h_2(k) = 1 + (k \bmod 11)$
- (B) $h_2(k) = 1 + 6 \cdot (k \bmod 2)$
- (C) $h_2(k) = 1 + 2 \cdot (k \bmod 6)$
- (D) $h_2(k) = 1 + 4 \cdot (k \bmod 3)$

Q.20 [MSQ · one or more correct | 2 marks | Python — sorted() with key functions and stability]

Let `words = ["kiwi", "fig", "apple", "date", "plum", "pear"]`. Which of the following statements is/are TRUE?

- (A) `sorted(words, key=len)[1:5]` equals `['kiwi', 'date', 'plum', 'pear']`
- (B) `sorted(words, key=lambda w: (-len(w), w))[1]` equals `'date'`
- (C) `min(words, key=lambda w: w[-1])` equals `'date'`
- (D) `sorted(words, key=len, reverse=True)[1]` equals `'pear'`

Answer Key — Mock Test 43

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — shallow vs deep copy
2	NAT	1	21	Python — string slicing and searching
3	NAT	1	9	Stacks — counting pop sequences
4	MSQ	1	(A), (B), (D)	Queue as a circular linked list
5	NAT	1	4	Counting BSTs with a fixed root
6	MCQ	1	(D)	Hash functions for strings
7	MCQ	1	(A)	Binary search — average successful cost
8	NAT	1	11	Insertion sort — comparisons
9	MSQ	1	(A), (C)	Degrees and Euler trails
10	MCQ	1	(A)	Recurrences — master theorem
11	MCQ	2	(B)	Python — try / except / finally
12	NAT	2	42	Monotonic stack — next greater element
13	MSQ	2	(A), (C), (D)	Linked lists — Floyd's cycle detection
14	NAT	2	45	Min-heap — extract-min and insert
15	MCQ	2	(D)	Dijkstra with a negative edge
16	NAT	2	24	BFS — shortest paths in a hypercube
17	MSQ	2	(A), (C), (D)	Quicksort — input sensitivity
18	MCQ	2	(C)	Bubble sort vs selection sort — swaps
19	MCQ	2	(B)	Double hashing — choosing h_2
20	MSQ	2	(A), (B)	Python — sorted() with key functions and stability

Detailed Solutions — Mock Test 43

Q.1 [MCQ · 1 mark · Python — shallow vs deep copy]

Answer: (C)

`a[:]` makes a **shallow** copy: a new outer list whose elements are the *same* inner list objects. `deepcopy` copies the inner lists too.

- `b[0].append(9)` mutates the inner list shared by a and b → `a[0] = [1, 2, 9]`.
- `b.append([7])` changes only b's outer list → `len(b) = 3`, a still has 2 elements.
- `c[1].append(8)` mutates c's private copy of [3] → `c[1] = [3, 8]`; `a[1]` stays [3].

Output: `[[1, 2, 9], [3]] 3 [3, 8]`.

- (A) thinks appending to b's outer list also affects a.
- (B) treats the slice as a deep copy.
- (D) thinks `deepcopy` still shares inner lists with a.

Trap: 'copy' in Python usually means one level deep. **Tip:** `list(a)`, `a.copy()` and `a[:]` are all shallow.

Q.2 [NAT · 1 mark · Python — string slicing and searching]

Answer: 21

Index the string: d0 a1 t2 a3 s4 c5 i6 e7 n8 c9 e10.

- `s[::2]` takes indices 0, 2, 4, 6, 8, 10 → "dtsine"; all six characters are distinct → `len(set(...)) = 6`.
- `s.find('c')` = first occurrence of 'c' = **5**.
- `s.rfind('e')` = last occurrence of 'e' = **10**.

Sum = 6 + 5 + 10 = **21**.

Traps: taking `s[::2]` as starting from index 1, using 1-based indices (giving 6 + 6 + 11 = 23), or confusing `rfind` with 'find from the right and count from the right'. `rfind` returns an ordinary left-based index.

Q.3 [NAT · 1 mark · Stacks — counting pop sequences]

Answer: 9

To print 3 first we must push 1, 2, 3 and pop 3. The state is now: stack [1, 2] (2 on top), inputs 4, 5 still to come. Count the completions.

Let $N(i, s)$ = number of ways to finish with i inputs left and s items on the stack. $N(0, s) = 1$ (just pop everything); $N(i, 0) = N(i - 1, 1)$; otherwise $N(i, s) = N(i - 1, s + 1)$ [push] + $N(i, s - 1)$ [pop].

- $N(1, 0) = 1$, $N(1, 1) = 2$, $N(1, 2) = 3$, $N(1, 3) = 4$
- $N(2, 0) = N(1, 1) = 2$
- $N(2, 1) = N(1, 2) + N(2, 0) = 3 + 2 = 5$
- $N(2, 2) = N(1, 3) + N(2, 1) = 4 + 5 = \mathbf{9}$

For instance 3 2 1 4 5, 3 2 4 1 5, 3 4 5 2 1, 3 5 4 2 1, ... — in all of them 2 precedes 1.

Trap: counting all orders of the remaining four values with 2 before 1 (12) — this also admits impossible ones such as 3 5 2 4 1 (popping 5 traps 4 above 2). Of all $42 = C_5$ stack permutations of 1...5, exactly 9 start with 3.

Q.4 [MSQ · 1 mark · Queue as a circular linked list]

Answer: (A), (B), (D)

With a circular list one pointer to the rear gives access to **both** ends: `rear = tail`, `front = tail.next`.

- (A) $f = \text{tail.next}$; $\text{tail.next} = f.\text{next}$ (if f is the only node, set $\text{tail} = \text{None}$). $\Theta(1)$. **True.**
- (B) new node z : $z.\text{next} = \text{tail.next}$; $\text{tail.next} = z$; $\text{tail} = z$. $\Theta(1)$. **True.**
- (C) removing tail requires its **predecessor** (to become the new tail), which can only be found by walking around the circle: $\Theta(n)$. **False.**
- (D) tail.next.val . $\Theta(1)$. **True.**

Trap: keeping a pointer to the *front* instead would make $\text{enqueue } \Theta(n)$, because reaching the rear needs a full traversal. **Tip:** this is why queues built on circular lists keep the rear pointer.

Q.5 [NAT · 1 mark · Counting BSTs with a fixed root]

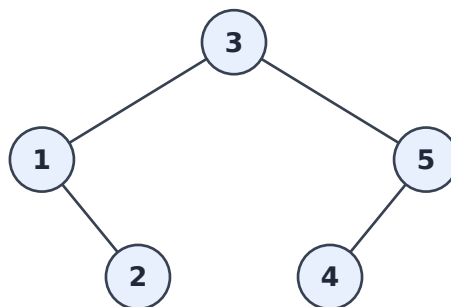
Answer: 4

If 3 is the root, the left subtree is a BST on $\{1, 2\}$ and the right subtree is a BST on $\{4, 5\}$, chosen independently.

- Number of BSTs on 2 keys = Catalan $C_2 = 2$ (smaller key as root with right child, or larger key as root with left child).
- Total = $2 \times 2 = 4$.

General formula: the number of BSTs on keys $1 \dots n$ with root r is $C_{r-1} \cdot C_{n-r}$. Summing over r gives the Catalan number $C_5 = 1 \cdot 1 + 1 \cdot 5 + 2 \cdot 2 + 5 \cdot 1 + 14 \cdot 1 = 42$.

Trap: answering $C_5 = 42$ (ignoring the root constraint) or $2 + 2 = 4$ by luck with the wrong rule — the subtrees combine by **multiplication**, not addition.



One of the 4 trees

Q.6 [MCQ · 1 mark · Hash functions for strings]

Answer: (D)

A sum of character codes ignores **order**, so any rearrangement (anagram) of a string has the same sum and therefore the same hash value.

- (A) "silence" is not an anagram (extra c and e); its slot cannot be predicted without computing.
- (B) "lister" replaces n by r — a different sum (differs by $\text{ord}('r') - \text{ord}('n') = 4$), so a different slot.
- (C) "listens" adds an 's' (code $115 \equiv 5 \pmod{11}$) → shifts the slot by 5.
- (D) "tinsel" uses exactly the letters l, i, s, t, e, n → identical sum → **same slot, guaranteed.**

(Indeed $h(\text{"listen"}) = h(\text{"tinsel"}) = 6$.)

Tip: good string hashes are position-dependent, e.g. the polynomial hash $\sum c_i \cdot b^i \pmod{m}$, which separates anagrams.

Q.7 [MCQ · 1 mark · Binary search — average successful cost]

Answer: (A)

The probes of binary search on $15 = 2^4 - 1$ elements form a perfectly balanced decision tree: level d ($d = 1 \dots 4$) contains 2^{d-1} elements, each found after exactly d probes.

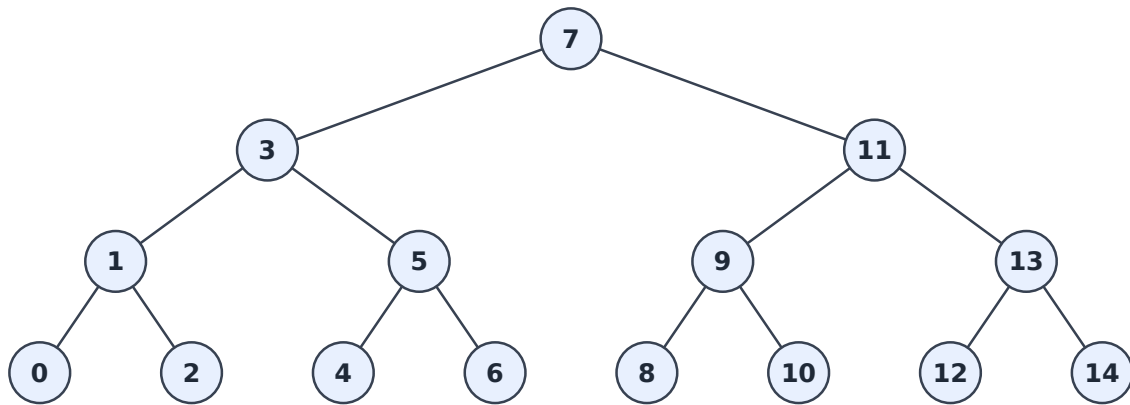
- 1 element (the middle, index 7) → 1 probe

- 2 elements (indices 3, 11) → 2 probes
- 4 elements → 3 probes
- 8 elements → 4 probes

Total = $1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 8 \cdot 4 = 1 + 4 + 12 + 32 = 49 \rightarrow \text{expected} = \mathbf{49/15 \approx 3.27}$.

- (B) 4 is the **worst case** (and the cost of every unsuccessful search).
- (C) $15/4$ is a mis-weighted average.
- (D) $3 = \log_2 8$ is a rough guess.

Tip: more than half of the elements sit on the deepest level, so the average is close to the worst case: about $\log_2 n - 1$.



Decision tree of indices probed

Q.8 [NAT · 1 mark · Insertion sort — comparisons]

Answer: 11

For each insertion: comparisons = (number of shifts) + 1, **unless** the key travels all the way to index 0 (then there is no final failing comparison).

- $i=1$, key 1: $4 > 1$ shift; reaches left end → 1 comparison → [1, 4, 3, 9, 7, 2]
- $i=2$, key 3: $4 > 3$ shift; $1 \leq 3$ stop → 2 → [1, 3, 4, 9, 7, 2]
- $i=3$, key 9: $4 \leq 9$ stop → 1
- $i=4$, key 7: $9 > 7$ shift; $4 \leq 7$ stop → 2 → [1, 3, 4, 7, 9, 2]
- $i=5$, key 2: 9, 7, 4, 3 shift; $1 \leq 2$ stop → 5 → [1, 2, 3, 4, 7, 9]

Total = $1 + 2 + 1 + 2 + 5 = \mathbf{11}$.

Check: shifts = inversions = $1 + 1 + 0 + 1 + 4 = 7$; comparisons = $7 + (5 \text{ insertions} - 1 \text{ that reached the left end}) = 11$.

Trap: counting comparisons = shifts (7) or = shifts + $n - 1$ (12).

Q.9 [MSQ · 1 mark · Degrees and Euler trails]

Answer: (A), (C)

Degrees: A 2, B 4 (A, C, D, E), C 3 (A, B, E), D 3 (B, E, F), E 4 (C, D, F, B), F 2. Sum = $18 = 2 \times 9$ edges. ✓

Euler's theorem (connected graph): Euler circuit $\Leftrightarrow$ all degrees even; Euler trail (not closed) $\Leftrightarrow$ exactly two odd-degree vertices, and the trail runs between them.

- (A) hence an Euler trail from C to D exists, but no circuit. **True.** (e.g. C-A-B-C-E-B-D-E-F-D)
- (B) A-B-C is a triangle (odd cycle). **False.**
- (C) odd vertices: C and D → exactly two. **True.**
- (D) deleting B-E makes B and E odd too → four odd vertices → no Euler circuit (not even a trail). **False.**

Tip: adding the edge C-D instead would make every degree even and create an Euler circuit.

Q.10 [MCQ · 1 mark · Recurrences — master theorem]

Answer: (A)

Master theorem with $a = 2$, $b = 4$: $n^{\log_b a} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$. The driving term $f(n) = \sqrt{n}$ is of the **same** order, so we are in case 2: $T(n) = \Theta(\sqrt{n} \log n)$.

Direct check with $n = 4^k$ ($\sqrt{n} = 2^k$): $T(4^k) = 2T(4^{k-1}) + 2^k$. Dividing by 2^k : $T(4^k)/2^k = T(4^{k-1})/2^{k-1} + 1$, so $T(4^k) = 2^k(k + 1) = \sqrt{n}(\log_4 n + 1)$.

- (B) $\Theta(\sqrt{n})$ would need $f(n)$ polynomially smaller than $\sqrt{n}$ (case 1).
- (C) $\Theta(n)$ mistakes $n^{\log_b a}$ for n .
- (D) $n^{1/4} \log n$ mistakes the critical exponent $\log_4 2 = 1/2$ for $1/4$.

Trap: each of the $\log_4 n$ levels of the recursion tree costs exactly $\sqrt{n}$ — equal work per level is the signature of case 2.

Q.11 [MCQ · 2 marks · Python — try / except / finally]

Answer: (B)

Two rules: (1) a **finally** block runs on **every** exit from the **try** statement, including **return**; (2) if **finally** itself executes **return**, that value replaces the pending one.

The arguments of the outer **print** are evaluated left to right **before** it prints:

- **f(1): return 'pos'** is pending → finally prints **F** → returns 'pos'.
- **f(0): ValueError** raised → caught → **return 'err'** pending → finally prints **F** → returns 'err'.
- **g(): return 1** pending → finally's **return 2** overrides → 2.
- Finally the outer print writes **pos err 2** after the two **F** already on the line.

Output: **F F pos err 2**.

- (A) assumes the outer print emits its values interleaved with the calls.
- (C) assumes each value is printed as soon as its call returns.
- (D) misses that **return** in **finally** overrides the earlier **return 1**.

Trap: **return** inside **finally** also silently swallows exceptions — avoid it in real code.

Q.12 [NAT · 2 marks · Monotonic stack — next greater element]

Answer: 42

The stack holds indices of elements still waiting for a larger element to their right; their values are non-increasing from bottom to top. When x arrives, every smaller waiting element gets x as its *next greater element*.

- 5: stack [5]
- 3: stack [5, 3]
- 8: pops 3 → 8, pops 5 → 8; stack [8]
- 2: stack [8, 2]
- 4: pops 2 → 4; stack [8, 4]
- 9: pops 4 → 9, pops 8 → 9; stack [9]
- 1: stack [9, 1]
- 6: pops 1 → 6; stack [9, 6]

Never popped: 9 and 6 → keep -1 .

$\text{res} = [8, 8, 9, 4, 9, -1, 6, -1]$; $\text{sum} = 8 + 8 + 9 + 4 + 9 - 1 + 6 - 1 = 42$.

Traps: forgetting the two -1 entries (sum 44), or taking the *first popped* order as the order of res .

Tip: each index is pushed and popped at most once → $\Theta(n)$.

res =

8	8	9	4	9	-1	6	-1
0	1	2	3	4	5	6	7

-1 marks elements with no greater element

Q.13 [MSQ · 2 marks · Linked lists — Floyd's cycle detection]**Answer: (A), (C), (D)**

Tail length $\mu = 3$ (nodes 1, 2, 3 before the cycle), cycle length $\lambda = 5$ (nodes 4-8).

Positions after each iteration (slow / fast):

- 1: 2 / 3
- 2: 3 / 5
- 3: 4 / 7
- 4: 5 / 4 (7 → 8 → 4)
- 5: **6 / 6** → meet
- (A) 4 → 5 → 6 → 7 → 8 → 4: $\lambda = 5$. **True.**
- (B) they meet after **5** iterations. **False.**
- (C) Phase 2: from node 1 and node 6, one step each: (2, 7), (3, 8), (4, 4) → meet at the cycle entrance, node 4, after $\mu = 3$ steps. **True.**
- (D) **True.**

Why phase 2 works: at the meeting point slow has walked k steps and fast $2k$, so k is a multiple of λ ; walking μ more steps from the meeting point lands exactly on the entrance.

Trap: assuming the meeting point is the cycle entrance — it generally is not.

Floyd trace (phase 1)

iter	slow	fast
0	1	1
1	2	3
2	3	5
3	4	7
4	5	4
5	6	6

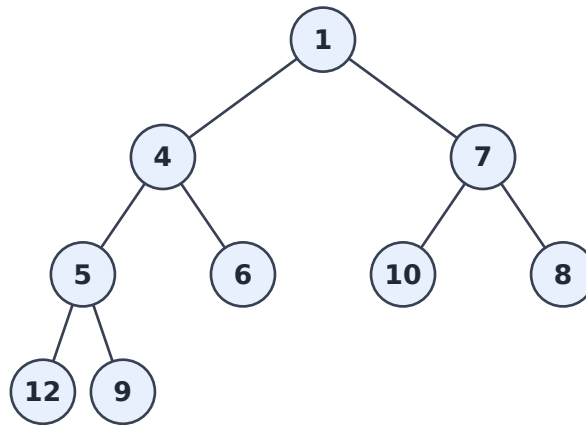
Q.14 [NAT · 2 marks · Min-heap — extract-min and insert]**Answer: 45**

- Extract-min #1 (removes 2): 7 moves to the root → [7, 5, 3, 9, 6, 4, 8, 12, 10]; 7 vs children 5, 3 → swap with 3; at index 2 children 4, 8 → swap with 4 → [3, 5, 4, 9, 6, 7, 8, 12, 10].
- Extract-min #2 (removes 3): 10 moves to the root → [10, 5, 4, 9, 6, 7, 8, 12]; swap with 4; at index 2 children 7, 8 → swap with 7 → [4, 5, 7, 9, 6, 10, 8, 12].
- Insert 1 at index 8: parent index 3 (9) → swap; parent index 1 (5) → swap; parent index 0 (4) → swap → [1, 4, 7, 5, 6, 10, 8, 12, 9].

With $n = 9$ nodes, the leaves are indices $\lfloor n/2 \rfloor \dots n - 1 = 4 \dots 8$: keys 6, 10, 8, 12, 9.

Sum = 6 + 10 + 8 + 12 + 9 = **45**.

Trap: in sift-down, swapping with the *left* child by habit (7 ↔ 5 in the first step) breaks the heap; always choose the smaller child. **Tip:** leaves of an n -node heap are exactly indices $\lfloor n/2 \rfloor$ to $n - 1$.



Final min-heap

Q.15 [MCQ · 2 marks · Dijkstra with a negative edge]**Answer: (D)**

Dijkstra assumes that once a vertex is extracted no later path can be shorter — false with negative edges.

Dijkstra run:

- Extract S (0): A = 4, B = 2
- Extract B (2): C = 5
- Extract A (4): A→B would give 1, but B is already finalised → ignored; D = 5
- Extract C (5, tie with D broken alphabetically): E = 6
- Extract D (5): $5 + 5 = 10 > 6$ → no change; extract E (6)

Dijkstra: A 4, B 2, C 5, D 5, E 6.

True distances (e.g. Bellman-Ford): B = $4 - 3 = 1$ via $S \rightarrow A \rightarrow B$, C = $1 + 3 = 4$, E = $4 + 1 = 5$, A = 4, D = 5.

Wrong vertices: **B, C, E** — the error at B propagates to everything reached through B.

- (A) D's distance 5 ($S \rightarrow A \rightarrow D$) is correct.
- (B) ignores propagation.
- (C) misses that E's best path also goes through C.

Tip: with negative edges use Bellman-Ford ($V - 1$ rounds of relaxing all edges).

Q.16 [NAT · 2 marks · BFS — shortest paths in a hypercube]**Answer: 24**

The distance between two vertices of a hypercube is the number of bits in which they differ (Hamming distance): $d(0000, 1111) = 4$. A shortest path must flip each of the 4 bits **exactly once**; flipping any bit twice wastes two steps.

So a shortest path is just an order in which to flip the 4 bits: $4! = 24$.

BFS path counting agrees: $\sigma(\text{level 1 vertex}) = 1$ (4 vertices), $\sigma(\text{level 2}) = 2$ (6 vertices, each reachable from 2 level-1 vertices), $\sigma(\text{level 3}) = 3 \cdot 2 = 6$, $\sigma(1111) = 4 \cdot 6 = 24$.

Trap: counting vertices at distance 2 ($C(4, 2) = 6$) or the number of edges (32) instead of paths.

Generalisation: in Q_n , two vertices at Hamming distance k are joined by $k!$ shortest paths.

Q.17 [MSQ · 2 marks · Quicksort — input sensitivity]**Answer: (A), (C), (D)**

Total comparisons = $\sum (s - 1)$ over all partitioned subarrays; everything depends on how balanced the splits are.

- (A) reverse-sorted: first pivot 1 is the minimum $\rightarrow [1 \mid 5 \ 4 \ 3 \ 2 \ 6]$; the next pivot 6 is the maximum, then 2 is the minimum, ... every split is still $(n-1, 0) \rightarrow 15$. **True.**
- (B) $[2, 1, 4, 3, 6, 5]$: pivot 5 $\rightarrow 5$ comparisons, split $[2 \ 1 \ 4 \ 3] \mid [6]$; pivot 3 $\rightarrow 3$, split $[2 \ 1] \mid [4]$; pivot 1 $\rightarrow 1 \rightarrow$ total $5 + 3 + 1 = 9$, not 8. **False.**
- (C) sorted input: the pivot is always the maximum $\rightarrow$ splits $(5, 0), (4, 0), \dots \rightarrow 5 + 4 + 3 + 2 + 1 = 15$. **True.**
- (D) The first partition always costs 5. The best split of the remaining 5 elements is $(2, 3) \rightarrow 1 + 2 = 3$ more (the 3-part costs 2 and splits $1 \mid 1$), total 8; every other split costs more (e.g. $(1, 4) \rightarrow 0 + 3 + \dots \geq 4$). For example $[3, 1, 2, 5, 6, 4]$ achieves 8. **True.**

Tip: for $n = 2^k - 1$ a perfect pivot sequence gives the minimum; for other n the minimum is found by the best balanced split at each level.

Q.18 [MCQ · 2 marks · Bubble sort vs selection sort — swaps]**Answer: (C)**

Bubble sort swaps only adjacent out-of-order pairs, and each swap removes exactly one inversion, so $X =$ number of inversions.

For each element, count larger elements before it: 8: 0, 3: 1, 6: 1, 1: 3, 7: 1, 2: 4, 5: 3, 4: 4 $\rightarrow X = 17$.

Selection sort trace:

- $i=0$: min 1 $\rightarrow$ swap with 8 $\rightarrow [1, 3, 6, 8, 7, 2, 5, 4]$
- $i=1$: min 2 $\rightarrow$ swap with 3 $\rightarrow [1, 2, 6, 8, 7, 3, 5, 4]$
- $i=2$: min 3 $\rightarrow$ swap with 6 $\rightarrow [1, 2, 3, 8, 7, 6, 5, 4]$
- $i=3$: min 4 $\rightarrow$ swap with 8 $\rightarrow [1, 2, 3, 4, 7, 6, 5, 8]$
- $i=4$: min 5 $\rightarrow$ swap with 7 $\rightarrow [1, 2, 3, 4, 5, 6, 7, 8]$
- $i=5, 6$: already in place

$Y = 5$, so $X - Y = 17 - 5 = 12$.

- (A) 17 is X alone.
- (B) 11 uses $Y = 6$ (counting a swap at $i=5$).
- (D) 13 uses $Y = 4$.

Tip: selection sort does at most $n - 1$ swaps ($\Theta(n)$) while bubble sort may do $\Theta(n^2)$ — selection sort is preferred when writes are expensive.

Q.19 [MCQ · 2 marks · Double hashing — choosing h_2]**Answer: (B)**

The sequence $h_1, h_1 + s, h_1 + 2s, \dots \pmod{m}$ visits all m slots iff $\gcd(s, m) = 1$. For $m = 12$ the good step sizes are $s \in \{1, 5, 7, 11\}$.

- (A) values 1 ... 11, including 2, 3, 4, 6, 8, 9, 10 $\rightarrow$ e.g. $k = 1$ gives $s = 2$, which visits only the 6 slots of one parity.
- (B) values $\{1, 7\}$ — both coprime to 12. **Always full coverage.**
- (C) values $\{1, 3, 5, 7, 9, 11\} \rightarrow s = 3$ or 9 visits only 4 slots ($\gcd(3)$).
- (D) values $\{1, 5, 9\} \rightarrow s = 9$ has $\gcd(9, 12) = 3 \rightarrow$ only 4 slots.

Tip: this is why double hashing usually takes m **prime** (then any s in $1 \dots m - 1$ works) or m a power of 2 with s forced odd.

Q.20 [MSQ · 2 marks · Python — `sorted()` with key functions and stability]**Answer: (A), (B)**

`sorted` is **stable**: items with equal keys keep their original relative order — and this remains true with `reverse=True` (Python reverses the comparison, not the result). `min/max` return the **first** item with the extreme key.

- (A) `key=len`: `fig(3)`; then the four 4-letter words in input order kiwi, date, plum, pear; then `apple(5)`. Slice `[1:5]` → `['kiwi', 'date', 'plum', 'pear']`. **True.**
- (B) `key=(-len, w)`: longest first, ties alphabetical → apple, date, kiwi, pear, plum, fig → index 1 is 'date'. **True.**
- (C) last letters: i, g, e, e, m, r → smallest 'e', first achieved by 'apple'. **False.**
- (D) `reverse=True` by length: apple, then the 4-letter words **still in input order** kiwi, date, plum, pear, then fig → index 1 is 'kiwi'. **False.**

Trap: (D) — many expect `reverse=True` to reverse the tie order as well (that would be `sorted(words, key=len)[::-1]`, which gives 'pear' at index 1).

Mock Test 44

Full-Syllabus Mock — Paper 14

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — `+=` versus `+` on lists]

What is printed by the following Python program?

Python

```
1 a = [1, 2]
2 b = a
3 b += [3]
4 c = a
5 c = c + [4]
6 print(a, c, a is b)
```

- (A) [1, 2] [1, 2, 4] False
- (B) [1, 2, 3] [1, 2, 3, 4] True
- (C) [1, 2, 3, 4] [1, 2, 3, 4] True
- (D) [1, 2, 3] [1, 2, 3, 4] False

Q.2 [NAT · numerical answer | 1 mark | Python — dictionary keys and overwriting]

What value is printed by the following Python program?

Python

```
1 d = {x % 4: x for x in [3, 8, 11, 14, 7, 20]}
2 e = {1: 'a', True: 'b', 1.0: 'c'}
3 print(sum(d.values()) + len(e))
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — prefix expression evaluation]

The prefix (Polish) expression

$- * + 4 3 2 / 8 ^ 2 2$

is evaluated with a stack by scanning the tokens from **right to left** (push operands; on an operator pop the top two values x then y and push $x \text{ op } y$). Here $/$ is integer division and $^$ is exponentiation. The result is

- (A) 14
- (B) -12
- (C) 12
- (D) 2

Q.4 [MCQ · single correct | 1 mark | Deques — bounded deque in Python]

What is printed by the following Python program?

Python

```
1 from collections import deque
2 dq = deque(maxlen=3)
3 for x in range(1, 6):
4     dq.append(x)
5 dq.appendleft(0)
6 dq.rotate(1)
7 print(list(dq))
```

(A) [4, 0, 3]

(B) [0, 3, 4]

(C) [5, 0, 3]

(D) [3, 4, 5]

Q.5 [NAT · numerical answer | 1 mark | Linked lists — middle node with slow/fast pointers]

What value is printed by the following Python program?

Python

```
1 class Node:
2     def __init__(self, val, nxt=None):
3         self.val, self.next = val, nxt
4
5 def mid1(h):
6     slow = fast = h
7     while fast and fast.next:
8         slow, fast = slow.next, fast.next.next
9     return slow
10
11 def mid2(h):
12     slow = fast = h
13     while fast.next and fast.next.next:
14         slow, fast = slow.next, fast.next.next
15     return slow
16
17 head = None
18 for v in reversed([12, 7, 30, 5, 18, 41, 9, 26]):
19     head = Node(v, head)
20 print(mid1(head).val - mid2(head).val)
```



Answer: _____ (enter a numerical value)

Q.6 [MSQ · one or more correct | 1 mark | BST — possible search paths]

A binary search tree contains integer keys from 1 to 100 (not necessarily all of them), and a search for the key **55** is performed. Which of the following could be the sequence of keys examined by the search?

(A) 10, 80, 40, 75, 45, 60, 55

(B) 25, 95, 50, 90, 52, 58, 53, 55

(C) 70, 30, 65, 40, 68, 55

(D) 90, 20, 70, 30, 60, 50, 55

Q.7 [MCQ · single correct | 1 mark | Hashing — linear probing with wrap-around]

Keys 4, 15, 9, 26, 20 are inserted in this order into an empty hash table with slots 0–10, using $h(k) = (2k + 3) \bmod 11$ and linear probing (slot after 10 is 0). In which slot is 20 stored?

- (A) 3
- (B) 0
- (C) 4
- (D) 2

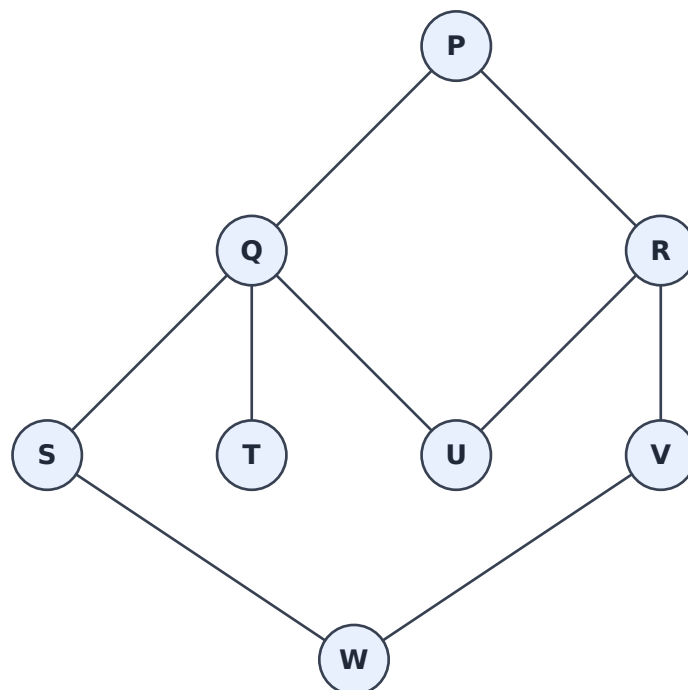
Q.8 [MSQ · one or more correct | 1 mark | Bubble sort — passes and swaps]

Bubble sort (each pass scans left to right over the unsorted prefix, swapping adjacent out-of-order pairs) is applied to $A = [62, 18, 45, 7, 53, 29, 11]$. Which of the following statements is/are TRUE?

- (A) The first 2 passes perform 9 swaps in total
- (B) After 2 passes, $A = [18, 7, 45, 29, 11, 53, 62]$
- (C) After any 2 passes of bubble sort on any array, the two largest elements are in their final positions
- (D) Sorting A completely requires 15 swaps

Q.9 [NAT · numerical answer | 1 mark | BFS — counting possible visiting orders]

BFS is run on the undirected graph below starting from P. When a vertex is dequeued, its undiscovered neighbours may be enqueued in **any** order. The number of distinct orders in which BFS can visit (dequeue) the vertices is _____.



Answer: _____ (enter a numerical value)

Q.10 [NAT · numerical answer | 1 mark | Recursion — counting calls]

How many times is the function **f** called (including the initial call) when **f(8)** is evaluated?

Python

```
1 def f(n):
2     if n <= 2:
3         return 1
4     return f(n - 1) + f(n - 3)
5
6 print(f(8))
```

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — class vs instance attributes]

What is printed by the following Python program?

Python

```
1 class C:
2     items = []
3     count = 0
4
5     def add(self, x):
6         self.items.append(x)
7         self.count += 1
8
9 a, b = C(), C()
10 a.add(1)
11 b.add(2)
12 a.add(3)
13 print(len(C.items), a.count, b.count, C.count)
```

(A) 2 2 1 0

(B) 3 2 1 0

(C) 3 3 3 3

(D) 3 2 1 3

Q.12 [NAT · numerical answer | 2 marks | Stacks — monotonic stack (stock span)]

The **span** of day *i* in a price list *P* is the number of consecutive days ending at day *i* (including day *i*) whose price is $\leq P[i]$. It is computed with a stack of indices: for each *i*, pop while the price at the stack top is $\leq P[i]$; then span = *i* + 1 if the stack is empty, otherwise *i* – (top index); finally push *i*. For

$P = [31, 27, 29, 35, 22, 24, 30, 40, 26]$

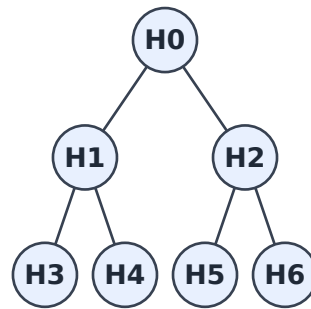
the sum of all nine spans is _____.

P =	31	27	29	35	22	24	30	40	26
	0	1	2	3	4	5	6	7	8

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Heaps — possible positions of keys]

The seven keys 1, 2, 3, 4, 5, 6, 7 are stored in an array-based binary **min**-heap $H[0..6]$ (children of index i are $2i + 1$ and $2i + 2$). Considering **all** valid min-heaps on these keys, which of the following statements is/are TRUE?



Index layout of a 7-element heap

- (A) Key 6 can be at index 2
- (B) Key 2 can be at index 3
- (C) Key 7 is always at one of the indices 3, 4, 5, 6
- (D) Key 3 can be at an index of depth 2 (indices 3–6)

Q.14 [MSQ · one or more correct | 2 marks | Quicksort — Hoare partition]

The Hoare partition below is called as **hoare**(A , 0, 8) on $A = [6, 11, 3, 9, 2, 8, 5, 1, 10]$. Which of the following statements is/are TRUE?

Python

```

1 def hoare(A, lo, hi):
2     p = A[lo]
3     i, j = lo - 1, hi + 1
4     while True:
5         i += 1
6         while A[i] < p:
7             i += 1
8         j -= 1
9         while A[j] > p:
10            j -= 1
11        if i >= j:
12            return j
13        A[i], A[j] = A[j], A[i]
  
```

- (A) The function returns 3
- (B) After the call, every element of $A[0..3]$ is $\leq$ every element of $A[4..8]$
- (C) Exactly 3 swaps are performed
- (D) After the call, the pivot value 6 is at index 3

Q.15 [MCQ · single correct | 2 marks | Binary search — rotated arrays (buggy variant)]

The function below is intended to return the minimum of a sorted array of distinct keys that has been rotated by an unknown amount. What does the program print?

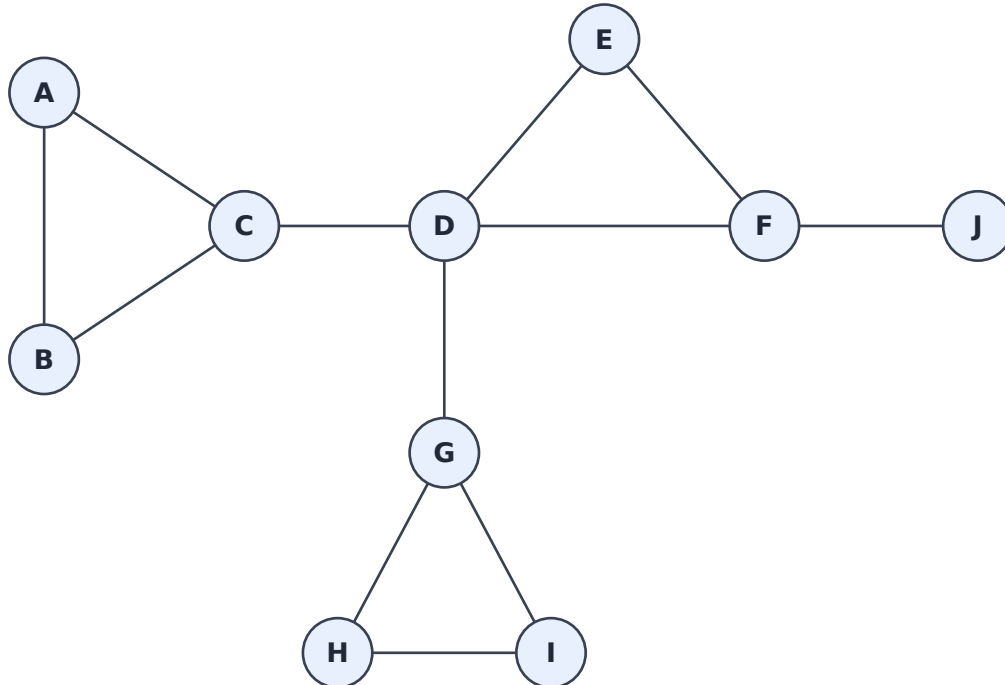
Python

```
1 def find_min(A):
2     lo, hi = 0, len(A) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if A[mid] >= A[lo]:
6             lo = mid + 1
7         else:
8             hi = mid
9     return A[lo]
10
11 print(find_min([15, 18, 22, 3, 6, 9, 12]),
12       find_min([7, 9, 11, 2, 5]),
13       find_min([2, 4, 6, 8, 10]))
```

- (A) 3 2 2
 (B) 22 5 10
 (C) 3 2 10
 (D) 3 5 10

Q.16 [NAT · numerical answer | 2 marks | Graph theory — cut vertices]

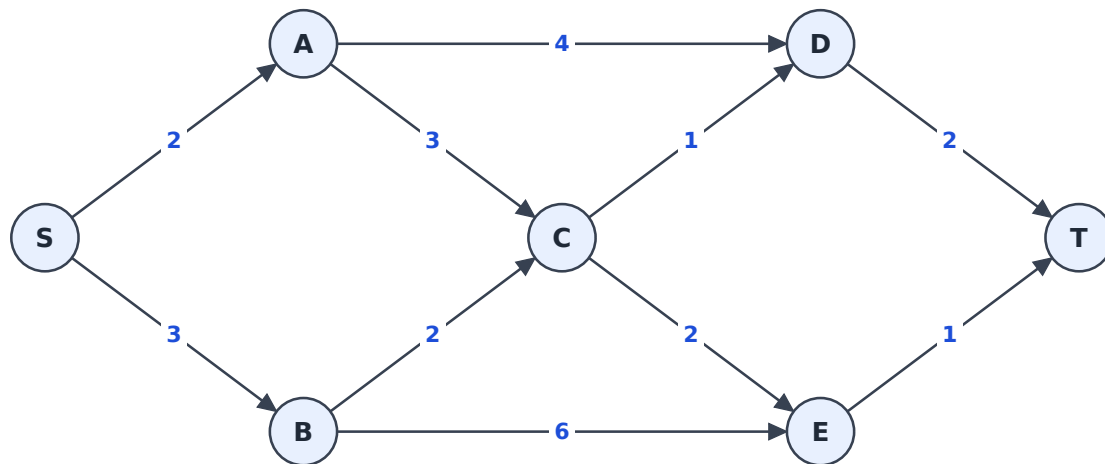
A vertex of a connected undirected graph is a **cut vertex** (articulation point) if deleting it together with its incident edges disconnects the graph. The number of cut vertices in the graph below is _____.



Answer: _____ (enter a numerical value)

Q.17 [NAT · numerical answer | 2 marks | Shortest paths — counting shortest weighted paths]

In the weighted directed graph below, the number of distinct shortest paths from S to T is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Complexity — nested loops with geometric steps]

The value returned by $f(n)$ below, as a function of n , is

Python

```

1 def f(n):
2     c = 0
3     i = 1
4     while i < n:
5         j = i
6         while j < n:
7             j *= 2
8             c += 1
9         i *= 3
10    return c
  
```

- (A) $\Theta(\log n)$
- (B) $\Theta(n)$
- (C) $\Theta(\log^2 n)$
- (D) $\Theta(n \log n)$

Q.19 [MSQ · one or more correct | 2 marks | Python — string immutability and slicing]

Consider the following Python program (it prints nothing). Which of the following statements is/are TRUE after it runs?

Python

```

1 s = "data"
2 t = s
3 s += "base"
4 u = s[::-1][1::2]
5 k = s.find("a", 2)
  
```

- (A) $k == 3$
- (B) $u == \text{"sbtd"}$
- (C) $s.\text{count}(\text{"a"}) == 2$
- (D) $t == \text{"data"}$

Q.20 [MCQ · single correct | 2 marks | Graph representations — BFS from an adjacency matrix]

An undirected graph on vertices 0-6 is given by the adjacency matrix below. How many vertices are at shortest-path distance **exactly 2** from vertex 0?

Adjacency matrix M

	0	1	2	3	4	5	6
0	0	1	0	0	1	0	0
1	1	0	1	0	0	0	1
2	0	1	0	1	0	0	0
3	0	0	1	0	0	1	0
4	1	0	0	0	0	1	0
5	0	0	0	1	1	0	1
6	0	1	0	0	0	1	0

- (A) 2
(B) 5
(C) 4
(D) 3

Answer Key — Mock Test 44

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — += versus + on lists
2	NAT	1	42	Python — dictionary keys and overwriting
3	MCQ	1	(C)	Stacks — prefix expression evaluation
4	MCQ	1	(A)	Deque — bounded deque in Python
5	NAT	1	13	Linked lists — middle node with slow/fast pointers
6	MSQ	1	(A), (B), (D)	BST — possible search paths
7	MCQ	1	(A)	Hashing — linear probing with wrap-around
8	MSQ	1	(A), (B), (C)	Bubble sort — passes and swaps
9	NAT	1	10	BFS — counting possible visiting orders
10	NAT	1	25	Recursion — counting calls
11	MCQ	2	(B)	Python — class vs instance attributes
12	NAT	2	23	Stacks — monotonic stack (stock span)
13	MSQ	2	(C), (D)	Heaps — possible positions of keys
14	MSQ	2	(A), (B), (C)	Quicksort — Hoare partition
15	MCQ	2	(D)	Binary search — rotated arrays (buggy variant)
16	NAT	2	4	Graph theory — cut vertices
17	NAT	2	5	Shortest paths — counting shortest weighted paths
18	MCQ	2	(C)	Complexity — nested loops with geometric steps
19	MSQ	2	(A), (B), (D)	Python — string immutability and slicing
20	MCQ	2	(D)	Graph representations — BFS from an adjacency matrix

Detailed Solutions — Mock Test 44

Q.1 [MCQ · 1 mark · Python — `+=` versus `+` on lists]

Answer: (B)

Concept. For lists, `b += [3]` calls `list.__iadd__`, which **extends the existing list in place** and rebinds `b` to the same object. `c = c + [4]` builds a **new** list and rebinds only `c`.

Trace.

- `b = a`: one list `[1, 2]`, two names.
- `b += [3]`: the shared list becomes `[1, 2, 3]`; `a` and `b` still refer to it.
- `c = a`, then `c = c + [4]`: a new list `[1, 2, 3, 4]` is bound to `c`; `a` is unchanged.
- `a is b` → `True`.

Output: `[1, 2, 3] [1, 2, 3, 4] True`.

Options.

- (A) treats `+=` like `+` (creating a new list for `b`).
- (C) treats `c = c + [4]` as an in-place change of `a`.
- (D) thinks `+=` rebinds `b` to a different object.

Trap: for immutable types (`int`, `str`, `tuple`) `x += y` always creates a new object; for lists it mutates — the same syntax, different semantics.

Q.2 [NAT · 1 mark · Python — dictionary keys and overwriting]

Answer: 42

Concept. In a dict comprehension a repeated key keeps its first insertion position but its value is overwritten by the *last* assignment. Keys are compared by hash and equality, and `1 == True == 1.0` with equal hashes, so they are the **same** key.

d. Keys `x % 4`: `3→3`, `8→0`, `11→3`, `14→2`, `7→3`, `20→0`.

- key 3: 3, then 11, then 7 → final 7
- key 0: 8, then 20 → final 20
- key 2: 14

`sum(d.values()) = 7 + 20 + 14 = 41`.

e. All three keys are equal, so `e` has a single entry `{1: 'c'}` (first key object kept, last value) → `len(e) = 1`.

Printed: `41 + 1 = 42`.

Trap: expecting `len(e) = 3` (keys look different) or summing all six numbers (63).

Q.3 [MCQ · 1 mark · Stacks — prefix expression evaluation]

Answer: (C)

Concept. Scanning a prefix expression right-to-left makes it behave like postfix, but the operand order flips: the **first** value popped is the **left** operand.

Trace (stack bottom → top):

- `2` → `[2]`; `2` → `[2, 2]`
- `^` : `x = 2, y = 2` → `2 ^ 2 = 4` → `[4]`
- `8` → `[4, 8]`
- `/` : `x = 8, y = 4` → `8 / 4 = 2` → `[2]`
- `2` → `[2, 2]`; `3` → `[2, 2, 3]`; `4` → `[2, 2, 3, 4]`
- `+` : `4 + 3 = 7` → `[2, 2, 7]`
- `*` : `7 × 2 = 14` → `[2, 14]`
- `-` : `x = 14, y = 2` → `14 - 2 = 12`.

Infix form: $((4 + 3) \times 2) - (8 / (2 ^ 2)) = 14 - 2 = 12$.

Options.

- (A) 14 and (D) 2 are the values of the two operand sub-expressions of the outer $-$.
- (B) -12 swaps the operands of $-$ (computes $y - x$).

Trap: with right-to-left scanning, popping order is the reverse of postfix evaluation.

Q.4 [MCQ · 1 mark · Deques — bounded deque in Python]

Answer: (A)

Concept. A deque with `maxlen` discards from the **opposite end** when full: `append` drops the leftmost item, `appendleft` drops the rightmost. `rotate(1)` moves the last element to the front.

Trace.

- Appending 1..5: $[1] \rightarrow [1, 2] \rightarrow [1, 2, 3] \rightarrow [2, 3, 4] \rightarrow [3, 4, 5]$.
- `appendleft(0)`: deque is full, so 5 is dropped from the right $\rightarrow [0, 3, 4]$.
- `rotate(1)`: last element 4 goes to the front $\rightarrow [4, 0, 3]$.

Options.

- (B) forgets the rotation.
- (C) assumes `appendleft` dropped from the left end (the 4) and kept 5.
- (D) assumes `appendleft` is ignored on a full deque.

Tip: a `deque(maxlen=k)` is a ready-made sliding window / circular buffer of the last k items, with $O(1)$ operations at both ends.



Final deque (left $\rightarrow$ right)

Q.5 [NAT · 1 mark · Linked lists — middle node with slow/fast pointers]

Answer: 13

Concept. For a list of even length n , the two loop conditions pick different middles: `while fast and fast.next` stops with slow at index $n/2$ (the **second** middle); `while fast.next and fast.next.next` stops with slow at index $n/2 - 1$ (the **first** middle).

mid1 (indices 0-based): (slow, fast) = $(0, 0) \rightarrow (1, 2) \rightarrow (2, 4) \rightarrow (3, 6) \rightarrow (4, 8 = \text{None})$. Loop stops; slow at index 4 $\rightarrow$ value 18.

mid2: $(0, 0) \rightarrow (1, 2) \rightarrow (2, 4) \rightarrow (3, 6)$; now fast.next is index 7 but fast.next.next is None $\rightarrow$ stop. slow at index 3 $\rightarrow$ value 5.

Printed: $18 - 5 = 13$.

Trap: both functions agree on odd-length lists; only even lengths reveal the difference. Merge sort on linked lists usually wants `mid2` so that both halves are non-empty for $n = 2$.

Q.6 [MSQ · 1 mark · BST — possible search paths]

Answer: (A), (B), (D)

Concept. Maintain the open interval (low, high) of keys still possible. Each examined key must lie inside the current interval; if it is greater than 55 it becomes the new high, if smaller the new low.

- (A) $10 \rightarrow (10, \infty)$; $80 \rightarrow (10, 80)$; $40 \rightarrow (40, 80)$; $75 \rightarrow (40, 75)$; $45 \rightarrow (45, 75)$; $60 \rightarrow (45, 60)$; $55 \checkmark$. **Possible.**
- (B) $25 \rightarrow (25, \infty)$; $95 \rightarrow (25, 95)$; $50 \rightarrow (50, 95)$; $90 \rightarrow (50, 90)$; $52 \rightarrow (52, 90)$; $58 \rightarrow (52, 58)$; $53 \rightarrow (53, 58)$; $55 \checkmark$. **Possible.**

- (C) $70 \rightarrow (-\infty, 70)$; $30 \rightarrow (30, 70)$; $65 \rightarrow (30, 65)$; $40 \rightarrow (40, 65)$; **68 is outside (40,65)** — after going left at 65, every later key must be < 65 . **Not possible.**
- (D) $(-\infty, \infty)$; $90 \rightarrow (-\infty, 90)$; $20 \rightarrow (20, 90)$; $70 \rightarrow (20, 70)$; $30 \rightarrow (30, 70)$; $60 \rightarrow (30, 60)$; $50 \rightarrow (50, 60)$; $55 \checkmark$. **Possible.**

Trap: checking only the immediate parent ($68 > 40$, so 'go right' looks fine) misses the constraint imposed by the earlier ancestor 65.

Q.7 [MCQ · 1 mark · Hashing — linear probing with wrap-around]

Answer: (A)

Concept. Linear probing scans $h, h+1, h+2, \dots$ modulo the table size, so a search that starts near the end of the table wraps around to slot 0.

Hash values. $4 \rightarrow 11 \bmod 11 = 0$; $15 \rightarrow 33 \bmod 11 = 0$; $9 \rightarrow 21 \bmod 11 = 10$; $26 \rightarrow 55 \bmod 11 = 0$; $20 \rightarrow 43 \bmod 11 = 10$.

Trace.

- $4 \rightarrow$ slot 0.
- $15 \rightarrow 0$ full $\rightarrow$ slot 1.
- $9 \rightarrow$ slot 10.
- $26 \rightarrow 0, 1$ full $\rightarrow$ slot 2.
- $20 \rightarrow 10$ full $\rightarrow 0 \rightarrow 1 \rightarrow 2$ full $\rightarrow$ slot **3**.

Options.

- (B) 0 is 20's position if wrap-around were applied without checking occupancy.
- (C) 4 skips one slot too many.
- (D) 2 is 26's slot — it would be 20's only if 20 had been inserted before 26.

Insight: keys with different home slots (0 and 10) merge into one cluster across the wrap-around — primary clustering does not respect the 'end' of the array.

0	4
1	15
2	26
3	20
4	
5	
6	
7	
8	
9	
10	9

Final table

Q.8 [MSQ · 1 mark · Bubble sort — passes and swaps]

Answer: (A), (B), (C)

Pass 1: 62 bubbles to the end, swapping with 18, 45, 7, 53, 29, 11 (6 swaps) $\rightarrow$ [18, 45, 7, 53, 29, 11, 62].

Pass 2: $18 < 45$ ok; $45 > 7$ swap; $45 < 53$ ok; $53 > 29$ swap; $53 > 11$ swap (3 swaps) $\rightarrow$ [18, 7, 45, 29, 11, 53, 62].

- (A) **True** — $6 + 3 = 9$.

- (B) **True**.
- (C) **True** — pass k carries the k -th largest element to position $n - k$; this invariant holds for every input.
- (D) **False** — bubble sort performs exactly one swap per inversion, and A has 14 inversions (62: 6, 18: 2, 45: 3, 7: 0, 53: 2, 29: 1, 11: 0), so 14 swaps.

Trap: counting comparisons (6 + 5 + 4 + ...) instead of swaps, or mis-tallying inversions.

Array after each pass							
start	62	18	45	7	53	29	11
pass 1	18	45	7	53	29	11	62
pass 2	18	7	45	29	11	53	62

Q.9 [NAT · 1 mark · BFS — counting possible visiting orders]

Answer: 10

Concept. A BFS order is fixed once we choose, for every dequeued vertex, the order in which its *newly discovered* neighbours are enqueued. Branch on these choices.

Case 1: Q before R (level 1 = Q, R).

- Q discovers S, T, U $\rightarrow 3! = 6$ orders. Then R discovers only V (U already seen).
- Level 3: W is discovered by the first of S, V to be dequeued — S (it precedes V). One way.
- Subtotal 6.

Case 2: R before Q.

- R discovers U, V $\rightarrow 2$ orders; Q then discovers S, T $\rightarrow 2$ orders.
- W is discovered by V (V precedes S). One way.
- Subtotal 4.

Total = 6 + 4 = **10**.

Trap: multiplying level sizes ($2! \times 4! \times 1 = 48$) ignores that a vertex discovered by one parent cannot appear among another parent's children — U's position depends on whether Q or R is dequeued first.

Q.10 [NAT · 1 mark · Recursion — counting calls]

Answer: 25

Concept. Let $C(n)$ be the number of calls made when evaluating $f(n)$. For $n \leq 2$ there is just the one call; otherwise $C(n) = 1 + C(n - 1) + C(n - 3)$.

Table.

- $C(0) = C(1) = C(2) = 1$ (note $f(0)$ is reached from $f(3)$).
- $C(3) = 1 + C(2) + C(0) = 3$
- $C(4) = 1 + C(3) + C(1) = 5$
- $C(5) = 1 + C(4) + C(2) = 7$
- $C(6) = 1 + C(5) + C(3) = 11$
- $C(7) = 1 + C(6) + C(4) = 17$
- $C(8) = 1 + C(7) + C(5) = \mathbf{25}$

(The printed value $f(8) = 13$ is a different quantity — it counts leaves, not calls.)

Trap: confusing the return value with the call count, or forgetting the '+1' for the current call. Memoisation would cut the calls to $O(n)$.

Q.11 [MCQ · 2 marks · Python — class vs instance attributes]**Answer: (B)**

Concept. Attribute *lookup* `self.x` searches the instance first, then the class. Attribute *assignment* `self.x = ...` always creates/updates an **instance** attribute.

- `self.items.append(x)` only *reads* `self.items` (finding the class's single list) and mutates it — all instances share that list.
- `self.count += 1` means `self.count = self.count + 1`: the read finds `C.count = 0` the first time, but the write creates an instance attribute, shadowing the class attribute from then on. `C.count` itself is never changed.

Trace.

- `a.add(1)`: `C.items = [1]`; `a.count = 0 + 1 = 1` (new instance attribute).
- `b.add(2)`: `C.items = [1, 2]`; `b.count = 1`.
- `a.add(3)`: `C.items = [1, 2, 3]`; `a.count = 2`.

Output: `3 2 1 0`.**Options.**

- (A) assumes each instance has its own `items`, yet counts them together.
- (C) assumes `count` is shared like `items`.
- (D) assumes `+=` updated the class attribute.

Trap: mutable class attributes are shared state — initialise per-instance lists in `__init__`.**Q.12 [NAT · 2 marks · Stacks — monotonic stack (stock span)]****Answer: 23**

Concept. The stack keeps indices of a strictly decreasing sequence of prices; the element below day *i* after popping is the previous day with a *higher* price. Each index is pushed and popped at most once → $O(n)$ total.

Trace (stack shown as indices, prices in brackets):

- `i=0 (31)`: empty → span 1; push 0. Stack [0]
- `i=1 (27)`: top 31 > 27 → span 1-0 = 1; push. [0, 1]
- `i=2 (29)`: pop 1 (27); top 0 (31) → span 2; push. [0, 2]
- `i=3 (35)`: pop 2, pop 0 → empty → span 4; push. [3]
- `i=4 (22)`: span 1; push. [3, 4]
- `i=5 (24)`: pop 4; top 3 → span 2; push. [3, 5]
- `i=6 (30)`: pop 5; top 3 → span 3; push. [3, 6]
- `i=7 (40)`: pop 6, pop 3 → empty → span 8; push. [7]
- `i=8 (26)`: top 40 → span 1. [7, 8]

Spans: 1, 1, 2, 4, 1, 2, 3, 8, 1 → sum = **23**.**Trap:** stopping at the first smaller price to the left (that would be the 'previous smaller element' problem); the span stops at the first strictly *greater* price.**Q.13 [MSQ · 2 marks · Heaps — possible positions of keys]****Answer: (C), (D)**

Concept. In a min-heap every key is smaller than all keys in its subtree. So a key at a node whose subtree has *s* nodes needs at least *s* - 1 larger keys; and a key can be at depth *d* only if its *d* ancestors are all smaller than it.

- (A) **False.** Index 2 roots a subtree of 3 nodes, so the key there needs 2 larger keys; only 7 is larger than 6.

- (B) **False.** 2 can only have the ancestor 1, so it must be at depth ≤ 1 — in fact it must be a child of the root (index 1 or 2), since its parent must be 1.
- (C) **True.** 7 is the maximum; if it had a child, the child would have to be larger. So 7 is at a leaf, i.e. index 3–6.
- (D) **True.** Depth 2 needs two smaller ancestors: 1 (root) and 2. Example heap [1, 2, 4, 3, 5, 6, 7] — 3 is at index 3, child of 2.

Fact: there are exactly 80 distinct min-heaps on 7 distinct keys — enumerating them confirms (C) and (D) and refutes (B) and (A).

Trap: assuming the k -th smallest must be at depth $k - 1$ or at depth $\lfloor \log_2 k \rfloor$; the k -th smallest can be anywhere from depth 1 down to depth $k - 1$.

Q.14 [MSQ · 2 marks · Quicksort — Hoare partition]

Answer: (A), (B), (C)

Concept. Hoare's scheme moves i right to an element $\geq$ pivot and j left to an element $\leq$ pivot, swaps them, and repeats until the pointers cross. It returns j such that $A[\text{lo}..j] \leq \text{pivot} \leq A[j+1..hi]$ — but, unlike Lomuto, it does **not** put the pivot in its final place.

Trace (pivot 6):

- $i \rightarrow 0$ ($6 \geq 6$), $j \rightarrow 7$ ($1 \leq 6$): swap $\rightarrow$ [1, 11, 3, 9, 2, 8, 5, 6, 10] (swap 1)
- $i \rightarrow 1$ (11), $j \rightarrow 6$ (5): swap $\rightarrow$ [1, 5, 3, 9, 2, 8, 11, 6, 10] (swap 2)
- $i \rightarrow 2$ ($3 < 6$) $\rightarrow$ 3 (9); $j \rightarrow 5$ ($8 > 6$) $\rightarrow$ 4 (2): swap $\rightarrow$ [1, 5, 3, 2, 9, 8, 11, 6, 10] (swap 3)
- $i \rightarrow 4$ (9); $j \rightarrow 3$ (2). $i \geq j \rightarrow$ return **3**.
- (A) **True.**
- (B) **True** — left part {1, 5, 3, 2} (max 5), right part {9, 8, 11, 6, 10} (min 6).
- (C) **True.**
- (D) **False** — 6 ended at index 7; index 3 holds 2.

Trap: recursing on $(\text{lo}, j - 1)$ and $(j + 1, \text{hi})$ as with Lomuto is a bug for Hoare; the correct calls are (lo, j) and $(j + 1, \text{hi})$.

Q.15 [MCQ · 2 marks · Binary search — rotated arrays (buggy variant)]

Answer: (D)

Concept. The correct algorithm compares $A[\text{mid}]$ with $A[\text{hi}]$: if $A[\text{mid}] > A[\text{hi}]$ the minimum is right of mid, otherwise it is at mid or to its left. Comparing with $A[\text{lo}]$ fails because ' $A[\text{mid}] \geq A[\text{lo}]$ ' is also true when the range $[\text{lo}, \text{hi}]$ is already sorted (unrotated), and it lets lo jump past the minimum.

Trace 1 [15, 18, 22, 3, 6, 9, 12]: (0,6) mid 3: $3 \geq 15$? no $\rightarrow$ hi 3; (0,3) mid 1: $18 \geq 15 \rightarrow$ lo 2; (2,3) mid 2: $22 \geq 22 \rightarrow$ lo 3; return **3** (correct by luck).

Trace 2 [7, 9, 11, 2, 5]: (0,4) mid 2: $11 \geq 7 \rightarrow$ lo 3; (3,4) mid 3: $2 \geq 2$ (mid = lo!) $\rightarrow$ lo 4; return **5** — wrong, the minimum 2 was skipped.

Trace 3 [2, 4, 6, 8, 10] (rotation 0): every test $A[\text{mid}] \geq A[\text{lo}]$ succeeds, lo keeps moving right; return **10** — the maximum!

Output: **3 5 10**.

Options. (A) is what a correct implementation prints; (C) and (B) mix correct and incorrect traces.

Trap: when $\text{lo} = \text{mid}$, the test $A[\text{mid}] \geq A[\text{lo}]$ is trivially true — a classic off-by-one hazard in binary-search variants.

Q.16 [NAT · 2 marks · Graph theory — cut vertices]**Answer: 4**

Concept. A vertex is a cut vertex iff some pair of other vertices has all its connecting paths passing through it. Vertices inside a cycle are protected by the alternative route around the cycle; vertices where blocks (cycles/bridges) meet are cut vertices.

Structure. Blocks: triangle ABC, bridge CD, triangle DEF, bridge DG, triangle GHI, bridge FJ.

Check each vertex.

- C: removing it separates {A, B} from the rest → **cut**.
- D: separates {A, B, C}, {E, F, J} and {G, H, I} → **cut**.
- F: separates J → **cut**.
- G: separates {H, I} → **cut**.
- A, B, E, H, I: each lies on a triangle with two other vertices that stay connected; J is a leaf. Removing any of them leaves the graph connected.

Cut vertices: C, D, F, G → **4**. (The bridges are CD, DG and FJ.)

Trap: a leaf (J) is never a cut vertex — but its neighbour (F) always is, if the graph has more than two vertices. E is not a cut vertex even though it looks 'central' in the drawing.

Q.17 [NAT · 2 marks · Shortest paths — counting shortest weighted paths]**Answer: 5**

Concept. Compute distances $d(\cdot)$ (Dijkstra), then count paths along tight edges only ($d(u) + w = d(v)$), processing vertices in increasing order of d : $\text{ways}(v) = \sum \text{ways}(u)$ over tight edges (u, v) .

Distances. S 0; A 2; B 3; C = $\min(2 + 3, 3 + 2) = 5$; D = $\min(2 + 4, 5 + 1) = 6$; E = $\min(5 + 2, 3 + 6) = 7$; T = $\min(6 + 2, 7 + 1) = 8$.

Counting.

- $\text{ways}(A) = 1$, $\text{ways}(B) = 1$.
- C: tight from A ($2+3$) and B ($3+2$) → 2.
- D: tight from A ($2+4 = 6$) and C ($5+1 = 6$) → $1 + 2 = 3$.
- E: tight from C only ($B \rightarrow E$ gives 9) → 2.
- T: tight from D ($6+2$) and E ($7+1$) → $3 + 2 = 5$.

The five paths (all of length 8): S-A-D-T, S-A-C-D-T, S-B-C-D-T, S-A-C-E-T, S-B-C-E-T.

Trap: Dijkstra's predecessor array stores only one parent per vertex, so reading paths off the shortest-path tree finds just one of the five.

Q.18 [MCQ · 2 marks · Complexity — nested loops with geometric steps]**Answer: (C)**

Concept. Count iterations exactly, then sum.

- The outer loop runs for $i = 3^0, 3^1, \dots, 3^k$ with $3^k < n$, i.e. about $\log_3 n$ times.
- For a given $i = 3^t$, the inner loop doubles j from i until $j \geq n$: about $\log_2(n / 3^t) = \log_2 n - t \cdot \log_2 3$ iterations.

Sum. With $L = \log_2 n$ and $K = \log_3 n = L / \log_2 3$:

$$\sum_{t=0}^K (L - t \cdot \log_2 3) \approx K \cdot L - \log_2 3 \cdot K^2/2 = L^2/\log_2 3 - L^2/(2 \log_2 3) = L^2 / (2 \log_2 3) = \Theta(\log^2 n).$$

(Numerically, $f(2^{40}) = 537$ and $537 / 40^2 \approx 0.34$, close to $1/(2 \log_2 3) \approx 0.32$.)

Options.

- (A) counts only the outer loop.
- (B) would need the inner loop to be linear in i (e.g. $j += 1$).
- (D) confuses this with a loop over all i from 1 to n .

Trap: the inner loop's trip count shrinks as i grows, but only *linearly in t* , so the sum is still quadratic in $\log n$ (an arithmetic series).

Q.19 [MSQ · 2 marks · Python — string immutability and slicing]

Answer: (A), (B), (D)

Concept. Strings are immutable: `s += "base"` builds a new string and rebinds `s`; `t` still refers to the old "data". Slices create new strings.

- (A) `s.find("a", 2)` searches from index 2: `d0 a1 t2 a3 ...` → first 'a' at or after index 2 is at index 3. **True.**
- (B) `s = "database"; s[::-1] = "esabatad"; [1::2]` takes indices 1, 3, 5, 7 → `s, b, t, d` → "sbtd". **True.**
- (C) "database" contains 'a' at indices 1, 3, 5 → count 3. **False.**
- (D) **True** — `t` is untouched: "data".

Trap: contrasting with lists — had `s` been a list, `s += [...]` would mutate in place and `t` would see the change (compare Q1 of this paper).

Q.20 [MCQ · 2 marks · Graph representations — BFS from an adjacency matrix]

Answer: (D)

Concept. Run BFS from 0, reading neighbours of u from row u of the matrix ($\Theta(V)$ per vertex, $\Theta(V^2)$ total).

Adjacency (from rows). 0: {1, 4}; 1: {0, 2, 6}; 2: {1, 3}; 3: {2, 5}; 4: {0, 5}; 5: {3, 4, 6}; 6: {1, 5}.

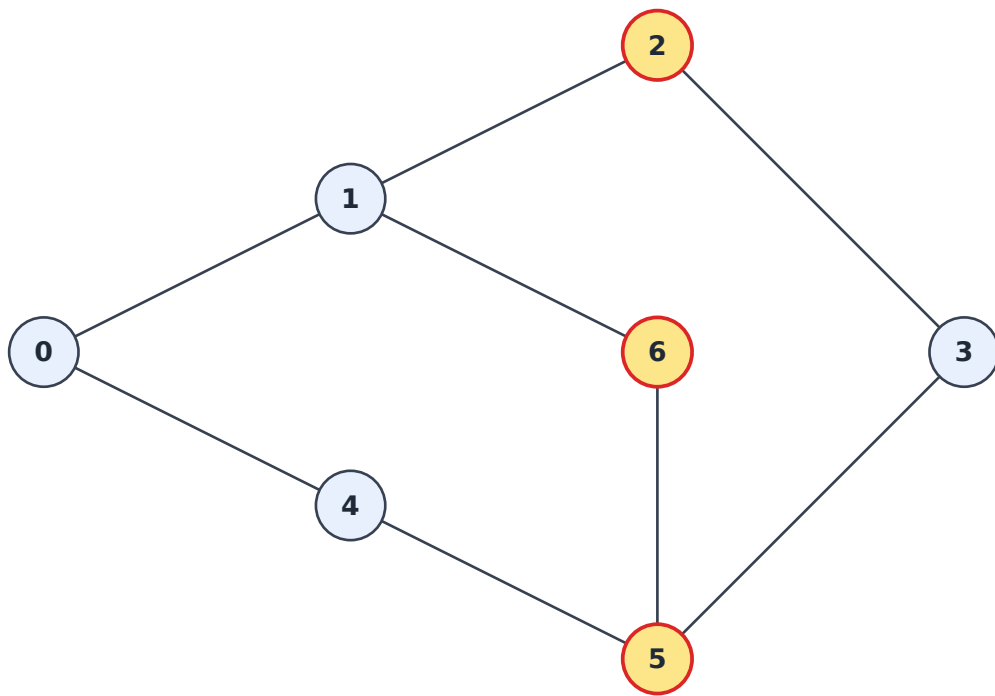
BFS levels.

- Level 0: {0}
- Level 1: {1, 4}
- Level 2: from 1 → 2, 6; from 4 → 5 → {2, 5, 6}
- Level 3: from 2 → 3 (5's neighbour 3 is also level 3) → {3}

Vertices at distance exactly 2: 2, 5, 6 → **3**.

Options. (A) misses 5 (reached via 4). (C) counts every vertex not adjacent to 0 (2, 3, 5, 6), but 3 needs three edges. (B) counts all vertices within distance ≤ 2 (1, 4, 2, 5, 6).

Alternative view: $(M^2)[0][v] > 0$ means a walk of length 2 from 0 to v exists; the vertices at distance exactly 2 are those with $(M^2)[0][v] > 0$, $M[0][v] = 0$ and $v \neq 0$.



The graph; distance-2 vertices highlighted

Mock Test 45

Full-Syllabus Mock — Paper 15

Difficulty: GATE-level

Focus areas: balanced paper across the whole Section 4 syllabus

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — list replication and in-place +=]

Consider the following Python program. What is printed?

Python

```
1 a = [[]] * 3
2 b = [[] for _ in range(3)]
3 a[0].append(1)
4 b[0].append(1)
5 a[1] += [2]
6 print(sum(map(len, a)), sum(map(len, b)))
```

- (A) 2 1
- (B) 3 1
- (C) 6 1
- (D) 4 1

Q.2 [NAT · numerical answer | 1 mark | Queues — deque operations in Python]

Consider the following Python program. The value printed is _____.

Python

```
1 from collections import deque
2
3 d = deque(range(1, 8))
4 d.rotate(3)
5 d.appendleft(d.pop())
6 d.rotate(-2)
7 print(d[0] * 10 + d[-1])
```

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Trees — full binary trees]

A *full* binary tree is one in which every node has either 0 or 2 children. Consider full binary trees with exactly 21 nodes (height = number of edges on the longest root-to-leaf path). Which of the following statements is/are TRUE?

- (A) The maximum possible height is 20
- (B) The minimum possible height is 4
- (C) Every such tree has exactly 11 leaves
- (D) Every such tree has exactly 20 edges

Q.4 [NAT · numerical answer | 1 mark | Hashing — expected number of empty slots]

Eight keys are inserted into a hash table with 8 slots using separate chaining. Assume simple uniform hashing (each key independently hashes to each slot with probability $1/8$). The expected number of slots that remain empty is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

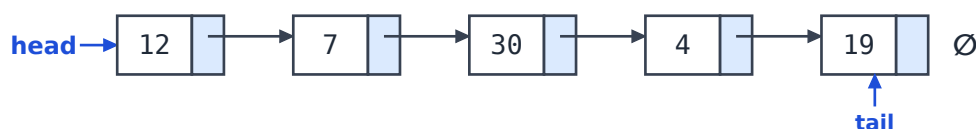
Q.5 [MCQ · single correct | 1 mark | Insertion sort — counting comparisons]

Insertion sort is applied to [3, 1, 2, 6, 4, 5]. When inserting key = $A[i]$, it compares key with $A[i-1]$, $A[i-2]$, ... moving larger elements right, and stops at the first element $\leq$ key or at the front of the array. Every evaluation of $A[j] > \text{key}$ counts as one comparison. The total number of comparisons is

- (A) 4
- (B) 15
- (C) 9
- (D) 8

Q.6 [MCQ · single correct | 1 mark | Linked lists — cost of operations]

A singly linked list maintains pointers to both its first node (head) and its last node (tail), and nothing else. Which of the following operations **cannot** be performed in $O(1)$ worst-case time on a list of n nodes?



Singly linked list with head and tail

- (A) Insert a new node before the first node
- (B) Delete the last node
- (C) Delete the first node
- (D) Insert a new node after the last node

Q.7 [NAT · numerical answer | 1 mark | Graphs — DFS tree and back edges]

Depth-first search is run on a **connected** simple undirected graph with 12 vertices and 20 edges. The number of edges that are **not** edges of the DFS tree is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Complexity — halving loop nest]

What is the time complexity of the following function in terms of n ?

Python

```

1 def g(n):
2     count = 0
3     i = n
4     while i > 0:
5         for j in range(i):
6             count += 1
7         i //= 2
8     return count
  
```

- (A) $\Theta(n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(\log n)$
- (D) $\Theta(n^2)$

Q.9 [NAT · numerical answer | 1 mark | Linear search — expected comparisons]

Linear search scans an array of 6 distinct keys from position 1 to 6, making one comparison per position examined, and stops when the key is found. The key searched for is always present, and it is at position i with probability proportional to i ($i = 1, \dots, 6$). The expected number of comparisons is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Binary search trees — valid pre-order sequences]

Which of the following sequences can be the pre-order traversal of a binary search tree with distinct keys?

- (A) 15, 10, 20, 18, 12
- (B) 30, 20, 25, 10, 40
- (C) 50, 30, 40, 35, 20
- (D) 30, 20, 10, 25, 40, 35, 50

Q.11 [NAT · numerical answer | 2 marks | Python — counting recursive calls]

Consider the following Python program. The value printed is _____.

Python

```
1 calls = 0
2
3 def f(n):
4     global calls
5     calls += 1
6     if n < 3:
7         return n
8     return f(n - 1) + f(n - 3)
9
10 v = f(8)
11 print(calls)
```

Answer: _____ (enter a numerical value)

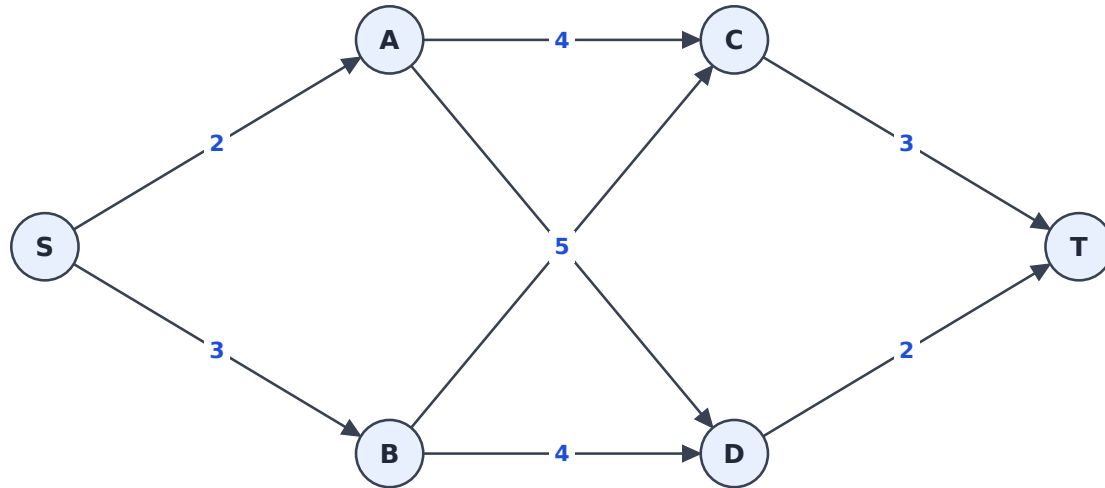
Q.12 [MSQ · one or more correct | 2 marks | Queues — input-restricted deque permutations]

The values 1, 2, 3, 4, 5 are inserted, in this order, at the **rear** of an initially empty deque (no insertion at the front). Deletions may be made from **either** end and may be interleaved arbitrarily with the insertions; each deleted value is appended to an output sequence. Which of the following output sequences is/are possible?

- (A) 5, 3, 1, 2, 4
- (B) 3, 1, 5, 2, 4
- (C) 5, 1, 4, 2, 3
- (D) 4, 2, 1, 3, 5

Q.13 [MCQ · single correct | 2 marks | Shortest paths — Dijkstra with tied path costs]

Dijkstra's algorithm is run from S on the weighted directed graph shown. A tentative distance (and the stored predecessor) is changed only when a **strictly** smaller value is found; when two vertices have equal tentative distance, the alphabetically smaller one is extracted first. Which S-T path is recorded through the predecessor pointers?



- (A) $S \rightarrow A \rightarrow D \rightarrow T$
- (B) $S \rightarrow B \rightarrow C \rightarrow T$
- (C) $S \rightarrow A \rightarrow C \rightarrow T$
- (D) $S \rightarrow B \rightarrow D \rightarrow T$

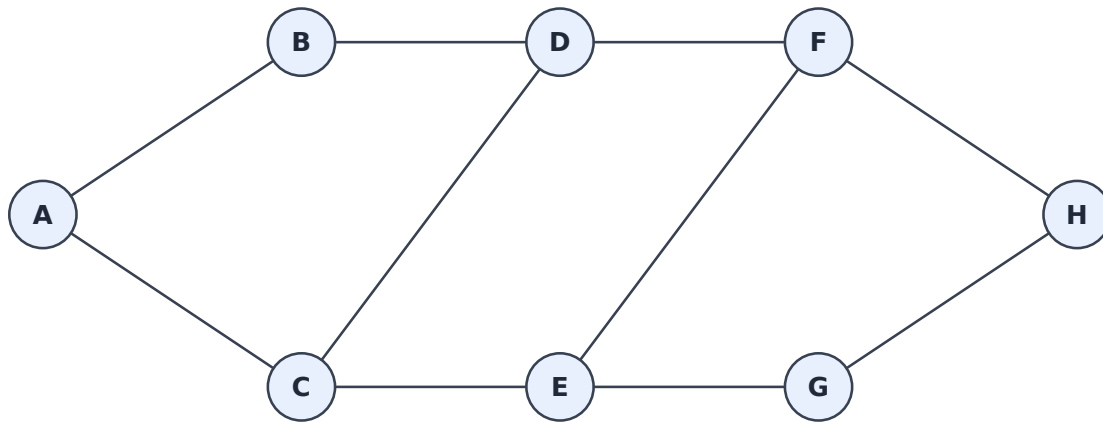
Q.14 [NAT · numerical answer | 2 marks | Heaps — possible positions of the k-th smallest key]

A binary min-heap holds 15 distinct keys in an array $H[0..14]$ (children of index i are $2i + 1$ and $2i + 2$), so it is a perfect binary tree of 4 levels. Over all possible such heaps, the number of distinct array indices at which the **3rd smallest** key can appear is _____.

Answer: _____ (enter a numerical value)

Q.15 [MSQ · one or more correct | 2 marks | Graphs — BFS levels and bipartiteness]

BFS is run from A on the undirected graph shown; neighbours are enqueued in alphabetical order. Which of the following statements is/are TRUE?



- (A) H is at BFS level 3
- (B) Adding the single edge B-C would make the graph non-bipartite
- (C) The BFS tree contains the edge C-E
- (D) The graph is bipartite

Q.16 [MSQ · one or more correct | 2 marks | Quicksort — full Lomuto trace]

Quicksort with the Lomuto partition (pivot = last element of the sub-array; partitioning m elements costs $m - 1$ comparisons) sorts $A = [5, 3, 8, 1, 9, 2, 7]$. Sub-arrays with fewer than 2 elements are not partitioned. Which of the following statements is/are TRUE?

Python

```

1 def partition(A, lo, hi):
2     pivot, i = A[hi], lo - 1
3     for j in range(lo, hi):
4         if A[j] <= pivot:
5             i += 1
6             A[i], A[j] = A[j], A[i]
7     A[i + 1], A[hi] = A[hi], A[i + 1]
8     return i + 1
9
10 def quicksort(A, lo, hi):
11     if lo < hi:
12         p = partition(A, lo, hi)
13         quicksort(A, lo, p - 1)
14         quicksort(A, p + 1, hi)

```

- (A) The second call to partition uses 5 as its pivot
- (B) partition is called exactly 4 times
- (C) After the first partition, $A = [5, 3, 1, 2, 7, 8, 9]$
- (D) The total number of key comparisons is 11

Q.17 [MCQ · single correct | 2 marks | Python — chained generators sharing an iterator]

Consider the following Python program. What is printed?

Python

```
1 def evens(it):
2     for x in it:
3         if x % 2 == 0:
4             yield x
5
6 def squares(it):
7     for x in it:
8         yield x * x
9
10 src = iter(range(10))
11 p = squares(evens(src))
12 first = next(p)
13 rest = list(zip(p, src))
14 print(first, rest)
```

- (A) 0 [(4, 3), (16, 5), (36, 7)]
- (B) 0 [(4, 3), (16, 5), (36, 7), (64, 9)]
- (C) 0 [(4, 0), (16, 1), (36, 2), (64, 3)]
- (D) 0 [(4, 1), (16, 3), (36, 5), (64, 7)]

Q.18 [MSQ · one or more correct | 2 marks | Elementary sorting — stability and counts]

The array $[4_a, 3, 4_b, 1]$ contains two equal keys 4, distinguished by the labels a and b. It is sorted in ascending order by (i) selection sort that picks the **first** minimum (strict $<$) and swaps it into place, (ii) bubble sort that swaps adjacent elements only if the left one is strictly greater, and (iii) insertion sort that shifts elements strictly greater than the key. Which of the following statements is/are TRUE?

- (A) Insertion sort performs exactly 5 shifts
- (B) Bubble sort produces $1, 3, 4_a, 4_b$
- (C) Selection sort produces $1, 3, 4_b, 4_a$
- (D) Selection sort performs exactly 2 swaps (a swap of an element with itself is not counted)

Q.19 [NAT · numerical answer | 2 marks | Hashing — chaining with insertion at the head]

The keys 15, 22, 9, 36, 30, 43, 12, 26 are inserted in that order into a hash table of 7 slots using $h(k) = k \bmod 7$ and separate chaining, where each new key is inserted at the **head** of its chain. Afterwards, each of the 8 keys is searched for exactly once; a search compares the key with chain elements from the head until it is found. The total number of key comparisons over the 8 searches is _____.

Answer: _____ (enter a numerical value)

Q.20 [MCQ · single correct | 2 marks | Binary trees — reconstruction from level-order and in-order]

A binary tree with distinct labels has

- level-order traversal: P, K, W, C, M, S, F, U
- in-order traversal: C, F, K, M, P, S, U, W

What is its post-order traversal?

- (A) F, C, M, K, U, S, W, P
- (B) F, C, M, K, S, U, W, P
- (C) C, F, M, K, U, S, W, P
- (D) F, C, K, M, U, S, W, P

Answer Key — Mock Test 45

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — list replication and in-place +=
2	NAT	1	65	Queues — deque operations in Python
3	MSQ	1	(B), (C), (D)	Trees — full binary trees
4	NAT	1	2.74 to 2.76	Hashing — expected number of empty slots
5	MCQ	1	(D)	Insertion sort — counting comparisons
6	MCQ	1	(B)	Linked lists — cost of operations
7	NAT	1	9	Graphs — DFS tree and back edges
8	MCQ	1	(A)	Complexity — halving loop nest
9	NAT	1	4.32 to 4.34	Linear search — expected comparisons
10	MCQ	1	(D)	Binary search trees — valid pre-order sequences
11	NAT	2	25	Python — counting recursive calls
12	MSQ	2	(B), (C)	Queues — input-restricted deque permutations
13	MCQ	2	(C)	Shortest paths — Dijkstra with tied path costs
14	NAT	2	6	Heaps — possible positions of the k-th smallest key
15	MSQ	2	(B), (C), (D)	Graphs — BFS levels and bipartiteness
16	MSQ	2	(B), (C), (D)	Quicksort — full Lomuto trace
17	MCQ	2	(B)	Python — chained generators sharing an iterator
18	MSQ	2	(B), (C)	Elementary sorting — stability and counts
19	NAT	2	16	Hashing — chaining with insertion at the head
20	MCQ	2	(A)	Binary trees — reconstruction from level-order and in-order

Detailed Solutions — Mock Test 45

Q.1 [MCQ · 1 mark · Python — list replication and in-place +=]

Answer: (C)

Concept. `[[[]] * 3]` copies the *reference* three times: all three slots name one list object. A comprehension evaluates `[]` afresh for every element. For lists, `x += y` mutates `x` in place (like `extend`) and then rebinds the same object.

- `a[0].append(1)`: the single shared list becomes `[1]`; `a = [[1], [1], [1]]`.
- `b[0].append(1)`: only `b[0]` changes; `b = [[1], [], []]`.
- `a[1] += [2]`: extends the shared list in place $\rightarrow [1, 2]$; then stores that same object back into `a[1]`. All slots still alias it: `a = [[1, 2], [1, 2], [1, 2]]`.

Sums of lengths: `a` $\rightarrow 2 + 2 + 2 = 6$; `b` $\rightarrow 1 + 0 + 0 = 1$. Output **6 1** $\rightarrow$ (C).

- (B) assumes `+=` creates a new list for `a[1]` only (true for tuples, not lists).
- (A) counts only one copy of the shared list.
- (D) treats `a[1]` as a separate list `[1, 2]` and the others as `[1]`.

Trap: `a[1] = a[1] + [2]` would create a new list and give `1 + 2 + 1 = 4` (option D) — `+=` and `+` differ for mutable sequences.

Q.2 [NAT · 1 mark · Queues — deque operations in Python]

Answer: 65

Concept. `rotate(k)` with `k > 0` moves the last `k` elements to the front (rotate right); `k < 0` rotates left. `pop()` removes from the right end, `appendleft` adds at the left end.

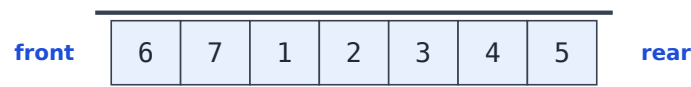
Trace:

- start: `[1, 2, 3, 4, 5, 6, 7]`
- `rotate(3)`: `[5, 6, 7, 1, 2, 3, 4]`
- `pop()` $\rightarrow 4$; `appendleft(4)`: `[4, 5, 6, 7, 1, 2, 3]` (net effect: one more right rotation)
- `rotate(-2)`: `[6, 7, 1, 2, 3, 4, 5]`

`d[0] = 6`, `d[-1] = 5` $\rightarrow 6 \times 10 + 5 = \mathbf{65}$.

Tip: the whole sequence is a net right rotation by `3 + 1 - 2 = 2`, which immediately gives `[6, 7, 1, 2, 3, 4, 5]`.

Trap: reading `rotate(3)` as a left rotation gives `[4, 5, 6, 7, 1, 2, 3]` and a different final answer.



Final deque (left end $\rightarrow$ right end)

Q.3 [MSQ · 1 mark · Trees — full binary trees]

Answer: (B), (C), (D)

Concept. In a full binary tree with `i` internal nodes, there are `i + 1` leaves, so `n = 2i + 1` is always odd. Height is extreme when the tree is as bushy or as stringy as possible.

- (A) The tallest full tree is a 'caterpillar': each internal node has one leaf child and one internal child, adding one level per internal node $\rightarrow$ height = `i` = 10, not 20. **False.**
- (B) A tree of height `h` has at most $2^{h+1} - 1$ nodes; `h = 3` allows only `15 < 21`, while `h = 4` allows `31  $\geq$  21` (and an odd count 21 is achievable). Minimum height 4. **True.**
- (C) `n = 21 = 2i + 1` $\rightarrow i = 10$ internal nodes, 11 leaves. **True.**
- (D) Any tree with `n` nodes has `n - 1 = 20` edges. **True.**

Trap: height 20 would need a path of 21 nodes, each with one child — not allowed in a full binary tree.

Q.4 [NAT · 1 mark · Hashing — expected number of empty slots]

Answer: 2.74 to 2.76

Concept. Linearity of expectation: define an indicator $X_j = 1$ if slot j is empty. Then $E[\text{\#empty}] = \sum P(\text{slot } j \text{ empty})$, and the slots need not be independent.

- $P(\text{a given key misses slot } j) = 7/8$.
- $P(\text{all 8 keys miss slot } j) = (7/8)^8 \approx 0.3436$.
- $E[\text{\#empty}] = 8 \times (7/8)^8 \approx 8 \times 0.3436 = \mathbf{2.75}$.

So even at load factor $\alpha = 1$, about 34% of the slots stay empty ($\approx e^{-1}$ for large tables), and some chains must therefore be longer than 1.

Trap: answering 0 ("8 keys fill 8 slots") or using $1/8$ instead of $(7/8)^8$. No independence between slots is needed — expectation is linear.

Q.5 [MCQ · 1 mark · Insertion sort — counting comparisons]

Answer: (D)

Concept. Inserting $A[i]$ costs (number of shifts) + 1 comparisons, except when the key travels all the way to the front, where the final failing comparison is replaced by the boundary check (no key comparison).

Trace:

- $i = 1$, key 1: $3 > 1$ shift; reaches front $\rightarrow$ 1 comparison. [1, 3, 2, 6, 4, 5]
- $i = 2$, key 2: $3 > 2$ shift; $1 > 2$? no $\rightarrow$ 2 comparisons. [1, 2, 3, 6, 4, 5]
- $i = 3$, key 6: $3 > 6$? no $\rightarrow$ 1 comparison.
- $i = 4$, key 4: $6 > 4$ shift; $3 > 4$? no $\rightarrow$ 2 comparisons. [1, 2, 3, 4, 6, 5]
- $i = 5$, key 5: $6 > 5$ shift; $4 > 5$? no $\rightarrow$ 2 comparisons.

Total = $1 + 2 + 1 + 2 + 2 = \mathbf{8} \rightarrow$ (D).

- (A) 4 is the number of shifts (= inversions).
- (B) $15 = n(n - 1)/2$ is the worst case.
- (C) $9 = \text{shifts} + (n - 1)$ wrongly charges a comparison when 1 reaches the front.

Trap: comparisons = inversions + $(n - 1)$ – (number of keys that reach the front) = $4 + 5 - 1 = 8$.

Q.6 [MCQ · 1 mark · Linked lists — cost of operations]

Answer: (B)

Concept. An operation is $O(1)$ only if every pointer that must change is reachable in $O(1)$ steps from head or tail.

- (A) $\text{new.next} = \text{head}$; $\text{head} = \text{new} \rightarrow O(1)$.
- (B) Deleting the last node requires the **predecessor** of tail, so that its next can be set to None and tail moved to it. In a singly linked list the only way to find it is to walk from head: $\Theta(n)$. **Cannot be $O(1)$.**
- (C) $\text{head} = \text{head.next} \rightarrow O(1)$.
- (D) $\text{tail.next} = \text{new}$; $\text{tail} = \text{new} \rightarrow O(1)$. (This is why a tail pointer makes a linked-list queue efficient.)

Answer (B).

Tip: a queue (enqueue at tail, dequeue at head) works in $O(1)$ with this structure; a deque needing removal at both ends needs a doubly linked list.

Trap: having a tail pointer helps insertion at the end, not deletion there.

Q.7 [NAT · 1 mark · Graphs — DFS tree and back edges]**Answer: 9**

Concept. DFS from one vertex of a connected graph visits every vertex, and each newly discovered vertex adds exactly one tree edge. So the DFS tree is a spanning tree with $n - 1$ edges. In an undirected graph every remaining edge is a back edge (there are no forward/cross edges in undirected DFS).

- Tree edges = $n - 1 = 11$.
- Non-tree (back) edges = $m - (n - 1) = 20 - 11 = 9$.

The answer is independent of the start vertex and of the neighbour order — those change *which* edges are back edges, not how many.

Trap: subtracting n (giving 8). Also note each back edge closes a distinct cycle with the tree: 9 is the cyclomatic number $m - n + 1$.

Q.8 [MCQ · 1 mark · Complexity — halving loop nest]**Answer: (A)**

Concept. When the inner loop's length shrinks geometrically, the total work is a geometric series dominated by its first term.

The outer loop runs with $i = n, n/2, n/4, \dots, 1$ (about $\log_2 n + 1$ times), but the inner loop runs i times, so

count = $n + \lfloor n/2 \rfloor + \lfloor n/4 \rfloor + \dots + 1 < 2n$.

Hence $\Theta(n) \rightarrow (A)$. For $n = 1024$: $1024 + 512 + \dots + 1 = 2047$.

- (B) multiplies the number of outer iterations ($\log n$) by the *largest* inner loop (n) — a valid upper bound, but not tight.
- (C) counts only the outer loop.
- (D) would need the inner loop to run $\sim n$ times on every one of $\sim n$ iterations.

Trap: 'two nested loops $\Rightarrow$ at least $n \log n$ '. Always sum the actual inner lengths.

Q.9 [NAT · 1 mark · Linear search — expected comparisons]**Answer: 4.32 to 4.34**

Concept. If the key is at position i , the search makes exactly i comparisons, so $E[\text{comparisons}] = \sum i \cdot P(i)$.

- Normalising: $P(i) = i / (1 + 2 + \dots + 6) = i / 21$.
- $E = \sum i \cdot i/21 = (1 + 4 + 9 + 16 + 25 + 36)/21 = 91/21 \approx 4.33$.

Compare with the uniform case ($P(i) = 1/6$): $E = (n + 1)/2 = 3.5$. Because later positions are more likely here, the cost is higher.

Tip: if the access probabilities are known, storing keys in **decreasing** order of probability minimises the expected cost — reversing this array would give $\sum (7 - i) \cdot i/21 = 56/21 \approx 2.67$.

Trap: using $\sum i/6$ or forgetting to normalise the probabilities.

Q.10 [MCQ · 1 mark · Binary search trees — valid pre-order sequences]**Answer: (D)**

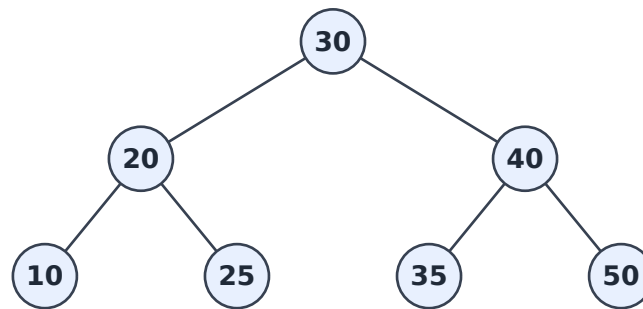
Concept. In a BST pre-order, after the root r come all keys $< r$ (left subtree) followed by all keys $> r$ (right subtree), recursively. Equivalently, once a key larger than some ancestor-to-the-left appears, no later key may drop below that ancestor.

- (A) 15 | 10 | 20, 18, 12: after 20 (> 15) we are in 15's right subtree, so $12 < 15$ is illegal. **Invalid.**

- (B) 30 | 20, 25, 10 | 40: inside the left part, 20 | (nothing smaller first) 25, 10 — after 25 (> 20) we are in 20's right subtree, so $10 < 20$ is illegal. **Invalid.**
- (C) 50 | 30, 40, 35, 20: inside 30's subtree, 40 (> 30) starts the right subtree, so $20 < 30$ cannot follow. **Invalid.**
- (D) 30 | 20, 10, 25 | 40, 35, 50. Left part: 20 | 10 | 25 ✓; right part: 40 | 35 | 50 ✓. **Valid.**

Answer (D).

Tip: a stack-based $O(n)$ check keeps a running lower bound: pop while the top $<$ current key, raising the bound to each popped value.



BST with pre-order (A)

Q.11 [NAT · 2 marks · Python — counting recursive calls]

Answer: 25

Concept. Without memoisation every call spawns its sub-calls again. Let $C(n)$ be the number of calls made by $f(n)$, including itself: $C(n) = 1$ for $n < 3$, and $C(n) = 1 + C(n - 1) + C(n - 3)$ otherwise.

Table:

- $C(0) = C(1) = C(2) = 1$
- $C(3) = 1 + C(2) + C(0) = 3$
- $C(4) = 1 + C(3) + C(1) = 5$
- $C(5) = 1 + C(4) + C(2) = 7$
- $C(6) = 1 + C(5) + C(3) = 11$
- $C(7) = 1 + C(6) + C(4) = 17$
- $C(8) = 1 + C(7) + C(5) = \mathbf{25}$

(For reference, $f(8)$ itself returns 15.)

Trap: computing the *value* recurrence $f(n)$ instead of the *call-count* recurrence, or forgetting the '+1' for the call itself. Note that `global calls` is required — without it, `calls += 1` raises `UnboundLocalError`.

Call counts $C(n)$

	0	1	2	3	4	5	6	7	8
$C(n)$	1	1	1	3	5	7	11	17	25

Q.12 [MSQ · 2 marks · Queues — input-restricted deque permutations]

Answer: (B), (C)

Concept. The deque content is always a contiguous run of the inserted values in increasing order with some removed from its two ends. A value can be output only when it is at the front or at the rear at that moment.

- (A) After outputting 5 first, the deque is [1, 2, 3, 4]; 3 is in the middle and cannot be deleted next. **Impossible.**
- (B) Insert 1, 2, 3; delete rear 3 → [1, 2]; delete front 1 → [2]; insert 4, 5 → [2, 4, 5]; delete rear 5; delete front 2; delete 4. **Possible.**
- (C) Insert 1-5, delete rear 5 → deque [1, 2, 3, 4]; front 1; rear 4; front 2; then 3. **Possible.**
- (D) 4 first means 1, 2, 3 are all in the deque [1, 2, 3]; 2 is in the middle. **Impossible.**

Tip: for an input-restricted deque the only way to fail is to need an element strictly inside the current run. Of the 120 permutations of 1-5, 90 are achievable.



Options (A)/(B): deque after 5 is deleted first

Q.13 [MCQ · 2 marks · Shortest paths — Dijkstra with tied path costs]

Answer: (C)

Concept. All four S-T paths here cost 9, so *which* one Dijkstra reports is decided purely by the order of relaxations and the strict-improvement rule.

Trace:

- Extract S (0): A = 2 (pred S), B = 3 (pred S).
- Extract A (2): C = 6 (pred A), D = 7 (pred A).
- Extract B (3): via B, C would be $3 + 3 = 6$ — not < 6 , unchanged; D would be $3 + 4 = 7$ — not < 7 , unchanged.
- Extract C (6): T = 9 (pred C).
- Extract D (7): via D, T would be $7 + 2 = 9$ — not < 9 , unchanged.
- Extract T (9).

Predecessors: $T \leftarrow C \leftarrow A \leftarrow S$, i.e. $S \rightarrow A \rightarrow C \rightarrow T \rightarrow (C)$.

- (B) would be recorded if ties replaced the predecessor ($\leq$ instead of $<$), since B is relaxed after A.
- (A) and (D) need T's predecessor to be D, but C is extracted before D and sets $T = 9$ first.

Trap: all four options are genuine shortest paths; the question is about the algorithm's *recorded* tree, not about optimality.

Q.14 [NAT · 2 marks · Heaps — possible positions of the k-th smallest key]

Answer: 6

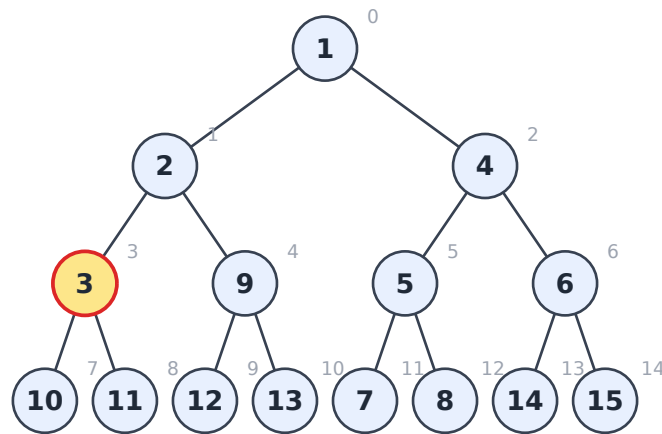
Concept. In a min-heap every ancestor of a key is smaller than it. So the k-th smallest key can have at most $k - 1$ ancestors, i.e. it lies at depth $\leq k - 1$. Also it can never be at the root (depth 0) unless $k = 1$.

For the 3rd smallest key x_3 :

- Depth 0 (index 0): impossible — the root is the minimum.
- Depth 1 (indices 1, 2): possible, e.g. the 2nd smallest is the other child of the root (or a child of x_3 's sibling).
- Depth 2 (indices 3, 4, 5, 6): possible when its two ancestors are the 1st and 2nd smallest — e.g. $H = [1, 2, 4, 3, \dots]$ puts 3 at index 3.
- Depth 3 (indices 7-14): impossible — it would need three smaller ancestors.

Possible indices: $2 + 4 = 6$.

Trap: answering 4 (only depth 2) or 2 (only depth 1). In general the k-th smallest can appear at any depth from 1 to $k - 1$ (for $k \geq 2$).



A valid min-heap with the 3rd smallest key at index 3

Q.15 [MSQ · 2 marks · Graphs — BFS levels and bipartiteness]**Answer: (B), (C), (D)**

Concept. BFS levels give a 2-colouring candidate (even/odd level). A connected graph is bipartite iff no edge joins two vertices on the **same** level (such an edge closes an odd cycle).

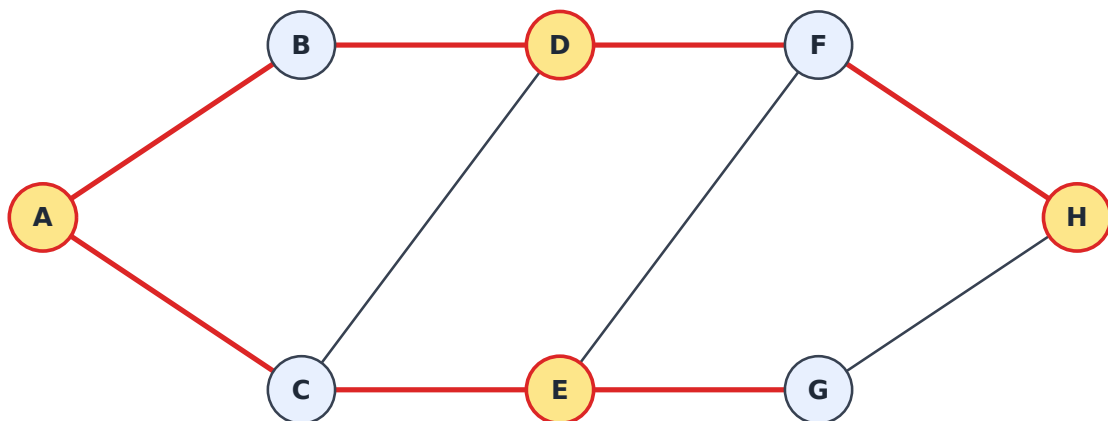
BFS from A:

- Level 0: A. Level 1: B, C.
- From B: D (tree edge B-D). From C: D already seen; E (tree edge C-E). Level 2: D, E.
- From D: F (tree edge D-F). From E: F seen; G (tree edge E-G). Level 3: F, G.
- From F: H (tree edge F-H). Level 4: H.

Every edge joins consecutive levels (A-B, A-C, B-D, C-D, C-E, D-F, E-F, E-G, F-H, G-H), so the graph is bipartite with parts {A, D, E, H} and {B, C, F, G}.

- (A) **False** — H is at level 4.
- (B) **True** — B and C are both on level 1; B-C would create the triangle A-B-C, an odd cycle.
- (C) **True** — E is first discovered from C.
- (D) **True.**

Trap: the graph has many cycles, but all of them have length 4 or 6 — cycles alone do not break bipartiteness, only *odd* cycles do.



BFS tree (red); one side of the bipartition highlighted

Q.16 [MSQ · 2 marks · Quicksort — full Lomuto trace]**Answer: (B), (C), (D)****Concept.** Trace partition calls in the order the recursion makes them (left sub-array first).

Call 1 — $A[0..6]$, pivot 7: elements ≤ 7 are 5, 3, 1, 2 $\rightarrow$ moved to the front in order; 8, 9 end up after. Swaps give [5, 3, 1, 2, 9, 8, 7], then the pivot swap $\rightarrow$ [5, 3, 1, 2, 7, 8, 9], $p = 4$. Comparisons 6.

Call 2 — $A[0..3] = [5, 3, 1, 2]$, pivot 2: only $1 \leq 2 \rightarrow [1, 3, 5, 2] \rightarrow$ pivot swap $\rightarrow [1, 2, 5, 3]$, $p = 1$. Comparisons 3.

- Left [1] (size 1) — no call. Right $A[2..3] = [5, 3]$.

Call 3 — $A[2..3] = [5, 3]$, pivot 3: $5 > 3 \rightarrow$ pivot swap $\rightarrow [3, 5]$. Comparisons 1.

Call 4 — $A[5..6] = [8, 9]$, pivot 9: $8 \leq 9 \rightarrow$ unchanged. Comparisons 1.

Totals: comparisons $6 + 3 + 1 + 1 = 11$; partition calls 4.

- (A) **False** — the second partition works on [5, 3, 1, 2] whose last element is 2.
- (B) **True.**
- (C) **True.**
- (D) **True.**

Trap: picking the *first* element (5) as pivot out of habit, or counting the calls on size-0/1 sub-arrays as partition calls.

Array after each partition call

	0	1	2	3	4	5	6
start	5	3	8	1	9	2	7
call 1	5	3	1	2	7	8	9
call 2	1	2	5	3	7	8	9
call 3	1	2	3	5	7	8	9
call 4	1	2	3	5	7	8	9

Q.17 [MCQ · 2 marks · Python — chained generators sharing an iterator]**Answer: (B)**

Concept. Generators are lazy and `src` is a single iterator shared by the pipeline and by `zip`. Every value taken by one consumer is gone for the other. `zip` pulls from its first argument first, then from the second.

Trace (`src` yields 0, 1, ..., 9):

- `next(p)`: evens takes 0 (even) $\rightarrow$ squares yields 0. first = 0.
- zip round 1: `p` resumes; evens takes 1 (skip), 2 $\rightarrow$ yields 4. `zip` then takes 3 from `src` $\rightarrow$ (4, 3).
- round 2: evens takes 4 $\rightarrow$ 16; `src` gives 5 $\rightarrow$ (16, 5).
- round 3: evens takes 6 $\rightarrow$ 36; `src` gives 7 $\rightarrow$ (36, 7).
- round 4: evens takes 8 $\rightarrow$ 64; `src` gives 9 $\rightarrow$ (64, 9).
- round 5: evens finds `src` exhausted $\rightarrow$ `p` raises `StopIteration` $\rightarrow$ `zip` stops.

Output 0 [(4, 3), (16, 5), (36, 7), (64, 9)] $\rightarrow$ (B).

- (A) stops one round early.
- (C) assumes `src` restarts from 0 for `zip` (as if it were a range, not an iterator).
- (D) assumes `zip` reads `src` *before* `p` in each round.

Trap: the odd numbers that `zip` consumes are exactly the ones evens would have skipped anyway, so the squares are unaffected — but this is luck of the data, not a property of the code.

Q.18 [MSQ · 2 marks · Elementary sorting — stability and counts]**Answer: (B), (C)**

Concept. Bubble and insertion sort move elements only past *strictly* larger neighbours, so equal keys never overtake each other (stable). Selection sort's long-distance swap can jump an element over an equal key (unstable).

Selection sort:

- Pass 0: minimum 1 (index 3) swapped with $4_a \rightarrow [1, 3, 4_b, 4_a]$ (1 swap).
- Pass 1: minimum of $[3, 4_b, 4_a]$ is 3 at index 1 $\rightarrow$ no swap.
- Pass 2: minimum of $[4_b, 4_a]$ with strict $<$ is 4_b (first) $\rightarrow$ no swap.
- Result $1, 3, 4_b, 4_a$ with **1** swap.

Bubble sort: pass 1: $4_a > 3$ swap $\rightarrow [3, 4_a, 4_b, 1]$; 4_a vs 4_b no swap; $4_b > 1$ swap $\rightarrow [3, 4_a, 1, 4_b]$; pass 2: ... $\rightarrow [3, 1, 4_a, 4_b]$; pass 3 $\rightarrow [1, 3, 4_a, 4_b]$.

Insertion sort shifts = number of inversions: $(4_a, 3), (4_a, 1), (3, 1), (4_b, 1) = 4$.

- (A) **False** — 4 shifts (equal keys do not form an inversion).
- (B) **True** — bubble sort is stable.
- (C) **True** — the equal keys end up in reversed order.
- (D) **False** — only 1 real swap.

Trap: counting $(4_a, 4_b)$ as an inversion, or assuming selection sort is stable because it uses strict $<$.

Q.19 [NAT · 2 marks · Hashing — chaining with insertion at the head]**Answer: 16**

Concept. With head insertion, the most recently inserted key of a chain is found with 1 comparison, the one before with 2, and so on. A chain of length L contributes $1 + 2 + \dots + L = L(L + 1)/2$.

Home slots: $15 \rightarrow 1, 22 \rightarrow 1, 9 \rightarrow 2, 36 \rightarrow 1, 30 \rightarrow 2, 43 \rightarrow 1, 12 \rightarrow 5, 26 \rightarrow 5$.

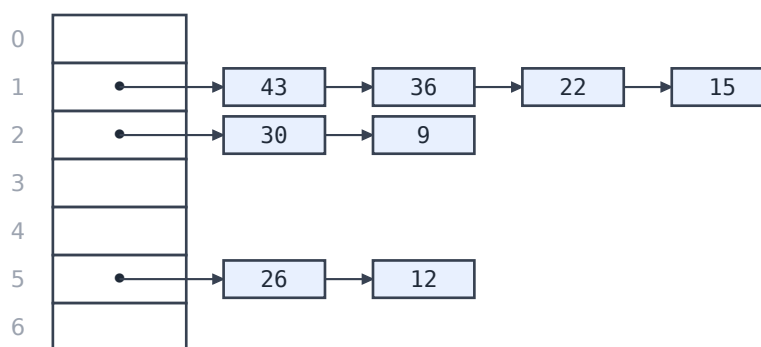
Chains (head first):

- slot 1: $43 \rightarrow 36 \rightarrow 22 \rightarrow 15$ ($L = 4$) $\rightarrow 1 + 2 + 3 + 4 = 10$
- slot 2: $30 \rightarrow 9$ ($L = 2$) $\rightarrow 3$
- slot 5: $26 \rightarrow 12$ ($L = 2$) $\rightarrow 3$

Total = $10 + 3 + 3 = 16$.

Tip: the total is the same for tail insertion — only *which* key is cheap changes. What matters is the chain-length distribution: $\sum L(L + 1)/2$.

Trap: 22, 36 and 43 all reduce to 1 mod 7 (21, 35 and 42 are multiples of 7), which makes one chain much longer than the load factor $8/7$ suggests.



Chains with head insertion

Q.20 [MCQ · 2 marks · Binary trees — reconstruction from level-order and in-order]**Answer: (A)**

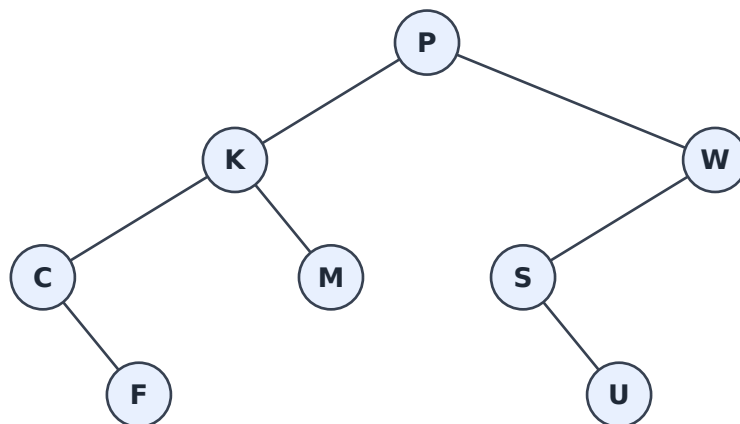
Concept. The first level-order symbol is the root. Split the in-order sequence at the root; for each side, the root of that subtree is the **first** symbol of the level-order sequence that belongs to that side.

- Root P. In-order: [C, F, K, M] | P | [S, U, W].
- Left side {C, F, K, M}: first in level order is K → root K; in-order [C, F] | K | [M]. Left of K: {C, F}, first in level order is C → C, with F right of C in in-order → F is C's right child. Right of K: M.
- Right side {S, U, W}: first in level order is W → root W; in-order [S, U] | W, so W has only a left subtree {S, U} with root S (S precedes U in level order) and U as S's right child.

Tree: $P(K(C(\cdot, F), M), W(S(\cdot, U), \cdot))$. Post-order: F, C, M, K, U, S, W, P → (A).

- (B) swaps U and S — U is S's child, so U must come first.
- (C) puts C before F, but F is C's child.
- (D) puts K before M, but M is K's child.

Trap: the level-order sequence is *not* split into contiguous blocks like pre-order; you must filter it by subtree membership.



Reconstructed tree

Mock Test 46

Challenge Mock — Paper 1

Difficulty: Hard

Focus areas: hardest, multi-concept, trap-heavy 2-mark questions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — short-circuit values, chained comparisons, identity]

Consider the following Python program. What is printed?

Python

```
1 a = 0 or [] or 'GATE'[1:3]
2 b = 1 < 3 > 2 == 2
3 c = [] is not [] and not 0
4 print(a, b, c, (5 > 3) + (2 > 1) * 2)
```

- (A) AT False True 3
- (B) AT True True 3
- (C) True True False 2
- (D) AT True False 3

Q.2 [NAT · numerical answer | 1 mark | Stack permutations — counting with a constraint]

The integers 1, 2, 3, 4, 5 are pushed in this order onto an initially empty stack, with pops interleaved arbitrarily; each popped value is output and the stack is empty at the end. The number of possible output sequences whose **first** element is 3 is _____.

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Complexity — doubling outer loop]

The time complexity of the following fragment, as a function of n , is

Python

```
1 i, s = 1, 0
2 while i < n:
3     for j in range(i):
4         s += 1
5     i *= 2
```

- (A) $\Theta(\log n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n)$
- (D) $\Theta(n^2)$

Q.4 [MSQ · one or more correct | 1 mark | Queue using two stacks]

A queue is implemented with two stacks IN and OUT: `enqueue(x)` pushes `x` on IN; `dequeue()` first, **only if OUT is empty**, pops every element of IN and pushes it on OUT, and then pops OUT.

Starting empty, the operations are:

enq 1, enq 2, enq 3, deq, enq 4, enq 5, deq, deq, deq, enq 6, deq.

Which of the following statements is/are TRUE?

- (A) At the end, OUT is empty and IN contains only 6
- (B) No single dequeue performs more than 5 push/pop operations
- (C) In total, 5 elements are moved from IN to OUT
- (D) The second dequeue moves elements from IN to OUT

Q.5 [NAT · numerical answer | 1 mark | Hashing — expected number of empty slots]

12 keys are inserted into a hash table with 8 slots using separate chaining. Assume simple uniform hashing (each key independently hashes to each slot with probability $1/8$). The expected number of slots that remain **empty** is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.6 [MCQ · single correct | 1 mark | Graph theory — graphical degree sequences]

Which of the following is **not** the degree sequence of any simple undirected graph?

- (A) (5, 5, 4, 3, 2, 2, 1)
- (B) (5, 5, 5, 3, 2, 1, 1)
- (C) (6, 5, 4, 3, 2, 2, 2)
- (D) (4, 4, 3, 3, 2, 2, 2)

Q.7 [MCQ · single correct | 1 mark | Binary search on the answer]

Consider the following Python program. What is printed?

Python

```
1 N = 1000
2 lo, hi, it = 0, N, 0
3 while lo < hi:
4     it += 1
5     m = (lo + hi + 1) // 2
6     if m * m <= N:
7         lo = m
8     else:
9         hi = m - 1
10 print(lo, it)
```

- (A) 32 9
- (B) 31 10
- (C) 31 9
- (D) 31 11

Q.8 [MCQ · single correct | 1 mark | Sorting — adjacent swaps versus arbitrary swaps]

For the array [4, 3, 1, 6, 5, 2, 8, 7], let `x` be the minimum number of swaps of **adjacent** elements needed to sort it in ascending order, and `y` the minimum number of swaps of **arbitrary** pairs of elements. The pair (`x`, `y`) is

- (A) (9, 5)
- (B) (9, 4)
- (C) (5, 5)
- (D) (8, 5)

Q.9 [MSQ · one or more correct | 1 mark | Python — scoping errors]

Each of the four snippets below is run **separately**, in a fresh interpreter. Which of them raise an exception?

Python

```
1 # Snippet (A)
2 x = 5
3 def f():
4     print(x)
5     x = 6
6 f()
7
8 # Snippet (B)
9 def g():
10     total = 0
11     def add(v):
12         total += v
13     add(3)
14     return total
15 g()
16
17 # Snippet (C)
18 fs = [lambda: i for i in range(3)]
19 fs[0]()
20
21 # Snippet (D)
22 def h(a, b=2, *args, c, **kw):
23     return a + b + c + len(kw)
24 h(1, c=3, d=4)
```

- (A) Snippet (D)
- (B) Snippet (C)
- (C) Snippet (B)
- (D) Snippet (A)

Q.10 [MCQ · single correct | 1 mark | Binary trees — counting shapes of maximum height]

The number of structurally distinct binary trees with **5** nodes whose height (number of edges on the longest root-to-leaf path) is **4** is

- (A) 8
- (B) 32
- (C) 14
- (D) 16

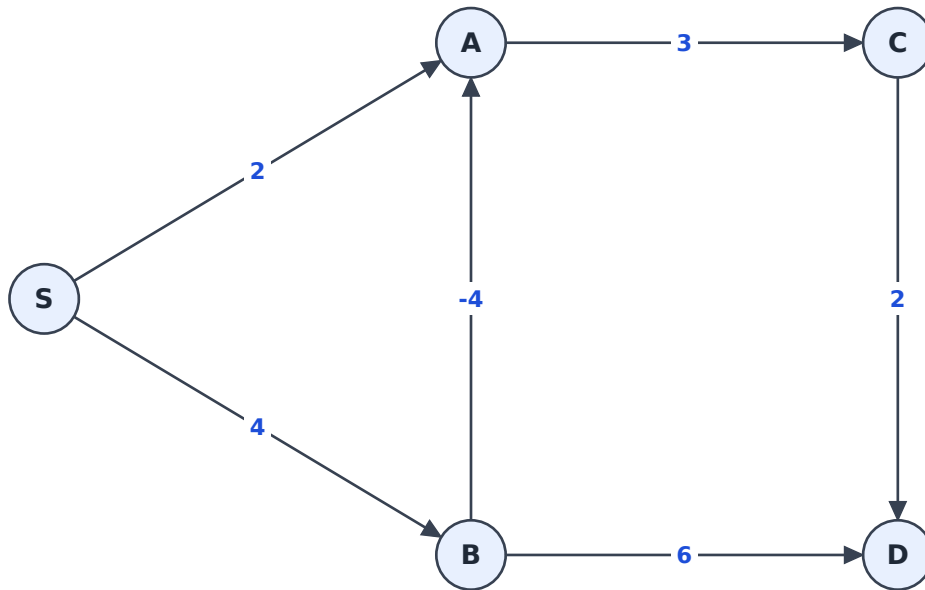
Q.11 [NAT · numerical answer | 2 marks | BST — insertion orders of minimum height]

The keys 1, 2, 3, 4, 5, 6 are inserted in some order into an initially empty BST. Of the 720 possible insertion orders, the number that produce a BST of the **minimum possible height** (height = number of edges on the longest root-to-leaf path) is _____.

Answer: _____ (enter a numerical value)

Q.12 [MSQ · one or more correct | 2 marks | Dijkstra with a negative edge]

The standard Dijkstra algorithm (a vertex, once extracted, is final and never updated again) is run from **S** on the directed graph below, which has one negative edge $B \rightarrow A$. Which of the following statements is/are TRUE?



- (A) Dijkstra reports the correct shortest distance to B
- (B) Adding 4 to every edge weight and running Dijkstra yields shortest paths that are also shortest in the original graph
- (C) Dijkstra reports the correct shortest distance to D
- (D) Dijkstra reports $d(A) = 2$, whereas the true shortest distance to A is 0

Q.13 [NAT · numerical answer | 2 marks | Divide and conquer — inversion counting]

Inversions of $A = [7, 2, 9, 4, 11, 1, 8, 3, 10, 5]$ are counted by the merge-sort method: the array is split into $A[0..4]$ and $A[5..9]$, each half is sorted recursively (counting its own inversions), and the remaining *split* inversions — pairs (i, j) with i in the left half, j in the right half and $A[i] > A[j]$ — are counted during the **final** merge. The number of inversions counted during that final merge is _____.

Answer: _____ (enter a numerical value)

Q.14 [MCQ · single correct | 2 marks | Python — mutable defaults, closures and try/finally]

Consider the following Python program. What is printed?

Python

```

1 def make(n, acc=[]):
2     acc.append(n)
3     return lambda k: [a * k for a in acc]
4
5 f = make(1)
6 g = make(2)
7 h = make(3, [])
8 print(f(10), g(1), h(2))
9
10 def t():
11     try:
12         return f(1)
13     finally:
14         make(4)
15
16 print(t(), g(1))

```

- (A) [10, 20] [1, 2] [6] / [1, 2] [1, 2, 4]
 (B) [10] [2] [6] / [1] [2]
 (C) [10, 20] [1, 2] [6] / [1, 2, 4] [1, 2, 4]
 (D) [10, 20] [1, 2] [2, 4, 6] / [1, 2] [1, 2, 4]

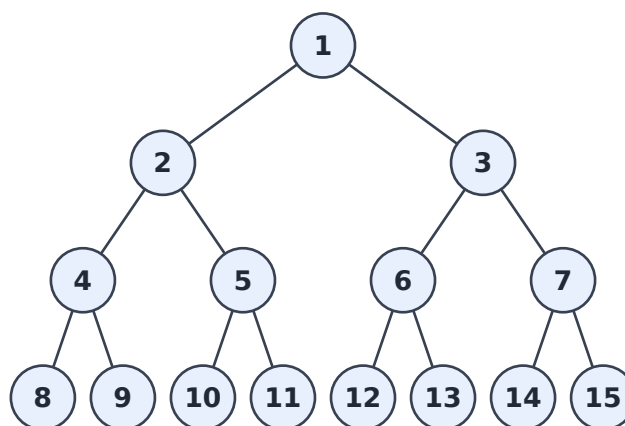
Q.15 [NAT · numerical answer | 2 marks | Linear probing — dependence on insertion order]

The keys 14, 24, 35, 4, 15 are inserted into an initially empty hash table with slots 0–9 using $h(k) = k \bmod 10$ and linear probing (step +1). Of the $5! = 120$ possible insertion orders, the number in which key **15** ends up in slot **5** is _____.

Answer: _____ (enter a numerical value)

Q.16 [MSQ · one or more correct | 2 marks | Heaps — structural bounds]

Consider binary heaps on **15 distinct** keys stored in $A[0..14]$ (children of i at $2i+1, 2i+2$). Which of the following statements is/are TRUE?

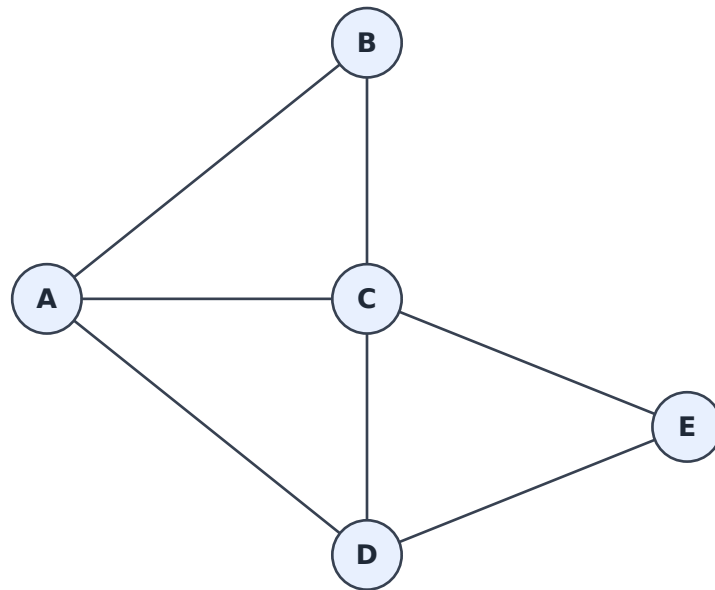


A min-heap on 15 keys (keys 1..15 in level order) — shape reference

- (A) Inserting the keys 1, 2, ..., 15 in this order, one at a time, into an initially empty **max**-heap performs 34 swaps in total
 (B) Bottom-up build-heap performs at most 11 swaps on any input of 15 keys
 (C) In a **min**-heap, the second smallest key is always at index 1 or 2
 (D) In a **min**-heap, the largest key can be at index 6

Q.17 [NAT · numerical answer | 2 marks | DFS — counting possible visit orders]

Depth-first search (recursive) is started at **A** on the undirected graph shown below. The order in which a vertex scans its neighbours is **arbitrary** (any order may be used at any vertex). The number of distinct orders in which the five vertices can be first visited is _____.



Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Output-restricted deque permutations]

The values 1, 2, 3, 4 arrive in this order. Each arriving value must be inserted into a deque at **either end**; at any time the element at the **front** may be removed and output (removal only at the front). Which of the following output sequences is **impossible**?



e.g. after inserting 1, 2 at front, 3 at rear

- (A) 4, 2, 1, 3
- (B) 3, 1, 2, 4
- (C) 1, 4, 2, 3
- (D) 4, 1, 3, 2

Q.19 [NAT · numerical answer | 2 marks | Recurrences — exact operation count]

Consider the following Python program. The value printed is _____.

Python

```
1 c = 0
2 def f(n):
3     global c
4     if n <= 1:
5         c += 1
6         return
7     for i in range(n // 2):
8         c += 1
9     f(n // 2)
10    f(n // 2)
11    f(n // 2)
12
13 f(16)
14 print(c)
```

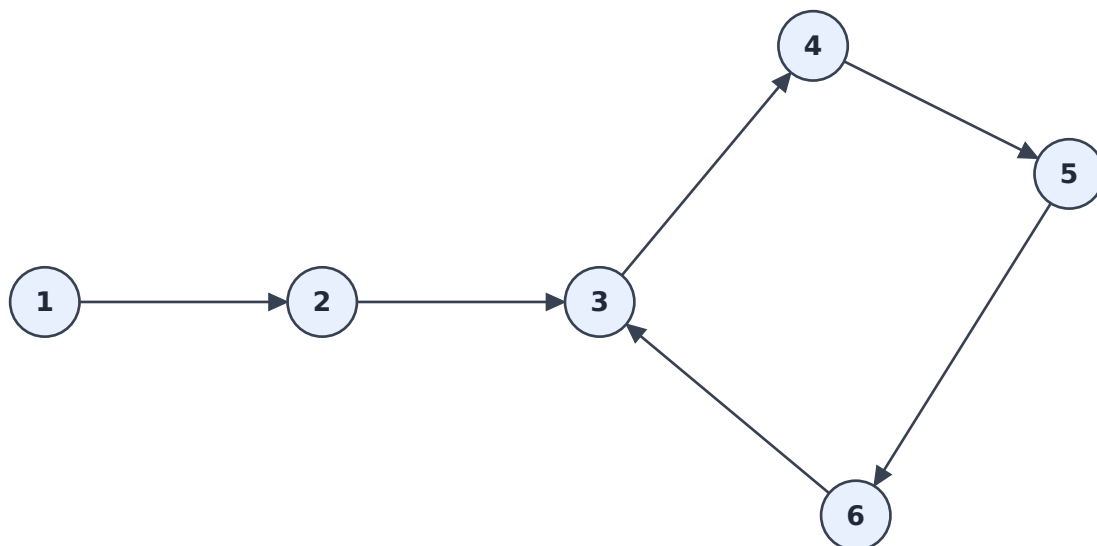
Answer: _____ (enter a numerical value)

Q.20 [MSQ · one or more correct | 2 marks | Reversing a linked list that contains a cycle]

A singly linked list has nodes with values 1, 2, 3, 4, 5, 6 linked in this order, and node 6 points back to node 3 (see figure). The standard iterative reversal below is applied with **head** = node 1. Which of the following statements is/are TRUE?

Python

```
1 prev, cur, steps = None, head, 0
2 while cur:
3     cur.nxt, prev, cur = prev, cur, cur.nxt
4     steps += 1
5 head = prev
```



- (A) After the loop, **head** refers to the node with value 1
- (B) The loop body executes exactly 9 times
- (C) The loop terminates
- (D) After the loop, following **nxt** from **head** visits 1, 2, 3, 4, 5, 6, 3, ...

Answer Key — Mock Test 46

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — short-circuit values, chained comparisons, identity
2	NAT	1	9	Stack permutations — counting with a constraint
3	MCQ	1	(C)	Complexity — doubling outer loop
4	MSQ	1	(A), (C)	Queue using two stacks
5	NAT	1	1.60 to 1.62	Hashing — expected number of empty slots
6	MCQ	1	(B)	Graph theory — graphical degree sequences
7	MCQ	1	(C)	Binary search on the answer
8	MCQ	1	(A)	Sorting — adjacent swaps versus arbitrary swaps
9	MSQ	1	(C), (D)	Python — scoping errors
10	MCQ	1	(D)	Binary trees — counting shapes of maximum height
11	NAT	2	80	BST — insertion orders of minimum height
12	MSQ	2	(A), (D)	Dijkstra with a negative edge
13	NAT	2	15	Divide and conquer — inversion counting
14	MCQ	2	(A)	Python — mutable defaults, closures and try/finally
15	NAT	2	42	Linear probing — dependence on insertion order
16	MSQ	2	(A), (B), (C)	Heaps — structural bounds
17	NAT	2	9	DFS — counting possible visit orders
18	MCQ	2	(D)	Output-restricted deque permutations
19	NAT	2	146	Recurrences — exact operation count
20	MSQ	2	(A), (B), (C)	Reversing a linked list that contains a cycle

Detailed Solutions — Mock Test 46

Q.1 [MCQ · 1 mark · Python — short-circuit values, chained comparisons, identity]

Answer: (B)

Four independent traps:

- `or` returns the **first truthy operand** (or the last one): `0` and `[]` are falsy, so `a = 'GATE'[1:3] = 'AT'` (indices 1 and 2) — a string, not `True`.
- Chained comparison `1 < 3 > 2 == 2` means `1 < 3 and 3 > 2 and 2 == 2` → `True` (it is *not* `(1 < 3) > 2`, which would be `False`).
- Each `[]` literal creates a new list, so `[] is not []` is `True`; `not 0` is `True`; `and` returns the last evaluated operand → `True`.
- Booleans are integers: `True + True * 2 = 1 + 2 = 3`.

Output: `AT True True 3`.

- (A) evaluates the chain left to right as `(1 < 3) > 2`.
- (C) treats `or` as returning a bool and adds without precedence.
- (D) thinks two empty lists are the same object.

Trap: `and/or` return operands, not booleans; `is` compares identity, not equality.

Q.2 [NAT · 1 mark · Stack permutations — counting with a constraint]

Answer: 9

To output 3 first, we must push 1, 2, 3 and pop 3 immediately. Now the stack holds [1, 2] (2 on top) and 4, 5 are still to be pushed.

Count the ways to finish. Think of it as interleaving: 2 must come out before 1 (stack order), and 4, 5 can be pushed/popped in between.

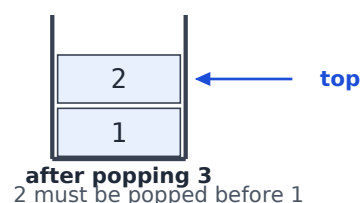
Enumerate by when 4 is pushed relative to popping 2 and 1:

- Pop 2, pop 1, then 4/5: {4 5, 5 4} → 2 sequences (3 2 1 4 5, 3 2 1 5 4).
- Pop 2, then 4/5 activity, then 1 at some point: 3 2 4 1 5, 3 2 4 5 1, 3 2 5 4 1 → 3.
- Push 4 before popping 2: 3 4 2 1 5, 3 4 2 5 1, 3 4 5 2 1, 3 5 4 2 1 → 4.

Total = 2 + 3 + 4 = **9**.

Check by the pattern rule: a sequence is achievable iff it has no `a...b...c` with `c < a < b`. After the leading 3, the remaining 4 values must avoid the pattern and keep 2 before 1.

Trap: answering $C_4 = 14$ (as if the stack were empty after outputting 3) ignores that 1 and 2 are already stacked in a fixed order.



Q.3 [MCQ · 1 mark · Complexity — doubling outer loop]

Answer: (C)

The outer loop runs about $\log_2 n$ times, but the inner loop's length **doubles** each time ($i = 1, 2, 4, \dots, < n$). Total inner iterations:

$1 + 2 + 4 + \dots + 2^k$ where $2^k < n \leq 2^{k+1} \rightarrow \text{sum} = 2^{k+1} - 1 < 2n$.

Hence the work is **$\Theta(n)$** (a geometric series is dominated by its last term).

- (A) counts only outer iterations.

- (B) multiplies $\log n$ iterations by the *maximum* inner length n — an upper bound that is not tight.
- (D) has no basis.

Trap: 'a loop inside a log-loop' is not automatically $n \log n$; sum the actual inner lengths.

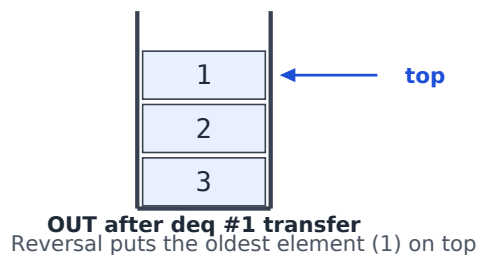
Q.4 [MSQ · 1 mark · Queue using two stacks]

Answer: (A), (C)

Elements are transferred only when OUT is empty, which is what makes the amortised cost $O(1)$.

- enq 1, 2, 3 $\rightarrow$ IN [1, 2, 3].
- deq #1: OUT empty $\rightarrow$ move 3 elements (OUT [3, 2, 1], top 1); pop 1. Cost 3 pops + 3 pushes + 1 pop = **7** operations.
- enq 4, 5 $\rightarrow$ IN [4, 5].
- deq #2: OUT = [3, 2] not empty $\rightarrow$ pop 2 (no transfer).
- deq #3: pop 3. OUT empty.
- deq #4: OUT empty $\rightarrow$ move 4, 5 (OUT [5, 4]); pop 4.
- enq 6 $\rightarrow$ IN [6].
- deq #5: OUT = [5] $\rightarrow$ pop 5. OUT empty; IN = [6].
- (A) True.
- (B) False — the first dequeue performs 7 push/pop operations.
- (C) $3 + 2 = 5$ transfers. True.
- (D) False — OUT still held 2 and 3.

Trap: transferring on every dequeue (not only when OUT is empty) would break FIFO order.



Q.5 [NAT · 1 mark · Hashing — expected number of empty slots]

Answer: 1.60 to 1.62

Use linearity of expectation with an indicator for each slot.

$$P(\text{a particular slot is empty}) = P(\text{none of the 12 keys hashes there}) = (1 - 1/8)^{12} = (7/8)^{12}.$$

$$(7/8)^2 = 0.765625; (7/8)^4 \approx 0.586182; (7/8)^8 \approx 0.343609; (7/8)^{12} \approx 0.343609 \times 0.586182 \approx 0.201417.$$

$$E[\text{empty slots}] = 8 \times 0.201417 \approx \mathbf{1.61}.$$

Trap: the events 'slot i empty' are *not* independent, but linearity of expectation does not need independence. Also, $n > m$ does not mean every slot is used — about 20% of slots are still empty on average.

Q.6 [MCQ · 1 mark · Graph theory — graphical degree sequences]

Answer: (B)

All four have an even sum and maximum degree $\leq 6 = n - 1$, so the quick checks pass. Use **Havel-Hakimi**: remove the largest degree d and subtract 1 from the next d degrees; repeat.

(B) (5, 5, 5, 3, 2, 1, 1): remove 5 $\rightarrow$ (4, 4, 2, 1, 0, 1) $\rightarrow$ sort (4, 4, 2, 1, 1, 0); remove 4 $\rightarrow$ (3, 1, 0, 0, 0) $\rightarrow$ remove 3 $\rightarrow$ needs three positive entries after it, but we get (0, -1, -1) $\rightarrow$ **not graphical**.

(Erdős-Gallai at $k = 3$: $5 + 5 + 5 = 15 > 3 \cdot 2 + \min(3, 3) + \min(2, 3) + 1 + 1 = 13$.) Intuitively, three

vertices of degree 5 each need 3 neighbours outside the trio, but the other four vertices have total degree only 7 and two of them have degree 1.

- (A) remove 5 $\rightarrow$ (4, 3, 2, 1, 1, 1) $\rightarrow$ remove 4 $\rightarrow$ (2, 1, 0, 0, 1) $\rightarrow$ (2, 1, 1, 0, 0) $\rightarrow$ (0, 0, 0, 0) ✓ graphical.
- (C) remove 6 $\rightarrow$ (4, 3, 2, 1, 1, 1) $\rightarrow$ same as above ✓.
- (D) remove 4 $\rightarrow$ (3, 2, 2, 1, 2, 2) $\rightarrow$ (3, 2, 2, 2, 2, 1) $\rightarrow$ (1, 1, 1, 2, 1) $\rightarrow$... ✓.

Trap: 'even sum and max degree $\leq n - 1$ ' is necessary but **not sufficient**.

Q.7 [MCQ · 1 mark · Binary search on the answer]

Answer: (C)

This finds the **largest m with $m^2 \leq N$** (integer square root). Using the *upper middle* $(lo+hi+1)//2$ together with $lo = m$ guarantees progress (with the lower middle the loop could stick at $hi = lo + 1$).

Trace (lo, hi) $\rightarrow$ m:

- (0, 1000) m=500 $>$ $\rightarrow$ hi=499
- (0, 499) m=250 $>$ $\rightarrow$ hi=249
- (0, 249) m=125 $>$ $\rightarrow$ hi=124
- (0, 124) m=62 $>$ $\rightarrow$ hi=61
- (0, 61) m=31: $961 \leq 1000 \rightarrow lo=31$
- (31, 61) m=46 $>$ $\rightarrow$ hi=45
- (31, 45) m=38 $>$ $\rightarrow$ hi=37
- (31, 37) m=34 $>$ $\rightarrow$ hi=33
- (31, 33) m=32: $1024 >$ $\rightarrow$ hi=31 $\rightarrow$ loop ends.

9 iterations, lo = 31. Output **31 9**.

- (A) 32 would be the ceiling square root; $32^2 = 1024 > 1000$.
- (B), (D) miscount iterations (e.g. adding a final check).

Trap: the iteration count is not simply $\lceil \log_2 1001 \rceil = 10$, because each step removes slightly more than half when it moves $hi = m - 1$.

Q.8 [MCQ · 1 mark · Sorting — adjacent swaps versus arbitrary swaps]

Answer: (A)

- An adjacent swap removes **exactly one inversion**, so x = number of inversions (this is the number of swaps bubble sort or insertion sort performs).
- An arbitrary swap can fix one element per swap within a permutation cycle, so $y = n - (\text{number of cycles})$.

Inversions: 4 > 3, 1, 2 (3); 3 > 1, 2 (2); 6 > 5, 2 (2); 5 > 2 (1); 8 > 7 (1) $\rightarrow x = 9$.

Cycles (value v belongs at index $v - 1$): index 0 holds 4 $\rightarrow$ index 3 holds 6 $\rightarrow$ index 5 holds 2 $\rightarrow$ index 1 holds 3 $\rightarrow$ index 2 holds 1 $\rightarrow$ back to index 0: a 5-cycle. Index 4 holds 5 (fixed point). Indices 6, 7 hold 8, 7: a 2-cycle. Cycles = 3 $\rightarrow y = 8 - 3 = 5$.

- (B) counts cycles of length > 1 as 3 and subtracts wrongly ($8 - 4$).
- (C) assumes adjacent swaps are as powerful as arbitrary ones.
- (D) misses one inversion.

Tip: selection sort achieves y (at most $n - 1$ swaps); bubble/insertion sort achieve x .

Q.9 [MSQ · 1 mark · Python — scoping errors]**Answer: (C), (D)**

A name that is **assigned anywhere** in a function body is local to that function for the whole body (decided at compile time).

- (A) `c` is keyword-only (after `*args`); the call supplies it, and `d=4` goes into `kw`. Returns `1 + 2 + 3 + 1 = 7`. No error.
- (B) The lambdas look up `i` at call time and see 2; `fs[0]()` returns 2. No error.
- (C) `total += v` assigns `total` inside `add`, making it local to `add`; reading it first fails → **UnboundLocalError** (needs `nonlocal total`).
- (D) `x = 6` makes `x` local to `f`, so `print(x)` reads an unassigned local → **UnboundLocalError**.

Trap: (D) fails even though a global `x` exists — the local assignment *later* in the body still shadows it from the first line.

Q.10 [MCQ · 1 mark · Binary trees — counting shapes of maximum height]**Answer: (D)**

Height 4 with 5 nodes means the longest path already contains all 5 nodes, so the tree is a **path** (every node except the last has exactly one child).

Each of the 4 parent→child links independently goes **left or right** → $2^4 = 16$ distinct trees.

- (A) 8 forgets one link.
- (B) $32 = 2^5$ counts a choice for the leaf too.
- (C) 14 is the Catalan number C_4 (all binary trees with 4 nodes) — irrelevant here.

Tip: in general, the number of n -node binary trees of height $n - 1$ is 2^{n-1} ; these are exactly the BSTs produced by insertion orders in which every new key is a new minimum or maximum of the remaining ones.

Q.11 [NAT · 2 marks · BST — insertion orders of minimum height]**Answer: 80**

With 6 keys the minimum height is $\lceil \log_2 7 \rceil - 1 = 2$ (a height-2 tree holds at most 7 keys).

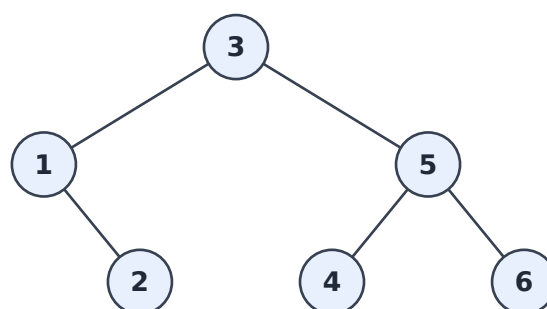
Which roots work? The two subtrees must each have height ≤ 1 , i.e. ≤ 3 keys. With 5 non-root keys, the split must be $2 + 3$ or $3 + 2$ → root is **3** or **4**.

Root 3: left $\{1, 2\}$ (any order: 2 orders, both give height 1); right $\{4, 5, 6\}$ must have height 1, so its root must be 5, i.e. 5 is inserted before 4 and 6 → 2 orders. The two subsequences can be interleaved in $C(5, 2) = 10$ ways. Count = $10 \times 2 \times 2 = 40$.

Root 4: symmetric (left $\{1, 2, 3\}$ needs 2 first; right $\{5, 6\}$ free) → 40.

Total = $40 + 40 = 80$.

Trap: remembering '80' as the classic answer for 7 keys and a *perfect* tree is a coincidence here — derive it: forgetting the interleaving factor $C(5, 2)$ gives only 8.



One minimum-height BST (e.g. order 3, 5, 1, 6, 2, 4)

Q.12 [MSQ · 2 marks · Dijkstra with a negative edge]**Answer: (A), (D)**

Dijkstra trace: extract S (0): A = 2, B = 4. Extract A (2): C = 5. Extract B (4): edge B → A would give 0, but A is already final → ignored; D = 10. Extract C (5): D = 7. Extract D (7).

Reported: A 2, B 4, C 5, D 7.

True distances (no negative cycle): A = $4 - 4 = 0$ (S→B→A), C = 3, D = 5 (S→B→A→C→D: $4 - 4 + 3 + 2$), B = 4.

- (A) $d(B) = 4$ is correct (no path to B uses the negative edge). **True.**
- (B) After +4: S→B→D costs $8 + 10 = 18$, while S→B→A→C→D costs $8 + 0 + 7 + 6 = 21$, so Dijkstra picks S→B→D (original cost 10), not the true shortest (cost 5). Adding a constant penalises paths with **more edges**. **False.**
- (C) Reported 7 vs true 5 — the error propagates through C. **False.**
- (D) **True** — A was finalised before B's negative edge was seen.

Trap: uniform reweighting is not Johnson's reweighting (which uses vertex potentials $w'(u,v) = w(u,v) + h(u) - h(v)$).

Reported vs true distances

	S	A	B	C	D
Dijkstra	0	2	4	5	7
true	0	0	4	3	5

Q.13 [NAT · 2 marks · Divide and conquer — inversion counting]**Answer: 15**

During a merge, whenever an element y of the right half is output before some remaining left elements, it forms an inversion with **every** remaining left element.

Sorted halves: L = [2, 4, 7, 9, 11], R = [1, 3, 5, 8, 10]. Merge:

- 1 (from R) output while all 5 of L remain → +5
- 2 (L), then 3 (R): L has 4, 7, 9, 11 left → +4
- 4 (L), then 5 (R): 7, 9, 11 left → +3
- 7 (L), then 8 (R): 9, 11 left → +2
- 9 (L), then 10 (R): 11 left → +1
- 11 (L).

Split inversions = $5 + 4 + 3 + 2 + 1 = 15$.

(For completeness: the left half [7, 2, 9, 4, 11] has 3 inversions and the right half [1, 8, 3, 10, 5] has 3, so the total is 21.)

Trap: counting the inversions of the whole array (21), or adding +1 per right element instead of +(number of remaining left elements).

Final merge: right elements and split inversions

	1	3	5	8	10
left items still pending	5	4	3	2	1

Q.14 [MCQ · 2 marks · Python — mutable defaults, closures and try/finally]**Answer: (A)**

(/ separates the two output lines.) Three mechanisms interact:

1. `acc=[]` is created **once**; `make(1)` and `make(2)` append to the same list $\rightarrow acc = [1, 2]$. `make(3, [])` uses a fresh list `[3]`.
2. Each lambda closes over *its* `acc` object and reads it **when called**. So `f` and `g` see the same list `[1, 2]`: `f(10) = [10, 20]`, `g(1) = [1, 2]`; `h(2) = [6]`.
3. In `t()`, the `return` expression `f(1)` is evaluated **before** the `finally` block runs: it builds the new list `[1, 2]`. Then `finally` calls `make(4)`, appending 4 to the shared `acc`. The already-computed return value is unaffected, but the later `g(1)` sees `[1, 2, 4]`.

Output:

```
[10, 20] [1, 2] [6]
```

```
[1, 2] [1, 2, 4]
```

- (B) ignores the shared default list.
- (C) assumes `finally` runs before the return value is computed.
- (D) assumes `make(3, [])` also used the shared list.

Trap: `finally` runs after the return expression is evaluated but before the function actually returns; it can mutate shared state (and could even override the return value with its own `return`).

Q.15 [NAT · 2 marks · Linear probing — dependence on insertion order]

Answer: 42

Keys 14, 24, 4 hash to slot 4; 35 and 15 hash to slot 5. All five end up in the cluster of slots 4–8, and 15 lands in slot 5 **iff slot 5 is still empty when 15 is inserted**.

Who can fill slot 5 before 15?

- 35 (its home slot), or
- a 4-key (14, 24 or 4) that finds slot 4 occupied, i.e. the **second** 4-key to be inserted.

So 15 gets slot 5 iff, among the keys inserted before 15, there is no 35 and at most one 4-key. Let p = number of keys before 15:

- $p = 0$: 15 first $\rightarrow$ any order of the other 4: $4! = 24$.
- $p = 1$: the key before 15 is one of the three 4-keys (3 ways); the remaining 3 keys follow in any order ($3! = 6$) $\rightarrow 18$.
- $p \geq 2$: impossible (either 35 or a second 4-key precedes 15).

Total = $24 + 18 = 42$.

Trap: assuming the first 4-key always takes slot 4 'and blocks nothing' — the second 4-key spills into slot 5 and steals 15's home.

0	
1	
2	
3	
4	14
5	15
6	24
7	35
8	4
9	

Result for the order 14, 15, 24, 35, 4

Q.16 [MSQ · 2 marks · Heaps — structural bounds]**Answer: (A), (B), (C)**

- (A) Each new key is the largest so far and sifts up to the root, making as many swaps as its depth: depths 0 (1 key), 1 (2 keys), 2 (4 keys), 3 (8 keys) $\rightarrow 0 + 2 + 8 + 24 = 34$. **True.**
- (B) A sift-down from a node of height h makes at most h swaps. Heights: root 3, two nodes 2, four nodes 1, eight leaves 0 $\rightarrow$ at most $3 + 2 \cdot 2 + 4 \cdot 1 = 11$ swaps (reached e.g. for ascending input to a max-heap). **True.**
- (C) The second smallest has only one smaller key (the root), so its parent must be the root $\rightarrow$ index 1 or 2. **True.**
- (D) Index 6 has children 13 and 14, which must be larger than it in a min-heap — so the largest key must be a **leaf** (indices 7–14). **False.**

Trap: (B) vs (A) shows why bottom-up build-heap is $O(n)$ while repeated insertion can be $\Theta(n \log n)$: bottom-up sums *heights* (mostly small), insertion sums *depths* (mostly large).

Q.17 [NAT · 2 marks · DFS — counting possible visit orders]**Answer: 9**

DFS always continues from the **most recently discovered** vertex that still has an unvisited neighbour. Branch on A's first choice.

A $\rightarrow$ B: B's only unvisited neighbour is C. From C: D then E (D-E), or E then D. $\rightarrow$ A B C D E, A B C E D (2).

A $\rightarrow$ C: C's unvisited neighbours are B, D, E.

- C $\rightarrow$ B: B is a dead end; back at C choose D ($\rightarrow$ E) or E ($\rightarrow$ D): A C B D E, A C B E D.
- C $\rightarrow$ D $\rightarrow$ E (E's neighbours C, D visited) $\rightarrow$ back to C $\rightarrow$ B: A C D E B.
- C $\rightarrow$ E $\rightarrow$ D $\rightarrow$ back to C $\rightarrow$ B: A C E D B.

(4)

A $\rightarrow$ D: D's unvisited neighbours C, E.

- D $\rightarrow$ C: then C $\rightarrow$ B (dead end) $\rightarrow$ C $\rightarrow$ E: A D C B E; or C $\rightarrow$ E first, then B: A D C E B.
- D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ B: A D E C B.

(3)

Total = 2 + 4 + 3 = **9**.

Trap: orders like A, B, D, ... are impossible — after B, DFS must continue from B (to C) before backtracking to A. Not every 'A first' permutation of a connected graph is a DFS order.

Q.18 [MCQ · 2 marks · Output-restricted deque permutations]**Answer: (D)**

Key invariant: every arriving value is **larger** than everything already in the deque and is placed at an end. Hence the deque contents are always **valley-shaped** — reading front to rear, the values decrease to a minimum and then increase (removing from the front keeps this shape). A sequence starting with 4 needs 1, 2, 3 to be inside the deque, front to rear, in exactly their output order before 4 is added.

- (A) 4, 2, 1, 3: insert 1; 2 at front $\rightarrow$ [2, 1]; 3 at rear $\rightarrow$ [2, 1, 3]; 4 at front $\rightarrow$ output 4, 2, 1, 3. **Possible.**
- (B) 3, 1, 2, 4: [1], 2 at rear $\rightarrow$ [1, 2], 3 at front $\rightarrow$ [3, 1, 2]; output 3, 1, 2; then 4. **Possible.**
- (C) 1, 4, 2, 3: insert 1, output 1; [2], 3 at rear $\rightarrow$ [2, 3], 4 at front $\rightarrow$ 4, 2, 3. **Possible.**
- (D) 4, 1, 3, 2: the deque would have to read 1, 3, 2 (front to rear) — up then down, a **peak**, not a valley. It can never be built. **Impossible.**

(The only impossible output sequences of length 4 are 4, 1, 3, 2 and 4, 2, 3, 1.)

Tip: the valley invariant immediately rules out any sequence 4, a, b, c with $a < b > c$, which is why 4, 2, 3, 1 is the other impossible sequence.

Q.19 [NAT · 2 marks · Recurrences — exact operation count]

Answer: 146

Let $T(n)$ be the final increase of c caused by $f(n)$. From the code:

$$T(1) = 1, T(n) = n/2 + 3T(n/2).$$

- $T(2) = 1 + 3 \cdot 1 = 4$
- $T(4) = 2 + 3 \cdot 4 = 14$
- $T(8) = 4 + 3 \cdot 14 = 46$
- $T(16) = 8 + 3 \cdot 46 = \mathbf{146}$

Closed form check: with $n = 2^k$, $T(n) = 3^k + (n/2) \cdot \sum_{i=0}^{k-1} (3/2)^i = 3^k + 3^k - 2^k = 2 \cdot 3^k - 2^k$; for $k = 4$: $2 \cdot 81 - 16 = 146 \checkmark$. Asymptotically $T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ (master theorem, case 1).

Trap: the loop runs $n/2$ times (not n), and leaves contribute 1 each ($3^4 = 81$ leaves).

Q.20 [MSQ · 2 marks · Reversing a linked list that contains a cycle]

Answer: (A), (B), (C)

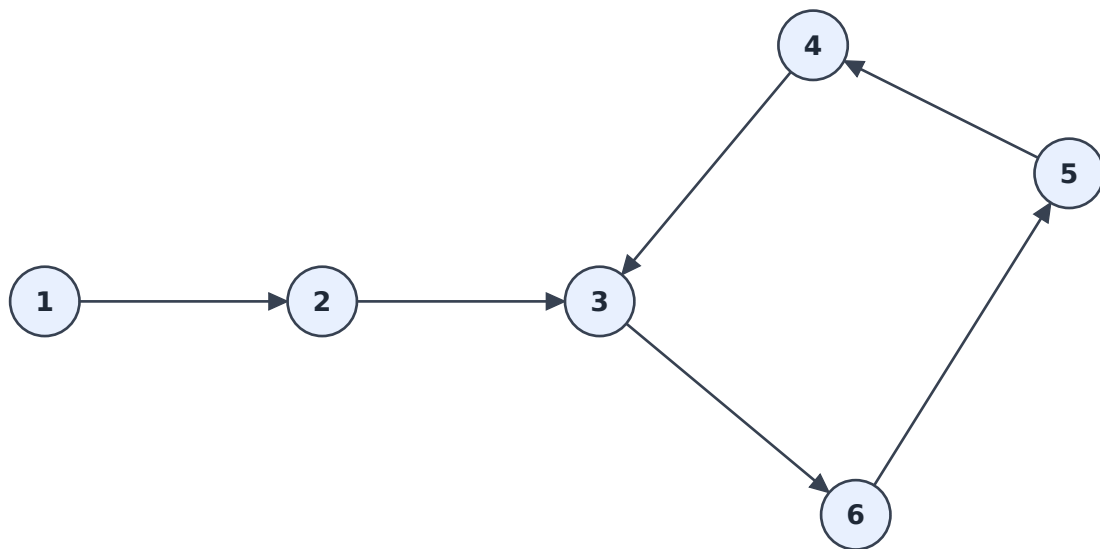
Trace (each line: node processed $\rightarrow$ its new nxt ; then prev , cur):

- 1: 1. nxt = None; $\text{cur} \rightarrow 2$
- 2: 2. nxt = 1; $\text{cur} \rightarrow 3$
- 3: 3. nxt = 2; $\text{cur} \rightarrow 4$
- 4: 4. nxt = 3; $\text{cur} \rightarrow 5$
- 5: 5. nxt = 4; $\text{cur} \rightarrow 6$
- 6: 6. nxt = 5; $\text{cur} \rightarrow \mathbf{3}$ (6's old nxt)
- 3 again: its current nxt is 2 (set in step 3), so $\text{cur} \rightarrow 2$; 3. nxt = 6
- 2 again: $\text{cur} \rightarrow 1$; 2. nxt = 3
- 1 again: $\text{cur} \rightarrow \text{None}$ (1. nxt was set to None); 1. nxt = 2. Loop ends.

9 iterations; prev = node 1. Final links: $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3$ (cycle).

- (A) **True** — reversing a ρ -shaped list returns the original head.
- (B) **True** — $9 = 2\mu + \lambda + 1$ with tail length $\mu = 2$ and cycle length $\lambda = 4$ (the tail is walked twice, the cycle once).
- (C) **True** — the walk re-enters the tail and follows the already-reversed tail links back to the head, whose nxt is None.
- (D) **False** — the tail $1 \rightarrow 2 \rightarrow 3$ is restored, but the **cycle is reversed**: $3 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3$.

Tip: comparing the returned head with the original head is a (slow) way to detect a cycle with $O(1)$ extra space.



Links after the loop: the cycle direction is reversed

Mock Test 47

Challenge Mock — Paper 2

Difficulty: Hard

Focus areas: hardest, multi-concept, trap-heavy 2-mark questions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — `+=` vs `+` on lists, identity]

Consider the following Python program. What is printed?

Python

```
1 a = [1, 2]
2 b = a
3 a += [3]
4 c = a
5 a = a + [4]
6 print(b, c is b, a is c, len(a))
```

- (A) [1, 2] False False 4
- (B) [1, 2, 3] True True 4
- (C) [1, 2, 3] True False 4
- (D) [1, 2, 3, 4] True True 4

Q.2 [NAT · numerical answer | 1 mark | Python — try/except/finally and return]

The value printed by the following Python program is ____.

Python

```
1 def f(n):
2     try:
3         if n % 2:
4             return n * 10
5         raise ValueError
6     except ValueError:
7         return n + 1
8     finally:
9         if n > 5:
10            return -n
11
12 print(sum(f(k) for k in range(8)))
```

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Input-restricted deque permutations]

An **input-restricted deque** allows insertion only at the rear but deletion at both ends. The values 1, 2, 3, 4 are inserted in this order; deletions may be interleaved with insertions, and each deleted value is output at once. Which of the following output sequences is/are possible?

- (A) 3, 1, 4, 2
- (B) 4, 2, 1, 3
- (C) 4, 1, 3, 2
- (D) 4, 2, 3, 1

Q.4 [NAT · numerical answer | 1 mark | Linear probing — expected probes]

A hash table with 10 slots (0-9) uses linear probing. Slots 1, 2, 3, 6 and 8 are occupied and the rest are empty, as shown. A new key is inserted whose home slot is uniformly distributed over 0-9. Counting every slot examined (including the one where the key is placed), the expected number of probes for this insertion is _____ (rounded off to two decimal places).

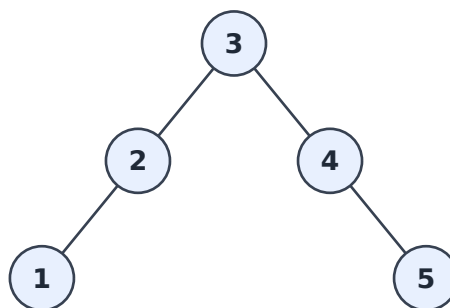
0	
1	31
2	52
3	13
4	
5	
6	96
7	
8	78
9	

Current table

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | BST — insertion orders]

How many permutations of the keys 1, 2, 3, 4, 5, when inserted in that order into an initially empty BST, produce exactly the tree shown?



Target BST

- (A) 6
- (B) 4
- (C) 8
- (D) 12

Q.6 [NAT · numerical answer | 1 mark | Binary search — worst-case iterations]

The standard iterative binary search ($lo = 0$, $hi = n - 1$; while $lo \leq hi$: $mid = \lfloor (lo + hi)/2 \rfloor$; compare; move lo or hi past mid) is run on a sorted array of n distinct keys. The **smallest** n for which some search (successful or unsuccessful) can execute the loop body 8 times is _____.

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Graph theory — edges forcing connectivity]

What is the minimum number m such that **every** simple undirected graph on 10 vertices with m edges is connected?

- (A) 9
- (B) 37
- (C) 36
- (D) 45

Q.8 [MSQ · one or more correct | 1 mark | Stability depends on the comparison]

Which of the following sorting procedures is/are **stable**?

- (A) Merge sort whose merge takes from the left run only when $L[i] \leq R[j]$
- (B) Quicksort with Lomuto partitioning (pivot = last element, $a[j] \leq \text{pivot}$ goes left)
- (C) Bubble sort that swaps adjacent elements when $A[j] \geq A[j+1]$
- (D) Insertion sort that shifts $A[j]$ right while $A[j] > \text{key}$

Q.9 [NAT · numerical answer | 1 mark | Counting heaps]

The number of distinct binary max-heaps that can be built with the six distinct keys 1, 2, 3, 4, 5, 6 is _____.

Answer: _____ (enter a numerical value)

Q.10 [MCQ · single correct | 1 mark | Python — multi-pass stable sorting]

Consider the following Python program. What is printed?

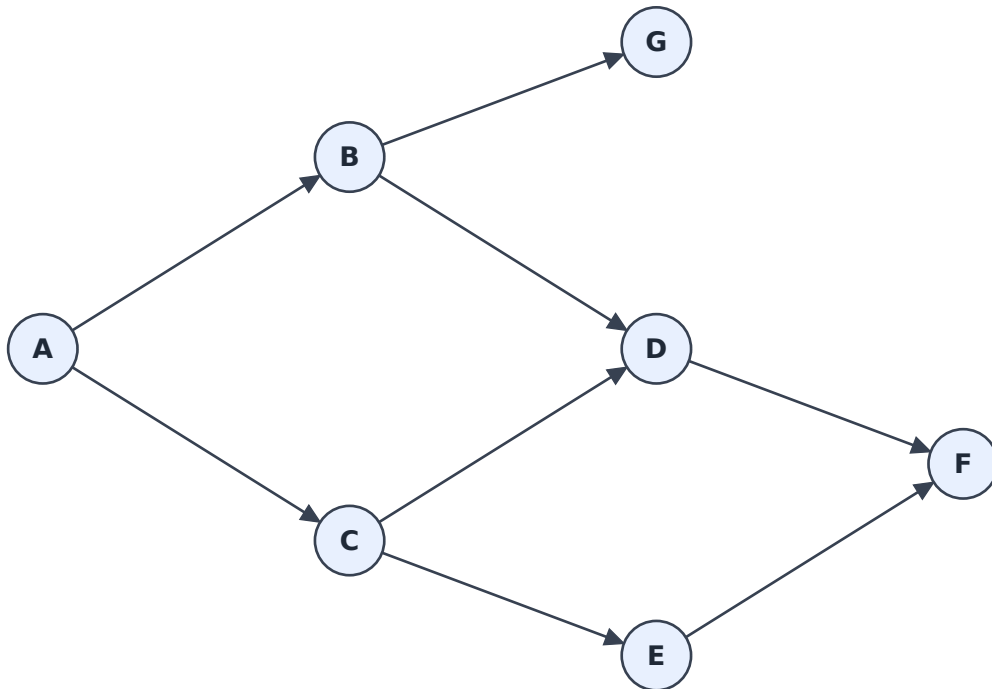
Python

```
1 data = ['b2', 'a3', 'c1', 'a1', 'b3', 'c2']
2 data.sort(key=lambda s: s[1])
3 data.sort(key=lambda s: s[0], reverse=True)
4 print(data)
```

- (A) ['c2', 'c1', 'b3', 'b2', 'a3', 'a1']
- (B) ['c1', 'c2', 'b2', 'b3', 'a1', 'a3']
- (C) ['c1', 'c2', 'b3', 'b2', 'a1', 'a3']
- (D) ['a1', 'a3', 'b2', 'b3', 'c1', 'c2']

Q.11 [NAT · numerical answer | 2 marks | Counting topological orders]

The number of distinct topological orderings of the directed acyclic graph shown is ____.



Answer: _____ (enter a numerical value)

Q.12 [MCQ · single correct | 2 marks | Generators + closures + default arguments]

Consider the following Python program. What is printed?

Python

```

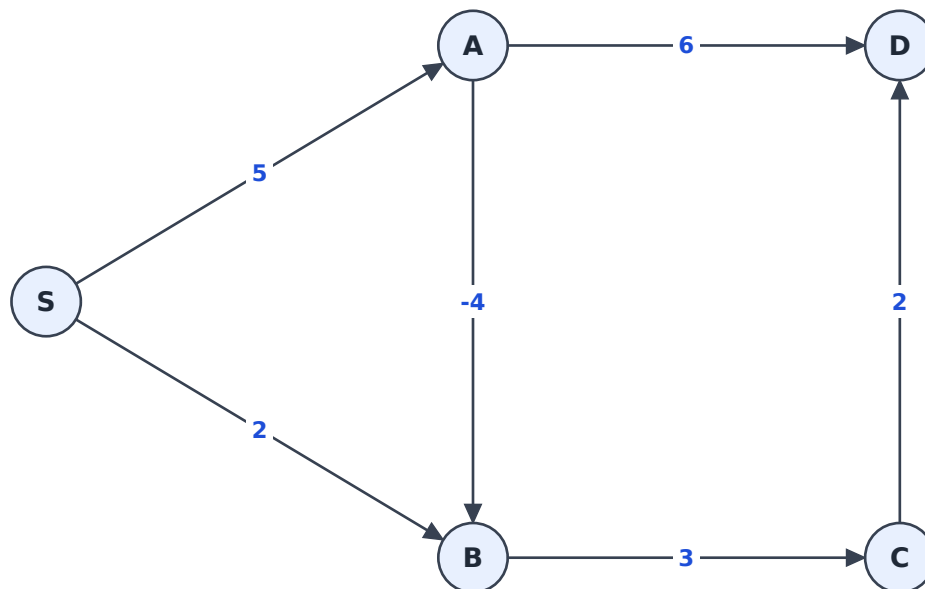
1 def gen():
2     fs = []
3     for i in range(3):
4         fs.append(lambda x, i=i: x + i)
5         fs.append(lambda x: x * i)
6         yield len(fs)
7     yield [f(2) for f in fs]
8
9 g = gen()
10 next(g)
11 print(next(g), list(g))

```

- (A) 4 [[2, 4, 3, 4, 4, 4]]
- (B) 4 [6, [2, 0, 3, 2, 4, 4]]
- (C) 2 [4, 6, [2, 4, 3, 4, 4, 4]]
- (D) 4 [6, [2, 4, 3, 4, 4, 4]]

Q.13 [MSQ · one or more correct | 2 marks | Dijkstra with a negative edge]

Dijkstra's algorithm is run from S on the directed graph shown, which has one negative edge ($A \rightarrow B$, weight -4) and no negative cycle. Implementation: repeatedly extract the non-finalized vertex with the smallest d (ties alphabetically), finalize it, and relax **all** its outgoing edges — d of an already-finalized vertex may still decrease, but a finalized vertex is never extracted again. Which of the following statements is/are TRUE?



- (A) Bellman-Ford on this graph would report a negative-weight cycle
- (B) The final value $d[B]$ equals the true shortest distance from S to B
- (C) The final value $d[C]$ is 5, whereas the true distance to C is 4
- (D) Exactly one vertex ends with an incorrect d value

Q.14 [NAT · numerical answer | 2 marks | Counting BSTs by height]

Consider all binary search trees containing exactly the keys 1, 2, ..., 7. The height of a tree is the number of edges on its longest root-to-leaf path. The number of these BSTs whose height is **exactly 3** is _____.

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Unbalanced divide-and-conquer recurrences]

Consider (with $T(n) = 1$ for $n \leq 1$)

$$T_1(n) = T_1(\lfloor n/2 \rfloor) + T_1(\lfloor n/3 \rfloor) + n$$

$$T_2(n) = T_2(\lfloor n/4 \rfloor) + T_2(\lfloor 3n/4 \rfloor) + n$$

Which one of the following is correct?

- (A) $T_1(n) = \Theta(n)$ and $T_2(n) = \Theta(n)$
- (B) $T_1(n) = \Theta(n \log n)$ and $T_2(n) = \Theta(n \log n)$
- (C) $T_1(n) = \Theta(n)$ and $T_2(n) = \Theta(n \log n)$
- (D) $T_1(n) = \Theta(n \log n)$ and $T_2(n) = \Theta(n^2)$

Q.16 [NAT · numerical answer | 2 marks | Linear probing with deletion (tombstones)]

A hash table of size 11 uses $h(k) = k \bmod 11$ with linear probing. Deletion replaces a key by a tombstone **DEL**. Insertion places the key in the first slot on its probe sequence that is empty or DEL. A search continues past DEL slots and stops at the key or at an empty slot.

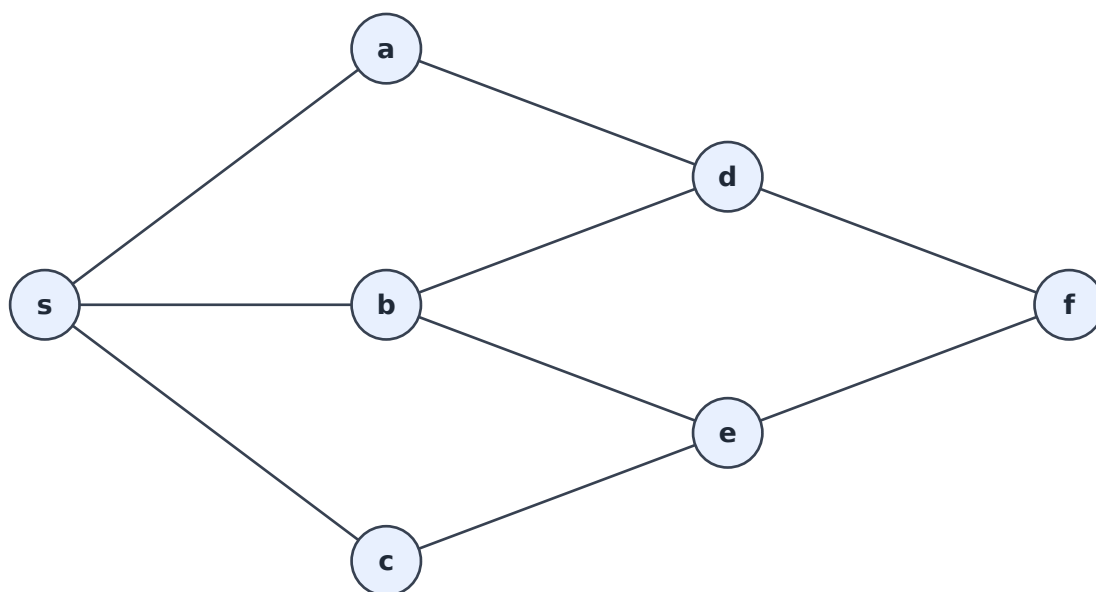
Operations: insert 22, 33, 13, 44, 24, 35; delete 33; delete 13; insert 46.

Then two unsuccessful searches are made, for 57 and for 55. Counting every slot examined (including the empty slot that ends a search), the total number of slots examined by the two searches is _____.

Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | All possible BFS orders]

BFS is run from s on the undirected graph shown. The adjacency lists may be ordered arbitrarily, so several visiting orders are possible. Which of the following statements is/are TRUE?



- (A) Exactly 8 distinct BFS visiting orders are possible
- (B) Some BFS order visits e before d
- (C) s, c, a, b, d, e, f is a possible BFS order
- (D) f is the last vertex visited in every BFS order

Q.18 [NAT · numerical answer | 2 marks | Memoization via a mutable default]

The value printed by the following Python program is _____.

Python

```

1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     f.calls += 1
5     r = n if n < 2 else f(n - 1) + f(n - 2)
6     memo[n] = r
7     return r
8
9 f.calls = 0
10 f(10)
11 a = f.calls
12 f(15)
13 b = f.calls
14 print(a * 100 + b)

```

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Linked lists — reversal in growing groups]

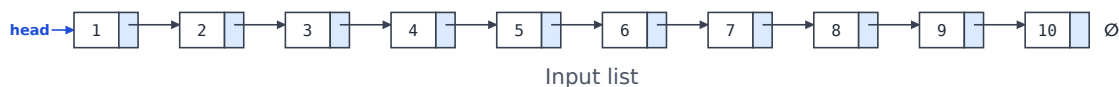
Consider the following Python program on the list $1 \rightarrow 2 \rightarrow \dots \rightarrow 10$. What is printed?

Python

```

1 class N:
2     def __init__(self, v, nxt=None):
3         self.v, self.next = v, nxt
4
5 def rev(h, k):
6     cur, prev, cnt = h, None, 0
7     while cur and cnt < k:
8         cur.next, prev, cur = prev, cur, cur.next
9         cnt += 1
10    if cur:
11        h.next = rev(cur, k + 1)
12    return prev
13
14 head = None
15 for v in range(10, 0, -1):
16     head = N(v, head)
17 head = rev(head, 2)
18 out = []
19 while head:
20     out.append(head.v)
21     head = head.next
22 print(*out)

```



- (A) 2 1 5 4 3 9 8 7 6 10
- (B) 2 1 4 3 6 5 8 7 10 9
- (C) 2 1 5 4 3 9 8 7 6
- (D) 2 1 5 4 3 10 9 8 7 6

Q.20 [MSQ · one or more correct | 2 marks | Heap facts and heapsort]

Which of the following statements about binary heaps with n distinct keys is/are TRUE?

- (A) Heapsort is a stable sorting algorithm
- (B) For $n \geq 7$, the third-largest key of a max-heap can occupy exactly 6 different array positions
- (C) The k -th smallest key of a min-heap can be found in $O(k \log k)$ time without modifying the heap
- (D) Two binary heaps of n keys each can be merged into one binary heap in $O(n)$ time

Answer Key — Mock Test 47

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — += vs + on lists, identity
2	NAT	1	86	Python — try/except/finally and return
3	MSQ	1	(A), (C)	Input-restricted deque permutations
4	NAT	1	1.79 to 1.81	Linear probing — expected probes
5	MCQ	1	(A)	BST — insertion orders
6	NAT	1	128	Binary search — worst-case iterations
7	MCQ	1	(B)	Graph theory — edges forcing connectivity
8	MSQ	1	(A), (D)	Stability depends on the comparison
9	NAT	1	20	Counting heaps
10	MCQ	1	(B)	Python — multi-pass stable sorting
11	NAT	2	21	Counting topological orders
12	MCQ	2	(D)	Generators + closures + default arguments
13	MSQ	2	(B), (C)	Dijkstra with a negative edge
14	NAT	2	68	Counting BSTs by height
15	MCQ	2	(C)	Unbalanced divide-and-conquer recurrences
16	NAT	2	12	Linear probing with deletion (tombstones)
17	MSQ	2	(A), (B), (D)	All possible BFS orders
18	NAT	2	1116	Memoization via a mutable default
19	MCQ	2	(A)	Linked lists — reversal in growing groups
20	MSQ	2	(B), (C), (D)	Heap facts and heapsort

Detailed Solutions — Mock Test 47

Q.1 [MCQ · 1 mark · Python — `+=` vs `+` on lists, identity]

Answer: (C)

For lists, `a += x` calls `list.__iadd__`, which **extends the same object in place**; `a = a + x` builds a **new** list and rebinds `a` to it.

- `b = a`: `b` and `a` name the same list `[1, 2]`.
- `a += [3]`: that shared list becomes `[1, 2, 3]` — `b` sees it.
- `c = a`: `c` is the same object as `b`.
- `a = a + [4]`: `a` now names a new list `[1, 2, 3, 4]`; `b` and `c` still name the old one.

So `b = [1, 2, 3]`, `c is b` → `True`, `a is c` → `False`, `len(a) = 4`.

- (A) treats `+=` as creating a new list (true for tuples/strings, not lists).
- (B) treats `a = a + [4]` as in-place.
- (D) treats both operations as in-place.

Trap: `x += y` and `x = x + y` are equivalent only for immutable types.

Q.2 [NAT · 1 mark · Python — try/except/finally and return]

Answer: 86

The **finally** block runs on every exit path. If it executes its own **return**, that value **replaces** whatever the **try** or **except** block was returning.

- `n = 0`: `even` → `ValueError` → `except` returns `1`; `finally` (`0 > 5` false) → **1**
- `n = 1`: `odd` → `return 10` → **10**
- `n = 2`: → **3**
- `n = 3`: → **30**
- `n = 4`: → **5**
- `n = 5`: → **50**
- `n = 6`: `except` would return `7`, but `finally` returns **-6**
- `n = 7`: `try` would return `70`, but `finally` returns **-7**

Sum = `1 + 10 + 3 + 30 + 5 + 50 - 6 - 7 = 86`.

Trap: assuming **return** ends the function immediately. With `n > 5` the pending return value (`7` or `70`) is discarded; ignoring **finally** would give `176`.

Q.3 [MSQ · 1 mark · Input-restricted deque permutations]

Answer: (A), (C)

When the first output is `4`, all of `1, 2, 3, 4` have already been inserted, in order, so the deque is `[1, 2, 3, 4]` and afterwards only the two **ends** can be removed.

- (A) Insert `1, 2, 3` → `[1,2,3]`; delete rear `3`; delete front `1` → `[2]`; insert `4` → `[2,4]`; delete rear `4`; delete `2`. **Possible.** (This sequence is *impossible* for a stack — the deque's extra front deletion makes it reachable.)
- (B) After `4` the deque is `[1, 2, 3]`; `2` is in the middle — neither end. **Impossible.**
- (C) `[1,2,3,4]`: delete rear `4` → `[1,2,3]`; delete front `1` → `[2,3]`; delete rear `3`; then `2`. **Possible.**
- (D) Again `2` would have to leave from the middle of `[1, 2, 3]`. **Impossible.**

In fact, of the `24` permutations of `1..4`, exactly these two (`4,2,1,3` and `4,2,3,1`) cannot be produced by an input-restricted deque.

Trap: applying the stack rule (forbidden `3-1-2` pattern) — it rejects (A) wrongly.

Q.4 [NAT · 1 mark · Linear probing — expected probes]**Answer: 1.79 to 1.81**

With linear probing, a key whose home slot lies inside a run of occupied slots must walk to the end of the run. Probes needed for each home slot h :

- $h = 0$: empty $\rightarrow 1$
- $h = 1$: 1, 2, 3 full, 4 empty $\rightarrow 4$
- $h = 2$: $\rightarrow 3$
- $h = 3$: $\rightarrow 2$
- $h = 4$: $\rightarrow 1$
- $h = 5$: $\rightarrow 1$
- $h = 6$: 6 full, 7 empty $\rightarrow 2$
- $h = 7$: $\rightarrow 1$
- $h = 8$: 8 full, 9 empty $\rightarrow 2$
- $h = 9$: $\rightarrow 1$

Total = $1 + 4 + 3 + 2 + 1 + 1 + 2 + 1 + 2 + 1 = 18$; expected = $18/10 = \mathbf{1.80}$.

Notice also that slot 4 receives the key with probability $4/10$ (home slots 1–4) while slot 5 gets only $1/10$ — long runs grow faster (primary clustering).

Trap: using the load-factor formula $\frac{1}{2}(1 + 1/(1 - \alpha)^2) = \frac{1}{2}(1 + 4) = 2.5$ — that is an asymptotic average over *random* tables, not this specific table.

Q.5 [MCQ · 1 mark · BST — insertion orders]**Answer: (A)**

A permutation produces this tree iff every node is inserted **before all its descendants**. Here that means: 3 first; 2 before 1; 4 before 5. Keys in different subtrees of 3 can be interleaved freely.

- The root 3 must come first.
- The left subtree sequence is forced: (2, 1). The right subtree sequence is forced: (4, 5).
- Interleavings of two fixed sequences of length 2 and 2: $C(4, 2) = \mathbf{6}$.

They are: 3 2 1 4 5, 3 2 4 1 5, 3 2 4 5 1, 3 4 2 1 5, 3 4 2 5 1, 3 4 5 2 1.

General rule: $\text{ways}(T) = C(|L| + |R|, |L|) \cdot \text{ways}(L) \cdot \text{ways}(R)$.

- (B) 4 forgets some interleavings. (C) 8 and (D) 12 allow 1 before 2 or 5 before 4, which would change the shape.

Trap: thinking the order must be level-order; any topological order of the tree (parents before children) works.

Q.6 [NAT · 1 mark · Binary search — worst-case iterations]**Answer: 128**

The loop examines the nodes of an implicit, height-balanced decision tree on n keys. Its worst case equals the number of levels, $\lfloor \log_2 n \rfloor + 1$, and an unsuccessful search never takes more iterations than that (it ends just below a node on the deepest level).

We need $\lfloor \log_2 n \rfloor + 1 \geq 8$, i.e. $\lfloor \log_2 n \rfloor \geq 7$, i.e. $n \geq 2^7 = \mathbf{128}$.

Check: with $n = 127$ the tree is perfect with 7 levels ($2^7 - 1$ nodes), so every search ends within 7 iterations. Adding one key ($n = 128$) creates an 8th level containing one node.

Trap: answering 255 or 256 (confusing “at most 8” with “at least 8”), or 127 (the largest n for which 7 iterations always suffice).

Q.7 [MCQ · 1 mark · Graph theory — edges forcing connectivity]**Answer: (B)**

Find the largest number of edges a **disconnected** simple graph on 10 vertices can have; m is one more than that.

A disconnected graph splits into parts of sizes k and $10 - k$ with no edges between them, so it has at most $C(k, 2) + C(10 - k, 2)$ edges, maximised at the extreme split $k = 1$: an isolated vertex plus $K_9 \rightarrow C(9, 2) = 36$ edges.

So 36 edges do not guarantee connectivity (K_9 + isolated vertex), but any graph with **37** edges must be connected.

- (A) $9 = n - 1$ is the minimum number of edges in *some* connected graph (a tree), not a guarantee.
- (C) 36 is the largest disconnected case.
- (D) $45 = C(10, 2)$ is the complete graph.

Trap: confusing “can be connected” ($n - 1$ edges) with “must be connected” ($C(n - 1, 2) + 1$ edges).

Q.8 [MSQ · 1 mark · Stability depends on the comparison]**Answer: (A), (D)**

A sort is stable if records with equal keys keep their input order. Whether an implementation is stable often hinges on a single $<$ versus $<=$.

- (A) **Stable.** On ties the merge takes from the **left** run, which holds the earlier records. (With strict $<$ it would take from the right on ties and be unstable.)
- (B) **Not stable.** The final pivot swap is a long-range exchange. Input $[(1,a), (1,b), (0,c)]$: pivot key 0, nothing goes left, and the pivot swap exchanges positions 0 and 2 $\rightarrow [(0,c), (1,b), (1,a)]$. The recursive call on $[(1,b), (1,a)]$ leaves it unchanged, so the 1-records come out as b, a — reversed.
- (C) **Not stable.** With $>=$, two adjacent equal keys are swapped, reversing their order; e.g. $(1,'x'), (1,'y')$ becomes $(1,'y'), (1,'x')$.
- (D) **Stable.** The key stops at the first element $\leq$ it, so it never jumps over an equal key.

Trap: classifying algorithms as stable/unstable without looking at the exact comparison.

Q.9 [NAT · 1 mark · Counting heaps]**Answer: 20**

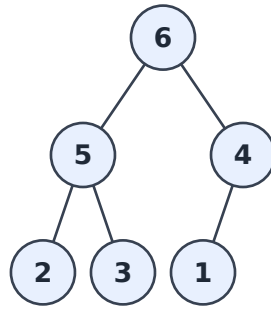
The shape of a 6-node heap is fixed (complete binary tree): the root's left subtree has 3 nodes (a root with two children) and the right subtree has 2 nodes (a root with one left child).

- The root must be 6.
- Choose which 3 of the remaining 5 keys go to the left subtree: $C(5, 3) = 10$. The other 2 go right.
- Left subtree (3 nodes): its largest key is its root; the other two can be placed in 2 ways $\rightarrow 2$.
- Right subtree (2 nodes): the larger is the parent, the smaller the child $\rightarrow 1$ way.

Total = $10 \times 2 \times 1 = \mathbf{20}$.

Recurrence: $H(n) = C(n - 1, L) \cdot H(L) \cdot H(R)$, where L, R are the subtree sizes of the complete tree. (For comparison $H(7) = C(6, 3) \cdot 2 \cdot 2 = 80$.)

Trap: assuming both subtrees have the same size (as for 7 nodes) or that the 2-node subtree allows 2 arrangements — the single child position is forced.



One of the 20 heaps (left subtree {5,2,3}, right {4,1})

Q.10 [MCQ · 1 mark · Python — multi-pass stable sorting]

Answer: (B)

Python's sort is **stable**, and `reverse=True` keeps stability: elements with equal keys stay in their original relative order (the sort does *not* simply reverse the stable ascending result).

- Pass 1 (by digit): 1s: c1, a1; 2s: b2, c2; 3s: a3, b3 → ['c1', 'a1', 'b2', 'c2', 'a3', 'b3']
- Pass 2 (by letter, descending): c-group in current order c1, c2; b-group b2, b3; a-group a1, a3 → ['c1', 'c2', 'b2', 'b3', 'a1', 'a3']

Net effect: letters descending, digits ascending within a letter — the classic “sort by secondary key first, then by primary key” idiom.

- (A) assumes `reverse=True` reverses ties too (it would if you sorted ascending and then called `reverse()`).
- (C) mixes the two behaviours.
- (D) ignores `reverse=True`.

Trap: `sorted(x, reverse=True) ≠ sorted(x)[::-1]` when there are ties.

Q.11 [NAT · 2 marks · Counting topological orders]

Answer: 21

Strategy: fix the forced parts, count the core, then insert the “loose” vertex G.

Forced: A is the only source → first. F must follow D and E, and D, E follow B and C (D after B, C; E after C), so among {B, C, D, E, F}, F is last.

Core {B, C, D, E} with $B \rightarrow D$, $C \rightarrow D$, $C \rightarrow E$:

- B first: then C must precede D and E → B C D E, B C E D (2)
- C first: remaining B, D, E with B before D → C B D E, C B E D, C E B D (3)

So 5 orders of B, C, D, E (each followed by F).

Insert G (only constraint: after B) into the 6 gaps of each 5-element sequence $X_1 \dots X_5$ — the gaps after B number 6 — $\text{pos}(B)$:

- B C D E F: pos 1 → 5
- B C E D F: → 5
- C B D E F: pos 2 → 4
- C B E D F: → 4
- C E B D F: pos 3 → 3

Total = $5 + 5 + 4 + 4 + 3 = 21$.

Trap: multiplying independent-looking counts (e.g. $5 \times 6 = 30$) — G's options depend on where B sits in each order.

Q.12 [MCQ · 2 marks · Generators + closures + default arguments]**Answer: (D)**

Two independent traps: (1) a generator resumes where it paused, and `list(g)` collects **all remaining** yields; (2) `i=i` freezes the current value in a default argument, while the plain lambda looks `i` up when called (late binding).

Generator progress:

- first `next(g)`: `i = 0`, two lambdas appended, yields 2 (discarded)
- second `next(g)`: `i = 1`, yields **4** (printed first)
- `list(g)`: `i = 2`, yields 6; loop ends; yields the list of results; then stops $\rightarrow [6, [\dots]]$

The list is computed after the loop, when `i = 2`:

- `i=0` pair: $2 + 0 = 2$, and $2 \times i = 2 \times 2 = 4$
- `i=1` pair: $2 + 1 = 3$, and 4
- `i=2` pair: $2 + 2 = 4$, and 4

 $\rightarrow [2, 4, 3, 4, 4, 4]$. Output: **4** `[6, [2, 4, 3, 4, 4, 4]]`.

- (A) forgets the third `yield len(fs)`.
- (B) evaluates the plain lambdas with the `i` of their creation (no late binding).
- (C) prints the discarded first yield as well.

Tip: the generator's frame (and its `i`) stays alive until exhausted, which is why the plain lambdas see `i = 2`.

Q.13 [MSQ · 2 marks · Dijkstra with a negative edge]**Answer: (B), (C)****Dijkstra trace:**

- Extract S (0): $d[A] = 5$, $d[B] = 2$.
- Extract B (2): $d[C] = 5$.
- Extract A (5; tie with C, A first): $d[B] = \min(2, 5 - 4) = \mathbf{1}$ (B is already finalized — its value changes but it is never re-processed); $d[D] = 11$.
- Extract C (5): $d[D] = \min(11, 7) = 7$.
- Extract D (7).

Final: S 0, A 5, B 1, C 5, D 7.

True distances (Bellman-Ford): S 0, A 5, B = $5 - 4 = 1$, C = $1 + 3 = 4$, D = $4 + 2 = 6$.

- (A) **False** — the graph has no directed cycle at all, so there is no negative cycle; Bellman-Ford simply returns the true distances.
- (B) **True** — $d[B] = 1$ is correct, but only because the late relaxation from A updated it.
- (C) **True** — C was relaxed from B while $d[B]$ was still 2, and the improvement of B never propagated: $d[C] = 5$ vs 4.
- (D) **False** — both C (5 vs 4) and D (7 vs 6) are wrong.

Trap: believing Dijkstra fails only at the endpoint of the negative edge — the damage is in everything *downstream* of it.

Q.14 [NAT · 2 marks · Counting BSTs by height]**Answer: 68**

Let $N(n, h)$ = number of BSTs on n keys with height $\leq h$ (empty tree has height -1 , so $N(0, h) = 1$ for $h \geq -1$). Choosing root i splits the keys into $i - 1$ and $n - i$:

$$N(n, h) = \sum_{i=1}^n N(i-1, h-1) \cdot N(n-i, h-1).$$

$$\text{Answer} = N(7, 3) - N(7, 2).$$

Height ≤ 2 (at most 7 nodes in 3 levels), for $n = 0..7$:

$N(n, 2) = 1, 1, 2, 5, 6, 6, 4, 1$.

(e.g. $n = 4$: of the 14 BSTs, the 8 “paths” have height 3, leaving 6; $n = 7$: only the perfect tree.)

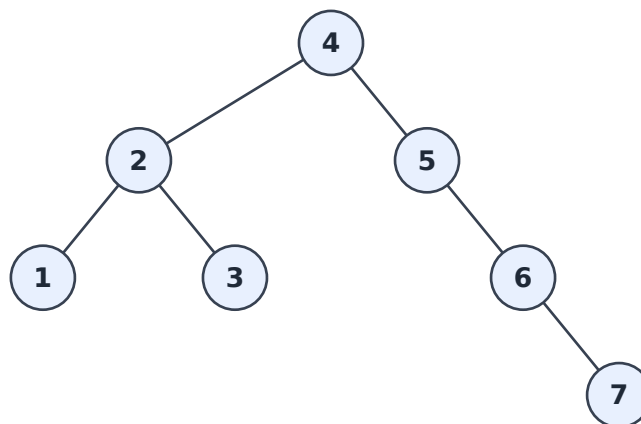
Height ≤ 3 , $n = 7$: sum over root position, left size $L = 0..6$, right size $6 - L$:

- $L = 0, 6$: $1 \times 4 = 4$ each
- $L = 1, 5$: $1 \times 6 = 6$ each
- $L = 2, 4$: $2 \times 6 = 12$ each
- $L = 3$: $5 \times 5 = 25$

$N(7, 3) = 4 + 6 + 12 + 25 + 12 + 6 + 4 = 69$.

Since $N(7, 2) = 1$ (the perfect tree), the count with height exactly 3 is $69 - 1 = \mathbf{68}$.

Trap: forgetting to subtract the height-2 tree, or using node-count height (where “height 3” would mean something else).



One BST of height 3

Q.15 [MCQ · 2 marks · Unbalanced divide-and-conquer recurrences]

Answer: (C)

Use the recursion tree: the work at a level is n times the sum of the fractions raised to the level number.

T₁: the subproblem sizes $n/2 + n/3 = (5/6)n$. Level i costs at most $(5/6)^i \cdot n$, a geometric series: total $\leq n \cdot 1/(1 - 5/6) = 6n$. With the root alone costing n , $T_1(n) = \mathbf{\Theta(n)}$.

T₂: $n/4 + 3n/4 = n$, so every full level costs exactly n . The shortest root-to-leaf path (always taking $n/4$) has $\log_4 n$ levels and the longest $\log_{4/3} n$ levels — both $\Theta(\log n)$. Hence $T_2(n) = \mathbf{\Theta(n \log n)}$ (like a merge sort with a 1:3 split).

Numerical check: $T_1(n)/n$ rises towards the bound 6 (5.1 at $n = 10^4$, 5.7 at $n = 10^6$), while $T_2(n)/(n \log_2 n)$ settles near a constant (≈ 1.2) and $T_2(n)/n$ keeps growing.

- (A) treats T_2 like T_1 — but its fractions sum to exactly 1.
- (B) treats T_1 like a balanced split — but the fractions sum to less than 1.
- (D) confuses an unbalanced *constant-fraction* split with quicksort's worst case $T(n) = T(n - 1) + n$.

Tip: for $T(n) = \sum T(\alpha_i n) + n$: $\sum \alpha_i < 1 \rightarrow \Theta(n)$; $= 1 \rightarrow \Theta(n \log n)$; $> 1 \rightarrow$ polynomially larger.

Q.16 [NAT · 2 marks · Linear probing with deletion (tombstones)]

Answer: 12

Home slots: $22 \rightarrow 0$, $33 \rightarrow 0$, $13 \rightarrow 2$, $44 \rightarrow 0$, $24 \rightarrow 2$, $35 \rightarrow 2$, $46 \rightarrow 2$, $57 \rightarrow 2$, $55 \rightarrow 0$.

Insertions:

- $22 \rightarrow 0$; $33 \rightarrow 0$ full $\rightarrow 1$; $13 \rightarrow 2$; $44 \rightarrow 0,1,2$ full $\rightarrow 3$; $24 \rightarrow 2,3$ full $\rightarrow 4$; $35 \rightarrow 2,3,4$ full $\rightarrow 5$.

- Table: [22, 33, 13, 44, 24, 35, -, -, -, -, -]

Deletions: 33 (slot 1) and 13 (slot 2) become DEL.

Insert 46 (home 2): slot 2 is DEL → 46 goes to slot 2. Table: [22, DEL, 46, 44, 24, 35, -, ...].

Search 57 (home 2): slots 2 (46), 3 (44), 4 (24), 5 (35), 6 (empty) → **5** examined.

Search 55 (home 0): 0 (22), 1 (DEL — continue), 2, 3, 4, 5, 6 (empty) → **7** examined.

Total = 5 + 7 = **12**.

Trap: stopping a search at a DEL slot (which would wrongly report keys such as 44 as absent), or treating deletion as emptying the slot — that would break the probe chain of 44, 24 and 35. Tombstones keep searches correct but make them longer, which is why tables are periodically rebuilt.

0	22
1	DEL
2	46
3	44
4	24
5	35
6	
7	
8	
9	
10	

Table before the two searches

Q.17 [MSQ · 2 marks · All possible BFS orders]

Answer: (A), (B), (D)

Levels are fixed: {s}, {a, b, c}, {d, e}, {f}. Within these, BFS order is constrained: the level-2 vertices are visited in the order they were **discovered**, which is determined by the level-1 order (d is discovered by a or b, e by b or c).

Enumerate the 6 orders of {a, b, c} and see who discovers d, e first:

- a b c → a finds d first → d, e (b's list order is irrelevant: d already found)
- a c b → a finds d → d, e
- b a c, b c a → b finds both; its adjacency order decides → d, e **or** e, d (2 each)
- c a b, c b a → c finds e first → e, d

Total orders = 1 + 1 + 2 + 2 + 1 + 1 = **8**.

- (A) **True**.
- (B) **True** — e.g. s, c, a, b, e, d, f.
- (C) **False** — if c is the first level-1 vertex, it discovers e before anyone discovers d, so e must precede d.
- (D) **True** — f is the only vertex at distance 3.

Trap: assuming any permutation within each level is achievable (that would give $3! \times 2! = 12$).

Q.18 [NAT · 2 marks · Memoization via a mutable default]**Answer: 1116**

The default `memo={}` is created **once** when `def` runs, so it persists across top-level calls — here that is exploited as a cache. `f.calls` counts only cache misses (calls that actually compute).

f(10): `f(10) → f(9) → ... → f(1); f(0)` is reached via `f(2)`'s second call. Every `n` from 0 to 10 is computed exactly once (later requests hit the memo) → 11 misses. `a = 11`.

f(15): values 0..10 are already cached; 15, 14, 13, 12, 11 are computed → 5 more misses. `b = 16`.

Printed: $11 \times 100 + 16 = \mathbf{1116}$.

Without memoisation `f(10)` alone would make 177 calls ($2 \cdot F(11) - 1$ with $F(11) = 89$); memoisation reduces this to linear.

Trap: expecting the memo to reset between calls (then `f(15)` would add 16 more misses, `b = 27`, printing 1127), or counting cache hits as calls.

Q.19 [MCQ · 2 marks · Linked lists — reversal in growing groups]**Answer: (A)**

`rev(h, k)` reverses the first `k` nodes, then recursively processes the rest with group size `k + 1`, and links the old first node `h` (now the group's tail) to the result.

The tuple assignment `cur.next, prev, cur = prev, cur, cur.next` evaluates the whole right side first (old `prev`, old `cur`, old `cur.next`) and then assigns left to right — `cur.next` is set on the *old* `cur` before `cur` is rebound, so it is a correct in-place reversal step.

- `k = 2`: group [1, 2] → 2 1; recurse from 3
- `k = 3`: group [3, 4, 5] → 5 4 3; recurse from 6
- `k = 4`: group [6, 7, 8, 9] → 9 8 7 6; recurse from 10
- `k = 5`: only [10] remains → loop stops after 1 node (`cur` becomes None) → 10

Joined: 2 1 5 4 3 9 8 7 6 10.

- (B) uses a constant group size 2.
- (C) loses the final short group (as if `if cur` were missing the last link).
- (D) treats the last group as 6..10 (sizes 2, 3, 5).

Trap: writing the assignment as `cur, cur.next, prev = cur.next, prev, cur` would set `.next` on the *new* `cur` and corrupt the list — order of targets matters.

Q.20 [MSQ · 2 marks · Heap facts and heapsort]**Answer: (B), (C), (D)**

- (A) **False.** E.g. records (1,x), (1,y): build-heap leaves them, then the first extraction swaps the root (1,x) to the end, giving the order y, x. Long-range swaps destroy stability.
- (B) **True.** The third-largest has exactly two larger keys, so it has at most two ancestors: depth 1 or 2 → array positions 2..7 (1-indexed), six positions. Each is achievable: at depth 1 with the second-largest as its sibling, at depth 2 with the second-largest as its parent.
- (C) **True.** Keep an auxiliary min-heap of *candidates*, starting with the root. Pop the smallest candidate and push its (at most two) children; the `k`-th pop is the `k`-th smallest. The auxiliary heap never exceeds `k + 1` entries, so the cost is $O(k \log k)$, independent of `n`.
- (D) **True.** Concatenate the two arrays ($O(n)$) and run bottom-up build-heap on the $2n$ keys, which is $O(n)$.

Trap: in (B), confusing *positions* (6) with *depths* (2); in (C), thinking the heap order forces the `k`-th smallest to be at depth $\leq \log k$ (it can be at depth `k - 1`).

Mock Test 48

Challenge Mock — Paper 3

Difficulty: Hard

Focus areas: hardest, multi-concept, trap-heavy 2-mark questions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — chained comparisons]

Consider the following Python program. What is printed?

Python

```
1 print(1 < 3 > 2 == 2 < 5, (1 < 3) > 2, [] == [] is not [])
```

- (A) True False False
- (B) True True True
- (C) False False True
- (D) True False True

Q.2 [NAT · numerical answer | 1 mark | Python — generators and iterator exhaustion]

Consider the following Python program. The value printed is _____.

Python

```
1 g = (x * x for x in range(6))
2 s1 = sum(x for x in g if x % 2)
3 s2 = sum(g)
4 it = iter(range(11))
5 pairs = list(zip(it, it))
6 rest = sum(it)
7 print(s1 + s2 + len(pairs) + rest)
```

Answer: _____ (enter a numerical value)

Q.3 [MSQ · one or more correct | 1 mark | Stacks — monotonic stack (next greater element)]

The function below is called on $a = [6, 2, 5, 1, 4, 8, 3, 7]$ (shown). Which of the following statements is/are TRUE?

Python

```
1 def nge(a):
2     res, st, pops = [-1] * len(a), [], 0
3     for i, x in enumerate(a):
4         while st and a[st[-1]] < x:
5             res[st.pop()] = x
6             pops += 1
7         st.append(i)
8     return res, pops, st
```

a =	6	2	5	1	4	8	3	7
	0	1	2	3	4	5	6	7

- (A) At some moment the stack holds 4 indices
- (B) `pops == 6`
- (C) When the function returns, the stack holds the indices of the values 8 and 7 (bottom to top)
- (D) `res == [8, 5, 8, 4, 8, -1, 7, -1]`

Q.4 [NAT · numerical answer | 1 mark | Deques — sliding-window maximum]

The program computes the maximum of every window of size 3 using a deque of indices. The value printed (the number of removals from the **back** of the deque) is _____.

Python

```
1 from collections import deque
2
3 a = [4, 2, 12, 3, 8, 6, 1, 9, 5, 7]
4 k, dq, back, out = 3, deque(), 0, []
5 for i, x in enumerate(a):
6     while dq and a[dq[-1]] <= x:
7         dq.pop()
8         back += 1
9     dq.append(i)
10    if dq[0] <= i - k:
11        dq.popleft()
12    if i >= k - 1:
13        out.append(a[dq[0]])
14 print(back)
```

Answer: _____ (enter a numerical value)

Q.5 [MCQ · single correct | 1 mark | Linked lists — split, reverse and interleave]

reorder is applied to the list $1 \rightarrow 2 \rightarrow \dots \rightarrow 7$ (shown). Reading the returned list from its head, the sequence of values is:

Python

```

1 def reorder(h):
2     slow, fast = h, h.nxt
3     while fast and fast.nxt:
4         slow, fast = slow.nxt, fast.nxt.nxt
5         second, slow.nxt = slow.nxt, None
6         prev = None
7         while second:
8             second.nxt, prev, second = prev, second, second.nxt
9         first = h
10        while prev:
11            a, b = first.nxt, prev.nxt
12            first.nxt, prev.nxt = prev, a
13            first, prev = a, b
14    return h

```



- (A) [1, 6, 2, 5, 3, 4, 7]
 (B) [1, 7, 2, 6, 3, 5]
 (C) [1, 5, 2, 6, 3, 7, 4]
 (D) [1, 7, 2, 6, 3, 5, 4]

Q.6 [NAT · numerical answer | 1 mark | Heaps — where can the k-th smallest be?]

A binary min-heap stores 1023 distinct keys in an array $A[0..1022]$ (children of i at $2i+1$ and $2i+2$). Considering all possible such heaps, the number of different indices at which the **4th smallest** key can appear is _____.

Answer: _____ (enter a numerical value)

Q.7 [MCQ · single correct | 1 mark | Python dicts — insertion order and deletion]

Consider the following Python program. What is printed?

Python

```

1 d = {}
2 for k in "banana":
3     d[k] = d.get(k, 0) + 1
4 del d['b']
5 d['b'] = 9
6 d['a'] += 1
7 print(list(d.items()))

```

- (A) [('n', 2), ('b', 9), ('a', 4)]
 (B) [('b', 9), ('a', 4), ('n', 2)]
 (C) [('a', 4), ('n', 2), ('b', 9)]
 (D) [('a', 3), ('n', 2), ('b', 9)]

Q.8 [MCQ · single correct | 1 mark | Binary search on the answer]

The program finds the smallest integer speed v such that piles of sizes P can be finished within H hours, eating at most v units per hour from one pile (time for a pile p is $\lceil p / v \rceil$). What is printed?

Python

```
1 import math
2 P, H = [30, 11, 23, 4, 20], 6
3 lo, hi, it = 1, max(P), 0
4 while lo < hi:
5     mid = (lo + hi) // 2
6     it += 1
7     if sum(math.ceil(p / mid) for p in P) <= H:
8         hi = mid
9     else:
10        lo = mid + 1
11 print(lo, it)
```

- (A) 23 5
- (B) 22 5
- (C) 23 4
- (D) 30 5

Q.9 [MCQ · single correct | 1 mark | Insertion sort — exact comparison count]

Insertion sort (ascending) is applied to $[5, 1, 4, 2, 3]$. Each evaluation of $a[j] > \text{key}$ counts as one comparison; the inner loop also stops (without a comparison) when j becomes -1 . The total number of comparisons is:

- (A) 6
- (B) 9
- (C) 10
- (D) 7

Q.10 [MSQ · one or more correct | 1 mark | Graph theory — cycles, degrees, connectivity]

Which of the following statements about simple undirected graphs is/are TRUE?

- (A) If every vertex has degree at least 2, the graph is connected
- (B) A connected graph with n vertices and n edges contains exactly one cycle
- (C) Every graph with $n \geq 2$ vertices has two vertices of equal degree
- (D) A graph with 10 vertices, 12 edges and exactly 2 connected components contains at least 4 distinct cycles

Q.11 [MCQ · single correct | 2 marks | Python — closures, class attributes, try/finally]

Consider the following Python program. What is printed?

Python

```

1 class Counter:
2     total = 0
3     def __init__(self):
4         self.fns = []
5         for i in range(3):
6             def f(x, i=i):
7                 Counter.total += x * i
8                 return Counter.total
9             self.fns.append(f)
10    def run(self, vals):
11        try:
12            for f, v in zip(self.fns, vals):
13                if v < 0:
14                    raise ValueError(v)
15                f(v)
16            return Counter.total
17        except ValueError as e:
18            return -e.args[0]
19        finally:
20            Counter.total += 100
21
22 c = Counter()
23 print(c.run([1, 2, 3]), c.run([4, -5, 6]), Counter.total)

```

- (A) 108 5 208
- (B) 8 5 208
- (C) 108 208 208
- (D) 8 -5 208

Q.12 [NAT · numerical answer | 2 marks | Recursion — memoisation via a mutable default]

Consider the following Python program. The value printed is _____.

Python

```

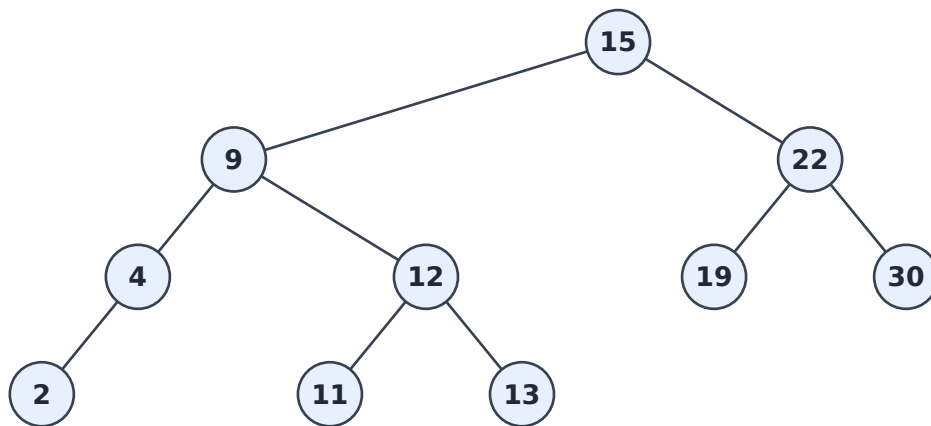
1 calls = 0
2
3 def f(n, memo={}):
4     global calls
5     calls += 1
6     if n in memo:
7         return memo[n]
8     r = 1 if n < 3 else f(n - 1) + f(n - 3)
9     memo[n] = r
10    return r
11
12 f(10)
13 f(12)
14 print(calls)

```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | BST vs binary heap built from the same keys]

The keys 15, 9, 22, 4, 12, 19, 30, 11, 13, 2 are inserted, in that order, (i) into an empty BST T (shown) and (ii) into an empty binary **min**-heap H stored as an array (0-based), using the standard sift-up insertion. Depth of the root is 0. Which of the following statements is/are TRUE?

Figure: BST T

- (A) Exactly two keys have the same depth in T as in H
- (B) The post-order traversal of T ends with 22, 15, and the last element of the array H is 13
- (C) The height of T is 3
- (D) In H , the root is 2 and its two children are 4 and 19

Q.14 [NAT · numerical answer | 2 marks | Heaps — counting distinct heaps]

The number of distinct binary **max**-heaps (as arrays of length 9, i.e. complete binary trees filled level by level from the left) that can be formed with the 9 distinct keys 1, 2, ..., 9 is _____.

Answer: _____ (enter a numerical value)

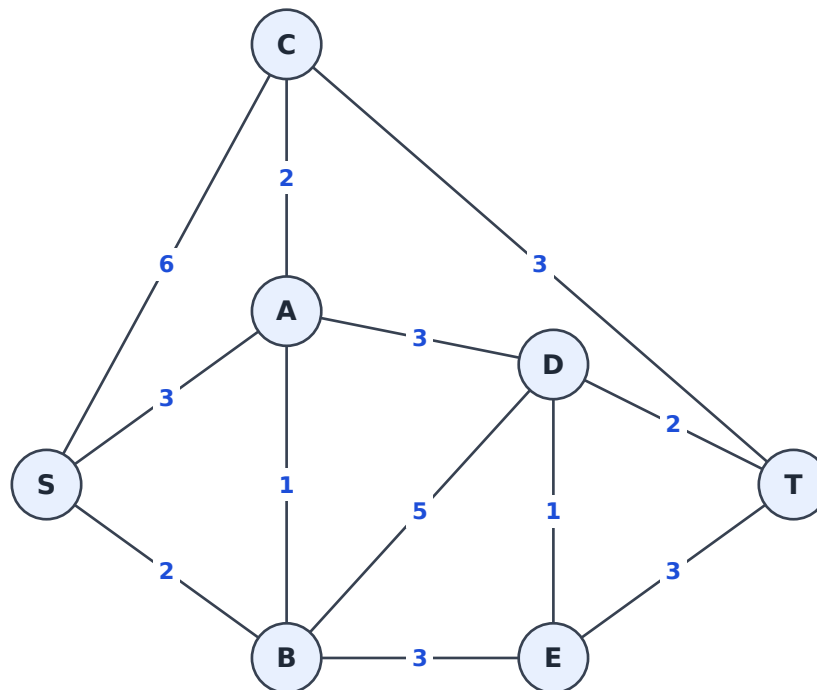
Q.15 [MCQ · single correct | 2 marks | Merge sort — bottom-up vs top-down]

Bottom-up (iterative) merge sort merges runs of width 1, 2, 4, ... from left to right; at each width, a run with no partner on its right is left unchanged. A merge compares the two front elements until one run is exhausted. The number of element comparisons it makes on [5, 2, 8, 6, 1, 9, 3] is:

- (A) 12
- (B) 14
- (C) 13
- (D) 11

Q.16 [NAT · numerical answer | 2 marks | Shortest paths — edges on some shortest path]

In the weighted undirected graph shown, the number of edges that lie on **at least one** shortest path from S to T is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Hashing — collision probabilities]

Three keys are inserted into a hash table with 5 slots using separate chaining. Each key independently hashes to each slot with probability $1/5$. Which of the following statements is/are TRUE?

- (A) The expected length of the longest chain is 1.6
- (B) The expected number of non-empty slots is 2.44
- (C) The probability that no two keys share a slot is $12/25$
- (D) The probability that all three keys land in the same slot is $1/25$

Q.18 [NAT · numerical answer | 2 marks | Stacks — prefix evaluation with floor division]

The function below evaluates a prefix (Polish) expression by scanning tokens from right to left. The value printed is _____.

Python

```
1 def ev(tokens):
2     st = []
3     for t in reversed(tokens):
4         if t in '+-*/':
5             a = st.pop()
6             b = st.pop()
7             st.append({'+': a + b, '-': a - b,
8                       '*': a * b, '/': a // b}[t])
9         else:
10            st.append(int(t))
11    return st[-1]
12
13 e = "* - / 7 -2 * 3 - 4 6 / - 5 * 2 6 + 1 3"
14 print(ev(e.split()))
```

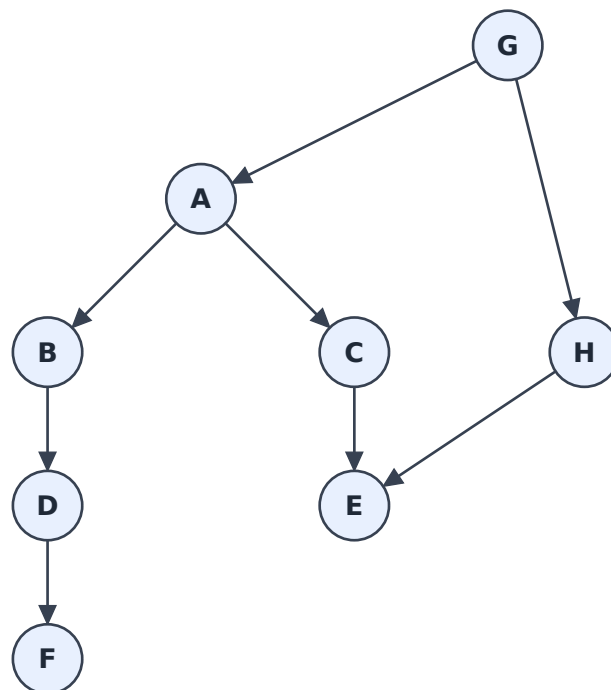
Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | DFS in Python — mutable default visited set]

The directed graph shown is stored as `g` (neighbours listed alphabetically). What does the program print?

Python

```
1 def dfs(g, u, seen=set(), order=None):
2     if order is None:
3         order = []
4     seen.add(u)
5     order.append(u)
6     for v in g[u]:
7         if v not in seen:
8             dfs(g, v, seen, order)
9     return order
10
11 g = {'A': 'BC', 'B': 'D', 'C': 'E', 'D': 'F',
12      'E': '', 'F': '', 'G': 'AH', 'H': 'E'}
13 print(dfs(g, 'A'), dfs(g, 'G'))
```



- (A) ['A', 'B', 'D', 'F', 'C', 'E'] ['G', 'H']
 (B) ['A', 'B', 'D', 'F', 'C', 'E'] ['G', 'A', 'B', 'D', 'F', 'C', 'E', 'H']
 (C) ['A', 'B', 'D', 'F', 'C', 'E'] ['G']
 (D) ['A', 'B', 'C', 'D', 'E', 'F'] ['G', 'H']

Q.20 [MSQ · one or more correct | 2 marks | Counting BSTs by height]

Consider all binary search trees containing exactly the keys 1, 2, 3, 4, 5. Height is the number of edges on the longest root-to-leaf path. Which of the following statements is/are TRUE?

- (A) Exactly 16 of them have height 4
 (B) Exactly 5 of them have 3 at the root
 (C) There are exactly 32 such BSTs
 (D) Exactly 6 of them have the minimum possible height, 2

Answer Key — Mock Test 48

Q	Type	Marks	Key	Topic
1	MCQ	1	(D)	Python — chained comparisons
2	NAT	1	40	Python — generators and iterator exhaustion
3	MSQ	1	(B), (C), (D)	Stacks — monotonic stack (next greater element)
4	NAT	1	7	Deque — sliding-window maximum
5	MCQ	1	(D)	Linked lists — split, reverse and interleave
6	NAT	1	14	Heaps — where can the k-th smallest be?
7	MCQ	1	(C)	Python dicts — insertion order and deletion
8	MCQ	1	(A)	Binary search on the answer
9	MCQ	1	(B)	Insertion sort — exact comparison count
10	MSQ	1	(B), (C), (D)	Graph theory — cycles, degrees, connectivity
11	MCQ	2	(B)	Python — closures, class attributes, try/finally
12	NAT	2	22	Recursion — memoisation via a mutable default
13	MSQ	2	(A), (C), (D)	BST vs binary heap built from the same keys
14	NAT	2	896	Heaps — counting distinct heaps
15	MCQ	2	(C)	Merge sort — bottom-up vs top-down
16	NAT	2	10	Shortest paths — edges on some shortest path
17	MSQ	2	(B), (C), (D)	Hashing — collision probabilities
18	NAT	2	-4	Stacks — prefix evaluation with floor division
19	MCQ	2	(A)	DFS in Python — mutable default visited set
20	MSQ	2	(A), (D)	Counting BSTs by height

Detailed Solutions — Mock Test 48

Q.1 [MCQ · 1 mark · Python — chained comparisons]

Answer: (D)

Python **chains** comparison operators: `a op1 b op2 c` means `(a op1 b) and (b op2 c)`, with `b` evaluated once. `is`, `is not`, `==`, `<` ... all belong to the same chaining family. Parentheses break the chain.

- `1 < 3 > 2 == 2 < 5` = `(1<3) and (3>2) and (2==2) and (2<5)` = **True**.
- `(1 < 3) > 2`: the parenthesised part is **True**, i.e. `1` as an int; `1 > 2` is **False**.
- `[] == [] is not []` = `([] == []) and ([] is not [])`. The middle list is shared by both comparisons, but the third literal is a *new* list object, so `is not` is True; the equality is True → **True**.

Output **True False True** → option **(D)**.

- (A) assumes `[] is not []` is False, i.e. that two empty-list literals are the same object — every `[]` creates a new list.
- (B) evaluates `(1 < 3) > 2` as if it were chained.
- (C) evaluates the first chain left-to-right as `((1<3) > 2) ...`, like C would.

Trap: in C/Java, `1 < 3 > 2` would compare a boolean with 2. Python's chaining is purely syntactic sugar for `and`.

Q.2 [NAT · 1 mark · Python — generators and iterator exhaustion]

Answer: 40

Generators and iterators are **single-pass**; and `zip` pulls from its arguments left to right, stopping as soon as *any* argument is exhausted.

- `g` yields 0, 1, 4, 9, 16, 25. The first `sum` consumes it completely, keeping odd squares `1 + 9 + 25` → `s1 = 35`.
- `sum(g)` on the exhausted generator → `s2 = 0`.
- `zip(it, it)` takes two consecutive items per tuple from the *same* iterator: (0,1), (2,3), (4,5), (6,7), (8,9) → 5 pairs. For the 6th tuple `zip` calls `next(it)` for the first argument and gets **10**, then the second `next` raises `StopIteration`, so the tuple is discarded — and 10 is **lost**.
- `rest = sum(it)` → 0.

Printed: `35 + 0 + 5 + 0 = 40`.

Trap: expecting `rest = 10` (answer 50), or expecting `s2 = 55` (re-iterating the generator). Use `itertools.zip_longest` or slicing when leftovers matter.

Q.3 [MSQ · 1 mark · Stacks — monotonic stack (next greater element)]

Answer: (B), (C), (D)

The stack keeps indices whose values are **strictly decreasing** from bottom to top; a new value pops every smaller value and becomes their next greater element. Each index is pushed once and popped at most once → $O(n)$.

Trace (stack shown as values):

- 6 → [6]
- 2 → [6, 2]
- 5 pops 2 (res=5) → [6, 5]
- 1 → [6, 5, 1] (size 3)
- 4 pops 1 (res=4) → [6, 5, 4] (size 3)
- 8 pops 4, 5, 6 (res=8 each) → [8]
- 3 → [8, 3]

- 7 pops 3 (res=7) → [8, 7]
- (A) The maximum size is 3. **False.**
- (B) pops = 1 + 1 + 3 + 1 = 6 (= 8 pushes – 2 left). **True.**
- (C) Final stack = indices 5, 7 = values 8, 7. **True.**
- (D) res = [8, 5, 8, 4, 8, -1, 7, -1]. **True.**

Trap: in (A), 4 does not stack on top of 1 — it pops 1 first, so the size stays 3.

Q.4 [NAT · 1 mark · Deques — sliding-window maximum]

Answer: 7

The deque holds indices whose values are decreasing from front to back. An incoming value removes from the back every value $\leq$ it (they can never be a future maximum); the front is dropped when it leaves the window.

Trace (deque as values):

- 4 → [4]; 2 → [4, 2]
- 12: pops 2, 4 (**2**) → [12]; max 12
- 3 → [12, 3]; max 12
- 8: pops 3 (**3**) → [12, 8]; max 12
- 6 → [12, 8, 6]; 12 (index 2) leaves the window (front pop) → [8, 6]; max 8
- 1 → [8, 6, 1]; max 8
- 9: pops 1, 6, 8 (**6**) → [9]; max 9
- 5 → [9, 5]; max 9
- 7: pops 5 (**7**) → [9, 7]; max 9

Back removals = **7**; window maxima = 12, 12, 12, 8, 8, 9, 9, 9.

Trap: counting the single front removal (12 leaving the window) as well, which gives 8. The total of all removals is at most n , which is why the algorithm is $O(n)$.

Q.5 [MCQ · 1 mark · Linked lists — split, reverse and interleave]

Answer: (D)

Three classic phases: (1) find the middle with slow/fast pointers, (2) cut and reverse the second half, (3) interleave.

- Phase 1 (fast starts at $h.next$): slow/fast = (1,2) → (2,4) → (3,6); fast.next = 7 → (4, None) stop. slow = **4**.
- Cut after 4: first half 1→2→3→4, second half 5→6→7.
- Phase 2: reverse the second half → 7→6→5 (prev = 7).
- Phase 3: 1→7, 7→2; 2→6, 6→3; 3→5, 5→4; prev becomes None → stop. 4's next is already None.

Result 1, 7, 2, 6, 3, 5, 4 → option **(D)** (pattern $L_0, L_n, L_1, L_{n-1}, \dots$).

- (A) cuts after 3 (using fast = h), leaving 4 in the second half.
- (B) loses the middle node 4 (would happen if the cut were made *before* the middle).
- (C) forgets to reverse the second half.

Trap: the multiple-assignment lines evaluate the entire right side first; writing them as sequential statements in the same order would lose pointers.

Q.6 [NAT · 1 mark · Heaps — where can the k -th smallest be?]

Answer: 14

Every ancestor of a node holds a smaller key. So if the 4th smallest key sits at depth d (root depth 0), all its d ancestors must be among the 3 smaller keys → $d \leq 3$. It cannot be the root ($d = 0$) because the root holds the minimum.

Conversely every node at depth 1, 2 or 3 is possible: put the 4th smallest there, place smaller keys on its ancestors (at most 3 needed) and any remaining smaller keys as other children of the root, then fill the rest with larger keys in heap order. With $1023 = 2^{10} - 1$ keys all of depths 1–3 exist.

Count = $2 + 4 + 8 = 14$ indices ($A[1] \dots A[14]$).

Trap: answering 15 (including the root) or 8 (only the deepest level 3, thinking the 4th smallest must be 'three levels down'). In general the k -th smallest ($k \geq 2$) can be at any depth from 1 to $k - 1$, i.e. at $2^k - 2$ indices.

Q.7 [MCQ · 1 mark · Python dicts — insertion order and deletion]

Answer: (C)

Since Python 3.7, a dict iterates in **insertion order** of its keys. Updating the value of an existing key does **not** change its position; deleting a key and inserting it again puts it at the **end**.

- Counting "banana": keys inserted in order b, a, n $\rightarrow \{b: 1, a: 3, n: 2\}$.
- `del d['b']` $\rightarrow \{a: 3, n: 2\}$.
- `d['b'] = 9` $\rightarrow$ re-inserted at the end $\rightarrow \{a: 3, n: 2, b: 9\}$.
- `d['a'] += 1` $\rightarrow$ value update in place $\rightarrow \{a: 4, n: 2, b: 9\}$.

Output `[('a', 4), ('n', 2), ('b', 9)]` $\rightarrow$ option **(C)**.

- (A) assumes updating 'a' moves it to the end.
- (B) assumes 'b' regains its original first position.
- (D) ignores the final increment.

Tip: underneath, CPython keeps a dense entries array plus a sparse hash index; deletion leaves a hole in the entries array, and a re-insert appends a new entry.

Q.8 [MCQ · 1 mark · Binary search on the answer]

Answer: (A)

The predicate 'speed v suffices' is **monotone** (if v works, every larger v works), so binary search finds the first v where it becomes true. The loop keeps the answer in $[lo, hi]$.

Hours needed: $hours(v) = \sum \lceil p/v \rceil$.

- it 1: $lo=1, hi=30, mid=15 \rightarrow 2+1+2+1+2 = 8 > 6 \rightarrow lo = 16$
- it 2: $mid=23 \rightarrow 2+1+1+1+1 = 6 \leq 6 \rightarrow hi = 23$
- it 3: $mid=19 \rightarrow 2+1+2+1+2 = 8 \rightarrow lo = 20$
- it 4: $mid=21 \rightarrow 2+1+2+1+1 = 7 \rightarrow lo = 22$
- it 5: $mid=22 \rightarrow 2+1+2+1+1 = 7 \rightarrow lo = 23$
- $lo = hi = 23 \rightarrow$ stop.

Output **23 5** $\rightarrow$ option **(A)**.

- (B) 22 needs 7 hours (the 23-pile takes 2 hours).
- (C) miscounts iterations (the search space 1..30 needs $\lceil \log_2 30 \rceil = 5$ halvings).
- (D) is a valid but not minimal speed.

Trap: writing `hi = mid - 1` when the predicate is true would skip the answer.

Q.9 [MCQ · 1 mark · Insertion sort — exact comparison count]

Answer: (B)

Comparisons = (number of shifts) + (number of passes that end with a *failing* comparison rather than at $j = -1$). Shifts = inversions.

Inversions of $[5, 1, 4, 2, 3]$: $(5,1), (5,4), (5,2), (5,3), (4,2), (4,3) \rightarrow 6$.

Pass by pass:

- insert 1: $5 > 1$ shift; $j = -1$ stop $\rightarrow 1$ comparison

- insert 4: $5 > 4$ shift; $1 > 4$? no $\rightarrow$ 2 comparisons
- insert 2: 5 shift, 4 shift, $1 > 2$? no $\rightarrow$ 3 comparisons
- insert 3: 5 shift, 4 shift, $2 > 3$? no $\rightarrow$ 3 comparisons

Total = $1 + 2 + 3 + 3 = 9$ = 6 shifts + 3 failing comparisons $\rightarrow$ option **(B)**.

- (A) counts only shifts.
- (C) adds a failing comparison to every pass, including the one that ran off the front.
- (D) undercounts the last pass.

Tip: for n elements, comparisons = inversions + $(n - 1)$ – (number of elements that become the new minimum of the sorted prefix).

Q.10 [MSQ · 1 mark · Graph theory — cycles, degrees, connectivity]

Answer: (B), (C), (D)

- (D) A spanning forest of a graph with n vertices and c components has $n - c$ edges. Each of the remaining $e - (n - c) = 12 - 8 = 4$ edges closes a different *fundamental cycle* with the forest (each contains its own non-forest edge, so they are distinct). **True.**
- (C) Degrees lie in $\{0, \dots, n-1\}$, but 0 and $n-1$ cannot both occur (a vertex of degree $n-1$ is adjacent to everything). So n vertices take at most $n-1$ distinct values $\rightarrow$ by pigeonhole two are equal. **True.**
- (B) Take a spanning tree ($n - 1$ edges); exactly one edge uv is left over. A tree has no cycle, so every cycle must use uv , and the rest of such a cycle is a u - v path inside the tree — which is unique. Hence exactly one cycle. **True.**
- (A) Two disjoint triangles: every degree is 2, but the graph has two components. **False.**

Trap: confusing 'minimum degree $\geq 2 \Rightarrow$ contains a cycle' (true) with ' $\Rightarrow$ connected' (false).

Q.11 [MCQ · 2 marks · Python — closures, class attributes, try/finally]

Answer: (B)

Three mechanisms interact:

- `i=i` freezes each closure's multiplier at 0, 1, 2 (no late-binding problem).
- `Counter.total` is a **class** attribute shared by all calls.
- In `return expr` inside `try`, `expr` is evaluated **before** `finally` runs; `finally` cannot change an already-computed return value (it does not itself return).

First call `run([1, 2, 3])`:

- `f0(1)`: `total += 0  $\rightarrow$  0`; `f1(2)`: `total += 2  $\rightarrow$  2`; `f2(3)`: `total += 6  $\rightarrow$  8`.
- `return Counter.total` computes **8**; then `finally` makes total 108.

Second call `run([4, -5, 6])`:

- `f0(4)`: `total += 0  $\rightarrow$  108`.
- `v = -5  $\rightarrow$  ValueError(-5)`; handler returns `-(-5) = 5`; `finally` $\rightarrow$ total 208.

The arguments of `print` are evaluated left to right, so `Counter.total` is read last $\rightarrow$ **208**.

Output **8 5 208** $\rightarrow$ option **(B)**.

- (A) assumes `finally` runs before the return value is computed.
- (C) additionally treats the except-branch return like the try-branch.
- (D) forgets the negation in `-e.args[0]`.

Trap: a `return` inside `finally` would override; a plain statement there only has side effects.

Q.12 [NAT · 2 marks · Recursion — memoisation via a mutable default]**Answer: 22**

The default dict `memo={}` is created once and **shared by every call**, so it acts as a persistent cache — even across the two top-level calls. Every call (hit or miss) increments `calls`.

First call `f(10)`:

- Each `n` is computed (a miss) at most once. `f(10) → f(9) → ... → f(3) → f(2)` (base) then `f(0)` (base); returning up, `f(4)` needs `f(1)` (base, new), and `f(n-3)` for $n \geq 5$ is always already cached.
- Computed values: `n = 0 ... 10 → 11` misses. Each computed $n \geq 3$ (8 values: 3 ... 10) makes exactly 2 calls $\rightarrow$ 16 calls, plus the top-level call $\rightarrow$ **17**.

Second call `f(12)`:

- `f(12)` miss $\rightarrow$ calls `f(11)` (miss) and `f(9)` (hit); `f(11)` calls `f(10)` (hit) and `f(8)` (hit).
- Calls: `f(12)`, `f(11)`, `f(10)`, `f(8)`, `f(9)` $\rightarrow$ **5**.

Total = $17 + 5 =$ **22**.

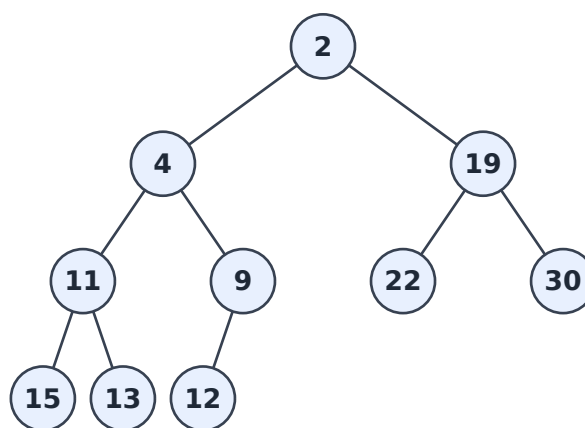
Trap: assuming the memo is empty again for `f(12)` (which would give $17 + 21 = 38$), or counting only cache misses (13).

Q.13 [MSQ · 2 marks · BST vs binary heap built from the same keys]**Answer: (A), (C), (D)**

Build H by sift-up insertion:

- 15 $\rightarrow$ [15]; 9 $\rightarrow$ swap $\rightarrow$ [9, 15]; 22 $\rightarrow$ [9, 15, 22].
- 4 (index 3): swap with 15, then with 9 $\rightarrow$ [4, 9, 22, 15].
- 12 (index 4): parent 9 $<$ 12 $\rightarrow$ [4, 9, 22, 15, 12].
- 19 (index 5): swap with 22 $\rightarrow$ [4, 9, 19, 15, 12, 22]; 30 $\rightarrow$ appended.
- 11 (index 7): swap with 15; parent 9 $<$ 11 $\rightarrow$ stop $\rightarrow$ [4, 9, 19, 11, 12, 22, 30, 15].
- 13 (index 8): parent 11 $<$ 13 $\rightarrow$ stays.
- 2 (index 9): swap with 12, then 9, then 4 $\rightarrow$ H = [2, 4, 19, 11, 9, 22, 30, 15, 13, 12].
- (A) Depths (T, H): 15 (0, 3), 9 (1, 2), 22 (1, 2), 4 (2, 1), 12 (2, 3), 19 (2, 1), 30 (2, 2) $\checkmark$, 11 (3, 2), 13 (3, 3) $\checkmark$, 2 (3, 0). Exactly 30 and 13 match. **True**.
- (B) Post-order of T = 2, 4, 11, 13, 12, 9, 19, 30, 22, 15 — ends with 22, 15 $\checkmark$, but the last array element of H is **12**, not 13. **False**.
- (C) Longest root-to-leaf path in T: 15 $\rightarrow$ 9 $\rightarrow$ 4 $\rightarrow$ 2 or 15 $\rightarrow$ 9 $\rightarrow$ 12 $\rightarrow$ 11 $\rightarrow$ 3 edges. **True**.
- (D) $H[0] = 2$, $H[1] = 4$, $H[2] = 19$. **True**.

Trap: in H the last inserted key 2 travels all the way to the root, dragging 12 (not 13) into the last slot.



Min-heap H after all insertions

Q.14 [NAT · 2 marks · Heaps — counting distinct heaps]**Answer: 896**

The maximum (9) must be at the root. The remaining 8 keys are split between the left and right subtrees, whose **shapes are fixed** by completeness; any choice of which keys go left works, and each side is then any heap of its own shape:

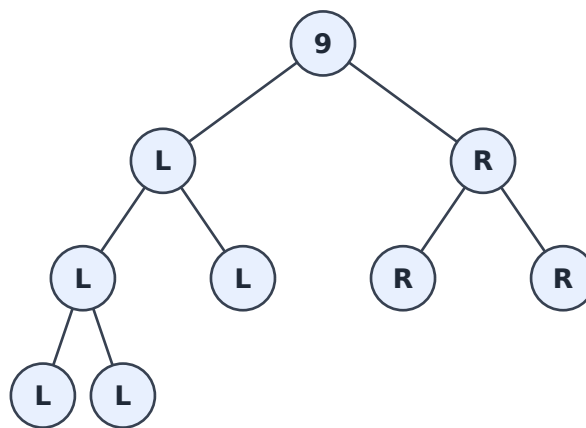
$H(n) = C(n-1, L) \cdot H(L) \cdot H(R)$, where L, R are the subtree sizes.

Shapes: a complete tree with 9 nodes has levels of 1, 2, 4 and 2 nodes. The two last-level nodes are both under the left child, so $L = 1 + 2 + 2 = 5$ (indices 1, 3, 4, 7, 8) and $R = 3$ (indices 2, 5, 6).

- $H(1) = 1$, $H(2) = 1$ (root + one child), $H(3) = C(2,1) \cdot 1 \cdot 1 = 2$.
- $H(5)$: $L = 3$, $R = 1 \rightarrow C(4,3) \cdot H(3) \cdot H(1) = 4 \cdot 2 = 8$.
- $H(9) = C(8,5) \cdot H(5) \cdot H(3) = 56 \cdot 8 \cdot 2 = \mathbf{896}$.

Sanity check: $9! = 362\,880$ permutations, of which only 896 satisfy the heap property (the brute-force count agrees).

Trap: using a balanced split $L = R = 4$ (would give $C(8,4) \cdot 3 \cdot 3 = 630$), or forgetting the binomial factor for choosing which keys go left.



Shape: 5 positions in the left subtree (L), 3 in the right (R)

Q.15 [MCQ · 2 marks · Merge sort — bottom-up vs top-down]**Answer: (C)**

Bottom-up and top-down merge sort split an odd-length array differently, so their comparison counts can differ.

Width 1 (pairs): $(5|2) \rightarrow 1$, $(8|6) \rightarrow 1$, $(1|9) \rightarrow 1$, $[3]$ has no partner $\rightarrow 3$ comparisons; array $[2, 5, 6, 8, 1, 9, 3]$.

Width 2: $[2, 5] + [6, 8]$: $2 < 6$, $5 < 6$, left exhausted $\rightarrow 2$. $[1, 9] + [3]$: $1 < 3$, $9 > 3$, right exhausted $\rightarrow 2$. Array $[2, 5, 6, 8, 1, 3, 9]$.

Width 4: $[2, 5, 6, 8] + [1, 3, 9]$: 1, 2, 3, 5, 6, 8 are output by 6 comparisons, then 9 is copied $\rightarrow 6$.

Total = $3 + 4 + 6 = 13 \rightarrow$ option **(C)**.

- (A), (D) miscount the unequal merges.
- (B) 14 is what **top-down** merge sort (split $\lfloor n/2 \rfloor = 3 | 4$) makes: $[5 | 2, 8] \rightarrow 1 + 2$, $[6, 1 | 9, 3] \rightarrow 1 + 1 + 3$, final $3 + 4$ run merge $\rightarrow 6$; total 14.

Trap: assuming both versions perform identical merges — they only coincide when n is a power of two.

Bottom-up passes

	width	array after pass	comparisons
pass 1	1	2 5 6 8 1 9 3	3
pass 2	2	2 5 6 8 1 3 9	4
pass 3	4	1 2 3 5 6 8 9	6

Q.16 [NAT · 2 marks · Shortest paths — edges on some shortest path]**Answer: 10**

Run Dijkstra from S **and** from T. An edge (u, v, w) lies on some shortest S-T path iff $d_S(u) + w + d_T(v) = d_S(T)$ (in either orientation).

Distances from S: S 0, A 3, B 2, C 5, D 6, E 5, T 8.

Distances to T: S 8, A 5, B 6, C 3, D 2, E 3, T 0.

Check each edge (best orientation):

- S-A: $0 + 3 + 5 = 8$ ✓ - S-B: $0 + 2 + 6 = 8$ ✓ - S-C: $0 + 6 + 3 = 9$ ✗
- A-B: $d_S(B) + 1 + d_T(A) = 2 + 1 + 5 = 8$ ✓ (path S-B-A-D-T)
- A-D: $3 + 3 + 2 = 8$ ✓ - B-D: $2 + 5 + 2 = 9$ ✗ - B-E: $2 + 3 + 3 = 8$ ✓
- D-T: $6 + 2 = 8$ ✓ - E-T: $5 + 3 = 8$ ✓
- D-E: $d_S(E) + 1 + d_T(D) = 5 + 1 + 2 = 8$ ✓ (path S-B-E-D-T)
- C-T: $5 + 3 = 8$ ✓ - A-C: $3 + 2 + 3 = 8$ ✓ (path S-A-C-T)

Edges on some shortest path: 12 - 2 = **10** (all except S-C and B-D).

Trap: only marking the edges of the single path Dijkstra happens to record in its predecessor tree, or testing only one orientation of each undirected edge (A-B and D-E are used 'backwards' relative to the letter order).

Q.17 [MSQ · 2 marks · Hashing — collision probabilities]**Answer: (B), (C), (D)**

There are $5^3 = 125$ equally likely outcomes.

- (A) Longest chain L: $P(L = 1) = 60/125$, $P(L = 3) = 5/125$, so $P(L = 2) = 60/125$. $E[L] = (60 \cdot 1 + 60 \cdot 2 + 5 \cdot 3)/125 = 195/125 = \mathbf{1.56}$, not 1.6. **False.**
- (B) Linearity of expectation: a slot stays empty with probability $(4/5)^3 = 64/125$, so $E[\text{non-empty}] = 5 \cdot (1 - 64/125) = 5 \cdot 61/125 = \mathbf{2.44}$. **True.**
- (C) All distinct: $5 \cdot 4 \cdot 3 = 60$ outcomes $\rightarrow 60/125 = \mathbf{12/25} = 0.48$. **True.**
- (D) All in one slot: 5 outcomes $\rightarrow 5/125 = \mathbf{1/25}$. **True.**

Even with load factor $\alpha = 0.6$, a collision happens with probability 0.52 — the birthday effect.

Trap: in (A), guessing $1 + \alpha = 1.6$ — that is the expected cost of a successful search pattern, not the expected *maximum* chain length.

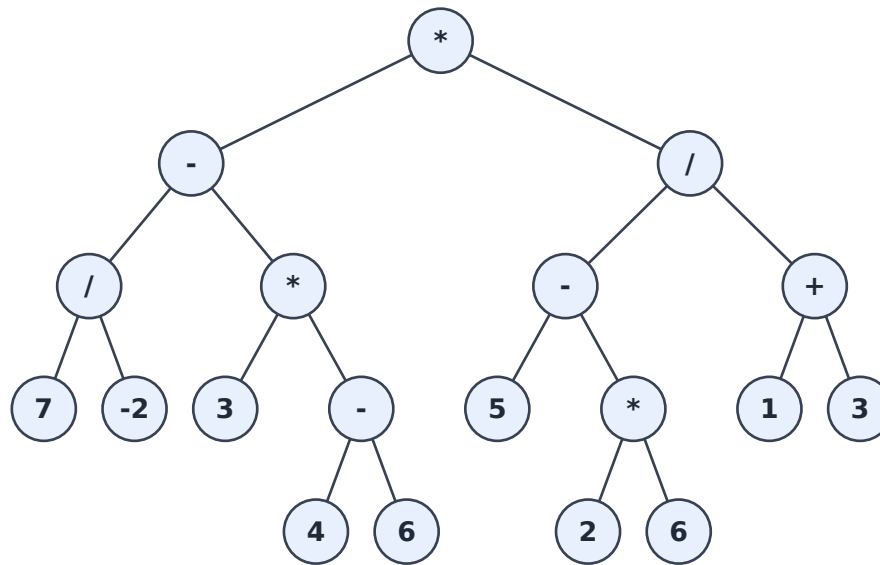
Q.18 [NAT · 2 marks · Stacks — prefix evaluation with floor division]**Answer: -4**

Scanning a prefix expression from the right, the **first** value popped is the **left** operand. Note '-2' in '+-*/' is False (the string "-2" is not a substring of "+-*/"), so -2 is an operand. / is Python's floor division.

Structure: * X Y with $X = - / 7 -2 * 3 - 4 6$ and $Y = / - 5 * 2 6 + 1 3$.

- X: $/ 7 -2 = 7 // -2 = \text{floor}(-3.5) = \mathbf{-4}$; $* 3 - 4 6 = 3 \times (4 - 6) = -6$; $X = -4 - (-6) = \mathbf{2}$.
- Y: $- 5 * 2 6 = 5 - 12 = -7$; $+ 1 3 = 4$; $Y = -7 // 4 = \text{floor}(-1.75) = \mathbf{-2}$.
- Result: $X \times Y = 2 \times (-2) = \mathbf{-4}$.

Trap: with truncating division (C/Java) $X = -3 + 6 = 3$ and $Y = -1$, giving -3 ; popping operands in the postfix order (right operand first) would compute $-2 // 7$ etc. and give a completely different value.



Expression tree of e

Q.19 [MCQ · 2 marks · DFS in Python — mutable default visited set]

Answer: (A)

`order` is correctly reset with the `None` idiom, but `seen=set()` is a mutable default evaluated **once** — the second top-level call starts with everything the first call visited already marked.

First call `dfs(g, 'A')` — recursive DFS, neighbours in listed order:

- $A \rightarrow B \rightarrow D \rightarrow F$ (dead end) $\rightarrow$ back to $A \rightarrow C \rightarrow E$.
- `order = [A, B, D, F, C, E]`; `seen = {A, B, C, D, E, F}`.

Second call `dfs(g, 'G')` with the same `seen`:

- G added. Neighbour A is already in `seen` $\rightarrow$ skipped. H is new $\rightarrow$ visit H; its neighbour E is in `seen` $\rightarrow$ skipped.
- `order = [G, H]`.

Output $\rightarrow$ option **(A)**.

- (B) is the correct DFS from G with a fresh visited set — the intended behaviour.
- (C) assumes G's neighbours are all seen — H was not reachable from A.
- (D) is a BFS order for the first call.

Trap: the bug is silent for the first call; only repeated calls reveal it. Always use `seen=None` and create the set inside.

Q.20 [MSQ · 2 marks · Counting BSTs by height]

Answer: (A), (D)

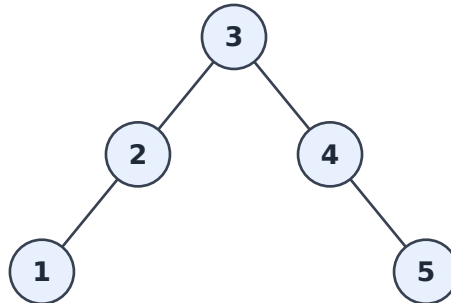
The number of BSTs on n keys is the Catalan number C_n : $C_0..C_5 = 1, 1, 2, 5, 14, 42$. With root r there are $C_{r-1} \cdot C_{5-r}$ trees.

- (A) Height 4 with 5 nodes means a path: the root is 1 or 5 (otherwise both sides are non-empty and the height ≤ 3), and the same holds at every level $\rightarrow$ each of the 4 non-leaf steps chooses min or max of the remaining keys $\rightarrow 2^4 = 16$. **True.**
- (B) Root 3: left $\{1, 2\} \rightarrow C_2 = 2$, right $\{4, 5\} \rightarrow 2 \rightarrow 4$ trees. **False.**
- (C) Total = $C_5 = 42$ ($= 14 + 5 + 4 + 5 + 14$ by root). **False.**

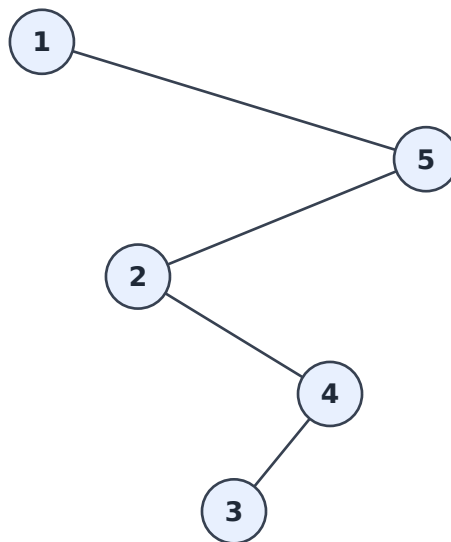
- (D) Height 2 holds at most 7 nodes; with 5 keys the root's two subtrees must both have height ≤ 1 , i.e. sizes ≤ 3 and summing to 4: (1, 3), (2, 2), (3, 1). A 3-node subtree of height 1 must be perfect (1 way); a 2-node subtree has 2 shapes. Count = $1 \cdot 1 + 2 \cdot 2 + 1 \cdot 1 = 6$. **True.**

Height distribution: $h=2$: 6, $h=3$: 20, $h=4$: 16 (sum 42).

Trap: in (C), confusing 2^n (or 2^{n-1} chains) with the Catalan count.



One of the 6 BSTs of height 2



One of the 16 BSTs of height 4 (a zig-zag path)

Mock Test 49

Challenge Mock — Paper 4

Difficulty: Hard

Focus areas: hardest, multi-concept, trap-heavy 2-mark questions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	8 / 5 / 7

Marking: MCQ — +1/+2 for correct, −1/3 (1-mark) or −2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — list multiplication aliasing]

Consider the following Python program. What is printed?

Python

```
1 grid = [[0] * 3] * 3
2 grid[0][1] = 5
3 row = grid[1][:]
4 row[2] = 7
5 print(grid[2], row, grid[0] is grid[2], row == grid[1])
```

- (A) [0, 0, 0] [0, 5, 7] False False
- (B) [0, 5, 0] [0, 5, 7] True False
- (C) [0, 5, 0] [0, 5, 7] True True
- (D) [0, 5, 0] [0, 0, 7] True False

Q.2 [NAT · numerical answer | 1 mark | Python — global vs nonlocal]

Consider the following Python program. The value printed is _____.

Python

```
1 x = 10
2 def outer():
3     x = 1
4     def inner():
5         nonlocal x
6         x += 2
7         return x
8     def other():
9         global x
10        x *= 3
11        return x
12    return inner() + other() + inner() + x
13
14 print(outer() + x)
```

Answer: _____ (enter a numerical value)

Q.3 [MCQ · single correct | 1 mark | Stacks — prefix expression evaluation]

The prefix (Polish) expression below is evaluated with a stack by scanning from **right to left**. $^$ is exponentiation and $/$ is exact division.

$- / 8 ^ 2 2 * + 4 2 3$

The value of the expression is

- (A) 2
- (B) 16
- (C) -16
- (D) 17.5

Q.4 [MCQ · single correct | 1 mark | Linked lists — reversing in groups of k]

Consider the following Python program. What is printed?

Python

```
1 class N:
2     def __init__(self, v, nx=None):
3         self.v, self.nx = v, nx
4
5 def rev_k(h, k):
6     cur, cnt = h, 0
7     while cur and cnt < k:
8         cur, cnt = cur.nx, cnt + 1
9     if cnt < k:
10        return h
11    prev, cur = rev_k(cur, k), h
12    for _ in range(k):
13        cur.nx, prev, cur = prev, cur, cur.nx
14    return prev
15
16 h = None
17 for v in range(8, 0, -1):
18     h = N(v, h)
19 h = rev_k(h, 3)
20 out = []
21 while h:
22     out.append(h.v)
23     h = h.nx
24 print(out)
```



List built by the loop

- (A) [3, 2, 1, 6, 5, 4, 7, 8]
- (B) [3, 2, 1, 6, 5, 4, 8, 7]
- (C) [8, 7, 6, 5, 4, 3, 2, 1]
- (D) [1, 2, 3, 6, 5, 4, 7, 8]

Q.5 [MSQ · one or more correct | 1 mark | Binary trees — counting facts]

Which of the following statements is/are TRUE?

- (A) A complete binary tree with 100 nodes has height 7 (height = edges on the longest root-to-leaf path)
- (B) A binary tree with n nodes has exactly $n + 1$ empty (None) child pointers
- (C) Every binary tree with 12 leaves has exactly 11 nodes with two children
- (D) A full binary tree (every node has 0 or 2 children) with 20 leaves has exactly 39 nodes

Q.6 [MCQ · single correct | 1 mark | Hashing — probability of no collision]

Three distinct keys are inserted into a hash table with 8 slots. Assume each key independently hashes to each slot with probability $1/8$. The probability that **no two** keys hash to the same slot is closest to

- (A) 0.670
- (B) 0.875
- (C) 0.344
- (D) 0.656

Q.7 [NAT · numerical answer | 1 mark | Binary search — average successful search]

A sorted array of $n = 1023$ distinct keys is searched with standard binary search ($\text{mid} = (\text{lo} + \text{hi}) // 2$; one three-way comparison per iteration). Each of the 1023 keys is equally likely to be the search key. The average number of comparisons for a **successful** search (rounded off to two decimal places) is _____.

Answer: _____ (enter a numerical value)

Q.8 [MCQ · single correct | 1 mark | Sorting — stability of selection sort]

Records (key, tag) are sorted by key with selection sort: for $i = 0, 1, \dots$, the record with the smallest key in positions $i \dots n-1$ (the **first** such record if there are ties) is swapped into position i . The input is

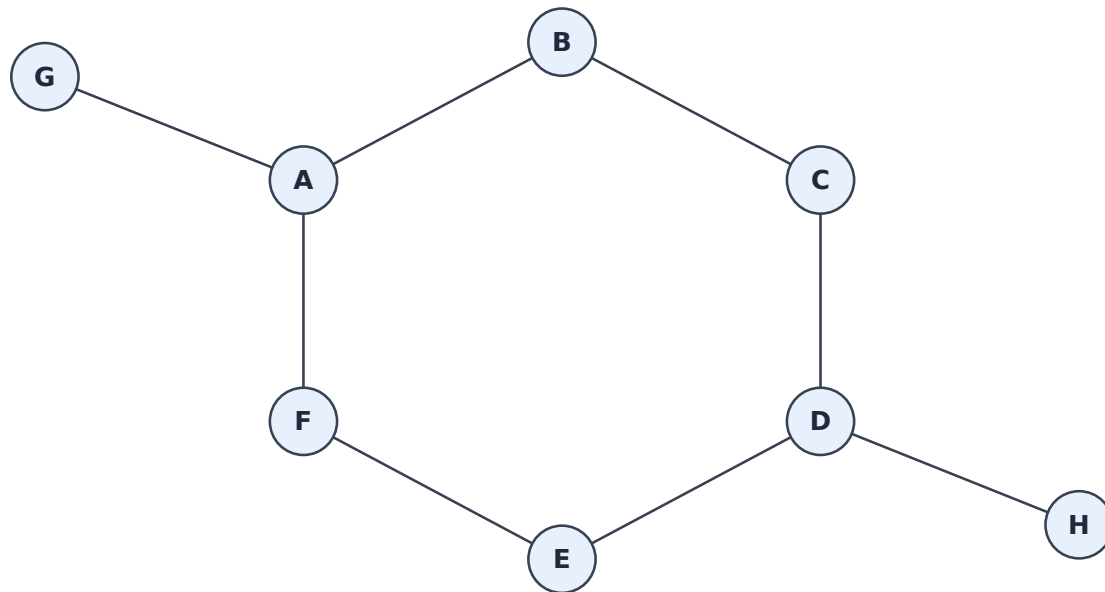
(4,p) (2,q) (4,r) (1,s) (2,t) (3,u).

The sequence of tags after sorting is

- (A) s q t u p r
- (B) s q t u r p
- (C) s t q u r p
- (D) s t q u p r

Q.9 [MSQ · one or more correct | 1 mark | Graphs — bipartiteness]

The undirected graph G below is bipartite. Each option proposes adding **one** edge to G (the options are considered separately). After which of the following additions is the graph still bipartite?



- (A) Edge F-G
- (B) Edge A-C
- (C) Edge B-E
- (D) Edge G-H

Q.10 [NAT · numerical answer | 1 mark | Loop counting — harmonic sums]

Consider the following Python program. The value printed is ____.

Python

```
1 c = 0
2 n = 20
3 for i in range(1, n + 1):
4     for j in range(i, n + 1, i):
5         c += 1
6 print(c)
```

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — sorted(): keys, case and stability]

Consider the following Python program. What is printed?

Python

```
1 data = ["kiwi", "Fig", "apple", "date", "Banana", "fig"]
2 r1 = sorted(data, key=len)
3 r2 = sorted(data, key=str.lower)
4 r3 = sorted(sorted(data), key=len, reverse=True)
5 print(r1[:3], r2[:2], r3[2:5])
```

- (A) ['Fig', 'fig', 'kiwi'] ['apple', 'Banana'] ['date', 'kiwi', 'Fig']
- (B) ['Fig', 'fig', 'kiwi'] ['apple', 'Banana'] ['kiwi', 'date', 'fig']
- (C) ['Fig', 'fig', 'date'] ['apple', 'Banana'] ['date', 'kiwi', 'Fig']
- (D) ['Fig', 'fig', 'kiwi'] ['Banana', 'Fig'] ['date', 'kiwi', 'Fig']

Q.12 [NAT · numerical answer | 2 marks | Python — memoisation through a mutable default]

Consider the following Python program. It prints four numbers; the last two are c1 and c2. The value of c1 + c2 is _____.

Python

```

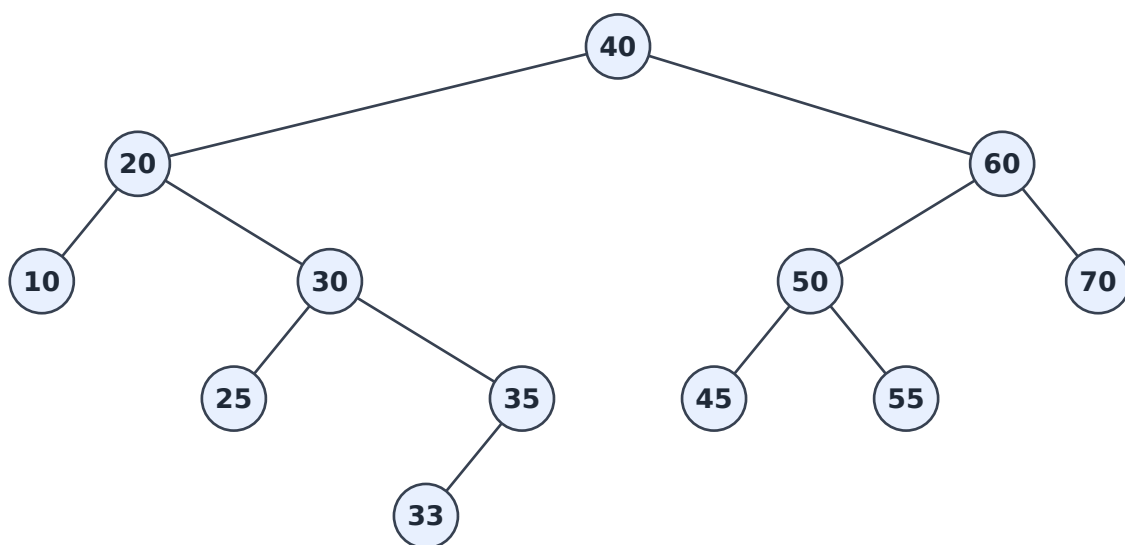
1 calls = 0
2 def T(n, memo={}):
3     global calls
4     calls += 1
5     if n in memo:
6         return memo[n]
7     val = 1 if n < 3 else T(n - 1) + T(n - 2) + T(n - 3)
8     memo[n] = val
9     return val
10
11 a = T(8)
12 c1 = calls
13 b = T(10)
14 c2 = calls - c1
15 print(a, b, c1, c2)

```

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | BST — deletion with two children]

The BST below is built by inserting 40, 20, 60, 10, 30, 50, 70, 25, 35, 45, 55, 33. Then **40** is deleted and afterwards **30** is deleted. A node with two children is deleted by copying its **in-order successor**'s key into it and deleting the successor node. Height = number of edges on the longest root-to-leaf path. Which of the following statements is/are TRUE about the final tree?



Initial BST

- (A) It has exactly 4 leaves
- (B) Its pre-order traversal is 45, 20, 10, 33, 25, 35, 60, 50, 55, 70
- (C) Its height is 4
- (D) If the in-order **predecessor** had been used for deleting 40 instead, the root after that deletion would be 35

Q.14 [NAT · numerical answer | 2 marks | Hashing — deletion with tombstones]

A hash table has 11 slots, $h(k) = k \bmod 11$, and linear probing. Deletion replaces a key by a DELETED marker (tombstone). **Insertion** places the key in the first slot of its probe sequence that is EMPTY or DELETED. **Search** probes until it finds the key or an EMPTY slot (DELETED slots are probed and skipped). The operations are:

insert 22, 33, 44, 13, 24; delete 33; delete 13; insert 55; insert 46; search 24; search 35.

Counting every slot examined as one probe, the total number of probes made by the two searches is _____.

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Quicksort — Hoare partition]

The Hoare partition below is called as `hoare(A, 0, 7)` on $A = [26, 41, 13, 26, 9, 37, 18, 52]$. What does it return, and what is A afterwards?

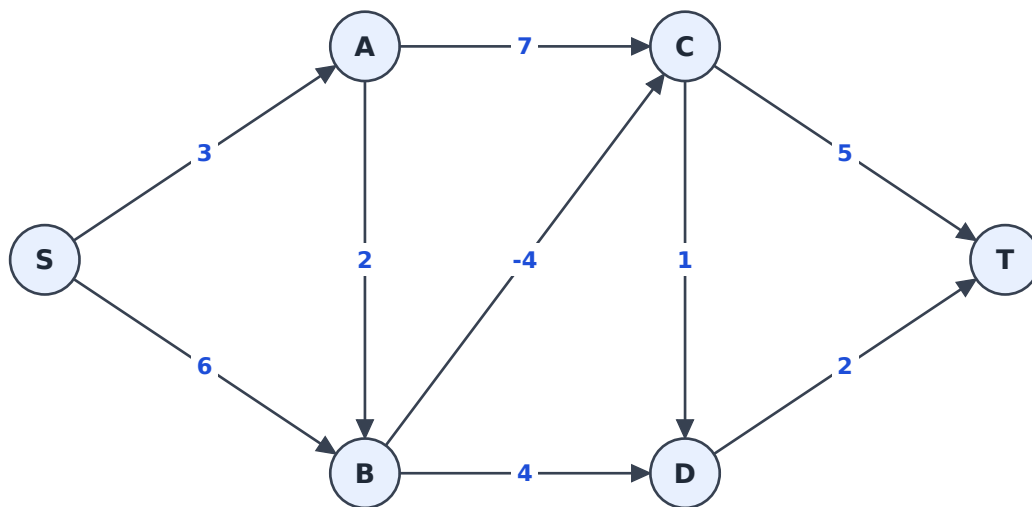
Python

```
1 def hoare(A, lo, hi):
2     p = A[lo]
3     i, j = lo - 1, hi + 1
4     while True:
5         i += 1
6         while A[i] < p:
7             i += 1
8         j -= 1
9         while A[j] > p:
10            j -= 1
11        if i >= j:
12            return j
13        A[i], A[j] = A[j], A[i]
```

- (A) Returns 4; A = [18, 9, 13, 26, 26, 37, 41, 52]
- (B) Returns 2; A = [18, 9, 13, 26, 41, 37, 26, 52]
- (C) Returns 3; A = [18, 9, 13, 26, 41, 37, 26, 52]
- (D) Returns 3; A = [18, 9, 13, 26, 37, 41, 26, 52]

Q.16 [MSQ · one or more correct | 2 marks | Shortest and longest paths in a DAG]

Consider the weighted DAG below (note the negative edge B→C). Which of the following statements is/are TRUE?



- (A) The DAG has exactly 2 topological orderings
- (B) Textbook Dijkstra from S (a vertex is final once extracted; edges into extracted vertices are ignored) computes the correct shortest distance for **every** vertex of this graph
- (C) The longest (maximum-weight) S→T path has weight 15
- (D) The shortest S→T distance is 4

Q.17 [NAT · numerical answer | 2 marks | Divide and conquer — inversion counting]

Consider the following Python program. The value printed is ____.

Python

```

1 def count(a):
2     if len(a) < 2:
3         return a, 0
4     m = len(a) // 2
5     L, x = count(a[:m])
6     R, y = count(a[m:])
7     out, i, j, inv = [], 0, 0, x + y
8     while i < len(L) and j < len(R):
9         if L[i] <= R[j]:
10            out.append(L[i]); i += 1
11        else:
12            out.append(R[j]); j += 1
13            inv += len(L) - i
14    out += L[i:] + R[j:]
15    return out, inv
16
17 print(count([5, 3, 5, 1, 3, 5, 2, 2])[1])

```

Answer: _____ (enter a numerical value)

Q.18 [MCQ · single correct | 2 marks | Heaps — top-k of a stream]

To keep the 3 largest values of a stream, a binary **min**-heap of capacity 3 is used (array, root at index 0). The first three values are inserted normally (append + sift-up). For each later value x : if $x > \text{root}$, the root is **replaced** by x and sifted down (swap with the smaller child while larger than it); otherwise x is discarded. For the stream

15, 4, 22, 9, 31, 7, 18, 26, 3

the final heap array is

- (A) [22, 26, 31]
- (B) [26, 31, 22]
- (C) [18, 31, 26]
- (D) [22, 31, 26]

Q.19 [NAT · numerical answer | 2 marks | Graph theory — maximum edges with components]

A simple undirected graph has 15 vertices and exactly 3 connected components, and **each component has at least 3 vertices**. The maximum possible number of edges is _____.

Answer: _____ (enter a numerical value)

Q.20 [MSQ · one or more correct | 2 marks | Binary search — counting occurrences]

Consider the following functions, where **a** is a list sorted in non-decreasing order. Which of the following statements is/are TRUE?

Python

```

1 def first_ge(a, x):
2     lo, hi = 0, len(a)
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if a[mid] < x:
6             lo = mid + 1
7         else:
8             hi = mid
9     return lo
10
11 def count(a, x):
12     return first_ge(a, x + 1) - first_ge(a, x)

```

- (A) For a list of length 1000, the loop in `first_ge` executes at most 10 times
- (B) `first_ge([], 7)` returns 0
- (C) `count([1, 2, 2, 2, 5], 2)` returns 3
- (D) `count(a, x)` returns the number of occurrences of x for every sorted list of **floats** a and float x

Answer Key — Mock Test 49

Q	Type	Marks	Key	Topic
1	MCQ	1	(B)	Python — list multiplication aliasing
2	NAT	1	73	Python — global vs nonlocal
3	MCQ	1	(C)	Stacks — prefix expression evaluation
4	MCQ	1	(A)	Linked lists — reversing in groups of k
5	MSQ	1	(B), (C), (D)	Binary trees — counting facts
6	MCQ	1	(D)	Hashing — probability of no collision
7	NAT	1	9.00 to 9.02	Binary search — average successful search
8	MCQ	1	(B)	Sorting — stability of selection sort
9	MSQ	1	(C), (D)	Graphs — bipartiteness
10	NAT	1	66	Loop counting — harmonic sums
11	MCQ	2	(A)	Python — sorted(): keys, case and stability
12	NAT	2	26	Python — memoisation through a mutable default
13	MSQ	2	(B), (D)	BST — deletion with two children
14	NAT	2	7	Hashing — deletion with tombstones
15	MCQ	2	(C)	Quicksort — Hoare partition
16	MSQ	2	(B), (C), (D)	Shortest and longest paths in a DAG
17	NAT	2	16	Divide and conquer — inversion counting
18	MCQ	2	(D)	Heaps — top-k of a stream
19	NAT	2	42	Graph theory — maximum edges with components
20	MSQ	2	(A), (B), (C)	Binary search — counting occurrences

Detailed Solutions — Mock Test 49

Q.1 [MCQ · 1 mark · Python — list multiplication aliasing]

Answer: (B)

`[[0] * 3] * 3` builds **one** inner list and an outer list holding three references to it. `[0] * 3` itself is fine, because its elements are immutable ints.

- `grid[0][1] = 5` mutates the single shared row → every 'row' reads `[0, 5, 0]`.
- `grid[1][:]` is a **shallow copy** of that row → a new list `[0, 5, 0]`; `row[2] = 7` changes only the copy → `row = [0, 5, 7]`.
- `grid[0] is grid[2]` → `True` (same object).
- `row == grid[1]` compares values: `[0, 5, 7]` vs `[0, 5, 0]` → `False`.

Output `[0, 5, 0] [0, 5, 7] True False` → option (B).

- (A) assumes three independent rows.
- (C) assumes the slice is an alias (it is a copy).
- (D) assumes the copy was taken before the assignment of 5 reached row 1.

Fix: `grid = [[0] * 3 for _ in range(3)]` creates three separate rows — the standard way to build a DP table or an adjacency matrix.

Q.2 [NAT · 1 mark · Python — global vs nonlocal]

Answer: 73

There are two different variables named `x`: the **global** `x` (10) and the **local** `x` of **outer** (1). **nonlocal** `x` in **inner** binds to outer's `x`; **global** `x` in **other** binds to the module-level `x`.

Inside **outer**, the sum is evaluated left to right:

- `inner()` → outer's `x = 1 + 2 = 3` → returns 3.
- `other()` → global `x = 10 × 3 = 30` → returns 30.
- `inner()` → outer's `x = 3 + 2 = 5` → returns 5.
- `x` (outer's local) → 5.
- `outer()` returns `3 + 30 + 5 + 5 = 43`.

In `print(outer() + x)`, `outer()` is evaluated first (43) and **then** the global `x` is read — it is already 30. Result `43 + 30 = 73`.

Traps: using the stale global 10 (→ 53), or letting **other** modify outer's `x` (→ different sums). Python evaluates the operands of `+` strictly left to right, so the side effect inside `outer()` is visible to the right operand.

Q.3 [MCQ · 1 mark · Stacks — prefix expression evaluation]

Answer: (C)

Scanning a prefix expression right to left, operands are pushed; for an operator the **first** popped value is the **left** operand (the opposite of postfix evaluation).

- `3, 2, 4` → stack (bottom→top) `[3, 2, 4]`.
- `+` → pop 4 (left), 2 (right) → `6` → `[3, 6]`.
- `*` → pop 6 (left), 3 (right) → `18` → `[18]`.
- `2, 2` → `[18, 2, 2]`; `^` → `22 = 4` → `[18, 4]`.
- `8` → `[18, 4, 8]`; `/` → pop 8 (left), 4 (right) → `2` → `[18, 2]`.
- `-` → pop 2 (left), 18 (right) → `2 - 18 = -16`.

Infix: $(8 / 2^2) - ((4 + 2) \times 3) = 2 - 18 = -16$ → option (C).

- (A) 2 is just the left operand `8 / 4`.
- (B) 16 uses the postfix pop order for the final `-` (`18 - 2`).

- (D) 17.5 applies the postfix pop order to *every* operator: $4 / 8 = 0.5$ and $18 - 0.5$.

Trap: in prefix the operator's left operand is the one nearer to it, which ends up on **top** of the stack.

Q.4 [MCQ · 1 mark · Linked lists — reversing in groups of k]

Answer: (A)

The loop builds $1 \rightarrow 2 \rightarrow \dots \rightarrow 8$ (each new node is put in front, from 8 down to 1).

rev_k first checks whether $k = 3$ nodes remain; if not, the group is returned unchanged. Otherwise it recursively processes the rest, and then reverses the current k nodes, pointing the first one at the processed remainder (**prev** starts as that remainder).

- Group 1, 2, 3 $\rightarrow$ becomes $3 \rightarrow 2 \rightarrow 1 \rightarrow$ (rest).
- Group 4, 5, 6 $\rightarrow$ 6 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ (rest).
- Remaining 7, 8: only $2 < 3$ nodes $\rightarrow$ returned as is.

Output [3, 2, 1, 6, 5, 4, 7, 8] $\rightarrow$ option (A).

The tuple assignment **cur.nx, prev, cur = prev, cur, cur.nx** is safe: the right-hand side is evaluated completely first (old prev, old cur, old cur.nx), then the targets are assigned left to right, and **cur.nx** is assigned while **cur** still names the old node.

- (B) also reverses the short final group.
- (C) is a full reversal.
- (D) skips the first group.

Trap: reordering the targets as **cur, cur.nx, prev = ...** would set **.nx** on the *new* cur — a classic bug.

Q.5 [MSQ · 1 mark · Binary trees — counting facts]

Answer: (B), (C), (D)

Let n_0, n_1, n_2 be the numbers of nodes with 0, 1, 2 children. Counting edges two ways: $n - 1 = n_1 + 2n_2$ and $n = n_0 + n_1 + n_2 \Rightarrow \mathbf{n_0 = n_2 + 1}$.

- (A) **False.** A complete tree of height h has between 2^h and $2^{h+1} - 1$ nodes; $2^6 = 64 \leq 100 \leq 127$, so the height is $\lfloor \log_2 100 \rfloor = \mathbf{6}$.
- (B) **True.** n nodes have $2n$ child pointers, and exactly $n - 1$ of them are used (one per non-root node), leaving $2n - (n - 1) = n + 1$ empty.
- (C) **True.** $n_2 = n_0 - 1 = 11$ for *every* binary tree, regardless of n_1 .
- (D) **True.** Full $\Rightarrow n_1 = 0$, so $n_2 = 20 - 1 = 19$ and $n = 20 + 19 = 39$.

Answer: (D), (C), (B).

Trap: (A) uses the node-count definition of height (7 levels); with edges it is 6.

Q.6 [MCQ · 1 mark · Hashing — probability of no collision]

Answer: (D)

Count favourable outcomes: the first key may go anywhere, the second must avoid one slot, the third must avoid two.

$P(\text{no collision}) = (8/8) \times (7/8) \times (6/8) = 336 / 512 = 21/32 \approx \mathbf{0.656} \rightarrow$ option (D).

- (A) $0.670 = (7/8)^3$ treats the three *pairs* of keys as independent events — they are not (if $A \neq B$ and $B \neq C$, the chance that $A \neq C$ changes).
- (B) $0.875 = 7/8$ is the answer for only **two** keys.
- (C) $0.344 = 1 - 0.656$ is the probability of **at least one** collision.

This is the birthday problem in miniature: with m slots, collisions become likely after only about $\sqrt{\pi m/2}$ keys — the reason why hash tables must resolve collisions rather than hope to avoid them.

Tip: the expected number of colliding pairs is $C(3, 2)/8 = 0.375$ — linearity works even though the pair events are dependent.

Q.7 [NAT · 1 mark · Binary search — average successful search]

Answer: 9.00 to 9.02

For $n = 2^{10} - 1$ the binary-search decision tree is **perfect** with 10 levels: level d ($d = 1 \dots 10$) holds 2^{d-1} keys, each found with exactly d comparisons.

Total = $\sum d \cdot 2^{d-1}$ for $d = 1 \dots 10 = (10 - 1) \cdot 2^{10} + 1 = 9 \cdot 1024 + 1 = 9217$.

(Identity: $\sum_{d=1}^k d \cdot 2^{d-1} = (k - 1) \cdot 2^k + 1$.)

Average = $9217 / 1023 \approx 9.01$.

Note how close this is to the worst case (10): half of all keys sit on the bottom level, so the average is only about one comparison below the maximum, roughly $\log_2 n - 1$.

Trap: answering $\log_2 1023 \approx 9.998$, or $(1 + 10)/2 = 5.5$ by assuming every number of comparisons is equally likely.

Q.8 [MCQ · 1 mark · Sorting — stability of selection sort]

Answer: (B)

Trace (only swaps shown):

- $i = 0$: smallest key 1 at (1,s), index 3 $\rightarrow$ swap with (4,p): (1,s) (2,q) (4,r) (4,p) (2,t) (3,u).
- $i = 1$: smallest key 2, first occurrence (2,q) at index 1 $\rightarrow$ no change.
- $i = 2$: smallest key 2 is (2,t) at index 4 $\rightarrow$ swap with (4,r): (1,s) (2,q) (2,t) (4,p) (4,r) (3,u).
- $i = 3$: smallest key 3 at index 5 $\rightarrow$ swap with (4,p): (1,s) (2,q) (2,t) (3,u) (4,r) (4,p).
- $i = 4$: smallest is (4,r) (first of the two 4s) $\rightarrow$ no change.

Tags: **s q t u r p** $\rightarrow$ option (B).

The two records with key 4 came out as r, p although the input had p before r: the long-distance swap at $i = 0$ threw (4,p) behind (4,r). Selection sort is **not stable**.

- (A) is the stable order (what insertion or merge sort would give).
- (C) and (D) also reorder the key-2 records, which never happens here.

Trap: concluding stability from the 2s (q before t is preserved) — one counter-example pair is enough to make a sort unstable.

Q.9 [MSQ · 1 mark · Graphs — bipartiteness]

Answer: (C), (D)

A connected graph is bipartite iff a BFS/DFS 2-colouring never puts the same colour on both ends of an edge (equivalently, it has no odd cycle). Colour G from A:

- A: 0; B, F, G: 1 (neighbours of A); C, E: 0; D: 1; H (neighbour of D): 0.

An added edge keeps the graph bipartite iff its endpoints have **different** colours.

- (A) F-G: both 1 $\rightarrow$ triangle A-F-G $\rightarrow$ **not** bipartite.
- (B) A-C: both 0 $\rightarrow$ odd cycle A-B-C-A of length 3 $\rightarrow$ **not** bipartite.
- (C) B-E: colours 1 and 0 $\rightarrow$ **still bipartite** (it creates cycles B-C-D-E-B and B-A-F-E-B of length 4).
- (D) G-H: colours 1 and 0 $\rightarrow$ **still bipartite** (new cycle G-A-B-C-D-H-G has length 6).

Answer: (C), (D).

Trap: judging by the picture — B-E and A-C both look like chords of the hexagon, but B-E joins vertices at odd distance 3 (closing even cycles) while A-C joins vertices at even distance 2 (closing a triangle).

Q.10 [NAT · 1 mark · Loop counting — harmonic sums]**Answer: 66**

For a fixed i the inner loop visits $j = i, 2i, 3i, \dots \leq n$, i.e. $\lfloor n/i \rfloor$ values. So

$$c = \sum_{i=1}^{20} \lfloor 20/i \rfloor.$$

- $i = 1 \dots 5$: $20 + 10 + 6 + 5 + 4 = 45$.
- $i = 6$: 3; $i = 7 \dots 10$: 2 each $\rightarrow 3 + 8 = 11$.
- $i = 11 \dots 20$: 1 each $\rightarrow 10$.

Total = $45 + 11 + 10 = 66$.

Equivalently, c counts pairs (i, j) with $i \mid j$, i.e. $\sum_{j=1}^{20} d(j)$, the total number of divisors of $1 \dots 20$.

Complexity: $\sum n/i = n \cdot H_n \approx n \ln n$, so the fragment is $\Theta(n \log n)$ — the same harmonic-sum pattern as the sieve of Eratosthenes (which, skipping non-primes, is even $\Theta(n \log \log n)$).

Trap: assuming two nested loops over $1 \dots n$ must be $\Theta(n^2)$; the step size i makes the inner loop shrink like n/i .

Q.11 [MCQ · 2 marks · Python — sorted(): keys, case and stability]**Answer: (A)**

Three facts decide everything: (1) **sorted** is **stable** — records with equal keys keep their relative input order; (2) default string order compares code points, so every uppercase letter precedes every lowercase letter; (3) **reverse=True** reverses the order of keys but still keeps equal-key records stable (it does **not** simply reverse the output list).

- $r1$ (by length): lengths kiwi 4, Fig 3, apple 5, date 4, Banana 6, fig 3. Length 3 in input order: Fig, fig; then length 4: kiwi, date $\rightarrow r1[:3] = ['Fig', 'fig', 'kiwi']$.
- $r2$ (case-insensitive): apple, banana, date, fig, fig, kiwi $\rightarrow r2[:2] = ['apple', 'Banana']$.
- $r3$: **sorted(data)** = ['Banana', 'Fig', 'apple', 'date', 'fig', 'kiwi'] (uppercase first). Then by length descending, ties kept in this order: Banana(6), apple(5), date(4), kiwi(4), Fig(3), fig(3) $\rightarrow r3[2:5] = ['date', 'kiwi', 'Fig']$.

Output $\rightarrow$ option (A).

- (B) assumes **reverse=True** reverses tied elements too (as **sorted(...)[::-1]** would).
- (C) sorts the length-4 words alphabetically in $r1$ — but **key=len** ignores the letters.
- (D) uses case-sensitive order for $r2$.

Tip: multi-key sorting by successive stable sorts (secondary key first, primary key last) relies exactly on this stability.

Q.12 [NAT · 2 marks · Python — memoisation through a mutable default]**Answer: 26**

The default **memo={}** is created **once** and shared by all calls — including the second top-level call **T(10)**. Every call (even a memo hit) increments **calls**.

c1 — calls made by T(8) (memo initially empty):

- The first-child chain $T(8) \rightarrow T(7) \rightarrow T(6) \rightarrow T(5) \rightarrow T(4) \rightarrow T(3) \rightarrow T(2)$: 7 calls; $T(3)$ then calls $T(1)$ and $T(0)$: 2 more (all three bases get memoised) $\rightarrow 9$.
- Returning upward, each of $T(4), T(5), T(6), T(7), T(8)$ makes two more calls ($n-2, n-3$) which are memo hits $\rightarrow 5 \times 2 = 10$.
- $c1 = 9 + 10 = 19$. (Value $a = T(8) = 57$: 1, 1, 1, 3, 5, 9, 17, 31, 57.)

c2 — calls made by T(10) (memo holds 0 ... 8):

- $T(10)$ (1) $\rightarrow T(9)$ (1, new) $\rightarrow T(8), T(7), T(6)$ all hits (3) $\rightarrow T(9) = 105$.
- Back in $T(10)$: $T(8), T(7)$ hits (2) $\rightarrow b = 193$.

- $c2 = 1 + 1 + 3 + 2 = 7$.

$$c1 + c2 = 19 + 7 = 26.$$

Traps: assuming the memo is reset between top-level calls ($c2$ would be 25), or counting only cache misses ($c1 = 9$, $c2 = 2$).

Q.13 [MSQ · 2 marks · BST — deletion with two children]

Answer: (B), (D)

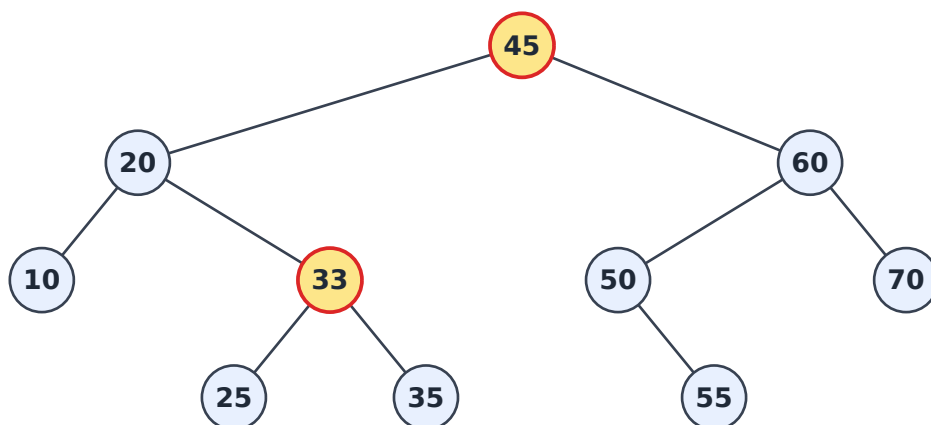
Delete 40 (two children): successor = minimum of the right subtree = 45 (leaf, left child of 50). Copy 45 into the root and remove the leaf 45 → 50 keeps only its right child 55.

Delete 30 (two children 25, 35): successor = minimum of 30's right subtree = 33 (left child of 35, a leaf). Copy 33 into the node and remove leaf 33 → 35 becomes a leaf.

Final tree: 45 → left 20 (10, 33 (25, 35)), right 60 (50 (–, 55), 70).

- (A) **False.** Leaves: 10, 25, 35, 55, 70 → 5.
- (B) **True.** Pre-order: 45, 20, 10, 33, 25, 35, 60, 50, 55, 70.
- (C) **False.** The longest paths are 45→20→33→25/35 and 45→60→50→55: 3 edges. (The original height 4 came from 40→20→30→35→33, and 33 has been moved up.)
- (D) **True.** The predecessor of 40 is the maximum of the left subtree: 20 → 30 → 35 (35 has no right child) → 35, so 35 would become the root (its left child 33 would move up).

Trap: taking the successor of 40 to be 50 (the right child) — the successor is the *leftmost* node of the right subtree.



After deleting 40 and then 30

Q.14 [NAT · 2 marks · Hashing — deletion with tombstones]

Answer: 7

Inserts: 22 → slot 0; 33 → 0 taken → 1; 44 → 0, 1 → 2; 13 (home 2) → 2 → 3; 24 (home 2) → 2, 3 → 4.

Deletes: 33 at slot 1 → DEL; 13 at slot 3 → DEL. Table: 0:22, 1:DEL, 2:44, 3:DEL, 4:24.

Insert 55 (home 0): slot 0 occupied, slot 1 DEL → reuse slot 1.

Insert 46 (home 2): slot 2 occupied, slot 3 DEL → reuse slot 3.

Table: 0:22, 1:55, 2:44, 3:46, 4:24, others EMPTY.

Search 24 (home 2): slots 2 (44), 3 (46), 4 (24) → found, **3** probes.

Search 35 (home 2, absent): slots 2, 3, 4, 5 (EMPTY) → **4** probes.

$$\text{Total} = 3 + 4 = 7.$$

Why tombstones matter: had the deletions simply emptied slots 1 and 3, a search for 24 *before* the re-insertions would have stopped at the empty slot 3 and wrongly reported 24 as absent.

Trap: forgetting that 55 and 46 recycle the tombstones — they would then land in slots 5 and 6, and the two searches would cost $3 + 6 = 9$ probes.

0	22
1	DEL
2	44
3	DEL
4	24
5	
6	
7	
8	
9	
10	

After the two deletions

0	22
1	55
2	44
3	46
4	24
5	
6	
7	
8	
9	
10	

Final table (tombstones reused)

Q.15 [MCQ · 2 marks · Quicksort — Hoare partition]

Answer: (C)

Hoare's scheme moves i right past elements $< p$ and j left past elements $> p$, then swaps; elements **equal** to the pivot stop both scans. It returns j such that $A[\text{lo}..j] \leq p \leq A[j+1..hi]$ — the pivot is **not** necessarily in its final position.

$p = 26$.

- $i \rightarrow 0$ ($A[0] = 26$ is not < 26). $j: 52 > 26 \rightarrow$ skip, stops at 6 (18). $i < j \rightarrow$ swap $A[0], A[6] \rightarrow [18, 41, 13, 26, 9, 37, 26, 52]$.
- $i \rightarrow 1$ (41 stops). $j: 37 > 26 \rightarrow$ skip, stops at 4 (9). Swap $\rightarrow [18, 9, 13, 26, 41, 37, 26, 52]$.
- $i \rightarrow 2$ ($13 < 26$), 3 (26 stops). $j \rightarrow 3$ (26 is not > 26). $i \geq j \rightarrow$ **return 3**.

Result: returns 3, $A = [18, 9, 13, 26, 41, 37, 26, 52] \rightarrow$ option (C). Quicksort then recurses on $A[0..3]$ and $A[4..7]$ (note the pivot value 26 appears on *both* sides).

- (A) is what a Lomuto-style partition that places the pivot would suggest — Hoare does not.
- (B) returns $i - 1$ instead of j .
- (D) performs an extra swap of 41 and 37, which never happens.

Trap: with Hoare partition the recursive calls must be on $[lo, j]$ and $[j + 1, hi]$ (not $j - 1$), otherwise elements can be lost or the recursion may not shrink.

Q.16 [MSQ · 2 marks · Shortest and longest paths in a DAG]

Answer: (B), (C), (D)

In a DAG, shortest **and** longest paths are found by relaxing edges in topological order (negative weights are fine). Here $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow T$ is a Hamiltonian path, so the topological order is forced: **S, A, B, C, D, T**.

Shortest distances in that order: $A = 3$; $B = \min(6, 3+2) = 5$; $C = \min(3+7, 5-4) = 1$; $D = \min(5+4, 1+1) = 2$; $T = \min(1+5, 2+2) = 4$.

Longest distances: $A = 3$; $B = \max(6, 5) = 6$; $C = \max(10, 2) = 10$; $D = \max(10, 11) = 11$; $T = \max(10+5, 11+2) = 15$.

- (A) **False** — exactly one topological order.
- (B) **True**, surprisingly. Dijkstra: extract $S \rightarrow A = 3$, $B = 6$; extract $A \rightarrow B = 5$, $C = 10$; extract $B \rightarrow C = 1$, $D = 9$; now the smallest label is $C (1) \rightarrow D = 2$, $T = 6$; extract $D \rightarrow T = 4$; extract T . The negative edge $B \rightarrow C$ is relaxed **before** C is extracted, so nothing goes wrong. Negative edges *can* break Dijkstra, but they do not have to.
- (C) **True** — 15 via $S \rightarrow A \rightarrow C \rightarrow T (3 + 7 + 5)$.
- (D) **True** — 4 via $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow T (3 + 2 - 4 + 1 + 2)$.

Trap: rejecting (B) by reflex. Dijkstra fails only when a vertex is extracted before a cheaper path through a negative edge reaches it.

Q.17 [NAT · 2 marks · Divide and conquer — inversion counting]

Answer: 16

This is merge sort that counts **inversions** (pairs $i < j$ with $a[i] > a[j]$): when $R[j]$ is output before the remaining $L[i...]$, it forms an inversion with each of those $\text{len}(L) - i$ elements. Because the test is $L[i] \leq R[j]$, **equal** keys are not counted — exactly as the strict definition requires.

Count by levels for $[5, 3, 5, 1 \mid 3, 5, 2, 2]$:

- Pairs: $[5,3] \rightarrow 1$; $[5,1] \rightarrow 1$; $[3,5] \rightarrow 0$; $[2,2] \rightarrow 0$ (equal). Subtotal 2.
- Merge $[3,5]+[1,5]$: 1 jumps over 3, $5 \rightarrow 2$. Merge $[3,5]+[2,2]$: each 2 jumps over 3, $5 \rightarrow 4$. Subtotal 6.
- Final merge $[1,3,5,5] + [2,2,3,5]$: each 2 jumps over 3, $5, 5 \rightarrow 3 + 3$; 3 jumps over 5, $5 \rightarrow 2$; the right 5 does not jump over the left 5s (equal). Subtotal 8.

Total = $2 + 6 + 8 = 16$.

Direct check: for each element count the smaller elements to its right — $5:5, 3:3, 5:4, 1:0, 3:2, 5:2, 2:0, 2:0 \rightarrow 16$.

Time: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$, versus $\Theta(n^2)$ for checking all pairs.

Trap: counting equal pairs (using $<$ instead of $\leq$ would also count 5-5, 3-3 and 2-2 crossings) or adding only 1 per jump instead of $\text{len}(L) - i$.

Inversions counted at each merge level

	merged runs	count
level 1	35 15 35 22	$1+1+0+0 = 2$
level 2	1355 2235	$2+4 = 6$
level 3	12233555	$3+3+2 = 8$

Q.18 [MCQ · 2 marks · Heaps — top-k of a stream]**Answer: (D)**

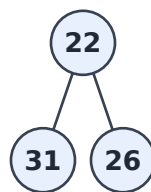
A min-heap of size k holds the k largest values seen so far; its root is the k -th largest, the threshold a new value must beat. Each step costs $O(\log k)$.

- $15 \rightarrow [15]$; $4 \rightarrow$ append, sift up $\rightarrow [4, 15]$; $22 \rightarrow [4, 15, 22]$.
- $9 > 4 \rightarrow$ root $:= 9 \rightarrow$ children 15, 22 $\rightarrow$ stays $\rightarrow [9, 15, 22]$.
- $31 > 9 \rightarrow$ root $:= 31 \rightarrow$ smaller child 15 $\rightarrow$ swap $\rightarrow [15, 31, 22]$.
- $7 < 15 \rightarrow$ discard.
- $18 > 15 \rightarrow$ root $:= 18 \rightarrow 18 < 31$, 22 $\rightarrow$ stays $\rightarrow [18, 31, 22]$.
- $26 > 18 \rightarrow$ root $:= 26 \rightarrow$ smaller child 22 $\rightarrow$ swap $\rightarrow [22, 31, 26]$.
- $3 < 22 \rightarrow$ discard.

Final **[22, 31, 26]** $\rightarrow$ option (D); the three largest values are 31, 26, 22 and the root 22 is the 3rd largest.

- (A) is the sorted order — a heap array need not be sorted.
- (B) forgets to sift 26 down in the last replacement.
- (C) keeps 18 — i.e. ignores that 26 beats the root.

Trap: using a *max*-heap for ‘ k largest’ — the min-heap is what lets the smallest of the kept values be evicted in $O(\log k)$.



Final min-heap

Q.19 [NAT · 2 marks · Graph theory — maximum edges with components]**Answer: 42**

Within each component the edge count is maximised by making it complete, so with component sizes $a + b + c = 15$ the maximum is $C(a, 2) + C(b, 2) + C(c, 2)$. Since $C(x, 2)$ is convex, the sum is largest when the sizes are as **unbalanced** as allowed.

- Without the size restriction: $1 + 1 + 13 \rightarrow C(13, 2) = 78$ (the classic answer, $(n - k)(n - k + 1)/2$).
- With every component ≥ 3 : make two components as small as possible: $3 + 3 + 9 \rightarrow 3 + 3 + C(9, 2) = 3 + 3 + 36 = 42$.

Check other splits: $3 + 4 + 8 \rightarrow 3 + 6 + 28 = 37$; $3 + 5 + 7 \rightarrow 3 + 10 + 21 = 34$; $5 + 5 + 5 \rightarrow 30$. Moving a vertex from a smaller to the largest component never decreases the total, so 42 is the maximum.

Trap: answering 78 by applying the textbook formula without the extra constraint, or choosing the balanced split (30) by intuition.

Q.20 [MSQ · 2 marks · Binary search — counting occurrences]**Answer: (A), (B), (C)**

`first_ge` is a lower-bound search on the half-open range $[lo, hi)$: it returns the first index whose element is $\geq x$ (len(a) if none).

- (A) **True.** The range size $s = hi - lo$ shrinks to at most $\lfloor s/2 \rfloor$ each iteration (either $[lo, mid)$ or $[mid + 1, hi)$). Starting from 1000: $1000 \rightarrow 500 \rightarrow 250 \rightarrow 125 \rightarrow 62 \rightarrow 31 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 0$ is

the slowest possible chain — exactly 10 iterations; in general $\lfloor \log_2 n \rfloor + 1$.

- (B) **True.** $lo = hi = 0 \rightarrow$ loop skipped $\rightarrow 0$.
- (C) **True.** $first_ge(a, 3) = 4$ (element 5) and $first_ge(a, 2) = 1 \rightarrow 4 - 1 = 3$.
- (D) **False.** $x + 1$ is the next value only for **integers**. For $a = [1.5, 2.0, 2.5]$ and $x = 2.0$: $first_ge(a, 3.0) = 3$ and $first_ge(a, 2.0) = 1$, giving 2 although 2.0 occurs once (2.5 was counted). The robust version uses an upper-bound search (first index with element $> x$) instead of $x + 1$.

Answer: (C), (B), (A).

Trap: (D) works on every integer test you try — the bug only appears when values can lie strictly between x and $x + 1$.

Mock Test 50

Challenge Mock — Paper 5

Difficulty: Hard

Focus areas: hardest, multi-concept, trap-heavy 2-mark questions

Questions	Total marks	Time	1-mark Qs	2-mark Qs	MCQ / MSQ / NAT
20	30	54 min	10	10	7 / 5 / 8

Marking: MCQ — +1/+2 for correct, -1/3 (1-mark) or -2/3 (2-mark) for wrong. MSQ and NAT — no negative marking; MSQ needs exactly all correct options. Programs are in Python 3 unless stated otherwise.

Q.1 [MCQ · single correct | 1 mark | Python — list replication and in-place +=]

Consider the following Python program. What is printed?

Python

```
1 grid = [[0] * 3] * 3
2 grid[0][1] = 5
3 row = grid[1]
4 row += [7]
5 print(grid[2], len(grid[0]))
```

- (A) [0, 0, 0] 3
- (B) [0, 5, 0] 3
- (C) [0, 5, 0, 7] 4
- (D) [0, 5, 0] 4

Q.2 [NAT · numerical answer | 1 mark | Python — closures with nonlocal state]

What value does the following program print?

Python

```
1 def make():
2     c = 0
3     def inc(k=1):
4         nonlocal c
5         c += k
6         return c
7     return inc
8
9 a, b = make(), make()
10 x = a() + a(3) + b(10) + a()
11 print(x)
```

Answer: _____ (enter a numerical value)

Q.3 [NAT · numerical answer | 1 mark | Amortised analysis — array-backed stack]

A stack is stored in a dynamic array whose initial capacity is 1. When a push finds the array full, a new array of **twice** the capacity is allocated and all current elements are copied into it before the push proceeds. Starting from an empty stack, 100 elements are pushed (no pops). The total number of element copies caused by resizing is _____.

Answer: _____ (enter a numerical value)

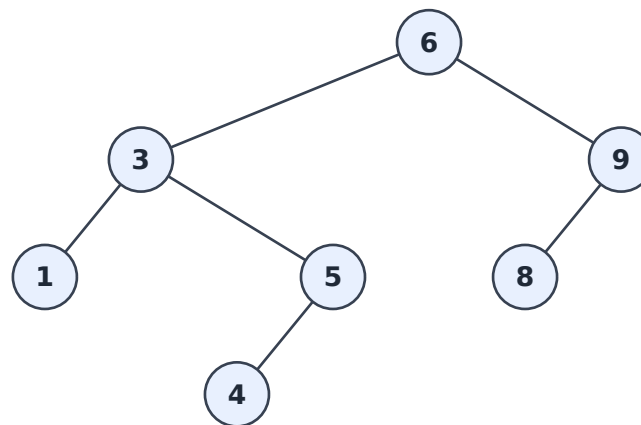
Q.4 [MCQ · single correct | 1 mark | Heaps — where can the k-th smallest be?]

A binary min-heap stores 31 distinct keys in a complete binary tree (root at level 0, leaves at level 4). At which levels can the **4th smallest** key be located (over all possible such heaps)?

- (A) Levels 1, 2 and 3 only
- (B) Levels 2 and 3 only
- (C) Levels 1, 2, 3 and 4
- (D) Level 3 only

Q.5 [MSQ · one or more correct | 1 mark | BSTs — insertion orders giving the same tree]

The sequence 6, 3, 9, 1, 5, 8, 4 is inserted into an empty BST (no rebalancing), giving the tree T shown. Which of the following insertion sequences produce **exactly the same tree T**?



Tree T

- (A) 6, 8, 9, 3, 5, 1, 4
- (B) 6, 3, 1, 9, 4, 5, 8
- (C) 6, 9, 3, 8, 1, 5, 4
- (D) 6, 3, 5, 4, 9, 1, 8

Q.6 [MCQ · single correct | 1 mark | Binary search — termination bug]

The following binary search contains a bug. For $a = [2, 4, 6, 8, 10]$, which of the following calls **terminates**?

Python

```

1 def search(a, x):
2     lo, hi = 0, len(a) - 1
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         if a[mid] == x:
6             return mid
7         if a[mid] < x:
8             lo = mid
9         else:
10            hi = mid - 1
11    return -1
  
```

- (A) `search(a, 4)`
- (B) `search(a, 1)`
- (C) `search(a, 7)`
- (D) `search(a, 10)`

Q.7 [NAT · numerical answer | 1 mark | Hashing — expected number of empty slots]

Six keys are inserted into a hash table with 10 slots using separate chaining. Assume simple uniform hashing: each key independently hashes to each slot with probability $1/10$. The expected number of slots that remain **empty** is _____ (rounded off to two decimal places).

Answer: _____ (enter a numerical value)

Q.8 [MSQ · one or more correct | 1 mark | Sorting — behaviour on all-equal keys]

An array of n keys that are **all equal** is sorted. Which of the following statements is/are TRUE? (Insertion sort shifts while $A[j] > \text{key}$; bubble sort has an early-exit flag; quicksort uses the stated partition scheme recursively.)

- (A) Bubble sort with early exit performs $n(n-1)/2$ comparisons
- (B) Quicksort with the Hoare partition (first element as pivot) performs $\Theta(n^2)$ comparisons
- (C) Quicksort with the Lomuto partition (last element as pivot, moving elements $\leq$ pivot left) performs $\Theta(n^2)$ comparisons
- (D) Insertion sort performs $\Theta(n)$ comparisons

Q.9 [MCQ · single correct | 1 mark | Graph theory — edges vs components]

The maximum number of edges in a simple undirected graph with 10 vertices and exactly 3 connected components is

- (A) 36
- (B) 28
- (C) 27
- (D) 21

Q.10 [NAT · numerical answer | 1 mark | Linked lists — Floyd's cycle detection]

A singly linked list has nodes with values 11, 22, 33, 44, 55, 66, 77, 88, 99 in order, and the **next** pointer of the last node (99) points back to the node 55. The function below is called with the head (11). What is the value of **s1 + s2** it returns (as the first component)?

Python

```
1 def floyd(head):
2     slow = fast = head
3     s1 = 0
4     while True:
5         slow, fast = slow.next, fast.next.next
6         s1 += 1
7         if slow is fast:
8             break
9     p, s2 = head, 0
10    while p is not slow:
11        p, slow = p.next, slow.next
12        s2 += 1
13    return s1 + s2, p.v
```



The last node's next points back to 55 (not drawn)

Answer: _____ (enter a numerical value)

Q.11 [MCQ · single correct | 2 marks | Python — class vs instance attributes, inheritance]

Consider the following Python program. What is printed?

Python

```

1 class A:
2     tags = []
3     n = 0
4     def __init__(self, t):
5         self.tags.append(t)
6         self.n += 1
7
8 class B(A):
9     tags = []
10    def __init__(self, t):
11        super().__init__(t)
12        self.tags = self.tags + [t * 2]
13
14 x, y, z = A(1), B(2), B(3)
15 print(A.tags, B.tags, z.tags, A.n + z.n)

```

- (A) [1, 2, 3] [2, 3] [2, 3, 6] 3
 (B) [1] [2, 3, 4, 6] [2, 3, 6] 1
 (C) [1] [2, 3] [2, 3, 6] 3
 (D) [1] [2, 3] [2, 3, 6] 1

Q.12 [NAT · numerical answer | 2 marks | Monotonic deque — sliding-window maximum]

The function below returns the maximum of every window of k consecutive elements using a deque of indices. For the array shown and $k = 3$, the sum of all the window maxima returned is _____.

Python

```

1 from collections import deque
2 def win_max(a, k):
3     dq, out = deque(), []
4     for i, x in enumerate(a):
5         while dq and a[dq[-1]] <= x:
6             dq.pop()
7         dq.append(i)
8         if dq[0] <= i - k:
9             dq.popleft()
10        if i >= k - 1:
11            out.append(a[dq[0]])
12    return out

```

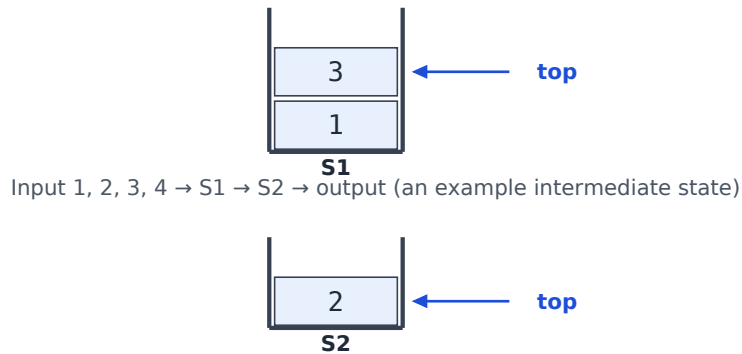
a =

4	2	12	3	8	5	1	9	6	7
0	1	2	3	4	5	6	7	8	9

Answer: _____ (enter a numerical value)

Q.13 [MSQ · one or more correct | 2 marks | Stack permutations — one stack vs two stacks in series]

The input 1, 2, 3, 4 (in this order) must be output as a permutation. In the **single stack** model each input value is pushed onto S and values are popped from S to the output. In the **two stacks in series** model the allowed moves are: push the next input onto S1; move the top of S1 onto S2; pop the top of S2 to the output. Which of the following statements is/are TRUE?



- (A) Every permutation of 1, 2, 3, 4 can be produced with two stacks in series
- (B) Exactly 14 permutations of 1, 2, 3, 4 can be produced with a single stack
- (C) The output 2 4 3 1 can be produced with a single stack
- (D) The output 4 1 3 2 can be produced with a single stack

Q.14 [NAT · numerical answer | 2 marks | Quicksort — counting worst-case inputs]

Quicksort with the Lomuto partition (last element as pivot; every element of the current range other than the pivot is compared with the pivot exactly once) is run on each of the 120 permutations of 1, 2, 3, 4, 5. The maximum possible total number of comparisons is 10. On how many of the 120 permutations is this maximum attained?

Python

```

1 def quicksort(A, lo, hi):
2     if lo < hi:
3         p, i = A[hi], lo - 1
4         for j in range(lo, hi):
5             if A[j] <= p:          # one comparison
6                 i += 1
7                 A[i], A[j] = A[j], A[i]
8         A[i + 1], A[hi] = A[hi], A[i + 1]
9         quicksort(A, lo, i)
10        quicksort(A, i + 2, hi)

```

Answer: _____ (enter a numerical value)

Q.15 [MCQ · single correct | 2 marks | Complexity — dependent logarithmic loops]

For n a power of 2, what is the order of the value returned by $f(n)$ (equivalently, its running time)?

Python

```

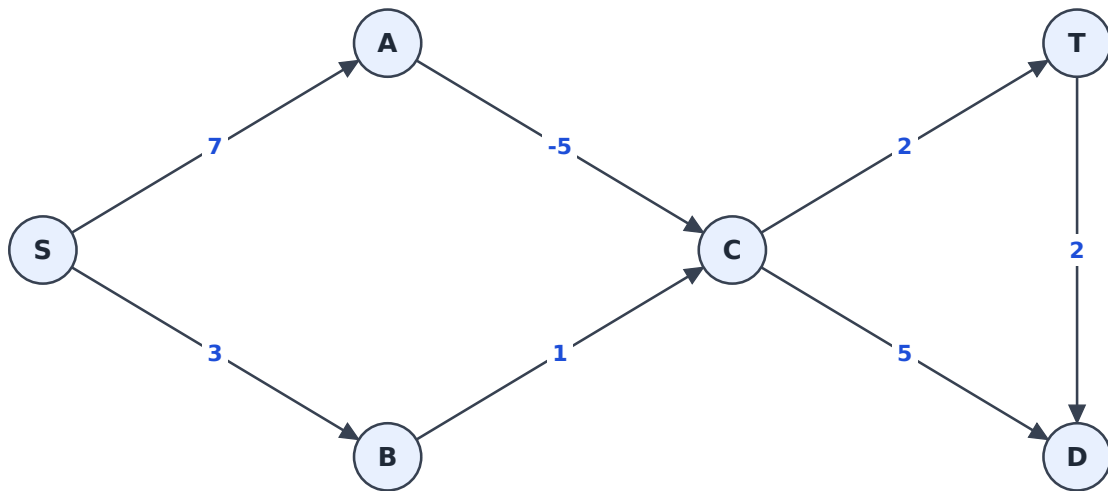
1 def f(n):
2     c = 0
3     i = n
4     while i > 1:
5         j = i
6         while j < n:
7             j *= 2
8             c += 1
9         i //= 2
10    return c

```

- (A) $\Theta(\log^2 n)$
- (B) $\Theta(\log n)$
- (C) $\Theta(n)$
- (D) $\Theta(n \log n)$

Q.16 [NAT · numerical answer | 2 marks | Dijkstra with a negative edge]

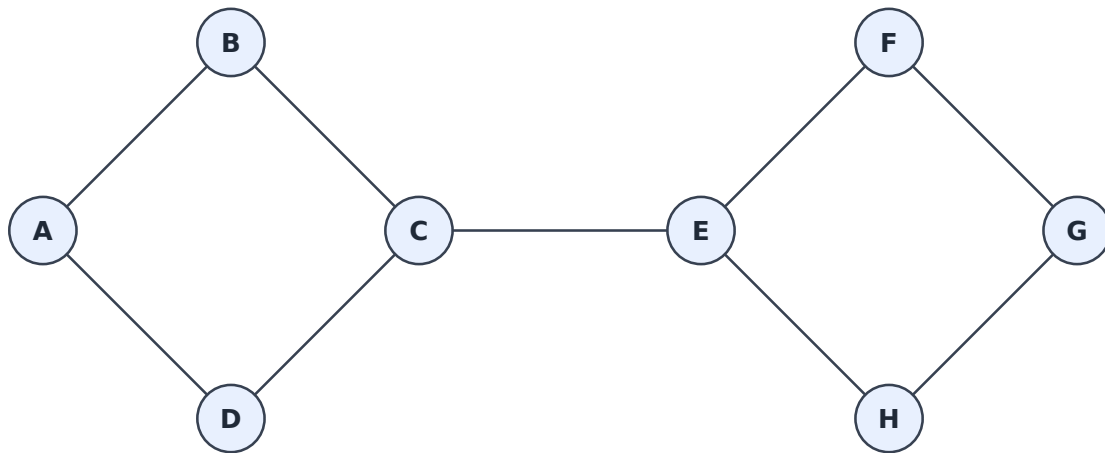
Dijkstra's algorithm is run from S on the directed graph below, which has one negative edge but no negative cycle. The implementation marks a vertex *final* when it is extracted (minimum d among non-final vertices) and relaxes an edge (u, v) only if v is not yet final. The number of vertices whose final d -value differs from the true shortest-path distance is _____.



Answer: _____ (enter a numerical value)

Q.17 [MSQ · one or more correct | 2 marks | Graph structure — bipartiteness, cut vertices, BFS levels]

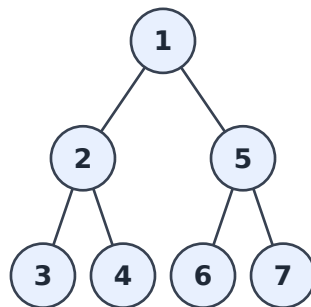
Consider the undirected graph below. Which of the following statements is/are TRUE?



- (A) Every spanning tree of the graph contains the edge C-E
- (B) The graph is bipartite
- (C) A BFS from A assigns vertices to exactly 6 distinct levels (level 0 to level 5)
- (D) The graph has exactly one cut vertex

Q.18 [NAT · numerical answer | 2 marks | Heaps — counting heaps with a constraint]

Consider all array-based binary **min-heaps** that store exactly the keys 1, 2, ..., 7 (so the tree is a perfect binary tree of height 2). In how many of them is key 2 the **left** child of the root?



One such heap (2 is the left child of the root)

Answer: _____ (enter a numerical value)

Q.19 [MCQ · single correct | 2 marks | Python — dict ordering, update and truthiness]

Consider the following Python program. What is printed?

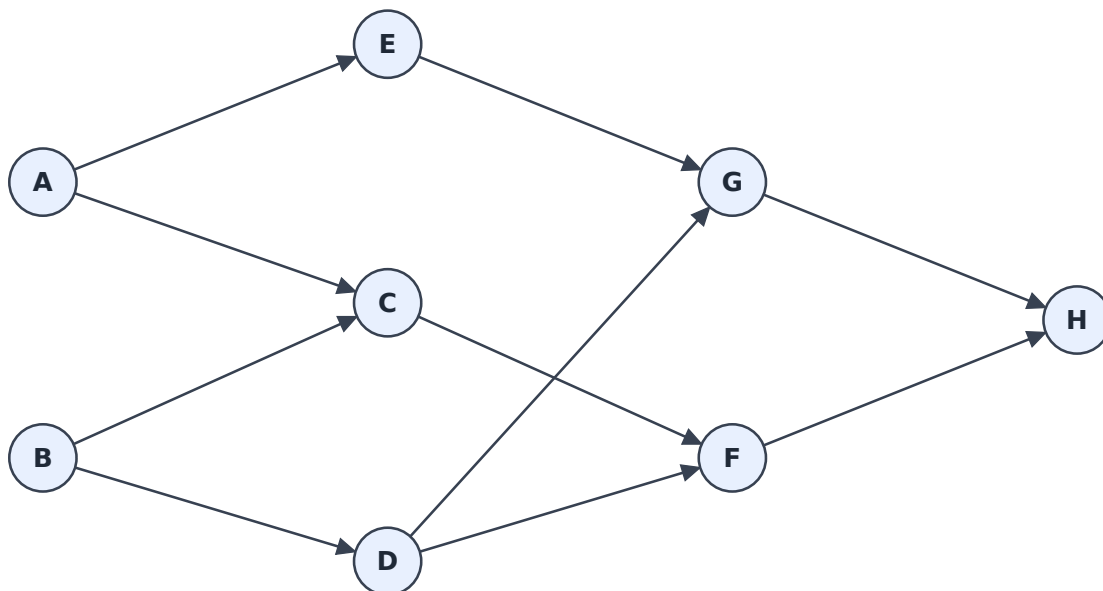
Python

```
1 d = {'b': 1, 'a': 2, 'c': 3}
2 d['a'] = 5
3 del d['b']
4 d['b'] = 7
5 d.update(c=0, e=9)
6 k = [x for x in d if d[x]]
7 print(k, sorted(d, key=d.get)[0], list(d.items())[1])
```

- (A) ['a', 'c', 'b', 'e'] c ('c', 0)
- (B) ['b', 'a', 'e'] c ('a', 5)
- (C) ['a', 'b', 'e'] c ('c', 0)
- (D) ['a', 'b', 'e'] c ('b', 7)

Q.20 [MSQ · one or more correct | 2 marks | DAGs — topological orders and Kahn's algorithm]

Consider the directed acyclic graph below. Which of the following statements is/are TRUE?



- (A) B D A G C E F H is a valid topological order
- (B) H is the last vertex in every topological order
- (C) Kahn's algorithm that always removes the alphabetically smallest vertex of in-degree 0 outputs A B C D E F G H
- (D) A precedes D in every topological order

Answer Key — Mock Test 50

Q	Type	Marks	Key	Topic
1	MCQ	1	(C)	Python — list replication and in-place +=
2	NAT	1	20	Python — closures with nonlocal state
3	NAT	1	127	Amortised analysis — array-backed stack
4	MCQ	1	(A)	Heaps — where can the k-th smallest be?
5	MSQ	1	(C), (D)	BSTs — insertion orders giving the same tree
6	MCQ	1	(B)	Binary search — termination bug
7	NAT	1	5.30 to 5.32	Hashing — expected number of empty slots
8	MSQ	1	(C), (D)	Sorting — behaviour on all-equal keys
9	MCQ	1	(B)	Graph theory — edges vs components
10	NAT	1	9	Linked lists — Floyd's cycle detection
11	MCQ	2	(D)	Python — class vs instance attributes, inheritance
12	NAT	2	79	Monotonic deque — sliding-window maximum
13	MSQ	2	(A), (B), (C)	Stack permutations — one stack vs two stacks in series
14	NAT	2	16	Quicksort — counting worst-case inputs
15	MCQ	2	(A)	Complexity — dependent logarithmic loops
16	NAT	2	3	Dijkstra with a negative edge
17	MSQ	2	(A), (B), (C)	Graph structure — bipartiteness, cut vertices, BFS levels
18	NAT	2	40	Heaps — counting heaps with a constraint
19	MCQ	2	(C)	Python — dict ordering, update and truthiness
20	MSQ	2	(B), (C)	DAGs — topological orders and Kahn's algorithm

Detailed Solutions — Mock Test 50

Q.1 [MCQ · 1 mark · Python — list replication and in-place +=]

Answer: (C)

Concept. `[[0] * 3] * 3` builds an outer list holding **three references to the same inner list**. For lists, `row += [7]` is `row.extend([7])` — it mutates the object in place rather than rebinding to a new list.

- `grid[0][1] = 5` changes the single shared row → every “row” shows `[0, 5, 0]`.
- `row = grid[1]` is yet another name for that same list.
- `row += [7]` extends it in place → `[0, 5, 0, 7]`, visible through `grid[0]`, `grid[1]`, `grid[2]`.

So `grid[2]` is `[0, 5, 0, 7]` and `len(grid[0])` is 4 → (C).

Options. (A) assumes independent rows. (B) assumes `+=` creates a new list (true for tuples and strings, not lists). (D) mixes the two: aliasing for the assignment but not for `+=`.

Trap: use `[[0] * 3 for _ in range(3)]` to obtain independent rows.

Q.2 [NAT · 1 mark · Python — closures with nonlocal state]

Answer: 20

Concept. Each call of `make()` creates a fresh local `c` and a fresh closure bound to it; `nonlocal` lets `inc` rebind that captured variable. So `a` and `b` have *independent* counters, and the state persists between calls of the same closure.

Evaluated left to right:

- `a()` → `a`'s `c = 1` → returns **1**
- `a(3)` → `a`'s `c = 4` → **4**
- `b(10)` → `b`'s `c = 10` → **10**
- `a()` → `a`'s `c = 5` → **5**

`x = 1 + 4 + 10 + 5 = 20`.

Trap: assuming `a` and `b` share one counter (giving `1 + 4 + 14 + 15 = 34`), or that `c` resets to 0 on every call (giving `1 + 3 + 10 + 1 = 15`). Without `nonlocal`, `c += k` would raise `UnboundLocalError`.

Q.3 [NAT · 1 mark · Amortised analysis — array-backed stack]

Answer: 127

Concept. A resize happens when the stack holds exactly as many elements as the capacity; it copies all of them. Capacities go `1 → 2 → 4 → ...`, so resizes occur at sizes 1, 2, 4, 8, 16, 32, 64.

- Push #2 finds 1 element (cap 1) → copy 1, cap 2.
- Push #3: copy 2, cap 4. Push #5: copy 4, cap 8. Push #9: copy 8, cap 16.
- Push #17: copy 16, cap 32. Push #33: copy 32, cap 64. Push #65: copy 64, cap 128.
- Pushes 66–100 fit in capacity 128.

Total copies = `1 + 2 + 4 + 8 + 16 + 32 + 64 = 127` (`< 2·100`), so each push costs $O(1)$ amortised.

Trap: including a resize at 128 (no — only 100 elements are pushed) or counting the 100 ordinary writes as copies.

Q.4 [MCQ · 1 mark · Heaps — where can the k-th smallest be?]

Answer: (A)

Concept. Every ancestor of a key in a min-heap is smaller than it. The 4th smallest has at most 3 smaller keys, so it has at most 3 ancestors → level ≤ 3 . It cannot be the root (that is the minimum), so its level is ≥ 1 .

All three levels are achievable:

- **Level 1:** root 1, children 2 and 4, with 3 below 2 → 4 is a child of the root.
- **Level 2:** e.g. the chain 1 → 2 → 4 with 3 as the other child of the root.
- **Level 3:** the chain 1 → 2 → 3 → 4 down the leftmost path.

Level 4 is impossible (it would need 4 smaller ancestors). Answer → (A).

Options. (B) wrongly excludes level 1 (see the first example). (C) includes level 4. (D) confuses “can be” with “must be”.

Tip: the k -th smallest lies at a level between 1 and $k - 1$ (for $k \geq 2$); hence finding it only requires inspecting the top $k - 1$ levels.

Q.5 [MSQ · 1 mark · BSTs — insertion orders giving the same tree]

Answer: (C), (D)

Concept. A sequence produces T iff every node is inserted **after all of its ancestors** in T . Unrelated subtrees may be interleaved freely.

Ancestor constraints in T : 6 first; 3 and 9 after 6; 1 and 5 after 3; 4 after 5; 8 after 9.

- (A) 8 is inserted before 9 → 8 becomes the right child of 6. **No.**
- (B) 4 comes before 5 → 4 becomes the right child of 3 and 5 then goes under 4. **No.**
- (C) 6, 9, 3, 8, 1, 5, 4: all constraints hold (8 after 9, 4 after 5). **Yes.**
- (D) 6, 3, 5, 4, 9, 1, 8: 5 after 3, 4 after 5, 1 after 3, 8 after 9. **Yes.**

(For the record, the number of insertion orders producing T is $C(6, 2) \times 3 = 45$: choose the positions of the right subtree $\{9, 8\}$ among the 6 non-root slots, and interleave $\{1\}$ with the chain $5 \rightarrow 4$ after 3 in 3 ways.)

Trap: checking only that each element lands on the correct *side* of the root — the constraint must hold at every level.

Q.6 [MCQ · 1 mark · Binary search — termination bug]

Answer: (B)

Concept. $lo = mid$ (instead of $mid + 1$) makes no progress when $hi = lo$ or $hi = lo + 1$, because then $mid = lo$. If at that moment $a[mid] < x$, the loop repeats forever.

- (A) $x = 4$: $mid = 2$ ($6 > 4$) → $hi = 1$; $mid = 0$ ($2 < 4$) → $lo = 0$ → $mid = 0$ again ... **infinite**, even though 4 is present (it sits at index $1 = hi$, never reached as mid).
- (B) $x = 1$: $mid = 2$ ($6 > 1$) → $hi = 1$; $mid = 0$ ($2 > 1$) → $hi = -1$; loop ends → returns -1 . **Terminates.**
- (C) $x = 7$: $lo \rightarrow 2$ ($6 < 7$); $mid = 3$ ($8 > 7$) → $hi = 2$; $mid = 2$ ($6 < 7$) → $lo = 2$... **infinite**.
- (D) $x = 10$: $lo \rightarrow 2$, then $lo \rightarrow 3$; with $(lo, hi) = (3, 4)$, $mid = 3$ and $8 < 10$ forever. **Infinite.**

Trap: the bug is only triggered on the right-moving branch, so searches that keep going left (like $x = 1$) or hit the key at a probed mid (6, 8, 2) still work — testing a few lucky inputs does not expose it.

Q.7 [NAT · 1 mark · Hashing — expected number of empty slots]

Answer: 5.30 to 5.32

Concept. Use linearity of expectation with an indicator per slot: $I_s = 1$ if slot s is empty after all insertions.

- $P(\text{a given key avoids slot } s) = 9/10$.
- Keys are independent, so $P(\text{slot } s \text{ empty}) = (9/10)^6 = 0.531441$.
- $E[\text{number of empty slots}] = \sum_s P(I_s = 1) = 10 \times 0.531441 = \mathbf{5.31441} \approx \mathbf{5.31}$.

(Equivalently, the expected number of *occupied* slots is $10 - 5.31 = 4.69 < 6$, reflecting collisions; the expected number of colliding pairs is $C(6, 2)/10 = 1.5$.)

Trap: answering $10 - 6 = 4$ assumes no collisions; using $(1 - 6/10) \times 10 = 4$ makes the same mistake. Indicators avoid any need for the (dependent) joint distribution.

Q.8 [MSQ · 1 mark · Sorting — behaviour on all-equal keys]

Answer: (C), (D)

- (A) The first pass makes no swap, so the algorithm stops after $n - 1$ comparisons. **FALSE.**
- (B) Hoare: both scans stop at every element equal to the pivot, so i and j advance in lock-step and meet in the middle — each partition splits the range roughly in half $\rightarrow \Theta(n \log n)$, not $\Theta(n^2)$. **FALSE.**
- (C) Lomuto: every element satisfies $\leq$ pivot, so the pivot ends at the right end; the recursion continues on $n - 1$ elements, then $n - 2, \dots \rightarrow n(n - 1)/2 = \Theta(n^2)$. **TRUE.**
- (D) Each key compares once with its left neighbour, finds it not greater, and stops: $n - 1$ comparisons. **TRUE.**

Trap: “quicksort is quadratic on equal keys” is true for Lomuto but *not* for Hoare — the stop-on-equal rule is precisely what balances Hoare’s splits.

Q.9 [MCQ · 1 mark · Graph theory — edges vs components]

Answer: (B)

Concept. With the component sizes $n_1 + n_2 + n_3 = 10$ fixed, the edge count is maximised by making each component complete: $\sum C(n_i, 2)$. Since $C(x, 2)$ is convex, the sum is largest when the sizes are as **unequal** as possible: 8, 1, 1.

- Sizes (8, 1, 1): $C(8, 2) = 28$.
- Sizes (7, 2, 1): $21 + 1 = 22$. Sizes (4, 3, 3): $6 + 3 + 3 = 12$. All smaller.

General formula: $(n - k)(n - k + 1)/2 = 7 \cdot 8/2 = 28$ for $n = 10, k = 3 \rightarrow$ (B).

Options. (A) $36 = C(9, 2)$ is the answer for **2** components. (C) 27 subtracts one edge per extra component from 28 — no basis. (D) $21 = C(7, 2)$ uses $n - k = 7$ vertices in the big component, an off-by-one.

Trap: balancing the components (the intuition for *minimising* edges) is exactly wrong here.

Q.10 [NAT · 1 mark · Linked lists — Floyd’s cycle detection]

Answer: 9

Concept. Tail length $\mu = 4$ (11, 22, 33, 44 are outside the cycle) and cycle length $\lambda = 5$ ($55 \rightarrow 66 \rightarrow 77 \rightarrow 88 \rightarrow 99 \rightarrow 55$). After the meeting, moving one pointer from the head and one from the meeting point at equal speed makes them meet at the cycle start after exactly μ steps.

Phase 1 (positions as indices 0–8; index $8 \rightarrow 4$):

- step 1: slow 1, fast 2; step 2: slow 2, fast 4; step 3: slow 3, fast 6;
- step 4: slow 4, fast 8; step 5: slow 5, fast 4 $\rightarrow$ 5 ($8 \rightarrow 4 \rightarrow 5$). Meet at index 5 (66) $\rightarrow s_1 = 5$.

Phase 2: p from index 0, slow from 5 $\rightarrow$ after 4 steps $p = 4$ and slow = 5 $\rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 4$. Meet at $55 \rightarrow s_2 = 4 = \mu$.

Return value (9, 55); $s_1 + s_2 = 9$.

Check: the first meeting occurs after s_1 steps where s_1 is the smallest multiple of λ that is $\geq \mu \rightarrow s_1 = 5$.

Trap: assuming the pointers meet at the cycle start in phase 1 (they meet at 66, one step past it).

Q.11 [MCQ · 2 marks · Python — class vs instance attributes, inheritance]**Answer: (D)**

Concept. Attribute *lookup* on an instance falls back to the class (and its bases), but attribute *assignment* on an instance always creates/overwrites an instance attribute. Mutating a found class attribute (`append`) changes the class-level object.

- **A(1):** `self.tags` finds `A.tags` → append 1 → `A.tags = [1]`. `self.n += 1` reads `A.n` (0) and **creates** `x.n = 1`; `A.n` stays 0.
- **B(2):** inside `A.__init__`, `self.tags` now finds **`B.tags`** (B overrides it) → `B.tags = [2]`; `y.n = 1`. Then `self.tags = self.tags + [4]` builds a *new* list `[2, 4]` as `y`'s instance attribute — `B.tags` is unchanged.
- **B(3):** `self.tags` (no instance attribute yet) finds `B.tags` → `B.tags = [2, 3]`; then `z.tags = [2, 3] + [6] = [2, 3, 6]`; `z.n = 1`.

Printed: `A.tags = [1]`, `B.tags = [2, 3]`, `z.tags = [2, 3, 6]`, `A.n + z.n = 0 + 1 = 1` → (D).

Option by option:

- (A) assumes B shares A's list and that `n` is a shared counter.
- (B) assumes `self.tags = self.tags + ...` mutates the class list (it rebinds).
- (C) assumes `self.n += 1` increments the class attribute `A.n`.
- (D) Correct.

Trap: `self.x += 1` on an immutable class attribute silently creates an instance copy, while `self.lst.append(...)` on a mutable class attribute is shared by every instance.

Q.12 [NAT · 2 marks · Monotonic deque — sliding-window maximum]**Answer: 79**

Concept. The deque holds indices whose values are strictly decreasing from front to back; the front is the current window maximum. An index leaves from the back when a larger-or-equal value arrives and from the front when it slides out of the window. Each index enters and leaves at most once → $\Theta(n)$.

Trace (deque shown as values):

- `i=0..2`: 4; 2 → `[4, 2]`; 12 pops both → `[12]`. Window `[4, 2, 12]` → **12**.
- `i=3` (3): `[12, 3]` → **12**. `i=4` (8): pop 3 → `[12, 8]` → **12** (index 2 still inside).
- `i=5` (5): `[12, 8, 5]`; index $2 \leq 5 - 3$ → pop left 12 → `[8, 5]` → **8**.
- `i=6` (1): `[8, 5, 1]` → **8**.
- `i=7` (9): pops 1, 5, 8 → `[9]` → **9**.
- `i=8` (6): `[9, 6]` → **9**. `i=9` (7): pop 6 → `[9, 7]` → **9**.

Maxima: 12, 12, 12, 8, 8, 9, 9, 9 → sum = **79**.

Trap: forgetting to expire 12 at `i = 5` (giving 12 instead of 8 for the window `[3, 8, 5]`), or counting `n - k = 7` windows instead of `n - k + 1 = 8`.

Q.13 [MSQ · 2 marks · Stack permutations — one stack vs two stacks in series]**Answer: (A), (B), (C)**

Single stack. The achievable outputs of `1..n` are counted by the Catalan number $C_n = (2n)! / ((n+1)! n!)$, so for `n = 4` there are **14** → (B) **TRUE**. A permutation is achievable iff it avoids the pattern "3 1 2" (some `c ... a ... b` with `a < b < c`).

- (C) 2 4 3 1: push 1, 2 pop 2; push 3, 4 pop 4; pop 3; pop 1. **TRUE**.
- (D) 4 1 3 2: when 4 is output, 1, 2, 3 are on the stack with 3 on top, so 1 cannot come next (4 1 3 is a 312-pattern). **FALSE**.

Two stacks in series. `S2` can hold a value while `S1` keeps accepting input, which lets the machine undo the single-stack obstruction. For example, the 312-type output 4 1 3 2:

- push 1, push 2; move 2 → S2 (S1 = [1], S2 = [2])
- push 3, push 4; move 4 → S2 and output **4**
- move 3 → S2, move 1 → S2 (S2 = [2, 3, 1], top 1)
- output **1, 3, 2**.

An exhaustive search over all move sequences (see the verify code) confirms that all 24 permutations of 1..4 are reachable → (A) **TRUE**.

Trap: assuming two stacks behave like one bigger stack; the transfer step adds genuine reordering power.

Q.14 [NAT · 2 marks · Quicksort — counting worst-case inputs]

Answer: 16

Concept. A call on m elements costs $m - 1$ comparisons. The total reaches $4 + 3 + 2 + 1 = 10$ only if **every** partition leaves one side empty, i.e. the pivot (last element of the current range) is the minimum or maximum of that range.

Count level by level:

- Top level: $A[4]$ must be 1 or 5 → **2** choices.
- If the pivot is the maximum, every element is $\leq$ pivot, each swap is a self-swap, and the remaining 4 elements keep their order. If the pivot is the minimum, nothing is $\leq$ pivot and the final swap exchanges $A[lo]$ with the pivot, so the remaining range is $A[lo+1..hi-1]$ followed by the old $A[lo]$. In both cases the new range is a **fixed, invertible rearrangement** of the other 4 elements.
- Hence, given the top-level choice, the number of good arrangements of the remaining 4 elements equals the number of worst-case permutations of size 4, and so on.

$W(1) = 1$, $W(m) = 2 \cdot W(m - 1) \rightarrow W(5) = 2^4 = \mathbf{16}$.

Example: 2 3 4 1 5. Pivot 5 (max) → range 2 3 4 1, pivot 1 (min) → swap gives 1 | 3 4 2, pivot 2 (min) → 2 | 4 3, pivot 3 (min) → 3 | 4. Total $4 + 3 + 2 + 1 = 10$ ✓.

Trap: assuming only the sorted and reverse-sorted inputs are worst cases (2 permutations); or assuming “pivot is extreme” must hold in the *original* array positions — Lomuto’s swaps move elements, so the condition is on the rearranged subarray.

Q.15 [MCQ · 2 marks · Complexity — dependent logarithmic loops]

Answer: (A)

Concept. Count inner iterations per outer iteration. Let $n = 2^k$. The outer loop takes $i = 2^k, 2^{k-1}, \dots, 2^1$ (k iterations). For $i = 2^t$, the inner loop doubles j from 2^t up to 2^k : exactly $k - t$ iterations.

Total $c = \sum_{t=1}^k (k - t) = 0 + 1 + \dots + (k - 1) = k(k - 1)/2$.

For $n = 1024$ ($k = 10$): $c = 45$; for $n = 2^{20}$: $c = 190$. With $k = \log_2 n$, $c = \Theta(\log^2 n) \rightarrow (A)$.

Options. (B) multiplies the *counts* incorrectly by treating the inner loop as $O(1)$. (C)/(D) treat one of the loops as linear — but both variables move geometrically.

Trap: the inner loop starts at the current i (not at 1), so it is short when i is large and long when i is small; the sum is an arithmetic series in k .

Q.16 [NAT · 2 marks · Dijkstra with a negative edge]

Answer: 3

Dijkstra run (extraction order with d -values):

- S (0): $A = 7$, $B = 3$.
- B (3): $C = 4$.
- C (4): $T = 6$, $D = 9$.

- T (6): $D = \min(9, 8) = 8$.
- A (7): edge $A \rightarrow C$ (-5) would give 2, but C is already final $\rightarrow$ **skipped**.
- D (8).

Reported: S 0, A 7, B 3, C 4, T 6, D 8.

True distances (e.g. Bellman-Ford): $C = \min(3 + 1, 7 - 5) = 2$, $T = 2 + 2 = 4$, $D = \min(2 + 5, 4 + 2) = 6$; S, A, B are unchanged (0, 7, 3).

Wrong vertices: C, T and D $\rightarrow$ **3**.

Concept. Dijkstra's correctness relies on the fact that a path can only get longer when extended; a negative edge into an already-final vertex breaks this, and the error propagates to everything downstream of that vertex.

Trap: counting only C (the head of the negative edge) — T and D inherit the mistake.

Reported vs true distances

	S	A	B	C	T	D
Dijkstra	0	7	3	4	6	8
true	0	7	3	2	4	6

Q.17 [MSQ · 2 marks · Graph structure — bipartiteness, cut vertices, BFS levels]

Answer: (A), (B), (C)

- (A) C-E is a **bridge** (its removal disconnects the graph), and every spanning tree must contain every bridge. **TRUE**.
- (B) The only cycles are A-B-C-D-A and E-F-G-H-E, both of length 4 (even). A graph is bipartite iff it has no odd cycle $\rightarrow$ 2-colouring $\{A, C, F, H\}$ vs $\{B, D, E, G\}$. **TRUE**.
- (C) BFS from A: level 0 $\{A\}$, 1 $\{B, D\}$, 2 $\{C\}$, 3 $\{E\}$, 4 $\{F, H\}$, 5 $\{G\}$ $\rightarrow$ 6 levels. **TRUE**.
- (D) Removing C separates $\{A, B, D\}$ from $\{E, F, G, H\}$; removing E separates $\{A, B, C, D\}$ from $\{F, G, H\}$. No other vertex disconnects the graph. **Two** cut vertices $\rightarrow$ **FALSE**.

Concept recap. A vertex is a cut vertex iff removing it increases the number of components; an edge lies in every spanning tree iff it is a bridge (it lies on no cycle).

Trap: in (D), noticing only the bridge's one endpoint — *both* endpoints of a bridge are cut vertices whenever each has degree ≥ 2 .

Q.18 [NAT · 2 marks · Heaps — counting heaps with a constraint]

Answer: 40

Concept. Count heaps recursively: the root must be the minimum; the remaining keys are split between the two subtrees, and each subtree must itself be a heap. For a 3-node subtree with a fixed set of keys, the smallest is its root and the other two can be arranged in 2 ways.

- Root = 1 (forced).
- Key 2 is the left child, so 2 is the root of the left 3-node subtree. Its two children are any 2 of the remaining keys $\{3, 4, 5, 6, 7\}$: $C(5, 2) = 10$ choices, arranged in **2** orders.
- The other 3 keys form the right subtree: its minimum is the root, and the other two can be ordered in **2** ways.

Total = $10 \times 2 \times 2 = 40$.

Cross-check: the total number of min-heaps on 7 keys is $C(6, 3) \times 2 \times 2 = 80$, and by symmetry key 2 (which must be a child of the root) is the left child in exactly half of them $\rightarrow 40 \checkmark$.

Trap: forgetting that the left subtree's children may appear in either order (giving 20), or requiring 3 to be the right child of the root (it need not be — 3 can sit under 2).

Q.19 [MCQ · 2 marks · Python — dict ordering, update and truthiness]**Answer: (C)**

Concept. Since Python 3.7, dicts preserve **insertion order**. Re-assigning an existing key keeps its position; deleting and re-inserting a key moves it to the end. **update** follows the same rules. Iterating a dict yields its keys.

- Start: b:1, a:2, c:3.
- `d['a'] = 5` → b:1, a:5, c:3 (position unchanged).
- `del d['b']` → a:5, c:3; `d['b'] = 7` → a:5, c:3, **b:7** (now last).
- `update(c=0, e=9)` → a:5, c:0, b:7, e:9.
- **k:** keys with truthy values → c (0) is excluded → ['a', 'b', 'e'].
- `sorted(d, key=d.get)` orders keys by value: c(0), a(5), b(7), e(9) → first is 'c'.
- `list(d.items())[1]` → ('c', 0).

Output ['a', 'b', 'e'] c ('c', 0) → (C).

Options. (A) forgets that 0 is falsy. (B) keeps 'b' at its original first position. (D) thinks re-inserted 'b' returns to its old slot right after 'a'.

Trap: `d[k] = v` on an *existing* key does not move it; only deletion followed by insertion does.

Q.20 [MSQ · 2 marks · DAGs — topological orders and Kahn's algorithm]**Answer: (B), (C)**

Concept. A sequence is a topological order iff every edge $u \rightarrow v$ has u before v . Kahn's algorithm repeatedly outputs a vertex of in-degree 0 and deletes its out-edges.

In-degrees: A 0, B 0, C 2, D 1, E 1, F 2, G 2, H 2.

- (A) In B D A G C E F H, G appears before E, but $E \rightarrow G$ is an edge. **FALSE.**
- (B) H is the only sink, and every other vertex has a path to H (via F or G), so H must come after all of them. **TRUE.**
- (C) Kahn with a min-priority queue: available {A, B} → A (frees E); {B, E} → B (frees C and D); {C, D, E} → C; {D, E} → D (F freed: its predecessors C, D are done); {E, F} → E (frees G); {F, G} → F; G; H. Output **A B C D E F G H. TRUE.**
- (D) A and D are incomparable (no path between them): e.g. B D A C E F G H is valid. **FALSE.**

Trap: in (C), students often output D before C because D was “freed” by B at the same time as C — the tie is broken alphabetically, so C wins.

Consolidated Answer Keys

Tests 01 - 10

Q	T01	T02	T03	T04	T05	T06	T07	T08	T09	T10
1	(C)	(B)	(A)	(C)	(A)	(A)	(C)	18	(B)	(C)
2	17	179	7	82	17	4	5	(B)	8	4
3	(B)	(C)	-27	(A),(B),(C)	(B)	(C)	(B)	(D)	39	(B)
4	(A)	(B),(D)	(A),(C)	(D)	(A),(B),(D)	(B),(C)	(C),(D)	7	(C),(D)	8
5	185	2	(B)	4	12	1.89 to 1.91	13	(C)	(D)	(B)
6	(A),(B),(D)	(A)	(C)	(D)	(C)	(B)	5	(C),(D)	5	(C),(D)
7	5	1.79 to 1.81	(D)	(B)	7	11	(D)	(A)	(A)	(C)
8	(D)	(B)	(B)	(A)	(B)	(D)	4	17	(B)	(B),(D)
9	(D)	(B),(D)	(C)	(B),(C),(D)	3	(A),(D)	(A),(D)	(B)	(A),(B),(D)	(D)
10	(C),(D)	6	(C),(D)	7	(A)	7	(B)	(B),(D)	(C)	(B),(C),(D)
11	40	14	8	(B)	(B),(C),(D)	(B)	8	12	37	(A)
12	(B)	(D)	25	9	16	210	(A)	(A)	(B),(C),(D)	22
13	(B),(D)	(A),(B),(C)	(A)	9	(D)	(C),(D)	(A),(B),(C)	(A),(D)	9	(A),(C)
14	(A),(D)	60	(B),(D)	(B),(C)	(C)	(A)	32	2.05 to 2.07	(D)	(A)
15	(C)	(A),(B),(D)	9	9	(B),(C),(D)	56	(A)	(C)	17	13
16	18	(C)	(D)	(A)	34	7	12	(A),(C),(D)	(A),(B),(D)	16
17	33	(B),(C),(D)	21	(C)	(D)	(A),(B),(C)	(C),(D)	(D)	33	(B),(C),(D)
18	(A),(B),(D)	24	(A),(B),(D)	10	7	33	9	36	(A)	10
19	(A)	(A)	83	(B),(C),(D)	(A),(B),(C)	(C)	(C)	(B),(C),(D)	(A),(C),(D)	(D)
20	7281821	33	(A),(C)	(A),(C),(D)	(A),(B),(D)	(C),(D)	(A),(B),(D)	7	(C)	13

Tests 11 - 20

Q	T11	T12	T13	T14	T15	T16	T17	T18	T19	T20
1	(D)	(A)	(B)	(B)	(A)	(B)	(C)	(A)	(C)	(B)
2	10	4	3	13	26	37	7	50	40	43
3	(A)	(A)	(A),(C)	(A)	(B)	(A)	(A)	(D)	(A)	(B)
4	(A),(D)	(A),(B)	6	(A),(B),(C)	(A),(B)	(B),(C),(D)	(A),(B),(D)	(A),(B),(D)	(A),(B)	3
5	15	-23	(D)	4	5	12	9	7	27	(A),(B)
6	(C)	(C)	(A)	(C)	(D)	(C)	(A)	(C)	(D)	(C)
7	19	6	20	4	22	4	9	6	(B)	2
8	(A),(B)	(D)	(A)	(D)	(A)	(C)	(B)	(B)	(D)	(A),(D)
9	(B)	(D)	(C),(D)	(A),(B)	3	(C),(D)	(A),(B),(D)	(B),(C)	63	(A),(C),(D)
10	9	(C),(D)	(B)	(D)	(D)	(A)	6	37	(C),(D)	4
11	(D)	(C)	16	6	16	23	(C)	(A)	(B)	25
12	37	13	(A),(D)	(C)	(A),(B),(D)	(D)	643	(A),(B),(D)	286	(A)
13	(A),(C)	(B),(C)	5	8	6	22	(A),(C),(D)	(A),(B),(C)	(B),(C),(D)	26
14	14	18	(C)	(B),(C),(D)	(C)	(A),(B),(D)	5819	14	573	(A),(B)
15	(A)	(B)	53	(B)	(C)	(D)	17	(D)	(B),(C),(D)	(D)
16	(B),(D)	17	(A),(B),(C)	12	(A),(D)	(A),(B),(C)	(A),(B),(D)	(B)	9	4
17	6	(A),(C)	11	(A),(C),(D)	6	(A),(C),(D)	9	16	(C)	(C)
18	(C)	7	(C)	35	(B),(C),(D)	9	(B)	(A),(B),(D)	(A),(B),(D)	(A)
19	(B)	(B)	(B),(C),(D)	(A)	(A),(B),(D)	(B)	(D)	(C)	119	(B),(C)
20	(B),(C),(D)	(B),(C),(D)	(D)	(A),(C),(D)	(B)	39	(A),(B),(C)	8	(A)	14

Tests 21 - 30

Q	T21	T22	T23	T24	T25	T26	T27	T28	T29	T30
1	10	(B)	(A)	(B)	18	(A)	(B)	6	(A)	(B)
2	(A)	14	0.34 to 0.36	7	(A)	8	3	(A)	14	92
3	(B)	(C)	3.79 to 3.81	(B),(D)	(D)	(A),(C)	(C)	(B),(D)	4	(C)
4	(A),(B),(C)	(A),(C),(D)	(C)	11	(A),(B)	4	(A),(B),(C)	3	(A),(B),(C)	(A),(C),(D)
5	17	-6	(B),(C),(D)	(A)	(B)	(B)	8	(C)	(D)	14
6	(C)	(B)	(A)	3	(C)	(A),(B),(C)	(A)	210	4	(C)
7	6	9	4	16	10	(D)	5	(B)	(C)	(A),(C),(D)
8	(D)	(A),(B),(D)	(B)	(A),(C),(D)	(B)	3	(D)	(B)	(B)	10
9	(D)	(A)	(C)	(B)	36	(B)	(B),(C),(D)	(C)	(A),(B),(D)	(B)
10	(C),(D)	(D)	(C),(D)	(D)	(C)	(C)	3	8	(C)	8
11	64	21	(A),(C),(D)	(B),(C),(D)	7	(C)	3	(B),(C)	(D)	(A)
12	68	(A)	2.79 to 2.81	24	(B),(C)	(B),(C),(D)	(B)	36	(A),(B)	17
13	(B)	(A),(D)	15	(C)	14	5	(A),(B),(D)	(A),(B),(D)	3	(A),(C),(D)
14	(B),(C)	14	(D)	9	(A)	(D)	30	56	(B)	7
15	45	(D)	11	(C)	(A),(B),(D)	(A),(B),(C)	(C)	(D)	6	(D)
16	(A)	(C)	(B)	(A),(B),(D)	(B),(C),(D)	75	18	(B),(C),(D)	(A),(C)	6
17	(A),(B),(C)	(B),(D)	35	7	(D)	(A)	(A),(C),(D)	9	8	(C),(D)
18	(A),(C)	210	(A),(C),(D)	(D)	43	6	28	(A)	(A)	6
19	(C)	(A),(D)	(D)	(B),(D)	(B),(D)	(C),(D)	(A)	(D)	(A),(C)	(A)
20	19483752	24	(A),(B),(C)	(A)	18	20	(B),(C)	(B),(C),(D)	4	(A),(B)

Tests 31 - 40

Q	T31	T32	T33	T34	T35	T36	T37	T38	T39	T40
1	(D)	(C)	(B)	(C)	(B)	(B)	(B)	(D)	(D)	(A)
2	34	16	-1	-5	4	3	6	-6	8	35
3	(A)	(B)	(A)	(D)	(B),(C),(D)	(C)	(C)	(B)	(C)	(D)
4	6	26	321456	6	12	6	0	1	5	(A),(B),(D)
5	(A),(C)	(A)	(C)	(B),(C)	(A)	(C),(D)	(A),(B)	(A)	(A),(B),(C)	6
6	(B)	3.26 to 3.28	48	(B)	(D)	(D)	(A)	(C)	(A),(B),(C)	(C)
7	(C)	(C)	(A)	(A)	4	15	405	(B),(D)	(D)	10
8	(B)	10	(A),(B),(D)	9	(A)	(A)	(C)	(D)	6.32 to 6.34	(A),(C),(D)
9	(A),(C),(D)	(A),(B),(C)	(A),(B),(C)	(A),(D)	-2	(A),(C),(D)	16	6	(A)	7
10	31	(A),(C)	(D)	8	(C)	(A)	(A),(C),(D)	(B),(D)	(A),(B),(C)	(B)
11	(C)	59	37	(C)	(C)	7	(B)	(C)	(A)	(A)
12	5	(A)	(C),(D)	41293765	(A),(B),(D)	(D)	11	16	56	110
13	(A),(D)	(A),(C),(D)	8	(B),(C),(D)	(A),(B),(C)	(A),(B),(D)	(B),(D)	(A),(C),(D)	(A),(B),(D)	(A),(C),(D)
14	12	15	(C)	(D)	92	13	12	405	(C),(D)	27
15	(D)	(B)	14	120	(A),(B),(D)	(B)	(D)	(A)	(B)	(B)
16	10	4	(B)	(A),(B),(D)	42	9	19	(A),(C),(D)	3.48 to 3.50	48
17	(A),(D)	(B),(C)	10	(B)	(B)	(A),(C)	(A),(B),(D)	(B)	(B)	(B),(C)
18	(A)	3.58 to 3.60	(B),(C),(D)	6	(A),(C)	13	17	12	14	19.39 to 19.41
19	12	(D)	(A),(B),(C)	(A),(B),(C)	11	(C)	(A)	(A),(C),(D)	(C)	(C)
20	(A),(C)	(B),(D)	(D)	(A)	(D)	(B),(C),(D)	(A),(C),(D)	14	17	(B),(D)

Tests 41 - 50

Q	T41	T42	T43	T44	T45	T46	T47	T48	T49	T50
1	(C)	(D)	(C)	(B)	(C)	(B)	(C)	(D)	(B)	(C)
2	30	11	21	42	65	9	86	40	73	20
3	(C),(D)	(A),(B),(D)	9	(C)	(B),(C),(D)	(C)	(A),(C)	(B),(C),(D)	(C)	127
4	(C)	16	(A),(B),(D)	(A)	2.74 to 2.76	(A),(C)	1.79 to 1.81	7	(A)	(A)
5	6	(A)	4	13	(D)	1.60 to 1.62	(A)	(D)	(B),(C),(D)	(C),(D)
6	(A)	16	(D)	(A),(B),(D)	(B)	(B)	128	14	(D)	(B)
7	2.92 to 2.94	(B)	(A)	(A)	9	(C)	(B)	(C)	9.00 to 9.02	5.30 to 5.32
8	(B),(C)	(A),(C),(D)	11	(A),(B),(C)	(A)	(A)	(A),(D)	(A)	(B)	(C),(D)
9	(A)	12	(A),(C)	10	4.32 to 4.34	(C),(D)	20	(B)	(C),(D)	(B)
10	(B)	(A)	(A)	25	(D)	(D)	(B)	(B),(C),(D)	66	9
11	23	(C)	(B)	(B)	25	80	21	(B)	(A)	(D)
12	(D)	16	42	23	(B),(C)	(A),(D)	(D)	22	26	79
13	(A),(B),(C)	(A),(C)	(A),(C),(D)	(C),(D)	(C)	15	(B),(C)	(A),(C),(D)	(B),(D)	(A),(B),(C)
14	39	18	45	(A),(B),(C)	6	(A)	68	896	7	16
15	(B)	(C)	(D)	(D)	(B),(C),(D)	42	(C)	(C)	(C)	(A)
16	26	8	24	4	(B),(C),(D)	(A),(B),(C)	12	10	(B),(C),(D)	3
17	(D)	(A),(C),(D)	(A),(C),(D)	5	(B)	9	(A),(B),(D)	(B),(C),(D)	16	(A),(B),(C)
18	(A),(C),(D)	45	(C)	(C)	(B),(C)	(D)	1116	-4	(D)	40
19	9	(B)	(B)	(A),(B),(D)	16	146	(A)	(A)	42	(C)
20	(A),(C)	(C),(D)	(A),(B)	(D)	(A)	(A),(B),(C)	(B),(C),(D)	(A),(D)	(A),(B),(C)	(B),(C)

Rows 1-10 are 1-mark questions and rows 11-20 are 2-mark questions. For NAT answers given as a range, any value within the range is correct.

Score Tracker

Test	Date	Time taken	Attempted	Correct	Wrong MCQs	Score/30	Weak topics
01							
02							
03							
04							
05							
06							
07							
08							
09							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							
20							
21							
22							
23							
24							
25							
26							
27							
28							
29							
30							
31							
32							
33							
34							
35							
36							
37							
38							
39							
40							
41							
42							
43							
44							
45							
46							
47							
48							
49							
50							